



Nuclei Microcontroller Software Interface Standard

NMSIS

Release 1.6.0

Nuclei

Jun 15, 2026

CONTENTS:

1	Nuclei MCU Software Interface Standard(NMSIS)	1
1.1	About NMSIS	1
1.2	NMSIS Components	1
1.3	NMSIS Design	2
1.4	How to Access	3
1.5	Coding Rules	3
1.6	Validation	3
1.7	License	4
2	NMSIS Core	5
2.1	Overview	5
2.1.1	Introduction	5
2.1.2	Processor Support	6
2.1.3	Toolchain Support	6
2.2	Using NMSIS in Embedded Applications	6
2.2.1	Introduction	6
2.2.2	Basic NMSIS Example	10
2.2.3	Using Interrupt and Exception/NMI	11
2.2.4	Using NMSIS with generic Nuclei Processors	11
2.2.5	Create generic Libraries with NMSIS	11
2.3	NMSIS-Core Device Templates	12
2.3.1	Introduction	12
2.3.2	NMSIS-Core Processor Files	13
2.3.3	Device Examples	13
2.3.4	Template Files	13
2.3.5	Adapt Template Files to a Device	14
2.3.6	Device Templates Explanation	15
2.4	Register Mapping	70
2.5	NMSIS Core API	71
2.5.1	Version Control	71
2.5.2	Compiler Control	72
2.5.3	Core CSR Register Access	75
2.5.4	Core CSR Encoding	85
2.5.5	Register Define and Type Definitions	135
2.5.6	CPU Intrinsic Functions	198
2.5.7	Intrinsic Functions for SIMD Instructions	211
2.5.8	Intrinsic Functions for Bitmanipulation Instructions	610
2.5.9	Intrinsic Functions for Vector Instructions	610
2.5.10	Peripheral Access	611
2.5.11	Systick Timer(SysTimer)	612

2.5.12	Interrupts and Exceptions	631
2.5.13	PLIC Interrupt	671
2.5.14	CIDU Functions	680
2.5.15	FPU Functions	690
2.5.16	PMP Functions	694
2.5.17	SPMP/sMPU Functions	700
2.5.18	PMA Functions	705
2.5.19	Cache Functions	712
2.5.20	SMPCC Functions	763
2.5.21	ECC Functions	791
2.5.22	System Device Configuration	804
2.5.23	ARM Compatiable Functions	819
2.5.24	NMSIS Bench and Test Helper Functions	822
3	NMSIS DSP	831
3.1	Overview	831
3.1.1	Introduction	831
3.1.2	Using the Library	832
3.1.3	Examples	832
3.1.4	Toolchain Support	832
3.1.5	Building the Library	832
3.1.6	Preprocessor Macros	832
3.2	Using NMSIS-DSP	832
3.2.1	Preparation	832
3.2.2	Tool Setup	833
3.2.3	Build NMSIS DSP Library	833
3.2.4	How to run	836
3.3	NMSIS DSP API	837
3.3.1	Examples	837
3.3.2	Basic Math Functions	853
3.3.3	Bayesian estimators	880
3.3.4	Complex Math Functions	881
3.3.5	Controllor Functions	893
3.3.6	Distance Functions	905
3.3.7	Fast Math Functions	919
3.3.8	Filtering Functions	928
3.3.9	Interpolation Functions	1018
3.3.10	Matrix Functions	1026
3.3.11	Quaternion Math Functions	1062
3.3.12	Statistics Functions	1066
3.3.13	Support Functions	1098
3.3.14	SVM Functions	1120
3.3.15	Transform Functions	1127
3.3.16	Window Functions	1249
3.3.17	Generic Types	1275
3.4	Changelog	1278
3.4.1	latest	1278
3.4.2	V1.3.1	1279
3.4.3	V1.3.0	1279
3.4.4	V1.2.1	1279
3.4.5	V1.2.0	1279
3.4.6	V1.1.1	1280
3.4.7	V1.1.0	1280
3.4.8	V1.0.3	1280

3.4.9	V1.0.2	1281
3.4.10	V1.0.1	1281
3.4.11	V1.0.0	1281
4	NMSIS NN	1283
4.1	Overview	1283
4.1.1	Introduction	1283
4.1.2	Block Diagram	1284
4.1.3	Examples	1284
4.1.4	Pre-processor Macros	1284
4.2	Using NMSIS-NN	1284
4.2.1	Preparation	1284
4.2.2	Tool Setup	1285
4.2.3	Build NMSIS NN Library	1285
4.2.4	How to run	1287
4.3	NMSIS NN API	1289
4.3.1	Convolutional Neural Network Example	1289
4.3.2	Gated Recurrent Unit Example	1290
4.3.3	Public	1291
4.3.4	Private	1398
4.4	Changelog	1430
4.4.1	latest	1430
4.4.2	V1.3.1	1430
4.4.3	V1.3.0	1431
4.4.4	V1.2.1	1431
4.4.5	V1.2.0	1431
4.4.6	V1.1.1	1432
4.4.7	V1.1.0	1432
4.4.8	V1.0.3	1432
4.4.9	V1.0.2	1432
4.4.10	V1.0.1	1432
4.4.11	V1.0.0	1433
5	Changelog	1435
5.1	V1.6.0	1435
5.2	V1.5.0	1437
5.3	V1.4.1	1439
5.4	V1.4.0	1439
5.5	V1.3.1	1440
5.6	V1.3.0	1441
5.7	V1.2.1	1442
5.8	V1.2.0	1443
5.9	V1.1.1	1444
5.10	V1.1.0	1445
5.11	V1.0.4	1446
5.12	V1.0.3	1447
5.13	V1.0.2	1448
5.14	V1.0.2-RC2	1448
5.15	V1.0.2-RC1	1449
5.16	V1.0.1	1449
5.17	V1.0.1-RC1	1449
5.18	V1.0.0-beta1	1450
5.19	V1.0.0-beta	1450
5.20	V1.0.0-alpha.1	1451

5.21 V1.0.0-alpha	1451
6 Glossary	1453
7 Appendix	1455
8 Indices and tables	1457
Index	1459

NUCLEI MCU SOFTWARE INTERFACE STANDARD(NMSIS)

1.1 About NMSIS

The **NMSIS** is a vendor-independent hardware abstraction layer for micro-controllers that are based on [Nuclei Processors](#)¹.

The **NMSIS** defines generic tool interfaces and enables consistent device support. It provides simple software interfaces to the processor and the peripherals, simplifying software re-use, reducing the learning curve for micro-controller developers, and reducing the time to market for new devices.

Note

To get a pdf version of this documentation, please click [NMSIS Document](#)²

1.2 NMSIS Components

NMSIS CORE

All Nuclei N/NX/UX Class Processors Standardized API for the Nuclei processor core and peripherals.

NMSIS DSP

All Nuclei N/NX/UX Class Processors DSP library collection with a lot of functions for various data types: fixed-point (fractional q7, q15, q31) and single precision floating-point (32-bit).

NMSIS NN

All Nuclei N/NX/UX Class Processors Collection of efficient neural network kernels developed to maximize the performance and minimize the memory footprint Nuclei processor cores.

Note

NMSIS DSP/NN library implementations are optimized for the Nuclei Processors which has P-ext or V-ext instruction set.

We also provide other **Nuclei Optimized Libraries** in other repositories:

- [Nuclei AI Library Optimized For RISC-V Vector](#)³

¹ <https://www.nucleisys.com/product.php>

² <https://doc.nucleisys.com/nmsis/nmsis.pdf>

³ <https://github.com/Nuclei-Software/nuclei-ai-library>

- MbedTLS Library Optimized for Nuclei Crypto Engine and RISC-V Crypto/Vector Crypto Extension⁴
- Various AI Models supported by Nuclei RISC-V Processor⁵
- Nuclei Audio Library Optimized For RISC-V Packed-SIMD and Vector⁶
- TensorFlow Lite Micro Optimized For Nuclei RISC-V Processor⁷
- TinyMaix(Tiny Inference Library) Optimized For Nuclei RISC-V Processor⁸

1.3 NMSIS Design

NMSIS is designed to help the Nuclei N/NX/UX Class Processors processors in standardization. It enables consistent software layers and device support across a wide range of development tools and micro-controllers.

NMSIS is a lightweight software interface layer that tried to standardize common Nuclei processor-based SOC, and it didn't define any standard peripherals. The silicon industry can therefore support the wide variations of Nuclei processor-based devices with this common standard.

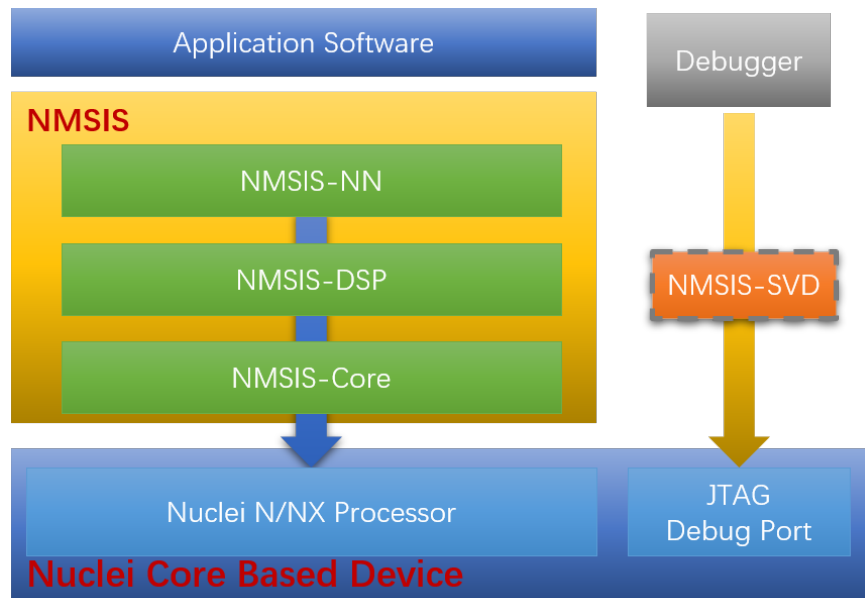


Fig. 1: NMSIS Design Diagram

We can achieve the following benefits of **NMSIS**:

- **NMSIS** reduces the learning curve, development costs, and time-to-market. Developers can write software quicker through a variety of easy-to-use, standardized software interfaces.
- Consistent software interfaces improve the software portability and re-usability. Generic software libraries and interfaces provide consistent software framework.
- It provides interfaces for debug connectivity, debug peripheral views, software delivery, and device support to reduce time-to-market for new micro-controller deployment.

⁴ <https://github.com/Nuclei-Software/mbedtls>

⁵ <https://github.com/Nuclei-Software/nuclei-ai-models>

⁶ <https://github.com/Nuclei-Software/nuclei-audio-library>

⁷ <https://github.com/Nuclei-Software/tflite-micro>

⁸ <https://github.com/Nuclei-Software/npk-tinymaix>

- Being a compiler independent layer, it allows to use the compiler of your choice. Thus, it is supported by main-stream compilers.
- It enhances program debugging with peripheral information for debuggers.

1.4 How to Access

If you want to access the code of **NMSIS**, you can visit our opensource [NMSIS Github Repository](https://github.com/Nuclei-Software/NMSIS)⁹.

1.5 Coding Rules

The **NMSIS** uses the following essential coding rules and conventions:

- Compliant with ANSI C (C99) and C++ (C++03).
- Uses ANSI C standard data types defined in **stdint.h**.
- Variables and parameters have a complete data type.
- Expressions for *#define* constants are enclosed in parenthesis.

In addition, the **NMSIS** recommends the following conventions for identifiers:

- **CAPITAL** names to identify Core Registers, Peripheral Registers, and CPU Instructions.
- **CamelCase** names to identify function names and interrupt functions.
- **Namespace_** prefixes avoid clashes with user identifiers and provide functional groups (i.e. for peripherals, RTOS, or DSP Library).

The **NMSIS** is documented within the source files with:

- Comments that use the C or C++ style.
- Doxygen compliant comments, which provide:
 - brief function, variable, macro overview.
 - detailed description of the function, variable, macro.
 - detailed parameter explanation.
 - detailed information about return values.

1.6 Validation

Nuclei uses RISC-V GCC/Clang/IAR Compiler in the various tests of **NMSIS**, and if more compiler is added, it could be easily supported by following the **NMSIS** compiler independent layer. For each component, the section **Validation** describes the scope of the various verifications.

NMSIS components are compatible with a range of C and C++ language standards.

As **NMSIS** defines API interfaces and functions that scale to a wide range of processors and devices, the scope of the run-time test coverage is limited. However, several components are validated using dedicated test suites.

⁹ <https://github.com/Nuclei-Software/NMSIS>

1.7 License

This **NMSIS** is modified based on open-source project **CMSIS** to match Nuclei requirements.

This **NMSIS** is provided free of charge by Nuclei under the [Apache 2.0 License](http://www.apache.org/licenses/LICENSE-2.0)¹⁰.

¹⁰ <http://www.apache.org/licenses/LICENSE-2.0>

NMSIS CORE

2.1 Overview

2.1.1 Introduction

NMSIS-Core implements the basic run-time system for a Nuclei N/NX/UX Class Processors based device and gives the user access to the processor core and the device peripherals. In detail it defines:

- **Hardware Abstraction Layer (HAL)** for Nuclei processor registers with standardized definitions for the **CSR Registers, TIMER, ECLIC, PLIC, CIDU, PMA, PMP/sPMP/SMPU Registers, DSP Registers, FPU registers, and Core Access Functions.**
- **Standard core exception/interrupt names** to interface to system exceptions or interrupts without having compatibility issues.
- **Methods to organize header files** that makes it easy to learn new Nuclei micro-controller products and improve software portability. This includes naming conventions for device-specific interrupts.
- **Methods for system initialization** to be used by each Device vendor. For example, the standardized *SystemInit()* (page 805) function is essential for configuring the clock system of the device.
- **Intrinsic functions** used to generate CPU instructions that are not supported by standard C functions.
- A variable *SystemCoreClock* (page 810) to determine the **system clock frequency** which simplifies the setup the timer.

The following sections provide details about the **NMSIS-Core**:

- *Using NMSIS in Embedded Applications* (page 6) describes the project setup and shows a simple program example
- *NMSIS-Core Device Templates* (page 12) describes the files of the *NMSIS Core* (page 5) in detail and explains how to adapt template files provided by Nuclei to silicon vendor devices.
- *NMSIS Core API* (page 71) describe the features and functions of the *Device Header File <Device.h>* (page 48) in detail.
- *Register Define and Type Definitions* (page 135) describe the data structures of the *Device Header File <Device.h>* (page 48) in detail.

2.1.2 Processor Support

NMSIS have provided support for all the Nuclei N/NX/UX Class Processors.

Nuclei ISA Spec:

Please contact with our sales about Nuclei Process Core Instruction Set Architecture Spec `Nuclei_RISC-V_ISA_Spec.pdf`.

Nuclei Processor Reference Manuals:

- 200 series¹¹
- 300 series¹²
- 600 series¹³
- 900 series¹⁴
- 1000 series¹⁵

2.1.3 Toolchain Support

The *NMSIS-Core Device Templates* (page 12) provided by Nuclei have been tested and verified using these toolchains:

- GNU GCC/LLVM Clang Toolchain for RISC-V modified by Nuclei, see <https://www.nucleisys.com/download.php#tools>
- IAR Compiler for RISC-V, see <https://www.iar.com/riscv>
- Terapines ZCC Compiler, see <https://www.terapines.com/products/zcc>

2.2 Using NMSIS in Embedded Applications

2.2.1 Introduction

To use the **NMSIS-Core**, the following files are added to the embedded application:

- *Startup File* `startup_<Device>.S` (page 15), which provided asm startup code and vector table.
- *Interrupt and Exception Handling File*: `intexc_<Device>.S` (page 26), which provided general exception handling code for non-vector interrupts and exceptions.
- *Device Linker Script*: `gcc_<device>.ld` (page 37), which provided linker script for the device.
- *System Configuration Files* `system_<Device>.c` and `system_<Device>.h` (page 43), which provided general device configuration (i.e. for clock and BUS setup).
- *Device Header File* `<Device>.h` (page 48) gives access to processor core and all peripherals.

¹¹ <https://www.nucleisys.com/product/200.php>

¹² <https://www.nucleisys.com/product/300.php>

¹³ <https://www.nucleisys.com/product/600.php>

¹⁴ <https://www.nucleisys.com/product/900.php>

¹⁵ <https://www.nucleisys.com/product/1000.php>

Note

The best example usage of NMSIS device template is our Nuclei SDK project's evalsoc support source code, it is modified based on NMSIS device template and may contains newer experimental features not in this device template, and it also provided very good user experience for both command line and IDE itegration like Nuclei Studio and IAR Workbench.

The files *Startup File startup_<Device>.S* (page 15), *Interrupt and Exception Handling File: intexc_<Device>.S* (page 26), *Device Linker Script: gcc_<device>.ld* (page 37) and *System Configuration Files system_<Device>.c and system_<Device>.h* (page 43) may require application specific adaptations and therefore should be copied into the application project folder prior configuration.

The *Device Header File <Device.h>* (page 48) is included in all source files that need device access and can be stored on a central include folder that is generic for all projects.

The *Startup File startup_<Device>.S* (page 15) is executed right after device reset, it will do necessary stack pointer initialization, exception and interrupt entry configuration, then call *SystemInit()* (page 805), after system initialization, will return to assemble startup code and do c/c++ runtime initialization which includes data, bss section initialization, c++ runtime initialization, then it will call *main()* function in the application code.

In the *Interrupt and Exception Handling File: intexc_<Device>.S* (page 26), it will contain all exception and interrupt vectors and implements a default function for every interrupt. It may also contain stack and heap configurations for the user application.

The *System Configuration Files system_<Device>.c and system_<Device>.h* (page 43) performs the setup for the processor clock. The variable *SystemCoreClock* (page 810) indicates the CPU clock speed. *Systick Timer(SysTimer)* (page 612) describes the minimum feature set. In addition the file may contain functions for the memory BUS setup and clock re-configuration.

The *Device Header File <Device.h>* (page 48) is the central include file that the application programmer is using in the C source code. It provides the following features:

- *Peripheral Access* (page 611) provides a standardized register layout for all peripherals. Optionally functions for device-specific peripherals may be available.
- *Interrupts and Exceptions* (page 631) can be accessed with standardized symbols and functions for the **ECLIC** are provided.
- *CPU Intrinsic Functions* (page 198) allow to access special instructions, for example for activating sleep mode or the NOP instruction.
- *Intrinsic Functions for SIMD Instructions* (page 211) provide access to the DSP-oriented instructions.
- *Systick Timer(SysTimer)* (page 612) function to configure and start a periodic timer interrupt.
- *Core CSR Register Access* (page 75) function to access the core csr registers.
- *Cache Functions* (page 712) to access the I-CACHE and D-CACHE unit
- *FPU Functions* (page 690) to access the Floating point unit.
- *PMP Functions* (page 694) to access the Physical Memory Protection unit
- *Version Control* (page 71) which defines NMSIS release specific macros.
- *Compiler Control* (page 72) is compiler agnostic *#define* symbols for generic C/C++ source code

The NMSIS-Core system files are device specific.

In addition, the *Startup File startup_<Device>.S* (page 15) is also compiler vendor specific, currently only GCC version is provided. The versions provided by NMSIS are only generic templates. The adopted versions for a concrete device

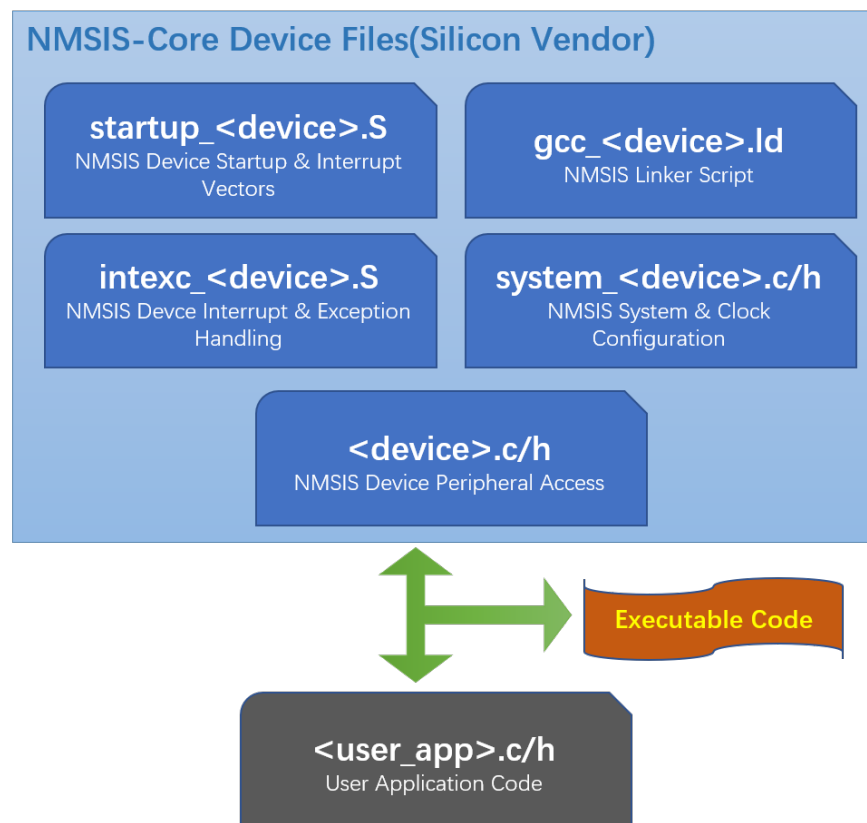


Fig. 1: NMSIS-Core User Files

are typically provided by the device vendor through the according device family package.

For example, the following files are provided by the **GD32VF103** device family pack:

Table 1: Files provided by GD32VF103 device family pack

File	Description
./Device/Source/GCC/startup_gd32vf103.S	Startup File startup_<device>.S for the GD32VF103 device variants.
./Device/Source/GCC/intexc_gd32vf103.S	Exception and Interrupt Handling File intexc_<device>.S for the GD32VF103 device variants.
./Device/Source/GCC/gcc_gd32vf103.ld	Linker script File gcc_<device>.ld for the GD32VF103 device variants.
./Device/Source/system_gd32vf103.c	System Configuration File system_<device>.c for the GD32VF103 device families
./Device/Include/system_gd32vf103.h	System Configuration File system_<device>.h for the GD32VF103 device families
./Device/Include/gd32vf103.h	Device Header File <device>.h for the GD32VF103 device families.

Note

The silicon vendors create these device-specific NMSIS-Core files based on *NMSIS-Core Device Templates* (page 12) provided by Nuclei.

Thereafter, the functions described under *NMSIS Core API* (page 71) can be used in the application.

2.2.2 Basic NMSIS Example

A typical example for using the NMSIS layer is provided below. The example is based on a GD32VF103 Device.

Listing 1: gd32vf103_example.c

```

1  #include <gd32vf103.h>                                // File name depends on device used
2
3  uint32_t volatile msTicks;                             // Counter for millisecond Interval
4  #define SysTick_Handler    eclic_mtip_handler
5  #define CONFIG_TICKS      (SOC_TIMER_FREQ / 1000)
6
7  void SysTick_Handler (void) {                          // SysTick Interrupt Handler
8      SysTick_Reload(CONFIG_TICKS);
9      msTicks++;                                         // Increment Counter
10 }
11
12 void WaitForTick (void) {
13     uint32_t curTicks;
14
15     curTicks = msTicks;                                // Save Current SysTick Value
16     while (msTicks == curTicks) {                      // Wait for next SysTick Interrupt
17         __WFI ();                                     // Power-Down until next Event/Interrupt
18     }
19 }
20
21 void TIMER0_UP_IRQHandler (void) {                     // Timer Interrupt Handler
22     ;                                                  // Add user code here
23 }
24
25 void timer0_init(int frequency) {                      // Set up Timer (device specific)
26     ECLIC_SetPriorityIRQ (TIMER0_UP_IRQn, 1);          // Set Timer priority
27     ECLIC_EnableIRQ (TIMER0_UP_IRQn);                  // Enable Timer Interrupt
28 }
29
30
31 void Device_Initialization (void) {                    // Configure & Initialize MCU
32     if (SysTick_Config (CONFIG_TICKS)) {
33         ; // Handle Error
34     }
35     timer0_init ();                                   // setup device-specific timer
36 }
37
38 // The processor clock is initialized by NMSIS startup + system file
39 void main (void) {                                     // user application starts here
40     Device_Initialization ();                          // Configure & Initialize MCU
41     while (1) {                                        // Endless Loop (the Super-Loop)
42         __disable_irq ();                             // Disable all interrupts
43         Get_InputValues ();                            // Read Values
44         __enable_irq ();                              // Enable all interrupts
45         Calculation_Response ();                       // Calculate Results
46         Output_Response ();                           // Output Results
47         WaitForTick ();                               // Synchronize to SysTick Timer

```

(continues on next page)

(continued from previous page)

```

48     }
49 }

```

2.2.3 Using Interrupt and Exception/NMI

Nuclei processors provide **NMI(Non-Maskable Interrupt)**, **Exception**, **Vector Interrupt** and **Non-Vector Interrupt** features.

2.2.4 Using NMSIS with generic Nuclei Processors

Nuclei provides NMSIS-Core Device template files for the supported Nuclei Processors and for various compiler vendors. These files can be used as templates, and you can modify based on it to match your processor design.

And you can also refer to [Nuclei SDK¹⁶](#) project for an quick and easily startup project to work with Nuclei RISC-V processor, it is based on NMSIS project and support build/debug c/c++ project in both command line or many different IDEs such as Nuclei Studio.

The table below lists the template folder.

Table 2: NMSIS Device Templates for the Nuclei processors

Folder	Processor	RISC-V	Description
./Device/_Template_Vendor	• 200 • 300 • 600 • 900	RV32 RV64	Contains Include and Source template files configured for the Nuclei RISC-V processor.

2.2.5 Create generic Libraries with NMSIS

The NMSIS Processor and Core Peripheral files allow also to create generic libraries. The **NMSIS-DSP** Libraries are an example for such a generic library.

To build a generic library set the define `__NMSIS_GENERIC` and include the `nmsis_core.h` NMSIS CPU & Core Access header file for the processor.

The define `__NMSIS_GENERIC` disables device-dependent features such as the **SysTick timer** and the **Interrupt System**.

¹⁶ <https://github.com/Nuclei-Software/nuclei-sdk>

Example

The following code section shows the usage of the *nmsis_core.h* header files to build a generic library for N200, N300, N600, NX600, UX600, N900, NX900, UX900, UX1000.

One of these defines needs to be provided on the compiler command line.

By using this header file, the source code can access the functions for *Core CSR Register Access* (page 75), *CPU Intrinsic Functions* (page 198) and *Intrinsic Functions for SIMD Instructions* (page 211).

Listing 2: core_generic.h

```
1 #define __NMSIS_GENERIC    // Disable Eclic and Systick functions
2 #include <nmsis_core.h>
```

2.3 NMSIS-Core Device Templates

2.3.1 Introduction

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of *evalsoc* in the *Nuclei SDK*¹⁷ for the latest reference template. This reference template may not be actively maintained in the future.

Nuclei supplies NMSIS-Core device template files for the all supported Nuclei N/NX/UX Class Processors and various compiler vendors. Refer to the list of *supported toolchain* (page 6) for compliance.

These NMSIS-Core device template files include the following:

- Register names of the Core Peripherals and names of the Core Exception/Interrupt Vectors.
- Functions to access core peripherals, special CPU instructions and SIMD instructions
- Generic startup code and system configuration code.

The detailed file structure of the NMSIS-Core device templates is shown in the following picture.

Nuclei SDK is an open source Nuclei RISC-V processor software development kit which is based on NMSIS project, using NMSIS Core/DSP/NN header files and prebuilt NMSIS DSP/NN library, and also create Nuclei FPGA evaluation SoC called evalsoc support package based on NMSIS device template, if you want to know how to use NMSIS, it is a good startup.

¹⁷ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

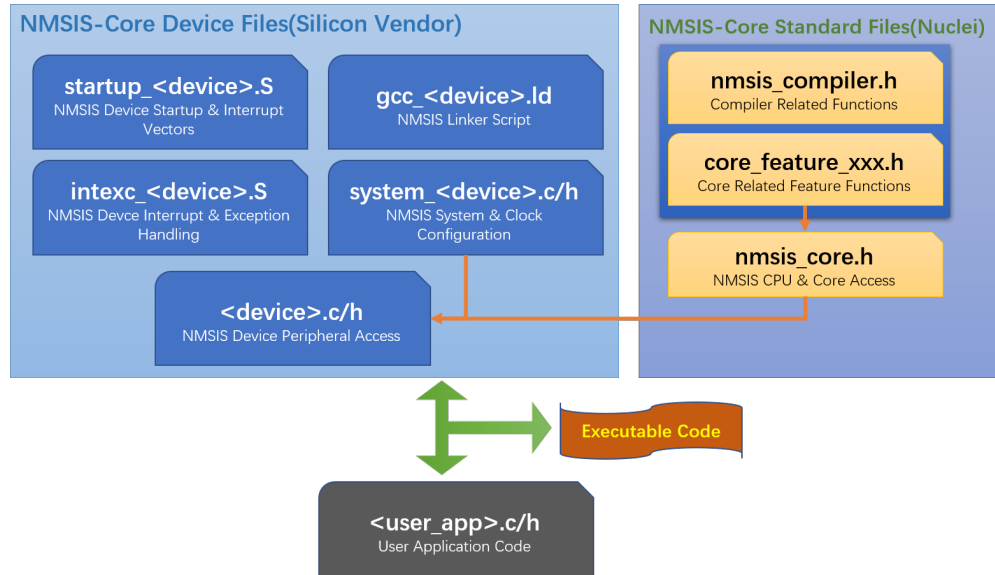


Fig. 2: NMSIS-Core Device Templates

2.3.2 NMSIS-Core Processor Files

The NMSIS-Core processor files provided by Nuclei are in the directory *NMSIS/Core/Include*.

These header files define all processor specific attributes do not need any modifications.

The *nmsis_core.h* defines the core peripherals and provides helper functions that access the core registers.

2.3.3 Device Examples

The NMSIS Software Pack defines several devices that are based on the Nuclei N/NX/U/UX processors.

The device related NMSIS-Core files are in the directory *Device/Nuclei*, but it is removed since NMSIS 1.2.0 release, and please directly refer to the **Template Files** or Nuclei SDK project's SoC/evalsoc support code.

2.3.4 Template Files

To simplify the creation of NMSIS-Core device files, the following template files are provided that should be extended by the silicon vendor to reflect the actual device and device peripherals.

Silicon vendors add to these template files the following information:

- **Device Peripheral Access Layer** that provides definitions for device-specific peripherals.
- **Access Functions for Peripherals** (optional) that provides additional helper functions to access device-specific peripherals.
- **Interrupt vectors** in the startup file that are device specific.

Table 3: NMSIS-Core Device Template Files

Template File (Under <code>./Device/_Template_Vendor/Vendor/</code>)	Description
<code>Device/Source/GCC/startup_Device.S</code>	Startup file template for GCC/Clang RISC-V Embedded Compiler.
<code>Device/Source/GCC/gcc_Device.ld</code>	Link Script file template for GCC/Clang RISC-V Embedded Compiler.
<code>Device/Source/GCC/intexc_Device.S</code>	Exception and Interrupt handling file template(machine mode) for GCC/Clang RISC-V Embedded Compiler.
<code>Device/Source/GCC/intexc_Device_s.S</code>	Exception and Interrupt handling file template(supervisor mode) for GCC/Clang RISC-V Embedded Compiler.
<code>Device/Source/IAR/startup_Device.c</code>	Startup file template for IAR RISC-V Compiler.
<code>Device/Source/IAR/iar_Device.icf</code>	Link Script file template for IAR RISC-V Compiler.
<code>Device/Source/IAR/intexc_Device.S</code>	Exception and Interrupt handling file template(machine mode) for IAR RISC-V Compiler.
<code>Device/Source/IAR/intexc_Device_s.S</code>	Exception and Interrupt handling file template(supervisor mode) for IAR RISC-V Compiler.
<code>Device/Source/system_Device.c</code>	Generic <code>system_Device.c</code> file for system configuration (i.e. processor clock and memory bus system).
<code>Device/Include/Device.h</code>	Generic device header file. Needs to be extended with the device-specific peripheral registers. Optionally functions that access the peripherals can be part of that file.
<code>Device/Include/system_Device.h</code>	Generic system device configuration include file.

Note

The template files for silicon vendors are placed under `./Device/_Template_Vendor/Vendor/`.
Please goto that folder to find the file list in the above table.

2.3.5 Adapt Template Files to a Device

The following steps describe how to adopt the template files to a specific device or device family.

Copy the complete all files in the template directory and replace:

- directory name `Vendor` with the abbreviation for the device vendor e.g.: **GD**.
- directory name `Device` with the specific device name e.g.: **GD32VF103**.
- in the file names `Device` with the specific device name e.g.: **GD32VF103**.

Each template file contains comments that start with **TODO**: that describe a required modification.

The template files contain place holders:

Table 4: Placeholders of Template files

Placeholder	Replaced with
<code><Device></code>	the specific device name or device family name; i.e. GD32VF103.
<code><DeviceInterrupt></code>	a specific interrupt name of the device; i.e. TIM1 for Timer 1.
<code><DeviceAbbreviation></code>	short name or abbreviation of the device family; i.e. GD32VF.
<code>Nuclei-N#</code>	the specific Nuclei Class name; i.e. Nuclei N or Nuclei NX.

2.3.6 Device Templates Explanation

Caution

The template files and its explanation maybe out of date, please directly take [evalsoc in the Nuclei SDK¹⁸](#) support source code as reference device templates and read the source code and comment inside it.

The device configuration of the template files is described in detail on the following pages:

We only explain machine mode intexc_Device.S template, its supervisor mode version is similar, please directly check the code, and for the IAR device templates, the flow is also similar, but it reused startup code provided in IAR compiler, and implement Nuclei dependent boot code in `__low_level_init` function.

Startup File startup_<Device>.S

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of [evalsoc in the Nuclei SDK¹⁹](#) for the latest reference template. This reference template may not be actively maintained in the future.

The Startup File startup_<device>.S contains:

- The reset handler which is executed after CPU reset and typically calls the [SystemInit\(\)](#) (page 805) function.
- The setup values for the stack pointer SP and global pointor GP for small data access.
- Exception vectors of the Nuclei Processor with weak functions that implement default routines.
- Interrupt vectors that are device specific with weak functions that implement default routines.

The processor level start flow is implemented in the *startup_<Device>.S*. Detail description as below picture:

The IAR version of startup code located in *startup_<Device>.c*.

Stage1: Interrupt and Exception initialization

- Disable Interrupt
- Initialize GP, SP for single core or smp core if existed
- Initialize NMI entry and set default NMI handler
- Initialize exception entry to early exception entry in *startup_<Device>.S*
- Initialize vector table entry and set default interrupt handler
- Initialize Interrupt mode as ECLIC mode. (ECLIC mode is proposed. Default mode is CLINT mode)

Stage2: Hardware initialization

- Enable FPU if necessary
- Enable VPU if necessary
- Enable Zc if necessary

¹⁸ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

¹⁹ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

Stage3: Section initialization

- Copy section, e.g. data section, text section if necessary.
- Clear Block Started by Symbol (BSS) section
- Call user defined `SystemInit()` (page 805) for system clock initialization.
- Call `__libc_fini_array` and `__libc_init_array` functions to do C library initialization
- Call `_premain_init` function to do initialization steps before main function
- Initialize exception entry to exception entry in `intexc_<Device>.S`
- Enable BPU of Nuclei CPU
- Jump Main

The file exists for each supported toolchain and is the only toolchain specific NMSIS file.

To adapt the file to a new device only the interrupt vector table needs to be extended with the device-specific interrupt handlers.

The naming convention for the interrupt handler names are `eclic_<interrupt_name>_handler`.

This table needs to be consistent with *IRQn_Type* (page 636) that defines all the IRQ numbers for each interrupt.

The following example shows the extension of the interrupt vector table for the GD32VF103 device family.

```

1  .section .text.vtable
2
3  .weak  eclic_msip_handler
4  .weak  eclic_mtip_handler
5  .weak  eclic_pmaf_handler
6  /* Adjusted for GD32VF103 interrupt handlers */
7  .weak  eclic_wwdgt_handler
8  .weak  eclic_lvd_handler
9  .weak  eclic_tamper_handler
10 :      :
11 :      :
12 .weak  eclic_can1_ewmc_handler
13 .weak  eclic_usbfs_handler
14
15 .globl vector_base
16 .type vector_base, @object
17 vector_base:
18     /* Run in FlashXIP download mode */
19     j _start                                /* 0: Reserved, Jump to _
↳start when reset for vector table not remapped cases.*/
20     .align LOG_REGBYTES                    /*    Need to align 4 byte
↳for RV32, 8 Byte for RV64 */
21     DECLARE_INT_HANDLER    default_intexc_handler    /* 1: Reserved */
22     DECLARE_INT_HANDLER    default_intexc_handler    /* 2: Reserved */
23     DECLARE_INT_HANDLER    eclic_msip_handler        /* 3: Machine software
↳interrupt */
24         :      :
25         :      :
26     /* Adjusted for Vendor Defined External Interrupts */
27     DECLARE_INT_HANDLER    eclic_wwdgt_handler        /* 19: Window watchDog timer
↳interrupt */

```

(continues on next page)

(continued from previous page)

```

28
29     DECLARE_INT_HANDLER    eclic_lvd_handler          /* 20: LVD through EXTI line
↪detect interrupt */
30     DECLARE_INT_HANDLER    eclic_tamper_handler        /* 21: tamper through EXTI
↪line detect */
31         :
32         :
33     DECLARE_INT_HANDLER    eclic_can1_ewmc_handler     /* 85: CAN1 EWMC interrupt */
34     DECLARE_INT_HANDLER    eclic_usbfs_handler         /* 86: USBFS global
↪interrupt */

```

startup_Device.S Template File

Here provided a riscv-gcc template startup assemble code template file as below. The files for other compilers can slightly differ from this version.

```

1  /*
2  * Copyright (c) 2019 Nuclei Limited. All rights reserved.
3  *
4  * SPDX-License-Identifier: Apache-2.0
5  *
6  * Licensed under the Apache License, Version 2.0 (the License); you may
7  * not use this file except in compliance with the License.
8  * You may obtain a copy of the License at
9  *
10 * www.apache.org/licenses/LICENSE-2.0
11 *
12 * Unless required by applicable law or agreed to in writing, software
13 * distributed under the License is distributed on an AS IS BASIS, WITHOUT
14 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15 * See the License for the specific language governing permissions and
16 * limitations under the License.
17 */
18 /*****
19 * \file      startup_<Device>.S
20 * \brief     NMSIS Nuclei N/NX Class Core based Core Device Startup File for
21 * Nuclei Eval SoC which support Nuclei N/NX class cores
22 * \version   V1.00
23 * \date      17. Dec 2019
24 *
25 *****/
26
27 /* Please check the TODO items */
28
29 #include "riscv_encoding.h"
30
31 /* TODO: Require Nuclei SDK >= 0.6.0, which introduced this cpufeature.h */
32 #include "cpufeature.h"
33
34 /* If BOOT_HARTID is not defined, default value is 0 */
35 #ifndef BOOT_HARTID

```

(continues on next page)

(continued from previous page)

```

36     .equ BOOT_HARTID,    0
37 #endif
38
39 .macro DECLARE_INT_HANDLER INT_HDL_NAME
40 #if defined(__riscv_xlen) && (__riscv_xlen == 32)
41     .word INT_HDL_NAME
42 #else
43     .dword INT_HDL_NAME
44 #endif
45 .endm
46
47 #ifdef CFG_HAS_CLIC
48     .section .text.vtable
49
50     .weak eclic_msip_handler
51     .weak eclic_mtip_handler
52     .weak eclic_uart0_int_handler
53     .weak eclic_inter_core_int_handler
54     .globl vector_base
55     .type vector_base, @object
56     .option push
57     .option norelax
58 vector_base:
59 #ifndef VECTOR_TABLE_REMAPPED
60     j _start                                /* 0: Reserved, Jump to _
61     ↪ start when reset for vector table not remapped cases.*/
62     .align LOG_REGBYTES                    /*    Need to align 4 byte
63     ↪ for RV32, 8 Byte for RV64 */
64 #else
65     DECLARE_INT_HANDLER    default_intexc_handler    /* 0: Reserved, default
66     ↪ handler for vector table remapped cases */
67 #endif
68     DECLARE_INT_HANDLER    default_intexc_handler    /* 1: Reserved */
69     DECLARE_INT_HANDLER    default_intexc_handler    /* 2: Reserved */
70     DECLARE_INT_HANDLER    eclic_msip_handler        /* 3: Machine software
71     ↪ interrupt */
72     DECLARE_INT_HANDLER    default_intexc_handler    /* 4: Reserved */
73     DECLARE_INT_HANDLER    default_intexc_handler    /* 5: Reserved */
74     DECLARE_INT_HANDLER    default_intexc_handler    /* 6: Reserved */
75     DECLARE_INT_HANDLER    eclic_mtip_handler        /* 7: Machine timer
76     ↪ interrupt */
77     DECLARE_INT_HANDLER    default_intexc_handler    /* 8: Reserved */
78     DECLARE_INT_HANDLER    default_intexc_handler    /* 9: Reserved */
79     DECLARE_INT_HANDLER    default_intexc_handler    /* 10: Reserved */
80     DECLARE_INT_HANDLER    default_intexc_handler    /* 11: Reserved */
81     DECLARE_INT_HANDLER    default_intexc_handler    /* 12: Reserved */
82     DECLARE_INT_HANDLER    default_intexc_handler    /* 13: Reserved */
83     DECLARE_INT_HANDLER    default_intexc_handler    /* 14: Reserved */
84     DECLARE_INT_HANDLER    default_intexc_handler    /* 15: Reserved */

```

(continues on next page)

(continued from previous page)

```

83 DECLARE_INT_HANDLER    eclic_inter_core_int_handler    /* 16: Reserved */
84 DECLARE_INT_HANDLER    default_intexc_handler         /* 17: Reserved */
85 DECLARE_INT_HANDLER    default_intexc_handler         /* 18: Reserved */
86 /* TODO Below are external interrupt handlers, please define them as your
87 requirements, you need to increase or decrease it, and define correct interrupt
handler name */
88 DECLARE_INT_HANDLER    default_intexc_handler         /* 19: Interrupt 19 */
89
90 DECLARE_INT_HANDLER    default_intexc_handler         /* 20: Interrupt 20 */
91 DECLARE_INT_HANDLER    default_intexc_handler         /* 21: Interrupt 21 */
92 DECLARE_INT_HANDLER    default_intexc_handler         /* 22: Interrupt 22 */
93 DECLARE_INT_HANDLER    default_intexc_handler         /* 23: Interrupt 23 */
94
95 DECLARE_INT_HANDLER    default_intexc_handler         /* 24: Interrupt 24 */
96 DECLARE_INT_HANDLER    default_intexc_handler         /* 25: Interrupt 25 */
97 DECLARE_INT_HANDLER    default_intexc_handler         /* 26: Interrupt 26 */
98 DECLARE_INT_HANDLER    default_intexc_handler         /* 27: Interrupt 27 */
99
100 DECLARE_INT_HANDLER    default_intexc_handler         /* 28: Interrupt 28 */
101 DECLARE_INT_HANDLER    default_intexc_handler         /* 29: Interrupt 29 */
102 DECLARE_INT_HANDLER    default_intexc_handler         /* 30: Interrupt 30 */
103 DECLARE_INT_HANDLER    default_intexc_handler         /* 31: Interrupt 31 */
104
105 DECLARE_INT_HANDLER    default_intexc_handler         /* 32: Interrupt 32 */
106 DECLARE_INT_HANDLER    default_intexc_handler         /* 33: Interrupt 33 */
107 DECLARE_INT_HANDLER    default_intexc_handler         /* 34: Interrupt 34 */
108 DECLARE_INT_HANDLER    default_intexc_handler         /* 35: Interrupt 35 */
109
110 DECLARE_INT_HANDLER    default_intexc_handler         /* 36: Interrupt 36 */
111 DECLARE_INT_HANDLER    default_intexc_handler         /* 37: Interrupt 37 */
112 DECLARE_INT_HANDLER    default_intexc_handler         /* 38: Interrupt 38 */
113 DECLARE_INT_HANDLER    default_intexc_handler         /* 39: Interrupt 39 */
114
115 DECLARE_INT_HANDLER    default_intexc_handler         /* 40: Interrupt 40 */
116 DECLARE_INT_HANDLER    default_intexc_handler         /* 41: Interrupt 41 */
117 DECLARE_INT_HANDLER    default_intexc_handler         /* 42: Interrupt 42 */
118 DECLARE_INT_HANDLER    default_intexc_handler         /* 43: Interrupt 43 */
119
120 DECLARE_INT_HANDLER    default_intexc_handler         /* 44: Interrupt 44 */
121 DECLARE_INT_HANDLER    default_intexc_handler         /* 45: Interrupt 45 */
122 DECLARE_INT_HANDLER    default_intexc_handler         /* 46: Interrupt 46 */
123 DECLARE_INT_HANDLER    default_intexc_handler         /* 47: Interrupt 47 */
124
125 DECLARE_INT_HANDLER    default_intexc_handler         /* 48: Interrupt 48 */
126 DECLARE_INT_HANDLER    default_intexc_handler         /* 49: Interrupt 49 */
127 DECLARE_INT_HANDLER    default_intexc_handler         /* 50: Interrupt 50 */
128 DECLARE_INT_HANDLER    eclic_uart0_int_handler        /* 51: Interrupt 51 */
129
130 DECLARE_INT_HANDLER    default_intexc_handler         /* 52: Interrupt 52 */
131 DECLARE_INT_HANDLER    default_intexc_handler         /* 53: Interrupt 53 */
132 DECLARE_INT_HANDLER    default_intexc_handler         /* 54: Interrupt 54 */

```

(continues on next page)

(continued from previous page)

```

133 DECLARE_INT_HANDLER    default_intexc_handler    /* 55: Interrupt 55 */
134
135 DECLARE_INT_HANDLER    default_intexc_handler    /* 56: Interrupt 56 */
136 DECLARE_INT_HANDLER    default_intexc_handler    /* 57: Interrupt 57 */
137 DECLARE_INT_HANDLER    default_intexc_handler    /* 58: Interrupt 58 */
138 DECLARE_INT_HANDLER    default_intexc_handler    /* 59: Interrupt 59 */
139
140 DECLARE_INT_HANDLER    default_intexc_handler    /* 60: Interrupt 60 */
141 DECLARE_INT_HANDLER    default_intexc_handler    /* 61: Interrupt 61 */
142 DECLARE_INT_HANDLER    default_intexc_handler    /* 62: Interrupt 62 */
143 DECLARE_INT_HANDLER    default_intexc_handler    /* 63: Interrupt 63 */
144
145 .option pop
146
147 .size vector_base, . - vector_base
148 #endif
149
150 .section .text.init
151 .globl _start
152 .type _start, @function
153
154 /**
155  * Reset Handler called on controller reset
156  */
157 _start:
158     /* ===== Startup Stage 1 ===== */
159     /* Disable Global Interrupt */
160     csrr CSR_MSTATUS, MSTATUS_MIE
161
162     /* If SMP_CPU_CNT is not defined,
163      * assume that only 1 core is allowed to run,
164      * the core hartid is defined via BOOT_HARTID.
165      * other harts if run to here, just do wfi in __amp_wait
166      */
167 #ifndef SMP_CPU_CNT
168     /* take bit 0-7 for hart id in a local cluster */
169     csrr a0, CSR_MHARTID
170     andi a0, a0, 0xFF
171     /* BOOT_HARTID is configurable in Makefile via BOOT_HARTID variable */
172     li a1, BOOT_HARTID
173     bne a0, a1, __amp_wait
174 #endif
175
176     /* Initialize GP and TP and jump table base when zcmt enabled */
177     .option push
178     .option norelax
179     la gp, __global_pointer$
180     la tp, __tls_base
181 #if defined(__riscv_zcmt)
182     la t0, __jvt_base$
183     csrw CSR_JVT, t0
184 #endif

```

(continues on next page)

(continued from previous page)

```

185     .option pop
186
187     /* TODO if don't have SMP, you can remove the SMP_CPU_CNT related code */
188     #if defined(SMP_CPU_CNT) && (SMP_CPU_CNT > 1)
189         /* Set correct sp for each cpu
190          * each stack size is __STACK_SIZE
191          * defined in linker script */
192         lui t0, %hi(__STACK_SIZE)
193         addi t0, t0, %lo(__STACK_SIZE)
194         la sp, _sp
195         csrr a0, CSR_MHARTID
196         andi a0, a0, 0xFF
197         li a1, 0
198     1:
199         beq a0, a1, 2f
200         sub sp, sp, t0
201         addi a1, a1, 1
202         j 1b
203     2:
204     #else
205         /* Set correct sp for current cpu */
206         la sp, _sp
207     #endif
208
209     /*
210      * Set the the NMI base mnvec to share
211      * with mtvec by setting CSR_MMISC_CTL
212      * bit 9 NMI_CAUSE_FFF to 1
213      */
214     li t0, MMISC_CTL_NMI_CAUSE_FFF
215     csrs CSR_MMISC_CTL, t0
216
217     /*
218      * Enable Zc feature when compiled zcmp & zcmt
219      */
220     li t0, MMISC_CTL_ZC
221     #if defined(__riscv_zcmp) || defined(__riscv_zcmt)
222         csrs CSR_MMISC_CTL, t0
223     #else
224         csrc CSR_MMISC_CTL, t0
225     #endif
226
227     /*
228      * Set Exception Entry MTVEC to early_exc_entry
229      * Due to settings above, Exception and NMI
230      * will share common entry.
231      * This early_exc_entry is only used during early
232      * boot stage before main
233      * Set default interrupt mode to CLINT interrupt mode
234      */
235     la t0, early_exc_entry
236     csrw CSR_MTVEC, t0

```

(continues on next page)

(continued from previous page)

```

237
238     /* ===== Startup Stage 2 ===== */
239
240     /* Enable FPU and Vector Unit if f/d/v exist in march */
241     #if defined(__riscv_flen) && __riscv_flen > 0
242         /* Enable FPU, and set state to initial */
243         li t0, MSTATUS_FS
244         csrc mstatus, t0
245         li t0, MSTATUS_FS_INITIAL
246         csrs mstatus, t0
247     #endif
248
249     #if defined(__riscv_vector)
250         /* Enable Vector, and set state to initial */
251         li t0, MSTATUS_VS
252         csrc mstatus, t0
253         li t0, MSTATUS_VS_INITIAL
254         csrs mstatus, t0
255     #endif
256
257     /* TODO: Enable I/D Cache if present determined by cpufeature.h */
258     /* This should be only used by nuclei_gen which generate a correct cpufeature.h */
259     /* We use CPU_ISA macro to determine whether this cpufeature.h is generated or hand-
    ↪written */
260     /* This is used to speedup data loading */
261     #ifndef CPU_ISA
262     #ifndef CFG_HAS_ECC
263         /* Only enable i/dcache when ecc not present */
264         #ifndef CFG_HAS_ICACHE
265             csrsi CSR_MCACHE_CTL, MCACHE_CTL_IC_EN
266         #endif
267         #ifndef CFG_HAS_DCACHE
268             li t0, MCACHE_CTL_DC_EN
269             csrs CSR_MCACHE_CTL, t0
270         #endif
271     #endif
272     #endif
273
274     /* Enable mcycle and minstret counter */
275     csrci CSR_MCOUNTINHIBIT, 0x5
276
277     #if defined(SMP_CPU_CNT) && (SMP_CPU_CNT > 1)
278         csrr a0, CSR_MHARTID
279         li a1, BOOT_HARTID
280         bne a0, a1, __skip_init
281     #endif
282
283     .size _start, . - _start
284
285     .type __init_common, @function
286     __init_common:
287         /* ===== Startup Stage 3 ===== */

```

(continues on next page)

(continued from previous page)

```

288  /*
289  * Load text section from CODE ROM to CODE RAM
290  * when text LMA is different with VMA
291  */
292  la a0, _text_lma
293  la a1, _text
294  /* If text LMA and VMA are equal
295  * then no need to copy text section */
296  beq a0, a1, 2f
297  la a2, _etext
298  bgeu a1, a2, 2f
299
300  1:
301  /* Load code section if necessary */
302  lw t0, (a0)
303  sw t0, (a1)
304  addi a0, a0, 4
305  addi a1, a1, 4
306  bltu a1, a2, 1b
307  2:
308  /* Load data section */
309  la a0, _data_lma
310  la a1, _data
311  /* If data vma=lma, no need to copy */
312  beq a0, a1, 2f
313  la a2, _edata
314  bgeu a1, a2, 2f
315  1:
316  lw t0, (a0)
317  sw t0, (a1)
318  addi a0, a0, 4
319  addi a1, a1, 4
320  bltu a1, a2, 1b
321  2:
322  /* Clear bss section */
323  la a0, __bss_start
324  la a1, _end
325  bgeu a0, a1, 2f
326  1:
327  sw zero, (a0)
328  addi a0, a0, 4
329  bltu a0, a1, 1b
330  2:
331
332  .size __init_common, . - __init_common
333
334  .globl _start_premain
335  .type _start_premain, @function
336  _start_premain:
337  /*
338  * Call vendor defined SystemInit to
339  * initialize the micro-controller system

```

(continues on next page)

(continued from previous page)

```

340     * SystemInit will just be called by boot cpu
341     */
342     call SystemInit
343
344     /*
345     * Call C/C++ constructor start up code,
346     * __libc_fini is defined in linker script,
347     * so register_fini function will be called
348     * and will run atexit (__libc_fini_array)
349     * to do previous call atexit function
350     */
351     call __libc_init_array
352
353     .size _start_premain, . - _start_premain
354
355     .type __skip_init, @function
356     __skip_init:
357         /* Sync all harts at this function */
358         call __sync_harts
359
360         /* do pre-init steps before main */
361         /* _premain_init will be called by each cpu
362         * please make sure the implementation of __premain_int
363         * considered this
364         * it will update mtvec according to eclic present or not
365         * mtvec will set to exc_entry
366         */
367         call _premain_init
368
369         /* BPU cold bringup need time, so enable BPU before enter to main */
370         li t0, MMISC_CTL_BPU
371         csrs CSR_MMISC_CTL, t0
372
373         // Interrupt is still disabled here
374         /* ===== Call SMP Main Function ===== */
375         /* argc = argv = 0 */
376         li a0, 0
377         li a1, 0
378         #if defined(SMP_CPU_CNT) && (SMP_CPU_CNT > 1)
379             /* The weak implementation of smp_main is in this file */
380             call smp_main
381         #else
382             #ifndef RTOS_RTTHREAD
383                 // Call entry function when using RT-Thread
384             #endif
385             #ifndef SMODE_RTOS
386                 call main_entry
387             #else
388                 call entry
389             #endif
390             #ifndef SMODE_RTOS
391                 call main
392             #endif

```

(continues on next page)

(continued from previous page)

```

392 #endif
393     /* do post-main steps after main
394      * this function will be called by each cpu */
395     call _postmain_fini
396
397     .size __skip_init, . - __skip_init
398
399 .type __amp_wait, @function
400 __amp_wait:
401 1:
402     wfi
403     j 1b
404
405     .size __amp_wait, . - __amp_wait
406
407 #if defined(SMP_CPU_CNT) && (SMP_CPU_CNT > 1)
408 /*
409  * You can re-implement smp_main function in your code
410  * to do smp boot process and handle multi harts
411  */
412 .weak smp_main
413 .type smp_main, @function
414 smp_main:
415     addi sp, sp, -2*REGBYTES
416     STORE ra, 0*REGBYTES(sp)
417     /* only boot hart goto main, other harts do wfi */
418     csrr t0, CSR_MHARTID
419     li t1, BOOT_HARTID
420     beq t0, t1, 2f
421 1:
422     wfi
423     j 1b
424 2:
425 #ifdef RTOS_RTTHREAD
426     // Call entry function when using RT-Thread
427 #ifdef SMODE_RTOS
428     call main_entry
429 #else
430     call entry
431 #endif
432 #else
433     call main
434 #endif
435     LOAD ra, 0*REGBYTES(sp)
436     addi sp, sp, 2*REGBYTES
437     ret
438
439     .size smp_main, . - smp_main
440 #endif
441
442 /* Early boot exception entry before main */
443 .align 6

```

(continues on next page)

(continued from previous page)

```

444 .global early_exc_entry
445 .type early_exc_entry, @function
446 early_exc_entry:
447     wfi
448     j early_exc_entry
449
450     .size early_exc_entry, . - early_exc_entry

```

Interrupt and Exception Handling File: intexc_<Device>.S

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of [evalsoc in the Nuclei SDK²⁰](#) for the latest reference template. This reference template may not be actively maintained in the future.

The intexc File intexc_<Device>.S contains:

- Macro to save caller register.
- Macro to restore caller register.
- Default Exception/NMI routine implementation.
- Default Non-Vector Interrupt routine implementation.

Nuclei processors provide **NMI(Non-Maskable Interrupt)**, **Exception**, **Vector Interrupt** and **Non-Vector Interrupt** features.

Note

To provide S-Mode interrupt and exception handling feature, we also provide a template file called intexc_<Device>_s.S

NMI(Non-Maskable Interrupt)

Click [NMI²¹](#) to learn about Nuclei Processor Core NMI in Nuclei ISA Spec.

NMI is used for urgent external HW error. It can't be masked and disabled.

When NMI happened, bit 9 of CSR `MMISIC_CTL` will be checked. If this bit value is 1, then NMI entry address will be the same as exception(`CSR_MTVEC`), and exception code for NMI will be 0xFFF, otherwise NMI entry will be same as `reset_vector`.

In NMSIS-Core, the bit 9 of CSR `MMISC_CTL` is set to 1 during core startup, so NMI will be treated as Exception and handled.

²⁰ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

²¹ https://doc.nucleisys.com/nuclei_spec/isa/nmi.html

Exception

Click [Exception²²](#) to learn about Nuclei Processor Core Exception in Nuclei ISA Spec.

For CPU exception, the entry for exception will be `exc_entry`, in this entry code, it will call default exception handler `core_exception_handler()` (page 812).

In the common exception routine(`exc_entry`) to get more information like exception code. Exception handle flow show as below picture:

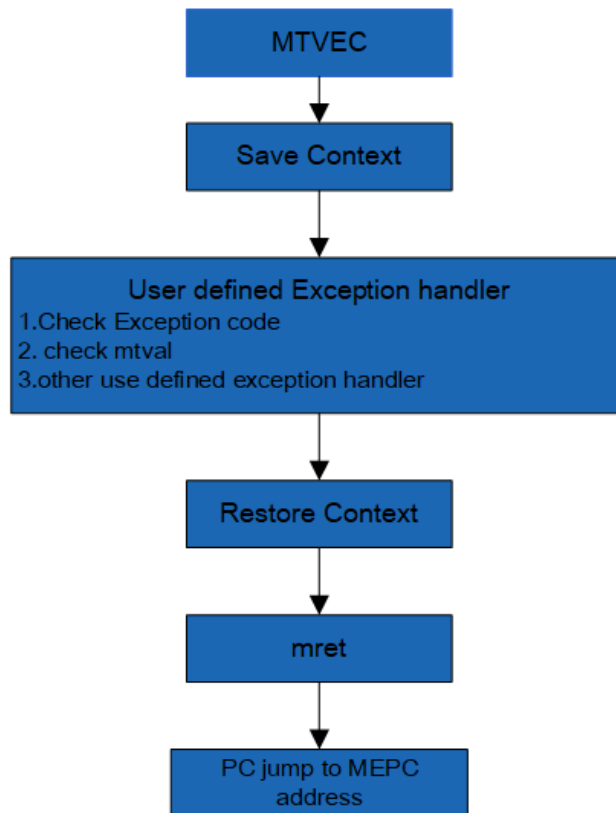


Fig. 3: Exception Handling Flow

NMI and exception could support nesting. Two levels of NMI/Exception state save stacks are supported.

We support three nesting mode as below:

- NMI nesting exception
- Exception nesting exception
- Exception nesting NMI

For software, we have provided the common entry for NMI and exception. Silicon vendor only need adapt the interface defined in *Interrupt Exception NMI Handling* (page 811).

Context save and restore have been handled by `exc_entry` interface.

When exception exception return it will run the instruction which trigger the exception again. It will cause software dead loop. So in the exception handler for each exception code, we propose to set CSR MEPC to be `MEPC+4`, then it will start from next instruction of MEPC.

²² https://doc.nucleisys.com/nuclei_spec/isa/exception.html

Interrupt

Click [Interrupt²³](#) to learn about Nuclei Processor Core Interrupt in Nuclei Spec.

Interrupt could be configured as **CLINT** mode or **ECILC** mode.

In NMSIS-Core, Interrupt has been configured as **ECLIC** mode during startup in *startup_<Devices>.S*, which is also recommended setting using Nuclei Processors.

ECLIC managed interrupt could be configured as **vector** and **non-vector** mode.

Detail interrupt handling process as below picture:

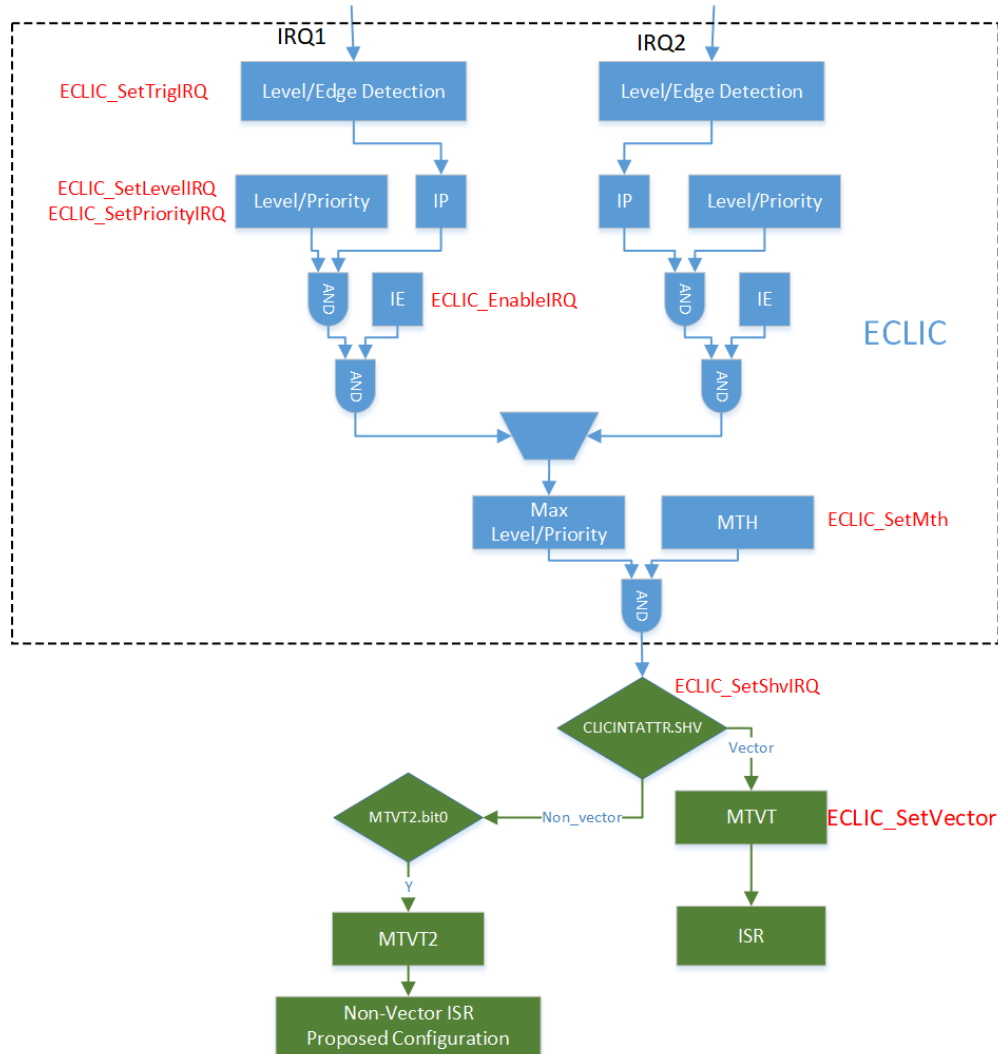


Fig. 4: Interrupt Handling Flow

To get highest priority interrupt we need compare the interrupt level first. If level is the same then compare the priority. High level interrupt could interrupt low level ISR and trigger interrupt nesting. If different priority with same level interrupt pending higher priority will be served first. Interrupt could be configured as vector mode and non-vector mode by vendor. For non-vector mode interrupt handler entry get from MTVT2 and exception/NMI handler entry get

²³ https://doc.nucleisys.com/nuclei_spec/isa/interrupt.html

from MTVVEC. If Vendor need set non vector mode interrupt handler entry from MTVVEC you need set MTVT2.BIT0 as 0.

Non-Vector Interrupt SW handler

For **non-vector** mode interrupt it will make the necessary CSR registers and context save and restore. Non-vector mode software handle flow show as below pciture:

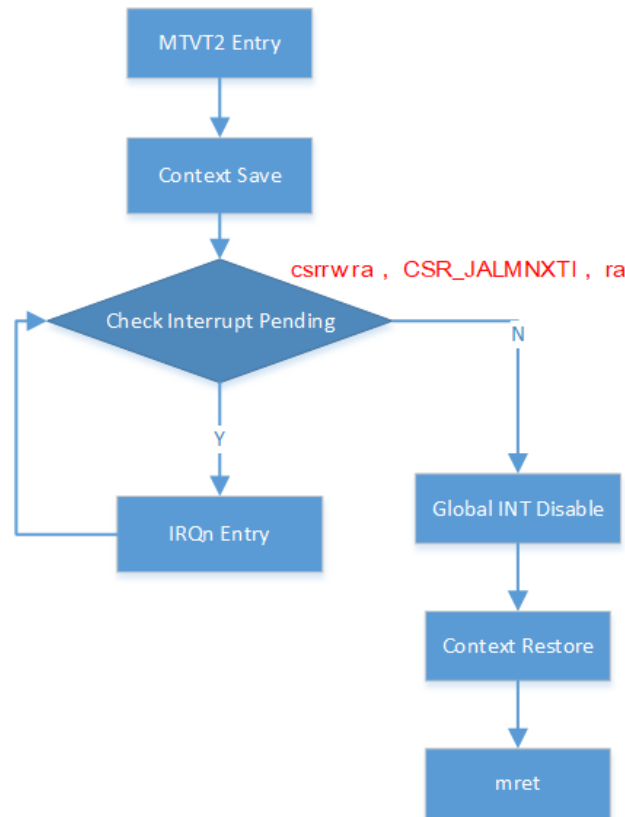


Fig. 5: Non-vector mode interrupt software handle flow

Detail description for non-vector mode interrupt handler as below steps:

1. Get non-vector mode handler entry from MTVT2 if MTVT2.BIT0 is 1(proposed configuration).
2. Context save to stack for cpu registers.
3. Save CSR registers MEPC/MCAUSE/MSUBM to stack.
4. Run instruction `csrrw ra, CSR_JALMNXTI, ra`. It will enable interrupt, check interrupt pending. If interrupt is pending then get highest priority interrupt and jump to interrupt handler entry in the vector table, otherwise it will go to step 6.
5. Execute the interrupt handler routine, when return from isr routine it will jump to step 4.
6. Global interrupt disable.
7. Restore CSR registers MEPC/MCAUSE/MSUBM.
8. Context restore from stack for cpu registers.
9. Execute `mret` to return from handler.

For **non-vector** mode interrupt it could support **interrupt nesting**.

Interrupt nesting handle flow show as below picture:

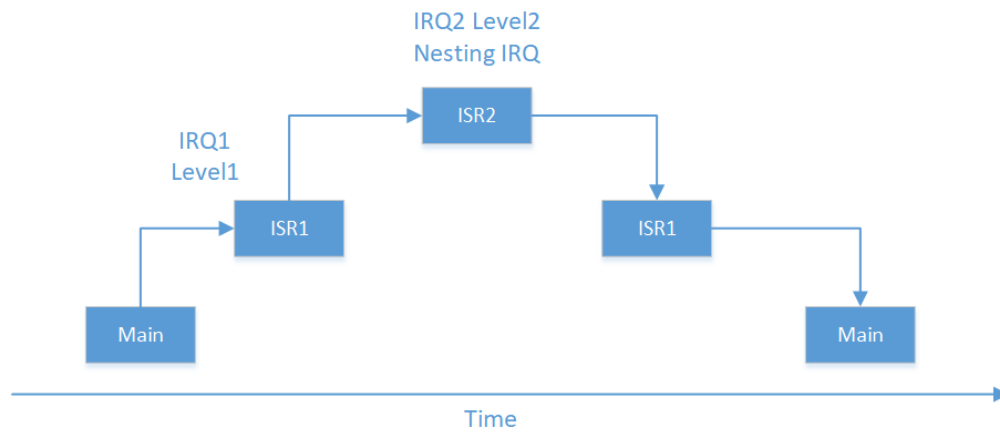


Fig. 6: Nesting interrupt handling flow

Vector Interrupt SW handler

If vector interrupt handler need support nesting or making function call Vector mode software handling flow show as below picture:

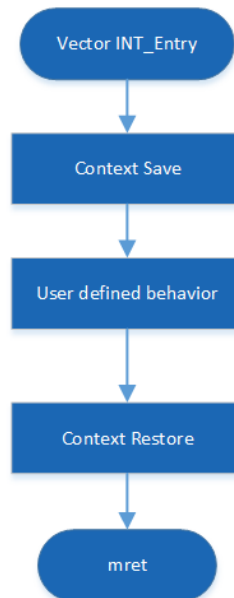


Fig. 7: Vector mode nesting interrupt handling flow

Detail description for nested vector mode interrupt handler as below steps:

1. Get vector mode handler from address of vector table entry MTVT added offset.
2. Context save to stack for cpu registers, done in each vector interrupt handler via `__INTERRUPT` (page 74)

3. Save CSR registers MEPC/MCAUSE/MSUBM to stack, done in each vector interrupt handler by read and save these CSRs into variables.
4. Execute the interrupt handling.
5. Restore CSR registers MEPC/MCAUSE/MSUBM from stack.
6. CSR registers restore from saved variables used in step 3.
7. Execute `mret` to return from handler

Here is sample code for above nested vector interrupt handling process:

```

1 // Vector interrupt handler for on-board button
2 __INTERUPT void SOC_BUTTON_1_HANDLER(void)
3 {
4     // save mepc,mcause,msubm enable interrupts
5     SAVE_IRQ_CSR_CONTEXT();
6
7     printf("%s", "----Begin button1 handler----Vector mode\r\n");
8
9     // Green LED toggle
10    gpio_toggle(GPIO, SOC_LED_GREEN_GPIO_MASK);
11
12    // Clear the GPIO Pending interrupt by writing 1.
13    gpio_clear_interrupt(GPIO, SOC_BUTTON_1_GPIO_OFS, GPIO_INT_RISE);
14
15    wait_seconds(1); // Wait for a while
16
17    printf("%s", "----End button1 handler\r\n");
18
19    // disable interrupts,restore mepc,mcause,msubm
20    RESTORE_IRQ_CSR_CONTEXT();
21 }

```

Detail description for non-nested vector mode interrupt handler as below

To improve the software response latency for vector mode vendor could remove context save/restore and **MEPC/MCAUSE/MSUBM** save/restore.

If so vector mode interrupt will not support nesting and interrupt handler can only be a leaf function which doesn't make any function calls.

Then the vector mode interrupt software flow will be described as below:

1. Get vector mode handler from address of vector table entry MTVT added offset.
2. Execute the interrupt handler(leaf function).
3. Execute `mret` to return from handler

Here is sample code for above non-nested vector interrupt handler which is a leaf function handling process:

```

1 static uint32_t btn_pressed = 0;
2 // Vector interrupt handler for on-board button
3 // This function is an leaf function, no function call is allowed
4 __INTERUPT void SOC_BUTTON_1_HANDLER(void)
5 {
6     btn_pressed ++;
7 }

```

intexc_Device.S Template File

The file exists for each supported toolchain and is the only toolchain specific NMSIS file.

Normally this file needn't adapt for different device. If CPU CSR registers have done some changes you may need some adaption.

Here we provided intexc_Device.S template file as below:

```

1  /*
2  * Copyright (c) 2019 Nuclei Limited. All rights reserved.
3  *
4  * SPDX-License-Identifier: Apache-2.0
5  *
6  * Licensed under the Apache License, Version 2.0 (the License); you may
7  * not use this file except in compliance with the License.
8  * You may obtain a copy of the License at
9  *
10 * www.apache.org/licenses/LICENSE-2.0
11 *
12 * Unless required by applicable law or agreed to in writing, software
13 * distributed under the License is distributed on an AS IS BASIS, WITHOUT
14 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15 * See the License for the specific language governing permissions and
16 * limitations under the License.
17 */
18 /*****
19 * \file      intexc_evalsoc.S
20 * \brief     NMSIS Interrupt and Exception Handling Template File
21 * for Nuclei Eval SoC which support Nuclei N/NX class cores
22 * \version   V1.00
23 * \date      17 Dec 2019
24 *
25 *****/
26
27 #include "riscv_encoding.h"
28
29 /* TODO: Require Nuclei SDK >= 0.6.0, which introduced this cpufeature.h */
30 #include "cpufeature.h"
31
32 /**
33 * \brief Global interrupt disabled
34 * \details
35 * This function disable global interrupt.
36 * \remarks
37 * - All the interrupt requests will be ignored by CPU.
38 */
39 .macro DISABLE_MIE
40     csrr CSR_MSTATUS, MSTATUS_MIE
41 .endm
42
43 /**
44 * \brief Macro for context save
45 * \details

```

(continues on next page)

(continued from previous page)

```

46  * This macro save ABI defined caller saved registers in the stack.
47  * \remarks
48  * - This Macro could use to save context when you enter to interrupt
49  * or exception
50  */
51  /* Save caller registers */
52  .macro SAVE_CONTEXT
53      /* Allocate stack space for context saving */
54  #ifndef __riscv_32e
55      addi sp, sp, -20*REGBYTES
56  #else
57      addi sp, sp, -14*REGBYTES
58  #endif /* __riscv_32e */
59
60      STORE x1, 0*REGBYTES(sp)
61      STORE x4, 1*REGBYTES(sp)
62      STORE x5, 2*REGBYTES(sp)
63      STORE x6, 3*REGBYTES(sp)
64      STORE x7, 4*REGBYTES(sp)
65      STORE x10, 5*REGBYTES(sp)
66      STORE x11, 6*REGBYTES(sp)
67      STORE x12, 7*REGBYTES(sp)
68      STORE x13, 8*REGBYTES(sp)
69      STORE x14, 9*REGBYTES(sp)
70      STORE x15, 10*REGBYTES(sp)
71  #ifndef __riscv_32e
72      STORE x16, 14*REGBYTES(sp)
73      STORE x17, 15*REGBYTES(sp)
74      STORE x28, 16*REGBYTES(sp)
75      STORE x29, 17*REGBYTES(sp)
76      STORE x30, 18*REGBYTES(sp)
77      STORE x31, 19*REGBYTES(sp)
78  #endif /* __riscv_32e */
79  .endm
80
81  /**
82   * \brief Macro for restore caller registers
83   * \details
84   * This macro restore ABI defined caller saved registers from stack.
85   * \remarks
86   * - You could use this macro to restore context before you want return
87   * from interrupt or exeception
88   */
89  /* Restore caller registers */
90  .macro RESTORE_CONTEXT
91      LOAD x1, 0*REGBYTES(sp)
92      LOAD x4, 1*REGBYTES(sp)
93      LOAD x5, 2*REGBYTES(sp)
94      LOAD x6, 3*REGBYTES(sp)
95      LOAD x7, 4*REGBYTES(sp)
96      LOAD x10, 5*REGBYTES(sp)
97      LOAD x11, 6*REGBYTES(sp)

```

(continues on next page)

(continued from previous page)

```

98     LOAD x12, 7*REGBYTES(sp)
99     LOAD x13, 8*REGBYTES(sp)
100    LOAD x14, 9*REGBYTES(sp)
101    LOAD x15, 10*REGBYTES(sp)
102    #ifndef __riscv_32e
103        LOAD x16, 14*REGBYTES(sp)
104        LOAD x17, 15*REGBYTES(sp)
105        LOAD x28, 16*REGBYTES(sp)
106        LOAD x29, 17*REGBYTES(sp)
107        LOAD x30, 18*REGBYTES(sp)
108        LOAD x31, 19*REGBYTES(sp)
109
110        /* De-allocate the stack space */
111        addi sp, sp, 20*REGBYTES
112    #else
113        /* De-allocate the stack space */
114        addi sp, sp, 14*REGBYTES
115    #endif /* __riscv_32e */
116
117    .endm
118
119    /**
120     * \brief Macro for save necessary CSRs to stack
121     * \details
122     * This macro store MCAUSE, MEPC, MSUBM to stack.
123     */
124    .macro SAVE_CSR_CONTEXT
125    #ifndef CFG_HAS_CLIC
126        /* Store CSR mcause to stack using pushmcause */
127        csrrwi x0, CSR_PUSHMCAUSE, 11
128        /* Store CSR mepc to stack using pushmepc */
129        csrrwi x0, CSR_PUSHMEPC, 12
130        /* Store CSR msub to stack using pushmsub */
131        csrrwi x0, CSR_PUSHMSUBM, 13
132    #else
133        csrr x5, CSR_MEPC
134        STORE x5, 12*REGBYTES(sp)
135        csrr x5, CSR_MCAUSE
136        STORE x5, 11*REGBYTES(sp)
137    #endif
138    .endm
139
140    /**
141     * \brief Macro for restore necessary CSRs from stack
142     * \details
143     * This macro restore MSUBM, MEPC, MCAUSE from stack.
144     */
145    .macro RESTORE_CSR_CONTEXT
146    #ifndef CFG_HAS_CLIC
147        LOAD x5, 13*REGBYTES(sp)
148        csrwr CSR_MSUBM, x5
149    #endif

```

(continues on next page)

(continued from previous page)

```

150     LOAD x5, 12*REGBYTES(sp)
151     csrwr CSR_MEPC, x5
152     LOAD x5, 11*REGBYTES(sp)
153     csrwr CSR_MCAUSE, x5
154 .endm
155
156 /**
157  * \brief Exception/NMI Entry
158  * \details
159  * This function provide common entry functions for exception/nmi and interrupt of plic_
160  * mode.
161  * \remarks
162  * This function provide a default exception/nmi entry.
163  * ABI defined caller save register and some CSR registers
164  * to be saved before enter interrupt handler and be restored before return.
165  */
166 .section .text.trap
167 /* In CLIC mode, the exeception entry must be 64bytes aligned */
168 .align 6
169 # gnu let .weak override .globl, but llvm will show warning
170 # see https://reviews.llvm.org/D90108
171 .weak exc_entry
172 .type exc_entry, @function
173 exc_entry:
174     /* Save the caller saving registers (context) */
175     SAVE_CONTEXT
176     /* Save the necessary CSR registers */
177     SAVE_CSR_CONTEXT
178
179     /*
180      * Set the exception handler function arguments
181      * argument 1: mcause value
182      * argument 2: current stack point(SP) value
183      */
184     csrr a0, mcause
185     mv a1, sp
186
187     /*
188      * TODO: Call the exception handler function
189      * By default, the function template is provided in
190      * system_Device.c, you can adjust it as you want
191      */
192     call core_exception_handler
193
194     /* Restore the necessary CSR registers */
195     RESTORE_CSR_CONTEXT
196     /* Restore the caller saving registers (context) */
197     RESTORE_CONTEXT
198
199     /* Return to regular code */
200     mret
201
202     .size exc_entry, . - exc_entry

```

(continues on next page)

(continued from previous page)

```

201  /**
202  * \brief Non-Vector Interrupt Entry
203  * \details
204  * This function provide common entry functions for handling
205  * non-vector interrupts
206  * \remarks
207  * This function provide a default non-vector interrupt entry.
208  * ABI defined caller save register and some CSR registers need
209  * to be saved before enter interrupt handler and be restored before return.
210  */
211
212 .section      .text.irq
213 /* In CLIC mode, the interrupt entry must be 4bytes aligned */
214 .align 2
215 # gnu let .weak override .globl, but llvm will show warning
216 # see https://reviews.llvm.org/D90108
217 .weak irq_entry
218 .type irq_entry, @function
219 /* This label will be set to MVT2 register */
220 irq_entry:
221     /* Save the caller saving registers (context) */
222     SAVE_CONTEXT
223     /* Save the necessary CSR registers */
224     SAVE_CSR_CONTEXT
225
226     /* This special CSR read/write operation, which is actually
227     * claim the CLIC to find its pending highest ID, if the ID
228     * is not 0, then automatically enable the mstatus.MIE, and
229     * jump to its vector-entry-label, and update the link register
230     */
231     csrrw ra, CSR_JALMNXTI, ra
232
233     /* Critical section with interrupts disabled */
234     DISABLE_MIE
235
236     /* Restore the necessary CSR registers */
237     RESTORE_CSR_CONTEXT
238     /* Restore the caller saving registers (context) */
239     RESTORE_CONTEXT
240
241     /* Return to regular code */
242     mret
243
244     .size irq_entry, . - irq_entry
245
246 /* Default Handler for Exceptions / Interrupts */
247 # gnu let .weak override .globl, but llvm will show warning
248 # see https://reviews.llvm.org/D90108
249 .weak default_intexc_handler
250 .type default_intexc_handler, @function
251 Undef_Handler:
252 default_intexc_handler:

```

(continues on next page)

(continued from previous page)

```

253 1:
254     j 1b
255
256     .size default_intexc_handler, . - default_intexc_handler

```

Device Linker Script: gcc_<device>.ld

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of **evalsoc** in the **Nuclei SDK**²⁴ for the latest reference template. This reference template may not be actively maintained in the future.

The Linker Script File gcc_<device>.ld contains:

- Memory base address and size.
- Code, data section, vector table etc. location.
- Stack & heap location and size.

The file exists for each supported toolchain and is the only toolchain specific NMSIS file.

To adapt the file to a new device only when you need change the memory base address, size, data and code location etc.

gcc_Device.ld Template File

Here we provided gcc_Device.ld template file as below:

```

1  /*
2   * Copyright (c) 2019 Nuclei Limited. All rights reserved.
3   *
4   * SPDX-License-Identifier: Apache-2.0
5   *
6   * Licensed under the Apache License, Version 2.0 (the License); you may
7   * not use this file except in compliance with the License.
8   * You may obtain a copy of the License at
9   *
10  * www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing, software
13  * distributed under the License is distributed on an AS IS BASIS, WITHOUT
14  * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15  * See the License for the specific language governing permissions and
16  * limitations under the License.
17  */
18  /*****
19   * @file      gcc_<Device>.ld
20   * @brief     GNU Linker Script for Device <Device>

```

(continues on next page)

²⁴ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

(continued from previous page)

```

21  * @version  V2.1.0
22  * @date     19. Dec 2023
23  *****/
24
25  /***** Use Configuration Wizard in Context Menu *****/
26
27  OUTPUT_ARCH( "riscv" )
28  /***** Flash Configuration *****/
29  * <h> Flash Configuration
30  * <o0> Flash Base Address <0x0-0xFFFFFFFF:8>
31  * <o1> Flash Size (in Bytes) <0x0-0xFFFFFFFF:8>
32  * </h>
33  */
34  __ROM_BASE = 0x20000000;
35  __ROM_SIZE = 0x00400000;
36
37  /*----- ILM RAM Configuration -----*/
38  * <h> ILM RAM Configuration
39  * <o0> ILM RAM Base Address <0x0-0xFFFFFFFF:8>
40  * <o1> ILM RAM Size (in Bytes) <0x0-0xFFFFFFFF:8>
41  * </h>
42  */
43  __ILM_RAM_BASE = 0x80000000;
44  __ILM_RAM_SIZE = 0x00010000;
45
46  /*----- Embedded RAM Configuration -----*/
47  * <h> RAM Configuration
48  * <o0> RAM Base Address <0x0-0xFFFFFFFF:8>
49  * <o1> RAM Size (in Bytes) <0x0-0xFFFFFFFF:8>
50  * </h>
51  */
52  __RAM_BASE = 0x90000000;
53  __RAM_SIZE = 0x00010000;
54
55  /***** Stack / Heap Configuration *****/
56  * <h> Stack / Heap Configuration
57  * <o0> Stack Size (in Bytes) <0x0-0xFFFFFFFF:8>
58  * <o1> Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
59  * </h>
60  */
61  __STACK_SIZE = 0x00000800;
62  __HEAP_SIZE = 0x00000800;
63
64  /***** end of configuration section *****/
65
66  /* Define entry label of program */
67  ENTRY(_start)
68  /* Define base address and length of flash and ram */
69  MEMORY
70  {
71      flash (rxa!w) : ORIGIN = __ROM_BASE, LENGTH = __ROM_SIZE
72      ram (wxa!r) : ORIGIN = __RAM_BASE, LENGTH = __RAM_SIZE

```

(continues on next page)

(continued from previous page)

```

73 }
74
75 REGION_ALIAS("ROM", flash)
76 REGION_ALIAS("RAM", ram)
77
78 /* Linker script to place sections and symbol values. Should be used together
79  * with other linker script that defines memory regions FLASH, ILM and RAM.
80  * It references following symbols, which must be defined in code:
81  *   _start : Entry of reset handler
82  *
83  * It defines following symbols, which code can use without definition:
84  *   _ilm_lma ; deprecated
85  *   _ilm      ; deprecated
86  *   _eilm      ; deprecated
87  *   _text_lma
88  *   _text
89  *   _etext
90  *   __etext
91  *   etext
92  *   __preinit_array_start
93  *   __preinit_array_end
94  *   __init_array_start
95  *   __init_array_end
96  *   __fini_array_start
97  *   __fini_array_end
98  *   _data_lma
99  *   _edata
100  *   edata
101  *   __data_end__
102  *   __bss_start
103  *   __fbss
104  *   _end
105  *   end
106  *   __heap_start
107  *   __heap_end
108  *   __heap_limit
109  *   __StackLimit
110  *   __StackBottom
111  *   __StackTop
112  *   __HEAP_SIZE
113  *   __STACK_SIZE
114 */
115
116 SECTIONS
117 {
118     /* To provide symbol __STACK_SIZE, __HEAP_SIZE and __SMP_CPU_CNT */
119     PROVIDE(__STACK_SIZE = 2K);
120     PROVIDE(__HEAP_SIZE = 2K);
121     PROVIDE(__SMP_CPU_CNT = 1);
122     __TOT_STACK_SIZE = __STACK_SIZE * __SMP_CPU_CNT;
123
124     .init      :

```

(continues on next page)

(continued from previous page)

```

125 {
126     /* vector table locate at ROM */
127     *(.text.vtable)
128     *(.text.vtable_s)
129     *(.text.init)
130     KEEP (*(SORT_NONE(.init)))
131     . = ALIGN(4);
132 } >ROM AT>ROM
133
134 /* Code section located at ROM */
135 .text          :
136 {
137     *(.text.unlikely .text.unlikely.*)
138     *(.text.startup .text.startup.*)
139     . = ALIGN(8);
140     PROVIDE( __jvt_base$ = . );
141     *(.text.tljal .text.tljal.*)
142     *(.text .text.*)
143     *(.gnu.linkonce.t.*)
144     /* readonly data placed in ROM */
145     . = ALIGN(8);
146     *(.srodata.cst16)
147     *(.srodata.cst8)
148     *(.srodata.cst4)
149     *(.srodata.cst2)
150     *(.srodata .srodata.*)
151     *(.rdata)
152     *(.rodata .rodata.*)
153     *(.gnu.linkonce.r.*)
154     /* rtt */
155     . = ALIGN(8);
156     __rt_init_start = .;
157     KEEP(*(SORT(.rti_fn*)))
158     __rt_init_end = .;
159     . = ALIGN(8);
160     __fsymtab_start = .;
161     KEEP(*(FSymTab))
162     __fsymtab_end = .;
163     . = ALIGN(8);
164     __vsymtab_start = .;
165     KEEP(*(VSymTab))
166     __vsymtab_end = .;
167     /* .fini */
168     . = ALIGN(8);
169     KEEP (*(SORT_NONE(.fini)))
170     /* .preinit_array */
171     . = ALIGN(8);
172     PROVIDE_HIDDEN (__preinit_array_start = .);
173     KEEP (*(preinit_array))
174     PROVIDE_HIDDEN (__preinit_array_end = .);
175     /* .init_array */
176     . = ALIGN(8);

```

(continues on next page)

(continued from previous page)

```

177 PROVIDE_HIDDEN (__init_array_start = .);
178 KEEP (*(SORT_BY_INIT_PRIORITY(.init_array.*) SORT_BY_INIT_PRIORITY(.ctors.*)))
179 KEEP (*(.(init_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .
↳ctors))
180 PROVIDE_HIDDEN (__init_array_end = .);
181 /* .fini_array */
182 . = ALIGN(8);
183 PROVIDE_HIDDEN (__fini_array_start = .);
184 PROVIDE_HIDDEN (__libc_fini = _fini);
185 KEEP (*(SORT_BY_INIT_PRIORITY(.fini_array.*) SORT_BY_INIT_PRIORITY(.dtors.*)))
186 KEEP (*(.(fini_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .
↳dtors))
187 PROVIDE_HIDDEN (__fini_array_end = .);
188 /* .ctors */
189 . = ALIGN(8);
190 KEEP (*crtbegin.o(.ctors))
191 KEEP (*crtbegin?.o(.ctors))
192 KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .ctors))
193 KEEP (*(SORT(.ctors.*)))
194 KEEP (*(.(ctors)))
195 /* .dtors */
196 . = ALIGN(8);
197 KEEP (*crtbegin.o(.dtors))
198 KEEP (*crtbegin?.o(.dtors))
199 KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .dtors))
200 KEEP (*(SORT(.dtors.*)))
201 KEEP (*(.(dtors)))
202 } >ROM AT>ROM
203
204 PROVIDE( _ilm_lma = LOADADDR(.text) );
205 PROVIDE( _ilm = ADDR(.text) );
206 PROVIDE( _eilm = . );
207 PROVIDE( _text_lma = LOADADDR(.text) );
208 PROVIDE( _text = ADDR(.text) );
209 PROVIDE( _etext = . );
210 PROVIDE( __etext = . );
211 PROVIDE( etext = . );
212
213 .data : ALIGN(8)
214 {
215     KEEP(*(.(data.ctest*))
216     *(.data .data.*)
217     *(.gnu.linkonce.d.*)
218     . = ALIGN(8);
219     PROVIDE( __global_pointer$ = . + 0x800 );
220     *(.sdata .sdata.* .sdata*)
221     *(.gnu.linkonce.s.*)
222     . = ALIGN(8);
223 } >RAM AT>ROM
224
225 .tdata : ALIGN(8)
226 {

```

(continues on next page)

(continued from previous page)

```

227     PROVIDE( __tls_base = . );
228     *(.tdata .tdata.* .gnu.linkonce.td.*)
229 } >RAM AT>ROM
230
231 PROVIDE( _data_lma = LOADADDR(.data) );
232 PROVIDE( _data = ADDR(.data) );
233 PROVIDE( _edata = . );
234 PROVIDE( edata = . );
235
236 PROVIDE( _fbss = . );
237 PROVIDE( __bss_start = . );
238
239 .tbss (NOLOAD) : ALIGN(8)
240 {
241     *(.tbss .tbss.* .gnu.linkonce.tb.*)
242     *(.tcommon)
243     PROVIDE( __tls_end = . );
244 } >RAM AT>RAM
245
246 .tbss_space (NOLOAD) : ALIGN(8)
247 {
248     . = . + SIZEOF(.tbss);
249 } >RAM AT>RAM
250
251 .bss (NOLOAD) : ALIGN(8)
252 {
253     *(.sbss*)
254     *(.gnu.linkonce.sb.*)
255     *(.bss .bss.*)
256     *(.gnu.linkonce.b.*)
257     *(COMMON)
258     . = ALIGN(4);
259 } >RAM AT>RAM
260
261 PROVIDE( _end = . );
262 PROVIDE( end = . );
263
264 /* Nuclei C Runtime Library requirements:
265 * 1. heap need to be align at 16 bytes
266 * 2. __heap_start and __heap_end symbol need to be defined
267 * 3. reserved at least __HEAP_SIZE space for heap
268 */
269 .heap (NOLOAD) : ALIGN(16)
270 {
271     . = ALIGN(16);
272     PROVIDE( __heap_start = . );
273     . += __HEAP_SIZE;
274     . = ALIGN(16);
275     PROVIDE( __heap_limit = . );
276 } >RAM AT>RAM
277
278 .stack ORIGIN(RAM) + LENGTH(RAM) - __TOT_STACK_SIZE (NOLOAD) :

```

(continues on next page)

(continued from previous page)

```

279 {
280     . = ALIGN(16);
281     PROVIDE( _heap_end = . );
282     PROVIDE( __heap_end = . );
283     PROVIDE( __StackLimit = . );
284     PROVIDE( __StackBottom = . );
285     . += __TOT_STACK_SIZE;
286     . = ALIGN(16);
287     PROVIDE( __StackTop = . );
288     PROVIDE( _sp = . );
289 } >RAM AT>RAM
290 }

```

System Configuration Files system_<Device>.c and system_<Device>.h

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of [evalsoc in the Nuclei SDK²⁵](#) for the latest reference template. This reference template may not be actively maintained in the future.

The **System Configuration Files system_<device>.c and system_<device>.h** provides as a minimum the functions described under *System Device Configuration* (page 804).

These functions are device specific and need adaptations. In addition, the file might have configuration settings for the device such as XTAL frequency or PLL prescaler settings, necessary system initialization, vendor customized interrupt, exception and nmi handling code, refer to *System Device Configuration* (page 804) for more details.

For devices with external memory BUS the system_<Device>.c also configures the BUS system.

The silicon vendor might expose other functions (i.e. for power configuration) in the system_<Device>.c file. In case of additional features the function prototypes need to be added to the system_<Device>.h header file.

system_Device.c Template File

Here we provided system_Device.c template file as below:

Please check file content in https://github.com/Nuclei-Software/NMSIS/blob/master/Device/_Template_Vendor/Vendor/Device/Source/system_Device.c

²⁵ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

system_Device.h Template File

Here we provided system_Device.h template file as below:

```

1  /*
2  * Copyright (c) 2009-2018 Arm Limited. All rights reserved.
3  * Copyright (c) 2019 Nuclei Limited. All rights reserved.
4  *
5  * SPDX-License-Identifier: Apache-2.0
6  *
7  * Licensed under the Apache License, Version 2.0 (the License); you may
8  * not use this file except in compliance with the License.
9  * You may obtain a copy of the License at
10 *
11 * www.apache.org/licenses/LICENSE-2.0
12 *
13 * Unless required by applicable law or agreed to in writing, software
14 * distributed under the License is distributed on an AS IS BASIS, WITHOUT
15 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
16 * See the License for the specific language governing permissions and
17 * limitations under the License.
18 */
19 /*****
20 * @file      system_<Device>.h
21 * @brief     NMSIS Nuclei N/NX Device Peripheral Access Layer Header File for
22 *           Device <Device>
23 * @version   V1.00
24 * @date      17. Dec 2019
25 *****/
26
27 #ifndef __SYSTEM_<Device>_H__  /* ToDo: replace '<Device>' with your device name */
28 #define __SYSTEM_<Device>_H__
29
30 // NOTE: Should never directly include this header, you should include <Device>.h
31
32 #ifdef __cplusplus
33 extern "C" {
34 #endif
35
36 #include <stdint.h>
37
38 extern volatile uint32_t SystemCoreClock;    /*!< System Clock Frequency (Core Clock) */
39
40 typedef struct EXC_Frame {
41     unsigned long ra;           /* ra: x1, return address for jump */
42     unsigned long tp;           /* tp: x4, thread pointer */
43     unsigned long t0;           /* t0: x5, temporary register 0 */
44     unsigned long t1;           /* t1: x6, temporary register 1 */
45     unsigned long t2;           /* t2: x7, temporary register 2 */
46     unsigned long a0;           /* a0: x10, return value or function argument 0 */
47     unsigned long a1;           /* a1: x11, return value or function argument 1 */
48     unsigned long a2;           /* a2: x12, function argument 2 */
49     unsigned long a3;           /* a3: x13, function argument 3 */

```

(continues on next page)

(continued from previous page)

```

50     unsigned long a4;           /* a4: x14, function argument 4 */
51     unsigned long a5;           /* a5: x15, function argument 5 */
52     unsigned long cause;        /* cause: machine/supervisor mode cause csr
↳ register */
53     unsigned long epc;          /* epc: machine/ supervisor mode exception program
↳ counter csr register */
54     unsigned long msubm;        /* msubm: machine sub-mode csr register, nuclei
↳ customized, exclusive to machine mode */
55 #ifndef __riscv_32e
56     unsigned long a6;           /* a6: x16, function argument 6 */
57     unsigned long a7;           /* a7: x17, function argument 7 */
58     unsigned long t3;           /* t3: x28, temporary register 3 */
59     unsigned long t4;           /* t4: x29, temporary register 4 */
60     unsigned long t5;           /* t5: x30, temporary register 5 */
61     unsigned long t6;           /* t6: x31, temporary register 6 */
62 #endif
63 } EXC_Frame_Type;
64
65 /**
66  * \brief Setup the microcontroller system.
67  * \details
68  * Initialize the System and update the SystemCoreClock variable.
69  */
70 extern void SystemInit(void);
71
72 /**
73  * \brief Update SystemCoreClock variable.
74  * \details
75  * Updates the SystemCoreClock with current core Clock retrieved from cpu registers.
76  */
77 extern void SystemCoreClockUpdate(void);
78
79 /**
80  * \brief Dump Exception Frame
81  */
82 void Exception_DumpFrame(unsigned long sp, uint8_t mode);
83
84 /**
85  * \brief Register a m-mode exception handler for exception code EXCn
86  */
87 extern void Exception_Register_EXC(uint32_t EXCn, unsigned long exc_handler);
88
89 /**
90  * \brief Get current m-mode exception handler for exception code EXCn
91  */
92 extern unsigned long Exception_Get_EXC(uint32_t EXCn);
93
94 /**
95  * \brief Initialize Interrupt as Clint interrupt mode
96  */
97 extern void CLINT_Interrupt_Init(void);
98

```

(continues on next page)

(continued from previous page)

```

99  /**
100  * \brief Initialize Interrupt
101  */
102  extern void Interrupt_Init(void);
103
104  #if defined(__ECLIC_PRESENT) && (__ECLIC_PRESENT == 1)
105  /**
106  * \brief Do ECLIC Interrupt configuration
107  */
108  extern void ECLIC_Interrupt_Init(void);
109
110  /**
111  * \brief Initialize a specific IRQ and register the handler
112  * \details
113  * This function set vector mode, trigger mode and polarity, interrupt level and
114  * priority,
115  * assign handler for specific IRQn.
116  */
117  extern int32_t ECLIC_Register_IRQ(IRQn_Type IRQn, uint8_t shv, ECLIC_TRIGGER_Type trig_
118  * mode, uint8_t lvl, uint8_t priority, void* handler);
119
120  #if defined(__TEE_PRESENT) && (__TEE_PRESENT == 1)
121  /**
122  * \brief Initialize a specific IRQ and register the handler of supervisor mode
123  * \details
124  * This function set vector mode, trigger mode and polarity, interrupt level and
125  * priority,
126  * assign handler for specific IRQn.
127  */
128  extern int32_t ECLIC_Register_IRQ_S(IRQn_Type IRQn, uint8_t shv, ECLIC_TRIGGER_Type trig_
129  * mode, uint8_t lvl, uint8_t priority, void* handler);
130  #endif
131
132  #endif
133
134  /**
135  * \brief Register a m-mode core interrupt handler for clint/plic interrupt mode
136  */
137  extern void Interrupt_Register_CoreIRQ(uint32_t irqn, unsigned long int_handler);
138
139  /**
140  * \brief Register a m-mode plic external interrupt handler for clint/plic interrupt mode
141  */
142  extern void Interrupt_Register_ExtIRQ(uint32_t irqn, unsigned long int_handler);
143
144  /**
145  * \brief Get a m-mode core interrupt handler for core interrupt number
146  */
147  extern unsigned long Interrupt_Get_CoreIRQ(uint32_t irqn);
148
149  /**
150  * \brief Get a m-mode external interrupt handler for external interrupt number

```

(continues on next page)

(continued from previous page)

```

147  */
148  extern unsigned long Interrupt_Get_ExtIRQ(uint32_t irqn);
149
150  /**
151   * \brief Register a m-mode riscv core interrupt and register the handler for clint/plic_
152   * \interrupt mode
153   */
154  extern int32_t Core_Register_IRQ(uint32_t irqn, void *handler);
155
156  #if defined(__PLIC_PRESENT) && (__PLIC_PRESENT == 1)
157  /**
158   * \brief Do plic interrupt configuration for clint/plic interrupt mode
159   */
160  extern void PLIC_Interrupt_Init(void);
161
162  /**
163   * \brief Register a m-mode specific plic interrupt and register the handler
164   */
165  extern int32_t PLIC_Register_IRQ(uint32_t source, uint8_t priority, void *handler);
166  #if defined(__SMODE_PRESENT) && (__SMODE_PRESENT == 1)
167  /**
168   * \brief Register a s-mode specific plic interrupt and register the handler
169   */
170  extern int32_t PLIC_Register_IRQ_S(uint32_t source, uint8_t priority, void *handler);
171  #endif
172  #endif
173
174  #if defined(__SMODE_PRESENT) && (__SMODE_PRESENT == 1)
175  /**
176   * \brief Register a s-mode exception handler for exception code EXCn
177   */
178  extern void Exception_Register_EXC_S(uint32_t EXCn, unsigned long exc_handler);
179
180  /**
181   * \brief Get current s-mode exception handler for exception code EXCn
182   */
183  extern unsigned long Exception_Get_EXC_S(uint32_t EXCn);
184
185  /**
186   * \brief Register a s-mode core interrupt handler for clint/plic interrupt mode
187   */
188  extern void Interrupt_Register_CoreIRQ_S(uint32_t irqn, unsigned long int_handler);
189
190  /**
191   * \brief Register a s-mode plic external interrupt handler for clint/plic interrupt mode
192   */
193  extern void Interrupt_Register_ExtIRQ_S(uint32_t irqn, unsigned long int_handler);
194
195  /**
196   * \brief Get a s-mode core interrupt handler for core interrupt number
197   */
198  extern unsigned long Interrupt_Get_CoreIRQ_S(uint32_t irqn);

```

(continues on next page)

(continued from previous page)

```

198 /**
199  * \brief Get a s-mode external interrupt handler for external interrupt number
200  */
201 extern unsigned long Interrupt_Get_ExtIRQ_S(uint32_t irqn);
202
203 /**
204  * \brief Register a s-mode riscv core interrupt and register the handler for clint/plic_
205  * \interrupt mode
206  */
207 extern int32_t Core_Register_IRQ_S(uint32_t irqn, void *handler);
208 #endif
209
210 #ifdef __cplusplus
211 }
212 #endif
213
214 #endif /* __SYSTEM_<Device>_H__ */

```

Device Header File <Device.h>

Caution

Please be informed that the **NMSIS-Core Device Templates** may not be updated in a timely manner and thus could become outdated. We suggest referring to the specific implementation of **evalsoc** in the **Nuclei SDK**²⁶ for the latest reference template. This reference template may not be actively maintained in the future.

The *Device Header File <Device.h>* (page 48) contains the following sections that are device specific:

- *Interrupt Number Definition* (page 49) provides interrupt numbers (IRQn) for all exceptions and interrupts of the device.
- *Configuration of the Processor and Core Peripherals* (page 50) reflect the features of the device.
- *Device Peripheral Access Layer* (page 53) provides definitions for the *Peripheral Access* (page 611) to all device peripherals. It contains all data structures and the address mapping for device-specific peripherals.
- **Access Functions for Peripherals (optional)** provide additional helper functions for peripherals that are useful for programming of these peripherals. Access Functions may be provided as inline functions or can be extern references to a device-specific library provided by the silicon vendor.

NMSIS Core API (page 71) describes the standard features and functions of the *Device Header File <Device.h>* (page 48) in detail.

²⁶ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

Interrupt Number Definition

Device Header File <Device.h> (page 48) contains the enumeration *IRQn_Type* (page 636) that defines all exceptions and interrupts of the device.

- Negative IRQn values represent processor core exceptions (internal interrupts).
- Positive IRQn values represent device-specific exceptions (external interrupts). The first device-specific interrupt has the IRQn value 0. The IRQn values needs extension to reflect the device-specific interrupt vector table in the *Startup File startup_<Device>.S* (page 15).

The following example shows the extension of the interrupt vector table for the GD32VF103 device family.

```

1 typedef enum IRQn {
2     /***** N200 Processor Exceptions Numbers *****/
3     Reserved0_IRQn = 0,      /*!< Internal reserved
4     */
5     Reserved1_IRQn = 1,      /*!< Internal reserved
6     */
7     Reserved2_IRQn = 2,      /*!< Internal reserved
8     */
9     SysTimerSW_IRQn = 3,     /*!< System Timer SW interrupt
10    */
11    Reserved3_IRQn = 4,       /*!< Internal reserved
12    */
13    Reserved4_IRQn = 5,       /*!< Internal reserved
14    */
15    Reserved5_IRQn = 6,       /*!< Internal reserved
16    */
17    SysTimer_IRQn = 7,        /*!< System Timer Interrupt
18    */
19    Reserved6_IRQn = 8,       /*!< Internal reserved
20    */
21    Reserved7_IRQn = 9,       /*!< Internal reserved
22    */
23    Reserved8_IRQn = 10,      /*!< Internal reserved
24    */
25    Reserved9_IRQn = 11,      /*!< Internal reserved
26    */
27    Reserved10_IRQn = 12,     /*!< Internal reserved
28    */
29    Reserved11_IRQn = 13,     /*!< Internal reserved
30    */
31    Reserved12_IRQn = 14,     /*!< Internal reserved
32    */
33    Reserved13_IRQn = 15,     /*!< Internal reserved
34    */
35    Reserved14_IRQn = 16,     /*!< Internal reserved
36    */
37    HardFault_IRQn = 17,      /*!< Hard Fault, storage access error
38    */
39    Reserved15_IRQn = 18,     /*!< Internal reserved
40    */
41 } IRQn_Type;

```

(continues on next page)

(continued from previous page)

```

23  /***** GD32VF103 Specific Interrupt Numbers *****/
24  WWDGT_IRQn      = 19,    /*!< window watchDog timer interrupt
25  */
26  LVD_IRQn        = 20,    /*!< LVD through EXTI line detect interrupt
27  */
28  TAMPER_IRQn     = 21,    /*!< tamper through EXTI line detect
29  */
30  CAN1_EWMC_IRQn  = 85,    /*!< CAN1 EWMC interrupt
31  */
32  USBFS_IRQn      = 86,    /*!< USBFS global interrupt
33  */
34  SOC_INT_MAX,    /*!< Number of total Interrupts
35  */
36  } IRQn_Type;

```

Configuration of the Processor and Core Peripherals

The *Device Header File* <Device.h> (page 48) configures the Nuclei N/NX/UX Class Processors and the core peripherals with #define that are set prior to including the file *nmsis_core.h*.

For recently released Nuclei 200/300/600/900 RISC-V CPU, the cpu private peripherals are also called internal regions(IREGION) and the address spaces are continuous with fixed size, cpu will be configured with only the base address of IREGION, such as ECLIC, TIMER, SMP, Cluster Cache, CIDU, PLIC unit, for more details, please check Nuclei ISA Spec and related CPU databook.

The following tables list the #define along with the possible values for N200, N300, N600, NX600, UX600, N900, NX900, UX900, UX1000. If these #define are missing default values are used.

nmsis_core.h

Note

- __NUCLEI_N_REV and __NUCLEI_NX_REV are deprecated since 1.2.0, please use __NUCLEI_CPU_REV and __NUCLEI_CPU_SERIES now.
- __HARTID_OFFSET and __SYSTIMER_HARTID is added since 1.2.0
- Please check Nuclei SDK evalsoc header file evalsoc.h for latest cpu macros.

Table 5: Macros used in nmsis_core.h

#define	Value Range	Default	Description
---------	-------------	---------	-------------

continues on next page

Table 5 – continued from previous page

		0x0100	
__NUCLEI_N_REV OR __NUCLEI_NX_REV	0x0100 0x0104		- For Nuclei N class device, define __NUCLEI_N_REV, for NX class device, define __NUCLEI_NX_REV. - Core revision number ([15:8] revision number, [7:0] patch number), 0x0100 -> 1.0, 0x0104 -> 1.4
__NUCLEI_CPU_REV	•	•	Define Nuclei CPU Revision Number, such as 0x030A01 means v3.10.1.
__NUCLEI_CPU_SERIES	•	•	Define Nuclei CPU Series, such as 0x0200, 0x0300, 0x0600, 0x0900 for 200/300/600/900 series.
__IREGION_BASEADDR	•	•	Define Nuclei CPU Internal Region Base Address.
__HARTID_OFFSET	•	•	Define the offset of the first cpu hart's hartid vs hart index, eg, cpu first hartid is 3, set it to 3.
__SYSTIMER_PRESENT	0 .. 1	1	Define whether Private System Timer is present or not. This SysTimer is a Memory Mapped Unit.
__SYS- TIMER_BASEADDR	•	0x18030000	Base address of the System Timer Unit.
__CLINT_TIMER_BASEAD	•	0x18040000	Base address of the CLINT compatible timer in System Timer Unit.
__SYSTIMER_HARTID	•	•	Optional, if you cpu system only has one hart, and the timer hartid is known, you can set it to known value
__SSTC_PRESENT	0 .. 1	1	Define whether Sstc(Supervisor-mode Timer Interrupts) extension is present or not
__ECLIC_PRESENT	0 .. 1	1	Define whether Enhanced Core Local Interrupt Controller (ECLIC) Unit is present or not
__ECLIC_BASEADDR	•	0x18020000	Base address of the ECLIC unit.
__CIDU_PRESENT	0 .. 1	0	Define whether Cluster Interrupt Distribution Unit (CIDU) is present or not
__CIDU_BASEADDR	•	0x18050000	Base address of the CIDU unit.
__ECLIC_INTCTLBITS	1 .. 8	1	Define the number of hardware bits are actually implemented in the clicintctl registers.
__ECLIC_INTNUM	1 .. 1024	1	Define the total interrupt number(including the internal core interrupts) of ECLIC Unit
__PLIC_PRESENT	0 .. 1	0	Define whether Platform-Level Interrupt Controller (PLIC) Unit is present or not
__PLIC_BASEADDR	•	0x1C000000	Base address of the PLIC unit.

continues on next page

Table 5 – continued from previous page

__SMPCC_PRESENT	0 .. 1	0	Define whether SMP and Cluster Cache Unit is present or not
__SMPCC_BASEADDR	•	0x18040000	Base address of the SMPCC unit.
__PLIC_INTNUM	1 .. 1024	1	Define the total external interrupt number of PLIC Unit
__PMP_PRESENT	0 .. 1	0	Define whether Physical Memory Protection (PMP) Unit is present or not.
__PMP_ENTRY_NUM	8 or 16	8	Define the numbers of PMP entries.
__TEE_PRESENT	0 .. 1	0	Define whether TEE Unit is present or not.
__SPMP_PRESENT	0 .. 1	0	Define whether SMode Physical Memory Protection (sPMP) Unit is present or not.
__SPMP_ENTRY_NUM	8 or 16	8	Define the numbers of sPMP entries.
__SMPU_PRESENT	0 .. 1	0	Define whether SMode Memory Protection Unit (sMPU) Unit is present or not.
__SMPU_ENTRY_NUM	8 or 16	8	Define the numbers of sMPU entries.
__FPU_PRESENT	0 .. 2	0	Define whether Floating Point Unit (FPU) is present or not. • 0: Not present • 1: Single precision FPU present • 2: Double precision FPU present
__BITMANIP_PRESENT	0 .. 1	0	Define whether Bitmainp Unit is present or not.
__DSP_PRESENT	0 .. 1	0	Define whether Digital Signal Processing Unit (DSP) is present or not.
__VECTOR_PRESENT	0 .. 1	0	Define whether Vector Unit is present or not.
__ICACHE_PRESENT	0 .. 1	0	Define whether I-Cache Unit is present or not.
__DCACHE_PRESENT	0 .. 1	0	Define whether D-Cache Unit is present or not.
__CCM_PRESENT	0 .. 1	0	Define whether Nuclei Cache Control and Maintenance Unit is present or not.
__PMA_PRESENT	0 .. 1	0	Define whether Physical memory attribute Unit is present or not.
__PMA_CSR_NUM	•	0	Define the numbers of PMA CSR Number.
__PMA_SEC_CSR_NUM	•	0	Define the numbers of PMA Secure CSR Number.
__PMA_MACRO_PRESENT	0 .. 1	0	Define whether Physical memory attribute rtl configuration macro is present or not.
__HPM_PRESENT	0 .. 1	0	Define whether High Performance Monitor(PMU) is present or not.
__HPM_VER	•	1	Define High Performance Monitor(PMU) version.
__NICE_PRESENT	0 .. 1	0	Define whether NICE is present or not.
__VNICE_PRESENT	0 .. 1	0	Define whether Vector NICE is present or not.
__INC_INTRINSIC_API	0 .. 1	0	Define whether toolchain provided intrinsic api headers are included or not.

continues on next page

Table 5 – continued from previous page

<code>__Vendor_SysTickConfig</code>	0 .. 1	0	If <code>__SYSTIMER_PRESENT</code> is 1, then the <code>__Vendor_SysTickConfig</code> can be set to 0, otherwise it can only set to 1. If this define is set to 1, then the default <code>SysTick_Config</code> and <code>SysTick_Reload</code> function is excluded. In this case, the file <code>Device.h</code> must contain a vendor specific implementation of this function.
-------------------------------------	--------	---	---

NMSIS Version and Processor Information

The following shows the defines in the `nmsis_core.h` file that may be used in the *NMSIS-Core Device Templates* (page 12) to verify a minimum version or ensure that the right Nuclei N/NX/U/UX class is used.

Device Peripheral Access Layer

The *Device Header File <Device.h>* (page 48) contains for each peripheral:

- Register Layout Typedef
- Base Address
- Access Definitions

The section *Peripheral Access* (page 611) shows examples for peripheral definitions.

Device.h Template File

Here we provided `Device.h` template file as below:

```

1  /*****
2  * @file      <Device>.h
3  * @brief     NMSIS Core Peripheral Access Layer Header File for
4  *            Nuclei Eval SoC which support Nuclei N/NX class cores
5  * @version   V1.00
6  * @date      22. Nov 2019
7  *****/
8  /*
9  * Copyright (c) 2019 Nuclei Limited. All rights reserved.
10 *
11 * SPDX-License-Identifier: Apache-2.0
12 *
13 * Licensed under the Apache License, Version 2.0 (the License); you may
14 * not use this file except in compliance with the License.
15 * You may obtain a copy of the License at
16 *
17 * www.apache.org/licenses/LICENSE-2.0
18 *
19 * Unless required by applicable law or agreed to in writing, software
20 * distributed under the License is distributed on an AS IS BASIS, WITHOUT
21 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.

```

(continues on next page)

(continued from previous page)

```

22  * See the License for the specific language governing permissions and
23  * limitations under the License.
24  */
25
26  #ifndef __<Device>_H__
27  #define __<Device>_H__
28
29  #include <stddef.h>
30
31  #ifdef __cplusplus
32  extern "C" {
33  #endif
34
35  // NOTE: this cpufeature.h header file is introduced in Nuclei SDK 0.6.0
36  #include "cpufeature.h"
37
38  /** @addtogroup Nuclei
39   * @{
40   */
41
42
43  /** @addtogroup <Device>
44   * @{
45   */
46
47
48  /** @addtogroup Configuration_of_NMSIS
49   * @{
50   */
51
52  /** \brief SoC Download mode definition */
53  typedef enum {
54      DOWNLOAD_MODE_FLASHXIP = 0,          /*!< Flashxip download mode */
55      DOWNLOAD_MODE_FLASH = 1,             /*!< Flash download mode */
56      DOWNLOAD_MODE_ILM = 2,               /*!< ilm download mode */
57      DOWNLOAD_MODE_DDR = 3,               /*!< ddr download mode */
58      DOWNLOAD_MODE_SRAM = 4,              /*!< sram download mode */
59      DOWNLOAD_MODE_SRAMXIP = 5,           /*!< sramxip download mode */
60      DOWNLOAD_MODE_MAX,
61  } DownloadMode_Type;
62
63  // IRegion_Info_Type structure is removed in Nuclei SDK 0.6.0 release
64
65  /* Simulation mode macros */
66  #define SIMULATION_MODE_XLSPIKE 0        /*!< xlspike simulation mode */
67  #define SIMULATION_MODE_QEMU 1          /*!< qemu simulation mode */
68
69  /*
70  =====
71  ===== */
72  */

```

Interrupt Number Definition

(continues on next page)

(continued from previous page)

```

71  /*
72  ↪=====
73  ↪*/
74  /* <Device>'s External IRQn ID is from the hard-wired persperctive, which has an offset
75  ↪mapped to the ECLIC IRQn.
76  eg.: uart0's external interrupt id in <Device> is 32, while its ECLIC IRQn is 51 */
77  #define SOC_EXTERNAL_MAP_TO_ECLIC_IRQn_OFFSET      19
78  /* get <Device>'s External IRQn from ECLIC external IRQn which indexs from 19 */
79  #define IRQn_MAP_TO_EXT_ID(IRQn)                   (IRQn - SOC_EXTERNAL_MAP_TO_ECLIC_
80  ↪IRQn_OFFSET)
81
82  typedef enum IRQn {
83  ↪/* ===== Nuclei Core Specific Interrupt Numbers
84  ↪===== */
85
86  Reserved0_IRQn      = 0,          /*!< Internal reserved */
87  SysTimerSW_S_IRQn   = 1,          /*!< System Timer supervisor mode SW
88  ↪interrupt triggered by ssip */
89  Reserved2_IRQn      = 2,          /*!< Internal reserved */
90  SysTimerSW_IRQn     = 3,          /*!< System Timer machine mode SW
91  ↪interrupt triggered by msip */
92  Reserved3_IRQn      = 4,          /*!< Internal reserved */
93  SysTimer_S_IRQn     = 5,          /*!< System Timer supervisor mode
94  ↪interrupt triggered by stimecmp csr */
95  Reserved5_IRQn      = 6,          /*!< Internal reserved */
96  SysTimer_IRQn       = 7,          /*!< System Timer machine mode
97  ↪interrupt triggered by mtimecmp */
98  Reserved6_IRQn      = 8,          /*!< Internal reserved */
99  Reserved7_IRQn      = 9,          /*!< Internal reserved */
100  Reserved8_IRQn      = 10,         /*!< Internal reserved */
101  Reserved9_IRQn      = 11,         /*!< Internal reserved */
102  Reserved10_IRQn     = 12,         /*!< Internal reserved */
103  Reserved11_IRQn     = 13,         /*!< Internal reserved */
104  Reserved12_IRQn     = 14,         /*!< Internal reserved */
105  Reserved13_IRQn     = 15,         /*!< Internal reserved */
106  InterCore_IRQn      = 16,         /*!< CIDU Inter Core Interrupt */
107  Reserved15_IRQn     = 17,         /*!< Internal reserved */
108  Reserved16_IRQn     = 18,         /*!< Internal reserved */
109
110  ↪/* ===== <Device> Specific Interrupt Numbers
111  ↪===== */
112  /* ToDo: add here your device specific external interrupt numbers. 19~1023 is
113  ↪reserved number for user. Maxmum interrupt supported
114  ↪could get from clicinfo.NUM_INTERRUPT. According the interrupt handlers
115  ↪defined in startup_Device.s
116  eg.: Interrupt for Timer#1      eclic_tim1_handler  ->  TIM1_IRQn */
117  SOC_INT19_IRQn       = 19,        /*!< Device Interrupt */
118  SOC_INT20_IRQn       = 20,        /*!< Device Interrupt */
119  SOC_INT21_IRQn       = 21,        /*!< Device Interrupt */
120  SOC_INT22_IRQn       = 22,        /*!< Device Interrupt */
121  SOC_INT23_IRQn       = 23,        /*!< Device Interrupt */

```

(continues on next page)

(continued from previous page)

```

111 SOC_INT24_IRQn      = 24,          /*!< Device Interrupt */
112 SOC_INT25_IRQn      = 25,          /*!< Device Interrupt */
113 SOC_INT26_IRQn      = 26,          /*!< Device Interrupt */
114 SOC_INT27_IRQn      = 27,          /*!< Device Interrupt */
115 SOC_INT28_IRQn      = 28,          /*!< Device Interrupt */
116 SOC_INT29_IRQn      = 29,          /*!< Device Interrupt */
117 SOC_INT30_IRQn      = 30,          /*!< Device Interrupt */
118 SOC_INT31_IRQn      = 31,          /*!< Device Interrupt */
119 SOC_INT32_IRQn      = 32,          /*!< Device Interrupt */
120 SOC_INT33_IRQn      = 33,          /*!< Device Interrupt */
121 SOC_INT34_IRQn      = 34,          /*!< Device Interrupt */
122 SOC_INT35_IRQn      = 35,          /*!< Device Interrupt */
123 SOC_INT36_IRQn      = 36,          /*!< Device Interrupt */
124 SOC_INT37_IRQn      = 37,          /*!< Device Interrupt */
125 SOC_INT38_IRQn      = 38,          /*!< Device Interrupt */
126 SOC_INT39_IRQn      = 39,          /*!< Device Interrupt */
127 SOC_INT40_IRQn      = 40,          /*!< Device Interrupt */
128 SOC_INT41_IRQn      = 41,          /*!< Device Interrupt */
129 SOC_INT42_IRQn      = 42,          /*!< Device Interrupt */
130 SOC_INT43_IRQn      = 43,          /*!< Device Interrupt */
131 SOC_INT44_IRQn      = 44,          /*!< Device Interrupt */
132 SOC_INT45_IRQn      = 45,          /*!< Device Interrupt */
133 SOC_INT46_IRQn      = 46,          /*!< Device Interrupt */
134 SOC_INT47_IRQn      = 47,          /*!< Device Interrupt */
135 SOC_INT48_IRQn      = 48,          /*!< Device Interrupt */
136 SOC_INT49_IRQn      = 49,          /*!< Device Interrupt */
137 SOC_INT50_IRQn      = 50,          /*!< Device Interrupt */
138 SOC_INT51_IRQn      = 51,          /*!< Device Interrupt */
139 SOC_INT52_IRQn      = 52,          /*!< Device Interrupt */
140 SOC_INT53_IRQn      = 53,          /*!< Device Interrupt */
141 SOC_INT54_IRQn      = 54,          /*!< Device Interrupt */
142 SOC_INT55_IRQn      = 55,          /*!< Device Interrupt */
143 SOC_INT56_IRQn      = 56,          /*!< Device Interrupt */
144 SOC_INT57_IRQn      = 57,          /*!< Device Interrupt */
145 SOC_INT58_IRQn      = 58,          /*!< Device Interrupt */
146 SOC_INT59_IRQn      = 59,          /*!< Device Interrupt */
147 SOC_INT60_IRQn      = 60,          /*!< Device Interrupt */
148 SOC_INT61_IRQn      = 61,          /*!< Device Interrupt */
149 SOC_INT62_IRQn      = 62,          /*!< Device Interrupt */
150 SOC_INT63_IRQn      = 63,          /*!< Device Interrupt */
151 #if defined(CFG_IRQ_NUM)
152     SOC_INT_MAX      = CFG_IRQ_NUM + SOC_EXTERNAL_MAP_TO_ECLIC_IRQn_OFFSET,
153 #else
154     SOC_INT_MAX,
155 #endif
156
157 PLIC_INT0_IRQn      = 0,
158 PLIC_INT1_IRQn      = 1,
159 PLIC_INT2_IRQn      = 2,
160 PLIC_INT3_IRQn      = 3,
161 PLIC_INT4_IRQn      = 4,
162 PLIC_INT5_IRQn      = 5,

```

(continues on next page)

(continued from previous page)

```

163     PLIC_INT6_IRQn           = 6,
164     PLIC_INT7_IRQn           = 7,
165     PLIC_INT8_IRQn           = 8,
166     PLIC_INT9_IRQn           = 9,
167     PLIC_INT10_IRQn          = 10,
168     PLIC_INT11_IRQn          = 11,
169     PLIC_INT12_IRQn          = 12,
170     PLIC_INT13_IRQn          = 13,
171     PLIC_INT14_IRQn          = 14,
172     PLIC_INT15_IRQn          = 15,
173     PLIC_INT16_IRQn          = 16,
174     PLIC_INT17_IRQn          = 17,
175     PLIC_INT18_IRQn          = 18,
176     PLIC_INT19_IRQn          = 19,
177     PLIC_INT20_IRQn          = 20,
178     PLIC_INT21_IRQn          = 21,
179     PLIC_INT22_IRQn          = 22,
180     PLIC_INT23_IRQn          = 23,
181     PLIC_INT24_IRQn          = 24,
182     PLIC_INT25_IRQn          = 25,
183     PLIC_INT26_IRQn          = 26,
184     PLIC_INT27_IRQn          = 27,
185     PLIC_INT28_IRQn          = 28,
186     PLIC_INT29_IRQn          = 29,
187     PLIC_INT30_IRQn          = 30,
188     PLIC_INT31_IRQn          = 31,
189     PLIC_INT32_IRQn          = 32,
190     PLIC_INT33_IRQn          = 33,
191     PLIC_INT34_IRQn          = 34,
192     PLIC_INT35_IRQn          = 35,
193     PLIC_INT36_IRQn          = 36,
194     PLIC_INT37_IRQn          = 37,
195     PLIC_INT38_IRQn          = 38,
196     PLIC_INT39_IRQn          = 39,
197     PLIC_INT40_IRQn          = 40,
198     PLIC_INT41_IRQn          = 41,
199     PLIC_INT42_IRQn          = 42,
200     PLIC_INT43_IRQn          = 43,
201     #if defined(CFG_IRQ_NUM)
202         PLIC_INIT_MAX          = CFG_IRQ_NUM + 1,
203     #else
204         PLIC_INIT_MAX,
205     #endif
206 } IRQn_Type;
207
208 #if defined(CFG_IRQ_NUM) && (CFG_IRQ_NUM > 38)
209 #define IRQn_OFFSET            0
210 #else
211 #define IRQn_OFFSET            32
212 #endif
213
214 #ifndef CFG_HAS_CLIC

```

(continues on next page)

(continued from previous page)

```

215  /* UART0 Interrupt */
216  /* NOTE: Take care the external uart irq may not work, it require a correct <Device> cpu_
↳ configuration */
217  /* NOTE: For latest 200/300 cpu, this UART0_IRQn maybe SOC_INT19_IRQn */
218  /* Please check Interrupts of Eval_SoC section in Nuclei_Processor_Integration_Guide.pdf_
↳ */
219  #define UART0_IRQn                (SOC_INT51_IRQn - IRQn_OFFSET)
220  /* QSPI Interrupt */
221  #define QSPI0_IRQn                (SOC_INT53_IRQn - IRQn_OFFSET)
222  #define QSPI1_IRQn                (SOC_INT54_IRQn - IRQn_OFFSET)
223  #define QSPI2_IRQn                (SOC_INT55_IRQn - IRQn_OFFSET)
224  #else
225  /* UART0 Interrupt */
226  #define UART0_IRQn                (PLIC_INT33_IRQn - IRQn_OFFSET)
227  /* QSPI Interrupt */
228  #define QSPI0_IRQn                (PLIC_INT35_IRQn - IRQn_OFFSET)
229  #define QSPI1_IRQn                (PLIC_INT36_IRQn - IRQn_OFFSET)
230  #define QSPI2_IRQn                (PLIC_INT37_IRQn - IRQn_OFFSET)
231  #endif
232
233  #define PLIC_UART0_IRQn            (PLIC_INT33_IRQn - IRQn_OFFSET)
234
235
236  /*_
↳
↳ */
237  /* ===== Exception Code Definition _
↳
↳ ===== */
238  /*_
↳
↳ */
239
240  typedef enum EXCn {
241      /* ===== Nuclei N/NX Specific Exception Code _
↳ ===== */
242      InsUnalign_EXCn                = 0,          /*!< Instruction address misaligned */
243      InsAccFault_EXCn               = 1,          /*!< Instruction access fault */
244      IlleIns_EXCn                   = 2,          /*!< Illegal instruction */
245      Break_EXCn                     = 3,          /*!< Beakpoint */
246      LdAddrUnalign_EXCn             = 4,          /*!< Load address misaligned */
247      LdFault_EXCn                   = 5,          /*!< Load access fault */
248      StAddrUnalign_EXCn             = 6,          /*!< Store or AMO address misaligned */
249      StAccessFault_EXCn             = 7,          /*!< Store or AMO access fault */
250      UmodeEcall_EXCn               = 8,          /*!< Environment call from User mode */
251      SmodeEcall_EXCn               = 9,          /*!< Environment call from S-mode */
252      MmodeEcall_EXCn               = 11,         /*!< Environment call from Machine_
↳ mode */
253      InsPageFault_EXCn              = 12,         /*!< Instruction page fault */
254      LdPageFault_EXCn               = 13,         /*!< Load page fault */
255      StPageFault_EXCn               = 15,         /*!< Store or AMO page fault */
256      StackOverflow_EXCn             = 24,         /*!< Stack overflow fault */
257      StackUnderflow_EXCn           = 25,         /*!< Stack underflow fault */

```

(continues on next page)

(continued from previous page)

```

258     NMI_EXCn                = 0xffff,          /*!< NMI interrupt */
259 } EXCn_Type;
260
261 /*
262  *===== Processor and Core Peripheral Section *
263  */
264 // NOTE: macros __NUCLEI_CORE_REV/__NUCLEI_N_REV/__NUCLEI_NX_REV are removed now
265 // NOTE: __FPU_PRESENT/__BITMANIP_PRESENT/__DSP_PRESENT/__VECTOR_PRESENT can be probed
266 // by compiler's -march= option
267 // See https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html
268
269 /*!< Set to 0, 1, or 2, 0 not present, 1 single floating point unit present, 2 double
270 floating point unit present */
271 #if !defined(__riscv_flen)
272 #define __FPU_PRESENT          0
273 #elif __riscv_flen == 32
274 #define __FPU_PRESENT          1
275 #else
276 #define __FPU_PRESENT          2
277 #endif
278
279 #if defined(__riscv_bitmanip)
280 #define __BITMANIP_PRESENT      1          /*!< Set to 1 if Bitmainpulation
281 extension is present */
282 #else
283 #define __BITMANIP_PRESENT      0          /*!< Set to 1 if Bitmainpulation
284 extension is present */
285 #endif
286
287 #if defined(__riscv_dsp)
288 #define __DSP_PRESENT           1          /*!< Set to 1 if Partial
289 SIMD(DSP) extension is present */
290 #else
291 #define __DSP_PRESENT           0          /*!< Set to 1 if Partial
292 SIMD(DSP) extension is present */
293 #endif
294
295 #if defined(__riscv_vector)
296 #define __VECTOR_PRESENT        1          /*!< Set to 1 if Vector
297 extension is present */
298 #else
299 #define __VECTOR_PRESENT        0          /*!< Set to 1 if Vector
300 extension is present */
301 #endif
302
303 // CPU IREGION Private Peripherals Offset against IREGION base specified by mirgb_info
304 // CSR

```

(continues on next page)

(continued from previous page)

```

296 // These offset information can be found in selected cpu series databook's IREGION_
    ↳ section.
297 // mirgb_info CSR can be found in Nuclei RISC-V ISA Spec
298 // You can also probe the cpu information using the general application/baremetal/
    ↳ cpuinfo case
299 // You can find the offset macros(IREGION_*_OFS) in riscv_encoding.h via search IREGION_
    ↳ Offsets
300
301 // NORMAL CPU Configuration: From Nuclei SDK 0.6.0, we recommend you to update the CPU_
    ↳ configuration macros defined in cpufeature.h
302 // Please don't modify the macros below directly
303
304 // NOTE: We use macros defined in cpufeature.h
305 // WARNING: Please dont modify macros directly below, you can change in cpufeature.h
306
307 // CPU Series and Version Configuration
308 // To set CPU REV and SERIES, just define CFG_CPU_VER/CFG_CPU_SERIES macros in_
    ↳ cpufeature.h
309 #define __NUCLEI_CPU_REV          CFG_CPU_VER          /*!< Nuclei CPU Core Revision,
    ↳ version X.Y.Z, this is for the CPU Core Version, you get from Nuclei, eg. N300 v3.10.1,
    ↳ it should be 0x030A01 */
310 #define __NUCLEI_CPU_SERIES      CFG_CPU_SERIES        /*!< Nuclei CPU Series, such_
    ↳ as 200/300/600/900, eg. 900 will be 0x0900 */
311
312 // CPU IREGION Base Address
313 // To set IREGION base, just define macro CFG_IREGION_BASE_ADDR in cpufeature.h
314 #ifndef CFG_IREGION_BASE_ADDR
315 // it is defined in system_<Device>.c, you should not use this variable CpuIRegionBase
316 // SystemIRegionInfo variable in previous release is removed, you should avoid to use it
317 // you should use macro __IREGION_BASEADDR defined in <Device>.h
318 extern volatile unsigned long CpuIRegionBase;
319 #define CPU_IREGION_BASE          CpuIRegionBase
320 #else
321 #define CPU_IREGION_BASE          CFG_IREGION_BASE_ADDR
322 #endif
323 #define __IREGION_BASEADDR        (CPU_IREGION_BASE)
324 #define __IINFO_BASEADDR          (__IREGION_BASEADDR + IREGION_IINFO_OFS)
325 #define __IINFO_MPASIZE_OFS       0
326 #define __IINFO_MPASIZE_ADDR      (__IINFO_BASEADDR + __IINFO_MPASIZE_OFS)
327
328 // ECLIC Configuration
329 // To enable ECLIC, just define macro CFG_HAS_CLIC/CFG_CLICINTCTLBITS/CFG_IRQ_NUM in_
    ↳ cpufeature.h
330 #ifdef CFG_HAS_CLIC
331 #define __ECLIC_PRESENT            1
332 #ifdef CFG_CLICINTCTLBITS
333 #define __ECLIC_INTCTLBITS        CFG_CLICINTCTLBITS
334 #endif
335 #define __ECLIC_INTNUM             (CFG_IRQ_NUM + SOC_EXTERNAL_MAP_TO_ECLIC_IRQn_OFFSET)
336 #else
337 #define __ECLIC_PRESENT            0
338 #endif

```

(continues on next page)

(continued from previous page)

```

339 #define __ECLIC_BASEADDR          (__IREGION_BASEADDR + IREGION_ECLIC_OFS)
340
341 // PLIC Configuration
342 // To enable PLIC, just define macro CFG_HAS_PLIC/CFG_IRQ_NUM in cpufeature.h
343 #ifdef CFG_HAS_PLIC
344 #define __PLIC_PRESENT              1
345 #define __PLIC_INTNUM              (CFG_IRQ_NUM + 1)
346 #else
347 #define __PLIC_PRESENT              0
348 #endif
349 #define __PLIC_BASEADDR            (__IREGION_BASEADDR + IREGION_PLIC_OFS)
350
351 // CPU System Timer Configuration
352 // To enable CPU System Timer, just define macro CFG_TMR_PRIVATE in cpufeature.h
353 #if defined(CFG_TMR_PRIVATE) || defined(CFG_TMR_BASE_ADDR)
354 #define __SYSTIMER_PRESENT          1
355 #else
356 #define __SYSTIMER_PRESENT          0
357 #endif
358 #ifdef CFG_TMR_BASE_ADDR // Maybe using timer out of cpu for <Device>
359 #define __SYSTIMER_BASEADDR         (CFG_TMR_BASE_ADDR)
360 #else
361 #define __SYSTIMER_BASEADDR         (__IREGION_BASEADDR + IREGION_TIMER_OFS)
362 #endif
363 #define __CLINT_TIMER_BASEADDR      (__SYSTIMER_BASEADDR + 0x1000)
364
365 // CPU System Timer SSTC Configuration
366 #if defined(CFG_HAS_ISA_SSTC)
367 #define __SSTC_PRESENT              1
368 #else
369 #define __SSTC_PRESENT              0
370 #endif
371
372 // CIDU Configuration
373 // To enable CIDU, just define macro CFG_HAS_IDU in cpufeature.h
374 #ifdef CFG_HAS_IDU
375 #define __CIDU_PRESENT              1
376 #else
377 #define __CIDU_PRESENT              0
378 #endif
379
380 #define __CIDU_BASEADDR              (__IREGION_BASEADDR + IREGION_IDU_OFS)
381
382 // SMP & CC Configuration
383 // To enable SMP & CC, just define macro CFG_HAS_SMP in cpufeature.h
384 #ifdef CFG_HAS_SMP
385 #define __SMPCC_PRESENT             1
386 #else
387 #define __SMPCC_PRESENT             0
388 #endif
389
390 #define __SMPCC_BASEADDR             (__IREGION_BASEADDR + IREGION_SMP_OFS)

```

(continues on next page)

(continued from previous page)

```

391
392 // PMP Configuration
393 // To enable PMP, just define macro CFG_HAS_PMP/__PMP_ENTRY_NUM in cpufeature.h
394 #ifndef CFG_HAS_PMP
395 #define __PMP_PRESENT                1
396 #define __PMP_ENTRY_NUM             CFG_PMP_ENTRY_NUM
397 #else
398 #define __PMP_PRESENT                0
399 #define __PMP_ENTRY_NUM             0
400 #endif
401
402 // TEE/sPMP Configuration
403 // To enable TEE, just define macro CFG_HAS_TEE in cpufeature.h
404 // TEE required PMP, please also make sure CFG_HAS_PMP defined
405 #ifndef CFG_HAS_TEE
406 #define __TEE_PRESENT                1
407 #define __SPMP_PRESENT               1
408 #define __SPMP_ENTRY_NUM            CFG_PMP_ENTRY_NUM
409 #ifndef CFG_HAS_SMPU
410 #define __SMPU_PRESENT               1
411 #define __SMPU_ENTRY_NUM            __SPMP_ENTRY_NUM
412 #endif
413 #else
414 #define __TEE_PRESENT                0
415 #define __SPMP_PRESENT               0
416 #define __SMPU_PRESENT               0
417 #define __SPMP_ENTRY_NUM            0
418 #endif
419
420 // ICache Configuration
421 // To enable ICACHE, just define macro CFG_HAS_ICACHE in cpufeature.h
422 #ifndef CFG_HAS_ICACHE
423 #define __ICACHE_PRESENT             1
424 #else
425 #define __ICACHE_PRESENT             0
426 #endif
427
428 // DCache Configuration
429 // To enable DCACHE, just define macro CFG_HAS_DCACHE in cpufeature.h
430 #ifndef CFG_HAS_DCACHE
431 #define __DCACHE_PRESENT             1
432 #else
433 #define __DCACHE_PRESENT             0
434 #endif
435
436 // CCM Configuration
437 // To enable CCM, just define macro CFG_HAS_IOCC in cpufeature.h
438 #ifndef CFG_HAS_IOCC
439 #define __CCM_PRESENT                1
440 #else
441 #define __CCM_PRESENT                0
442 #endif

```

(continues on next page)

(continued from previous page)

```

443
444 // PMA Configuration
445 // To enable PMA, just define macro CFG_HAS_PMA in cpufeature.h
446 #ifndef CFG_HAS_PMA
447 #define __PMA_PRESENT          1
448 #define __PMA_CSR_NUM         CFG_PMA_CSR_NUM
449 #define __PMA_SEC_CSR_NUM     CFG_PMA_SEC_CSR_NUM
450
451 #ifndef CFG_HAS_PMA_MACRO
452 #define __PMA_MACRO_PRESENT    1
453 #else
454 #define __PMA_MACRO_PRESENT    0
455 #endif
456 #else
457 #define __PMA_PRESENT          0
458 #define __PMA_MACRO_PRESENT    0
459 #endif
460
461 // HPM Configuration
462 // To enable HPM, just define macro CFG_HAS_HPM in cpufeature.h
463 #ifndef CFG_HAS_HPM
464 #define __HPM_PRESENT          1
465 #define __HPM_VER              CFG_HPM_VER
466 #endif
467
468 // SMODE Configuration
469 // To enable S-Mode, just define macro CFG_HAS_SMODE in cpufeature.h
470 #ifndef CFG_HAS_SMODE
471 #define __SMODE_PRESENT        1
472 #endif
473
474 // NICE Configuration
475 // To enable NICE, just define macro CFG_HAS_NICE in cpufeature.h
476 #ifndef CFG_HAS_NICE
477 #define __NICE_PRESENT          1
478 #else
479 #define __NICE_PRESENT          0
480 #endif
481
482 // VNICE Configuration
483 // To enable Vector NICE, just define macro CFG_HAS_VNICE in cpufeature.h
484 #ifndef CFG_HAS_VNICE
485 #define __VNICE_PRESENT         1
486 #else
487 #define __VNICE_PRESENT         0
488 #endif
489
490 #ifndef __INC_INTRINSIC_API
491 #define __INC_INTRINSIC_API      0          /*!< Set to 1 if intrinsic api_
↳ header files need to be included */
492 #endif
493

```

(continues on next page)

(continued from previous page)

```

494 #define __Vendor_SysTickConfig      0          /*!< Set to 1 if different_
↳ SysTick Config is used */
495 #define __Vendor_EXCEPTION          0          /*!< Set to 1 if vendor_
↳ exception handler is present */
496
497
498 /*!< Set this timer hartid if you have only 1 hart in your cpu, and you know the timer_
↳ hartid, just set it */
499 // #define __SYSTIMER_HARTID          0
500
501
502 #ifndef __HARTID_OFFSET
503 /**
504  * If your hart index is different to your hartid, you must define this __HARTID_OFFSET_
↳ macro.
505  * For example, if your cpu has 4 harts, and hartid start from 3, so the __HARTID_OFFSET_
↳ should set to 3.
506  * Which means hartid 3-6 means hart index 0-3, this is useful for the timer software_
↳ interrupt and timer interrupt trigger register location
507  */
508 #define __HARTID_OFFSET              0
509 #endif
510
511 /* Define boot hart id */
512 #ifndef BOOT_HARTID
513 /**
514  * Chosen boot hart id in current cluster when in soc system, need to align with the_
↳ value defined in startup_<Device>.S,
515  * should start from 0, taken the mhartid bit 0-7 value
516  */
517 #define BOOT_HARTID                  0
518 #endif
519
520 /** @} */ /* End of group Configuration_of_NMSIS */
521
522 // NOTE: Run Mode Control internally used Nuclei
523 // RUNMODE_* macros are defined in SoC/<Device>/runmode.mk, for internal usage not_
↳ intend for widely usage
524 #ifdef RUNMODE_IC_EN
525 #undef __ICACHE_PRESENT
526 #define __ICACHE_PRESENT              RUNMODE_IC_EN      /*!< Controlled by macro_
↳ RUNMODE_IC_EN */
527 #endif
528
529 #ifdef RUNMODE_DC_EN
530 #undef __DCACHE_PRESENT
531 #define __DCACHE_PRESENT              RUNMODE_DC_EN      /*!< Controlled by macro_
↳ RUNMODE_DC_EN */
532 #endif
533
534 #ifdef RUNMODE_CCM_EN
535 #undef __CCM_PRESENT

```

(continues on next page)

(continued from previous page)

```

536 #define __CCM_PRESENT          RUNMODE_CCM_EN          /*!< Controlled by macro
    ↳ RUNMODE_CCM_EN */
537 #endif
538
539 #include <nmsis_core.h>          /*!< Nuclei N/NX class processor and
    ↳ core peripherals */
540 #include "system_<Device>.h"    /*!< <Device> System */
541
542 /* ===== Start of section using anonymous unions
    ↳ ===== */
543
544 #ifdef <Device>_RTC_FREQ
545 #define RTC_FREQ                <Device>_RTC_FREQ
546 #else
547 #define RTC_FREQ                32768
548 #endif
549 // The TIMER frequency is just the RTC frequency
550 #define SOC_TIMER_FREQ          RTC_FREQ
551
552
553 /*
    ↳
    ↳
    ↳
554 /* ===== Device Specific Peripheral Section
    ↳
    ↳
    ↳
555 /*
    ↳
    ↳
    ↳
556
557
558 /** @addtogroup Device_Peripheral_peripherals
559     * @{
560     */
561
562 /**
563     * Platform definitions
564     */
565 // Interrupt Numbers, will be removed in future, please use SOC_INT_MAX - SOC_EXTERNAL_
    ↳ MAP_TO_ECLIC_IRQn_OFFSET
566 #define SOC_ECLIC_NUM_INTERRUPTS (SOC_INT_MAX - SOC_EXTERNAL_MAP_TO_ECLIC_IRQn_OFFSET)
567
568 // Interrupt Handler Definitions
569 #define SOC_MTIMER_HANDLER      eclic_mtip_handler
570 #define SOC_SOFTINT_HANDLER    eclic_msip_handler
571
572 /**
573     * @brief UART
574     */
575 typedef struct {
576     __IOM uint32_t TXFIFO;
577     __IOM uint32_t RXFIFO;
578     __IOM uint32_t TXCTRL;

```

(continues on next page)

(continued from previous page)

```

579     __IOM uint32_t RXCTRL;
580     __IOM uint32_t IE;
581     __IOM uint32_t IP;
582     __IOM uint32_t DIV;
583 } UART_TypeDef;
584
585 /**
586  * @brief QSPI
587  */
588 typedef struct {
589     __IOM uint32_t SCKDIV;
590     __IOM uint32_t SCKMODE;
591     __IOM uint32_t RESERVED0[1];
592     __IOM uint32_t FORCE;
593     __IOM uint32_t CSID;
594     __IOM uint32_t CSDEF;
595     __IOM uint32_t CSMODE;
596     __IOM uint32_t VERSION;
597     __IOM uint32_t RESERVED1[2];
598     __IOM uint32_t DELAY0;
599     __IOM uint32_t DELAY1;
600     __IOM uint32_t RESERVED2[4];
601     __IOM uint32_t FMT;
602     __IOM uint32_t RESERVED3;
603     __IOM uint32_t TXDATA;
604     __IOM uint32_t RXDATA;
605     __IOM uint32_t TXMARK;
606     __IOM uint32_t RXMARK;
607     __IOM uint32_t RESERVED4[2];
608     __IOM uint32_t FCTRL;
609     __IOM uint32_t FFMT;
610     __IOM uint32_t RESERVED5[2];
611     __IOM uint32_t IE;
612     __IOM uint32_t IP;
613     __IOM uint32_t FFMT1;
614     __IOM uint32_t STATUS;
615 } QSPI_TypeDef;
616
617 /*@}*/ /* end of group <Device>_Peripherals */
618
619
620 /* ===== End of section using anonymous unions =====
621 ↪ ===== */
622
623 /* Macros for memory access operations */
624 #define _REG8P(p, i) ((volatile uint8_t *) ((uintptr_t)(p) + ↪
625 ↪ (i)))
626 #define _REG16P(p, i) ((volatile uint16_t *) ((uintptr_t)(p) + ↪
627 ↪ (i)))
628 #define _REG32P(p, i) ((volatile uint32_t *) ((uintptr_t)(p) + ↪
629 ↪ (i)))
630 #define _REG64P(p, i) ((volatile uint64_t *) ((uintptr_t)(p) + ↪

```

(continues on next page)

(continued from previous page)

```

    ↪(i))))
627 #define _REG8(p, i)                ((*_REG8P(p, i)))
628 #define _REG16(p, i)               ((*_REG16P(p, i)))
629 #define _REG32(p, i)               ((*_REG32P(p, i)))
630 #define _REG64(p, i)               ((*_REG64P(p, i)))
631 #define REG8(addr)                 _REG8((addr), 0)
632 #define REG16(addr)                _REG16((addr), 0)
633 #define REG32(addr)                _REG32((addr), 0)
634 #define REG64(addr)                _REG64((addr), 0)
635
636 /* Macros for address type convert and access operations */
637 #define ADDR16(addr)                ((uint16_t)(uintptr_t)(addr))
638 #define ADDR32(addr)                ((uint32_t)(uintptr_t)(addr))
639 #define ADDR64(addr)                ((uint64_t)(uintptr_t)(addr))
640 #define ADDR8P(addr)                ((uint8_t *) (uintptr_t)(addr))
641 #define ADDR16P(addr)               ((uint16_t *) (uintptr_t)(addr))
642 #define ADDR32P(addr)               ((uint32_t *) (uintptr_t)(addr))
643 #define ADDR64P(addr)               ((uint64_t *) (uintptr_t)(addr))
644
645 /* Macros for Bit Operations */
646 #if __riscv_xlen == 32
647 #define BITMASK_MAX                  0xFFFFFFFFUL
648 #define BITOFS_MAX                   31
649 #else
650 #define BITMASK_MAX                  0xFFFFFFFFFFFFFFFFULL
651 #define BITOFS_MAX                   63
652 #endif
653
654 // BIT/BITS only support bit mask for __riscv_xlen
655 // For RISC-V 32 bit, it support mask 32 bit wide
656 // For RISC-V 64 bit, it support mask 64 bit wide
657 #define BIT(ofs)                     (0x1UL << (ofs))
658 #define BITS(start, end)              ((BITMASK_MAX) << (start) & (BITMASK_MAX) >>
    ↪(BITOFS_MAX - (end)))
659 #define GET_BIT(regval, bitofs)        (((regval) >> (bitofs)) & 0x1)
660 #define SET_BIT(regval, bitofs)        ((regval) |= BIT(bitofs))
661 #define CLR_BIT(regval, bitofs)        ((regval) &= (~BIT(bitofs)))
662 #define FLIP_BIT(regval, bitofs)       ((regval) ^= BIT(bitofs))
663 #define WRITE_BIT(regval, bitofs, val) CLR_BIT(regval, bitofs); ((regval) |= ((val)
    ↪<< bitofs) & BIT(bitofs))
664 #define CHECK_BIT(regval, bitofs)      (!((regval) & (0x1UL<<(bitofs))))
665 #define GET_BITS(regval, start, end)   (((regval) & BITS((start), (end))) >>
    ↪(start))
666 #define SET_BITS(regval, start, end)    ((regval) |= BITS((start), (end)))
667 #define CLR_BITS(regval, start, end)    ((regval) &= (~BITS((start), (end))))
668 #define FLIP_BITS(regval, start, end)   ((regval) ^= BITS((start), (end)))
669 #define WRITE_BITS(regval, start, end, val) CLR_BITS(regval, start, end); ((regval) |=
    ↪((val) << start) & BITS((start), (end)))
670 #define CHECK_BITS_ALL(regval, start, end) (!((~(regval)) & BITS((start), (end))))
671 #define CHECK_BITS_ANY(regval, start, end) ((regval) & BITS((start), (end)))
672
673 #define BITMASK_SET(regval, mask)      ((regval) |= (mask))

```

(continues on next page)

(continued from previous page)

```

674 #define BITMASK_CLR(regval, mask)      ((regval) &= (~(mask)))
675 #define BITMASK_FLIP(regval, mask)     ((regval) ^= (mask))
676 #define BITMASK_CHECK_ALL(regval, mask) (!((~(regval)) & (mask)))
677 #define BITMASK_CHECK_ANY(regval, mask) ((regval) & (mask))
678
679 /*
680  ===== Device Specific Peripheral Address Map
681  */
682
683 /* ToDo: add here your device peripherals base addresses
684    following is an example for timer */
685 /** @addtogroup Device_Peripheral_peripheralAddr
686     * @{
687     */
688 /* ILM/DLM/FLASHXIP and Peripheral base address */
689 #ifndef <Device>_FLASH_XIP_BASE
690 #define QSPI_FLASH_BASE      (0x200000000UL)      /*!< (FLASH      ) Base
691    ↳Address */
692 #else
693 #define QSPI_FLASH_BASE      (<Device>_FLASH_XIP_BASE) /*!< (FLASH      ) Base
694    ↳Address */
695 #endif
696
697 #ifndef CFG_ILM_BASE_ADDR
698 #define ONCHIP_ILM_BASE      (0x800000000UL)      /*!< (ILM      ) Base
699    ↳Address */
700 #else
701 #define ONCHIP_ILM_BASE      (CFG_ILM_BASE_ADDR)  /*!< (ILM      ) Base
702    ↳Address */
703 #endif
704
705 #ifndef CFG_DLM_BASE_ADDR
706 #define ONCHIP_DLM_BASE      (0x900000000UL)      /*!< (DLM      ) Base
707    ↳Address */
708 #else
709 #define ONCHIP_DLM_BASE      (CFG_DLM_BASE_ADDR)  /*!< (DLM      ) Base
710    ↳Address */
711 #endif
712
713 #ifndef <Device>_SYSMEM_BASE
714 #define ONCHIP_SRAM_BASE      (0xA00000000UL)      /*!< (SRAM      ) Base
715    ↳Address */
716 #else
717 #define ONCHIP_SRAM_BASE      (<Device>_SYSMEM_BASE) /*!< (SRAM      ) Base
718    ↳Address */
719 #endif

```

(continues on next page)

(continued from previous page)

```

713 #ifndef <Device>_PERIPS_BASE
714 #define <Device>_PERIPH_BASE      (0x10000000UL)          /*!< (Peripheral) Base
↳Address */
715 #else
716 #define <Device>_PERIPH_BASE      (<Device>_PERIPS_BASE)  /*!< (Peripheral) Base
↳Address */
717 #endif
718
719 /* Peripheral memory map */
720 #define UART0_BASE                (<Device>_PERIPH_BASE + 0x13000)  /*!< (UART0)
↳Base Address */
721 #define QSPI0_BASE                (<Device>_PERIPH_BASE + 0x14000)  /*!< (QSPI0)
↳Base Address */
722 #define UART1_BASE                (<Device>_PERIPH_BASE + 0x23000)  /*!< (UART1)
↳Base Address */
723 #define QSPI1_BASE                (<Device>_PERIPH_BASE + 0x24000)  /*!< (QSPI1)
↳Base Address */
724 #define QSPI2_BASE                (<Device>_PERIPH_BASE + 0x34000)  /*!< (QSPI2)
↳Base Address */
725
726 /** @} */ /* End of group Device_Peripheral_peripheralAddr */
727
728
729 /*
↳=====
↳*/
730 /* ===== Peripheral declaration
↳===== */
731 /*
↳=====
↳*/
732
733
734 /* ToDo: add here your device peripherals pointer definitions
following is an example for timer */
735 /** @addtogroup Device_Peripheral_declaration
* @{
736 *
737 */
738
739 #define UART0                    ((UART_TypeDef *) UART0_BASE)
740 #define QSPI0                    ((QSPI_TypeDef *) QSPI0_BASE)
741 #define UART1                    ((UART_TypeDef *) UART1_BASE)
742 #define QSPI1                    ((QSPI_TypeDef *) QSPI1_BASE)
743 #define QSPI2                    ((QSPI_TypeDef *) QSPI2_BASE)
744
745 #define SPI0_REG(offset)         _REG32(QSPI0_BASE, offset)
746 #define SPI1_REG(offset)         _REG32(QSPI1_BASE, offset)
747 #define SPI2_REG(offset)         _REG32(QSPI2_BASE, offset)
748 #define UART0_REG(offset)        _REG32(UART0_BASE, offset)
749 #define UART1_REG(offset)        _REG32(UART1_BASE, offset)
750
751 // Misc
752

```

(continues on next page)

(continued from previous page)

```

753 // Only used by Nuclei Internally, please dont use it
754 #define SIMULATION_EXIT(ret)    { __WMB(); UART0->RXFIFO = (ret);      \
755                                while (UART0->TXFIFO & (1<<31));        \
756                                UART0->TXFIFO = 4; }
757
758 extern uint32_t get_cpu_freq(void);
759 extern void delay_lms(uint32_t count);
760
761 /** @} */ /* End of group <Device> */
762
763 /** @} */ /* End of group Nuclei */
764
765 #ifdef __cplusplus
766 }
767 #endif
768
769 #endif /* __<Device>_H_ */

```

2.4 Register Mapping

The table below associates some common register names used in NMSIS to the register names used in Nuclei ISA Spec.

Note

The below register mapping maybe out of date, please refer to Nuclei ISA Spec for an updated version.

Table 6: Register names used in NMSIS related with the register names in ISA

NMSIS Register Name	200, 300, 600, 900	Register Description
Enhanced Core Local Interrupt Controller(ECLIC)		
ECLIC->CFG	cliccfg	ECLIC Global Configuration Register
ECLIC->INFO	clicinfo	ECLIC Global Information Register
ECLIC->MTH	mtm	ECLIC Global Machine Mode Threshold Register
ECLIC->CTRL[i].INTIP	clicintip[i]	ECLIC Interrupt Pending Register
ECLIC->CTRL[i].INTIE	clicintie[i]	ECLIC Interrupt Enable Register
ECLIC->CTRL[i].INTATTR	clicintattr[i]	ECLIC Interrupt Attribute Register
ECLIC->CTRL[i].INTCTRL	clicintctl[i]	ECLIC Interrupt Input Control Register
System Timer Unit(SysTimer)		
SysTimer->MTIMER	mtime_hi<<32 + mtime_lo	System Timer current value 64bits Register
SysTimer->MTIMERCMP	mtimecmp_hi<<32 + mtimecmp_lo	System Timer compare value 64bits Register
SysTimer->MSTOP	mstop	System Timer Stop Register
SysTimer->MSIP	msip	System Timer SW interrupt Register

2.5 NMSIS Core API

If you want to access doxygen generated NMSIS Core API, please click [NMSIS Core Doxygen API Documentation](#).

2.5.1 Version Control

group Version Control

Version #define symbols for NMSIS release specific C/C++ source code.

We followed the [semantic versioning 2.0.0²⁷](#) to control NMSIS version. The version format is **MAJOR.MINOR.PATCH**, increment the:

1. MAJOR version when you make incompatible API changes,
2. MINOR version when you add functionality in a backwards compatible manner, and
3. PATCH version when you make backwards compatible bug fixes.

The header file `nmsis_version.h` is included by each core header so that these definitions are available.

Example Usage for NMSIS Version Check:

```
#if defined(__NMSIS_VERSION) && (__NMSIS_VERSION >= 0x00010105)
    #warning "Yes, we have NMSIS 1.1.5 or later"
#else
    #error "We need NMSIS 1.1.5 or later!"
#endif
```

Unnamed Group

__NUCLEI_N_REV (0x0309)

Nuclei N class core revision number.

Reversion number format: [15:8] revision number, [7:0] patch number

Attention

Deprecated, this define is exclusive with [__NUCLEI_NX_REV](#) (page 71)

__NUCLEI_NX_REV (0x0207)

Nuclei NX class core revision number.

Reversion number format: [15:8] revision number, [7:0] patch number

Attention

Deprecated, this define is exclusive with [__NUCLEI_N_REV](#) (page 71)

__NUCLEI_CPU_REV (0x030A01)

Nuclei CPU core revision number.

Nuclei RISC-V CPU Revision Number vX.Y.Z, eg. v3.10.1

Attention

This define is exclusive with [__NUCLEI_CPU_SERIES](#) (page 72)

__NUCLEI_CPU_SERIES (0x0200)

Nuclei CPU core series.

Nuclei RISC-V CPU Series Number, eg, 0x200, 0x300, 0x600, 0x900 for 200, 300, 600, 900 series.

Attention

This define is used together with [__NUCLEI_CPU_REV](#) (page 71)

Defines**__NMSIS_VERSION_MAJOR** (1U)

Represent the NMSIS major version.

The NMSIS major version can be used to differentiate between NMSIS major releases.

__NMSIS_VERSION_MINOR (6U)

Represent the NMSIS minor version.

The NMSIS minor version can be used to query a NMSIS release update including new features.

__NMSIS_VERSION_PATCH (0U)

Represent the NMSIS patch version.

The NMSIS patch version can be used to show bug fixes in this package.

__NMSIS_VERSION (([__NMSIS_VERSION_MAJOR](#) (page 72) << 16U) | ([__NMSIS_VERSION_MINOR](#) (page 72) << 8) | [__NMSIS_VERSION_PATCH](#) (page 72))

Represent the NMSIS Version.

NMSIS Version format: **MAJOR.MINOR.PATCH**

- MAJOR: [__NMSIS_VERSION_MAJOR](#) (page 72), stored in bits [31:16] of [__NMSIS_VERSION](#) (page 72)
- MINOR: [__NMSIS_VERSION_MINOR](#) (page 72), stored in bits [15:8] of [__NMSIS_VERSION](#) (page 72)
- PATCH: [__NMSIS_VERSION_PATCH](#) (page 72), stored in bits [7:0] of [__NMSIS_VERSION](#) (page 72)

2.5.2 Compiler Control

group **Compiler Control**

Compiler agnostic #define symbols for generic c/c++ source code.

The NMSIS-Core provides the header file **nmsis_compiler.h** with consistent #define symbols for generate C or C++ source files that should be compiler agnostic. Each NMSIS compliant compiler should support the functionality described in this section.

The header file **nmsis_compiler.h** is also included by each Device Header File <device.h> so that these definitions are available.

²⁷ <https://semver.org/>

Defines

__has_builtin(x) (0)

__ASM __asm

Pass information from the compiler to the assembler.

__INLINE inline

Recommend that function should be inlined by the compiler.

__STATIC_INLINE static inline

Define a static function that may be inlined by the compiler.

__STATIC_FORCEINLINE __attribute__((always_inline)) static inline

Define a static function that should be always inlined by the compiler.

__NO_RETURN __attribute__((__noreturn__))

Inform the compiler that a function does not return.

__USED __attribute__((used))

Inform that a variable shall be retained in executable image.

__WEAK __attribute__((weak))

restrict pointer qualifier to enable additional optimizations.

__VECTOR_SIZE(x) __attribute__((vector_size(x)))

specified the vector size of the variable, measured in bytes

__PACKED __attribute__((packed, aligned(1)))

Request smallest possible alignment.

__PACKED_STRUCT struct __attribute__((packed, aligned(1)))

Request smallest possible alignment for a structure.

__PACKED_UNION union __attribute__((packed, aligned(1)))

Request smallest possible alignment for a union.

__UNALIGNED_UINT16_WRITE(addr, val) (void) (((struct [T_UINT16_WRITE](#) (page 74) *) (void *) (addr)) -> v) = (val))

Pointer for unaligned write of a uint16_t variable.

__UNALIGNED_UINT16_READ(addr) (((const struct [T_UINT16_READ](#) (page 74) *) (const void *) (addr)) -> v)

Pointer for unaligned read of a uint16_t variable.

__UNALIGNED_UINT32_WRITE(addr, val) (void) (((struct [T_UINT32_WRITE](#) (page 74) *) (void *) (addr)) -> v) = (val))

Pointer for unaligned write of a uint32_t variable.

__UNALIGNED_UINT32_READ(addr) (((const struct *T_UINT32_READ* (page 74) *) (const void *) (addr)) -> v)

Pointer for unaligned read of a uint32_t variable.

__ALIGNED(x) __attribute__((aligned(x)))

Minimum x bytes alignment for a variable.

__RESTRICT __restrict

restrict pointer qualifier to enable additional optimizations.

__COMPILER_BARRIER() *__ASM* (page 73) volatile("": "memory")

Barrier to prevent compiler from reordering instructions.

__USUALLY(exp) __builtin_expect((exp), 1)

provide the compiler with branch prediction information, the branch is usually true

__RARELY(exp) __builtin_expect((exp), 0)

provide the compiler with branch prediction information, the branch is rarely true

__INTERRUPT __attribute__((interrupt))

Use this attribute to indicate that the specified function is an interrupt handler run in Machine Mode.

__MACHINE_INTERRUPT __attribute__((interrupt ("machine")))

Use this attribute to indicate that the specified function is an interrupt handler run in Machine Mode.

__SUPERVISOR_INTERRUPT __attribute__((interrupt ("supervisor")))

Use this attribute to indicate that the specified function is an interrupt handler run in Supervisor Mode.

__USER_INTERRUPT __attribute__((interrupt ("user")))

Use this attribute to indicate that the specified function is an interrupt handler run in User Mode.

Variables

__PACKED_STRUCT T_UINT16_WRITE

Packed struct for unaligned uint16_t write access.

__PACKED_STRUCT T_UINT16_READ

Packed struct for unaligned uint16_t read access.

__PACKED_STRUCT T_UINT32_WRITE

Packed struct for unaligned uint32_t write access.

__PACKED_STRUCT T_UINT32_READ

Packed struct for unaligned uint32_t read access.

2.5.3 Core CSR Register Access

Click [Nuclei Core CSR](#)²⁸ to learn about Core CSR in Nuclei ISA Spec.

group Core CSR Register Access

Functions to access the Core CSR Registers.

The following functions or macros provide access to Core CSR registers.

- [Core CSR Encodings](#) (page 113)
- [Core CSR Registers](#) (page 86)

Defines

__RV_CSR_SWAP(csr, val)

CSR operation Macro for csrrw instruction.

Read the content of csr register to __v, then write content of val into csr register, then return __v

Parameters

- **csr** – CSR macro definition defined in [Core CSR Registers](#) (page 86), eg. [CSR_MSTATUS](#) (page 91)
- **val** – value to store into the CSR register

Returns

the CSR register value before written

__RV_CSR_READ(csr)

CSR operation Macro for csrr instruction.

Read the content of csr register to __v and return it

Parameters

- **csr** – CSR macro definition defined in [Core CSR Registers](#) (page 86), eg. [CSR_MSTATUS](#) (page 91)

Returns

the CSR register value

__RV_CSR_WRITE(csr, val)

CSR operation Macro for csrw instruction.

Write the content of val to csr register

Parameters

- **csr** – CSR macro definition defined in [Core CSR Registers](#) (page 86), eg. [CSR_MSTATUS](#) (page 91)
- **val** – value to store into the CSR register

__RV_CSR_READ_SET(csr, val)

CSR operation Macro for csrrs instruction.

Read the content of csr register to __v, then set csr register to be __v | val, then return __v

Parameters

²⁸ https://doc.nucleisys.com/nuclei_spec/isa/core_csr.html

- **csr** – CSR macro definition defined in *Core CSR Registers* (page 86), eg. *CSR_MSTATUS* (page 91)
- **val** – Mask value to be used with csrrs instruction

Returns

the CSR register value before written

__RV_CSR_SET(csr, val)

CSR operation Macro for csrrs instruction.

Set csr register to be csr_content | val

Parameters

- **csr** – CSR macro definition defined in *Core CSR Registers* (page 86), eg. *CSR_MSTATUS* (page 91)
- **val** – Mask value to be used with csrrs instruction

__RV_CSR_READ_CLEAR(csr, val)

CSR operation Macro for csrrc instruction.

Read the content of csr register to __v, then set csr register to be __v & ~val, then return __v

Parameters

- **csr** – CSR macro definition defined in *Core CSR Registers* (page 86), eg. *CSR_MSTATUS* (page 91)
- **val** – Mask value to be used with csrrc instruction

Returns

the CSR register value before written

__RV_CSR_CLEAR(csr, val)

CSR operation Macro for csrrc instruction.

Set csr register to be csr_content & ~val

Parameters

- **csr** – CSR macro definition defined in *Core CSR Registers* (page 86), eg. *CSR_MSTATUS* (page 91)
- **val** – Mask value to be used with csrrc instruction

__FENCE(p, s) *__ASM* (page 73) volatile ("fence " #p ", " #s : : "memory")

Execute fence instruction, p -> pred, s -> succ.

the FENCE instruction ensures that all memory accesses from instructions preceding the fence in program order (the predecessor set) appear earlier in the global memory order than memory accesses from instructions appearing after the fence in program order (the successor set). For details, please refer to The RISC-V Instruction Set Manual

Parameters

- **p** – predecessor set, such as iorw, rw, r, w
- **s** – successor set, such as iorw, rw, r, w

__RWMB() *__FENCE* (page 76)(iorw,iorw)

Read & Write Memory barrier.

__RMB() *__FENCE* (page 76)(ir,ir)
Read Memory barrier.

__WMB() *__FENCE* (page 76)(ow,ow)
Write Memory barrier.

__SMP_RWMB() *__FENCE* (page 76)(rw,rw)
SMP Read & Write Memory barrier.

__SMP_RMB() *__FENCE* (page 76)(r,r)
SMP Read Memory barrier.

__SMP_WMB() *__FENCE* (page 76)(w,w)
SMP Write Memory barrier.

__CPU_RELAX() *__ASM* (page 73) volatile ("" : : "memory")
CPU relax for busy loop.

Functions

__STATIC_FORCEINLINE void __FENCE_I (void)
Fence.i Instruction.
The FENCE.I instruction is used to synchronize the instruction and data streams.

__STATIC_INLINE void __switch_mode (uint8_t mode, uintptr_t stack, void(*entry_point)(void))
switch privilege from machine mode to others.
Execute into entry_point in mode(supervisor or user) with given stack

Parameters

- **mode** – privilege mode
- **stack** – predefined stack, size should set enough
- **entry_point** – a function pointer to execute

__STATIC_INLINE void __s_switch_mode (uint8_t mode, uintptr_t stack, void(*entry_point)(void))
switch privilege from supervisor mode to others.
Execute into entry_point in mode(user) with given stack

Parameters

- **mode** – privilege mode
- **stack** – predefined stack, size should set enough
- **entry_point** – a function pointer to execute

__STATIC_FORCEINLINE void __enable_irq (void)
Enable IRQ Interrupts.
Enables IRQ interrupts by setting the MIE-bit in the MSTATUS Register.

Remark

Can only be executed in Privileged modes.

__STATIC_FORCEINLINE void __disable_irq (void)

Disable IRQ Interrupts.

Disables IRQ interrupts by clearing the MIE-bit in the MSTATUS Register.

Remark

Can only be executed in Privileged modes.

__STATIC_FORCEINLINE void __enable_ext_irq (void)

Enable External IRQ Interrupts.

Enables External IRQ interrupts by setting the MEIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_ext_irq (void)

Disable External IRQ Interrupts.

Disables External IRQ interrupts by clearing the MEIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_timer_irq (void)

Enable Timer IRQ Interrupts.

Enables Timer IRQ interrupts by setting the MTIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_timer_irq (void)

Disable Timer IRQ Interrupts.

Disables Timer IRQ interrupts by clearing the MTIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_sw_irq (void)

Enable software IRQ Interrupts.

Enables software IRQ interrupts by setting the MSIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_sw_irq (void)

Disable software IRQ Interrupts.

Disables software IRQ interrupts by clearing the MSIE-bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_core_irq (uint32_t irq)

Disable Core IRQ Interrupt.

Disable Core IRQ interrupt by clearing the irq bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_core_irq (uint32_t irq)

Enable Core IRQ Interrupt.

Enable Core IRQ interrupt by setting the irq bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE uint32_t __get_core_irq_pending (uint32_t irq)

Get Core IRQ Interrupt Pending status.

Get Core IRQ interrupt pending status of irq bit.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __clear_core_irq_pending (uint32_t irq)

Clear Core IRQ Interrupt Pending status.

Clear Core IRQ interrupt pending status of irq bit.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_irq_s (void)

Enable IRQ Interrupts in supervisor mode.

Enables IRQ interrupts by setting the SIE-bit in the SSTATUS Register.

Remark

Can only be executed in Privileged modes.

__STATIC_FORCEINLINE void __disable_irq_s (void)

Disable IRQ Interrupts in supervisor mode.

Disables IRQ interrupts by clearing the SIE-bit in the SSTATUS Register.

Remark

Can only be executed in Privileged modes.

__STATIC_FORCEINLINE void __enable_ext_irq_s (void)

Enable External IRQ Interrupts in supervisor mode.

Enables External IRQ interrupts by setting the SEIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_ext_irq_s (void)

Disable External IRQ Interrupts in supervisor mode.

Disables External IRQ interrupts by clearing the SEIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_timer_irq_s (void)

Enable Timer IRQ Interrupts in supervisor mode.

Enables Timer IRQ interrupts by setting the STIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_timer_irq_s (void)

Disable Timer IRQ Interrupts in supervisor mode.

Disables Timer IRQ interrupts by clearing the STIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_sw_irq_s (void)

Enable software IRQ Interrupts in supervisor mode.

Enables software IRQ interrupts by setting the SSIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_sw_irq_s (void)

Disable software IRQ Interrupts in supervisor mode.

Disables software IRQ interrupts by clearing the SSIE-bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __disable_core_irq_s (uint32_t irq)

Disable Core IRQ Interrupt in supervisor mode.

Disable Core IRQ interrupt by clearing the irq bit in the SIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __enable_core_irq_s (uint32_t irq)

Enable Core IRQ Interrupt in supervisor mode.

Enable Core IRQ interrupt by setting the irq bit in the MIE Register.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE uint32_t __get_core_irq_pending_s (uint32_t irq)

Get Core IRQ Interrupt Pending status in supervisor mode.

Get Core IRQ interrupt pending status of irq bit.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_FORCEINLINE void __clear_core_irq_pending_s (uint32_t irq)

Clear Core IRQ Interrupt Pending status in supervisor mode.

Clear Core IRQ interrupt pending status of irq bit.

Remark

Can only be executed in Privileged modes, available for plic interrupt mode.

__STATIC_INLINE rv_counter_t __get_rv_cycle (void)

Read whole 64 bits value of mcycle counter.

This function will read the whole 64 bits of MCYCLE register

Remark

It will work for both RV32 and RV64 to get full 64bits value of MCYCLE

Returns

The whole 64 bits value of MCYCLE

__STATIC_FORCEINLINE void __set_rv_cycle (rv_counter_t cycle)

Set whole 64 bits value of mcycle counter.

This function will set the whole 64 bits of MCYCLE register

Remark

It will work for both RV32 and RV64 to set full 64bits value of MCYCLE

__STATIC_INLINE rv_counter_t __get_rv_instret (void)

Read whole 64 bits value of machine instruction-retired counter.

This function will read the whole 64 bits of MINSTRET register

Remark

It will work for both RV32 and RV64 to get full 64bits value of MINSTRET

Returns

The whole 64 bits value of MINSTRET

```
__STATIC_FORCEINLINE void __set_rv_instret (rv_counter_t instret)
```

Set whole 64 bits value of machine instruction-retired counter.

This function will set the whole 64 bits of MINSTRET register

Remark

It will work for both RV32 and RV64 to set full 64bits value of MINSTRET

```
__STATIC_INLINE rv_counter_t __get_rv_time (void)
```

Read whole 64 bits value of real-time clock.

This function will read the whole 64 bits of TIME register

Remark

It will work for both RV32 and RV64 to get full 64bits value of TIME

Attention

only available when user mode available

Returns

The whole 64 bits value of TIME CSR

```
__STATIC_FORCEINLINE unsigned long __read_cycle_csr (void)
```

Read the CYCLE register.

This function will read the CYCLE register without taking the CYCLEH register into account

Returns

32 bits value when XLEN=32 64 bits value when XLEN=64 TODO: XLEN=128 need to be supported

```
__STATIC_FORCEINLINE unsigned long __read_instret_csr (void)
```

Read the INSTRET register.

This function will read the INSTRET register without taking the INSTRETH register into account

Returns

32 bits value when XLEN=32 64 bits value when XLEN=64 TODO: XLEN=128 need to be supported

```
__STATIC_FORCEINLINE unsigned long __read_time_csr (void)
```

Read the TIME register.

This function will read the TIME register without taking the TIMEH register into account

Returns

32 bits value when XLEN=32 64 bits value when XLEN=64 TODO: XLEN=128 need to be supported

__STATIC_FORCEINLINE unsigned long __get_cluster_id (void)

Get cluster id of current cluster.

This function will get cluster id of current cluster in a multiple cluster system

Remark

mhartid bit 15-8 is designed for cluster id in nuclei subsystem reference design

Attention

function is allowed in machine mode only

Returns

The cluster id of current cluster

__STATIC_FORCEINLINE unsigned long __get_hart_index (void)

Get hart index of current cluster.

This function will get hart index of current cluster in a multiple cluster system, hart index is hartid - hartid offset, for example if your hartid is 1, and offset is 1, then hart index is 0

Attention

function is allowed in machine mode only

Returns

The hart index of current cluster

__STATIC_FORCEINLINE unsigned long __get_hart_id (void)

Get hart id of current cluster.

This function will get hart id of current cluster in a multiple cluster system

Remark

it will return full hartid not part of it for reference subsystem design, if your reference subsystem design has hartid offset, please define `__HARTID_OFFSET` in `<Device>.h`

Attention

function is allowed in machine mode only

Returns

The hart id of current cluster

__STATIC_FORCEINLINE unsigned long __get_cluster_id_s (void)

Get cluster id of current cluster in supervisor mode.

This function will get cluster id of current cluster in a multiple cluster system

Remark

hartid bit 15-8 is designed for cluster id in nuclei subsystem reference design

Attention

function is allowed in machine/supervisor mode, currently only present in 600/900 series from 2024 released version

Returns

The cluster id of current cluster

__STATIC_FORCEINLINE unsigned long __get_hart_index_s (void)

Get hart index of current cluster in supervisor mode.

This function will get hart index of current cluster in a multiple cluster system, hart index is hartid - hartid offset, for example if your hartid is 1, and offset is 1, then hart index is 0

Attention

function is allowed in machine/supervisor mode, currently only present in 600/900 series from 2024 released version

Returns

The hart index of current cluster

__STATIC_FORCEINLINE unsigned long __get_hart_id_s (void)

Get hart id of current cluster in supervisor mode.

This function will get hart id of current cluster in a multiple cluster system

Remark

it will return full hartid not part of it for reference subsystem design, if your reference subsystem design has hartid offset, please define `__HARTID_OFFSET` in `<Device>.h`

Attention

function is allowed in machine/supervisor mode, currently only present in 600/900 series from 2024 released version

Returns

The hart id of current cluster

2.5.4 Core CSR Encoding

Click [Nuclei Core CSR²⁹](#) to learn about Core CSR in Nuclei ISA Spec.

²⁹ https://doc.nucleisys.com/nuclei_spec/isa/core_csr.html

Core CSR Register Definitions

group **Core CSR Registers**

NMSIS Core CSR Register Definitions.

The following macros are used for CSR Register Definitions.

Defines

CSR_USTATUS 0x0

CSR_FFLAGS 0x1

CSR_FRM 0x2

CSR_FCSR 0x3

CSR_VSTART 0x8

CSR_VXSAT 0x9

CSR_VXRM 0xa

CSR_VCSR 0xf

CSR_SEED 0x15

CSR_JVT 0x17

CSR_CYCLE 0xc00

CSR_TIME 0xc01

CSR_INSTRET 0xc02

CSR_HPM_COUNTER3 0xc03

CSR_HPM_COUNTER4 0xc04

CSR_HPM_COUNTER5 0xc05

CSR_HPM_COUNTER6 0xc06

CSR_HPMCounter7 0xc07

CSR_HPMCounter8 0xc08

CSR_HPMCounter9 0xc09

CSR_HPMCounter10 0xc0a

CSR_HPMCounter11 0xc0b

CSR_HPMCounter12 0xc0c

CSR_HPMCounter13 0xc0d

CSR_HPMCounter14 0xc0e

CSR_HPMCounter15 0xc0f

CSR_HPMCounter16 0xc10

CSR_HPMCounter17 0xc11

CSR_HPMCounter18 0xc12

CSR_HPMCounter19 0xc13

CSR_HPMCounter20 0xc14

CSR_HPMCounter21 0xc15

CSR_HPMCounter22 0xc16

CSR_HPMCounter23 0xc17

CSR_HPMCounter24 0xc18

CSR_HPMCounter25 0xc19

CSR_HPMCounter26 0xc1a

CSR_HPMCounter27 0xc1b

CSR_HPMCOUNTER28 0xc1c

CSR_HPMCOUNTER29 0xc1d

CSR_HPMCOUNTER30 0xc1e

CSR_HPMCOUNTER31 0xc1f

CSR_VL 0xc20

CSR_VTYPE 0xc21

CSR_VLENB 0xc22

CSR_TIMEH 0xc81

CSR_TIMEH 0xc81

CSR_SSTATUS 0x100

CSR_SEDELEG 0x102

CSR_SIDELEG 0x103

CSR_SIE 0x104

CSR_STVEC 0x105

CSR_STVT 0x107

CSR_STVT 0x107

CSR_SCOUNTEREN 0x106

CSR_SENVCFG 0x10a

CSR_SSTATEEN0 0x10c

CSR_SSTATEEN1 0x10d

CSR_SSTATEEN2 0x10e

CSR_SSTATEEN3 0x10f

CSR_SSCRATCH 0x140

CSR_SEPC 0x141

CSR_SCAUSE 0x142

CSR_STVAL 0x143

CSR_SIP 0x144

CSR_STIMECMP 0x14d

CSR_STIMECMPH 0x15d

CSR_STIMECMPH 0x15d

CSR_SATP 0x180

CSR_SCONTEXT 0x5a8

CSR_VSSTATUS 0x200

CSR_VSIE 0x204

CSR_VSTVEC 0x205

CSR_VSSCRATCH 0x240

CSR_VSEPC 0x241

CSR_VSCAUSE 0x242

CSR_VSTVAL 0x243

CSR_VSIP 0x244

CSR_VSTIMECMP 0x24d

CSR_VSATP 0x280

CSR_HSTATUS 0x600

CSR_HEDELEG 0x602

CSR_HIDELEG 0x603

CSR_HIE 0x604

CSR_HTIMEDELTA 0x605

CSR_HCOUNTEREN 0x606

CSR_HGEIE 0x607

CSR_HENVCFG 0x60a

CSR_HSTATEEN0 0x60c

CSR_HSTATEEN1 0x60d

CSR_HSTATEEN2 0x60e

CSR_HSTATEEN3 0x60f

CSR_HTVAL 0x643

CSR_HIP 0x644

CSR_HVIP 0x645

CSR_HTINST 0x64a

CSR_HGATP 0x680

CSR_HCONTEXT 0x6a8

CSR_HGEIP 0xe12

CSR_SCOUNTOVF 0xda0

CSR_UTVT 0x7

CSR_UNXTI 0x45

CSR_UINTSTATUS 0x46

CSR_USCRATCHCSW 0x48

CSR_USCRATCHCSWL 0x49

CSR_SNXTI 0x145

CSR_SINTSTATUS 0x146

CSR_SSCRATCHCSW 0x148

CSR_SSCRATCHCSWL 0x149

CSR_MTVT 0x307

CSR_MTVT 0x307

CSR_MNXTI 0x345

CSR_MNXTI 0x345

CSR_MINTSTATUS 0x346

CSR_MINTSTATUS 0x346

CSR_MSCRATCHCSW 0x348

CSR_MSCRATCHCSW 0x348

CSR_MSCRATCHCSWL 0x349

CSR_MSCRATCHCSWL 0x349

CSR_MSTATUS 0x300

CSR_MISA 0x301

CSR_MEDELEG 0x302

CSR_MIDELEG 0x303

CSR_MIE 0x304

CSR_MTVEC 0x305

CSR_MCOUNTEREN 0x306

CSR_MENVCFG 0x30a

CSR_MENVCFGH 0x31a

CSR_MENVCFGH 0x31a

CSR_MSTATEEN0 0x30c

CSR_MSTATEEN1 0x30d

CSR_MSTATEEN2 0x30e

CSR_MSTATEEN3 0x30f

CSR_MCOUNTINHIBIT 0x320

CSR_MSCRATCH 0x340

CSR_MEPC 0x341

CSR_MCAUSE 0x342

CSR_MTVAL 0x343

CSR_MBADADDR 0x343

CSR_MIP 0x344

CSR_MTINST 0x34a

CSR_MTVAL2 0x34b

CSR_PMPCFG0 0x3a0

CSR_PMPCFG1 0x3a1

CSR_PMPCFG2 0x3a2

CSR_PMPCFG3 0x3a3

CSR_PMPCFG4 0x3a4

CSR_PMPCFG5 0x3a5

CSR_PMPCFG6 0x3a6

CSR_PMPCFG7 0x3a7

CSR_PMPCFG8 0x3a8

CSR_PMPCFG9 0x3a9

CSR_PMPCFG10 0x3aa

CSR_PMPCFG11 0x3ab

CSR_PMPCFG12 0x3ac

CSR_PMPCFG13 0x3ad

CSR_PMPCFG14 0x3ae

CSR_PMPCFG15 0x3af

CSR_PMPADDR0 0x3b0

CSR_PMPADDR1 0x3b1

CSR_PMPADDR2 0x3b2

CSR_PMPADDR3 0x3b3

CSR_PMPADDR4 0x3b4

CSR_PMPADDR5 0x3b5

CSR_PMPADDR6 0x3b6

CSR_PMPADDR7 0x3b7

CSR_PMPADDR8 0x3b8

CSR_PMPADDR9 0x3b9

CSR_PMPADDR10 0x3ba

CSR_PMPADDR11 0x3bb

CSR_PMPADDR12 0x3bc

CSR_PMPADDR13 0x3bd

CSR_PMPADDR14 0x3be

CSR_PMPADDR15 0x3bf

CSR_PMPADDR16 0x3c0

CSR_PMPADDR17 0x3c1

CSR_PMPADDR18 0x3c2

CSR_PMPADDR19 0x3c3

CSR_PMPADDR20 0x3c4

CSR_PMPADDR21 0x3c5

CSR_PMPADDR22 0x3c6

CSR_PMPADDR23 0x3c7

CSR_PMPADDR24 0x3c8

CSR_PMPADDR25 0x3c9

CSR_PMPADDR26 0x3ca

CSR_PMPADDR27 0x3cb

CSR_PMPADDR28 0x3cc

CSR_PMPADDR29 0x3cd

CSR_PMPADDR30 0x3ce

CSR_PMPADDR31 0x3cf

CSR_PMPADDR32 0x3d0

CSR_PMPADDR33 0x3d1

CSR_PMPADDR34 0x3d2

CSR_PMPADDR35 0x3d3

CSR_PMPADDR36 0x3d4

CSR_PMPADDR37 0x3d5

CSR_PMPADDR38 0x3d6

CSR_PMPADDR39 0x3d7

CSR_PMPADDR40 0x3d8

CSR_PMPADDR41 0x3d9

CSR_PMPADDR42 0x3da

CSR_PMPADDR43 0x3db

CSR_PMPADDR44 0x3dc

CSR_PMPADDR45 0x3dd

CSR_PMPADDR46 0x3de

CSR_PMPADDR47 0x3df

CSR_PMPADDR48 0x3e0

CSR_PMPADDR49 0x3e1

CSR_PMPADDR50 0x3e2

CSR_PMPADDR51 0x3e3

CSR_PMPADDR52 0x3e4

CSR_PMPADDR53 0x3e5

CSR_PMPADDR54 0x3e6

CSR_PMPADDR55 0x3e7

CSR_PMPADDR56 0x3e8

CSR_PMPADDR57 0x3e9

CSR_PMPADDR58 0x3ea

CSR_PMPADDR59 0x3eb

CSR_PMPADDR60 0x3ec

CSR_PMPADDR61 0x3ed

CSR_PMPADDR62 0x3ee

CSR_PMPADDR63 0x3ef

CSR_MSECCFG 0x747

CSR_TSELECT 0x7a0

CSR_TDATA1 0x7a1

CSR_TDATA2 0x7a2

CSR_TDATA3 0x7a3

CSR_TINFO 0x7a4

CSR_TCONTROL 0x7a5

CSR_MCONTEXT 0x7a8

CSR_MSCONTEXT 0x7aa

CSR_DCSR 0x7b0

CSR_DPC 0x7b1

CSR_DSCRATCH0 0x7b2

CSR_DSCRATCH1 0x7b3

CSR_MCYCLE 0xb00

CSR_MINSTRET 0xb02

CSR_MHPMCOUNTER3 0xb03

CSR_MHPMCOUNTER4 0xb04

CSR_MHPMCOUNTER5 0xb05

CSR_MHPMCOUNTER6 0xb06

CSR_MHPMCOUNTER7 0xb07

CSR_MHPMCOUNTER8 0xb08

CSR_MHPMCOUNTER9 0xb09

CSR_MHPMCOUNTER10 0xb0a

CSR_MHPMCOUNTER11 0xb0b

CSR_MHPMCOUNTER12 0xb0c

CSR_MHPMCOUNTER13 0xb0d

CSR_MHPMCOUNTER14 0xb0e

CSR_MHPMCOUNTER15 0xb0f

CSR_MHPMCOUNTER16 0xb10

CSR_MHPMCOUNTER17 0xb11

CSR_MHPMCOUNTER18 0xb12

CSR_MHPMCOUNTER19 0xb13

CSR_MHPMCOUNTER20 0xb14

CSR_MHPMCOUNTER21 0xb15

CSR_MHPMCOUNTER22 0xb16

CSR_MHPMCOUNTER23 0xb17

CSR_MHPMCOUNTER24 0xb18

CSR_MHPMCOUNTER25 0xb19

CSR_MHPMCOUNTER26 0xb1a

CSR_MHPMCOUNTER27 0xb1b

CSR_MHPMCOUNTER28 0xb1c

CSR_MHPMCOUNTER29 0xb1d

CSR_MHPMCOUNTER30 0xb1e

CSR_MHPMCOUNTER31 0xb1f

CSR_MHPMEVENT3 0x323

CSR_MHPMEVENT4 0x324

CSR_MHPMEVENT5 0x325

CSR_MHPMEVENT6 0x326

CSR_MHPMEVENT7 0x327

CSR_MHPMEVENT8 0x328

CSR_MHPMEVENT9 0x329

CSR_MHPMEVENT10 0x32a

CSR_MHPMEVENT11 0x32b

CSR_MHPMEVENT12 0x32c

CSR_MHPMEVENT13 0x32d

CSR_MHPMEVENT14 0x32e

CSR_MHPMEVENT15 0x32f

CSR_MHPMEVENT16 0x330

CSR_MHPMEVENT17 0x331

CSR_MHPMEVENT18 0x332

CSR_MHPMEVENT19 0x333

CSR_MHPMEVENT20 0x334

CSR_MHPMEVENT21 0x335

CSR_MHPMEVENT22 0x336

CSR_MHPMEVENT23 0x337

CSR_MHPMEVENT24 0x338

CSR_MHPMEVENT25 0x339

CSR_MHPMEVENT26 0x33a

CSR_MHPMEVENT27 0x33b

CSR_MHPMEVENT28 0x33c

CSR_MHPMEVENT29 0x33d

CSR_MHPMEVENT30 0x33e

CSR_MHPMEVENT31 0x33f

CSR_MVENDORID 0xf11

CSR_MARCHID 0xf12

CSR_MIMPID 0xf13

CSR_MHARTID 0xf14

CSR_MCONFIGPTR 0xf15

CSR_VSTIMECMPH 0x25d

CSR_HTIMEDELTAH 0x615

CSR_HENVCFGH 0x61a

CSR_HSTATEEN0H 0x61c

CSR_HSTATEEN1H 0x61d

CSR_HSTATEEN2H 0x61e

CSR_HSTATEEN3H 0x61f

CSR_CYCLEH 0xc80

CSR_INSTRETH 0xc82

CSR_HPMCOUNTER3H 0xc83

CSR_HPMCOUNTER4H 0xc84

CSR_HPMCOUNTER5H 0xc85

CSR_HPMCOUNTER6H 0xc86

CSR_HPMCOUNTER7H 0xc87

CSR_HPMCOUNTER8H 0xc88

CSR_HPMCOUNTER9H 0xc89

CSR_HPMCOUNTER10H 0xc8a

CSR_HPMCOUNTER11H 0xc8b

CSR_HPMCOUNTER12H 0xc8c

CSR_HPMCOUNTER13H 0xc8d

CSR_HPMCOUNTER14H 0xc8e

CSR_HPMCOUNTER15H 0xc8f

CSR_HPMCOUNTER16H 0xc90

CSR_HPMCOUNTER17H 0xc91

CSR_HPMCOUNTER18H 0xc92

CSR_HPMCOUNTER19H 0xc93

CSR_HPMCOUNTER20H 0xc94

CSR_HPMCOUNTER21H 0xc95

CSR_HPMCOUNTER22H 0xc96

CSR_HPMCOUNTER23H 0xc97

CSR_HPMCOUNTER24H 0xc98

CSR_HPMCOUNTER25H 0xc99

CSR_HPMCOUNTER26H 0xc9a

CSR_HPMCOUNTER27H 0xc9b

CSR_HPMCOUNTER28H 0xc9c

CSR_HPMCOUNTER29H 0xc9d

CSR_HPMCOUNTER30H 0xc9e

CSR_HPMCOUNTER31H 0xc9f

CSR_MSTATUSH 0x310

CSR_MSTATEEN0H 0x31c

CSR_MSTATEEN1H 0x31d

CSR_MSTATEEN2H 0x31e

CSR_MSTATEEN3H 0x31f

CSR_MHPMEVENT3H 0x723

CSR_MHPMEVENT4H 0x724

CSR_MHPMEVENT5H 0x725

CSR_MHPMEVENT6H 0x726

CSR_MHPMEVENT7H 0x727

CSR_MHPMEVENT8H 0x728

CSR_MHPMEVENT9H 0x729

CSR_MHPMEVENT10H 0x72a

CSR_MHPMEVENT11H 0x72b

CSR_MHPMEVENT12H 0x72c

CSR_MHPMEVENT13H 0x72d

CSR_MHPMEVENT14H 0x72e

CSR_MHPMEVENT15H 0x72f

CSR_MHPMEVENT16H 0x730

CSR_MHPMEVENT17H 0x731

CSR_MHPMEVENT18H 0x732

CSR_MHPMEVENT19H 0x733

CSR_MHPMEVENT20H 0x734

CSR_MHPMEVENT21H 0x735

CSR_MHPMEVENT22H 0x736

CSR_MHPMEVENT23H 0x737

CSR_MHPMEVENT24H 0x738

CSR_MHPMEVENT25H 0x739

CSR_MHPMEVENT26H 0x73a

CSR_MHPMEVENT27H 0x73b

CSR_MHPMEVENT28H 0x73c

CSR_MHPMEVENT29H 0x73d

CSR_MHPMEVENT30H 0x73e

CSR_MHPMEVENT31H 0x73f

CSR_MSECCFGH 0x757

CSR_MCYCLEH 0xb80

CSR_MINSTRETH 0xb82

CSR_MHPMCOUNTER3H 0xb83

CSR_MHPMCOUNTER4H 0xb84

CSR_MHPMCOUNTER5H 0xb85

CSR_MHPMCOUNTER6H 0xb86

CSR_MHPMCOUNTER7H 0xb87

CSR_MHPMCOUNTER8H 0xb88

CSR_MHPMCOUNTER9H 0xb89

CSR_MHPMCOUNTER10H 0xb8a

CSR_MHPMCOUNTER11H 0xb8b

CSR_MHPMCOUNTER12H 0xb8c

CSR_MHPMCOUNTER13H 0xb8d

CSR_MHPMCOUNTER14H 0xb8e

CSR_MHPMCOUNTER15H 0xb8f

CSR_MHPMCOUNTER16H 0xb90

CSR_MHPMCOUNTER17H 0xb91

CSR_MHPMCOUNTER18H 0xb92

CSR_MHPMCOUNTER19H 0xb93

CSR_MHPMCOUNTER20H 0xb94

CSR_MHPMCOUNTER21H 0xb95

CSR_MHPMCOUNTER22H 0xb96

CSR_MHPMCOUNTER23H 0xb97

CSR_MHPMCOUNTER24H 0xb98

CSR_MHPMCOUNTER25H 0xb99

CSR_MHPMCOUNTER26H 0xb9a

CSR_MHPMCOUNTER27H 0xb9b

CSR_MHPMCOUNTER28H 0xb9c

CSR_MHPMCOUNTER29H 0xb9d

CSR_MHPMCOUNTER30H 0xb9e

CSR_MHPMCOUNTER31H 0xb9f

CSR_SPMPCFG0 0x1A0

CSR_SPMPCFG1 0x1A1

CSR_SPMPCFG2 0x1A2

CSR_SPMPCFG3 0x1A3

CSR_SPMADDR0 0x1B0

CSR_SPMADDR1 0x1B1

CSR_SPMADDR2 0x1B2

CSR_SPMADDR3 0x1B3

CSR_SPMADDR4 0x1B4

CSR_SPMADDR5 0x1B5

CSR_SPMADDR6 0x1B6

CSR_SPMADDR7 0x1B7

CSR_SPMPADDR8 0x1B8

CSR_SPMPADDR9 0x1B9

CSR_SPMPADDR10 0x1BA

CSR_SPMPADDR11 0x1BB

CSR_SPMPADDR12 0x1BC

CSR_SPMPADDR13 0x1BD

CSR_SPMPADDR14 0x1BE

CSR_SPMPADDR15 0x1BF

CSR_SMPUCFG0 0x1A0

CSR_SMPUCFG1 0x1A1

CSR_SMPUCFG2 0x1A2

CSR_SMPUCFG3 0x1A3

CSR_SMPUADDR0 0x1B0

CSR_SMPUADDR1 0x1B1

CSR_SMPUADDR2 0x1B2

CSR_SMPUADDR3 0x1B3

CSR_SMPUADDR4 0x1B4

CSR_SMPUADDR5 0x1B5

CSR_SMPUADDR6 0x1B6

CSR_SMPUADDR7 0x1B7

CSR_SMPUADDR8 0x1B8

CSR_SMPUADDR9 0x1B9

CSR_SMPUADDR10 0x1BA

CSR_SMPUADDR11 0x1BB

CSR_SMPUADDR12 0x1BC

CSR_SMPUADDR13 0x1BD

CSR_SMPUADDR14 0x1BE

CSR_SMPUADDR15 0x1BF

CSR_SMPUSWITCH0 0x170

CSR_SMPUSWITCH1 0x171

CSR_MCLICBASE 0x350

CSR_UCODE 0x801

CSR_MILM_CTL 0x7C0

CSR_MDLM_CTL 0x7C1

CSR_MECC_CODE 0x7C2

CSR_MNVEC 0x7C3

CSR_MSUBM 0x7C4

CSR_MDCAUSE 0x7C9

CSR_MCACHE_CTL 0x7CA

CSR_MMISC_CTL 0x7D0

CSR_MTSPCSW 0x7D5

CSR_MSAVESTATUS 0x7D6

CSR_MSAVEEPC1 0x7D7

CSR_MSAVECAUSE1 0x7D8

CSR_MSAVEEPC2 0x7D9

CSR_MSAVECAUSE2 0x7DA

CSR_MSAVEDCAUSE1 0x7DB

CSR_MSAVEDCAUSE2 0x7DC

CSR_MTLB_CTL 0x7DD

CSR_MECC_LOCK 0x7DE

CSR_MFP16MODE 0x7E2

CSR_MMISC_CTL1 0x7E2

CSR_MSHADGPRLVL0 0x7E3

CSR_MSHADGPRLVL1 0x7E4

CSR_MECLIC_CTL 0x7E5

CSR_MTSP 0x7E6

CSR_LSTEPFORC 0x7E9

CSR_PUSHMSUBM 0x7EB

CSR_MTVT2 0x7EC

CSR_JALMNXTI 0x7ED

CSR_PUSHMCAUSE 0x7EE

CSR_PUSHMEPC 0x7EF

CSR_MPPICFG_INFO 0x7F0

CSR_MFIOCFG_INFO 0x7F1

CSR_MDEVB 0x7F3

CSR_MDEVM 0x7F4

CSR_MNOCB 0x7F5

CSR_MNOCM 0x7F6

CSR_MMACRO_DEV_EN 0xBC8

CSR_MMACRO_NOC_EN 0xBC9

CSR_MMACRO_CA_EN 0xBCA

CSR_MATTRI0_BASE 0x7F3

CSR_MATTRI0_MASK 0x7F4

CSR_MATTRI1_BASE 0x7F5

CSR_MATTRI1_MASK 0x7F6

CSR_MATTRI2_BASE 0x7F9

CSR_MATTRI2_MASK 0x7FA

CSR_MATTRI3_BASE 0x7FB

CSR_MATTRI3_MASK 0x7FC

CSR_MATTRI4_BASE 0x7FD

CSR_MATTRI4_MASK 0x7FE

CSR_MATTRI5_BASE 0xBE0

CSR_MATTRI5_MASK 0xBE1

CSR_MATTRI6_BASE 0xBE2

CSR_MATTRI6_MASK 0xBE3

CSR_MATTRI7_BASE 0xBE4

CSR_MATTRI7_MASK 0xBE5

CSR_SATTRI0_BASE 0x5F0

CSR_SATTRI0_MASK 0x5F1

CSR_SATTRI1_BASE 0x5F2

CSR_SATTRI1_MASK 0x5F3

CSR_SATTRI2_BASE 0x5F4

CSR_SATTRI2_MASK 0x5F5

CSR_SATTRI3_BASE 0x5F6

CSR_SATTRI3_MASK 0x5F7

CSR_SATTRI4_BASE 0x5F8

CSR_SATTRI4_MASK 0x5F9

CSR_SATTRI5_BASE 0x5FA

CSR_SATTRI5_MASK 0x5FB

CSR_SATTRI6_BASE 0x5FC

CSR_SATTRI6_MASK 0x5FD

CSR_SATTRI7_BASE 0x5FE

CSR_SATTRI7_MASK 0x5FF

CSR_MSMPCFG_INFO 0x7F7

CSR_MIRGB_INFO 0x7F7

CSR_SLEEPVALUE 0x811

CSR_TXEVT 0x812

CSR_WFE 0x810

CSR_JALSNXTI 0x947

CSR_STVT2 0x948

CSR_PUSHSCAUSE 0x949

CSR_PUSHSEPC 0x94A

CSR_PUSHSSUBM 0x94B

CSR_POPXRET 0x94C

CSR_STSPCSW 0x94D

CSR_SDCAUSE 0x9C0

CSR_SSUBM 0x9C4

CSR_SSHADGPRLVL0 0x9E3

CSR_SSHADGPRLVL1 0x9E4

CSR_SECLIC_CTL 0x9E5

CSR_STSP 0x9E6

CSR_MICFG_INFO 0xFC0

CSR_MDCFG_INFO 0xFC1

CSR_MCFG_INFO 0xFC2

CSR_MTLBCFG_INFO 0xFC3

CSR_MECC_CTL 0xBC0

CSR_MECC_STATUS 0xBC4

CSR_SAFETY_CRC_CTL 0x813

CSR_SAFETY_STL_STATUS 0x814

CSR_MSTACK_CTRL 0x7C6

CSR_MSTACK_CTL 0x7C6

CSR_MSTACK_BOUND 0x7C7

CSR_MSTACK_BASE 0x7C8

CSR_CCM_MBEGINADDR 0x7CB

CSR_CCM_MCOMMAND 0x7CC

CSR_CCM_MDATA 0x7CD

CSR_CCM_SUEN 0x7CE

CSR_CCM_SBEGINADDR 0x5CB

CSR_CCM_SCOMMAND 0x5CC

CSR_CCM_SDATA 0x5CD

CSR_CCM_UBEGINADDR 0x4CB

CSR_CCM_UCOMMAND 0x4CC

CSR_CCM_UDATA 0x4CD

CSR_CCM_FPIPE 0x4CF

CSR_SHARTID 0xDC0

CSR_MLWID 0x390

CSR_MWIDDELEG 0x738

CSR_SLWID 0x190

CSR_MSIP 0xBD8

CSR_MTIMECMP 0xBD9

CSR_MTIME 0xBDA

CSR_MSTOP 0xBDB

CSR_IRQCIP 0xBD0

CSR_IRQCIE 0xBD1

Other Core Related Macros

group Core CSR Encodings

NMSIS Core CSR Encodings.

The following macros are used for CSR encodings

Defines

MSTATUS_UIE 0x00000001

MSTATUS_SIE 0x00000002

MSTATUS_HIE 0x00000004

MSTATUS_MIE 0x00000008

MSTATUS_UPIE 0x00000010

MSTATUS_SPIE 0x00000020

MSTATUS_UBE 0x00000040

MSTATUS_MPIE 0x00000080

MSTATUS_SPP 0x00000100

MSTATUS_VS 0x00000600

MSTATUS_MPP 0x00001800

MSTATUS_FS 0x00006000

MSTATUS_XS 0x00018000

MSTATUS_MPRV 0x00020000

MSTATUS_SUM 0x00040000

MSTATUS_MXR 0x00080000

MSTATUS_TVM 0x00100000

MSTATUS_TW 0x00200000

MSTATUS_TSR 0x00400000

MSTATUS32_SD 0x80000000

MSTATUS_UXL 0x0000000300000000

MSTATUS_SXL 0x0000000C00000000

MSTATUS_SBE 0x0000001000000000

MSTATUS_MBE 0x0000002000000000

MSTATUS_GVA 0x0000004000000000

MSTATUS_MPV 0x0000008000000000

MSTATUS64_SD 0x8000000000000000

MSTATUS_FS_INITIAL 0x00002000

MSTATUS_FS_CLEAN 0x00004000

MSTATUS_FS_DIRTY 0x00006000

MSTATUS_VS_INITIAL 0x00000200

MSTATUS_VS_CLEAN 0x00000400

MSTATUS_VS_DIRTY 0x00000600

MSTATUSH_SBE 0x00000010

MSTATUSH_MBE 0x00000020

MSTATUSH_GVA 0x00000040

MSTATUSH_MPV 0x00000080

SSTATUS_UIE 0x00000001

SSTATUS_SIE 0x00000002

SSTATUS_UPIE 0x00000010

SSTATUS_SPIE 0x00000020

SSTATUS_UBE 0x00000040

SSTATUS_SPP 0x00000100

SSTATUS_VS 0x00000600

SSTATUS_FS 0x00006000

SSTATUS_XS 0x00018000

SSTATUS_SUM 0x00040000

SSTATUS_MXR 0x00080000

SSTATUS32_SD 0x80000000

SSTATUS_UXL 0x0000000300000000

SSTATUS64_SD 0x8000000000000000

USTATUS_UIE 0x00000001

USTATUS_UPIE 0x00000010

DCSR_XDEBUGVER (3U<<30)

DCSR_NDRESET (1<<29)

DCSR_FULLRESET (1<<28)

DCSR_EBREAKM (1<<15)

DCSR_EBREAKH (1<<14)

DCSR_EBREAKS (1<<13)

DCSR_EBREAKU (1<<12)

DCSR_STOPCYCLE (1<<10)

DCSR_STOPTIME (1<<9)

DCSR_CAUSE (7<<6)

DCSR_DEBUGINT (1<<5)

DCSR_HALT (1<<3)

DCSR_STEP (1<<2)

DCSR_PRV (3<<0)

DCSR_CAUSE_NONE 0

DCSR_CAUSE_SWBP 1

DCSR_CAUSE_HWBP 2

DCSR_CAUSE_DEBUGINT 3

DCSR_CAUSE_STEP 4

DCSR_CAUSE_HALT 5

MCONTROL_TYPE(xlen) (0xfULL<<((xlen)-4))

MCONTROL_DMODE(xlen) (1ULL<<((xlen)-5))

MCONTROL_MASKMAX(xlen) (0x3fULL<<((xlen)-11))

MCONTROL_SELECT (1<<19)

MCONTROL_TIMING (1<<18)

MCONTROL_ACTION (0x3f<<12)

MCONTROL_CHAIN (1<<11)

MCONTROL_MATCH (0xf<<7)

MCONTROL_M (1<<6)

MCONTROL_H (1<<5)

MCONTROL_S (1<<4)

MCONTROL_U (1<<3)

MCONTROL_EXECUTE (1<<2)

MCONTROL_STORE (1<<1)

MCONTROL_LOAD (1<<0)

MCONTROL_TYPE_NONE 0

MCONTROL_TYPE_MATCH 2

MCONTROL_ACTION_DEBUG_EXCEPTION 0

MCONTROL_ACTION_DEBUG_MODE 1

MCONTROL_ACTION_TRACE_START 2

MCONTROL_ACTION_TRACE_STOP 3

MCONTROL_ACTION_TRACE_EMIT 4

MCONTROL_MATCH_EQUAL 0

MCONTROL_MATCH_NAPOT 1

MCONTROL_MATCH_GE 2

MCONTROL_MATCH_LT 3

MCONTROL_MATCH_MASK_LOW 4

MCONTROL_MATCH_MASK_HIGH 5

MIP_SSIP (1 << *IRQ_S_SOFT* (page 129))

MIP_HSIP (1 << *IRQ_H_SOFT* (page 129))

MIP_MSIP (1 << *IRQ_M_SOFT* (page 129))

MIP_STIP (1 << *IRQ_S_TIMER* (page 129))

MIP_HTIP (1 << *IRQ_H_TIMER* (page 129))

MIP_MTIP (1 << *IRQ_M_TIMER* (page 129))

MIP_SEIP (1 << *IRQ_S_EXT* (page 129))

MIP_HEIP (1 << *IRQ_H_EXT* (page 129))

MIP_MEIP (1 << *IRQ_M_EXT* (page 129))

MIE_SSIE *MIP_SSIP* (page 118)

MIE_HSIE *MIP_HSIP* (page 118)

MIE_MSIE *MIP_MSIP* (page 118)

MIE_STIE *MIP_STIP* (page 118)

MIE_HTIE *MIP_HTIP* (page 118)

MIE_MTIE *MIP_MTIP* (page 118)

MIE_SEIE *MIP_SEIP* (page 118)

MIE_HEIE *MIP_HEIP* (page 118)

MIE_MEIE *MIP_MEIP* (page 118)

SIP_SSIP *MIP_SSIP* (page 118)

SIP_SSIP *MIP_SSIP* (page 118)

SIP_STIP *MIP_STIP* (page 118)

SIP_STIP *MIP_STIP* (page 118)

SIP_SEIP *MIP_SEIP* (page 118)

SIE_SSIE *MIP_SSIP* (page 118)

SIE_STIE *MIP_STIP* (page 118)

SIE_SEIE *MIP_SEIP* (page 118)

MIDELEG_SSIE *MIP_SSIP* (page 118)

MIDELEG_STIE *MIP_STIP* (page 118)

MIDELEG_SEIE *MIP_SEIP* (page 118)

MCAUSE_INTR (1ULL << (__riscv_xlen - 1))

MCAUSE_CAUSE 0x00000FFFUL

SCAUSE_INTR *MCAUSE_INTR* (page 119)

SCAUSE_CAUSE 0x000003FFUL

MENVCFG_CBIE_EN (0x11 << 4)

MENVCFG_CBIE_FLUSH (0x01 << 4)

MENVCFG_CBIE_INVALID (0x11 << 4)

SENVCFG_CBIE_EN (0x11 << 4)

SENVCFG_CBIE_FLUSH (0x01 << 4)

SENVCFG_CBIE_INVALID (0x11 << 4)

MENVCFG_FIOM 0x00000001

MENVCFG_LPE 0x00000004

MENVCFG_SSE 0x00000008

MENVCFG_CBIE 0x00000030

MENVCFG_CBCFE 0x00000040

MENVCFG_CBZE 0x00000080

MENVCFG_PMM 0x0000000300000000

MENVCFG_DTE 0x0800000000000000

MENVCFG_ADUE 0x2000000000000000

MENVCFG_PBMTE 0x4000000000000000

MENVCFG_STCE 0x8000000000000000

MENVCFGH_DTE 0x08000000

MENVCFGH_ADUE 0x20000000

MENVCFGH_PBMTE 0x40000000

MENVCFGH_STCE 0x80000000

SENVCFG_FIOM 0x00000001

SENVCFG_LPE 0x00000004

SENVCFG_SSE 0x00000008

SENVCFG_CBIE 0x00000030

SENVCFG_CBCFE 0x00000040

SENVCFG_CBZE 0x00000080

SENVCFG_PMM 0x0000000300000000

UCODE_OV (0x1)

CSR_MCACHE_CTL_IE 0x00000001

CSR_MCACHE_CTL_DE 0x00010000

WFE_WFE (0x1)

TXEVT_TXEVT (0x1)

SLEEPVALUE_SLEEPVALUE (0x1)

MCOUNTEREN_CY_SHIFT 0

MCOUNTEREN_TIME_SHIFT 1

MCOUNTEREN_IR_SHIFT 2

MCOUNTEREN_CY (1U << *MCOUNTEREN_CY_SHIFT* (page 121))

MCOUNTEREN_TIME (1U << *MCOUNTEREN_TIME_SHIFT* (page 121))

MCOUNTEREN_IR (1U << *MCOUNTEREN_IR_SHIFT* (page 121))

MCOUNTINHIBIT_CY *MCOUNTEREN_CY* (page 121)

MCOUNTINHIBIT_IR *MCOUNTEREN_IR* (page 121)

MILM_CTL_ILM_BPA (((1ULL<<((__riscv_xlen)-10))-1)<<10)

MILM_CTL_ILM_ECC_CHK_EN (1<<4)

MILM_CTL_ILM_RWECC (1<<3)

MILM_CTL_ILM_ECC_INJ_EN (1<<3)

MILM_CTL_ILM_ECC_EXCP_EN (1<<2)

MILM_CTL_ILM_ECC_EN (1<<1)

MILM_CTL_ILM_EN (1<<0)

MDLM_CTL_DLM_BPA (((1ULL<<((__riscv_xlen)-10))-1)<<10)

MDLM_CTL_DLM_ECC_CHK_EN (1<<4)

MDLM_CTL_DLM_RWECC (1<<3)

MDLM_CTL_DLM_ECC_INJ_EN (1<<3)

MDLM_CTL_DLM_ECC_EXCP_EN (1<<2)

MDLM_CTL_DLM_ECC_EN (1<<1)

MDLM_CTL_DLM_EN (1<<0)

MSUBM_PGPRIDX (0x1F<<15)

MSUBM_GPRIDX (0x1F<<10)

MSUBM_PTYPE (0x3<<8)

MSUBM_TTYPE (0x3<<6)

SSUBM_PGPRIDX (0x1F<<15)

SSUBM_GPRIDX (0x1F<<10)

SSUBM_PTYPE (0x3<<8)

SSUBM_TTYPE (0x3<<6)

MDCAUSE_MDCAUSE (0x7)

MMISC_CTL_HW_AUTO_CONTEXT (1<<21)

MMISC_CTL_LDSPEC_ENABLE (1<<12)

MMISC_CTL_SIJUMP_ENABLE (1<<11)

MMISC_CTL_IMRETURN_ENABLE (1<<10)

MMISC_CTL_NMI_CAUSE_FFF (1<<9)

MMISC_CTL_CODE_BUS_ERR (1<<8)

MMISC_CTL_MISALIGN (1<<6)

MMISC_CTL_ZC (1<<7)

MMISC_CTL_BPU (1<<3)

MECLIC_CTL_SHADOW_EN (1<<2)

MECLIC_CTL_TSP_EN (1<<1)

MECLIC_CTL_FEAT_EN (1<<0)

SECLIC_CTL_SHADOW_EN (1<<2)

SECLIC_CTL_TSP_EN (1<<1)

SECLIC_CTL_FEAT_EN (1<<0)

MCACHE_CTL_IC_EN (1<<0)

MCACHE_CTL_IC_SCPD_MOD (1<<1)

MCACHE_CTL_IC_ECC_EN (1<<2)

MCACHE_CTL_IC_ECC_EXCP_EN (1<<3)

MCACHE_CTL_IC_TRAM_ECC_INJ_EN (1<<4)

MCACHE_CTL_IC_RWTECC (1<<4)

MCACHE_CTL_IC_RWDECC (1<<5)

MCACHE_CTL_IC_DRAM_ECC_INJ_EN (1<<5)

MCACHE_CTL_IC_PF_EN (1<<6)

MCACHE_CTL_IC_CANCEL_EN (1<<7)

MCACHE_CTL_IC_ECC_CHK_EN (1<<8)

MCACHE_CTL_IC_CMO_PF_EN (1<<9)

MCACHE_CTL_DC_EN (1<<16)

MCACHE_CTL_DC_ECC_EN (1<<17)

MCACHE_CTL_DC_ECC_EXCP_EN (1<<18)

MCACHE_CTL_DC_TRAM_ECC_INJ_EN (1<<19)

MCACHE_CTL_DC_RWTECC (1<<19)

MCACHE_CTL_DC_RWDECC (1<<20)

MCACHE_CTL_DC_DRAM_ECC_INJ_EN (1<<20)

MCACHE_CTL_DC_ECC_CHK_EN (1<<21)

MCACHE_CTL_DC_CMO_PF_EN (1<<22)

MTVT2_MTVT2EN (1<<0)

MTVT2_COMMON_CODE_ENTRY (((1ULL<<((__riscv_xlen)-2))-1)<<2)

MCFG_INFO_TEE (1<<0)

MCFG_INFO_ECC (1<<1)

MCFG_INFO_CLIC (1<<2)

MCFG_INFO_PLIC (1<<3)

MCFG_INFO_FIO (1<<4)

MCFG_INFO_PPI (1<<5)

MCFG_INFO_NICE (1<<6)

MCFG_INFO_ILM (1<<7)

MCFG_INFO_DLM (1<<8)

MCFG_INFO_ICACHE (1<<9)

MCFG_INFO_DCACHE (1<<10)

MCFG_INFO_SMP (1<<11)

MCFG_INFO_DSP_N1 (1<<12)

MCFG_INFO_DSP_N2 (1<<13)

MCFG_INFO_DSP_N3 (1<<14)

MCFG_INFO_ZC_XLCZ_EXT (1<<15)

MCFG_INFO_IREGION_EXIST (1<<16)

MCFG_INFO_VP (0x3<<17)

MCFG_INFO_SEC_MODE (1<<19)

MCFG_INFO_ETRACE (1<<20)

MCFG_INFO_SAFETY_MECHANISM (0x3<<21)

MCFG_INFO_VNICE (1<<23)

MCFG_INFO_XLCZ (1<<24)

MCFG_INFO_ZILSD (1<<25)

MCFG_INFO_SSTC (1<<26)

MICFG_IC_SET (0xF<<0)

MICFG_IC_WAY (0x7<<4)

MICFG_IC_LSIZE (0x7<<7)

MICFG_IC_ECC (0x1<<10)

MICFG_ILM_SIZE (0x1F<<16)

MICFG_ILM_XONLY (0x1<<21)

MICFG_ILM_ECC (0x1<<22)

MDCFG_DC_SET (0xF<<0)

MDCFG_DC_WAY (0x7<<4)

MDCFG_DC_LSIZE (0x7<<7)

MDCFG_DC_ECC (0x1<<10)

MDCFG_DLM_SIZE (0x1F<<16)

MDCFG_DLM_ECC (0x1<<21)

MIRGB_INFO_IRG_BASE_ADDR_BOFS (10)

MIRGB_INFO_IREGION_SIZE_BOFS (1)

MPPICFG_INFO_PPI_SIZE (0x1F<<1)

MPPICFG_INFO_PPI_BPA (((1ULL<<((__riscv_xlen)-10))-1)<<10)

MFIOCFG_INFO_FIO_SIZE (0x1F<<1)

MFIOCFG_INFO_FIO_BPA (((1ULL<<((__riscv_xlen)-10))-1)<<10)

MECC_LOCK_ECC_LOCK (0x1)

MECC_CODE_CODE (0x1FF)

MECC_CODE_RAMID (0x1F<<16)

MECC_CODE_SRAMID (0x1F<<24)

MECC_CODE_RAMID_IC (0x1<<16)

MECC_CODE_RAMID_DC (0x1<<17)

MECC_CODE_RAMID_TLB (0x1<<18)

MECC_CODE_RAMID_ILM (0x1<<19)

MECC_CODE_RAMID_DLM (0x1<<20)

MECC_CODE_SRAMID_IC (0x1<<24)

MECC_CODE_SRAMID_DC (0x1<<25)

MECC_CODE_SRAMID_TLB (0x1<<26)

MECC_CODE_SRAMID_ILM (0x1<<27)

MECC_CODE_SRAMID_DLM (0x1<<28)

CCM_SUEN_SUEN (0x1<<0)

CCM_DATA_DATA (0x7<<0)

CCM_COMMAND_COMMAND (0x1F<<0)

IREGION_IINFO_OFS (0x0)

IREGION_DEBUG_OFS (0x10000)

IREGION_ECLIC_OFS (0x20000)

IREGION_TIMER_OFS (0x30000)

IREGION_SMP_OFS (0x40000)

IREGION_IDU_OFS (0x50000)

IREGION_PL2_OFS (0x60000)

IREGION_DPREFETCH_OFS (0x70000)

IREGION_PLIC_OFS (0x4000000)

MSTACK_CTRL_MODE (0x1<<2)

MSTACK_CTRL_UDF_EN (0x1<<1)

MSTACK_CTRL_OVF_TRACK_EN (0x1)

PRV_U 0

PRV_S 1

PRV_H 2

PRV_M 3

VM_MBARE 0

VM_MBB 1

VM_MBBID 2

VM_SV32 8

VM_SV39 9

VM_SV48 10

SATP32_MODE 0x80000000

SATP32_ASID 0x7FC00000

SATP32_PPN 0x003FFFFF

SATP64_MODE 0xF000000000000000

SATP64_ASID 0x0FFFF00000000000

SATP64_PPN 0x00000FFFFFFFFFFFFF

SATP_MODE_OFF 0

SATP_MODE_SV32 1

SATP_MODE_SV39 8

SATP_MODE_SV48 9

SATP_MODE_SV57 10

SATP_MODE_SV64 11

IRQ_S_SOFT 1

IRQ_H_SOFT 2

IRQ_M_SOFT 3

IRQ_S_TIMER 5

IRQ_H_TIMER 6

IRQ_M_TIMER 7

IRQ_S_EXT 9

IRQ_H_EXT 10

IRQ_M_EXT 11

IRQ_COP 12

IRQ_HOST 13

FRM_RNDMODE_RNE 0x0

FPU Round to Nearest, ties to Even.

FRM_RNDMODE_RTZ 0x1

FPU Round Towards Zero.

FRM_RNDMODE_RDN 0x2

FPU Round Down (towards -inf)

FRM_RNDMODE_RUP 0x3

FPU Round Up (towards +inf)

FRM_RNDMODE_RMM 0x4

FPU Round to nearest, ties to Max Magnitude.

FRM_RNDMODE_DYN 0x7

In instruction's rm, selects dynamic rounding mode.

In Rounding Mode register, Invalid

FFLAGS_AE_NX (1<<0)

FPU Inexact.

FFLAGS_AE_UF (1<<1)

FPU Underflow.

FFLAGS_AE_OF (1<<2)

FPU Overflow.

FFLAGS_AE_DZ (1<<3)

FPU Divide by Zero.

FFLAGS_AE_NV (1<<4)

FPU Invalid Operation.

FREG(idx) f##idx

Floating Point Register f0-f31, eg.

f0 -> *FREG(0)* (page 130)

PMP_R 0x01

PMP_W 0x02

PMP_X 0x04

PMP_A 0x18

PMP_A_TOR 0x08

PMP_A_NA4 0x10

PMP_A_NAPOT 0x18

PMP_L 0x80

PMP_SHIFT 2

PMP_COUNT 16

SPMP_R *PMP_R* (page 130)

SPMP_W *PMP_W* (page 130)

SPMP_X *PMP_X* (page 130)

SPMP_A *PMP_A* (page 130)

SPMP_A_TOR *PMP_A_TOR* (page 130)

SPMP_A_NA4 *PMP_A_NA4* (page 130)

SPMP_A_NAPOT *PMP_A_NAPOT* (page 130)

SPMP_U 0x40

SPMP_L *PMP_L* (page 130)

SPMP_SHIFT *PMP_SHIFT* (page 131)

SPMP_COUNT 16

SMPU_R *SPMP_R* (page 131)

SMPU_W *SPMP_W* (page 131)

SMPU_X *SPMP_X* (page 131)

SMPU_A *SPMP_A* (page 131)

SMPU_A_TOR *SPMP_A_TOR* (page 131)

SMPU_A_NA4 *SPMP_A_NA4* (page 131)

SMPU_A_NAPOT *SPMP_A_NAPOT* (page 131)

SMPU_S 0x80

SMPU_SHIFT *PMP_SHIFT* (page 131)

PTE_V 0x001

PTE_R 0x002

PTE_W 0x004

PTE_X 0x008

PTE_U 0x010

PTE_G 0x020

PTE_A 0x040

PTE_D 0x080

PTE_SOFT 0x300

PTE_PPN_SHIFT 10

PTE_TABLE(PTE) (((PTE) & (*PTE_V* (page 132) | *PTE_R* (page 132) | *PTE_W* (page 132) | *PTE_X* (page 132))) == *PTE_V* (page 132))

CAUSE_MISALIGNED_FETCH 0x0

End of Doxygen Group NMSIS_Core_CSR_Registers.

CAUSE_FAULT_FETCH 0x1

CAUSE_ILLEGAL_INSTRUCTION 0x2

CAUSE_BREAKPOINT 0x3

CAUSE_MISALIGNED_LOAD 0x4

CAUSE_FAULT_LOAD 0x5

CAUSE_MISALIGNED_STORE 0x6

CAUSE_FAULT_STORE 0x7

CAUSE_USER_ECALL 0x8

CAUSE_SUPERVISOR_ECALL 0x9

CAUSE_HYPERVISOR_ECALL 0xa

CAUSE_MACHINE_ECALL 0xb

CAUSE_FETCH_PAGE_FAULT 0xc

CAUSE_LOAD_PAGE_FAULT 0xd

CAUSE_STORE_PAGE_FAULT 0xf

MISALIGNED_FETCH (1 << *CAUSE_MISALIGNED_FETCH* (page 132))

FAULT_FETCH (1 << *CAUSE_FAULT_FETCH* (page 132))

ILLEGAL_INSTRUCTION (1 << *CAUSE_ILLEGAL_INSTRUCTION* (page 132))

BREAKPOINT (1 << *CAUSE_BREAKPOINT* (page 132))

MISALIGNED_LOAD (1 << *CAUSE_MISALIGNED_LOAD* (page 132))

FAULT_LOAD (1 << *CAUSE_FAULT_LOAD* (page 132))

MISALIGNED_STORE (1 << *CAUSE_MISALIGNED_STORE* (page 132))

FAULT_STORE (1 << *CAUSE_FAULT_STORE* (page 132))

USER_ECALL (1 << *CAUSE_USER_ECALL* (page 132))

FETCH_PAGE_FAULT (1 << *CAUSE_FETCH_PAGE_FAULT* (page 133))

LOAD_PAGE_FAULT (1 << *CAUSE_LOAD_PAGE_FAULT* (page 133))

STORE_PAGE_FAULT (1 << *CAUSE_STORE_PAGE_FAULT* (page 133))

DCAUSE_FAULT_FETCH_PMP 0x1

DCAUSE_FAULT_FETCH_INST 0x2

DCAUSE_FAULT_LOAD_PMP 0x1

DCAUSE_FAULT_LOAD_INST 0x2

DCAUSE_FAULT_LOAD_NICE 0x3

DCAUSE_FAULT_STORE_PMP 0x1

DCAUSE_FAULT_STORE_INST 0x2

CSR_XSTATUS *CSR_MSTATUS* (page 91)

CSR_XTVEC *CSR_MTVEC* (page 92)

CSR_XCOUNTEREN *CSR_MCOUNTEREN* (page 92)

CSR_XIE *CSR_MIE* (page 92)

CSR_XIP *CSR_MIP* (page 92)

CSR_XSCRATCH *CSR_MSCRATCH* (page 92)

CSR_XEPC *CSR_MEPC* (page 92)

CSR_XCAUSE *CSR_MCAUSE* (page 92)

CSR_XSUBM *CSR_MSUBM* (page 107)

CSR_XTVAL *CSR_MTVAL* (page 92)

CSR_XENVCFG *CSR_MENVCFG* (page 92)

CSR_XTVT CSR_MTVT

CSR_XTVT2 *CSR_MTVT2* (page 108)

CSR_XSCRATCHCSWL CSR_MSCRATCHCSWL

CSR_XSCRATCHCSW CSR_MSCRATCHCSW

CSR_XDCAUSE *CSR_MDCAUSE* (page 107)

CSR_XTSP *CSR_MTSP* (page 108)

CSR_XECLIC_CTL *CSR_MECLIC_CTL* (page 108)

CSR_JALNXNTI *CSR_JALMNXTI* (page 108)

CSR_XINTSTATUS **CSR_MINTSTATUS**

CSR_XNXTI **CSR_MNXTI**

CSR_PUSHXEPC *CSR_PUSHMEPC* (page 108)

CSR_PUSHXCAUSE *CSR_PUSHMCAUSE* (page 108)

CSR_PUSHXSUBM *CSR_PUSHMSUBM* (page 108)

XRET mret

eclic_xsip_handler eclic_msip_handler

eclic_xtip_handler eclic_mtip_handler

XSTATUS_XIE *MSTATUS_MIE* (page 113)

XECLIC_CTL_TSP_EN *MECLIC_CTL_TSP_EN* (page 123)

x_exc_entry *exc_entry* (page 805)

x_irq_entry *irq_entry* (page 805)

2.5.5 Register Define and Type Definitions

group Register Define and Type Definitions

Type definitions and defines for core registers.

Defines

__RISCV_XLEN 32

Refer to the width of an integer register in bits(either 32 or 64)

Typedefs

typedef unsigned long **rv_csr_t**

Type of Control and Status Register(CSR), depends on the XLEN defined in RISC-V.

typedef uint64_t **rv_counter_t**

Type of RISC-V Counter such as cycle, instret, time, depends on the XLEN defined in RISC-V, but for n100, it will be 32bit max.

Core

group **Base Register Define and Type Definitions**

Type definitions and defines for base core registers.

Typedefs

typedef *CSR_MMISCCTRL_Type* (page 144) **CSR_MMISCCTL_Type**

typedef *CSR_MMISCCTRL_Type* (page 144) **CSR_MMISC_CTL_Type**

typedef *CSR_MCACHECTL_Type* (page 147) **CSR_MCACHE_CTL_Type**

typedef *CSR_MILMCTL_Type* (page 149) **CSR_MILM_CTL_Type**

typedef *CSR_MDLMCTL_Type* (page 150) **CSR_DILM_CTL_Type**

typedef *CSR_MCFGINFO_Type* (page 151) **CSR_MCFG_INFO_Type**

typedef *CSR_MICFGINFO_Type* (page 153) **CSR_MICFG_INFO_Type**

typedef *CSR_MDCFGINFO_Type* (page 154) **CSR_MDCFG_INFO_Type**

typedef *CSR_MTLBCFGINFO_Type* (page 155) **CSR_MTLBCFG_INFO_Type**

typedef *CSR_MPPICFGINFO_Type* (page 157) **CSR_MPPICFG_INFO_Type**

typedef *CSR_MFIOCFGINFO_Type* (page 157) **CSR_MFIOCFG_INFO_Type**

typedef *CSR_MECCLOCK_Type* (page 158) **CSR_MECC_LOCK_Type**

typedef *CSR_MECCCODE_Type* (page 158) **CSR_MECC_CODE_Type**

union **CSR_MISA_Type**

#include <core_feature_base.h> Union type to access MISA CSR register.

Public Members

rv_csr_t (page 136) **a**

bit: 0 Atomic extension

rv_csr_t (page 136) **b**

bit: 1 B extension

rv_csr_t (page 136) **c**

bit: 2 Compressed extension

rv_csr_t (page 136) **d**

bit: 3 Double-precision floating-point extension

Type used for csr data access.

rv_csr_t (page 136) **e**

bit: 4 RV32E/64E base ISA

rv_csr_t (page 136) **f**

bit: 5 Single-precision floating-point extension

rv_csr_t (page 136) **g**

bit: 6 Reserved

rv_csr_t (page 136) **h**

bit: 7 Hypervisor extension

rv_csr_t (page 136) **i**

bit: 8 RV32I/64I/128I base ISA

rv_csr_t (page 136) **j**

bit: 9 Reserved

rv_csr_t (page 136) **k**

bit: 10 Reserved

rv_csr_t (page 136) **l**

bit: 11 Reserved

rv_csr_t (page 136) **m**

bit: 12 Integer Multiply/Divide extension

rv_csr_t (page 136) **n**

bit: 13 Tentatively reserved for User-Level Interrupts extension

rv_csr_t (page 136) **o**

bit: 14 Reserved

rv_csr_t (page 136) **p**

bit: 15 Tentatively reserved for Packed-SIMD extension

rv_csr_t (page 136) **q**

bit: 16 Quad-precision floating-point extension

rv_csr_t (page 136) **r**

bit: 17 Reserved

rv_csr_t (page 136) **s**

bit: 18 Supervisor mode implemented

rv_csr_t (page 136) **t**

bit: 19 Reserved

rv_csr_t (page 136) **u**

bit: 20 User mode implemented

rv_csr_t (page 136) **v**

bit: 21 Vector extension

rv_csr_t (page 136) **w**

bit: 22 Reserved

rv_csr_t (page 136) **x**

bit: 23 Non-standard extensions present

rv_csr_t (page 136) **y**

bit: 24 Reserved

rv_csr_t (page 136) **z**

bit: 25 Reserved

rv_csr_t (page 136) **_reserved0**

bit: 26..XLEN-3 Reserved

rv_csr_t (page 136) **mx1**

bit: XLEN-2..XLEN-1 Machine XLEN

struct *CSR_MISA_Type* (page 136)::[anonymous] **b**

Structure used for bit access.

union **CSR_MSTATUS_Type**

#include <core_feature_base.h> Union type to access MSTATUS CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0 Reserved

rv_csr_t (page 136) **sie**

bit: 1 supervisor interrupt enable flag

rv_csr_t (page 136) **_reserved1**

bit: 2 Reserved

rv_csr_t (page 136) **mie**

bit: 3 machine mode interrupt enable flag

rv_csr_t (page 136) **_reserved2**

bit: 4 Reserved

rv_csr_t (page 136) **spie**

bit: 5 supervisor mode interrupt enable flag

rv_csr_t (page 136) **ube**

bit: 6 U-mode non-instruction-fetch memory access big-endian enable flag

rv_csr_t (page 136) **mpie**

bit: 7 machine mode previous interrupt enable flag

rv_csr_t (page 136) **spp**

bit: 8 supervisor previous privilege mode

rv_csr_t (page 136) **vs**

bit: 9..10 vector status flag

rv_csr_t (page 136) **mpp**

bit: 11..12 machine previous privilege mode

rv_csr_t (page 136) **fs**

bit: 13..14 FS status flag

rv_csr_t (page 136) **xs**

bit: 15..16 XS status flag

rv_csr_t (page 136) **mprv**

bit: 17 Modify PRiVilege

rv_csr_t (page 136) **sum**

bit: 18 Supervisor Mode load and store protection

rv_csr_t (page 136) **mxr**

bit: 19 Make eXecutable Readable

rv_csr_t (page 136) **tvm**

bit: 20 Trap Virtual Memory

rv_csr_t (page 136) **tw**

bit: 21 Timeout Wait

rv_csr_t (page 136) **tsr**

bit: 22 Trap SRET

rv_csr_t (page 136) **spelp**

bit: 23 Supervisor mode Previous Expected Landing Pad (ELP) State

rv_csr_t (page 136) **sdt**

bit: 24 S-mode-disable-trap

rv_csr_t (page 136) **_reserved3**

bit: 25..30 Reserved

rv_csr_t (page 136) **sd**

bit: 31 Dirty status for XS or FS

struct *CSR_MSTATUS_Type* (page 139)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MSTATUSH_Type**

#include <core_feature_base.h> Union type to access MSTATUSH CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0..3 Reserved

rv_csr_t (page 136) **sbe**

bit: 4 S-mode non-instruction-fetch memory access big-endian enable flag

rv_csr_t (page 136) **mbe**

bit: 5 M-mode non-instruction-fetch memory access big-endian enable flag

rv_csr_t (page 136) **gva**

bit: 6 Guest Virtual Address

rv_csr_t (page 136) **mpv**

bit: 7 Machine Previous Virtualization Mode

rv_csr_t (page 136) **_reserved1**

bit: 8 Reserved

rv_csr_t (page 136) **mpelp**

bit: 9 Machine mode Previous Expected Landing Pad (ELP) State

rv_csr_t (page 136) **mdt**

bit: 10 M-mode-disable-trap

rv_csr_t (page 136) **_reserved5**

bit: 11..31 Reserved

struct *CSR_MSTATUSH_Type* (page 140)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MTVEC_Type**

#include <core_feature_base.h> Union type to access MTVEC CSR register.

Public Members

rv_csr_t (page 136) **mode**

bit: 0..5 interrupt mode control

rv_csr_t (page 136) **addr**

bit: 6..XLEN-1 mtvec address

struct *CSR_MTVEC_Type* (page 141)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MCAUSE_Type**

#include <core_feature_base.h> Union type to access MCAUSE CSR register.

Public Members

rv_csr_t (page 136) **exccode**

bit: 0..11 exception or interrupt code

rv_csr_t (page 136) **_reserved0**

bit: 12..15 Reserved

rv_csr_t (page 136) **mpil**

bit: 16..23 Previous interrupt level

rv_csr_t (page 136) **_reserved1**

bit: 24..26 Reserved

rv_csr_t (page 136) **mpie**

bit: 27 Interrupt enable flag before enter interrupt

rv_csr_t (page 136) **mpp**

bit: 28..29 Priviledge mode flag before enter interrupt

rv_csr_t (page 136) **minhv**

bit: 30 Machine interrupt vector table

rv_csr_t (page 136) **interrupt**

bit: XLEN-1 trap type.

0 means exception and 1 means interrupt

struct *CSR_MCAUSE_Type* (page 142)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MCOUNTINHIBIT_Type**

#include <core_feature_base.h> Union type to access MCOUNTINHIBIT CSR register.

Public Members

rv_csr_t (page 136) **cy**

bit: 0 1 means disable mcycle counter

rv_csr_t (page 136) **_reserved0**

bit: 1 Reserved

rv_csr_t (page 136) **ir**

bit: 2 1 means disable minstret counter

rv_csr_t (page 136) **_reserved1**

bit: 3..XLEN-1 Reserved

struct *CSR_MCOUNTINHIBIT_Type* (page 143)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MSUBM_Type**

#include <core_feature_base.h> Union type to access MSUBM CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0..5 Reserved 0

rv_csr_t (page 136) **typ**

bit: 6..7 Current sub-mode: 0: Normal Machine Mode; 1: Interrupt Handling Mode; 2: Exception Handling Mode; 3: NMI Handling Mode.

rv_csr_t (page 136) **ptyp**

bit: 8..9 sub-mode before entering the trap: 0: Normal Machine Mode; 1: Interrupt Handling Mode; 2: Exception Handling Mode; 3: NMI Handling Mode.

rv_csr_t (page 136) **gpridx**

bit: 10..14 Current Register Group Select

rv_csr_t (page 136) **pgpridx**

bit: 15..19 Previous Register Group Select

rv_csr_t (page 136) **_reserved1**

bit: 20..XLEN-1 Reserved 0

struct *CSR_MSUBM_Type* (page 143)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MDCAUSE_Type**

#include <core_feature_base.h> Union type to access MDCAUSE CSR register.

Public Members

rv_csr_t (page 136) **mdcause**

bit: 0..2 More detailed exception information as MCAUSE supplement

rv_csr_t (page 136) **_reserved0**

bit: 3..XLEN-1 Reserved

struct *CSR_MDCAUSE_Type* (page 144)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MMISCCTRL_Type**

#include <core_feature_base.h> Union type to access MMISC_CTRL CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0 Reserved

rv_csr_t (page 136) **zclsd_en**

bit: 1 Control the Zclsd will uses the Zcf extension encoding or not

rv_csr_t (page 136) **_reserved1**

bit: 2 Reserved

rv_csr_t (page 136) **bpu**

bit: 3 dynamic prediction enable flag

rv_csr_t (page 136) **_reserved2**

bit: 4..5 Reserved

rv_csr_t (page 136) **misalign**

bit: 6 misaligned access support flag

rv_csr_t (page 136) **zcmt_zcmp**

bit: 7 Zc Ext uses the cfdsp of D Ext's encoding or not

rv_csr_t (page 136) **core_buserr**

bit: 8 core bus error exception or interrupt

rv_csr_t (page 136) **nmi_cause**

bit: 9 mnvec control and nmi mcase exccode

rv_csr_t (page 136) **imreturn_en**

bit: 10 IMRETURN mode of trace

rv_csr_t (page 136) **sijump_en**

bit: 11 SIJUMP mode of trace

rv_csr_t (page 136) **ldspec_en**

bit: 12 enable load speculative goes to mem interface

rv_csr_t (page 136) **_reserved3**

bit: 13 Reserved

rv_csr_t (page 136) **dbg_sec**

bit: 14 debug access mode, removed in latest releases

rv_csr_t (page 136) **_reserved4**

bit: 15..16 Reserved

rv_csr_t (page 136) **csr_excl_enable**

bit: 17 Exclusive instruction(lr,sc) on Non-cacheable/Device memory can send exclusive flag in memory bus

rv_csr_t (page 136) **_reserved5**

bit: 18..19 Reserved

rv_csr_t (page 136) **lsu_allow_diff_en**

bit: 20 LSU access allows the next operation can outstanding transactions when the current transaction has not been completed

rv_csr_t (page 136) **hw_auto_context**

bit: 21 Hardware auto context saving and restoring enable

rv_csr_t (page 136) **_reserved6**

bit: 22..XLEN-1 Reserved

struct *CSR_MMISCCTRL_Type* (page 144)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MMISC_CTL1_Type**

#include <core_feature_base.h> Union type to access MMISC_CTL1 CSR register.

Public Members

rv_csr_t (page 136) **fp16mode**

bit: 0 16 bit float precision mode

rv_csr_t (page 136) **vlsu_ooo_4k_mode**

bit: 1 Control the size of address check region for vlsu ooo

rv_csr_t (page 136) **vlsu_ooo_force_va_4k**

bit: 2 Control the size of virtual address check region for vlsu ooo

rv_csr_t (page 136) **vlsu_ooo_en**

bit: 3 Control the enable of vlsu ooo feature

rv_csr_t (page 136) **vlsu_cof_en**

bit: 4 Control the enable of vlsu check-only first feature

rv_csr_t (page 136) **vlm_path_en**

bit: 5 Control vlm dedicated path enable

rv_csr_t (page 136) **rvv_v1_0_cmpt**

bit: 6 Control some vpu instruction behaviour is compatible with rvv1.0

rv_csr_t (page 136) **_reserved0**

bit: 7..XLEN-1 Reserved

struct *CSR_MMISC_CTL1_Type* (page 146)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MCACHECTL_Type**

#include <core_feature_base.h> Union type to access MCACHE_CTL CSR register.

Public Members

rv_csr_t (page 136) **ic_en**

bit: 0 I-Cache enable

rv_csr_t (page 136) **ic_scpd_mod**

bit: 1 Scratchpad mode, 0: Scratchpad as ICache Data RAM, 1: Scratchpad as ILM SRAM

rv_csr_t (page 136) **ic_ecc_en**

bit: 2 I-Cache ECC enable

rv_csr_t (page 136) **ic_ecc_excp_en**

bit: 3 I-Cache 2bit ECC error exception enable

rv_csr_t (page 136) **ic_rwtecc**

bit: 4 Control I-Cache Tag Ram ECC code injection

rv_csr_t (page 136) **ic_rwdecc**

bit: 5 Control I-Cache Data Ram ECC code injection

rv_csr_t (page 136) **ic_pf_en**

bit: 6 I-Cache prefetch enable

rv_csr_t (page 136) **ic_cancel_en**

bit: 7 I-Cache change flow canceling enable control

rv_csr_t (page 136) **ic_ecc_chk_en**

bit: 8 I-Cache check ECC codes enable

rv_csr_t (page 136) **ic_prefetch_en**

bit: 9 I-Cache CMO prefetch enable control

rv_csr_t (page 136) **ic_burst_type**

bit: 10 I-Cache Burst type control

rv_csr_t (page 136) **_reserved0**

bit: 11..15 Reserved

rv_csr_t (page 136) **dc_en**

bit: 16 DCache enable

rv_csr_t (page 136) **dc_ecc_en**

bit: 17 D-Cache ECC enable

rv_csr_t (page 136) **dc_ecc_excp_en**

bit: 18 D-Cache 2bit ECC error exception enable

rv_csr_t (page 136) **dc_rwt ecc**

bit: 19 Control D-Cache Tag Ram ECC code injection

rv_csr_t (page 136) **dc_rwdecc**

bit: 20 Control D-Cache Data Ram ECC code injection

rv_csr_t (page 136) **dc_ecc_chk_en**

bit: 21 D-Cache check ECC codes enable

rv_csr_t (page 136) **dc_prefetch_en**

bit: 22 D-Cache CMO prefetch enable control

rv_csr_t (page 136) **dc_burst_type**

bit: 23 D-Cache Burst type control

rv_csr_t (page 136) **_reserved1**

bit: 24..XLEN-1 Reserved

struct *CSR_MCACHECTL_Type* (page 147)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MSAVESTATUS_Type**

#include <core_feature_base.h> Union type to access MSAVESTATUS CSR register.

Public Members

rv_csr_t (page 136) **mpie1**

bit: 0 interrupt enable flag of first level NMI/exception nesting

rv_csr_t (page 136) **mpp1**

bit: 1..2 privileged mode of first level NMI/exception nesting

rv_csr_t (page 136) **_reserved0**

bit: 3..5 Reserved

rv_csr_t (page 136) **ptyp1**

bit: 6..7 NMI/exception type of before first nesting

rv_csr_t (page 136) **mpie2**

bit: 8 interrupt enable flag of second level NMI/exception nesting

rv_csr_t (page 136) **mpp2**

bit: 9..10 privileged mode of second level NMI/exception nesting

rv_csr_t (page 136) **_reserved1**

bit: 11..13 Reserved

rv_csr_t (page 136) **ptyp2**

bit: 14..15 NMI/exception type of before second nesting

rv_csr_t (page 136) **_reserved2**

bit: 16..XLEN-1 Reserved

struct *CSR_MSAVESTATUS_Type* (page 148)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **w**

Type used for csr data access.

union **CSR_MILMCTL_Type**

#include <core_feature_base.h> Union type to access MILM_CTL CSR register.

Public Members

rv_csr_t (page 136) **ilm_en**

bit: 0 ILM enable

rv_csr_t (page 136) **ilm_ecc_en**

bit: 1 ILM ECC enable

rv_csr_t (page 136) **ilm_ecc_excp_en**

bit: 2 ILM ECC exception enable

rv_csr_t (page 136) **ilm_rwecc**

bit: 3 Control mecc_code write to ilm, simulate error injection

rv_csr_t (page 136) **ilm_ecc_chk_en**

bit: 4 ILM check ECC codes enable

rv_csr_t (page 136) **ilm_va_en**

bit: 5 Using virtual address to judge ILM access

rv_csr_t (page 136) **dis_lsu_ilm**

bit: 6 Disable lsu access ILM

rv_csr_t (page 136) **_reserved0**

bit: 7..9 Reserved

rv_csr_t (page 136) **ilm_bpa**

bit: 10..XLEN-1 ILM base physical address

struct *CSR_MILMCTL_Type* (page 149)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MDLMCTL_Type**

#include <core_feature_base.h> Union type to access MDLM_CTL CSR register.

Public Members

rv_csr_t (page 136) **d1m_en**

bit: 0 DLM enable

rv_csr_t (page 136) **d1m_ecc_en**

bit: 1 DLM ECC enable

rv_csr_t (page 136) **d1m_ecc_excp_en**

bit: 2 DLM ECC exception enable

rv_csr_t (page 136) **d1m_rwecc**

bit: 3 Control mecc_code write to d1m, simulate error injection

rv_csr_t (page 136) **d1m_ecc_chk_en**

bit: 4 DLM check ECC codes enable

rv_csr_t (page 136) **d1m_va_en**

bit: 5 Using virtual address to judge DLM access

rv_csr_t (page 136) **dis_lsu_d1m**

bit: 6 Disable LSU access DLM

rv_csr_t (page 136) **_reserved0**

bit: 7..9 Reserved

rv_csr_t (page 136) **d1m_bpa**

bit: 10..XLEN-1 DLM base address

struct *CSR_MDLCTL_Type* (page 150)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MCFGINFO_Type**

#include <core_feature_base.h> Union type to access MCFG_INFO CSR register.

Public Members

rv_csr_t (page 136) **tee**

bit: 0 TEE present

rv_csr_t (page 136) **ecc**

bit: 1 ECC present

rv_csr_t (page 136) **clic**

bit: 2 CLIC present

rv_csr_t (page 136) **plic**

bit: 3 PLIC present

rv_csr_t (page 136) **fio**

bit: 4 FIO present

rv_csr_t (page 136) **ppi**

bit: 5 PPI present

rv_csr_t (page 136) **nice**

bit: 6 NICE present

rv_csr_t (page 136) **ilm**

bit: 7 ILM present

rv_csr_t (page 136) **dlim**

bit: 8 DLM present

rv_csr_t (page 136) **icache**

bit: 9 ICache present

rv_csr_t (page 136) **dcache**

bit: 10 DCache present

rv_csr_t (page 136) **smp**

bit: 11 SMP present

rv_csr_t (page 136) **dsp_n1**

bit: 12 DSP N1 present

rv_csr_t (page 136) **dsp_n2**

bit: 13 DSP N2 present

rv_csr_t (page 136) **dsp_n3**

bit: 14 DSP N3 present

rv_csr_t (page 136) **zc_xlcz**

bit: 15 Zc and xlcz extension present

rv_csr_t (page 136) **iregion**

bit: 16 IREGION present

rv_csr_t (page 136) **vpu_degree**

bit: 17..18 Indicate the VPU degree of parallel

rv_csr_t (page 136) **sec_mode**

bit: 19 Smwg extension present

rv_csr_t (page 136) **etrace**

bit: 20 Etrace present

rv_csr_t (page 136) **safety_mecha**

bit: 21..22 Indicate Core's safety mechanism

rv_csr_t (page 136) **vnice**

bit: 23 VNICE present

rv_csr_t (page 136) **xlcz**

bit: 24 XLCZ extension present

rv_csr_t (page 136) **zilsd**

bit: 25 Zilsd/Zclsd extension present

rv_csr_t (page 136) **sstc**

bit: 26 SSTC extension present

rv_csr_t (page 136) **_reserved1**

bit: 27..XLEN-1 Reserved

struct *CSR_MICFGINFO_Type* (page 151)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MICFGINFO_Type**

#include <core_feature_base.h> Union type to access MICFG_INFO CSR register.

Public Members

rv_csr_t (page 136) **set**

bit: 0..3 I-Cache sets per way

rv_csr_t (page 136) **way**

bit: 4..6 I-Cache way

rv_csr_t (page 136) **lsize**

bit: 7..9 I-Cache line size

rv_csr_t (page 136) **ecc**

bit: 10 I-Cache ECC support

rv_csr_t (page 136) **_reserved0**

bit: 11..15 Reserved

rv_csr_t (page 136) **lm_size**

bit: 16..20 ILM size, need to be 2^n size

rv_csr_t (page 136) **lm_xonly**

bit: 21 ILM Execute only permission or Reserved

rv_csr_t (page 136) **lm_ecc**

bit: 22 ILM ECC support

rv_csr_t (page 136) **i_share_dlm**

bit: 23 Support IFU fetch instructions from DLM

rv_csr_t (page 136) **_reserved1**

bit: 24..XLEN-1 Reserved

struct *CSR_MICFGINFO_Type* (page 153)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MDCFGINFO_Type**

#include <core_feature_base.h> Union type to access MDCFG_INFO CSR register.

Public Members

rv_csr_t (page 136) **set**

bit: 0..3 D-Cache sets per way

rv_csr_t (page 136) **way**

bit: 4..6 D-Cache way

rv_csr_t (page 136) **lsize**

bit: 7..9 D-Cache line size

rv_csr_t (page 136) **ecc**

bit: 10 D-Cache ECC support

rv_csr_t (page 136) **_reserved0**

bit: 11..15 Reserved

rv_csr_t (page 136) **lm_size**

bit: 16..20 DLM size, need to be 2^n size

rv_csr_t (page 136) **lm_ecc**

bit: 21 DLM ECC present

rv_csr_t (page 136) **_reserved1**

bit: 22..XLEN-1 Reserved

struct *CSR_MDCFGINFO_Type* (page 154)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MTLBCFGINFO_Type**

#include <core_feature_base.h> Union type to access MTLBCFG_INFO CSR register.

NOTE: the MTLBCFG_INFO CSR supports two different mapping layouts. Use the `b.mapping` or `nb.mapping` field to determine the active mapping type. If `mapping` field is set, use the `nb` structure for field access. Otherwise, use the `b` structure for field access.

Public Members

rv_csr_t (page 136) **set**

bit: 0..3 Main TLB entry per way

rv_csr_t (page 136) **way**

bit: 4..6 Main TLB ways

rv_csr_t (page 136) **lsize**

bit: 7..9 Main TLB line size or Reserved

rv_csr_t (page 136) **ecc**

bit: 10 Main TLB supports ECC or not

rv_csr_t (page 136) **napot**

bit: 11 TLB supports Svnapot or not

rv_csr_t (page 136) **_reserved1**

bit: 12..15 Reserved 0

rv_csr_t (page 136) **i_size**

bit: 16..18 ITLB size

bit: 12..18 ITLB size

rv_csr_t (page 136) **d_size**

bit: 19..21 DTLB size

bit: 19..26 DTLB size

rv_csr_t (page 136) **_reserved2**

bit: 22..XLEN-2 Reserved 0

rv_csr_t (page 136) **mapping**

bit: XLEN-1 mapping type

bit: XLEN-1 TLB mapping type

struct *CSR_MTLBCFGINFO_Type* (page 155)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **_reserved0**

bit: 27..XLEN-2 Reserved 0

struct *CSR_MTLBCFGINFO_Type* (page 155)::[anonymous] **nb**

Structure used for new mapping bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MPPICFGINFO_Type**

#include <core_feature_base.h> Union type to access MPPICFG_INFO CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0 Reserved 1

rv_csr_t (page 136) **ppi_size**

bit: 1..5 PPI size, need to be 2^n size

rv_csr_t (page 136) **_reserved1**

bit: 6..8 Reserved 0

rv_csr_t (page 136) **ppi_en**

bit: 9 PPI Enable.

Software can write this bit to control PPI

rv_csr_t (page 136) **ppi_bpa**

bit: 10..XLEN-1 PPI base address

struct *CSR_MPPICFGINFO_Type* (page 157)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MFIOCFGINFO_Type**

#include <core_feature_base.h> Union type to access MFIOCFG_INFO CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0 Reserved

rv_csr_t (page 136) **fio_size**

bit: 1..5 FIO size, need to be 2^n size

rv_csr_t (page 136) **_reserved1**

bit: 6..9 Reserved

rv_csr_t (page 136) **fio_bpa**

bit: 10..XLEN-1 FIO base address

struct *CSR_MFIOCFGINFO_Type* (page 157)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MECCLOCK_Type**

#include <core_feature_base.h> Union type to access MECC_LOCK CSR register.

Public Members

rv_csr_t (page 136) **ecc_lock**

bit: 0 RW permission, ECC Lock configure

rv_csr_t (page 136) **_reserved0**

bit: 1..XLEN-1 Reserved

struct *CSR_MECCLOCK_Type* (page 158)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MECCCODE_Type**

#include <core_feature_base.h> Union type to access MECC_CODE CSR register.

Public Members

rv_csr_t (page 136) **code**

bit: 0..8 Used to inject ECC check code

rv_csr_t (page 136) **_reserved0**

bit: 9..15 Reserved 0

rv_csr_t (page 136) **ramid**

bit: 16..20 The ID of RAM that has 2bit ECC error, software can clear these bits

rv_csr_t (page 136) **_reserved1**

bit: 21..23 Reserved 0

rv_csr_t (page 136) **sramid**

bit: 24..28 The ID of RAM that has 1bit ECC error, software can clear these bits

rv_csr_t (page 136) **_reserved2**

bit: 29..30 Reserved 0

rv_csr_t (page 136) **ecc_inj_mode**

bit: 31 ECC injection mode

struct *CSR_MECCCODE_Type* (page 158)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MECC_CTL_Type**

#include <core_feature_base.h> Union type to access MECC_CTL CSR register.

Public Members

rv_csr_t (page 136) **ilm_fch_msk**

bit: 0 Write 1 to disable aggregate ILM fetch ECC fatal error to safety_error output

rv_csr_t (page 136) **ilm_acc_msk**

bit: 1 Write 1 to disable aggregate ILM load/store access ECC fatal error to safety_error output

rv_csr_t (page 136) **dlim_acc_msk**

bit: 2 Write 1 to disable aggregate DLM access ECC fatal error to safety_error output

rv_csr_t (page 136) **ic_fch_msk**

bit: 3 Write 1 to disable aggregate ICache fetch ECC fatal error to safety_error output

rv_csr_t (page 136) **dc_acc_msk**

bit: 4 Write 1 to disable aggregate DCache access ECC fatal error to safety_error output

rv_csr_t (page 136) **ilm_ext_msk**

bit: 5 Write 1 to disable aggregate ILM external access ECC fatal error to safety_error output

rv_csr_t (page 136) **dlim_ext_msk**

bit: 6 Write 1 to disable aggregate DLM external access ECC fatal error to safety_error output

rv_csr_t (page 136) **ic_ccm_msk**

bit: 7 Write 1 to disable aggregate ICache CCM ECC fatal error to safety_error output

rv_csr_t (page 136) **dc_ccm_msk**

bit: 8 Write 1 to disable aggregate DCache CCM ECC fatal error to safety_error output

rv_csr_t (page 136) **dc_cpbk_msk**

bit: 9 Write 1 to disable aggregate DCache CPBK ECC fatal error to safety_error output

rv_csr_t (page 136) **_reserved0**

bit: 10..30 Reserved 0

rv_csr_t (page 136) **io_prot_chk_en**

bit: 31 Controls to check the IO interface

struct *CSR_MECC_CTL_Type* (page 159)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MECC_STATUS_Type**

#include <core_feature_base.h> Union type to access MECC_STATUS CSR register.

Public Members

rv_csr_t (page 136) **ilm_fch_err**

bit: 0 ILM fetch ECC fatal error has occurred

rv_csr_t (page 136) **ilm_acc_err**

bit: 1 ILM load/store access ECC fatal error has occurred

rv_csr_t (page 136) **dlim_acc_err**

bit: 2 DLM access ECC fatal error has occurred

rv_csr_t (page 136) **ic_fch_err**

bit: 3 ICache fetch ECC fatal error has occurred

rv_csr_t (page 136) **dc_acc_err**

bit: 4 DCache access ECC fatal error has occurred

rv_csr_t (page 136) **ilm_ext_err**

bit: 5 ILM external access ECC fatal error has occurred

rv_csr_t (page 136) **dlim_ext_err**

bit: 6 DLM external access ECC fatal error has occurred

rv_csr_t (page 136) **ic_ccm_err**

bit: 7 ICache CCM ECC fatal error has occurred

rv_csr_t (page 136) **dc_ccm_err**

bit: 8 DCache CCM ECC fatal error has occurred

rv_csr_t (page 136) **dc_cpbk_err**

bit: 9 DCache CPBK ECC fatal error has occurred

rv_csr_t (page 136) **_reserved0**

bit: 10..XLEN-1 Reserved 0

struct *CSR_MECC_STATUS_Type* (page 160)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MIRGB_INFO_Type**

#include <core_feature_base.h> Union type to access MIRGB_INFO CSR register.

Public Members

rv_csr_t (page 136) **_reserved0**

bit: 0 Reserved

rv_csr_t (page 136) **iregion_size**

bit: 1..5 Indicates the size of IREGION and it should be power of 2

rv_csr_t (page 136) **_reserved1**

bit: 6..9 Reserved

rv_csr_t (page 136) **iregion_base**

bit: 10..PA_SIZE IREGION Base Address

struct *CSR_MIRGB_INFO_Type* (page 161)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MSTACK_CTL_Type**

#include <core_feature_base.h> Union type to access MSTACK_CTL CSR register.

Public Members

rv_csr_t (page 136) **ovf_track_en**

bit: 0 Stack overflow check or track enable

rv_csr_t (page 136) **udf_en**

bit: 1 Stack underflow check enable

rv_csr_t (page 136) **mode**

bit: 2 Mode of stack checking

rv_csr_t (page 136) **_reserved0**

bit: 3..XLEN-1 Reserved

struct *CSR_MSTACK_CTL_Type* (page 161)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

union **CSR_MTLB_CTL_Type**

#include <core_feature_base.h> Union type to access MTLB_CTL CSR register.

Public Members

rv_csr_t (page 136) **tlb_ecc_en**

bit: 0 MTLB ECC enable

rv_csr_t (page 136) **tlb_ecc_excp_en**

bit: 1 MTLB double bit ECC exception enable control

rv_csr_t (page 136) **tlb_tram_ecc_inj_en**

bit: 2 Controls to inject the ECC Code in CSR mecc_code to MTLB tag rams

rv_csr_t (page 136) **tlb_dram_ecc_inj_en**

bit: 3 Controls to inject the ECC Code in CSR mecc_code to MTLB data rams

rv_csr_t (page 136) **_reserved0**

bit: 4..5 Reserved

rv_csr_t (page 136) **tlb_ecc_chk_en**

bit: 6 Controls to check the ECC when core access to MTLB

rv_csr_t (page 136) **napot_en**

bit: 7 NAPOT page enable

rv_csr_t (page 136) **_reserved1**

bit: 8..XLEN-1 Reserved

struct *CSR_MTLB_CTL_Type* (page 162)::[anonymous] **b**

Structure used for bit access.

rv_csr_t (page 136) **d**

Type used for csr data access.

ECLIC

group Register Define and Type Definitions Of ECLIC

Type definitions and defines for eclic registers.

Defines

CLIC_CLICCFG_NLBIT_Pos 1U

CLIC CLICCFG: NLBIT Position.

CLIC_CLICCFG_NLBIT_Msk (0xFUL << *CLIC_CLICCFG_NLBIT_Pos* (page 163))

CLIC CLICCFG: NLBIT Mask.

CLIC_CLICINFO_CTLBIT_Pos 21U

CLIC INTINFO: CLICINTCTLBITS Position.

CLIC_CLICINFO_CTLBIT_Msk (0xFUL << *CLIC_CLICINFO_CTLBIT_Pos* (page 163))

CLIC INTINFO: CLICINTCTLBITS Mask.

CLIC_CLICINFO_VER_Pos 13U

CLIC CLICINFO: VERSION Position.

CLIC_CLICINFO_VER_Msk (0xFFUL << *CLIC_CLICINFO_VER_Pos* (page 163))

CLIC CLICINFO: VERSION Mask.

CLIC_CLICINFO_NUM_Pos 0U

CLIC CLICINFO: NUM_INTERRUPT Position.

CLIC_CLICINFO_NUM_Msk (0x1FFFUL << *CLIC_CLICINFO_NUM_Pos* (page 163))

CLIC CLICINFO: NUM_INTERRUPT Mask.

CLIC_CLICINFO_SHD_NUM_Pos 25U

CLIC CLICINFO: SHD_NUM Position.

CLIC_CLICINFO_SHD_NUM_Msk (0xFUL << *CLIC_CLICINFO_SHD_NUM_Pos* (page 163))

CLIC CLICINFO: SHD_NUM Mask.

CLIC_INTIP_IP_Pos 0U

CLIC INTIP: IP Position.

CLIC_INTIP_IP_Msk (0x1UL << *CLIC_INTIP_IP_Pos* (page 164))

CLIC INTIP: IP Mask.

CLIC_INTIE_IE_Pos 0U

CLIC INTIE: IE Position.

CLIC_INTIE_IE_Msk (0x1UL << *CLIC_INTIE_IE_Pos* (page 164))

CLIC INTIE: IE Mask.

CLIC_INTATTR_MODE_Pos 6U

CLIC INTATTA: Mode Position.

CLIC_INTATTR_MODE_Msk (0x3U << *CLIC_INTATTR_MODE_Pos* (page 164))

CLIC INTATTA: Mode Mask.

CLIC_INTATTR_TRIG_Pos 1U

CLIC INTATTR: TRIG Position.

CLIC_INTATTR_TRIG_Msk (0x3UL << *CLIC_INTATTR_TRIG_Pos* (page 164))

CLIC INTATTR: TRIG Mask.

CLIC_INTATTR_SHV_Pos 0U

CLIC INTATTR: SHV Position.

CLIC_INTATTR_SHV_Msk (0x1UL << *CLIC_INTATTR_SHV_Pos* (page 164))

CLIC INTATTR: SHV Mask.

ECLIC_MAX_NLBITS 8U

Max nbit of the CLICINTCTLBITS.

ECLIC_MODE_MTVEC_Msk 3U

ECLIC Mode mask for MTVT CSR Register.

ECLIC_NON_VECTOR_INTERRUPT 0x0

Non-Vector Interrupt Mode of ECLIC.

ECLIC_VECTOR_INTERRUPT 0x1

Vector Interrupt Mode of ECLIC.

__ECLIC_VERSION

ECLIC_BASE **__ECLIC_BASEADDR**

ECLIC Base Address.

ECLIC ((*CLIC_Type* (page 166) *) *ECLIC_BASE* (page 165))

CLIC configuration struct.

Enums

enum **ECLIC_TRIGGER_Type**

ECLIC Trigger Enum for different Trigger Type.

Values:

enumerator **ECLIC_LEVEL_TRIGGER**

Level Triggerred, trig[0] = 0.

enumerator **ECLIC_POSTIVE_EDGE_TRIGGER**

Postive/Rising Edge Triggered, trig[0] = 1, trig[1] = 0.

enumerator **ECLIC_NEGTIVE_EDGE_TRIGGER**

Negative/Falling Edge Triggered, trig[0] = 1, trig[1] = 1.

enumerator **ECLIC_MAX_TRIGGER**

MAX Supported Trigger Mode.

union **CLICCFG_Type**

#include <core_feature_eclic.h> Union type to access CLICCFG configure register.

Public Members

__IM uint8_t _reserved0

__IOM uint8_t nlbits

bit: 1..4 specified the bit-width of level and priority in the register clicintctl[i]

__IM uint8_t nmbits

bit: 5..6 ties to 1 if supervisor-level interrupt supported, or else it's reserved

__IM uint8_t _reserved1

struct *CLICCFG_Type* (page 165)::[anonymous] **b**

Structure used for bit access.

`uint8_t w`

Type used for byte access.

union **CLICINFO_Type**

#include <core_feature_eclic.h> Union type to access CLICINFO information register.

Public Members

__IM uint32_t numint

bit: 0..12 number of maximum interrupt inputs supported

__IM uint32_t version

bit: 13..20 Hardware implementation version number.

1: version 1. 2: version 2, support hardware context saving and restoring.

__IM uint32_t intctlbits

bit: 21..24 specifies how many hardware bits are actually implemented in the clicintctl registers

__IM uint32_t shd_num

bit: 25..28 number of shadow register groups for single mode(M/S mode)

__IM uint32_t _reserved0

bit: 29..31 Reserved

struct *CLICINFO_Type* (page 166)::[anonymous] **b**

Structure used for bit access.

__IM uint32_t w

Type used for word access.

struct **CLIC_CTRL_Type**

#include <core_feature_eclic.h> Access to the machine mode register structure of INTIP, INTIE, INTATTR, INTCTL.

struct **CLIC_Type**

#include <core_feature_eclic.h> Access to the structure of ECLIC Memory Map.

PLIC

group Register Define and Type Definitions Of PLIC

Type definitions and defines for plic registers.

Defines

PLIC_PRIORITY_OFFSET _AC(0x0000,UL)

PLIC Priority register offset.

PLIC_PRIORITY_SHIFT_PER_SOURCE 2

$\log_2(4)$, shift per source

PLIC_PENDING_OFFSET _AC(0x1000,UL)

PLIC Pending register offset.

PLIC_PENDING_SHIFT_PER_SOURCE 0

bit index within word

PLIC_ENABLE_OFFSET _AC(0x2000,UL)

PLIC Enable register offset.

PLIC_ENABLE_SHIFT_PER_CONTEXT 7

$\log_2(0x80)$, shift per context

PLIC_THRESHOLD_OFFSET _AC(0x200000,UL)

PLIC Threshold register offset.

PLIC_CLAIM_OFFSET _AC(0x200004,UL)

PLIC Claim/Complete register offset.

PLIC_THRESHOLD_SHIFT_PER_CONTEXT 12

$\log_2(0x1000)$, shift per context

PLIC_CLAIM_SHIFT_PER_CONTEXT 12

$\log_2(0x1000)$, shift per context

PLIC_BASE __PLIC_BASEADDR

PLIC Base Address.

PLIC_GetHartID() ([__get_hart_index](#) (page 84)())

[PLIC_GetHartID\(\)](#) (page 167) is used to get plic hartid which might not be the same as cpu hart id, for example, cpu hartid may be 1, but plic hartid may be 0, then plic hartid offset is 1.

If defined __PLIC_HARTID, it will use __PLIC_HARTID as plic hartid, otherwise, it will use [__get_hart_index\(\)](#) (page 84). The cpu hartid is get by using [__get_hart_id](#) function

PLIC_GetHartID_S() (*__get_hart_index_s* (page 85)())

PLIC_GetHartMContextID() (*PLIC_GetHartID* (page 167)() << 1)

PLIC_GetHartSContextID() ((*PLIC_GetHartID_S* (page 168)() << 1) + 1)

PLIC_PRIORITY_REGADDR(source) ((*PLIC_BASE* (page 167)) + (*PLIC_PRIORITY_OFFSET* (page 167)) + ((source) << *PLIC_PRIORITY_SHIFT_PER_SOURCE* (page 167)))

PLIC_PENDING_REGADDR(source) ((*PLIC_BASE* (page 167)) + (*PLIC_PENDING_OFFSET* (page 167)) + (((source) >> 5) * 4))

PLIC_ENABLE_REGADDR(ctxid, source) ((*PLIC_BASE* (page 167)) + (*PLIC_ENABLE_OFFSET* (page 167)) + ((ctxid) << *PLIC_ENABLE_SHIFT_PER_CONTEXT* (page 167)) + ((source) >> 5) * 4)

PLIC_THRESHOLD_REGADDR(ctxid) ((*PLIC_BASE* (page 167)) + (*PLIC_THRESHOLD_OFFSET* (page 167)) + ((ctxid) << *PLIC_THRESHOLD_SHIFT_PER_CONTEXT* (page 167)))

PLIC_CLAIM_REGADDR(ctxid) ((*PLIC_BASE* (page 167)) + (*PLIC_CLAIM_OFFSET* (page 167)) + ((ctxid) << *PLIC_CLAIM_SHIFT_PER_CONTEXT* (page 167)))

PLIC_COMPLETE_REGADDR(ctxid) (*PLIC_CLAIM_REGADDR* (page 168)(ctxid))

SMPCC

group Register Define and Type Definitions Of SMPCC

Type definitions and defines for smpcc registers.

Defines

SMPCC_CTRL_CC_EN_Pos 0U

SMPCC CC_CTRL CC_EN Position.

SMPCC_CTRL_CC_EN_Msk (0x1UL << *SMPCC_CTRL_CC_EN_Pos* (page 168))

SMPCC CC_CTRL CC_EN Mask.

SMPCC_CTRL_CC_EN_ENABLE 1U

SMPCC CC_CTRL CC_EN Enable.

SMPCC_CTRL_CC_EN_DISABLE 0U

SMPCC CC_CTRL CC_EN Disable.

SMPCC_CTRL_CC_ECC_EN_Pos 1U

SMPCC CC_CTRL CC_ECC_EN Position.

SMPCC_CTRL_CC_ECC_EN_Msk (0x1UL << *SMPCC_CTRL_CC_ECC_EN_Pos* (page 168))

SMPCC CC_CTRL CC_ECC_EN Mask.

SMPCC_CTRL_CC_ECC_EN_ENABLE 1U

SMPCC CC_CTRL CC_ECC_EN Enable.

SMPCC_CTRL_CC_ECC_EN_DISABLE 0U

SMPCC CC_CTRL CC_ECC_EN Disable.

SMPCC_CTRL_CC_ECC_EXCP_EN_Pos 2U

SMPCC CC_CTRL ECC_EXCP_EN Position.

SMPCC_CTRL_CC_ECC_EXCP_EN_Msk (0x1UL << [SMPCC_CTRL_CC_ECC_EXCP_EN_Pos](#) (page 169))

SMPCC CC_CTRL ECC_EXCP_EN Mask.

SMPCC_CTRL_CC_ECC_EXCP_EN_ENABLE 1U

SMPCC CC_CTRL ECC_EXCP_EN Enable.

SMPCC_CTRL_CC_ECC_EXCP_EN_DISABLE 0U

SMPCC CC_CTRL ECC_EXCP_EN Disable.

SMPCC_CTRL_LOCK_ECC_CFG_Pos 3U

SMPCC CC_CTRL LOCK_ECC_CFG Position.

SMPCC_CTRL_LOCK_ECC_CFG_Msk (0x1UL << [SMPCC_CTRL_LOCK_ECC_CFG_Pos](#) (page 169))

SMPCC CC_CTRL LOCK_ECC_CFG Mask.

SMPCC_CTRL_LOCK_ECC_CFG_LOCK 1U

SMPCC CC_CTRL LOCK_ECC_CFG Lock.

SMPCC_CTRL_LOCK_ECC_ERR_INJ_Pos 4U

SMPCC CC_CTRL LOCK_ECC_ERR_INJ Position.

SMPCC_CTRL_LOCK_ECC_ERR_INJ_Msk (0x1UL << [SMPCC_CTRL_LOCK_ECC_ERR_INJ_Pos](#) (page 169))

SMPCC CC_CTRL LOCK_ECC_ERR_INJ Mask.

SMPCC_CTRL_LOCK_ECC_ERR_INJ_LOCK 1U

SMPCC CC_CTRL LOCK_ECC_ERR_INJ Lock.

SMPCC_CTRL_RECV_ERR_IRQ_EN_Pos 5U

SMPCC CC_CTRL RECV_ERR_IRQ_EN Position.

SMPCC_CTRL_RECV_ERR_IRQ_EN_Msk (0x1UL << [SMPCC_CTRL_RECV_ERR_IRQ_EN_Pos](#) (page 169))

SMPCC CC_CTRL RECV_ERR_IRQ_EN Mask.

SMPCC_CTRL_RECV_ERR_IRQ_EN_ENABLE 1U

SMPCC CC_CTRL RECV_ERR_IRQ_EN Enable.

SMPCC_CTRL_RECV_ERR_IRQ_EN_DISABLE 0U

SMPCC CC_CTRL_RECV_ERR_IRQ_EN Disable.

SMPCC_CTRL_FATAL_ERR_IRQ_EN_Pos 6U

SMPCC CC_CTRL_FATAL_ERR_IRQ_EN Position.

SMPCC_CTRL_FATAL_ERR_IRQ_EN_Msk (0x1UL << [SMPCC_CTRL_FATAL_ERR_IRQ_EN_Pos](#) (page 170))

SMPCC CC_CTRL_FATAL_ERR_IRQ_EN Mask.

SMPCC_CTRL_FATAL_ERR_IRQ_EN_ENABLE 1U

SMPCC CC_CTRL_FATAL_ERR_IRQ_EN Enable.

SMPCC_CTRL_FATAL_ERR_IRQ_EN_DISABLE 0U

SMPCC CC_CTRL_FATAL_ERR_IRQ_EN Disable.

SMPCC_CTRL_BUS_ERR_PEND_Pos 7U

SMPCC CC_CTRL_BUS_ERR_PEND Position.

SMPCC_CTRL_BUS_ERR_PEND_Msk (0x1UL << [SMPCC_CTRL_BUS_ERR_PEND_Pos](#) (page 170))

SMPCC CC_CTRL_BUS_ERR_PEND Mask.

SMPCC_CTRL_BUS_ERR_IRQ_EN_Pos 8U

SMPCC CC_CTRL_BUS_ERR_IRQ_EN Position.

SMPCC_CTRL_BUS_ERR_IRQ_EN_Msk (0x1UL << [SMPCC_CTRL_BUS_ERR_IRQ_EN_Pos](#) (page 170))

SMPCC CC_CTRL_BUS_ERR_IRQ_EN Mask.

SMPCC_CTRL_BUS_ERR_IRQ_EN_ENABLE 1U

SMPCC CC_CTRL_BUS_ERR_IRQ_EN Enable.

SMPCC_CTRL_BUS_ERR_IRQ_EN_DISABLE 0U

SMPCC CC_CTRL_BUS_ERR_IRQ_EN Disable.

SMPCC_CTRL_SUP_CMD_EN_Pos 9U

SMPCC CC_CTRL_SUP_CMD_EN Position.

SMPCC_CTRL_SUP_CMD_EN_Msk (0x1UL << [SMPCC_CTRL_SUP_CMD_EN_Pos](#) (page 170))

SMPCC CC_CTRL_SUP_CMD_EN Mask.

SMPCC_CTRL_SUP_CMD_EN_ENABLE 1U

SMPCC CC_CTRL_SUP_CMD_EN Enable.

SMPCC_CTRL_SUP_CMD_EN_DISABLE 0U

SMPCC CC_CTRL_SUP_CMD_EN Disable.

SMPCC_CTRL_USE_CMD_EN_Pos 10U

SMPCC CC_CTRL USE_CMD_EN Position.

SMPCC_CTRL_USE_CMD_EN_Msk (0x1UL << [SMPCC_CTRL_USE_CMD_EN_Pos](#) (page 171))

SMPCC CC_CTRL USE_CMD_EN Mask.

SMPCC_CTRL_USE_CMD_EN_ENABLE 1U

SMPCC CC_CTRL USE_CMD_EN Enable.

SMPCC_CTRL_USE_CMD_EN_DISABLE 0U

SMPCC CC_CTRL USE_CMD_EN Disable.

SMPCC_CTRL_ECC_CHK_EN_Pos 11U

SMPCC CC_CTRL ECC_CHK_EN Position.

SMPCC_CTRL_ECC_CHK_EN_Msk (0x1UL << [SMPCC_CTRL_ECC_CHK_EN_Pos](#) (page 171))

SMPCC CC_CTRL ECC_CHK_EN Mask.

SMPCC_CTRL_ECC_CHK_EN_ENABLE 1U

SMPCC CC_CTRL ECC_CHK_EN Enable.

SMPCC_CTRL_ECC_CHK_EN_DISABLE 0U

SMPCC CC_CTRL ECC_CHK_EN Disable.

SMPCC_CTRL_CLM_ECC_EN_Pos 12U

SMPCC CC_CTRL CLM_ECC_EN Position.

SMPCC_CTRL_CLM_ECC_EN_Msk (0x1UL << [SMPCC_CTRL_CLM_ECC_EN_Pos](#) (page 171))

SMPCC CC_CTRL CLM_ECC_EN Mask.

SMPCC_CTRL_CLM_ECC_EN_ENABLE 1U

SMPCC CC_CTRL CLM_ECC_EN Enable.

SMPCC_CTRL_CLM_ECC_EN_DISABLE 0U

SMPCC CC_CTRL CLM_ECC_EN Disable.

SMPCC_CTRL_CLM_EXCP_EN_Pos 13U

SMPCC CC_CTRL CLM_EXCP_EN Position.

SMPCC_CTRL_CLM_EXCP_EN_Msk (0x1UL << [SMPCC_CTRL_CLM_EXCP_EN_Pos](#) (page 171))

SMPCC CC_CTRL CLM_EXCP_EN Mask.

SMPCC_CTRL_CLM_EXCP_EN_ENABLE 1U

SMPCC CC_CTRL CLM_EXCP_EN Enable.

SMPCC_CTRL_CLM_EXCP_EN_DISABLE 0U

SMPCC CC_CTRL CLM_EXCP_EN Disable.

SMPCC_CTRL_CLM_ECC_CHK_EN_Pos 14U

SMPCC CC_CTRL CLM_ECC_CHK_EN Position.

SMPCC_CTRL_CLM_ECC_CHK_EN_Msk (0x1UL << *SMPCC_CTRL_CLM_ECC_CHK_EN_Pos* (page 172))

SMPCC CC_CTRL CLM_ECC_CHK_EN Mask.

SMPCC_CTRL_CLM_ECC_CHK_EN_ENABLE 1U

SMPCC CC_CTRL CLM_ECC_CHK_EN Enable.

SMPCC_CTRL_CLM_ECC_CHK_EN_DISABLE 0U

SMPCC CC_CTRL CLM_ECC_CHK_EN Disable.

SMPCC_CTRL_PF_SH_CL_EN_Pos 15U

SMPCC CC_CTRL PF_SH_CL_EN Position.

SMPCC_CTRL_PF_SH_CL_EN_Msk (0x1UL << *SMPCC_CTRL_PF_SH_CL_EN_Pos* (page 172))

SMPCC CC_CTRL PF_SH_CL_EN Mask.

SMPCC_CTRL_PF_SH_CL_EN_ENABLE 1U

SMPCC CC_CTRL PF_SH_CL_EN Enable.

SMPCC_CTRL_PF_SH_CL_EN_DISABLE 0U

SMPCC CC_CTRL PF_SH_CL_EN Disable.

SMPCC_CTRL_PF_L2_EARLY_EN_Pos 16U

SMPCC CC_CTRL PF_L2_EARLY_EN Position.

SMPCC_CTRL_PF_L2_EARLY_EN_Msk (0x1UL << *SMPCC_CTRL_PF_L2_EARLY_EN_Pos* (page 172))

SMPCC CC_CTRL PF_L2_EARLY_EN Mask.

SMPCC_CTRL_PF_L2_EARLY_EN_ENABLE 1U

SMPCC CC_CTRL PF_L2_EARLY_EN Enable.

SMPCC_CTRL_PF_L2_EARLY_EN_DISABLE 0U

SMPCC CC_CTRL PF_L2_EARLY_EN Disable.

SMPCC_CTRL_PF_BIU_OUTS_EN_Pos 17U

SMPCC CC_CTRL PF_BIU_OUTS_EN Position.

SMPCC_CTRL_PF_BIU_OUTS_EN_Msk (0x1UL << *SMPCC_CTRL_PF_BIU_OUTS_EN_Pos* (page 172))

SMPCC CC_CTRL PF_BIU_OUTS_EN Mask.

SMPCC_CTRL_PF_BIU_OUTS_EN_ENABLE 1U

SMPCC CC_CTRL PF_BIU_OUTS_EN Enable.

SMPCC_CTRL_PF_BIU_OUTS_EN_DISABLE 0U

SMPCC CC_CTRL PF_BIU_OUTS_EN Disable.

SMPCC_CTRL_I_SNOOP_D_EN_Pos 18U

SMPCC CC_CTRL I_SNOOP_D_EN Position.

SMPCC_CTRL_I_SNOOP_D_EN_Msk (0x1UL << [SMPCC_CTRL_I_SNOOP_D_EN_Pos](#) (page 173))

SMPCC CC_CTRL I_SNOOP_D_EN Mask.

SMPCC_CTRL_I_SNOOP_D_EN_ENABLE 1U

SMPCC CC_CTRL I_SNOOP_D_EN Enable.

SMPCC_CTRL_I_SNOOP_D_EN_DISABLE 0U

SMPCC CC_CTRL I_SNOOP_D_EN Disable.

SMPCC_CTRL_IOCC_ERR_Pos 19U

SMPCC CC_CTRL IOCC_ERR Position.

SMPCC_CTRL_IOCC_ERR_Msk (0x1UL << [SMPCC_CTRL_IOCC_ERR_Pos](#) (page 173))

SMPCC CC_CTRL IOCC_ERR Mask.

SMPCC_CTRL_EARLY_WR_ERR_Pos 20U

SMPCC CC_CTRL EARLY_WR_ERR Position.

SMPCC_CTRL_EARLY_WR_ERR_Msk (0x1UL << [SMPCC_CTRL_EARLY_WR_ERR_Pos](#) (page 173))

SMPCC CC_CTRL EARLY_WR_ERR Mask.

SMPCC_CTRL_PF_NO_WB_Pos 21U

SMPCC CC_CTRL PF_NO_WB Position.

SMPCC_CTRL_PF_NO_WB_Msk (0x1UL << [SMPCC_CTRL_PF_NO_WB_Pos](#) (page 173))

SMPCC CC_CTRL PF_NO_WB Mask.

SMPCC_CTRL_PF_NO_WB_ENABLE 1U

SMPCC CC_CTRL PF_NO_WB Enable.

SMPCC_CTRL_PF_NO_WB_DISABLE 0U

SMPCC CC_CTRL PF_NO_WB Disable.

SMPCC_ERR_INJ_INJDATA_Pos 0U

SMPCC CC_ERR_INJ INJDATA Position.

SMPCC_ERR_INJ_INJDATA_Msk (0x1UL << *SMPCC_ERR_INJ_INJDATA_Pos* (page 173))

SMPCC CC_ERR_INJ INJDATA Mask.

SMPCC_ERR_INJ_INJDATA_ENABLE 1U

SMPCC CC_ERR_INJ INJDATA Enable.

SMPCC_ERR_INJ_INJDATA_DISABLE 0U

SMPCC CC_ERR_INJ INJDATA Disable.

SMPCC_ERR_INJ_INJTAG_Pos 1U

SMPCC CC_ERR_INJ INJTAG Position.

SMPCC_ERR_INJ_INJTAG_Msk (0x1UL << *SMPCC_ERR_INJ_INJTAG_Pos* (page 174))

SMPCC CC_ERR_INJ INJTAG Mask.

SMPCC_ERR_INJ_INJTAG_ENABLE 1U

SMPCC CC_ERR_INJ INJTAG Enable.

SMPCC_ERR_INJ_INJTAG_DISABLE 0U

SMPCC CC_ERR_INJ INJTAG Disable.

SMPCC_ERR_INJ_INJCLM_Pos 2U

SMPCC CC_ERR_INJ INJCLM Position.

SMPCC_ERR_INJ_INJCLM_Msk (0x1UL << *SMPCC_ERR_INJ_INJCLM_Pos* (page 174))

SMPCC CC_ERR_INJ INJCLM Mask.

SMPCC_ERR_INJ_INJCLM_ENABLE 1U

SMPCC CC_ERR_INJ INJCLM Enable.

SMPCC_ERR_INJ_INJCLM_DISABLE 0U

SMPCC CC_ERR_INJ INJCLM Disable.

SMPCC_ERR_INJ_INJMODE_Pos 3U

SMPCC CC_ERR_INJ INJMODE Position.

SMPCC_ERR_INJ_INJMODE_Msk (0x1UL << *SMPCC_ERR_INJ_INJMODE_Pos* (page 174))

SMPCC CC_ERR_INJ INJMODE Mask.

SMPCC_ERR_INJ_INJMODE_DIRECT 0U

SMPCC CC_ERR_INJ INJMODE Direct write mode.

SMPCC_ERR_INJ_INJMODE_XOR 1U

SMPCC CC_ERR_INJ INJMODE XOR write mode.

SMPCC_ERR_INJ_CS_Pos 4U

SMPCC CC_ERR_INJ CS Position.

SMPCC_ERR_INJ_CS_Msk (0x1UL << *SMPCC_ERR_INJ_CS_Pos* (page 175))

SMPCC CC_ERR_INJ CS Mask.

SMPCC_ERR_INJ_CS_START 1U

SMPCC CC_ERR_INJ CS Start.

SMPCC_ERR_INJ_INJECCCODE_Pos 24U

SMPCC CC_ERR_INJ INJECCCODE Position.

SMPCC_ERR_INJ_INJECCCODE_Msk (0xFFUL << *SMPCC_ERR_INJ_INJECCCODE_Pos* (page 175))

SMPCC CC_ERR_INJ INJECCCODE Mask.

SMPCC_CLIERRSTS_READ_BUS_ERR_Pos 0U

SMPCC CLIENT ERROR STATUS READ_BUS_ERR Position.

SMPCC_CLIERRSTS_READ_BUS_ERR_Msk (0x1UL << *SMPCC_CLIERRSTS_READ_BUS_ERR_Pos* (page 175))

SMPCC CLIENT ERROR STATUS READ_BUS_ERR Mask.

SMPCC_CLIERRSTS_WRITE_BUS_ERR_Pos 1U

SMPCC CLIENT ERROR STATUS WRITE_BUS_ERR Position.

SMPCC_CLIERRSTS_WRITE_BUS_ERR_Msk (0x1UL << *SMPCC_CLIERRSTS_WRITE_BUS_ERR_Pos* (page 175))

SMPCC CLIENT ERROR STATUS WRITE_BUS_ERR Mask.

SMPCC_CLIERRSTS_CC_SCU_ECC_ERR_Pos 2U

SMPCC CLIENT ERROR STATUS CC_SCU_ECC_ERR Position.

SMPCC_CLIERRSTS_CC_SCU_ECC_ERR_Msk (0x1UL << *SMPCC_CLIERRSTS_CC_SCU_ECC_ERR_Pos* (page 175))

SMPCC CLIENT ERROR STATUS CC_SCU_ECC_ERR Mask.

SMPCC_CLIERRSTS_IOC_P_BUS_ERR_Pos 3U

SMPCC CLIENT ERROR STATUS IOC_P_BUS_ERR Position.

SMPCC_CLIERRSTS_IOC_P_BUS_ERR_Msk (0x1UL << *SMPCC_CLIERRSTS_IOC_P_BUS_ERR_Pos* (page 175))

SMPCC CLIENT ERROR STATUS IOC_P_BUS_ERR Mask.

SMPCC_STMCTRL_RD_STM_EN_Pos 0U

SMPCC READ Stream Enable Position.

SMPCC_STMCTRL_RD_STM_EN_Msk (0x1UL << *SMPCC_STMCTRL_RD_STM_EN_Pos* (page 175))
SMPCC READ Stream Enable Mask.

SMPCC_STMCTRL_RD_STM_EN_ENABLE 1U
SMPCC READ Stream Enable Enable.

SMPCC_STMCTRL_RD_STM_EN_DISABLE 0U
SMPCC READ Stream Enable Disable.

SMPCC_STMCTRL_WR_STM_EN_Pos 1U
SMPCC WRITE Stream Enable Position.

SMPCC_STMCTRL_WR_STM_EN_Msk (0x1UL << *SMPCC_STMCTRL_WR_STM_EN_Pos* (page 176))
SMPCC WRITE Stream Enable Mask.

SMPCC_STMCTRL_WR_STM_EN_ENABLE 1U
SMPCC WRITE Stream Enable Enable.

SMPCC_STMCTRL_WR_STM_EN_DISABLE 0U
SMPCC WRITE Stream Enable Disable.

SMPCC_STMCTRL_TRANS_ALLOC_Pos 2U
SMPCC TRANSLATE ALLOC ATTRIBUTE Enable Position.

SMPCC_STMCTRL_TRANS_ALLOC_Msk (0x1UL << *SMPCC_STMCTRL_TRANS_ALLOC_Pos* (page 176))
SMPCC TRANSLATE ALLOC ATTRIBUTE Enable Mask.

SMPCC_STMCTRL_TRANS_ALLOC_ENABLE 1U
SMPCC TRANSLATE ALLOC ATTRIBUTE Enable.

SMPCC_STMCTRL_TRANS_ALLOC_DISABLE 0U
SMPCC TRANSLATE ALLOC ATTRIBUTE Disable.

SMPCC_STMCTRL_RD_MERGE_EN_Pos 3U
SMPCC READ Merge Enable Position.

SMPCC_STMCTRL_RD_MERGE_EN_Msk (0x1UL << *SMPCC_STMCTRL_RD_MERGE_EN_Pos* (page 176))
SMPCC READ Merge Enable Mask.

SMPCC_STMCTRL_RD_MERGE_EN_ENABLE 1U
SMPCC READ Merge Enable Enable.

SMPCC_STMCTRL_RD_MERGE_EN_DISABLE 0U
SMPCC READ Merge Enable Disable.

SMPCC_STMCTRL_CROSS_EN_Pos 4U

SMPCC READ STREAM CROSS 4K Enable Position.

SMPCC_STMCTRL_CROSS_EN_Msk (0x1UL << *SMPCC_STMCTRL_CROSS_EN_Pos* (page 177))

SMPCC READ STREAM CROSS 4K Enable Mask.

SMPCC_STMCTRL_CROSS_EN_ENABLE 1U

SMPCC READ STREAM CROSS 4K Enable.

SMPCC_STMCTRL_CROSS_EN_DISABLE 0U

SMPCC READ STREAM CROSS 4K Disable.

SMPCC_STMCFG_RD_BYTE_THRE_Pos 0U

SMPCC READ BYTE THRESHOLD Position.

SMPCC_STMCFG_RD_BYTE_THRE_Msk (0x3FFUL << *SMPCC_STMCFG_RD_BYTE_THRE_Pos* (page 177))

SMPCC READ BYTE THRESHOLD Mask.

SMPCC_STMCFG_RD_DEGREE_Pos 12U

SMPCC READ DEGREE Position.

SMPCC_STMCFG_RD_DEGREE_Msk (0x7UL << *SMPCC_STMCFG_RD_DEGREE_Pos* (page 177))

SMPCC READ DEGREE Mask.

SMPCC_STMCFG_RD_DISTANCE_Pos 16U

SMPCC READ DISTANCE Position.

SMPCC_STMCFG_RD_DISTANCE_Msk (0x7UL << *SMPCC_STMCFG_RD_DISTANCE_Pos* (page 177))

SMPCC READ DISTANCE Mask.

SMPCC_STMCFG_WR_BYTE_THRE_Pos 20U

SMPCC WRITE BYTE THRESHOLD Position.

SMPCC_STMCFG_WR_BYTE_THRE_Msk (0x7FFUL << *SMPCC_STMCFG_WR_BYTE_THRE_Pos* (page 177))

SMPCC WRITE BYTE THRESHOLD Mask.

SMPCC_DFF_PROT_CHK_EN_Pos 0U

SMPCC DFF PROTECT CHECK ENABLE Position.

SMPCC_DFF_PROT_CHK_EN_Msk (0x3UL << *SMPCC_DFF_PROT_CHK_EN_Pos* (page 177))

SMPCC DFF PROTECT CHECK ENABLE Mask.

SMPCC_DFF_PROT_CHK_EN_ENABLE 2U

SMPCC DFF PROTECT CHECK ENABLE ENABLE.

SMPCC_DFF_PROT_CHK_EN_DISABLE 1U

SMPCC DFF PROTECT CHECK ENABLE DISABLE.

SMPCC_NS_RG_CFG_Pos 0U

SMPCC Non-Shareable Region CFG Position.

SMPCC_NS_RG_CFG_Msk (0x3UL << *SMPCC_NS_RG_CFG_Pos* (page 178))

SMPCC Non-Shareable Region CFG Mask.

SMPCC_NS_RG_CFG_DISABLE 0x00U

SMPCC Non-Shareable Region CFG DISABLE.

SMPCC_NS_RG_CFG_NACL 0x10U

SMPCC Non-Shareable Region CFG NACL.

SMPCC_NS_RG_CFG_NAPOT 0x11U

SMPCC Non-Shareable Region CFG NAPOT.

SMPCC_PMON_EVENT_SEL_Pos 0U

SMPCC PMON EVENT SEL Position.

SMPCC_PMON_EVENT_SEL_Msk (0xFFFFFUL << *SMPCC_PMON_EVENT_SEL_Pos* (page 178))

SMPCC PMON EVENT SEL Mask.

SMPCC_PMON_EVENT_DISABLE 0U

SMPCC PMON EVENT DISABLE.

SMPCC_PMON_EVENT_DATA_READ_COUNT 1U

SMPCC PMON EVENT DATA READ COUNT.

SMPCC_PMON_EVENT_DATA_WRITE_COUNT 2U

SMPCC PMON EVENT DATA WRITE COUNT SABLE.

SMPCC_PMON_EVENT_INSTR_READ_COUNT 3U

SMPCC PMON EVENT INSTR READ COUNT.

SMPCC_PMON_EVENT_DATA_READ_HIT_COUNT 4U

SMPCC PMON EVENT DATA READ HIT COUNT.

SMPCC_PMON_EVENT_DATA_WRITE_REPLACE_COUNT 5U

SMPCC PMON EVENT DATA WRITE REPLACE COUNT.

SMPCC_PMON_EVENT_DATA_READ_REPLACE_COUNT 6U

SMPCC PMON EVENT DATA READ REPLACE COUNT.

SMPCC_PMON_EVENT_DATA_READ_MISS_COUNT 7U
SMPCC PMON EVENT DATA READ MISS COUNT.

SMPCC_PMON_EVENT_INSTR_READ_HIT_COUNT 8U
SMPCC PMON EVENT INSTR READ HIT COUNT.

SMPCC_PMON_EVENT_INSTR_READ_MISS_COUNT 9U
SMPCC PMON EVENT INSTR READ MISS COUNT.

SMPCC_PMON_EVENT_INSTR_READ_REPLACE_COUNT 10U
SMPCC PMON EVENT INSTR READ REPLACE COUNT.

SMPCC_PMON_CLIENT_SEL_Pos 16U
SMPCC PMON CLIENT SEL Position.

SMPCC_PMON_CLIENT_SEL_Msk (0x1FUL << *SMPCC_PMON_CLIENT_SEL_Pos* (page 179))
SMPCC PMON CLIENT SEL Mask.

SMPCC_PMON_EVENT(event, client)

SMPCC_BASE __SMPCC_BASEADDR
SMPCC Base Address.

SMPCC ((*SMPCC_Type* (page 195) *)*SMPCC_BASE* (page 179))
SMPCC configuration struct.

Typedefs

__IO uint64_t CC_BUS_ERR_ADDR_Type
Type to access CC_BUS_ERR_ADDR register.

__IO uint64_t SMP_PMON_CNT_Type
Type to access SMP_PMON_CNT register.

__IO uint64_t CLIENT_ERR_ADDR_Type
Type to access CLIENT_ERR_ADDR register.

__IO uint64_t CC_INV_RANGE_START_Type
Type to access CC_INV_RANGE_START register.

__IO uint64_t CC_INV_RANGE_END_Type
Type to access CC_INV_RANGE_END register.

__IO uint64_t CC_ECC_INJ_ADDR_Type
Type to access CC_ECC_INJ_ADDR register.

__IO uint32_t CC_ECC_INJ_DATA_Type

Type to access CC_ECC_INJ_DATA register.

union **SMP_VER_Type**

#include <core_feature_smpcc.h> Union type to access SMP_VER information register.

Public Members

__IM uint32_t mic_ver

bit: 0..7 micro version number

__IM uint32_t min_ver

bit: 8..15 minor version number

__IM uint32_t maj_ver

bit: 16..23 major version number

__IM uint32_t _reserved

bit: 24..31 reserved

struct *SMP_VER_Type* (page 180)::[anonymous] **b**

Structure used for bit access.

__IM uint32_t w

Type used for word access.

union **SMP_CFG_Type**

#include <core_feature_smpcc.h> Union type to access SMP_CFG information register.

Public Members

__IM uint32_t cc_present

bit: 0 cluster cache present or not

__IM uint32_t smp_core_num

bit: 1..6 core number in cluster

__IM uint32_t iocp_num

bit: 7..12 IO coherency port number in the cluster

__IM uint32_t pmon_num

bit: 13..18 performance monitor number in the cluster

__IM uint32_t _reserved

bit: 19..31 reserved

struct *SMP_CFG_Type* (page 180)::[anonymous] **b**

Structure used for bit access.

__IM uint32_t w

Type used for word access.

union **CC_CFG_Type**

#include <core_feature_smpcc.h> Union type to access CC_CFG information register.

Public Members

__IM uint32_t cc_set

bit: 0..3 cluster cache set number = 2^{cc_set}

__IM uint32_t cc_way

bit: 4..7 cluster cache way number = $cc_way + 1$

__IM uint32_t cc_lsize

bit: 8..10 cluster cache line size = $2^{cc_lsize + 2}$

__IM uint32_t cc_ecc

bit: 11 cluster cache ECC supports ECC or not

__IM uint32_t cc_tcycle

bit: 12..14 L2 tag ram access cycle = $cc_tcycle + 1$

__IM uint32_t cc_dcycle

bit: 15..17 L2 Data sram access cycle = $cc_dcycle + 1$

__IM uint32_t _reserved

bit: 18..31 reserved

struct *CC_CFG_Type* (page 181)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **SMP_ENB_Type**

#include <core_feature_smpcc.h> Union type to access SMP_ENB configure register.

Public Members

__IOM uint32_t smp_enable

bit: 0..15 SMP enable bits for clients in cluster

__IM uint32_t _reserved

bit: 16..31 reserved

struct *SMP_ENB_Type* (page 181)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **CC_CTRL_Type**

#include <core_feature_smpcc.h> Union type to access CC_CTRL configure register.

Public Members

__IOM uint32_t cc_en

bit: 0 cluster cache enable bit

__IOM uint32_t cc_ecc_en

bit: 1 cluster cache ECC enable bit

__IOM uint32_t ecc_excp_en

bit: 2 cluster cache ECC exception enable bit

__IOM uint32_t lock_ecc_cfg

bit: 3 lock the cc ecc configuration bit

__IOM uint32_t lock_ecc_err_inj

bit: 4 lock cc ecc error injection register

__IOM uint32_t recv_err_irq_en

bit: 5 enable the interrupt when recoverable error count exceeds the threshold

__IOM uint32_t fatal_err_irq_en

bit: 6 enable the interrupt when fatal error count exceeds the threshold

__IOM uint32_t bus_err_pend

bit: 7 indicate if there is bus error pending

__IOM uint32_t bus_err_irq_en

bit: 8 enable the buss error interrupt of cc maintain operation

__IOM uint32_t sup_cmd_en

bit: 9 enable S mode can operate register CC_sCMD and SMP_PMON_SEL

__IOM uint32_t use_cmd_en

bit: 10 enable U mode can operate register CC_uCMD and SMP_PMON_SEL

__IOM uint32_t ecc_chk_en

bit: 11 cc ecc check enable bit

__IOM uint32_t clm_ecc_en

bit: 12 clm ecc enable bit

__IOM uint32_t clm_excp_en

bit: 13 clm ecc exception enable bit

__IOM uint32_t clm_ecc_chk_en

bit: 14 clm ecc check enable bit

__IOM uint32_t pf_sh_cl_en

bit: 15 enable L1 prefetch to snoop and share cacheline from other cores

__IOM uint32_t pf_l2_early_en

bit: 16 enable L2 prefetch to initialize external bus read access while lookup the cluster cache

__IOM uint32_t pf_biu_outs_en

bit: 17 enable the limit of outstanding L2 prefetch to the number of L2 prefetch line-buffer

__IOM uint32_t i_snoop_d_en

bit: 18 snoop to dcache for icache refill reads enable

__IOM uint32_t iocc_err

bit: 19 iocc has error

__IOM uint32_t early_wr_err

bit: 20 early write response has error

__IOM uint32_t pf_no_wb

bit: 21 enable L2 prefetch to abort and avoid dirty cacheline write back when filling the cluster cache

__IM uint32_t _reserved

bit: 22..31 reserved

struct *CC_CTRL_Type* (page 182)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **CC_CMD_Type**

#include <core_feature_smpcc.h> Union type to access CC_CMD register.

This type is suitable for all m/s/u mode registers.

Public Members

__IOM uint32_t cmd

bit: 0..4 cluster cache maintain command code

__IM uint32_t _reserved

bit: 5..22 reserved

__IOM uint32_t reisc

bit: 23 recoverable error interrupt status, write 1 to clean

__IOM uint32_t feisc

bit: 24 fatal error interrupt status, write 1 to clean

__IOM uint32_t besc

bit: 25 bus error status, write 1 to clean

__IM uint32_t result_code

bit: 26..30 result code

__IM uint32_t complete

bit: 31 completion status

struct *CC_CMD_Type* (page 184)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **CC_ERR_INJ_Type**

#include <core_feature_smpcc.h> Union type to access CC_ERR_INJ register.

Public Members

__IOM uint32_t inj_data

bit: 0 ECC error injection to data ram

__IOM uint32_t inj_tag

bit: 1 ECC error injection to tag ram

__IOM uint32_t inj_clm

bit: 2 ECC error injection to clm ram

__IOM uint32_t inj_mode

bit: 3 ECC error injection mode: 0-direct write mode, 1-xor write mode

__IOM uint32_t cs

bit: 4 Precise injection error control and status.

Write 1 to start, automatic cleared when finished.

__IM uint32_t _reserved0

bit: 5..23 reserved

__IM uint32_t inj_ecc_code

24..32 ECC code for injection

Write to this bit field may use sb instruction (write only one byte), which is not allowed for SMPCC registers. So this bit field is read-only here, but actually it can be written. To write this bit field, you should write the whole 32-bit register

struct *CC_ERR_INJ_Type* (page 184)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_RECV_CNT_Type**

#include <core_feature_smpcc.h> Union type to access CC_RECV_CNT register.

Public Members

__IOM uint32_t cnt

bit: 0..15 count of the recoverable error

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CC_RECV_CNT_Type* (page 185)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_FATAL_CNT_Type**

#include <core_feature_smpcc.h> Union type to access CC_FATAL_CNT register.

Public Members

__IOM uint32_t cnt

bit: 0..15 count of the fatal error

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CC_FATAL_CNT_Type* (page 186)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_RECV_THV_Type**

#include <core_feature_smpcc.h> Union type to access CC_RECV_THV register.

Public Members

__IOM uint32_t cnt

bit: 0..15 count of the recoverable error threshold value

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CC_RECV_THV_Type* (page 186)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_FATAL_THV_Type**

#include <core_feature_smpcc.h> Union type to access CC_FATAL_THV register.

Public Members

__IOM uint32_t cnt

bit: 0..15 count of the fatal error threshold value

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CC_FATAL_THV_Type* (page 186)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CLIENT_ERR_STATUS_Type**

#include <core_feature_smpcc.h> Union type to access CLIENT_ERR_STATUS register.

Public Members

__IOM uint32_t read_bus_err

bit: 0 read bus error

__IOM uint32_t write_bus_err

bit: 1 write bus error

__IOM uint32_t cc_scu_ecc_err

bit: 2 cc scu ecc error

__IOM uint32_t iocp_bus_err

bit: 3 iocp bus error

__IM uint32_t _reserved

bit: 4..31 reserved

struct *CLIENT_ERR_STATUS_Type* (page 187)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **SNOOP_PENDING_Type**

#include <core_feature_smpcc.h> Union type to access SNOOP_PENDING register.

Public Members

__IM uint32_t snoop_pending

bit: 0..15 snoop pending bit for each client

__IM uint32_t _reserved

bit: 16..31 reserved

struct *SNOOP_PENDING_Type* (page 187)::[anonymous] **b**

Structure used for bit access.

__IM uint32_t w

Type used for word access.

union **TRANS_PENDING_Type**

#include <core_feature_smpcc.h> Union type to access TRANS_PENDING register.

Public Members

__IM uint32_t trans_pending

bit: 0..15 transaction pending bit for each client

__IM uint32_t _reserved

bit: 16..30 reserved

__IM uint32_t ext_trans

bit: 31 external memory bus transaction pending

struct *TRANS_PENDING_Type* (page 188)::[anonymous] **b**

Structure used for bit access.

__IM uint32_t w

Type used for word access.

union **CLM_ADDR_BASE_Type**

#include <core_feature_smpcc.h> Union type to access CLM_ADDR_BASE register.

Public Members

__IOM uint32_t clm_base32

__IM uint32_t _reserved

struct *CLM_ADDR_BASE_Type* (page 188)::[anonymous] **clm32**

Structure used access only low 32-bits.

uint64_t clm_base64

Type used access whole 64-bits.

union **CLM_WAY_EN_Type**

#include <core_feature_smpcc.h> Union type to access CLM_WAY_EN register.

Public Members

__IOM uint32_t ena

bit: 0..15 This way is used as CLM or not

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CLM_WAY_EN_Type* (page 189)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **CC_INVALID_ALL_Type**

#include <core_feature_smpcc.h> Union type to access CC_INVALID_ALL register.

Public Members

__IOM uint32_t cs

bit: 0 write 1 to invalid all cluster cache, and hardware auto clean when operation is done

__IM uint32_t _reserved

bit: 1..31 reserved

struct *CC_INVALID_ALL_Type* (page 189)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **STM_CTRL_Type**

#include <core_feature_smpcc.h> Union type to access STM_CTRL register.

Public Members

__IOM uint32_t rd_stm_en

bit: 0 read stream enable

__IOM uint32_t wr_stm_en

bit: 1 write stream enable

__IOM uint32_t trans_alloc

bit: 2 translate allocate attribute to non-alloc attribute enable

__IOM uint32_t rd_merge_en

bit: 3 non-cacheable attribute read merge enable

__IOM uint32_t cross_en

bit: 4 read stream cross 4k enable

__IM uint32_t _reserved

bit: 5..31 reserved

struct *STM_CTRL_Type* (page 190)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **STM_CFG_Type**

#include <core_feature_smpcc.h> Union type to access STM_CFG register.

Public Members

__IOM uint32_t rd_byte_threshold

bit: 0..9 the prefetch number for read stream

__IM uint32_t _reserved0

bit: 10..11 reserved

__IOM uint32_t rd_degree

bit: 12..14 the delta between prefetch address and current bus address

__IM uint32_t _reserved1

bit: 15 reserved

__IOM uint32_t rd_distance

bit: 16..18 the threshold bytes matching write stream training successfully

__IM uint32_t _reserved2

bit: 19 reserved

__IOM uint32_t wr_byte_threshold

bit: 20..29 the line buffer timeout free time when no same cacheline transactions

__IM uint32_t _reserved3

bit: 30..31 reserved

struct *STM_CFG_Type* (page 190)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **STM_TIMEOUT_Type**

#include <core_feature_smpcc.h> Union type to access STM_TIMEOUT register.

Public Members

__IOM uint32_t timeout

bit: 0..10 write streaming wait clk num

__IM uint32_t _reserved

bit: 11..31 reserved

struct *STM_TIMEOUT_Type* (page 191)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **DFF_PROT_Type**

#include <core_feature_smpcc.h> Union type to access DFF_PROT register.

Public Members

__IOM uint32_t chk_en

bit: 0..1 register protect check enable.

2'b01: disable; 2'b10: enable

__IM uint32_t _reserved

bit: 2..31 reserved

struct *DFF_PROT_Type* (page 191)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **ECC_ERR_MSK_Type**

#include <core_feature_smpcc.h> Union type to access ECC_ERR_MSK register.

Public Members

__IOM uint32_t cc_l2_err_msk

bit: 0 mask L2 double bit error output

__IOM uint32_t cc_core_err_mask

bit: 1 mask core double bit error output

__IM uint32_t _reserved

bit: 2..31 reserved

struct *ECC_ERR_MSK_Type* (page 192)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **NS_RG_Type**

#include <core_feature_smpcc.h> Union type to access NS_RG register.

Public Members

__IOM uint64_t cfg

bit: 0..1 0x00: disable region; 0x10:NACL; 0x11: NAPOT

__IOM uint64_t addr_hi

bit: 2..63 address of the region

struct *NS_RG_Type* (page 192)::[anonymous] **b**

Structure used for bit access.

__IOM uint64_t dw

Type used for double word access.

union **SMP_PMON_SEL_Type**

#include <core_feature_smpcc.h> Union type to access SMP_PMON_SEL register.

Public Members

__IOM uint32_t event_sel

bit: 0..15 select the event for this performance monitor counter

__IOM uint32_t client_sel

bit: 16..20 specify the core in the cluster or external master number hooked to I/O coherency port

__IM uint32_t _reserved

bit: 21..31 reserved

struct *SMP_PMON_SEL_Type* (page 193)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **CLIENT_WAY_MASK_Type**

#include <core_feature_smpcc.h> Union type to access CLIENT_WAY_MASK register.

Public Members

__IOM uint32_t mask

bit: 0..15 mask this way for the client

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CLIENT_WAY_MASK_Type* (page 193)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_INV_RANGE_Type**

#include <core_feature_smpcc.h> Union type to access CC_INV_RANGE register.

Public Members

__IOM uint32_t cs

bit: 0 Set cs can enable inv range, it clear by hardware

__IM uint32_t _reserved

bit: 1..31 reserved

struct *CC_INV_RANGE_Type* (page 194)::[anonymous] **b**

Structure used for bit access.

uint32_t **w**

Type used for word access.

union **CC_ECC_INJ_WAY_Type**

#include <core_feature_smpcc.h> Union type to access CC_ECC_INJ_WAY register.

Public Members

__IOM uint32_t num

bit: 0..15 Set precise ecc inject way number, one-hot

__IM uint32_t _reserved

bit: 16..31 reserved

struct *CC_ECC_INJ_WAY_Type* (page 194)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

union **IOCP_ATTR_RMP_Type**

#include <core_feature_smpcc.h> Union type to access IOCP_ATTR_RMP register.

Public Members

__IOM uint32_t wr_rmp_en

bit: 0 Set write trans attribute remap enable

__IOM uint32_t wr_attri

bit: 1 Remap allocate and non-alloc.

0: Remap non-alloc to alloc, 1: Remap allocate to non-alloc

__IOM uint32_t rd_rmp_en

bit: 2 Set read trans attribute remap enable

__IOM uint32_t rd_attri

bit: 3 Remap allocate and non-alloc.

0: Remap non-alloc to allocate, 1: Remap allocate to non-alloc.

__IM uint32_t _reserved

bit: 4..31 reserved

struct *IOCP_ATTR_RMP_Type* (page 195)::[anonymous] **b**

Structure used for bit access.

uint32_t w

Type used for word access.

struct **SMPCC_Type**

#include <core_feature_smpcc.h> Access to the structure of SMPCC Memory Map.

Remark

Write to these memory-mapped registers should write with full register width.

SysTimer

group Register Define and Type Definitions Of System Timer

Type definitions and defines for system timer registers.

Defines

SysTimer_MTIMECTL_TIMESTOP_Pos 0U

SysTick Timer MTIMECTL: TIMESTOP bit Position.

SysTimer_MTIMECTL_TIMESTOP_Msk (1UL << *SysTimer_MTIMECTL_TIMESTOP_Pos* (page 196))

SysTick Timer MTIMECTL: TIMESTOP Mask.

SysTimer_MTIMECTL_CMPCLREN_Pos 1U

SysTick Timer MTIMECTL: CMPCLREN bit Position.

SysTimer_MTIMECTL_CMPCLREN_Msk (1UL << *SysTimer_MTIMECTL_CMPCLREN_Pos* (page 196))

SysTick Timer MTIMECTL: CMPCLREN Mask.

SysTimer_MTIMECTL_CLKSRC_Pos 2U

SysTick Timer MTIMECTL: CLKSRC bit Position.

SysTimer_MTIMECTL_CLKSRC_Msk (1UL << *SysTimer_MTIMECTL_CLKSRC_Pos* (page 196))

SysTick Timer MTIMECTL: CLKSRC Mask.

SysTimer_MTIMECTL_HDBG_Pos 3U

SysTick Timer MTIMECTL: HDBG bit Position.

SysTimer_MTIMECTL_HDBG_Msk (1UL << *SysTimer_MTIMECTL_HDBG_Pos* (page 196))

SysTick Timer MTIMECTL: HDBG Mask.

SysTimer_MTIMECTL_MTIME_SRC_Pos 4U

SysTick Timer MTIMECTL: MTIME_SRC bit Position.

SysTimer_MTIMECTL_MTIME_SRC_Msk (1UL << *SysTimer_MTIMECTL_MTIME_SRC_Pos* (page 196))

SysTick Timer MTIMECTL: MTIME_SRC Mask.

SysTimer_MSIP_MSIP_Pos 0U

SysTick Timer MSIP: MSIP bit Position.

SysTimer_MSIP_MSIP_Msk (1UL << *SysTimer_MSIP_MSIP_Pos* (page 196))

SysTick Timer MSIP: MSIP Mask.

SysTimer_SSIP_SSIP_Pos 0U

SysTick Timer SSIP: SSIP bit Position.

SysTimer_SSIP_SSIP_Msk (1UL << *SysTimer_SSIP_SSIP_Pos* (page 196))

SysTick Timer SSIP: SSIP Mask.

SysTimer_MTIME_SRW_CTRL_SRW_Pos 0U

SysTick Timer MTIME_SRW_CTRL: SRW bit Position.

SysTimer_MTIME_SRW_CTRL_SRW_Msk (1UL << *SysTimer_MTIME_SRW_CTRL_SRW_Pos* (page 197))

SysTick Timer MTIME_SRW_CTRL: SRW Mask.

SysTimer_MTIMER_Msk (0xFFFFFFFFFFFFFFFFULL)

SysTick Timer MTIMER value Mask.

SysTimer_MTIMERCMP_Msk (0xFFFFFFFFFFFFFFFFULL)

SysTick Timer MTIMERCMP value Mask.

SysTimer_MTIMECTL_Msk (0xFFFFFFFFFUL)

SysTick Timer MTIMECTL/MSTOP value Mask.

SysTimer_MSIP_Msk (0xFFFFFFFFFUL)

SysTick Timer MSIP value Mask.

SysTimer_MSFRST_Msk (0xFFFFFFFFFUL)

SysTick Timer MSFRST value Mask.

SysTimer_MSFRST_KEY (0x80000A5FUL)

SysTick Timer Software Reset Request Key.

SysTimer_CLINT_MSIP_OFS (0x1000UL)

Machine Mode Software interrupt register offset of clint mode in SysTick Timer.

SysTimer_CLINT_MTIMECMP_OFS (0x5000UL)

MTIMECMP register offset of clint mode in SysTick Timer.

SysTimer_CLINT_MTIME_OFS (0xCF8UL)

MTIME register offset of clint mode in SysTick Timer.

SysTimer_CLINT_SSIP_OFS (0xD000UL)

Supervisor Mode Software interrupt register offset of clint mode in SysTick Timer.

SysTimer_BASE __SYSTIMER_BASEADDR

SysTick Base Address.

SysTimer ((*SysTimer_Type* (page 198) *) *SysTimer_BASE* (page 197))

SysTick configuration struct.

SysTimer_CLINT_MSIP_BASE(hartid) (unsigned long)((*SysTimer_BASE* (page 197)) +
(*SysTimer_CLINT_MSIP_OFS* (page 197)) + ((hartid) << 2))

SysTimer_CLINT_MTIMECMP_BASE(hartid) (unsigned long)((*SysTimer_BASE* (page 197)) +
(*SysTimer_CLINT_MTIMECMP_OFS* (page 197)) + ((hartid) << 3))

SysTimer_CLINT_MTIME_BASE (unsigned long)((*SysTimer_BASE* (page 197)) +
(*SysTimer_CLINT_MTIME_OFS* (page 197)))

SysTimer_CLINT_SSIIP_BASE(hartid) (unsigned long)((*SysTimer_BASE* (page 197)) +
(*SysTimer_CLINT_SSIIP_OFS* (page 197)) + ((hartid) << 2))

struct **SysTimer_Type**

#include <core_feature_timer.h> Structure type to access the System Timer (SysTimer).

Structure definition to access the system timer(SysTimer).

Remark

- MSFTRST register is introduced in Nuclei N Core version 1.3(*__NUCLEI_N_REV* (page 71) >= 0x0103)
- MSTOP register is renamed to MTIMECTL register in Nuclei N Core version 1.4(*__NUCLEI_N_REV* (page 71) >= 0x0104)
- CMPCLREN and CLKSRC bit in MTIMECTL register is introduced in Nuclei N Core version 1.4(*__NUCLEI_N_REV* (page 71) >= 0x0104)

2.5.6 CPU Intrinsic Functions

enum **WFI_SleepMode_Type**

Values:

enumerator **WFI_SHALLOW_SLEEP**

enumerator **WFI_DEEP_SLEEP**

__STATIC_FORCEINLINE void __NOP (void)

__STATIC_FORCEINLINE void __WFI (void)

__STATIC_FORCEINLINE void __WFE (void)

__STATIC_FORCEINLINE void __EBREAK (void)

__STATIC_FORCEINLINE void __ECALL (void)

```
__STATIC_FORCEINLINE void __set_wfi_sleepmode (WFI_SleepMode_Type mode)

__STATIC_FORCEINLINE void __TXEVT (void)

__STATIC_FORCEINLINE void __enable_mcycle_counter (void)

__STATIC_FORCEINLINE void __disable_mcycle_counter (void)

__STATIC_FORCEINLINE void __enable_minstret_counter (void)

__STATIC_FORCEINLINE void __disable_minstret_counter (void)

__STATIC_FORCEINLINE void __enable_mhpm_counter (unsigned long idx)

__STATIC_FORCEINLINE void __disable_mhpm_counter (unsigned long idx)

__STATIC_FORCEINLINE void __enable_mhpm_counters (unsigned long mask)

__STATIC_FORCEINLINE void __disable_mhpm_counters (unsigned long mask)

__STATIC_FORCEINLINE void __enable_all_counter (void)

__STATIC_FORCEINLINE void __disable_all_counter (void)

__STATIC_INLINE void __set_hpm_event (unsigned long idx, unsigned long event)

__STATIC_INLINE unsigned long __get_hpm_event (unsigned long idx)

__STATIC_INLINE void __set_hpm_counter (unsigned long idx, uint64_t value)

__STATIC_INLINE uint64_t __get_hpm_counter (unsigned long idx)

__STATIC_INLINE unsigned long __read_hpm_counter (unsigned long idx)

__STATIC_FORCEINLINE void __set_medeleg (unsigned long mask)

__STATIC_FORCEINLINE void __set_mideleg (unsigned long mask)

__STATIC_FORCEINLINE uint8_t __LB (volatile void *addr)

__STATIC_FORCEINLINE uint16_t __LH (volatile void *addr)
```

```
__STATIC_FORCEINLINE uint32_t __LW (volatile void *addr)

__STATIC_FORCEINLINE void __SB (volatile void *addr, uint8_t val)

__STATIC_FORCEINLINE void __SH (volatile void *addr, uint16_t val)

__STATIC_FORCEINLINE void __SW (volatile void *addr, uint32_t val)

__STATIC_INLINE uint32_t __CAS_W (volatile uint32_t *addr, uint32_t oldval,
uint32_t newval)

__STATIC_FORCEINLINE uint32_t __AMOSWAP_W (volatile uint32_t *addr, uint32_t newval)

__STATIC_FORCEINLINE int32_t __AMOADD_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE int32_t __AMOAND_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE int32_t __AMOOR_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE int32_t __AMOXOR_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE uint32_t __AMOMAXU_W (volatile uint32_t *addr, uint32_t value)

__STATIC_FORCEINLINE int32_t __AMOMAX_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE uint32_t __AMOMINU_W (volatile uint32_t *addr, uint32_t value)

__STATIC_FORCEINLINE int32_t __AMOMIN_W (volatile int32_t *addr, int32_t value)

__STATIC_FORCEINLINE void __enable_ic_prefetch (void)

__STATIC_FORCEINLINE void __disable_ic_prefetch (void)

__STATIC_FORCEINLINE void __enable_ic_cmo_prefetch (void)

__STATIC_FORCEINLINE void __disable_ic_cmo_prefetch (void)

__STATIC_FORCEINLINE void __enable_dc_cmo_prefetch (void)

__STATIC_FORCEINLINE void __disable_dc_cmo_prefetch (void)

__STATIC_FORCEINLINE void __cmo_prefetch_i (const void *addr)
```

```
__STATIC_FORCEINLINE void __cmo_prefetch_r (const void *addr)
```

```
__STATIC_FORCEINLINE void __cmo_prefetch_w (const void *addr)
```

group **Intrinsic Functions for CPU Instructions**

Functions that generate RISC-V CPU instructions.

The following functions generate specified RISC-V instructions that cannot be directly accessed by compiler.

Enums

enum **WFI_SleepMode_Type**

WFI Sleep Mode enumeration.

Values:

enumerator **WFI_SHALLOW_SLEEP**

Shallow sleep mode, the core_clk will poweroff.

enumerator **WFI_DEEP_SLEEP**

Deep sleep mode, the core_clk and core_ano_clk will poweroff.

Functions

```
__STATIC_FORCEINLINE void __NOP (void)
```

NOP Instruction.

No Operation does nothing. This instruction can be used for code alignment purposes.

```
__STATIC_FORCEINLINE void __WFI (void)
```

Wait For Interrupt.

Wait For Interrupt is executed using CSR_WFE.WFE=0 and WFI instruction. It will suspends execution until interrupt, NMI or Debug happened. When Core is waked up by interrupt, if

- a. mstatus.MIE == 1(interrupt enabled), Core will enter ISR code
- b. mstatus.MIE == 0(interrupt disabled), Core will resume previous execution

```
__STATIC_FORCEINLINE void __WFE (void)
```

Wait For Event.

Wait For Event is executed using CSR_WFE.WFE=1 and WFI instruction. It will suspends execution until event, NMI or Debug happened. When Core is waked up, Core will resume previous execution

```
__STATIC_FORCEINLINE void __EBREAK (void)
```

Breakpoint Instruction.

Causes the processor to enter Debug state. Debug tools can use this to investigate system state when the instruction at a particular address is reached.

__STATIC_FORCEINLINE void __ECALL (void)

Environment Call Instruction.

The ECALL instruction is used to make a service request to the execution environment.

__STATIC_FORCEINLINE void __set_wfi_sleepmode (WFI_SleepMode_Type mode)

Set Sleep mode of WFI.

Set the SLEEPVALUE CSR register to control the WFI Sleep mode.

Parameters

mode – [in] The sleep mode to be set

__STATIC_FORCEINLINE void __TXEVT (void)

Send TX Event.

Set the CSR TXEVT to control send a TX Event. The Core will output signal tx_evt as output event signal.

__STATIC_FORCEINLINE void __enable_mcycle_counter (void)

Enable MCYCLE counter.

Clear the CY bit of MCOUNTINHIBIT to 0 to enable MCYCLE Counter

__STATIC_FORCEINLINE void __disable_mcycle_counter (void)

Disable MCYCLE counter.

Set the CY bit of MCOUNTINHIBIT to 1 to disable MCYCLE Counter

__STATIC_FORCEINLINE void __enable_minstret_counter (void)

Enable MINSTRET counter.

Clear the IR bit of MCOUNTINHIBIT to 0 to enable MINSTRET Counter

__STATIC_FORCEINLINE void __disable_minstret_counter (void)

Disable MINSTRET counter.

Set the IR bit of MCOUNTINHIBIT to 1 to disable MINSTRET Counter

__STATIC_FORCEINLINE void __enable_mhpm_counter (unsigned long idx)

Enable selected hardware performance monitor counter.

enable selected hardware performance monitor counter mhpmcounterx.

Parameters

idx – [in] the index of the hardware performance monitor counter

__STATIC_FORCEINLINE void __disable_mhpm_counter (unsigned long idx)

Disable selected hardware performance monitor counter.

Disable selected hardware performance monitor counter mhpmcounterx.

Parameters

idx – [in] the index of the hardware performance monitor counter

__STATIC_FORCEINLINE void __enable_mhpm_counters (unsigned long mask)

Enable hardware performance counters with mask.

enable mhpmcounterx with mask, only the masked ones will be enabled. mhpmcounter3-mhpmcount31 are for high performance monitor counters.

Parameters

mask – [in] mask of selected hardware performance monitor counters

__STATIC_FORCEINLINE void __disable_mhpm_counters (unsigned long mask)

Disable hardware performance counters with mask.

Disable mhpmcounterx with mask, only the masked ones will be disabled. mhpmcounter3-mhpmcount31 are for high performance monitor counters.

Parameters

mask – [in] mask of selected hardware performance monitor counters

__STATIC_FORCEINLINE void __enable_all_counter (void)

Enable all MCYCLE & MINSTRET & MHPMCOUNTER counter.

Clear all to zero to enable all counters, such as cycle, instret, high performance monitor counters

__STATIC_FORCEINLINE void __disable_all_counter (void)

Disable all MCYCLE & MINSTRET & MHPMCOUNTER counter.

Set all to one to disable all counters, such as cycle, instret, high performance monitor counters

__STATIC_INLINE void __set_hpm_event (unsigned long idx, unsigned long event)

Set event for selected high performance monitor event.

Set event for high performance monitor event register

Parameters

- **idx** – [in] HPMEVENTx CSR index(3-31)
- **event** – [in] HPMEVENTx Register value to set

__STATIC_INLINE unsigned long __get_hpm_event (unsigned long idx)

Get event for selected high performance monitor event.

Get high performance monitor event register value

Parameters

- **idx** – [in] HPMEVENTx CSR index(3-31)
- **event** – [in] HPMEVENTx Register value to set

Returns

HPMEVENTx Register value

__STATIC_INLINE void __set_hpm_counter (unsigned long idx, uint64_t value)

Set value for selected high performance monitor counter.

Set value for high performance monitor counter register

Parameters

- **idx** – [in] HPMCOUNTERx CSR index(3-31)
- **value** – [in] HPMCOUNTERx Register value to set

__STATIC_INLINE uint64_t __get_hpm_counter (unsigned long idx)

Get value of selected high performance monitor counter.

Get high performance monitor counter register value

Parameters**idx** – [in] HPMCOUNTERx CSR index(3-31)**Returns**

HPMCOUNTERx Register value

__STATIC_INLINE unsigned long __read_hpm_counter (unsigned long idx)

Get value of selected high performance monitor counter.

Get high performance monitor counter register value without high 32 bits when XLEN=32

Parameters**idx** – [in] HPMCOUNTERx CSR index(3-31)**Returns**

HPMCOUNTERx Register value

__STATIC_FORCEINLINE void __set_medeleg (unsigned long mask)

Set exceptions delegation to S mode.

Set certain exceptions of supervisor mode or user mode delegated from machine mode to supervisor mode.

Remark

Exception should trigger in supervisor mode or user mode.

__STATIC_FORCEINLINE void __set_mideleg (unsigned long mask)

Set interrupt delegation to S mode.

Set certain interrupt of supervisor mode or user mode delegated from machine mode to supervisor mode.

Remark

interrupt should trigger in supervisor mode or user mode.

__STATIC_FORCEINLINE uint8_t __LB (volatile void *addr)

Load 8bit value from address (8 bit)

Load 8 bit value.

Parameters

addr – [in] Address pointer to data

Returns

value of type uint8_t at (*addr)

__STATIC_FORCEINLINE uint16_t __LH (volatile void *addr)

Load 16bit value from address (16 bit)

Load 16 bit value.

Parameters

addr – [in] Address pointer to data

Returns

value of type uint16_t at (*addr)

__STATIC_FORCEINLINE uint32_t __LW (volatile void *addr)

Load 32bit value from address (32 bit)

Load 32 bit value.

Parameters

addr – [in] Address pointer to data

Returns

value of type uint32_t at (*addr)

__STATIC_FORCEINLINE void __SB (volatile void *addr, uint8_t val)

Write 8bit value to address (8 bit)

Write 8 bit value.

Parameters

- **addr** – [in] Address pointer to data
- **val** – [in] Value to set

__STATIC_FORCEINLINE void __SH (volatile void *addr, uint16_t val)

Write 16bit value to address (16 bit)

Write 16 bit value.

Parameters

- **addr** – [in] Address pointer to data
- **val** – [in] Value to set

__STATIC_FORCEINLINE void __SW (volatile void *addr, uint32_t val)

Write 32bit value to address (32 bit)

Write 32 bit value.

Parameters

- **addr** – [in] Address pointer to data
- **val** – [in] Value to set

__STATIC_INLINE uint32_t __CAS_W (volatile uint32_t *addr, uint32_t oldval, uint32_t newval)

Compare and Swap 32bit value using LR and SC.

Compare old value with memory, if identical, store new value in memory. Return the initial value in memory. Success is indicated by comparing the return value with OLD.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **oldval** – [in] Old value of the data in address
- **newval** – [in] New value to be stored into the address

Returns

return the initial value in memory

__STATIC_FORCEINLINE uint32_t __AMOSWAP_W (volatile uint32_t *addr, uint32_t newval)

Atomic Swap 32bit value into memory.

Atomically swap new 32bit value into memory using amoswap.w. This helper uses a plain AMO without aq/rl ordering bits.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **newval** – [in] New value to be stored into the address

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOADD_W (volatile int32_t *addr, int32_t value)

Atomic Add with 32bit value.

Atomically ADD 32bit value with value in memory using amoadd.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be ADDED

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOAND_W (volatile int32_t *addr, int32_t value)

Atomic And with 32bit value.

Atomically AND 32bit value with value in memory using amoand.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be ANDed

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOOR_W (volatile int32_t *addr, int32_t value)

Atomic OR with 32bit value.

Atomically OR 32bit value with value in memory using amoor.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be ORed

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOXOR_W (volatile int32_t *addr, int32_t value)

Atomic XOR with 32bit value.

Atomically XOR 32bit value with value in memory using amoxor.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be XORed

Returns

return the original value in memory

__STATIC_FORCEINLINE uint32_t __AMOMAXU_W (volatile uint32_t *addr, uint32_t value)

Atomic unsigned MAX with 32bit value.

Atomically unsigned max compare 32bit value with value in memory using amomaxu.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be compared

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOMAX_W (volatile int32_t *addr, int32_t value)

Atomic signed MAX with 32bit value.

Atomically signed max compare 32bit value with value in memory using amomax.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be compared

Returns

return the original value in memory

__STATIC_FORCEINLINE uint32_t __AMOMINU_W (volatile uint32_t *addr, uint32_t value)

Atomic unsigned MIN with 32bit value.

Atomically unsigned min compare 32bit value with value in memory using amominu.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be compared

Returns

return the original value in memory

__STATIC_FORCEINLINE int32_t __AMOMIN_W (volatile int32_t *addr, int32_t value)

Atomic signed MIN with 32bit value.

Atomically signed min compare 32bit value with value in memory using amomin.w. This helper uses a plain AMO without aq/rl ordering bits. On RV64, the hardware sign-extends the word written to rd; this API returns the 32-bit value.

Parameters

- **addr** – [in] Address pointer to data, address must be naturally aligned to 4 bytes
- **value** – [in] value to be compared

Returns

return the original value in memory

__STATIC_FORCEINLINE void __enable_ic_prefetch (void)

Enable ICache prefetch.

Set the IC_PF_EN bit in the MCACHE_CTL CSR to enable ICache prefetch.

__STATIC_FORCEINLINE void __disable_ic_prefetch (void)

Disable ICache prefetch.

Clear the IC_PF_EN bit in the MCACHE_CTL CSR to disable ICache prefetch.

__STATIC_FORCEINLINE void __enable_ic_cmo_prefetch (void)

Enable ICache CMO prefetch.

Set the IC_CMO_PF_EN bit in the MCACHE_CTL CSR to enable ICache prefetch.

__STATIC_FORCEINLINE void __disable_ic_cmo_prefetch (void)

Disable ICache CMO prefetch.

Clear the IC_CMO_PF_EN bit in the MCACHE_CTL CSR to disable ICache prefetch.

__STATIC_FORCEINLINE void __enable_dc_cmo_prefetch (void)

Enable DCache CMO prefetch.

Set the DC_CMO_PF_EN bit in the MCACHE_CTL CSR to enable DCache prefetch.

__STATIC_FORCEINLINE void __disable_dc_cmo_prefetch (void)

Disable DCache CMO prefetch.

Clear the DC_CMO_PF_EN bit in the MCACHE_CTL CSR to disable DCache prefetch.

__STATIC_FORCEINLINE void __cmo_prefetch_i (const void *addr)

Instruction prefetch operation.

Performs an instruction prefetch operation for the specified address using the RISC-V prefetch.i instruction.

```
if (IINFO_IsCMOPrefetchSupported()) {
    __cmo_prefetch_i(func);
}
```

Remark

Before calling this function, ensure that the hardware supports CMO prefetch and that the code is compiled with the `_zicbop` extension enabled. Here is a code example to check if CMO prefetch is supported:

See also

- `IINFO_IsCMOPrefetchSupported`

Parameters

addr – [in] Address to prefetch

__STATIC_FORCEINLINE void __cmo_prefetch_r (const void *addr)

Read prefetch operation.

Performs a read prefetch operation for the specified address using the RISC-V prefetch.r instruction.

```
if (IINFO_IsCMOPrefetchSupported()) {  
    __cmo_prefetch_r(data);  
}
```

Remark

Before calling this function, ensure that the hardware supports CMO prefetch and that the code is compiled with the `_zicbop` extension enabled. Here is a code example to check if CMO prefetch is supported:

See also

- `IINFO_IsCMOPrefetchSupported`

Parameters

addr – [in] Address to prefetch

__STATIC_FORCEINLINE void __cmo_prefetch_w (const void *addr)

Write prefetch operation.

Performs a write prefetch operation for the specified address using the RISC-V prefetch.w instruction.

```
if (IINFO_IsCMOPrefetchSupported()) {  
    __cmo_prefetch_w(data);  
}
```

Remark

Before calling this function, ensure that the hardware supports CMO prefetch and that the code is compiled with the `_zicbop` extension enabled. Here is a code example to check if CMO prefetch is supported:

See also

- `IINFO_IsCMOPrefetchSupported`

Parameters

addr – [in] Address to prefetch

2.5.7 Intrinsic Functions for SIMD Instructions

SIMD Data Processing Instructions

SIMD 16-bit Add/Subtract Instructions

```

__STATIC_FORCEINLINE unsigned long __RV_ADD16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_CRAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_CRSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KADD16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KCRAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KCRSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KSTAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KSTSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KSUB16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RADD16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RCRAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RCRSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSTAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSTSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSUB16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_STAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_STSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_SUB16 (unsigned long a, unsigned long b)

```

```
__STATIC_FORCEINLINE unsigned long __RV_UKADD16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKCRAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKCRSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSTAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSTSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSUB16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URADD16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URCRAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URCRSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSTAS16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSTSA16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSUB16 (unsigned long a, unsigned long b)
```

group **SIMD 16-bit Add/Subtract Instructions**

SIMD 16-bit Add/Subtract Instructions.

Based on the combination of the types of the two 16-bit arithmetic operations, the SIMD 16-bit add/subtract instructions can be classified into 6 main categories: Addition (two 16-bit addition), Subtraction (two 16-bit subtraction), Crossed Add & Sub (one addition and one subtraction), and Crossed Sub & Add (one subtraction and one addition), Straight Add & Sub (one addition and one subtraction), and Straight Sub & Add (one subtraction and one addition). Based on the way of how an overflow condition is handled, the SIMD 16-bit add/subtract instructions can be classified into 5 groups: Wrap-around (dropping overflow), Signed Halving (keeping overflow by dropping 1 LSB bit), Unsigned Halving, Signed Saturation (clipping overflow), and Unsigned Saturation. Together, there are 30 SIMD 16-bit add/subtract instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_ADD16 (unsigned long a, unsigned long b)
```

ADD16 (SIMD 16-bit Addition)

Type: SIMD

Syntax:

```
ADD16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit integer element additions simultaneously.

Description :

This instruction adds the 16-bit integer elements in Rs1 with the 16-bit integer elements in Rs2, and then writes the 16-bit element results to Rd.

Note :

This instruction can be used for either signed or unsigned addition.

Operations:

```
Rd.H[x] = Rs1.H[x] + Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_CRAS16 (unsigned long a, unsigned long b)
```

CRAS16 (SIMD 16-bit Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
CRAS16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit integer element addition and 16-bit integer element subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction adds the 16-bit integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit integer element in [15:0] of 32-bit chunks in Rs2, and writes the result to [31:16] of 32-bit chunks in Rd; at the same time, it subtracts the 16-bit integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit integer element in [15:0] of 32-bit chunks, and writes the result to [15:0] of 32-bit chunks in Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[x][31:16] = Rs1.W[x][31:16] + Rs2.W[x][15:0];
Rd.W[x][15:0] = Rs1.W[x][15:0] - Rs2.W[x][31:16];
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CRSA16 (unsigned long a, unsigned long b)

CRSA16 (SIMD 16-bit Cross Subtraction & Addition)

Type: SIMD

Syntax:

CRSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit integer element subtraction and 16-bit integer element addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit integer element in [31:16] of 32-bit chunks in Rs1, and writes the result to [31:16] of 32-bit chunks in Rd; at the same time, it adds the 16-bit integer element in [31:16] of 32-bit chunks in Rs2 with the 16-bit integer element in [15:0] of 32-bit chunks in Rs1, and writes the result to [15:0] of 32-bit chunks in Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[x][31:16] = Rs1.W[x][31:16] - Rs2.W[x][15:0];
Rd.W[x][15:0] = Rs1.W[x][15:0] + Rs2.W[x][31:16];
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KADD16 (unsigned long a, unsigned long b)

KADD16 (SIMD 16-bit Signed Saturating Addition)

Type: SIMD

Syntax:

KADD16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.H[x] + Rs2.H[x];
if (res[x] > 32767) {
    res[x] = 32767;
    OV = 1;
} else if (res[x] < -32768) {
    res[x] = -32768;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KCRAS16 (unsigned long a, unsigned long b)

KCRAS16 (SIMD 16-bit Signed Saturating Cross Addition & Subtraction)

Type: SIMD

Syntax:

KCRAS16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element saturating addition and 16-bit signed integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for addition and [15:0] of 32-bit chunks in Rd for subtraction.

Operations:

```

res1 = Rs1.W[x][31:16] + Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] - Rs2.W[x][31:16];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KCRSA16 (unsigned long a, unsigned long b)

KCRSA16 (SIMD 16-bit Signed Saturating Cross Subtraction & Addition)

Type: SIMD

Syntax:

KCRSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element saturating subtraction and 16-bit signed integer element saturating addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```

res1 = Rs1.W[x][31:16] - Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] + Rs2.W[x][31:16];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    }
}

```

(continues on next page)

(continued from previous page)

```

    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSTAS16 (unsigned long a, unsigned long b)

KSTAS16 (SIMD 16-bit Signed Saturating Straight Addition & Subtraction)

Type: SIMD

Syntax:

```
KSTAS16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer element saturating addition and 16-bit signed integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for addition and [15:0] of 32-bit chunks in Rd for subtraction.

Operations:

```

res1 = Rs1.W[x][31:16] + Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] - Rs2.W[x][15:0];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;

```

(continues on next page)

(continued from previous page)

```

Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KTSA16 (unsigned long a, unsigned long b)

KTSA16 (SIMD 16-bit Signed Saturating Straight Subtraction & Addition)

Type: SIMD

Syntax:

```
KTSA16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer element saturating subtraction and 16-bit signed integer element saturating addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```

res1 = Rs1.W[x][31:16] - Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] + Rs2.W[x][15:0];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a

- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSUB16 (unsigned long a, unsigned long b)

KSUB16 (SIMD 16-bit Signed Saturating Subtraction)

Type: SIMD

Syntax:

```
KSUB16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 16-bit signed integer elements in Rs2 from the 16-bit signed integer elements in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.H[x] - Rs2.H[x];
if (res[x] > (2^15)-1) {
    res[x] = (2^15)-1;
    OV = 1;
} else if (res[x] < -2^15) {
    res[x] = -2^15;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RADD16 (unsigned long a, unsigned long b)

RADD16 (SIMD 16-bit Signed Halving Addition)

Type: SIMD

Syntax:

```
RADD16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Rs1 = 0x7FFF, Rs2 = 0x7FFF, Rd = 0x7FFF
* Rs1 = 0x8000, Rs2 = 0x8000, Rd = 0x8000
* Rs1 = 0x4000, Rs2 = 0x8000, Rd = 0xE000
```

Operations:

```
Rd.H[x] = (Rs1.H[x] + Rs2.H[x]) s>> 1; for RV32: x=1...0, for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RCRAS16 (unsigned long a, unsigned long b)

RCRAS16 (SIMD 16-bit Signed Halving Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
RCRAS16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer element addition and 16-bit signed integer element subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2, and subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `RADD16` and `RSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] + Rs2.W[x][15:0]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] - Rs2.W[x][31:16]) s>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RCRSA16 (unsigned long a, unsigned long b)

RCRSA16 (SIMD 16-bit Signed Halving Cross Subtraction & Addition)

Type: SIMD

Syntax:

RCRSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element subtraction and 16-bit signed integer element addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1, and adds the 16-bit signed element integer in [15:0] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2. The two results are first arithmetically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `RADD16` and `RSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] - Rs2.W[x][15:0]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] + Rs2.W[x][31:16]) s>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSTAS16 (unsigned long a, unsigned long b)

RSTAS16 (SIMD 16-bit Signed Halving Straight Addition & Subtraction)

Type: SIMD

Syntax:

RSTAS16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element addition and 16-bit signed integer element subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2, and subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `RADD16` and `RSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] + Rs2.W[x][31:16]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] - Rs2.W[x][15:0]) s>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSTSA16 (unsigned long a, unsigned long b)

RSTSA16 (SIMD 16-bit Signed Halving Straight Subtraction & Addition)

Type: SIMD

Syntax:

RSTSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element subtraction and 16-bit signed integer element addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1, and adds the 16-bit signed element integer in [15:0] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2. The two results are first arithmetically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `RADD16` and `RSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] - Rs2.W[x][31:16]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] + Rs2.W[x][15:0]) s>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSUB16 (unsigned long a, unsigned long b)

RSUB16 (SIMD 16-bit Signed Halving Subtraction)

Type: SIMD

Syntax:

RSUB16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit signed integer elements in Rs2 from the 16-bit signed integer elements in Rs1. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7FFF, Rb = 0x8000, Rt = 0x7FFF
* Ra = 0x8000, Rb = 0x7FFF, Rt = 0x8000
* Ra = 0x8000, Rb = 0x4000, Rt = 0xA000
```

Operations:

```
Rd.H[x] = (Rs1.H[x] - Rs2.H[x]) s>> 1;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_STAS16 (unsigned long a, unsigned long b)

STAS16 (SIMD 16-bit Straight Addition & Subtraction)

Type: SIMD

Syntax:

STAS16 Rd, Rs1, Rs2

Purpose :

Do 16-bit integer element addition and 16-bit integer element subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction adds the 16-bit integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit integer element in [31:16] of 32-bit chunks in Rs2, and writes the result to [31:16] of 32-bit chunks in Rd; at the same time, it subtracts the 16-bit integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit integer element in [15:0] of 32-bit chunks, and writes the result to [15:0] of 32-bit chunks in Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[x][31:16] = Rs1.W[x][31:16] + Rs2.W[x][31:16];
Rd.W[x][15:0] = Rs1.W[x][15:0] - Rs2.W[x][15:0];
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_STSA16 (unsigned long a, unsigned long b)

STSA16 (SIMD 16-bit Straight Subtraction & Addition)

Type: SIMD

Syntax:

STSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit integer element subtraction and 16-bit integer element addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit integer element in [31:16] of 32-bit chunks in Rs1, and writes the result to [31:16] of 32-bit chunks in Rd; at the same time, it adds the 16-bit integer element in [15:0] of 32-bit chunks in Rs2 with the 16-bit integer element in [15:0] of 32-bit chunks in Rs1, and writes the result to [15:0] of 32-bit chunks in Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[x][31:16] = Rs1.W[x][31:16] - Rs2.W[x][31:16];
Rd.W[x][15:0] = Rs1.W[x][15:0] + Rs2.W[x][15:0];
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUB16 (unsigned long a, unsigned long b)

SUB16 (SIMD 16-bit Subtraction)

Type: SIMD

Syntax:

```
SUB16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit integer element subtractions simultaneously.

Description :

This instruction subtracts the 16-bit integer elements in Rs2 from the 16-bit integer elements in Rs1, and then writes the result to Rd.

Note :

This instruction can be used for either signed or unsigned subtraction.

Operations:

```
Rd.H[x] = Rs1.H[x] - Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKADD16 (unsigned long a, unsigned long b)

UKADD16 (SIMD 16-bit Unsigned Saturating Addition)

Type: SIMD

Syntax:

```
UKADD16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer element saturating additions simultaneously.

Description :

This instruction adds the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.H[x] + Rs2.H[x];
if (res[x] > (2^16)-1) {
    res[x] = (2^16)-1;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKCRAS16 (unsigned long a, unsigned long b)

UKCRAS16 (SIMD 16-bit Unsigned Saturating Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
UKCRAS16 Rd, Rs1, Rs2
```

Purpose :

Do one 16-bit unsigned integer element saturating addition and one 16-bit unsigned integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction adds the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for addition and [15:0] of 32-bit chunks in Rd for subtraction.

Operations:


```

res1 = Rs1.W[x][31:16] + Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] - Rs2.W[x][31:16];
if (res1 > (2^16)-1) {
    res1 = (2^16)-1;
    OV = 1;
}
if (res2 < 0) {
    res2 = 0;
    OV = 1;
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKCRSA16 (unsigned long a, unsigned long b)

UKCRSA16 (SIMD 16-bit Unsigned Saturating Cross Subtraction & Addition)

Type: SIMD

Syntax:

UKCRSA16 Rd, Rs1, Rs2

Purpose :

Do one 16-bit unsigned integer element saturating subtraction and one 16-bit unsigned integer element saturating addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs2 with the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```

res1 = Rs1.W[x][31:16] - Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] + Rs2.W[x][31:16];
if (res1 < 0) {
    res1 = 0;
    OV = 1;
} else if (res2 > (2^16)-1) {
    res2 = (2^16)-1;
}

```

(continues on next page)

(continued from previous page)

```

    OV = 1;
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSTAS16 (unsigned long a, unsigned long b)

UKSTAS16 (SIMD 16-bit Unsigned Saturating Straight Addition & Subtraction)

Type: SIMD

Syntax:

```
UKSTAS16 Rd, Rs1, Rs2
```

Purpose :

Do one 16-bit unsigned integer element saturating addition and one 16-bit unsigned integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction adds the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for addition and [15:0] of 32-bit chunks in Rd for subtraction.

Operations:

```

res1 = Rs1.W[x][31:16] + Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] - Rs2.W[x][15:0];
if (res1 > (2^16)-1) {
    res1 = (2^16)-1;
    OV = 1;
}
if (res2 < 0) {
    res2 = 0;
    OV = 1;
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSTSA16 (unsigned long a, unsigned long b)

UKSTSA16 (SIMD 16-bit Unsigned Saturating Straight Subtraction & Addition)

Type: SIMD

Syntax:

UKSTSA16 Rd, Rs1, Rs2

Purpose :

Do one 16-bit unsigned integer element saturating subtraction and one 16-bit unsigned integer element saturating addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit unsigned integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs2 with the 16-bit unsigned integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```
res1 = Rs1.W[x][31:16] - Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] + Rs2.W[x][15:0];
if (res1 < 0) {
    res1 = 0;
    OV = 1;
} else if (res2 > (2^16)-1) {
    res2 = (2^16)-1;
    OV = 1;
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSUB16 (unsigned long a, unsigned long b)
UKSUB16 (SIMD 16-bit Unsigned Saturating Subtraction)

Type: SIMD

Syntax:

UKSUB16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 16-bit unsigned integer elements in Rs2 from the 16-bit unsigned integer elements in Rs1. If any of the results are beyond the 16-bit unsigned number range ($0 \leq \text{RES} \leq 2^{16}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.H[x] - Rs2.H[x];
if (res[x] < 0) {
    res[x] = 0;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URADD16 (unsigned long a, unsigned long b)
URADD16 (SIMD 16-bit Unsigned Halving Addition)

Type: SIMD

Syntax:

URADD16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7FFF, Rb = 0x7FFF Rt = 0x7FFF
* Ra = 0x8000, Rb = 0x8000 Rt = 0x8000
* Ra = 0x4000, Rb = 0x8000 Rt = 0x6000
```

Operations:

```
Rd.H[x] = (Rs1.H[x] + Rs2.H[x]) u>> 1;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URCRAS16 (unsigned long a, unsigned long b)

URCRAS16 (SIMD 16-bit Unsigned Halving Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
URCRAS16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer element addition and 16-bit unsigned integer element subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs1 with the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs2, and subtracts the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs2 from the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs1. The element results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `URADD16` and `URSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] + Rs2.W[x][15:0]) u>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] - Rs2.W[x][31:16]) u>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URCSA16 (unsigned long a, unsigned long b)

URCSA16 (SIMD 16-bit Unsigned Halving Cross Subtraction & Addition)

Type: SIMD**Syntax:**

URCSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer element subtraction and 16-bit unsigned integer element addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs2 from the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs1, and adds the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs1 with the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs2. The two results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `URADD16` and `URSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] - Rs2.W[x][15:0]) u>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] + Rs2.W[x][31:16]) u>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSTAS16 (unsigned long a, unsigned long b)

URSTAS16 (SIMD 16-bit Unsigned Halving Straight Addition & Subtraction)

Type: SIMD**Syntax:**

URSTAS16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer element addition and 16-bit unsigned integer element subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs1 with the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs2, and subtracts the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs2 from the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs1. The element results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `URADD16` and `URSUB16` instructions.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] + Rs2.W[x][31:16]) u>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] - Rs2.W[x][15:0]) u>> 1;
for RV32, x=0
for RV64, x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSTSA16 (unsigned long a, unsigned long b)

URSTSA16 (SIMD 16-bit Unsigned Halving Straight Subtraction & Addition)

Type: SIMD

Syntax:

URCRSA16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer element subtraction and 16-bit unsigned integer element addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs2 from the 16-bit unsigned integer in [31:16] of 32-bit chunks in Rs1, and adds the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs1 with the 16-bit unsigned integer in [15:0] of 32-bit chunks in Rs2. The two results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Examples:

Please see `URADD16` and `URSUB16` instructions.

Operations:

```

Rd.W[x][31:16] = (Rs1.W[x][31:16] - Rs2.W[x][31:16]) u>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] + Rs2.W[x][15:0]) u>> 1;
for RV32, x=0
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSUB16 (unsigned long a, unsigned long b)

URSUB16 (SIMD 16-bit Unsigned Halving Subtraction)

Type: SIMD

Syntax:

```
URSUB16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit unsigned integer elements in Rs2 from the 16-bit unsigned integer elements in Rs1. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```

* Ra = 0x7FFF, Rb = 0x8000 Rt = 0xFFFF
* Ra = 0x8000, Rb = 0x7FFF Rt = 0x0000
* Ra = 0x8000, Rb = 0x4000 Rt = 0x2000

```

Operations:

```

Rd.H[x] = (Rs1.H[x] - Rs2.H[x]) u>> 1;
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 8-bit Addition & Subtraction Instructions

```

__STATIC_FORCEINLINE unsigned long __RV_ADD8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KADD8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KSUB8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RADD8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSUB8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_SUB8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKADD8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSUB8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URADD8 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSUB8 (unsigned long a, unsigned long b)

```

group SIMD 8-bit Addition & Subtraction Instructions

SIMD 8-bit Addition & Subtraction Instructions.

Based on the types of the four 8-bit arithmetic operations, the SIMD 8-bit add/subtract instructions can be classified into 2 main categories: Addition (four 8-bit addition), and Subtraction (four 8-bit subtraction). Based on the way of how an overflow condition is handled for signed or unsigned operation, the SIMD 8-bit add/subtract instructions can be classified into 5 groups: Wrap-around (dropping overflow), Signed Halving (keeping overflow by dropping 1 LSB bit), Unsigned Halving, Signed Saturation (clipping overflow), and Unsigned Saturation. Together, there are 10 SIMD 8-bit add/subtract instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_ADD8 (unsigned long a, unsigned long b)
```

ADD8 (SIMD 8-bit Addition)

Type: SIMD

Syntax:

```
ADD8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit integer element additions simultaneously.

Description :

This instruction adds the 8-bit integer elements in Rs1 with the 8-bit integer elements in Rs2, and then writes the 8-bit element results to Rd.

Note :

This instruction can be used for either signed or unsigned addition.

Operations:

```
Rd.B[x] = Rs1.B[x] + Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KADD8 (unsigned long a, unsigned long b)

KADD8 (SIMD 8-bit Signed Saturating Addition)

Type: SIMD

Syntax:

```
KADD8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2. If any of the results are beyond the Q7 number range ($-2^7 \leq Q7 \leq 2^7-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.B[x] + Rs2.B[x];
if (res[x] > 127) {
    res[x] = 127;
    OV = 1;
} else if (res[x] < -128) {
    res[x] = -128;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSUB8 (unsigned long a, unsigned long b)

KSUB8 (SIMD 8-bit Signed Saturating Subtraction)

Type: SIMD

Syntax:

```
KSUB8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 8-bit signed integer elements in Rs2 from the 8-bit signed integer elements in Rs1. If any of the results are beyond the Q7 number range ($-2^7 \leq Q7 \leq 2^7 - 1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.B[x] - Rs2.B[x];
if (res[x] > (2^7)-1) {
    res[x] = (2^7)-1;
    OV = 1;
} else if (res[x] < -2^7) {
    res[x] = -2^7;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RADD8 (unsigned long a, unsigned long b)

RADD8 (SIMD 8-bit Signed Halving Addition)

Type: SIMD

Syntax:

```
RADD8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer element additions simultaneously. The element results are halved to avoid overflow or saturation.

Description :

This instruction adds the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Rs1 = 0x7F, Rs2 = 0x7F, Rd = 0x7F
* Rs1 = 0x80, Rs2 = 0x80, Rd = 0x80
* Rs1 = 0x40, Rs2 = 0x80, Rd = 0xE0
```

Operations:

```
Rd.B[x] = (Rs1.B[x] + Rs2.B[x]) s>> 1; for RV32: x=3...0, for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSUB8 (unsigned long a, unsigned long b)

RSUB8 (SIMD 8-bit Signed Halving Subtraction)

Type: SIMD

Syntax:

```
RSUB8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 8-bit signed integer elements in Rs2 from the 8-bit signed integer elements in Rs1. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Rs1 = 0x7F, Rs2 = 0x80, Rd = 0x7F
* Rs1 = 0x80, Rs2 = 0x7F, Rd = 0x80
* Rs1 = 0x80, Rs2 = 0x40, Rd = 0xA0
```

Operations:

```
Rd.B[x] = (Rs1.B[x] - Rs2.B[x]) s>> 1;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUB8 (unsigned long a, unsigned long b)

SUB8 (SIMD 8-bit Subtraction)

Type: SIMD

Syntax:

SUB8 Rd, Rs1, Rs2

Purpose :

Do 8-bit integer element subtractions simultaneously.

Description :

This instruction subtracts the 8-bit integer elements in Rs2 from the 8-bit integer elements in Rs1, and then writes the result to Rd.

Note :

This instruction can be used for either signed or unsigned subtraction.

Operations:

```
Rd.B[x] = Rs1.B[x] - Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKADD8 (unsigned long a, unsigned long b)

UKADD8 (SIMD 8-bit Unsigned Saturating Addition)

Type: SIMD

Syntax:

UKADD8 Rd, Rs1, Rs2

Purpose :

Do 8-bit unsigned integer element saturating additions simultaneously.

Description :

This instruction adds the 8-bit unsigned integer elements in Rs1 with the 8-bit unsigned integer elements in Rs2. If any of the results are beyond the 8-bit unsigned number range ($0 \leq \text{RES} \leq 28-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```

res[x] = Rs1.B[x] + Rs2.B[x];
if (res[x] > (2^8)-1) {
    res[x] = (2^8)-1;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSUB8 (unsigned long a, unsigned long b)

UKSUB8 (SIMD 8-bit Unsigned Saturating Subtraction)

Type: SIMD

Syntax:

```
UKSUB8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit unsigned integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 8-bit unsigned integer elements in Rs2 from the 8-bit unsigned integer elements in Rs1. If any of the results are beyond the 8-bit unsigned number range ($0 \leq \text{RES} \leq 28-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```

res[x] = Rs1.B[x] - Rs2.B[x];
if (res[x] < 0) {
    res[x] = 0;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URADD8 (unsigned long a, unsigned long b)

URADD8 (SIMD 8-bit Unsigned Halving Addition)

Type: SIMD

Syntax:

```
URADD8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit unsigned integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 8-bit unsigned integer elements in Rs1 with the 8-bit unsigned integer elements in Rs2. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7F, Rb = 0x7F, Rt = 0x7F
* Ra = 0x80, Rb = 0x80, Rt = 0x80
* Ra = 0x40, Rb = 0x80, Rt = 0x60
```

Operations:

```
Rd.B[x] = (Rs1.B[x] + Rs2.B[x]) u>> 1;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSUB8 (unsigned long a, unsigned long b)

URSUB8 (SIMD 8-bit Unsigned Halving Subtraction)

Type: SIMD

Syntax:

```
URSUB8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit unsigned integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 8-bit unsigned integer elements in Rs2 from the 8-bit unsigned integer elements in Rs1. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7F, Rb = 0x80 Rt = 0xFF
* Ra = 0x80, Rb = 0x7F Rt = 0x00
* Ra = 0x80, Rb = 0x40 Rt = 0x20
```

Operations:

```
Rd.B[x] = (Rs1.B[x] - Rs2.B[x]) u>> 1;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 16-bit Shift Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KSL16 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA16 (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA16_U (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SLL16 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRA16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRA16_U (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRL16 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRL16_U (unsigned long a, unsigned int b)
```

```
__RV_KSL16(a, b)
```

```
__RV_SLL16(a, b)
```

```
__RV_SRA16(a, b)
```

```
__RV_SRA16_U(a, b)
```

```
__RV_SRL16(a, b)
```

```
__RV_SRL16_U(a, b)
```


group SIMD 16-bit Shift Instructions

SIMD 16-bit Shift Instructions.

there are 14 SIMD 16-bit shift instructions.

Defines

__RV_KSLLI16(a, b)

KSLLI16 (SIMD 16-bit Saturating Shift Left Logical Immediate)

Type: SIMD

Syntax:

```
KSLLI16 Rd, Rs1, imm4u
```

Purpose :

Do 16-bit elements logical left shift operations with saturation simultaneously. The shift amount is an immediate value.

Description :

The 16-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm4u constant. Any shifted value greater than $2^{15}-1$ is saturated to $2^{15}-1$. Any shifted value smaller than -2^{15} is saturated to -2^{15} . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```
sa = imm4u[3:0];
if (sa != 0) {
    res[(15+sa):0] = Rs1.H[x] << sa;
    if (res > (2^15)-1) {
        res = 0x7fff; OV = 1;
    } else if (res < -2^15) {
        res = 0x8000; OV = 1;
    }
    Rd.H[x] = res[15:0];
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SLLI16(a, b)

SLLI16 (SIMD 16-bit Shift Left Logical Immediate)

Type: SIMD**Syntax:**

SLLI16 Rd, Rs1, imm4[3:0]

Purpose :

Do 16-bit element logical left shift operations simultaneously. The shift amount is an immediate value.

Description :

The 16-bit elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm4[3:0] constant. And the results are written to Rd.

Operations:

sa = imm4[3:0];
Rd.H[x] = Rs1.H[x] << sa;
for RV32: x=1...0,
for RV64: x=3...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI16(a, b)

SRAI16 (SIMD 16-bit Shift Right Arithmetic Immediate)

Type: SIMD**Syntax:**

SRAI16 Rd, Rs1, imm4u
SRAI16.u Rd, Rs1, imm4u

Purpose :

Do 16-bit elements arithmetic right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the 16-bit data elements. The shift amount is specified by the imm4u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data to calculate the final results. And the results are written to Rd.

Operations:

```

sa = imm4u[3:0];
if (sa > 0) {
    if (`.u` form) { // SRAI16.u
        res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[15:0];
    } else { // SRAI16
        Rd.H[x] = SE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI16_U(a, b)

SRAI16.u (SIMD 16-bit Rounding Shift Right Arithmetic Immediate)

Type: SIMD

Syntax:

```

SRAI16 Rd, Rs1, imm4u
SRAI16.u Rd, Rs1, imm4u

```

Purpose :

Do 16-bit elements arithmetic right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the 16-bit data elements. The shift amount is specified by the imm4u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data to calculate the final results. And the results are written to Rd.

Operations:

```

sa = imm4u[3:0];
if (sa > 0) {
    if (`.u` form) { // SRAI16.u
        res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[15:0];
    } else { // SRAI16
        Rd.H[x] = SE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}

```

(continues on next page)

(continued from previous page)

```

}
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI16(a, b)

SRLI16 (SIMD 16-bit Shift Right Logical Immediate)

Type: SIMD

Syntax:

```

SRLI16 Rt, Ra, imm4u
SRLI16.u Rt, Ra, imm4u

```

Purpose :

Do 16-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm4u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = imm4u;
if (sa > 0) {
    if (`.u` form) { // SRLI16.u
        res[16:0] = ZE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[16:1];
    } else { // SRLI16
        Rd.H[x] = ZE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI16_U(a, b)

SRLI16.u (SIMD 16-bit Rounding Shift Right Logical Immediate)

Type: SIMD

Syntax:

```
SRLI16 Rt, Ra, imm4u
SRLI16.u Rt, Ra, imm4u
```

Purpose :

Do 16-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm4u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = imm4u;
if (sa > 0) {
    if (`.u` form) { // SRLI16.u
        res[16:0] = ZE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[16:1];
    } else { // SRLI16
        Rd.H[x] = ZE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_KSL16 (unsigned long a, unsigned int b)

KSL16 (SIMD 16-bit Saturating Shift Left Logical)

Type: SIMD

Syntax:

```
KSL16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit elements logical left shift operations with saturation simultaneously. The shift amount is a variable from a GPR.

Description :

The 16-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the low-order 4-bits of the value in the Rs2 register. Any shifted value greater than $2^{15}-1$ is saturated to $2^{15}-1$. Any shifted value smaller than -2^{15} is saturated to -2^{15} . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```
sa = Rs2[3:0];
if (sa != 0) {
    res[(15+sa):0] = Rs1.H[x] << sa;
    if (res > (2^15)-1) {
        res = 0x7fff; OV = 1;
    } else if (res < -2^15) {
        res = 0x8000; OV = 1;
    }
    Rd.H[x] = res[15:0];
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA16 (unsigned long a, int b)

KSLRA16 (SIMD 16-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD

Syntax:

```
KSLRA16 Rd, Rs1, Rs2
KSLRA16.u Rd, Rs1, Rs2
```

Purpose :

Do 16-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q15 saturation for the left shift. The `.u` form performs additional rounding up operations for the right shift.

Description :

The 16-bit data elements of `Rs1` are left-shifted logically or right-shifted arithmetically based on the value of `Rs2[4:0]`. `Rs2[4:0]` is in the signed range of $[-2^4, 2^4-1]$. A positive `Rs2[4:0]` means logical left shift and a negative `Rs2[4:0]` means arithmetic right shift. The shift amount is the absolute value of `Rs2[4:0]`. However, the behavior of `Rs2[4:0] == -2^4 (0x10)` is defined to be equivalent to the behavior of `Rs2[4:0] == -(2^4-1) (0x11)`. The left-shifted results are saturated to the 16-bit signed integer range of $[-2^{15}, 2^{15}-1]$. For the `.u` form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to `Rd`. If any saturation happens, this instruction sets the OV flag. The value of `Rs2[31:5]` will not affect this instruction.

Operations:

```
if (Rs2[4:0] < 0) {
    sa = -Rs2[4:0];
    sa = (sa == 16)? 15 : sa;
    if (`.u` form) {
        res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[15:0];
    } else {
        Rd.H[x] = SE16(Rs1.H[x][15:sa]);
    }
} else {
    sa = Rs2[3:0];
    res[(15+sa):0] = Rs1.H[x] <<(logic) sa;
    if (res > (2^15)-1) {
        res[15:0] = 0x7fff; OV = 1;
    } else if (res < -2^15) {
        res[15:0] = 0x8000; OV = 1;
    }
    d.H[x] = res[15:0];
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA16_U (unsigned long a, int b)

KSLRA16.u (SIMD 16-bit Shift Left Logical with Saturation or Rounding Shift Right Arithmetic)

Type: SIMD

Syntax:

```
KSLRA16 Rd, Rs1, Rs2
KSLRA16.u Rd, Rs1, Rs2
```

Purpose :

Do 16-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q15 saturation for the left shift. The .u form performs additional rounding up operations for the right shift.

Description :

The 16-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[4:0]. Rs2[4:0] is in the signed range of $[-2^4, 2^4-1]$. A positive Rs2[4:0] means logical left shift and a negative Rs2[4:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[4:0]. However, the behavior of Rs2[4:0]==-2⁴ (0x10) is defined to be equivalent to the behavior of Rs2[4:0]==-(2⁴-1) (0x11). The left-shifted results are saturated to the 16-bit signed integer range of $[-2^{15}, 2^{15}-1]$. For the .u form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:5] will not affect this instruction.

Operations:

```
if (Rs2[4:0] < 0) {
    sa = -Rs2[4:0];
    sa = (sa == 16)? 15 : sa;
    if (`.u` form) {
        res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[15:0];
    } else {
        Rd.H[x] = SE16(Rs1.H[x][15:sa]);
    }
} else {
    sa = Rs2[3:0];
    res[(15+sa):0] = Rs1.H[x] <<(logic) sa;
    if (res > (2^15)-1) {
        res[15:0] = 0x7fff; OV = 1;
    } else if (res < -2^15) {
        res[15:0] = 0x8000; OV = 1;
    }
    d.H[x] = res[15:0];
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SLL16 (unsigned long a, unsigned int b)

SLL16 (SIMD 16-bit Shift Left Logical)

Type: SIMD

Syntax:

```
SLL16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit elements logical left shift operations simultaneously. The shift amount is a variable from a GPR.

Description :

The 16-bit elements in Rs1 are left-shifted logically. And the results are written to Rd. The shifted out bits are filled with zero and the shift amount is specified by the low-order 4-bits of the value in the Rs2 register.

Operations:

```
sa = Rs2[3:0];
Rd.H[x] = Rs1.H[x] << sa;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA16 (unsigned long a, unsigned long b)

SRA16 (SIMD 16-bit Shift Right Arithmetic)

Type: SIMD

Syntax:

```
SRA16 Rd, Rs1, Rs2
SRA16.u Rd, Rs1, Rs2
```

Purpose :

Do 16-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 4-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[3:0];
if (sa != 0) {
    if (`.u` form) { // SRA16.u
        res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[15:0];
    } else { // SRA16
```

(continues on next page)

(continued from previous page)

```

    Rd.H[x] = SE16(Rs1.H[x][15:sa])
  }
} else {
  Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA16_U (unsigned long a, unsigned long b)

SRA16.u (SIMD 16-bit Rounding Shift Right Arithmetic)

Type: SIMD

Syntax:

```

SRA16 Rd, Rs1, Rs2
SRA16.u Rd, Rs1, Rs2

```

Purpose :

Do 16-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 4-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = Rs2[3:0];
if (sa != 0) {
  if (`.u` form) { // SRA16.u
    res[15:-1] = SE17(Rs1.H[x][15:sa-1]) + 1;
    Rd.H[x] = res[15:0];
  } else { // SRA16
    Rd.H[x] = SE16(Rs1.H[x][15:sa])
  }
} else {
  Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL16 (unsigned long a, unsigned int b)

SRL16 (SIMD 16-bit Shift Right Logical)

Type: SIMD

Syntax:

```
SRL16 Rt, Ra, Rb
SRL16.u Rt, Ra, Rb
```

Purpose :

Do 16-bit elements logical right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding upoperations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 4-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[3:0];
if (sa > 0) {
    if (`.u` form) { // SRL16.u
        res[16:0] = ZE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[16:1];
    } else { // SRL16
        Rd.H[x] = ZE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL16_U (unsigned long a, unsigned int b)

SRL16.u (SIMD 16-bit Rounding Shift Right Logical)

Type: SIMD

Syntax:

```
SRL16 Rt, Ra, Rb
SRL16.u Rt, Ra, Rb
```

Purpose :

Do 16-bit elements logical right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding upoperations on the shifted results.

Description :

The 16-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 4-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 16-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[3:0];
if (sa > 0) {
    if (`.u` form) { // SRL16.u
        res[16:0] = ZE17(Rs1.H[x][15:sa-1]) + 1;
        Rd.H[x] = res[16:1];
    } else { // SRL16
        Rd.H[x] = ZE16(Rs1.H[x][15:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

SIMD 8-bit Shift Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KSLL8 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA8 (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA8_U (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SLL8 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRA8 (unsigned long a, unsigned int b)
```

```

__STATIC_FORCEINLINE unsigned long __RV_SRA8_U (unsigned long a, unsigned int b)

__STATIC_FORCEINLINE unsigned long __RV_SRL8 (unsigned long a, unsigned int b)

__STATIC_FORCEINLINE unsigned long __RV_SRL8_U (unsigned long a, unsigned int b)

__RV_KSLLI8(a, b)
__RV_SLLI8(a, b)
__RV_SRAI8(a, b)
__RV_SRAI8_U(a, b)
__RV_SRLI8(a, b)
__RV_SRLI8_U(a, b)

```

group SIMD 8-bit Shift Instructions

SIMD 8-bit Shift Instructions.

there are 14 SIMD 8-bit shift instructions.

Defines

__RV_KSLLI8(a, b)
KSLLI8 (SIMD 8-bit Saturating Shift Left Logical Immediate)

Type: SIMD

Syntax:

```
KSLLI8 Rd, Rs1, imm3u
```

Purpose :

Do 8-bit elements logical left shift operations with saturation simultaneously. The shift amount is an immediate value.

Description :

The 8-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm3u constant. Any shifted value greater than 2^7-1 is saturated to 2^7-1 . Any shifted value smaller than -2^7 is saturated to -2^7 . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```

sa = imm3u[2:0];
if (sa != 0) {
    res[(7+sa):0] = Rs1.B[x] << sa;
    if (res > (2^7)-1) {
        res = 0x7f; OV = 1;
    } else if (res < -2^7) {
        res = 0x80; OV = 1;
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    Rd.B[x] = res[7:0];
  } else {
    Rd = Rs1;
  }
  for RV32: x=3...0,
  for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SLLI8(a, b)

SLLI8 (SIMD 8-bit Shift Left Logical Immediate)

Type: SIMD

Syntax:

```
SLLI8 Rd, Rs1, imm3u
```

Purpose :

Do 8-bit elements logical left shift operations simultaneously. The shift amount is an immediate value.

Description :

The 8-bit elements in Rs1 are left-shifted logically. And the results are written to Rd. The shifted out bits are filled with zero and the shift amount is specified by the imm3u constant.

Operations:

```

sa = imm3u[2:0];
Rd.B[x] = Rs1.B[x] << sa;
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI8(a, b)

SRAI8 (SIMD 8-bit Shift Right Arithmetic Immediate)

Type: SIMD

Syntax:

```
SRAI8 Rd, Rs1, imm3u
SRAI8.u Rd, Rs1, imm3u
```

Purpose :

Do 8-bit element arithmetic right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the imm3u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = imm3u[2:0];
if (sa > 0) {
    if (`.u` form) { // SRA8.u
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else { // SRA8
        Rd.B[x] = SE8(Rd.B[x][7:sa])
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI8_U(a, b)

SRAI8.u (SIMD 8-bit Rounding Shift Right Arithmetic Immediate)

Type: SIMD

Syntax:

```
SRAI8 Rd, Rs1, imm3u
SRAI8.u Rd, Rs1, imm3u
```

Purpose :

Do 8-bit element arithmetic right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the imm3u constant. For the rounding

operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = imm3u[2:0];
if (sa > 0) {
    if (`.u` form) { // SRA8.u
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else { // SRA8
        Rd.B[x] = SE8(Rd.B[x][7:sa])
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI8(a, b)

SRLI8 (SIMD 8-bit Shift Right Logical Immediate)

Type: SIMD

Syntax:

```
SRLI8 Rt, Ra, imm3u
SRLI8.u Rt, Ra, imm3u
```

Purpose :

Do 8-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm3u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = imm3u[2:0];
if (sa > 0) {
    if (`.u` form) { // SRLI8.u
        res[8:0] = ZE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[8:1];
    } else { // SRLI8
```

(continues on next page)

(continued from previous page)

```

    Rd.B[x] = ZE8(Rs1.B[x][7:sa]);
}
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI8_U(a, b)

SRLI8.u (SIMD 8-bit Rounding Shift Right Logical Immediate)

Type: SIMD

Syntax:

```

SRLI8 Rt, Ra, imm3u
SRLI8.u Rt, Ra, imm3u

```

Purpose :

Do 8-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm3u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = imm3u[2:0];
if (sa > 0) {
    if (`.u` form) { // SRLI8.u
        res[8:0] = ZE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[8:1];
    } else { // SRLI8
        Rd.B[x] = ZE8(Rs1.B[x][7:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_KSLL8 (unsigned long a, unsigned int b)

KSLL8 (SIMD 8-bit Saturating Shift Left Logical)

Type: SIMD

Syntax:

KSLL8 Rd, Rs1, Rs2

Purpose :

Do 8-bit elements logical left shift operations with saturation simultaneously. The shift amount is a variable from a GPR.

Description :

The 8-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the low-order 3-bits of the value in the Rs2 register. Any shifted value greater than 2^7-1 is saturated to 2^7-1 . Any shifted value smaller than -2^7 is saturated to -2^7 . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```
sa = Rs2[2:0];
if (sa != 0) {
    res[(7+sa):0] = Rs1.B[x] << sa;
    if (res > (2^7)-1) {
        res = 0x7f; OV = 1;
    } else if (res < -2^7) {
        res = 0x80; OV = 1;
    }
    Rd.B[x] = res[7:0];
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA8 (unsigned long a, int b)

KSLRA8 (SIMD 8-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD

Syntax:

```
KSLRA8 Rd, Rs1, Rs2
KSLRA8.u Rd, Rs1, Rs2
```

Purpose :

Do 8-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q7 saturation for the left shift. The .u form performs additional rounding up operations for the right shift.

Description :

The 8-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[3:0]. Rs2[3:0] is in the signed range of $[-2^3, 2^3-1]$. A positive Rs2[3:0] means logical left shift and a negative Rs2[3:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[3:0]. However, the behavior of Rs2[3:0] == -2³ (0x8) is defined to be equivalent to the behavior of Rs2[3:0] == -(2³-1) (0x9). The left-shifted results are saturated to the 8-bit signed integer range of $[-2^7, 2^7-1]$. For the .u form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:4] will not affect this instruction.

Operations:

```
if (Rs2[3:0] < 0) {
    sa = -Rs2[3:0];
    sa = (sa == 8)? 7 : sa;
    if (`.u` form) {
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else {
        Rd.B[x] = SE8(Rs1.B[x][7:sa]);
    }
} else {
    sa = Rs2[2:0];
    res[(7+sa):0] = Rs1.B[x] <<(logic) sa;
    if (res > (2^7)-1) {
        res[7:0] = 0x7f; OV = 1;
    } else if (res < -2^7) {
        res[7:0] = 0x80; OV = 1;
    }
    Rd.B[x] = res[7:0];
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA8_U (unsigned long a, int b)

KSLRA8.u (SIMD 8-bit Shift Left Logical with Saturation or Rounding Shift Right Arithmetic)

Type: SIMD

Syntax:

```
KSLRA8 Rd, Rs1, Rs2
KSLRA8.u Rd, Rs1, Rs2
```

Purpose :

Do 8-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q7 saturation for the left shift. The .u form performs additional rounding up operations for the right shift.

Description :

The 8-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[3:0]. Rs2[3:0] is in the signed range of $[-2^3, 2^3-1]$. A positive Rs2[3:0] means logical left shift and a negative Rs2[3:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[3:0]. However, the behavior of $Rs2[3:0] == -2^3$ (0x8) is defined to be equivalent to the behavior of $Rs2[3:0] == -(2^3-1)$ (0x9). The left-shifted results are saturated to the 8-bit signed integer range of $[-2^7, 2^7-1]$. For the .u form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:4] will not affect this instruction.

Operations:

```
if (Rs2[3:0] < 0) {
    sa = -Rs2[3:0];
    sa = (sa == 8)? 7 : sa;
    if (`.u` form) {
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else {
        Rd.B[x] = SE8(Rs1.B[x][7:sa]);
    }
} else {
    sa = Rs2[2:0];
    res[(7+sa):0] = Rs1.B[x] <<(logic) sa;
    if (res > (2^7)-1) {
        res[7:0] = 0x7f; OV = 1;
    } else if (res < -2^7) {
        res[7:0] = 0x80; OV = 1;
    }
    Rd.B[x] = res[7:0];
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a

- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SLL8 (unsigned long a, unsigned int b)

SLL8 (SIMD 8-bit Shift Left Logical)

Type: SIMD

Syntax:

```
SLL8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit elements logical left shift operations simultaneously. The shift amount is a variable from a GPR.

Description :

The 8-bit elements in Rs1 are left-shifted logically. And the results are written to Rd. The shifted out bits are filled with zero and the shift amount is specified by the low-order 3-bits of the value in the Rs2 register.

Operations:

```
sa = Rs2[2:0];
Rd.B[x] = Rs1.B[x] << sa;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA8 (unsigned long a, unsigned int b)

SRA8 (SIMD 8-bit Shift Right Arithmetic)

Type: SIMD

Syntax:

```
SRA8 Rd, Rs1, Rs2
SRA8.u Rd, Rs1, Rs2
```

Purpose :

Do 8-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 3-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = Rs2[2:0];
if (sa > 0) {
    if (`.u` form) { // SRA8.u
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else { // SRA8
        Rd.B[x] = SE8(Rd.B[x][7:sa])
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA8_U (unsigned long a, unsigned int b)

SRA8.u (SIMD 8-bit Rounding Shift Right Arithmetic)

Type: SIMD

Syntax:

```

SRA8 Rd, Rs1, Rs2
SRA8.u Rd, Rs1, Rs2

```

Purpose :

Do 8-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 3-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = Rs2[2:0];
if (sa > 0) {
    if (`.u` form) { // SRA8.u
        res[7:-1] = SE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[7:0];
    } else { // SRA8
        Rd.B[x] = SE8(Rd.B[x][7:sa])
    }
}

```

(continues on next page)

(continued from previous page)

```

} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL8 (unsigned long a, unsigned int b)

SRL8 (SIMD 8-bit Shift Right Logical)

Type: SIMD

Syntax:

```

SRL8 Rt, Ra, Rb
SRL8.u Rt, Ra, Rb

```

Purpose :

Do 8-bit elements logical right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 3-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = Rs2[2:0];
if (sa > 0) {
    if (`.u` form) { // SRL8.u
        res[8:0] = ZE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[8:1];
    } else { // SRL8
        Rd.B[x] = ZE8(Rs1.B[x][7:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a

- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL8_U (unsigned long a, unsigned int b)

SRL8.u (SIMD 8-bit Rounding Shift Right Logical)

Type: SIMD

Syntax:

```
SRL8 Rt, Ra, Rb
SRL8.u Rt, Ra, Rb
```

Purpose :

Do 8-bit elements logical right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 8-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 3-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 8-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[2:0];
if (sa > 0) {
    if (`.u` form) { // SRL8.u
        res[8:0] = ZE9(Rs1.B[x][7:sa-1]) + 1;
        Rd.B[x] = res[8:1];
    } else { // SRL8
        Rd.B[x] = ZE8(Rs1.B[x][7:sa]);
    }
} else {
    Rd = Rs1;
}
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

SIMD 16-bit Compare Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_CMPEQ16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SCMPL16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SCMPLT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UCMPLE16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UCMPLT16 (unsigned long a, unsigned long b)
```

group SIMD 16-bit Compare Instructions

SIMD 16-bit Compare Instructions.

there are 5 SIMD 16-bit Compare instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_CMPEQ16 (unsigned long a, unsigned long b)
```

CMPEQ16 (SIMD 16-bit Integer Compare Equal)

Type: SIMD

Syntax:

```
CMPEQ16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit integer elements equal comparisons simultaneously.

Description :

This instruction compares the 16-bit integer elements in Rs1 with the 16-bit integer elements in Rs2 to see if they are equal. If they are equal, the result is 0xFFFF; otherwise, the result is 0x0. The 16-bit element comparison results are written to Rt.

Note :

This instruction can be used for either signed or unsigned numbers.

Operations:

```
Rd.H[x] = (Rs1.H[x] == Rs2.H[x])? 0xffff : 0x0;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SCMPL16 (unsigned long a, unsigned long b)

SCMPLE16 (SIMD 16-bit Signed Compare Less Than & Equal)

Type: SIMD

Syntax:

SCMPLE16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer elements less than & equal comparisons simultaneously.

Description :

This instruction compares the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2 to see if the one in Rs1 is less than or equal to the one in Rs2. If it is true, the result is 0xFFFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

Rd.H[x] = (Rs1.H[x] {le} Rs2.H[x])? 0xffff : 0x0;
for RV32: x=1...0,
for RV64: x=3...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SCMPLT16 (unsigned long a, unsigned long b)

SCMPLT16 (SIMD 16-bit Signed Compare Less Than)

Type: SIMD

Syntax:

SCMPLT16 Rd, Rs1, Rs2

Purpose :

Do 16-bit signed integer elements less than comparisons simultaneously.

Description :

This instruction compares the 16-bit signed integer elements in Rs1 with the two 16-bit signed integer elements in Rs2 to see if the one in Rs1 is less than the one in Rs2. If it is true, the result is 0xFFFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] < Rs2.H[x])? 0xffff : 0x0;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UCMPLE16 (unsigned long a, unsigned long b)

UCMPLE16 (SIMD 16-bit Unsigned Compare Less Than & Equal)

Type: SIMD

Syntax:

```
UCMPLE16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer elements less than & equal comparisons simultaneously.

Description :

This instruction compares the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2 to see if the one in Rs1 is less than or equal to the one in Rs2. If it is true, the result is 0xFFFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] <=u Rs2.H[x])? 0xffff : 0x0;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UCMPLT16 (unsigned long a, unsigned long b)

UCMPLT16 (SIMD 16-bit Unsigned Compare Less Than)

Type: SIMD

Syntax:

```
UCMPLT16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer elements less than comparisons simultaneously.

Description :

This instruction compares the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2 to see if the one in Rs1 is less than the one in Rs2. If it is true, the result is 0xFFFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] <u Rs2.H[x])? 0xffff : 0x0;
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 8-bit Compare Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_CMPEQ8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SCMPLT8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SCMPLE8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UCMPLE8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UCMPLT8 (unsigned long a, unsigned long b)
```

group **SIMD 8-bit Compare Instructions**

SIMD 8-bit Compare Instructions.

there are 5 SIMD 8-bit Compare instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_CMPEQ8 (unsigned long a, unsigned long b)
```

CMPEQ8 (SIMD 8-bit Integer Compare Equal)

Type: SIMD

Syntax:

```
CMPEQ8 Rs, Rs1, Rs2
```

Purpose :

Do 8-bit integer elements equal comparisons simultaneously.

Description :

This instruction compares the 8-bit integer elements in Rs1 with the 8-bit integer elements in Rs2 to see if they are equal. If they are equal, the result is 0xFF; otherwise, the result is 0x0. The 8-bit element comparison results are written to Rd.

Note :

This instruction can be used for either signed or unsigned numbers.

Operations:

```
Rd.B[x] = (Rs1.B[x] == Rs2.B[x])? 0xff : 0x0;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SCMPL8 (unsigned long a, unsigned long b)

SCMPLE8 (SIMD 8-bit Signed Compare Less Than & Equal)

Type: SIMD

Syntax:

```
SCMPLE8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer elements less than & equal comparisons simultaneously.

Description :

This instruction compares the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2 to see if the one in Rs1 is less than or equal to the one in Rs2. If it is true, the result is 0xFF; otherwise, the result is 0x0. The element comparison results are written to Rd

Operations:

```
Rd.B[x] = (Rs1.B[x] {le} Rs2.B[x])? 0xff : 0x0;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SCMLT8 (unsigned long a, unsigned long b)

SCMLT8 (SIMD 8-bit Signed Compare Less Than)

Type: SIMD

Syntax:

SCMLT8 Rd, Rs1, Rs2

Purpose :

Do 8-bit signed integer elements less than comparisons simultaneously.

Description :

This instruction compares the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2 to see if the one in Rs1 is less than the one in Rs2. If it is true, the result is 0xFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

Rd.B[x] = (Rs1.B[x] < Rs2.B[x])? 0xff : 0x0;
for RV32: x=3...0,
for RV64: x=7...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UCMPLE8 (unsigned long a, unsigned long b)

UCMPLE8 (SIMD 8-bit Unsigned Compare Less Than & Equal)

Type: SIMD

Syntax:

UCMPLE8 Rd, Rs1, Rs2

Purpose :

Do 8-bit unsigned integer elements less than & equal comparisons simultaneously.

Description :

This instruction compares the 8-bit unsigned integer elements in Rs1 with the 8-bit unsigned integer elements in Rs2 to see if the one in Rs1 is less than or equal to the one in Rs2. If it is true, the result is 0xFF; otherwise, the result is 0x0. The four comparison results are written to Rd.

Operations:

Rd.B[x] = (Rs1.B[x] <=u Rs2.B[x])? 0xff : 0x0;
for RV32: x=3...0,
for RV64: x=7...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UCMPLT8 (unsigned long a, unsigned long b)
 UCMPLT8 (SIMD 8-bit Unsigned Compare Less Than)

Type: SIMD

Syntax:

```
UCMPLT8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit unsigned integer elements less than comparisons simultaneously.

Description :

This instruction compares the 8-bit unsigned integer elements in Rs1 with the 8-bit unsigned integer elements in Rs2 to see if the one in Rs1 is less than the one in Rs2. If it is true, the result is 0xFF; otherwise, the result is 0x0. The element comparison results are written to Rd.

Operations:

```
Rd.B[x] = (Rs1.B[x] <u Rs2.B[x])? 0xff : 0x0;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 16-bit Multiply Instructions

__STATIC_FORCEINLINE unsigned long __RV_KHM16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_KHMX16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long long __RV_SMUL16 (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE unsigned long long __RV_SMULX16 (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE unsigned long long __RV_UMUL16 (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE unsigned long long __RV_UMULX16 (unsigned int a, unsigned int b)

group **SIMD 16-bit Multiply Instructions**

SIMD 16-bit Multiply Instructions.

there are 6 SIMD 16-bit Multiply instructions.

Functions

__STATIC_FORCEINLINE unsigned long __RV_KHM16 (unsigned long a, unsigned long b)

KHM16 (SIMD Signed Saturating Q15 Multiply)

Type: SIMD

Syntax:

```
KHM16 Rd, Rs1, Rs2
KHM16 Rd, Rs1, Rs2
```

Purpose :

Do Q15xQ15 element multiplications simultaneously. The Q30 results are then reduced to Q15 numbers again.

Description :

For the KHM16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. For the KHM16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. The Q30 results are then right-shifted 15-bits and saturated into Q15 values. The Q15 results are then written into Rd. When both the two Q15 inputs of a multiplication are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
if (is `KHM16`) {
    op1t = Rs1.H[x+1]; op2t = Rs2.H[x+1]; // top
    op1b = Rs1.H[x]; op2b = Rs2.H[x]; // bottom
} else if (is `KHM16`) {
    op1t = Rs1.H[x+1]; op2t = Rs2.H[x]; // Rs1 top
    op1b = Rs1.H[x]; op2b = Rs2.H[x+1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x8000 != aop | 0x8000 != bop) {
        res = (aop s* bop) >> 15;
    } else {
        res= 0x7FFF;
        OV = 1;
    }
}
Rd.W[x/2] = concat(rest, resb);
for RV32: x=0
for RV64: x=0,2
```


Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KHM16 (unsigned long a, unsigned long b)

KHM16 (SIMD Signed Saturating Crossed Q15 Multiply)

Type: SIMD

Syntax:

```
KHM16 Rd, Rs1, Rs2
KHM16 Rd, Rs1, Rs2
```

Purpose :

Do Q15xQ15 element multiplications simultaneously. The Q30 results are then reduced to Q15 numbers again.

Description :

For the KHM16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. For the KHM16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. The Q30 results are then right-shifted 15-bits and saturated into Q15 values. The Q15 results are then written into Rd. When both the two Q15 inputs of a multiplication are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
if (is `KHM16`) {
    op1t = Rs1.H[x+1]; op2t = Rs2.H[x+1]; // top
    op1b = Rs1.H[x]; op2b = Rs2.H[x]; // bottom
} else if (is `KHM16`) {
    op1t = Rs1.H[x+1]; op2t = Rs2.H[x]; // Rs1 top
    op1b = Rs1.H[x]; op2b = Rs2.H[x+1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x8000 != aop | 0x8000 != bop) {
        res = (aop s* bop) >> 15;
    } else {
        res= 0x7FFF;
        OV = 1;
    }
}
Rd.W[x/2] = concat(rest, resb);
for RV32: x=0
for RV64: x=0,2
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long long __RV_SMUL16 (unsigned int a, unsigned int b)
 SMUL16 (SIMD Signed 16-bit Multiply)

Type: SIMD

Syntax:

```
SMUL16 Rd, Rs1, Rs2
SMULX16 Rd, Rs1, Rs2
```

Purpose :

Do signed 16-bit multiplications and generate two 32-bit results simultaneously.

RV32 Description :

For the SMUL16 instruction, multiply the top 16-bit Q15 content of Rs1 with the top 16-bit Q15 content of Rs2. At the same time, multiply the bottom 16-bit Q15 content of Rs1 with the bottom 16-bit Q15 content of Rs2. For the SMULX16 instruction, multiply the top 16-bit Q15 content of Rs1 with the bottom 16-bit Q15 content of Rs2. At the same time, multiply the bottom 16-bit Q15 content of Rs1 with the top 16-bit Q15 content of Rs2. The two Q30 results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the 32-bit result calculated from the top part of Rs1 and the even 2d register of the pair contains the 32-bit result calculated from the bottom part of Rs1.

RV64 Description :

For the SMUL16 instruction, multiply the top 16-bit Q15 content of the lower 32-bit word in Rs1 with the top 16-bit Q15 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit Q15 content of the lower 32-bit word in Rs1 with the bottom 16-bit Q15 content of the lower 32-bit word in Rs2. For the SMULX16 instruction, multiply the top 16-bit Q15 content of the lower 32-bit word in Rs1 with the bottom 16-bit Q15 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit Q15 content of the lower 32-bit word in Rs1 with the top 16-bit Q15 content of the lower 32-bit word in Rs2. The two 32-bit Q30 results are then written into Rd. The result calculated from the top 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[1]. And the result calculated from the bottom 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[0]

Operations:

```
* RV32:
if (is `SMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `SMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop s* bop;
}
```

(continues on next page)

(continued from previous page)

```

t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H] = rest;
R[t_L] = resb;
* RV64:
if (is `SMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `SMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop s* bop;
}
Rd.W[1] = rest;
Rd.W[0] = resb;

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_SMULX16 (unsigned int a, unsigned int b)

SMULX16 (SIMD Signed Crossed 16-bit Multiply)

Type: SIMD

Syntax:

```

SMUL16 Rd, Rs1, Rs2
SMULX16 Rd, Rs1, Rs2

```

Purpose :

Do signed 16-bit multiplications and generate two 32-bit results simultaneously.

RV32 Description :

For the SMUL16 instruction, multiply the top 16-bit Q15 content of Rs1 with the top 16-bit Q15 content of Rs2. At the same time, multiply the bottom 16-bit Q15 content of Rs1 with the bottom 16-bit Q15 content of Rs2. For the SMULX16 instruction, multiply the top 16-bit Q15 content of Rs1 with the bottom 16-bit Q15 content of Rs2. At the same time, multiply the bottom 16-bit Q15 content of Rs1 with the top 16-bit Q15 content of Rs2. The two Q30 results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the 32-bit result calculated from the top part of Rs1 and the even 2d register of the pair contains the 32-bit result calculated from the bottom part of Rs1.

RV64 Description :

For the SMUL16 instruction, multiply the top 16-bit Q15 content of the lower 32-bit word in Rs1 with the top 16-bit Q15 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit Q15

content of the lower 32-bit word in Rs1 with the bottom 16-bit Q15 content of the lower 32-bit word in Rs2. For the SMULX16 instruction, multiply the top 16-bit Q15 content of the lower 32-bit word in Rs1 with the bottom 16-bit Q15 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit Q15 content of the lower 32-bit word in Rs1 with the top 16-bit Q15 content of the lower 32-bit word in Rs2. The two 32-bit Q30 results are then written into Rd. The result calculated from the top 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[1]. And the result calculated from the bottom 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[0]

Operations:

```
* RV32:
if (is `SMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `SMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop s* bop;
}
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H] = rest;
R[t_L] = resb;
* RV64:
if (is `SMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `SMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop s* bop;
}
Rd.W[1] = rest;
Rd.W[0] = resb;
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMUL16 (unsigned int a, unsigned int b)
UMUL16 (SIMD Unsigned 16-bit Multiply)

Type: SIMD

Syntax:

```
UMUL16 Rd, Rs1, Rs2
UMULX16 Rd, Rs1, Rs2
```

Purpose :

Do unsigned 16-bit multiplications and generate two 32-bit results simultaneously.

RV32 Description :

For the UMUL16 instruction, multiply the top 16-bit U16 content of Rs1 with the top 16-bit U16 content of Rs2. At the same time, multiply the bottom 16-bit U16 content of Rs1 with the bottom 16-bit U16 content of Rs2. For the UMULX16 instruction, multiply the top 16-bit U16 content of Rs1 with the bottom 16-bit U16 content of Rs2. At the same time, multiply the bottom 16-bit U16 content of Rs1 with the top 16-bit U16 content of Rs2. The two U32 results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the 32-bit result calculated from the top part of Rs1 and the even 2d register of the pair contains the 32-bit result calculated from the bottom part of Rs1.

RV64 Description :

For the UMUL16 instruction, multiply the top 16-bit U16 content of the lower 32-bit word in Rs1 with the top 16-bit U16 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit U16 content of the lower 32-bit word in Rs1 with the bottom 16-bit U16 content of the lower 32-bit word in Rs2. For the UMULX16 instruction, multiply the top 16-bit U16 content of the lower 32-bit word in Rs1 with the bottom 16-bit U16 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit U16 content of the lower 32-bit word in Rs1 with the top 16-bit U16 content of the lower 32-bit word in Rs2. The two 32-bit U32 results are then written into Rd. The result calculated from the top 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[1]. And the result calculated from the bottom 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[0]

Operations:

```
* RV32:
if (is `UMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `UMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop u* bop;
}
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H] = rest;
R[t_L] = resb;
* RV64:
if (is `UMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `UMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop u* bop;
}
Rd.W[1] = rest;
Rd.W[0] = resb;
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMULX16 (unsigned int a, unsigned int b)

UMULX16 (SIMD Unsigned Crossed 16-bit Multiply)

Type: SIMD

Syntax:

```
UMUL16 Rd, Rs1, Rs2
UMULX16 Rd, Rs1, Rs2
```

Purpose :

Do unsigned 16-bit multiplications and generate two 32-bit results simultaneously.

RV32 Description :

For the UMUL16 instruction, multiply the top 16-bit U16 content of Rs1 with the top 16-bit U16 content of Rs2. At the same time, multiply the bottom 16-bit U16 content of Rs1 with the bottom 16-bit U16 content of Rs2. For the UMULX16 instruction, multiply the top 16-bit U16 content of Rs1 with the bottom 16-bit U16 content of Rs2. At the same time, multiply the bottom 16-bit U16 content of Rs1 with the top 16-bit U16 content of Rs2. The two U32 results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the 32-bit result calculated from the top part of Rs1 and the even 2d register of the pair contains the 32-bit result calculated from the bottom part of Rs1.

RV64 Description :

For the UMUL16 instruction, multiply the top 16-bit U16 content of the lower 32-bit word in Rs1 with the top 16-bit U16 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit U16 content of the lower 32-bit word in Rs1 with the bottom 16-bit U16 content of the lower 32-bit word in Rs2. For the UMULX16 instruction, multiply the top 16-bit U16 content of the lower 32-bit word in Rs1 with the bottom 16-bit U16 content of the lower 32-bit word in Rs2. At the same time, multiply the bottom 16-bit U16 content of the lower 32-bit word in Rs1 with the top 16-bit U16 content of the lower 32-bit word in Rs2. The two 32-bit U32 results are then written into Rd. The result calculated from the top 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[1]. And the result calculated from the bottom 16-bit of the lower 32-bit word in Rs1 is written to Rd.W[0]

Operations:

```
* RV32:
if (is `UMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `UMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
```

(continues on next page)

(continued from previous page)

```

    res = aop u* bop;
}
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H] = rest;
R[t_L] = resb;
* RV64:
if (is `UMUL16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[1]; // top
    op1b = Rs1.H[0]; op2b = Rs2.H[0]; // bottom
} else if (is `UMULX16`) {
    op1t = Rs1.H[1]; op2t = Rs2.H[0]; // Rs1 top
    op1b = Rs1.H[0]; op2b = Rs2.H[1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = aop u* bop;
}
Rd.W[1] = rest;
Rd.W[0] = resb;

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

SIMD 8-bit Multiply Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KHM8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KHMX8 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_SMUL8 (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_SMULX8 (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UMUL8 (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UMULX8 (unsigned int a, unsigned int b)
```

group SIMD 8-bit Multiply Instructions

SIMD 8-bit Multiply Instructions.

there are 6 SIMD 8-bit Multiply instructions.

Functions

__STATIC_FORCEINLINE unsigned long __RV_KHM8 (unsigned long a, unsigned long b)

KHM8 (SIMD Signed Saturating Q7 Multiply)

Type: SIMD

Syntax:

```
KHM8 Rd, Rs1, Rs2
KHMx8 Rd, Rs1, Rs2
```

Purpose :

Do Q7xQ7 element multiplications simultaneously. The Q14 results are then reduced to Q7 numbers again.

Description :

For the KHM8 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2. For the KHMx8 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2. The Q14 results are then right-shifted 7-bits and saturated into Q7 values. The Q7 results are then written into Rd. When both the two Q7 inputs of a multiplication are 0x80, saturation will happen. The result will be saturated to 0x7F and the overflow flag OV will be set.

Operations:

```
if (is `KHM8`) {
    op1t = Rs1.B[x+1]; op2t = Rs2.B[x+1]; // top
    op1b = Rs1.B[x]; op2b = Rs2.B[x]; // bottom
} else if (is `KHMx8`) {
    op1t = Rs1.B[x+1]; op2t = Rs2.B[x]; // Rs1 top
    op1b = Rs1.B[x]; op2b = Rs2.B[x+1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x80 != aop | 0x80 != bop) {
        res = (aop s* bop) >> 7;
    } else {
        res= 0x7F;
        OV = 1;
    }
}
Rd.H[x/2] = concat(rest, resb);
for RV32, x=0,2
for RV64, x=0,2,4,6
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KHM8 (unsigned long a, unsigned long b)

KHM8 (SIMD Signed Saturating Crossed Q7 Multiply)

Type: SIMD

Syntax:

```
KHM8 Rd, Rs1, Rs2
KHM8 Rd, Rs1, Rs2
```

Purpose :

Do Q7xQ7 element multiplications simultaneously. The Q14 results are then reduced to Q7 numbers again.

Description :

For the KHM8 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2. For the KHM16 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2. The Q14 results are then right-shifted 7-bits and saturated into Q7 values. The Q7 results are then written into Rd. When both the two Q7 inputs of a multiplication are 0x80, saturation will happen. The result will be saturated to 0x7F and the overflow flag OV will be set.

Operations:

```
if (is `KHM8`) {
    op1t = Rs1.B[x+1]; op2t = Rs2.B[x+1]; // top
    op1b = Rs1.B[x]; op2b = Rs2.B[x]; // bottom
} else if (is `KHM16`) {
    op1t = Rs1.B[x+1]; op2t = Rs2.B[x]; // Rs1 top
    op1b = Rs1.B[x]; op2b = Rs2.B[x+1]; // Rs1 bottom
}
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x80 != aop | 0x80 != bop) {
        res = (aop s* bop) >> 7;
    } else {
        res= 0x7F;
        OV = 1;
    }
}
Rd.H[x/2] = concat(rest, resb);
for RV32, x=0,2
for RV64, x=0,2,4,6
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long long __RV_SMUL8 (unsigned int a, unsigned int b)

SMUL8 (SIMD Signed 8-bit Multiply)

Type: SIMD

Syntax:

```
SMUL8 Rd, Rs1, Rs2
SMULX8 Rd, Rs1, Rs2
```

Purpose :

Do signed 8-bit multiplications and generate four 16-bit results simultaneously.

RV32 Description :

For the SMUL8 instruction, multiply the 8-bit data elements of Rs1 with the corresponding 8-bit data elements of Rs2. For the SMULX8 instruction, multiply the first and second 8-bit data elements of Rs1 with the second and first 8-bit data elements of Rs2. At the same time, multiply the third and fourth 8-bit data elements of Rs1 with the fourth and third 8-bit data elements of Rs2. The four 16-bit results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the two 16-bit results calculated from the top part of Rs1 and the even 2d register of the pair contains the two 16-bit results calculated from the bottom part of Rs1.

RV64 Description :

For the SMUL8 instruction, multiply the 8-bit data elements of Rs1 with the corresponding 8-bit data elements of Rs2. For the SMULX8 instruction, multiply the first and second 8-bit data elements of Rs1 with the second and first 8-bit data elements of Rs2. At the same time, multiply the third and fourth 8-bit data elements of Rs1 with the fourth and third 8-bit data elements of Rs2. The four 16-bit results are then written into Rd. The Rd.W[1] contains the two 16-bit results calculated from the top part of Rs1 and the Rd.W[0] contains the two 16-bit results calculated from the bottom part of Rs1.

Operations:

```
* RV32:
if (is `SMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `SMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] s* op2t[x/2];
resb[x/2] = op1b[x/2] s* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].H[1] = rest[1]; R[t_H].H[0] = resb[1];
R[t_L].H[1] = rest[0]; R[t_L].H[0] = resb[0];
x = 0 and 2
* RV64:
if (is `SMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `SMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] s* op2t[x/2];
resb[x/2] = op1b[x/2] s* op2b[x/2];
```

(continues on next page)

(continued from previous page)

```

t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
Rd.W[1].H[1] = rest[1]; Rd.W[1].H[0] = resb[1];
Rd.W[0].H[1] = rest[0]; Rd.W[0].H[0] = resb[0];
x = 0 and 2

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_SMULX8 (unsigned int a, unsigned int b)
 SMULX8 (SIMD Signed Crossed 8-bit Multiply)

Type: SIMD

Syntax:

```

SMUL8 Rd, Rs1, Rs2
SMULX8 Rd, Rs1, Rs2

```

Purpose :

Do signed 8-bit multiplications and generate four 16-bit results simultaneously.

RV32 Description :

For the SMUL8 instruction, multiply the 8-bit data elements of Rs1 with the corresponding 8-bit data elements of Rs2. For the SMULX8 instruction, multiply the first and second 8-bit data elements of Rs1 with the second and first 8-bit data elements of Rs2. At the same time, multiply the third and fourth 8-bit data elements of Rs1 with the fourth and third 8-bit data elements of Rs2. The four 16-bit results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the two 16-bit results calculated from the top part of Rs1 and the even 2d register of the pair contains the two 16-bit results calculated from the bottom part of Rs1.

RV64 Description :

For the SMUL8 instruction, multiply the 8-bit data elements of Rs1 with the corresponding 8-bit data elements of Rs2. For the SMULX8 instruction, multiply the first and second 8-bit data elements of Rs1 with the second and first 8-bit data elements of Rs2. At the same time, multiply the third and fourth 8-bit data elements of Rs1 with the fourth and third 8-bit data elements of Rs2. The four 16-bit results are then written into Rd. The Rd.W[1] contains the two 16-bit results calculated from the top part of Rs1 and the Rd.W[0] contains the two 16-bit results calculated from the bottom part of Rs1.

Operations:

```

* RV32:
if (is `SMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `SMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom

```

(continues on next page)

(continued from previous page)

```

}
rest[x/2] = op1t[x/2] s* op2t[x/2];
resb[x/2] = op1b[x/2] s* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].H[1] = rest[1]; R[t_H].H[0] = resb[1];
R[t_L].H[1] = rest[0]; R[t_L].H[0] = resb[0];
x = 0 and 2
* RV64:
if (is `SMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `SMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] s* op2t[x/2];
resb[x/2] = op1b[x/2] s* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
Rd.W[1].H[1] = rest[1]; Rd.W[1].H[0] = resb[1];
Rd.W[0].H[1] = rest[0]; Rd.W[0].H[0] = resb[0];
x = 0 and 2

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMUL8 (unsigned int a, unsigned int b)

UMUL8 (SIMD Unsigned 8-bit Multiply)

Type: SIMD

Syntax:

```

UMUL8 Rd, Rs1, Rs2
UMULX8 Rd, Rs1, Rs2

```

Purpose :

Do unsigned 8-bit multiplications and generate four 16-bit results simultaneously.

RV32 Description :

For the UMUL8 instruction, multiply the unsigned 8-bit data elements of Rs1 with the corresponding unsigned 8-bit data elements of Rs2. For the UMULX8 instruction, multiply the first and second unsigned 8-bit data elements of Rs1 with the second and first unsigned 8-bit data elements of Rs2. At the same time, multiply the third and fourth unsigned 8-bit data elements of Rs1 with the fourth and third unsigned 8-bit data elements of Rs2. The four 16-bit results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the two 16-bit results calculated from the top part of Rs1 and the even 2d register of the pair contains the two 16-bit results calculated from the bottom part of Rs1.

RV64 Description :

For the UMUL8 instruction, multiply the unsigned 8-bit data elements of Rs1 with the corresponding unsigned 8-bit data elements of Rs2. For the UMULX8 instruction, multiply the first and second unsigned 8-bit data elements of Rs1 with the second and first unsigned 8-bit data elements of Rs2. At the same time, multiply the third and fourth unsigned 8-bit data elements of Rs1 with the fourth and third unsigned 8-bit data elements of Rs2. The four 16-bit results are then written into Rd. The Rd.W[1] contains the two 16-bit results calculated from the top part of Rs1 and the Rd.W[0] contains the two 16-bit results calculated from the bottom part of Rs1.

Operations:

```
* RV32:
if (is `UMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `UMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] u* op2t[x/2];
resb[x/2] = op1b[x/2] u* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].H[1] = rest[1]; R[t_H].H[0] = resb[1];
R[t_L].H[1] = rest[0]; R[t_L].H[0] = resb[0];
x = 0 and 2

* RV64:
if (is `UMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `UMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] u* op2t[x/2];
resb[x/2] = op1b[x/2] u* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
Rd.W[1].H[1] = rest[1]; Rd.W[1].H[0] = resb[1];
Rd.W[0].H[1] = rest[0]; Rd.W[0].H[0] = resb[0]; x = 0 and 2
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMULX8 (unsigned int a, unsigned int b)
 UMULX8 (SIMD Unsigned Crossed 8-bit Multiply)

Type: SIMD

Syntax:

UMUL8 Rd, Rs1, Rs2
 UMULX8 Rd, Rs1, Rs2

Purpose :

Do unsigned 8-bit multiplications and generate four 16-bit results simultaneously.

RV32 Description :

For the UMUL8 instruction, multiply the unsigned 8-bit data elements of Rs1 with the corresponding unsigned 8-bit data elements of Rs2. For the UMULX8 instruction, multiply the first and second unsigned 8-bit data elements of Rs1 with the second and first unsigned 8-bit data elements of Rs2. At the same time, multiply the third and fourth unsigned 8-bit data elements of Rs1 with the fourth and third unsigned 8-bit data elements of Rs2. The four 16-bit results are then written into an even/odd pair of registers specified by Rd(4,1). Rd(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the two 16-bit results calculated from the top part of Rs1 and the even 2d register of the pair contains the two 16-bit results calculated from the bottom part of Rs1.

RV64 Description :

For the UMUL8 instruction, multiply the unsigned 8-bit data elements of Rs1 with the corresponding unsigned 8-bit data elements of Rs2. For the UMULX8 instruction, multiply the first and second unsigned 8-bit data elements of Rs1 with the second and first unsigned 8-bit data elements of Rs2. At the same time, multiply the third and fourth unsigned 8-bit data elements of Rs1 with the fourth and third unsigned 8-bit data elements of Rs2. The four 16-bit results are then written into Rd. The Rd.W[1] contains the two 16-bit results calculated from the top part of Rs1 and the Rd.W[0] contains the two 16-bit results calculated from the bottom part of Rs1.

Operations:

```
* RV32:
if (is `UMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `UMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] u* op2t[x/2];
resb[x/2] = op1b[x/2] u* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].H[1] = rest[1]; R[t_H].H[0] = resb[1];
R[t_L].H[1] = rest[0]; R[t_L].H[0] = resb[0];
x = 0 and 2
* RV64:
if (is `UMUL8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x+1]; // top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x]; // bottom
} else if (is `UMULX8`) {
    op1t[x/2] = Rs1.B[x+1]; op2t[x/2] = Rs2.B[x]; // Rs1 top
    op1b[x/2] = Rs1.B[x]; op2b[x/2] = Rs2.B[x+1]; // Rs1 bottom
}
rest[x/2] = op1t[x/2] u* op2t[x/2];
resb[x/2] = op1b[x/2] u* op2b[x/2];
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
```

(continues on next page)

(continued from previous page)

```
Rd.W[1].H[1] = rest[1]; Rd.W[1].H[0] = resb[1];
Rd.W[0].H[1] = rest[0]; Rd.W[0].H[0] = resb[0]; x = 0 and 2
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long long type

SIMD 16-bit Miscellaneous Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_CLRS16 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CLO16 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CLZ16 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KABS16 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SMAX16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SMIN16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UMAX16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UMIN16 (unsigned long a, unsigned long b)
```

```
__RV_SCLIP16(a, b)
```

```
__RV_UCLIP16(a, b)
```

group **SIMD 16-bit Miscellaneous Instructions**

SIMD 16-bit Miscellaneous Instructions.

there are 10 SIMD 16-bit Misc instructions.

Defines

__RV_SCLIP16(a, b)

SCLIP16 (SIMD 16-bit Signed Clip Value)

Type: SIMD

Syntax:

```
SCLIP16 Rd, Rs1, imm4u[3:0]
```

Purpose :

Limit the 16-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 16-bit signed integer elements stored in Rs1 into a signed integer range between $2^{\text{imm4u}}-1$ and -2^{imm4u} , and writes the limited results to Rd. For example, if imm4u is 3, the 16-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.H[x];
if (src > (2^imm4u)-1) {
    src = (2^imm4u)-1;
    OV = 1;
} else if (src < -2^imm4u) {
    src = -2^imm4u;
    OV = 1;
}
Rd.H[x] = src
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_UCLIP16(a, b)

UCLIP16 (SIMD 16-bit Unsigned Clip Value)

Type: SIMD

Syntax:

```
UCLIP16 Rt, Ra, imm4u
```

Purpose :

Limit the 16-bit signed elements of a register into an unsigned range simultaneously.

Description :

This instruction limits the 16-bit signed elements stored in Rs1 into an unsigned integer range between $2^{\text{imm4u}}-1$ and 0, and writes the limited results to Rd. For example, if imm4u is 3, the 16-bit input values should be saturated between 7 and 0. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.H[x];
if (src > (2^imm4u)-1) {
    src = (2^imm4u)-1;
    OV = 1;
} else if (src < 0) {
    src = 0;
    OV = 1;
}
Rd.H[x] = src;
for RV32: x=1..0,
for RV64: x=3..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_CLRS16 (unsigned long a)

CLRS16 (SIMD 16-bit Count Leading Redundant Sign)

Type: SIMD

Syntax:

CLRS16 Rd, Rs1

Purpose :

Count the number of redundant sign bits of the 16-bit elements of a general register.

Description :

Starting from the bits next to the sign bits of the 16-bit elements of Rs1, this instruction counts the number of redundant sign bits and writes the result to the corresponding 16-bit elements of Rd.

Operations:

```
snum[x] = Rs1.H[x];
cnt[x] = 0;
for (i = 14 to 0) {
    if (snum[x](i) == snum[x](15)) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
```

(continues on next page)

(continued from previous page)

```

}
Rd.H[x] = cnt[x];
for RV32: x=1...0
for RV64: x=3...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLO16 (unsigned long a)

CLO16 (SIMD 16-bit Count Leading One)

Type: SIMD

Syntax:

CLO16 Rd, Rs1

Purpose :

Count the number of leading one bits of the 16-bit elements of a general register.

Description :

Starting from the most significant bits of the 16-bit elements of Rs1, this instruction counts the number of leading one bits and writes the results to the corresponding 16-bit elements of Rd.

Operations:

```

snum[x] = Rs1.H[x];
cnt[x] = 0;
for (i = 15 to 0) {
    if (snum[x](i) == 1) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.H[x] = cnt[x];
for RV32: x=1...0
for RV64: x=3...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLZ16 (unsigned long a)

CLZ16 (SIMD 16-bit Count Leading Zero)

Type: SIMD

Syntax:

CLZ16 Rd, Rs1

Purpose :

Count the number of leading zero bits of the 16-bit elements of a general register.

Description :

Starting from the most significant bits of the 16-bit elements of Rs1, this instruction counts the number of leading zero bits and writes the results to the corresponding 16-bit elements of Rd.

Operations:

```
snum[x] = Rs1.H[x];
cnt[x] = 0;
for (i = 15 to 0) {
    if (snum[x](i) == 0) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.H[x] = cnt[x];
for RV32: x=1...0
for RV64: x=3...0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KABS16 (unsigned long a)

KABS16 (SIMD 16-bit Saturating Absolute)

Type: SIMD

Syntax:

KABS16 Rd, Rs1

Purpose :

Get the absolute value of 16-bit signed integer elements simultaneously.

Description :

This instruction calculates the absolute value of 16-bit signed integer elements stored in Rs1 and writes the element results to Rd. If the input number is 0x8000, this instruction generates 0x7fff as the output and sets the OV bit to 1.

Operations:

```
src = Rs1.H[x];
if (src == 0x8000) {
    src = 0x7fff;
    OV = 1;
} else if (src[15] == 1)
```

(continues on next page)

(continued from previous page)

```

    src = -src;
}
Rd.H[x] = src;
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMAX16 (unsigned long a, unsigned long b)

SMAX16 (SIMD 16-bit Signed Maximum)

Type: SIMD

Syntax:

```
SMAX16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2 and selects the numbers that is greater than the other one. The selected results are written to Rd.

Operations:

```

Rd.H[x] = (Rs1.H[x] > Rs2.H[x])? Rs1.H[x] : Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMIN16 (unsigned long a, unsigned long b)

SMIN16 (SIMD 16-bit Signed Minimum)

Type: SIMD

Syntax:

```
SMIN16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit signed integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] < Rs2.H[x])? Rs1.H[x] : Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UMAX16 (unsigned long a, unsigned long b)

UMAX16 (SIMD 16-bit Unsigned Maximum)

Type: SIMD

Syntax:

```
UMAX16 Rd, Rs1, Rs2
```

Purpose :

Do 16-bit unsigned integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2 and selects the numbers that is greater than the other one. The selected results are written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] >u Rs2.H[x])? Rs1.H[x] : Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UMIN16 (unsigned long a, unsigned long b)

UMIN16 (SIMD 16-bit Unsigned Minimum)

Type: SIMD

Syntax:

UMIN16 Rd, Rs1, Rs2

Purpose :

Do 16-bit unsigned integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 16-bit unsigned integer elements in Rs1 with the 16-bit unsigned integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

Rd.H[x] = (Rs1.H[x] <u Rs2.H[x])? Rs1.H[x] : Rs2.H[x];
for RV32: x=1...0,
for RV64: x=3...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 8-bit Miscellaneous Instructions

`__STATIC_FORCEINLINE unsigned long __RV_CLRS8 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_CLO8 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_CLZ8 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_KABS8 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_SMAX8 (unsigned long a, unsigned long b)`

`__STATIC_FORCEINLINE unsigned long __RV_SMIN8 (unsigned long a, unsigned long b)`

`__STATIC_FORCEINLINE unsigned long __RV_UMAX8 (unsigned long a, unsigned long b)`

`__STATIC_FORCEINLINE unsigned long __RV_UMIN8 (unsigned long a, unsigned long b)`

`__RV_SCLIP8(a, b)`

`__RV_UCLIP8(a, b)`

group **SIMD 8-bit Miscellaneous Instructions**

SIMD 8-bit Miscellaneous Instructions.

there are 10 SIMD 8-bit Miscellaneous instructions.

Defines

__RV_SCLIP8(a, b)

SCLIP8 (SIMD 8-bit Signed Clip Value)

Type: SIMD

Syntax:

```
SCLIP8 Rd, Rs1, imm3u[2:0]
```

Purpose :

Limit the 8-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 8-bit signed integer elements stored in Rs1 into a signed integer range between $2^{\text{imm3u}-1}$ and -2^{imm3u} , and writes the limited results to Rd. For example, if imm3u is 3, the 8-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.B[x];
if (src > (2^imm3u)-1) {
    src = (2^imm3u)-1;
    OV = 1;
} else if (src < -2^imm3u) {
    src = -2^imm3u;
    OV = 1;
}
Rd.B[x] = src
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_UCLIP8(a, b)

UCLIP8 (SIMD 8-bit Unsigned Clip Value)

Type: SIMD

Syntax:

```
UCLIP8 Rt, Ra, imm3u
```

Purpose :

Limit the 8-bit signed elements of a register into an unsigned range simultaneously.

Description :

This instruction limits the 8-bit signed elements stored in Rs1 into an unsigned integer range between $2^{\text{imm3u}}-1$ and 0, and writes the limited results to Rd. For example, if imm3u is 3, the 8-bit input values should be saturated between 7 and 0. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.H[x];
if (src > (2^imm3u)-1) {
    src = (2^imm3u)-1;
    OV = 1;
} else if (src < 0) {
    src = 0;
    OV = 1;
}
Rd.H[x] = src;
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_CLRS8 (unsigned long a)

CLRS8 (SIMD 8-bit Count Leading Redundant Sign)

Type: SIMD

Syntax:

CLRS8 Rd, Rs1

Purpose :

Count the number of redundant sign bits of the 8-bit elements of a general register.

Description :

Starting from the bits next to the sign bits of the 8-bit elements of Rs1, this instruction counts the number of redundant sign bits and writes the result to the corresponding 8-bit elements of Rd.

Operations:

```
snum[x] = Rs1.B[x];
cnt[x] = 0;
for (i = 6 to 0) {
```

(continues on next page)

(continued from previous page)

```

    if (snum[x](i) == snum[x](7)) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.B[x] = cnt[x];
for RV32: x=3...0
for RV64: x=7...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLO8 (unsigned long a)

CLO8 (SIMD 8-bit Count Leading One)

Type: SIMD

Syntax:

CLO8 Rd, Rs1

Purpose :

Count the number of leading one bits of the 8-bit elements of a general register.

Description :

Starting from the most significant bits of the 8-bit elements of Rs1, this instruction counts the number of leading one bits and writes the results to the corresponding 8-bit elements of Rd.

Operations:

```

snum[x] = Rs1.B[x];
cnt[x] = 0;
for (i = 7 to 0) {
    if (snum[x](i) == 1) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.B[x] = cnt[x];
for RV32: x=3...0
for RV64: x=7...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLZ8 (unsigned long a)

CLZ8 (SIMD 8-bit Count Leading Zero)

Type: SIMD**Syntax:**

CLZ8 Rd, Rs1

Purpose :

Count the number of leading zero bits of the 8-bit elements of a general register.

Description :

Starting from the most significant bits of the 8-bit elements of Rs1, this instruction counts the number of leading zero bits and writes the results to the corresponding 8-bit elements of Rd.

Operations:

```
snum[x] = Rs1.B[x];
cnt[x] = 0;
for (i = 7 to 0) {
    if (snum[x](i) == 0) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.B[x] = cnt[x];
for RV32: x=3...0
for RV64: x=7...0
```

Parameters**a** – [in] unsigned long type of value stored in a**Returns**

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KABS8 (unsigned long a)

KABS8 (SIMD 8-bit Saturating Absolute)

Type: SIMD**Syntax:**

KABS8 Rd, Rs1

Purpose :

Get the absolute value of 8-bit signed integer elements simultaneously.

Description :

This instruction calculates the absolute value of 8-bit signed integer elements stored in Rs1 and writes the element results to Rd. If the input number is 0x80, this instruction generates 0x7f as the output and sets the OV bit to 1.

Operations:

```

src = Rs1.B[x];
if (src == 0x80) {
    src = 0x7f;
    OV = 1;
} else if (src[7] == 1)
    src = -src;
}
Rd.B[x] = src;
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMAX8 (unsigned long a, unsigned long b)

SMAX8 (SIMD 8-bit Signed Maximum)

Type: SIMD

Syntax:

```
SMAX8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2 and selects the numbers that is greater than the other one. The selected results are written to Rd.

Operations:

```

Rd.B[x] = (Rs1.B[x] > Rs2.B[x])? Rs1.B[x] : Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMIN8 (unsigned long a, unsigned long b)

SMIN8 (SIMD 8-bit Signed Minimum)

Type: SIMD

Syntax:

```
SMIN8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit signed integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

```
Rd.B[x] = (Rs1.B[x] < Rs2.B[x])? Rs1.B[x] : Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_UMAX8 (unsigned long a, unsigned long b)
```

UMAX8 (SIMD 8-bit Unsigned Maximum)

Type: SIMD

Syntax:

```
UMAX8 Rd, Rs1, Rs2
```

Purpose :

Do 8-bit unsigned integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 8-bit unsigned integer elements in Rs1 with the four 8-bit unsigned integer elements in Rs2 and selects the numbers that is greater than the other one. The two selected results are written to Rd.

Operations:

```
Rd.B[x] = (Rs1.B[x] >u Rs2.B[x])? Rs1.B[x] : Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UMIN8 (unsigned long a, unsigned long b)

UMIN8 (SIMD 8-bit Unsigned Minimum)

Type: SIMD

Syntax:

UMIN8 Rd, Rs1, Rs2

Purpose :

Do 8-bit unsigned integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 8-bit unsigned integer elements in Rs1 with the 8-bit unsigned integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

Rd.B[x] = (Rs1.B[x] <u Rs2.B[x])? Rs1.B[x] : Rs2.B[x];
for RV32: x=3...0,
for RV64: x=7...0

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

SIMD 8-bit Unpacking Instructions

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD810 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD820 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD830 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD831 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD832 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD810 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD820 (unsigned long a)

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD830 (unsigned long a)

```
__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD831 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD832 (unsigned long a)
```

group SIMD 8-bit Unpacking Instructions

SIMD 8-bit Unpacking Instructions.

there are 8 SIMD 8-bit Unpacking instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_SUNPKD810 (unsigned long a)
```

SUNPKD810 (Signed Unpacking Bytes 1 & 0)

Type: DSP

Syntax:

```
SUNPKD8xy Rd, Rs1  
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the SUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[x])  
Rd.W[m].H[0] = SE16(Rs1.W[m].B[y])  
// SUNPKD810, x=1,y=0  
// SUNPKD820, x=2,y=0  
// SUNPKD830, x=3,y=0  
// SUNPKD831, x=3,y=1  
// SUNPKD832, x=3,y=2  
for RV32: m=0,  
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_SUNPKD820 (unsigned long a)
```

SUNPKD820 (Signed Unpacking Bytes 2 & 0)

Type: DSP

Syntax:

```
SUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the SUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[y])
// SUNPKD810, x=1,y=0
// SUNPKD820, x=2,y=0
// SUNPKD830, x=3,y=0
// SUNPKD831, x=3,y=1
// SUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD830 (unsigned long a)

SUNPKD830 (Signed Unpacking Bytes 3 & 0)

Type: DSP

Syntax:

```
SUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the SUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[y])
// SUNPKD810, x=1,y=0
// SUNPKD820, x=2,y=0
// SUNPKD830, x=3,y=0
```

(continues on next page)

(continued from previous page)

```
// SUNPKD831, x=3,y=1
// SUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD831 (unsigned long a)

SUNPKD831 (Signed Unpacking Bytes 3 & 1)

Type: DSP

Syntax:

```
SUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the SUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[y])
// SUNPKD810, x=1,y=0
// SUNPKD820, x=2,y=0
// SUNPKD830, x=3,y=0
// SUNPKD831, x=3,y=1
// SUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUNPKD832 (unsigned long a)

SUNPKD832 (Signed Unpacking Bytes 3 & 2)

Type: DSP

Syntax:


```
SUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the SUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[y])
// SUNPKD810, x=1,y=0
// SUNPKD820, x=2,y=0
// SUNPKD830, x=3,y=0
// SUNPKD831, x=3,y=1
// SUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD810 (unsigned long a)
```

ZUNPKD810 (Unsigned Unpacking Bytes 1 & 0)

Type: DSP

Syntax:

```
ZUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the ZUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = ZE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = ZE16(Rs1.W[m].B[y])
// ZUNPKD810, x=1,y=0
// ZUNPKD820, x=2,y=0
// ZUNPKD830, x=3,y=0
```

(continues on next page)

(continued from previous page)

```
// ZUNPKD831, x=3,y=1
// ZUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD820 (unsigned long a)

ZUNPKD820 (Unsigned Unpacking Bytes 2 & 0)

Type: DSP

Syntax:

```
ZUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte x and byte y of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the ZUNPKD8(x)(*y*) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = ZE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = ZE16(Rs1.W[m].B[y])
// ZUNPKD810, x=1,y=0
// ZUNPKD820, x=2,y=0
// ZUNPKD830, x=3,y=0
// ZUNPKD831, x=3,y=1
// ZUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD830 (unsigned long a)

ZUNPKD830 (Unsigned Unpacking Bytes 3 & 0)

Type: DSP

Syntax:

```
ZUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the ZUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = ZE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = ZE16(Rs1.W[m].B[y])
// ZUNPKD810, x=1,y=0
// ZUNPKD820, x=2,y=0
// ZUNPKD830, x=3,y=0
// ZUNPKD831, x=3,y=1
// ZUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD831 (unsigned long a)
```

ZUNPKD831 (Unsigned Unpacking Bytes 3 & 1)

Type: DSP

Syntax:

```
ZUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte *x* and byte *y* of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the ZUNPKD8(*x*)(**y**) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = ZE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = ZE16(Rs1.W[m].B[y])
// ZUNPKD810, x=1,y=0
// ZUNPKD820, x=2,y=0
// ZUNPKD830, x=3,y=0
```

(continues on next page)

(continued from previous page)

```
// ZUNPKD831, x=3,y=1
// ZUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_ZUNPKD832 (unsigned long a)

ZUNPKD832 (Unsigned Unpacking Bytes 3 & 2)

Type: DSP

Syntax:

```
ZUNPKD8xy Rd, Rs1
xy = {10, 20, 30, 31, 32}
```

Purpose :

Unpack byte x and byte y of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the ZUNPKD8(x)(*y*) instruction, it unpacks byte *x* and byte *y* of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = ZE16(Rs1.W[m].B[x])
Rd.W[m].H[0] = ZE16(Rs1.W[m].B[y])
// ZUNPKD810, x=1,y=0
// ZUNPKD820, x=2,y=0
// ZUNPKD830, x=3,y=0
// ZUNPKD831, x=3,y=1
// ZUNPKD832, x=3,y=2
for RV32: m=0,
for RV64: m=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

group **SIMD Data Processing Instructions**

SIMD Data Processing Instructions.

Non-SIMD Instructions

Non-SIMD Q15 saturation ALU Instructions

```
__STATIC_FORCEINLINE long __RV_KADDH (int a, int b)
```

```
__STATIC_FORCEINLINE long __RV_KHMBB (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE long __RV_KHMBT (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE long __RV_KHMTT (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE long __RV_KSUBH (int a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UKADDH (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UKSUBH (unsigned int a, unsigned int b)
```

group Non-SIMD Q15 saturation ALU Instructions

Non-SIMD Q15 saturation ALU Instructions.

there are 7 Non-SIMD Q15 saturation ALU Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KADDH (int a, int b)
```

KADDH (Signed Addition with Q15 Saturation)

Type: DSP

Syntax:

```
KADDH Rd, Rs1, Rs2
```

Purpose :

Add the signed lower 32-bit content of two registers with Q15 saturation.

Description :

The signed lower 32-bit content of Rs1 is added with the signed lower 32-bit content of Rs2. And the result is saturated to the 16-bit signed integer range of $[-2^{15}, 2^{15}-1]$ and then sign- extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```
tmp = Rs1.W[0] + Rs2.W[0];
if (tmp > 32767) {
    res = 32767;
    OV = 1;
}
```

(continues on next page)

(continued from previous page)

```

} else if (tmp < -32768) {
    res = -32768;
    OV = 1;
} else {
    res = tmp;
}
Rd = SE(tmp[15:0]);

```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KHMBB (unsigned int a, unsigned int b)

KHMBB (Signed Saturating Half Multiply B16 x B16)

Type: DSP

Syntax:

```
KHMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)
```

Purpose :

Multiply the signed Q15 number contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then right-shift 15 bits to turn the Q30 result into a Q15 number again and saturate the Q15 result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then right- shifted 15-bits and saturated into a Q15 value. The Q15 value is then sing-extended and written into Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```

aop = Rs1.H[0]; bop = Rs2.H[0]; // KHMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KHMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KHMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd = SE32(res[15:0]); // Rv32
Rd = SE64(res[15:0]); // RV64

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KHMBT (unsigned int a, unsigned int b)

KHMBT (Signed Saturating Half Multiply B16 x T16)

Type: DSP

Syntax:

KHMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 number contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then right-shift 15 bits to turn the Q30 result into a Q15 number again and saturate the Q15 result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then right- shifted 15-bits and saturated into a Q15 value. The Q15 value is then sing-extended and written into Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
aop = Rs1.H[0]; bop = Rs2.H[0]; // KHMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KHMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KHMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd = SE32(res[15:0]); // Rv32
Rd = SE64(res[15:0]); // RV64
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KHMTT (unsigned int a, unsigned int b)

KHMTT (Signed Saturating Half Multiply T16 x T16)

Type: DSP

Syntax:

```
KHMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)
```

Purpose :

Multiply the signed Q15 number contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then right-shift 15 bits to turn the Q30 result into a Q15 number again and saturate the Q15 result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then right- shifted 15-bits and saturated into a Q15 value. The Q15 value is then sing-extended and written into Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
aop = Rs1.H[0]; bop = Rs2.H[0]; // KHMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KHMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KHMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd = SE32(res[15:0]); // Rv32
Rd = SE64(res[15:0]); // RV64
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KSUBH (int a, int b)
```

KSUBH (Signed Subtraction with Q15 Saturation)

Type: DSP

Syntax:

```
KSUBH Rd, Rs1, Rs2
```

Purpose :

Subtract the signed lower 32-bit content of two registers with Q15 saturation.

Description :

The signed lower 32-bit content of Rs2 is subtracted from the signed lower 32-bit content of Rs1. And the result is saturated to the 16-bit signed integer range of $[-2^{15}, 2^{15}-1]$ and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```

tmp = Rs1.W[0] - Rs2.W[0];
if (tmp > (2^15)-1) {
    res = (2^15)-1;
    OV = 1;
} else if (tmp < -2^15) {
    res = -2^15;
    OV = 1;
} else {
    res = tmp;
}
Rd = SE(res[15:0]);

```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_UKADDH (unsigned int a, unsigned int b)

UKADDH (Unsigned Addition with U16 Saturation)

Type: DSP

Syntax:

```
UKADDH Rd, Rs1, Rs2
```

Purpose :

Add the unsigned lower 32-bit content of two registers with U16 saturation.

Description :

The unsigned lower 32-bit content of Rs1 is added with the unsigned lower 32-bit content of Rs2. And the result is saturated to the 16-bit unsigned integer range of [0, 2¹⁶-1] and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```

tmp = Rs1.W[0] + Rs2.W[0];
if (tmp > (2^16)-1) {
    tmp = (2^16)-1;
    OV = 1;
}
Rd = SE(tmp[15:0]);

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSUBH (unsigned int a, unsigned int b)

UKSUBH (Unsigned Subtraction with U16 Saturation)

Type: DSP

Syntax:

UKSUBH Rd, Rs1, Rs2

Purpose :

Subtract the unsigned lower 32-bit content of two registers with U16 saturation.

Description :

The unsigned lower 32-bit content of Rs2 is subtracted from the unsigned lower 32-bit content of Rs1. And the result is saturated to the 16-bit unsigned integer range of $[0, 2^{16}-1]$ and then zero-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```
tmp = Rs1.W[0] - Rs2.W[0];
if (tmp > (2^16)-1) {
    tmp = (2^16)-1;
    OV = 1;
}
else if (tmp < 0) {
    tmp = 0;
    OV = 1;
}
Rd = ZE(tmp[15:0]);
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Non-SIMD Q31 saturation ALU Instructions

__STATIC_FORCEINLINE unsigned long __RV_KABSW (signed long a)

__STATIC_FORCEINLINE long __RV_KADDW (int a, int b)

__STATIC_FORCEINLINE long __RV_KDMBB (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KDMBT (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KDMTT (unsigned int a, unsigned int b)

```

__STATIC_FORCEINLINE long __RV_KDMABB (long t, unsigned int a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KDMABT (long t, unsigned int a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KDMATT (long t, unsigned int a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KSLW (long a, unsigned int b)

__STATIC_FORCEINLINE long __RV_KSLRAW (int a, int b)

__STATIC_FORCEINLINE long __RV_KSLRAW_U (int a, int b)

__STATIC_FORCEINLINE long __RV_KSUBW (int a, int b)

__STATIC_FORCEINLINE unsigned long __RV_UKADDW (unsigned int a, unsigned int b)

__STATIC_FORCEINLINE unsigned long __RV_UKSUBW (unsigned int a, unsigned int b)

__RV_KSLLIW(a, b)

```

group Non-SIMD Q31 saturation ALU Instructions

Non-SIMD Q31 saturation ALU Instructions.

there are Non-SIMD Q31 saturation ALU Instructions

Defines

__RV_KSLLIW(a, b)

KSLLIW (Saturating Shift Left Logical Immediate for Word)

Type: DSP

Syntax:

KSLLIW Rd, Rs1, imm5u

Purpose :

Do logical left shift operation with saturation on a 32-bit word. The shift amount is an immediate value.

Description :

The first word data in Rs1 is left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm5u constant. Any shifted value greater than $2^{31}-1$ is saturated to $2^{31}-1$. Any shifted value smaller than -2^{31} is saturated to -2^{31} . And the saturated result is sign-extended and written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```

sa = imm5u;
res[(31+sa):0] = Rs1.W[0] << sa;
if (res > (2^31)-1) {
    res = 0x7fffffff; OV = 1;
} else if (res < -2^31) {
    res = 0x80000000; OV = 1;
}
Rd[31:0] = res[31:0]; // RV32
Rd[63:0] = SE(res[31:0]); // RV64

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_KABSW (signed long a)

KABSW (Scalar 32-bit Absolute Value with Saturation)

Type: DSP

Syntax:

KABSW Rd, Rs1

Purpose :

Get the absolute value of a signed 32-bit integer in a general register.

Description :

This instruction calculates the absolute value of a signed 32-bit integer stored in Rs1. The result is sign-extended (for RV64) and written to Rd. This instruction with the minimum negative integer input of 0x80000000 will produce a saturated output of maximum positive integer of 0x7fffffff and the OV flag will be set to 1.

Operations:

```

if (Rs1.W[0] >= 0) {
    res = Rs1.W[0];
} else {
    If (Rs1.W[0] == 0x80000000) {
        res = 0x7fffffff;
        OV = 1;
    } else {
        res = -Rs1.W[0];
    }
}
Rd = SE32(res);

```

Parameters

a – [in] signed long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE long __RV_KADDW (int a, int b)

KADDW (Signed Addition with Q31 Saturation)

Type: DSP

Syntax:

KADDW Rd, Rs1, Rs2

Purpose :

Add the lower 32-bit signed content of two registers with Q31 saturation.

Description :

The lower 32-bit signed content of Rs1 is added with the lower 32-bit signed content of Rs2. And the result is saturated to the 32-bit signed integer range of $[-2^{31}, 2^{31}-1]$ and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```
tmp = Rs1.W[0] + Rs2.W[0];
if (tmp > (2^31)-1) {
    res = (2^31)-1;
    OV = 1;
} else if (tmp < -2^31) {
    res = -2^31;
    OV = 1;
} else {
    res = tmp;
}
Rd = res[31:0]; // RV32
Rd = SE(res[31:0]) // RV64
```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMBB (unsigned int a, unsigned int b)

KDMBB (Signed Saturating Double Multiply B16 x B16)

Type: DSP

Syntax:

KDMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result. The result is written into the destination register for RV32 or sign-extended to 64-bits and written into the destination register for RV64. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then written into Rd (sign-extended in RV64). When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```
aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;
    resQ31 = Mresult << 1;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
} else {
    resQ31 = 0x7FFFFFFF;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
    OV = 1;
}
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMBT (unsigned int a, unsigned int b)

KDMBT (Signed Saturating Double Multiply B16 x T16)

Type: DSP

Syntax:

KDMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result. The result is written into the destination register for RV32 or sign-extended to 64-bits and written into the destination register for RV64. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then written into Rd (sign-extended in RV64). When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```
aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;
    resQ31 = Mresult << 1;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
} else {
    resQ31 = 0x7FFFFFFF;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
    OV = 1;
}
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMTT (unsigned int a, unsigned int b)

KDMTT (Signed Saturating Double Multiply T16 x T16)

Type: DSP

Syntax:

KDMxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result. The result is written into the destination register for RV32 or sign-extended to 64-bits and written into the destination register for RV64. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then written into Rd (sign-extended in RV64). When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```

aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMBB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMBT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMTT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;
    resQ31 = Mresult << 1;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
} else {
    resQ31 = 0x7FFFFFFF;
    Rd = resQ31; // RV32
    Rd = SE(resQ31); // RV64
    OV = 1;
}

```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMABB (long t, unsigned int a, unsigned int b)

KDMABB (Signed Saturating Double Multiply Addition B16 x B16)

Type: DSP

Syntax:

KDMAxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result, add the result with the sign-extended lower 32-bit chunk destination register and write the saturated addition result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then added with the content of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV flag is set to 1. The result after saturation is written to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```

aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMABB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMABT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMATT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;

```

(continues on next page)

(continued from previous page)

```

    resQ31 = Mresult << 1;
} else {
    resQ31 = 0x7FFFFFFF;
    OV = 1;
}
resadd = Rd + resQ31; // RV32
resadd = Rd.W[0] + resQ31; // RV64
if (resadd > (2^31)-1) {
    resadd = (2^31)-1;
    OV = 1;
} else if (resadd < -2^31) {
    resadd = -2^31;
    OV = 1;
}
Rd = resadd; // RV32
Rd = SE(resadd); // RV64

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMABT (long t, unsigned int a, unsigned int b)

KDMABT (Signed Saturating Double Multiply Addition B16 x T16)

Type: DSP

Syntax:

KDMAxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result, add the result with the sign-extended lower 32-bit chunk destination register and write the saturated addition result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then added with the content of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV flag is set to 1. The result after saturation is written to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```

aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMABB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMABT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMATT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;
    resQ31 = Mresult << 1;
} else {
    resQ31 = 0x7FFFFFFF;
    OV = 1;
}
resadd = Rd + resQ31; // RV32
resadd = Rd.W[0] + resQ31; // RV64
if (resadd > (2^31)-1) {
    resadd = (2^31)-1;
    OV = 1;
} else if (resadd < -2^31) {
    resadd = -2^31;
    OV = 1;
}
Rd = resadd; // RV32
Rd = SE(resadd); // RV64

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KDMATT (long t, unsigned int a, unsigned int b)

KDMATT (Signed Saturating Double Multiply Addition T16 x T16)

Type: DSP

Syntax:

KDMAxy Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the lower 32-bit chunk in registers and then double and saturate the Q31 result, add the result with the sign-extended lower 32-bit chunk destination register and write the saturated addition result into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs1 with the top or bottom 16-bit Q15 content of the lower 32-bit portion in Rs2. The Q30 result is then doubled and saturated into a Q31 value. The Q31 value is then added with the content of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV flag is set to 1. The result after saturation is written to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```

aop = Rs1.H[0]; bop = Rs2.H[0]; // KDMABB
aop = Rs1.H[0]; bop = Rs2.H[1]; // KDMABT
aop = Rs1.H[1]; bop = Rs2.H[1]; // KDMATT
If (0x8000 != aop | 0x8000 != bop) {
    Mresult = aop * bop;
    resQ31 = Mresult << 1;
} else {
    resQ31 = 0x7FFFFFFF;
    OV = 1;
}
resadd = Rd + resQ31; // RV32
resadd = Rd.W[0] + resQ31; // RV64
if (resadd > (2^31)-1) {
    resadd = (2^31)-1;
    OV = 1;
} else if (resadd < -2^31) {
    resadd = -2^31;
    OV = 1;
}
Rd = resadd; // RV32
Rd = SE(resadd); // RV64

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KSLW (long a, unsigned int b)
```

KSLW (Saturating Shift Left Logical for Word)

Type: DSP

Syntax:

```
KSLW Rd, Rs1, Rs2
```

Purpose :

Do logical left shift operation with saturation on a 32-bit word. The shift amount is a variable from a GPR.

Description :

The first word data in Rs1 is left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the low-order 5-bits of the value in the Rs2 register. Any shifted value greater than $2^{31}-1$ is saturated to $2^{31}-1$. Any shifted value smaller than -2^{31} is saturated to -2^{31} . And the saturated result is sign-extended and written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```

sa = Rs2[4:0];
res[(31+sa):0] = Rs1.W[0] << sa;
if (res > (2^31)-1) {
    res = 0x7fffffff; OV = 1;
} else if (res < -2^31) {
    res = 0x80000000; OV = 1;
}
Rd[31:0] = res[31:0]; // RV32
Rd[63:0] = SE(res[31:0]); // RV64

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KSLRAW (int a, int b)

KSLRAW (Shift Left Logical with Q31 Saturation or Shift Right Arithmetic)

Type: DSP

Syntax:

KSLRAW Rd, Rs1, Rs2

Purpose :

Perform a logical left (positive) or arithmetic right (negative) shift operation with Q31 saturation for the left shift on a 32-bit data.

Description :

The lower 32-bit content of Rs1 is left-shifted logically or right-shifted arithmetically based on the value of Rs2[5:0]. Rs2[5:0] is in the signed range of [-25, 25-1]. A positive Rs2[5:0] means logical left shift and a negative Rs2[5:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[5:0] clamped to the actual shift range of [0, 31]. The left-shifted result is saturated to the 32-bit signed integer range of [-2^31, 2^31-1]. After the shift operation, the final result is bit-31 sign-extended and written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:6] will not affected the operation of this instruction.

Operations:

```

if (Rs2[5:0] < 0) {
    sa = -Rs2[5:0];
    sa = (sa == 32)? 31 : sa;
    res[31:0] = Rs1.W[0] >>(arith) sa;
} else {
    sa = Rs2[5:0];
    tmp = Rs1.W[0] <<(logic) sa;
    if (tmp > (2^31)-1) {
        res[31:0] = (2^31)-1;
        OV = 1;
    } else if (tmp < -2^31) {

```

(continues on next page)

(continued from previous page)

```

    res[31:0] = -2^31;
    OV = 1
  } else {
    res[31:0] = tmp[31:0];
  }
}
Rd = res[31:0]; // RV32
Rd = SE64(res[31:0]); // RV64

```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KSLRAW_U (int a, int b)

KSLRAW.u (Shift Left Logical with Q31 Saturation or Rounding Shift Right Arithmetic)

Type: DSP

Syntax:

```
KSLRAW.u Rd, Rs1, Rs2
```

Purpose :

Perform a logical left (positive) or arithmetic right (negative) shift operation with Q31 saturation for the left shift and a rounding up operation for the right shift on a 32-bit data.

Description :

The lower 32-bit content of Rs1 is left-shifted logically or right-shifted arithmetically based on the value of Rs2[5:0]. Rs2[5:0] is in the signed range of [-25, 25-1]. A positive Rs2[5:0] means logical left shift and a negative Rs2[5:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[5:0] clamped to the actual shift range of [0, 31]. The left-shifted result is saturated to the 32-bit signed integer range of [-2^31, 2^31-1]. The right-shifted result is added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final result is bit-31 sign-extended and written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:6] will not affect the operation of this instruction.

Operations:

```

if (Rs2[5:0] < 0) {
    sa = -Rs2[5:0];
    sa = (sa == 32)? 31 : sa;
    res[31:-1] = SE33(Rs1[31:(sa-1)]) + 1;
    rst[31:0] = res[31:0];
} else {
    sa = Rs2[5:0];
    tmp = Rs1.W[0] <<(logic) sa;
    if (tmp > (2^31)-1) {
        rst[31:0] = (2^31)-1;
        OV = 1;
    }
}

```

(continues on next page)

(continued from previous page)

```

    } else if (tmp < -2^31) {
        rst[31:0] = -2^31;
        OV = 1
    } else {
        rst[31:0] = tmp[31:0];
    }
}
Rd = rst[31:0]; // RV32
Rd = SE64(rst[31:0]); // RV64

```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KSUBW (int a, int b)

KSUBW (Signed Subtraction with Q31 Saturation)

Type: DSP

Syntax:

```
KSUBW Rd, Rs1, Rs2
```

Purpose :

Subtract the signed lower 32-bit content of two registers with Q31 saturation.

Description :

The signed lower 32-bit content of Rs2 is subtracted from the signed lower 32-bit content of Rs1. And the result is saturated to the 32-bit signed integer range of $[-2^{31}, 2^{31}-1]$ and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```

tmp = Rs1.W[0] - Rs2.W[0];
if (tmp > (2^31)-1) {
    res = (2^31)-1;
    OV = 1;
} else if (tmp < -2^31) {
    res = -2^31;
    OV = 1
} else {
    res = tmp;
}
Rd = res[31:0]; // RV32
Rd = SE(res[31:0]); // RV64

```

Parameters

- **a** – [in] int type of value stored in a

- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_UKADDW (unsigned int a, unsigned int b)

UKADDW (Unsigned Addition with U32 Saturation)

Type: DSP

Syntax:

```
UKADDW Rd, Rs1, Rs2
```

Purpose :

Add the unsigned lower 32-bit content of two registers with U32 saturation.

Description :

The unsigned lower 32-bit content of Rs1 is added with the unsigned lower 32-bit content of Rs2. And the result is saturated to the 32-bit unsigned integer range of $[0, 2^{32}-1]$ and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```
tmp = Rs1.W[0] + Rs2.W[0];
if (tmp > (2^32)-1) {
    tmp[31:0] = (2^32)-1;
    OV = 1;
}
Rd = tmp[31:0]; // RV32
Rd = SE(tmp[31:0]); // RV64
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSUBW (unsigned int a, unsigned int b)

UKSUBW (Unsigned Subtraction with U32 Saturation)

Type: DSP

Syntax:

```
UKSUBW Rd, Rs1, Rs2
```

Purpose :

Subtract the unsigned lower 32-bit content of two registers with unsigned 32-bit saturation.

Description :

The unsigned lower 32-bit content of Rs2 is subtracted from the unsigned lower 32-bit content of Rs1. And the result is saturated to the 32-bit unsigned integer range of $[0, 2^{32}-1]$ and then sign-extended and written to Rd. If saturation happens, this instruction sets the OV flag.

Operations:

```
tmp = Rs1.W[0] - Rs2.W[0];  
if (tmp < 0) {  
    tmp[31:0] = 0;  
    OV = 1;  
}  
Rd = tmp[31:0]; // RV32  
Rd = SE(tmp[31:0]); // RV64
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

32-bit Computation Instructions

```
__STATIC_FORCEINLINE long __RV_MAXW (int a, int b)
```

```
__STATIC_FORCEINLINE long __RV_MINW (int a, int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_MULR64 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_MULSR64 (long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_RADDW (int a, int b)
```

```
__STATIC_FORCEINLINE long __RV_RSUBW (int a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_URADDW (unsigned int a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_URSUBW (unsigned int a, unsigned int b)
```

group **32-bit Computation Instructions**

32-bit Computation Instructions

there are 8 32-bit Computation Instructions

Functions

__STATIC_FORCEINLINE long __RV_MAXW (int a, int b)

MAXW (32-bit Signed Word Maximum)

Type: DSP

Syntax:

MAXW Rd, Rs1, Rs2

Purpose :

Get the larger value from the 32-bit contents of two general registers.

Description :

This instruction compares two signed 32-bit integers stored in Rs1 and Rs2, picks the larger value as the result, and writes the result to Rd.

Operations:

```
if (Rs1.W[0] >= Rs2.W[0]) {
    Rd = SE(Rs1.W[0]);
} else {
    Rd = SE(Rs2.W[0]);
}
```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_MINW (int a, int b)

MINW (32-bit Signed Word Minimum)

Type: DSP

Syntax:

MINW Rd, Rs1, Rs2

Purpose :

Get the smaller value from the 32-bit contents of two general registers.

Description :

This instruction compares two signed 32-bit integers stored in Rs1 and Rs2, picks the smaller value as the result, and writes the result to Rd.

Operations:

```
if (Rs1.W[0] >= Rs2.W[0]) { Rd = SE(Rs2.W[0]); } else { Rd = SE(Rs1.W[0]); }
```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long long __RV_MULR64 (unsigned long a, unsigned long b)

MULR64 (Multiply Word Unsigned to 64-bit Data)

Type: DSP

Syntax:

MULR64 Rd, Rs1, Rs2

Purpose :

Multiply the 32-bit unsigned integer contents of two registers and write the 64-bit result.

RV32 Description :

This instruction multiplies the 32-bit content of Rs1 with that of Rs2 and writes the 64-bit multiplication result to an even/odd pair of registers containing Rd. Rd(4,1) index d determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result. The lower 32-bit contents of Rs1 and Rs2 are treated as unsigned integers.

RV64 Description :

This instruction multiplies the lower 32-bit content of Rs1 with that of Rs2 and writes the 64-bit multiplication result to Rd. The lower 32-bit contents of Rs1 and Rs2 are treated as unsigned integers.

Operations:

```
RV32:
Mresult = CONCAT(1`b0,Rs1) u* CONCAT(1`b0,Rs2);
R[Rd(4,1).1(0)][31:0] = Mresult[63:32];
R[Rd(4,1).0(0)][31:0] = Mresult[31:0];
RV64:
Rd = Mresult[63:0];
Mresult = CONCAT(1`b0,Rs1.W[0]) u* CONCAT(1`b0,Rs2.W[0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE long long __RV_MULSR64 (long a, long b)

MULSR64 (Multiply Word Signed to 64-bit Data)

Type: DSP

Syntax:

```
MULSR64 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit signed integer contents of two registers and write the 64-bit result.

RV32 Description :

This instruction multiplies the lower 32-bit content of Rs1 with the lower 32-bit content of Rs2 and writes the 64-bit multiplication result to an even/odd pair of registers containing Rd. Rd(4,1) index d determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result. The lower 32-bit contents of Rs1 and Rs2 are treated as signed integers.

RV64 Description :

This instruction multiplies the lower 32-bit content of Rs1 with the lower 32-bit content of Rs2 and writes the 64-bit multiplication result to Rd. The lower 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
RV32:
Mresult = Ra.s * Rb;
R[Rd(4,1).1(0)][31:0] = Mresult[63:32];
R[Rd(4,1).0(0)][31:0] = Mresult[31:0];
RV64:
Mresult = Ra.W[0].s * Rb.W[0];
Rd = Mresult[63:0];
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long __RV_RADDW (int a, int b)
```

RADDW (32-bit Signed Halving Addition)

Type: DSP

Syntax:

```
RADDW Rd, Rs1, Rs2
```

Purpose :

Add 32-bit signed integers and the results are halved to avoid overflow or saturation.

Description :

This instruction adds the first 32-bit signed integer in Rs1 with the first 32-bit signed integer in Rs2. The result is first arithmetically right-shifted by 1 bit and then sign-extended and written to Rd.

Examples:

```
* Rs1 = 0x7FFFFFFF, Rs2 = 0x7FFFFFFF, Rd = 0x7FFFFFFF
* Rs1 = 0x80000000, Rs2 = 0x80000000, Rd = 0x80000000
* Rs1 = 0x40000000, Rs2 = 0x80000000, Rd = 0xE0000000
```

Operations:

```
RV32:
Rd[31:0] = (Rs1[31:0] + Rs2[31:0]) s>> 1;
RV64:
resw[31:0] = (Rs1[31:0] + Rs2[31:0]) s>> 1;
Rd[63:0] = SE(resw[31:0]);
```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_RSUBW (int a, int b)
```

RSUBW (32-bit Signed Halving Subtraction)

Type: DSP

Syntax:

```
RSUBW Rd, Rs1, Rs2
```

Purpose :

Subtract 32-bit signed integers and the result is halved to avoid overflow or saturation.

Description :

This instruction subtracts the first 32-bit signed integer in Rs2 from the first 32-bit signed integer in Rs1. The result is first arithmetically right-shifted by 1 bit and then sign-extended and written to Rd.

Examples:

```
* Rs1 = 0x7FFFFFFF, Rs2 = 0x80000000, Rd = 0x7FFFFFFF
* Rs1 = 0x80000000, Rs2 = 0x7FFFFFFF, Rd = 0x80000000
* Rs1 = 0x80000000, Rs2 = 0x40000000, Rd = 0xA0000000
```

Operations:

```
RV32:
Rd[31:0] = (Rs1[31:0] - Rs2[31:0]) s>> 1;
RV64:
resw[31:0] = (Rs1[31:0] - Rs2[31:0]) s>> 1;
Rd[63:0] = SE(resw[31:0]);
```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_URADDW (unsigned int a, unsigned int b)

URADDW (32-bit Unsigned Halving Addition)

Type: DSP

Syntax:

```
URADDW Rd, Rs1, Rs2
```

Purpose :

Add 32-bit unsigned integers and the results are halved to avoid overflow or saturation.

Description :

This instruction adds the first 32-bit unsigned integer in Rs1 with the first 32-bit unsigned integer in Rs2. The result is first logically right-shifted by 1 bit and then sign-extended and written to Rd.

Examples:

```
* Ra = 0x7FFFFFFF, Rb = 0x7FFFFFFF Rt = 0x7FFFFFFF
* Ra = 0x80000000, Rb = 0x80000000 Rt = 0x80000000
* Ra = 0x40000000, Rb = 0x80000000 Rt = 0x60000000
```

Operations:

```
* RV32:
Rd[31:0] = (Rs1[31:0] + Rs2[31:0]) u>> 1;
* RV64:
resw[31:0] = (Rs1[31:0] + Rs2[31:0]) u>> 1;
Rd[63:0] = SE(resw[31:0]);
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSUBW (unsigned int a, unsigned int b)

URSUBW (32-bit Unsigned Halving Subtraction)

Type: DSP

Syntax:

```
URSUBW Rd, Rs1, Rs2
```

Purpose :

Subtract 32-bit unsigned integers and the result is halved to avoid overflow or saturation.

Description :

This instruction subtracts the first 32-bit signed integer in Rs2 from the first 32-bit signed integer in Rs1. The result is first logically right-shifted by 1 bit and then sign-extended and written to Rd.

Examples:

```
* Ra = 0x7FFFFFFF, Rb = 0x80000000 Rt = 0xFFFFFFFF
* Ra = 0x80000000, Rb = 0x7FFFFFFF Rt = 0x00000000
* Ra = 0x80000000, Rb = 0x40000000 Rt = 0x20000000
```

Operations:

```
* RV32:
Rd[31:0] = (Rs1[31:0] - Rs2[31:0]) u>> 1;
* RV64:
resw[31:0] = (Rs1[31:0] - Rs2[31:0]) u>> 1;
Rd[63:0] = SE(resw[31:0]);
```

Parameters

- **a** – [in] unsigned int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

OV (Overflow) flag Set/Clear Instructions

```
__STATIC_FORCEINLINE void __RV_CLROV (void)
```

```
__STATIC_FORCEINLINE unsigned long __RV_RDOV (void)
```

group OV (Overflow) flag Set/Clear Instructions

OV (Overflow) flag Set/Clear Instructions.

The following table lists the user instructions related to Overflow (OV) flag manipulation. there are 2 OV (Overflow) flag Set/Clear Instructions

Functions

```
__STATIC_FORCEINLINE void __RV_CLROV (void)
```

CLROV (Clear OV flag)

Type: DSP

Syntax:

```
CLROV # pseudo mnemonic
```

Purpose :

This pseudo instruction is an alias to CSRRCI x0, ucode, 1 instruction.

```
__STATIC_FORCEINLINE unsigned long __RV_RDOV (void)
```

RDOV (Read OV flag)

Type: DSP

Syntax:

```
RDOV Rd # pseudo mnemonic
```

Purpose :

This pseudo instruction is an alias to CSRR Rd, ucode instruction which maps to the real instruction of CSRRS Rd, ucode, x0.

Returns

value stored in unsigned long type

Non-SIMD Miscellaneous Instructions

```
__STATIC_FORCEINLINE long __RV_AVE (long a, long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_BITREV (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_BPICK (unsigned long a, unsigned long b,  
unsigned long c)
```

```
__STATIC_FORCEINLINE unsigned long __RV_MADDR32 (unsigned long t, unsigned long a,  
unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_MSUBR32 (unsigned long t, unsigned long a,  
unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SRA_U (long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SWAP8 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SWAP16 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_WEXT (long long a, unsigned int b)
```

```
__RV_BITREVI(a, b)
```

```
__RV_INSB(t, a, b)
```

```
__RV_SRAI_U(a, b)
```

```
__RV_WEXTI(a, b)
```

group **Non-SIMD Miscellaneous Instructions**

Non-SIMD Miscellaneous Instructions.

There are 13 Miscellaneous Instructions here.

Defines

__RV_BITREVI(a, b)

BITREVI (Bit Reverse Immediate)

Type: DSP

Syntax:

```
(RV32) BITREVI Rd, Rs1, imm[4:0]
(RV64) BITREVI Rd, Rs1, imm[5:0]
```

Purpose :

Reverse the bit positions of the source operand within a specified width starting from bit 0. The reversed width is an immediate value.

Description :

This instruction reverses the bit positions of the content of Rs1. The reversed bit width is calculated as imm[4:0]+1 (RV32) or imm[5:0]+1 (RV64). The upper bits beyond the reversed width are filled with zeros. After the bit reverse operation, the result is written to Rd.

Operations:

```
msb = imm[4:0]; (RV32)
msb = imm[5:0]; (RV64)
rev[0:msb] = Rs1[msb:0];
Rd = ZE(rev[msb:0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__RV_INSB(t, a, b)

INSB (Insert Byte)

Type: DSP

Syntax:

```
(RV32) INSB Rd, Rs1, imm[1:0]
(RV64) INSB Rd, Rs1, imm[2:0]
```

Purpose :

Insert byte 0 of a 32-bit or 64-bit register into one of the byte elements of another register.

Description :

This instruction inserts byte 0 of Rs1 into byte `imm[1:0]` (RV32) or `imm[2:0]` (RV64) of Rd.

Operations:

```
bpos = imm[1:0]; (RV32)
bpos = imm[2:0]; (RV64)
Rd.B[bpos] = Rs1.B[0]
```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI_U(a, b)

SRAI.u (Rounding Shift Right Arithmetic Immediate)

Type: DSP

Syntax:

```
SRAI.u Rd, Rs1, imm6u[4:0] (RV32)
SRAI.u Rd, Rs1, imm6u[5:0] (RV64)
```

Purpose :

Perform an arithmetic right shift operation with rounding. The shift amount is an immediate value.

Description :

This instruction right-shifts the content of Rs1 arithmetically. The shifted out bits are filled with the sign-bit and the shift amount is specified by the `imm6u[4:0]` (RV32) or `imm6u[5:0]` (RV64) constant . For the rounding operation, a value of 1 is added to the most significant discarded bit of the data to calculate the final result. And the result is written to Rd.

Operations:

```
* RV32:
sa = imm6u[4:0];
if (sa > 0) {
    res[31:-1] = SE33(Rs1[31:(sa-1)]) + 1;
    Rd = res[31:0];
} else {
    Rd = Rs1;
}

* RV64:
sa = imm6u[5:0];
if (sa > 0) {
    res[63:-1] = SE65(Rs1[63:(sa-1)]) + 1;
    Rd = res[63:0];
} else {
    Rd = Rs1;
}
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__RV_WEXTI(a, b)

WEXTI (Extract Word from 64-bit Immediate)

Type: DSP

Syntax:

WEXTI Rd, Rs1, #LSBloc

Purpose :

Extract a 32-bit word from a 64-bit value stored in an even/odd pair of registers (RV32) or a register (RV64) starting from a specified immediate LSB bit position.

RV32 Description :

This instruction extracts a 32-bit word from a 64-bit value of an even/odd pair of registers specified by Rs1(4,1) starting from a specified immediate LSB bit position, #LSBloc. The extracted word is written to Rd. Rs1(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the 64-bit value and the even 2d register of the pair contains the low 32-bit of the 64-bit value.

RV64 Description :

This instruction extracts a 32-bit word from a 64-bit value in Rs1 starting from a specified immediate LSB bit position, #LSBloc. The extracted word is sign-extended and written to lower 32-bit of Rd.

Operations:

```
* RV32:
Idx0 = CONCAT(Rs1(4,1), 1'b0); Idx1 = CONCAT(Rs2(4,1), 1'b1);
src[63:0] = Concat(R[Idx1], R[Idx0]);
Rd = src[31+LSBloc:LSBloc];
* RV64:
ExtractW = Rs1[31+LSBloc:LSBloc];
Rd = SE(ExtractW)
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE long __RV_AVE (long a, long b)

AVE (Average with Rounding)

Type: DSP

Syntax:

AVE Rd, Rs1, Rs2

Purpose :

Calculate the average of the contents of two general registers.

Description :

This instruction calculates the average value of two signed integers stored in Rs1 and Rs2, rounds up a half-integer result to the nearest integer, and writes the result to Rd.

Operations:

```
Sum = CONCAT(Rs1[MSB],Rs1[MSB:0]) + CONCAT(Rs2[MSB],Rs2[MSB:0]) + 1;
Rd = Sum[(MSB+1):1];
for RV32: MSB=31,
for RV64: MSB=63
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_BITREV (unsigned long a, unsigned long b)

BITREV (Bit Reverse)

Type: DSP

Syntax:

BITREV Rd, Rs1, Rs2

Purpose :

Reverse the bit positions of the source operand within a specified width starting from bit 0. The reversed width is a variable from a GPR.

Description :

This instruction reverses the bit positions of the content of Rs1. The reversed bit width is calculated as Rs2[4:0]+1 (RV32) or Rs2[5:0]+1 (RV64). The upper bits beyond the reversed width are filled with zeros. After the bit reverse operation, the result is written to Rd.

Operations:

```
msb = Rs2[4:0]; (for RV32)
msb = Rs2[5:0]; (for RV64)
rev[0:msb] = Rs1[msb:0];
Rd = ZE(rev[msb:0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_BPICK (unsigned long a, unsigned long b, unsigned long c)

BPICK (Bit-wise Pick)

Type: DSP

Syntax:

```
BPICK Rd, Rs1, Rs2, Rc
```

Purpose :

Select from two source operands based on a bit mask in the third operand.

Description :

This instruction selects individual bits from Rs1 or Rs2, based on the bit mask value in Rc. If a bit in Rc is 1, the corresponding bit is from Rs1; otherwise, the corresponding bit is from Rs2. The selection results are written to Rd.

Operations:

```
Rd[x] = Rc[x]? Rs1[x] : Rs2[x];
for RV32, x=31...0
for RV64, x=63...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b
- **c** – [in] unsigned long type of value stored in c

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_MADDR32 (unsigned long t, unsigned long a, unsigned long b)

MADDR32 (Multiply and Add to 32-Bit Word)

Type: DSP

Syntax:

```
MADDR32 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit contents of two registers and add the lower 32-bit multiplication result to the 32-bit content of a destination register. Write the final result back to the destination register.

Description :

This instruction multiplies the lower 32-bit content of Rs1 with that of Rs2. It adds the lower 32-bit multiplication result to the lower 32-bit content of Rd and writes the final result (RV32) or sign-extended result (RV64) back to Rd. The contents of Rs1 and Rs2 can be either signed or unsigned integers.

Operations:

```
RV32:
Mresult = Rs1 * Rs2;
Rd = Rd + Mresult.W[0];
RV64:
Mresult = Rs1.W[0] * Rs2.W[0];
tres[31:0] = Rd.W[0] + Mresult.W[0];
Rd = SE64(tres[31:0]);
```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_MSUBR32 (unsigned long t, unsigned long a,
unsigned long b)
```

MSUBR32 (Multiply and Subtract from 32-Bit Word)

Type: DSP

Syntax:

```
MSUBR32 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit contents of two registers and subtract the lower 32-bit multiplication result from the 32-bit content of a destination register. Write the final result back to the destination register.

Description :

This instruction multiplies the lower 32-bit content of Rs1 with that of Rs2, subtracts the lower 32-bit multiplication result from the lower 32-bit content of Rd, then writes the final result (RV32) or sign-extended result (RV64) back to Rd. The contents of Rs1 and Rs2 can be either signed or unsigned integers.

Operations:

```
RV32:
Mresult = Rs1 * Rs2;
```

(continues on next page)

(continued from previous page)

```

Rd = Rd - Mresult.W[0];
RV64:
Mresult = Rs1.W[0] * Rs2.W[0];
tres[31:0] = Rd.W[0] - Mresult.W[0];
Rd = SE64(tres[31:0]);

```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE long __RV_SRA_U (long a, unsigned int b)

SRA.u (Rounding Shift Right Arithmetic)

Type: DSP

Syntax:

```
SRA.u Rd, Rs1, Rs2
```

Purpose :

Perform an arithmetic right shift operation with rounding. The shift amount is a variable from a GPR.

Description :

This instruction right-shifts the content of Rs1 arithmetically. The shifted out bits are filled with the sign-bit and the shift amount is specified by the low-order 5-bits (RV32) or 6-bits (RV64) of the Rs2 register. For the rounding operation, a value of 1 is added to the most significant discarded bit of the data to calculate the final result. And the result is written to Rd.

Operations:

```

* RV32:
sa = Rs2[4:0];
if (sa > 0) {
    res[31:-1] = SE33(Rs1[31:(sa-1)]) + 1;
    Rd = res[31:0];
} else {
    Rd = Rs1;
}
* RV64:
sa = Rs2[5:0];
if (sa > 0) {
    res[63:-1] = SE65(Rs1[63:(sa-1)]) + 1;
    Rd = res[63:0];
} else {
    Rd = Rs1;
}

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_SWAP8 (unsigned long a)

SWAP8 (Swap Byte within Halfword)

Type: DSP

Syntax:

SWAP8 Rd, Rs1

Purpose :

Swap the bytes within each halfword of a register.

Description :

This instruction swaps the bytes within each halfword of Rs1 and writes the result to Rd.

Operations:

```
Rd.H[x] = CONCAT(Rs1.H[x][7:0],Rs1.H[x][15:8]);
for RV32: x=1...0,
for RV64: x=3...0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SWAP16 (unsigned long a)

SWAP16 (Swap Halfword within Word)

Type: DSP

Syntax:

SWAP16 Rd, Rs1

Purpose :

Swap the 16-bit halfwords within each word of a register.

Description :

This instruction swaps the 16-bit halfwords within each word of Rs1 and writes the result to Rd.

Operations:

```
Rd.W[x] = CONCAT(Rs1.W[x][15:0],Rs1.H[x][31:16]);
for RV32: x=0,
for RV64: x=1...0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_WEXT (long long a, unsigned int b)

WEXT (Extract Word from 64-bit)

Type: DSP

Syntax:

WEXT Rd, Rs1, Rs2

Purpose :

Extract a 32-bit word from a 64-bit value stored in an even/odd pair of registers (RV32) or a register (RV64) starting from a specified LSB bit position in a register.

RV32 Description :

This instruction extracts a 32-bit word from a 64-bit value of an even/odd pair of registers specified by Rs1(4,1) starting from a specified LSB bit position, specified in Rs2[4:0]. The extracted word is written to Rd. Rs1(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the 64-bit value and the even 2d register of the pair contains the low 32-bit of the 64-bit value.

Operations:

```
* RV32:
Idx0 = CONCAT(Rs1(4,1), 1'b0); Idx1 = CONCAT(Rs1(4,1), 1'b1);
src[63:0] = Concat(R[Idx1], R[Idx0]);
LSBloc = Rs2[4:0];
Rd = src[31+LSBloc:LSBloc];
* RV64:
LSBloc = Rs2[4:0];
ExtractW = Rs1[31+LSBloc:LSBloc];
Rd = SE(ExtractW)
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

group **Non-SIMD Instructions**

Non-SIMD Instructions.

Partial-SIMD Data Processing Instructions

SIMD 16-bit Packing Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_PKBB16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKBT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKTT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKTB16 (unsigned long a, unsigned long b)
```

group SIMD 16-bit Packing Instructions

SIMD 16-bit Packing Instructions.

there are 4 SIMD16-bit Packing Instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_PKBB16 (unsigned long a, unsigned long b)
```

PKBB16 (Pack Two 16-bit Data from Both Bottom Half)

Type: DSP

Syntax:

```
PKBB16 Rd, Rs1, Rs2
PKBT16 Rd, Rs1, Rs2
PKTT16 Rd, Rs1, Rs2
PKTB16 Rd, Rs1, Rs2
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBB16: bottom.bottom
- PKBT16 bottom.top
- PKTT16 top.top
- PKTB16 top.bottom

Description :

(PKBB16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x] [15:0] to Rd.W[x] [15:0].
 (PKBT16) moves Rs1.W[x] [15:0] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].
 (PKTT16) moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].
 (PKTB16) moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [15:0] to Rd.W[x] [15:0].

Operations:

```

Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][15:0]); // PKBB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][31:16]); // PKBT16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][15:0]); // PKTB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][31:16]); // PKTT16
for RV32: x=0,
for RV64: x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKBT16 (unsigned long a, unsigned long b)

PKBT16 (Pack Two 16-bit Data from Bottom and Top Half)

Type: DSP

Syntax:

```

PKBB16 Rd, Rs1, Rs2
PKBT16 Rd, Rs1, Rs2
PKTT16 Rd, Rs1, Rs2
PKTB16 Rd, Rs1, Rs2

```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBB16: bottom.bottom
- PKBT16 bottom.top
- PKTT16 top.top
- PKTB16 top.bottom

Description :

(PKBB16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x][15:0] to Rd.W[x][15:0].
(PKBT16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x][31:16] to Rd.W[x][15:0].
(PKTT16) moves Rs1.W[x][31:16] to Rd.W[x][31:16] and moves Rs2.W[x][31:16] to Rd.W[x][15:0].
(PKTB16) moves Rs1.W[x][31:16] to Rd.W[x][31:16] and moves Rs2.W[x][15:0] to Rd.W[x][15:0].

Operations:

```

Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][15:0]); // PKBB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][31:16]); // PKBT16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][15:0]); // PKTB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][31:16]); // PKTT16
for RV32: x=0,
for RV64: x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a

- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKTT16 (unsigned long a, unsigned long b)

PKTT16 (Pack Two 16-bit Data from Both Top Half)

Type: DSP

Syntax:

```
PKBB16 Rd, Rs1, Rs2
PKBT16 Rd, Rs1, Rs2
PKTT16 Rd, Rs1, Rs2
PKTB16 Rd, Rs1, Rs2
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBB16: bottom.bottom
- PKBT16 bottom.top
- PKTT16 top.top
- PKTB16 top.bottom

Description :

(PKBB16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x][15:0] to Rd.W[x][15:0].
 (PKBT16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x][31:16] to Rd.W[x][15:0].
 (PKTT16) moves Rs1.W[x][31:16] to Rd.W[x][31:16] and moves Rs2.W[x][31:16] to Rd.W[x][15:0].
 (PKTB16) moves Rs1.W[x][31:16] to Rd.W[x][31:16] and moves Rs2.W[x][15:0] to Rd.W[x][15:0].

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][15:0]); // PKBB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][31:16]); // PKBT16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][15:0]); // PKTB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][31:16]); // PKTT16
for RV32: x=0,
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKTB16 (unsigned long a, unsigned long b)

PKTB16 (Pack Two 16-bit Data from Top and Bottom Half)

Type: DSP

Syntax:

```

PKBB16 Rd, Rs1, Rs2
PKBT16 Rd, Rs1, Rs2
PKTT16 Rd, Rs1, Rs2
PKTB16 Rd, Rs1, Rs2

```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBB16: bottom.bottom
- PKBT16 bottom.top
- PKTT16 top.top
- PKTB16 top.bottom

Description :

(PKBB16) moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x] [15:0] to Rd.W[x] [15:0].
(PKBT16) moves Rs1.W[x] [15:0] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].
(PKTT16) moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].
(PKTB16) moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [15:0] to Rd.W[x] [15:0].

Operations:

```

Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][15:0]); // PKBB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][31:16]); // PKBT16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][15:0]); // PKTB16
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][31:16]); // PKTT16
for RV32: x=0,
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

Signed MSW 32x32 Multiply and Add Instructions

```
__STATIC_FORCEINLINE long __RV_KMMAC (long t, long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAC_U (long t, long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMSB (long t, long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMSB_U (long t, long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_KWMMUL (long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_KWMUL_U (long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_SMMUL (long a, long b)
```

```
__STATIC_FORCEINLINE long __RV_SMMUL_U (long a, long b)
```

group Signed MSW 32x32 Multiply and Add Instructions

Signed MSW 32x32 Multiply and Add Instructions.

there are 8 Signed MSW 32x32 Multiply and Add Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KMMAC (long t, long a, long b)
```

KMMAC (SIMD Saturating MSW Signed Multiply Word and Add)

Type: SIMD

Syntax:

```
KMMAC Rd, Rs1, Rs2
KMMAC.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of two registers and add the most significant 32-bit results with the signed 32-bit integer elements of a third register. The addition results are saturated first and then written back to the third register. The .u form performs an additional rounding up operation on the multiplication results before adding the most significant 32-bit part of the results.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```
Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][63:32];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
```

(continues on next page)

(continued from previous page)

```

}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMAC_U (long t, long a, long b)

KMMAC.u (SIMD Saturating MSW Signed Multiply Word and Add with Rounding)

Type: SIMD

Syntax:

```

KMMAC Rd, Rs1, Rs2
KMMAC.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of two registers and add the most significant 32-bit results with the signed 32-bit integer elements of a third register. The addition results are saturated first and then written back to the third register. The .u form performs an additional rounding up operation on the multiplication results before adding the most significant 32-bit part of the results.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][63:32];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}

```

(continues on next page)

(continued from previous page)

```

}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMSB (long t, long a, long b)

KMMSB (SIMD Saturating MSW Signed Multiply Word and Subtract)

Type: SIMD

Syntax:

```

KMMSB Rd, Rs1, Rs2
KMMSB.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of two registers and subtract the most significant 32-bit results from the signed 32-bit elements of a third register. The subtraction results are written to the third register. The .u form performs an additional rounding up operation on the multiplication results before subtracting the most significant 32-bit part of the results.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and subtracts the most significant 32-bit multiplication results from the signed 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    res[x] = Rd.W[x] - Round[x][32:1];
} else {
    res[x] = Rd.W[x] - Mres[x][63:32];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
}

```

(continues on next page)

(continued from previous page)

```

    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMMSB_U (long t, long a, long b)
```

KMMSB.u (SIMD Saturating MSW Signed Multiply Word and Subtraction with Rounding)

Type: SIMD**Syntax:**

```

KMMSB Rd, Rs1, Rs2
KMMSB.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of two registers and subtract the most significant 32-bit results from the signed 32-bit elements of a third register. The subtraction results are written to the third register. The .u form performs an additional rounding up operation on the multiplication results before subtracting the most significant 32-bit part of the results.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and subtracts the most significant 32-bit multiplication results from the signed 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    res[x] = Rd.W[x] - Round[x][32:1];
} else {
    res[x] = Rd.W[x] - Mres[x][63:32];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {

```

(continues on next page)

(continued from previous page)

```

    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KWMUL (long a, long b)

KWMUL (SIMD Saturating MSW Signed Multiply Word & Double)

Type: SIMD

Syntax:

```

KWMUL Rd, Rs1, Rs2
KWMUL.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of two registers, shift the results left 1-bit, saturate, and write the most significant 32-bit results to a register. The .u form additionally rounds up the multiplication results from the most signification discarded bit.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2. It then shifts the multiplication results one bit to the left and takes the most significant 32-bit results. If the shifted result is greater than $2^{31}-1$, it is saturated to $2^{31}-1$ and the OV flag is set to 1. The final element result is written to Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction additionally rounds up the 64-bit multiplication results by adding a 1 to bit 30 before the shift and saturation operations.

Operations:

```

if ((0x80000000 != Rs1.W[x]) | (0x80000000 != Rs2.W[x])) {
    Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
    if (`.u` form) {
        Round[x][33:0] = Mres[x][63:30] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][62:31];
    }
} else {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
}

```

(continues on next page)

(continued from previous page)

```

}
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KWMUL_U (long a, long b)

KWMUL.u (SIMD Saturating MSW Signed Multiply Word & Double with Rounding)

Type: SIMD

Syntax:

```

KWMUL Rd, Rs1, Rs2
KWMUL.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of two registers, shift the results left 1-bit, saturate, and write the most significant 32-bit results to a register. The .u form additionally rounds up the multiplication results from the most significant discarded bit.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2. It then shifts the multiplication results one bit to the left and takes the most significant 32-bit results. If the shifted result is greater than $2^{31}-1$, it is saturated to $2^{31}-1$ and the OV flag is set to 1. The final element result is written to Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction additionally rounds up the 64-bit multiplication results by adding a 1 to bit 30 before the shift and saturation operations.

Operations:

```

if ((0x80000000 != Rs1.W[x]) | (0x80000000 != Rs2.W[x])) {
    Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
    if (`.u` form) {
        Round[x][33:0] = Mres[x][63:30] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][62:31];
    }
} else {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
}
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMUL (long a, long b)

SMMUL (SIMD MSW Signed Multiply Word)

Type: SIMD

Syntax:

```
SMMUL Rd, Rs1, Rs2
SMMUL.u Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit signed integer elements of two registers and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form performs an additional rounding up operation on the multiplication results before taking the most significant 32-bit part of the results.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

- For smmul/RV32 instruction, it is an alias to mulh/RV32 instruction.

Operations:

```
Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][63:32];
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMUL_U (long a, long b)

SMMUL.u (SIMD MSW Signed Multiply Word with Rounding)

Type: SIMD

Syntax:

```
SMMUL Rd, Rs1, Rs2
SMMUL.u Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit signed integer elements of two registers and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form performs an additional rounding up operation on the multiplication results before taking the most significant 32-bit part of the results.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

- For smmul/RV32 instruction, it is an alias to mulh/RV32 instruction.

Operations:

```
Mres[x][63:0] = Rs1.W[x] * Rs2.W[x];
if (`.u` form) {
    Round[x][32:0] = Mres[x][63:31] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][63:32];
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long type

Signed MSW 32x16 Multiply and Add Instructions

```
__STATIC_FORCEINLINE long __RV_KMMAWB (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAWB_U (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAWB2 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAWB2_U (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAWT (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMMAWT_U (long t, unsigned long a, unsigned long b)
```

```

__STATIC_FORCEINLINE long __RV_KMMAWT2 (long t, unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMMAWT2_U (long t, unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMMWB2 (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMMWB2_U (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMMWT2 (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMMWT2_U (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMMWB (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMMWB_U (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMMWT (long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMMWT_U (long a, unsigned long b)

```

group Signed MSW 32x16 Multiply and Add Instructions

Signed MSW 32x16 Multiply and Add Instructions.

there are 15 Signed MSW 32x16 Multiply and Add Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KMMAWB (long t, unsigned long a, unsigned long b)
```

KMMAWB (SIMD Saturating MSW Signed Multiply Word and Bottom Half and Add)

Type: SIMD

Syntax:

```

KMMAWB Rd, Rs1, Rs2
KMMAWB.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register and add the most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The addition result is written to the corresponding 32-bit elements of the third register. The .u form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed bottom 16-bit content of the corresponding 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the

corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The `.u` form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the result before the addition operations.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[0];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][47:16];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMAWB_U (long t, unsigned long a, unsigned long b)

KMMAWB.u (SIMD Saturating MSW Signed Multiply Word and Bottom Half and Add with Rounding)

Type: SIMD

Syntax:

```
KMMAWB Rd, Rs1, Rs2
KMMAWB.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register and add the most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The addition result is written to the corresponding 32-bit elements of the third register. The `.u` form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed bottom 16-bit content of the corresponding 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The `.u` form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the result before the addition operations.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[0];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][47:16];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMAWB2 (long t, unsigned long a, unsigned long b)

KMMAWB2 (SIMD Saturating MSW Signed Multiply Word and Bottom Half & 2 and Add)

Type: SIMD

Syntax:

```
KMMAWB2 Rd, Rs1, Rs2
KMMAWB2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and add the saturated most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The saturated addition result is written to the corresponding 32-bit elements of the third register. The `.u` form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed bottom 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and adds the saturated most significant 32-bit Q31 multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the result before the addition operations.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[0] == 0x8000)) {
    addop.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[0];
    if (`.u` form) {
        Mres[x][47:14] = Mres[x][47:14] + 1;
    }
    addop.W[x] = Mres[x][46:15]; // doubling
}
res[x] = Rd.W[x] + addop.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMAWB2_U (long t, unsigned long a, unsigned long b)

KMMAWB2.u (SIMD Saturating MSW Signed Multiply Word and Bottom Half & 2 and Add with Rounding)

Type: SIMD

Syntax:

```
KMMAWB2 Rd, Rs1, Rs2
KMMAWB2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and add the saturated most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The saturated addition result is written to the corresponding 32-bit elements of the third register. The `.u` form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed bottom 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and adds the saturated most significant 32-bit Q31 multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The `.u` form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the result before the addition operations.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[0] == 0x8000)) {
    addop.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[0];
    if (`.u` form) {
        Mres[x][47:14] = Mres[x][47:14] + 1;
    }
    addop.W[x] = Mres[x][46:15]; // doubling
}
res[x] = Rd.W[x] + addop.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMAWT (long t, unsigned long a, unsigned long b)

KMMAWT (SIMD Saturating MSW Signed Multiply Word and Top Half and Add)

Type: SIMD

Syntax:

```
KMMAWT Rd, Rs1, Rs2
KMMAWT.u Rd Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the signed top 16-bit of the corresponding 32-bit elements of another register and add the most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The addition results are written to the corresponding 32-bit elements of the third register. The .u form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed top 16-bit of the corresponding 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the result before the addition operations.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[1];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][47:16];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMMAWT_U (long t, unsigned long a, unsigned long b)
```

KMMAWT.u (SIMD Saturating MSW Signed Multiply Word and Top Half and Add with Rounding)

Type: SIMD

Syntax:

```
KMMAWT Rd, Rs1, Rs2
KMMAWT.u Rd Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the signed top 16-bit of the corresponding 32-bit elements of another register and add the most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The addition results are written to the corresponding 32-bit elements of the third register. The .u form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed top 16-bit of the corresponding 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the result before the addition operations.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[1];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    res[x] = Rd.W[x] + Round[x][32:1];
} else {
    res[x] = Rd.W[x] + Mres[x][47:16];
}
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMMAWT2 (long t, unsigned long a, unsigned long b)
```

KMMAWT2 (SIMD Saturating MSW Signed Multiply Word and Top Half & 2 and Add)

Type: SIMD

Syntax:

```
KMMAWT2 Rd, Rs1, Rs2
KMMAWT2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and add the saturated most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The saturated addition result is written to the corresponding 32-bit elements of the third register. The `.u` form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed top 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and adds the saturated most significant 32-bit Q31 multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The `.u` form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the result before the addition operations.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[1] == 0x8000)) {
    addop.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[1];
    if (`.u` form) {
        Mres[x][47:14] = Mres[x][47:14] + 1;
    }
    addop.W[x] = Mres[x][46:15]; // doubling
}
res[x] = Rd.W[x] + addop.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMMAWT2_U (long t, unsigned long a, unsigned long b)
```

KMMAWT2.u (SIMD Saturating MSW Signed Multiply Word and Top Half & 2 and Add with Rounding)

Type: SIMD

Syntax:

```
KMMAWT2 Rd, Rs1, Rs2
KMMAWT2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and add the saturated most significant 32-bit results with the corresponding signed 32-bit elements of a third register. The saturated addition result is written to the corresponding 32-bit elements of the third register. The .u form rounds up the multiplication results from the most significant discarded bit before the addition operations.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed top 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and adds the saturated most significant 32-bit Q31 multiplication results with the corresponding signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the result before the addition operations.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[1] == 0x8000)) {
    addop.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[1];
    if (`u` form) {
        Mres[x][47:14] = Mres[x][47:14] + 1;
    }
    addop.W[x] = Mres[x][46:15]; // doubling
}
res[x] = Rd.W[x] + addop.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a

- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMWB2 (long a, unsigned long b)

KMMWB2 (SIMD Saturating MSW Signed Multiply Word and Bottom Half & 2)

Type: SIMD

Syntax:

```
KMMWB2 Rd, Rs1, Rs2
KMMWB2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and write the saturated most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed bottom 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and writes the saturated most significant 32-bit Q31 multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the results.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[0] == 0x8000)) {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[0];
    if (`.u` form) {
        Round[x][32:0] = Mres[x][46:14] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][46:15];
    }
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMWB2_U (long a, unsigned long b)

KMMWB2.u (SIMD Saturating MSW Signed Multiply Word and Bottom Half & 2 with Rounding)

Type: SIMD

Syntax:

```
KMMWB2 Rd, Rs1, Rs2
KMMWB2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and write the saturated most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed bottom 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and writes the saturated most significant 32-bit Q31 multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the results.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[0] == 0x8000)) {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[0];
    if (`u` form) {
        Round[x][32:0] = Mres[x][46:14] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][46:15];
    }
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMWT2 (long a, unsigned long b)

KMMWT2 (SIMD Saturating MSW Signed Multiply Word and Top Half & 2)

Type: SIMD

Syntax:

```
KMMWT2 Rd, Rs1, Rs2
KMMWT2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and write the saturated most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed top 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and writes the saturated most significant 32-bit Q31 multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the results.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[1] == 0x8000)) {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[1];
    if (`.u` form) {
        Round[x][32:0] = Mres[x][46:14] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][46:15];
    }
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMMWT2_U (long a, unsigned long b)

KMMWT2.u (SIMD Saturating MSW Signed Multiply Word and Top Half & 2 with Rounding)

Type: SIMD

Syntax:

```
KMMWT2 Rd, Rs1, Rs2
KMMWT2.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, double the multiplication results and write the saturated most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit Q31 elements of Rs1 with the signed top 16-bit Q15 content of the corresponding 32-bit elements of Rs2, doubles the Q46 results to Q47 numbers and writes the saturated most significant 32-bit Q31 multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit Q47 multiplication results by adding a 1 to bit 15 (i.e., bit 14 before doubling) of the results.

Operations:

```
if ((Rs1.W[x] == 0x80000000) & (Rs2.W[x].H[1] == 0x8000)) {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
} else {
    Mres[x][47:0] = Rs1.W[x] s* Rs2.W[x].H[1];
    if (`.u` form) {
        Round[x][32:0] = Mres[x][46:14] + 1;
        Rd.W[x] = Round[x][32:1];
    } else {
        Rd.W[x] = Mres[x][46:15];
    }
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMWB (long a, unsigned long b)

SMMWB (SIMD MSW Signed Multiply Word and Bottom Half)

Type: SIMD

Syntax:

```
SMMWB Rd, Rs1, Rs2
SMMWB.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed bottom 16-bit content of the corresponding 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the results.

Operations:

```

Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[0];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][47:16];
}
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMWB_U (long a, unsigned long b)

SMMWB.u (SIMD MSW Signed Multiply Word and Bottom Half with Rounding)

Type: SIMD

Syntax:

```

SMMWB Rd, Rs1, Rs2
SMMWB.u Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 32-bit integer elements of one register and the bottom 16-bit of the corresponding 32-bit elements of another register, and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed bottom 16-bit content of the corresponding 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the results.

Operations:

```

Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[0];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][47:16];
}
for RV32: x=0
for RV64: x=1...0

```

Parameters

- **a** – [in] long type of value stored in a

- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMWT (long a, unsigned long b)

SMMWT (SIMD MSW Signed Multiply Word and Top Half)

Type: SIMD

Syntax:

```
SMMWT Rd, Rs1, Rs2
SMMWT.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the top signed 16-bit content of the corresponding 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the results.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[1];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][47:16];
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMMWT_U (long a, unsigned long b)

SMMWT.u (SIMD MSW Signed Multiply Word and Top Half with Rounding)

Type: SIMD

Syntax:

```
SMMWT Rd, Rs1, Rs2
SMMWT.u Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit integer elements of one register and the top 16-bit of the corresponding 32-bit elements of another register, and write the most significant 32-bit results to the corresponding 32-bit elements of a register. The .u form rounds up the results from the most significant discarded bit.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the top signed 16-bit content of the corresponding 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The .u form of the instruction rounds up the most significant 32-bit of the 48-bit multiplication results by adding a 1 to bit 15 of the results.

Operations:

```
Mres[x][47:0] = Rs1.W[x] * Rs2.W[x].H[1];
if (`.u` form) {
    Round[x][32:0] = Mres[x][47:15] + 1;
    Rd.W[x] = Round[x][32:1];
} else {
    Rd.W[x] = Mres[x][47:16];
}
for RV32: x=0
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

Signed 16-bit Multiply 32-bit Add/Subtract Instructions

```
__STATIC_FORCEINLINE long __RV_KMABB (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMABT (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMATT (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMADA (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMAXDA (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMADS (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMADRS (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMAXDS (long t, unsigned long a, unsigned long b)
```

```

__STATIC_FORCEINLINE long __RV_KMDA (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMXDA (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMSDA (long t, unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_KMSXDA (long t, unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMBB16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMBT16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMTT16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMDS (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMDRS (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE long __RV_SMXDS (unsigned long a, unsigned long b)

```

group **Signed 16-bit Multiply 32-bit Add/Subtract Instructions**

Signed 16-bit Multiply 32-bit Add/Subtract Instructions.

there are 18 Signed 16-bit Multiply 32-bit Add/Subtract Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KMABB (long t, unsigned long a, unsigned long b)
```

KMABB (SIMD Saturating Signed Multiply Bottom Halfs & Add)

Type: SIMD

Syntax:

```

KMABB Rd, Rs1, Rs2
KMABT Rd, Rs1, Rs2
KMATT Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 16-bit content of 32-bit elements in a register with the 16-bit content of 32-bit elements in another register and add the result to the content of 32-bit elements in the third register. The addition result may be saturated and is written to the third register.

- KMABB: $rd.W[x] + bottom * bottom$ (per 32-bit element)
- KMABT $rd.W[x] + bottom * top$ (per 32-bit element)
- KMATT $rd.W[x] + top * top$ (per 32-bit element)

Description :

For the KMABB instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMABT instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMATT instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. The multiplication result is added to the content of 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to

- a. The results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]); // KMABB
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[1]); // KMABT
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]); // KMATT
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMABT (long t, unsigned long a, unsigned long b)
```

KMABT (SIMD Saturating Signed Multiply Bottom & Top Halfs & Add)

Type: SIMD

Syntax:

```
KMABB Rd, Rs1, Rs2
KMABT Rd, Rs1, Rs2
KMATT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of 32-bit elements in a register with the 16-bit content of 32-bit elements in another register and add the result to the content of 32-bit elements in the third register. The addition result may be saturated and is written to the third register.

- KMABB: $rd.W[x] + bottom * bottom$ (per 32-bit element)

- KMABB rd.W[x] + bottom*top (per 32-bit element)
- KMATT rd.W[x] + top*top (per 32-bit element)

Description :

For the KMABB instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMABT instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMATT instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. The multiplication result is added to the content of 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to

- The results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]); // KMABB
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[1]); // KMABT
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]); // KMATT
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMATT (long t, unsigned long a, unsigned long b)
```

KMATT (SIMD Saturating Signed Multiply Top Halves & Add)

Type: SIMD

Syntax:

```
KMABB Rd, Rs1, Rs2
KMABT Rd, Rs1, Rs2
KMATT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of 32-bit elements in a register with the 16-bit content of 32-bit elements in another register and add the result to the content of 32-bit elements in the third register. The addition

result may be saturated and is written to the third register.

- KMABB: $rd.W[x] + bottom * bottom$ (per 32-bit element)
- KMABT $rd.W[x] + bottom * top$ (per 32-bit element)
- KMATT $rd.W[x] + top * top$ (per 32-bit element)

Description :

For the KMABB instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMABT instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMATT instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. The multiplication result is added to the content of 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to

- The results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]); // KMABB
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[1]); // KMABT
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]); // KMATT
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMADA (long t, unsigned long a, unsigned long b)

KMADA (SIMD Saturating Signed Multiply Two Halfs and Two Adds)

Type: SIMD

Syntax:

```
KMADA Rd, Rs1, Rs2
KMAXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then adds the two 32-bit results and 32-bit elements in a third register together. The addition result may be saturated.

- KMADA: $rd.W[x] + top*top + bottom*bottom$ (per 32-bit element)
- KMAXDA: $rd.W[x] + top*bottom + bottom*top$ (per 32-bit element)

Description :

For the `KMADA` instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMAXDA instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. The result is added to the content of 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The 32-bit results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
// KMADA
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]) + (Rs1.W[x].H[0] * Rs2.W[x].
↪H[0]);
// KMAXDA
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[0]) + (Rs1.W[x].H[0] * Rs2.W[x].
↪H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMAXDA (long t, unsigned long a, unsigned long b)

KMAXDA (SIMD Saturating Signed Crossed Multiply Two Halfs and Two Adds)

Type: SIMD

Syntax:

```
KMADA Rd, Rs1, Rs2
KMAXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then adds the two 32-bit results and 32-bit elements in a third register together. The addition result may be saturated.

- KMADA: $rd.W[x] + top*top + bottom*bottom$ (per 32-bit element)
- KMAXDA: $rd.W[x] + top*bottom + bottom*top$ (per 32-bit element)

Description :

For the `KMADA instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMAXDA instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. The result is added to the content of 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The 32-bit results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
// KMADA
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]) + (Rs1.W[x].H[0] * Rs2.W[x].
H[0]);
// KMAXDA
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[0]) + (Rs1.W[x].H[0] * Rs2.W[x].
H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMADS (long t, unsigned long a, unsigned long b)
```

KMADS (SIMD Saturating Signed Multiply Two Halfs & Subtract & Add)

Type: SIMD

Syntax:

```

KMADS Rd, Rs1, Rs2
KMADRS Rd, Rs1, Rs2
KMAXDS Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the corresponding 32-bit elements in a third register. The addition result may be saturated.

- KMADS: $rd.W[x] + (top*top - bottom*bottom)$ (per 32-bit element)
- KMADRS: $rd.W[x] + (bottom*bottom - top*top)$ (per 32-bit element)
- KMAXDS: $rd.W[x] + (top*bottom - bottom*top)$ (per 32-bit element)

Description :

For the KMADS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMADRS instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMAXDS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. The subtraction result is then added to the content of the corresponding 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The 32-bit results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

// KMADS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].
    ↪H[0]);
// KMADRS
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].
    ↪H[1]);
// KMAXDS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].
    ↪H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0

```

Parameters

- **t** – [in] long type of value stored in t

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMADRS (long t, unsigned long a, unsigned long b)

KMADRS (SIMD Saturating Signed Multiply Two Halfs & Reverse Subtract & Add)

Type: SIMD**Syntax:**

```
KMADS Rd, Rs1, Rs2
KMADRS Rd, Rs1, Rs2
KMAXDS Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the corresponding 32-bit elements in a third register. The addition result may be saturated.

- KMADS: $rd.W[x] + (top * top - bottom * bottom)$ (per 32-bit element)
- KMADRS: $rd.W[x] + (bottom * bottom - top * top)$ (per 32-bit element)
- KMAXDS: $rd.W[x] + (top * bottom - bottom * top)$ (per 32-bit element)

Description :

For the KMADS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMADRS instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMAXDS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. The subtraction result is then added to the content of the corresponding 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The 32-bit results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
// KMADS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].
    ↪H[0]);
// KMADRS
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].
    ↪H[1]);
// KMAXDS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].
    ↪H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
```

(continues on next page)

(continued from previous page)

```

    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMAXDS (long t, unsigned long a, unsigned long b)

KMAXDS (SIMD Saturating Signed Crossed Multiply Two Halfs & Subtract & Add)

Type: SIMD

Syntax:

```

KMADS Rd, Rs1, Rs2
KMADRS Rd, Rs1, Rs2
KMAXDS Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the corresponding 32-bit elements in a third register. The addition result may be saturated.

- KMADS: $rd.W[x] + (top*top - bottom*bottom)$ (per 32-bit element)
- KMADRS: $rd.W[x] + (bottom*bottom - top*top)$ (per 32-bit element)
- KMAXDS: $rd.W[x] + (top*bottom - bottom*top)$ (per 32-bit element)

Description :

For the KMADS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2. For the KMADRS instruction, it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. For the KMAXDS instruction, it multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2. The subtraction result is then added to the content of the corresponding 32-bit elements in Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The 32-bit results after saturation are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

// KMADS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].
    H[0]);
// KMADRS
res[x] = Rd.W[x] + (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].
    H[1]);
// KMAXDS
res[x] = Rd.W[x] + (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].
    H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMDA (unsigned long a, unsigned long b)

KMDA (SIMD Signed Multiply Two Halfs and Add)

Type: SIMD

Syntax:

```

KMDA Rd, Rs1, Rs2
KMXDA Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results together. The addition result may be saturated.

- KMDA: top*top + bottom*bottom (per 32-bit element)
- KMXDA: top*bottom + bottom*top (per 32-bit element)

Description :

For the KMDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the KMXDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top

16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{31}-1$. The final results are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```
if Rs1.W[x] != 0x80008000) or (Rs2.W[x] != 0x80008000 { // KMDA Rd.
    W[x] = Rs1.W[x].H[1] *
Rs2.W[x].H[1]) + (Rs1.W[x].H[0] * Rs2.W[x].H[0]; // KMXDA Rd.W[x] = Rs1.W[x].
    H[1] * Rs2.W[x].H[0])
+ (Rs1.W[x].H[0] * Rs2.W[x].H[1]; } else { Rd.W[x] = 0x7fffffff; OV
= 1; } for RV32: x=0 for RV64:
x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMXDA (unsigned long a, unsigned long b)

KMXDA (SIMD Signed Crossed Multiply Two Halfs and Add)

Type: SIMD

Syntax:

```
KMDA Rd, Rs1, Rs2
KMXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results together. The addition result may be saturated.

- KMDA: top*top + bottom*bottom (per 32-bit element)
- KMXDA: top*bottom + bottom*top (per 32-bit element)

Description :

For the KMDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the KMXDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{31}-1$. The final results are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```
if Rs1.W[x] != 0x80008000) or (Rs2.W[x] != 0x80008000 { // KMDA Rd.
    W[x] = Rs1.W[x].H[1] *
Rs2.W[x].H[1]) + (Rs1.W[x].H[0] * Rs2.W[x].H[0]; // KMXDA Rd.W[x] = Rs1.W[x].
    H[1] * Rs2.W[x].H[0])
```

(continues on next page)

(continued from previous page)

```
+ (Rs1.W[x].H[0] * Rs2.W[x].H[1]; } else { Rd.W[x] = 0xffffffff; OV
←= 1; } for RV32: x=0 for RV64:
x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMSDA (long t, unsigned long a, unsigned long b)
```

KMSDA (SIMD Saturating Signed Multiply Two Halfs & Add & Subtract)

Type: SIMD**Syntax:**

```
KMSDA Rd, Rs1, Rs2
KMSXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then subtracts the two 32-bit results from the corresponding 32-bit elements of a third register. The subtraction result may be saturated.

- KMSDA: $rd.W[x] - top*top - bottom*bottom$ (per 32-bit element)
- KMSXDA: $rd.W[x] - top*bottom - bottom*top$ (per 32-bit element)

Description :

For the KMSDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the KMSXDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The two 32-bit multiplication results are then subtracted from the content of the corresponding 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```
// KMSDA
res[x] = Rd.W[x] - (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].
←H[0]);
// KMSXDA
res[x] = Rd.W[x] - (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].
←H[1]);
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
```

(continues on next page)

(continued from previous page)

```

    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV32: x=0
for RV64: x=1..0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMSXDA (long t, unsigned long a, unsigned long b)

KMSXDA (SIMD Saturating Signed Crossed Multiply Two Halfs & Add & Subtract)

Type: SIMD

Syntax:

```

KMSDA Rd, Rs1, Rs2
KMSXDA Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then subtracts the two 32-bit results from the corresponding 32-bit elements of a third register. The subtraction result may be saturated.

- KMSDA: $rd.W[x] - top * top - bottom * bottom$ (per 32-bit element)
- KMSXDA: $rd.W[x] - top * bottom - bottom * top$ (per 32-bit element)

Description :

For the KMSDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the KMSXDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The two 32-bit multiplication results are then subtracted from the content of the corresponding 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```

// KMSDA
res[x] = Rd.W[x] - (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].
H[0]);
// KMSXDA
res[x] = Rd.W[x] - (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].

```

(continues on next page)

(continued from previous page)

```

    H[1]);
    if (res[x] > (2^31)-1) {
        res[x] = (2^31)-1;
        OV = 1;
    } else if (res[x] < -2^31) {
        res[x] = -2^31;
        OV = 1;
    }
    Rd.W[x] = res[x];
    for RV32: x=0
    for RV64: x=1..0

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMBB16 (unsigned long a, unsigned long b)

SMBB16 (SIMD Signed Multiply Bottom Half & Bottom Half)

Type: SIMD

Syntax:

```

SMBB16 Rd, Rs1, Rs2
SMBT16 Rd, Rs1, Rs2
SMTT16 Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- SMBB16: $W[x].bottom * W[x].bottom$
- SMBT16: $W[x].bottom * W[x].top$
- SMTT16: $W[x].top * W[x].top$

Description :

For the SMBB16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMBT16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMTT16 instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[0]; // SMBB16
Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[1]; // SMBT16
Rd.W[x] = Rs1.W[x].H[1] * Rs2.W[x].H[1]; // SMTT16
for RV32: x=0,
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMBT16 (unsigned long a, unsigned long b)

SMBT16 (SIMD Signed Multiply Bottom Half & Top Half)

Type: SIMD

Syntax:

```

SMBB16 Rd, Rs1, Rs2
SMBT16 Rd, Rs1, Rs2
SMTT16 Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- SMBB16: $W[x].bottom * W[x].bottom$
- SMBT16: $W[x].bottom * W[x].top$
- SMTT16: $W[x].top * W[x].top$

Description :

For the SMBB16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMBT16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMTT16 instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[0]; // SMBB16
Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[1]; // SMBT16
Rd.W[x] = Rs1.W[x].H[1] * Rs2.W[x].H[1]; // SMTT16
for RV32: x=0,
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMTT16 (unsigned long a, unsigned long b)

SMTT16 (SIMD Signed Multiply Top Half & Top Half)

Type: SIMD**Syntax:**

```

SMBB16 Rd, Rs1, Rs2
SMBT16 Rd, Rs1, Rs2
SMTT16 Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- SMBB16: $W[x].bottom * W[x].bottom$
- SMBT16: $W[x].bottom * W[x].top$
- SMTT16: $W[x].top * W[x].top$

Description :

For the SMBB16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMBT16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMTT16 instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[0]; // SMBB16
Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[1]; // SMBT16
Rd.W[x] = Rs1.W[x].H[1] * Rs2.W[x].H[1]; // SMTT16
for RV32: x=0,
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMDS (unsigned long a, unsigned long b)

SMDS (SIMD Signed Multiply Two Halfs and Subtract)

Type: SIMD**Syntax:**

```

SMDS Rd, Rs1, Rs2
SMDRS Rd, Rs1, Rs2
SMXDS Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results.

- SMDS: top*top - bottom*bottom (per 32-bit element)
- SMDRS: bottom*bottom - top*top (per 32-bit element)
- SMXDS: top*bottom - bottom*top (per 32-bit element)

Description :

For the SMDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction result is written to the corresponding 32-bit element of Rd. The 16-bit contents of multiplication are treated as signed integers.

Operations:

```

* SMDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].H[0]);
* SMDRS:
Rd.W[x] = (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].H[1]);
* SMXDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].H[1]);

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_SMDRS (unsigned long a, unsigned long b)
```

SMDRS (SIMD Signed Multiply Two Halfs and Reverse Subtract)

Type: SIMD

Syntax:

```

SMDS Rd, Rs1, Rs2
SMDRS Rd, Rs1, Rs2
SMXDS Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results.

- SMDS: top*top - bottom*bottom (per 32-bit element)
- SMDRS: bottom*bottom - top*top (per 32-bit element)
- SMXDS: top*bottom - bottom*top (per 32-bit element)

Description :

For the SMDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction result is written to the corresponding 32-bit element of Rd. The 16-bit contents of multiplication are treated as signed integers.

Operations:

```
* SMDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].H[0]);
* SMDRS:
Rd.W[x] = (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].H[1]);
* SMXDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].H[1]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMXDS (unsigned long a, unsigned long b)

SMXDS (SIMD Signed Crossed Multiply Two Halfs and Subtract)

Type: SIMD

Syntax:

```
SMDS Rd, Rs1, Rs2
SMDRS Rd, Rs1, Rs2
SMXDS Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results.

- SMDS: top*top - bottom*bottom (per 32-bit element)

- SMDRS: bottom*bottom - top*top (per 32-bit element)
- SMXDS: top*bottom - bottom*top (per 32-bit element)

Description :

For the SMDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction result is written to the corresponding 32-bit element of Rd. The 16-bit contents of multiplication are treated as signed integers.

Operations:

```
* SMDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[1]) - (Rs1.W[x].H[0] * Rs2.W[x].H[0]);
* SMDRS:
Rd.W[x] = (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].H[1]);
* SMXDS:
Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].H[1]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

Partial-SIMD Miscellaneous Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_CLRS32 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CLO32 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CLZ32 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PBSAD (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PBSADA (unsigned long t, unsigned long a,
unsigned long b)
```

```
__RV_SCLIP32(a, b)
```

```
__RV_UCLIP32(a, b)
```

group **Partial-SIMD Miscellaneous Instructions**

Partial-SIMD Miscellaneous Instructions.

there are 7 Partial-SIMD Miscellaneous Instructions

Defines

__RV_SCLIP32(a, b)

SCLIP32 (SIMD 32-bit Signed Clip Value)

Type: DSP

Syntax:

```
SCLIP32 Rd, Rs1, imm5u[4:0]
```

Purpose :

Limit the 32-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 32-bit signed integer elements stored in Rs1 into a signed integer range between $2^{\text{imm5u}}-1$ and -2^{imm5u} , and writes the limited results to Rd. For example, if imm5u is 3, the 32-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.W[x];
if (src > (2^imm5u)-1) {
    src = (2^imm5u)-1;
    OV = 1;
} else if (src < -2^imm5u) {
    src = -2^imm5u;
    OV = 1;
}
Rd.W[x] = src
for RV32: x=0,
for RV64: x=1...0
```

Parameters

- **a** – [in] long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

__RV_UCLIP32(a, b)

UCLIP32 (SIMD 32-bit Unsigned Clip Value)

Type: SIMD

Syntax:

```
UCLIP32 Rd, Rs1, imm5u[4:0]
```


Purpose :

Limit the 32-bit signed integer elements of a register into an unsigned range simultaneously.

Description :

This instruction limits the 32-bit signed integer elements stored in Rs1 into an unsigned integer range between $2^{\text{imm5u}}-1$ and 0, and writes the limited results to Rd. For example, if imm5u is 3, the 32-bit input values should be saturated between 7 and 0. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.W[x];
if (src > (2^imm5u)-1) {
    src = (2^imm5u)-1;
    OV = 1;
} else if (src < 0) {
    src = 0;
    OV = 1;
}
Rd.W[x] = src
for RV32: x=0,
for RV64: x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_CLRS32 (unsigned long a)

CLRS32 (SIMD 32-bit Count Leading Redundant Sign)

Type: SIMD

Syntax:

CLRS32 Rd, Rs1

Purpose :

Count the number of redundant sign bits of the 32-bit elements of a general register.

Description :

Starting from the bits next to the sign bits of the 32-bit elements of Rs1, this instruction counts the number of redundant sign bits and writes the result to the corresponding 32-bit elements of Rd.

Operations:

```
snum[x] = Rs1.W[x];
cnt[x] = 0;
for (i = 30 to 0) {
```

(continues on next page)

(continued from previous page)

```

    if (snum[x](i) == snum[x](31)) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.W[x] = cnt[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLO32 (unsigned long a)

CLO32 (SIMD 32-bit Count Leading One)

Type: SIMD

Syntax:

CLO32 Rd, Rs1

Purpose :

Count the number of leading one bits of the 32-bit elements of a general register.

Description :

Starting from the most significant bits of the 32-bit elements of Rs1, this instruction counts the number of leading one bits and writes the results to the corresponding 32-bit elements of Rd.

Operations:

```

snum[x] = Rs1.W[x];
cnt[x] = 0;
for (i = 31 to 0) {
    if (snum[x](i) == 1) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.W[x] = cnt[x];
for RV32: x=0
for RV64: x=1...0

```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CLZ32 (unsigned long a)

CLZ32 (SIMD 32-bit Count Leading Zero)

Type: SIMD

Syntax:

CLZ32 Rd, Rs1

Purpose :

Count the number of leading zero bits of the 32-bit elements of a general register.

Description :

Starting from the most significant bits of the 32-bit elements of Rs1, this instruction counts the number of leading zero bits and writes the results to the corresponding 32-bit elements of Rd.

Operations:

```
snum[x] = Rs1.W[x];
cnt[x] = 0;
for (i = 31 to 0) {
    if (snum[x](i) == 0) {
        cnt[x] = cnt[x] + 1;
    } else {
        break;
    }
}
Rd.W[x] = cnt[x];
for RV32: x=0
for RV64: x=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PBSAD (unsigned long a, unsigned long b)

PBSAD (Parallel Byte Sum of Absolute Difference)

Type: DSP

Syntax:

PBSAD Rd, Rs1, Rs2

Purpose :

Calculate the sum of absolute difference of unsigned 8-bit data elements.

Description :

This instruction subtracts the un-signed 8-bit elements of Rs2 from those of Rs1. Then it adds the absolute value of each difference together and writes the result to Rd.

Operations:

```
absdiff[x] = ABS(Rs1.B[x] - Rs2.B[x]);
Rd = SUM(absdiff[x]);
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **a** – **[in]** unsigned long type of value stored in a
- **b** – **[in]** unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PBSADA (unsigned long t, unsigned long a, unsigned long b)

PBSADA (Parallel Byte Sum of Absolute Difference Accum)

Type: DSP

Syntax:

```
PBSADA Rd, Rs1, Rs2
```

Purpose :

Calculate the sum of absolute difference of four unsigned 8-bit data elements and accumulate it into a register.

Description :

This instruction subtracts the un-signed 8-bit elements of Rs2 from those of Rs1. It then adds the absolute value of each difference together along with the content of Rd and writes the accumulated result back to Rd.

Operations:

```
absdiff[x] = ABS(Rs1.B[x] - Rs2.B[x]);
Rd = Rd + SUM(absdiff[x]);
for RV32: x=3...0,
for RV64: x=7...0
```

Parameters

- **t** – **[in]** unsigned long type of value stored in t
- **a** – **[in]** unsigned long type of value stored in a
- **b** – **[in]** unsigned long type of value stored in b

Returns

value stored in unsigned long type

8-bit Multiply with 32-bit Add Instructions

```
__STATIC_FORCEINLINE long __RV_SMAQA (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMAQA_SU (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UMAQA (unsigned long t, unsigned long a,
unsigned long b)
```

group 8-bit Multiply with 32-bit Add Instructions

8-bit Multiply with 32-bit Add Instructions

there are 3 8-bit Multiply with 32-bit Add Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_SMAQA (long t, unsigned long a, unsigned long b)
```

SMAQA (Signed Multiply Four Bytes with 32-bit Adds)

Type: Partial-SIMD (Reduction)

Syntax:

```
SMAQA Rd, Rs1, Rs2
```

Purpose :

Do four signed 8-bit multiplications from 32-bit chunks of two registers; and then adds the four 16-bit results and the content of corresponding 32-bit chunks of a third register together.

Description :

This instruction multiplies the four signed 8-bit elements of 32-bit chunks of Rs1 with the four signed 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the signed content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```
res[x] = Rd.W[x] +
    (Rs1.W[x].B[3] s* Rs2.W[x].B[3]) + (Rs1.W[x].B[2] s* Rs2.W[x].B[2]) +
    (Rs1.W[x].B[1] s* Rs2.W[x].B[1]) + (Rs1.W[x].B[0] s* Rs2.W[x].B[0]);
Rd.W[x] = res[x];
for RV32: x=0,
for RV64: x=1,0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMAQA_SU (long t, unsigned long a, unsigned long b)

SMAQA.SU (Signed and Unsigned Multiply Four Bytes with 32-bit Adds)

Type: Partial-SIMD (Reduction)**Syntax:**

SMAQA.SU Rd, Rs1, Rs2

Purpose :

Do four signed x unsigned 8-bit multiplications from 32-bit chunks of two registers; and then adds the four 16-bit results and the content of corresponding 32-bit chunks of a third register together.

Description :

This instruction multiplies the four signed 8-bit elements of 32-bit chunks of Rs1 with the four unsigned 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the signed content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```
res[x] = Rd.W[x] +
  (Rs1.W[x].B[3] su* Rs2.W[x].B[3]) + (Rs1.W[x].B[2] su* Rs2.W[x].B[2]) +
  (Rs1.W[x].B[1] su* Rs2.W[x].B[1]) + (Rs1.W[x].B[0] su* Rs2.W[x].B[0]);
Rd.W[x] = res[x];
for RV32: x=0,
for RV64: x=1...0
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long __RV_UMAQA (unsigned long t, unsigned long a, unsigned long b)

UMAQA (Unsigned Multiply Four Bytes with 32- bit Adds)

Type: DSP**Syntax:**

UMAQA Rd, Rs1, Rs2

Purpose :

Do four unsigned 8-bit multiplications from 32-bit chunks of two registers; and then adds the four 16-bit results and the content of corresponding 32-bit chunks of a third register together.

Description :

This instruction multiplies the four unsigned 8-bit elements of 32-bit chunks of Rs1 with the four unsigned 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the unsigned content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```
res[x] = Rd.W[x] + (Rs1.W[x].B[3] u* Rs2.W[x].B[3]) +
                (Rs1.W[x].B[2] u* Rs2.W[x].B[2]) + (Rs1.W[x].B[1] u* Rs2.W[x].B[1]) +
                (Rs1.W[x].B[0] u* Rs2.W[x].B[0]);
Rd.W[x] = res[x];
for RV32: x=0,
for RV64: x=1..0
```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

group Partial-SIMD Data Processing Instructions

Partial-SIMD Data Processing Instructions.

64-bit Profile Instructions

64-bit Addition & Subtraction Instructions

```
__STATIC_FORCEINLINE unsigned long long __RV_ADD64 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE long long __RV_KADD64 (long long a, long long b)
```

```
__STATIC_FORCEINLINE long long __RV_KSUB64 (long long a, long long b)
```

```
__STATIC_FORCEINLINE long long __RV_RADD64 (long long a, long long b)
```

```
__STATIC_FORCEINLINE long long __RV_RSUB64 (long long a, long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_SUB64 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UKADD64 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UKSUB64 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_URADD64 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_URSUB64 (unsigned long long a,
unsigned long long b)
```

group 64-bit Addition & Subtraction Instructions

64-bit Addition & Subtraction Instructions

there are 10 64-bit Addition & Subtraction Instructions.

Functions

```
__STATIC_FORCEINLINE unsigned long long __RV_ADD64 (unsigned long long a,
unsigned long long b)
```

ADD64 (64-bit Addition)

Type: 64-bit Profile

Syntax:

```
ADD64 Rd, Rs1, Rs2
```

Purpose :

Add two 64-bit signed or unsigned integers.

RV32 Description :

This instruction adds the 64-bit integer of an even/odd pair of registers specified by Rs1(4,1) with the 64-bit integer of an even/odd pair of registers specified by Rs2(4,1), and then writes the 64-bit result to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., value d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction has the same behavior as the ADD instruction in RV64I.

Note :

This instruction can be used for either signed or unsigned addition.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
a_L = CONCAT(Rs1(4,1), 1'b0); a_H = CONCAT(Rs1(4,1), 1'b1);
b_L = CONCAT(Rs2(4,1), 1'b0); b_H = CONCAT(Rs2(4,1), 1'b1);
R[t_H].R[t_L] = R[a_H].R[a_L] + R[b_H].R[b_L];
RV64:
Rd = Rs1 + Rs2;
```


Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE long long __RV_KADD64 (long long a, long long b)

KADD64 (64-bit Signed Saturating Addition)

Type: DSP (64-bit Profile)

Syntax:

KADD64 Rd, Rs1, Rs2

Purpose :

Add two 64-bit signed integers. The result is saturated to the Q63 range.

RV32 Description :

This instruction adds the 64-bit signed integer of an even/odd pair of registers specified by Rs1(4,1) with the 64-bit signed integer of an even/odd pair of registers specified by Rs2(4,1). If the 64-bit result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., value d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction adds the 64-bit signed integer in Rs1 with the 64-bit signed integer in Rs2. If the result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written to Rd.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
a_L = CONCAT(Rs1(4,1), 1'b0); a_H = CONCAT(Rs1(4,1), 1'b1);
b_L = CONCAT(Rs2(4,1), 1'b0); b_H = CONCAT(Rs2(4,1), 1'b1);
result = R[a_H].R[a_L] + R[b_H].R[b_L];
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
R[t_H].R[t_L] = result;
RV64:
result = Rs1 + Rs2;
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
Rd = result;
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_KSUB64 (long long a, long long b)

KSUB64 (64-bit Signed Saturating Subtraction)

Type: DSP (64-bit Profile)

Syntax:

KSUB64 Rd, Rs1, Rs2

Purpose :

Perform a 64-bit signed integer subtraction. The result is saturated to the Q63 range.

RV32 Description :

This instruction subtracts the 64-bit signed integer of an even/odd pair of registers specified by Rs2(4,1) from the 64-bit signed integer of an even/odd pair of registers specified by Rs1(4,1). If the 64-bit result is beyond the Q63 number range ($-2^{63} \leq \text{Q63} \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is then written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

This instruction subtracts the 64-bit signed integer of Rs2 from the 64-bit signed integer of Rs1. If the 64-bit result is beyond the Q63 number range ($-2^{63} \leq \text{Q63} \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is then written to Rd.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
a_L = CONCAT(Rs1(4,1), 1'b0); a_H = CONCAT(Rs1(4,1), 1'b1);
b_L = CONCAT(Rs2(4,1), 1'b0); b_H = CONCAT(Rs2(4,1), 1'b1);
result = R[a_H].R[a_L] - R[b_H].R[b_L];
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
R[t_H].R[t_L] = result;
RV64:
result = Rs1 - Rs2;
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
Rd = result;
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_RADD64 (long long a, long long b)

RADD64 (64-bit Signed Halving Addition)

Type: DSP (64-bit Profile)

Syntax:

RADD64 Rd, Rs1, Rs2

Purpose :

Add two 64-bit signed integers. The result is halved to avoid overflow or saturation.

RV32 Description :

This instruction adds the 64-bit signed integer of an even/odd pair of registers specified by Rs1(4,1) with the 64-bit signed integer of an even/odd pair of registers specified by Rs2(4,1). The 64-bit addition result is first arithmetically right-shifted by 1 bit and then written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., value d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction adds the 64-bit signed integer in Rs1 with the 64-bit signed integer in Rs2. The 64-bit addition result is first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
a_L = CONCAT(Rs1(4,1), 1'b0); a_H = CONCAT(Rs1(4,1), 1'b1);
b_L = CONCAT(Rs2(4,1), 1'b0); b_H = CONCAT(Rs2(4,1), 1'b1);
R[t_H].R[t_L] = (R[a_H].R[a_L] + R[b_H].R[b_L]) s>> 1;
RV64:
Rd = (Rs1 + Rs2) s>> 1;
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_RSUB64 (long long a, long long b)

RSUB64 (64-bit Signed Halving Subtraction)

Type: DSP (64-bit Profile)

Syntax:

RSUB64 Rd, Rs1, Rs2

Purpose :

Perform a 64-bit signed integer subtraction. The result is halved to avoid overflow or saturation.

RV32 Description :

This instruction subtracts the 64-bit signed integer of an even/odd pair of registers specified by Rb(4,1) from the 64-bit signed integer of an even/odd pair of registers specified by Ra(4,1). The subtraction result is first arithmetically right-shifted by 1 bit and then written to an even/odd pair of registers specified by Rt(4,1). Rx(4,1), i.e., value d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction subtracts the 64-bit signed integer in Rs2 from the 64-bit signed integer in Rs1. The 64-bit subtraction result is first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

RV32:

```
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
a_L = CONCAT(Rs1(4,1), 1'b0); a_H = CONCAT(Rs1(4,1), 1'b1);
b_L = CONCAT(Rs2(4,1), 1'b0); b_H = CONCAT(Rs2(4,1), 1'b1);
R[t_H].R[t_L] = (R[a_H].R[a_L] - R[b_H].R[b_L]) s>> 1;
```

RV64:

```
Rd = (Rs1 - Rs2) s>> 1;
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE unsigned long long __RV_SUB64 (unsigned long long a, unsigned long long b)

SUB64 (64-bit Subtraction)

Type: DSP (64-bit Profile)

Syntax:

SUB64 Rd, Rs1, Rs2

Purpose :

Perform a 64-bit signed or unsigned integer subtraction.

RV32 Description :

This instruction subtracts the 64-bit integer of an even/odd pair of registers specified by Rs2(4,1) from the 64-bit integer of an even/odd pair of registers specified by Rs1(4,1), and then writes the 64-bit result to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair

contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

This instruction subtracts the 64-bit integer of Rs2 from the 64-bit integer of Rs1, and then writes the 64-bit result to Rd.

Note :

This instruction can be used for either signed or unsigned subtraction.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
a_L = CONCAT(Rs1(4,1),1'b0); a_H = CONCAT(Rs1(4,1),1'b1);
b_L = CONCAT(Rs2(4,1),1'b0); b_H = CONCAT(Rs2(4,1),1'b1);
R[t_H].R[t_L] = R[a_H].R[a_L] - R[b_H].R[b_L];
* RV64:
Rd = Rs1 - Rs2;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UKADD64 (unsigned long long a, unsigned long long b)

UKADD64 (64-bit Unsigned Saturating Addition)

Type: DSP (64-bit Profile)

Syntax:

```
UKADD64 Rd, Rs1, Rs2
```

Purpose :

Add two 64-bit unsigned integers. The result is saturated to the U64 range.

RV32 Description :

This instruction adds the 64-bit unsigned integer of an even/odd pair of registers specified by Rs1(4,1) with the 64-bit unsigned integer of an even/odd pair of registers specified by Rs2(4,1). If the 64-bit result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction adds the 64-bit unsigned integer in Rs1 with the 64-bit unsigned integer in Rs2. If the 64-bit result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written to Rd.

Operations:

```

* RV32:
t_L = CONCAT(Rt(4,1),1'b0); t_H = CONCAT(Rt(4,1),1'b1);
a_L = CONCAT(Ra(4,1),1'b0); a_H = CONCAT(Ra(4,1),1'b1);
b_L = CONCAT(Rb(4,1),1'b0); b_H = CONCAT(Rb(4,1),1'b1);
result = R[a_H].R[a_L] + R[b_H].R[b_L];
if (result > (2^64)-1) {
    result = (2^64)-1; OV = 1;
}
R[t_H].R[t_L] = result;
* RV64:
result = Rs1 + Rs2;
if (result > (2^64)-1) {
    result = (2^64)-1; OV = 1;
}
Rd = result;

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UKSUB64 (unsigned long long a, unsigned long long b)

UKSUB64 (64-bit Unsigned Saturating Subtraction)

Type: DSP (64-bit Profile)

Syntax:

UKSUB64 Rd, Rs1, Rs2

Purpose :

Perform a 64-bit signed integer subtraction. The result is saturated to the U64 range.

RV32 Description :

This instruction subtracts the 64-bit unsigned integer of an even/odd pair of registers specified by Rs2(4,1) from the 64-bit unsigned integer of an even/odd pair of registers specified by Rs1(4,1). If the 64-bit result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is then written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

This instruction subtracts the 64-bit unsigned integer of Rs2 from the 64-bit unsigned integer of an even/odd pair of Rs1. If the 64-bit result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is then written to Rd.

Operations:

```

* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
a_L = CONCAT(Rs1(4,1),1'b0); a_H = CONCAT(Rs1(4,1),1'b1);
b_L = CONCAT(Rs2(4,1),1'b0); b_H = CONCAT(Rs2(4,1),1'b1);
result = R[a_H].R[a_L] - R[b_H].R[b_L];
if (result < 0) {
    result = 0; OV = 1;
}
R[t_H].R[t_L] = result;
* RV64
result = Rs1 - Rs2;
if (result < 0) {
    result = 0; OV = 1;
}
Rd = result;

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_URADD64 (unsigned long long a, unsigned long long b)

URADD64 (64-bit Unsigned Halving Addition)

Type: DSP (64-bit Profile)

Syntax:

```
URADD64 Rd, Rs1, Rs2
```

Purpose :

Add two 64-bit unsigned integers. The result is halved to avoid overflow or saturation.

RV32 Description :

This instruction adds the 64-bit unsigned integer of an even/odd pair of registers specified by Rs1(4,1) with the 64-bit unsigned integer of an even/odd pair of registers specified by Rs2(4,1). The 64-bit addition result is first logically right-shifted by 1 bit and then written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction adds the 64-bit unsigned integer in Rs1 with the 64-bit unsigned integer Rs2. The 64-bit addition result is first logically right-shifted by 1 bit and then written to Rd.

Operations:

```

* RV32:
t_L = CONCAT(Rt(4,1),1'b0); t_H = CONCAT(Rt(4,1),1'b1);
a_L = CONCAT(Ra(4,1),1'b0); a_H = CONCAT(Ra(4,1),1'b1);

```

(continues on next page)

(continued from previous page)

```

b_L = CONCAT(Rb(4,1),1'b0); b_H = CONCAT(Rb(4,1),1'b1);
R[t_H].R[t_L] = (R[a_H].R[a_L] + R[b_H].R[b_L]) u>> 1;
* RV64:
Rd = (Rs1 + Rs2) u>> 1;

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_URSUB64 (unsigned long long a, unsigned long long b)

URSUB64 (64-bit Unsigned Halving Subtraction)

Type: DSP (64-bit Profile)

Syntax:

```
URSUB64 Rd, Rs1, Rs2
```

Purpose :

Perform a 64-bit unsigned integer subtraction. The result is halved to avoid overflow or saturation.

RV32 Description :

This instruction subtracts the 64-bit unsigned integer of an even/odd pair of registers specified by Rs2(4,1) from the 64-bit unsigned integer of an even/odd pair of registers specified by Rs1(4,1). The subtraction result is first logically right-shifted by 1 bit and then written to an even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction subtracts the 64-bit unsigned integer in Rs2 from the 64-bit unsigned integer in Rs1. The subtraction result is first logically right-shifted by 1 bit and then written to Rd.

Operations:

```

* RV32:
t_L = CONCAT(Rt(4,1),1'b0); t_H = CONCAT(Rt(4,1),1'b1);
a_L = CONCAT(Ra(4,1),1'b0); a_H = CONCAT(Ra(4,1),1'b1);
b_L = CONCAT(Rb(4,1),1'b0); b_H = CONCAT(Rb(4,1),1'b1);
R[t_H].R[t_L] = (R[a_H].R[a_L] - R[b_H].R[b_L]) u>> 1;
* RV64:
Rd = (Rs1 - Rs2) u>> 1;

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

32-bit Multiply with 64-bit Add/Subtract Instructions

```
__STATIC_FORCEINLINE long long __RV_KMAR64 (long long t, long a, long b)
```

```
__STATIC_FORCEINLINE long long __RV_KMSR64 (long long t, long a, long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMAR64 (long long t, long a, long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMSR64 (long long t, long a, long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UKMAR64 (unsigned long long t,
unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UKMSR64 (unsigned long long t,
unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UMAR64 (unsigned long long t,
unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_UMSR64 (unsigned long long t,
unsigned long a, unsigned long b)
```

group **32-bit Multiply with 64-bit Add/Subtract Instructions**

32-bit Multiply with 64-bit Add/Subtract Instructions

there are 32-bit Multiply 64-bit Add/Subtract Instructions

Functions

```
__STATIC_FORCEINLINE long long __RV_KMAR64 (long long t, long a, long b)
```

KMAR64 (Signed Multiply and Saturating Add to 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

```
KMAR64 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit signed elements in two registers and add the 64-bit multiplication results to the 64-bit signed data of a pair of registers (RV32) or a register (RV64). The result is saturated to the Q63 range and written back to the pair of registers (RV32) or the register (RV64).

RV32 Description :

This instruction multiplies the 32-bit signed data of Rs1 with that of Rs2. It adds the 64-bit multiplication result to the 64-bit signed data of an even/odd pair of registers specified by Rd(4,1) with unlimited precision. If the 64-bit addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., value d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit signed elements of Rs1 with that of Rs2. It adds the 64-bit multiplication results to the 64-bit signed data of Rd with unlimited precision. If the 64-bit addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to Rd.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
result = R[t_H].R[t_L] + (Rs1 * Rs2);
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
R[t_H].R[t_L] = result;
RV64:
// `result` has unlimited precision
result = Rd + (Rs1.W[0] * Rs2.W[0]) + (Rs1.W[1] * Rs2.W[1]);
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
Rd = result;
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_KMSR64 (long long t, long a, long b)

KMSR64 (Signed Multiply and Saturating Subtract from 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

KMSR64 Rd, Rs1, Rs2

Purpose :

Multiply the 32-bit signed elements in two registers and subtract the 64-bit multiplication results from the 64-bit signed data of a pair of registers (RV32) or a register (RV64). The result is saturated to the Q63 range and written back to the pair of registers (RV32) or the register (RV64).

RV32 Description :

This instruction multiplies the 32-bit signed data of Rs1 with that of Rs2. It subtracts the 64-bit multiplication result from the 64-bit signed data of an even/odd pair of registers specified by Rd(4,1) with unlimited precision. If the 64-bit subtraction result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit signed elements of Rs1 with that of Rs2. It subtracts the 64-bit multiplication results from the 64-bit signed data in Rd with unlimited precision. If the 64-bit subtraction result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to Rd.

Operations:

```
RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
result = R[t_H].R[t_L] - (Rs1 * Rs2);
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
R[t_H].R[t_L] = result;
RV64:
// `result` has unlimited precision
result = Rd - (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]);
if (result > (2^63)-1) {
    result = (2^63)-1; OV = 1;
} else if (result < -2^63) {
    result = -2^63; OV = 1;
}
Rd = result;
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMAR64 (long long t, long a, long b)

SMAR64 (Signed Multiply and Add to 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

SMAR64 Rd, Rs1, Rs2

Purpose :

Multiply the 32-bit signed elements in two registers and add the 64-bit multiplication result to the 64-bit signed data of a pair of registers (RV32) or a register (RV64). The result is written back to the pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the 32-bit signed data of Rs1 with that of Rs2. It adds the 64-bit multiplication result to the 64-bit signed data of an even/odd pair of registers specified by Rd(4,1). The addition result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit signed elements of Rs1 with that of Rs2. It adds the 64-bit multiplication results to the 64-bit signed data of Rd. The addition result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
R[t_H].R[t_L] = R[t_H].R[t_L] + (Rs1 * Rs2);
* RV64:
Rd = Rd + (Rs1.W[0] * Rs2.W[0]) + (Rs1.W[1] * Rs2.W[1]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMSR64 (long long t, long a, long b)

SMSR64 (Signed Multiply and Subtract from 64- Bit Data)

Type: DSP (64-bit Profile)

Syntax:

SMSR64 Rd, Rs1, Rs2

Purpose :

Multiply the 32-bit signed elements in two registers and subtract the 64-bit multiplication results from the 64-bit signed data of a pair of registers (RV32) or a register (RV64). The result is written back to the pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the 32-bit signed data of Rs1 with that of Rs2. It subtracts the 64-bit multiplication result from the 64-bit signed data of an even/odd pair of registers specified by Rd(4,1). The subtraction

result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit signed elements of Rs1 with that of Rs2. It subtracts the 64-bit multiplication results from the 64-bit signed data of Rd. The subtraction result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1), 1'b0); t_H = CONCAT(Rd(4,1), 1'b1);
R[t_H].R[t_L] = R[t_H].R[t_L] - (Rs1 * Rs2);
* RV64:
Rd = Rd - (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] long type of value stored in a
- **b** – [in] long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE unsigned long long __RV_UKMAR64 (unsigned long long t, unsigned long a, unsigned long b)

UKMAR64 (Unsigned Multiply and Saturating Add to 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

```
UKMAR64 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit unsigned elements in two registers and add the 64-bit multiplication results to the 64-bit unsigned data of a pair of registers (RV32) or a register (RV64). The result is saturated to the U64 range and written back to the pair of registers (RV32) or the register (RV64).

RV32 Description :

This instruction multiplies the 32-bit unsigned data of Rs1 with that of Rs2. It adds the 64-bit multiplication result to the 64-bit unsigned data of an even/odd pair of registers specified by Rd(4,1) with unlimited precision. If the 64-bit addition result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit unsigned elements of Rs1 with that of Rs2. It adds the 64-bit multiplication results to the 64-bit unsigned data in Rd with unlimited precision. If the 64-bit addition result is

beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
result = R[t_H].R[t_L] + (Rs1 * Rs2);
if (result > (2^64)-1) {
    result = (2^64)-1; OV = 1;
}
R[t_H].R[t_L] = result;
* RV64:
// `result` has unlimited precision
result = Rd + (Rs1.W[0] u* Rs2.W[0]) + (Rs1.W[1] u* Rs2.W[1]);
if (result > (2^64)-1) {
    result = (2^64)-1; OV = 1;
}
Rd = result;
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UKMSR64 (unsigned long long t, unsigned long a, unsigned long b)

UKMSR64 (Unsigned Multiply and Saturating Subtract from 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

```
UKMSR64 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit unsigned elements in two registers and subtract the 64-bit multiplication results from the 64-bit unsigned data of a pair of registers (RV32) or a register (RV64). The result is saturated to the U64 range and written back to the pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the 32-bit unsigned data of Rs1 with that of Rs2. It subtracts the 64-bit multiplication result from the 64-bit unsigned data of an even/odd pair of registers specified by Rd(4,1) with unlimited precision. If the 64-bit subtraction result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit unsigned elements of Rs1 with that of Rs2. It subtracts the 64-bit multiplication results from the 64-bit unsigned data of Rd with unlimited precision. If the 64-bit subtraction result is beyond the U64 number range ($0 \leq U64 \leq 2^{64}-1$), it is saturated to the range and the OV bit is set to 1. The saturated result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
result = R[t_H].R[t_L] - (Rs1 u* Rs2);
if (result < 0) {
    result = 0; OV = 1;
}
R[t_H].R[t_L] = result;
* RV64:
// `result` has unlimited precision
result = Rd - (Rs1.W[0] u* Rs2.W[0]) - (Rs1.W[1] u* Rs2.W[1]);
if (result < 0) {
    result = 0; OV = 1;
}
Rd = result;
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMAR64 (unsigned long long t, unsigned long a, unsigned long b)

UMAR64 (Unsigned Multiply and Add to 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

```
UMAR64 Rd, Rs1, Rs2
```

Purpose :

Multiply the 32-bit unsigned elements in two registers and add the 64-bit multiplication results to the 64-bit unsigned data of a pair of registers (RV32) or a register (RV64). The result is written back to the pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the 32-bit unsigned data of Rs1 with that of Rs2. It adds the 64-bit multiplication result to the 64-bit unsigned data of an even/odd pair of registers specified by Rd(4,1). The addition result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit unsigned elements of Rs1 with that of Rs2. It adds the 64-bit multiplication results to the 64-bit unsigned data of Rd. The addition result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].R[t_L] = R[t_H].R[t_L] + (Rs1 * Rs2);
* RV64:
Rd = Rd + (Rs1.W[0] u* Rs2.W[0]) + (Rs1.W[1] u* Rs2.W[1]);
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_UMSR64 (unsigned long long t, unsigned long a, unsigned long b)

UMSR64 (Unsigned Multiply and Subtract from 64-Bit Data)

Type: DSP (64-bit Profile)

Syntax:

UMSR64 Rd, Rs1, Rs2

Purpose :

Multiply the 32-bit unsigned elements in two registers and subtract the 64-bit multiplication results from the 64-bit unsigned data of a pair of registers (RV32) or a register (RV64). The result is written back to the pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the 32-bit unsigned data of Rs1 with that of Rs2. It subtracts the 64-bit multiplication result from the 64-bit unsigned data of an even/odd pair of registers specified by Rd(4,1). The subtraction result is written back to the even/odd pair of registers specified by Rd(4,1). Rx(4,1), i.e., d, determines the even/odd pair group of two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

This instruction multiplies the 32-bit unsigned elements of Rs1 with that of Rs2. It subtracts the 64-bit multiplication results from the 64-bit unsigned data of Rd. The subtraction result is written back to Rd.

Operations:

```
* RV32:
t_L = CONCAT(Rd(4,1),1'b0); t_H = CONCAT(Rd(4,1),1'b1);
R[t_H].R[t_L] = R[t_H].R[t_L] - (Rs1 * Rs2);
* RV64:
Rd = Rd - (Rs1.W[0] u* Rs2.W[0]) - (Rs1.W[1] u* Rs2.W[1]);
```


Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

Signed 16-bit Multiply 64-bit Add/Subtract Instructions

```
__STATIC_FORCEINLINE long long __RV_SMAL (long long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALBB (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALBT (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALTT (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALDA (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALXDA (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALDS (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALDRS (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMALXDS (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMSLDA (long long t, unsigned long a,
unsigned long b)
```

```
__STATIC_FORCEINLINE long long __RV_SMSLXDA (long long t, unsigned long a,
unsigned long b)
```

group **Signed 16-bit Multiply 64-bit Add/Subtract Instructions**

Signed 16-bit Multiply 64-bit Add/Subtract Instructions.

Signed 16-bit Multiply with 64-bit Add/Subtract Instructions.

there is Signed 16-bit Multiply 64-bit Add/Subtract Instructions

there are 10 Signed 16-bit Multiply with 64-bit Add/Subtract Instructions

Functions

__STATIC_FORCEINLINE long long __RV_SMAL (long long a, unsigned long b)

SMAL (Signed Multiply Halfs & Add 64-bit)

Type: Partial-SIMD

Syntax:

SMAL Rd, Rs1, Rs2

Purpose :

Multiply the signed bottom 16-bit content of the 32-bit elements of a register with the top 16-bit content of the same 32-bit elements of the same register, and add the results with a 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to another even/odd pair of registers (RV32) or a register (RV64).

RV32 Description :

This instruction multiplies the bottom 16-bit content of the lower 32-bit of Rs2 with the top 16-bit content of the lower 32-bit of Rs2 and adds the result with the 64-bit value of an even/odd pair of registers specified by Rs1(4,1). The 64-bit addition result is written back to an even/odd pair of registers specified by Rd(4,1). The 16-bit values of Rs2, and the 64-bit value of the Rs1(4,1) register- pair are treated as signed integers. Rx(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

This instruction multiplies the bottom 16-bit content of the 32-bit elements of Rs2 with the top 16-bit content of the same 32-bit elements of Rs2 and adds the results with the 64-bit value of Rs1. The 64-bit addition result is written back to Rd. The 16-bit values of Rs2, and the 64-bit value of Rs1 are treated as signed integers.

Operations:

```
RV32:
Mres[31:0] = Rs2.H[1] * Rs2.H[0];
Idx0 = CONCAT(Rs1(4,1), 1'b0); Idx1 = CONCAT(Rs1(4,1), 1'b1); +
Idx2 = CONCAT(Rd(4,1), 1'b0); Idx3 = CONCAT(Rd(4,1), 1'b1);
R[Idx3].R[Idx2] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);
RV64:
Mres[0][31:0] = Rs2.W[0].H[1] * Rs2.W[0].H[0];
Mres[1][31:0] = Rs2.W[1].H[1] * Rs2.W[1].H[0];
Rd = Rs1 + SE64(Mres[1][31:0]) + SE64(Mres[0][31:0]);
```

Parameters

- **a** – [in] long long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_SMALBB (long long t, unsigned long a,
unsigned long b)
```

SMALBB (Signed Multiply Bottom Halves & Add 64-bit)

Type: DSP (64-bit Profile)**Syntax:**

```
SMALBB Rd, Rs1, Rs2
SMALBT Rd, Rs1, Rs2
SMALTT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to the register-pair (RV32) or the register (RV64).

- SMALBB: rt pair + bottom*bottom (all 32-bit elements)
- SMALBT rt pair + bottom*top (all 32-bit elements)
- SMALTT rt pair + top*top (all 32-bit elements)

RV32 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2. The multiplication result is added with the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
RV32:
Mres[31:0] = Rs1.H[0] * Rs2.H[0]; // SMALBB
Mres[31:0] = Rs1.H[0] * Rs2.H[1]; // SMALBT
Mres[31:0] = Rs1.H[1] * Rs2.H[1]; // SMALTT
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);
RV64:
```

(continues on next page)

(continued from previous page)

```

// SMALBB
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[0];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[0];
// SMALBT
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[1];
// SMALTT
Mres[0][31:0] = Rs1.W[0].H[1] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[1] * Rs2.W[1].H[1];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALBT (long long t, unsigned long a, unsigned long b)

SMALBT (Signed Multiply Bottom Half & Top Half & Add 64-bit)

Type: DSP (64-bit Profile)**Syntax:**

```

SMALBB Rd, Rs1, Rs2
SMALBT Rd, Rs1, Rs2
SMALTT Rd, Rs1, Rs2

```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to the register-pair (RV32) or the register (RV64).

- SMALBB: rt pair + bottom*bottom (all 32-bit elements)
- SMALBT rt pair + bottom*top (all 32-bit elements)
- SMALTT rt pair + top*top (all 32-bit elements)

RV32 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2. The multiplication result is added with the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
RV32:
Mres[31:0] = Rs1.H[0] * Rs2.H[0]; // SMALBB
Mres[31:0] = Rs1.H[0] * Rs2.H[1]; // SMALBT
Mres[31:0] = Rs1.H[1] * Rs2.H[1]; // SMALTT
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);
RV64:
// SMALBB
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[0];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[0];
// SMALBT
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[1];
// SMALTT
Mres[0][31:0] = Rs1.W[0].H[1] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[1] * Rs2.W[1].H[1];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALTT (long long t, unsigned long a, unsigned long b)

SMALTT (Signed Multiply Top Halfs & Add 64-bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMALBB Rd, Rs1, Rs2
SMALBT Rd, Rs1, Rs2
SMALTT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of

registers (RV32) or a register (RV64). The addition result is written back to the register-pair (RV32) or the register (RV64).

- SMALBB: $rt\ pair + bottom * bottom$ (all 32-bit elements)
- SMALBT $rt\ pair + bottom * top$ (all 32-bit elements)
- SMALTT $rt\ pair + top * top$ (all 32-bit elements)

RV32 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2. The multiplication result is added with the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALBB instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALBT instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALTT instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
RV32:
Mres[31:0] = Rs1.H[0] * Rs2.H[0]; // SMALBB
Mres[31:0] = Rs1.H[0] * Rs2.H[1]; // SMALBT
Mres[31:0] = Rs1.H[1] * Rs2.H[1]; // SMALTT
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);
RV64:
// SMALBB
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[0];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[0];
// SMALBT
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[1];
// SMALTT
Mres[0][31:0] = Rs1.W[0].H[1] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[1] * Rs2.W[1].H[1];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALDA (long long t, unsigned long a, unsigned long b)

SMALDA (Signed Multiply Two Halfs and Two Adds 64-bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMALDA Rd, Rs1, Rs2
SMALXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results and the 64-bit value of an even/odd pair of registers together.

- SMALDA: $rt\ pair + top * top + bottom * bottom$ (all 32-bit elements)
- SMALXDA: $rt\ pair + top * bottom + bottom * top$ (all 32-bit elements)

RV32 Description :

For the SMALDA instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision. For the SMALXDA instruction, it multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision. The result is added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 with unlimited precision. For the SMALXDA instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 with unlimited precision. The results are added to the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
RV32:
// SMALDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[0]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[1]);
// SMALXDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[1]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[0]);
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
```

(continues on next page)

(continued from previous page)

```

R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres0[31:0]) + SE64(Mres1[31:0]);
RV64:
// SMALDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]);
// SMALXDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[1]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[1]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]);
Rd = Rd + SE64(Mres0[0][31:0]) + SE64(Mres1[0][31:0]) + SE64(Mres0[1][31:0]) +
SE64(Mres1[1][31:0]);

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALXDA (long long t, unsigned long a, unsigned long b)

SMALXDA (Signed Crossed Multiply Two Halfs and Two Adds 64-bit)

Type: DSP (64-bit Profile)

Syntax:

```

SMALDA Rd, Rs1, Rs2
SMALXDA Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results and the 64-bit value of an even/odd pair of registers together.

- SMALDA: rt pair+ top*top + bottom*bottom (all 32-bit elements)
- SMALXDA: rt pair+ top*bottom + bottom*top (all 32-bit elements)

RV32 Description :

For the SMALDA instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision. For the SMALXDA instruction, it multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision. The result is added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 with unlimited precision. For the SMALXDA instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 with unlimited precision. The results are added to the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
RV32:
// SMALDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[0]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[1]);
// SMALXDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[1]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[0]);
Idx0 = CONCAT(Rd(4,1),1'b0); Idx1 = CONCAT(Rd(4,1),1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres0[31:0]) + SE64(Mres1[31:0]);
RV64:
// SMALDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]);
// SMALXDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[1]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[1]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]);
Rd = Rd + SE64(Mres0[0][31:0]) + SE64(Mres1[0][31:0]) + SE64(Mres0[1][31:0]) +
SE64(Mres1[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALDS (long long t, unsigned long a, unsigned long b)

SMALDS (Signed Multiply Two Halfs & Subtract & Add 64-bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMALDS Rd, Rs1, Rs2
SMALDRS Rd, Rs1, Rs2
SMALXDS Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to the register-pair.

- SMALDS: $rt\ pair + (top * top - bottom * bottom)$ (all 32-bit elements)
- SMALDRS: $rt\ pair + (bottom * bottom - top * top)$ (all 32-bit elements)
- SMALXDS: $rt\ pair + (top * bottom - bottom * top)$ (all 32-bit elements)

RV32 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2. The subtraction result is then added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction results are then added to the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
* RV32:
Mres[31:0] = (Rs1.H[1] * Rs2.H[1]) - (Rs1.H[0] * Rs2.H[0]); // SMALDS
Mres[31:0] = (Rs1.H[0] * Rs2.H[0]) - (Rs1.H[1] * Rs2.H[1]); // SMALDRS
Mres[31:0] = (Rs1.H[1] * Rs2.H[0]) - (Rs1.H[0] * Rs2.H[1]); // SMALXDS
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);
* RV64:
```

(continues on next page)

(continued from previous page)

```

// SMALDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]) - (Rs1.W[0].H[0] * Rs2.W[0].
↪H[0]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]) - (Rs1.W[1].H[0] * Rs2.W[1].
↪H[0]);
// SMALDRS
Mres[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]) - (Rs1.W[0].H[1] * Rs2.W[0].
↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]) - (Rs1.W[1].H[1] * Rs2.W[1].
↪H[1]);
// SMALXDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]) - (Rs1.W[0].H[0] * Rs2.W[0].
↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]) - (Rs1.W[1].H[0] * Rs2.W[1].
↪H[1]);
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALDRS (long long t, unsigned long a, unsigned long b)

SMALDRS (Signed Multiply Two Halfs & Reverse Subtract & Add 64- bit)

Type: DSP (64-bit Profile)

Syntax:

```

SMALDS Rd, Rs1, Rs2
SMALDRS Rd, Rs1, Rs2
SMALXDS Rd, Rs1, Rs2

```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to the register-pair.

- SMALDS: rt pair + (top*top - bottom*bottom) (all 32-bit elements)
- SMALDRS: rt pair + (bottom*bottom - top*top) (all 32-bit elements)
- SMALXDS: rt pair + (top*bottom - bottom*top) (all 32-bit elements)

RV32 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of Rs1 with

the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2. The subtraction result is then added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction results are then added to the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
* RV32:
Mres[31:0] = (Rs1.H[1] * Rs2.H[1]) - (Rs1.H[0] * Rs2.H[0]); // SMALDS
Mres[31:0] = (Rs1.H[0] * Rs2.H[0]) - (Rs1.H[1] * Rs2.H[1]); // SMALDRS
Mres[31:0] = (Rs1.H[1] * Rs2.H[0]) - (Rs1.H[0] * Rs2.H[1]); // SMALXDS
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);

* RV64:
// SMALDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]) - (Rs1.W[0].H[0] * Rs2.W[0].
↪H[0]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]) - (Rs1.W[1].H[0] * Rs2.W[1].
↪H[0]);
// SMALDRS
Mres[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]) - (Rs1.W[0].H[1] * Rs2.W[0].
↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]) - (Rs1.W[1].H[1] * Rs2.W[1].
↪H[1]);
// SMALXDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]) - (Rs1.W[0].H[0] * Rs2.W[0].
↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]) - (Rs1.W[1].H[0] * Rs2.W[1].
↪H[1]);
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMALXDS (long long t, unsigned long a, unsigned long b)

SMALXDS (Signed Crossed Multiply Two Halfs & Subtract & Add 64- bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMALDS Rd, Rs1, Rs2
SMALDRS Rd, Rs1, Rs2
SMALXDS Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The addition result is written back to the register-pair.

- SMALDS: $rt\ pair + (top * top - bottom * bottom)$ (all 32-bit elements)
- SMALDRS: $rt\ pair + (bottom * bottom - top * top)$ (all 32-bit elements)
- SMALXDS: $rt\ pair + (top * bottom - bottom * top)$ (all 32-bit elements)

RV32 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2. The subtraction result is then added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the operand and the even 2d register of the pair contains the low 32-bit of the operand.

RV64 Description :

For the SMALDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMALDRS instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. For the SMALXDS instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction results are then added to the 64-bit value of Rd. The 64-bit

addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
* RV32:
Mres[31:0] = (Rs1.H[1] * Rs2.H[1]) - (Rs1.H[0] * Rs2.H[0]); // SMALDS
Mres[31:0] = (Rs1.H[0] * Rs2.H[0]) - (Rs1.H[1] * Rs2.H[1]); // SMALDRS
Mres[31:0] = (Rs1.H[1] * Rs2.H[0]) - (Rs1.H[0] * Rs2.H[1]); // SMALXDS
Idx0 = CONCAT(Rd(4,1),1'b0); Idx1 = CONCAT(Rd(4,1),1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] + SE64(Mres[31:0]);

* RV64:
// SMALDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]) - (Rs1.W[0].H[0] * Rs2.W[0].
    ↪H[0]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]) - (Rs1.W[1].H[0] * Rs2.W[1].
    ↪H[0]);
// SMALDRS
Mres[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]) - (Rs1.W[0].H[1] * Rs2.W[0].
    ↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]) - (Rs1.W[1].H[1] * Rs2.W[1].
    ↪H[1]);
// SMALXDS
Mres[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]) - (Rs1.W[0].H[0] * Rs2.W[0].
    ↪H[1]);
Mres[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]) - (Rs1.W[1].H[0] * Rs2.W[1].
    ↪H[1]);
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMSLDA (long long t, unsigned long a, unsigned long b)

SMSLDA (Signed Multiply Two Halfs & Add & Subtract 64-bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMSLDA Rd, Rs1, Rs2
SMSLXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then subtracts the two 32-bit results from the 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The subtraction result is written back to the register-pair.

- SMSLDA: rd pair - top*top - bottom*bottom (all 32-bit elements)

- SMSLXDA: rd pair - top*bottom - bottom*top (all 32-bit elements)

RV32 Description :

For the SMSLDA instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content Rs2 and multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMSLXDA instruction, it multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2. The two multiplication results are subtracted from the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit subtraction result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

For the SMSLDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMSLXDA instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The four multiplication results are subtracted from the 64-bit value of Rd. The 64-bit subtraction result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
* RV32:
// SMSLDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[0]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[1]);
// SMSLXDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[1]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[0]);
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] - SE64(Mres0[31:0]) - SE64(Mres1[31:0]);
* RV64:
// SMSLDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]);
// SMSLXDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[1]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[1]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]);
Rd = Rd - SE64(Mres0[0][31:0]) - SE64(Mres1[0][31:0]) - SE64(Mres0[1][31:0]) -
SE64(Mres1[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_SMSLXDA (long long t, unsigned long a, unsigned long b)

SMSLXDA (Signed Crossed Multiply Two Halfs & Add & Subtract 64- bit)

Type: DSP (64-bit Profile)

Syntax:

```
SMSLDA Rd, Rs1, Rs2
SMSLXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then subtracts the two 32-bit results from the 64-bit value of an even/odd pair of registers (RV32) or a register (RV64). The subtraction result is written back to the register-pair.

- SMSLDA: rd pair - top*top - bottom*bottom (all 32-bit elements)
- SMSLXDA: rd pair - top*bottom - bottom*top (all 32-bit elements)

RV32 Description :

For the SMSLDA instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content Rs2 and multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2. For the SMSLXDA instruction, it multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2. The two multiplication results are subtracted from the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit subtraction result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 64-bit value of the register-pair are treated as signed integers. Rd(4,1), i.e., d, determines the even/odd pair group of the two registers. Specifically, the register pair includes register 2d and 2d+1. The odd 2d+1 register of the pair contains the high 32-bit of the result and the even 2d register of the pair contains the low 32-bit of the result.

RV64 Description :

For the SMSLDA instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. For the SMSLXDA instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The four multiplication results are subtracted from the 64-bit value of Rd. The 64-bit subtraction result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
* RV32:
// SMSLDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[0]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[1]);
// SMSLXDA
Mres0[31:0] = (Rs1.H[0] * Rs2.H[1]);
Mres1[31:0] = (Rs1.H[1] * Rs2.H[0]);
Idx0 = CONCAT(Rd(4,1), 1'b0); Idx1 = CONCAT(Rd(4,1), 1'b1);
R[Idx1].R[Idx0] = R[Idx1].R[Idx0] - SE64(Mres0[31:0]) - SE64(Mres1[31:0]);
```

(continues on next page)

(continued from previous page)

```

* RV64:
// SMSLDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]);
// SMSLXDA
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[1]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[1]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]);
Rd = Rd - SE64(Mres0[0][31:0]) - SE64(Mres1[0][31:0]) - SE64(Mres0[1][31:0]) -
SE64(Mres1[1][31:0]);

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long long type

group 64-bit Profile Instructions

64-bit Profile Instructions

RV64 Only Instructions**(RV64 Only) SIMD 32-bit Add/Subtract Instructions**

```
__STATIC_FORCEINLINE unsigned long __RV_ADD32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CRAS32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_CRSA32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KADD32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KCRAS32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KCRSA32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSTAS32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSTSA32 (unsigned long a, unsigned long b)
```

__STATIC_FORCEINLINE unsigned long __RV_KSUB32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RADD32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RCRAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RCRSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSTAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSTSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_RSUB32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_STAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_STSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_SUB32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKADD32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKCRAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKCRSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSTAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSTSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_UKSUB32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URADD32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URCRAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URCRSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSTAS32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSTSA32 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long __RV_URSUB32 (unsigned long a, unsigned long b)

group (RV64 Only) SIMD 32-bit Add/Subtract Instructions

(RV64 Only) SIMD 32-bit Add/Subtract Instructions

The following tables list instructions that are only present in RV64. There are 30 SIMD 32-bit addition or subtraction instructions. There are 4 SIMD16-bit Packing Instructions.

Functions

__STATIC_FORCEINLINE unsigned long __RV_ADD32 (unsigned long a, unsigned long b)

ADD32 (SIMD 32-bit Addition)

Type: SIMD (RV64 Only)

Syntax:

ADD32 Rd, Rs1, Rs2

Purpose :

Do 32-bit integer element additions simultaneously.

Description :

This instruction adds the 32-bit integer elements in Rs1 with the 32-bit integer elements in Rs2, and then writes the 32-bit element results to Rd.

Note :

This instruction can be used for either signed or unsigned addition.

Operations:

$Rd.W[x] = Rs1.W[x] + Rs2.W[x];$
for RV64: $x=1 \dots 0$

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CRAS32 (unsigned long a, unsigned long b)

CRAS32 (SIMD 32-bit Cross Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

CRAS32 Rd, Rs1, Rs2

Purpose :

Do 32-bit integer element addition and 32-bit integer element subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [31:0] of Rs2, and writes the result to [63:32] of Rd; at the same time, it subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [31:0] of Rs1, and writes the result to [31:0] of Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[1] = Rs1.W[1] + Rs2.W[0];
Rd.W[0] = Rs1.W[0] - Rs2.W[1];
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_CRSA32 (unsigned long a, unsigned long b)

CRSA32 (SIMD 32-bit Cross Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

```
CRSA32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit integer element subtraction and 32-bit integer element addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. *Description: * This instruction subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [63:32] of Rs1, and writes the result to [63:32] of Rd; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [63:32] of Rs2, and writes the result to [31:0] of Rd

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[1] = Rs1.W[1] - Rs2.W[0];
Rd.W[0] = Rs1.W[0] + Rs2.W[1];
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KADD32 (unsigned long a, unsigned long b)

KADD32 (SIMD 32-bit Signed Saturating Addition)

Type: SIMD (RV64 Only)

Syntax:

```
KADD32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] + Rs2.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KCRAS32 (unsigned long a, unsigned long b)

KCRAS32 (SIMD 32-bit Signed Saturating Cross Addition & Subtraction)

Type: SIM (RV64 Only)

Syntax:

```
KCRAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element saturating addition and 32-bit signed integer element saturating subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [31:0] of Rs2; at the same time, it subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [31:0] of Rs1. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```
res[1] = Rs1.W[1] + Rs2.W[0];
res[0] = Rs1.W[0] - Rs2.W[1];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];
for RV64, x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KCRSA32 (unsigned long a, unsigned long b)

KCRSA32 (SIMD 32-bit Signed Saturating Cross Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

KCRSA32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer element saturating subtraction and 32-bit signed integer element saturating addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. *Description: * This instruction subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [63:32] of Rs1; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [63:32] of Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```
res[1] = Rs1.W[1] - Rs2.W[0];
res[0] = Rs1.W[0] + Rs2.W[1];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res < -2^31) {
```

(continues on next page)

(continued from previous page)

```

    res[x] = -231;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSTAS32 (unsigned long a, unsigned long b)

KSTAS32 (SIMD 32-bit Signed Saturating Straight Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
KSTAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element saturating addition and 32-bit signed integer element saturating subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [63:32] of Rs2; at the same time, it subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [31:0] of Rs1. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```

res[1] = Rs1.W[1] + Rs2.W[1];
res[0] = Rs1.W[0] - Rs2.W[0];
if (res[x] > (231)-1) {
    res[x] = (231)-1;
    OV = 1;
} else if (res < -231) {
    res[x] = -231;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];
for RV64, x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a

- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSTSA32 (unsigned long a, unsigned long b)

KSTSA32 (SIMD 32-bit Signed Saturating Straight Subtraction & Addition)

Type: SIM (RV64 Only)

Syntax:

KSTSA32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer element saturating subtraction and 32-bit signed integer element saturating addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. *Description: * This instruction subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [63:32] of Rs1; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [31:0] of Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```
res[1] = Rs1.W[1] - Rs2.W[1];
res[0] = Rs1.W[0] + Rs2.W[0];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];
for RV64, x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSUB32 (unsigned long a, unsigned long b)

KSUB32 (SIMD 32-bit Signed Saturating Subtraction)

Type: SIMD (RV64 Only)

Syntax:

KSUB32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 32-bit signed integer elements in Rs2 from the 32-bit signed integer elements in Rs1. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] - Rs2.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[x] = res[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RADD32 (unsigned long a, unsigned long b)

RADD32 (SIMD 32-bit Signed Halving Addition)

Type: SIMD (RV64 Only)

Syntax:

RADD32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Rs1 = 0x7FFFFFFF, Rs2 = 0x7FFFFFFF Rd = 0x7FFFFFFF
* Rs1 = 0x80000000, Rs2 = 0x80000000 Rd = 0x80000000
* Rs1 = 0x40000000, Rs2 = 0x80000000 Rd = 0xE0000000
```

Operations:

```
Rd.W[x] = (Rs1.W[x] + Rs2.W[x]) s>> 1;
for RV64: x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RCRAS32 (unsigned long a, unsigned long b)

RCRAS32 (SIMD 32-bit Signed Halving Cross Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
RCRAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element addition and 32-bit signed integer element subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit signed integer element in [63:32] of Rs1 with the 32-bit signed integer element in [31:0] of Rs2, and subtracts the 32-bit signed integer element in [63:32] of Rs2 from the 32-bit signed integer element in [31:0] of Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Examples:

Please see `RADD32` and `RSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] + Rs2.W[0]) s>> 1;
Rd.W[0] = (Rs1.W[0] - Rs2.W[1]) s>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RCRSA32 (unsigned long a, unsigned long b)

RCRSA32 (SIMD 32-bit Signed Halving Cross Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

RCRSA32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer element subtraction and 32-bit signed integer element addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit signed integer element in [31:0] of Rs2 from the 32-bit signed integer element in [63:32] of Rs1, and adds the 32-bit signed integer element in [31:0] of Rs1 with the 32-bit signed integer element in [63:32] of Rs2. The two results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Examples:

Please see `RADD32` and `RSUB32` instructions.

Operations:

Rd.W[1] = (Rs1.W[1] - Rs2.W[0]) s>> 1;
Rd.W[0] = (Rs1.W[0] + Rs2.W[1]) s>> 1;

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSTAS32 (unsigned long a, unsigned long b)

RSTAS32 (SIMD 32-bit Signed Halving Straight Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

RSTAS32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer element addition and 32-bit signed integer element subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit signed integer element in [63:32] of Rs1 with the 32-bit signed integer element in [63:32] of Rs2, and subtracts the 32-bit signed integer element in [31:0] of Rs2 from the 32-bit signed integer element in [31:0] of Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Examples:

Please see `RADD32` and `RSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] + Rs2.W[1]) s>> 1;
Rd.W[0] = (Rs1.W[0] - Rs2.W[0]) s>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSTSA32 (unsigned long a, unsigned long b)

RSTSA32 (SIMD 32-bit Signed Halving Straight Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

```
RSTSA32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element subtraction and 32-bit signed integer element addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit signed integer element in [63:32] of Rs2 from the 32-bit signed integer element in [63:32] of Rs1, and adds the 32-bit signed element integer in [31:0] of Rs1 with the 32-bit signed integer element in [31:0] of Rs2. The two results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Examples:

Please see `RADD32` and `RSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] - Rs2.W[1]) s>> 1;
Rd.W[0] = (Rs1.W[0] + Rs2.W[0]) s>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_RSUB32 (unsigned long a, unsigned long b)

RSUB32 (SIMD 32-bit Signed Halving Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
RSUB32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit signed integer elements in Rs2 from the 32-bit signed integer elements in Rs1. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7FFFFFFF, Rb = 0x80000000 Rt = 0x7FFFFFFF
* Ra = 0x80000000, Rb = 0x7FFFFFFF Rt = 0x80000000
* Ra = 0x80000000, Rb = 0x40000000 Rt = 0xA0000000
```

Operations:

```
Rd.W[x] = (Rs1.W[x] - Rs2.W[x]) s>> 1;
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_STAS32 (unsigned long a, unsigned long b)
```

STAS32 (SIMD 32-bit Straight Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
STAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit integer element addition and 32-bit integer element subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [63:32] of Rs2, and writes the result to [63:32] of Rd; at the same time, it subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [31:0] of Rs1, and writes the result to [31:0] of Rd.

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

```
Rd.W[1] = Rs1.W[1] + Rs2.W[1];
Rd.W[0] = Rs1.W[0] - Rs2.W[0];
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_STSA32 (unsigned long a, unsigned long b)

STSA32 (SIMD 32-bit Straight Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

STSA32 Rd, Rs1, Rs2

Purpose :

Do 32-bit integer element subtraction and 32-bit integer element addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. *Description: * This instruction subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [63:32] of Rs1, and writes the result to [63:32] of Rd; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [31:0] of Rs2, and writes the result to [31:0] of Rd

Note :

This instruction can be used for either signed or unsigned operations.

Operations:

Rd.W[1] = Rs1.W[1] - Rs2.W[1];
Rd.W[0] = Rs1.W[0] + Rs2.W[0];

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SUB32 (unsigned long a, unsigned long b)

SUB32 (SIMD 32-bit Subtraction)

Type: DSP (RV64 Only)

Syntax:

SUB32 Rd, Rs1, Rs2

Purpose :

Do 32-bit integer element subtractions simultaneously.

Description :

This instruction subtracts the 32-bit integer elements in Rs2 from the 32-bit integer elements in Rs1, and then writes the results to Rd.

Note :

This instruction can be used for either signed or unsigned subtraction.

Operations:

```
Rd.W[x] = Rs1.W[x] - Rs2.W[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKADD32 (unsigned long a, unsigned long b)

UKADD32 (SIMD 32-bit Unsigned Saturating Addition)

Type: SIMD (RV64 Only)

Syntax:

```
UKADD32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element saturating additions simultaneously.

Description :

This instruction adds the 32-bit unsigned integer elements in Rs1 with the 32-bit unsigned integer elements in Rs2. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] + Rs2.W[x];
if (res[x] > (2^32)-1) {
    res[x] = (2^32)-1;
    OV = 1;
}
Rd.W[x] = res[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKCRAS32 (unsigned long a, unsigned long b)

UKCRAS32 (SIMD 32-bit Unsigned Saturating Cross Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
UKCRAS32 Rd, Rs1, Rs2
```

Purpose :

Do one 32-bit unsigned integer element saturating addition and one 32-bit unsigned integer element saturating subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction adds the 32-bit unsigned integer element in [63:32] of Rs1 with the 32-bit unsigned integer element in [31:0] of Rs2; at the same time, it subtracts the 32-bit unsigned integer element in [63:32] of Rs2 from the 32-bit unsigned integer element in [31:0] of Rs1. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```
res1 = Rs1.W[1] + Rs2.W[0];
res2 = Rs1.W[0] - Rs2.W[1];
if (res1 > (2^32)-1) {
    res1 = (2^32)-1;
    OV = 1;
}
if (res2 < 0) {
    res2 = 0;
    OV = 1;
}
Rd.W[1] = res1;
Rd.W[0] = res2;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_UKCRSA32 (unsigned long a, unsigned long b)
```

UKCRSA32 (SIMD 32-bit Unsigned Saturating Cross Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

```
UKCRSA32 Rd, Rs1, Rs2
```

Purpose :

Do one 32-bit unsigned integer element saturating subtraction and one 32-bit unsigned integer element saturating addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction subtracts the 32-bit unsigned integer element in [31:0] of Rs2 from the 32-bit unsigned integer element in [63:32] of Rs1; at the same time, it adds the 32-bit unsigned integer element in [63:32]

of Rs2 with the 32-bit unsigned integer element in [31:0] Rs1. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```
res1 = Rs1.W[1] - Rs2.W[0];
res2 = Rs1.W[0] + Rs2.W[1];
if (res1 < 0) {
    res1 = 0;
    OV = 1;
} else if (res2 > (2^32)-1) {
    res2 = (2^32)-1;
    OV = 1;
}
Rd.W[1] = res1;
Rd.W[0] = res2;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSTAS32 (unsigned long a, unsigned long b)

UKSTAS32 (SIMD 32-bit Unsigned Saturating Straight Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

UKSTAS32 Rd, Rs1, Rs2

Purpose :

Do one 32-bit unsigned integer element saturating addition and one 32-bit unsigned integer element saturating subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction adds the 32-bit unsigned integer element in [63:32] of Rs1 with the 32-bit unsigned integer element in [63:32] of Rs2; at the same time, it subtracts the 32-bit unsigned integer element in [31:0] of Rs2 from the 32-bit unsigned integer element in [31:0] Rs1. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```
res1 = Rs1.W[1] + Rs2.W[1];
res2 = Rs1.W[0] - Rs2.W[0];
if (res1 > (2^32)-1) {
    res1 = (2^32)-1;
    OV = 1;
}
if (res2 < 0) {
```

(continues on next page)

(continued from previous page)

```

    res2 = 0;
    OV = 1;
}
Rd.W[1] = res1;
Rd.W[0] = res2;

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSTSA32 (unsigned long a, unsigned long b)

UKSTSA32 (SIMD 32-bit Unsigned Saturating Straight Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

```
UKSTSA32 Rd, Rs1, Rs2
```

Purpose :

Do one 32-bit unsigned integer element saturating subtraction and one 32-bit unsigned integer element saturating addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction subtracts the 32-bit unsigned integer element in [63:32] of Rs2 from the 32-bit unsigned integer element in [63:32] of Rs1; at the same time, it adds the 32-bit unsigned integer element in [31:0] of Rs2 with the 32-bit unsigned integer element in [31:0] Rs1. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```

res1 = Rs1.W[1] - Rs2.W[1];
res2 = Rs1.W[0] + Rs2.W[0];
if (res1 < 0) {
    res1 = 0;
    OV = 1;
} else if (res2 > (2^32)-1) {
    res2 = (2^32)-1;
    OV = 1;
}
Rd.W[1] = res1;
Rd.W[0] = res2;

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UKSUB32 (unsigned long a, unsigned long b)

UKSUB32 (SIMD 32-bit Unsigned Saturating Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
UKSUB32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 32-bit unsigned integer elements in Rs2 from the 32-bit unsigned integer elements in Rs1. If any of the results are beyond the 32-bit unsigned number range ($0 \leq \text{RES} \leq 2^{32}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] - Rs2.W[x];
if (res[x] < 0) {
    res[x] = 0;
    OV = 1;
}
Rd.W[x] = res[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URADD32 (unsigned long a, unsigned long b)

URADD32 (SIMD 32-bit Unsigned Halving Addition)

Type: SIMD (RV64 Only)

Syntax:

```
URADD32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit unsigned integer elements in Rs1 with the 32-bit unsigned integer elements in Rs2. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7FFFFFFF, Rb = 0x7FFFFFFF Rt = 0x7FFFFFFF
* Ra = 0x80000000, Rb = 0x80000000 Rt = 0x80000000
* Ra = 0x40000000, Rb = 0x80000000 Rt = 0x60000000
```

Operations:

```
Rd.W[x] = (Rs1.W[x] + Rs2.W[x]) u>> 1;
for RV64: x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URCRAS32 (unsigned long a, unsigned long b)

URCRAS32 (SIMD 32-bit Unsigned Halving Cross Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
URCRAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element addition and 32-bit unsigned integer element subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit unsigned integer element in [63:32] of Rs1 with the 32-bit unsigned integer element in [31:0] of Rs2, and subtracts the 32-bit unsigned integer element in [63:32] of Rs2 from the 32-bit unsigned integer element in [31:0] of Rs1. The element results are first logically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Examples:

Please see `URADD32` and `URSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] + Rs2.W[0]) u>> 1;
Rd.W[0] = (Rs1.W[0] - Rs2.W[1]) u>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URCRSA32 (unsigned long a, unsigned long b)

URCRSA32 (SIMD 32-bit Unsigned Halving Cross Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

```
URCRSA32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element subtraction and 32-bit unsigned integer element addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit unsigned integer element in [31:0] of Rs2 from the 32-bit unsigned integer element in [63:32] of Rs1, and adds the 32-bit unsigned integer element in [31:0] of Rs1 with the 32-bit unsigned integer element in [63:32] of Rs2. The two results are first logically right-shifted by 1 bit and then written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Examples:

Please see `URADD32` and `URSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] - Rs2.W[0]) u>> 1;
Rd.W[0] = (Rs1.W[0] + Rs2.W[1]) u>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSTAS32 (unsigned long a, unsigned long b)

URSTAS32 (SIMD 32-bit Unsigned Halving Straight Addition & Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
URSTAS32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element addition and 32-bit unsigned integer element subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit unsigned integer element in [63:32] of Rs1 with the 32-bit unsigned integer element in [63:32] of Rs2, and subtracts the 32-bit unsigned integer element in [31:0] of Rs2 from the 32-bit

unsigned integer element in [31:0] of Rs1. The element results are first logically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Examples:

Please see `URADD32` and `URSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] + Rs2.W[1]) u>> 1;
Rd.W[0] = (Rs1.W[0] - Rs2.W[0]) u>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_URSTSA32 (unsigned long a, unsigned long b)

URSTSA32 (SIMD 32-bit Unsigned Halving Straight Subtraction & Addition)

Type: SIMD (RV64 Only)

Syntax:

URSTSA32 Rd, Rs1, Rs2

Purpose :

Do 32-bit unsigned integer element subtraction and 32-bit unsigned integer element addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit unsigned integer element in [63:32] of Rs2 from the 32-bit unsigned integer element in [63:32] of Rs1, and adds the 32-bit unsigned element integer in [31:0] of Rs1 with the 32-bit unsigned integer element in [31:0] of Rs2. The two results are first logically right-shifted by 1 bit and then written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Examples:

Please see `URADD32` and `URSUB32` instructions.

Operations:

```
Rd.W[1] = (Rs1.W[1] - Rs2.W[1]) u>> 1;
Rd.W[0] = (Rs1.W[0] + Rs2.W[0]) u>> 1;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_URSUB32 (unsigned long a, unsigned long b)
```

URSUB32 (SIMD 32-bit Unsigned Halving Subtraction)

Type: SIMD (RV64 Only)

Syntax:

```
URSUB32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit unsigned integer elements in Rs2 from the 32-bit unsigned integer elements in Rs1. The results are first logically right-shifted by 1 bit and then written to Rd.

Examples:

```
* Ra = 0x7FFFFFFF, Rb = 0x80000000, Rt = 0xFFFFFFFF
* Ra = 0x80000000, Rb = 0x7FFFFFFF, Rt = 0x00000000
* Ra = 0x80000000, Rb = 0x40000000, Rt = 0x20000000
```

Operations:

```
Rd.W[x] = (Rs1.W[x] - Rs2.W[x]) u>> 1;
for RV64: x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

(RV64 Only) SIMD 32-bit Shift Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KSLL32 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA32 (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KSLRA32_U (unsigned long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SLL32 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRA32 (unsigned long a, unsigned int b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SRA32_U (unsigned long a, unsigned int b)
```

```

__STATIC_FORCEINLINE unsigned long __RV_SRL32 (unsigned long a, unsigned int b)

__STATIC_FORCEINLINE unsigned long __RV_SRL32_U (unsigned long a, unsigned int b)

__RV_KSLLI32(a, b)
__RV_SLLI32(a, b)
__RV_SRAI32(a, b)
__RV_SRAI32_U(a, b)
__RV_SRLI32(a, b)
__RV_SRLI32_U(a, b)

```

group (RV64 Only) SIMD 32-bit Shift Instructions

(RV64 Only) SIMD 32-bit Shift Instructions

there are 14 (RV64 Only) SIMD 32-bit Shift Instructions

Defines

__RV_KSLLI32(a, b)
KSLLI32 (SIMD 32-bit Saturating Shift Left Logical Immediate)

Type: SIMD (RV64 Only)

Syntax:

```
KSLLI32 Rd, Rs1, imm5u
```

Purpose :

Do 32-bit elements logical left shift operations with saturation simultaneously. The shift amount is an immediate value.

Description :

The 32-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm5u constant. Any shifted value greater than $2^{31}-1$ is saturated to $2^{31}-1$. Any shifted value smaller than -2^{31} is saturated to -2^{31} . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```

sa = imm5u[4:0];
if (sa != 0) {
    res[(31+sa):0] = Rs1.W[x] << sa;
    if (res > (2^31)-1) {
        res = 0x7fffffff; OV = 1;
    } else if (res < -2^31) {
        res = 0x80000000; OV = 1;
    }
    Rd.W[x] = res[31:0];
} else {

```

(continues on next page)

(continued from previous page)

```

    Rd = Rs1;
}
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SLLI32(a, b)

SLLI32 (SIMD 32-bit Shift Left Logical Immediate)

Type: SIMD (RV64 Only)

Syntax:

```
SLLI32 Rd, Rs1, imm5u[4:0]
```

Purpose :

Do 32-bit element logical left shift operations simultaneously. The shift amount is an immediate value.

Description :

The 32-bit elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the imm5u[4:0] constant. And the results are written to Rd.

Operations:

```

sa = imm5u[4:0];
Rd.W[x] = Rs1.W[x] << sa;
for RV64: x=1..0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI32(a, b)

SRAI32 (SIMD 32-bit Shift Right Arithmetic Immediate)

Type: DSP (RV64 Only)

Syntax:

```

SRAI32 Rd, Rs1, imm5u
SRAI32.u Rd, Rs1, imm5u

```

Purpose :

Do 32-bit elements arithmetic right shift operations simultaneously. The shift amount is an immediate value. The `.u` form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in `Rs1` are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the 32-bit data elements. The shift amount is specified by the `imm5u` constant. For the rounding operation of the `.u` form, a value of 1 is added to the most significant discarded bit of each 32-bit data to calculate the final results. And the results are written to `Rd`.

Operations:

```
sa = imm5u[4:0];
if (sa > 0) {
  if (`.u` form) { // SRAI32.u
    res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
    Rd.W[x] = res[31:0];
  } else { // SRAI32
    Rd.W[x] = SE32(Rs1.W[x][31:sa]);
  }
} else {
  Rd = Rs1;
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRAI32_U(a, b)

SRAI32.u (SIMD 32-bit Rounding Shift Right Arithmetic Immediate)

Type: DSP (RV64 Only)

Syntax:

```
SRAI32 Rd, Rs1, imm5u
SRAI32.u Rd, Rs1, imm5u
```

Purpose :

Do 32-bit elements arithmetic right shift operations simultaneously. The shift amount is an immediate value. The `.u` form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in `Rs1` are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the 32-bit data elements. The shift amount is specified by the `imm5u` constant. For the rounding operation of the `.u` form, a value of 1 is added to the most significant discarded bit of each 32-bit data to calculate the final results. And the results are written to `Rd`.

Operations:

```

sa = imm5u[4:0];
if (sa > 0) {
  if (`.u` form) { // SRAI32.u
    res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
    Rd.W[x] = res[31:0];
  } else { // SRAI32
    Rd.W[x] = SE32(Rs1.W[x][31:sa]);
  }
} else {
  Rd = Rs1;
}
for RV64: x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI32(a, b)

SRLI32 (SIMD 32-bit Shift Right Logical Immediate)

Type: SIMD (RV64 Only)

Syntax:

```

SRLI32 Rd, Rs1, imm5u
SRLI32.u Rd, Rs1, imm5u

```

Purpose :

Do 32-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm5u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 32-bit data to calculate the final results. And the results are written to Rd.

Operations:

```

sa = imm5u[4:0];
if (sa > 0) {
  if (`.u` form) { // SRLI32.u
    res[31:-1] = ZE33(Rs1.W[x][31:sa-1]) + 1;
    Rd.W[x] = res[31:0];
  } else { // SRLI32
    Rd.W[x] = ZE32(Rs1.W[x][31:sa]);
  }
} else {
  Rd = Rs1;
}

```

(continues on next page)

(continued from previous page)

```
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__RV_SRLI32_U(a, b)

SRLI32.u (SIMD 32-bit Rounding Shift Right Logical Immediate)

Type: SIMD (RV64 Only)

Syntax:

```
SRLI32 Rd, Rs1, imm5u
SRLI32.u Rd, Rs1, imm5u
```

Purpose :

Do 32-bit elements logical right shift operations simultaneously. The shift amount is an immediate value. The .u form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the imm5u constant. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 32-bit data to calculate the final results. And the results are written to Rd.

Operations:

```
sa = imm5u[4:0];
if (sa > 0) {
    if (`.u` form) { // SRLI32.u
        res[31:-1] = ZE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else { // SRLI32
        Rd.W[x] = ZE32(Rs1.W[x][31:sa]);
    }
} else {
    Rd = Rs1;
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

Functions

__STATIC_FORCEINLINE unsigned long __RV_KSLL32 (unsigned long a, unsigned int b)

KSLL32 (SIMD 32-bit Saturating Shift Left Logical)

Type: SIMD (RV64 Only)

Syntax:

KSLL32 Rd, Rs1, Rs2

Purpose :

Do 32-bit elements logical left shift operations with saturation simultaneously. The shift amount is a variable from a GPR.

Description :

The 32-bit data elements in Rs1 are left-shifted logically. The shifted out bits are filled with zero and the shift amount is specified by the low-order 5-bits of the value in the Rs2 register. Any shifted value greater than $2^{31}-1$ is saturated to $2^{31}-1$. Any shifted value smaller than -2^{31} is saturated to -2^{31} . And the saturated results are written to Rd. If any saturation is performed, set OV bit to 1.

Operations:

```
sa = Rs2[4:0];
if (sa != 0) {
    res[(31+sa):0] = Rs1.W[x] << sa;
    if (res > (2^31)-1) {
        res = 0x7fffffff; OV = 1;
    } else if (res < -2^31) {
        res = 0x80000000; OV = 1;
    }
    Rd.W[x] = res[31:0];
} else {
    Rd = Rs1;
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA32 (unsigned long a, int b)

KSLRA32 (SIMD 32-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD (RV64 Only)

Syntax:

KSLRA32 Rd, Rs1, Rs2
KSLRA32.u Rd, Rs1, Rs2

Purpose :

Do 32-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q31 saturation for the left shift. The .u form performs additional rounding up operations for the right shift.

Description :

The 32-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[5:0]. Rs2[5:0] is in the signed range of [-25, 25-1]. A positive Rs2[5:0] means logical left shift and a negative Rs2[5:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[5:0]. However, the behavior of Rs2[5:0]==-25 (0x20) is defined to be equivalent to the behavior of Rs2[5:0]==-(25-1) (0x21). The left-shifted results are saturated to the 32-bit signed integer range of [-2³¹, 2³¹-1]. For the .u form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:6] will not affect this instruction.

Operations:

```
if (Rs2[5:0] < 0) {
    sa = -Rs2[5:0];
    sa = (sa == 32)? 31 : sa;
    if (`.u` form) {
        res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else {
        Rd.W[x] = SE32(Rs1.W[x][31:sa]);
    }
} else {
    sa = Rs2[4:0];
    res[(31+sa):0] = Rs1.W[x] <<(logic) sa;
    if (res > (2^31)-1) {
        res[31:0] = 0x7fffffff; OV = 1;
    } else if (res < -2^31) {
        res[31:0] = 0x80000000; OV = 1;
    }
    Rd.W[x] = res[31:0];
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KSLRA32_U (unsigned long a, int b)

KSLRA32.u (SIMD 32-bit Shift Left Logical with Saturation or Rounding Shift Right Arithmetic)

Type: SIMD (RV64 Only)

Syntax:

```
KSLRA32 Rd, Rs1, Rs2
KSLRA32.u Rd, Rs1, Rs2
```

Purpose :

Do 32-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q31 saturation for the left shift. The .u form performs additional rounding up operations for the right shift.

Description :

The 32-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[5:0]. Rs2[5:0] is in the signed range of [-25, 25-1]. A positive Rs2[5:0] means logical left shift and a negative Rs2[5:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[5:0]. However, the behavior of Rs2[5:0]==-25 (0x20) is defined to be equivalent to the behavior of Rs2[5:0]==-(25-1) (0x21). The left-shifted results are saturated to the 32-bit signed integer range of $[-2^{31}, 2^{31}-1]$. For the .u form of the instruction, the right-shifted results are added a 1 to the most significant discarded bit position for rounding effect. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:6] will not affect this instruction.

Operations:

```
if (Rs2[5:0] < 0) {
    sa = -Rs2[5:0];
    sa = (sa == 32)? 31 : sa;
    if (`.u` form) {
        res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else {
        Rd.W[x] = SE32(Rs1.W[x][31:sa]);
    }
} else {
    sa = Rs2[4:0];
    res[(31+sa):0] = Rs1.W[x] <<(logic) sa;
    if (res > (2^31)-1) {
        res[31:0] = 0x7fffffff; OV = 1;
    } else if (res < -2^31) {
        res[31:0] = 0x80000000; OV = 1;
    }
    Rd.W[x] = res[31:0];
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long type

```
__STATIC_FORCEINLINE unsigned long __RV_SLL32 (unsigned long a, unsigned int b)
```

SLL32 (SIMD 32-bit Shift Left Logical)

Type: SIMD (RV64 Only)

Syntax:

```
SLL32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit elements logical left shift operations simultaneously. The shift amount is a variable from a GPR.

Description :

The 32-bit elements in Rs1 are left-shifted logically. And the results are written to Rd. The shifted out bits are filled with zero and the shift amount is specified by the low-order 5-bits of the value in the Rs2 register.

Operations:

```
sa = Rs2[4:0];
Rd.W[x] = Rs1.W[x] << sa;
for RV64: x=1..0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA32 (unsigned long a, unsigned int b)

SRA32 (SIMD 32-bit Shift Right Arithmetic)

Type: SIMD (RV64 Only)

Syntax:

```
SRA32 Rd, Rs1, Rs2
SRA32.u Rd, Rs1, Rs2
```

Purpose :

Do 32-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 5-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 32-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[4:0];
if (sa > 0) {
    if (`.u` form) { // SRA32.u
        res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else { // SRA32
        Rd.W[x] = SE32(Rs1.W[x][31:sa])
    }
}
```

(continues on next page)

(continued from previous page)

```

    }
  } else {
    Rd = Rs1;
  }
  for RV64: x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRA32_U (unsigned long a, unsigned int b)

SRA32.u (SIMD 32-bit Rounding Shift Right Arithmetic)

Type: SIMD (RV64 Only)

Syntax:

```

SRA32 Rd, Rs1, Rs2
SRA32.u Rd, Rs1, Rs2

```

Purpose :

Do 32-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 5-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 32-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```

sa = Rs2[4:0];
if (sa > 0) {
  if (`.u` form) { // SRA32.u
    res[31:-1] = SE33(Rs1.W[x][31:sa-1]) + 1;
    Rd.W[x] = res[31:0];
  } else { // SRA32
    Rd.W[x] = SE32(Rs1.W[x][31:sa])
  }
} else {
  Rd = Rs1;
}
for RV64: x=1...0

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL32 (unsigned long a, unsigned int b)

SRL32 (SIMD 32-bit Shift Right Logical)

Type: SIMD (RV64 Only)**Syntax:**

```
SRL32 Rd, Rs1, Rs2
SRL32.u Rd, Rs1, Rs2
```

Purpose :

Do 32-bit element logical right shift operations simultaneously. The shift amount is a variable from a GPR. The .u form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in Rs1 are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 5-bits of the value in the Rs2 register. For the rounding operation of the .u form, a value of 1 is added to the most significant discarded bit of each 32-bit data element to calculate the final results. And the results are written to Rd.

Operations:

```
sa = Rs2[4:0];
if (sa > 0) {
    if (`.u` form) { // SRA32.u
        res[31:-1] = ZE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else { // SRA32
        Rd.W[x] = ZE32(Rs1.W[x][31:sa])
    }
} else {
    Rd = Rs1;
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SRL32_U (unsigned long a, unsigned int b)

SRL32.u (SIMD 32-bit Rounding Shift Right Logical)

Type: SIMD (RV64 Only)**Syntax:**

```
SRL32 Rd, Rs1, Rs2
SRL32.u Rd, Rs1, Rs2
```

Purpose :

Do 32-bit element logical right shift operations simultaneously. The shift amount is a variable from a GPR. The `.u` form performs additional rounding up operations on the shifted results.

Description :

The 32-bit data elements in `Rs1` are right-shifted logically, that is, the shifted out bits are filled with zero. The shift amount is specified by the low-order 5-bits of the value in the `Rs2` register. For the rounding operation of the `.u` form, a value of 1 is added to the most significant discarded bit of each 32-bit data element to calculate the final results. And the results are written to `Rd`.

Operations:

```
sa = Rs2[4:0];
if (sa > 0) {
    if (`.u` form) { // SRA32.u
        res[31:-1] = ZE33(Rs1.W[x][31:sa-1]) + 1;
        Rd.W[x] = res[31:0];
    } else { // SRA32
        Rd.W[x] = ZE32(Rs1.W[x][31:sa])
    }
} else {
    Rd = Rs1;
}
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in unsigned long type

(RV64 Only) SIMD 32-bit Miscellaneous Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KABS32 (unsigned long a)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SMAX32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_SMIN32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UMAX32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_UMIN32 (unsigned long a, unsigned long b)
```

group (RV64 Only) SIMD 32-bit Miscellaneous Instructions

(RV64 Only) SIMD 32-bit Miscellaneous Instructions

there are 5 (RV64 Only) SIMD 32-bit Miscellaneous Instructions

Functions

__STATIC_FORCEINLINE unsigned long __RV_KABS32 (unsigned long a)

KABS32 (Scalar 32-bit Absolute Value with Saturation)

Type: DSP (RV64 Only) 24 20 19 15 14 12 11 7 KABS32 10010 Rs1 000 Rd 6 0 GE80B 1111111

Syntax:

KABS32 Rd, Rs1

Purpose :

Get the absolute value of signed 32-bit integer elements in a general register.

Description :

This instruction calculates the absolute value of signed 32-bit integer elements stored in Rs1. The results are written to Rd. This instruction with the minimum negative integer input of 0x80000000 will produce a saturated output of maximum positive integer of 0x7fffffff and the OV flag will be set to 1.

Operations:

```
if (Rs1.W[x] >= 0) {
    res[x] = Rs1.W[x];
} else {
    If (Rs1.W[x] == 0x80000000) {
        res[x] = 0x7fffffff;
        OV = 1;
    } else {
        res[x] = -Rs1.W[x];
    }
}
Rd.W[x] = res[x];
for RV64: x=1..0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMAX32 (unsigned long a, unsigned long b)

SMAX32 (SIMD 32-bit Signed Maximum)

Type: SIMD (RV64 Only)

Syntax:

SMAX32 Rd, Rs1, Rs2

Purpose :

Do 32-bit signed integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2 and selects the numbers that is greater than the other one. The selected results are written to Rd.

Operations:

```
Rd.W[x] = (Rs1.W[x] > Rs2.W[x])? Rs1.W[x] : Rs2.W[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_SMIN32 (unsigned long a, unsigned long b)

SMIN32 (SIMD 32-bit Signed Minimum)

Type: SIMD (RV64 Only)

Syntax:

```
SMIN32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit signed integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

```
Rd.W[x] = (Rs1.W[x] < Rs2.W[x])? Rs1.W[x] : Rs2.W[x];
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_UMAX32 (unsigned long a, unsigned long b)

UMAX32 (SIMD 32-bit Unsigned Maximum)

Type: SIMD (RV64 Only)

Syntax:

```
UMAX32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer elements finding maximum operations simultaneously.

Description :

This instruction compares the 32-bit unsigned integer elements in Rs1 with the 32-bit unsigned integer elements in Rs2 and selects the numbers that is greater than the other one. The selected results are written to Rd.

Operations:

```
Rd.W[x] = (Rs1.W[x] u> Rs2.W[x])? Rs1.W[x] : Rs2.W[x];  
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

`__STATIC_FORCEINLINE unsigned long __RV_UMIN32 (unsigned long a, unsigned long b)`

UMIN32 (SIMD 32-bit Unsigned Minimum)

Type: SIMD (RV64 Only)

Syntax:

```
UMIN32 Rd, Rs1, Rs2
```

Purpose :

Do 32-bit unsigned integer elements finding minimum operations simultaneously.

Description :

This instruction compares the 32-bit unsigned integer elements in Rs1 with the 32-bit unsigned integer elements in Rs2 and selects the numbers that is less than the other one. The selected results are written to Rd.

Operations:

```
Rd.W[x] = (Rs1.W[x] <u Rs2.W[x])? Rs1.W[x] : Rs2.W[x];  
for RV64: x=1...0
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

(RV64 Only) SIMD Q15 Saturating Multiply Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_KDMBB16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KDMBT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KDMTT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KDMABB16 (unsigned long t, unsigned long a,  
unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KDMABT16 (unsigned long t, unsigned long a,  
unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KDMATT16 (unsigned long t, unsigned long a,  
unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KHMBB16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KHMBT16 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_KHMTT16 (unsigned long a, unsigned long b)
```

group **(RV64 Only) SIMD Q15 Saturating Multiply Instructions**

(RV64 Only) SIMD Q15 Saturating Multiply Instructions

there are 9 (RV64 Only) SIMD Q15 saturating Multiply Instructions

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_KDMBB16 (unsigned long a, unsigned long b)
```

KDMBB16 (SIMD Signed Saturating Double Multiply B16 x B16)

Type: SIMD (RV64 only)

Syntax:

```
KDMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)
```

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results into the 32-bit chunks in the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values.

The Q31 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```
// KDMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
Rd.W[z] = resQ31[z];
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KDMBT16 (unsigned long a, unsigned long b)

KDMBT16 (SIMD Signed Saturating Double Multiply B16 x T16)

Type: SIMD (RV64 only)

Syntax:

KDMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results into the 32-bit chunks in the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values. The Q31 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```
// KDMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
}
```

(continues on next page)

(continued from previous page)

```

} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
Rd.W[z] = resQ31[z];

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KDMTT16 (unsigned long a, unsigned long b)

KDMTT16 (SIMD Signed Saturating Double Multiply T16 x T16)

Type: SIMD (RV64 only)

Syntax:

KDMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results into the 32-bit chunks in the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values. The Q31 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFFFFFF and the overflow flag OV will be set.

Operations:

```

// KDMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
Rd.W[z] = resQ31[z];

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

`__STATIC_FORCEINLINE unsigned long __RV_KDMABB16 (unsigned long t, unsigned long a, unsigned long b)`

KDMABB16 (SIMD Signed Saturating Double Multiply Addition B16 x B16)

Type: SIMD (RV64 only)

Syntax:

KDMAxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results, add the results with the values of the corresponding 32-bit chunks from the destination register and write the saturated addition results back into the corresponding 32-bit chunks of the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the corresponding 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values. The Q31 values are then added with the content of the corresponding 32-bit portions of Rd. If the addition results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV flag is set to 1. The results after saturation are written back to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```
// KDMABB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMABT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMATT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
resadd[z] = Rd.W[z] + resQ31[z];
if (resadd[z] > (2^31)-1) {
    resadd[z] = (2^31)-1;
    OV = 1;
} else if (resadd[z] < -2^31) {
    resadd[z] = -2^31;
    OV = 1;
}
Rd.W[z] = resadd[z];
```

Parameters

- **t** – [in] unsigned long type of value stored in t

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KDMABT16 (unsigned long t, unsigned long a, unsigned long b)

KDMABT16 (SIMD Signed Saturating Double Multiply Addition B16 x T16)

Type: SIMD (RV64 only)

Syntax:

KDMAxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results, add the results with the values of the corresponding 32-bit chunks from the destination register and write the saturated addition results back into the corresponding 32-bit chunks of the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the corresponding 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values. The Q31 values are then added with the content of the corresponding 32-bit portions of Rd. If the addition results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV flag is set to 1. The results after saturation are written back to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```
// KDMABB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMABT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMATT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
resadd[z] = Rd.W[z] + resQ31[z];
if (resadd[z] > (2^31)-1) {
    resadd[z] = (2^31)-1;
    OV = 1;
} else if (resadd[z] < -2^31) {
    resadd[z] = -2^31;
    OV = 1;
}
Rd.W[z] = resadd[z];
```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KDMATT16 (unsigned long t, unsigned long a, unsigned long b)

KDMATT16 (SIMD Signed Saturating Double Multiply Addition T16 x T16)

Type: SIMD (RV64 only)

Syntax:

KDMAxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then double and saturate the Q31 results, add the results with the values of the corresponding 32-bit chunks from the destination register and write the saturated addition results back into the corresponding 32-bit chunks of the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the corresponding 32-bit portions in Rs2. The Q30 results are then doubled and saturated into Q31 values. The Q31 values are then added with the content of the corresponding 32-bit portions of Rd. If the addition results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV flag is set to 1. The results after saturation are written back to Rd. When both the two Q15 inputs are 0x8000, saturation will happen and the overflow flag OV will be set.

Operations:

```
// KDMABB16: (x,y,z)=(0,0,0),(2,2,1)
// KDMABT16: (x,y,z)=(0,1,0),(2,3,1)
// KDMATT16: (x,y,z)=(1,1,0),(3,3,1)
aop[z] = Rs1.H[x]; bop[z] = Rs2.H[y];
If (0x8000 != aop[z] | 0x8000 != bop[z]) {
    Mresult[z] = aop[z] * bop[z];
    resQ31[z] = Mresult[z] << 1;
} else {
    resQ31[z] = 0x7FFFFFFF;
    OV = 1;
}
resadd[z] = Rd.W[z] + resQ31[z];
if (resadd[z] > (2^31)-1) {
    resadd[z] = (2^31)-1;
    OV = 1;
} else if (resadd[z] < -2^31) {
    resadd[z] = -2^31;
    OV = 1;
}
Rd.W[z] = resadd[z];
```

Parameters

- **t** – [in] unsigned long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KHMBB16 (unsigned long a, unsigned long b)

KHMBB16 (SIMD Signed Saturating Half Multiply B16 x B16)

Type: SIMD (RV64 Only)

Syntax:

KHMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then right-shift 15 bits to turn the Q30 results into Q15 numbers again and saturate the Q15 results into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portion in Rs2. The Q30 results are then right-shifted 15- bits and saturated into Q15 values. The 32-bit Q15 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
// KHMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KHMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KHMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop = Rs1.H[x]; bop = Rs2.H[y];
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd.W[z] = SE32(res[15:0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KHMBT16 (unsigned long a, unsigned long b)

KHMBT16 (SIMD Signed Saturating Half Multiply B16 x T16)

Type: SIMD (RV64 Only)

Syntax:

KHMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then right-shift 15 bits to turn the Q30 results into Q15 numbers again and saturate the Q15 results into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portion in Rs2. The Q30 results are then right-shifted 15- bits and saturated into Q15 values. The 32-bit Q15 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
// KHMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KHMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KHMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop = Rs1.H[x]; bop = Rs2.H[y];
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd.W[z] = SE32(res[15:0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_KHMTT16 (unsigned long a, unsigned long b)

KHMTT16 (SIMD Signed Saturating Half Multiply T16 x T16)

Type: SIMD (RV64 Only)

Syntax:

KHMxy16 Rd, Rs1, Rs2 (xy = BB, BT, TT)

Purpose :

Multiply the signed Q15 integer contents of two 16-bit data in the corresponding portion of the 32-bit chunks in registers and then right-shift 15 bits to turn the Q30 results into Q15 numbers again and saturate the Q15 results into the destination register. If saturation happens, an overflow flag OV will be set.

Description :

Multiply the top or bottom 16-bit Q15 content of the 32-bit portions in Rs1 with the top or bottom 16-bit Q15 content of the 32-bit portion in Rs2. The Q30 results are then right-shifted 15- bits and saturated into Q15 values. The 32-bit Q15 values are then written into the 32-bit chunks in Rd. When both the two Q15 inputs are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
// KHMBB16: (x,y,z)=(0,0,0),(2,2,1)
// KHMBT16: (x,y,z)=(0,1,0),(2,3,1)
// KHMTT16: (x,y,z)=(1,1,0),(3,3,1)
aop = Rs1.H[x]; bop = Rs2.H[y];
If (0x8000 != aop | 0x8000 != bop) {
    Mresult[31:0] = aop * bop;
    res[15:0] = Mresult[30:15];
} else {
    res[15:0] = 0x7FFF;
    OV = 1;
}
Rd.W[z] = SE32(res[15:0]);
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

(RV64 Only) 32-bit Multiply Instructions

```
__STATIC_FORCEINLINE long __RV_SMBB32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMBT32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMTT32 (unsigned long a, unsigned long b)
```

group (RV64 Only) 32-bit Multiply Instructions

(RV64 Only) 32-bit Multiply Instructions

there is 3 RV64 Only) 32-bit Multiply Instructions

Functions

__STATIC_FORCEINLINE long __RV_SMBB32 (unsigned long a, unsigned long b)

SMBB32 (Signed Multiply Bottom Word & Bottom Word)

Type: DSP (RV64 Only)

Syntax:

```
SMBB32 Rd, Rs1, Rs2
SMBT32 Rd, Rs1, Rs2
SMTT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- SMBB32: bottom*bottom
- SMBT32: bottom*top
- SMTT32: top*top

Description :

For the SMBB32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. It is actually an alias of MULSR64 instruction. For the SMBT32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMTT32 instruction, it multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[0] * Rs2.W[0]; // SMBB32
res = Rs1.W[0] * Rs2.W[1]; // SMBT32
res = Rs1.W[1] * Rs2.W[1]; // SMTT32
Rd = res;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMBT32 (unsigned long a, unsigned long b)

SMBT32 (Signed Multiply Bottom Word & Top Word)

Type: DSP (RV64 Only)

Syntax:

```
SMBB32 Rd, Rs1, Rs2
SMBT32 Rd, Rs1, Rs2
SMTT32 Rd, Rs1, Rs2
```


Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- SMBB32: bottom*bottom
- SMBT32: bottom*top
- SMTT32: top*top

Description :

For the SMBB32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. It is actually an alias of MULSR64 instruction. For the SMBT32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMTT32 instruction, it multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[0] * Rs2.W[0]; // SMBB32 res = Rs1.W[0] * Rs2.w[1]; // SMBT32 res =
Rs1.W[1] * Rs2.W[1];
// SMTT32 Rd = res;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_SMTT32 (unsigned long a, unsigned long b)
```

SMTT32 (Signed Multiply Top Word & Top Word)

Type: DSP (RV64 Only)

Syntax:

```
SMBB32 Rd, Rs1, Rs2
SMBT32 Rd, Rs1, Rs2
SMTT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- SMBB32: bottom*bottom
- SMBT32: bottom*top
- SMTT32: top*top

Description :

For the SMBB32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. It is actually an alias of MULSR64 instruction. For the SMBT32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMTT32 instruction, it multiplies the top

32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[0] * Rs2.W[0]; // SMBB32 res = Rs1.W[0] * Rs2.w[1]; // SMBT32 res =
Rs1.W[1] * Rs2.W[1];
// SMTT32 Rd = res;
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

(RV64 Only) 32-bit Multiply & Add Instructions

```
__STATIC_FORCEINLINE long __RV_KMABB32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMABT32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMATT32 (long t, unsigned long a, unsigned long b)
```

group (RV64 Only) 32-bit Multiply & Add Instructions

(RV64 Only) 32-bit Multiply & Add Instructions

there are 3 (RV64 Only) 32-bit Multiply & Add Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KMABB32 (long t, unsigned long a, unsigned long b)
```

KMABB32 (Saturating Signed Multiply Bottom Words & Add)

Type: DSP (RV64 Only)

Syntax:

```
KMABB32 Rd, Rs1, Rs2
KMABT32 Rd, Rs1, Rs2
KMATT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- KMABB32: $rd + bottom * bottom$
- KMABT32: $rd + bottom * top$

- KMATT32: $rd + top * top$

Description :

For the KMABB32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMABT32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMATT32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[0] * Rs2.W[0]); // KMABB32
res = Rd + (Rs1.W[0] * Rs2.W[1]); // KMABT32
res = Rd + (Rs1.W[1] * Rs2.W[1]); // KMATT32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
*Exceptions:* None
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMABT32 (long t, unsigned long a, unsigned long b)

KMABT32 (Saturating Signed Multiply Bottom & Top Words & Add)

Type: DSP (RV64 Only)

Syntax:

```
KMABB32 Rd, Rs1, Rs2
KMABT32 Rd, Rs1, Rs2
KMATT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- KMABB32: $rd + bottom * bottom$
- KMABT32: $rd + bottom * top$
- KMATT32: $rd + top * top$

Description :

For the KMABB32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMABT32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMATT32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[0] * Rs2.W[0]); // KMABB32
res = Rd + (Rs1.W[0] * Rs2.W[1]); // KMABT32
res = Rd + (Rs1.W[1] * Rs2.W[1]); // KMATT32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
*Exceptions: * None
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_KMATT32 (long t, unsigned long a, unsigned long b)
```

KMATT32 (Saturating Signed Multiply Top Words & Add)

Type: DSP (RV64 Only)

Syntax:

```
KMABB32 Rd, Rs1, Rs2
KMABT32 Rd, Rs1, Rs2
KMATT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- KMABB32: $rd + bottom * bottom$
- KMABT32: $rd + bottom * top$
- KMATT32: $rd + top * top$

Description :

For the KMABB32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMABT32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMATT32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[0] * Rs2.W[0]); // KMABB32
res = Rd + (Rs1.W[0] * Rs2.W[1]); // KMABT32
res = Rd + (Rs1.W[1] * Rs2.W[1]); // KMATT32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
*Exceptions:* None
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

(RV64 Only) 32-bit Parallel Multiply & Add Instructions

```
__STATIC_FORCEINLINE long __RV_KMADA32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMAXDA32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMDA32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMXDA32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMADS32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMADRS32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMAXDS32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMSDA32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_KMSXDA32 (long t, unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMDS32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMDRS32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE long __RV_SMXDS32 (unsigned long a, unsigned long b)
```

group (RV64 Only) 32-bit Parallel Multiply & Add Instructions

(RV64 Only) 32-bit Parallel Multiply & Add Instructions

there are 12 (RV64 Only) 32-bit Parallel Multiply & Add Instructions

Functions

```
__STATIC_FORCEINLINE long __RV_KMADA32 (long t, unsigned long a, unsigned long b)
```

KMADA32 (Saturating Signed Multiply Two Words and Two Adds)

Type: DSP (RV64 Only)

Syntax:

```
KMADA32 Rd, Rs1, Rs2
KMAXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from 32-bit data in two registers; and then adds the two 64-bit results and 64-bit data in a third register together. The addition result may be saturated.

- KMADA32: $rd + top * top + bottom * bottom$
- KMAXDA32: $rd + top * bottom + bottom * top$

Description :

For the KMADA32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2. It is actually an alias of the KMAR64 instruction. For the KMAXDA32 instruction, it multiplies the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. The result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The 64-bit result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[1] * Rs2.w[1]) + (Rs1.W[0] * Rs2.W[0]); // KMADA32
res = Rd + (Rs1.W[1] * Rs2.W[0]) + (Rs1.W[0] * Rs2.W[1]); // KMAXDA32
if (res > (2^63)-1) {
    res = (2^63)-1;
```

(continues on next page)

(continued from previous page)

```

    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMAXDA32 (long t, unsigned long a, unsigned long b)

KMAXDA32 (Saturating Signed Crossed Multiply Two Words and Two Adds)

Type: DSP (RV64 Only)

Syntax:

```

KMADA32 Rd, Rs1, Rs2
KMAXDA32 Rd, Rs1, Rs2

```

Purpose :

Do two signed 32-bit multiplications from 32-bit data in two registers; and then adds the two 64-bit results and 64-bit data in a third register together. The addition result may be saturated.

- KMADA32: $rd + top * top + bottom * bottom$
- KMAXDA32: $rd + top * bottom + bottom * top$

Description :

For the KMADA32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2. It is actually an alias of the KMAR64 instruction. For the KMAXDA32 instruction, it multiplies the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. The result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The 64-bit result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

res = Rd + (Rs1.W[1] * Rs2.w[1]) + (Rs1.W[0] * Rs2.W[0]); // KMADA32
res = Rd + (Rs1.W[1] * Rs2.W[0]) + (Rs1.W[0] * Rs2.W[1]); // KMAXDA32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;

```

(continues on next page)

(continued from previous page)

```

    OV = 1;
}
Rd = res;

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMDA32 (unsigned long a, unsigned long b)

KMDA32 (Signed Multiply Two Words and Add)

Type: DSP (RV64 Only)

Syntax:

```

KMDA32 Rd, Rs1, Rs2
KMXDA32 Rd, Rs1, Rs2

```

Purpose :

Do two signed 32-bit multiplications from the 32-bit element of two registers; and then adds the two 64-bit results together. The addition result may be saturated.

- KMDA32: top*top + bottom*bottom
- KMXDA32: top*bottom + bottom*top

Description :

For the KMDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the KMXDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{63}-1$. The final result is written to Rd. The 32-bit contents are treated as signed integers.

Operations:

```

if ((Rs1 != 0x8000000008000000) or (Rs2 != 0x8000000008000000)) {
    Rd = (Rs1.W[1] * Rs2.W[1]) + (Rs1.W[0] * Rs2.W[0]); // KMDA32
    Rd = (Rs1.W[1] * Rs2.W[0]) + (Rs1.W[0] * Rs2.W[1]); // KMXDA32
} else {
    Rd = 0x7fffffffffffffff;
    OV = 1;
}

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMXDA32 (unsigned long a, unsigned long b)

KMXDA32 (Signed Crossed Multiply Two Words and Add)

Type: DSP (RV64 Only)**Syntax:**

```
KMDA32 Rd, Rs1, Rs2
KMXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 32-bit element of two registers; and then adds the two 64-bit results together. The addition result may be saturated.

- KMDA32: top*top + bottom*bottom
- KMXDA32: top*bottom + bottom*top

Description :

For the KMDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the KMXDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{63}-1$. The final result is written to Rd. The 32-bit contents are treated as signed integers.

Operations:

```
if ((Rs1 != 0x8000000008000000) or (Rs2 != 0x8000000008000000)) {
    Rd = (Rs1.W[1] * Rs2.W[1]) + (Rs1.W[0] * Rs2.W[0]); // KMDA32
    Rd = (Rs1.W[1] * Rs2.W[0]) + (Rs1.W[0] * Rs2.W[1]); // KMXDA32
} else {
    Rd = 0x7fffffffffffffff;
    OV = 1;
}
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMADS32 (long t, unsigned long a, unsigned long b)

KMADS32 (Saturating Signed Multiply Two Words & Subtract & Add)

Type: DSP (RV64 Only)**Syntax:**

```

KMADS32 Rd, Rs1, Rs2
KMADRS32 Rd, Rs1, Rs2
KMAXDS32 Rd, Rs1, Rs2

```

Purpose :

Do two signed 32-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 64-bit results. Then add the subtraction result to 64-bit data in a third register. The addition result may be saturated.

- KMADS32: $rd + (top * top - bottom * bottom)$
- KMADRS32: $rd + (bottom * bottom - top * top)$
- KMAXDS32: $rd + (top * bottom - bottom * top)$

Description :

For the KMADS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMADRS32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMAXDS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2. The subtraction result is then added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to

- The 64-bit result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

res = Rd + (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // KMADS32
res = Rd + (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // KMADRS32
res = Rd + (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // KMAXDS32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;

```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```

__STATIC_FORCEINLINE long __RV_KMADRS32 (long t, unsigned long a, unsigned long b)
KMADRS32 (Saturating Signed Multiply Two Words & Reverse Subtract & Add)

```

Type: DSP (RV64 Only)

Syntax:

```
KMADS32 Rd, Rs1, Rs2
KMADRS32 Rd, Rs1, Rs2
KMAXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 64-bit results. Then add the subtraction result to 64-bit data in a third register. The addition result may be saturated.

- KMADS32: $rd + (top * top - bottom * bottom)$
- KMADRS32: $rd + (bottom * bottom - top * top)$
- KMAXDS32: $rd + (top * bottom - bottom * top)$

Description :

For the KMADS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMADRS32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMAXDS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2. The subtraction result is then added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to

- a. The 64-bit result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // KMADS32
res = Rd + (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // KMADRS32
res = Rd + (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // KMAXDS32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMAXDS32 (long t, unsigned long a, unsigned long b)

KMAXDS32 (Saturating Signed Crossed Multiply Two Words & Subtract & Add)

Type: DSP (RV64 Only)

Syntax:

```
KMADS32 Rd, Rs1, Rs2
KMADRS32 Rd, Rs1, Rs2
KMAXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 64-bit results. Then add the subtraction result to 64-bit data in a third register. The addition result may be saturated.

- KMADS32: $rd + (top * top - bottom * bottom)$
- KMADRS32: $rd + (bottom * bottom - top * top)$
- KMAXDS32: $rd + (top * bottom - bottom * top)$

Description :

For the KMADS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2. For the KMADRS32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. For the KMAXDS32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2. The subtraction result is then added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to

- The 64-bit result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // KMADS32
res = Rd + (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // KMADRS32
res = Rd + (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // KMAXDS32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMSDA32 (long t, unsigned long a, unsigned long b)

KMSDA32 (Saturating Signed Multiply Two Words & Add & Subtract)

Type: DSP (RV64 Only)**Syntax:**

```
KMSDA32 Rd, Rs1, Rs2
KMSXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 32-bit element of two registers; and then subtracts the two 64-bit results from a third register. The subtraction result may be saturated.

- KMSDA: $rd = top * top - bottom * bottom$
- KMSXDA: $rd = top * bottom - bottom * top$

Description :

For the KMSDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the KMSXDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The two 64-bit multiplication results are then subtracted from the content of Rd. If the subtraction result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents are treated as signed integers.

Operations:

```
res = Rd - (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // KMSDA32
res = Rd - (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // KMSXDA32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_KMSXDA32 (long t, unsigned long a, unsigned long b)

KMSXDA32 (Saturating Signed Crossed Multiply Two Words & Add & Subtract)

Type: DSP (RV64 Only)

Syntax:

```
KMSDA32 Rd, Rs1, Rs2
KMSXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 32-bit element of two registers; and then subtracts the two 64-bit results from a third register. The subtraction result may be saturated.

- KMSDA: $rd = top * top - bottom * bottom$
- KMSXDA: $rd = top * bottom - bottom * top$

Description :

For the KMSDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the KMSXDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The two 64-bit multiplication results are then subtracted from the content of Rd. If the subtraction result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents are treated as signed integers.

Operations:

```
res = Rd - (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // KMSDA32
res = Rd - (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // KMSXDA32
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
```

Parameters

- **t** – [in] long type of value stored in t
- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMDS32 (unsigned long a, unsigned long b)

SMDS32 (Signed Multiply Two Words and Subtract)

Type: DSP (RV64 Only)

Syntax:

```
SMDS32 Rd, Rs1, Rs2
SMDRS32 Rd, Rs1, Rs2
SMXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 1 32-bit element of two registers; and then perform a subtraction operation between the two 64-bit results.

- SMDS32: top*top - bottom*bottom
- SMDRS32: bottom*bottom - top*top
- SMXDS32: top*bottom - bottom*top

Description :

For the SMDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMDRS32 instruction, it multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. For the SMXDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The subtraction result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
Rt = (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // SMDS32
Rt = (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // SMDRS32
Rt = (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // SMXDS32
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_SMDRS32 (unsigned long a, unsigned long b)
```

SMDRS32 (Signed Multiply Two Words and Reverse Subtract)

Type: DSP (RV64 Only)

Syntax:

```
SMDS32 Rd, Rs1, Rs2
SMDRS32 Rd, Rs1, Rs2
SMXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 1 32-bit element of two registers; and then perform a subtraction operation between the two 64-bit results.

- SMDS32: top*top - bottom*bottom
- SMDRS32: bottom*bottom - top*top
- SMXDS32: top*bottom - bottom*top

Description :

For the SMDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMDRS32 instruction, it multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. For the SMXDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The subtraction result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
Rt = (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // SMDS32
Rt = (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // SMDRS32
Rt = (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // SMXDS32
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long __RV_SMXDS32 (unsigned long a, unsigned long b)

SMXDS32 (Signed Crossed Multiply Two Words and Subtract)

Type: DSP (RV64 Only)

Syntax:

```
SMDS32 Rd, Rs1, Rs2
SMDRS32 Rd, Rs1, Rs2
SMXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32-bit multiplications from the 1 32-bit element of two registers; and then perform a subtraction operation between the two 64-bit results.

- SMDS32: top*top - bottom*bottom
- SMDRS32: bottom*bottom - top*top
- SMXDS32: top*bottom - bottom*top

Description :

For the SMDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2. For the SMDRS32 instruction, it multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. For the SMXDS32 instruction, it multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The subtraction result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:


```

Rt = (Rs1.W[1] * Rs2.W[1]) - (Rs1.W[0] * Rs2.W[0]); // SMDS32
Rt = (Rs1.W[0] * Rs2.W[0]) - (Rs1.W[1] * Rs2.W[1]); // SMDRS32
Rt = (Rs1.W[1] * Rs2.W[0]) - (Rs1.W[0] * Rs2.W[1]); // SMXDS32

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in long type

(RV64 Only) Non-SIMD 32-bit Shift Instructions

__RV_SRAIW_U(a, b)

group (RV64 Only) Non-SIMD 32-bit Shift Instructions

(RV64 Only) Non-SIMD 32-bit Shift Instructions

there are 1 (RV64 Only) Non-SIMD 32-bit Shift Instructions

Defines

__RV_SRAIW_U(a, b)

SRAIW.u (Rounding Shift Right Arithmetic Immediate Word)

Type: DSP (RV64 only)

Syntax:

```
SRAIW.u Rd, Rs1, imm5u
```

Purpose :

Perform a 32-bit arithmetic right shift operation with rounding. The shift amount is an immediate value.

Description :

This instruction right-shifts the lower 32-bit content of Rs1 arithmetically. The shifted out bits are filled with the sign-bit Rs1(31) and the shift amount is specified by the imm5u constant. For the rounding operation, a value of 1 is added to the most significant discarded bit of the data to calculate the final result. And the result is sign-extended and written to Rd.

Operations:

```

sa = imm5u;
if (sa != 0) {
    res[31:-1] = SE32(Rs1[31:(sa-1)]) + 1;
    Rd = SE32(res[31:0]);
} else {
    Rd = SE32(Rs1.W[0]);
}

```

Parameters

- **a** – [in] int type of value stored in a
- **b** – [in] unsigned int type of value stored in b

Returns

value stored in long type

32-bit Packing Instructions

```
__STATIC_FORCEINLINE unsigned long __RV_PKBB32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKBT32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKTT32 (unsigned long a, unsigned long b)
```

```
__STATIC_FORCEINLINE unsigned long __RV_PKTB32 (unsigned long a, unsigned long b)
```

group 32-bit Packing Instructions

32-bit Packing Instructions

There are four 32-bit packing instructions here

Functions

```
__STATIC_FORCEINLINE unsigned long __RV_PKBB32 (unsigned long a, unsigned long b)
```

PKBB32 (Pack Two 32-bit Data from Both Bottom Half)

Type: DSP (RV64 Only)

Syntax:

```
PKBB32 Rd, Rs1, Rs2
PKBT32 Rd, Rs1, Rs2
PKTT32 Rd, Rs1, Rs2
PKTB32 Rd, Rs1, Rs2
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- PKBB32: bottom.bottom
- PKBT32: bottom.top
- PKTT32: top.top
- PKTB32: top.bottom

Description :

(PKBB32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0]. (PKBT32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTT32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTB32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```

Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*0*_]); // PKBB32
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*1*_]); // PKBT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*1*_]); // PKTT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*0*_]); // PKTB32

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKBT32 (unsigned long a, unsigned long b)

PKBT32 (Pack Two 32-bit Data from Bottom and Top Half)

Type: DSP (RV64 Only)

Syntax:

```

PKBB32 Rd, Rs1, Rs2
PKBT32 Rd, Rs1, Rs2
PKTT32 Rd, Rs1, Rs2
PKTB32 Rd, Rs1, Rs2

```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- PKBB32: bottom.bottom
- PKBT32: bottom.top
- PKTT32: top.top
- PKTB32: top.bottom

Description :

(PKBB32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0]. (PKBT32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTT32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTB32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```

Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*0*_]); // PKBB32
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*1*_]); // PKBT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*1*_]); // PKTT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*0*_]); // PKTB32

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKTT32 (unsigned long a, unsigned long b)

PKTT32 (Pack Two 32-bit Data from Both Top Half)

Type: DSP (RV64 Only)

Syntax:

```
PKBB32 Rd, Rs1, Rs2
PKBT32 Rd, Rs1, Rs2
PKTT32 Rd, Rs1, Rs2
PKTB32 Rd, Rs1, Rs2
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- PKBB32: bottom.bottom
- PKBT32: bottom.top
- PKTT32: top.top
- PKTB32: top.bottom

Description :

(PKBB32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0]. (PKBT32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTT32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTB32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*0*_]); // PKBB32
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*1*_]); // PKBT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*1*_]); // PKTT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*0*_]); // PKTB32
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_PKTB32 (unsigned long a, unsigned long b)

PKTB32 (Pack Two 32-bit Data from Top and Bottom Half)

Type: DSP (RV64 Only)

Syntax:

```
PKBB32 Rd, Rs1, Rs2
PKBT32 Rd, Rs1, Rs2
PKTT32 Rd, Rs1, Rs2
PKTB32 Rd, Rs1, Rs2
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- PKBB32: bottom.bottom
- PKBT32: bottom.top
- PKTT32: top.top
- PKTB32: top.bottom

Description :

(PKBB32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0]. (PKBT32) moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTT32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0]. (PKTB32) moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*0*_]); // PKBB32
Rd = CONCAT(Rs1.W[_*0*_], Rs2.W[_*1*_]); // PKBT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*1*_]); // PKTT32
Rd = CONCAT(Rs1.W[_*1*_], Rs2.W[_*0*_]); // PKTB32
```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long type

group **RV64 Only Instructions**

RV64 Only Instructions.

Nuclei Default SIMD DSP Additional Instructions

`__STATIC_FORCEINLINE unsigned long __RV_EXPD80 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_EXPD81 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_EXPD82 (unsigned long a)`

`__STATIC_FORCEINLINE unsigned long __RV_EXPD83 (unsigned long a)`

group **Nuclei Default SIMD DSP Additional Instructions**

(RV32 & RV64)Nuclei Customized DSP Instructions

This is Nuclei customized DSP instructions for both RV32 and RV64

Functions

__STATIC_FORCEINLINE unsigned long __RV_EXP80 (unsigned long a)

EXP80 (Expand and Copy Byte 0 to 32bit(when rv32) or 64bit(when rv64))

Type: DSP

Syntax:

```
EXP80 Rd, Rs1
```

Purpose :

When rv32, Copy 8-bit data from 32-bit chunks into 4 bytes in a register. When rv64, Copy 8-bit data from 64-bit chunks into 8 bytes in a register.

Description :

Moves Rs1.B[0][7:0] to Rd.[0][7:0], Rd.[1][7:0], Rd.[2][7:0], Rd.[3][7:0]

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.B[0][7:0], Rs1.B[0][7:0], Rs1.B[0][7:0], Rs1.
↳B[0][7:0]);
for RV32: x=0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_EXP81 (unsigned long a)

EXP81 (Expand and Copy Byte 1 to 32bit(rv32) or 64bit(when rv64))

Type: DSP

Syntax:

```
EXP81 Rd, Rs1
```

Purpose :

Copy 8-bit data from 32-bit chunks into 4 bytes in a register.

Description :

Moves Rs1.B[1][7:0] to Rd.[0][7:0], Rd.[1][7:0], Rd.[2][7:0], Rd.[3][7:0]

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.B[1][7:0], Rs1.B[1][7:0], Rs1.B[1][7:0], Rs1.
↳B[1][7:0]);
for RV32: x=0
```

Parameters

a – [in] unsigned long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_EXP82 (unsigned long a)

EXP82 (Expand and Copy Byte 2 to 32bit(rv32) or 64bit(when rv64))

Type: DSP**Syntax:**

EXP82 Rd, Rs1

Purpose :

Copy 8-bit data from 32-bit chunks into 4 bytes in a register.

Description :

Moves Rs1.B[2][7:0] to Rd.[0][7:0], Rd.[1][7:0], Rd.[2][7:0], Rd.[3][7:0]

Operations:

```

Rd.W[x][31:0] = CONCAT(Rs1.B[2][7:0], Rs1.B[2][7:0], Rs1.B[2][7:0], Rs1.
↪B[2][7:0]);
for RV32: x=0

```

Parameters**a** – [in] unsigned long type of value stored in a**Returns**

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_EXP83 (unsigned long a)

EXP83 (Expand and Copy Byte 3 to 32bit(rv32) or 64bit(when rv64))

Type: DSP**Syntax:**

EXP83 Rd, Rs1

Purpose :

Copy 8-bit data from 32-bit chunks into 4 bytes in a register.

Description :

Moves Rs1.B[3][7:0] to Rd.[0][7:0], Rd.[1][7:0], Rd.[2][7:0], Rd.[3][7:0]

Operations:

```

Rd.W[x][31:0] = CONCAT(Rs1.B[3][7:0], Rs1.B[3][7:0], Rs1.B[3][7:0], Rs1.
↪B[3][7:0]);
for RV32: x=0

```

Parameters**a** – [in] unsigned long type of value stored in a**Returns**

value stored in unsigned long type

Nuclei N1 SIMD DSP Additional Instructions

```
__STATIC_FORCEINLINE unsigned long long __RV_DKHM8 (unsigned long long a,  
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKHM16 (unsigned long long a,  
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKABS8 (unsigned long long a)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKABS16 (unsigned long long a)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA8 (unsigned long long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA16 (unsigned long long a, int b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKADD8 (unsigned long long a,  
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKADD16 (unsigned long long a,  
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKSUB8 (unsigned long long a,  
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKSUB16 (unsigned long long a,  
unsigned long long b)
```

group Nuclei N1 SIMD DSP Additional Instructions

(RV32 only)Nuclei Customized N1 DSP Instructions

This is Nuclei customized DSP N1 instructions only for RV32

Functions

```
__STATIC_FORCEINLINE unsigned long long __RV_DKHM8 (unsigned long long a,  
unsigned long long b)
```

DKHM8 (64-bit SIMD Signed Saturating Q7 Multiply)

Type: SIMD

Syntax:

```
DKHM8 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```


Purpose :

Do Q7xQ7 element multiplications simultaneously. The Q14 results are then reduced to Q7 numbers again.

Description :

For the DKHM8 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2.

The Q14 results are then right-shifted 7-bits and saturated into Q7 values. The Q7 results are then written into Rd. When both the two Q7 inputs of a multiplication are 0x80, saturation will happen. The result will be saturated to 0x7F and the overflow flag OV will be set.

Operations:

```
op1t = Rs1.B[x+1]; op2t = Rs2.B[x+1]; // top
op1b = Rs1.B[x]; op2b = Rs2.B[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x80 != aop | 0x80 != bop) {
        res = (aop s* bop) >> 7;
    } else {
        res= 0x7F;
        OV = 1;
    }
}
Rd.H[x/2] = concat(rest, resb);
for RV32, x=0,2,4,6
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKHM16 (unsigned long long a, unsigned long long b)

DKHM16 (64-bit SIMD Signed Saturating Q15 Multiply)

Type: SIMD

Syntax:

```
DKHM16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do Q15xQ15 element multiplications simultaneously. The Q30 results are then reduced to Q15 numbers again.

Description :

For the DKHM16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2.

The Q30 results are then right-shifted 15-bits and saturated into Q15 values. The Q15 results are then written into Rd. When both the two Q15 inputs of a multiplication are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```
op1t = Rs1.H[x+1]; op2t = Rs2.H[x+1]; // top
op1b = Rs1.H[x]; op2b = Rs2.H[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x8000 != aop | 0x8000 != bop) {
        res = (aop s* bop) >> 15;
    } else {
        res= 0x7FFF;
        OV = 1;
    }
}
Rd.W[x/2] = concat(rest, resb);
for RV32: x=0, 2
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKABS8 (unsigned long long a)

DKABS8 (64-bit SIMD 8-bit Saturating Absolute)

Type: SIMD

Syntax:

```
DKABS8 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Get the absolute value of 8-bit signed integer elements simultaneously.

Description :

This instruction calculates the absolute value of 8-bit signed integer elements stored in Rs1 and writes the element results to Rd. If the input number is 0x80, this instruction generates 0x7f as the output and sets the OV bit to 1.

Operations:

```
src = Rs1.B[x];
if (src == 0x80) {
    src = 0x7f;
    OV = 1;
} else if (src[7] == 1)
    src = -src;
}
```

(continues on next page)

(continued from previous page)

```
Rd.B[x] = src;
for RV32: x=7...0,
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKABS16 (unsigned long long a)

DKABS16 (64-bit SIMD 16-bit Saturating Absolute)

Type: SIMD

Syntax:

```
DKABS16 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Get the absolute value of 16-bit signed integer elements simultaneously.

Description :

This instruction calculates the absolute value of 16-bit signed integer elements stored in Rs1 and writes the element results to Rd. If the input number is 0x8000, this instruction generates 0x7fff as the output and sets the OV bit to 1.

Operations:

```
src = Rs1.H[x];
if (src == 0x8000) {
    src = 0x7fff;
    OV = 1;
} else if (src[15] == 1)
    src = -src;
Rd.H[x] = src;
for RV32: x=3...0,
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA8 (unsigned long long a, int b)

DKSLRA8 (64-bit SIMD 8-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD

Syntax:

```
DKSLRA8 Rd, Rs1, Rs2
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Do 8-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q7 saturation for the left shift.

Description :

The 8-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[3:0]. Rs2[3:0] is in the signed range of $[-2^3, 2^3-1]$. A positive Rs2[3:0] means logical left shift and a negative Rs2[3:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[3:0]. However, the behavior of Rs2[3:0] == -2^3 (0x8) is defined to be equivalent to the behavior of Rs2[3:0] == $-(2^3-1)$ (0x9). The left-shifted results are saturated to the 8-bit signed integer range of $[-2^7, 2^7-1]$. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:4] will not affect this instruction.

Operations:

```
if (Rs2[3:0] < 0) {
    sa = -Rs2[3:0];
    sa = (sa == 8)? 7 : sa;
    Rd.B[x] = SE8(Rs1.B[x][7:sa]);
} else {
    sa = Rs2[2:0];
    res[(7+sa):0] = Rs1.B[x] <<(logic) sa;
    if (res > (2^7)-1) {
        res[7:0] = 0x7f; OV = 1;
    } else if (res < -2^7) {
        res[7:0] = 0x80; OV = 1;
    }
    Rd.B[x] = res[7:0];
}
for RV32: x=7...0,
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA16 (unsigned long long a, int b)

DKSLRA16 (64-bit SIMD 16-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD

Syntax:

```
DKSLRA16 Rd, Rs1, Rs2
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Do 16-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q15 saturation for the left shift.

Description :

The 16-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[4:0]. Rs2[4:0] is in the signed range of $[-2^4, 2^4-1]$. A positive Rs2[4:0] means logical left shift and a negative Rs2[4:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[4:0]. However, the behavior of Rs2[4:0]==-2⁴ (0x10) is defined to be equivalent to the behavior of Rs2[4:0]==-(2⁴-1) (0x11). The left-shifted results are saturated to the 16-bit signed integer range of $[-2^{15}, 2^{15}-1]$. After the shift, saturation, or rounding, the final results are written to Rd. If any saturation happens, this instruction sets the OV flag. The value of Rs2[31:5] will not affect this instruction.

Operations:

```
if (Rs2[4:0] < 0) {
    sa = -Rs2[4:0];
    sa = (sa == 16)? 15 : sa;
    Rd.H[x] = SE16(Rs1.H[x][15:sa]);
} else {
    sa = Rs2[3:0];
    res[(15+sa):0] = Rs1.H[x] <<(logic) sa;
    if (res > (2^15)-1) {
        res[15:0] = 0x7fff; OV = 1;
    } else if (res < -2^15) {
        res[15:0] = 0x8000; OV = 1;
    }
    d.H[x] = res[15:0];
}
for RV32: x=3...0,
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKADD8 (unsigned long long a, unsigned long long b)

DKADD8 (64-bit SIMD 8-bit Signed Saturating Addition)

Type: SIMD

Syntax:

```
DKADD8 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 8-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 8-bit signed integer elements in Rs1 with the 8-bit signed integer elements in Rs2. If any of the results are beyond the Q7 number range ($-2^7 \leq Q7 \leq 2^7-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```

res[x] = Rs1.B[x] + Rs2.B[x];
if (res[x] > 127) {
    res[x] = 127;
    OV = 1;
} else if (res[x] < -128) {
    res[x] = -128;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=7...0,

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKADD16 (unsigned long long a, unsigned long long b)

DKADD16 (64-bit SIMD 16-bit Signed Saturating Addition)

Type: SIMD

Syntax:

```

DKADD16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 16-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```

res[x] = Rs1.H[x] + Rs2.H[x];
if (res[x] > 32767) {
    res[x] = 32767;
    OV = 1;
} else if (res[x] < -32768) {
    res[x] = -32768;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=3...0,

```

Parameters

- **a** – [in] unsigned long long type of value stored in a

- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSUB8 (unsigned long long a, unsigned long long b)

DKSUB8 (64-bit SIMD 8-bit Signed Saturating Subtraction)

Type: SIMD

Syntax:

```
DKSUB8 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 8-bit signed elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 8-bit signed integer elements in Rs2 from the 8-bit signed integer elements in Rs1. If any of the results are beyond the Q7 number range ($-2^7 \leq Q7 \leq 2^7-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.B[x] - Rs2.B[x];
if (res[x] > (2^7)-1) {
    res[x] = (2^7)-1;
    OV = 1;
} else if (res[x] < -2^7) {
    res[x] = -2^7;
    OV = 1;
}
Rd.B[x] = res[x];
for RV32: x=7...0,
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSUB16 (unsigned long long a, unsigned long long b)

DKSUB16 (64-bit SIMD 16-bit Signed Saturating Subtraction)

Type: SIMD

Syntax:

```
DKSUB16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 16-bit signed integer elements in Rs2 from the 16-bit signed integer elements in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.H[x] - Rs2.H[x];
if (res[x] > (2^15)-1) {
    res[x] = (2^15)-1;
    OV = 1;
} else if (res[x] < -2^15) {
    res[x] = -2^15;
    OV = 1;
}
Rd.H[x] = res[x];
for RV32: x=3...0,
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

Nuclei N2 SIMD DSP Additional Instructions

```
__STATIC_FORCEINLINE unsigned long long __RV_DKHMx8 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKHMx16 (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DSMMUL (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DSMMUL_U (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKWMUL (unsigned long long a,
unsigned long long b)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DKWMUL_U (unsigned long long a,
unsigned long long b)
```



```
__STATIC_FORCEINLINE unsigned long long __RV_DKABS32 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA32 (unsigned long long a, int b)

__STATIC_FORCEINLINE unsigned long long __RV_DKADD32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKSUB32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRADD16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSUB16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRADD32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSUB32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DMSR16 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long long __RV_DMSR17 (unsigned long a, unsigned long b)

__STATIC_FORCEINLINE unsigned long long __RV_DMSR33 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DMXS33 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long __RV_DREDAS16 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long __RV_DREDSA16 (unsigned long long a)

__STATIC_FORCEINLINE int16_t __RV_DKCLIP64 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DKMDA (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMXDA (unsigned long long a,
unsigned long long b)
```

__STATIC_FORCEINLINE unsigned long long __RV_DSMDRS (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMXDS (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBB32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBB32_SRA14 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBB32_SRA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBT32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBT32_SRA14 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMBT32_SRA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMTT32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMTT32_SRA14 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMTT32_SRA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKBB32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKBT32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKTT32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKTB32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKTB16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKBB16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKBT16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPKTT16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSRA16 (unsigned long long a,
unsigned long b)

__STATIC_FORCEINLINE unsigned long long __RV_DADD16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DADD32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMBB16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMBT16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMTT16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRCRSA16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRCRSA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRCRAS16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRCRAS32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKCRAS16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKCRSA16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRSUB16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSTSA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSTAS32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKCRSA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKCRAS32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DCRSA32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DCRAS32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKSTSA16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKSTAS16 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DRSUB32 (unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DPACK32 (signed long a, signed long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD810 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD820 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD830 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD831 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD832 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD810 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD820 (unsigned long long a)

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD830 (unsigned long long a)

```
__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD831 (unsigned long long a)
```

```
__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD832 (unsigned long long a)
```

```
__RV_DSCLIP8(a, b)
```

```
__RV_DSCLIP16(a, b)
```

```
__RV_DSCLIP32(a, b)
```

group Nuclei N2 SIMD DSP Additional Instructions

(RV32 only)Nuclei Customized N2 DSP Instructions

This is Nuclei customized DSP N2 instructions only for RV32

Defines

```
__RV_DSCLIP8(a, b)
```

DSCLIP8 (8-bit Signed Saturation and Clip)

Type: SIMD

Syntax:

```
DSCLIP8 Rd, Rs1, imm3u[2:0]
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Limit the 8-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 8-bit signed integer elements stored in Rs1 into a signed integer range between -2^{imm3u} and $2^{\text{imm3u}}-1$, and writes the limited results to Rd. For example, if imm3u is 3, the 8-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.B[x];
if (src > (2^imm3u)-1) {
    src = (2^imm3u)-1;
    OV = 1;
} else if (src < -2^imm3u) {
    src = -2^imm3u;
    OV = 1;
}
Rd.B[x] = src
x=7...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__RV_DSCLIP16(a, b)

DSCLIP16 (16-bit Signed Saturation and Clip)

Type: SIMD

Syntax:

```
DSCLIP16 Rd, Rs1, imm4u[3:0]  
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Limit the 16-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 16-bit signed integer elements stored in Rs1 into a signed integer range between -2^{imm4u} and $2^{\text{imm4u}}-1$, and writes the limited results to Rd. For example, if imm4u is 3, the 32-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.H[x];  
if (src > (2imm4u)-1) {  
    src = (2imm4u)-1;  
    OV = 1;  
} else if (src < -2imm4u) {  
    src = -2imm4u;  
    OV = 1;  
}  
Rd.H[x] = src  
x=3...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__RV_DSCLIP32(a, b)

DSCLIP32 (32-bit Signed Saturation and Clip)

Type: SIMD

Syntax:

```
DSCLIP32 Rd, Rs1, imm5u[4:0]  
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Limit the 32-bit signed integer elements of a register into a signed range simultaneously.

Description :

This instruction limits the 32-bit signed integer elements stored in Rs1 into a signed integer range between -2^{imm5u} and $2^{\text{imm5u}}-1$, and writes the limited results to Rd. For example, if imm5u is 3, the 32-bit input values should be saturated between 7 and -8. If saturation is performed, set OV bit to 1.

Operations:

```
src = Rs1.W[x];
if (src > (2^imm5u)-1) {
    src = (2^imm5u)-1;
    OV = 1;
} else if (src < -2^imm5u) {
    src = -2^imm5u;
    OV = 1;
}
Rd.W[x] = src
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

Functions

__STATIC_FORCEINLINE unsigned long long __RV_DKHMx8 (unsigned long long a, unsigned long long b)

DKHMx8 (64-bit SIMD Signed Crossed Saturating Q7 Multiply)

Type: SIMD

Syntax:

```
DKHMx8 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do Q7xQ7 element crossed multiplications simultaneously. The Q15 results are then reduced to Q7 numbers again.

Description :

For the KHM8 instruction, multiply the top 8-bit Q7 content of 16-bit chunks in Rs1 with the bottom 8-bit Q7 content of 16-bit chunks in Rs2. At the same time, multiply the bottom 8-bit Q7 content of 16-bit chunks in Rs1 with the top 8-bit Q7 content of 16-bit chunks in Rs2.

The Q14 results are then right-shifted 7-bits and saturated into Q7 values. The Q7 results are then written into Rd. When both the two Q7 inputs of a multiplication are 0x80, saturation will happen. The result will be saturated to 0x7F and the overflow flag OV will be set.

Operations:

```

op1t = Rs1.B[x+1]; op2t = Rs2.B[x]; // top
op1b = Rs1.B[x]; op2b = Rs2.B[x+1]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x80 != aop | 0x80 != bop) {
        res = (aop s* bop) >> 7;
    } else {
        res= 0x7F;
        OV = 1;
    }
}
Rd.H[x/2] = concat(rest, resb);
for RV32, x=0,2,4,6

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKHMx16 (unsigned long long a, unsigned long long b)

DKHMx16 (64-bit SIMD Signed Crossed Saturating Q15 Multiply)

Type: SIMD

Syntax:

```

DKHMx16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do Q15xQ15 element crossed multiplications simultaneously. The Q31 results are then reduced to Q15 numbers again.

Description :

For the KHMx16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2.

The Q30 results are then right-shifted 15-bits and saturated into Q15 values. The Q15 results are then written into Rd. When both the two Q15 inputs of a multiplication are 0x8000, saturation will happen. The result will be saturated to 0x7FFF and the overflow flag OV will be set.

Operations:

```

op1t = Rs1.H[x+1]; op2t = Rs2.H[x]; // top
op1b = Rs1.H[x]; op2b = Rs2.H[x+1]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    if (0x8000 != aop | 0x8000 != bop) {
        res = (aop s* bop) >> 15;
    } else {
        res= 0x7FFF;
    }
}

```

(continues on next page)

(continued from previous page)

```

    OV = 1;
  }
}
Rd.W[x/2] = concat(rest, resb);
for RV32, x=0,2

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMMUL (unsigned long long a, unsigned long long b)

DSMMUL (64-bit MSW 32x32 Signed Multiply)

Type: SIMD

Syntax:

```

DSMMUL Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do MSW 32x32 element signed multiplications simultaneously. The results are written into Rd.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = (aop s* bop)[63:32];
}
Rd = concat(rest, resb);
x=0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DSMMUL_U (unsigned long long a,
unsigned long long b)
```

DSMMUL.u (64-bit MSW 32x32 signed Multiply with Rounding)

Type: SIMD

Syntax:

```
DSMMUL.u Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element signed multiplications simultaneously. The results are written into Rd.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2 and writes the most significant 32-bit multiplication results to the corresponding 32-bit elements of Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = RUND(aop u* bop)[63:32];
}
Rd = concat(rest, resb);
x=x+1
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DKWMUL (unsigned long long a,
unsigned long long b)
```

DKWMUL (64-bit MSW 32x32 Signed Multiply & Double)

Type: SIMD

Syntax:

```
DKWMUL Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element signed multiplications simultaneously and double. The results are written into Rd.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2. It then shifts the multiplication results one bit to the left and takes the most significant 32-bit results. If the shifted result is greater than $2^{31}-1$, it is saturated to $2^{31}-1$ and the OV flag is set to 1. The final element result is written to Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction additionally rounds up the 64-bit multiplication results by adding a 1 to bit 30 before the shift and saturation operations.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = sat.q31((aop s* bop) << 1)[63:32];
}
Rd = concat(rest, resb);
x=x+1
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKWMUL_U (unsigned long long a, unsigned long long b)

DKWMUL.u (64-bit MSW 32x32 Unsigned Multiply & Double with Rounding)

Type: SIMD

Syntax:

```
DKWMUL.u Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element unsigned multiplications simultaneously and double. The results are written into Rd.

Description :

This instruction multiplies the 32-bit elements of Rs1 with the 32-bit elements of Rs2. It then shifts the multiplication results one bit to the left and takes the most significant 32-bit results. If the shifted result is greater than $2^{31}-1$, it is saturated to $2^{31}-1$ and the OV flag is set to 1. The final element result is written to Rd. The 32-bit elements of Rs1 and Rs2 are treated as signed integers. The .u form of the instruction additionally rounds up the 64-bit multiplication results by adding a 1 to bit 30 before the shift and saturation operations.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
for ((aop,bop,res) in [(op1t,op2t,rest), (op1b,op2b,resb)]) {
    res = sat.q31(RUND(aop u* bop) << 1)[63:32];
}
```

(continues on next page)

(continued from previous page)

```
Rd = concat(rest, resb);
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKABS32 (unsigned long long a)

DKABS32 (64-bit SIMD 32-bit Saturating Absolute)

Type: SIMD

Syntax:

```
DKABS32 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Get the absolute value of 32-bit signed integer elements simultaneously.

Description :

This instruction calculates the absolute value of 32-bit signed integer elements stored in Rs1 and writes the element results to Rd. If the input number is 0x8000_0000, this instruction generates 0x7fff_ffff as the output and sets the OV bit to 1.

Operations:

```
src = Rs1.W[x];
if (src == 0x8000_0000) {
    src = 0x7fff_ffff;
    OV = 1;
} else if (src[31] == 1)
    src = -src;
}
Rd.W[x] = src;
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSLRA32 (unsigned long long a, int b)

DKSLRA32 (64-bit SIMD 32-bit Shift Left Logical with Saturation or Shift Right Arithmetic)

Type: SIMD

Syntax:

DKSLRA32 Rd, Rs1, Rs2

Rd, Rs1, Rs2 are all even/odd pair of registers

Purpose :

Do 31-bit elements logical left (positive) or arithmetic right (negative) shift operation with Q31 saturation for the left shift.

Description :

The 31-bit data elements of Rs1 are left-shifted logically or right-shifted arithmetically based on the value of Rs2[5:0]. Rs2[5:0] is in the signed range of $[-2^5, 2^5-1]$. A positive Rs2[5:0] means logical left shift and a negative Rs2[4:0] means arithmetic right shift. The shift amount is the absolute value of Rs2[5:0]. However, the behavior of Rs2[5:0]==- 2⁵ (0x20) is defined to be equivalent to the behavior of Rs2[5:0]==-(2⁵-1) (0x21).

Operations:

```
if (Rs2[5:0] < 0) {
    sa = -Rs2[5:0];
    sa = (sa == 32)? 31 : sa;
    Rd.W[x] = SE32(Rs1.W[x][31:sa]);
} else {
    sa = Rs2[4:0];
    res[(31+sa):0] = Rs1.W[x] <<(logic) sa;
    if (res > (2^31)-1) {
        res[31:0] = 0x7fff_ffff; OV = 1;
    } else if (res < -2^31) {
        res[31:0] = 0x8000_0000; OV = 1;
    }
    Rd.W[x] = res[31:0];
}
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] int type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKADD32 (unsigned long long a, unsigned long long b)

DKADD32(64-bit SIMD 32-bit Signed Saturating Addition)

Type: SIMD

Syntax:

DKADD32 Rd, Rs1, Rs2

Rd, Rs1, Rs2 are all even/odd pair of registers

Purpose :

Do 32-bit signed integer element saturating additions simultaneously.

Description :

This instruction adds the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] + Rs2.W[x];
if (res[x] > 0x7fff_ffff) {
    res[x] = 0x7fff_ffff;
    OV = 1;
} else if (res[x] < 0x8000_0000) {
    res[x] = 0x8000_0000;
    OV = 1;
}
Rd.W[x] = res[x];
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSUB32 (unsigned long long a, unsigned long long b)

DKSUB32 (64-bit SIMD 32-bit Signed Saturating Subtraction)

Type: SIMD

Syntax:

```
DKSUB32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer elements saturating subtractions simultaneously.

Description :

This instruction subtracts the 32-bit signed integer elements in Rs2 from the 32-bit signed integer elements in Rs1. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to Rd.

Operations:

```
res[x] = Rs1.W[x] - Rs2.W[x];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res[x] < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
```

(continues on next page)

(continued from previous page)

```

}
Rd.W[x] = res[x];
x=1...0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DRADD16 (unsigned long long a, unsigned long long b)

DRADD16 (64-bit SIMD 16-bit Halving Signed Addition)

Type: SIMD

Syntax:

```

DRADD16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 16-bit signed integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```

Rd.H[x] = [(Rs1.H[x]) + (Rs2.H[x])] s>> 1;
x=3...0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUB16 (unsigned long long a, unsigned long long b)

DSUB16 (64-bit SIMD 16-bit Halving Signed Subtraction)

Type: SIMD

Syntax:

```

DSUB16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 16-bit integer element subtractions simultaneously.

Description :

This instruction adds the 16-bit signed integer elements in Rs1 with the 16-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```
Rd.H[x] = [(Rs1.H[x]) - (Rs2.H[x])] ;  
x=3...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DRADD32 (unsigned long long a,  
unsigned long long b)
```

DRADD32 (64-bit SIMD 32-bit Halving Signed Addition)

Type: SIMD

Syntax:

```
DRADD32 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer element additions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit signed integer elements in Rs1 with the 32-bit signed integer elements in Rs2. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```
Rd.W[x] = [(Rs1.W[x]) + (Rs2.W[x])] s>> 1;  
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DSUB32 (unsigned long long a,  
unsigned long long b)
```


DSUB32 (64-bit SIMD 32-bit Halving Signed Subtraction)

Type: SIMD

Syntax:

```
DSUB32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit integer element subtractions simultaneously.

Description :

This instruction subtracts the 32-bit signed integer elements in Rs2 from the 32-bit signed integer elements in Rs1 . The results are written to Rd.

Operations:

```
Rd.W[x] = [(Rs1.E[x]) - (Rs2.E[x])] ;
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DMSR16 (unsigned long a, unsigned long b)

DMSR16 (Signed Multiply Halfs with Right Shift 16-bit and Cross Multiply Halfs with Right Shift 16-bit)

Type: SIMD

Syntax:

```
DMSR16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do two signed 16-bit multiplications and cross multiplications from the 16-bit elements of two registers; and each multiplications performs a right shift operation.

Description :

For the DMSR16 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2 and multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. The Q31 results are then right-shifted 16-bits and clipped to Q15 values. The Q15 results are then written into Rd.

Operations:

```

Rd.H[0] = (Rs1.H[0] s* Rs2.H[0]) s>> 16
Rd.H[1] = (Rs1.H[1] s* Rs2.H[1]) s>> 16
Rd.H[2] = (Rs1.H[1] s* Rs2.H[0]) s>> 16
Rd.H[3] = (Rs1.H[0] s* Rs2.H[1]) s>> 16

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DMSR17 (unsigned long a, unsigned long b)

DMSR17 (Signed Multiply Halves with Right Shift 17-bit and Cross Multiply Halves with Right Shift 17-bit)

Type: SIMD

Syntax:

```

DMSR17 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do two signed 16-bit multiplications and cross multiplications from the 16-bit elements of two registers; and each multiplications performs a right shift operation.

Description :

For the DMSR17 instruction, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2, multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2. At the same time, multiply the top 16-bit Q15 content of 32-bit chunks in Rs1 with the bottom 16-bit Q15 content of 32-bit chunks in Rs2 and multiply the bottom 16-bit Q15 content of 32-bit chunks in Rs1 with the top 16-bit Q15 content of 32-bit chunks in Rs2. The Q31 results are then right-shifted 17-bits and clipped to Q15 values. The Q15 results are then written into Rd.

Operations:

```

Rd.H[0] = (Rs1.H[0] s* Rs2.H[0]) s>> 17
Rd.H[1] = (Rs1.H[1] s* Rs2.H[1]) s>> 17
Rd.H[2] = (Rs1.H[1] s* Rs2.H[0]) s>> 17
Rd.H[3] = (Rs1.H[0] s* Rs2.H[1]) s>> 17

```

Parameters

- **a** – [in] unsigned long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DMSR33 (unsigned long long a, unsigned long long b)

DMSR33 (Signed Multiply with Right Shift 33-bit and Cross Multiply with Right Shift 33-bit)

Type: SIMD

Syntax:

```
DMSR33 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do two signed 32-bit multiplications from the 32-bit elements of two registers, and each multiplications performs a right shift operation.

Description :

For the DMSR33 instruction, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. The Q64 results are then right-shifted 33-bits and clipped to Q31 values. The Q31 results are then written into Rd.

Operations:

```
Rd.W[0] = (Rs1.W[0] s* Rs2.W[0]) s>> 33
Rd.W[1] = (Rs1.W[1] s* Rs2.W[1]) s>> 33
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DMCSR33 (unsigned long long a, unsigned long long b)

DMCSR33 (Signed Multiply with Right Shift 33-bit and Cross Multiply with Right Shift 33-bit)

Type: SIMD

Syntax:

```
DMCSR33 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do two signed 32-bit cross multiplications from the 32-bit elements of two registers, and each multiplications performs a right shift operation.

Description :

For the DMCSR33 instruction, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. The Q63 results are then right-shifted 33-bits and clipped to Q31 values. The Q31 results are then written into Rd.

Operations:

```
Rd.W[0] = (Rs1.W[0] s* Rs2.W[1]) s>> 33
Rd.W[1] = (Rs1.W[1] s* Rs2.W[0]) s>> 33
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long __RV_DREDAS16 (unsigned long long a)

DREDAS16 (Reduced Addition and Reduced Subtraction)

Type: SIMD

Syntax:

```
DREDAS16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do halves reduced subtraction and halves reduced addition from a register. The result is written to Rd.

Description :

For the DREDAS16 instruction, subtract the top 16-bit Q15 element from the bottom 16-bit Q15 element of the bottom 32-bit Q31 content of 64-bit chunks in Rs1. At the same time, add the the top16-bit Q15 element with the bottom16-bit Q15 element of the top 32-bit Q31 content of 64-bit chunks in Rs1. The two Q15 results are then written into Rd.

Operations:

```
Rd.H[0] = Rs1.H[0] - Rs1.H[1]
Rd.H[1] = Rs1.H[2] + Rs1.H[3]
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE unsigned long __RV_DREDSA16 (unsigned long long a)

DREDSA16 (Reduced Subtraction and Reduced Addition)

Type: SIMD

Syntax:

```
DREDSA16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do halves reduced subtraction and halves reduced addition from a register. The result is written to Rd.

Description :

For the DREDSA16 instruction, add the top 16-bit Q15 element from the bottom 16-bit Q15 element of the bottom 32-bit Q31 content of 64-bit chunks in Rs1. At the same time, subtract the the top16-bit Q15 element with the bottom16-bit Q15 element of the top 32-bit Q31 content of 64-bit chunks in Rs1. The two Q15 results are then written into Rd.

Operations:

```
Rd.H[0] = Rs1.H[0] + Rs1.H[1]
Rd.H[1] = Rs1.H[2] - Rs1.H[3]
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long type

__STATIC_FORCEINLINE int16_t __RV_DKCLIP64 (unsigned long long a)

DKCLIP64 (64-bit Clipped to 16-bit Saturation Value)

Type: SIMD

Syntax:

```
DKCLIP64 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 15-bit element arithmetic right shift operations and limit result into 32-bit int, then do saturate operation to 16-bit and clip result to 16-bit Q15.

Description :

For the DKCLIP64 instruction, shift the input 15 bits to the right and data convert the result to 32-bit int type, after which the input is saturated to limit the data to between $2^{15}-1$ and -2^{15} . the result is converted to 16-bits q15 type. The final results are written to Rd.

Operations:

```
const int32_t max = (int32_t)((1U << 15U) - 1U);
const int32_t min = -1 - max;
int32_t val = (int32_t)(Rs s >> 15);
if (val > max) {
    Rd = max;
} else if (val < min) {
    Rd = min;
} else {
    Rd = (int16_t)val;
}
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in int16_t type

```
__STATIC_FORCEINLINE unsigned long long __RV_DKMDA (unsigned long long a,
unsigned long long b)
```

DKMDA (Signed Multiply Two Halfs and Add)

Type: SIMD

Syntax:

```
DKMDA Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results together. The addition result may be saturated.

Description :

This instruction multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{31}-1$. The final results are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```
if (Rs1.W[x] != 0x80008000) or (Rs2.W[x] != 0x80008000){
    Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[1]) + (Rs1.W[x].H[0] * Rs2.W[x].H[0]);
} else {
    Rd.W[x] = 0x7fffffff;
    OV = 1;
}
x=1..0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DKMXDA (unsigned long long a,
unsigned long long b)
```

DKMXDA (Signed Crossed Multiply Two Halfs and Add)

Type: SIMD

Syntax:

```
DKMXDA Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then adds the two 32-bit results together. The addition result may be saturated.

- DKMXDA: top*bottom + bottom*top (per 32-bit element)

Description :

This instruction multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The addition result is checked for saturation. If saturation happens, the result is saturated to $2^{31}-1$. The final results are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```
if (Rs1.W[x] != 0x80008000) or (Rs2.W[x] != 0x80008000){
  Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[0]) + (Rs1.W[x].H[0] * Rs2.W[x].H[1]);
} else {
  Rd.W[x] = 0x7fffffff;
  OV = 1;
}
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMDRS (unsigned long long a, unsigned long long b)

DSMDRS (Signed Multiply Two Halfs and Reverse Subtract)

Type: SIMD

Syntax:

DSMDRS Rd, Rs1, Rs2

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results.

- DSMDRS: bottom*bottom - top*top (per 32-bit element)

Description :

This instruction multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction result is written to the corresponding 32-bit element of Rd (The 16-bit contents of multiplication are treated as signed integers).

Operations:

```
Rd.W[x] = (Rs1.W[x].H[0] * Rs2.W[x].H[0]) - (Rs1.W[x].H[1] * Rs2.W[x].H[1]); x=
↪ 1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

- **b** – [in] unsigned long long type of value stored in **b**

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMXDS (unsigned long long a, unsigned long long b)

DSMXDS (Signed Crossed Multiply Two Halfs and Subtract)

Type: SIMD

Syntax:

DSMXDS Rd, Rs1, Rs2

Purpose :

Do two signed 16-bit multiplications from the 32-bit elements of two registers; and then perform a subtraction operation between the two 32-bit results.

- DSMXDS: top*bottom - bottom*top (per 32-bit element)

Description :

This instruction multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The subtraction result is written to the corresponding 32-bit element of Rd. The 16-bit contents of multiplication are treated as signed integers.

Operations:

$$Rd.W[x] = (Rs1.W[x].H[1] * Rs2.W[x].H[0]) - (Rs1.W[x].H[0] * Rs2.W[x].H[1]); x \in \{0, 1, 2, 3\}$$

Parameters

- **a** – [in] unsigned long long type of value stored in **a**
- **b** – [in] unsigned long long type of value stored in **b**

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE long long __RV_DSMBB32 (unsigned long long a, unsigned long long b)

DSMBB32 (Signed Multiply Bottom Word & Bottom Word)

Type: SIMD

Syntax:

DSMBB32 Rd, Rs1, Rs2

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- DSMBB32: bottom*bottom

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]);
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMBB32_SRA14 (unsigned long long a, unsigned long long b)

DSMBB32.sra14 (Signed Crossed Multiply Two Halfs and Subtract with Right Shift 14)

Type: SIMD

Syntax:

```
DSMBB32.sra14 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 14- bit,finally write the 64-bit result to a third register.

- DSMBB32.sra14: bottom*bottom s>> 14

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 14-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]) s>> 14;
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMBB32_SRA32 (unsigned long long a, unsigned long long b)

DSMBB32.sra32 (Signed Crossed Multiply Two Halfs and Subtract with Right Shift 32)

Type: SIMD

Syntax:

```
DSMBB32.sra32 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 32-bit, finally write the 64-bit result to a third register.

- DSMBB32.sra32: bottom*bottom s >> 32

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 32-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]) s>> 32;  
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

**__STATIC_FORCEINLINE long long __RV_DSMBT32 (unsigned long long a,
unsigned long long b)**

DSMBT32 (Signed Multiply Bottom Word & Top Word)

Type: SIMD

Syntax:

```
DSMBT32 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- DSMBT32: bottom*top

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]);
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMBT32_SRA14 (unsigned long long a, unsigned long long b)

DSMBT32.sra14 (Signed Multiply Bottom Word & Top Word with Right Shift 14)

Type: SIMD

Syntax:

```
DSMBT32.sra14 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 14- bit, finally write the 64-bit result to a third register.

- DSMBT32.sra14: bottom*bottom s>> 14

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 14-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]) s>> 14;
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMBT32_SRA32 (unsigned long long a, unsigned long long b)

DSMBT32.sra32 (Signed Crossed Multiply Two Halfs and Subtract with Right Shift 32)

Type: SIMD

Syntax:

```
DSMBT32.sra32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 32-bit, finally write the 64-bit result to a third register.

- DSMBT32.sra32: bottom*bottom s>> 32

Description :

This instruction multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 32-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = (Rs1.W[0] * Rs2.W[0]) s>> 32;
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DSMTT32 (unsigned long long a,
unsigned long long b)
```

DSMTT32 (Signed Multiply Top Word & Top Word)

Type: SIMD

Syntax:

```
DSMTT32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register and write the 64-bit result to a third register.

- DSMTT32: top*top

Description :

This instruction multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[1] * Rs2.W[1];
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMTT32_SRA14 (unsigned long long a, unsigned long long b)

DSMTT32.sra14 (Signed Multiply Top Word & Top Word with Right Shift 14-bit)

Type: SIMD

Syntax:

```
DSMTT32.sra14 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 14-bit, finally write the 64-bit result to a third register.

- DSMTT32.sra14: $\text{top} * \text{top} \gg 14$

Description :

This instruction multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 14-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[1] * Rs2.W[1] >> 14;
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMTT32_SRA32 (unsigned long long a, unsigned long long b)

DSMTT32.sra32 (Signed Multiply Top Word & Top Word with Right Shift 32-bit)

Type: SIMD

Syntax:

```
DSMTT32.sra32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 32-bit element of a register with the signed 32-bit element of another register, then right shift 32-bit, finally write the 64-bit result to a third register.

- DSMTT32.sra32: $\text{top} * \text{top} \gg 32$

Description :

This instruction multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2. The 64-bit multiplication result is written to Rd after right shift 32-bit. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rs1.W[1] * Rs2.W[1] >> 32;  
Rd = res;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKBB32 (unsigned long long a, unsigned long long b)

DPKBB32 (Pack Two 32-bit Data from Both Bottom Half)

Type: SIMD

Syntax:

```
DPKBB32 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- DPKBB32: bottom.bottom

Description :

This instruction moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[0], Rs2.W[0]);
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKBT32 (unsigned long long a, unsigned long long b)

DPKBT32 (Pack Two 32-bit Data from Bottom and Top Half)

Type: SIMD

Syntax:

```
DPKBT32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- DPKBT32: bottom.top

Description :

This instruction moves Rs1.W[0] to Rd.W[1] and moves Rs2.W[1] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[0], Rs2.W[1]);
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DPKTT32 (unsigned long long a,
unsigned long long b)
```

DPKTT32 (Pack Two 32-bit Data from Both Top Half)

Type: SIMD

Syntax:

```
DPKTT32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- DPKTT32: top.top

Description :

This instruction moves Rs1.W[1] to Rd.W[0] and moves Rs2.W[1] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[1], Rs2.W[1]);
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKTB32 (unsigned long long a, unsigned long long b)

DPKTB32 (Pack Two 32-bit Data from Top and Bottom Half)

Type: SIMD

Syntax:

```
DPKTB32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 32-bit data from 64-bit chunks in two registers.

- DPKTB32: top.bottom

Description :

This instruction moves Rs1.W[1] to Rd.W[1] and moves Rs2.W[0] to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W[1], Rs2.W[0]);
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKTB16 (unsigned long long a, unsigned long long b)

DPKTB16 (Pack Two 32-bit Data from Top and Bottom Half)

Type: SIMD

Syntax:

```
DPKTB16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- DPKTB16: top.bottom

Description :

This instruction moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [15:0] to Rd.W[x] [15:0].

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][15:0]);
x=1...0
```


Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKBB16 (unsigned long long a, unsigned long long b)

DPKBB16 (Pack Two 16-bit Data from Both Bottom Half)

Type: SIMD

Syntax:

```
DPKBB16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBB16: bottom.bottom

Description :

This instruction moves Rs1.W[x][15:0] to Rd.W[x][31:16] and moves Rs2.W[x][15:0] to Rd.W[x][15:0].

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][15:0]);
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKBT16 (unsigned long long a, unsigned long long b)

DPKBT16 (Pack Two 16-bit Data from Bottom and Top Half)

Type: SIMD

Syntax:

```
DPKBT16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKBT16: bottom.top

Description :

This instruction moves Rs1.W[x] [15:0] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.W[x][15:0], Rs2.W[x][31:16]);
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPKTT16 (unsigned long long a, unsigned long long b)

DPKTT16 (Pack Two 16-bit Data from Both Top Half)

Type: SIMD

Syntax:

```
DPKTT16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Pack 16-bit data from 32-bit chunks in two registers.

- PKTT16 top.top

Description :

This instruction moves Rs1.W[x] [31:16] to Rd.W[x] [31:16] and moves Rs2.W[x] [31:16] to Rd.W[x] [15:0].

Operations:

```
Rd.W[x][31:0] = CONCAT(Rs1.W[x][31:16], Rs2.W[x][31:16]);
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSRA16 (unsigned long long a, unsigned long long b)

DSRA16 (SIMD 16-bit Shift Right Arithmetic)

Type: SIMD

Syntax:

```
DSRA16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit element arithmetic right shift operations simultaneously. The shift amount is a variable from a GPR.

Description :

The 16-bit data elements in Rs1 are right-shifted arithmetically, that is, the shifted out bits are filled with the sign-bit of the data elements. The shift amount is specified by the low-order 4-bits of the value in the Rs2 register. And the results are written to Rd.

Operations:

```
sa = Rs2[3:0];
if (sa != 0)
{
  Rd.H[x] = SE16(Rs1.H[x][15:sa]);
} else {
  Rd = Rs1;
}
x=3...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DADD16 (unsigned long long a,
unsigned long long b)
```

DADD16 (16-bit Addition)

Type: SIMD

Syntax:

```
DADD16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit integer element additions simultaneously.

Description :

This instruction adds the 16-bit integer elements in Rs1 with the 16-bit integer elements in Rs2. And the results are written to Rd.

Note :

This instruction can be used for either signed or unsigned addition.

Operations:

```
Rd.H[x] = Rs1.H[x] + Rs2.H[x];  
x=3...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DADD32 (unsigned long long a,
unsigned long long b)**

DADD32 (32-bit Addition)

Type: SIMD

Syntax:

```
DADD32 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit integer element additions simultaneously.

Description :

This instruction adds the 32-bit integer elements in Rs1 with the 32-bit integer elements in Rs2, and then writes the 32-bit element results to Rd.

Operations:

```
Rd.W[x] = Rs1.W[x] + Rs2.W[x];  
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DSMBB16 (unsigned long long a,
unsigned long long b)**

DSMBB16 (Signed Multiply Bottom Half & Bottom Half)

Type: SIMD

Syntax:

```
DSMBB16 Rd, Rs1, Rs2  
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- DSMBB16: $W[x].bottom * W[x].bottom$

Description :

For the DSMBB16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[0];
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMBT16 (unsigned long long a, unsigned long long b)

DSMBT16 (Signed Multiply Bottom Half & Top Half)

Type: SIMD

Syntax:

```
DSMBT16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- DSMBT16: $W[x].bottom * W[x].top$

Description :

For the DSMBT16 instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
Rd.W[x] = Rs1.W[x].H[0] * Rs2.W[x].H[1];
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMTT16 (unsigned long long a, unsigned long long b)

DSMTT16 (Signed Multiply Top Half & Top Half)

Type: SIMD

Syntax:

```
DSMTT16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the signed 16-bit content of the 32-bit elements of another register and write the result to a third register.

- DSMTT16: $W[x].top * W[x].top$

Description :

For the DSMTT16 instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are written to Rd. The 16-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
Rd.W[x] = Rs1.W[x].H[1] * Rs2.W[x].H[1];
x=1..0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DRCRSA16 (unsigned long long a, unsigned long long b)

DRCRSA16 (16-bit Signed Halving Cross Subtraction & Addition)

Type: SIMD

Syntax:

```
DRCRSA16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer element subtraction and 16-bit signed integer element addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit signed integer in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer in [15:0] of 32-bit chunks in Rs2, and adds the 16-bit signed integer in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer in [15:0] of 32-bit chunks in Rs1. The element results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] - Rs2.W[x][15:0]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] + Rs2.W[x][31:16]) s>> 1;
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DRCRSA32 (unsigned long long a, unsigned long long b)

DRCRSA32 (32-bit Signed Halving Cross Subtraction & Addition)

Type: SIMD

Syntax:

```
DRCRSA32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer element subtraction and 32-bit signed integer element addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit signed integer element in [63:32] of Rs1 with the 32-bit signed integer element in [31:0] of Rs2, and adds the 32-bit signed integer element in [63:32] of Rs2 from the 32-bit signed integer element in [31:0] of Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```
Rd.W[1] = (Rs1.W[1] - Rs2.W[0]) s>> 1;
Rd.W[0] = (Rs1.W[0] + Rs2.W[1]) s>> 1;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DRCRAS16 (unsigned long long a,
unsigned long long b)
```

DRCRAS16 (16-bit Signed Halving Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
DRCRAS16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer element addition and 16-bit signed integer element subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 16-bit signed integer in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer in [15:0] of 32-bit chunks in Rs2, and subtracts the 16-bit signed integer in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer in [15:0] of 32-bit chunks in Rs1. The element results are first logically right-shifted by 1 bit and then written to [31:16] of 32-bit chunks in Rd and [15:0] of 32-bit chunks in Rd.

Operations:

```
Rd.W[x][31:16] = (Rs1.W[x][31:16] + Rs2.W[x][15:0]) s>> 1;
Rd.W[x][15:0] = (Rs1.W[x][15:0] - Rs2.W[x][31:16]) s>> 1;
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DRCRAS32 (unsigned long long a,
unsigned long long b)
```

DRCRAS32 (32-bit Signed Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
DRCRAS32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer element addition and 32-bit signed integer element subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements. The results are halved to avoid overflow or saturation.

Description :

This instruction adds the 32-bit signed integer element in [63:32] of Rs1 with the 32-bit signed integer element in [31:0] of Rs2, and subtracts the 32-bit signed integer element in [63:32] of Rs2 from the 32-bit

signed integer element in [31:0] of Rs1. The element results are first arithmetically right-shifted by 1 bit and then written to [63:32] of Rd for addition and [31:0] of Rd for subtraction.

Operations:

```
Rd.W[1] = (Rs1.W[1] + Rs2.W[0]) s>> 1;
Rd.W[0] = (Rs1.W[0] - Rs2.W[1]) s>> 1;
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKCRAS16 (unsigned long long a, unsigned long long b)

DKCRAS16 (16-bit Signed Saturating Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
DKCRAS16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer element saturating addition and 16-bit signed integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for addition and [15:0] of 32-bit chunks in Rd for subtraction.

Operations:

```
res1 = Rs1.W[x][31:16] + Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] - Rs2.W[x][31:16];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
x=1...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKCRSA16 (unsigned long long a, unsigned long long b)

DKCRSA16 (16-bit Signed Saturating Cross Subtraction & Addition)

Type: SIMD

Syntax:

DKCRSA16 Rd, Rs1, Rs2
Rd, Rs1, Rs2 are all even/odd pair of registers

Purpose :

Do 16-bit signed integer element saturating subtraction and 16-bit signed integer element saturating addition in a 32-bit chunk simultaneously. Operands are from crossed positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```
res1 = Rs1.W[x][31:16] - Rs2.W[x][15:0];
res2 = Rs1.W[x][15:0] + Rs2.W[x][31:16];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
x=1..0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DRSUB16 (unsigned long long a, unsigned long long b)

DRSUB16 (16-bit Signed Halving Subtraction)

Type: SIMD

Syntax:

```
DRSUB16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 16-bit signed integer elements in Rs2 from the 16-bit signed integer elements in Rs1. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```
Rd.H[x] = (Rs1.H[x] - Rs2.H[x]) s>> 1;
x=3...0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSTSA32 (unsigned long long a, unsigned long long b)

DSTSA32 (32-bit Straight Subtraction & Addition)

Type: SIMD

Syntax:

```
DSTSA32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit integer element subtraction and 32-bit integer element addition in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [63:32] of Rs1, and writes the result to [63:32] of Rd; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [31:0] of Rs2, and writes the result to [31:0] of Rd.

Operations:

```
Rd.W[1] = Rs1.W[1] - Rs2.W[1];
Rd.W[0] = Rs1.W[0] + Rs2.W[0];
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSTAS32 (unsigned long long a, unsigned long long b)

DSTAS32 (SIMD 32-bit Straight Addition & Subtraction)

Type: SIMD

Syntax:

```
DSTAS32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit integer element addition and 32-bit integer element subtraction in a 64-bit chunk simultaneously. Operands are from corresponding 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [63:32] of Rs2, and writes the result to [63:32] of Rd; at the same time, it subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [31:0] of Rs1, and writes the result to [31:0] of Rd.

Operations:

```
Rd.W[1] = Rs1.W[1] + Rs2.W[1];
Rd.W[0] = Rs1.W[0] - Rs2.W[0];
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKCRSA32 (unsigned long long a, unsigned long long b)

DKCRSA32 (32-bit Signed Saturating Cross Subtraction & Addition)

Type: SIMD

Syntax:

```
DKCRSA32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer element saturating subtraction and 32-bit signed integer element saturating addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction subtracts the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [63:32] of Rs1; at the same time, it adds the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [63:32] of Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```
res[1] = Rs1.W[1] - Rs2.W[0];
res[0] = Rs1.W[0] + Rs2.W[1];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKCRAS32 (unsigned long long a, unsigned long long b)

DKCRAS32 (32-bit Signed Saturating Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
DKCRAS32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit signed integer element saturating subtraction and 32-bit signed integer element saturating addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [31:0] of Rs2 from the 32-bit integer element in [63:32] of Rs1; at the same time, it subtracts the 32-bit integer element in [31:0] of Rs1 with the 32-bit integer element in [63:32] of Rs2. If any of the results are beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [63:32] of Rd for subtraction and [31:0] of Rd for addition.

Operations:

```

res[1] = Rs1.W[1] + Rs2.W[0];
res[0] = Rs1.W[0] - Rs2.W[1];
if (res[x] > (2^31)-1) {
    res[x] = (2^31)-1;
    OV = 1;
} else if (res < -2^31) {
    res[x] = -2^31;
    OV = 1;
}
Rd.W[1] = res[1];
Rd.W[0] = res[0];

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DCRSA32 (unsigned long long a, unsigned long long b)

DCRSA32 (32-bit Cross Subtraction & Addition)

Type: SIMD

Syntax:

```

DCRSA32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 32-bit integer element subtraction and 32-bit integer element addition in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction adds the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [31:0] of Rs2, and writes the result to [63:32] of Rd; at the same time, it subtracts the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [31:0] of Rs1, and writes the result to [31:0] of Rd.

Operations:

```

res[1] = Rs1.W[1] - Rs2.W[0];
res[0] = Rs1.W[0] + Rs2.W[1];

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DCRAS32 (unsigned long long a, unsigned long long b)

DCRAS32 (32-bit Cross Addition & Subtraction)

Type: SIMD

Syntax:

```
DCRAS32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 32-bit integer element addition and 32-bit integer element subtraction in a 64-bit chunk simultaneously. Operands are from crossed 32-bit elements.

Description :

This instruction subtracts the 32-bit integer element in [63:32] of Rs1 with the 32-bit integer element in [31:0] of Rs2, and writes the result to [63:32] of Rd; at the same time, it adds the 32-bit integer element in [63:32] of Rs2 from the 32-bit integer element in [31:0] of Rs1, and writes the result to [31:0] of Rd.

Operations:

```
res[1] = Rs1.W[1] - Rs2.W[0];
res[0] = Rs1.W[0] + Rs2.W[1];
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSTSA16 (unsigned long long a, unsigned long long b)

DKSTSA16 (16-bit Signed Saturating Straight Subtraction & Addition)

Type: SIMD

Syntax:

```
DKSTSA16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do 16-bit signed integer element saturating subtraction and 16-bit signed integer element saturating addition in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction subtracts the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1; at the same time, it adds the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 with the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```

res1 = Rs1.W[x][31:16] - Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] + Rs2.W[x][15:0];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {
        res = (2^15)-1;
        OV = 1;
    } else if (res < -2^15) {
        res = -2^15;
        OV = 1;
    }
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
x=1...0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSTAS16 (unsigned long long a, unsigned long long b)

DKSTAS16 (16-bit Signed Saturating Straight Addition & Subtraction)

Type: SIMD

Syntax:

```

DKSTAS16 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 16-bit signed integer element saturating addition and 16-bit signed integer element saturating subtraction in a 32-bit chunk simultaneously. Operands are from corresponding positions in 32-bit chunks.

Description :

This instruction adds the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs1 with the 16-bit signed integer element in [31:16] of 32-bit chunks in Rs2; at the same time, it subtracts the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs2 from the 16-bit signed integer element in [15:0] of 32-bit chunks in Rs1. If any of the results are beyond the Q15 number range ($-2^{15} \leq Q15 \leq 2^{15}-1$), they are saturated to the range and the OV bit is set to 1. The saturated results are written to [31:16] of 32-bit chunks in Rd for subtraction and [15:0] of 32-bit chunks in Rd for addition.

Operations:

```

res1 = Rs1.W[x][31:16] + Rs2.W[x][31:16];
res2 = Rs1.W[x][15:0] - Rs2.W[x][15:0];
for (res in [res1, res2]) {
    if (res > (2^15)-1) {

```

(continues on next page)

(continued from previous page)

```

    res = (2^15)-1;
    OV = 1;
} else if (res < -2^15) {
    res = -2^15;
    OV = 1;
}
}
Rd.W[x][31:16] = res1;
Rd.W[x][15:0] = res2;
x=1...0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DRSUB32 (unsigned long long a, unsigned long long b)

DRSUB32 (32-bit Signed Halving Subtraction)

Type: SIMD

Syntax:

```

DRSUB32 Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do 32-bit signed integer element subtractions simultaneously. The results are halved to avoid overflow or saturation.

Description :

This instruction subtracts the 32-bit signed integer elements in Rs2 from the 32-bit signed integer elements in Rs1. The results are first arithmetically right-shifted by 1 bit and then written to Rd.

Operations:

```

Rd.W[x] = (Rs1.W[x] - Rs2.W[x]) s>> 1;
x=1...0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DPACK32 (signed long a, signed long b)

DPACK32 (SIMD Pack Two 32-bit Data To 64-bit)

Type: SIMD

Syntax:

```
DPACK32 Rd, Rs1, Rs2  
# Rd is even/odd pair of register
```

Purpose :

Pack two 32-bit datas which from two registers into a 64-bit data.

Description :

This instruction moves 32-bit Rs1 to Rd.W[1] and moves 32-bit Rs2 to Rd.W[0].

Operations:

```
Rd = CONCAT(Rs1.W , Rs2.W);
```

Parameters

- **a** – [in] signed long type of value stored in a
- **b** – [in] signed long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD810 (unsigned long long a)

DSUNPKD810 (Signed Unpacking Bytes 1 & 0)

Type: SIMD

Syntax:

```
DSUNPKD810 Rd, Rs1  
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 1 and byte 0 of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the DSUNPKD810 instruction, it unpacks byte 1 and byte 0 of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[1])  
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD820 (unsigned long long a)

DSUNPKD820 (Signed Unpacking Bytes 2 & 0)

Type: SIMD

Syntax:

```
DSUNPKD820 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 2 and byte 0 of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the DSUNPKD820 instruction, it unpacks byte 2 and byte 0 of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[2])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD830 (unsigned long long a)

DSUNPKD830 (Signed Unpacking Bytes 3 & 0)

Type: SIMD

Syntax:

```
DSUNPKD830 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 3 and byte 0 of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the DSUNPKD830 instruction, it unpacks byte 3 and byte 0 of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD831 (unsigned long long a)

DSUNPKD831 (Signed Unpacking Bytes 3 & 1)

Type: SIMD

Syntax:

DSUNPKD831 Rd, Rs1
Rd, Rs1 are all even/odd pair of registers

Purpose :

Unpack byte 3 and byte 1 of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the DSUNPKD831 instruction, it unpacks byte 3 and byte 1 of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

$Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])$
 $Rd.W[m].H[0] = SE16(Rs1.W[m].B[1])$

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSUNPKD832 (unsigned long long a)

DSUNPKD832 (Signed Unpacking Bytes 3 & 2)

Type: SIMD

Syntax:

DSUNPKD832 Rd, Rs1
Rd, Rs1 are all even/odd pair of registers

Purpose :

Unpack byte 3 and byte 2 of 32-bit chunks in a register into two 16-bit signed halfwords of 32-bit chunks in a register.

Description :

For the DSUNPKD832 instruction, it unpacks byte 3 and byte 2 of 32-bit chunks in Rs1 into two 16-bit signed halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

$Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])$
 $Rd.W[m].H[0] = SE16(Rs1.W[m].B[2])$

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD810 (unsigned long long a)

DZUNPKD810 (UnSigned Unpacking Bytes 1 & 0)

Type: SIMD

Syntax:

```
DZUNPKD810 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 1 and byte 0 of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the DZUNPKD810 instruction, it unpacks byte 1 and byte 0 of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[1])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD820 (unsigned long long a)

DZUNPKD820 (UnSigned Unpacking Bytes 2 & 0)

Type: SIMD

Syntax:

```
DZUNPKD820 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 2 and byte 0 of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the DZUNPKD820 instruction, it unpacks byte 2 and byte 0 of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[2])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD830 (unsigned long long a)

DZUNPKD830 (UnSigned Unpacking Bytes 3 & 0)

Type: SIMD

Syntax:

```
DZUNPKD830 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 3 and byte 0 of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the DZUNPKD830 instruction, it unpacks byte 3 and byte 0 of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[0])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD831 (unsigned long long a)

DZUNPKD831 (UnSigned Unpacking Bytes 3 & 1)

Type: SIMD

Syntax:

```
DZUNPKD831 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 3 and byte 1 of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the DZUNPKD831 instruction, it unpacks byte 3 and byte 1 of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[1])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DZUNPKD832 (unsigned long long a)

DZUNPKD832 (UnSigned Unpacking Bytes 3 & 2)

Type: SIMD

Syntax:

```
DZUNPKD832 Rd, Rs1
# Rd, Rs1 are all even/odd pair of registers
```

Purpose :

Unpack byte 3 and byte 2 of 32-bit chunks in a register into two 16-bit unsigned halfwords of 32-bit chunks in a register.

Description :

For the DZUNPKD832 instruction, it unpacks byte 3 and byte 2 of 32-bit chunks in Rs1 into two 16-bit unsigned halfwords and writes the results to the top part and the bottom part of 32-bit chunks in Rd.

Operations:

```
Rd.W[m].H[1] = SE16(Rs1.W[m].B[3])
Rd.W[m].H[0] = SE16(Rs1.W[m].B[2])
```

Parameters

a – [in] unsigned long long type of value stored in a

Returns

value stored in unsigned long long type

Nuclei N3 SIMD DSP Additional Instructions

__STATIC_FORCEINLINE unsigned long long __RV_DKMMAC (unsigned long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMMAC_U (unsigned long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMMSB (unsigned long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMSB_U (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMADA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMAXDA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMADS (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMADRS (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMAXDS (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMSDA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DKMSXDA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMAQA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DSMAQA_SU (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE unsigned long long __RV_DUMAQA (unsigned long long t,
unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMDA32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMXDA32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMADA32 (long long t, unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMAXDA32 (long long t, unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMADS32 (long long t, unsigned long long a,
unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMADRS32 (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMAXDS32 (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMSDA32 (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DKMSXDA32 (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMDS32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMDRS32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMXDS32 (unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMALDA (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMALXDA (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMALDS (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMALDRS (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMALXDS (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMSLDA (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DSMSLXDA (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DD SMAQA (long long t, unsigned long long a, unsigned long long b)

__STATIC_FORCEINLINE long long __RV_DD SMAQA_SU (long long t, unsigned long long a, unsigned long long b)

`__STATIC_FORCEINLINE long long __RV_DDUMAQA (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMA32_U (unsigned long long a, unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMXS32_U (unsigned long long a, unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMXA32_U (unsigned long long a, unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMS32_U (unsigned long long a, unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMA16 (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DSMA16 (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE unsigned long long __RV_DKSMS32_U (unsigned long long t,
unsigned long long a, unsigned long long b)`

`__STATIC_FORCEINLINE long __RV_DMADA32 (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DSMALBB (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DSMALBT (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DSMALTT (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DKMABB32 (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DKMABT32 (long long t, unsigned long long a,
unsigned long long b)`

`__STATIC_FORCEINLINE long long __RV_DKMATT32 (long long t, unsigned long long a,
unsigned long long b)`

group **Nuclei N3 SIMD DSP Additional Instructions**

(RV32 only)Nuclei Customized N3 DSP Instructions

This is Nuclei customized DSP N3 instructions only for RV32

Functions

__STATIC_FORCEINLINE unsigned long long __RV_DKMMAC (unsigned long long t, unsigned long long a, unsigned long long b)

DKMMAC (64-bit MSW 32x32 Signed Multiply and Saturating Add)

Type: SIMD

Syntax:

```
DKMMAC Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element signed multiplications and saturating addition simultaneously. The results are written into Rd.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom
for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    res = sat.q31(dop + (aop s* bop)[63:32]);
}
Rd = concat(rest, resb);
x=x+1
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMMAC_U (unsigned long long t, unsigned long long a, unsigned long long b)

DKMMAC.u (64-bit MSW 32x32 Unsigned Multiply and Saturating Add with Rounding)

Type: SIMD

Syntax:

```
DKMMAC.u Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element signed multiplications and saturating addition simultaneously. The results are written into Rd.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and adds the most significant 32-bit multiplication results with the signed 32-bit elements of Rd. If the addition result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom
for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    res = sat.q31(dop + RUND(aop u* bop)[63:32]);
}
Rd = concat(rest, resb);
x=x+1
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DKMSB (unsigned long long t,
unsigned long long a, unsigned long long b)**

DKMSB (64-bit MSW 32x32 Signed Multiply and Saturating Sub)

Type: SIMD

Syntax:

```
DKMSB Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers
```

Purpose :

Do MSW 32x32 element signed multiplications and saturating subtraction simultaneously. The results are written into Rd.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and subtracts the most significant 32-bit multiplication results from the signed 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom
for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    res = sat.q31(dop - (aop s* bop)[63:32]);
}
Rd = concat(rest, resb);
x=0

```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMSB_U (unsigned long long t, unsigned long long a, unsigned long long b)

DKMSB.u (64-bit MSW 32x32 Unsigned Multiply and Saturating Sub with Rounding)

Type: SIMD

Syntax:

```

DKMSB.u Rd, Rs1, Rs2
# Rd, Rs1, Rs2 are all even/odd pair of registers

```

Purpose :

Do MSW 32x32 element signed multiplications and saturating subtraction simultaneously. The results are written into Rd.

Description :

This instruction multiplies the signed 32-bit elements of Rs1 with the signed 32-bit elements of Rs2 and subtracts the most significant 32-bit multiplication results from the signed 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The .u form of the instruction additionally rounds up the most significant 32-bit of the 64-bit multiplication results by adding a 1 to bit 31 of the results.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom
for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    res = sat.q31(dop - (aop u* bop)[63:32]);
}
Rd = concat(rest, resb);
x=0

```

Parameters

- **t** – [in] unsigned long long type of value stored in t

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMADA (unsigned long long t, unsigned long long a, unsigned long long b)

DKMADA (Saturating Signed Multiply Two Halfs and Two Adds)

Type: DSP

Syntax:

```
DKMADA Rd, Rs1, Rs2
```

Purpose :

Do two 16x16 with 32-bit signed double addition simultaneously. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[1];
    mul2 = aop.H[0] s* bop.H[0];
    res = sat.q31(dop + mul1 + mul2);
}
Rd = concat(rest, resb);
x=x+2
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMAXDA (unsigned long long t, unsigned long long a, unsigned long long b)

DKMAXDA (Two Cross 16x16 with 32-bit Signed Double Add)

Type: DSP

Syntax:

DKMAXDA Rd, Rs1, Rs2

Purpose :

Do two cross 16x16 with 32-bit signed double addition simultaneously. The results are written into Rd.

Description :

It multiplies the top 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in elements in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[0];
    mul2 = aop.H[0] s* bop.H[1];
    res = sat.q31(dop + mul1 + mul2);
}
Rd = concat(rest, resb);
x=x+1
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DKMADS (unsigned long long t,
unsigned long long a, unsigned long long b)**

DKMADS (Two 16x16 with 32-bit Signed Add and Sub)

Type: DSP

Syntax:

DKMADS Rd, Rs1, Rs2

Purpose :

Do two 16x16 with 32-bit signed addition and subtraction simultaneously. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[1];
    mul2 = aop.H[0] s* bop.H[0];
    res = sat.q31(dop + mul1 - mul2);
}
Rd = concat(rest, resb);
x=x+2

```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMADRS (unsigned long long t, unsigned long long a, unsigned long long b)

DKMADRS (Two 16x16 with 32-bit Signed Add and Reversed Sub)

Type: DSP

Syntax:

```
DKMADRS Rd, Rs1, Rs2
```

Purpose :

Do two 16x16 with 32-bit signed addition and reverred subtraction simultaneously. The results are written into Rd.

Description :

it multiplies the top 16-bit content of 32-bit elements in Rs1 with the top 16-bit content of 32-bit elements in Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of 32-bit elements in Rs1 with the bottom 16-bit content of 32-bit elements in Rs2

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[1];
    mul2 = aop.H[0] s* bop.H[0];
    res = sat.q31(dop - mul1 + mul2);
}
Rd = concat(rest, resb);
x=x+2

```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMAXDS (unsigned long long t, unsigned long long a, unsigned long long b)

DKMAXDS (Saturating Signed Crossed Multiply Two Halfs & Subtract & Add)

Type: DSP

Syntax:

```
DKMAXDS Rd, Rs1, Rs2
```

Purpose :

Do two cross 16x16 with 32-bit signed addition and subtraction simultaneously. The results are written into Rd.

Description :

Do two signed 16-bit multiplications from 32-bit elements in two registers; and then perform a subtraction operation between the two 32-bit results. Then add the subtraction result to the corresponding 32-bit elements in a third register. The addition result may be saturated.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[0];
    mul2 = aop.H[0] s* bop.H[1];
    res = sat.q31(dop + mul1 - mul2);
}
Rd = concat(rest, resb);
x=x+2
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DKMSDA (unsigned long long t, unsigned long long a, unsigned long long b)

DKMSDA (Two 16x16 with 32-bit Signed Double Sub)

Type: DSP

Syntax:

DKMSDA Rd, Rs1, Rs2

Purpose :

Do two 16x16 with 32-bit signed double subtraction simultaneously. The results are written into Rd.

Description :

it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[0];
    mul2 = aop.H[0] s* bop.H[1];
    res = sat.q31(dop - mul1 - mul2);
}
Rd = concat(rest, resb);
x=x+2
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DKMSXDA (unsigned long long t,
unsigned long long a, unsigned long long b)**

DKMSXDA (Two Cross 16x16 with 32-bit Signed Double Sub)

Type: DSP

Syntax:

DKMSXDA Rd, Rs1, Rs2

Purpose :

Do two cross 16x16 with 32-bit signed double subtraction simultaneously. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 and multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2. The two 32-bit multiplication results are then subtracted from the content of the corresponding 32-bit elements of Rd. If the subtraction result is beyond the Q31 number range ($-2^{31} \leq Q31 \leq 2^{31}-1$), it is saturated to the range and the OV bit is set to 1. The results after saturation are written to Rd. The 16-bit contents are treated as signed integers.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    mul1 = aop.H[1] s* bop.H[0];
    mul2 = aop.H[0] s* bop.H[1];
    res = sat.q31(dop - mul1 - mul2);
}
Rd = concat(rest, resb);
x=0

```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

**__STATIC_FORCEINLINE unsigned long long __RV_DSMAQA (unsigned long long t,
unsigned long long a, unsigned long long b)**

DSMAQA (Four Signed 8x8 with 32-bit Signed Add)

Type: DSP

Syntax:

DSMAQA Rd, Rs1, Rs2

Purpose :

Do four signed 8x8 with 32-bit signed addition simultaneously. The results are written into Rd.

Description :

This instruction multiplies the four signed 8-bit elements of 32-bit chunks of Rs1 with the four signed 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the signed content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```

op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    m0 = aop.B[0] s* bop.B[0];
    m1 = aop.B[1] s* bop.B[1];
    m2 = aop.B[2] s* bop.B[2];
    m3 = aop.B[3] s* bop.B[3];
    res = dop + m0 + m1 + m2 + m3;
}
Rd = concat(rest, resb);
x=0

```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE unsigned long long __RV_DSMAQA_SU (unsigned long long t, unsigned long long a, unsigned long long b)

DSMAQA.SU (Four Signed 8 x Unsigned 8 with 32-bit Signed Add)

Type: DSP

Syntax:

DSMAQA.SU Rd, Rs1, Rs2

Purpose :

Do four Signed 8 x Unsigned 8 with 32-bit unsigned addition simultaneously. The results are written into Rd.

Description :

This instruction multiplies the four unsigned 8-bit elements of 32-bit chunks of Rs1 with the four signed 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the unsigned content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom

for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {
    m0 = aop.B[0] su* bop.B[0];
    m1 = aop.B[1] su* bop.B[1];
    m2 = aop.B[2] su* bop.B[2];
    m3 = aop.B[3] su* bop.B[3];
    res = dop + m0 + m1 + m2 + m3;
}
Rd = concat(rest, resb);
x=x+4
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE unsigned long long __RV_DUMAQA (unsigned long long t,  
unsigned long long a, unsigned long long b)
```

DUMAQA (Four Unsigned 8x8 with 32-bit Unsigned Add)

Type: DSP

Syntax:

```
DUMAQA Rd, Rs1, Rs2
```

Purpose :

Do four unsigned 8x8 with 32-bit unsigned addition simultaneously. The results are written into Rd.

Description :

This instruction multiplies the four unsigned 8-bit elements of 32-bit chunks of Rs1 with the four unsigned 8-bit elements of 32-bit chunks of Rs2 and then adds the four results together with the unsigned content of the corresponding 32-bit chunks of Rd. The final results are written back to the corresponding 32-bit chunks in Rd.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; op3t = Rd.W[x+1] // top  
op1b = Rs1.W[x]; op2b = Rs2.W[x]; op3b = Rd.W[x] // bottom  
  
for ((aop,bop,dop,res) in [(op1t,op2t,op3t,rest), (op1b,op2b,op3b,resb)]) {  
    m0 = aop.B[0] su* bop.B[0];  
    m1 = aop.B[1] su* bop.B[1];  
    m2 = aop.B[2] su* bop.B[2];  
    m3 = aop.B[3] su* bop.B[3];  
    res = dop + m0 + m1 + m2 + m3;  
}  
Rd = concat(rest, resb);  
x=x+1
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

```
__STATIC_FORCEINLINE long long __RV_DKMDA32 (unsigned long long a,  
unsigned long long b)
```

DKMDA32 (Two Signed 32x32 with 64-bit Saturation Add)

Type: DSP

Syntax:

```
DKMDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 add the signed multiplication results with Q63 saturation. The results are written into Rd.

Description :

For the KMDA32 instruction, it multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
t0 = op1b s* op2b;
t1 = op1t s* op2t;
Rd = sat.q63(t0 + t1);
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DKMXDA32 (unsigned long long a, unsigned long long b)

DKMXDA32 (Two Cross Signed 32x32 with 64-bit Saturation Add)

Type: DSP

Syntax:

```
DKMXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two cross signed 32x32 and add the signed multiplication results with Q63 saturation. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
t01 = op1b s* op2t;
t10 = op1t s* op2b;
Rd = sat.q63(t01 + t10);
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DKMADA32 (long long t, unsigned long long a,  
unsigned long long b)
```

DKMADA32 (Two Signed 32x32 with 64-bit Saturation Add)

Type: DSP

Syntax:

```
DKMADA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add the signed multiplication results and a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then adds the result to the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top  
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom  
t0 = op1b s* op2b;  
t1 = op1t s* op2t;  
Rd = sat.q63(Rd + t0 + t1);  
x=0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DKMAXDA32 (long long t, unsigned long long a,  
unsigned long long b)
```

DKMAXDA32 (Two Cross Signed 32x32 with 64-bit Saturation Add)

Type: DSP

Syntax:

```
DKMAXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two cross signed 32x32 and add the signed multiplication results and a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
t01 = op1b s* op2t;
t10 = op1t s* op2b;
Rd = sat.q63(Rd + t01 + t10);
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DKMADS32 (long long t, unsigned long long a, unsigned long long b)

DKMADS32 (Two Signed 32x32 with 64-bit Saturation Add and Sub)

Type: DSP

Syntax:

```
DKMADS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add the top signed multiplication results and subtraction bottom signed multiplication results and add a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then subtracts the result to the result of multiplying the top 32-bit element in Rs1 with the top 32-bit element in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t0 = op1b s* op2b;
t1 = op1t s* op2t;
Rd = sat.q63(Rd - t0 + t1);
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DKMADRS32 (long long t, unsigned long long a,  
unsigned long long b)
```

DKMADRS32 (Two Signed 32x32 with 64-bit Saturation Reversed Add and Sub)

Type: DSP

Syntax:

```
DKMADRS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add the signed multiplication results and a third register with Q63 saturation. The results are written into Rd. Do two signed 32x32 and subtraction the top signed multiplication results and add bottom signed multiplication results and add a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top  
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom  
t0 = op1b s* op2b;  
t1 = op1t s* op2t;  
Rd = sat.q63(Rd + t0 - t1);  
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DKMAXDS32 (long long t, unsigned long long a,  
unsigned long long b)
```

DKMAXDS32 (Two Cross Signed 32x32 with 64-bit Saturation Add and Sub)

Type: DSP

Syntax:

```
DKMAXDS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add the top signed multiplication results and subtraction bottom signed multiplication results and add a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2 and then subtracts the result from the result of multiplying the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t01 = op1b s* op2t;
t10 = op1t s* op2b;
Rd = sat.q63(Rd - t01 + t10);
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DKMSDA32 (long long t, unsigned long long a, unsigned long long b)

DKMSDA32 (Two Signed 32x32 with 64-bit Saturation Sub)

Type: DSP

Syntax:

```
DKMSDA32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and subtraction the top signed multiplication results and subtraction bottom signed multiplication results and add a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t0 = op1b s* op2b;
t1 = op1t s* op2t;
Rd = sat.q63(Rd - t0 - t1);
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DKMSXDA32 (long long t, unsigned long long a, unsigned long long b)

DKMSXDA32 (Two Cross Signed 32x32 with 64-bit Saturation Sub)

Type: DSP

Syntax:

```
DKMSXDA32 Rd, Rs1, Rs2
```

Purpose :

Do two cross signed 32x32 and subtraction the top signed multiplication results and subtraction bottom signed multiplication results and add a third register with Q63 saturation. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and multiplies the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t0 = op1b s* op2t;
t1 = op1t s* op2b;
Rd = sat.q63(Rd - t0 - t1);
x=x+2
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMD32 (unsigned long long a, unsigned long long b)

DSMD32 (Two Signed 32x32 with 64-bit Sub)

Type: DSP

Syntax:

```
DSMD32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add the top signed multiplication results and subtraction bottom signed multiplication. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the top 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t0 = op1b s* op2t;
t1 = op1t s* op2b;
Rd = t1 - t0;
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMDRS32 (unsigned long long a, unsigned long long b)

DSMDRS32 (Two Signed 32x32 with 64-bit Reversed Sub)

Type: DSP

Syntax:

```
DSMDRS32 Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and subtraction the top signed multiplication results and add bottom signed multiplication. The results are written into Rd

Description :

It multiplies the top 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the bottom 32-bit element of Rs1 with the bottom 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t0 = op1b s* op2b;
t1 = op1t s* op2t;
Rd = t1 - t0;
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a

- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMDXS32 (unsigned long long a, unsigned long long b)

DSMDXS32 (Two Cross Signed 32x32 with 64-bit Sub)

Type: DSP

Syntax:

DSMDXS32 Rd, Rs1, Rs2

Purpose :

Do two cross signed 32x32 and add the top signed multiplication results and subtraction bottom signed multiplication. The results are written into Rd.

Description :

It multiplies the bottom 32-bit element of Rs1 with the top 32-bit element of Rs2 and then subtracts the result from the result of multiplying the top 32-bit element of Rs1 with the bottom 32-bit element of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

t01 = op1b s* op2t;
t10 = op1t s* op2b;
Rd = t1 - t0;
x=x+2
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMDALDA (long long t, unsigned long long a, unsigned long long b)

DSMDALDA (Four Signed 16x16 with 64-bit Add)

Type: DSP

Syntax:

DSMDALDA Rd, Rs1, Rs2

Purpose :

Do four signed 16x16 and add signed multiplication results and a third register. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[0] s* op2b.H[0];
m1 = op1b.H[1] s* op2b.H[1];
m2 = op1t.H[0] s* op2t.H[0];
m3 = op1t.H[1] s* op2t.H[1];

Rd = Rd + m0 + m1 + m2 + m3;
x=x+2
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMLXDA (long long t, unsigned long long a, unsigned long long b)

DSMLXDA (Four Signed 16x16 with 64-bit Add)

Type: DSP

Syntax:

```
DSMLXDA Rd, Rs1, Rs2
```

Purpose :

Do four cross signed 16x16 and add signed multiplication results and a third register. The results are written into Rd.

Description :

It multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 with unlimited precision.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[0] s* op2b.H[1];
m1 = op1b.H[1] s* op2b.H[0];
m2 = op1t.H[0] s* op2t.H[1];
m3 = op1t.H[1] s* op2t.H[0];
```

(continues on next page)

(continued from previous page)

```
Rd = Rd + m0 + m1 + m2 + m3;
x=0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMALDS (long long t, unsigned long long a, unsigned long long b)

DSMALDS (Four Signed 16x16 with 64-bit Add and Sub)

Type: DSP

Syntax:

```
DSMALDS Rd, Rs1, Rs2
```

Purpose :

Do four signed 16x16 and add and subtraction signed multiplication results and a third register. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the top 16-bit content of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[1] s* op2b.H[1];
m1 = op1b.H[0] s* op2b.H[0];
m2 = op1t.H[1] s* op2t.H[1];
m3 = op1t.H[0] s* op2t.H[0];

Rd = Rd + m0 - m1 + m2 - m3;
x=0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DSMDLDRS (long long t, unsigned long long a,  
unsigned long long b)
```

DSMDLDRS (Four Signed 16x16 with 64-bit Add and Reversed Sub)

Type: DSP

Syntax:

```
DSMDLDRS Rd, Rs1, Rs2
```

Purpose :

Do two signed 16x16 and add and reversed subtraction signed multiplication results and a third register. The results are written into Rd.

Description :

It multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top  
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom  
  
m0 = op1b.H[0] s* op2b.H[0];  
m1 = op1b.H[1] s* op2b.H[1];  
m2 = op1t.H[0] s* op2t.H[0];  
m3 = op1t.H[1] s* op2t.H[1];  
  
Rd = Rd + m0 - m1 + m2 - m3;  
x=x+2
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DSMDLXDS (long long t, unsigned long long a,  
unsigned long long b)
```

DSMDLXDS (Four Cross Signed 16x16 with 64-bit Add and Sub)

Type: DSP

Syntax:

```
DSMDLXDS Rd, Rs1, Rs2
```

Purpose :

Do four cross signed 16x16 and add and subtraction signed multiplication results and a third register. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2 and then subtracts the result from the result of multiplying the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[1] s* op2b.H[0];
m1 = op1b.H[0] s* op2b.H[1];
m2 = op1t.H[1] s* op2t.H[0];
m3 = op1t.H[0] s* op2t.H[1];

Rd = Rd + m0 - m1 + m2 - m3;
x=x+0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMSLDA (long long t, unsigned long long a, unsigned long long b)

DSMSLDA (Four Signed 16x16 with 64-bit Sub)

Type: DSP

Syntax:

```
DSMSLDA Rd, Rs1, Rs2
```

Purpose :

Do four signed 16x16 and subtraction signed multiplication results and add a third register. The results are written into Rd.

Description :

It multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content Rs2 and multiplies the top 16-bit content of Rs1 with the top 16-bit content of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[0] s* op2b.H[0];
m1 = op1b.H[1] s* op2b.H[1];
m2 = op1t.H[0] s* op2t.H[0];
m3 = op1t.H[1] s* op2t.H[1];

Rd = Rd - m0 - m1 - m2 - m3;
x=x+0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMSLXDA (long long t, unsigned long long a, unsigned long long b)

DSMSLXDA (Four Cross Signed 16x16 with 64-bit Sub)

Type: DSP

Syntax:

DSMSLXDA Rd, Rs1, Rs2

Purpose :

Do four signed 16x16 and subtraction signed multiplication results and add a third register. The results are written into Rd.

Description :

It multiplies the top 16-bit content of Rs1 with the bottom 16-bit content of Rs2 and multiplies the bottom 16-bit content of Rs1 with the top 16-bit content of Rs2.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.H[0] s* op2b.H[1];
m1 = op1b.H[1] s* op2b.H[0];
m2 = op1t.H[0] s* op2t.H[1];
m3 = op1t.H[1] s* op2t.H[0];

Rd = Rd - m0 - m1 - m2 - m3;
x=x+0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DDMSMAQA (long long t, unsigned long long a, unsigned long long b)

DDMSMAQA (Eight Signed 8x8 with 64-bit Add)

Type: DSP

Syntax:

```
DDSMAQA Rd, Rs1, Rs2
```

Purpose :

Do eight signed 8x8 and add signed multiplication results and a third register. The results are written into Rd.

Description :

Do eight signed 8-bit multiplications from eight 8-bit chunks of two registers; and then adds the eight 16-bit results and the content of 64-bit chunks of a third register.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.B[0] s* op2b.B[0];
m1 = op1b.B[1] s* op2b.B[1];
m2 = op1b.B[2] s* op2b.B[2];
m3 = op1b.B[3] s* op2b.B[3];
m4 = op1t.B[0] s* op2t.B[0];
m5 = op1t.B[1] s* op2t.B[1];
m6 = op1t.B[2] s* op2t.B[2];
m7 = op1t.B[3] s* op2t.B[3];

s0 = m0 + m1 + m2 + m3;
s1 = m4 + m5 + m6 + m7;
Rd = Rd + s0 + s1;
x=x+1
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DDSMAQA_SU (long long t, unsigned long long a,
unsigned long long b)
```

DDSMAQA.SU (Eight Signed 8 x Unsigned 8 with 64-bit Add)

Type: DSP

Syntax:

```
DDSMAQA.SU Rd, Rs1, Rs2
```

Purpose :

Do eight signed 8 x unsigned 8 and add signed multiplication results and a third register. The results are written into Rd.

Description :

Do eight signed 8 x unsigned 8 and add signed multiplication results and a third register; and then adds the eight 16-bit results and the content of 64-bit chunks of a third register.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom

m0 = op1b.B[0] su* op2b.B[0];
m1 = op1b.B[1] su* op2b.B[1];
m2 = op1b.B[2] su* op2b.B[2];
m3 = op1b.B[3] su* op2b.B[3];
m4 = op1t.B[0] su* op2t.B[0];
m5 = op1t.B[1] su* op2t.B[1];
m6 = op1t.B[2] su* op2t.B[2];
m7 = op1t.B[3] su* op2t.B[3];

s0 = m0 + m1 + m2 + m3;
s1 = m4 + m5 + m6 + m7;
Rd = Rd + s0 + s1;
x=x+0
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DDUMAQA (long long t, unsigned long long a, unsigned long long b)

DDUMAQA (Eight Unsigned 8x8 with 64-bit Unsigned Add)

Type: DSP

Syntax:

```
DDUMAQA Rd, Rs1, Rs2
```

Purpose :

Do eight unsigned 8x8 and add unsigned multiplication results and a third register. The results are written into Rd.

Description :

Do eight unsigned 8x8 and add unsigned multiplication results and a third register; and then adds the eight 16-bit results and the content of 64-bit chunks of a third register.

Operations:

```
op1t = Rs1.W[x+1]; op2t = Rs2.W[x+1]; // top
op1b = Rs1.W[x]; op2b = Rs2.W[x]; // bottom
```

(continues on next page)

(continued from previous page)

```

m0 = op1b.B[0] u* op2b.B[0];
m1 = op1b.B[1] u* op2b.B[1];
m2 = op1b.B[2] u* op2b.B[2];
m3 = op1b.B[3] u* op2b.B[3];
m4 = op1t.B[0] u* op2t.B[0];
m5 = op1t.B[1] u* op2t.B[1];
m6 = op1t.B[2] u* op2t.B[2];
m7 = op1t.B[3] u* op2t.B[3];

s0 = m0 + m1 + m2 + m3;
s1 = m4 + m5 + m6 + m7;
Rd = Rd + s0 + s1;
x=0

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long __RV_DSMA32_U (unsigned long long a, unsigned long long b)

DSMA32.u (64-bit SIMD 32-bit Signed Multiply Addition With Rounding and Clip)

Type: DSP

Syntax:

```
DSMA32.u Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and add signed multiplication results with Rounding, then right shift 32-bit and clip q63 to q31. The result is written to Rd.

Description :

For the DSMA32.u instruction, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. Then, do the addition for the results above and perform the additional rounding operations, and then move the data to the right by 32-bit, and clip the 64-bit data into 32-bit. The result is written to Rd.

Operations:

```

Rd = (q31_t)((Rs1.W[x] s* Rs2.W[x] + Rs1.W[x + 1] s* Rs2.W[x + 1] +
0x80000000LL) s>> 32);
x=0

```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_DSMXS32_U (unsigned long long a,  
unsigned long long b)
```

DSMXS32.u (64-bit SIMD 32-bit Signed Multiply Cross Subtraction With Rounding and Clip)

Type: DSP**Syntax:**

```
DSMXS32.u Rd, Rs1, Rs2
```

Purpose :

Do two cross signed 32x32 and sub signed multiplication results with Rounding, then right shift 32-bit and clip q63 to q31. The result is written to Rd.

Description :

For the DSMXS32.u instruction, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. Then, do the subtraction for the results above and perform the additional rounding operations, and then move the data to the right by 32-bit, and clip the 64-bit data into 32-bit. The result is written to Rd.

Operations:

```
Rd = (q31_t)((Rs1.W[x + 1] s* Rs2.W[x] - Rs1.W[x] s* Rs2.W[x + 1] +  
0x80000000LL) s>> 32);  
x=0
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_DSMXA32_U (unsigned long long a,  
unsigned long long b)
```

DSMXA32.u (64-bit SIMD 32-bit Signed Cross Multiply Addition with Rounding and Clip)

Type: DSP**Syntax:**

```
DSMXA32.u Rd, Rs1, Rs2
```

Purpose :

Do two cross signed 32x32 and add signed multiplication results with Rounding, then right shift 32-bit and clip q63 to q31. The result is written to Rd.

Description :

For the DSMXA32.u instruction, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. Then, do the addition for the results above and perform the additional rounding operations, and then move the data to the right by 32-bit, and clip the 64-bit data into 32-bit. The result is written to Rd.

64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. Then, do the addition for the results above and perform the additional rounding operations, and then move the data to the right by 32-bit, and clip the 64-bit data into 32-bit. The result is written to Rd.

Operations:

```
Rd = (q31_t)((Rs1.W[x + 1] s* Rs2.W[x] + Rs1.W[x] s* Rs2.W[x + 1] +
0x80000000LL) s>> 32);
x=x+1
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_DSMS32_U (unsigned long long a, unsigned long long b)
```

DSMS32.u (64-bit SIMD 32-bit Signed Multiply Subtraction with Rounding and Clip)

Type: DSP

Syntax:

```
DSMS32.u Rd, Rs1, Rs2
```

Purpose :

Do two signed 32x32 and sub signed multiplication results with Rounding, then right shift 32-bit and clip q63 to q31. The result is written to Rd.

Description :

For the DSMS32.u instruction, multiply the bottom 32-bit Q31 content of 64-bit chunks in Rs1 with the bottom 32-bit Q31 content of 64-bit chunks in Rs2. At the same time, multiply the top 32-bit Q31 content of 64-bit chunks in Rs1 with the top 32-bit Q31 content of 64-bit chunks in Rs2. Then, do the subtraction for the results above and perform the additional rounding operations, and then move the data to the right by 32-bit, and clip the 64-bit data into 32-bit. The result is written to Rd.

Operations:

```
Rd = (q31_t)((Rs1.W[x] s* Rs2.W[x] - Rs1.W[x + 1] s* Rs2.W[x + 1] +
0x80000000LL) s>> 32);
x=x+1
```

Parameters

- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_DSMADA16 (long long t, unsigned long long a,
unsigned long long b)
```

DSMADA16 (Signed Multiply Two Halfs and Two Adds 32-bit)

Type: SIMD

Syntax:

```
DSMADA16 Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications of two 32-bit registers; and then adds the 32-bit results and the 32-bit value of an even/odd pair of registers together.

- DSMADA16: $rt\ pair + top * top + bottom * bottom$

Description :

This instruction multiplies the per 16-bit content of the 32-bit elements of Rs1 with the corresponding 16-bit content of the 32-bit elements of Rs2. The result is added to the 32-bit value of an even/odd pair of registers specified by Rd(4,1). The 32-bit addition result is written back to the register-pair. The 16-bit values of Rs1 and Rs2, and the 32-bit value of the register-pair are treated as signed integers.

Operations:

```
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[0]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[1]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[0]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[1]);
Rd = Rd + SE32(Mres0[0][31:0]) + SE32(Mres1[0][31:0]) + SE32(Mres0[1][31:0]) +
SE32(Mres1[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

```
__STATIC_FORCEINLINE long __RV_DSMAXDA16 (long long t, unsigned long long a,
unsigned long long b)
```

DSMAXDA16 (Signed Crossed Multiply Two Halfs and Two Adds 32-bit)

Type: SIMD

Syntax:

```
DSMAXDA16 Rd, Rs1, Rs2
```

Purpose :

Do two signed 16-bit multiplications of two 32-bit registers; and then adds the 32-bit results and the 32-bit value of an even/odd pair of registers together.

- DSMAXDA: $rt\ pair + top * bottom + bottom * top$ (all 32-bit elements)

Description :

This instruction crossly multiplies the top 16-bit content of the 32-bit elements of Rs1 with the bottom 16-bit content of the 32-bit elements of Rs2 and then adds the result to the result of multiplying the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2 with unlimited precision. The result is added to the 64-bit value of an even/odd pair of registers specified by Rd(4,1). The 64-bit addition result is clipped to 32-bit result.

Operations:

```
Mres0[0][31:0] = (Rs1.W[0].H[0] * Rs2.W[0].H[1]);
Mres1[0][31:0] = (Rs1.W[0].H[1] * Rs2.W[0].H[0]);
Mres0[1][31:0] = (Rs1.W[1].H[0] * Rs2.W[1].H[1]);
Mres1[1][31:0] = (Rs1.W[1].H[1] * Rs2.W[1].H[0]);
Rd.W = Rd.W + SE32(Mres0[0][31:0]) + SE32(Mres1[0][31:0]) +
SE32(Mres0[1][31:0]) + SE32(Mres1[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE unsigned long long __RV_DKSMS32_U (unsigned long long t, unsigned long long a, unsigned long long b)

DKSMS32.u (Two Signed Multiply Shift-clip and Saturation with Rounding)

Type: SIMD

Syntax:

```
DKSMS32.u Rd, Rs1, Rs2
```

Purpose :

Computes saturated multiplication of two pairs of q31 type with shifted rounding.

Description :

Compute the multiplication of Rs1 and Rs2 of type q31_t, intercept [47:16] for the resulting 64-bit product to get the 32-bit number, then add 1 to it to do rounding, and finally saturate the result after rounding.

Operations:

```
Mres[x][63:0] = Rs1.W[x] s* Rs2.W[x];
Round[x][32:0] = Mres[x][47:15] + 1;
Rd.W[x] = sat.31(Rd.W[x] + Round[x][32:1]);
x=1...0
```

Parameters

- **t** – [in] unsigned long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

__STATIC_FORCEINLINE long __RV_DMADA32 (long long t, unsigned long long a, unsigned long long b)

DMADA32 ((Two Cross Signed 32x32 with 64-bit Add and Clip to 32-bit)

Type: SIMD

Syntax:

DMADA32 Rd, Rs1, Rs2

Purpose :

Do two cross signed 32x32 and add the signed multiplication results to q63, then clip the q63 result to q31, the final results are written into Rd.

Description :

For the DMADA32 instruction, it multiplies the top 32-bit element in Rs1 with the bottom 32-bit element in Rs2 and then adds the result to the result of multiplying the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2, then clip the q63 result to q31.

Operations:

```
res = (q31_t) (((q63_t) Rd.w[0] << 32) + (q63_t)Rs1.w[0] s* Rs2.w[1] + (q63_t)Rs1.w[1] s* Rs2.w[0]) s>> 32);
rd = res;
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long type

__STATIC_FORCEINLINE long long __RV_DSMALBB (long long t, unsigned long long a, unsigned long long b)

DSMALBB (Signed Multiply Bottom Halfs & Add 64-bit)

Type: SIMD

Syntax:

DSMALBB Rd, Rs1, Rs2

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of registers. The addition result is written back to the register-pair.

- DSMALBB: rt pair + bottom*bottom (all 32-bit elements)

Description :

For the DSMALBB instruction, it multiplies the bottom 16-bit content of Rs1 with the bottom 16-bit content of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd.

Operations:

```
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[0];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[0];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DSMBLBT (long long t, unsigned long long a, unsigned long long b)

DSMBLBT (Signed Multiply Bottom Half & Top Half & Add 64-bit)

Type: SIMD

Syntax:

```
DSMBLBT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of registers. The addition result is written back to the register-pair.

- DSMBLBT: rt pair + bottom*top (all 32-bit elements)

Description :

For the DSMBLBT instruction, it multiplies the bottom 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers

Operations:

```
Mres[0][31:0] = Rs1.W[0].H[0] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[0] * Rs2.W[1].H[1];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DSMLTT (long long t, unsigned long long a,
unsigned long long b)
```

DSMLTT (Signed Multiply Top Half & Add 64-bit)

Type: SIMD**Syntax:**

```
DSMLTT Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 16-bit content of the 32-bit elements of a register with the 16-bit content of the corresponding 32-bit elements of another register and add the results with a 64-bit value of an even/odd pair of registers. The addition result is written back to the register-pair.

- DSMLTT: DSMLTT rt pair + top*top (all 32-bit elements)

Description :

For the DSMLTT instruction, it multiplies the top 16-bit content of the 32-bit elements of Rs1 with the top 16-bit content of the 32-bit elements of Rs2. The multiplication results are added with the 64-bit value of Rd. The 64-bit addition result is written back to Rd. The 16-bit values of Rs1 and Rs2, and the 64-bit value of Rd are treated as signed integers.

Operations:

```
Mres[0][31:0] = Rs1.W[0].H[1] * Rs2.W[0].H[1];
Mres[1][31:0] = Rs1.W[1].H[1] * Rs2.W[1].H[1];
Rd = Rd + SE64(Mres[0][31:0]) + SE64(Mres[1][31:0]);
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

```
__STATIC_FORCEINLINE long long __RV_DKMABB32 (long long t, unsigned long long a,
unsigned long long b)
```

DKMABB32 (Saturating Signed Multiply Bottom Words & Add)

Type: SIMD**Syntax:**

```
DKMABB32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- DKMABB32: $rd + bottom * bottom$

Description :

For the DKMABB32 instruction, it multiplies the bottom 32-bit element in Rs1 with the bottom 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```
res = Rd + (Rs1.W[0] * Rs2.W[0]);
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;
```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

`__STATIC_FORCEINLINE long long __RV_DKMABT32 (long long t, unsigned long long a, unsigned long long b)`

DKMABT32 (Saturating Multiply Bottom & Top Words & Add)

Type: SIMD

Syntax:

DKMABT32 Rd, Rs1, Rs2

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- DKMABT32: $rd + bottom * top$

Description :

For the DKMABT32 instruction, it multiplies the bottom 32-bit element in Rs1 with the top 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

res = Rd + (Rs1.W[0] * Rs2.W[1]);
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in long long type

__STATIC_FORCEINLINE long long __RV_DKMATT32 (long long t, unsigned long long a, unsigned long long b)

DKMATT32 (Saturating Signed Multiply Top Words & Add)

Type: SIMD

Syntax:

```
DKMATT32 Rd, Rs1, Rs2
```

Purpose :

Multiply the signed 32-bit element in a register with the 32-bit element in another register and add the result to the content of 64-bit data in the third register. The addition result may be saturated and is written to the third register.

- DKMATT32: $rd + top * top$

Description :

For the DKMATT32 instruction, it multiplies the top 32-bit element in Rs1 with the top 32-bit element in Rs2. The multiplication result is added to the content of 64-bit data in Rd. If the addition result is beyond the Q63 number range ($-2^{63} \leq Q63 \leq 2^{63}-1$), it is saturated to the range and the OV bit is set to 1. The result after saturation is written to Rd. The 32-bit contents of Rs1 and Rs2 are treated as signed integers.

Operations:

```

res = Rd + (Rs1.W[1] * Rs2.W[1]);
if (res > (2^63)-1) {
    res = (2^63)-1;
    OV = 1;
} else if (res < -2^63) {
    res = -2^63;
    OV = 1;
}
Rd = res;

```

Parameters

- **t** – [in] long long type of value stored in t
- **a** – [in] unsigned long long type of value stored in a
- **b** – [in] unsigned long long type of value stored in b

Returns

value stored in unsigned long long type

group **Intrinsic Functions for SIMD Instructions**

Functions that generate RISC-V DSP SIMD instructions.

The following functions generate specified RISC-V SIMD instructions that cannot be directly accessed by compiler.

- **DSP ISA Extension Instruction Summary**

- **Shorthand Definitions**

- * **r.H** == r.H1: r[31:16], **r.L** == r.H0: r[15:0]
- * **r.B3**: r[31:24], **r.B2**: r[23:16], **r.B1**: r[15:8], **r.B0**: r[7:0]
- * **r.B[x]**: r[(x*8+7):(x*8+0)]
- * **r.H[x]**: r[(x*16+7):(x*16+0)]
- * **r.W[x]**: r[(x*32+31):(x*32+0)]
- * **r[xU]**: the upper 32-bit of a 64-bit number; xU represents the GPR number that contains this upper part 32-bit value.
- * **r[xL]**: the lower 32-bit of a 64-bit number; xL represents the GPR number that contains this lower part 32-bit value.
- * **r[xU].r[xL]**: a 64-bit number that is formed from a pair of GPRs.
- * **s>>**: signed arithmetic right shift:
- * **u>>**: unsigned logical right shift
- * **SAT.Qn()**: Saturate to the range of $[-2^n, 2^n-1]$, if saturation happens, set PSW.OV.
- * **SAT.Um()**: Saturate to the range of $[0, 2^m-1]$, if saturation happens, set PSW.OV.
- * **RUND()**: Indicate **rounding**, i.e., add 1 to the most significant discarded bit for right shift or MSW-type multiplication instructions.
- * **Sign or Zero Extending functions**:
 - **SEm(data)**: Sign-Extend data to m-bit.:
 - **ZEm(data)**: Zero-Extend data to m-bit.
- * **ABS(x)**: Calculate the absolute value of **x**.
- * **CONCAT(x,y)**: Concatenate **x** and **y** to form a value.
- * **u<**: Unsinged less than comparison.
- * **u<=**: Unsinged less than & equal comparison.
- * **u>**: Unsinged greater than comparison.
- * **s***: Signed multiplication.

* u*: Unsigned multiplication.

2.5.8 Intrinsic Functions for Bitmanipulation Instructions

group **Intrinsic Functions for Bitmanipulation Instructions**

Functions that generate RISC-V Bitmanipulation instructions.

RISC-V Bitmanipulation Intrinsic APIs are provided directly through compiler generated intrinsic function.

This intrinsic function support in compiler is introduced in nuclei riscv gcc 10.2.

API header file can be found in lib/gcc/riscv-nuclei-elf/<gcc_ver>/include/rvintrin.h

For Nuclei GCC 13/Clang 17 and later version, this intrinsic header no longer existed, please take care.

2.5.9 Intrinsic Functions for Vector Instructions

__STATIC_FORCEINLINE void __enable_vector (void)

__STATIC_FORCEINLINE void __disable_vector (void)

group **Intrinsic Functions for Vector Instructions**

Functions that generate RISC-V Vector instructions.

RISC-V Vector Intrinsic APIs are provided directly through compiler generated intrinsic function.

This intrinsic function support by compiler:

For Nuclei RISC-V GCC 10.2, it is an very old and not ratified version(no longer supported).

- API header file can be found in lib/gcc/riscv-nuclei-elf/<gcc_ver>/include/riscv_vector.h

For Nuclei RISC-V GCC 13/Clang 17, the intrinsic API supported is v0.12 version, see <https://github.com/riscv-non-isa/rvv-intrinsic-doc/releases/tag/v0.12.0>

For Nuclei RISC-V GCC 14.2/Clang 19, the intrinsic API supported is v0.11.x version, see <https://github.com/riscv-non-isa/rvv-intrinsic-doc/tree/v0.11.x>

Functions

__STATIC_FORCEINLINE void __enable_vector (void)

Enable Vector Unit.

Set vector context status bits to enable vector unit, and set state to initial

__STATIC_FORCEINLINE void __disable_vector (void)

Disable Vector Unit.

Clear vector context status bits to disable vector unit

2.5.10 Peripheral Access

__I volatile const

__O volatile

__IO volatile

__IM volatile const

__OM volatile

__IOM volatile

_VAL2FLD(field, value) (((uint32_t)(value) << field ## _Pos) & field ## _Msk)

_FLD2VAL(field, value) (((uint32_t)(value) & field ## _Msk) >> field ## _Pos)

group **Peripheral Access**

Naming conventions and optional features for accessing peripherals.

The section below describes the naming conventions, requirements, and optional features for accessing device specific peripherals. Most of the rules also apply to the core peripherals.

The **Device Header File <device.h>** contains typically these definition and also includes the core specific header files.

Defines

__I volatile const

Defines 'read only' permissions.

__O volatile

Defines 'write only' permissions.

__IO volatile

Defines 'read / write' permissions.

__IM volatile const

Defines 'read only' structure member permissions.

__OM volatile

Defines 'write only' structure member permissions.

__IOM volatile

Defines 'read/write' structure member permissions.

_VAL2FLD(field, value) (((uint32_t)(value) << field ## _Pos) & field ## _Msk)

Mask and shift a bit field value for use in a register bit range.

The macro **_VAL2FLD** uses the `#define`'s `_Pos` and `_Msk` of the related bit field to shift bit-field values for assigning to a register.

Example:

```
ECLIC->CFG = _VAL2FLD(CLIC_CLICCFG_NLBIT, 3);
```

Parameters

- **field** – [in] Name of the register bit field.
- **value** – [in] Value of the bit field. This parameter is interpreted as an `uint32_t` type.

Returns

Masked and shifted value.

_FLD2VAL(field, value) (((uint32_t)(value) & field ## _Msk) >> field ## _Pos)

Mask and shift a register value to extract a bit filed value.

The macro **_FLD2VAL** uses the `#define`'s `_Pos` and `_Msk` of the related bit field to extract the value of a bit field from a register.

Example:

```
nlbits = _FLD2VAL(CLIC_CLICCFG_NLBIT, ECLIC->CFG);
```

Parameters

- **field** – [in] Name of the register bit field.
- **value** – [in] Value of register. This parameter is interpreted as an `uint32_t` type.

Returns

Masked and shifted bit field value.

2.5.11 SysTick Timer(SysTimer)

Click [Nuclei Timer Unit](https://doc.nucleisys.com/nuclei_spec/isa/timer.html)³⁰ to learn about Core Timer Unit in Nuclei ISA Spec.

SysTimer API

```
__STATIC_INLINE void SysTimer_SetLoadValue (rv_counter_t value)
```

```
__STATIC_INLINE rv_counter_t SysTimer_GetLoadValue (void)
```

```
__STATIC_INLINE void SysTimer_SetHartCompareValue (rv_counter_t value,  
unsigned long hartid)
```

```
__STATIC_FORCEINLINE void SysTimer_SetCompareValue (rv_counter_t value)
```

³⁰ https://doc.nucleisys.com/nuclei_spec/isa/timer.html

```
__STATIC_INLINE rv_counter_t SysTimer_GetHartCompareValue (unsigned long hartid)

__STATIC_FORCEINLINE rv_counter_t SysTimer_GetCompareValue (void)

__STATIC_FORCEINLINE void SysTimer_Start (void)

__STATIC_FORCEINLINE void SysTimer_Stop (void)

__STATIC_FORCEINLINE void SysTimer_SetControlValue (uint32_t mctl)

__STATIC_FORCEINLINE uint32_t SysTimer_GetControlValue (void)

__STATIC_FORCEINLINE void SysTimer_SetHartSWIRQ (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_SetSWIRQ (void)

__STATIC_FORCEINLINE void SysTimer_ClearHartSWIRQ (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_ClearSWIRQ (void)

__STATIC_FORCEINLINE uint32_t SysTimer_GetHartMsipValue (unsigned long hartid)

__STATIC_FORCEINLINE uint32_t SysTimer_GetMsipValue (void)

__STATIC_FORCEINLINE void SysTimer_SetHartMsipValue (uint32_t msip, unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_SetMsipValue (uint32_t msip)

__STATIC_INLINE void SysTimer_SetCompareValue_S (uint64_t value)

__STATIC_INLINE uint64_t SysTimer_GetCompareValue_S (void)

__STATIC_FORCEINLINE void SysTimer_SetHartSWIRQ_S (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_SetSWIRQ_S (void)

__STATIC_FORCEINLINE void SysTimer_ClearHartSWIRQ_S (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_ClearSWIRQ_S (void)

__STATIC_FORCEINLINE uint32_t SysTimer_GetHartSsipValue (unsigned long hartid)
```

```
__STATIC_FORCEINLINE uint32_t SysTimer_GetSsipValue (void)

__STATIC_FORCEINLINE void SysTimer_SetHartSsipValue (uint32_t ssip, unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_SetSsipValue (uint32_t ssip)

__STATIC_FORCEINLINE void SysTimer_SendIPI_S (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_ClearIPI_S (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_EnableSSTC (void)

__STATIC_FORCEINLINE void SysTimer_DisableSSTC (void)

__STATIC_FORCEINLINE void SysTimer_SoftwareReset (void)

__STATIC_FORCEINLINE void SysTimer_SendIPI (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_ClearIPI (unsigned long hartid)

__STATIC_FORCEINLINE void SysTimer_EnableSAccess (void)

__STATIC_FORCEINLINE void SysTimer_DisableSAccess (void)

__STATIC_INLINE uint32_t SysTick_Config (rv_counter_t ticks)

__STATIC_INLINE uint32_t SysTick_HartConfig (rv_counter_t ticks, unsigned long hartid)

__STATIC_INLINE uint32_t SysTick_Reload (rv_counter_t ticks)

__STATIC_INLINE uint32_t SysTick_HartReload (rv_counter_t ticks, unsigned long hartid)

__STATIC_INLINE uint32_t SysTick_Config_S (rv_counter_t ticks)

__STATIC_INLINE uint32_t SysTick_Reload_S (rv_counter_t ticks)

SysTimer_GetHartID() (\_\_get\_hart\_index (page 84)())

SysTimer_GetHartID_S() (\_\_get\_hart\_index\_s (page 85)())
```

group **SysTimer Functions**

Functions that configure the Core System Timer.

Defines

SysTimer_GetHartID() (*__get_hart_index* (page 84)())

SysTimer_GetHartID() is used to get timer hartid which might not be the same as cpu hart id, for example, cpu hartid may be 1, but timer hartid may be 0, then timer hartid offset is 1.

If defined `__SYSTIMER_HARTID`, it will use `__SYSTIMER_HARTID` as timer hartid, otherwise, it will use *__get_hart_index()* (page 84). The cpu hartid is get by using `__get_hart_id` function

SysTimer_GetHartID_S() (*__get_hart_index_s* (page 85)())

Functions

__STATIC_INLINE void SysTimer_SetLoadValue (rv_counter_t value)

Set system timer load value.

This function set the system timer load value in MTIMER register.

Remark

- Load value is 64bits wide for non-n100 cpu
- SysTimer_GetLoadValue

Parameters

value – [in] value to set system timer MTIMER register.

__STATIC_INLINE rv_counter_t SysTimer_GetLoadValue (void)

Get system timer load value.

This function get the system timer current value in MTIMER register.

Remark

- Load value is 64bits wide.
- SysTimer_SetLoadValue

Returns

current value(64bit) of system timer MTIMER register.

__STATIC_INLINE void SysTimer_SetHartCompareValue (rv_counter_t value, unsigned long hartid)

Set system timer compare value by hartid.

This function set the system Timer compare value in MTIMERCMP register.

Remark

- Compare value is 64bits wide.
- If compare value is larger than current value timer interrupt generate.
- Modify the load value or compare value less to clear the interrupt.
- In S-mode, hartid can't be get by using `__get_hart_id` function, so this api suits S-mode particularly.
- `SysTimer_GetHartCompareValue`

Parameters

- **value** – [in] compare value to set system timer MTIMERCMP register.
- **hartid** – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_SetCompareValue (rv_counter_t value)

Set system timer compare value in machine mode.

This function set the system Timer compare value in MTIMERCMP register.

Remark

- Compare value is 64bits wide.
- If compare value is larger than current value timer interrupt generate.
- Modify the load value or compare value less to clear the interrupt.
- `__get_hart_id` function can only be accessed in machine mode, or else exception will occur.
- `SysTimer_GetCompareValue`

Parameters

- **value** – [in] compare value to set system timer MTIMERCMP register.

__STATIC_INLINE rv_counter_t SysTimer_GetHartCompareValue (unsigned long hartid)

Get system timer compare value by hartid.

This function get the system timer compare value in MTIMERCMP register.

Remark

- Compare value is 64bits wide.
- In S-mode, hartid can't be get by using `__get_hart_id` function, so this api suits S-mode particularly.

- SysTimer_SetHartCompareValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

Returns

compare value of system timer MTIMERCMP register.

__STATIC_FORCEINLINE rv_counter_t SysTimer_GetCompareValue (void)

Get system timer compare value in machine mode.

This function get the system timer compare value in MTIMERCMP register.

Remark

- Compare value is 64bits wide.
- SysTimer_SetCompareValue

Returns

compare value of system timer MTIMERCMP register.

__STATIC_FORCEINLINE void SysTimer_Start (void)

Enable system timer counter running.

Enable system timer counter running by clear TIMESTOP bit in MTIMECTL register.

__STATIC_FORCEINLINE void SysTimer_Stop (void)

Stop system timer counter running.

Stop system timer counter running by set TIMESTOP bit in MTIMECTL register.

__STATIC_FORCEINLINE void SysTimer_SetControlValue (uint32_t mctl)

Set system timer control value.

This function set the system timer MTIMECTL register value.

Remark

- Bit TIMESTOP is used to start and stop timer. Clear TIMESTOP bit to 0 to start timer, otherwise to stop timer.
- Bit CMPCLREN is used to enable auto MTIMER clear to zero when MTIMER >= MTIMER-CMP. Clear CMPCLREN bit to 0 to stop auto clear MTIMER feature, otherwise to enable it.
- Bit CLKSRC is used to select timer clock source. Clear CLKSRC bit to 0 to use *mtime_toggle_a*, otherwise use *core_clk_aon*
- SysTimer_GetControlValue

Parameters

mctl – [in] value to set MTIMECTL register

__STATIC_FORCEINLINE uint32_t SysTimer_GetControlValue (void)

Get system timer control value.

This function get the system timer MTIMECTL register value.

Remark

- SysTimer_SetControlValue

Returns

MTIMECTL register value

__STATIC_FORCEINLINE void SysTimer_SetHartSWIRQ (unsigned long hartid)

Trigger or set software interrupt via system timer by hartid.

This function set the system timer MSIP bit in MSIP register.

Remark

- Set system timer MSIP bit and generate a machine mode software interrupt.
- In S-mode, hartid can't be get by using __get_hart_id function, so this api suite S-mode particularly when MTIME_SRW_CTRL.SRW equal 0.
- SysTimer_ClearHartSWIRQ
- SysTimer_GetHartMsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_SetSWIRQ (void)

Trigger or set software interrupt via system timer in machine mode.

This function set the system timer MSIP bit in MSIP register.

Remark

- Set system timer MSIP bit and generate a SW interrupt.
- SysTimer_ClearSWIRQ
- SysTimer_GetMsipValue

__STATIC_FORCEINLINE void SysTimer_ClearHartSWIRQ (unsigned long hartid)

Clear system timer software interrupt pending request by hartid.

This function clear the system timer MSIP bit in MSIP register.

Remark

- Clear system timer MSIP bit in MSIP register to clear the software interrupt pending.
- In S-mode, hartid can't be get by using `__get_hart_id` function, so this api suits S-mode particularly.
- SysTimer_SetHartSWIRQ
- SysTimer_GetHartMsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_ClearSWIRQ (void)

Clear system timer software interrupt pending request in machine mode.

This function clear the system timer MSIP bit in MSIP register.

Remark

- Clear system timer MSIP bit in MSIP register to clear the software interrupt pending.
- SysTimer_SetSWIRQ
- SysTimer_GetMsipValue

__STATIC_FORCEINLINE uint32_t SysTimer_GetHartMsipValue (unsigned long hartid)

Get system timer MSIP register value by hartid.

This function get the system timer MSIP register value.

Remark

- Bit0 is SW interrupt flag. Bit0 is 1 then SW interrupt set. Bit0 is 0 then SW interrupt clear.
- In S-mode, hartid can't be get by using `__get_hart_id` function, so this api suits S-mode particularly.
- SysTimer_SetHartSWIRQ
- SysTimer_ClearHartSWIRQ
- SysTimer_SetHartMsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

Returns

Value of Timer MSIP register.

__STATIC_FORCEINLINE uint32_t SysTimer_GetMsipValue (void)

Get system timer MSIP register value in machine mode.

This function get the system timer MSIP register value.

Remark

- Bit0 is SW interrupt flag. Bit0 is 1 then SW interrupt set. Bit0 is 0 then SW interrupt clear.
- SysTimer_SetSWIRQ
- SysTimer_ClearSWIRQ
- SysTimer_SetMsipValue

Returns

Value of Timer MSIP register.

__STATIC_FORCEINLINE void SysTimer_SetHartMsipValue (uint32_t msip, unsigned long hartid)

Set system timer MSIP register value by hartid.

This function set the system timer MSIP register value.

Remark

- In S-mode, hartid can't be get using __get_hart_id function, so this api suits S-mode particularly.
- SysTimer_GetHartMsipValue

Parameters

- **msip** – [in] value to set MSIP register
- **hartid** – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_SetMsipValue (uint32_t msip)

Set system timer MSIP register value in machine mode.

This function set the system timer MSIP register value.

Parameters

- msip** – [in] value to set MSIP register
- SysTimer_GetMsipValue

__STATIC_INLINE void SysTimer_SetCompareValue_S (uint64_t value)

Set system timer smode compare value.

This function set the system timer smode compare value in STIMECMP and STIMECMPH CSR register.

Remark

- smode timer compare register required the SSTC extension
- smode timer compare value located in stimecmp/stimecmph for rv32 and stimecmp for rv64.
- If compare value is larger than current value smode timer interrupt generate.
- Modify the load value or compare value less to clear the interrupt.
- SysTimer_GetCompareValue_S

Parameters

value – [in] compare value to set system timer smode compare register.

__STATIC_INLINE uint64_t SysTimer_GetCompareValue_S (void)

Get system timer smode compare value.

This function get the system timer smode compare value in STIMERCOMP and STIMECMPH CSR register.

Remark

- Compare value is 64bits wide.
- SysTimer_SetCompareValue_S

Returns

compare value of system timer smode compare csr register in 64bit.

__STATIC_FORCEINLINE void SysTimer_SetHartSWIRQ_S (unsigned long hartid)

Trigger or set supervisor mode software interrupt via system timer by hartid.

This function set the system timer SSIP bit in SSIP register.

Remark

- Set system timer SSIP bit and generate a supervisor mode software interrupt.
- SysTimer_ClearHartSWIRQ_S
- SysTimer_GetHartSsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_SetSWIRQ_S (void)

Trigger or set supervisor software interrupt via system timer.

This function set the system timer SSIP bit in SSIP register.

Remark

- Set system timer SSIP bit and generate a supervisor mode SW interrupt.
- SysTimer_ClearSWIRQ_S
- SysTimer_GetSsipValue

__STATIC_FORCEINLINE void SysTimer_ClearHartSWIRQ_S (unsigned long hartid)

Clear system timer supervisor mode software interrupt pending request by hartid.

This function clear the system timer SSIP bit in SSIP register.

Remark

- Clear system timer SSIP bit in SSIP register to clear the supervisor mode software interrupt pending.
- SysTimer_SetHartSWIRQ_S
- SysTimer_GetHartSsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_ClearSWIRQ_S (void)

Clear system timer supervisor mode software interrupt pending request.

This function clear the system timer SSIP bit in SSIP register.

Remark

- Clear system timer SSIP bit in SSIP register to clear the supervisor mode software interrupt pending.
- SysTimer_SetSWIRQ_S
- SysTimer_GetSsipValue

__STATIC_FORCEINLINE uint32_t SysTimer_GetHartSsipValue (unsigned long hartid)

Get system timer SSIP register value by hartid.

This function get the system timer SSIP register value.

Remark

- Bit0 is SW interrupt flag. Bit0 is 1 then SW interrupt set. Bit0 is 0 then SW interrupt clear.
- SysTimer_SetHartSWIRQ_S
- SysTimer_ClearHartSWIRQ_S
- SysTimer_SetHartSsipValue

Parameters

hartid – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

Returns

Value of Timer SSIP register.

__STATIC_FORCEINLINE uint32_t SysTimer_GetSsipValue (void)

Get system timer SSIP register value.

This function get the system timer MSIP register value.

Remark

- Bit0 is SW interrupt flag. Bit0 is 1 then SW interrupt set. Bit0 is 0 then SW interrupt clear.
- SysTimer_SetSWIRQ_S
- SysTimer_ClearSWIRQ_S
- SysTimer_SetSsipValue

Returns

Value of Timer MSIP register.

__STATIC_FORCEINLINE void SysTimer_SetHartSsipValue (uint32_t ssip, unsigned long hartid)

Set system timer SSIP register value by hartid.

This function set the system timer SSIP register value.

Remark

- SysTimer_GetHartSsipValue

Parameters

- **ssip** – [in] value to set SSIP register
- **hartid** – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

__STATIC_FORCEINLINE void SysTimer_SetSsipValue (uint32_t ssip)

Set system timer SSIP register value.

This function set the system timer SSIP register value.

Parameters

msip – [in] value to set SSIP register

- SysTimer_GetSsipValue

__STATIC_FORCEINLINE void SysTimer_SendIPI_S (unsigned long hartid)

send ipi to target hart using Systimer Clint SSIP

This function send ipi using clint timer's sSIP register.

Parameters

hart – [in] target hart

__STATIC_FORCEINLINE void SysTimer_ClearIPI_S (unsigned long hartid)

clear ipi to target hart using Systimer Clint SSIP

This function clear ipi using clint timer's SSIP register.

Parameters

hart – [in] target hart

__STATIC_FORCEINLINE void SysTimer_EnableSSTC (void)

Enable SSTC access right for supervisor mode.

This will enable SSTC stimecmp(h) csr register read/write access right in supervisor mode and make time(h) csr can be read

Attention

This can be only executed in machine mode

__STATIC_FORCEINLINE void SysTimer_DisableSSTC (void)

Disable SSTC stimecmp csr access right for supervisor mode.

This will disable SSTC stimecmp(h) csr register read/write access right in supervisor mode and make time(h) csr can not be read

__STATIC_FORCEINLINE void SysTimer_SoftwareReset (void)

Do software reset request.

This function will do software reset request through MTIMER

- Software need to write [SysTimer_MSFRST_KEY](#) (page 197) to generate software reset request
- The software request flag can be cleared by reset operation to clear

Remark

- The software reset is sent to SoC, SoC need to generate reset signal and send back to Core
- This function will not return, it will do while(1) to wait the Core reset happened

__STATIC_FORCEINLINE void SysTimer_SendIPI (unsigned long hartid)

send ipi to target hart using Systimer Clint MSIP

This function send ipi using clint timer's MSIP register.

Parameters

hart – [in] target hart

__STATIC_FORCEINLINE void SysTimer_ClearIPI (unsigned long hartid)

clear ipi to target hart using Systimer Clint MSIP

This function clear ipi using clint timer's MSIP register.

Parameters

hart – [in] target hart

__STATIC_FORCEINLINE void SysTimer_EnableSAccess (void)

Enable supervisor mode systimer register read/write access right.

This will make some systimer registers can be accessed in supervisor mode, which is reset state.

Attention

This can be only executed in machine mode

__STATIC_FORCEINLINE void SysTimer_DisableSAccess (void)

Disable supervisor mode systimer register read/write access right.

This will make some systimer registers can not be accessed in supervisor mode.

Attention

This can be only executed in machine mode

__STATIC_INLINE uint32_t SysTick_Config (rv_counter_t ticks)

System Tick Configuration.

Initializes the System Timer and its non-vector interrupt, and starts the System Tick Timer.

In our default implementation, the timer counter will be set to zero, and it will start a timer compare non-vector interrupt when it matches the ticks user set, during the timer interrupt user should reload the system tick using SysTick_Reload function or similar function written by user, so it can produce period timer interrupt.

Remark

- For [__NUCLEI_N_REV](#) (page 71) $\geq 0x0104$, the CMPCLREN bit in MTIMECTL is introduced, but we assume that the CMPCLREN bit is set to 0, so MTIMER register will not be auto cleared to 0 when MTIMER $\geq$ MTIMERCMP.
- When the variable `__Vendor_SysTickConfig` is set to 1, then the function SysTick_Config is not included.
- In this case, the file `<Device>.h` must contain a vendor-specific implementation of this function.
- If user need this function to start a period timer interrupt, then in timer interrupt handler routine code, user should call SysTick_Reload with ticks to reload the timer.

- This function only available when `__SYSTIMER_PRESENT == 1` and `__ECLIC_PRESENT == 1` and `__Vendor_SysTickConfig == 0`

See also

- `SysTimer_SetCompareValue`; `SysTimer_SetLoadValue`

Parameters

ticks – [in] Number of ticks between two interrupts.

Returns

0 Function succeeded.

Returns

1 Function failed.

__STATIC_INLINE uint32_t SysTick_HartConfig (rv_counter_t ticks, unsigned long hartid)

System Tick Configuration By hartid.

Initializes the System Timer and its non-vector interrupt, and starts the System Tick Timer.

In our default implementation, the timer counter will be set to zero, and it will start a timer compare non-vector interrupt when it matches the ticks user set, during the timer interrupt user should reload the system tick using `SysTick_Reload` function or similar function written by user, so it can produce period timer interrupt.

Remark

- For `__NUCLEI_N_REV` (page 71) $\geq 0x0104$, the `CMPCLREN` bit in `MTIMECTL` is introduced, but we assume that the `CMPCLREN` bit is set to 0, so `MTIMER` register will not be auto cleared to 0 when `MTIMER >= MTIMERCMP`.
- When the variable `__Vendor_SysTickConfig` is set to 1, then the function `SysTick_Config` is not included.
- In this case, the file `<Device>.h` must contain a vendor-specific implementation of this function.
- If user need this function to start a period timer interrupt, then in timer interrupt handler routine code, user should call `SysTick_Reload` with ticks to reload the timer.
- This function only available `__Vendor_SysTickConfig == 0`
- When `ECLIC` present, it will set `eclic` timer interrupt as non-vector mode, and level configured to 0, and enable it
- In S-mode, `hartid` can't be get by using `__get_hart_id` function, so this api suits S-mode particularly.

See also

- SysTimer_SetCompareValue; SysTimer_SetLoadValue

Parameters

- **ticks** – [in] Number of ticks between two interrupts.
- **hartid** – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

Returns

0 Function succeeded.

Returns

1 Function failed.

__STATIC_INLINE uint32_t SysTick_Reload (rv_counter_t ticks)

System Tick Reload.

Reload the System Timer Tick when the MTIMERCMP reached TIME value

Remark

- For *__NUCLEI_N_REV* (page 71) $\geq 0x0104$, the CMPCLREN bit in MTIMECTL is introduced, but for this SysTick_Config function, we assume this CMPCLREN bit is set to 0, so in interrupt handler function, user still need to set the MTIMERCMP or MTIMER to reload the system tick, if vendor want to use this timer's auto clear feature, they can define `__Vendor_SysTickConfig` to 1, and implement SysTick_Config and SysTick_Reload functions.
- When the variable `__Vendor_SysTickConfig` is set to 1, then the function SysTick_Reload is not included.
- In this case, the file **<Device>.h** must contain a vendor-specific implementation of this function.
- This function only available when `__SYSTIMER_PRESENT == 1`
- Since the MTIMERCMP value might overflow, if overflowed, MTIMER will be set to 0, and MTIMERCMP set to ticks

See also

- SysTimer_SetCompareValue
- SysTimer_SetLoadValue

Parameters

ticks – [in] Number of ticks between two interrupts.

Returns

0 Function succeeded.

Returns

1 Function failed.

```
__STATIC_INLINE uint32_t SysTick_HartReload (rv_counter_t ticks,
unsigned long hartid)
```

System Tick Reload By hartid.

Reload the System Timer Tick when the MTIMECMP reached TIME value

Remark

- For *__NUCLEI_N_REV* (page 71) $\geq 0x0104$, the CMPCLREN bit in MTIMECTL is introduced, but for this SysTick_Config function, we assume this CMPCLREN bit is set to 0, so in interrupt handler function, user still need to set the MTIMERCMP or MTIMER to reload the system tick, if vendor want to use this timer's auto clear feature, they can define *__Vendor_SysTickConfig* to 1, and implement SysTick_Config and SysTick_Reload functions.
- When the variable *__Vendor_SysTickConfig* is set to 1, then the function SysTick_Reload is not included.
- In this case, the file **<Device>.h** must contain a vendor-specific implementation of this function.
- This function only available when *__SYSTIMER_PRESENT* == 1 and *__ECLIC_PRESENT* == 1 and *__Vendor_SysTickConfig* == 0
- Since the MTIMERCMP value might overflow, if overflowed, MTIMER will be set to 0, and MTIMERCMP set to ticks
- In S-mode, hartid can't be get by using *__get_hart_id* function, so this api suits S-mode particularly.

See also

- SysTimer_SetCompareValue
- SysTimer_SetLoadValue

Parameters

- **ticks** – [in] Number of ticks between two interrupts.
- **hartid** – [in] hart ID, one hart is required to have a known hart ID of 0, other harts ID can be in 1~1023.

Returns

0 Function succeeded.

Returns

1 Function failed.

```
__STATIC_INLINE uint32_t SysTick_Config_S (rv_counter_t ticks)
```

System Tick Configuration in supervisor mode.

Initializes the system timer supervisor mode interrupt and registered as non-vector interrupt

The smode timecmp csr register will be set to proper value according to the ticks passed when it matches the ticks user set, during the smode timer interrupt user should reload the system tick using SysTick_Reload_S function or similar function written by user, so it can produce period smode timer interrupt.

Parameters

ticks – [in] Number of ticks between two interrupts.

Returns

0 Function succeeded.

__STATIC_INLINE uint32_t SysTick_Reload_S (rv_counter_t ticks)

System Tick Reload in supervisor mode.

Reload the System Timer Tick when the STIMECMP CSR reached TIME value

Parameters

ticks – [in] Number of ticks between two interrupts.

Returns

0 Function succeeded.

Returns

1 Function failed.

SysTick Code Example

The code below shows the usage of the function SysTick_Config() and SysTick_Reload() with an GD32VF103 SoC.

Listing 3: gd32vf103_systick_example.c

```

1  #include "gd32vf103.h"
2
3  volatile uint32_t msTicks = 0;           /* Variable to store_
   ↳ millisecond ticks */
4
5  #define CONFIG_TICKS      (SOC_TIMER_FREQ / 1000)
6  #define SysTick_Handler  eclic_mtip_handler
7
8  void SysTick_Handler(void) {             /* SysTick interrupt Handler.
   ↳ */
9      SysTimer_Reload(CONFIG_TICKS);       /* Call SysTick_Reload to_
   ↳ reload timer. */
10     msTicks++;                           /* See startup file startup_
   ↳ gd32vf103.S for SysTick vector */
11 }
12
13 int main (void) {
14     uint32_t returnCode;
15
16     returnCode = SysTick_Config(CONFIG_TICKS); /* Configure SysTick to_
   ↳ generate an interrupt every millisecond */
17
18     if (returnCode != 0) {                 /* Check return code for_

```

(continues on next page)

(continued from previous page)

```

19 ↪ errors */
20     // Error Handling
21 }
22 while(1);
23 }

```

SysTimer Interrupt Code Example

The code below shows the usage of various NMSIS Timer Interrupt functions with an GD32VF103 device.

Listing 4: gd32vf103_timer_example1.c

```

1  #include "gd32vf103.h"
2
3  void eclic_mtip_handler(void)
4  {
5      uint64_t now = SysTimer_GetLoadValue();
6      SysTimer_SetCompareValue(now + SOC_TIMER_FREQ/100);
7  }
8
9  static uint32_t int_cnt = 0;
10 void eclic_msip_handler(void)
11 {
12     SysTimer_ClearSWIRQ();
13     int_cnt++;
14 }
15
16 void eclic_global_initialize(void)
17 {
18     ECLIC_SetMth(0);
19     ECLIC_SetCfgNlbits(3);
20 }
21
22 int eclic_register_interrupt(IRQn_Type IRQn, uint8_t shv, uint32_t trig_mode, uint32_t lvl,
23 ↪ uint32_t priority, void * handler)
24 {
25     ECLIC_SetShvIRQ(IRQn, shv);
26     ECLIC_SetTrigIRQ(IRQn, trig_mode);
27     ECLIC_SetLevelIRQ(IRQn, lvl);
28     ECLIC_SetPriorityIRQ(IRQn, priority);
29     ECLIC_SetVector(IRQn, (rv_csr_t)(handler));
30     ECLIC_EnableIRQ(IRQn);
31     return 0;
32 }
33
34 void setup_timer(void)
35 {
36     SysTimer_SetLoadValue(0);
37     SysTimer_SetCompareValue(SOC_TIMER_FREQ/100);
38 }

```

(continues on next page)

(continued from previous page)

```

38
39 int main (void)
40 {
41     uint32_t returnCode;
42
43     eclic_global_initialize();           /* initialize ECLIC */
44
45     setup_timer();                     /* initialize timer */
46
47     returnCode = eclic_register_interrupt(SysTimer_IRQn,1,2,8,0,eclic_mtip_handler); /*
↳register system timer interrupt */
48
49     returnCode = eclic_register_interrupt(SysTimerSW_IRQn,1,2,8,0,eclic_msip_handler); /*
↳register system timer SW interrupt */
50
51     __enable_irq();                   /* enable global interrupt
↳*/
52
53     SysTimer_SetSWIRQ();              /* trigger timer SW
↳interrupt */
54
55     if (returnCode != 0) {             /* Check return code for
↳errors */
56         // Error Handling
57     }
58
59     while(1);
60 }

```

2.5.12 Interrupts and Exceptions

Description

This section explains how to use interrupts and exceptions and access functions for the Enhanced Core Local Interrupt Controller (ECLIC)³¹.

Nuclei provides a template file `startup_device` for each supported compiler. The file must be adapted by the silicon vendor to include interrupt vectors for all device-specific interrupt handlers. Each interrupt handler is defined as a weak function to an dummy handler. These interrupt handlers can be used directly in application software without being adapted by the programmer.

Click [Interrupt](#)³² to learn more about interrupt handling in Nuclei processor core.

³¹ https://doc.nucleisys.com/nuclei_spec/isa/eclic.html

³² https://doc.nucleisys.com/nuclei_spec/isa/interrupt.html

NMI Interrupt

NMI interrupt³³ entry is stored by **CSR_MNVEC**. If CSR_MMSIC[9] is 1 then NMI entry is the same as **Exception** which get from CSR_MTVEC. If CSR_MMSIC[9] is 0 NMI entry is reset vector.

Exception

Exception³⁴ has only 1 entry address which stored by CSR_MTVEC. All the exceptions will jump to the same entry `exc_entry` defined in `intexc_<Device>.S`.

The table below lists the core exception code of the Nuclei N/NX processors.

Table 7: Core exception code of the Nuclei N/NX processors

Exception Code	Value	Description
InsUnalign_EXCn	0	Instruction address misaligned
InsAccFault_EXCn	1	Instruction access fault
IlleIns_EXCn	2	Illegal instruction
Break_EXCn	3	Beakpoint
LdAddrUnalign_EXCn	4	Load address misaligned
LdFault_EXCn	5	Load access fault
StAddrUnalign_EXCn	6	Store or AMO address misaligned
StAccessFault_EXCn	7	Store or AMO access fault
UmodeEcall_EXCn	8	Environment call from User mode
SmodeEcall_EXCn	9	Environment call from Supervisor Mode
MmodeEcall_EXCn	11	Environment call from Machine mode
InsPageFault_EXCn	12	Instruction page fault
LdPageFault_EXCn	13	Load page fault
StPageFault_EXCn	15	Store or AMO page fault
StackOverflow_EXCn	24	Stack overflow fault
StackUnderflow_EXCn	25	Stack overflow fault
NMI_EXCn	0xff	NMI interrupt

Vector Table

The Vector Table defines the entry addresses of the ECLIC managed interrupts.

It is typically located at the beginning of the program memory, and you can modify CSR MTVT to reallocate the base address of this vector table, but you need to take care of the base address alignment according to the number of interrupts.

Table 8: base address alignment according to the number of interrupts

Number of Interrupt	Alignment Requirements of CSR MTVT
0 to 16	64-byte
17 to 32	128-byte
33 to 64	256-byte
65 to 128	512-byte
129 to 256	1KB
257 to 512	2KB
513 to 1024	4KB

³³ https://doc.nucleisys.com/nuclei_spec/isa/nmi.html

³⁴ https://doc.nucleisys.com/nuclei_spec/isa/exception.html

Interrupt number 0~18 is reserved by Nuclei Core. 19~1023 could be used by Silicon Vendor Device.

Below is an example interrupt allocated table:

```

1 typedef enum IRQn {
2     ***** Nuclei N/NX Processor Core Internal Interrupt Numbers_
3     *****
4     Reserved0_IRQn      = 0,      /*!< Internal reserved
5     */
6     Reserved1_IRQn      = 1,      /*!< Internal reserved
7     */
8     Reserved2_IRQn      = 2,      /*!< Internal reserved
9     */
10    SysTimerSW_IRQn      = 3,      /*!< System Timer SW interrupt
11    */
12    Reserved3_IRQn      = 4,      /*!< Internal reserved
13    */
14    Reserved4_IRQn      = 5,      /*!< Internal reserved
15    */
16    Reserved5_IRQn      = 6,      /*!< Internal reserved
17    */
18    SysTimer_IRQn        = 7,      /*!< System Timer Interrupt
19    */
20    Reserved6_IRQn      = 8,      /*!< Internal reserved
21    */
22    Reserved7_IRQn      = 9,      /*!< Internal reserved
23    */
24    Reserved8_IRQn      = 10,     /*!< Internal reserved
25    */
26    Reserved9_IRQn      = 11,     /*!< Internal reserved
27    */
28    Reserved10_IRQn     = 12,     /*!< Internal reserved
29    */
30    Reserved11_IRQn     = 13,     /*!< Internal reserved
31    */
32    Reserved12_IRQn     = 14,     /*!< Internal reserved
33    */
34    Reserved13_IRQn     = 15,     /*!< Internal reserved
35    */
36    Reserved14_IRQn     = 16,     /*!< Internal reserved
37    */
38    HardFault_IRQn      = 17,     /*!< Hard Fault, storage access error
39    */
40    Reserved15_IRQn     = 18,     /*!< Internal reserved
41    */
42
43    ***** GD32VF103 Specific External Interrupt Numbers_
44    *****
45    WWDGT_IRQn          = 19,     /*!< window watchDog timer interrupt
46    */
47    LVD_IRQn            = 20,     /*!< LVD through EXTI line detect interrupt
48    */
49    TAMPER_IRQn         = 21,     /*!< tamper through EXTI line detect
50    */

```

(continues on next page)

(continued from previous page)

```

27         :      :
28         :      :
29     CAN1_EWMC_IRQn      = 85,      /*!< CAN1 EWMC interrupt
↪    */
30     USBFS_IRQn          = 86,      /*!< USBFS global interrupt
↪    */
31     SOC_INT_MAX,        /*!< Number of total Interrupts
↪    */
32 } IRQn_Type;

```

ECLIC API Definitions

When macro `NMSIS_ECLIC_VIRTUAL` is defined, the ECLIC access functions in the table below must be implemented for virtualizing ECLIC access.

These functions should be implemented in a separate source module. The original NMSIS-Core `__ECLIC_xxx` functions are always available independent of `NMSIS_ECLIC_VIRTUAL` macro.

Table 9: ECLIC Access Functions

ECLIC ACCESS FUNCTIONS	NMSIS-CORE FUNCTIONS FOR ECLIC
ECLIC_SetCfgNlbits (page 640)	<code>__ECLIC_SetCfgNlbits()</code>
ECLIC_GetCfgNlbits (page 640)	<code>__ECLIC_GetCfgNlbits()</code>
ECLIC_GetInfoVer (page 640)	<code>__ECLIC_GetInfoVer()</code>
ECLIC_GetInfoCtlbits (page 640)	<code>__ECLIC_GetInfoCtlbits()</code>
ECLIC_GetInfoNum (page 640)	<code>__ECLIC_GetInfoNum()</code>
ECLIC_SetMth (page 640)	<code>__ECLIC_SetMth()</code>
ECLIC_GetMth (page 640)	<code>__ECLIC_GetMth()</code>
ECLIC_EnableIRQ (page 640)	<code>__ECLIC_EnableIRQ()</code>
ECLIC_GetEnableIRQ (page 640)	<code>__ECLIC_GetEnableIRQ()</code>
ECLIC_DisableIRQ (page 640)	<code>__ECLIC_DisableIRQ()</code>
ECLIC_SetPendingIRQ (page 640)	<code>__ECLIC_SetPendingIRQ()</code>
ECLIC_GetPendingIRQ (page 640)	<code>__ECLIC_GetPendingIRQ()</code>
ECLIC_ClearPendingIRQ (page 640)	<code>__ECLIC_ClearPendingIRQ()</code>
ECLIC_SetTrigIRQ (page 640)	<code>__ECLIC_SetTrigIRQ()</code>
ECLIC_GetTrigIRQ (page 640)	<code>__ECLIC_GetTrigIRQ()</code>
ECLIC_SetShvIRQ (page 640)	<code>__ECLIC_SetShvIRQ()</code>
ECLIC_GetShvIRQ (page 640)	<code>__ECLIC_GetShvIRQ()</code>
ECLIC_SetCtrlIRQ (page 640)	<code>__ECLIC_SetCtrlIRQ()</code>
ECLIC_GetCtrlIRQ (page 641)	<code>__ECLIC_GetCtrlIRQ()</code>
ECLIC_SetLevelIRQ (page 641)	<code>__ECLIC_SetLevelIRQ()</code>
ECLIC_GetLevelIRQ (page 641)	<code>__ECLIC_GetLevelIRQ()</code>
ECLIC_SetPriorityIRQ (page 641)	<code>__ECLIC_SetPriorityIRQ()</code>
ECLIC_GetPriorityIRQ (page 641)	<code>__ECLIC_GetPriorityIRQ()</code>

When `NMSIS_VECTAB_VIRTUAL` macro is defined, the functions in the table below must be replaced to virtualize the API access functions to the interrupt vector table.

The ECLIC vector table API should be implemented in a separate source module.

This allows, for example, alternate implementations to relocate the vector table from flash to RAM on the first vector table update.

The original NMSIS-Core functions are always available, but prefixed with `__ECLIC`.

Table 10: ECLIC Vector Access Functions

ECLIC Vector Table Access	NMSIS-CORE FUNCTIONS
<i>ECLIC_SetVector</i> (page 642)	<code>__ECLIC_SetVector()</code>
<i>ECLIC_GetVector</i> (page 642)	<code>__ECLIC_GetVector()</code>

ECLIC Function Usage

The code below shows the usage of various NMSIS ECLIC flow with an GD32VF103 device.

Listing 5: gd32vf103_interrupt_example1.c

```

1  #include "gd32vf103.h"
2
3  // Vector interrupt which could be nested
4  __INTERUPT void eclic_button1_handler(void)
5  {
6      SAVE_IRQ_CSR_CONTEXT();                                /* save mepc,
7      ↪ mcause, msubm enable interrupts */
8
9      GPIO_REG(GPIO_OUTPUT_VAL) |= (1 << GREEN_LED_GPIO_OFFSET); /* Green LED On */
10     GPIO_REG(GPIO_RISE_IP) = (0x1 << BUTTON_1_GPIO_OFFSET);    /* Clear the GPIO_
11     ↪ Pending interrupt by writing 1. */
12
13     RESTORE_IRQ_CSR_CONTEXT();                                /* disable_
14     ↪ interrupts, restore mepc, mcause, msubm */
15 }
16
17 // Non-vector interrupt
18 void eclic_button2_handler(void)
19 {
20     GPIO_REG(GPIO_OUTPUT_VAL) |= (1 << GREEN_LED_GPIO_OFFSET); /* Green LED On */
21     GPIO_REG(GPIO_RISE_IP) = (0x1 << BUTTON_2_GPIO_OFFSET);    /* Clear the GPIO_
22     ↪ Pending interrupt by writing 1. */
23 }
24
25 void eclic_global_initialize(void)
26 {
27     ECLIC_SetMth(0);
28     ECLIC_SetCfgNlbits(3);
29 }
30
31 int eclic_register_interrupt(IRQn_Type IRQn, uint8_t shv, uint32_t trig_mode, uint32_t_
32 ↪ lvl, uint32_t priority, void * handler)
33 {
34     ECLIC_SetShvIRQ(IRQn, shv);
35     ECLIC_SetTrigIRQ(IRQn, trig_mode);
36     ECLIC_SetLevelIRQ(IRQn, lvl);
37     ECLIC_SetPriorityIRQ(IRQn, priority);
38     ECLIC_SetVector(IRQn, (rv_csr_t)(handler));
39     ECLIC_EnableIRQ(IRQn);

```

(continues on next page)

(continued from previous page)

```

35     return 0;
36 }
37
38 int main (void)
39 {
40     uint32_t returnCode;
41
42     eclic_global_initialize();           /* initialize ECLIC */
43
44     GPIO_init();                       /* initialize GPIO */
45
46     returnCode = eclic_register_interrupt(BTN1_IRQn,1,2,1,0,Button1_IRQHandler); /*
↳register system button1 interrupt */
47     returnCode = eclic_register_interrupt(BTN2_IRQn,0,2,2,0,Button2_IRQHandler); /*
↳register system button2 interrupt */
48
49     __enable_irq();                   /* enable global interrupt
↳*/
50
51     if (returnCode != 0) {             /* Check return code for
↳errors */
52         // Error Handling
53     }
54
55     while(1);
56 }

```

Interrupt and Exception API

enum **IRQn_Type**

Values:

enumerator **Reserved0_IRQn**

enumerator **Reserved1_IRQn**

enumerator **Reserved2_IRQn**

enumerator **SysTimerSW_IRQn**

enumerator **Reserved3_IRQn**

enumerator **Reserved4_IRQn**

enumerator **Reserved5_IRQn**

enumerator **SysTimer_IRQn**

enumerator **Reserved6_IRQn**

enumerator **Reserved7_IRQn**

enumerator **Reserved8_IRQn**

enumerator **Reserved9_IRQn**

enumerator **Reserved10_IRQn**

enumerator **Reserved11_IRQn**

enumerator **Reserved12_IRQn**

enumerator **Reserved13_IRQn**

enumerator **Reserved14_IRQn**

enumerator **Reserved15_IRQn**

enumerator **Reserved16_IRQn**

enumerator **FirstDeviceSpecificInterrupt_IRQn**

enumerator **SOC_INT_MAX**

__STATIC_FORCEINLINE void __ECLIC_SetCfgNlbits (uint32_t nlbits)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetCfgNlbits (void)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoVer (void)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoCtlbits (void)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoNum (void)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoShadowNum (void)

__STATIC_FORCEINLINE void __ECLIC_SetMth (uint8_t mth)

```
__STATIC_FORCEINLINE uint8_t __ECLIC_GetMth (void)

__STATIC_FORCEINLINE void __ECLIC_EnableIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetEnableIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_DisableIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE int32_t __ECLIC_GetPendingIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetPendingIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_ClearPendingIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetTrigIRQ (IRQn_Type IRQn, uint32_t trig)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetTrigIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetShvIRQ (IRQn_Type IRQn, uint32_t shv)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetShvIRQ (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetCtrlIRQ (IRQn_Type IRQn, uint8_t intctrl)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetCtrlIRQ (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetLevelIRQ (IRQn_Type IRQn, uint8_t lvl_abs)

__STATIC_INLINE uint8_t __ECLIC_GetLevelIRQ (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetPriorityIRQ (IRQn_Type IRQn, uint8_t pri)

__STATIC_INLINE uint8_t __ECLIC_GetPriorityIRQ (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetVector (IRQn_Type IRQn, rv_csr_t vector)

__STATIC_FORCEINLINE rv_csr_t __ECLIC_GetVector (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetModeIRQ (IRQn_Type IRQn, uint32_t mode)

__STATIC_FORCEINLINE void __ECLIC_SetSth (uint8_t sth)
```

```

__STATIC_FORCEINLINE uint8_t __ECLIC_GetSth (void)

__STATIC_FORCEINLINE void __ECLIC_SetPendingIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE int32_t __ECLIC_GetPendingIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_ClearPendingIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetTrigIRQ_S (IRQn_Type IRQn, uint32_t trig)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetTrigIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetShvIRQ_S (IRQn_Type IRQn, uint32_t shv)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetShvIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_SetCtrlIRQ_S (IRQn_Type IRQn, uint8_t intctrl)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetCtrlIRQ_S (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetLevelIRQ_S (IRQn_Type IRQn, uint8_t lvl_abs)

__STATIC_INLINE uint8_t __ECLIC_GetLevelIRQ_S (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetPriorityIRQ_S (IRQn_Type IRQn, uint8_t pri)

__STATIC_INLINE uint8_t __ECLIC_GetPriorityIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_EnableIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetEnableIRQ_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __ECLIC_DisableIRQ_S (IRQn_Type IRQn)

__STATIC_INLINE void __ECLIC_SetVector_S (IRQn_Type IRQn, rv_csr_t vector)

__STATIC_FORCEINLINE rv_csr_t __ECLIC_GetVector_S (IRQn_Type IRQn)

__STATIC_FORCEINLINE void __set_exc_entry (rv_csr_t addr)

__STATIC_FORCEINLINE rv_csr_t __get_exc_entry (void)

```

`__STATIC_INLINE void __set_nonvec_entry (rv_csr_t addr)`

`__STATIC_INLINE rv_csr_t __get_nonvec_entry (void)`

`__STATIC_FORCEINLINE rv_csr_t __get_nmi_entry (void)`

`ECLIC_SetCfgNlbits __ECLIC_SetCfgNlbits`

`ECLIC_GetCfgNlbits __ECLIC_GetCfgNlbits`

`ECLIC_GetInfoVer __ECLIC_GetInfoVer`

`ECLIC_GetInfoCtlbits __ECLIC_GetInfoCtlbits`

`ECLIC_GetInfoNum __ECLIC_GetInfoNum`

`ECLIC_GetInfoShadowNum __ECLIC_GetInfoShadowNum`

`ECLIC_SetMth __ECLIC_SetMth`

`ECLIC_GetMth __ECLIC_GetMth`

`ECLIC_EnableIRQ __ECLIC_EnableIRQ`

`ECLIC_GetEnableIRQ __ECLIC_GetEnableIRQ`

`ECLIC_DisableIRQ __ECLIC_DisableIRQ`

`ECLIC_SetPendingIRQ __ECLIC_SetPendingIRQ`

`ECLIC_GetPendingIRQ __ECLIC_GetPendingIRQ`

`ECLIC_ClearPendingIRQ __ECLIC_ClearPendingIRQ`

`ECLIC_SetTrigIRQ __ECLIC_SetTrigIRQ`

`ECLIC_GetTrigIRQ __ECLIC_GetTrigIRQ`

`ECLIC_SetShvIRQ __ECLIC_SetShvIRQ`

`ECLIC_GetShvIRQ __ECLIC_GetShvIRQ`

ECLIC_SetCtrlIRQ __ECLIC_SetCtrlIRQ

ECLIC_GetCtrlIRQ __ECLIC_GetCtrlIRQ

ECLIC_SetLevelIRQ __ECLIC_SetLevelIRQ

ECLIC_GetLevelIRQ __ECLIC_GetLevelIRQ

ECLIC_SetPriorityIRQ __ECLIC_SetPriorityIRQ

ECLIC_GetPriorityIRQ __ECLIC_GetPriorityIRQ

ECLIC_SetModeIRQ __ECLIC_SetModeIRQ

ECLIC_SetSth __ECLIC_SetSth

ECLIC_GetSth __ECLIC_GetSth

ECLIC_SetPendingIRQ_S __ECLIC_SetPendingIRQ_S

ECLIC_GetPendingIRQ_S __ECLIC_GetPendingIRQ_S

ECLIC_ClearPendingIRQ_S __ECLIC_ClearPendingIRQ_S

ECLIC_SetTrigIRQ_S __ECLIC_SetTrigIRQ_S

ECLIC_GetTrigIRQ_S __ECLIC_GetTrigIRQ_S

ECLIC_SetShvIRQ_S __ECLIC_SetShvIRQ_S

ECLIC_GetShvIRQ_S __ECLIC_GetShvIRQ_S

ECLIC_SetCtrlIRQ_S __ECLIC_SetCtrlIRQ_S

ECLIC_GetCtrlIRQ_S __ECLIC_GetCtrlIRQ_S

ECLIC_SetLevelIRQ_S __ECLIC_SetLevelIRQ_S

ECLIC_GetLevelIRQ_S __ECLIC_GetLevelIRQ_S

ECLIC_SetPriorityIRQ_S __ECLIC_SetPriorityIRQ_S

ECLIC_GetPriorityIRQ_S __ECLIC_GetPriorityIRQ_S

ECLIC_EnableIRQ_S __ECLIC_EnableIRQ_S

ECLIC_GetEnableIRQ_S __ECLIC_GetEnableIRQ_S

ECLIC_DisableIRQ_S __ECLIC_DisableIRQ_S

ECLIC_SetVector __ECLIC_SetVector

ECLIC_GetVector __ECLIC_GetVector

ECLIC_SetVector_S __ECLIC_SetVector_S

ECLIC_GetVector_S __ECLIC_GetVector_S

SAVE_SSUBM_VAR()

RESTORE_SSUBM_VAR()

SAVE_IRQ_CSR_CONTEXT()

SAVE_IRQ_CSR_CONTEXT_S()

RESTORE_IRQ_CSR_CONTEXT()

RESTORE_IRQ_CSR_CONTEXT_S()

group **Interrupts and Exceptions**

Functions that manage interrupts and exceptions via the ECLIC.

Defines

ECLIC_SetCfgNlbits __ECLIC_SetCfgNlbits

ECLIC_GetCfgNlbits __ECLIC_GetCfgNlbits

ECLIC_GetInfoVer __ECLIC_GetInfoVer

ECLIC_GetInfoCtlbits __ECLIC_GetInfoCtlbits

ECLIC_GetInfoNum __ECLIC_GetInfoNum

ECLIC_GetInfoShadowNum __ECLIC_GetInfoShadowNum

ECLIC_SetMth __ECLIC_SetMth

ECLIC_GetMth __ECLIC_GetMth

ECLIC_EnableIRQ __ECLIC_EnableIRQ

ECLIC_GetEnableIRQ __ECLIC_GetEnableIRQ

ECLIC_DisableIRQ __ECLIC_DisableIRQ

ECLIC_SetPendingIRQ __ECLIC_SetPendingIRQ

ECLIC_GetPendingIRQ __ECLIC_GetPendingIRQ

ECLIC_ClearPendingIRQ __ECLIC_ClearPendingIRQ

ECLIC_SetTrigIRQ __ECLIC_SetTrigIRQ

ECLIC_GetTrigIRQ __ECLIC_GetTrigIRQ

ECLIC_SetShvIRQ __ECLIC_SetShvIRQ

ECLIC_GetShvIRQ __ECLIC_GetShvIRQ

ECLIC_SetCtrlIRQ __ECLIC_SetCtrlIRQ

ECLIC_GetCtrlIRQ __ECLIC_GetCtrlIRQ

ECLIC_SetLevelIRQ __ECLIC_SetLevelIRQ

ECLIC_GetLevelIRQ __ECLIC_GetLevelIRQ

ECLIC_SetPriorityIRQ __ECLIC_SetPriorityIRQ

ECLIC_GetPriorityIRQ __ECLIC_GetPriorityIRQ

ECLIC_SetModeIRQ __ECLIC_SetModeIRQ

ECLIC_SetSth __ECLIC_SetSth

ECLIC_GetSth __ECLIC_GetSth

ECLIC_SetPendingIRQ_S __ECLIC_SetPendingIRQ_S

ECLIC_GetPendingIRQ_S __ECLIC_GetPendingIRQ_S

ECLIC_ClearPendingIRQ_S __ECLIC_ClearPendingIRQ_S

ECLIC_SetTrigIRQ_S __ECLIC_SetTrigIRQ_S

ECLIC_GetTrigIRQ_S __ECLIC_GetTrigIRQ_S

ECLIC_SetShvIRQ_S __ECLIC_SetShvIRQ_S

ECLIC_GetShvIRQ_S __ECLIC_GetShvIRQ_S

ECLIC_SetCtrlIRQ_S __ECLIC_SetCtrlIRQ_S

ECLIC_GetCtrlIRQ_S __ECLIC_GetCtrlIRQ_S

ECLIC_SetLevelIRQ_S __ECLIC_SetLevelIRQ_S

ECLIC_GetLevelIRQ_S __ECLIC_GetLevelIRQ_S

ECLIC_SetPriorityIRQ_S __ECLIC_SetPriorityIRQ_S

ECLIC_GetPriorityIRQ_S __ECLIC_GetPriorityIRQ_S

ECLIC_EnableIRQ_S __ECLIC_EnableIRQ_S

ECLIC_GetEnableIRQ_S __ECLIC_GetEnableIRQ_S

ECLIC_DisableIRQ_S __ECLIC_DisableIRQ_S

ECLIC_SetVector __ECLIC_SetVector

ECLIC_GetVector __ECLIC_GetVector

ECLIC_SetVector_S __ECLIC_SetVector_S

ECLIC_GetVector_S __ECLIC_GetVector_S

SAVE_SSUBM_VAR()

RESTORE_SSUBM_VAR()

SAVE_IRQ_CSR_CONTEXT()

Save necessary CSRs into variables for vector interrupt nesting.

This macro is used to declare variables which are used for saving CSRs(MCAUSE, MEPC, MSUB), and it will read these CSR content into these variables, it need to be used in a vector-interrupt if nesting is required.

Remark

- Interrupt will be enabled after this macro is called
- It need to be used together with RESTORE_IRQ_CSR_CONTEXT
- Don't use variable names __mcause, __mpec, __msubm in your ISR code
- If you want to enable interrupt nesting feature for vector interrupt, you can do it like this:

```
// __INTERRUPT attribute will generates function entry and exit sequences suitable
// for use in an interrupt handler when this attribute is present
__INTERRUPT void eclic_mtip_handler(void)
{
    // Must call this to save CSRs
    SAVE_IRQ_CSR_CONTEXT();
    // !!!Interrupt is enabled here!!!
    // !!!Higher priority interrupt could nest it!!!

    // put you own interrupt handling code here

    // Must call this to restore CSRs
    RESTORE_IRQ_CSR_CONTEXT();
}
```

SAVE_IRQ_CSR_CONTEXT_S()

Save necessary CSRs into variables for vector interrupt nesting in supervisor mode.

RESTORE_IRQ_CSR_CONTEXT()

Restore necessary CSRs from variables for vector interrupt nesting.

This macro is used restore CSRs(MCAUSE, MEPC, MSUB) from pre-defined variables in SAVE_IRQ_CSR_CONTEXT macro.

Remark

- Interrupt will be disabled after this macro is called
- It need to be used together with SAVE_IRQ_CSR_CONTEXT

RESTORE_IRQ_CSR_CONTEXT_S()

Restore necessary CSRs from variables for vector interrupt nesting in supervisor mode.

Enums

enum **IRQn_Type**

Definition of IRQn numbers.

The core interrupt enumeration names for IRQn values are defined in the file **<Device>.h**.

- Interrupt ID(IRQn) from 0 to 18 are reserved for core internal interrupts.
- Interrupt ID(IRQn) start from 19 represent device-specific external interrupts.
- The first device-specific interrupt has the IRQn value 19.

The table below describes the core interrupt names and their availability in various Nuclei Cores.

Values:

enumerator **Reserved0_IRQn**

Internal reserved.

enumerator **Reserved1_IRQn**

Internal reserved.

enumerator **Reserved2_IRQn**

Internal reserved.

enumerator **SysTimerSW_IRQn**

System Timer SW interrupt.

enumerator **Reserved3_IRQn**

Internal reserved.

enumerator **Reserved4_IRQn**

Internal reserved.

enumerator **Reserved5_IRQn**

Internal reserved.

enumerator **SysTimer_IRQn**

System Timer Interrupt.

enumerator **Reserved6_IRQn**

Internal reserved.

enumerator **Reserved7_IRQn**

Internal reserved.

enumerator **Reserved8_IRQn**

Internal reserved.

enumerator **Reserved9_IRQn**

Internal reserved.

enumerator **Reserved10_IRQn**

Internal reserved.

enumerator **Reserved11_IRQn**

Internal reserved.

enumerator **Reserved12_IRQn**

Internal reserved.

enumerator **Reserved13_IRQn**

Internal reserved.

enumerator **Reserved14_IRQn**

Internal reserved.

enumerator **Reserved15_IRQn**

Internal reserved.

enumerator **Reserved16_IRQn**

Internal reserved.

enumerator **FirstDeviceSpecificInterrupt_IRQn**

First Device Specific Interrupt.

enumerator **SOC_INT_MAX**

Number of total interrupts.

Functions

__STATIC_FORCEINLINE void __ECLIC_SetCfgNlbits (uint32_t nlbits)

Set nlbits value.

This function set the nlbits value of CLICCFG register.

Remark

- nlbits is used to set the width of level in the CLICINTCTL[i].

See also

- ECLIC_GetCfgNlbits

Parameters

nlbits – [in] nlbits value

__STATIC_FORCEINLINE uint32_t __ECLIC_GetCfgNlbits (void)

Get nlbits value.

This function get the nlbits value of CLICCFG register.

Remark

- nlbits is used to set the width of level in the CLICINTCTL[i].

See also

- ECLIC_SetCfgNlbits

Returns

nlbits value of CLICCFG register

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoVer (void)

Get the ECLIC version number.

This function gets the hardware version information from CLICINFO register.

Remark

- This function gets hardware version information from CLICINFO register.
- Bit 20:17 for architecture version, bit 16:13 for implementation version.

See also

- ECLIC_GetInfoNum

Returns

hardware version number in CLICINFO register.

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoCtlbits (void)

Get CLICINTCTLBITS.

This function gets CLICINTCTLBITS from CLICINFO register.

Remark

- In the CLICINTCTL[i] registers, with $2 \leq \text{CLICINTCTLBITS} \leq 8$.
- The implemented bits are kept left-justified in the most-significant bits of each 8-bit CLICINTCTL[I] register, with the lower unimplemented bits hardwired to 1.

See also

- ECLIC_GetInfoNum

Returns

CLICINTCTLBITS from CLICINFO register.

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoNum (void)

Get number of maximum interrupt inputs supported.

This function gets number of maximum interrupt inputs supported from CLICINFO register.

Remark

- This function gets number of maximum interrupt inputs supported from CLICINFO register.
- The num_interrupt field specifies the actual number of maximum interrupt inputs supported in this implementation.

See also

- ECLIC_GetInfoCtlbits

Returns

number of maximum interrupt inputs supported from CLICINFO register.

__STATIC_FORCEINLINE uint32_t __ECLIC_GetInfoShadowNum (void)

Get number of shadow register groups.

This function gets the number of shadow register groups from the CLICINFO register. This includes both the first-come-first-served dedicated interrupt shadow registers and the shadow registers designed for different interrupt levels. This number represents the total count for M mode; if S Mode is present, the number is the same as M Mode. Note that shadow register 0 is fixed for first-come-first-served m/s-mode interrupt and cannot be configured.

Remark

- This function is only valid for ECLICv2
- This function gets the number of shadow register groups from the CLICINFO register.

See also

- ECLIC_GetInfoNum

Returns

number of shadow register groups from the CLICINFO register.

__STATIC_FORCEINLINE void __ECLIC_SetMth (uint8_t mth)

Set Machine Mode Interrupt Level Threshold.

This function sets machine mode interrupt level threshold. This threshold is not effective immediately, if you want to safely change it, you need to disable all interrupts first, and then change it, and do fence and then enable all interrupts.

See also

- ECLIC_GetMth

Parameters

mth – [in] Interrupt Level Threshold.

__STATIC_FORCEINLINE uint8_t __ECLIC_GetMth (void)

Get Machine Mode Interrupt Level Threshold.

This function gets machine mode interrupt level threshold.

See also

- ECLIC_SetMth

Returns

Interrupt Level Threshold.

__STATIC_FORCEINLINE void __ECLIC_EnableIRQ (IRQn_Type IRQn)

Enable a specific interrupt.

This function enables the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_DisableIRQ

Parameters

IRQn – [in] Interrupt number

__STATIC_FORCEINLINE uint32_t __ECLIC_GetEnableIRQ (IRQn_Type IRQn)

Get a specific interrupt enable status.

This function returns the interrupt enable status for the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_EnableIRQ
- ECLIC_DisableIRQ

Parameters

IRQn – [in] Interrupt number

Returns

- 0 Interrupt is not enabled
- 1 Interrupt is pending

__STATIC_FORCEINLINE void __ECLIC_DisableIRQ (IRQn_Type IRQn)

Disable a specific interrupt.

This function disables the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_EnableIRQ

Parameters

IRQn – [in] Number of the external interrupt to disable

__STATIC_FORCEINLINE int32_t __ECLIC_GetPendingIRQ (IRQn_Type IRQn)

Get the pending specific interrupt.

This function returns the pending status of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetPendingIRQ
- ECLIC_ClearPendingIRQ

Parameters

IRQn – [in] Interrupt number

Returns

- 0 Interrupt is not pending
- 1 Interrupt is pending

__STATIC_FORCEINLINE void __ECLIC_SetPendingIRQ (IRQn_Type IRQn)

Set a specific interrupt to pending.

This function sets the pending bit for the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_GetPendingIRQ

- `ECLIC_ClearPendingIRQ`

Parameters

IRQn – [in] Interrupt number

__STATIC_FORCEINLINE void __ECLIC_ClearPendingIRQ (IRQn_Type IRQn)

Clear a specific interrupt from pending.

This function removes the pending state of the specific interrupt *IRQn*. *IRQn* cannot be a negative number.

Remark

- IRQn must not be negative.

See also

- `ECLIC_SetPendingIRQ`
- `ECLIC_GetPendingIRQ`

Parameters

IRQn – [in] Interrupt number

__STATIC_FORCEINLINE void __ECLIC_SetTrigIRQ (IRQn_Type IRQn, uint32_t trig)

Set trigger mode and polarity for a specific interrupt.

This function set trigger mode and polarity of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- `ECLIC_GetTrigIRQ`

Parameters

- **IRQn** – [in] Interrupt number
- **trig** – [in]
 - 00 level trigger, [ECLIC_LEVEL_TRIGGER](#) (page 165)
 - 01 positive edge trigger, [ECLIC_POSTIVE_EDGE_TRIGGER](#) (page 165)

- 02 level trigger, [ECLIC_LEVEL_TRIGGER](#) (page 165)
- 03 negative edge trigger, [ECLIC_NEGTIVE_EDGE_TRIGGER](#) (page 165)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetTrigIRQ (IRQn_Type IRQn)

Get trigger mode and polarity for a specific interrupt.

This function get trigger mode and polarity of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetTrigIRQ

Parameters

IRQn – [in] Interrupt number

Returns

- 00 level trigger, [ECLIC_LEVEL_TRIGGER](#) (page 165)
- 01 positive edge trigger, [ECLIC_POSTIVE_EDGE_TRIGGER](#) (page 165)
- 02 level trigger, [ECLIC_LEVEL_TRIGGER](#) (page 165)
- 03 negative edge trigger, [ECLIC_NEGTIVE_EDGE_TRIGGER](#) (page 165)

__STATIC_FORCEINLINE void __ECLIC_SetShvIRQ (IRQn_Type IRQn, uint32_t shv)

Set interrupt working mode for a specific interrupt.

This function set selective hardware vector or non-vector working mode of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_GetShvIRQ

Parameters

- **IRQn** – [in] Interrupt number
- **shv** – [in]

- 0 non-vector mode, *ECLIC_NON_VECTOR_INTERRUPT* (page 164)
- 1 vector mode, *ECLIC_VECTOR_INTERRUPT* (page 164)

__STATIC_FORCEINLINE uint32_t __ECLIC_GetShvIRQ (IRQn_Type IRQn)

Get interrupt working mode for a specific interrupt.

This function get selective hardware vector or non-vector working mode of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetShvIRQ

Parameters

IRQn – [in] Interrupt number

Returns

shv

- 0 non-vector mode, *ECLIC_NON_VECTOR_INTERRUPT* (page 164)
- 1 vector mode, *ECLIC_VECTOR_INTERRUPT* (page 164)

__STATIC_FORCEINLINE void __ECLIC_SetCtrlIRQ (IRQn_Type IRQn, uint8_t intctrl)

Modify ECLIC Interrupt Input Control Register for a specific interrupt.

This function modify ECLIC Interrupt Input Control(CLICINTCTL[i]) register of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_GetCtrlIRQ

Parameters

- **IRQn** – [in] Interrupt number
- **intctrl** – [in] Set value for CLICINTCTL[i] register

```
__STATIC_FORCEINLINE uint8_t __ECLIC_GetCtrlIRQ (IRQn_Type IRQn)
```

Get ECLIC Interrupt Input Control Register value for a specific interrupt.

This function modify ECLIC Interrupt Input Control register of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.

See also

- `ECLIC_SetCtrlIRQ`

Parameters

IRQn – [in] Interrupt number

Returns

value of ECLIC Interrupt Input Control register

```
__STATIC_INLINE void __ECLIC_SetLevelIRQ (IRQn_Type IRQn, uint8_t lvl_abs)
```

Set ECLIC Interrupt level of a specific interrupt.

This function set interrupt level of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.
- If *lvl_abs* to be set is larger than the max level allowed, it will be force to be max level.
- When you set level value you need use `clciinfo.nlbits` to get the width of level. Then we could know the maximum of level. `CLICINTCTLBITS` is how many total bits are present in the `CLICINTCTL` register.

See also

- `ECLIC_GetLevelIRQ`

Parameters

- ***IRQn*** – [in] Interrupt number
- ***lvl_abs*** – [in] Interrupt level

```
__STATIC_INLINE uint8_t __ECLIC_GetLevelIRQ (IRQn_Type IRQn)
```

Get ECLIC Interrupt level of a specific interrupt.

This function get interrupt level of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.

See also

- `ECLIC_SetLevelIRQ`

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt level

`__STATIC_INLINE void __ECLIC_SetPriorityIRQ (IRQn_Type IRQn, uint8_t pri)`

Get ECLIC Interrupt priority of a specific interrupt.

This function get interrupt priority of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.
- If *pri* to be set is larger than the max priority allowed, it will be force to be max priority.
- Priority width is `CLICINTCTLBITS` minus `clciinfo.nbits` if `clciinfo.nbits` is less than `CLICINTCTLBITS`. Otherwise priority width is 0.

See also

- `ECLIC_GetPriorityIRQ`

Parameters

- ***IRQn*** – [in] Interrupt number
- ***pri*** – [in] Interrupt priority

`__STATIC_INLINE uint8_t __ECLIC_GetPriorityIRQ (IRQn_Type IRQn)`

Get ECLIC Interrupt priority of a specific interrupt.

This function get interrupt priority of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetPriorityIRQ

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt priority

__STATIC_INLINE void __ECLIC_SetVector (IRQn_Type IRQn, rv_csr_t vector)

Set Interrupt Vector of a specific interrupt.

This function set interrupt handler address of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.
- You can set the CSR_CSR_MTVT to set interrupt vector table entry address.
- If your vector table is placed in readonly section, the vector for IRQn will not be modified. For this case, you need to use the correct irq handler name defined in your vector table as your irq handler function name.
- This function will only work correctly when the vector table is placed in an read-write enabled section.

See also

- ECLIC_GetVector

Parameters

- **IRQn** – [in] Interrupt number
- **vector** – [in] Interrupt handler address

__STATIC_FORCEINLINE rv_csr_t __ECLIC_GetVector (IRQn_Type IRQn)

Get Interrupt Vector of a specific interrupt.

This function get interrupt handler address of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.
- You can read CSR_CSR_MTVT to get interrupt vector table entry address.

See also

- ECLIC_SetVector

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt handler address

__STATIC_FORCEINLINE void __ECLIC_SetModeIRQ (IRQn_Type IRQn, uint32_t mode)

Set privilege mode of a specific interrupt.

This function set in which privilege mode the interrupts *IRQn* should be taken.

Remark

- IRQn must not be negative.
- mode must be 1(Supervisor Mode) or 3(Machine Mode), other values are ignored.
- M-mode can R/W this field, but S-mode can only read. And ECLIC with S-Mode does not reply on CSR mideleg to delegate interrupts.
- Mode of S-mode ECLIC region's clicintattr can be omitted to set, which is mirror to M-mode ECLIC region's. Only the low 6 bits of clicintattr [i] can be written via the S-mode memory region.

Parameters

- **IRQn** – [in] Interrupt number
- **mode** – [in] Privilege mode

__STATIC_FORCEINLINE void __ECLIC_SetSth (uint8_t sth)

Set supervisor-mode Interrupt Level Threshold in supervisor mode.

This function sets supervisor-mode interrupt level threshold.

Remark

- S-mode ECLIC region `sinthresh.sth` is a mirror to M-mode ECLIC region's `minthresh.sth`, and will be updated synchronously, here operate on `minthresh.sth`.
- Similiar to `ECLIC_SetMth`, also recommended to disable interrupts before setting interrupt level threshold.

See also

- `ECLIC_GetSth`

Parameters

sth – [in] Interrupt Level Threshold.

__STATIC_FORCEINLINE uint8_t __ECLIC_GetSth (void)

Get supervisor-mode Interrupt Level Threshold in supervisor mode.

This function gets supervisor mode interrupt level threshold.

Remark

- S-mode ECLIC region `sinthresh.sth` is a mirror to M-mode ECLIC region's `minthresh.sth`, and will be updated synchronously, here operate on `minthresh.sth`.

See also

- `ECLIC_SetSth`

Returns

Interrupt Level Threshold.

__STATIC_FORCEINLINE void __ECLIC_SetPendingIRQ_S (IRQn_Type IRQn)

Set a specific interrupt to pending in supervisor mode.

This function sets the pending bit for the specific interrupt *IRQn* in supervisor mode.

Remark

- *IRQn* must not be negative.

See also

- ECLIC_GetPendingIRQ_S
- ECLIC_ClearPendingIRQ_S

Parameters

IRQn – [in] Interrupt number

__STATIC_FORCEINLINE int32_t __ECLIC_GetPendingIRQ_S (IRQn_Type IRQn)

Get the pending specific interrupt in supervisor mode.

This function returns the pending status of the specific interrupt *IRQn* in supervisor mode.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetPendingIRQ_S
- ECLIC_ClearPendingIRQ_S

Parameters

IRQn – [in] Interrupt number

Returns

- 0 Interrupt is not pending
- 1 Interrupt is pending

__STATIC_FORCEINLINE void __ECLIC_ClearPendingIRQ_S (IRQn_Type IRQn)

Clear a specific interrupt from pending in supervisor mode.

This function removes the pending state of the specific interrupt *IRQn* in supervisor mode. *IRQn* cannot be a negative number.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetPendingIRQ_S

- `ECLIC_GetPendingIRQ_S`

Parameters

IRQn – [in] Interrupt number

__STATIC_FORCEINLINE void __ECLIC_SetTrigIRQ_S (IRQn_Type IRQn, uint32_t trig)

Set trigger mode and polarity for a specific interrupt in supervisor mode.

This function set trigger mode and polarity of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- `ECLIC_GetTrigIRQ_S`

Parameters

- **IRQn** – [in] Interrupt number
- **trig** – [in]
 - 00 level trigger, [*ECLIC_LEVEL_TRIGGER*](#) (page 165)
 - 01 positive edge trigger, [*ECLIC_POSTIVE_EDGE_TRIGGER*](#) (page 165)
 - 02 level trigger, [*ECLIC_LEVEL_TRIGGER*](#) (page 165)
 - 03 negative edge trigger, [*ECLIC_NEGTIVE_EDGE_TRIGGER*](#) (page 165)

__STATIC_FORCEINLINE uint8_t __ECLIC_GetTrigIRQ_S (IRQn_Type IRQn)

Get trigger mode and polarity for a specific interrupt in supervisor mode.

This function get trigger mode and polarity of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- `ECLIC_SetTrigIRQ_S`

Parameters**IRQn** – [in] Interrupt number**Returns**

- 00 level trigger, *ECLIC_LEVEL_TRIGGER* (page 165)
- 01 positive edge trigger, *ECLIC_POSTIVE_EDGE_TRIGGER* (page 165)
- 02 level trigger, *ECLIC_LEVEL_TRIGGER* (page 165)
- 03 negative edge trigger, *ECLIC_NEGTIVE_EDGE_TRIGGER* (page 165)

```
__STATIC_FORCEINLINE void __ECLIC_SetShvIRQ_S (IRQn_Type IRQn, uint32_t shv)
```

Set interrupt working mode for a specific interrupt in supervisor mode.

This function set selective hardware vector or non-vector working mode of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- *ECLIC_GetShvIRQ_S*

Parameters

- **IRQn** – [in] Interrupt number
- **shv** – [in]
 - 0 non-vector mode, *ECLIC_NON_VECTOR_INTERRUPT* (page 164)
 - 1 vector mode, *ECLIC_VECTOR_INTERRUPT* (page 164)

```
__STATIC_FORCEINLINE uint8_t __ECLIC_GetShvIRQ_S (IRQn_Type IRQn)
```

Get interrupt working mode for a specific interrupt in supervisor mode.

This function get selective hardware vector or non-vector working mode of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- *ECLIC_SMODE_SetShvIRQ*

Parameters**IRQn** – [in] Interrupt number**Returns**

shv

- 0 non-vector mode, *ECLIC_NON_VECTOR_INTERRUPT* (page 164)
- 1 vector mode, *ECLIC_VECTOR_INTERRUPT* (page 164)

__STATIC_FORCEINLINE void __ECLIC_SetCtrlIRQ_S (IRQn_Type IRQn, uint8_t intctrl)

Modify ECLIC Interrupt Input Control Register for a specific interrupt in supervisor mode.

This function modify ECLIC Interrupt Input Control(CLICINTCTL[i]) register of the specific interrupt *IRQn*.**Remark**

- IRQn must not be negative.

See also

- ECLIC_GetCtrlIRQ_S

Parameters

- **IRQn** – [in] Interrupt number
- **intctrl** – [in] Set value for CLICINTCTL[i] register

__STATIC_FORCEINLINE uint8_t __ECLIC_GetCtrlIRQ_S (IRQn_Type IRQn)

Get ECLIC Interrupt Input Control Register value for a specific interrupt in supervisor mode.

This function modify ECLIC Interrupt Input Control register of the specific interrupt *IRQn*.**Remark**

- IRQn must not be negative.

See also

- ECLIC_SetCtrlIRQ_S

Parameters**IRQn** – [in] Interrupt number

Returns

value of ECLIC Interrupt Input Control register

```
__STATIC_INLINE void __ECLIC_SetLevelIRQ_S (IRQn_Type IRQn, uint8_t lvl_abs)
```

Set ECLIC Interrupt level of a specific interrupt in supervisor mode.

This function set interrupt level of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.
- If *lvl_abs* to be set is larger than the max level allowed, it will be force to be max level.
- When you set level value you need use *clciinfo.nbits* to get the width of level. Then we could know the maximum of level. *CLICINTCTLBITS* is how many total bits are present in the *CLICINTCTL* register.

See also

- *ECLIC_GetLevelIRQ_S*

Parameters

- *IRQn* – [in] Interrupt number
- *lvl_abs* – [in] Interrupt level

```
__STATIC_INLINE uint8_t __ECLIC_GetLevelIRQ_S (IRQn_Type IRQn)
```

Get ECLIC Interrupt level of a specific interrupt.

This function get interrupt level of the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.

See also

- *ECLIC_SetLevelIRQ_S*

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt level

```
__STATIC_INLINE void __ECLIC_SetPriorityIRQ_S (IRQn_Type IRQn, uint8_t pri)
```

Set ECLIC Interrupt priority of a specific interrupt in supervisor mode.

This function get interrupt priority of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.
- If pri to be set is larger than the max priority allowed, it will be force to be max priority.
- Priority width is CLICINTCTLBITS minus clciinfo.nlbits if clciinfo.nlbits is less than CLICINTCTLBITS. Otherwise priority width is 0.

See also

- ECLIC_GetPriorityIRQ_S

Parameters

- **IRQn** – [in] Interrupt number
- **pri** – [in] Interrupt priority

```
__STATIC_INLINE uint8_t __ECLIC_GetPriorityIRQ_S (IRQn_Type IRQn)
```

Get ECLIC Interrupt priority of a specific interrupt in supervisor mode.

This function get interrupt priority of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_SetPriorityIRQ_S

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt priority

```
__STATIC_FORCEINLINE void __ECLIC_EnableIRQ_S (IRQn_Type IRQn)
```

Enable a specific interrupt in supervisor mode.

This function enables the specific interrupt *IRQn*.

Remark

- *IRQn* must not be negative.

See also

- `ECLIC_DisableIRQ`

Parameters

IRQn – [in] Interrupt number

`__STATIC_FORCEINLINE uint8_t __ECLIC_GetEnableIRQ_S (IRQn_Type IRQn)`

Get a specific interrupt enable status in supervisor mode.

This function returns the interrupt enable status for the specific interrupt *IRQn* in S MODE.

Remark

- *IRQn* must not be negative.

See also

- `ECLIC_EnableIRQ_S`
- `ECLIC_DisableIRQ_S`

Parameters

IRQn – [in] Interrupt number

Returns

- 0 Interrupt is not masked
- 1 Interrupt is enabled

`__STATIC_FORCEINLINE void __ECLIC_DisableIRQ_S (IRQn_Type IRQn)`

Disable a specific interrupt in supervisor mode.

This function disables the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

See also

- ECLIC_EnableIRQ

Parameters

IRQn – [in] Number of the external interrupt to disable

__STATIC_INLINE void __ECLIC_SetVector_S (IRQn_Type IRQn, rv_csr_t vector)

Set Interrupt Vector of a specific interrupt in supervisor mode.

This function set interrupt handler address of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.
- You can set the CSR_CSR_MTVT to set interrupt vector table entry address.
- If your vector table is placed in readonly section, the vector for IRQn will not be modified. For this case, you need to use the correct irq handler name defined in your vector table as your irq handler function name.
- This function will only work correctly when the vector table is placed in an read-write enabled section.

See also

- ECLIC_GetVector_S

Parameters

- **IRQn** – [in] Interrupt number
- **vector** – [in] Interrupt handler address

__STATIC_FORCEINLINE rv_csr_t __ECLIC_GetVector_S (IRQn_Type IRQn)

Get Interrupt Vector of a specific interrupt in supervisor mode.

This function get interrupt handler address of the specific interrupt *IRQn*.

Remark

- IRQn must not be negative.

- You can read CSR_CSR_MTVT to get interrupt vector table entry address.

See also

- ECLIC_SMODE_SetVector

Parameters

IRQn – [in] Interrupt number

Returns

Interrupt handler address

__STATIC_FORCEINLINE void __set_exc_entry (rv_csr_t addr)

Set Exception entry address.

This function set exception handler address to 'CSR_MTVEC'.

Remark

- This function use to set exception handler address to 'CSR_MTVEC'. Address need to be aligned to 64 bytes.

See also

- __get_exc_entry

Parameters

addr – [in] Exception handler address

__STATIC_FORCEINLINE rv_csr_t __get_exc_entry (void)

Get Exception entry address.

This function get exception handler address from 'CSR_MTVEC'.

Remark

- This function use to get exception handler address from 'CSR_MTVEC'. Address need to be aligned to 64 bytes.

See also

- `__set_exc_entry`

Returns

Exception handler address

`__STATIC_INLINE void __set_nonvec_entry (rv_csr_t addr)`

Set Non-vector interrupt entry address.

This function set Non-vector interrupt address.

Remark

- This function use to set non-vector interrupt entry address to 'CSR_MTVT2' if
- CSR_MTVT2 bit0 is 1. If 'CSR_MTVT2' bit0 is 0 then set address to 'CSR_MTVEC'

See also

- `__get_nonvec_entry`

Parameters

addr – [in] Non-vector interrupt entry address

`__STATIC_INLINE rv_csr_t __get_nonvec_entry (void)`

Get Non-vector interrupt entry address.

This function get Non-vector interrupt address.

Remark

- This function use to get non-vector interrupt entry address from 'CSR_MTVT2' if
- CSR_MTVT2 bit0 is 1. If 'CSR_MTVT2' bit0 is 0 then get address from 'CSR_MTVEC'.

See also

- `__set_nonvec_entry`

Returns

Non-vector interrupt handler address

`__STATIC_FORCEINLINE rv_csr_t __get_nmi_entry (void)`

Get NMI interrupt entry from 'CSR_MNVEC'.

This function get NMI interrupt address from ‘CSR_MNVEC’.

Remark

- This function use to get NMI interrupt handler address from ‘CSR_MNVEC’. If CSR_MMISC_CTL[9] = 1 ‘CSR_MNVEC’
- will be equal as mtvec. If CSR_MMISC_CTL[9] = 0 ‘CSR_MNVEC’ will be equal as reset vector.
- NMI entry is defined via [CSR_MMISC_CTL](#) (page 107), writing to [CSR_MNVEC](#) (page 107) will be ignored.

Returns

NMI interrupt handler address

2.5.13 PLIC Interrupt

```
__STATIC_FORCEINLINE void PLIC_SetContextThreshold (uint32_t ctxid, uint32_t thresh)
```

```
__STATIC_FORCEINLINE uint32_t PLIC_GetContextThreshold (uint32_t ctxid)
```

```
__STATIC_FORCEINLINE void PLIC_EnableContextInterrupt (uint32_t ctxid, uint32_t source)
```

```
__STATIC_FORCEINLINE void PLIC_DisableContextInterrupt (uint32_t ctxid, uint32_t source)
```

```
__STATIC_FORCEINLINE uint32_t PLIC_GetContextInterruptEnable (uint32_t ctxid,  
uint32_t source)
```

```
__STATIC_FORCEINLINE void PLIC_SetInterruptPending (uint32_t source)
```

```
__STATIC_FORCEINLINE void PLIC_ClearInterruptPending (uint32_t source)
```

```
__STATIC_FORCEINLINE uint32_t PLIC_GetInterruptPending (uint32_t source)
```

```
__STATIC_FORCEINLINE void PLIC_SetPriority (uint32_t source, uint32_t priority)
```

```
__STATIC_FORCEINLINE uint32_t PLIC_GetPriority (uint32_t source)
```

```
__STATIC_FORCEINLINE uint32_t PLIC_ClaimContextInterrupt (uint32_t ctxid)
```

```
__STATIC_FORCEINLINE void PLIC_CompleteContextInterrupt (uint32_t ctxid, uint32_t source)
```

```
__STATIC_INLINE void PLIC_Context_Init (uint32_t ctxid, uint32_t num_sources,  
uint32_t enable, uint32_t thresh)
```

```
__STATIC_FORCEINLINE void __set_trap_entry (rv_csr_t addr)
```

```
__STATIC_FORCEINLINE rv_csr_t __get_trap_entry (void)
```

```
PLIC_Init(num_sources, enable, thresh) PLIC_Context_Init(PLIC_GetHartMContextID (page 168)(),  
(num_sources), (enable), (thresh))
```

```
PLIC_Init_S(num_sources, enable, thresh) PLIC_Context_Init(PLIC_GetHartSContextID (page 168)(),  
(num_sources), (enable), (thresh))
```

```
PLIC_ClaimInterrupt() PLIC_ClaimContextInterrupt(PLIC_GetHartMContextID (page 168)())
```

```
PLIC_ClaimInterrupt_S() PLIC_ClaimContextInterrupt(PLIC_GetHartSContextID (page 168)())
```

```
PLIC_CompleteInterrupt(source) PLIC_CompleteContextInterrupt(PLIC_GetHartMContextID (page 168)(),  
(source))
```

```
PLIC_CompleteInterrupt_S(source) PLIC_CompleteContextInterrupt(PLIC_GetHartSContextID (page 168)(),  
(source))
```

```
PLIC_GetInterruptEnable(source) PLIC_GetContextInterruptEnable(PLIC_GetHartMContextID (page 168)(),  
(source))
```

```
PLIC_GetInterruptEnable_S(source) PLIC_GetContextInterruptEnable(PLIC_GetHartSContextID  
(page 168)(), (source))
```

```
PLIC_EnableInterrupt(source) PLIC_EnableContextInterrupt(PLIC_GetHartMContextID (page 168)(),  
(source))
```

```
PLIC_EnableInterrupt_S(source) PLIC_EnableContextInterrupt(PLIC_GetHartSContextID (page 168)(),  
(source))
```

```
PLIC_DisableInterrupt(source) PLIC_DisableContextInterrupt(PLIC_GetHartMContextID (page 168)(),  
(source))
```

```
PLIC_DisableInterrupt_S(source) PLIC_DisableContextInterrupt(PLIC_GetHartSContextID (page 168)(),  
(source))
```

```
PLIC_SetThreshold(thresh) PLIC_SetContextThreshold(PLIC_GetHartMContextID (page 168)(), (thresh))
```

```
PLIC_SetThreshold_S(thresh) PLIC_SetContextThreshold(PLIC_GetHartSContextID (page 168)(), (thresh))
```

```
PLIC_GetThreshold() PLIC_GetContextThreshold(PLIC_GetHartMContextID (page 168)())
```

```
PLIC_GetThreshold_S() PLIC_GetContextThreshold(PLIC_GetHartSContextID (page 168)())
```

group **PLIC Interrupt**

Functions that manage interrupts via the PLIC.

Defines

PLIC_Init(num_sources, enable, thresh) **PLIC_Context_Init**(*PLIC_GetHartMContextID* (page 168)(), (num_sources), (enable), (thresh))

PLIC_Init_S(num_sources, enable, thresh) **PLIC_Context_Init**(*PLIC_GetHartSContextID* (page 168)(), (num_sources), (enable), (thresh))

PLIC_ClaimInterrupt() **PLIC_ClaimContextInterrupt**(*PLIC_GetHartMContextID* (page 168)())

PLIC_ClaimInterrupt_S() **PLIC_ClaimContextInterrupt**(*PLIC_GetHartSContextID* (page 168)())

PLIC_CompleteInterrupt(source) **PLIC_CompleteContextInterrupt**(*PLIC_GetHartMContextID* (page 168)(), (source))

PLIC_CompleteInterrupt_S(source) **PLIC_CompleteContextInterrupt**(*PLIC_GetHartSContextID* (page 168)(), (source))

PLIC_GetInterruptEnable(source) **PLIC_GetContextInterruptEnable**(*PLIC_GetHartMContextID* (page 168)(), (source))

PLIC_GetInterruptEnable_S(source) **PLIC_GetContextInterruptEnable**(*PLIC_GetHartSContextID* (page 168)(), (source))

PLIC_EnableInterrupt(source) **PLIC_EnableContextInterrupt**(*PLIC_GetHartMContextID* (page 168)(), (source))

PLIC_EnableInterrupt_S(source) **PLIC_EnableContextInterrupt**(*PLIC_GetHartSContextID* (page 168)(), (source))

PLIC_DisableInterrupt(source) **PLIC_DisableContextInterrupt**(*PLIC_GetHartMContextID* (page 168)(), (source))

PLIC_DisableInterrupt_S(source) **PLIC_DisableContextInterrupt**(*PLIC_GetHartSContextID* (page 168)(), (source))

PLIC_SetThreshold(thresh) **PLIC_SetContextThreshold**(*PLIC_GetHartMContextID* (page 168)(), (thresh))

PLIC_SetThreshold_S(thresh) **PLIC_SetContextThreshold**(*PLIC_GetHartSContextID* (page 168)(), (thresh))

PLIC_GetThreshold() **PLIC_GetContextThreshold**(*PLIC_GetHartMContextID* (page 168)())

PLIC_GetThreshold_S() **PLIC_GetContextThreshold**(*PLIC_GetHartSContextID* (page 168)())

Functions

__STATIC_FORCEINLINE void PLIC_SetContextThreshold (uint32_t ctxid, uint32_t thresh)

Set priority threshold value of plic for selected context.

This function set priority threshold value of plic for selected context.

Remark

See also

- `PLIC_GetContextThreshold`

Parameters

- **ctxid** – [in] selected context id
- **thresh** – [in] threshold value

__STATIC_FORCEINLINE uint32_t PLIC_GetContextThreshold (uint32_t ctxid)

Get priority threshold value of plic for selected context.

This function get priority threshold value of plic for selected context.

Remark**See also**

- `PLIC_SetContextThreshold`

Parameters

ctxid – [in] selected context id

Returns

priority threshold value for selected context

__STATIC_FORCEINLINE void PLIC_EnableContextInterrupt (uint32_t ctxid, uint32_t source)

Enable interrupt of selected source plic for selected context.

This function enable interrupt of selected source plic for selected context.

Remark**See also**

- `PLIC_DisableContextInterrupt`

Parameters

- **ctxid** – [in] selected context id

- **source** – [in] interrupt source

```
__STATIC_FORCEINLINE void PLIC_DisableContextInterrupt (uint32_t ctxid,  
uint32_t source)
```

Disable interrupt of selected source plic for selected context.

This function disable interrupt of selected source plic for selected context

Remark

See also

- PLIC_EnableContextInterrupt

Parameters

- **ctxid** – [in] selected context id
- **source** – [in] interrupt source

```
__STATIC_FORCEINLINE uint32_t PLIC_GetContextInterruptEnable (uint32_t ctxid,  
uint32_t source)
```

Get interrupt enable status of selected source plic for selected context.

This function get interrupt enable of selected source plic for selected context.

Remark

See also

- PLIC_EnableContextInterrupt
- PLIC_DisableContextInterrupt

Parameters

- **ctxid** – [in] selected context id
- **source** – [in] interrupt source

Returns

enable status for selected interrupt source for selected context

```
__STATIC_FORCEINLINE void PLIC_SetInterruptPending (uint32_t source)
```

Set interrupt pending of selected source plic.

This function set interrupt pending of selected source plic.

Remark**See also**

- PLIC_GetInterruptPending
- PLIC_ClearInterruptPending

Parameters

source – [in] interrupt source

__STATIC_FORCEINLINE void PLIC_ClearInterruptPending (uint32_t source)

Clear interrupt pending of selected source plic.

This function clear interrupt pending of selected source plic

Remark**See also**

- PLIC_SetInterruptPending
- PLIC_GetInterruptPending

Parameters

source – [in] interrupt source

__STATIC_FORCEINLINE uint32_t PLIC_GetInterruptPending (uint32_t source)

Get interrupt pending status of selected source plic.

This function get interrupt pending of selected source plic

Remark**See also**

- PLIC_SetInterruptPending

- PLIC_ClearInterruptPending

Parameters

source – [in] interrupt source

Returns

interrupt pending status for selected interrupt source

__STATIC_FORCEINLINE void PLIC_SetPriority (uint32_t source, uint32_t priority)

Set interrupt priority for selected source plic.

This function set interrupt priority for selected source plic.

Remark**See also**

- PLIC_GetPriority

Parameters

- **source** – [in] interrupt source
- **priority** – [in] interrupt priority

__STATIC_FORCEINLINE uint32_t PLIC_GetPriority (uint32_t source)

Get interrupt priority for selected source plic.

This function get interrupt priority for selected source plic.

Remark**See also**

- PLIC_SetPriority

Parameters

- **source** – [in] interrupt source
- **priority** – [in] interrupt priority

__STATIC_FORCEINLINE uint32_t PLIC_ClaimContextInterrupt (uint32_t ctxid)

Claim interrupt for plic for selected context.

This function claim interrupt for plic for selected context.

Remark

A successful claim will also atomically clear the corresponding pending bit on the interrupt source. The PLIC can perform a claim at any time and the claim operation is not affected by the setting of the priority threshold register.

See also

- PLIC_CompleteContextInterrupt

Parameters

ctxid – [in] selected context id

Returns

the ID of the highest priority pending interrupt or zero if there is no pending interrupt

__STATIC_FORCEINLINE void PLIC_CompleteContextInterrupt (uint32_t ctxid, uint32_t source)

Complete interrupt for plic for selected context.

This function complete interrupt for plic for selected context.

Remark

The PLIC signals it has completed executing an interrupt handler by writing the interrupt ID it received from the claim to the claim/complete register. The PLIC does not check whether the completion ID is the same as the last claim ID for that context. If the completion ID does not match an interrupt source that is currently enabled for the context, the completion is silently ignored.

See also

- PLIC_ClaimContextInterrupt

Parameters

- **ctxid** – [in] selected context id
- **source** – [in] interrupt source ID to complete

__STATIC_INLINE void PLIC_Context_Init (uint32_t ctxid, uint32_t num_sources, uint32_t enable, uint32_t thresh)

Perform init for plic for selected context.

This function perform initialization steps for plic for selected context.

Remark

- Disable all interrupts
- Set priority threshold to value specified by thresh

Parameters

- **ctxid** – [in] selected context id
- **num_sources** – [in] plic interrupt source count number
- **enable** – [in] plic interrupt enable or not
- **thresh** – [in] plic interrupt threshold

__STATIC_FORCEINLINE void __set_trap_entry (rv_csr_t addr)

Set Trap entry address.

This function set trap entry address to 'CSR_MTVEC'.

Remark

- This function use to set trap entry address to 'CSR_MTVEC'.

See also

- `__get_trap_entry`

Parameters

addr – [in] trap entry address

__STATIC_FORCEINLINE rv_csr_t __get_trap_entry (void)

Get trap entry address.

This function get trap entry address from 'CSR_MTVEC'.

Remark

- This function use to get trap entry address from 'CSR_MTVEC'.

See also

- `__set_trap_entry`

Returns

trap entry address

2.5.14 CIDU Functions

General

`__STATIC_FORCEINLINE uint32_t CIDU_GetCoreNum (void)``__STATIC_FORCEINLINE uint32_t CIDU_GetIntNum (void)``CIDU_BASE __CIDU_BASEADDR``CIDU_RECEIVE_INTERRUPT_EN(core_id) (0x1UL << core_id)``CIDU_CORE_INT_STATUS_OFS 0x0``CIDU_SEMAPHORE_OFS 0x80``CIDU_ICI_SHADOW_OFS 0x3FFC``CIDU_INT_INDICATOR_OFS 0x4000``CIDU_INT_MASK_OFS 0x8000``CIDU_CORE_NUM_OFS 0xC084``CIDU_INT_NUM_OFS 0xC090``CIDU_CORE_INT_STATUS_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_CORE_INT_STATUS_OFS) + ((n) << 2))``CIDU_SEMAPHORE_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_SEMAPHORE_OFS) + ((n) << 2))``CIDU_ICI_SHADOW_ADDR (unsigned long)((CIDU_BASE) + (CIDU_ICI_SHADOW_OFS))``CIDU_INT_INDICATOR_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_INT_INDICATOR_OFS) + ((n) << 2))``CIDU_INT_MASK_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_INT_MASK_OFS) + ((n) << 2))`

CIDU_CORE_NUM_ADDR (unsigned long)((CIDU_BASE) + (CIDU_CORE_NUM_OFS))

CIDU_INT_NUM_ADDR (unsigned long)((CIDU_BASE) + (CIDU_INT_NUM_OFS))

CIDU_ICI_SEND_CORE_ID_POS 16

group **CIDU Functions**

Functions that manage external interrupts, inter core interrupts and semaphores.

Nuclei provide Cluster Interrupt Distribution Unit (CIDU) for scenarios that a SMP system is designed for real time application or both Linux and real time application, and Nuclei processor core can optionally support CIDU. The CIDU is used to distribute external interrupts to the core's ECLIC, also it provides Inter Core Interrupt (ICI) and Semaphores Mechanism. Its features are as follows:

- Support up to 16 Cores in one cluster
- Support up to 4096 external interrupts sources
- Support up to 16 Inter Core Interrupts
- Support 32 Semaphores

Defines

CIDU_BASE __CIDU_BASEADDR

CIDU_RECEIVE_INTERRUPT_EN(core_id) (0x1UL << core_id)

Indicates the core can receive corresponding interrupt.

CIDU_CORE_INT_STATUS_OFS 0x0

Core n Inter Core Interrupt status register base offset.

CIDU_SEMAPHORE_OFS 0x80

Semaphore n register base offset.

CIDU_ICI_SHADOW_OFS 0x3FFC

ICI Interrupt source core ID and target core ID register offset.

CIDU_INT_INDICATOR_OFS 0x4000

External interrupt n indicator register base offset.

CIDU_INT_MASK_OFS 0x8000

External interrupt n mask (mask interrupt n to cores or not when interrupt n indicator on)register base offset.

CIDU_CORE_NUM_OFS 0xC084

Static configuration core num register offset.

CIDU_INT_NUM_OFS 0xC090

Static configuration external interrupt number register offset.

CIDU_CORE_INT_STATUS_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_CORE_INT_STATUS_OFS) + ((n) << 2))

Core n Inter Core Interrupt status register address.

CIDU_SEMAPHORE_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_SEMAPHORE_OFS) + ((n) << 2))

Semaphore n register address.

CIDU_ICI_SHADOW_ADDR (unsigned long)((CIDU_BASE) + (CIDU_ICI_SHADOW_OFS))

ICI Interrupt source core ID and target core ID register address.

CIDU_INT_INDICATOR_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_INT_INDICATOR_OFS) + ((n) << 2))

External interrupt n indicator register address.

CIDU_INT_MASK_ADDR(n) (unsigned long)((CIDU_BASE) + (CIDU_INT_MASK_OFS) + ((n) << 2))

External interrupt n mask (mask interrupt n to cores or not when interrupt n indicator on) register address.

CIDU_CORE_NUM_ADDR (unsigned long)((CIDU_BASE) + (CIDU_CORE_NUM_OFS))

Static configuration core num register address.

CIDU_INT_NUM_ADDR (unsigned long)((CIDU_BASE) + (CIDU_INT_NUM_OFS))

Static configuration external interrupt number register address.

CIDU_ICI_SEND_CORE_ID_POS 16

Functions

__STATIC_FORCEINLINE uint32_t CIDU_GetCoreNum (void)

Get core number in the cluster.

Indicate the static configuration core num in the cluster.

Remark

- In a Nulcei multi-core system, each core has an identifiable serial number, the serial number starts from 0 and is continuous, also the number is static.
- CORE_NUM register is read only.

Returns

core number configured

__STATIC_FORCEINLINE uint32_t CIDU_GetIntNum (void)

Get external interrupt number.

Indicate the static configuration external interrupt number

Remark

- INT_NUM register is read only.

Returns

interrupt number configured

External Interrupt Distribution Functions

```
__STATIC_FORCEINLINE void CIDU_BroadcastExtInterrupt (uint32_t int_id, uint32_t to_cores)
```

```
__STATIC_FORCEINLINE uint32_t CIDU_GetBroadcastModeStatus (uint32_t int_id)
```

```
__STATIC_INLINE long CIDU_SetFirstClaimMode (uint32_t int_id, uint32_t core_id)
```

```
__STATIC_FORCEINLINE void CIDU_ResetFirstClaimMode (uint32_t int_id)
```

```
__STATIC_FORCEINLINE uint32_t CIDU_GetClaimStatus (uint32_t int_id)
```

group **External Interrupt Distribution Functions**

Functions that distribute external interrupts to cores.

Functions

```
__STATIC_FORCEINLINE void CIDU_BroadcastExtInterrupt (uint32_t int_id,  
uint32_t to_cores)
```

Broadcast external interrupt to cores.

This function broadcasts external interrupt which id is int_id to some/all target cores

Remark

- External IRQn ID(int_id) is from the hard-wired perspective, which has an offset mapped to the ECLIC IRQn, see Interrupt Number Definition in <Device.h>
- By default on reset, only core 0 can receive interrupt which id is int_id

Parameters

- **int_id** – [in] external interrupt id
- **to_cores** – [in] target cores which can receive interrupt, use bitwise inclusive or of CIDU_RECEIVE_INTERRUPT_EN(core_id)

__STATIC_FORCEINLINE uint32_t CIDU_GetBroadcastModeStatus (uint32_t int_id)

get broadcast mode status

Just query the INTn_INDICATOR register value

Remark

- External IRQn ID(int_id) is from the hard-wired perspective, which has an offset mapped to the ECLIC IRQn, see Interrupt Number Definition in <Device.h>
- By default on reset, only core 0 can receive interrupt which id is int_id

Parameters

int_id – [in] external interrupt id

Returns

INTn_INDICATOR register value

__STATIC_INLINE long CIDU_SetFirstClaimMode (uint32_t int_id, uint32_t core_id)

Let the first coming core to first claim the interrupt.

In external interrupt broadcast mode, make the first coming core to claim this interrupt and then can handle it.

Remark

- External IRQn ID(int_id) is from the hard-wired perspective, which has an offset mapped to the ECLIC IRQn, see Interrupt Number Definition in <Device.h>.
- If it fails to claim the interrupt, it should quit the interrupt n's handler of all cores
- When a core claims the interrupt successfully and has handled it, it must call CIDU_ResetFirstClaimMode to reset the claim.

See also

- CIDU_BroadcastExtInterrupt
- CIDU_ResetFirstClaimMode

Parameters

- **int_id** – [in] external interrupt id
- **core_id** – [in] core id that receive the interrupt

Returns

-1 if it fails to claim the interrupt, else it can continue to handle the interrupt

__STATIC_FORCEINLINE void CIDU_ResetFirstClaimMode (uint32_t int_id)

Reset the claim mode mask.

Reset the claim mode mask by Writing the reset value (all 1) to it

Remark

- External IRQn ID(int_id) is from the hard-wired perspective, which has an offset mapped to the ECLIC IRQn, see Interrupt Number Definition in <Device.h>
- When a core claims the interrupt successfully and handle it, it must call CIDU_ResetFirstClaimMode to reset the claim

See also

- CIDU_SetFirstClaimMode

Parameters

int_id – [in] external interrupt id

__STATIC_FORCEINLINE uint32_t CIDU_GetClaimStatus (uint32_t int_id)

Get the claim mask status.

Get the claim mode status, each bit[n] indicates whether core n has claimed interrupt successfully, 1 means yes, 0 means no.

Remark

- External IRQn ID(int_id) is from the hard-wired perspective, which has an offset mapped to the ECLIC IRQn, see Interrupt Number Definition in <Device.h>

See also

- CIDU_ResetFirstClaimMode
- CIDU_SetFirstClaimMode

Parameters

int_id – [in] external interrupt id

Returns

claim mode register INTn_MASK value

Inter Core Interrupt Functions

```
__STATIC_FORCEINLINE void CIDU_TriggerInterCoreInt (uint32_t send_core_id,  
uint32_t recv_core_id)
```

```
__STATIC_FORCEINLINE uint32_t CIDU_QueryCoreIntSenderMask (uint32_t recv_core_id)
```

```
__STATIC_FORCEINLINE void CIDU_ClearInterCoreIntReq (uint32_t send_core_id,  
uint32_t recv_core_id)
```

group Inter Core Interrupt Functions

Functions that implement Inter Core Interrupt mechanism.

Inter Core Interrupt (ICI) means that one core can send interrupt to another core in a multi-core cluster. CIDU ICI belongs to Internal Interrupt.

- CIDU ICI Interrupt ID is fixed to 16.

Functions

```
__STATIC_FORCEINLINE void CIDU_TriggerInterCoreInt (uint32_t send_core_id,  
uint32_t recv_core_id)
```

Trigger interrupt to another core in a multi-core cluster.

When called by core `send_core_id`, CIDU will trigger ICI to core `recv_core_id` automatically. and core `recv_core_id` could query `CIDU_GetCoreIntSenderId` to know the sender.

Remark

- The core `recv_core_id` need to call `CIDU_ClearInterCoreIntReq` to clear the corresponding bit/bits of its own `COREn_INT_STATUS`.
- It supports that multiple cores call `CIDU_TriggerInterCoreInt` simultaneously.

See also

- `CIDU_GetCoreIntSenderId`
- `CIDU_ClearInterCoreIntReq`

Parameters

- `send_core_id` – [in] the core id which want to send the inter core interrupt
- `recv_core_id` – [in] the core id which will receive the inter core interrupt

```
__STATIC_FORCEINLINE uint32_t CIDU_QueryCoreIntSenderMask (uint32_t recv_core_id)
```

Core `recv_core_id` queries out who sends inter core interrupt to itself.

In the ISR of ICI, receive core can query if `bit[n]` of this return value is 1, core `n` sends the current ICI, if `bit[m]` is 1, then core `m` also sends, etc.

Remark

- If the ICI ISR has finished the job, should call `CIDU_ClearInterCoreIntReq` to clear the IRQ

Parameters

`recv_core_id` – [in] the core id which receives the inter core interrupt

Returns

Value that shows sender core's ID `n` whose `bitn` is 1

```
__STATIC_FORCEINLINE void CIDU_ClearInterCoreIntReq (uint32_t send_core_id,
uint32_t recv_core_id)
```

Clear the corresponding bit/bits of ICI request triggered by sender core.

Core `recv_core_id` write 1 to clear the bit `send_core_id` of the core `recv_core_id`'s `COREn_INT_STATUS`.

Remark

- If the ICI ISR has finished the job of `send_core_id_n`'s ICI, then clear bit `send_core_id_n`; if it has finished `send_core_id_n` and `send_core_id_m`'s, then should clear both the bits, etc.

Parameters

- **`send_core_id`** – [in] the core id which wants to send the inter core interrupt
- **`recv_core_id`** – [in] the core id which will receive the inter core interrupt

Semaphore Functions

```
__STATIC_FORCEINLINE uint32_t CIDU_GetSemaphoreStatus (uint32_t semph_n)
```

```
__STATIC_INLINE long CIDU_CheckSemaphoreAcquired (uint32_t semph_n, uint32_t core_id)
```

```
__STATIC_INLINE long CIDU_AcquireSemaphore (uint32_t semph_n, uint32_t core_id)
```

```
__STATIC_INLINE void CIDU_AcquireSemaphore_Block (uint32_t semph_n, uint32_t core_id)
```

```
__STATIC_FORCEINLINE void CIDU_ReleaseSemaphore (uint32_t semph_n)
```

group Semaphore Functions

Functions that configure and use semaphores

Semaphore is very useful for multi-core cluster without SMP enable.

- All Cores in the cluster agree on using SEMAPHORE_n register to protect a critical resource (an UART device for example).
- If Core n wants to access the critical resource, it should try to own the SEMPAPHORE_n register, or else it can't access the critical resource.
- When the Core n owns the register SEMPAPHORE_n and finishes the job related the critical resource, then it should release the register by writing all 1 to it.

Functions

__STATIC_FORCEINLINE uint32_t CIDU_GetSemaphoreStatus (uint32_t semph_n)

Get SEMAPHOREn's value.

Just query the semaphore n's value

Parameters

semph_n – [in] the semaphore id used to protect a critical resource

Returns

register SEMAPHOREn_STATUS value

__STATIC_INLINE long CIDU_CheckSemaphoreAcquired (uint32_t semph_n, uint32_t core_id)

check SEMAPHOREn's acquired status

Query that whether SEMAPHOREn has been acquired by one core successfully.

Remark

- When the core n owns the register SEMPAPHORE_n and finishes the job related the critical resource, it should call CIDU_ReleaseSemaphore to release it.

See also

- CIDU_GetSemaphoreStatus
- CIDU_ReleaseSemaphore

Parameters

- **semph_n** – [in] the semaphore id used to protect a critical resource
- **core_id** – [in] the core id that wants to access the critical resource

Returns

0 if core_id has acquired this semaphore successfully, or else -1 if failed

__STATIC_INLINE long CIDU_AcquireSemaphore (uint32_t semph_n, uint32_t core_id)

Acquire the SEMAPHOREn.

Acquire the SEMAPHOREn, and check the acquired status

Remark

- When the core n owns the register SEMPAPHORE_n and finishes the job related the critical resource, it should call CIDU_ReleaseSemaphore to release it.

See also

- CIDU_CheckSemaphoreAcquired
- CIDU_ReleaseSemaphore

Parameters

- **semph_n** – [in] the semaphore id used to protect a critical resource
- **core_id** – [in] the core id that wants to access the critical resource

Returns

0 if core_id has acquired this semaphore successfully, or else -1 if failed

__STATIC_INLINE void CIDU_AcquireSemaphore_Block (uint32_t semph_n, uint32_t core_id)

Keep acquiring the SEMAPHOREn until it has acquired this semaphore successfully.

Query that whether SEMAPHOREn has been owned by one core successfully, if not, keep trying.

Remark

- Core n will block here acquiring, so take care that core should release the semaphore when related job done.

See also

- CIDU_AcquireSemaphore
- CIDU_ReleaseSemaphore

Parameters

- **semph_n** – [in] the semaphore id used to protect a critical resource
- **core_id** – [in] the core id that wants to access the critical resource

__STATIC_FORCEINLINE void CIDU_ReleaseSemaphore (uint32_t semph_n)

Release the SEMAPHOREn.

Release the SEMAPHOREn by writing all 1 to SEMAPHOREn register.

Remark

- When the core finishes the job related to the critical resource, it should release the corresponding semaphore.

See also

- CIDU_AcquireSemaphore_Block

Parameters

semph_n – [in] the semaphore id used to protect a critical resource

2.5.15 FPU Functions

group FPU Functions

Functions that related to the RISC-V FPU (F and D extension).

Nuclei provided floating point unit by RISC-V F and D extension.

- **F extension** adds single-precision floating-point computational instructions compliant with the IEEE 754-2008 arithmetic standard, `__RISCV_FLEN = 32`. The F extension adds 32 floating-point registers, f0-f31, each 32 bits wide, and a floating-point control and status register fcsr, which contains the operating mode and exception status of the floating-point unit.
- **D extension** adds double-precision floating-point computational instructions compliant with the IEEE 754-2008 arithmetic standard. The D extension widens the 32 floating-point registers, f0-f31, to 64 bits, `__RISCV_FLEN = 64`

Defines

`__RISCV_FLEN` 64

`__get_FCSR()` [__RV_CSR_READ](#) (page 75)([CSR_FCSR](#) (page 86))

Get FCSR CSR Register.

`__set_FCSR(val)` [__RV_CSR_WRITE](#) (page 75)([CSR_FCSR](#) (page 86), (val))

Set FCSR CSR Register with val.

__get_FRM() [__RV_CSR_READ](#) (page 75)([CSR_FRM](#) (page 86))

Get FRM CSR Register.

__set_FRM(val) [__RV_CSR_WRITE](#) (page 75)([CSR_FRM](#) (page 86), (val))

Set FRM CSR Register with val.

__get_FFLAGS() [__RV_CSR_READ](#) (page 75)([CSR_FFLAGS](#) (page 86))

Get FFLAGS CSR Register.

__set_FFLAGS(val) [__RV_CSR_WRITE](#) (page 75)([CSR_FFLAGS](#) (page 86), (val))

Set FFLAGS CSR Register with val.

__enable_FPU()

Enable FPU Unit, and set state to initial.

__disable_FPU() [__RV_CSR_CLEAR](#) (page 76)([CSR_MSTATUS](#) (page 91), [MSTATUS_FS](#) (page 114))

Disable FPU Unit.

- We can save power by disable FPU Unit.
- When FPU Unit is disabled, any access to FPU related CSR registers and FPU instructions will cause illegal Instruction Exception.

__RV_FLW(freg, addr, ofs)

Load a single-precision value from memory into float point register freg using flw instruction.

The FLW instruction loads a single-precision floating point value from memory address (addr + ofs) into floating point register freg(f0-f31)

Remark

- FLW and FSW operations need to make sure the address is 4 bytes aligned, otherwise it will cause exception code 4(Load address misaligned) or 6 (Store/AMO address misaligned)
- FLW and FSW do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved

Parameters

- **freg** – **[in]** The floating point register, eg. [FREG\(0\)](#) (page 130), f0
- **addr** – **[in]** The memory base address, 4 byte aligned required
- **ofs** – **[in]** a 12-bit immediate signed byte offset value, should be an const value

__RV_FSW(freg, addr, ofs)

Store a single-precision value from float point freg into memory using fsw instruction.

The FSW instruction stores a single-precision value from floating point register to memory

Remark

- FLW and FSW operations need to make sure the address is 4 bytes aligned, otherwise it will cause exception code 4(Load address misaligned) or 6 (Store/AMO address misaligned)
- FLW and FSW do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved

Parameters

- **freg** – [in] The floating point register(f0-f31), eg. *FREG(0)* (page 130), f0
- **addr** – [in] The memory base address, 4 byte aligned required
- **ofs** – [in] a 12-bit immediate signed byte offset value, should be an const value

__RV_FLD(freg, addr, ofs)

Load a double-precision value from memory into float point register freg using fld instruction.

The FLD instruction loads a double-precision floating point value from memory address (addr + ofs) into floating point register freg(f0-f31)

Remark

- FLD and FSD operations need to make sure the address is 8 bytes aligned, otherwise it will cause exception code 4(Load address misaligned) or 6 (Store/AMO address misaligned)
- FLD and FSD do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

Attention

- Function only available for double precision floating point unit, FLEN = 64

Parameters

- **freg** – [in] The floating point register, eg. *FREG(0)* (page 130), f0
- **addr** – [in] The memory base address, 8 byte aligned required
- **ofs** – [in] a 12-bit immediate signed byte offset value, should be an const value

__RV_FSD(freg, addr, ofs)

Store a double-precision value from float point freg into memory using fsd instruction.

The FSD instruction stores double-precision value from floating point register to memory

Remark

- FLD and FSD operations need to make sure the address is 8 bytes aligned, otherwise it will cause exception code 4(Load address misaligned) or 6 (Store/AMO address misaligned)
- FLD and FSD do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

Attention

- Function only available for double precision floating point unit, FLEN = 64

Parameters

- **freg** – [in] The floating point register(f0-f31), eg. *FREG(0)* (page 130), f0
- **addr** – [in] The memory base address, 8 byte aligned required
- **ofs** – [in] a 12-bit immediate signed byte offset value, should be an const value

__RV_FLOAD __RV_FLD (page 692)

Load a float point value from memory into float point register freg using flw/fld instruction.

- For Single-Precision Floating-Point Mode(__FPU_PRESENT == 1, __RISCV_FLEN == 32): It will call *__RV_FLW* (page 691) to load a single-precision floating point value from memory to floating point register
- For Double-Precision Floating-Point Mode(__FPU_PRESENT == 2, __RISCV_FLEN == 64): It will call *__RV_FLD* (page 692) to load a double-precision floating point value from memory to floating point register

Attention

Function behaviour is different for __FPU_PRESENT = 1 or 2, please see the real function this macro represent

__RV_FSTORE __RV_FSD (page 692)

Store a float value from float point freg into memory using fsw/fsd instruction.

- For Single-Precision Floating-Point Mode(__FPU_PRESENT == 1, __RISCV_FLEN == 32): It will call *__RV_FSW* (page 691) to store floating point register into memory
- For Double-Precision Floating-Point Mode(__FPU_PRESENT == 2, __RISCV_FLEN == 64): It will call *__RV_FSD* (page 692) to store floating point register into memory

Attention

Function behaviour is different for __FPU_PRESENT = 1 or 2, please see the real function this macro represent

SAVE_FPU_CONTEXT()

Save FPU context into variables for interrupt nesting.

This macro is used to declare variables which are used for saving FPU context + FCSR, and it will store the nessary fpu registers into these variables, it need to be used in a interrupt when in this interrupt fpu registers are used.

Remark

- It need to be used together with *RESTORE_FPU_CONTEXT* (page 694)

- Don't use variable names `__fpu_context` in your ISR code
- If your ISR code will use FPU registers, and this interrupt is nested. Then you can do it like this:

```
void eclic_mtip_handler(void)
{
    // !!!Interrupt is enabled here!!!
    // !!!Higher priority interrupt could nest it!!!

    // Necessary only when you need to use fpu registers
    // in this isr handler functions
    SAVE_FPU_CONTEXT();

    // put your own interrupt handling code here

    // pair of SAVE_FPU_CONTEXT()
    RESTORE_FPU_CONTEXT();
}
```

RESTORE_FPU_CONTEXT()

Restore necessary FPU registers from variables for interrupt nesting.

This macro is used to restore necessary FPU registers and FCSR from pre-defined variables in [SAVE_FPU_CONTEXT](#) (page 693) macro.

Remark

- It needs to be used together with [SAVE_FPU_CONTEXT](#) (page 693)

Typedefs

```
typedef uint64_t rv_fpu_t
```

Type of FPU register, depends on the FLEN defined in RISC-V.

2.5.16 PMP Functions

Click [Nuclei PMP Unit](#)³⁵ to learn about Core PMP Unit in Nuclei ISA Spec.

```
__STATIC_INLINE rv_csr_t __get_PMPCFGx (uint32_t csr_idx)
```

```
__STATIC_INLINE void __set_PMPCFGx (uint32_t csr_idx, rv_csr_t pmpcfg)
```

```
__STATIC_INLINE uint8_t __get_PMPxCFG (uint32_t entry_idx)
```

```
__STATIC_INLINE void __set_PMPxCFG (uint32_t entry_idx, uint8_t pmpxcfg)
```

³⁵ https://doc.nucleisys.com/nuclei_spec/isa/pmp.html

```

__STATIC_INLINE rv_csr_t __get_PMPADDRx (uint32_t csr_idx)

__STATIC_INLINE void __set_PMPADDRx (uint32_t csr_idx, rv_csr_t pmpaddr)

__STATIC_INLINE void __set_PMPENTRYx (uint32_t entry_idx, const pmp_config *pmp_cfg)

__STATIC_INLINE int __get_PMPENTRYx (unsigned int entry_idx, pmp_config *pmp_cfg)

PMPCFG_LIST_RV32_0_7 X(0) X(1) X(2) X(3) X(4) X(5) X(6) X(7)

PMPCFG_LIST_RV32_8_15 X(8) X(9) X(10) X(11) X(12) X(13) X(14) X(15)

PMPCFG_LIST_RV64_0_6 X(0) X(2) X(4) X(6)

PMPCFG_LIST_RV64_8_14 X(8) X(10) X(12) X(14)

PMPCFG_LIST PMPCFG_LIST_RV32_0_7

PMPADDR_LIST_BASE X(0) X(1) X(2) X(3) X(4) X(5) X(6) X(7)

PMPADDR_LIST_8_15 X(8) X(9) X(10) X(11) X(12) X(13) X(14) X(15)

PMPADDR_LIST_16_31

PMPADDR_LIST_32_63

PMPADDR_LIST PMPADDR_LIST_BASE PMPADDR_LIST_8_15

struct pmp_config

```

group **PMP Functions**

Functions that related to the RISC-V Physical Memory Protection.

Optional physical memory protection (PMP) unit provides per-hart machine-mode control registers to allow physical memory access privileges (read, write, execute) to be specified for each physical memory region.

The PMP can supports region access control settings as small as four bytes.

Defines

PMPCFG_LIST_RV32_0_7 X(0) X(1) X(2) X(3) X(4) X(5) X(6) X(7)

PMPCFG list for RV32 (indices 0-7)

For RV32, each PMPCFG register holds 4 PMP entries. This list covers the first 32 entries (registers 0-7).

PMPCFG_LIST_RV32_8_15 X(8) X(9) X(10) X(11) X(12) X(13) X(14) X(15)

PMPCFG list for RV32 (indices 8-15)

For RV32, each PMPCFG register holds 4 PMP entries. This list covers entries 33-64 (registers 8-15).

PMPCFG_LIST_RV64_0_6 X(0) X(2) X(4) X(6)

PMPCFG list for RV64 (even indices 0-6)

For RV64, each PMPCFG register holds 8 PMP entries. This list covers the first 32 entries (registers 0,2,4,6).

PMPCFG_LIST_RV64_8_14 X(8) X(10) X(12) X(14)

PMPCFG list for RV64 (even indices 8-14)

For RV64, each PMPCFG register holds 8 PMP entries. This list covers entries 33-64 (registers 8,10,12,14).

PMPCFG_LIST PMPCFG_LIST_RV32_0_7

Select appropriate PMPCFG list based on architecture and PMP entry count.

This macro defines the complete PMPCFG list according to:

- RISC-V architecture (RV32/RV64)
- Number of PMP entries (`__PMP_ENTRY_NUM`)

Note

Valid indices depend on configuration:

- RV32:
 - <=32 entries: indices 0-7
 - <=64 entries: indices 0-15
- RV64:
 - <=32 entries: indices 0,2,4,6
 - <=64 entries: indices 0,2,4,6,8,10,12,14

PMPADDR_LIST_BASE X(0) X(1) X(2) X(3) X(4) X(5) X(6) X(7)

Base PMPADDR list (indices 0-7)

PMPADDR_LIST_8_15 X(8) X(9) X(10) X(11) X(12) X(13) X(14) X(15)

Extended PMPADDR list (indices 8-15)

PMPADDR_LIST_16_31

Extended PMPADDR list (indices 16-31)

PMPADDR_LIST_32_63

Extended PMPADDR list (indices 32-63)

PMPADDR_LIST PMPADDR_LIST_BASE PMPADDR_LIST_8_15

Select appropriate PMPADDR list based on PMP_ENTRY_NUM.

This macro defines the complete PMPADDR list by combining base and extended lists according to the configured number of PMP entries.

Note

The actual list is determined by the __PMP_ENTRY_NUM configuration:

- <=8: Only base list (0-7)
- <=16: Base + 8-15
- <=32: Base + 8-15 + 16-31
- <=64: Base + 8-15 + 16-31 + 32-63

Functions**__STATIC_INLINE rv_csr_t __get_PMPCFGx (uint32_t csr_idx)**

Get PMPCFGx Register by CSR index.

Return the content of the PMPCFGx Register.

Remark

The function returns 0 for invalid indices to prevent illegal accesses.

Note

Architecture-specific behavior:

- RV32:
 - Each register holds 4 PMP entries
 - Valid indices: 0 to ceil(__PMP_ENTRY_NUM/4)-1
 - 64 entries require indices 0-15
- RV64:
 - Each register holds 8 PMP entries
 - Only even indices are valid
 - 64 entries require indices 0,2,4,6,8,10,12,14

Parameters

csr_idx – [in] PMPCFG CSR index (0-15 for RV32, 0,2,4,6,8,10,12,14 for RV64)

Returns

PMPCFGx Register value

__STATIC_INLINE void __set_PMPxCFG (uint32_t csr_idx, rv_csr_t pmpcfg)

Set PMPCFGx by CSR index.

Write the given value to the PMPCFGx Register.

Remark

The function does nothing for invalid indices to prevent illegal accesses.

Note

Architecture-specific behavior:

- RV32:
 - Each register holds 4 PMP entries
 - Valid indices: 0 to ceil(__PMP_ENTRY_NUM/4)-1
 - 64 entries require indices 0-15
- RV64:
 - Each register holds 8 PMP entries
 - Only even indices are valid
 - 64 entries require indices 0,2,4,6,8,10,12,14

Parameters

- **csr_idx** – [in] PMPCFG CSR index (0-15 for RV32, 0,2,4,6,8,10,12,14 for RV64)
- **pmpcfg** – [in] PMPCFGx Register value to set

__STATIC_INLINE uint8_t __get_PMPxCFG (uint32_t entry_idx)

Get 8bit PMPxCFG Register by PMP entry index.

Return the content of the PMPxCFG Register.

Parameters

entry_idx – [in] PMP region index(0-63)

Returns

PMPxCFG Register value

__STATIC_INLINE void __set_PMPxCFG (uint32_t entry_idx, uint8_t pmpxcfg)

Set 8bit PMPxCFG by pmp entry index.

Set the given pmpxcfg value to the PMPxCFG Register.

Remark

- For RV32, 4 pmpxcfgs are densely packed into one CSR in order. For RV64, 8 pmpxcfgs are densely packed into one CSR in order.

Parameters

- **entry_idx** – [in] PMPx region index(0-63)
- **pmpxcfg** – [in] PMPxCFG register value to set

__STATIC_INLINE rv_csr_t __get_PMPADDRx (uint32_t csr_idx)

Get PMPADDRx Register by CSR index.

Return the content of the PMPADDRx Register.

Parameters

csr_idx – [in] PMP region CSR index(0-63)

Returns

PMPADDRx Register value

__STATIC_INLINE void __set_PMPADDRx (uint32_t csr_idx, rv_csr_t pmpaddr)

Set PMPADDRx by CSR index.

Write the given value to the PMPADDRx Register.

Parameters

- **csr_idx** – [in] PMP region CSR index(0-63)
- **pmpaddr** – [in] PMPADDRx Register value to set

__STATIC_INLINE void __set_PMPENTRYx (uint32_t entry_idx, const pmp_config *pmp_cfg)

Set PMP entry by entry idx.

Write the given value to the PMPxCFG Register and PMPADDRx.

Remark

- If the size of memory region is 2^{12} (4KB) range, pmp_cfg->order makes 12, and the like.
- Suppose the size of memory region is 2^X bytes range, if $X \geq 3$, the NA4 mode is not selectable, NAPOT is selected.
- TOR of A field in PMP configuration register is not considered here.

Parameters

- **entry_idx** – [in] PMP entry index(0-63)
- **pmp_cfg** – [in] structure of L, X, W, R field of PMP configuration register, memory region base address and size of memory region as power of 2

```
__STATIC_INLINE int __get_PMPENTRYx (unsigned int entry_idx, pmp_config *pmp_cfg)
```

Get PMP entry by entry idx.

Write the given value to the PMPxCFG Register and PMPADDRx.

Remark

- If the size of memory region is 2¹²(4KB) range, pmp_cfg->order makes 12, and the like.
- TOR of A field in PMP configuration register is not considered here.

Parameters

- **entry_idx** – [in] PMP entry index(0-63)
- **pmp_cfg** – [out] structure of L, X, W, R, A field of PMP configuration register, memory region base address and size of memory region as power of 2

Returns

-1 failure, else 0 success

```
struct pmp_config
#include <core_feature_pmp.h>
```

2.5.17 SPMP/sMPU Functions

Click [TEE Introduction](#)³⁶ to learn about Core sPMP/sMPU Unit in Nuclei ISA Spec.

typedef *spmp_config* (page 701) **mpu_config**

```
__STATIC_INLINE rv_csr_t __get_sPMPCFGx (uint32_t csr_idx)
```

```
__STATIC_INLINE void __set_sPMPCFGx (uint32_t csr_idx, rv_csr_t spmpcfg)
```

```
__STATIC_INLINE uint8_t __get_sPMPxCFG (uint32_t entry_idx)
```

```
__STATIC_INLINE void __set_sPMPxCFG (uint32_t entry_idx, uint8_t spmpxcfg)
```

```
__STATIC_INLINE rv_csr_t __get_sPMPADDRx (uint32_t csr_idx)
```

```
__STATIC_INLINE void __set_sPMPADDRx (uint32_t csr_idx, rv_csr_t spmpaddr)
```

```
__STATIC_INLINE void __set_sPMPENTRYx (uint32_t entry_idx, const spmp_config *spmp_cfg)
```

³⁶ https://doc.nucleisys.com/nuclei_spec/isa/tee.html

```

__STATIC_INLINE int __get_sPMPENTRYx (unsigned int entry_idx, spmp_config *spmp_cfg)

__STATIC_INLINE void __set_SMPUSWITCHx (uint64_t val)

__STATIC_INLINE uint64_t __get_SMPUSWITCHx (void)

__get_SMPUCFGx __get_sPMPCFGx

__set_SMPUCFGx __set_sPMPCFGx

__get_SMPUxCFG __get_sPMPxCFG

__set_SMPUxCFG __set_sPMPxCFG

__get_SMPUADDRx __get_sPMPADDRx

__set_SMPUADDRx __set_sPMPADDRx

__set_SMPUENTRYx __set_sPMPENTRYx

__get_SMPUENTRYx __get_sPMPENTRYx

struct spmp_config

```

group sPMP or sMPU Functions

Functions that related to the RISC-V supervisor-mode Physical Memory Protection.

Optional supervisor physical memory protection (sPMP) unit provides per-hart supervisor-mode control registers to allow physical memory access privileges (read, write, execute) to be specified for each physical memory region. The sPMP values are checked after the physical address to be accessed pass PMP checks described in the RISC-V privileged spec.

Like PMP, the sPMP can supports region access control settings as small as four bytes.

Defines

```

__get_SMPUCFGx __get_sPMPCFGx

__set_SMPUCFGx __set_sPMPCFGx

__get_SMPUxCFG __get_sPMPxCFG

__set_SMPUxCFG __set_sPMPxCFG

```

`__get_SMPUADDRx __get_sPMPADDRx`

`__set_SMPUADDRx __set_sPMPADDRx`

`__set_SMPUENTRYx __set_sPMPENTRYx`

`__get_SMPUENTRYx __get_sPMPENTRYx`

Typedefs

typedef *smpu_config* (page 701) **smpu_config**

sPMP has upgraded to S-mode Memory Protection Unit, renamed as SMPU, but still share the apis with sPMP's

Functions

`__STATIC_INLINE rv_csr_t __get_sPMPCFGx (uint32_t csr_idx)`

Get sPMPCFGx Register by csr index.

Return the content of the sPMPCFGx Register.

Remark

- For RV64, only `csr_idx = 0` and `csr_idx = 2` is allowed. `smpcfg0` and `smpcfg2` hold the configurations for the 16 sPMP entries, `smpcfg1` and `smpcfg3` are illegal
- For RV32, `smpcfg0`–`smpcfg3`, hold the configurations `smp0cfg`–`smp15cfg` for the 16 sPMP entries

Parameters

csr_idx – [in] sPMPCFG CSR index(0-3)

Returns

sPMPCFGx Register value

`__STATIC_INLINE void __set_sPMPCFGx (uint32_t csr_idx, rv_csr_t smpcfg)`

Set sPMPCFGx by csr index.

Write the given value to the sPMPCFGx Register.

Remark

- For RV64, only `csr_idx = 0` and `csr_idx = 2` is allowed. `smpcfg0` and `smpcfg2` hold the configurations for the 16 sPMP entries, `smpcfg1` and `smpcfg3` are illegal

- For RV32, smpcfg0–smpcfg3, hold the configurations smp0cfg–smp15cfg for the 16 sPMP entries

Parameters

- **csr_idx** – [in] sPMPCFG CSR index(0-3)
- **smpcfg** – [in] sPMPCFGx Register value to set

__STATIC_INLINE uint8_t __get_sPMPxCFG (uint32_t entry_idx)

Get 8bit sPMPxCFG Register by sPMP entry index.

Return the content of the sPMPxCFG Register.

Parameters

- **entry_idx** – [in] sPMP region index(0-15)

Returns

sPMPxCFG Register value

__STATIC_INLINE void __set_sPMPxCFG (uint32_t entry_idx, uint8_t smpxcfg)

Set 8bit sPMPxCFG by smp entry index.

Set the given smpxcfg value to the sPMPxCFG Register.

Remark

- For RV32, 4 smpxcfgs are densely packed into one CSR in order For RV64, 8 smpxcfgs are densely packed into one CSR in order

Parameters

- **entry_idx** – [in] sPMPx region index(0-15)
- **smpxcfg** – [in] sPMPxCFG register value to set

__STATIC_INLINE rv_csr_t __get_sPMPADDRx (uint32_t csr_idx)

Get sPMPADDRx Register by CSR index.

Return the content of the sPMPADDRx Register.

Parameters

- **csr_idx** – [in] sPMP region CSR index(0-15)

Returns

sPMPADDRx Register value

__STATIC_INLINE void __set_sPMPADDRx (uint32_t csr_idx, rv_csr_t smpaddr)

Set sPMPADDRx by CSR index.

Write the given value to the sPMPADDRx Register.

Parameters

- **csr_idx** – [in] sPMP region CSR index(0-15)

- **spmpaddr** – [in] sPMPADDRx Register value to set

```
__STATIC_INLINE void __set_sPMPENTRYx (uint32_t entry_idx,  
const spmp_config *spmp_cfg)
```

Set sPMP entry by entry idx.

Write the given value to the sPMPxCFG Register and sPMPADDRx.

Remark

- If the size of memory region is 2^{12} (4KB) range, spmp_cfg->order makes 12, and the like.
- Suppose the size of memory region is 2^X bytes range, if $X \geq 3$, the NA4 mode is not selectable, NAPOT is selected.
- TOR of A field in sPMP configuration register is not considered here.

Parameters

- **entry_idx** – [in] sPMP entry index(0-15)
- **spmp_cfg** – [in] structure of L,U,X,W,R field of sPMP configuration register, memory region base address and size of memory region as power of 2

```
__STATIC_INLINE int __get_sPMPENTRYx (unsigned int entry_idx, spmp_config *spmp_cfg)
```

Get sPMP entry by entry idx.

Write the given value to the sPMPxCFG Register and sPMPADDRx.

Remark

- If the size of memory region is 2^{12} (4KB) range, spmp_cfg->order makes 12, and the like.
- TOR of A field in PMP configuration register is not considered here.

Parameters

- **entry_idx** – [in] sPMP entry index(0-15)
- **spmp_cfg** – [out] structure of L, U, X, W, R, A field of sPMP configuration register, memory region base address and size of memory region as power of 2

Returns

-1 failure, else 0 success

```
__STATIC_INLINE void __set_SMPUSWITCHx (uint64_t val)
```

Set SMPU each entry's on/off status.

Write the given value to the SMPUSWITCHx Register.

Remark

- Each bit of this register holds on/off status of the corresponding SMPU entry respectively.
- An SMPU entry is activated only when both corresponding bits in smpuswitch and A field of smpuicfg are set. (i.e., smpuswitch[i] & smpu[i]cfg.A).

Parameters

val – [in] activate each entry(max to 64) or not

__STATIC_INLINE uint64_t __get_SMPUSWITCHx (void)

Get SMPU each entry's on/off status.

Get the value of the SMPUSWITCHx Register.

Remark

- Each bit of this register holds on/off status of the corresponding SMPU entry respectively.
- An SMPU entry is activated only when both corresponding bits in smpuswitch and A field of smpuicfg are set. (i.e., smpuswitch[i] & smpu[i]cfg.A).

```
struct smp_config
    #include <core_feature_smp.h>
```

2.5.18 PMA Functions

```
__STATIC_INLINE long PMA_SetRegion (unsigned long entry_idx, pma_config *pma_cfg)
```

```
__STATIC_INLINE long PMA_GetRegion (unsigned long entry_idx, pma_config *pma_cfg)
```

```
__STATIC_INLINE long PMA_SetRegion_S (unsigned long entry_idx, pma_config *pma_cfg)
```

```
__STATIC_INLINE long PMA_GetRegion_S (unsigned long entry_idx, pma_config *pma_cfg)
```

```
__STATIC_FORCEINLINE void PMA_EnableHwDevRegion (unsigned long entry_idx)
```

```
__STATIC_FORCEINLINE void PMA_DisableHwDevRegion (unsigned long entry_idx)
```

```
__STATIC_FORCEINLINE void PMA_EnableHwNCRegion (unsigned long entry_idx)
```

```
__STATIC_FORCEINLINE void PMA_DisableHwNCRegion (unsigned long entry_idx)
```

```
__STATIC_FORCEINLINE void PMA_EnableHwCAREgion (unsigned long entry_idx)
```

```
__STATIC_FORCEINLINE void PMA_DisableHwCARegion (unsigned long entry_idx)
```

```
PMA_REGION_TYPE_SECSHARE (1UL << 3)
```

```
PMA_REGION_TYPE_NC (1UL << 2)
```

```
PMA_REGION_TYPE_DEV (1UL << 1)
```

```
PMA_REGION_TYPE_CA (0)
```

```
PMA_REGION_ENA (1UL << 0)
```

```
PMA_REGION_DIS (0)
```

```
struct pma_config
```

group **PMA Functions**

Functions that set/disable/enable different attribute type(Device/Non-Cacheable/Cacheable) memory regions, or get region info.

Nuclei provide Physical Memory Attribute(PMA) to define the attribute of memory.PMA will affect CPU access memory behavior.

PMA are split into three attributes:

- Device(abbreviated as 'DEV') Attribute
- Non-Cacheable(abbreviated as 'NC') Attribute
- Cacheable(abbreviated as 'CA') Attribute And correspondingly, the whole memory region are split into three regions: DEV Region/NC Region/CA Region

Hardware provide some software CSR to set the pma by mattri(n)_base/mattri(n)_mask/sattri(n)_base/sattri(n)_mask

Hardware defined PMA regions(up to 8 DEV/NC/CA regions) can be disable or enable by CSR mmacro_dev_en/mmacro_noc_en/mmacro_ca_en

Defines

```
PMA_REGION_TYPE_SECSHARE (1UL << 3)
```

Set this region shareable between secure world and non-secure world, or else default is invalid.

```
PMA_REGION_TYPE_NC (1UL << 2)
```

Set this region Non-Cacheable, or else default is invalid.

```
PMA_REGION_TYPE_DEV (1UL << 1)
```

Set this region Device, or else default is invalid.

PMA_REGION_TYPE_CA (0)

Set this region Cacheable, which is default.

PMA_REGION_ENA (1UL << 0)

Enable this region, then the region type will take effect.

PMA_REGION_DIS (0)

Disable this region.

Functions

__STATIC_INLINE long PMA_SetRegion (unsigned long entry_idx, pma_config *pma_cfg)

Configure one region in machine mode.

Set the region(0-n) info of base address/region type/region size/enable status

Remark

- The entry_idx(0-n) depends on number of paired mattri(n)_mask and mattri(n)_base, refer to Nuclei ISA specifications
- Not all the entry_idx(0-n) could set to all the types freely, refer to Nuclei ISA specifications
- The mattri(n)_mask must be written first, before mattri(n)_base, which the api takes care of.
- The higher bits of mattri(n)_mask should be continuously 1, the remaining lower bits should be all 0 and the number (N) of 0 means the size of this region(2^N bytes)
- Region granularity is 4KB, so the low 12-bits of mattri(n)_mask must be 0, which the api takes care of
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

Parameters

- **entry_idx** – [in] Index(0-n) of paired mattri(n)_mask and mattri(n)_base
- **pma_cfg** – [in] Region info to configure

Returns

-1 failure, else 0 success

__STATIC_INLINE long PMA_GetRegion (unsigned long entry_idx, pma_config *pma_cfg)

Get the region info in machine mode.

Read the region(0-n) info of base address/region type/region size/enable status

Remark

- `entry_idx(0-n)` depends on number of paired `mattri(n)_mask` and `mattri(n)_base`, refer to Nuclei ISA specifications
- Not all the `entry_idx(0-n)` could set to all the types freely, refer to Nuclei ISA specifications

Parameters

- **`entry_idx`** – [in] Index(0-n) of paired `mattri(n)_mask` and `mattri(n)_base`
- **`pma_cfg`** – [out] Region info read

Returns

0 if success, else -1

```
__STATIC_INLINE long PMA_SetRegion_S (unsigned long entry_idx, pma_config *pma_cfg)
```

Configure one region for Secure S-Mode world to share with Non-Secure S-Mode world.

Set the region(0-7) info of base address/region size/enable status

Remark

- `sattri(n)_mask` must be written first, before `sattri(n)_base`, which the api takes care of
- The higher bits of `sattri(n)_mask` should be continuously 1, the remaining lower bits should be all 0 and the number (N) of 0 means the size of this region(2^N bytes)
- Region granularity is 4KB, so the low 12-bits of `sattri(n)_mask` must be 0, which the api takes care of
- Unlike `mattri(n)_base`, there's no DEV Region/NC Region/CA Region type

Parameters

- **`entry_idx`** – [in] Index(0-7) of paired `sattri(n)_mask` and `sattri(n)_base`
- **`pma_cfg`** – [in] Region info to configure

Returns

0 if success, else -1

```
__STATIC_INLINE long PMA_GetRegion_S (unsigned long entry_idx, pma_config *pma_cfg)
```

Get the region info of Secure S-Mode world sharing with Non-Secure S-Mode world.

Read the region(0-7) info of base address/region size/enable status

Remark

- Unlike `mattri(n)_base`, there's no DEV Region/NC Region/CA Region type

Parameters

- **`entry_idx`** – [in] Index(0-7) of paired `sattri(n)_mask` and `sattri(n)_base`
- **`pma_cfg`** – [out] Region info read

Returns

0 if success, else -1

__STATIC_FORCEINLINE void PMA_EnableHwDevRegion (unsigned long entry_idx)

Enable hardware defined Device regions.

Enable Device region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_DisableHwDevRegion

Parameters

entry_idx – [in] Index(0-7)

__STATIC_FORCEINLINE void PMA_DisableHwDevRegion (unsigned long entry_idx)

Disable hardware defined Device regions.

Disable Device region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_EnableHwDevRegion

Parameters**entry_idx** – [in] Index(0-7)**__STATIC_FORCEINLINE void PMA_EnableHwNCRegion (unsigned long entry_idx)**

Enable hardware defined Non-Cacheable regions.

Enable Non-Cacheable region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_DisableHwNCRegion

Parameters**entry_idx** – [in] Index(0-7)**__STATIC_FORCEINLINE void PMA_DisableHwNCRegion (unsigned long entry_idx)**

Disable hardware defined Non-Cacheable regions.

Disable Non-Cacheable region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_EnableHwNCRegion

Parameters**entry_idx** – [in] Index(0-7)**__STATIC_FORCEINLINE void PMA_EnableHwCARegion (unsigned long entry_idx)**

Enable hardware defined Cacheable regions.

Enable Cacheable region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_DisableHwCARegion

Parameters**entry_idx** – [in] Index(0-7)**__STATIC_FORCEINLINE void PMA_DisableHwCARegion (unsigned long entry_idx)**

Disable hardware defined Cacheable regions.

Disable Cacheable region by corresponding index

Remark

- This function can be called in M-Mode only.
- When there is no entry_idx, this field is tied to 0
- For each region entry's address/size/attribute detail refers to RTL Configuration Stage
- The regions can be overlapped as the priority: Non-Cacheable > Cacheable > Device, but especially be careful not to overlap software's instruction/data sections by Device, or overlap Device(like uart) memory by Cacheable

See also

- PMA_EnableHwCARegion

Parameters**entry_idx** – [in] Index(0-7)

```
struct pma_config  
    #include <core_feature_pma.h>
```

2.5.19 Cache Functions

General

enum **CCM_OP_FINFO_Type***Values:*enumerator **CCM_OP_SUCCESS**enumerator **CCM_OP_EXCEED_ERR**enumerator **CCM_OP_PERM_CHECK_ERR**enumerator **CCM_OP_REFILL_BUS_ERR**enumerator **CCM_OP_ECC_ERR**enum **CCM_CMD_Type***Values:*enumerator **CCM_DC_INVALID**enumerator **CCM_DC_WB**enumerator **CCM_DC_WBINVAL**enumerator **CCM_DC_LOCK**enumerator **CCM_DC_UNLOCK**enumerator **CCM_DC_WBINVAL_ALL**enumerator **CCM_DC_WB_ALL**enumerator **CCM_DC_INVALID_ALL**enumerator **CCM_IC_INVALID**

enumerator **CCM_IC_LOCK**

enumerator **CCM_IC_UNLOCK**

enumerator **CCM_IC_INVALID_ALL**

enumerator **CCM_CC_LOCK**

enumerator **CCM_CC_UNLOCK**

__STATIC_FORCEINLINE void EnableSUCCM (void)

__STATIC_FORCEINLINE void DisableSUCCM (void)

__STATIC_FORCEINLINE void FlushPipeCCM (void)

CCM_SUEN_SUEN_Msk (0xFFFFFFFFFFFFFFFFFUL)

struct **CacheInfo_Type**

group **Cache Functions**

Functions that configure Instruction and Data Cache.

Nuclei provide Cache Control and Maintenance (CCM) for software to control and maintain the internal L1 I/D Cache and Cluster Cache of the RISC-V Core, software can manage the cache flexibly to meet the actual application scenarios.

The CCM operations have 3 types: by single address, by all and flush pipeline. The CCM operations are done via CSR registers, M/S/U mode has its own CSR registers to do CCM operations. By default, CCM operations are not allowed in S/U mode, you can execute EnableSUCCM in M-Mode to enable it.

- API names started with M<operations>, such as MInvalICacheLine must be called in M-Mode only.
- API names started with S<operations>, such as SInvalICacheLine should be called in S-Mode.
- API names started with U<operations>, such as UInvalICacheLine should be called in U-Mode.

Defines

CCM_SUEN_SUEN_Msk (0xFFFFFFFFFFFFFFFFFUL)

CSR CCM_SUEN: SUEN Mask.

Enums

enum **CCM_OP_FINFO_Type**

Cache CCM Operation Fail Info.

Values:

enumerator **CCM_OP_SUCCESS**

Lock Succeed.

enumerator **CCM_OP_EXCEED_ERR**

Exceed the the number of lockable ways(N-Way I/D-Cache, lockable is N-1)

enumerator **CCM_OP_PERM_CHECK_ERR**

PMP/sPMP/Page-Table X(I-Cache)/R(D-Cache) permission check failed, or belong to Device/Non-Cacheable address range.

enumerator **CCM_OP_REFILL_BUS_ERR**

Refill has Bus Error.

enumerator **CCM_OP_ECC_ERR**

Deprecated, ECC Error, this error code is removed in later Nuclei CCM RTL design, please don't use it.

enum **CCM_CMD_Type**

Cache CCM Command Types.

Values:

enumerator **CCM_DC_INVALID**

Unlock and invalidate D-Cache line and Cluster Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_DC_WB**

Flush the specific D-Cache line and Cluster Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_DC_WBINVAL**

Unlock, flush and invalidate the specific D-Cache line and Cluster Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_DC_LOCK**

Lock the specific D-Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_DC_UNLOCK**

Unlock the specific D-Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_DC_WBINVAL_ALL**

Unlock and flush and invalidate all the valid and dirty D-Cache lines.

enumerator **CCM_DC_WB_ALL**

Flush all the valid and dirty D-Cache lines.

enumerator **CCM_DC_INVALID_ALL**

Unlock and invalidate all the D-Cache lines.

enumerator **CCM_IC_INVALID**

Unlock and invalidate I-Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_IC_LOCK**

Lock the specific I-Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_IC_UNLOCK**

Unlock the specific I-Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_IC_INVALID_ALL**

Unlock and invalidate all the I-Cache lines.

enumerator **CCM_CC_LOCK**

Lock the specific Cluster Cache line specified by CSR CCM_XBEGINADDR.

enumerator **CCM_CC_UNLOCK**

Unlock the specific Cluster Cache line specified by CSR CCM_XBEGINADDR.

Functions

__STATIC_FORCEINLINE void EnableSUCCM (void)

Enable CCM operation in Supervisor/User Mode.

This function enable CCM operation in Supervisor/User Mode. If enabled, CCM operations in supervisor/user mode will be allowed. Besides CCM registers, CC_sCMD and CC_uCMD registers which belong to SMPCC module are also allowed.

Remark

- This function can be called in M-Mode only.

See also

- DisableSUCCM

__STATIC_FORCEINLINE void DisableSUCCM (void)

Disable CCM operation in Supervisor/User Mode.

This function disable CCM operation in Supervisor/User Mode. If not enabled, CCM operations in supervisor/user mode will trigger a *illegal instruction* exception, access to CC_sCMD and CC_uCMD register is also forbidden.

Remark

- This function can be called in M-Mode only.

See also

- EnableSUCCM

__STATIC_FORCEINLINE void FlushPipeCCM (void)

Flush pipeline after CCM operation.

This function is used to flush pipeline after CCM operations on Cache, it will ensure latest instructions or data can be seen by pipeline.

Remark

- This function can be called in M/S/U-Mode only.

struct **CacheInfo_Type**

#include <core_feature_cache.h> Cache Information Type.

I-Cache Functions

__STATIC_INLINE int32_t ICachePresent (void)

__STATIC_FORCEINLINE void EnableICache (void)

__STATIC_FORCEINLINE void DisableICache (void)

__STATIC_FORCEINLINE void EnableICacheECC (void)

__STATIC_FORCEINLINE void DisableICacheECC (void)

```
__STATIC_INLINE int32_t GetICacheInfo (CacheInfo_Type *info)

__STATIC_INLINE void MInvalICacheLine (unsigned long addr)

__STATIC_INLINE void MInvalICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void SInvalICacheLine (unsigned long addr)

__STATIC_INLINE void SInvalICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UInvalICacheLine (unsigned long addr)

__STATIC_INLINE void UInvalICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MInvalICacheCCacheLine (unsigned long addr)

__STATIC_INLINE void MInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void SInvalICacheCCacheLine (unsigned long addr)

__STATIC_INLINE void SInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UInvalICacheCCacheLine (unsigned long addr)

__STATIC_INLINE void UInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long MLockICacheLine (unsigned long addr)

__STATIC_INLINE unsigned long MLockICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long SLockICacheLine (unsigned long addr)

__STATIC_INLINE unsigned long SLockICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long ULockICacheLine (unsigned long addr)

__STATIC_INLINE unsigned long ULockICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MUnlockICacheLine (unsigned long addr)

__STATIC_INLINE void MUnlockICacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void SUnlockICacheLine (unsigned long addr)

__STATIC_INLINE void SUnlockICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UUnlockICacheLine (unsigned long addr)

__STATIC_INLINE void UUnlockICacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MInvalICache (void)

__STATIC_INLINE void SInvalICache (void)

__STATIC_INLINE void UInvalICache (void)

__STATIC_INLINE void MInvalCCache (void)

__STATIC_INLINE void SInvalCCache (void)

__STATIC_INLINE void UInvalCCache (void)

__STATIC_INLINE void MInvalICacheCCache (void)

__STATIC_INLINE void SInvalICacheCCache (void)

__STATIC_INLINE void UInvalICacheCCache (void)
```

group **I-Cache Functions**

Functions that configure Instruction Cache.

Functions

```
__STATIC_INLINE int32_t ICachePresent (void)
```

Check ICache Unit Present or Not.

This function check icache unit present or not via mcfg_info csr

Remark

- This function might not work for some old nuclei processors
- Please make sure the version of your nuclei processor contain ICACHE bit in mcfg_info

Returns

1 if present otherwise 0

__STATIC_FORCEINLINE void EnableICache (void)

Enable ICache.

This function enable I-Cache

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control I Cache enable.

See also

- DisableICache

__STATIC_FORCEINLINE void DisableICache (void)

Disable ICache.

This function Disable I-Cache

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control I Cache enable.

See also

- EnableICache

__STATIC_FORCEINLINE void EnableICacheECC (void)

Enable ICache ECC.

This function enable I-Cache ECC

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control I Cache ECC enable.

See also

- `DisableICacheECC`

`__STATIC_FORCEINLINE void DisableICacheECC (void)`

Disable ICache ECC.

This function disable I-Cache ECC

Remark

- This function can be called in M-Mode only.
- This [*CSR_MCACHE_CTL*](#) (page 107) register control I Cache ECC enable.

See also

- `EnableICacheECC`

`__STATIC_INLINE int32_t GetICacheInfo (CacheInfo_Type *info)`

Get I-Cache Information.

This function get I-Cache Information

Remark

- This function can be called in M-Mode only.
- You can use this function in combination with cache lines operations

See also

- `GetDCacheInfo`

`__STATIC_INLINE void MInvalidCacheLine (unsigned long addr)`

Invalidate one I-Cache line specified by address in M-Mode.

This function unlock and invalidate one I-Cache line specified by the address. Command `CCM_IC_INVALID` is written to CSR [*CSR_CCM_MCOMMAND*](#) (page 112).**Remark**

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void MInvalidCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache lines specified by address in M-Mode.

This function unlock and invalidate several I-Cache lines specified by the address and line count. Command CCM_IC_INVALID is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void SInvalidCacheLine (unsigned long addr)

Invalidate one I-Cache line specified by address in S-Mode.

This function unlock and invalidate one I-Cache line specified by the address. Command CCM_IC_INVALID is written to CSR [CSR_CCM_SCOMMAND](#) (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void SInvalidCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache lines specified by address in S-Mode.

This function unlock and invalidate several I-Cache lines specified by the address and line count. Command CCM_IC_INVALID is written to CSR [CSR_CCM_SCOMMAND](#) (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void UInvalICacheLine (unsigned long addr)

Invalidate one I-Cache line specified by address in U-Mode.

This function unlock and invalidate one I-Cache line specified by the address. Command CCM_IC_INVALID is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void UInvalICacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache lines specified by address in U-Mode.

This function unlock and invalidate several I-Cache lines specified by the address and line count. Command CCM_IC_INVALID is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void MInvalICacheCCacheLine (unsigned long addr)

Invalidate one I-Cache and Cluster Cache line specified by address in M-Mode.

This function unlock and invalidate one I-Cache line and corresponding Cluster Cache line specified by the address. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void MInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache and Cluster Cache lines specified by address in M-Mode.

This function unlock and invalidate several I-Cache lines and corresponding Cluster Cache lines specified by the address and line count. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void SInvalICacheCCacheLine (unsigned long addr)

Invalidate one I-Cache and Cluster Cache line specified by address in S-Mode.

This function unlock and invalidate one I-Cache line and corresponding Cluster Cache line specified by the address. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void SInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache and Cluster Cache lines specified by address in S-Mode.

This function unlock and invalidate several I-Cache lines and corresponding Cluster Cache lines specified by the address and line count. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void UInvalICacheCCacheLine (unsigned long addr)

Invalidate one I-Cache and Cluster Cache line specified by address in U-Mode.

This function unlock and invalidate one I-Cache line and corresponding Cluster Cache line specified by the address. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void UInvalICacheCCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several I-Cache and Cluster Cache lines specified by address in U-Mode.

This function unlock and invalidate several I-Cache lines and corresponding Cluster Cache lines specified by the address and line count. Commands CCM_IC_INVALID and CCM_DC_INVALID are written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE unsigned long MLockICacheLine (unsigned long addr)

Lock one I-Cache line specified by address in M-Mode.

This function lock one I-Cache line specified by the address. Command CCM_IC_LOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long MLockICacheLines (unsigned long addr, unsigned long cnt)

Lock several I-Cache lines specified by address in M-Mode.

This function lock several I-Cache lines specified by the address and line count. Command CCM_IC_LOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockICacheLine (unsigned long addr)

Lock one I-Cache line specified by address in S-Mode.

This function lock one I-Cache line specified by the address. Command CCM_IC_LOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockICacheLines (unsigned long addr, unsigned long cnt)

Lock several I-Cache lines specified by address in S-Mode.

This function lock several I-Cache lines specified by the address and line count. Command CCM_IC_LOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockICacheLine (unsigned long addr)

Lock one I-Cache line specified by address in U-Mode.

This function lock one I-Cache line specified by the address. Command CCM_IC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockICacheLines (unsigned long addr, unsigned long cnt)

Lock several I-Cache lines specified by address in U-Mode.

This function lock several I-Cache lines specified by the address and line count. Command CCM_IC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE void MUnlockICacheLine (unsigned long addr)

Unlock one I-Cache line specified by address in M-Mode.

This function unlock one I-Cache line specified by the address. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void MUnlockICacheLines (unsigned long addr, unsigned long cnt)

Unlock several I-Cache lines specified by address in M-Mode.

This function unlock several I-Cache lines specified by the address and line count. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void SUnlockICacheLine (unsigned long addr)

Unlock one I-Cache line specified by address in S-Mode.

This function unlock one I-Cache line specified by the address. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void SUnlockICacheLines (unsigned long addr, unsigned long cnt)

Unlock several I-Cache lines specified by address in S-Mode.

This function unlock several I-Cache lines specified by the address and line count. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void UUnlockICacheLine (unsigned long addr)

Unlock one I-Cache line specified by address in U-Mode.

This function unlock one I-Cache line specified by the address. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void UUnlockICacheLines (unsigned long addr, unsigned long cnt)

Unlock several I-Cache lines specified by address in U-Mode.

This function unlock several I-Cache lines specified by the address and line count. Command CCM_IC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void MInvalidCache (void)

Invalidate all I-Cache lines in M-Mode.

This function invalidate all I-Cache lines. Command CCM_IC_INVALID_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void SInvalidCache (void)

Invalidate all I-Cache lines in S-Mode.

This function invalidate all I-Cache lines. Command CCM_IC_INVALID_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void UInvalidCache (void)

Invalidate all I-Cache lines in U-Mode.

This function invalidate all I-Cache lines. Command CCM_IC_INVALID_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void MInvalCCache (void)

Invalidate all Cluster Cache in M-Mode.

This function invalidate all Cluster Cache.

Remark

This function must be executed in M-Mode only.

__STATIC_INLINE void SInvalCCache (void)

Invalidate all Cluster Cache in S-Mode.

This function invalidate all Cluster Cache.

Remark

This function must be executed in M/S-Mode only.

__STATIC_INLINE void UInvalCCache (void)

Invalidate all Cluster Cache in U-Mode.

This function invalidate all Cluster Cache.

Remark

This function must be executed in M/S/U-Mode only.

__STATIC_INLINE void MInvalICacheCCache (void)

Invalidate all I-Cache and Cluster Cache in M-Mode.

This function unlock and invalidate all I-Cache and Cluster Cache. Command CCM_IC_INVALID_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

__STATIC_INLINE void SInvalICacheCCache (void)

Invalidate all I-Cache and Cluster Cache in S-Mode.

This function unlock and invalidate all I-Cache and Cluster Cache. Command CCM_IC_INVALID_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

```
__STATIC_INLINE void UInvalICacheCCache (void)
```

Invalidate all I-Cache and Cluster Cache in U-Mode.

This function unlock and invalidate all I-Cache and Cluster Cache. Command CCM_IC_INVALID_ALL is written to CSR [CSR_CCM_UCOMMAND](#) (page 112).

Remark
This function must be executed in M/S/U-Mode only.

D-Cache Functions

```
__STATIC_INLINE int32_t DCachePresent (void)
```

```
__STATIC_FORCEINLINE void EnabledDCache (void)
```

```
__STATIC_FORCEINLINE void DisabledDCache (void)
```

```
__STATIC_FORCEINLINE void EnabledDCacheECC (void)
```

```
__STATIC_FORCEINLINE void DisabledDCacheECC (void)
```

```
__STATIC_INLINE int32_t GetDCacheInfo (CacheInfo_Type *info)
```

```
__STATIC_INLINE void MInvalDCacheLine (unsigned long addr)
```

```
__STATIC_INLINE void MInvalDCacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void SInvalDCacheLine (unsigned long addr)
```

```
__STATIC_INLINE void SInvalDCacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void UInvalDCacheLine (unsigned long addr)
```

```
__STATIC_INLINE void UInvalDCacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void MFlushDCacheLine (unsigned long addr)
```

```
__STATIC_INLINE void MFlushDCacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void SFlushDCacheLine (unsigned long addr)
```

```
__STATIC_INLINE void SFlushDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UFlushDCacheLine (unsigned long addr)

__STATIC_INLINE void UFlushDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MFlushInvalDCacheLine (unsigned long addr)

__STATIC_INLINE void MFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void SFlushInvalDCacheLine (unsigned long addr)

__STATIC_INLINE void SFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UFlushInvalDCacheLine (unsigned long addr)

__STATIC_INLINE void UFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long MLockDCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long MLockDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long SLockDCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long SLockDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long ULockDCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long ULockDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MUnlockDCacheLine (unsigned long addr)

__STATIC_INLINE void MUnlockDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void SUnlockDCacheLine (unsigned long addr)

__STATIC_INLINE void SUnlockDCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UUnlockDCacheLine (unsigned long addr)

__STATIC_INLINE void UUnlockDCacheLines (unsigned long addr, unsigned long cnt)
```

```
__STATIC_INLINE void MInvalDCache (void)

__STATIC_INLINE void SInvalDCache (void)

__STATIC_INLINE void UInvalDCache (void)

__STATIC_INLINE void MFlushDCache (void)

__STATIC_INLINE void SFlushDCache (void)

__STATIC_INLINE void UFlushDCache (void)

__STATIC_INLINE void MFlushInvalDCache (void)

__STATIC_INLINE void SFlushInvalDCache (void)

__STATIC_INLINE void UFlushInvalDCache (void)

__STATIC_INLINE int32_t MFlushCCache (void)

__STATIC_INLINE int32_t SFlushCCache (void)

__STATIC_INLINE int32_t UFlushCCache (void)

__STATIC_INLINE int32_t MFlushInvalCCache (void)

__STATIC_INLINE int32_t SFlushInvalCCache (void)

__STATIC_INLINE int32_t UFlushInvalCCache (void)

__STATIC_INLINE void MInvalDCacheCCache (void)

__STATIC_INLINE void SInvalDCacheCCache (void)

__STATIC_INLINE void UInvalDCacheCCache (void)

__STATIC_INLINE int32_t MFlushDCacheCCache (void)

__STATIC_INLINE int32_t SFlushDCacheCCache (void)

__STATIC_INLINE int32_t UFlushDCacheCCache (void)
```

```

__STATIC_INLINE int32_t MFlushInvalDCacheCCache (void)

__STATIC_INLINE int32_t SFlushInvalDCacheCCache (void)

__STATIC_INLINE int32_t UFlushInvalDCacheCCache (void)

MInvalDCacheCCacheLine(addr) MInvalDCacheLine(addr)
MInvalDCacheCCacheLines(addr, cnt) MInvalDCacheLines(addr, cnt)
SInvalDCacheCCacheLine(addr) SInvalDCacheLine(addr)
SInvalDCacheCCacheLines(addr, cnt) SInvalDCacheLines(addr, cnt)
UInvalDCacheCCacheLine(addr) UInvalDCacheLine(addr)
UInvalDCacheCCacheLines(addr, cnt) UInvalDCacheLines(addr, cnt)
MFlushDCacheCCacheLine(addr) MFlushDCacheLine(addr)
MFlushDCacheCCacheLines(addr, cnt) MFlushDCacheLines(addr, cnt)
SFlushDCacheCCacheLine(addr) SFlushDCacheLine(addr)
SFlushDCacheCCacheLines(addr, cnt) SFlushDCacheLines(addr, cnt)
UFlushDCacheCCacheLine(addr) UFlushDCacheLine(addr)
UFlushDCacheCCacheLines(addr, cnt) UFlushDCacheLines(addr, cnt)
MFlushInvalDCacheCCacheLine(addr) MFlushInvalDCacheLine(addr)
MFlushInvalDCacheCCacheLines(addr, cnt) MFlushInvalDCacheLines(addr, cnt)
SFlushInvalDCacheCCacheLine(addr) SFlushInvalDCacheLine(addr)
SFlushInvalDCacheCCacheLines(addr, cnt) SFlushInvalDCacheLines(addr, cnt)
UFlushInvalDCacheCCacheLine(addr) UFlushInvalDCacheLine(addr)
UFlushInvalDCacheCCacheLines(addr, cnt) UFlushInvalDCacheLines(addr, cnt)

```

group **D-Cache Functions**

Functions that configure Data Cache.

Defines

MInvalDCacheCCacheLine(addr) MInvalDCacheLine(addr)

Invalidate one D-Cache and Cluster Cache line specified by address in M-Mode.

This macro is an alias for MInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be invalidated

MInvalDCacheCCacheLines(addr, cnt) MInvalDCacheLines(addr, cnt)

Invalidate several D-Cache and Cluster Cache lines specified by address in M-Mode.

This macro is an alias for MInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

SInvalDCacheCCacheLine(addr) SInvalDCacheLine(addr)

Invalidate one D-Cache and Cluster Cache line specified by address in S-Mode.

This macro is an alias for SInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be invalidated

SInvalDCacheCCacheLines(addr, cnt) SInvalDCacheLines(addr, cnt)

Invalidate several D-Cache and Cluster Cache lines specified by address in S-Mode.

This macro is an alias for SInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

UInvalDCacheCCacheLine(addr) UInvalDCacheLine(addr)

Invalidate one D-Cache and Cluster Cache line specified by address in U-Mode.

This macro is an alias for UInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be invalidated

UInvalDCacheCCacheLines(addr, cnt) UInvalDCacheLines(addr, cnt)

Invalidate several D-Cache and Cluster Cache lines specified by address in U-Mode.

This macro is an alias for UInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

MFlushDCacheCCacheLine(addr) MFlushDCacheLine(addr)

Flush one D-Cache and Cluster Cache line specified by address in M-Mode.

This macro is an alias for MFlushDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed

MFlushDCacheCCacheLines(addr, cnt) MFlushDCacheLines(addr, cnt)

Flush several D-Cache and Cluster Cache lines specified by address in M-Mode.

This macro is an alias for MFlushDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

SFlushDCacheCCacheLine(addr) SFlushDCacheLine(addr)

Flush one D-Cache and Cluster Cache line specified by address in S-Mode.

This macro is an alias for SFlushDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed

SFlushDCacheCCacheLines(addr, cnt) SFlushDCacheLines(addr, cnt)

Flush several D-Cache and Cluster Cache lines specified by address in S-Mode.

This macro is an alias for SFlushDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

UFlushDCacheCCacheLine(addr) UFlushDCacheLine(addr)

Flush one D-Cache and Cluster Cache line specified by address in U-Mode.

This macro is an alias for UFlushDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed

UFlushDCacheCCacheLines(addr, cnt) UFlushDCacheLines(addr, cnt)

Flush several D-Cache and Cluster Cache lines specified by address in U-Mode.

This macro is an alias for UFlushDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

MFlushInvalDCacheCCacheLine(addr) MFlushInvalDCacheLine(addr)

Flush and invalidate one D-Cache and Cluster Cache line specified by address in M-Mode.

This macro is an alias for MFlushInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated

MFlushInvalDCacheCCacheLines(addr, cnt) MFlushInvalDCacheLines(addr, cnt)

Flush and invalidate several D-Cache and Cluster Cache lines specified by address in M-Mode.

This macro is an alias for MFlushInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

SFlushInvalDCacheCCacheLine(addr) SFlushInvalDCacheLine(addr)

Flush and invalidate one D-Cache and Cluster Cache line specified by address in S-Mode.

This macro is an alias for SFlushInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated

SFlushInvalDCacheCCacheLines(addr, cnt) SFlushInvalDCacheLines(addr, cnt)

Flush and invalidate several D-Cache and Cluster Cache lines specified by address in S-Mode.

This macro is an alias for SFlushInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

UFlushInvalDCacheCCacheLine(addr) UFlushInvalDCacheLine(addr)

Flush and invalidate one D-Cache and Cluster Cache line specified by address in U-Mode.

This macro is an alias for UFlushInvalDCacheLine. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated

UFlushInvalDCacheCCacheLines(addr, cnt) UFlushInvalDCacheLines(addr, cnt)

Flush and invalidate several D-Cache and Cluster Cache lines specified by address in U-Mode.

This macro is an alias for UFlushInvalDCacheLines. D-Cache CCM operations are also effective for Cluster Cache.

Remark

This macro must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

Functions

__STATIC_INLINE int32_t DCachePresent (void)

Check DCache Unit Present or Not.

This function check dcache unit present or not via mcfg_info csr

Remark

- This function might not work for some old nuclei processors

- Please make sure the version of your nuclei processor contain DCACHE bit in mcfg_info

Returns

1 if present otherwise 0

__STATIC_FORCEINLINE void EnabledDCache (void)

Enable DCache.

This function enable D-Cache

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control D Cache enable.

See also

- DisableDCache

__STATIC_FORCEINLINE void DisabledDCache (void)

Disable DCache.

This function Disable D-Cache

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control D Cache enable.

See also

- EnabledDCache

__STATIC_FORCEINLINE void EnabledDCacheECC (void)

Enable DCache ECC.

This function enable D-Cache ECC

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control D Cache ECC enable.

See also

- DisableDCacheECC

__STATIC_FORCEINLINE void DisableDCacheECC (void)

Disable DCache ECC.

This function disable D-Cache ECC

Remark

- This function can be called in M-Mode only.
- This [CSR_MCACHE_CTL](#) (page 107) register control D Cache ECC enable.

See also

- EnableDCacheECC

__STATIC_INLINE int32_t GetDCacheInfo (CacheInfo_Type *info)

Get D-Cache Information.

This function get D-Cache Information

Remark

- This function can be called in M-Mode only.
- You can use this function in combination with cache lines operations

See also

- GetICacheInfo

__STATIC_INLINE void MInvalidDCacheLine (unsigned long addr)

Invalidate one D-Cache line specified by address in M-Mode.

This function unlock and invalidate one D-Cache line specified by the address. Command CCM_DC_INVALID is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void MInvalDCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several D-Cache lines specified by address in M-Mode.

This function unlock and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_INVALID is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void SInvalDCacheLine (unsigned long addr)

Invalidate one D-Cache line specified by address in S-Mode.

This function unlock and invalidate one D-Cache line specified by the address. Command CCM_DC_INVALID is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void SInvalDCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several D-Cache lines specified by address in S-Mode.

This function unlock and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_INVALID is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void UInvalDCacheLine (unsigned long addr)

Invalidate one D-Cache line specified by address in U-Mode.

This function unlock and invalidate one D-Cache line specified by the address. Command CCM_DC_INVAL is written to CSR [CSR_CCM_UCOMMAND](#) (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void UInvalDCacheLines (unsigned long addr, unsigned long cnt)

Invalidate several D-Cache lines specified by address in U-Mode.

This function unlock and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_INVAL is written to CSR [CSR_CCM_UCOMMAND](#) (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be invalidated
- **cnt** – [in] count of cache lines to be invalidated

__STATIC_INLINE void MFlushDCacheLine (unsigned long addr)

Flush one D-Cache line specified by address in M-Mode.

This function flush one D-Cache line specified by the address. Command CCM_DC_WB is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be flushed

__STATIC_INLINE void MFlushDCacheLines (unsigned long addr, unsigned long cnt)

Flush several D-Cache lines specified by address in M-Mode.

This function flush several D-Cache lines specified by the address and line count. Command CCM_DC_WB is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

__STATIC_INLINE void SFlushDCacheLine (unsigned long addr)

Flush one D-Cache line specified by address in S-Mode.

This function flush one D-Cache line specified by the address. Command CCM_DC_WB is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be flushed

__STATIC_INLINE void SFlushDCacheLines (unsigned long addr, unsigned long cnt)

Flush several D-Cache lines specified by address in S-Mode.

This function flush several D-Cache lines specified by the address and line count. Command CCM_DC_WB is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

__STATIC_INLINE void UFlushDCacheLine (unsigned long addr)

Flush one D-Cache line specified by address in U-Mode.

This function flush one D-Cache line specified by the address. Command CCM_DC_WB is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be flushed

__STATIC_INLINE void UFlushDCacheLines (unsigned long addr, unsigned long cnt)

Flush several D-Cache lines specified by address in U-Mode.

This function flush several D-Cache lines specified by the address and line count. Command CCM_DC_WB is written to CSR [CSR_CCM_UCOMMAND](#) (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed
- **cnt** – [in] count of cache lines to be flushed

__STATIC_INLINE void MFlushInvalDCacheLine (unsigned long addr)

Flush and invalidate one D-Cache line specified by address in M-Mode.

This function flush and invalidate one D-Cache line specified by the address. Command CCM_DC_WBINVAL is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be flushed and invalidated

__STATIC_INLINE void MFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

Flush and invalidate several D-Cache lines specified by address in M-Mode.

This function flush and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_WBINVAL is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

__STATIC_INLINE void SFlushInvalDCacheLine (unsigned long addr)

Flush and invalidate one D-Cache line specified by address in S-Mode.

This function flush and invalidate one D-Cache line specified by the address. Command CCM_DC_WBINVAL is written to CSR [CSR_CCM_SCOMMAND](#) (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be flushed and invalidated

__STATIC_INLINE void SFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

Flush and invalidate several D-Cache lines specified by address in S-Mode.

This function flush and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_WBINVAL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

__STATIC_INLINE void UFlushInvalDCacheLine (unsigned long addr)

Flush and invalidate one D-Cache line specified by address in U-Mode.

This function flush and invalidate one D-Cache line specified by the address. Command CCM_DC_WBINVAL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be flushed and invalidated

__STATIC_INLINE void UFlushInvalDCacheLines (unsigned long addr, unsigned long cnt)

Flush and invalidate several D-Cache lines specified by address in U-Mode.

This function flush and invalidate several D-Cache lines specified by the address and line count. Command CCM_DC_WBINVAL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be flushed and invalidated
- **cnt** – [in] count of cache lines to be flushed and invalidated

__STATIC_INLINE unsigned long MLockDCacheLine (unsigned long addr)

Lock one D-Cache line specified by address in M-Mode.

This function lock one D-Cache line specified by the address. Command CCM_DC_LOCK is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long MLockDCacheLines (unsigned long addr, unsigned long cnt)

Lock several D-Cache lines specified by address in M-Mode.

This function lock several D-Cache lines specified by the address and line count. Command CCM_DC_LOCK is written to CSR [CSR_CCM_MCOMMAND](#) (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockDCacheLine (unsigned long addr)

Lock one D-Cache line specified by address in S-Mode.

This function lock one D-Cache line specified by the address. Command CCM_DC_LOCK is written to CSR [CSR_CCM_SCOMMAND](#) (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockDCacheLines (unsigned long addr, unsigned long cnt)

Lock several D-Cache lines specified by address in S-Mode.

This function lock several D-Cache lines specified by the address and line count. Command CCM_DC_LOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockDCacheLine (unsigned long addr)

Lock one D-Cache line specified by address in U-Mode.

This function lock one D-Cache line specified by the address. Command CCM_DC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockDCacheLines (unsigned long addr, unsigned long cnt)

Lock several D-Cache lines specified by address in U-Mode.

This function lock several D-Cache lines specified by the address and line count. Command CCM_DC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE void MUnlockDCacheLine (unsigned long addr)

Unlock one D-Cache line specified by address in M-Mode.

This function unlock one D-Cache line specified by the address. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void MUnlockDCacheLines (unsigned long addr, unsigned long cnt)

Unlock several D-Cache lines specified by address in M-Mode.

This function unlock several D-Cache lines specified by the address and line count. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void SUnlockDCacheLine (unsigned long addr)

Unlock one D-Cache line specified by address in S-Mode.

This function unlock one D-Cache line specified by the address. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void SUnlockDCacheLines (unsigned long addr, unsigned long cnt)

Unlock several D-Cache lines specified by address in S-Mode.

This function unlock several D-Cache lines specified by the address and line count. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void UUnlockDCacheLine (unsigned long addr)

Unlock one D-Cache line specified by address in U-Mode.

This function unlock one D-Cache line specified by the address. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void UUnlockDCacheLines (unsigned long addr, unsigned long cnt)

Unlock several D-Cache lines specified by address in U-Mode.

This function unlock several D-Cache lines specified by the address and line count. Command CCM_DC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void MInvalidDCache (void)

Invalidate all D-Cache lines in M-Mode.

This function invalidate all D-Cache lines. Command CCM_DC_INVALID_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void SInvalDCache (void)

Invalidate all D-Cache lines in S-Mode.

This function invalidate all D-Cache lines. Command CCM_DC_INVALID_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void UInvalDCache (void)

Invalidate all D-Cache lines in U-Mode.

This function invalidate all D-Cache lines. In U-Mode, this operation will be automatically translated to flush and invalidate operations by hardware. Command CCM_DC_INVALID_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be invalidated

__STATIC_INLINE void MFlushDCache (void)

Flush all D-Cache lines in M-Mode.

This function flush all D-Cache lines. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be flushed

__STATIC_INLINE void SFlushDCache (void)

Flush all D-Cache lines in S-Mode.

This function flush all D-Cache lines. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters**addr** – [in] start address to be flushed**__STATIC_INLINE void UFlushDCache (void)**

Flush all D-Cache lines in U-Mode.

This function flush all D-Cache lines. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters**addr** – [in] start address to be flushed**__STATIC_INLINE void MFlushInvalDCache (void)**

Flush and invalidate all D-Cache lines in M-Mode.

This function flush and invalidate all D-Cache lines. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters**addr** – [in] start address to be flushed and locked**__STATIC_INLINE void SFlushInvalDCache (void)**

Flush and invalidate all D-Cache lines in S-Mode.

This function flush and invalidate all D-Cache lines. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters**addr** – [in] start address to be flushed and locked**__STATIC_INLINE void UFlushInvalDCache (void)**

Flush and invalidate all D-Cache lines in U-Mode.

This function flush and invalidate all D-Cache lines. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be flushed and locked

__STATIC_INLINE int32_t MFlushCCache (void)

Flush all Cluster Cache in M-Mode.

This function flush all Cluster Cache. Command SMPCC_CMD_WB_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t SFlushCCache (void)

Flush all Cluster Cache in S-Mode.

This function flush all Cluster Cache. Command SMPCC_CMD_WB_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M/S-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t UFlushCCache (void)

Flush all Cluster Cache in U-Mode.

This function flush all Cluster Cache. Command SMPCC_CMD_WB_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M/S/U-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t MFlushInvalCCache (void)

Flush and invalidate all Cluster Cache in M-Mode.

This function flush and invalidate all Cluster Cache. Command SMPCC_CMD_WBINVAL_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t SFlushInvalCCache (void)

Flush and invalidate all Cluster Cache in S-Mode.

This function flush and invalidate all Cluster Cache. Command SMPCC_CMD_WBINVAL_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M/S-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t UFlushInvalCCache (void)

Flush and invalidate all Cluster Cache in U-Mode.

This function flush and invalidate all Cluster Cache. Command SMPCC_CMD_WBINVAL_ALL is written to SMPCC CMD register.

Remark

This function must be executed in M/S/U-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE void MInvalDCacheCCache (void)

Invalidate all D-Cache and Cluster Cache in M-Mode.

This function unlock and invalidate all D-Cache and Cluster Cache. Command CCM_DC_INVAL_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

__STATIC_INLINE void SInvalDCacheCCache (void)

Invalidate all D-Cache and Cluster Cache in S-Mode.

This function unlock and invalidate all D-Cache and Cluster Cache. Command CCM_DC_INVALID_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

__STATIC_INLINE void UInvalDCacheCCache (void)

Invalidate all D-Cache and Cluster Cache in U-Mode.

This function unlock and invalidate all D-Cache and Cluster Cache. Command CCM_DC_INVALID_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

__STATIC_INLINE int32_t MFlushDCacheCCache (void)

Flush all D-Cache and Cluster Cache in M-Mode.

This function flush all D-Cache and Cluster Cache. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t SFlushDCacheCCache (void)

Flush all D-Cache and Cluster Cache in S-Mode.

This function flush all D-Cache and Cluster Cache. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t UFlushDCacheCCache (void)

Flush all D-Cache and Cluster Cache in U-Mode.

This function flush all D-Cache and Cluster Cache. Command CCM_DC_WB_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t MFlushInvalDCacheCCache (void)

Flush and invalidate all D-Cache and Cluster Cache in M-Mode.

This function flush and invalidate all D-Cache and Cluster Cache. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t SFlushInvalDCacheCCache (void)

Flush and invalidate all D-Cache and Cluster Cache in S-Mode.

This function flush and invalidate all D-Cache and Cluster Cache. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

__STATIC_INLINE int32_t UFlushInvalDCacheCCache (void)

Flush and invalidate all D-Cache and Cluster Cache in U-Mode.

This function flush and invalidate all D-Cache and Cluster Cache. Command CCM_DC_WBINVAL_ALL is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Returns

Operation result, see enum SMPCC_CMD_RESULT

Cluster-Cache Functions

```
__STATIC_INLINE unsigned long MLockCCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long MLockCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long SLockCCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long SLockCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE unsigned long ULockCCacheLine (unsigned long addr)

__STATIC_INLINE unsigned long ULockCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void MUnlockCCacheLine (unsigned long addr)

__STATIC_INLINE void MUnlockCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void SUnlockCCacheLine (unsigned long addr)

__STATIC_INLINE void SUnlockCCacheLines (unsigned long addr, unsigned long cnt)

__STATIC_INLINE void UUnlockCCacheLine (unsigned long addr)

__STATIC_INLINE void UUnlockCCacheLines (unsigned long addr, unsigned long cnt)

SMPCC_CMD_CTRL_SUP_EN_Pos 9U

SMPCC_CMD_CTRL_SUP_EN_Msk (0x1UL << SMPCC_CMD_CTRL_SUP_EN_Pos)

SMPCC_CMD_CTRL_SUP_EN_ENABLE 1U

SMPCC_CMD_CTRL_SUP_EN_DISABLE 0U

SMPCC_CMD_CTRL_USE_EN_Pos 10U

SMPCC_CMD_CTRL_USE_EN_Msk (0x1UL << SMPCC_CMD_CTRL_USE_EN_Pos)

SMPCC_CMD_CTRL_USE_EN_ENABLE 1U

SMPCC_CMD_CTRL_USE_EN_DISABLE 0U
```

SMPCC_CMD_xCMD_CMD_Pos 0U

SMPCC_CMD_xCMD_CMD_Msk (0x1FUL << SMPCC_CMD_xCMD_CMD_Pos)

SMPCC_CMD_xCMD_CMD_WB_ALL 0x7U

SMPCC_CMD_xCMD_CMD_WBINVAL_ALL 0x6U

SMPCC_CMD_xCMD_STATUS_Pos 23U

SMPCC_CMD_xCMD_STATUS_Msk (0x7UL << SMPCC_CMD_xCMD_STATUS_Pos)

SMPCC_CMD_xCMD_RESULT_Pos 26U

SMPCC_CMD_xCMD_RESULT_Msk (0x1FUL << SMPCC_CMD_xCMD_RESULT_Pos)

SMPCC_CMD_xCMD_RESULT_SUCCESS 0x0U

SMPCC_CMD_xCMD_RESULT_ENTRY_EXCEED_LIMIT 0x1U

SMPCC_CMD_xCMD_RESULT_REFILL_BUS_ERROR 0x3U

SMPCC_CMD_xCMD_RESULT_ECC_ERROR 0x4U

SMPCC_CMD_xCMD_RESULT_CPBACK_BUS_ERROR 0x5U

SMPCC_CMD_xCMD_COMPLETE_Pos 31U

SMPCC_CMD_xCMD_COMPLETE_Msk (0x1UL << SMPCC_CMD_xCMD_COMPLETE_Pos)

SMPCC_CMD_INVALID_ALL_Pos 0U

SMPCC_CMD_INVALID_ALL_Msk (0x1UL << SMPCC_CMD_INVALID_ALL_Pos)

SMPCC_CMD_BASE __SMPCC_BASEADDR

SMPCC_CMD ((SMPCC_CMD_Type *)SMPCC_CMD_BASE)

struct **SMPCC_CMD_Type**

group **Cluster-Cache Functions**

Functions that configure Cluster Cache by CCM.

Unnamed Group

__STATIC_INLINE unsigned long MLockCCacheLine (unsigned long addr)

Lock one Cluster Cache line specified by address in M-Mode.

This function lock one Cluster Cache line specified by the address. Command CCM_CC_LOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long MLockCCacheLines (unsigned long addr, unsigned long cnt)

Lock several Cluster Cache lines specified by address in M-Mode.

This function lock several Cluster Cache lines specified by the address and line count. Command CCM_CC_LOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockCCacheLine (unsigned long addr)

Lock one Cluster Cache line specified by address in S-Mode.

This function lock one Cluster Cache line specified by the address. Command CCM_CC_LOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long SLockCCacheLines (unsigned long addr, unsigned long cnt)

Lock several Cluster Cache lines specified by address in S-Mode.

This function lock several Cluster Cache lines specified by the address and line count. Command CCM_CC_LOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockCCacheLine (unsigned long addr)

Lock one Cluster Cache line specified by address in U-Mode.

This function lock one Cluster Cache line specified by the address. Command CCM_CC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE unsigned long ULockCCacheLines (unsigned long addr, unsigned long cnt)

Lock several Cluster Cache lines specified by address in U-Mode.

This function lock several Cluster Cache lines specified by the address and line count. Command CCM_CC_LOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be locked
- **cnt** – [in] count of cache lines to be locked

Returns

result of CCM lock operation, see enum CCM_OP_FINFO

__STATIC_INLINE void MUnlockCCacheLine (unsigned long addr)

Unlock one Cluster Cache line specified by address in M-Mode.

This function unlock one Cluster Cache line specified by the address. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void MUnlockCCacheLines (unsigned long addr, unsigned long cnt)

Unlock several Cluster Cache lines specified by address in M-Mode.

This function unlock several Cluster Cache lines specified by the address and line count. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_MCOMMAND* (page 112).

Remark

This function must be executed in M-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void SUnlockCCacheLine (unsigned long addr)

Unlock one Cluster Cache line specified by address in S-Mode.

This function unlock one Cluster Cache line specified by the address. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void SUnlockCCacheLines (unsigned long addr, unsigned long cnt)

Unlock several Cluster Cache lines specified by address in S-Mode.

This function unlock several Cluster Cache lines specified by the address and line count. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_SCOMMAND* (page 112).

Remark

This function must be executed in M/S-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

__STATIC_INLINE void UUnlockCCacheLine (unsigned long addr)

Unlock one Cluster Cache line specified by address in U-Mode.

This function unlock one Cluster Cache line specified by the address. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

addr – [in] start address to be unlocked

__STATIC_INLINE void UUnlockCCacheLines (unsigned long addr, unsigned long cnt)

Unlock several Cluster Cache lines specified by address in U-Mode.

This function unlock several Cluster Cache lines specified by the address and line count. Command CCM_CC_UNLOCK is written to CSR *CSR_CCM_UCOMMAND* (page 112).

Remark

This function must be executed in M/S/U-Mode only.

Parameters

- **addr** – [in] start address to be unlocked
- **cnt** – [in] count of cache lines to be unlocked

Defines

SMPCC_CMD_CTRL_SUP_EN_Pos 9U

SMPCC_CMD CC_CTRL SUP_EN Position.

SMPCC_CMD_CTRL_SUP_EN_Msk (0x1UL << SMPCC_CMD_CTRL_SUP_EN_Pos)

SMPCC_CMD CC_CTRL SUP_EN Mask.

SMPCC_CMD_CTRL_SUP_EN_ENABLE 1U

SMPCC_CMD CC_CTRL SUP_EN Enable.

SMPCC_CMD_CTRL_SUP_EN_DISABLE 0U

SMPCC_CMD CC_CTRL SUP_EN Disable.

SMPCC_CMD_CTRL_USE_EN_Pos 10U

SMPCC_CMD CC_CTRL USE_EN Position.

SMPCC_CMD_CTRL_USE_EN_Msk (0x1UL << SMPCC_CMD_CTRL_USE_EN_Pos)

SMPCC_CMD CC_CTRL USE_EN Mask.

SMPCC_CMD_CTRL_USE_EN_ENABLE 1U

SMPCC_CMD CC_CTRL USE_EN Enable.

SMPCC_CMD_CTRL_USE_EN_DISABLE 0U

SMPCC_CMD CC_CTRL USE_EN Disable.

SMPCC_CMD_xCMD_CMD_Pos 0U

SMPCC_CMD register xCMD field CMD Position.

SMPCC_CMD_xCMD_CMD_Msk (0x1FUL << SMPCC_CMD_xCMD_CMD_Pos)

SMPCC_CMD register xCMD field CMD Mask.

SMPCC_CMD_xCMD_CMD_WB_ALL 0x7U

SMPCC_CMD xCMD CMD WB_ALL.

SMPCC_CMD_xCMD_CMD_WBINVAL_ALL 0x6U

SMPCC_CMD xCMD CMD WBINVAL_ALL.

SMPCC_CMD_xCMD_STATUS_Pos 23U

SMPCC_CMD register xCMD field RESC/FESC/BESC Position.

SMPCC_CMD_xCMD_STATUS_Msk (0x7UL << SMPCC_CMD_xCMD_STATUS_Pos)

SMPCC_CMD register xCMD field RESC/FESC/BESC Mask.

SMPCC_CMD_xCMD_RESULT_Pos 26U

SMPCC_CMD xCMD RESULT Position.

SMPCC_CMD_xCMD_RESULT_Msk (0x1FUL << SMPCC_CMD_xCMD_RESULT_Pos)

SMPCC_CMD xCMD RESULT Mask.

SMPCC_CMD_xCMD_RESULT_SUCCESS 0x0U

SMPCC_CMD xCMD RESULT Success.

SMPCC_CMD_xCMD_RESULT_ENTRY_EXCEED_LIMIT 0x1U

SMPCC_CMD xCMD RESULT Exceed the upper entry num of lockable way.

SMPCC_CMD_xCMD_RESULT_REFILL_BUS_ERROR 0x3U

SMPCC_CMD xCMD RESULT Refill Bus Error.

SMPCC_CMD_xCMD_RESULT_ECC_ERROR 0x4U

SMPCC_CMD xCMD RESULT ECC Error.

SMPCC_CMD_xCMD_RESULT_CPBACK_BUS_ERROR 0x5U

SMPCC_CMD xCMD RESULT Copy Back Bus Error.

SMPCC_CMD_xCMD_COMPLETE_Pos 31U

SMPCC_CMD xCMD COMPLETE Position.

SMPCC_CMD_xCMD_COMPLETE_Msk (0x1UL << SMPCC_CMD_xCMD_COMPLETE_Pos)

SMPCC_CMD xCMD COMPLETE Mask.

SMPCC_CMD_INVALID_ALL_Pos 0U

SMPCC_CMD INVALID_ALL Position.

SMPCC_CMD_INVALID_ALL_Msk (0x1UL << SMPCC_CMD_INVALID_ALL_Pos)

SMPCC_CMD INVALID_ALL Mask.

SMPCC_CMD_BASE __SMPCC_BASEADDR

SMPCC CMD Base Address.

SMPCC_CMD ((SMPCC_CMD_Type *)SMPCC_CMD_BASE)

SMPCC CMD configuration struct.

struct **SMPCC_CMD_Type**

#include <core_feature_cache.h> Cluster Cache Control and Command Registers.

Remark

This structure only holds a part of SMPCC registers, which are related to Cluster Cache CCM operations. Other more registers of SMPCC are listed in *core_feature_smpcc.h*

2.5.20 SMPCC Functions

__STATIC_FORCEINLINE SMP_VER_Type SMPCC_GetVersion (void)

__STATIC_FORCEINLINE uint8_t SMPCC_IsCCachePresent (void)

__STATIC_FORCEINLINE uint8_t SMPCC_GetCoreNum (void)

```
__STATIC_FORCEINLINE uint8_t SMPCC_GetIOCPNum (void)

__STATIC_FORCEINLINE uint8_t SMPCC_GetPMONNum (void)

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheSetNum (void)

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheWayNum (void)

__STATIC_FORCEINLINE uint8_t SMPCC_GetCCacheLineSize (void)

__STATIC_FORCEINLINE uint8_t SMPCC_IsCCacheSupportECC (void)

__STATIC_FORCEINLINE void SMPCC_EnableSnoop (uint16_t client_msk)

__STATIC_FORCEINLINE void SMPCC_DisableSnoop (uint16_t client_msk)

__STATIC_FORCEINLINE void SMPCC_EnableCCache (void)

__STATIC_FORCEINLINE void SMPCC_DisableCCache (void)

__STATIC_FORCEINLINE int32_t SMPCC_IsCCacheEnabled (void)

__STATIC_FORCEINLINE uint8_t SMPCC_IsAnySnoopPending (uint32_t client_msk)

__STATIC_FORCEINLINE uint8_t SMPCC_IsAnyTransactionPending (uint32_t client_msk)

__STATIC_FORCEINLINE void SMPCC_SetCLMAllWays (uint64_t addr)

__STATIC_FORCEINLINE void SMPCC_SetCLMNWays (uint64_t addr, uint32_t way_msk)

__STATIC_FORCEINLINE void SMPCC_SetCLMNoWay (void)

__STATIC_FORCEINLINE void SMPCC_SetCCacheControl (uint32_t val)

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheControl (void)

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECC (void)

__STATIC_FORCEINLINE void SMPCC_DisableCCacheECC (void)

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECCExcp (void)
```

```
__STATIC_FORCEINLINE void SMPCC_DisableCCacheECCExcp (void)

__STATIC_FORCEINLINE void SMPCC_LockECCConfig (void)

__STATIC_FORCEINLINE void SMPCC_LockECCErrInjection (void)

__STATIC_FORCEINLINE void SMPCC_EnableRecvErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_DisableRecvErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_EnableFatalErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_ClearBusErrPending (void)

__STATIC_FORCEINLINE void SMPCC_DisableFatalErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_EnableBusErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_DisableBusErrIrq (void)

__STATIC_FORCEINLINE void SMPCC_EnableSModeCmd (void)

__STATIC_FORCEINLINE void SMPCC_DisableSModeCmd (void)

__STATIC_FORCEINLINE void SMPCC_EnableUModeCmd (void)

__STATIC_FORCEINLINE void SMPCC_DisableUModeCmd (void)

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECCCheck (void)

__STATIC_FORCEINLINE void SMPCC_DisableCCacheECCCheck (void)

__STATIC_FORCEINLINE void SMPCC_EnableCLMECC (void)

__STATIC_FORCEINLINE void SMPCC_DisableCLMECC (void)

__STATIC_FORCEINLINE void SMPCC_EnableCLMECCCheck (void)

__STATIC_FORCEINLINE void SMPCC_DisableCLMECCCheck (void)

__STATIC_FORCEINLINE void SMPCC_EnableCLMECCExcp (void)
```

```
__STATIC_FORCEINLINE void SMPCC_DisableCLMECCExcp (void)

__STATIC_FORCEINLINE void SMPCC_EnableL1PrefetchShareCacheline (void)

__STATIC_FORCEINLINE void SMPCC_DisableL1PrefetchShareCacheline (void)

__STATIC_FORCEINLINE void SMPCC_EnableCCacheEarlyPrefetch (void)

__STATIC_FORCEINLINE void SMPCC_DisableCCacheEarlyPrefetch (void)

__STATIC_FORCEINLINE void SMPCC_LimitCCachePrefetchOutsNum (void)

__STATIC_FORCEINLINE void SMPCC_UnlimitCCachePrefetchOutsNum (void)

__STATIC_FORCEINLINE void SMPCC_EnableCCachePrefetchNoWb (void)

__STATIC_FORCEINLINE void SMPCC_DisableCCachePrefetchNoWb (void)

__STATIC_FORCEINLINE void SMPCC_EnableICacheSnoopDCache (void)

__STATIC_FORCEINLINE void SMPCC_DisableICacheSnoopDCache (void)

__STATIC_FORCEINLINE uint32_t SMPCC_GetRecvErrCount (void)

__STATIC_FORCEINLINE void SMPCC_ClearRecvErrCount (void)

__STATIC_FORCEINLINE uint32_t SMPCC_GetFatalErrCount (void)

__STATIC_FORCEINLINE void SMPCC_ClearFatalErrCount (void)

__STATIC_FORCEINLINE void SMPCC_SetRecvErrCntThreshold (uint16_t threshold)

__STATIC_FORCEINLINE uint16_t SMPCC_GetRecvErrCntThreshold (void)

__STATIC_FORCEINLINE void SMPCC_SetFatalErrCntThreshold (uint16_t threshold)

__STATIC_FORCEINLINE uint16_t SMPCC_GetFatalErrCntThreshold (void)

__STATIC_FORCEINLINE uint32_t SMPCC_GetClientErrStatus (uint8_t client_id)

__STATIC_FORCEINLINE void SMPCC_SetSTMControl (uint32_t val)
```

```
__STATIC_FORCEINLINE uint32_t SMPCC_GetSTMControl (void)

__STATIC_FORCEINLINE void SMPCC_EnableStreamRead (void)

__STATIC_FORCEINLINE void SMPCC_DisableStreamRead (void)

__STATIC_FORCEINLINE void SMPCC_EnableStreamWrite (void)

__STATIC_FORCEINLINE void SMPCC_DisableStreamWrite (void)

__STATIC_FORCEINLINE void SMPCC_EnableStreamTransAlloc (void)

__STATIC_FORCEINLINE void SMPCC_DisableStreamTransAlloc (void)

__STATIC_FORCEINLINE void SMPCC_EnableStreamMergeNCRead (void)

__STATIC_FORCEINLINE void SMPCC_DisableStreamMergeNCRead (void)

__STATIC_FORCEINLINE void SMPCC_EnableStreamReadCross4K (void)

__STATIC_FORCEINLINE void SMPCC_DisableStreamReadCross4K (void)

__STATIC_FORCEINLINE void SMPCC_SetNSRegionNACL (uint8_t region_id, uint64_t addr)

__STATIC_FORCEINLINE void SMPCC_SetNSRegionNAPOT (uint8_t region_id, uint64_t addr)

__STATIC_FORCEINLINE void SMPCC_DisableNSRegion (uint8_t region_id)

__STATIC_FORCEINLINE void SMPCC_SetPMONEventSelect (uint8_t idx, uint8_t client_id,
uint8_t event)

__STATIC_FORCEINLINE uint32_t SMPCC_GetPMONEventSelect (uint8_t idx)

__STATIC_FORCEINLINE uint64_t SMPCC_GetPMONCount (uint8_t idx)

__STATIC_FORCEINLINE void SMPCC_ClearPMONCount (uint8_t idx)

__STATIC_FORCEINLINE uint64_t SMPCC_GetClientErrAddr (uint8_t client_id)

__STATIC_FORCEINLINE void SMPCC_MaskClientCCacheWays (uint8_t client_id,
uint32_t way_msk)
```

```
__STATIC_FORCEINLINE int32_t SMPCC_IsXorErrorInjectMode (void)
```

```
__STATIC_FORCEINLINE void SMPCC_SetECCCode (uint32_t ecc_code)
```

```
__STATIC_FORCEINLINE void SMPCC_CCacheTramErrInject (uint32_t ecc_code, uint32_t *addr,  
uint16_t way)
```

```
__STATIC_FORCEINLINE void SMPCC_CCacheDramErrInject (uint32_t ecc_code, uint32_t *addr,  
uint16_t way)
```

```
__STATIC_FORCEINLINE void SMPCC_CLMErrInject (uint32_t ecc_code, uint32_t *addr)
```

```
__STATIC_FORCEINLINE void SMPCC_CCacheErrRestore (void *addr)
```

group **SMPCC Functions**

SMPCC related functions.

Functions

```
__STATIC_FORCEINLINE SMP_VER_Type SMPCC_GetVersion (void)
```

Get the SMP version number.

This function gets the hardware version information from SMP_VER register.

Returns

hardware version number in SMP_VER register.

```
__STATIC_FORCEINLINE uint8_t SMPCC_IsCCachePresent (void)
```

Check if cluster cache is present.

This function checks if the cluster cache is present in the system.

Returns

1 if cluster cache is present, 0 otherwise

```
__STATIC_FORCEINLINE uint8_t SMPCC_GetCoreNum (void)
```

Get the number of cores in the cluster.

This function returns the number of cores in the SMP cluster.

Returns

Number of cores in the cluster

```
__STATIC_FORCEINLINE uint8_t SMPCC_GetIOCPNum (void)
```

Get the number of IO coherency ports.

This function returns the number of IO coherency ports in the cluster.

Returns

Number of IO coherency ports

__STATIC_FORCEINLINE uint8_t SMPCC_GetPMONNum (void)

Get the number of performance monitors.

This function returns the number of performance monitors in the cluster.

Returns

Number of performance monitors

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheSetNum (void)

Get the number of cache sets.

This function returns the number of cache sets in the cluster cache ($2^{\text{cc_set}}$).

Returns

Number of cache sets

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheWayNum (void)

Get the number of cache ways.

This function returns the number of cache ways in the cluster cache ($\text{cc_way} + 1$).

Returns

Number of cache ways

__STATIC_FORCEINLINE uint8_t SMPCC_GetCCacheLineSize (void)

Get the cache line size.

This function returns the cache line size in the cluster cache ($2^{(\text{cc_lsize} + 2)}$).

Returns

Cache line size in bytes

__STATIC_FORCEINLINE uint8_t SMPCC_IsCCacheSupportECC (void)

Check if cluster cache supports ECC.

This function checks if the cluster cache supports ECC functionality.

Returns

1 if ECC is supported, 0 otherwise

__STATIC_FORCEINLINE void SMPCC_EnableSnoop (uint16_t client_msk)

Enable snoop for specific clients.

This function enables snoop functionality for specified client mask.

See also

- SMPCC_DisableSnoop

Parameters

client_msk – [in] Client mask to enable snoop for

__STATIC_FORCEINLINE void SMPCC_DisableSnoop (uint16_t client_msk)

Disable snoop for specific clients.

This function disables snoop functionality for specified client mask.

See also

- SMPCC_EnableSnoop

Parameters

client_msk – [in] Client mask to disable snoop for

__STATIC_FORCEINLINE void SMPCC_EnableCCache (void)

Enable cluster cache.

This function enables the cluster cache.

See also

- SMPCC_DisableCCache

__STATIC_FORCEINLINE void SMPCC_DisableCCache (void)

Disable cluster cache.

This function disables the cluster cache.

See also

- SMPCC_EnableCCache

__STATIC_FORCEINLINE int32_t SMPCC_IsCCacheEnabled (void)

Get status of cluster cache.

This function returns the enable status of the cluster cache.

See also

- SMPCC_EnableCCache

Returns

Return the status of cluster cache (1 - enabled, 0 - disabled)

__STATIC_FORCEINLINE uint8_t SMPCC_IsAnySnoopPending (uint32_t client_msk)

Check if any client is being snooped.

This function checks if any client specified in the mask is currently being snooped.

Parameters

client_msk – [in] Client mask to check

Returns

1 if any client is being snooped, 0 otherwise

__STATIC_FORCEINLINE uint8_t SMPCC_IsAnyTransactionPending (uint32_t client_msk)

Check if any transaction is pending for clients.

This function checks if any transaction is pending for clients specified in the mask.

Parameters

client_msk – [in] Client mask to check

Returns

1 if any transaction is pending, 0 otherwise

__STATIC_FORCEINLINE void SMPCC_SetCLMAllWays (uint64_t addr)

Set Cluster Local Memory to use all ways.

This function configures the Cluster Local Memory to use all cache ways at the specified address.

Example usage:

```
SMPCC_DisableCCache();
MFlushInvalCCache();
SMPCC_SetCLMAllWays(addr);
```

Remark

- Access to the CLM_ADDR_BASE register depends on the physical address (PA) size. If the PA size is not larger than 32 bits, only the low 32 bits of the address can be accessed.
- Before changing the Cluster Cache to CLM mode, ensure that:
 - The Cluster Cache is disabled,
 - Its contents have been invalidated and flushed to memory.

See also

- SMPCC_SetCLMNWays
- SMPCC_SetCLMNoWay

Parameters

addr – [in] Base address for Cluster Local Memory.

__STATIC_FORCEINLINE void SMPCC_SetCLMNWays (uint64_t addr, uint32_t way_msk)

Set Cluster Local Memory to use specific ways.

This function configures the Cluster Local Memory to use specific cache ways at the specified address.

Example usage:

```
SMPCC_DisableCCache();  
MFlushInvalCCache();  
SMPCC_SetCLMNWays(addr, way_msk);  
SMPCC_EnableCCache(); // Enable the remaining CCache
```

Remark

- Access to the CLM_ADDR_BASE register depends on the physical address (PA) size. If the PA size is not larger than 32 bits, only the low 32 bits of the address can be accessed.
- Before changing the Cluster Cache to CLM mode, ensure that:
 - The Cluster Cache is disabled,
 - Its contents have been invalidated and flushed to memory.

See also

- SMPCC_SetCLMAliWays
- SMPCC_SetCLMNoWay

Parameters

- **addr** – [in] Base address for Cluster Local Memory
- **way_msk** – [in] Way mask to configure

__STATIC_FORCEINLINE void SMPCC_SetCLMNoWay (void)

Configure Cluster Local Memory to use no ways.

This function configures the Cluster Local Memory to not use any cache ways.

See also

- SMPCC_SetCLMAliWays

- SMPCC_SetCLMNWays

__STATIC_FORCEINLINE void SMPCC_SetCCacheControl (uint32_t val)

Set Cluster Cache Control register.

This function sets the value of the Cluster Cache Control register.

See also

- SMPCC_GetCCacheControl

Parameters

val – [in] Value to set in the control register

__STATIC_FORCEINLINE uint32_t SMPCC_GetCCacheControl (void)

Get Cluster Cache Control register value.

This function returns the current value of the Cluster Cache Control register.

See also

- SMPCC_SetCCacheControl

Returns

Current value of the control register

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECC (void)

Enable Cluster Cache ECC.

This function enables ECC functionality for the cluster cache.

See also

- SMPCC_DisableCCacheECC

__STATIC_FORCEINLINE void SMPCC_DisableCCacheECC (void)

Disable Cluster Cache ECC.

This function disables ECC functionality for the cluster cache.

See also

- SMPCC_EnableCCacheECC

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECCExcp (void)

Enable Cluster Cache ECC Exception.

This function enables ECC exception handling for the cluster cache.

See also

- SMPCC_DisableCCacheECCExcp

__STATIC_FORCEINLINE void SMPCC_DisableCCacheECCExcp (void)

Disable Cluster Cache ECC Exception.

This function disables ECC exception handling for the cluster cache.

See also

- SMPCC_EnableCCacheECCExcp

__STATIC_FORCEINLINE void SMPCC_LockECCConfig (void)

Lock ECC Configuration.

This function locks the ECC configuration to prevent further changes.

See also

- SMPCC_LockECCErrInjection

__STATIC_FORCEINLINE void SMPCC_LockECCErrInjection (void)

Lock ECC Error Injection Register.

This function locks the ECC error injection register to prevent further changes.

See also

- SMPCC_LockECCConfig

__STATIC_FORCEINLINE void SMPCC_EnableRecvErrIrq (void)

Enable Recoverable Error Interrupt.

This function enables interrupt generation when recoverable error count exceeds the threshold.

See also

- SMPCC_DisableRecvErrIrq

__STATIC_FORCEINLINE void SMPCC_DisableRecvErrIrq (void)

Disable Recoverable Error Interrupt.

This function disables interrupt generation when recoverable error count exceeds the threshold.

See also

- SMPCC_EnableRecvErrIrq

__STATIC_FORCEINLINE void SMPCC_EnableFatalErrIrq (void)

Enable Fatal Error Interrupt.

This function enables interrupt generation when fatal error count exceeds the threshold.

See also

- SMPCC_DisableFatalErrIrq

__STATIC_FORCEINLINE void SMPCC_ClearBusErrPending (void)

Clear Bus Error Pending Status.

This function clears the pending status for bus errors in cluster cache maintenance operations.

See also

- SMPCC_EnableBusErrIrq
- SMPCC_DisableBusErrIrq

__STATIC_FORCEINLINE void SMPCC_DisableFatalErrIrq (void)

Disable Fatal Error Interrupt.

This function disables interrupt generation when fatal error count exceeds the threshold.

See also

- SMPCC_EnableFatalErrIrq

__STATIC_FORCEINLINE void SMPCC_EnableBusErrIrq (void)

Enable Bus Error Interrupt.

This function enables interrupt generation for bus errors in cluster cache maintenance operations.

See also

- SMPCC_DisableBusErrIrq

__STATIC_FORCEINLINE void SMPCC_DisableBusErrIrq (void)

Disable Bus Error Interrupt.

This function disables interrupt generation for bus errors in cluster cache maintenance operations.

See also

- SMPCC_EnableBusErrIrq

__STATIC_FORCEINLINE void SMPCC_EnableSModeCmd (void)

Enable Supervisor Mode Commands.

This function enables supervisor mode to operate CC_sCMD and SMP_PMON_SEL registers.

See also

- SMPCC_DisableSModeCmd

__STATIC_FORCEINLINE void SMPCC_DisableSModeCmd (void)

Disable Supervisor Mode Commands.

This function disables supervisor mode from operating CC_sCMD and SMP_PMON_SEL registers.

See also

- SMPCC_EnableSModeCmd

__STATIC_FORCEINLINE void SMPCC_EnableUModeCmd (void)

Enable User Mode Commands.

This function enables user mode to operate CC_uCMD and SMP_PMON_SEL registers.

See also

- SMPCC_DisableUModeCmd

__STATIC_FORCEINLINE void SMPCC_DisableUModeCmd (void)

Disable User Mode Commands.

This function disables user mode from operating CC_uCMD and SMP_PMON_SEL registers.

See also

- SMPCC_EnableUModeCmd

__STATIC_FORCEINLINE void SMPCC_EnableCCacheECCCheck (void)

Enable Cluster Cache ECC Check.

This function enables ECC check functionality for the cluster cache.

See also

- SMPCC_DisableCCacheECCCheck

__STATIC_FORCEINLINE void SMPCC_DisableCCacheECCCheck (void)

Disable Cluster Cache ECC Check.

This function disables ECC check functionality for the cluster cache.

See also

- SMPCC_EnableCCacheECCCheck

__STATIC_FORCEINLINE void SMPCC_EnableCLMECC (void)

Enable Cluster Local Memory ECC.

This function enables ECC functionality for the Cluster Local Memory.

See also

- SMPCC_DisableCLMECC

__STATIC_FORCEINLINE void SMPCC_DisableCLMECC (void)

Disable Cluster Local Memory ECC.

This function disables ECC functionality for the Cluster Local Memory.

See also

- SMPCC_EnableCLMECC

__STATIC_FORCEINLINE void SMPCC_EnableCLMECCCheck (void)

Enable Cluster Local Memory ECC Check.

This function enables ECC check functionality for the Cluster Local Memory.

See also

- SMPCC_DisableCLMECCCheck

__STATIC_FORCEINLINE void SMPCC_DisableCLMECCCheck (void)

Disable Cluster Local Memory ECC Check.

This function disables ECC check functionality for the Cluster Local Memory.

See also

- SMPCC_EnableCLMECCCheck

__STATIC_FORCEINLINE void SMPCC_EnableCLMECCExcp (void)

Enable Cluster Local Memory ECC Exception.

This function enables ECC exception handling for the Cluster Local Memory.

See also

- SMPCC_DisableCLMECCExcp

__STATIC_FORCEINLINE void SMPCC_DisableCLMECCExcp (void)

Disable Cluster Local Memory ECC Exception.

This function disables ECC exception handling for the Cluster Local Memory.

See also

- SMPCC_EnableCLMECCExcp

__STATIC_FORCEINLINE void SMPCC_EnableL1PrefetchShareCacheline (void)

Enable L1 Prefetch to Snoop and Share Cacheline.

This function enables L1 prefetch to snoop and share cacheline from other cores.

See also

- SMPCC_DisableL1PrefetchShareCacheline

__STATIC_FORCEINLINE void SMPCC_DisableL1PrefetchShareCacheline (void)

Disable L1 Prefetch to Snoop and Share Cacheline.

This function disables L1 prefetch to snoop and share cacheline from other cores.

See also

- SMPCC_EnableL1PrefetchShareCacheline

__STATIC_FORCEINLINE void SMPCC_EnableCCacheEarlyPrefetch (void)

Enable Cluster Cache Early Prefetch.

This function enables L2 prefetch to initialize external bus read access while looking up the cluster cache.

See also

- SMPCC_DisableCCacheEarlyPrefetch

__STATIC_FORCEINLINE void SMPCC_DisableCCacheEarlyPrefetch (void)

Disable Cluster Cache Early Prefetch.

This function disables L2 prefetch to initialize external bus read access while looking up the cluster cache.

See also

- SMPCC_EnableCCacheEarlyPrefetch

__STATIC_FORCEINLINE void SMPCC_LimitCCachePrefetchOutsNum (void)

Limit Cluster Cache Prefetch Outstanding Number.

This function enables the limit of outstanding L2 prefetch to the number of L2 prefetch line-buffer.

See also

- SMPCC_UnlimitCCachePrefetchOutsNum

__STATIC_FORCEINLINE void SMPCC_UnlimitCCachePrefetchOutsNum (void)

Unlimit Cluster Cache Prefetch Outstanding Number.

This function disables the limit of outstanding L2 prefetch to the number of L2 prefetch line-buffer.

See also

- SMPCC_LimitCCachePrefetchOutsNum

__STATIC_FORCEINLINE void SMPCC_EnableCCachePrefetchNoWb (void)

Enable Cluster Cache Prefetch to Avoid Write Back.

This function enables L2 prefetch to abort and avoid dirty cacheline write back when filling the cluster cache.

See also

- SMPCC_DisableCCachePrefetchNoWb

__STATIC_FORCEINLINE void SMPCC_DisableCCachePrefetchNoWb (void)

Disable Cluster Cache Prefetch to Avoid Write Back.

This function disables L2 prefetch from aborting and avoiding dirty cacheline write back when filling the cluster cache.

See also

- SMPCC_EnableCCachePrefetchNoWb

__STATIC_FORCEINLINE void SMPCC_EnableICacheSnoopDCache (void)

Enable ICache to Snoop DCache.

This function enables snoop to dcache for icache refill reads.

See also

- SMPCC_DisableICacheSnoopDCache

__STATIC_FORCEINLINE void SMPCC_DisableICacheSnoopDCache (void)

Disable ICache to Snoop DCache.

This function disables snoop to dcache for icache refill reads.

See also

- SMPCC_EnableICacheSnoopDCache

__STATIC_FORCEINLINE uint32_t SMPCC_GetRecvErrCount (void)

Get Recoverable Error Count.

This function returns the current count of recoverable errors.

See also

- SMPCC_ClearRecvErrCount

Returns

Current count of recoverable errors

__STATIC_FORCEINLINE void SMPCC_ClearRecvErrCount (void)

Clear Recoverable Error Count.

This function clears the recoverable error count register.

See also

- SMPCC_GetRecvErrCount

__STATIC_FORCEINLINE uint32_t SMPCC_GetFatalErrCount (void)

Get Fatal Error Count.

This function returns the current count of fatal errors.

See also

- SMPCC_ClearFatalErrCount

Returns

Current count of fatal errors

__STATIC_FORCEINLINE void SMPCC_ClearFatalErrCount (void)

Clear Fatal Error Count.

This function clears the fatal error count register.

See also

- SMPCC_GetFatalErrCount

__STATIC_FORCEINLINE void SMPCC_SetRecvErrCntThreshold (uint16_t threshold)

Set Recoverable Error Count Threshold.

This function sets the threshold value for recoverable error count.

See also

- SMPCC_GetRecvErrCntThreshold

Parameters

threshold – [in] Threshold value to set

__STATIC_FORCEINLINE uint16_t SMPCC_GetRecvErrCntThreshold (void)

Get Recoverable Error Count Threshold.

This function returns the current threshold value for recoverable error count.

See also

- SMPCC_SetRecvErrCntThreshold

Returns

Current threshold value for recoverable error count

__STATIC_FORCEINLINE void SMPCC_SetFatalErrCntThreshold (uint16_t threshold)

Set Fatal Error Count Threshold.

This function sets the threshold value for fatal error count.

See also

- SMPCC_GetFatalErrCntThreshold

Parameters

threshold – [in] Threshold value to set

__STATIC_FORCEINLINE uint16_t SMPCC_GetFatalErrCntThreshold (void)

Get Fatal Error Count Threshold.

This function returns the current threshold value for fatal error count.

See also

- SMPCC_SetFatalErrCntThreshold

Returns

Current threshold value for fatal error count

__STATIC_FORCEINLINE uint32_t SMPCC_GetClientErrStatus (uint8_t client_id)

Get Client Error Status.

This function returns the error status for a specific client.

Parameters

client_id – [in] ID of the client to get error status for

Returns

Error status of the specified client

__STATIC_FORCEINLINE void SMPCC_SetSTMControl (uint32_t val)

Set Stream Control Register.

This function sets the value of the Stream Control register.

See also

- SMPCC_GetSTMControl

Parameters

val – [in] Value to set in the stream control register

__STATIC_FORCEINLINE uint32_t SMPCC_GetSTMControl (void)

Get Stream Control Register Value.

This function returns the current value of the Stream Control register.

See also

- SMPCC_SetSTMControl

Returns

Current value of the stream control register

__STATIC_FORCEINLINE void SMPCC_EnableStreamRead (void)

Enable Stream Read.

This function enables stream read functionality.

See also

- SMPCC_DisableStreamRead

__STATIC_FORCEINLINE void SMPCC_DisableStreamRead (void)

Disable Stream Read.

This function disables stream read functionality.

See also

- SMPCC_EnableStreamRead

__STATIC_FORCEINLINE void SMPCC_EnableStreamWrite (void)

Enable Stream Write.

This function enables stream write functionality.

See also

- SMPCC_DisableStreamWrite

__STATIC_FORCEINLINE void SMPCC_DisableStreamWrite (void)

Disable Stream Write.

This function disables stream write functionality.

See also

- SMPCC_EnableStreamWrite

__STATIC_FORCEINLINE void SMPCC_EnableStreamTransAlloc (void)

Enable Stream Translate Allocate.

This function enables translation of allocate attribute to non-alloc attribute.

See also

- SMPCC_DisableStreamTransAlloc

__STATIC_FORCEINLINE void SMPCC_DisableStreamTransAlloc (void)

Disable Stream Translate Allocate.

This function disables translation of allocate attribute to non-alloc attribute.

See also

- SMPCC_EnableStreamTransAlloc

__STATIC_FORCEINLINE void SMPCC_EnableStreamMergeNCRead (void)

Enable Stream Merge Non-Cacheable Read.

This function enables non-cacheable attribute read merge functionality.

See also

- SMPCC_DisableStreamMergeNCRead

__STATIC_FORCEINLINE void SMPCC_DisableStreamMergeNCRead (void)

Disable Stream Merge Non-Cacheable Read.

This function disables non-cacheable attribute read merge functionality.

See also

- SMPCC_EnableStreamMergeNCRead

__STATIC_FORCEINLINE void SMPCC_EnableStreamReadCross4K (void)

Enable Stream Read Cross 4K Boundary.

This function enables read stream to cross 4K boundary.

See also

- SMPCC_DisableStreamReadCross4K

__STATIC_FORCEINLINE void SMPCC_DisableStreamReadCross4K (void)

Disable Stream Read Cross 4K Boundary.

This function disables read stream from crossing 4K boundary.

See also

- SMPCC_EnableStreamReadCross4K

__STATIC_FORCEINLINE void SMPCC_SetNSRegionNACL (uint8_t region_id, uint64_t addr)

Set Non-Shareable Region to NACL.

This function configures a non-shareable region as NACL (NAPOT/CA with length) at the specified address.

See also

- SMPCC_SetNSRegionNAPOT
- SMPCC_DisableNSRegion

Parameters

- **region_id** – [in] ID of the region to configure
- **addr** – [in] Address for the region

__STATIC_FORCEINLINE void SMPCC_SetNSRegionNAPOT (uint8_t region_id, uint64_t addr)

Set Non-Shareable Region to NAPOT.

This function configures a non-shareable region as NAPOT (Naturally Aligned Power of Two) at the specified address.

See also

- SMPCC_SetNSRegionNACL
- SMPCC_DisableNSRegion

Parameters

- **region_id** – [in] ID of the region to configure
- **addr** – [in] Address for the region

__STATIC_FORCEINLINE void SMPCC_DisableNSRegion (uint8_t region_id)

Disable Non-Shareable Region.

This function disables a non-shareable region.

See also

- SMPCC_SetNSRegionNAPOT

- SMPCC_SetNSRegionNACL

Parameters

region_id – [in] ID of the region to disable

__STATIC_FORCEINLINE void SMPCC_SetPMONEventSelect (uint8_t idx, uint8_t client_id, uint8_t event)

Set Performance Monitor Event Selection.

This function configures a performance monitor to select a specific event and client.

See also

- SMPCC_GetPMONEventSelect
- SMPCC_GetPMONCount
- SMPCC_ClearPMONCount

Parameters

- **idx** – [in] Index of the performance monitor
- **client_id** – [in] ID of the client to monitor
- **event** – [in] Event to monitor

__STATIC_FORCEINLINE uint32_t SMPCC_GetPMONEventSelect (uint8_t idx)

Get Performance Monitor Event Selection.

This function gets a performance monitor configuration value.

See also

- SMPCC_SetPMONEventSelect
- SMPCC_GetPMONCount
- SMPCC_ClearPMONCount

Parameters

idx – [in] Index of the performance monitor

Returns

The performance monitor configuration value.

__STATIC_FORCEINLINE uint64_t SMPCC_GetPMONCount (uint8_t idx)

Get Performance Monitor Count.

This function returns the current count value of a performance monitor.

See also

- SMPCC_SetPMONEventSelect
- SMPCC_GetPMONEventSelect
- SMPCC_ClearPMONCount

Parameters

idx – [in] Index of the performance monitor

Returns

Current count value of the performance monitor

__STATIC_FORCEINLINE void SMPCC_ClearPMONCount (uint8_t idx)

Clear Performance Monitor Count.

This function clears the count value of a performance monitor.

See also

- SMPCC_SetPMONEventSelect
- SMPCC_GetPMONEventSelect
- SMPCC_GetPMONCount

Parameters

idx – [in] Index of the performance monitor to clear

__STATIC_FORCEINLINE uint64_t SMPCC_GetClientErrAddr (uint8_t client_id)

Get Client Error Address.

This function returns the error address for a specific client.

Parameters

client_id – [in] ID of the client to get error address for

Returns

Error address of the specified client

__STATIC_FORCEINLINE void SMPCC_MaskClientCCacheWays (uint8_t client_id, uint32_t way_msk)

Mask Client Cluster Cache Ways.

This function masks specific ways in the cluster cache for a specific client.

Parameters

- **client_id** – [in] ID of the client
- **way_msk** – [in] Way mask to apply

__STATIC_FORCEINLINE int32_t SMPCC_IsXorErrorInjectMode (void)

Check if ECC error injection mode is XOR mode.

This function checks which ECC error injection mode is supported. Returns 1 if XOR mode is supported, 0 if direct write mode is supported.

Returns

1 if XOR mode is supported, 0 if direct write mode is supported

__STATIC_FORCEINLINE void SMPCC_SetECCCode (uint32_t ecc_code)

Set ECC code for error injection.

This function sets the ECC code to be used for error injection.

Parameters

ecc_code – ECC code to be set for error injection

Returns

None

__STATIC_FORCEINLINE void SMPCC_CCacheTramErrInject (uint32_t ecc_code, uint32_t *addr, uint16_t way)

Inject ECC error to cluster cache tag RAM.

This function injects an ECC error into the cluster cache tag RAM at the specified address.

Remark

The 4-byte data pointed to by addr is used to refill the CCache line. The cache line is larger than 4 bytes, this data is replicated to fill the entire line.

Remark

The way value must be one-hot. For example, if the cache has 2 ways, valid values are 0x01 and 0x02; all other values are invalid.

Parameters

- **ecc_code** – ECC code to be injected
- **addr** – Address where the error should be injected
- **way** – Cache way number used for precise injection

Returns

None

__STATIC_FORCEINLINE void SMPCC_CCacheDramErrInject (uint32_t ecc_code, uint32_t *addr, uint16_t way)

Inject ECC error to cluster cache data RAM.

This function injects an ECC error into the cluster cache data RAM at the specified address.

Remark

The 4-byte data pointed to by `addr` is used to refill the CCache line. The cache line is larger than 4 bytes, this data is replicated to fill the entire line.

Remark

The way value must be one-hot. For example, if the cache has 2 ways, valid values are 0x01 and 0x02; all other values are invalid.

Parameters

- **`ecc_code`** – ECC code to be injected
- **`addr`** – Address where the error should be injected
- **`way`** – Cache way number used for precise injection

Returns

None

`__STATIC_FORCEINLINE void SMPCC_CLMErrInject (uint32_t ecc_code, uint32_t *addr)`

Inject ECC error to CLM (Cluster Local Memory)

This function injects an ECC error into the CLM at the specified address. Only the ecc code can be injected, the data will keep as it is.

Parameters

- **`ecc_code`** – ECC code to be injected
- **`addr`** – Address where the error should be injected

Returns

None

`__STATIC_FORCEINLINE void SMPCC_CCacheErrRestore (void *addr)`

Restore cluster cache after error injection.

This function restores the cluster cache after an error injection operation.

Parameters

`addr` – Address to be restored

Returns

None

2.5.21 ECC Functions

```
__STATIC_FORCEINLINE int32_t ECC_IsGlobalSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsICacheSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsDCacheSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsTLBSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsILMSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsDLMSupportECC (void)

__STATIC_FORCEINLINE int32_t ECC_IsXorErrorInjectMode (void)

__STATIC_FORCEINLINE void ECC_EnableICacheECC (void)

__STATIC_FORCEINLINE void ECC_DisableICacheECC (void)

__STATIC_FORCEINLINE void ECC_EnableICacheECCExcp (void)

__STATIC_FORCEINLINE void ECC_DisableICacheECCExcp (void)

__STATIC_FORCEINLINE void ECC_EnableICacheECCCheck (void)

__STATIC_FORCEINLINE void ECC_DisableICacheECCCheck (void)

__STATIC_FORCEINLINE void ECC_EnableDCacheECC (void)

__STATIC_FORCEINLINE void ECC_DisableDCacheECC (void)

__STATIC_FORCEINLINE void ECC_EnableDCacheECCExcp (void)

__STATIC_FORCEINLINE void ECC_DisableDCacheECCExcp (void)

__STATIC_FORCEINLINE void ECC_EnableDCacheECCCheck (void)

__STATIC_FORCEINLINE void ECC_DisableDCacheECCCheck (void)

__STATIC_FORCEINLINE void ECC_ICacheTRamErrInject (uint32_t ecc_code, void *addr)
```

```
__STATIC_FORCEINLINE void ECC_ICacheDRamErrInject (uint32_t ecc_code, void *addr)

__STATIC_FORCEINLINE void ECC_ICacheErrRestore (void *addr)

__STATIC_FORCEINLINE void ECC_DCacheTRamErrInject (uint32_t ecc_code, void *addr)

__STATIC_FORCEINLINE void ECC_DCacheDRamErrInject (uint32_t ecc_code, void *addr)

__STATIC_FORCEINLINE void ECC_DCacheErrRestore (void *addr)

__STATIC_FORCEINLINE void ECC_EnableILM (void)

__STATIC_FORCEINLINE void ECC_DisableILM (void)

__STATIC_FORCEINLINE void ECC_EnableILMECC (void)

__STATIC_FORCEINLINE void ECC_DisableILMECC (void)

__STATIC_FORCEINLINE void ECC_EnableILMECCExcp (void)

__STATIC_FORCEINLINE void ECC_DisableILMECCExcp (void)

__STATIC_FORCEINLINE void ECC_EnableILMECCCheck (void)

__STATIC_FORCEINLINE void ECC_DisableILMECCCheck (void)

__STATIC_FORCEINLINE void ECC_ILMErrInject (uint32_t ecc_code, void *addr)

__STATIC_FORCEINLINE void ECC_ILMErrRestore (void *addr)

__STATIC_FORCEINLINE void ECC_EnableDLM (void)

__STATIC_FORCEINLINE void ECC_DisableDLM (void)

__STATIC_FORCEINLINE void ECC_EnableDLMECC (void)

__STATIC_FORCEINLINE void ECC_DisableDLMECC (void)

__STATIC_FORCEINLINE void ECC_EnableDLMECCExcp (void)

__STATIC_FORCEINLINE void ECC_DisableDLMECCExcp (void)
```

```
__STATIC_FORCEINLINE void ECC_EnableDLMECCCheck (void)

__STATIC_FORCEINLINE void ECC_DisableDLMECCCheck (void)

__STATIC_FORCEINLINE void ECC_DLMErrInject (uint32_t ecc_code, void *addr)

__STATIC_FORCEINLINE void ECC_DLMErrRestore (void *addr)

__STATIC_FORCEINLINE int32_t ECC_IsAnySingleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsICacheSingleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsDCacheSingleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsTLBSingleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsILMSingleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsDLMSingleBitErrorOccured (void)

__STATIC_FORCEINLINE void ECC_ClearAllSingleBitError (void)

__STATIC_FORCEINLINE void ECC_ClearICacheSingleBitError (void)

__STATIC_FORCEINLINE void ECC_ClearDCacheSingleBitError (void)

__STATIC_FORCEINLINE void ECC_ClearTLBSingleBitError (void)

__STATIC_FORCEINLINE void ECC_ClearILMSingleBitError (void)

__STATIC_FORCEINLINE void ECC_ClearDLMSingleBitError (void)

__STATIC_FORCEINLINE int32_t ECC_IsAnyDoubleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsICacheDoubleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsDCacheDoubleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsTLBDoubleBitErrorOccured (void)

__STATIC_FORCEINLINE int32_t ECC_IsILMDoubleBitErrorOccured (void)
```

```
__STATIC_FORCEINLINE int32_t ECC_IsDLMDoubleBitErrorOccured (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearAllDoubleBitError (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearICacheDoubleBitError (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearDCacheDoubleBitError (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearTLBDoubleBitError (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearILMDoubleBitError (void)
```

```
__STATIC_FORCEINLINE void ECC_ClearDLMDoubleBitError (void)
```

```
static uint8_t ECC_GenerateECCCodeU32(uint32_t a)
```

```
static uint8_t ECC_GenerateECCCodeU64(uint64_t a)
```

```
ECC_ERROR_RAMID_MASK_ICACHE 1U
```

```
ECC_ERROR_RAMID_MASK_DCACHE 2U
```

```
ECC_ERROR_RAMID_MASK_TLB 4U
```

```
ECC_ERROR_RAMID_MASK_ILM 8U
```

```
ECC_ERROR_RAMID_MASK_DLM 16U
```

group **ECC Functions**

Functions that related to the ECC feature.

These functions provide access to the Error Correction Code (ECC) feature available in Nuclei N/NX processor cores. ECC is a memory protection mechanism that can detect and correct single-bit errors and detect double-bit errors in memory systems such as caches and local memories.

The ECC feature includes:

- Detection of ECC support in various memory components (I-Cache, D-Cache, ILM, DLM, TLB)
- Control functions to enable/disable ECC for different memory components
- Error injection capabilities for testing purposes
- Error detection and clearing functions for both single-bit and double-bit errors
- Configuration of ECC exception handling

Defines

ECC_ERROR_RAMID_MASK_ICACHE 1U

ECC_ERROR_RAMID_MASK_DCACHE 2U

ECC_ERROR_RAMID_MASK_TLB 4U

ECC_ERROR_RAMID_MASK_ILM 8U

ECC_ERROR_RAMID_MASK_DLM 16U

Functions

__STATIC_FORCEINLINE int32_t ECC_IsGlobalSupportECC (void)

Check if the core globally supports ECC.

This function reads the machine configuration info CSR and checks if ECC is supported globally in the core.

Returns

1 if ECC is globally supported, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsICacheSupportECC (void)

Check if I-Cache supports ECC.

This function checks if both I-Cache and ECC are supported in the core.

Returns

1 if I-Cache supports ECC, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDCacheSupportECC (void)

Check if D-Cache supports ECC.

This function checks if both D-Cache and ECC are supported in the core.

Returns

1 if D-Cache supports ECC, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsTLBSupportECC (void)

Check if TLB supports ECC.

This function checks if TLB ECC is supported in the core.

Returns

1 if TLB supports ECC, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsILMSupportECC (void)

Check if ILM supports ECC.

This function checks if both ILM and ECC are supported in the core.

Returns

1 if ILM supports ECC, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDLMSupportECC (void)

Check if DLM supports ECC.

This function checks if both DLM and ECC are supported in the core.

Returns

1 if DLM supports ECC, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsXorErrorInjectMode (void)

Check if XOR error injection mode is supported.

This function reads the machine ECC code CSR and checks if error injection mode is supported.

Returns

1 if XOR error injection mode is enabled, 0 otherwise

__STATIC_FORCEINLINE void ECC_EnableICacheECC (void)

Enable ECC for I-Cache.

This function enables ECC for the instruction cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableICacheECC (void)

Disable ECC for I-Cache.

This function disables ECC for the instruction cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_EnableICacheECCExcp (void)

Enable ECC exception for I-Cache.

This function enables ECC exception for the instruction cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableICacheECCExcp (void)

Disable ECC exception for I-Cache.

This function disables ECC exception for the instruction cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_EnableICacheECCCheck (void)

Enable ECC checking for I-Cache.

This function enables ECC checking for the instruction cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableICacheECCCheck (void)

Disable ECC checking for I-Cache.

This function disables ECC checking for the instruction cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_EnableDCacheECC (void)

Enable ECC for D-Cache.

This function enables ECC for the data cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableDCacheECC (void)

Disable ECC for D-Cache.

This function disables ECC for the data cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_EnableDCacheECCExcp (void)

Enable ECC exception for D-Cache.

This function enables ECC exception for the data cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableDCacheECCExcp (void)

Disable ECC exception for D-Cache.

This function disables ECC exception for the data cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_EnableDCacheECCCheck (void)

Enable ECC checking for D-Cache.

This function enables ECC checking for the data cache by setting the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_DisableDCacheECCCheck (void)

Disable ECC checking for D-Cache.

This function disables ECC checking for the data cache by clearing the appropriate bit in the machine cache control CSR.

__STATIC_FORCEINLINE void ECC_ICacheTRamErrInject (uint32_t ecc_code, void *addr)

Inject error into I-Cache Tag RAM.

This function injects an error into the I-Cache Tag RAM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject
- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_ICacheDRamErrInject (uint32_t ecc_code, void *addr)

Inject error into I-Cache Data RAM.

This function injects an error into the I-Cache Data RAM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject

- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_ICacheErrRestore (void *addr)

Restore I-Cache error at specified address.

This function restores the correct ECC code for the I-Cache line at the specified address.

Parameters

- **addr** – [in] Address to restore

__STATIC_FORCEINLINE void ECC_DCacheTRamErrInject (uint32_t ecc_code, void *addr)

Inject error into D-Cache Tag RAM.

This function injects an error into the D-Cache Tag RAM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject
- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_DCacheDRamErrInject (uint32_t ecc_code, void *addr)

Inject error into D-Cache Data RAM.

This function injects an error into the D-Cache Data RAM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject
- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_DCacheErrRestore (void *addr)

Restore D-Cache error at specified address.

This function restores the correct ECC code for the D-Cache line at the specified address.

Parameters

- **addr** – [in] Address to restore

__STATIC_FORCEINLINE void ECC_EnableILM (void)

Enable ILM.

This function enables ILM by setting the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_DisableILM (void)

Disable ILM.

This function disables ILM by clearing the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_EnableILMECC (void)

Enable ECC for ILM.

This function enables ECC for ILM by setting the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_DisableILMECC (void)

Disable ECC for ILM.

This function disables ECC for ILM by clearing the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_EnableILMECCExcp (void)

Enable ECC exception for ILM.

This function enables ECC exception for ILM by setting the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_DisableILMECCExcp (void)

Disable ECC exception for ILM.

This function disables ECC exception for ILM by clearing the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_EnableILMECCCheck (void)

Enable ECC checking for ILM.

This function enables ECC checking for ILM by setting the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_DisableILMECCCheck (void)

Disable ECC checking for ILM.

This function disables ECC checking for ILM by clearing the appropriate bit in the machine ILM control CSR.

__STATIC_FORCEINLINE void ECC_ILMErrInject (uint32_t ecc_code, void *addr)

Inject error into ILM.

This function injects an error into the ILM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject
- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_ILMErrRestore (void *addr)

Restore ILM error at specified address.

This function restores the correct ECC code for the ILM at the specified address.

Parameters

addr – [in] Address to restore

__STATIC_FORCEINLINE void ECC_EnableDLM (void)

Enable DLM.

This function enables DLM by setting the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_DisableDLM (void)

Disable DLM.

This function disables DLM by clearing the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_EnableDLMECC (void)

Enable ECC for DLM.

This function enables ECC for DLM by setting the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_DisableDLMECC (void)

Disable ECC for DLM.

This function disables ECC for DLM by clearing the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_EnableDLMECCExcp (void)

Enable ECC exception for DLM.

This function enables ECC exception for DLM by setting the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_DisableDLMECCExcp (void)

Disable ECC exception for DLM.

This function disables ECC exception for DLM by clearing the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_EnableDLMECCCheck (void)

Enable ECC checking for DLM.

This function enables ECC checking for DLM by setting the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_DisableDLMECCCheck (void)

Disable ECC checking for DLM.

This function disables ECC checking for DLM by clearing the appropriate bit in the machine DLM control CSR.

__STATIC_FORCEINLINE void ECC_DLMErrInject (uint32_t ecc_code, void *addr)

Inject error into DLM.

This function injects an error into the DLM at the specified address with the given ECC code.

Parameters

- **ecc_code** – [in] ECC code to inject
- **addr** – [in] Address where error should be injected

__STATIC_FORCEINLINE void ECC_DLMErrRestore (void *addr)

Restore DLM error at specified address.

This function restores the correct ECC code for the DLM at the specified address.

Parameters**addr** – [in] Address to restore**__STATIC_FORCEINLINE int32_t ECC_IsAnySingleBitErrorOccured (void)**

Check if any single-bit error has occurred.

This function checks if any single-bit error has occurred by reading the machine ECC code CSR.

Returns

1 if any single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsICacheSingleBitErrorOccured (void)

Check if I-Cache single-bit error has occurred.

This function checks if a single-bit error has occurred in the I-Cache.

Returns

1 if I-Cache single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDCacheSingleBitErrorOccured (void)

Check if D-Cache single-bit error has occurred.

This function checks if a single-bit error has occurred in the D-Cache.

Returns

1 if D-Cache single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsTLBSingleBitErrorOccured (void)

Check if TLB single-bit error has occurred.

This function checks if a single-bit error has occurred in the TLB.

Returns

1 if TLB single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsILMSingleBitErrorOccured (void)

Check if ILM single-bit error has occurred.

This function checks if a single-bit error has occurred in the ILM.

Returns

1 if ILM single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDLMSingleBitErrorOccured (void)

Check if DLM single-bit error has occurred.

This function checks if a single-bit error has occurred in the DLM.

Returns

1 if DLM single-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE void ECC_ClearAllSingleBitError (void)

Clear all single-bit errors.

This function clears all single-bit errors by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearICacheSingleBitError (void)

Clear I-Cache single-bit error.

This function clears the single-bit error in the I-Cache by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearDCacheSingleBitError (void)

Clear D-Cache single-bit error.

This function clears the single-bit error in the D-Cache by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearTLBSingleBitError (void)

Clear TLB single-bit error.

This function clears the single-bit error in the TLB by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearILMSingleBitError (void)

Clear ILM single-bit error.

This function clears the single-bit error in the ILM by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearDLMSingleBitError (void)

Clear DLM single-bit error.

This function clears the single-bit error in the DLM by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE int32_t ECC_IsAnyDoubleBitErrorOccured (void)

Check if any double-bit error has occurred.

This function checks if any double-bit error has occurred by reading the machine ECC code CSR.

Returns

1 if any double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsICacheDoubleBitErrorOccured (void)

Check if I-Cache double-bit error has occurred.

This function checks if a double-bit error has occurred in the I-Cache.

Returns

1 if I-Cache double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDCacheDoubleBitErrorOccured (void)

Check if D-Cache double-bit error has occurred.

This function checks if a double-bit error has occurred in the D-Cache.

Returns

1 if D-Cache double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsTLBDoubleBitErrorOccured (void)

Check if TLB double-bit error has occurred.

This function checks if a double-bit error has occurred in the TLB.

Returns

1 if TLB double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsILMDoubleBitErrorOccured (void)

Check if ILM double-bit error has occurred.

This function checks if a double-bit error has occurred in the ILM.

Returns

1 if ILM double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE int32_t ECC_IsDLMDoubleBitErrorOccured (void)

Check if DLM double-bit error has occurred.

This function checks if a double-bit error has occurred in the DLM.

Returns

1 if DLM double-bit error has occurred, 0 otherwise

__STATIC_FORCEINLINE void ECC_ClearAllDoubleBitError (void)

Clear all double-bit errors.

This function clears all double-bit errors by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearICacheDoubleBitError (void)

Clear I-Cache double-bit error.

This function clears the double-bit error in the I-Cache by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearDCacheDoubleBitError (void)

Clear D-Cache double-bit error.

This function clears the double-bit error in the D-Cache by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearTLBDoubleBitError (void)

Clear TLB double-bit error.

This function clears the double-bit error in the TLB by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearILMDoubleBitError (void)

Clear ILM double-bit error.

This function clears the double-bit error in the ILM by clearing the appropriate bits in the machine ECC code CSR.

__STATIC_FORCEINLINE void ECC_ClearDLMDoubleBitError (void)

Clear DLM double-bit error.

This function clears the double-bit error in the DLM by clearing the appropriate bits in the machine ECC code CSR.

static uint8_t **ECC_GenerateECCCodeU32**(uint32_t a)

Generate ECC code for a 32-bit value.

This function calculates the ECC code for a 32-bit input value using Hamming code algorithm. It generates a 7-bit ECC code that can be used to detect and correct single-bit errors and detect double-bit errors.

Parameters

a – [in] 32-bit value for which ECC code is to be generated

Returns

8-bit ECC code (7 bits used, 1 bit unused)

static uint8_t **ECC_GenerateECCCodeU64**(uint64_t a)

Generate ECC code for a 64-bit value.

This function calculates the ECC for a 64-bit input value using Hamming code algorithm. It generates an 8-bit ECC code that can be used to detect and correct single-bit errors and detect double-bit errors.

Parameters

a – [in] 64-bit value for which ECC code is to be generated

Returns

8-bit ECC code

2.5.22 System Device Configuration

group **System Device Configuration**

Functions for system and clock setup available in `system_<device>.c`.

Nuclei provides a template file `system_Device.c` that must be adapted by the silicon vendor to match their actual device. As a **minimum requirement**, this file must provide:

- A device-specific system configuration function, *SystemInit* (page 805).
- Global c library *_premain_init* (page 809) and *_postmain_fini* (page 810) functions called right before calling main function.
 - A global variable that contains the system frequency, *SystemCoreClock* (page 810).
 - A global eclic configuration initialization, *ECLIC_Init*.
 - A global exception and trap configuration initialization, *Trap_Init* (page 809) and *Exception_Init* (page 815).
- Vendor customized interrupt, exception and nmi handling code, see *Interrupt and Exception and NMI Handling* (page 811)

The file configures the device and, typically, initializes the oscillator (PLL) that is part of the microcontroller device. This file might export other functions or variables that provide a more flexible configuration of the microcontroller system.

And this file also provided common interrupt, exception and NMI exception handling framework template, Silicon vendor can customize these template code as they want.

Attention

Be aware that a value stored to `SystemCoreClock` during low level initialization (i.e. `SystemInit()` (page 805)) might get overwritten by C library startup code and/or .bss section initialization. Thus its highly recommended to call `SystemCoreClockUpdate` (page 805) at the beginning of the user `main()` routine.

Note

Please pay special attention to the static variable `SystemCoreClock`. This variable might be used throughout the whole system initialization and runtime to calculate frequency/time related values. Thus one must assure that the variable always reflects the actual system clock speed.

Defines

CLINT_MSIP(base, hartid) (*(volatile uint32_t *)((uintptr_t)((base) + ((hartid) * 4))))

SMP_CTRLREG(base, ofs) (*(volatile uint32_t *)((uintptr_t)((base) + (ofs))))

Functions

void **exc_entry_s**(void)

void **irq_entry_s**(void)

default entry for s-mode non-vector irq entry

void **default_intexc_handler**(void)

default entry for s-mode exception entry

default eclic interrupt or exception interrupt handler

void **SystemCoreClockUpdate**(void)

Function to update the variable `SystemCoreClock` (page 810).

Updates the variable `SystemCoreClock` (page 810) and must be called whenever the core clock is changed during program execution. The function evaluates the clock register settings and calculates the current core clock.

void **SystemInit**(void)

Function to Initialize the system.

Initializes the microcontroller system. Typically, this function configures the oscillator (PLL) that is part of the microcontroller device. For systems with a variable clock speed, it updates the variable `SystemCoreClock` (page 810). `SystemInit` is called from the file `startup`.

void **SystemBannerPrint**(void)

Banner Print for Nuclei SDK.

void **irq_entry**(void)

void **exc_entry**(void)

void **ECLIC_Interrupt_Init**(void)

Do ECLIC Interrupt configuration.

This function will initialize cpu interrupt mode to eclic mode. It will

- set common non-vector entry to irq_entry
- set vector interrupt table to vector_base
- set exception entry to exc_entry
- set eclic mth to 0, and nlbits to the biggest bits it supports
- set s-mode common non-vector entry to irq_entry_s if tee present
- set s-mode vector interrupt table to vector_base_s if tee present
- set s-mode exception entry to exc_entry_s if tee present
- set eclic sth to 0 if tee present

void **CLINT_Interrupt_Init**(void)

Do CLINT Interrupt configuration.

This function will initialize cpu interrupt mode to clint mode. It will

- Set exception/interrupt entry to exc_entry, now interrupt and exception share the same entry point
- Register interrupt handling routine system_core_interrupt_handler to core_interrupt_handler function, which will be called in core_exception_handler function

void **PLIC_Interrupt_Init**(void)

Do PLIC Interrupt configuration.

This function will initialize cpu interrupt mode to clint/plic mode. It will

- Initialize a software maintained SystemM/SExtInterruptHandlers and SystemCoreInterruptHandlers to default value
- Set exception/interrupt entry to exc_entry, now interrupt and exception share the same entry point

void **Interrupt_Init**(void)

initialize interrupt controller

Do CPU interrupt initialization, if plic present, init it, then init eclic if present. So if ECLIC present, the interrupt will default configured to ECLIC interrupt mode, if you want to switch to PLIC interrupt mode, you need to call PLIC_Interrupt_Init in you application code.

By default, if ECLIC present, eclic interrupt mode will be set, otherwise it will be clint/plic interrupt mode

Remark
This function previously was ECLIC_Init, now ECLIC_Init is removed

int32_t **ECLIC_Register_IRQ**(*IRQn_Type* (page 636) *IRQn*, uint8_t *shv*, *ECLIC_TRIGGER_Type* (page 165) *trig_mode*, uint8_t *lvl*, uint8_t *priority*, void **handler*)

Initialize a specific IRQ and register the handler.

This function set vector mode, trigger mode and polarity, interrupt level and priority, assign handler for specific IRQn.

Remark

- This function use to configure specific eclic interrupt and register its interrupt handler and enable its interrupt.
- If the vector table is placed in read-only section(FLASHXIP mode), handler could not be installed

Parameters

- **IRQn** – [in] NMI interrupt handler address
- **shv** – [in] *ECLIC_NON_VECTOR_INTERRUPT* (page 164) means non-vector mode, and *ECLIC_VECTOR_INTERRUPT* (page 164) is vector mode
- **trig_mode** – [in] see *ECLIC_TRIGGER_Type* (page 165)
- **lvl** – [in] interrupt level
- **priority** – [in] interrupt priority
- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

int32_t **Core_Register_IRQ**(uint32_t irqn, void *handler)

Register a m-mode riscv core interrupt and register the handler.

This function set interrupt handler for core interrupt in non-eclic mode

Remark

- This function use to configure riscv core interrupt and register its interrupt handler and enable its interrupt.
- You can only use it when you are not in eclic interrupt mode

Parameters

- **irqn** – [in] interrupt number
- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

int32_t **Core_Register_IRQ_S**(uint32_t irqn, void *handler)

Register a riscv s-mode core interrupt and register the handler.

This function set interrupt handler for core interrupt in non-eclic mode

Remark

- This function use to configure riscv core interrupt and register its interrupt handler and enable its interrupt.
- You can only use it when you are not in eclic interrupt mode

Parameters

- **irqn** – [in] interrupt number
- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

int32_t **PLIC_Register_IRQ**(uint32_t source, uint8_t priority, void *handler)

Register a m-mode specific plic interrupt and register the handler.

This function set priority and handler for m-mode plic interrupt

Remark

- This function use to configure specific plic interrupt and register its interrupt handler and enable its interrupt.
- You can only use it when you are in plic interrupt mode

Parameters

- **source** – [in] interrupt source
- **priority** – [in] interrupt priority
- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

int32_t **PLIC_Register_IRQ_S**(uint32_t source, uint8_t priority, void *handler)

Register a s-mode specific plic interrupt and register the handler.

This function set priority and handler for s-mode plic interrupt

Remark

- This function use to configure specific plic interrupt and register its interrupt handler and enable its interrupt.
- You can only use it when you are in plic interrupt mode

Parameters

- **source** – [in] interrupt source
- **priority** – [in] interrupt priority

- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

int32_t **ECLIC_Register_IRQ_S**(*IRQn_Type* (page 636) IRQn, uint8_t shv, *ECLIC_TRIGGER_Type* (page 165) trig_mode, uint8_t lvl, uint8_t priority, void *handler)

Initialize a specific IRQ and register the handler for supervisor mode.

This function set vector mode, trigger mode and polarity, interrupt level and priority, assign handler for specific IRQn.

Remark

- This function use to configure specific eclic S-mode interrupt and register its interrupt handler and enable its interrupt.
- If the vector table is placed in read-only section (FLASHXIP mode), handler could not be installed.

Parameters

- **IRQn** – [in] NMI interrupt handler address
- **shv** – [in] *ECLIC_NON_VECTOR_INTERRUPT* (page 164) means non-vector mode, and *ECLIC_VECTOR_INTERRUPT* (page 164) is vector mode
- **trig_mode** – [in] see *ECLIC_TRIGGER_Type* (page 165)
- **lvl** – [in] interrupt level
- **priority** – [in] interrupt priority
- **handler** – [in] interrupt handler, if NULL, handler will not be installed

Returns

-1 means invalid input parameter. 0 means successful.

void **__sync_harts**(void)

Synchronize all harts.

This function is used to synchronize all the harts, especially to wait the boot hart finish initialization of data section, bss section and c runtimes initialization. This function must be placed in .text.init section, since section initialization is not ready, global variable and static variable should be avoid to use in this function, and avoid to call other functions.

static void **Trap_Init**(void)

do the init for trap

void **_premain_init**(void)

early init function before main

This function is executed right before main function. For RISC-V gnu toolchain, _init function might not be called by __libc_init_array function, so we defined a new function to do initialization.

void **_postmain_fini**(int status)
finish function after main

This function is executed right after main function. For RISC-V gnu toolchain, _fini function might not be called by __libc_fini_array function, so we defined a new function to do initialization

Parameters

status – [in] status code return from main

void **_init**(void)
_init function called in __libc_init_array()

This __libc_init_array() function is called during startup code, user need to implement this function, otherwise when link it will error init.c:(.text.__libc_init_array+0x26): undefined reference to _init

Note

Please use [_premain_init](#) (page 809) function now

void **_fini**(void)
_fini function called in __libc_fini_array()

This __libc_fini_array() function is called when exit main. user need to implement this function, otherwise when link it will error fini.c:(.text.__libc_fini_array+0x28): undefined reference to _fini

Note

Please use [_postmain_fini](#) (page 810) function now

Variables

uint32_t **SystemCoreClock** = SYSTEM_CLOCK

Variable to hold the system core clock value.

Holds the system core clock, which is the system clock frequency supplied to the SysTick timer and the processor core clock. This variable can be used by debuggers to query the frequency of the debug timer or to configure the trace clock speed.

Attention

Compilers must be configured to avoid removing this variable in case the application program is not using it. Debugging systems require the variable to be physically present in memory so that it can be examined to configure the debugger.

unsigned long **vector_base**[]

unsigned long **CpuIRegionBase** = 0xFFFFFFFF

Nuclei RISC-V CPU IRegion Base Address Probed, you should avoid to use it in your application code, please use __IREGION_BASEADDR if you want.

Interrupt Exception NMI Handling

group **Interrupt and Exception and NMI Handling**

Functions for interrupt, exception and nmi handle available in system_<device>.c.

Nuclei provide a template for interrupt, exception and NMI handling. Silicon Vendor could adapt according to their requirement. Silicon vendor could implement interface for different exception code and replace current implementation.

Defines

MAX_SYSTEM_EXCEPTION_NUM 20

Max exception handler number, don't include the NMI(0xFFFF) one.

SYSTEM_CORE_INTNUM 16

Typedefs

typedef void (***EXC_HANDLER**)(unsigned long cause, unsigned long sp)

Exception Handler Function Typedef.

Note

This typedef is only used internal in this system_<Device>.c file. It is used to do type conversion for registered exception handler before calling it.

typedef void (***INT_HANDLER**)(unsigned long cause, unsigned long sp)

Functions

static void **system_mmode_extirq_handler**(unsigned long exccode, unsigned long sp)

M-Mode external interrupt handler common entry for plic interrupt mode.

This function provide common entry for m-mode external interrupt for plic interrupt mode.

Parameters

- **exccode** – [in] exception code indicating the reason that caused the trap in machine mode
- **sp** – [in] stack pointer

static void **core_interrupt_handler**(unsigned long exccode, unsigned long sp)

M-Mode Common Interrupt handler entry when in clint/plic mode.

This function provided a command entry for interrupt in clint/plic mode

Remark

- This is not used for clic interrupt mode, which is only used for clint/plic interrupt mode, you should call *CLINT_Interrupt_Init* (page 806) or *PLIC_Interrupt_Init* (page 806) first to make sure this handler entry registered
- If you are not in eclic interrupt mode, please use *Interrupt_Register_CoreIRQ* (page 813) to register internal interrupt and use *Interrupt_Register_ExtIRQ* (page 814) to register external interrupt

Parameters

- **exccode** – [in] Exception Code
- **sp** – [in] stack pointer

uint32_t **core_exception_handler**(unsigned long mcause, unsigned long sp)

M-Mode Common NMI/Exception/Interrupt handler entry.

This function provided a command entry for NMI and exception. Silicon Vendor could modify this template implementation according to requirement.

Remark

- RISC-V provided common entry for all types of exception and interrupt if not in eclic mode. This is proposed code template for exception entry function, Silicon Vendor could modify the implementation.
- For the *core_exception_handler* template, we provided exception register function *Exception_Register_EXC* (page 814) which can help developer to register your exception handler for specific exception number.
- If you are in eclic interrupt mode, please use *ECLIC_Register_IRQ* (page 806) to register both internal and external interrupt
- If you are not in eclic interrupt mode, please use *Interrupt_Register_CoreIRQ* (page 813) to register internal interrupt and use *Interrupt_Register_ExtIRQ* (page 814) to register external interrupt

Parameters

- **mcause** – [in] code indicating the reason that caused the trap in machine mode
- **sp** – [in] stack pointer

static void **system_default_interrupt_handler_s**(unsigned long scause, unsigned long sp)

s-mode System Default Interrupt Handler for CLINT/PLIC Interrupt Mode

This function provided a default interrupt handling code for all interrupt ids.

static void **system_smode_extirq_handler**(unsigned long exccode, unsigned long sp)

S-Mode external interrupt handler common entry for plic interrupt mode.

This function provide common entry for s-mode external interrupt for plic interrupt mode.

Parameters

- **exccode** – [in] exception code indicating the reason that caused the trap in supervisor mode

- **sp** – [in] stack pointer

static void **core_interrupt_handler_s**(unsigned long exccode, unsigned long sp)

S-Mode Common Interrupt handler entry when in clint/plic mode.

This function provided a command entry for interrupt in clint/plic mode

Remark

- This is not used for clic interrupt mode, which is only used for clint/plic interrupt mode, you should call *CLINT_Interrupt_Init* (page 806) or *PLIC_Interrupt_Init* (page 806) first to make sure this handler entry registered
- If you are not in eclic interrupt mode, please use please use *Interrupt_Register_CoreIRQ* (page 813) to register internal interrupt and use *Interrupt_Register_ExtIRQ* (page 814) to register external interrupt

Parameters

- **exccode** – [in] Exception Code
- **sp** – [in] stack pointer

void **Exception_DumpFrame**(unsigned long sp, uint8_t mode)

Dump Exception Frame.

This function provided feature to dump exception frame stored in stack.

Parameters

- **sp** – [in] stackpoint
- **mode** – [in] privileged mode to decide whether to dump msubm CSR

static void **system_default_interrupt_handler**(unsigned long mcause, unsigned long sp)

M-Mode System Default Interrupt Handler for CLINT/PLIC Interrupt Mode.

This function provided a default interrupt handling code for all interrupt ids.

void **Interrupt_Register_CoreIRQ**(uint32_t irqn, unsigned long int_handler)

Register a m-mode core interrupt handler for core interrupt number.

- For irqn <= SYSTEM_CORE_INTNUM, it will be registered into SystemCoreInterruptHandlers[irqn-1], only used in non-eclic mode.

Remark

You can only use it when you are not in ECLIC interrupt mode.

Parameters

- **irqn** – See IRQn
- **int_handler** – The core interrupt handler for this interrupt code irqn

unsigned long **Interrupt_Get_CoreIRQ**(uint32_t irqn)

Get a m-mode core interrupt handler for core interrupt number.

Remark

You can only use it when you are not in ECLIC interrupt mode.

Parameters

irqn – See IRQn

Returns

The core interrupt handler for this interrupt code irqn, only used in non-eclic mode.

void **Interrupt_Register_ExtIRQ**(uint32_t irqn, unsigned long int_handler)

Register a m-mode external interrupt handler for plic external interrupt number.

- For irqn <= __PLIC_INTNUM, it will be registered into SystemMExtInterruptHandlers[irqn-1].

Remark

You can only use it when you are in PLIC interrupt mode.

Parameters

- **irqn** – See IRQn
- **int_handler** – The external interrupt handler for this interrupt code irqn

unsigned long **Interrupt_Get_ExtIRQ**(uint32_t irqn)

Get a m-mode external interrupt handler for external interrupt number.

Remark

You can only use it when you are in PLIC interrupt mode.

Parameters

irqn – See IRQn

Returns

The external interrupt handler for this interrupt code irqn

void **Exception_Register_EXC**(uint32_t EXCn, unsigned long exc_handler)

Register a m-mode exception handler for exception code EXCn.

- For EXCn < [MAX_SYSTEM_EXCEPTION_NUM](#) (page 811), it will be registered into SystemExceptionHandler[EXCn-1].

- For `EXCn == NMI_EXCn`, it will be registered into `SystemExceptionHandlers[MAX_SYSTEM_EXCEPTION_NUM]`.

Parameters

- **EXCn** – [in] See `EXCn_Type`
- **exc_handler** – [in] The exception handler for this exception code `EXCn`

unsigned long **Exception_Get_EXC**(uint32_t EXCn)

Get current m-mode exception handler for exception code `EXCn`.

- For `EXCn < MAX_SYSTEM_EXCEPTION_NUM` (page 811), it will return `SystemExceptionHandlers[EXCn-1]`.
- For `EXCn == NMI_EXCn`, it will return `SystemExceptionHandlers[MAX_SYSTEM_EXCEPTION_NUM]`.

Parameters

EXCn – [in] See `EXCn_Type`

Returns

Current exception handler for exception code `EXCn`, if not found, return 0.

static void **Exception_Init**(void)

Initialize all the default core exception handlers.

The core exception handler for each exception id will be initialized to *system_default_exception_handler* (page 817).

Note

Called in *_init* (page 810) function, used to initialize default exception handlers for all exception IDs `SystemExceptionHandlers` contains `NMI`, but `SystemExceptionHandlers_S` not, because `NMI` can't be delegated to S-mode.

uint32_t **core_exception_handler_s**(unsigned long scause, unsigned long sp)

common Exception handler entry of supervisor mode

This function provided a supervisor mode common entry for exception. Silicon Vendor could modify this template implementation according to requirement.

Remark

- RISC-V provided supervisor mode common entry for all types of exception. This is proposed code template for exception entry function, Silicon Vendor could modify the implementation.
- For the `core_exception_handler_s` template, we provided exception register function *Exception_Register_EXC_S* (page 816) which can help developer to register your exception handler for specific exception number.

Parameters

- **scause** – [in] code indicating the reason that caused the trap in supervisor mode
- **sp** – [in] stack pointer

void **Exception_Register_EXC_S**(uint32_t EXCn, unsigned long exc_handler)

Register an exception handler for exception code EXCn of supervisor mode.

-For EXCn < *MAX_SYSTEM_EXCEPTION_NUM* (page 811), it will be registered into SystemExceptionHandlers_S[EXCn-1]. -For EXCn == NMI_EXCn, The NMI (Non-maskable-interrupt) cannot be trapped to the supervisor-mode or user-mode for any configuration, so NMI won't be registered into SystemExceptionHandlers_S.

Parameters

- **EXCn** – [in] See EXCn_Type
- **exc_handler** – [in] The exception handler for this exception code EXCn

unsigned long **Exception_Get_EXC_S**(uint32_t EXCn)

Get current exception handler for exception code EXCn of supervisor mode.

- For EXCn < *MAX_SYSTEM_EXCEPTION_NUM* (page 811), it will return SystemExceptionHandlers_S[EXCn-1].

Parameters

EXCn – [in] See EXCn_Type

Returns

Current exception handler for exception code EXCn, if not found, return 0.

void **Interrupt_Register_CoreIRQ_S**(uint32_t irqn, unsigned long int_handler)

Register an s-mode core interrupt handler for core interrupt number.

- For irqn <= SYSTEM_CORE_INTNUM, it will be registered into SystemCoreInterruptHandlers[irqn-1], only used in non-ecltic mode.

Remark

You can only use it when you are not in ECLIC interrupt mode.

Parameters

- **irqn** – See IRQn
- **int_handler** – The core interrupt handler for this interrupt code irqn

unsigned long **Interrupt_Get_CoreIRQ_S**(uint32_t irqn)

Get a s-mode core interrupt handler for core interrupt number.

Remark

You can only use it when you are not in ECLIC interrupt mode.

Parameters**irqn** – See IRQn**Returns**

The core interrupt handler for this interrupt code irqn, only used in non-eclic mode.

void **Interrupt_Register_ExtIRQ_S**(uint32_t irqn, unsigned long int_handler)

Register an s-mode external interrupt handler for plic external interrupt number.

- For irqn <= __PLIC_INTNUM, it will be registered into SystemSExtInterruptHandlers[irqn-1].

Remark

You can only use it when you are in PLIC interrupt mode.

Parameters

- **irqn** – See IRQn
- **int_handler** – The external interrupt handler for this interrupt code irqn

unsigned long **Interrupt_Get_ExtIRQ_S**(uint32_t irqn)

Get an s-mode external interrupt handler for external interrupt number.

Remark

You can only use it when you are in PLIC interrupt mode.

Parameters**irqn** – See IRQn**Returns**

The external interrupt handler for this interrupt code irqn

static void **system_default_exception_handler**(unsigned long mcause, unsigned long sp)

M-Mode System Default Exception Handler.

This function provides a default exception and NMI handler for all exception ids. By default, It will just print some information for debug, Vendor can customize it according to its requirements.

Parameters

- **mcause** – [in] code indicating the reason that caused the trap in machine mode
- **sp** – [in] stack pointer

static void **system_default_exception_handler_s**(unsigned long scause, unsigned long sp)

Supervisor mode system Default Exception Handler.

This function provided a default supervisor mode exception and NMI handling code for all exception ids. By default, It will just print some information for debug, Vendor can customize it according to its requirements.

Parameters

- **scause** – [in] code indicating the reason that caused the trap in supervisor mode
- **sp** – [in] stack pointer

Variables

static unsigned long **SystemExceptionHandlers**[MAX_SYSTEM_EXCEPTION_NUM + 1]

Store the exception handlers for each exception ID.

Note

- This SystemExceptionHandlers are used to store all the handlers for all the exception codes Nuclei N/NX core provided.
- Exception code 0 - 19, totally 20 exceptions are mapped to SystemExceptionHandlers[0:19]
- Exception for NMI is also re-routed to exception handling(exception code 0xFFFF) in startup code configuration, the handler itself is mapped to SystemExceptionHandlers[MAX_SYSTEM_EXCEPTION_NUM]

static unsigned long **SystemMExtInterruptHandlers**[__PLIC_INTNUM]

static unsigned long **SystemCoreInterruptHandlers**[SYSTEM_CORE_INTNUM]

static *INT_HANDLER* (page 811) **system_core_interrupt_handler** = NULL

static unsigned long **SystemExceptionHandlers_S**[MAX_SYSTEM_EXCEPTION_NUM]

Store the exception handlers for each exception ID in supervisor mode.

Note

- This SystemExceptionHandlers_S are used to store all the handlers for all the exception codes Nuclei N/NX core provided.
- Exception code 0 - 19, totally 20 exceptions are mapped to SystemExceptionHandlers_S[0:19]

static *INT_HANDLER* (page 811) **system_core_interrupt_handler_s** = NULL

static unsigned long **SystemCoreInterruptHandlers_S**[SYSTEM_CORE_INTNUM]

static unsigned long **SystemSExtInterruptHandlers**[__PLIC_INTNUM]

2.5.23 ARM Compatible Functions

group **ARM Compatible Functions**

A few functions that compatible with ARM CMSIS-Core.

Here we provided a few functions that compatible with ARM CMSIS-Core, mostly used in the DSP and NN library.

Defines

__ISB() *__RWMB* (page 76)()

Instruction Synchronization Barrier, compatible with ARM.

__DSB() *__RWMB* (page 76)()

Data Synchronization Barrier, compatible with ARM.

__DMB() *__RWMB* (page 76)()

Data Memory Barrier, compatible with ARM.

__LDRBT(ptr) *__LB*((ptr))

LDRT Unprivileged (8 bit), ARM Compatible.

__LDRHT(ptr) *__LH*((ptr))

LDRT Unprivileged (16 bit), ARM Compatible.

__LDRT(ptr) *__LW*((ptr))

LDRT Unprivileged (32 bit), ARM Compatible.

__STRBT(val, ptr) *__SB*((ptr), (val))

STRT Unprivileged (8 bit), ARM Compatible.

__STRHT(val, ptr) *__SH*((ptr), (val))

STRT Unprivileged (16 bit), ARM Compatible.

__STRT(val, ptr) *__SW*((ptr), (val))

STRT Unprivileged (32 bit), ARM Compatible.

__SSAT(val, sat) *__RV_SCLIP32*((val), (sat-1))

Signed Saturate.

Saturates a signed value.

Parameters

- **value** – [in] Value to be saturated
- **sat** – [in] Bit position to saturate to (1..32)

Returns

Saturated value

__USAT(val, sat) *__RV_UCLIP32*((val), (sat))

Unsigned Saturate.

Saturates an unsigned value.

Parameters

- **value** – [in] Value to be saturated

- **sat** – [in] Bit position to saturate to (0..31)

Returns

Saturated value

__RBIT(value) **__RV_BITREVI**((value), 31)

Reverse bit order of value.

Reverses the bit order of the given value.

Parameters

- **value** – [in] Value to reverse

Returns

Reversed value

__CLZ(data) **__RV_CLZ32**(data)

Count leading zeros.

Counts the number of leading zeros of a data value.

Parameters

- **data** – [in] Value to count the leading zeros

Returns

number of leading zeros in value

__EXPD_BYTE(x)

Expand byte to unsigned long value.

Expand byte value x to unsigned long value's each byte.

Parameters

- **x** – [in] the byte value to be expand, the input must be uint8_t type

Returns

Expanded value in unsigned long

Functions

__STATIC_FORCEINLINE uint32_t **__REV** (uint32_t value)

Reverse byte order (32 bit)

Reverses the byte order in unsigned integer value. For example, 0x12345678 becomes 0x78563412.

Parameters

value – [in] Value to reverse

Returns

Reversed value

__STATIC_FORCEINLINE uint32_t **__REV16** (uint32_t value)

Reverse byte order (16 bit)

Reverses the byte order within each halfword of a word. For example, 0x12345678 becomes 0x34127856.

Parameters

value – [in] Value to reverse

Returns

Reversed value

__STATIC_FORCEINLINE int16_t __REVSH (int16_t value)

Reverse byte order (16 bit)

Reverses the byte order in a 16-bit value and returns the signed 16-bit result. For example, 0x0080 becomes 0x8000.

Parameters**value** – [in] Value to reverse**Returns**

Reversed value

__STATIC_FORCEINLINE uint32_t __ROR (uint32_t op1, uint32_t op2)

Rotate Right in unsigned value (32 bit)

Rotate Right (immediate) provides the value of the contents of a register rotated by a variable number of bits.

Parameters

- **op1** – [in] Value to rotate
- **op2** – [in] Number of Bits to rotate(0-31)

Returns

Rotated value

__STATIC_INLINE uint64_t __ROR64 (uint64_t op1, uint32_t op2)

Rotate Right in uint32x2 value (64 bit)

Rotate Right (immediate) provides the value of the contents of a register rotated by a variable number of bits.

Parameters

- **op1** – [in] Value to rotate([63:32] and [31:0] rotate separately)
- **op2** – [in] Number of Bits to rotate

Returns

Rotated value([63:32] | [31:0])

__STATIC_INLINE unsigned long __CTZ (unsigned long data)

Count trailing zero.

Return the count of least-significant bit zero.for example, return 3 if x=0bxxx1000

Remark

- The value mustn't be 0, or else it will spin here

Parameters**data** – [in] Value to count the trailing zeros

Returns

number of trailing zeros in value

2.5.24 NMSIS Bench and Test Helper Functions

group **NMSIS Bench and Test Related Helper Functions**

Functions that used to do benchmark and test suite.

NMSIS benchmark and test related helper functions are provided to help do benchmark and test case pass/fail assertion.

If you want to do calculate cpu cycle cost of a process, you can use `BENCH_xxx` macros defined in this.

In a single c source code file, you should include `nmsis_bench.h`, and then you should place `BENCH_DECLARE_VAR()` (page 822); before call other `BENCH_xxx` macros. If you want to start to do benchmark, you should only call `BENCH_INIT()` (page 822); once in your source code, and then place `BENCH_START(proc_name)` (page 822); and `BENCH_END(proc_name)` (page 822) before and after the process you want to measure. You can refer to `<nuclei-sdk>/application/baremetal/demo_dsp` for how to use it.

If you want to disable the benchmark calculation, you can place `#define DISABLE_NMSIS_BENCH` before include `nmsis_bench.h`

If in your c test source code, you can add `NMSIS_TEST_PASS()` (page 829); and `NMSIS_TEST_FAIL()` (page 829); to mark c test is pass or fail.

Defines

READ_CYCLE `__get_rv_cycle` (page 82)

When `XLEN=32`, reading the full 64-bit `CYCLE` register incurs additional overhead.

`BENCH_XLEN_MODE` skips reading the upper 32 bits, reducing the extra cycle cost and allowing for more accurate measurements of small cycle counts.

NOTE: It is only applicable when the total cycle count does not exceed 2^{32} . Read the whole 64 bits value of `MCYCLE` register

BENCH_DECLARE_VAR()

Declare benchmark required variables, need to be placed above all `BENCH_xxx` macros in each c source code if `BENCH_xxx` used.

BENCH_INIT()

Initialize benchmark environment, need to called in before other `BENCH_xxx` macros are called.

BENCH_RESET(proc) `_bc_sumcyc = 0; _bc_usecyc = 0; _bc_lpcnt = 0; _bc_ercd = 0;`

Reset benchmark sum cycle and use cycle for proc.

BENCH_START(proc)

Start to do benchmark for proc, and record start cycle, and reset error code.

BENCH_SAMPLE(proc)

Sample a benchmark for proc, and record this start -> sample cost cycle, and accumulate it to sum cycle.

BENCH_END(proc)

Mark end of benchmark for proc, and calc used cycle, and print it.

BENCH_STOP(proc) printf("CSV, %s, %lu\n", #proc, (unsigned long)_bc_sumcyc);
 Mark stop of benchmark, start -> sample -> sample -> stop, and print the sum cycle of a proc.

BENCH_STAT(proc) printf("STAT, %s, %lu, %lu\n", #proc, (unsigned long)_bc_lpcnt, (unsigned long)_bc_sumcyc);
 Show statistics of benchmark, format: STAT, proc, loopcnt, sumcyc.

BENCH_GET_USECYC() (_bc_usecyc)
 Get benchmark use cycle.

BENCH_GET_SUMCYC() (_bc_sumcyc)
 Get benchmark sum cycle.

BENCH_GET_LPCNT() (_bc_lpcnt)
 Get benchmark loop count.

BENCH_ERROR(proc) _bc_ercd = 1;
 Mark benchmark for proc is errored.

BENCH_STATUS(proc)
 Show the status of the benchmark.

EVENT_SEL_INSTRUCTION_COMMIT 0

EVENT_SEL_MEMORY_ACCESS 1

EVENT_SEL_TYPE_0 0

EVENT_SEL_TYPE_1 1

EVENT_SEL_TYPE_2 2

EVENT_SEL_TYPE_3 3

EVENT_INSTRUCTION_COMMIT_CYCLE_COUNT 1

EVENT_INSTRUCTION_COMMIT_RETIRED_COUNT 2

EVENT_INSTRUCTION_COMMIT_INTEGER_LOAD 3

EVENT_INSTRUCTION_COMMIT_INTEGER_STORE 4

EVENT_INSTRUCTION_COMMIT_ATOMIC_MEMORY_OPERATION 5

EVENT_INSTRUCTION_COMMIT_SYSTEM 6

EVENT_INSTRUCTION_COMMIT_INTEGER_COMPUTATIONAL 7

EVENT_INSTRUCTION_COMMIT_CONDITIONAL_BRANCH 8

EVENT_INSTRUCTION_COMMIT_TAKEN_CONDITIONAL_BRANCH 9

EVENT_INSTRUCTION_COMMIT_JAL 10

EVENT_INSTRUCTION_COMMIT_JALR 11

EVENT_INSTRUCTION_COMMIT_RETURN 12

EVENT_INSTRUCTION_COMMIT_CONTROL_TRANSFER 13

EVENT_INSTRUCTION_COMMIT_FENCE_INSTRUCTION 14

EVENT_INSTRUCTION_COMMIT_INTEGER_MULTIPLICATION 15

EVENT_INSTRUCTION_COMMIT_INTEGER_DIVISION_REMAINDER 16

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_LOAD 17

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_STORE 18

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_ADDITION_SUBTRACTION 19

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_MULTIPLICATION 20

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_FUSED_MULTIPLY_ADD_SUB 21

EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_DIVISION_OR_SQUARE_ROOT 22

EVENT_INSTRUCTION_COMMIT_OTHER_FLOATING_POINT_INSTRUCTION 23

EVENT_INSTRUCTION_COMMIT_CONDITIONAL_BRANCH_PREDICTION_FAIL 24

EVENT_INSTRUCTION_COMMIT_JALR_PREDICTION_FAIL 25

EVENT_INSTRUCTION_COMMIT_POP_PREDICTION_FAIL 26

EVENT_INSTRUCTION_COMMIT_FENCEI_INSTRUCTION 27

EVENT_INSTRUCTION_COMMIT_SFENCE_INSTRUCTION 28

EVENT_INSTRUCTION_COMMIT_ECALL_INSTRUCTION 29

EVENT_INSTRUCTION_COMMIT_EXCEPTION_INSTRUCTION 30

EVENT_INSTRUCTION_COMMIT_INTERRUPT_INSTRUCTION 31

EVENT_MEMORY_ACCESS_ICACHE_MISS 1

EVENT_MEMORY_ACCESS_DCACHE_MISS 2

EVENT_MEMORY_ACCESS_ITLB_MISS 3

EVENT_MEMORY_ACCESS_DTLB_MISS 4

EVENT_MEMORY_ACCESS_MAIN_DTLB_MISS 5

EVENT_MEMORY_ACCESS_MAIN_TLB_MISS 5

EVENT_MEMORY_ACCESS_L2_CACHE_ACCESS 8

EVENT_MEMORY_ACCESS_L2_CACHE_MISS 9

EVENT_MEMORY_ACCESS_MEMORY_BUS_REQUEST 10

EVENT_MEMORY_ACCESS_IFU_STALL_CYCLE 11

EVENT_MEMORY_ACCESS_EXU_STALL_CYCLE 12

EVENT_MEMORY_ACCESS_TIMER 13

EVENT_TYPE_0_CYCLE_COUNT 1

EVENT_TYPE_0_RETIRED_COUNT 2

EVENT_TYPE_0_INTEGER_LOAD 3

EVENT_TYPE_0_INTEGER_STORE 4

EVENT_TYPE_0_ATOMIC_MEMORY_OPERATION 5

EVENT_TYPE_0_SYSTEM 6

EVENT_TYPE_0_INTEGER_COMPUTATIONAL 7

EVENT_TYPE_0_CONDITIONAL_BRANCH 8

EVENT_TYPE_0_TAKEN_CONDITIONAL_BRANCH 9

EVENT_TYPE_0_JAL 10

EVENT_TYPE_0_JALR 11

EVENT_TYPE_0_RETURN 12

EVENT_TYPE_0_CONTROL_TRANSFER 13

EVENT_TYPE_0_FENCE_INSTRUCTION 14

EVENT_TYPE_0_INTEGER_MULTIPLICATION 15

EVENT_TYPE_0_INTEGER_DIVISION_REMAINDER 16

EVENT_TYPE_0_FLOATING_POINT_LOAD 17

EVENT_TYPE_0_FLOATING_POINT_STORE 18

EVENT_TYPE_0_FLOATING_POINT_ADDITION_SUBTRACTION 19

EVENT_TYPE_0_FLOATING_POINT_MULTIPLICATION 20

EVENT_TYPE_0_FLOATING_POINT_FUSED_MULTIPLY_ADD_SUB 21

EVENT_TYPE_0_FLOATING_POINT_DIVISION_OR_SQUARE_ROOT 22

EVENT_TYPE_0_OTHER_FLOATING_POINT_INSTRUCTION 23

EVENT_TYPE_0_CONDITIONAL_BRANCH_PREDICTION_FAIL 24

EVENT_TYPE_0_JALR_PREDICTION_FAIL 25

EVENT_TYPE_0_POP_PREDICTION_FAIL 26

EVENT_TYPE_0_FENCEI_INSTRUCTION 27

EVENT_TYPE_0_SFENCE_INSTRUCTION 28

EVENT_TYPE_0_ECALL_INSTRUCTION 29

EVENT_TYPE_0_EXCEPTION_INSTRUCTION 30

EVENT_TYPE_0_INTERRUPT_INSTRUCTION 31

EVENT_TYPE_1_ICACHE_READ_MISS 1

EVENT_TYPE_1_DCACHE_RW_MISS 2

EVENT_TYPE_1_ITLB_READ_MISS 3

EVENT_TYPE_1_DTLB_RW_MISS 4

EVENT_TYPE_1_MAIN_TLB_MISS 5

EVENT_TYPE_1_L2_CACHE_ACCESS 8

EVENT_TYPE_1_L2_CACHE_MISS 9

EVENT_TYPE_1_MEMORY_BUS_REQUEST 10

EVENT_TYPE_1_IFU_STALL_CYCLE 11

EVENT_TYPE_1_EXU_STALL_CYCLE 12

EVENT_TYPE_1_TIMER 13

EVENT_TYPE_2_BRANCH_INSTRUCTION_COMMIT 2

EVENT_TYPE_2_BRANCH_PREDICT_FAIL_COMMIT 3

EVENT_TYPE_3_DCACHE_READ 0

EVENT_TYPE_3_DCACHE_READ_MISS 1

EVENT_TYPE_3_DCACHE_WRITE 2

EVENT_TYPE_3_DCACHE_WRITE_MISS 3

EVENT_TYPE_3_DCACHE_PREFETCH 4

EVENT_TYPE_3_DCACHE_PREFETCH_MISS 5

EVENT_TYPE_3_ICACHE_READ 6

EVENT_TYPE_3_ICACHE_PREFETCH 8

EVENT_TYPE_3_ICACHE_PREFETCH_MISS 9

EVENT_TYPE_3_L2_CACHE_READ 10

EVENT_TYPE_3_L2_CACHE_READ_MISS 11

EVENT_TYPE_3_L2_CACHE_WRITE 12

EVENT_TYPE_3_L2_CACHE_WRITE_MISS 13

EVENT_TYPE_3_L2_CACHE_PREFETCH_HIT 14

EVENT_TYPE_3_L2_CACHE_PREFETCH_MISS 15

EVENT_TYPE_3_DTLB_READ 16

EVENT_TYPE_3_DTLB_READ_MISS 17

EVENT_TYPE_3_DTLB_WRITE 18

EVENT_TYPE_3_DTLB_WRITE_MISS 19

EVENT_TYPE_3_ITLB_READ 20

EVENT_TYPE_3_BTB_READ 22

EVENT_TYPE_3_BTB_READ_MISS 23

EVENT_TYPE_3_BTB_WRITE 24

EVENT_TYPE_3_BTB_WRITE_MISS 25

MSU_EVENT_ENABLE 0x0F

MEVENT_EN 0x08

SEVENT_EN 0x02

UEVENT_EN 0x01

READ_HPM_COUNTER __get_hpm_counter

HPM_DECLARE_VAR(idx)

Declare high performance monitor counter idx benchmark required variables, need to be placed above all HPM_xxx macros in each c source code if HPM_xxx used.

HPM_SEL_ENABLE(ena) (ena << 28)

HPM_SEL_EVENT(sel, idx) ((sel) | (idx << 4))

HPM_EVENT(sel, idx, ena) (*HPM_SEL_ENABLE* (page 829)(ena) | *HPM_SEL_EVENT* (page 829)(sel, idx))

Construct a event variable to be set(sel -> event_sel, idx -> event_idx, ena -> m/s/u_enable)

HPM_INIT()

Initialize high performance monitor environment, need to called in before other HPM_xxx macros are called.

HPM_RESET(idx, proc, event) __hpm_sumcyc##idx = 0; __hpm_lpcnt##idx = 0;

Reset high performance benchmark for proc using counter which index is idx.

HPM_START(idx, proc, event)

Start to do high performance benchmark for proc, and record start hpm counter.

HPM_SAMPLE(idx, proc, event)

Do high performance benchmark sample for proc, and sum it into sum counter.

HPM_END(idx, proc, event)

Mark end of high performance benchmark for proc, and calc used hpm counter value.

HPM_STOP(idx, proc, event) printf("HPM%d:0x%x, %s, %lu\n", idx, event, #proc, (unsigned long)__hpm_sumcyc##idx);

Mark stop of hpm benchmark, start -> sample -> sample -> stop, and print the sum cycle of a proc.

HPM_STAT(idx, proc, event) printf("STATHPM%d:0x%x, %s, %lu, %lu\n", idx, event, #proc, (unsigned long)__hpm_lpcnt##idx, (unsigned long)__hpm_sumcyc##idx);

Show statistics of hpm benchmark, format: STATHPM::idx:event, proc, loopcnt, sumcyc.

HPM_GET_USECYC(idx) (__hpm_usecyc##idx)

Get hpm benchmark use cycle for counter idx.

HPM_GET_SUMCYC(idx) (__hpm_sumcyc##idx)

Get hpm benchmark sum cycle for counter idx.

HPM_GET_LPCNT(idx) (__hpm_lpcnt##idx)

Get hpm benchmark loop count for counter idx.

NMSIS_TEST_PASS() printf("\nNMSIS_TEST_PASS\n");

Mark test or application passed.

NMSIS_TEST_FAIL() printf("\nNMSIS_TEST_FAIL\n");

Mark test or application failed.

Functions

__STATIC_FORCEINLINE void __prepare_bench_env (void)

Prepare benchmark environment.

Prepare benchmark required environment, such as turn on necessary units like vpu, cycle, instret counters, hpm counters

3.1 Overview

3.1.1 Introduction

This user manual describes the NMSIS DSP software library, a suite of common signal processing functions for use on Nuclei N/NX/UX Class Processors based devices.

The library is divided into a number of functions each covering a specific category:

- Basic math functions
- Bayes functions
- Complex math functions
- Controller functions
- Distance functions
- Fast math functions
- Filtering functions
- Interpolation functions
- Matrix functions
- Quaternion math functions
- Statistical functions
- Support functions
- SVM functions
- Transform functions
- Window functions

The library has separate functions for operating on 8-bit integers, 16-bit integers, 32-bit integers and 32-bit floating-point values.

3.1.2 Using the Library

The library functions are declared in the public file `riscv_math.h` which is placed in the `NMSIS/DSP/Include` and `NMSIS/DSP/PrivateInclude` folder.

Simply include this file and link the appropriate library in the application and begin calling the library functions.

The Library supports single public header file `riscv_math.h` for Nuclei N/NX/UX Class Processors cores with little endian. Same header file will be used for floating point unit(FPU) variants.

3.1.3 Examples

The library ships with *a number of examples* (page 837) which demonstrate how to use the library functions.

3.1.4 Toolchain Support

The library has been developed and tested with RISC-V GCC Toolchain.

The library is being tested in GCC toolchain and updates on this activity will be made available shortly.

3.1.5 Building the Library

The library installer contains a **Makefile** to rebuild libraries on Nuclei RISC-V GCC toolchain in the **NMSIS/** folder.

The library can be built by run `make gen_dsp_lib`, it will build and install DSP library into **Library/DSP/GCC** folder.

3.1.6 Preprocessor Macros

Each library project have different pre-processor macros controlled via `CMakeLists.txt`.

This library is only built for little endian targets.

3.2 Using NMSIS-DSP

Here we will describe how to run the `nmsis dsp` examples in Nuclei QEMU.

3.2.1 Preparation

- Nuclei SDK, master branch(>= 0.9.0 release)
- Nuclei RISC-V GNU Toolchain 2025.10
- Nuclei QEMU 2025.10
- CMake >= 3.14
- Python 3 and pip package requirements located in
 - `<nuclei-sdk>/tools/scripts/requirements.txt`
 - `<NMSIS>/NMSIS/Scripts/requirements.txt`

3.2.2 Tool Setup

1. Export **PATH** correctly for qemu and riscv64-unknown-elf-gcc

```
export PATH=/path/to/qemu/bin:/path/to/gcc/bin:$PATH
```

3.2.3 Build NMSIS DSP Library

1. Download or clone NMSIS source code into **NMSIS** directory.
2. cd to **NMSIS/NMSIS/** directory, if you want to add or remove more arches to be build, you can modify **Scripts/Build/nmsis_dsp.json** file
3. Build NMSIS DSP library and strip debug information using `make gen_dsp_lib`
4. The dsp library will be generated into **./Library/DSP/GCC** folder
5. The dsp libraries will be look like this:

```
$ ls -lhG Library/DSP/GCC/
total 361M
-rw-rw-r-- 1 3.8M Oct 20 11:52 libnmsis_dsp_rv32imac.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_xxldsp.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_xxldspn1x.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_xxldspn2x.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_xxldspn3x.a
-rw-rw-r-- 1 3.8M Oct 20 11:52 libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 3.5M Oct 20 11:52 libnmsis_dsp_rv32imafc.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_xxldsp.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_xxldspn1x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_xxldspn2x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_xxldspn3x.a
-rw-rw-r-- 1 3.5M Oct 20 11:52 libnmsis_dsp_rv32imafc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 3.9M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f_xxldsp.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f_xxldspn1x.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f_zba_zbb_zbc_zbs_
↪xxldsp.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafc_zfh_zvfz_zve32f_zba_zbb_zbc_zbs_
↪xxldspn1x.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_xxldsp.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_xxldspn1x.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_xxldspn2x.a
```

(continues on next page)

(continued from previous page)

```

-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_xxldspn3x.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 3.5M Oct 20 11:52 libnmsis_dsp_rv32imafdc.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_xxldsp.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_xxldspn1x.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_xxldspn2x.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_xxldspn3x.a
-rw-rw-r-- 1 3.4M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 3.7M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 3.8M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh.a
-rw-rw-r-- 1 3.5M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f_xxldsp.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f_xxldspn1x.a
-rw-rw-r-- 1 3.5M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f_zba_zbb_zbc_zbs_
↪xxldsp.a
-rw-rw-r-- 1 3.6M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zfh_zvfh_zve32f_zba_zbb_zbc_zbs_
↪xxldspn1x.a
-rw-rw-r-- 1 3.1M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_xxldsp.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_xxldspn1x.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_xxldspn2x.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_xxldspn3x.a
-rw-rw-r-- 1 3.1M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 3.2M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 3.3M Oct 20 11:52 libnmsis_dsp_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 5.0M Oct 20 11:52 libnmsis_dsp_rv64imac.a
-rw-rw-r-- 1 5.4M Oct 20 11:52 libnmsis_dsp_rv64imac_xxldsp.a
-rw-rw-r-- 1 4.9M Oct 20 11:52 libnmsis_dsp_rv64imac_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 5.4M Oct 20 11:52 libnmsis_dsp_rv64imac_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 4.6M Oct 20 11:52 libnmsis_dsp_rv64imafc.a
-rw-rw-r-- 1 5.0M Oct 20 11:52 libnmsis_dsp_rv64imafc_xxldsp.a
-rw-rw-r-- 1 4.5M Oct 20 11:52 libnmsis_dsp_rv64imafc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 5.0M Oct 20 11:52 libnmsis_dsp_rv64imafc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 5.1M Oct 20 11:52 libnmsis_dsp_rv64imafc_zfh.a
-rw-rw-r-- 1 4.5M Oct 20 11:52 libnmsis_dsp_rv64imafc_zfh_zvfh_zve64f.a
-rw-rw-r-- 1 4.7M Oct 20 11:52 libnmsis_dsp_rv64imafc_zfh_zvfh_zve64f_xxldsp.a
-rw-rw-r-- 1 4.5M Oct 20 11:52 libnmsis_dsp_rv64imafc_zfh_zvfh_zve64f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 4.7M Oct 20 11:52 libnmsis_dsp_rv64imafc_zfh_zvfh_zve64f_zba_zbb_zbc_zbs_
↪xxldsp.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv64imafc_zve64f.a
-rw-rw-r-- 1 4.1M Oct 20 11:52 libnmsis_dsp_rv64imafc_zve64f_xxldsp.a
-rw-rw-r-- 1 4.0M Oct 20 11:52 libnmsis_dsp_rv64imafc_zve64f_zba_zbb_zbc_zbs.a

```

(continues on next page)

(continued from previous page)

```

-rw-rw-r-- 1 4.1M Oct 20 11:52 libnmsis_dsp_rv64imafc_zve64f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 4.5M Oct 20 11:52 libnmsis_dsp_rv64imafdc.a
-rw-rw-r-- 1 3.9M Oct 20 11:52 libnmsis_dsp_rv64imafdcv.a
-rw-rw-r-- 1 4.1M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_xxldsp.a
-rw-rw-r-- 1 3.9M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 4.1M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 4.5M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zfh_zvfh.a
-rw-rw-r-- 1 4.6M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zfh_zvfh_xxldsp.a
-rw-rw-r-- 1 4.4M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zfh_zvfh_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 4.6M Oct 20 11:52 libnmsis_dsp_rv64imafdcv_zfh_zvfh_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 4.9M Oct 20 11:52 libnmsis_dsp_rv64imafdc_xxldsp.a
-rw-rw-r-- 1 4.4M Oct 20 11:52 libnmsis_dsp_rv64imafdc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 4.9M Oct 20 11:52 libnmsis_dsp_rv64imafdc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 5.0M Oct 20 11:52 libnmsis_dsp_rv64imafdc_zfh.a

```

7. library name with extra `_xxldsp` `_xxldspn1x` `_xxldspn2x` `_xxldspn3x` is built with RISC-V DSP enabled

The examples are as follows:

- `libnmsis_dsp_rv32imac.a`: Build for **RISCV_ARCH=rv32imac** without DSP
- `libnmsis_dsp_rv32imac_xxldsp.a`: Build for **RISCV_ARCH=rv32imac_xxldsp** with Nuclei DSP enabled
- `libnmsis_dsp_rv32imac_xxldspn1x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn1x** with Nuclei N1 DSP extension enabled
- `libnmsis_dsp_rv32imac_xxldspn2x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn2x** with Nuclei N1/N2 DSP extension enabled
- `libnmsis_dsp_rv32imac_xxldspn3x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn3x** with Nuclei N1/N2/N3 DSP extension enabled

8. library name with extra `_zve32f` `_zve64f` `v` is built with RISC-V Vector enabled

The examples are as follows:

- `libnmsis_dsp_rv32imafc_zve32f.a`: Build for **RISCV_ARCH=rv32imafc_zve32f** with Vector enabled
- `libnmsis_dsp_rv32imafdc_zve32f.a`: Build for **RISCV_ARCH=rv32imafdc_zve32f** with Vector enabled
- `libnmsis_dsp_rv64imafc_zve64f.a`: Build for **RISCV_ARCH=rv64imafc_zve64f** with Vector enabled
- `libnmsis_dsp_rv64imafdcv.a`: Build for **RISCV_ARCH=rv64imafdcv** with Vector enabled

9. library name with extra `_zfh` is built for float16

The examples are as follows:

- `libnmsis_dsp_rv32imafc_zfh.a`: Build for **RISCV_ARCH=rv32imafc_zfh**.
- `libnmsis_dsp_rv32imafdc_zfh_zve32f.a`: Build for **RISCV_ARCH=rv32imafdc_zfh_zve32f** with Vector enabled.
- `libnmsis_dsp_rv64imafc_zfh_zvfh_zve64f.a`: Build for **RISCV_ARCH=rv64imafc_zfh_zvfh_zve64f** with Vector enabled.
- `libnmsis_dsp_rv64imafdcv_zfh_zvfh.a`: Build for **RISCV_ARCH=rv64imafdcv_zfh_zvfh** with Vector enabled

Note

- This NMSIS 1.2.0 is a big change version, will no longer support old gcc 10 version, and it now only support Nuclei Toolchain 2023.10. The `--march` option has changed a lot, such as:
 - b extension changed to `_zba_zbb_zbc_zbs` extension,
 - p extension changed to `_xxldsp`, `_xxldspn1x`, `_xxldspn2x`, `_xxldspn3x` extensions which means standard DSP extension, Nuclei N1, N2, N3 DSP extensions
 - v extension changed to `v`, `_zve32f`, `_zve64f` extensions
- The name of libraries has changed with `-march`, for examples, the library named `libnmsis_dsp_rv32imacb.a` is now named `libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs.a` since b extension changed to `_zba_zbb_zbc_zbs`
- `_xxldspn1x` `_xxldspn2x` `_xxldspn3x` only valid for RISC-V 32bit processor. `_xxldsp` is valid for RISC-V 32/64 bit processor
- You can also directly build both DSP and NN library using `make gen`
- DSP and Vector extension can be combined, such as `_xxldsp, v` and `v_xxldsp`, should notice the extension order
- Vector extension currently enabled for RISC-V 32/64 bit processor

3.2.4 How to run

1. Set environment variables `NUCLEI_SDK_ROOT` and `NUCLEI_SDK_NMSIS`, and set Nuclei SDK SoC to `evalsoc`, and change ilm/dlm size from 64K to 512K.

```
export NUCLEI_SDK_ROOT=/path/to/nuclei_sdk
export NUCLEI_SDK_NMSIS=/path/to/NMSIS/NMSIS
# Setup SDK development environment
cd $NUCLEI_SDK_ROOT
source setup.sh
cd -
# !!!!Take Care!!!!
# change this link script will make compiled example can only run on bitstream which has
↪ 512K ILM/DLM
# For Nuclei SDK < 0.7.0
sed -i "s/64K/512K/g" $NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_
↪ evalsoc_ilm.ld
# For Nuclei SDK >= 0.7.0
sed -i 's/\([ID]LM_MEMORY_SIZE\).*\1 = 0x80000;/' $NUCLEI_SDK_ROOT/SoC/evalsoc/Board/
↪ nuclei_fpga_eval/Source/GCC/evalsoc.memory
export SOC=evalsoc
```

2. Due to many of the examples could not be placed in 64K ILM and 64K DLM, and we are running using qemu, the ILM/DLM size in it are set to be 32MB, so we can change ilm/dlm to 512K/512K in the link script `$NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld` or `$NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/evalsoc.memory`

```
--- a/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld
+++ b/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld
```

(continues on next page)

(continued from previous page)

```

@@ -30,8 +30,8 @@ __HEAP_SIZE = 2K;

MEMORY
{
- ilm (rxa!w) : ORIGIN = 0x80000000, LENGTH = 64K
- ram (wxa!r) : ORIGIN = 0x90000000, LENGTH = 64K
+ ilm (rxa!w) : ORIGIN = 0x80000000, LENGTH = 512K
+ ram (wxa!r) : ORIGIN = 0x90000000, LENGTH = 512K
}

```

3. Let us take riscv_class_marks_example for example:

```
cd $NUCLEI_SDK_NMSIS/DSP/Examples/RISCV/riscv_class_marks_example
```

4. Run with RISC-V DSP enabled and Vector enabled NMSIS-DSP library for CORE nx900fd

```

# Clean project
make ARCH_EXT=v_xldsp CORE=nx900fd clean
# Build project, enable ``v`` and ``xldsp`` optimize
make ARCH_EXT=v_xldsp CORE=nx900fd all
# Run application using qemu
make ARCH_EXT=v_xldsp CORE=nx900fd run_qemu

```

5. Run with RISC-V DSP disabled and Vector disabled NMSIS-DSP library for CORE nx900fd

```

make ARCH_EXT= CORE=nx900fd clean
make ARCH_EXT= CORE=nx900fd all
make ARCH_EXT= CORE=nx900fd run_qemu

```

Note

- You can easily run this example in your hardware, if you have enough memory to run it, just modify the SOC to the one you are using in step 1.

3.3 NMSIS DSP API

If you want to access doxygen generated NMSIS DSP API, please click [NMSIS DSP API Doxygen Documentation](#).

3.3.1 Examples

Bayes Example

group Bayes Example

Description:

Demonstrates the use of Bayesian classifier functions. It is complementing the tutorial about classical ML with CMSIS-DSP and python scikit-learn: <https://>

[developer.arm.com/solutions/machine-learning-on-arm/developer-material/how-to-guides/
implement-classical-ml-with-arm-cmsis-dsp-libraries](https://developer.arm.com/solutions/machine-learning-on-arm/developer-material/how-to-guides/implement-classical-ml-with-arm-cmsis-dsp-libraries)

Class Marks Example

group **Class Marks Example**

Refer riscv_class_marks_example_f32.c

Description:

Demonstrates the use the Maximum, Minimum, Mean, Standard Deviation, Variance and Matrix functions to calculate statistical values of marks obtained in a class.

Variables Description:

- `testMarks_f32` points to the marks scored by 20 students in 4 subjects
- `max_marks` Maximum of all marks
- `min_marks` Minimum of all marks
- `mean` Mean of all marks
- `var` Variance of the marks
- `std` Standard deviation of the marks
- `numStudents` Total number of students in the class

NMSIS DSP Software Library Functions Used:

- `riscv_mat_init_f32()`
- `riscv_mat_mult_f32()`
- `riscv_max_f32()`
- `riscv_min_f32()`
- `riscv_mean_f32()`
- `riscv_std_f32()`
- `riscv_var_f32()`

Note

This example also demonstrates the usage of static initialization.

Convolution Example

group Convolution Example

Refer riscv_convolution_example_f32.c

Description:

Demonstrates the convolution theorem with the use of the Complex FFT, Complex-by-Complex Multiplication, and Support Functions.

Algorithm:

The convolution theorem states that convolution in the time domain corresponds to multiplication in the frequency domain. Therefore, the Fourier transform of the convolution of two signals is equal to the product of their individual Fourier transforms. The Fourier transform of a signal can be evaluated efficiently using the Fast Fourier Transform (FFT).

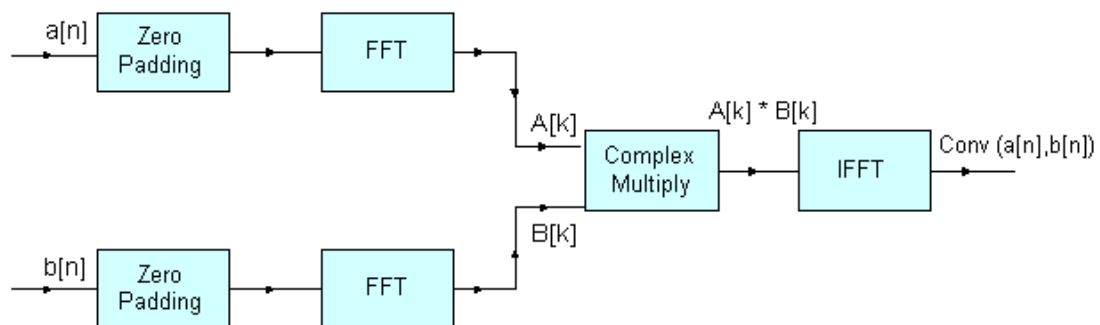
Two input signals, $a[n]$ and $b[n]$, with lengths $n1$ and $n2$ respectively, are zero padded so that their lengths become N , which is greater than or equal to $(n1+n2-1)$ and is a power of 4 as FFT implementation is radix-4. The convolution of $a[n]$ and $b[n]$ is obtained by taking the FFT of the input signals, multiplying the Fourier transforms of the two signals, and taking the inverse FFT of the multiplied result.

This is denoted by the following equations:

$$\begin{aligned} A[k] &= \text{FFT}(a[n], N) \\ B[k] &= \text{FFT}(b[n], N) \\ \text{conv}(a[n], b[n]) &= \text{IFFT}(A[k] * B[k], N) \end{aligned}$$

where $A[k]$ and $B[k]$ are the N -point FFTs of the signals $a[n]$ and $b[n]$ respectively. The length of the convolved signal is $(n1+n2-1)$.

Block Diagram:



Variables Description:

- testInputA_f32 points to the first input sequence
- srcALen length of the first input sequence
- testInputB_f32 points to the second input sequence

- srcBLen length of the second input sequence
- outLen length of convolution output sequence, (srcALen + srcBLen - 1)
- AxB points to the output array where the product of individual FFTs of inputs is stored.

NMSIS DSP Software Library Functions Used:

- riscv_fill_f32()
- riscv_copy_f32()
- riscv_cfft_radix4_init_f32()
- riscv_cfft_radix4_f32()
- riscv_cmplx_mult_cmplx_f32()

Dot Product Example*group* **Dot Product Example**

Refer riscv_dotproduct_example_f32.c

Description:

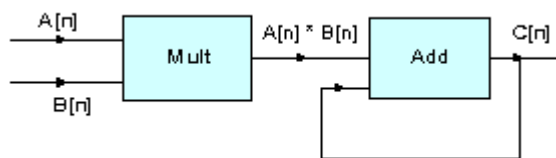
Demonstrates the use of the Multiply and Add functions to perform the dot product. The dot product of two vectors is obtained by multiplying corresponding elements and summing the products.

Algorithm:

The two input vectors A and B with length n, are multiplied element-by-element and then added to obtain dot product.

This is denoted by the following equation:

$$\text{dotProduct} = A[0] * B[0] + A[1] * B[1] + \dots + A[n-1] * B[n-1]$$

Block Diagram:**Variables Description:**

- srcA_buf_f32 points to first input vector
- srcB_buf_f32 points to second input vector
- testOutput stores dot product of the two input vectors.

NMSIS DSP Software Library Functions Used:

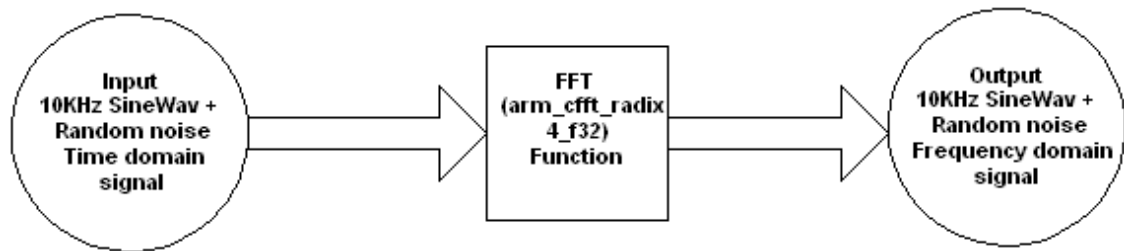
- `riscv_mult_f32()`
- `riscv_add_f32()`

Frequency Bin Example*group* **Frequency Bin Example****Refer** `riscv_fft_bin_example_f32.c`**Description**

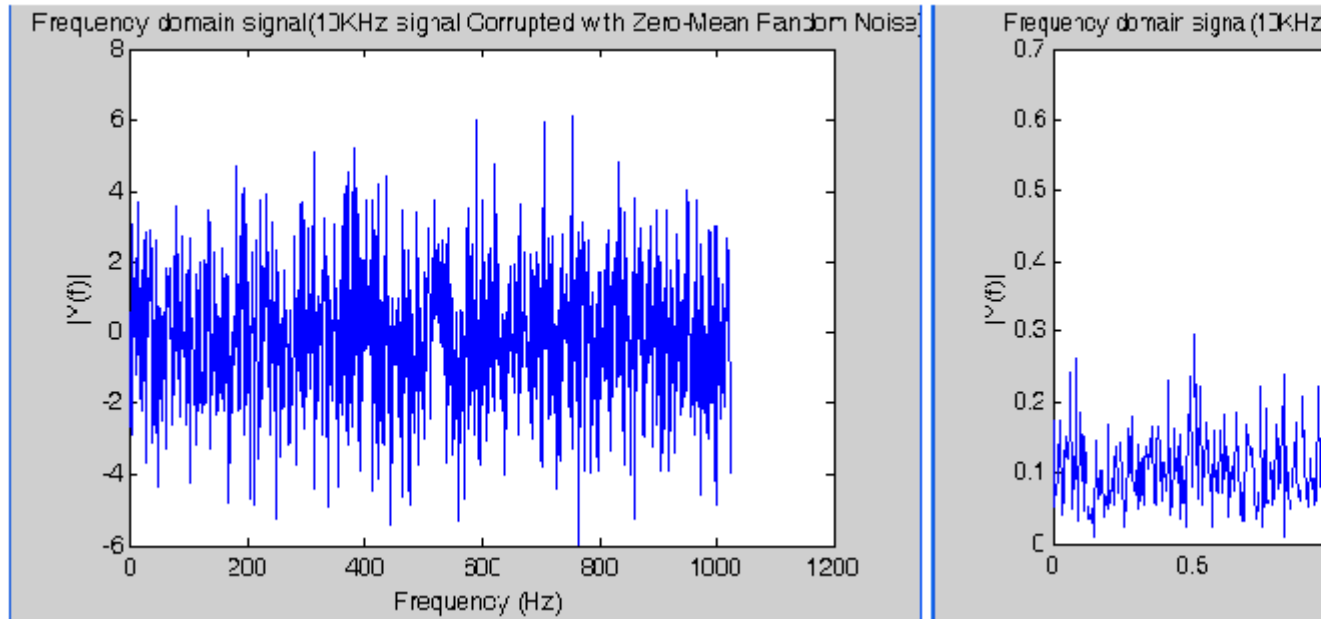
Demonstrates the calculation of the maximum energy bin in the frequency domain of the input signal with the use of Complex FFT, Complex Magnitude, and Maximum functions.

Algorithm:

The input test signal contains a 10 kHz signal with uniformly distributed white noise. Calculating the FFT of the input signal will give us the maximum energy of the bin corresponding to the input frequency of 10 kHz.

Block Diagram:

The figure below shows the time domain signal of 10 kHz signal with uniformly distributed white noise, and the next figure shows the input in the frequency domain. The bin with maximum energy corresponds to 10 kHz signal.



Variables Description:

- testInput_f32_10khz points to the input data
- testOutput points to the output data
- fftSize length of FFT
- ifftFlag flag for the selection of CFFT/CIFFT
- doBitReverse Flag for selection of normal order or bit reversed order
- refIndex reference index value at which maximum energy of bin occurs
- testIndex calculated index value at which maximum energy of bin occurs

NMSIS DSP Software Library Functions Used:

- riscv_cfft_f32()
- riscv_cmplx_mag_f32()
- riscv_max_f32()

FIR Lowpass Filter Example

group FIR Lowpass Filter Example

Refer riscv_fir_example_f32.c

Description:

Removes high frequency signal components from the input using an FIR lowpass filter. The example

demonstrates how to configure an FIR filter and then pass data through it in a block-by-block fashion.



Algorithm:

The input signal is a sum of two sine waves: 1 kHz and 15 kHz. This is processed by an FIR lowpass filter with cutoff frequency 6 kHz. The lowpass filter eliminates the 15 kHz signal leaving only the 1 kHz sine wave at the output.

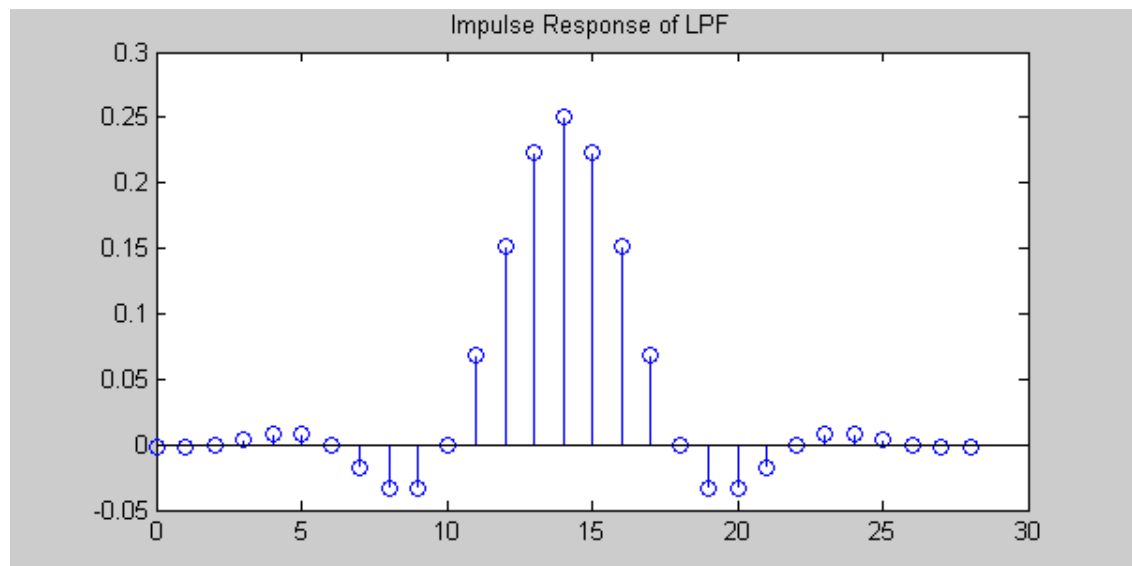
The lowpass filter was designed using MATLAB with a sample rate of 48 kHz and a length of 29 points. The MATLAB code to generate the filter coefficients is shown below:

```
h = fir1(28, 6/24);
```

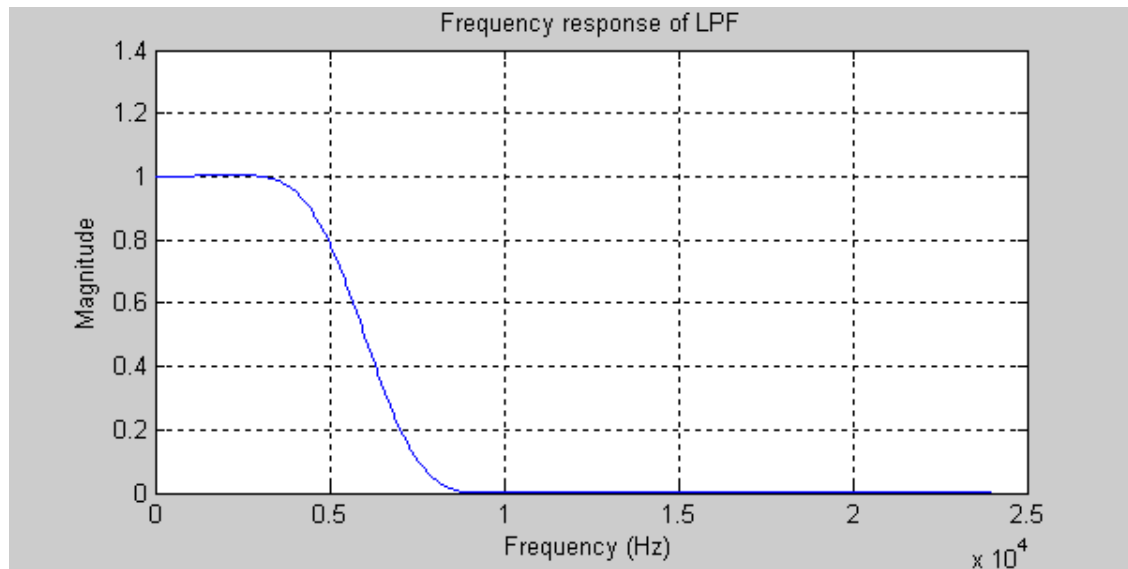
The first argument is the “order” of the filter and is always one less than the desired length. The second argument is the normalized cutoff frequency. This is in the range 0 (DC) to 1.0 (Nyquist). A 6 kHz cutoff with a Nyquist frequency of 24 kHz lies at a normalized frequency of $6/24 = 0.25$. The NMSIS FIR filter function requires the coefficients to be in time reversed order.

```
flip1r(h)
```

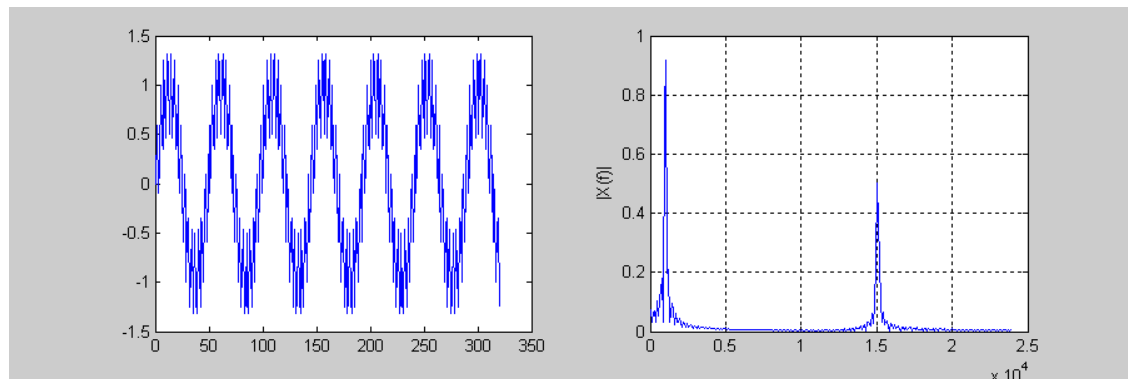
The resulting filter coefficients are shown below. Note that the filter is symmetric (a property of linear phase FIR filters) and the point of symmetry is sample 14. Thus the filter will have a delay of 14 samples for all frequencies.



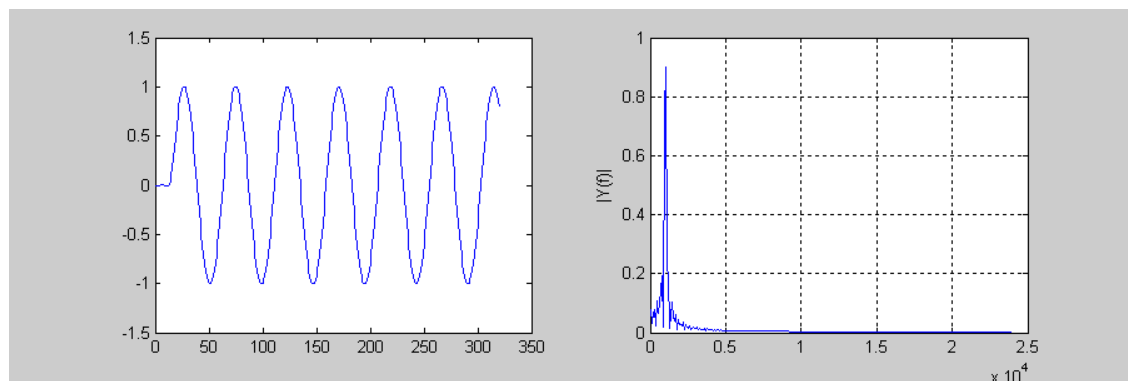
The frequency response of the filter is shown next. The passband gain of the filter is 1.0 and it reaches 0.5 at the cutoff frequency 6 kHz.



The input signal is shown below. The left hand side shows the signal in the time domain while the right hand side is a frequency domain representation. The two sine wave components can be clearly seen.



The output of the filter is shown below. The 15 kHz component has been eliminated.



Variables Description:

- `testInput_f32_1kHz_15kHz` points to the input data
- `refOutput` points to the reference output data
- `testOutput` points to the test output data
- `firStateF32` points to state buffer
- `firCoeffs32` points to coefficient buffer
- `blockSize` number of samples processed at a time
- `numBlocks` number of frames

NMSIS DSP Software Library Functions Used:

- `riscv_fir_init_f32()`
- `riscv_fir_f32()`

Graphic Audio Equalizer Example

group Graphic Audio Equalizer Example

Refer `riscv_graphic_equalizer_example_q31.c`

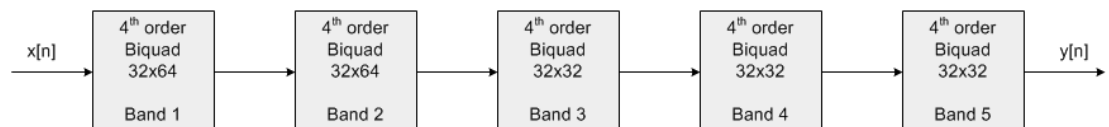
Description:

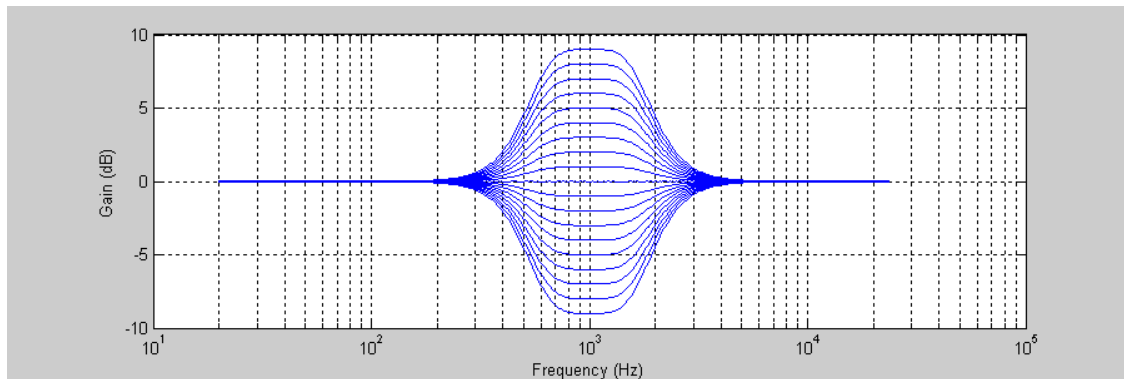
This example demonstrates how a 5-band graphic equalizer can be constructed using the Biquad cascade functions. A graphic equalizer is used in audio applications to vary the tonal quality of the audio.

Block Diagram:

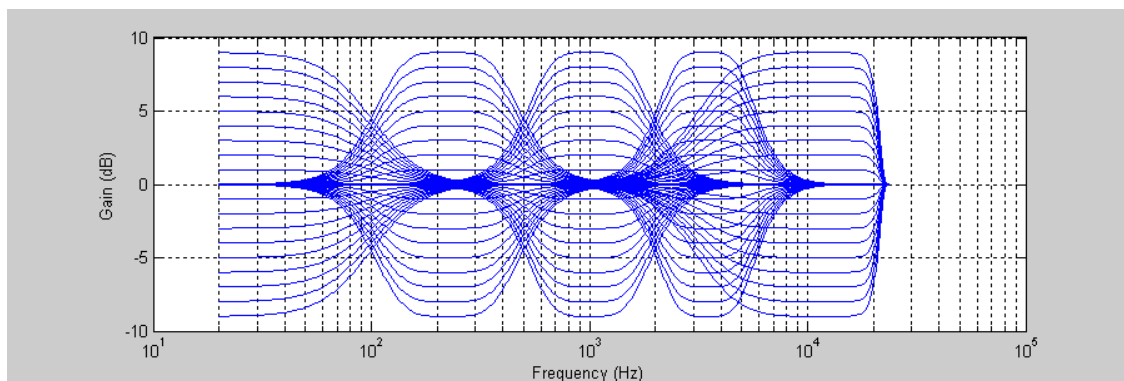
The design is based on a cascade of 5 filter sections.

Each filter section is 4th order and consists of a cascade of two Biquads. Each filter has a nominal gain of 0 dB (1.0 in linear units) and boosts or cuts signals within a specific frequency range. The edge frequencies between the 5 bands are 100, 500, 2000, and 6000 Hz. Each band has an adjustable boost or cut in the range of +/- 9 dB. For example, the band that extends from 500 to 2000 Hz has the response shown below:

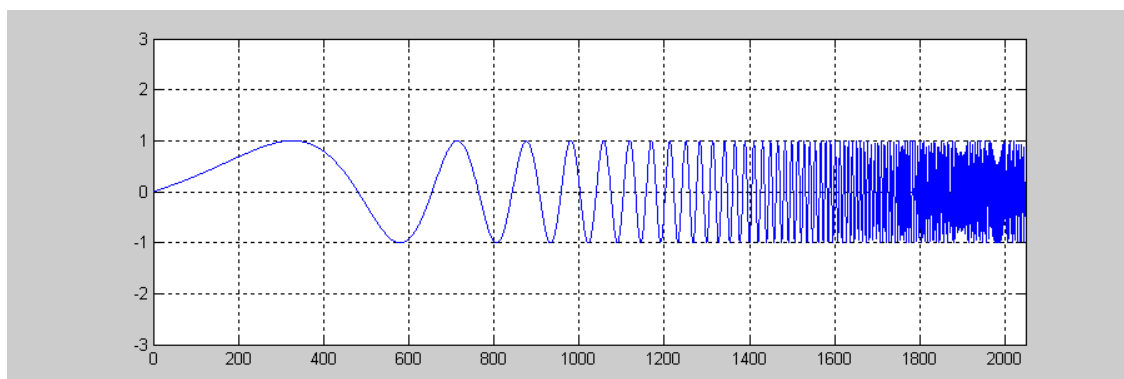




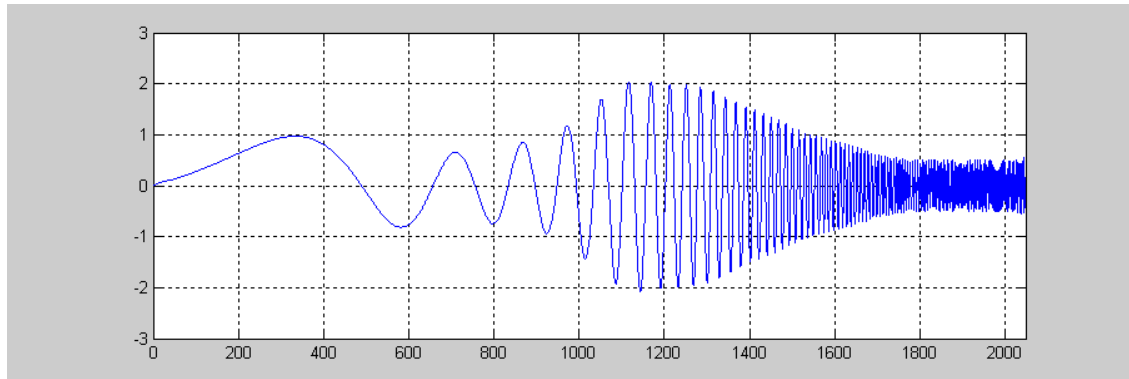
With 1 dB steps, each filter has a total of 19 different settings. The filter coefficients for all possible 19 settings were precomputed in MATLAB and stored in a table. With 5 different tables, there are a total of $5 \times 19 = 95$ different 4th order filters. All 95 responses are shown below:



Each 4th order filter has 10 coefficients for a grand total of 950 different filter coefficients that must be tabulated. The input and output data is in Q31 format. For better noise performance, the two low frequency bands are implemented using the high precision 32x64-bit Biquad filters. The remaining 3 high frequency bands use standard 32x32-bit Biquad filters. The input signal used in the example is a logarithmic chirp.



The array `bandGains` specifies the gain in dB to apply in each band. For example, if `bandGains={0, -3, 6, 4, -6}`; then the output signal will be:



Variables Description:

- testInput_f32 points to the input data
- testRefOutput_f32 points to the reference output data
- testOutput points to the test output data
- inputQ31 temporary input buffer
- outputQ31 temporary output buffer
- biquadStateBand1Q31 points to state buffer for band1
- biquadStateBand2Q31 points to state buffer for band2
- biquadStateBand3Q31 points to state buffer for band3
- biquadStateBand4Q31 points to state buffer for band4
- biquadStateBand5Q31 points to state buffer for band5
- coeffTable points to coefficient buffer for all bands
- gainDB gain buffer which has gains applied for all the bands

NMSIS DSP Software Library Functions Used:

- riscv_biquad_cas_df1_32x64_init_q31()
- riscv_biquad_cas_df1_32x64_q31()
- riscv_biquad_cascade_df1_init_q31()
- riscv_biquad_cascade_df1_q31()
- riscv_scale_q31()
- riscv_scale_f32()
- riscv_float_to_q31()
- riscv_q31_to_float()

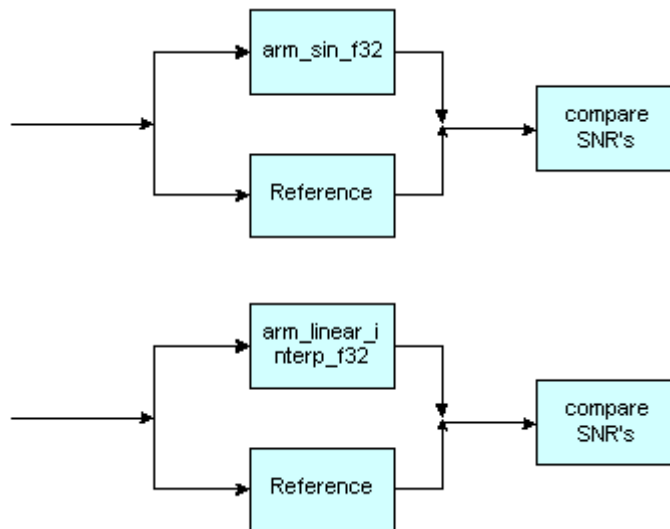
Note

The output chirp signal follows the gain or boost of each band.

Linear Interpolate Example*group* **Linear Interpolate Example****NMSIS DSP Software Library Linear Interpolate Example**

Description This example demonstrates usage of linear interpolate modules and fast math modules. Method 1 uses fast math sine function to calculate sine values using cubic interpolation and method 2 uses linear interpolation function and results are compared to reference output. Example shows linear interpolation function can be used to get higher precision compared to fast math sin calculation.

Refer riscv_linear_interp_example_f32.c

Block Diagram:**Variables Description:**

- testInputSin_f32 points to the input values for sine calculation
- testRefSinOutput32_f32 points to the reference values calculated from sin() matlab function
- testOutput points to output buffer calculation from cubic interpolation
- testLinIntOutput points to output buffer calculation from linear interpolation
- snr1 Signal to noise ratio for reference and cubic interpolation output
- snr2 Signal to noise ratio for reference and linear interpolation output

NMSIS DSP Software Library Functions Used:

- riscv_sin_f32()

- riscv_linear_interp_f32()

Matrix Example

group **Matrix Example**

Refer riscv_matrix_example_f32.c

Description:

Demonstrates the use of Matrix Transpose, Matrix Multiplication, and Matrix Inverse functions to apply least squares fitting to input data. Least squares fitting is the procedure for finding the best-fitting curve that minimizes the sum of the squares of the offsets (least square error) from a given set of data.

Algorithm:

The linear combination of parameters considered is as follows:

$A * X = B$, where X is the unknown value and can be estimated from A & B .

The least squares estimate X is given by the following equation:

$$X = \text{Inverse}(A * A) * A * B$$

Block Diagram:

Variables Description:

- A_f32 input matrix in the linear combination equation
- B_f32 output matrix in the linear combination equation
- X_f32 unknown matrix estimated using A_f32 & B_f32 matrices

NMSIS DSP Software Library Functions Used:

- riscv_mat_init_f32()
- riscv_mat_trans_f32()
- riscv_mat_mult_f32()
- riscv_mat_inverse_f32()

Signal Convergence Example

group Signal Convergence Example

Refer riscv_signal_converge_example_f32.c

Description:

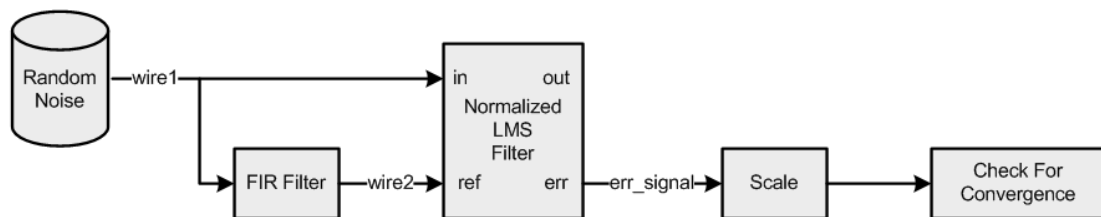
Demonstrates the ability of an adaptive filter to “learn” the transfer function of a FIR lowpass filter using the Normalized LMS Filter, Finite Impulse Response (FIR) Filter, and Basic Math Functions.

Algorithm:

The figure below illustrates the signal flow in this example. Uniformly distributed white noise is passed through an FIR lowpass filter. The output of the FIR filter serves as the reference input of the adaptive filter (normalized LMS filter). The white noise is input to the adaptive filter. The adaptive filter learns the transfer function of the FIR filter. The filter outputs two signals: (1) the output of the internal adaptive FIR filter, and (2) the error signal which is the difference between the adaptive filter and the reference output of the FIR filter. Over time as the adaptive filter learns the transfer function of the FIR filter, the first output approaches the reference output of the FIR filter, and the error signal approaches zero.

The adaptive filter converges properly even if the input signal has a large dynamic range (i.e., varies from small to large values). The coefficients of the adaptive filter are initially zero, and then converge over 1536 samples. The internal function test_signal_converge() implements the stopping condition. The function checks if all of the values of the error signal have a magnitude below a threshold DELTA.

Block Diagram:



Variables Description:

- testInput_f32 points to the input data
- firStateF32 points to FIR state buffer
- lmsStateF32 points to Normalised Least mean square FIR filter state buffer
- FIRCoeff_f32 points to coefficient buffer
- lmsNormCoeff_f32 points to Normalised Least mean square FIR filter coefficient buffer
- wire1, wir2, wire3 temporary buffers
- errOutput, err_signal temporary error buffers

NMSIS DSP Software Library Functions Used:

- riscv_lms_norm_init_f32()

- riscv_fir_init_f32()
- riscv_fir_f32()
- riscv_lms_norm_f32()
- riscv_scale_f32()
- riscv_abs_f32()
- riscv_sub_f32()
- riscv_min_f32()
- riscv_copy_f32()

SineCosine Example

group SineCosine Example

Refer riscv_sin_cos_example_f32.c

Description:

Demonstrates the Pythagorean trigonometric identity with the use of Cosine, Sine, Vector Multiplication, and Vector Addition functions.

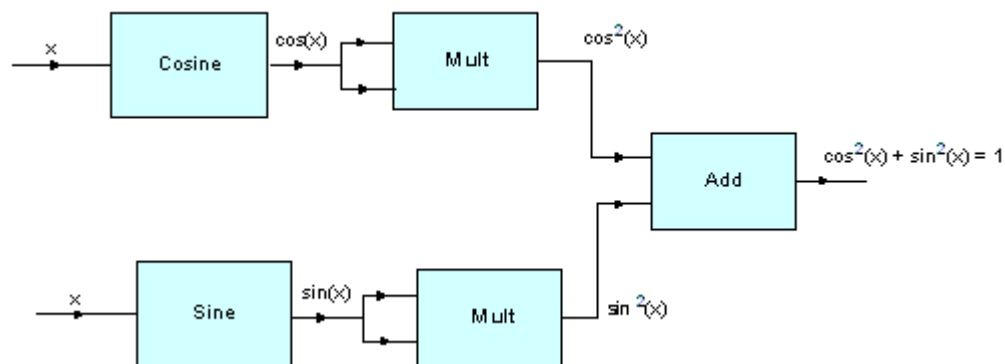
Algorithm:

Mathematically, the Pythagorean trigonometric identity is defined by the following equation:

$$\sin(x) * \sin(x) + \cos(x) * \cos(x) = 1$$

where x is the angle in radians.

Block Diagram:



Variables Description:

- testInput_f32 array of input angle in radians
- testOutput stores sum of the squares of sine and cosine values of input angle

NMSIS DSP Software Library Functions Used:

- `riscv_cos_f32()`
- `riscv_sin_f32()`
- `riscv_mult_f32()`
- `riscv_add_f32()`

SVM Example

group SVM Example

Description:

Demonstrates the use of SVM functions. It is complementing the tutorial about classical ML with CMSIS-DSP and python scikit-learn: <https://developer.arm.com/solutions/machine-learning-on-arm/developer-material/how-to-guides/implement-classical-ml-with-arm-cmsis-dsp-libraries>

Variance Example

group Variance Example

Refer `riscv_variance_example_f32.c`

Description:

Demonstrates the use of Basic Math and Support Functions to calculate the variance of an input sequence with N samples. Uniformly distributed white noise is taken as input.

Algorithm:

The variance of a sequence is the mean of the squared deviation of the sequence from its mean.

This is denoted by the following equation:

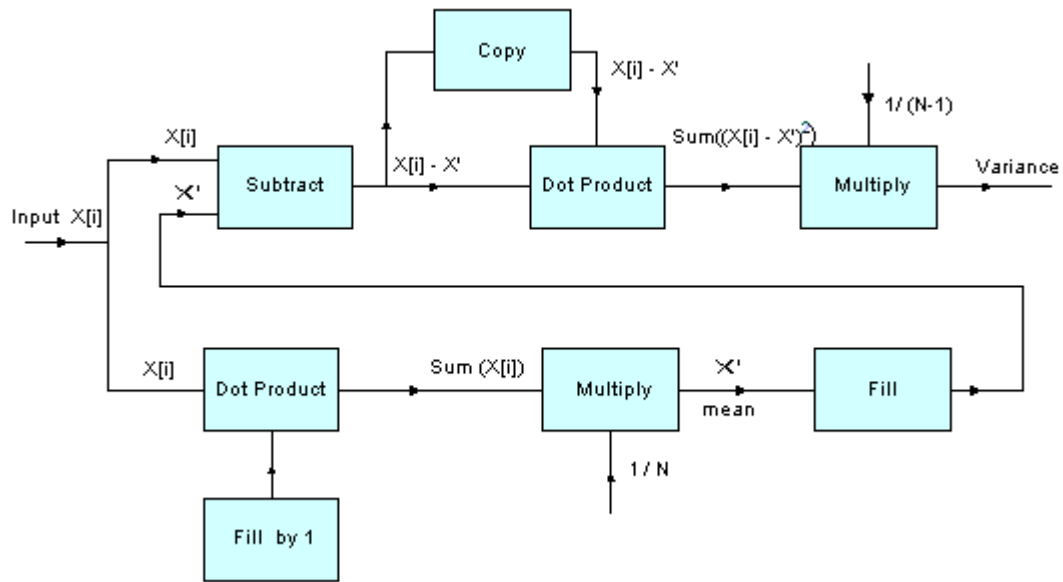
$$\text{variance} = ((x[0] - x') * (x[0] - x') + (x[1] - x') * (x[1] - x') + \dots + (x[n-1] - x') * (x[n-1] - x')) / (N-1)$$

where, $x[n]$ is the input sequence, N is the number of input samples, and x' is the mean value of the input sequence, $x[n]$.

The mean value x' is defined as:

$$x' = (x[0] + x[1] + \dots + x[n-1]) / N$$

Block Diagram:



Variables Description:

- testInput_f32 points to the input data
- wire1, wir2, wire3 temporary buffers
- blockSize number of samples processed at a time
- refVarianceOut reference variance value

NMSIS DSP Software Library Functions Used:

- riscv_dot_prod_f32()
- riscv_mult_f32()
- riscv_sub_f32()
- riscv_fill_f32()
- riscv_copy_f32()

group Examples

3.3.2 Basic Math Functions

Vector Absolute Value

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f64 (const float64_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_abs_q15 (const q15_t *pSrc, q15_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_abs_q31 (const q31_t *pSrc, q31_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_abs_q7 (const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)
```

group **Vector Absolute Value**

Computes the absolute value of a vector on an element-by-element basis.

$$pDst[n] = \text{abs}(pSrc[n]), \quad 0 \leq n < \text{blockSize}.$$

The functions support in-place computation allowing the source and destination pointers to reference the same memory buffer. There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f16 (const float16_t *pSrc, float16_t *pDst,  
uint32_t blockSize)
```

Floating-point vector absolute value.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f32 (const float32_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

Floating-point vector absolute value.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_abs_f64 (const float64_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

Floating-point vector absolute value.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector

- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_abs_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Q15 vector absolute value.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q15 value -1 (0x8000) will be saturated to the maximum allowable positive value 0x7FFF.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_abs_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Q31 vector absolute value.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q31 value -1 (0x80000000) will be saturated to the maximum allowable positive value 0x7FFFFFFF.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_abs_q7 (const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)

Q7 vector absolute value.

Conditions for optimum performance

Input and output buffers should be aligned by 32-bit

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q7 value -1 (0x80) will be saturated to the maximum allowable positive value 0x7F.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector Addition

```
RISCV_DSP_ATTRIBUTE void riscv_add_f16 (const float16_t *pSrcA, const float16_t *pSrcB,
float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_add_f32 (const float32_t *pSrcA, const float32_t *pSrcB,
float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_add_f64 (const float64_t *pSrcA, const float64_t *pSrcB,
float64_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_add_q15 (const q15_t *pSrcA, const q15_t *pSrcB,
q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_add_q31 (const q31_t *pSrcA, const q31_t *pSrcB,
q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_add_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst,
uint32_t blockSize)
```

group Vector Addition

Element-by-element addition of two vectors.

$$pDst[n] = pSrcA[n] + pSrcB[n], \quad 0 \leq n < blockSize.$$

There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_add_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, float16_t *pDst, uint32_t blockSize)
```

Floating-point vector addition.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_add_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, float32_t *pDst, uint32_t blockSize)
```

Floating-point vector addition.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector

- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_add_f64 (const float64_t *pSrcA, const float64_t *pSrcB, float64_t *pDst, uint32_t blockSize)

Floating-point vector addition.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_add_q15 (const q15_t *pSrcA, const q15_t *pSrcB, q15_t *pDst, uint32_t blockSize)

Q15 vector addition.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_add_q31 (const q31_t *pSrcA, const q31_t *pSrcB, q31_t *pDst, uint32_t blockSize)

Q31 vector addition.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_add_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst, uint32_t blockSize)

Q7 vector addition.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector bitwise AND

```
RISCV_DSP_ATTRIBUTE void riscv_and_u16 (const uint16_t *pSrcA, const uint16_t *pSrcB,  
uint16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_and_u32 (const uint32_t *pSrcA, const uint32_t *pSrcB,  
uint32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_and_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB,  
uint8_t *pDst, uint32_t blockSize)
```

group Vector bitwise AND

Compute the logical bitwise AND.

There are separate functions for uint32_t, uint16_t, and uint7_t data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_and_u16 (const uint16_t *pSrcA,  
const uint16_t *pSrcB, uint16_t *pDst, uint32_t blockSize)
```

Compute the logical bitwise AND of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_and_u32 (const uint32_t *pSrcA,  
const uint32_t *pSrcB, uint32_t *pDst, uint32_t blockSize)
```

Compute the logical bitwise AND of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_and_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB, uint8_t *pDst, uint32_t blockSize)

Compute the logical bitwise AND of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Elementwise clipping

RISCV_DSP_ATTRIBUTE void riscv_clip_f16 (const float16_t *pSrc, float16_t *pDst, float16_t low, float16_t high, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_clip_f32 (const float32_t *pSrc, float32_t *pDst, float32_t low, float32_t high, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_clip_q15 (const q15_t *pSrc, q15_t *pDst, q15_t low, q15_t high, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_clip_q31 (const q31_t *pSrc, q31_t *pDst, q31_t low, q31_t high, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_clip_q7 (const q7_t *pSrc, q7_t *pDst, q7_t low, q7_t high, uint32_t numSamples)

group **Elementwise clipping**

Element-by-element clipping of a value.

The value is constrained between 2 bounds.

There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_clip_f16 (const float16_t *pSrc, float16_t *pDst, float16_t low, float16_t high, uint32_t numSamples)

Elementwise floating-point clipping.

Parameters

- **pSrc** – [in] points to input values
- **pDst** – [out] points to output clipped values
- **low** – [in] lower bound
- **high** – [in] higher bound
- **numSamples** – [in] number of samples to clip

RISCV_DSP_ATTRIBUTE void riscv_clip_f32 (const float32_t *pSrc, float32_t *pDst, float32_t low, float32_t high, uint32_t numSamples)

Elementwise floating-point clipping.

Parameters

- **pSrc** – [in] points to input values
- **pDst** – [out] points to output clipped values
- **low** – [in] lower bound
- **high** – [in] higher bound
- **numSamples** – [in] number of samples to clip

RISCV_DSP_ATTRIBUTE void riscv_clip_q15 (const q15_t *pSrc, q15_t *pDst, q15_t low, q15_t high, uint32_t numSamples)

Elementwise fixed-point clipping.

Parameters

- **pSrc** – [in] points to input values
- **pDst** – [out] points to output clipped values
- **low** – [in] lower bound
- **high** – [in] higher bound
- **numSamples** – [in] number of samples to clip

RISCV_DSP_ATTRIBUTE void riscv_clip_q31 (const q31_t *pSrc, q31_t *pDst, q31_t low, q31_t high, uint32_t numSamples)

Elementwise fixed-point clipping.

Parameters

- **pSrc** – [in] points to input values
- **pDst** – [out] points to output clipped values
- **low** – [in] lower bound
- **high** – [in] higher bound

- **numSamples** – [in] number of samples to clip

RISCV_DSP_ATTRIBUTE void riscv_clip_q7 (const q7_t *pSrc, q7_t *pDst, q7_t low, q7_t high, uint32_t numSamples)

Elementwise fixed-point clipping.

Parameters

- **pSrc** – [in] points to input values
- **pDst** – [out] points to output clipped values
- **low** – [in] lower bound
- **high** – [in] higher bound
- **numSamples** – [in] number of samples to clip

Vector Dot Product

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f16 (const float16_t *pSrcA, const float16_t *pSrcB, uint32_t blockSize, float16_t *result)

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t blockSize, float32_t *result)

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f64 (const float64_t *pSrcA, const float64_t *pSrcB, uint32_t blockSize, float64_t *result)

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q15 (const q15_t *pSrcA, const q15_t *pSrcB, uint32_t blockSize, q63_t *result)

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q31 (const q31_t *pSrcA, const q31_t *pSrcB, uint32_t blockSize, q63_t *result)

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q7 (const q7_t *pSrcA, const q7_t *pSrcB, uint32_t blockSize, q31_t *result)

group Vector Dot Product

Computes the dot product of two vectors. The vectors are multiplied element-by-element and then summed.

```
sum = pSrcA[0]*pSrcB[0] + pSrcA[1]*pSrcB[1] + ... + pSrcA[blockSize-1]*pSrcB[blockSize-1]
```

There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f16 (const float16_t *pSrcA, const float16_t *pSrcB, uint32_t blockSize, float16_t *result)

Dot product of floating-point vectors.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **blockSize** – [in] number of samples in each vector.
- **result** – [out] output result returned here.

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t blockSize, float32_t *result)

Dot product of floating-point vectors.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **blockSize** – [in] number of samples in each vector.
- **result** – [out] output result returned here.

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_f64 (const float64_t *pSrcA, const float64_t *pSrcB, uint32_t blockSize, float64_t *result)

Dot product of floating-point vectors.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **blockSize** – [in] number of samples in each vector.
- **result** – [out] output result returned here.

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q15 (const q15_t *pSrcA, const q15_t *pSrcB, uint32_t blockSize, q63_t *result)

Dot product of Q15 vectors.

Scaling and Overflow Behavior

The intermediate multiplications are in $1.15 \times 1.15 = 2.30$ format and these results are added to a 64-bit accumulator in 34.30 format. Nonsaturating additions are used and given that there are 33 guard bits in the accumulator there is no risk of overflow. The return result is in 34.30 format.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in each vector

- **result** – [out] output result returned here

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q31 (const q31_t *pSrcA, const q31_t *pSrcB, uint32_t blockSize, q63_t *result)

Dot product of Q31 vectors.

Scaling and Overflow Behavior

The intermediate multiplications are in $1.31 \times 1.31 = 2.62$ format and these are truncated to 2.48 format by discarding the lower 14 bits. The 2.48 result is then added without saturation to a 64-bit accumulator in 16.48 format. There are 15 guard bits in the accumulator and there is no risk of overflow as long as the length of the vectors is less than 2^{16} elements. The return result is in 16.48 format.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **blockSize** – [in] number of samples in each vector.
- **result** – [out] output result returned here.

RISCV_DSP_ATTRIBUTE void riscv_dot_prod_q7 (const q7_t *pSrcA, const q7_t *pSrcB, uint32_t blockSize, q31_t *result)

Dot product of Q7 vectors.

Scaling and Overflow Behavior

The intermediate multiplications are in $1.7 \times 1.7 = 2.14$ format and these results are added to an accumulator in 18.14 format. Nonsaturating additions are used and there is no danger of wrap around as long as the vectors are less than 2^{18} elements long. The return result is in 18.14 format.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in each vector
- **result** – [out] output result returned here

Vector Multiplication

RISCV_DSP_ATTRIBUTE void riscv_mult_f16 (const float16_t *pSrcA, const float16_t *pSrcB, float16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_mult_f32 (const float32_t *pSrcA, const float32_t *pSrcB, float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_mult_f64 (const float64_t *pSrcA, const float64_t *pSrcB, float64_t *pDst, uint32_t blockSize)

```
RISCV_DSP_ATTRIBUTE void riscv_mult_q15 (const q15_t *pSrcA, const q15_t *pSrcB,
q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mult_q31 (const q31_t *pSrcA, const q31_t *pSrcB,
q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mult_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst,
uint32_t blockSize)
```

group **Vector Multiplication**

Element-by-element multiplication of two vectors.

$$pDst[n] = pSrcA[n] * pSrcB[n], \quad 0 \leq n < blockSize.$$

There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_mult_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, float16_t *pDst, uint32_t blockSize)
```

Floating-point vector multiplication.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

```
RISCV_DSP_ATTRIBUTE void riscv_mult_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, float32_t *pDst, uint32_t blockSize)
```

Floating-point vector multiplication.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

```
RISCV_DSP_ATTRIBUTE void riscv_mult_f64 (const float64_t *pSrcA,
const float64_t *pSrcB, float64_t *pDst, uint32_t blockSize)
```

Floating-point vector multiplication.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.

- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_mult_q15 (const q15_t *pSrcA, const q15_t *pSrcB, q15_t *pDst, uint32_t blockSize)

Q15 vector multiplication.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_mult_q31 (const q31_t *pSrcA, const q31_t *pSrcB, q31_t *pDst, uint32_t blockSize)

Q31 vector multiplication.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector.
- **pSrcB** – [in] points to the second input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_mult_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst, uint32_t blockSize)

Q7 vector multiplication.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector

- **blockSize** – [in] number of samples in each vector

Vector Negate

```
RISCV_DSP_ATTRIBUTE void riscv_negate_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_negate_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_negate_f64 (const float64_t *pSrc, float64_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_negate_q15 (const q15_t *pSrc, q15_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_negate_q31 (const q31_t *pSrc, q31_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_negate_q7 (const q7_t *pSrc, q7_t *pDst,
uint32_t blockSize)
```

group Vector Negate

Negates the elements of a vector.

$$pDst[n] = -pSrc[n], \quad 0 \leq n < blockSize.$$

The functions support in-place computation allowing the source and destination pointers to reference the same memory buffer. There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_negate_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

Negates the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector.
- **pDst** – [out] points to output vector.
- **blockSize** – [in] number of samples in each vector.

```
RISCV_DSP_ATTRIBUTE void riscv_negate_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

Negates the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector.

- **pDst** – [out] points to output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_negate_f64 (const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)

Negates the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector.
- **pDst** – [out] points to output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_negate_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Negates the elements of a Q15 vector.

Conditions for optimum performance

Input and output buffers should be aligned by 32-bit

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q15 value -1 (0x8000) is saturated to the maximum allowable positive value 0x7FFF.

Parameters

- **pSrc** – [in] points to the input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_negate_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Negates the elements of a Q31 vector.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q31 value -1 (0x80000000) is saturated to the maximum allowable positive value 0x7FFFFFFF.

Parameters

- **pSrc** – [in] points to the input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

RISCV_DSP_ATTRIBUTE void riscv_negate_q7 (const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)

Negates the elements of a Q7 vector.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q7 value -1 (0x80) is saturated to the maximum allowable positive value 0x7F.

Parameters

- **pSrc** – [in] points to the input vector.
- **pDst** – [out] points to the output vector.
- **blockSize** – [in] number of samples in each vector.

Vector bitwise NOT

```
RISCV_DSP_ATTRIBUTE void riscv_not_u16 (const uint16_t *pSrc, uint16_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_not_u32 (const uint32_t *pSrc, uint32_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_not_u8 (const uint8_t *pSrc, uint8_t *pDst,  
uint32_t blockSize)
```

group **Vector bitwise NOT**

Compute the logical bitwise NOT.

There are separate functions for uint32_t, uint16_t, and uint8_t data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_not_u16 (const uint16_t *pSrc, uint16_t *pDst,  
uint32_t blockSize)
```

Compute the logical bitwise NOT of a fixed-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_not_u32 (const uint32_t *pSrc, uint32_t *pDst,  
uint32_t blockSize)
```

Compute the logical bitwise NOT of a fixed-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_not_u8 (const uint8_t *pSrc, uint8_t *pDst,
uint32_t blockSize)
```

Compute the logical bitwise NOT of a fixed-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Vector Offset

```
RISCV_DSP_ATTRIBUTE void riscv_offset_f16 (const float16_t *pSrc, float16_t offset,
float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_offset_f32 (const float32_t *pSrc, float32_t offset,
float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_offset_f64 (const float64_t *pSrc, float64_t offset,
float64_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_offset_q15 (const q15_t *pSrc, q15_t offset, q15_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_offset_q31 (const q31_t *pSrc, q31_t offset, q31_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_offset_q7 (const q7_t *pSrc, q7_t offset, q7_t *pDst,
uint32_t blockSize)
```

group Vector Offset

Adds a constant offset to each element of a vector.

$$pDst[n] = pSrc[n] + offset, \quad 0 \leq n < blockSize.$$

The functions support in-place computation allowing the source and destination pointers to reference the same memory buffer. There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_offset_f16 (const float16_t *pSrc, float16_t offset,
float16_t *pDst, uint32_t blockSize)
```

Adds a constant offset to a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added

- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_offset_f32 (const float32_t *pSrc, float32_t offset, float32_t *pDst, uint32_t blockSize)

Adds a constant offset to a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_offset_f64 (const float64_t *pSrc, float64_t offset, float64_t *pDst, uint32_t blockSize)

Adds a constant offset to a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_offset_q15 (const q15_t *pSrc, q15_t offset, q15_t *pDst, uint32_t blockSize)

Adds a constant offset to a Q15 vector.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_offset_q31 (const q31_t *pSrc, q31_t offset, q31_t *pDst, uint32_t blockSize)

Adds a constant offset to a Q31 vector.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_offset_q7 (const q7_t *pSrc, q7_t offset, q7_t *pDst, uint32_t blockSize)

Adds a constant offset to a Q7 vector.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] are saturated.

Parameters

- **pSrc** – [in] points to the input vector
- **offset** – [in] is the offset to be added
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector bitwise inclusive OR

RISCV_DSP_ATTRIBUTE void riscv_or_u16 (const uint16_t *pSrcA, const uint16_t *pSrcB, uint16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_or_u32 (const uint32_t *pSrcA, const uint32_t *pSrcB, uint32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_or_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB, uint8_t *pDst, uint32_t blockSize)

group **Vector bitwise inclusive OR**

Compute the logical bitwise OR.

There are separate functions for uint32_t, uint16_t, and uint8_t data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_or_u16 (const uint16_t *pSrcA, const uint16_t *pSrcB, uint16_t *pDst, uint32_t blockSize)

Compute the logical bitwise OR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_or_u32 (const uint32_t *pSrcA, const uint32_t *pSrcB, uint32_t *pDst, uint32_t blockSize)

Compute the logical bitwise OR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_or_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB, uint8_t *pDst, uint32_t blockSize)

Compute the logical bitwise OR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Vector Scale

RISCV_DSP_ATTRIBUTE void riscv_scale_f16 (const float16_t *pSrc, float16_t scale, float16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_scale_f32 (const float32_t *pSrc, float32_t scale, float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_scale_f64 (const float64_t *pSrc, float64_t scale, float64_t *pDst, uint32_t blockSize)

```
RISCV_DSP_ATTRIBUTE void riscv_scale_q15 (const q15_t *pSrc, q15_t scaleFract,
int8_t shift, q15_t *pDst, uint32_t blockSize)
```

```
void riscv_scale_q31(const q31_t (page 1277) *pSrc, q31_t (page 1277) scaleFract, int8_t shift, q31_t
(page 1277) *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_scale_q7 (const q7_t *pSrc, q7_t scaleFract, int8_t shift,
q7_t *pDst, uint32_t blockSize)
```

group Vector Scale

Multiply a vector by a scalar value. For floating-point data, the algorithm used is:

$$pDst[n] = pSrc[n] * scale, \quad 0 \leq n < blockSize.$$

In the fixed-point Q7, Q15, and Q31 functions, `scale` is represented by a fractional multiplication `scaleFract` and an arithmetic shift `shift`. The shift allows the gain of the scaling operation to exceed 1.0. The algorithm used with fixed-point data is:

$$pDst[n] = (pSrc[n] * scaleFract) \ll shift, \quad 0 \leq n < blockSize.$$

The overall scale factor applied to the fixed-point data is

$$scale = scaleFract * 2^{shift}.$$

The functions support in-place computation allowing the source and destination pointers to reference the same memory buffer.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_scale_f16 (const float16_t *pSrc, float16_t scale,
float16_t *pDst, uint32_t blockSize)
```

Multiplies a floating-point vector by a scalar.

Parameters

- **pSrc** – [in] points to the input vector
- **scale** – [in] scale factor to be applied
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_scale_f32 (const float32_t *pSrc, float32_t scale,
float32_t *pDst, uint32_t blockSize)
```

Multiplies a floating-point vector by a scalar.

Parameters

- **pSrc** – [in] points to the input vector
- **scale** – [in] scale factor to be applied
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_scale_f64 (const float64_t *pSrc, float64_t scale, float64_t *pDst, uint32_t blockSize)

Multiplies a floating-point vector by a scalar.

Parameters

- **pSrc** – [in] points to the input vector
- **scale** – [in] scale factor to be applied
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_scale_q15 (const q15_t *pSrc, q15_t scaleFract, int8_t shift, q15_t *pDst, uint32_t blockSize)

Multiplies a Q15 vector by a scalar.

Scaling and Overflow Behavior

The input data *pSrc and scaleFract are in 1.15 format. These are multiplied to yield a 2.30 intermediate result and this is shifted with saturation to 1.15 format.

Parameters

- **pSrc** – [in] points to the input vector
- **scaleFract** – [in] fractional portion of the scale value
- **shift** – [in] number of bits to shift the result by
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

void **riscv_scale_q31**(const q31_t (page 1277) *pSrc, q31_t (page 1277) scaleFract, int8_t shift, q31_t (page 1277) *pDst, uint32_t blockSize)

Multiplies a Q31 vector by a scalar.

Scaling and Overflow Behavior

The input data *pSrc and scaleFract are in 1.31 format. These are multiplied to yield a 2.62 intermediate result and this is shifted with saturation to 1.31 format. There is an intermediate shift by 32 to go from the 2.62 to 1.31 format. The shift argument is applied on the 1.31 result and not to the intermediate 2.62 format.

Parameters

- **pSrc** – [in] points to the input vector
- **scaleFract** – [in] fractional portion of the scale value
- **shift** – [in] number of bits to shift the result by
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector


```
RISCV_DSP_ATTRIBUTE void riscv_scale_q7 (const q7_t *pSrc, q7_t scaleFract,
int8_t shift, q7_t *pDst, uint32_t blockSize)
```

Multiplies a Q7 vector by a scalar.

Scaling and Overflow Behavior

The input data **pSrc* and *scaleFract* are in 1.7 format. These are multiplied to yield a 2.14 intermediate result and this is shifted with saturation to 1.7 format.

Parameters

- **pSrc** – [in] points to the input vector
- **scaleFract** – [in] fractional portion of the scale value
- **shift** – [in] number of bits to shift the result by
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector Shift

```
RISCV_DSP_ATTRIBUTE void riscv_shift_q15 (const q15_t *pSrc, int8_t shiftBits,
q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_shift_q31 (const q31_t *pSrc, int8_t shiftBits,
q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_shift_q7 (const q7_t *pSrc, int8_t shiftBits, q7_t *pDst,
uint32_t blockSize)
```

group Vector Shift

Shifts the elements of a fixed-point vector by a specified number of bits. There are separate functions for Q7, Q15, and Q31 data types. The underlying algorithm used is:

```
pDst[n] = pSrc[n] << shift, 0 <= n < blockSize.
```

If *shift* is positive then the elements of the vector are shifted to the left. If *shift* is negative then the elements of the vector are shifted to the right.

The functions support in-place computation allowing the source and destination pointers to reference the same memory buffer.

Functions

RISCV_DSP_ATTRIBUTE void **riscv_shift_q15** (const q15_t *pSrc, int8_t shiftBits, q15_t *pDst, uint32_t blockSize)

Shifts the elements of a Q15 vector a specified number of bits.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrc** – [in] points to the input vector
- **shiftBits** – [in] number of bits to shift. A positive value shifts left; a negative value shifts right.
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void **riscv_shift_q31** (const q31_t *pSrc, int8_t shiftBits, q31_t *pDst, uint32_t blockSize)

Shifts the elements of a Q31 vector a specified number of bits.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrc** – [in] points to the input vector
- **shiftBits** – [in] number of bits to shift. A positive value shifts left; a negative value shifts right.
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in the vector

RISCV_DSP_ATTRIBUTE void **riscv_shift_q7** (const q7_t *pSrc, int8_t shiftBits, q7_t *pDst, uint32_t blockSize)

Shifts the elements of a Q7 vector a specified number of bits.

Conditions for optimum performance

Input and output buffers should be aligned by 32-bit

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] are saturated.

Parameters

- **pSrc** – [in] points to the input vector

- **shiftBits** – [in] number of bits to shift. A positive value shifts left; a negative value shifts right.
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector Subtraction

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f16 (const float16_t *pSrcA, const float16_t *pSrcB,
float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f32 (const float32_t *pSrcA, const float32_t *pSrcB,
float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f64 (const float64_t *pSrcA, const float64_t *pSrcB,
float64_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sub_q15 (const q15_t *pSrcA, const q15_t *pSrcB,
q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sub_q31 (const q31_t *pSrcA, const q31_t *pSrcB,
q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sub_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst,
uint32_t blockSize)
```

group Vector Subtraction

Element-by-element subtraction of two vectors.

```
pDst[n] = pSrcA[n] - pSrcB[n],    0 <= n < blockSize.
```

There are separate functions for floating-point, Q7, Q15, and Q31 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, float16_t *pDst, uint32_t blockSize)
```

Floating-point vector subtraction.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f32 (const float32_t *pSrcA,  
const float32_t *pSrcB, float32_t *pDst, uint32_t blockSize)
```

Floating-point vector subtraction.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_sub_f64 (const float64_t *pSrcA,  
const float64_t *pSrcB, float64_t *pDst, uint32_t blockSize)
```

Floating-point vector subtraction.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_sub_q15 (const q15_t *pSrcA, const q15_t *pSrcB,  
q15_t *pDst, uint32_t blockSize)
```

Q15 vector subtraction.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_sub_q31 (const q31_t *pSrcA, const q31_t *pSrcB,  
q31_t *pDst, uint32_t blockSize)
```

Q31 vector subtraction.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input vector

- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_sub_q7 (const q7_t *pSrcA, const q7_t *pSrcB, q7_t *pDst, uint32_t blockSize)

Q7 vector subtraction.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] will be saturated.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector bitwise exclusive OR

RISCV_DSP_ATTRIBUTE void riscv_xor_u16 (const uint16_t *pSrcA, const uint16_t *pSrcB, uint16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_xor_u32 (const uint32_t *pSrcA, const uint32_t *pSrcB, uint32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_xor_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB, uint8_t *pDst, uint32_t blockSize)

group Vector bitwise exclusive OR

Compute the logical bitwise XOR.

There are separate functions for uint32_t, uint16_t, and uint8_t data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_xor_u16 (const uint16_t *pSrcA, const uint16_t *pSrcB, uint16_t *pDst, uint32_t blockSize)

Compute the logical bitwise XOR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector

- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_xor_u32 (const uint32_t *pSrcA, const uint32_t *pSrcB, uint32_t *pDst, uint32_t blockSize)

Compute the logical bitwise XOR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_xor_u8 (const uint8_t *pSrcA, const uint8_t *pSrcB, uint8_t *pDst, uint32_t blockSize)

Compute the logical bitwise XOR of two fixed-point vectors.

Parameters

- **pSrcA** – [in] points to input vector A
- **pSrcB** – [in] points to input vector B
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

group **Basic Math Functions**

3.3.3 Bayesian estimators

RISCV_DSP_ATTRIBUTE uint32_t riscv_gaussian_naive_bayes_predict_f16 (const riscv_gaussian_naive_bayes_in_t *in, float16_t *pOutputProbabilities, float16_t *pBufferB)

RISCV_DSP_ATTRIBUTE uint32_t riscv_gaussian_naive_bayes_predict_f32 (const riscv_gaussian_naive_bayes_in_t *in, float32_t *pOutputProbabilities, float32_t *pBufferB)

group **Bayesian estimators**

Implement the naive gaussian Bayes estimator. The training must be done from scikit-learn.

The parameters can be easily generated from the scikit-learn object. Some examples are given in DSP/Testing/PatternGeneration/Bayes.py

Functions

RISCV_DSP_ATTRIBUTE uint32_t riscv_gaussian_naive_bayes_predict_f16 (const riscv_gaussian_naive_bayes_instance_t *pInstance, const float16_t *in, float16_t *pOutputProbabilities, float16_t *pBufferB)

Naive Gaussian Bayesian Estimator.

Parameters

- ***S** – [in] points to a naive bayes instance structure
- ***in** – [in] points to the elements of the input vector.
- ***pOutputProbabilities** – [out] points to a buffer of length numberOfClasses containing estimated probabilities
- ***pBufferB** – [out] points to a temporary buffer of length numberOfClasses

Returns

The predicted class

RISCV_DSP_ATTRIBUTE uint32_t riscv_gaussian_naive_bayes_predict_f32 (const riscv_gaussian_naive_bayes_instance_t *pInstance, const float32_t *in, float32_t *pOutputProbabilities, float32_t *pBufferB)

Naive Gaussian Bayesian Estimator.

Parameters

- ***S** – [in] points to a naive bayes instance structure
- ***in** – [in] points to the elements of the input vector.
- ***pOutputProbabilities** – [out] points to a buffer of length numberOfClasses containing estimated probabilities
- ***pBufferB** – [out] points to a temporary buffer of length numberOfClasses

Returns

The predicted class

3.3.4 Complex Math Functions

Complex Conjugate

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_f16 (const float16_t *pSrc, float16_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t numSamples)

group **Complex Conjugate**

Conjugates the elements of a complex data vector.

The `pSrc` points to the source data and `pDst` points to the destination data where the result should be written. `numSamples` specifies the number of complex samples and the data in each array is stored in an interleaved fashion (real, imag, real, imag, ...). Each array has a total of `2*numSamples` values.

The underlying algorithm is used:

```
for (n = 0; n < numSamples; n++) {  
    pDst[(2*n) ] = pSrc[(2*n) ];    // real part  
    pDst[(2*n)+1] = -pSrc[(2*n)+1]; // imag part  
}
```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_f16 (const float16_t *pSrc, float16_t *pDst, uint32_t numSamples)

Floating-point complex conjugate.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t numSamples)

Floating-point complex conjugate.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t numSamples)

Q15 complex conjugate.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q15 value -1 (0x8000) is saturated to the maximum allowable positive value 0x7FFF.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector

- **numSamples** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_conj_q31 (const q31_t *pSrc, q31_t *pDst,
uint32_t numSamples)
```

Q31 complex conjugate.

Scaling and Overflow Behavior

The function uses saturating arithmetic. The Q31 value -1 (0x80000000) is saturated to the maximum allowable positive value 0x7FFFFFFF.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **numSamples** – [in] number of samples in each vector

Complex Dot Product

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, uint32_t numSamples, float16_t *realResult,
float16_t *imagResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, uint32_t numSamples, float32_t *realResult,
float32_t *imagResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_q15 (const q15_t *pSrcA,
const q15_t *pSrcB, uint32_t numSamples, q31_t *realResult, q31_t *imagResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_q31 (const q31_t *pSrcA,
const q31_t *pSrcB, uint32_t numSamples, q63_t *realResult, q63_t *imagResult)
```

group Complex Dot Product

Computes the dot product of two complex vectors. The vectors are multiplied element-by-element and then summed.

The pSrcA points to the first complex input vector and pSrcB points to the second complex input vector. numSamples specifies the number of complex samples and the data in each array is stored in an interleaved fashion (real, imag, real, imag, ...). Each array has a total of 2*numSamples values.

The underlying algorithm is used:

```
realResult = 0;
imagResult = 0;
for (n = 0; n < numSamples; n++) {
    realResult += pSrcA[(2*n)+0] * pSrcB[(2*n)+0] - pSrcA[(2*n)+1] * pSrcB[(2*n)+1];
    imagResult += pSrcA[(2*n)+0] * pSrcB[(2*n)+1] + pSrcA[(2*n)+1] * pSrcB[(2*n)+0];
}
```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_f16 (const float16_t *pSrcA, const float16_t *pSrcB, uint32_t numSamples, float16_t *realResult, float16_t *imagResult)

Floating-point complex dot product.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **numSamples** – [in] number of samples in each vector
- **realResult** – [out] real part of the result returned here
- **imagResult** – [out] imaginary part of the result returned here

RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t numSamples, float32_t *realResult, float32_t *imagResult)

Floating-point complex dot product.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **numSamples** – [in] number of samples in each vector
- **realResult** – [out] real part of the result returned here
- **imagResult** – [out] imaginary part of the result returned here

RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_q15 (const q15_t *pSrcA, const q15_t *pSrcB, uint32_t numSamples, q31_t *realResult, q31_t *imagResult)

Q15 complex dot product.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The intermediate 1.15 by 1.15 multiplications are performed with full precision and yield a 2.30 result. These are accumulated in a 64-bit accumulator with 34.30 precision. As a final step, the accumulators are converted to 8.24 format. The return results **realResult** and **imagResult** are in 8.24 format.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **numSamples** – [in] number of samples in each vector
- **realResult** – [out] real part of the result returned here
- **imagResult** – [out] imaginary part of the result returned her

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_dot_prod_q31 (const q31_t *pSrcA,
const q31_t *pSrcB, uint32_t numSamples, q63_t *realResult, q63_t *imagResult)
```

Q31 complex dot product.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The intermediate 1.31 by 1.31 multiplications are performed with 64-bit precision and then shifted to 16.48 format. The internal real and imaginary accumulators are in 16.48 format and provide 15 guard bits. Additions are nonsaturating and no overflow will occur as long as `numSamples` is less than 32768. The return results `realResult` and `imagResult` are in 16.48 format. Input down scaling is not required.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **numSamples** – [in] number of samples in each vector
- **realResult** – [out] real part of the result returned here
- **imagResult** – [out] imaginary part of the result returned here

Complex Magnitude

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f64 (const float64_t *pSrc, float64_t *pDst,
uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_fast_q15 (const q15_t *pSrc, q15_t *pDst,
uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_q15 (const q15_t *pSrc, q15_t *pDst,
uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_q31 (const q31_t *pSrc, q31_t *pDst,
uint32_t numSamples)
```

group Complex Magnitude

Computes the magnitude of the elements of a complex data vector.

The `pSrc` points to the source data and `pDst` points to the where the result should be written. `numSamples` specifies the number of complex samples in the input array and the data is stored in an interleaved fashion (real, imag, real, imag, ...). The input array has a total of `2*numSamples` values; the output array has a total of `numSamples` values.

The underlying algorithm is used:

```
for (n = 0; n < numSamples; n++) {  
    pDst[n] = sqrt(pSrc[(2*n)+0]^2 + pSrc[(2*n)+1]^2);  
}
```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

**RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f16 (const float16_t *pSrc,
float16_t *pDst, uint32_t numSamples)**

Floating-point complex magnitude.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

**RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f32 (const float32_t *pSrc,
float32_t *pDst, uint32_t numSamples)**

Floating-point complex magnitude.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

**RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_f64 (const float64_t *pSrc,
float64_t *pDst, uint32_t numSamples)**

Floating-point complex magnitude.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

**RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_fast_q15 (const q15_t *pSrc, q15_t *pDst,
uint32_t numSamples)**

Q15 complex magnitude.

Scaling and Overflow Behavior

The function implements 1.15 by 1.15 multiplications and finally output is converted into 2.14 format. Fast functions are less accurate. This function will tend to clamp to 0 the too small values. So $\text{sqrt}(x*x) = x$ will not always be true.

Parameters

- **pSrc** – [in] points to input vector

- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t numSamples)

Q15 complex magnitude.

Scaling and Overflow Behavior

The function implements 1.15 by 1.15 multiplications and finally output is converted into 2.14 format.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t numSamples)

Q31 complex magnitude.

Scaling and Overflow Behavior

The function implements 1.31 by 1.31 multiplications and finally output is converted into 2.30 format. Input down scaling is not required.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

Complex Magnitude Squared

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f16 (const float16_t *pSrc, float16_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f64 (const float64_t *pSrc, float64_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t numSamples)

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t numSamples)

group **Complex Magnitude Squared**

Computes the magnitude squared of the elements of a complex data vector.

The `pSrc` points to the source data and `pDst` points to the where the result should be written. `numSamples` specifies the number of complex samples in the input array and the data is stored in an interleaved fashion (real, imag, real, imag, ...). The input array has a total of $2 \times \text{numSamples}$ values; the output array has a total of `numSamples` values.

The underlying algorithm is used:

```
for (n = 0; n < numSamples; n++) {  
    pDst[n] = pSrc[(2*n)+0]^2 + pSrc[(2*n)+1]^2;  
}
```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f16 (const float16_t *pSrc, float16_t *pDst, uint32_t numSamples)

Floating-point complex magnitude squared.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t numSamples)

Floating-point complex magnitude squared.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_f64 (const float64_t *pSrc, float64_t *pDst, uint32_t numSamples)

Floating-point complex magnitude squared.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t numSamples)

Q15 complex magnitude squared.

Scaling and Overflow Behavior

The function implements 1.15 by 1.15 multiplications and finally output is converted into 3.13 format.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mag_squared_q31 (const q31_t *pSrc,
q31_t *pDst, uint32_t numSamples)
```

Q31 complex magnitude squared.

Scaling and Overflow Behavior

The function implements 1.31 by 1.31 multiplications and finally output is converted into 3.29 format. Input down scaling is not required.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

Complex-by-Complex Multiplication

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, float16_t *pDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, float32_t *pDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f64 (const float64_t *pSrcA,
const float64_t *pSrcB, float64_t *pDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_q15 (const q15_t *pSrcA,
const q15_t *pSrcB, q15_t *pDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_q31 (const q31_t *pSrcA,
const q31_t *pSrcB, q31_t *pDst, uint32_t numSamples)
```

group Complex-by-Complex Multiplication

Multiplies a complex vector by another complex vector and generates a complex result. The data in the complex arrays is stored in an interleaved fashion (real, imag, real, imag, ...). The parameter numSamples represents the number of complex samples processed. The complex arrays have a total of 2*numSamples real values.

The underlying algorithm is used:

```

for (n = 0; n < numSamples; n++) {
    pDst[(2*n)+0] = pSrcA[(2*n)+0] * pSrcB[(2*n)+0] - pSrcA[(2*n)+1] *
↪ pSrcB[(2*n)+1];
    pDst[(2*n)+1] = pSrcA[(2*n)+0] * pSrcB[(2*n)+1] + pSrcA[(2*n)+1] *
↪ pSrcB[(2*n)+0];
}

```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f16 (const float16_t *pSrcA, const float16_t *pSrcB, float16_t *pDst, uint32_t numSamples)

Floating-point complex-by-complex multiplication.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f32 (const float32_t *pSrcA, const float32_t *pSrcB, float32_t *pDst, uint32_t numSamples)

Floating-point complex-by-complex multiplication.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_f64 (const float64_t *pSrcA, const float64_t *pSrcB, float64_t *pDst, uint32_t numSamples)

Floating-point complex-by-complex multiplication.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_q15 (const q15_t *pSrcA, const q15_t *pSrcB, q15_t *pDst, uint32_t numSamples)

Q15 complex-by-complex multiplication.

Scaling and Overflow Behavior

The function implements 1.15 by 1.15 multiplications and finally output is converted into 3.13 format.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_cmplx_q31 (const q31_t *pSrcA,
const q31_t *pSrcB, q31_t *pDst, uint32_t numSamples)
```

Q31 complex-by-complex multiplication.

Scaling and Overflow Behavior

The function implements 1.31 by 1.31 multiplications and finally output is converted into 3.29 format. Input down scaling is not required.

Parameters

- **pSrcA** – [in] points to first input vector
- **pSrcB** – [in] points to second input vector
- **pDst** – [out] points to output vector
- **numSamples** – [in] number of samples in each vector

Complex-by-Real Multiplication

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_f16 (const float16_t *pSrcCmplx,
const float16_t *pSrcReal, float16_t *pCmplxDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_f32 (const float32_t *pSrcCmplx,
const float32_t *pSrcReal, float32_t *pCmplxDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_q15 (const q15_t *pSrcCmplx,
const q15_t *pSrcReal, q15_t *pCmplxDst, uint32_t numSamples)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_q31 (const q31_t *pSrcCmplx,
const q31_t *pSrcReal, q31_t *pCmplxDst, uint32_t numSamples)
```

group **Complex-by-Real Multiplication**

Multiplies a complex vector by a real vector and generates a complex result. The data in the complex arrays is stored in an interleaved fashion (real, imag, real, imag, ...). The parameter numSamples represents the number of complex samples processed. The complex arrays have a total of 2*numSamples real values while the real array has a total of numSamples real values.

The underlying algorithm is used:

```

for (n = 0; n < numSamples; n++) {
    pCmplxDst[(2*n)+0] = pSrcCmplx[(2*n)+0] * pSrcReal[n];
    pCmplxDst[(2*n)+1] = pSrcCmplx[(2*n)+1] * pSrcReal[n];
}

```

There are separate functions for floating-point, Q15, and Q31 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_f16 (const float16_t *pSrcCmplx, const float16_t *pSrcReal, float16_t *pCmplxDst, uint32_t numSamples)

Floating-point complex-by-real multiplication.

Parameters

- **pSrcCmplx** – [in] points to complex input vector
- **pSrcReal** – [in] points to real input vector
- **pCmplxDst** – [out] points to complex output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_f32 (const float32_t *pSrcCmplx, const float32_t *pSrcReal, float32_t *pCmplxDst, uint32_t numSamples)

Floating-point complex-by-real multiplication.

Parameters

- **pSrcCmplx** – [in] points to complex input vector
- **pSrcReal** – [in] points to real input vector
- **pCmplxDst** – [out] points to complex output vector
- **numSamples** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_q15 (const q15_t *pSrcCmplx, const q15_t *pSrcReal, q15_t *pCmplxDst, uint32_t numSamples)

Q15 complex-by-real multiplication.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcCmplx** – [in] points to complex input vector
- **pSrcReal** – [in] points to real input vector
- **pCmplxDst** – [out] points to complex output vector
- **numSamples** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_cmplx_mult_real_q31 (const q31_t *pSrcCmplx,
const q31_t *pSrcReal, q31_t *pCmplxDst, uint32_t numSamples)
```

Q31 complex-by-real multiplication.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range[0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcCmplx** – [in] points to complex input vector
- **pSrcReal** – [in] points to real input vector
- **pCmplxDst** – [out] points to complex output vector
- **numSamples** – [in] number of samples in each vector

group Complex Math Functions

This set of functions operates on complex data vectors. The data in the complex arrays is stored in an interleaved fashion (real, imag, real, imag, ...). In the API functions, the number of samples in a complex array refers to the number of complex values; the array contains twice this number of real values.

3.3.5 Controller Functions

Sine Cosine

```
RISCV_DSP_ATTRIBUTE void riscv_sin_cos_f32 (float32_t theta, float32_t *pSinVal,
float32_t *pCosVal)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sin_cos_q31 (q31_t theta, q31_t *pSinVal, q31_t *pCosVal)
```

group Sine Cosine

Computes the trigonometric sine and cosine values using a combination of table lookup and linear interpolation. There are separate functions for Q31 and floating-point data types. The input to the floating-point version is in degrees while the fixed-point Q31 have a scaled input with the range [-1 0.9999] mapping to [-180 +180] degrees.

The floating point function also allows values that are out of the usual range. When this happens, the function will take extra time to adjust the input value to the range of [-180 180].

The result is accurate to 5 digits after the decimal point.

The implementation is based on table lookup using 360 values together with linear interpolation. The steps used are:

1. Calculation of the nearest integer table index.
2. Compute the fractional portion (fract) of the input.
3. Fetch the value corresponding to `index` from sine table to `y0` and also value from `index+1` to `y1`.
4. Sine value is computed as `*psinVal = y0 + (fract * (y1 - y0))`.
5. Fetch the value corresponding to `index` from cosine table to `y0` and also value from `index+1` to `y1`.

6. Cosine value is computed as $*pCosVal = y0 + (fract * (y1 - y0))$.

Functions

RISCV_DSP_ATTRIBUTE void riscv_sin_cos_f32 (float32_t theta, float32_t *pSinVal, float32_t *pCosVal)

Floating-point sin_cos function.

Parameters

- **theta** – [in] input value in degrees
- **pSinVal** – [out] points to processed sine output
- **pCosVal** – [out] points to processed cosine output

RISCV_DSP_ATTRIBUTE void riscv_sin_cos_q31 (q31_t theta, q31_t *pSinVal, q31_t *pCosVal)

Q31 sin_cos function.

The Q31 input value is in the range [-1 0.999999] and is mapped to a degree value in the range [-180 179].

Parameters

- **theta** – [in] scaled input value in degrees
- **pSinVal** – [out] points to processed sine output
- **pCosVal** – [out] points to processed cosine output

PID Motor Control

__STATIC_FORCEINLINE float32_t riscv_pid_f32 (riscv_pid_instance_f32 *S, float32_t in)

__STATIC_FORCEINLINE q31_t riscv_pid_q31 (riscv_pid_instance_q31 *S, q31_t in)

__STATIC_FORCEINLINE q15_t riscv_pid_q15 (riscv_pid_instance_q15 *S, q15_t in)

RISCV_DSP_ATTRIBUTE void riscv_pid_init_f32 (riscv_pid_instance_f32 *S, int32_t resetStateFlag)

RISCV_DSP_ATTRIBUTE void riscv_pid_init_q15 (riscv_pid_instance_q15 *S, int32_t resetStateFlag)

RISCV_DSP_ATTRIBUTE void riscv_pid_init_q31 (riscv_pid_instance_q31 *S, int32_t resetStateFlag)

RISCV_DSP_ATTRIBUTE void riscv_pid_reset_f32 (riscv_pid_instance_f32 *S)

```
RISCV_DSP_ATTRIBUTE void riscv_pid_reset_q15 (riscv_pid_instance_q15 *S)
```

```
RISCV_DSP_ATTRIBUTE void riscv_pid_reset_q31 (riscv_pid_instance_q31 *S)
```

```
struct riscv_pid_instance_q15
```

```
struct riscv_pid_instance_q31
```

```
struct riscv_pid_instance_f32
```

group PID Motor Control

A Proportional Integral Derivative (PID) controller is a generic feedback control loop mechanism widely used in industrial control systems. A PID controller is the most commonly used type of feedback controller.

This set of functions implements (PID) controllers for Q15, Q31, and floating-point data types. The functions operate on a single sample of data and each call to the function returns a single processed value. S points to an instance of the PID control data structure. in is the input sample value. The functions return the output value.

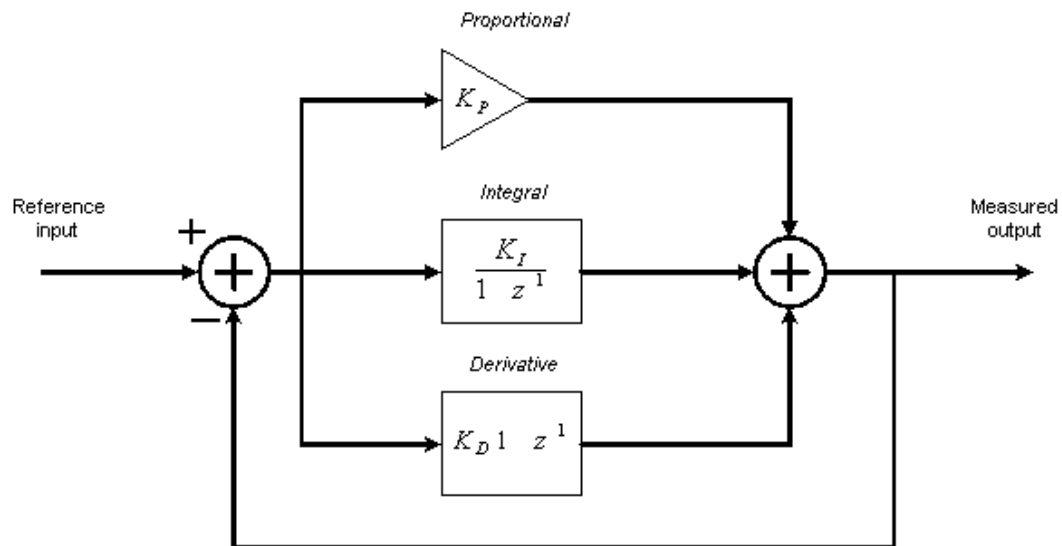
Algorithm:

```

y[n] = y[n-1] + A0 * x[n] + A1 * x[n-1] + A2 * x[n-2]
A0 = Kp + Ki + Kd
A1 = (-Kp) - (2 * Kd)
A2 = Kd

```

where Kp is proportional constant, Ki is Integral constant and Kd is Derivative constant



The PID controller calculates an “error” value as the difference between the measured output and the reference input. The controller attempts to minimize the error by adjusting the process control inputs. The

proportional value determines the reaction to the current error, the integral value determines the reaction based on the sum of recent errors, and the derivative value determines the reaction based on the rate at which the error has been changing.

Instance Structure

The Gains A0, A1, A2 and state variables for a PID controller are stored together in an instance data structure. A separate instance structure must be defined for each PID Controller. There are separate instance structure declarations for each of the 3 supported data types.

Reset Functions

There is also an associated reset function for each data type which clears the state array.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Initializes the Gains A0, A1, A2 from Kp, Ki, Kd gains.
- Zeros out the values in the state buffer.

Instance structure cannot be placed into a const data section and it is recommended to use the initialization function.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the PID Controller functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

```
__STATIC_FORCEINLINE float32_t riscv_pid_f32 (riscv_pid_instance_f32 *S,  
float32_t in)
```

Process function for the floating-point PID Control.

Parameters

- **S** – [inout] is an instance of the floating-point PID Control structure
- **in** – [in] input sample to process

Returns

processed output sample.

```
__STATIC_FORCEINLINE q31_t riscv_pid_q31 (riscv_pid_instance_q31 *S, q31_t in)
```

Process function for the Q31 PID Control.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by 2 bits as there are four additions. After all multiply-accumulates are performed, the 2.62 accumulator is truncated to 1.32 format and then saturated to 1.31 format.

Parameters

- **S** – [inout] points to an instance of the Q31 PID Control structure
- **in** – [in] input sample to process

Returns

processed output sample.

```
__STATIC_FORCEINLINE q15_t riscv_pid_q15 (riscv_pid_instance_q15 *S, q15_t in)
```

Process function for the Q15 PID Control.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both Gains and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

Parameters

- **S** – [inout] points to an instance of the Q15 PID Control structure
- **in** – [in] input sample to process

Returns

processed output sample.

```
RISCV_DSP_ATTRIBUTE void riscv_pid_init_f32 (riscv_pid_instance_f32 *S,  
int32_t resetStateFlag)
```

Initialization function for the floating-point PID Control.

Details

The resetStateFlag specifies whether to set state to zero or not.

The function computes the structure fields: **A0**, **A1** **A2** using the proportional gain(**Kp**), integral gain(**Ki**) and derivative gain(**Kd**) also sets the state variables to all zeros.

Parameters

- **S** – [inout] points to an instance of the PID structure
- **resetStateFlag** – [in]
 - value = 0: no change in state
 - value = 1: reset state

```
RISCV_DSP_ATTRIBUTE void riscv_pid_init_q15 (riscv_pid_instance_q15 *S,  
int32_t resetStateFlag)
```

Initialization function for the Q15 PID Control.

Details

The resetStateFlag specifies whether to set state to zero or not.

The function computes the structure fields: **A0**, **A1** **A2** using the proportional gain(**Kp**), integral gain(**Ki**) and derivative gain(**Kd**) also sets the state variables to all zeros.

Parameters

- **S** – **[inout]** points to an instance of the Q15 PID structure
- **resetStateFlag** – **[in]**
 - value = 0: no change in state
 - value = 1: reset state

RISCV_DSP_ATTRIBUTE void riscv_pid_init_q31 (riscv_pid_instance_q31 *S, int32_t resetStateFlag)

Initialization function for the Q31 PID Control.

Details

The **resetStateFlag** specifies whether to set state to zero or not.

The function computes the structure fields: **A0**, **A1** **A2** using the proportional gain(**Kp**), integral gain(**Ki**) and derivative gain(**Kd**) also sets the state variables to all zeros.

Parameters

- **S** – **[inout]** points to an instance of the Q31 PID structure
- **resetStateFlag** – **[in]**
 - value = 0: no change in state
 - value = 1: reset state

RISCV_DSP_ATTRIBUTE void riscv_pid_reset_f32 (riscv_pid_instance_f32 *S)

Reset function for the floating-point PID Control.

Details

The function resets the state buffer to zeros.

Parameters

S – **[inout]** points to an instance of the floating-point PID structure

RISCV_DSP_ATTRIBUTE void riscv_pid_reset_q15 (riscv_pid_instance_q15 *S)

Reset function for the Q15 PID Control.

Details

The function resets the state buffer to zeros.

Parameters

S – **[inout]** points to an instance of the Q15 PID structure

RISCV_DSP_ATTRIBUTE void riscv_pid_reset_q31 (riscv_pid_instance_q31 *S)

Reset function for the Q31 PID Control.

Details

The function resets the state buffer to zeros.

Parameters

S – [inout] points to an instance of the Q31 PID structure

struct **riscv_pid_instance_q15**

#include <controller_functions.h> Instance structure for the Q15 PID Control.

struct **riscv_pid_instance_q31**

#include <controller_functions.h> Instance structure for the Q31 PID Control.

struct **riscv_pid_instance_f32**

#include <controller_functions.h> Instance structure for the floating-point PID Control.

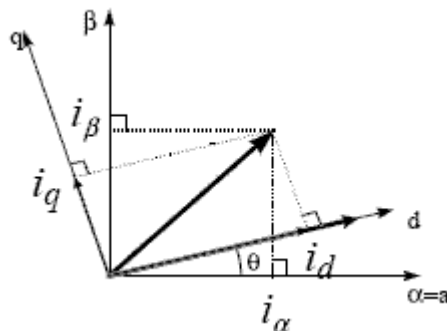
Vector Park Transform

__STATIC_FORCEINLINE void riscv_park_f32 (float32_t Ialpha, float32_t Ibeta, float32_t *pId, float32_t *pIq, float32_t sinVal, float32_t cosVal)

__STATIC_FORCEINLINE void riscv_park_q31 (q31_t Ialpha, q31_t Ibeta, q31_t *pId, q31_t *pIq, q31_t sinVal, q31_t cosVal)

group Vector Park Transform

Forward Park transform converts the input two-coordinate vector to flux and torque components. The Park transform can be used to realize the transformation of the Ialpha and the Ibeta currents from the stationary to the moving reference frame and control the spatial relationship between the stator vector current and rotor flux vector. If we consider the d axis aligned with the rotor flux, the diagram below shows the current vector and the



relationship from the two reference frames:

The function operates on a single sample of data and each call to the function returns the processed output. The library provides separate functions for Q31 and floating-point data types.

Algorithm

where `Ialpha` and `Ibeta` are the stator vector components, `pId` and `pIq` are rotor vector components and `cosVal` and `sinVal` are the cosine and sine values of `theta` (rotor flux position).

$$\begin{aligned} pId &= Ialpha * cosVal + Ibeta * sinVal \\ pIq &= - Ialpha sinVal + Ibeta * cosVal \end{aligned}$$

Fixed-Point Behavior

Care must be taken when using the Q31 version of the Park transform. In particular, the overflow and saturation behavior of the accumulator used must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

```
__STATIC_FORCEINLINE void riscv_park_f32 (float32_t Ialpha, float32_t Ibeta,  
float32_t *pId, float32_t *pIq, float32_t sinVal, float32_t cosVal)
```

Floating-point Park transform.

The function implements the forward Park transform.

Parameters

- **Ialpha** – [in] input two-phase vector coordinate alpha
- **Ibeta** – [in] input two-phase vector coordinate beta
- **pId** – [out] points to output rotor reference frame d
- **pIq** – [out] points to output rotor reference frame q
- **sinVal** – [in] sine value of rotation angle theta
- **cosVal** – [in] cosine value of rotation angle theta

```
__STATIC_FORCEINLINE void riscv_park_q31 (q31_t Ialpha, q31_t Ibeta, q31_t *pId,  
q31_t *pIq, q31_t sinVal, q31_t cosVal)
```

Park transform for Q31 version.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The accumulator maintains 1.31 format by truncating lower 31 bits of the intermediate multiplication in 2.62 format. There is saturation on the addition and subtraction, hence there is no risk of overflow.

Parameters

- **Ialpha** – [in] input two-phase vector coordinate alpha
- **Ibeta** – [in] input two-phase vector coordinate beta
- **pId** – [out] points to output rotor reference frame d
- **pIq** – [out] points to output rotor reference frame q
- **sinVal** – [in] sine value of rotation angle theta
- **cosVal** – [in] cosine value of rotation angle theta

Vector Inverse Park transform

```
__STATIC_FORCEINLINE void riscv_inv_park_f32 (float32_t Id, float32_t Iq,
float32_t *pIalpha, float32_t *pIbeta, float32_t sinVal, float32_t cosVal)
```

```
__STATIC_FORCEINLINE void riscv_inv_park_q31 (q31_t Id, q31_t Iq, q31_t *pIalpha,
q31_t *pIbeta, q31_t sinVal, q31_t cosVal)
```

group Vector Inverse Park transform

Inverse Park transform converts the input flux and torque components to two-coordinate vector.

The function operates on a single sample of data and each call to the function returns the processed output. The library provides separate functions for Q31 and floating-point data types.

Algorithm

where pIalpha and pIbeta are the stator vector components, Id and Iq are rotor vector components and cosVal and sinVal are the cosine and sine values of theta (rotor flux position).

$$\begin{aligned} pIalpha &= Id * \cosVal - Iq * \sinVal \\ pIbeta &= Id * \sinVal + Iq * \cosVal \end{aligned}$$

Fixed-Point Behavior

Care must be taken when using the Q31 version of the Park transform. In particular, the overflow and saturation behavior of the accumulator used must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

```
__STATIC_FORCEINLINE void riscv_inv_park_f32 (float32_t Id, float32_t Iq,
float32_t *pIalpha, float32_t *pIbeta, float32_t sinVal, float32_t cosVal)
```

Floating-point Inverse Park transform.

Parameters

- **Id** – [in] input coordinate of rotor reference frame d
- **Iq** – [in] input coordinate of rotor reference frame q
- **pIalpha** – [out] points to output two-phase orthogonal vector axis alpha
- **pIbeta** – [out] points to output two-phase orthogonal vector axis beta
- **sinVal** – [in] sine value of rotation angle theta
- **cosVal** – [in] cosine value of rotation angle theta

```
__STATIC_FORCEINLINE void riscv_inv_park_q31 (q31_t Id, q31_t Iq, q31_t *pIalpha,
q31_t *pIbeta, q31_t sinVal, q31_t cosVal)
```

Inverse Park transform for Q31 version.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The accumulator maintains 1.31 format by truncating lower 31 bits of the intermediate multiplication in 2.62 format. There is saturation on the addition, hence there is no risk of overflow.

Parameters

- **Id** – [in] input coordinate of rotor reference frame d
- **Iq** – [in] input coordinate of rotor reference frame q
- **pIalpha** – [out] points to output two-phase orthogonal vector axis alpha
- **pIbeta** – [out] points to output two-phase orthogonal vector axis beta
- **sinVal** – [in] sine value of rotation angle theta
- **cosVal** – [in] cosine value of rotation angle theta

Vector Clarke Transform

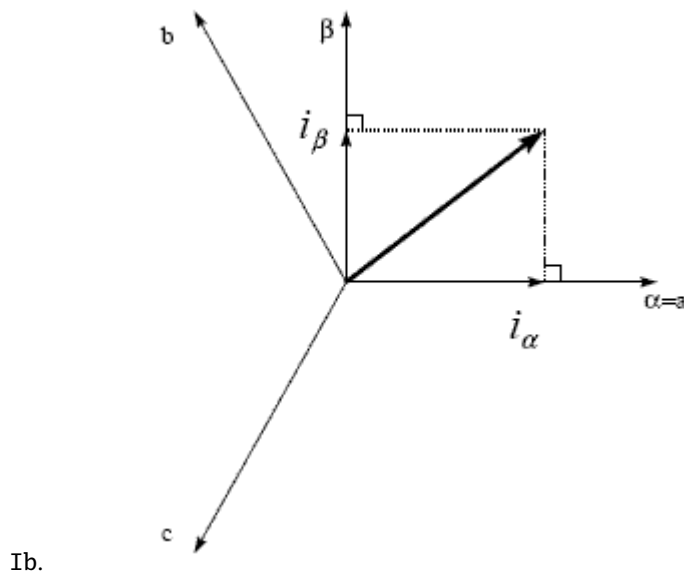
```
__STATIC_FORCEINLINE void riscv_clarke_f32 (float32_t Ia, float32_t Ib,
float32_t *pIalpha, float32_t *pIbeta)
```

```
__STATIC_FORCEINLINE void riscv_clarke_q31 (q31_t Ia, q31_t Ib, q31_t *pIalpha,
q31_t *pIbeta)
```

group Vector Clarke Transform

Forward Clarke transform converts the instantaneous stator phases into a two-coordinate time invariant vector. Generally the Clarke transform uses three-phase currents I_a , I_b and I_c to calculate currents in the two-phase orthogonal stator axis I_{α} and I_{β} . When I_{α} is superposed with I_a as shown in the figure below

and $I_a + I_b + I_c = 0$, in this condition I_{α} and I_{β} can be calculated using only I_a and



The function operates on a single sample of data and each call to the function returns the processed output. The library provides separate functions for Q31 and floating-point data types.

Algorithm

where I_a and I_b are the instantaneous stator phases and pI_{α} and pI_{β} are the two coordinates of

$$pI_{\alpha} = I_a$$

$$pI_{\beta} = (1/\sqrt{3}) I_a + (2/\sqrt{3}) I_b$$

time invariant vector.

Fixed-Point Behavior

Care must be taken when using the Q31 version of the Clarke transform. In particular, the overflow and saturation behavior of the accumulator used must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

```
__STATIC_FORCEINLINE void riscv_clarke_f32 (float32_t  $I_a$ , float32_t  $I_b$ ,
float32_t * $pI_{\alpha}$ , float32_t * $pI_{\beta}$ )
```

Floating-point Clarke transform.

Parameters

- I_a – [in] input three-phase coordinate a
- I_b – [in] input three-phase coordinate b
- pI_{α} – [out] points to output two-phase orthogonal vector axis alpha
- pI_{β} – [out] points to output two-phase orthogonal vector axis beta

```
__STATIC_FORCEINLINE void riscv_clarke_q31 (q31_t  $I_a$ , q31_t  $I_b$ , q31_t * $pI_{\alpha}$ ,
q31_t * $pI_{\beta}$ )
```

Clarke transform for Q31 version.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The accumulator maintains 1.31 format by truncating lower 31 bits of the intermediate multiplication in 2.62 format. There is saturation on the addition, hence there is no risk of overflow.

Parameters

- I_a – [in] input three-phase coordinate a
- I_b – [in] input three-phase coordinate b
- pI_{α} – [out] points to output two-phase orthogonal vector axis alpha
- pI_{β} – [out] points to output two-phase orthogonal vector axis beta

Vector Inverse Clarke Transform

```
__STATIC_FORCEINLINE void riscv_inv_clarke_f32 (float32_t Ialpha, float32_t Ibeta,  
float32_t *pIa, float32_t *pIb)
```

```
__STATIC_FORCEINLINE void riscv_inv_clarke_q31 (q31_t Ialpha, q31_t Ibeta, q31_t *pIa,  
q31_t *pIb)
```

group Vector Inverse Clarke Transform

Inverse Clarke transform converts the two-coordinate time invariant vector into instantaneous stator phases.

The function operates on a single sample of data and each call to the function returns the processed output. The library provides separate functions for Q31 and floating-point data types.

Algorithm

where pIa and pIb are the instantaneous stator phases and Ialpha and Ibeta are the two coordinates of

$$pIa = Ialpha$$

$$pIb = (-1/2) Ialpha + (\sqrt{3}/2) Ibeta$$

time invariant vector.

Fixed-Point Behavior

Care must be taken when using the Q31 version of the Clarke transform. In particular, the overflow and saturation behavior of the accumulator used must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

```
__STATIC_FORCEINLINE void riscv_inv_clarke_f32 (float32_t Ialpha, float32_t Ibeta,  
float32_t *pIa, float32_t *pIb)
```

Floating-point Inverse Clarke transform.

Parameters

- **Ialpha** – [in] input two-phase orthogonal vector axis alpha
- **Ibeta** – [in] input two-phase orthogonal vector axis beta
- **pIa** – [out] points to output three-phase coordinate a
- **pIb** – [out] points to output three-phase coordinate b

```
__STATIC_FORCEINLINE void riscv_inv_clarke_q31 (q31_t Ialpha, q31_t Ibeta,  
q31_t *pIa, q31_t *pIb)
```

Inverse Clarke transform for Q31 version.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The accumulator maintains 1.31 format by truncating lower 31 bits of the intermediate multiplication in 2.62 format. There is saturation on the subtraction, hence there is no risk of overflow.

Parameters

- **Ialpha** – [in] input two-phase orthogonal vector axis alpha
- **Ibeta** – [in] input two-phase orthogonal vector axis beta
- **pIa** – [out] points to output three-phase coordinate a
- **pIb** – [out] points to output three-phase coordinate b

group **Controller Functions**

3.3.6 Distance Functions

Float Distances

Bray-Curtis distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_braycurtis_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_braycurtis_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

group **Bray-Curtis distance**

Bray-Curtis distance between two vectors

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_braycurtis_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

Bray-Curtis distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_braycurtis_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

Bray-Curtis distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Canberra distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_canberra_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_canberra_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

group **Canberra distance**

Canberra distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_canberra_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

Canberra distance between two vectors.

This function may divide by zero when samples pA[i] and pB[i] are both zero. The result of the computation will be correct. So the division per zero may be ignored.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_canberra_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

Canberra distance between two vectors.

This function may divide by zero when samples pA[i] and pB[i] are both zero. The result of the computation will be correct. So the division per zero may be ignored.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Chebyshev distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_chebyshev_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_chebyshev_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float64_t riscv_chebyshev_distance_f64 (const float64_t *pA,
const float64_t *pB, uint32_t blockSize)
```

group Chebyshev distance

Chebyshev distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_chebyshev_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

Chebyshev distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_chebyshev_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

Chebyshev distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float64_t riscv_chebyshev_distance_f64 (const float64_t *pA,
const float64_t *pB, uint32_t blockSize)
```

Chebyshev distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector

- **blockSize** – [in] vector length

Returns

distance

Cityblock (Manhattan) distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_cityblock_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_cityblock_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float64_t riscv_cityblock_distance_f64 (const float64_t *pA,  
const float64_t *pB, uint32_t blockSize)
```

group **Cityblock (Manhattan) distance**

Cityblock (Manhattan) distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_cityblock_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

Cityblock (Manhattan) distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_cityblock_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

Cityblock (Manhattan) distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float64_t riscv_cityblock_distance_f64 (const float64_t *pA,  
const float64_t *pB, uint32_t blockSize)
```

Cityblock (Manhattan) distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Correlation distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_correlation_distance_f16 (float16_t *pA,
float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_correlation_distance_f32 (float32_t *pA,
float32_t *pB, uint32_t blockSize)
```

group **Correlation distance**

Correlation distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_correlation_distance_f16 (float16_t *pA,
float16_t *pB, uint32_t blockSize)
```

Correlation distance between two vectors.

The input vectors are modified in place !

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_correlation_distance_f32 (float32_t *pA,
float32_t *pB, uint32_t blockSize)
```

Correlation distance between two vectors.

The input vectors are modified in place !

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Cosine distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_cosine_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_cosine_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float64_t riscv_cosine_distance_f64 (const float64_t *pA,  
const float64_t *pB, uint32_t blockSize)
```

group Cosine distance

Cosine distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_cosine_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

Cosine distance between two vectors.

Description

$\text{cosine_distance}(u,v)$ is $1 - u \cdot v / (\text{Norm}(u) \text{Norm}(v))$

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_cosine_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

Cosine distance between two vectors.

Description

$\text{cosine_distance}(u,v)$ is $1 - u \cdot v / (\text{Norm}(u) \text{Norm}(v))$

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float64_t riscv_cosine_distance_f64 (const float64_t *pA,
const float64_t *pB, uint32_t blockSize)
```

Cosine distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Dynamic Time Warping Distance

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dtw_distance_f32 (const riscv_matrix_instance_f32 *pDistance,
const riscv_matrix_instance_q7 *pWindow, riscv_matrix_instance_f32 *pDTW,
float32_t *distance)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dtw_init_window_q7 (const riscv_dtw_window windowType,
const int32_t windowSize, riscv_matrix_instance_q7 *pWindow)
```

```
RISCV_DSP_ATTRIBUTE void riscv_dtw_path_f32 (const riscv_matrix_instance_f32 *pDTW,
int16_t *pPath, uint32_t *pathLength)
```

group Dynamic Time Warping Distance

Dynamic Time Warping Distance.

This is not really a distance since triangular inequality is not respected.

The step pattern used is symmetric2. Future versions of this function will provide more customization options.

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dtw_distance_f32 (const riscv_matrix_instance_f32 *pDistance,
const riscv_matrix_instance_q7 *pWindow, riscv_matrix_instance_f32 *pDTW,
float32_t *distance)
```

Dynamic Time Warping distance.

The windowing matrix is used to impose some constraints on the search for a path. The algorithm will run faster (smaller search path) but may not be able to find a solution.

Windowing matrix

The distance matrix must be initialized only where the windowing matrix is containing 1. Thus, use of a window also decreases the number of distances which must be computed.

Parameters

- **pDistance** – [in] Distance matrix (Query rows * Template columns)

- **pWindow** – [in] Windowing matrix (can be NULL if no windowing used)
- **pDTW** – [out] Temporary cost buffer (same size)
- **distance** – [out] Distance

Returns

RISCV_MATH_ARGUMENT_ERROR in case no path can be found with window constraint

RISCV_DSP_ATTRIBUTE riscv_status riscv_dtw_init_window_q7 (const riscv_dtw_window windowType, const int32_t windowSize, riscv_matrix_instance_q7 *pWindow)

Window for dynamic time warping computation.

The input matrix must already contain a buffer and the number of rows (query length) and columns (template length) must be initialized. The function will fill the matrix with 0 and 1.

Windowing matrix

The window matrix will contain 1 for the position which are accepted and 0 for the positions which are rejected.

Parameters

- **windowType** – [in] Type of window
- **windowSize** – [in] Window size
- **pWindow** – [inout] Window

Returns

Error if window type not recognized

RISCV_DSP_ATTRIBUTE void riscv_dtw_path_f32 (const riscv_matrix_instance_f32 *pDTW, int16_t *pPath, uint32_t *pathLength)

Mapping between query and template.

The warping path has length which is at most 2*(query length + template length) in float. 2 because it is a list of coordinates : (query index, template index) coordinate.

Warping path

The buffer pPath must be big enough to contain the warping path.

pathLength is the number of points in the returned path. The returned path may be smaller than query + template.

Parameters

- **pDTW** – [in] Cost matrix (Query rows * Template columns)
- **pPath** – [out] Warping path in cost matrix 2*(nb rows + nb columns)
- **pathLength** – [out] Length of path in number of points

Euclidean distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_euclidean_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_euclidean_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float64_t riscv_euclidean_distance_f64 (const float64_t *pA,
const float64_t *pB, uint32_t blockSize)
```

group Euclidean distance

Euclidean distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_euclidean_distance_f16 (const float16_t *pA,
const float16_t *pB, uint32_t blockSize)
```

Euclidean distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_euclidean_distance_f32 (const float32_t *pA,
const float32_t *pB, uint32_t blockSize)
```

Euclidean distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
RISCV_DSP_ATTRIBUTE float64_t riscv_euclidean_distance_f64 (const float64_t *pA,
const float64_t *pB, uint32_t blockSize)
```

Euclidean distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector

- **blockSize** – [in] vector length

Returns

distance

Jensen-Shannon distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_jensenshannon_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_jensenshannon_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

group **Jensen-Shannon distance**

Jensen-Shannon distance

Functions

```
__STATIC_INLINE float16_t rel_entr (float16_t x, float16_t y)
```

```
RISCV_DSP_ATTRIBUTE float16_t riscv_jensenshannon_distance_f16 (const float16_t *pA,  
const float16_t *pB, uint32_t blockSize)
```

Jensen-Shannon distance between two vectors.

This function is assuming that elements of second vector are > 0 and 0 only when the corresponding element of first vector is 0 . Otherwise the result of the computation does not make sense and for speed reasons, the cases returning NaN or Infinity are not managed.

When the function is computing $x \log (x / y)$ with $x == 0$ and $y == 0$, it will compute the right result (0) but a division by zero will occur and should be ignored in client code.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

```
__STATIC_INLINE float32_t rel_entr (float32_t x, float32_t y)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_jensenshannon_distance_f32 (const float32_t *pA,  
const float32_t *pB, uint32_t blockSize)
```

Jensen-Shannon distance between two vectors.

This function is assuming that elements of second vector are > 0 and 0 only when the corresponding element of first vector is 0 . Otherwise the result of the computation does not make sense and for speed reasons, the cases returning NaN or Infinity are not managed.

When the function is computing $x \log(x/y)$ with $x == 0$ and $y == 0$, it will compute the right result (0) but a division by zero will occur and should be ignored in client code.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **blockSize** – [in] vector length

Returns

distance

Minkowski distance

```
RISCV_DSP_ATTRIBUTE float16_t riscv_minkowski_distance_f16 (const float16_t *pA,
const float16_t *pB, int32_t order, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_minkowski_distance_f32 (const float32_t *pA,
const float32_t *pB, int32_t order, uint32_t blockSize)
```

group **Minkowski distance**

Minkowski distance

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_minkowski_distance_f16 (const float16_t *pA,
const float16_t *pB, int32_t order, uint32_t blockSize)
```

Minkowski distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **order** – [in] Distance order
- **blockSize** – [in] Number of samples

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_minkowski_distance_f32 (const float32_t *pA,
const float32_t *pB, int32_t order, uint32_t blockSize)
```

Minkowski distance between two vectors.

Parameters

- **pA** – [in] First vector
- **pB** – [in] Second vector
- **order** – [in] Distance order
- **blockSize** – [in] Number of samples

Returns

distance

group **Float Distances**

Distances between two vectors of float values.

Boolean Distances

```
RISCV_DSP_ATTRIBUTE float32_t riscv_dice_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_hamming_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_jaccard_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_kulsinski_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_rogerstanimoto_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_russellrao_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_sokalmichener_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_sokalsneath_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_yule_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

group **Boolean Distances**

Distances between two vectors of boolean values.

Booleans are packed in 32 bit words. numberOfBooleans argument is the number of booleans and not the number of words.

Bits are packed in big-endian mode (because of behavior of numpy packbits in in version < 1.17)

Unnamed Group

RISCV_DSP_ATTRIBUTE float32_t riscv_dice_distance (const uint32_t *pA, const uint32_t *pB, uint32_t numberOfBools)

Dice distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

Functions

RISCV_DSP_ATTRIBUTE float32_t riscv_hamming_distance (const uint32_t *pA, const uint32_t *pB, uint32_t numberOfBools)

Hamming distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

RISCV_DSP_ATTRIBUTE float32_t riscv_jaccard_distance (const uint32_t *pA, const uint32_t *pB, uint32_t numberOfBools)

Jaccard distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

RISCV_DSP_ATTRIBUTE float32_t riscv_kulsinski_distance (const uint32_t *pA, const uint32_t *pB, uint32_t numberOfBools)

Kulsinski distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_rogerstanimoto_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

Rogers Tanimoto distance between two vectors.

Roger Stanimoto distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_russellrao_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

Russell-Rao distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_sokalmichener_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

Sokal-Michener distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_sokalsneath_distance (const uint32_t *pA,  
const uint32_t *pB, uint32_t numberOfBools)
```

Sokal-Sneath distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

```
RISCV_DSP_ATTRIBUTE float32_t riscv_yule_distance (const uint32_t *pA,
const uint32_t *pB, uint32_t numberOfBools)
```

Yule distance between two vectors.

Parameters

- **pA** – [in] First vector of packed booleans
- **pB** – [in] Second vector of packed booleans
- **numberOfBools** – [in] Number of booleans

Returns

distance

group **Distance Functions**

Distance functions for use with clustering algorithms. There are distance functions for float vectors and boolean vectors.

3.3.7 Fast Math Functions**ArcTan2**

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_f16 (float16_t y, float16_t x,
float16_t *result)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_f32 (float32_t y, float32_t x,
float32_t *result)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_q15 (q15_t y, q15_t x, q15_t *result)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_q31 (q31_t y, q31_t x, q31_t *result)
```

group **ArcTan2**

Computing Arc tangent only using the ratio y/x is not enough to determine the angle since there is an indeterminacy. Opposite quadrants are giving the same ratio.

ArcTan2 is not using y/x to compute the angle but y and x and use the sign of y and x to determine the quadrant.

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_f16 (float16_t y, float16_t x, float16_t *result)

Arc Tangent of y/x using sign of y and x to get right quadrant.

Arc tangent in radian of y/x using sign of x and y to determine right quadrant.

Compute the Arc tangent of y/x :

The sign of y and x are used to determine the right quadrant and compute the right angle. Returned value is between $-Pi$ and Pi .

Parameters

- **y** – [in] y coordinate
- **x** – [in] x coordinate
- **result** – [out] Result

Returns

error status.

RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_f32 (float32_t y, float32_t x, float32_t *result)

Arc Tangent of y/x using sign of y and x to get right quadrant.

Arc tangent in radian of y/x using sign of x and y to determine right quadrant.

Compute the Arc tangent of y/x :

The sign of y and x are used to determine the right quadrant and compute the right angle. Returned value is between $-Pi$ and Pi .

Parameters

- **y** – [in] y coordinate
- **x** – [in] x coordinate
- **result** – [out] Result

Returns

error status.

RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_q15 (q15_t y, q15_t x, q15_t *result)

Arc Tangent of y/x using sign of y and x to get right quadrant.

Arc tangent in radian of y/x using sign of x and y to determine right quadrant.

Compute the Arc tangent of y/x :

The sign of y and x are used to determine the right quadrant and compute the right angle. Returned value is between $-Pi$ and Pi .

Parameters

- **y** – [in] y coordinate
- **x** – [in] x coordinate
- **result** – [out] Result in Q2.13

Returns

error status.

RISCV_DSP_ATTRIBUTE riscv_status riscv_atan2_q31 (q31_t y, q31_t x, q31_t *result)

Arc Tangent of y/x using sign of y and x to get right quadrant.

Arc tangent in radian of y/x using sign of x and y to determine right quadrant.

Compute the Arc tangent of y/x:

The sign of y and x are used to determine the right quadrant and compute the right angle. Returned value is between -Pi and Pi.

Parameters

- **y** – [in] y coordinate
- **x** – [in] x coordinate
- **result** – [out] Result in Q2.29

Returns

error status.

Cosine

RISCV_DSP_ATTRIBUTE float32_t riscv_cos_f32 (float32_t x)

RISCV_DSP_ATTRIBUTE q15_t riscv_cos_q15 (q15_t x)

RISCV_DSP_ATTRIBUTE q31_t riscv_cos_q31 (q31_t x)

group **Cosine**

Computes the trigonometric cosine function using a combination of table lookup and linear interpolation. There are separate functions for Q15, Q31, and floating-point data types. The input to the floating-point version is in radians while the fixed-point Q15 and Q31 have a scaled input with the range [0 +0.9999] mapping to [0 2*pi). The fixed-point range is chosen so that a value of 2*pi wraps around to 0.

The implementation is based on table lookup using 512 values together with linear interpolation. The steps used are:

1. Calculation of the nearest integer table index
2. Compute the fractional portion (fract) of the table index.
3. The final result equals $(1.0f - \text{fract}) * a + \text{fract} * b$;

where

```
a = Table[index];
b = Table[index+1];
```

Functions

RISCV_DSP_ATTRIBUTE float32_t riscv_cos_f32 (float32_t x)

Fast approximation to the trigonometric cosine function for floating-point data.

Parameters

x – [in] input value in radians

Returns

cos(x)

RISCV_DSP_ATTRIBUTE q15_t riscv_cos_q15 (q15_t x)

Fast approximation to the trigonometric cosine function for Q15 data.

The Q15 input value is in the range [0 +0.9999] and is mapped to a radian value in the range [0 2*PI).

Parameters

x – [in] Scaled input value in radians

Returns

cos(x)

RISCV_DSP_ATTRIBUTE q31_t riscv_cos_q31 (q31_t x)

Fast approximation to the trigonometric cosine function for Q31 data.

The Q31 input value is in the range [0 +0.9999] and is mapped to a radian value in the range [0 2*PI).

Parameters

x – [in] Scaled input value in radians

Returns

cos(x)

Fixed point division

RISCV_DSP_ATTRIBUTE riscv_status riscv_divide_q15 (q15_t numerator, q15_t denominator, q15_t *quotient, int16_t *shift)

RISCV_DSP_ATTRIBUTE riscv_status riscv_divide_q31 (q31_t numerator, q31_t denominator, q31_t *quotient, int16_t *shift)

group **Fixed point division**

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_divide_q15 (q15_t numerator, q15_t denominator, q15_t *quotient, int16_t *shift)

Fixed point division.

When dividing by 0, an error RISC_V_MATH_NANINF is returned. And the quotient is forced to the saturated negative or positive value.

Parameters

- **numerator** – [in] Numerator
- **denominator** – [in] Denominator
- **quotient** – [out] Quotient value normalized between -1.0 and 1.0
- **shift** – [out] Shift left value to get the unnormalized quotient

Returns

error status

RISCV_DSP_ATTRIBUTE riscv_status riscv_divide_q31 (q31_t numerator, q31_t denominator, q31_t *quotient, int16_t *shift)

Fixed point division.

When dividing by 0, an error RISC_V_MATH_NANINF is returned. And the quotient is forced to the saturated negative or positive value.

Parameters

- **numerator** – [in] Numerator
- **denominator** – [in] Denominator
- **quotient** – [out] Quotient value normalized between -1.0 and 1.0
- **shift** – [out] Shift left value to get the unnormalized quotient

Returns

error status

Sine

RISCV_DSP_ATTRIBUTE float32_t riscv_sin_f32 (float32_t x)

RISCV_DSP_ATTRIBUTE q15_t riscv_sin_q15 (q15_t x)

RISCV_DSP_ATTRIBUTE q31_t riscv_sin_q31 (q31_t x)

group **Sine**

Computes the trigonometric sine function using a combination of table lookup and linear interpolation. There are separate functions for Q15, Q31, and floating-point data types. The input to the floating-point version is in radians while the fixed-point Q15 and Q31 have a scaled input with the range $[0 +0.9999]$ mapping to $[0 2\pi]$. The fixed-point range is chosen so that a value of 2π wraps around to 0.

The implementation is based on table lookup using 512 values together with linear interpolation. The steps used are:

1. Calculation of the nearest integer table index
2. Compute the fractional portion (fract) of the table index.
3. The final result equals $(1.0f - \text{fract}) * a + \text{fract} * b$;

where

```
b = Table[index];  
c = Table[index+1];
```

Functions**RISCV_DSP_ATTRIBUTE float32_t riscv_sin_f32 (float32_t x)**

Fast approximation to the trigonometric sine function for floating-point data.

Parameters

x – [in] input value in radians.

Returns

sin(x)

RISCV_DSP_ATTRIBUTE q15_t riscv_sin_q15 (q15_t x)

Fast approximation to the trigonometric sine function for Q15 data.

The Q15 input value is in the range $[0 +0.9999]$ and is mapped to a radian value in the range $[0 2\pi]$.

Parameters

x – [in] Scaled input value in radians

Returns

sin(x)

RISCV_DSP_ATTRIBUTE q31_t riscv_sin_q31 (q31_t x)

Fast approximation to the trigonometric sine function for Q31 data.

The Q31 input value is in the range $[0 +0.9999]$ and is mapped to a radian value in the range $[0 2\pi]$.

Parameters

x – [in] Scaled input value in radians

Returns

sin(x)

Vector Exponential

```
RISCV_DSP_ATTRIBUTE void riscv_vexp_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_vexp_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

group Vector Exponential

Compute the exp values of a vector of samples.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_vexp_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

Floating-point vector of exp values.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_vexp_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

Floating-point vector of exp values.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

Vector Log

```
RISCV_DSP_ATTRIBUTE void riscv_vlog_f16 (const float16_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_vlog_f32 (const float32_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_vlog_q15 (const q15_t *pSrc, q15_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_vlog_q31 (const q31_t *pSrc, q31_t *pDst,
uint32_t blockSize)
```

group **Vector Log**

Compute the log values of a vector of samples.

Functions

RISCV_DSP_ATTRIBUTE void riscv_vlog_f16 (const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)

Floating-point vector of log values.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_vlog_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Floating-point vector of log values.

Parameters

- **pSrc** – [in] points to the input vector
- **pDst** – [out] points to the output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_vlog_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

q15 vector of log values.

Parameters

- **pSrc** – [in] points to the input vector in q15
- **pDst** – [out] points to the output vector in q4.11
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_vlog_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

q31 vector of log values.

Parameters

- **pSrc** – [in] points to the input vector in q31
- **pDst** – [out] points to the output vector q5.26
- **blockSize** – [in] number of samples in each vector

Square Root

```
__STATIC_FORCEINLINE riscv_status riscv_sqrt_f32 (const float32_t in, float32_t *pOut)
```

riscv_status (page 1276) **riscv_sqrt_q31**(*q31_t* (page 1277) in, *q31_t* (page 1277) *pOut)

riscv_status (page 1276) **riscv_sqrt_q15**(*q15_t* (page 1277) in, *q15_t* (page 1277) *pOut)

```
__STATIC_FORCEINLINE riscv_status riscv_sqrt_f16 (float16_t in, float16_t *pOut)
```

group Square Root

Computes the square root of a number. There are separate functions for Q15, Q31, and floating-point data types. The square root function is computed using the Newton-Raphson algorithm. This is an iterative algorithm of the form:

$$x1 = x0 - f(x0)/f'(x0)$$

where $x1$ is the current estimate, $x0$ is the previous estimate, and $f'(x0)$ is the derivative of $f()$ evaluated at $x0$. For the square root function, the algorithm reduces to:

$x0 = in/2$	[initial guess]
$x1 = 1/2 * (x0 + in / x0)$	[each iteration]

Functions

```
__STATIC_FORCEINLINE riscv_status riscv_sqrt_f32 (const float32_t in,  
float32_t *pOut)
```

Floating-point square root function.

Parameters

- **in** – [in] input value
- **pOut** – [out] square root of input value

Returns

execution status

- RISC_V_MATH_SUCCESS : input value is positive
- RISC_V_MATH_ARGUMENT_ERROR : input value is negative; *pOut is set to 0

riscv_status (page 1276) **riscv_sqrt_q31**(*q31_t* (page 1277) in, *q31_t* (page 1277) *pOut)

Q31 square root function.

Parameters

- **in** – [in] input value. The range of the input value is $[0 + 1)$ or 0x00000000 to 0x7FFFFFFF
- **pOut** – [out] points to square root of input value

Returns

execution status

- RISC_V_MATH_SUCCESS : input value is positive
- RISC_V_MATH_ARGUMENT_ERROR : input value is negative; *pOut is set to 0

riscv_status (page 1276) **riscv_sqrt_q15**(*q15_t* (page 1277) *in*, *q15_t* (page 1277) **pOut*)

Q15 square root function.

Parameters

- **in** – [**in**] input value. The range of the input value is [0 +1) or 0x0000 to 0x7FFF
- **pOut** – [**out**] points to square root of input value

Returns

execution status

- RISC_V_MATH_SUCCESS : input value is positive
- RISC_V_MATH_ARGUMENT_ERROR : input value is negative; **pOut* is set to 0

__STATIC_FORCEINLINE riscv_status riscv_sqrt_f16 (*float16_t in*, *float16_t *pOut*)

Floating-point square root function.

Parameters

- **in** – [**in**] input value
- **pOut** – [**out**] square root of input value

Returns

execution status

- RISC_V_MATH_SUCCESS : input value is positive
- RISC_V_MATH_ARGUMENT_ERROR : input value is negative; **pOut* is set to 0

group Fast Math Functions

This set of functions provides a fast approximation to sine, cosine, and square root. As compared to most of the other functions in the NMSIS math library, the fast math functions operate on individual values and not arrays. There are separate functions for Q15, Q31, and floating-point data.

3.3.8 Filtering Functions

High Precision Q31 Biquad Cascade Filter

RISC_V_DSP_ATTRIBUTE void **riscv_biquad_cas_df1_32x64_init_q31** (*riscv_biquad_cas_df1_32x64_ins_q31 *S*, *uint8_t numStages*, *const q31_t *pCoeffs*, *q63_t *pState*, *uint8_t postShift*)

RISC_V_DSP_ATTRIBUTE void **riscv_biquad_cas_df1_32x64_q31** (*const riscv_biquad_cas_df1_32x64_ins_q31 *S*, *const q31_t *pSrc*, *q31_t *pDst*, *uint32_t blockSize*)

group High Precision Q31 Biquad Cascade Filter

This function implements a high precision Biquad cascade filter which operates on Q31 data values. The filter coefficients are in 1.31 format and the state variables are in 1.63 format. The double precision state variables reduce quantization noise in the filter and provide a cleaner output. These filters are particularly useful when implementing filters in which the singularities are close to the unit circle. This is common for low pass or high pass filters with very low cutoff frequencies.

The function operates on blocks of input and output data and each call to the function processes *blockSize* samples through the filter. *pSrc* and *pDst* points to input and output arrays containing *blockSize* Q31 values.

Algorithm

Each Biquad stage implements a second order filter using the difference equation:

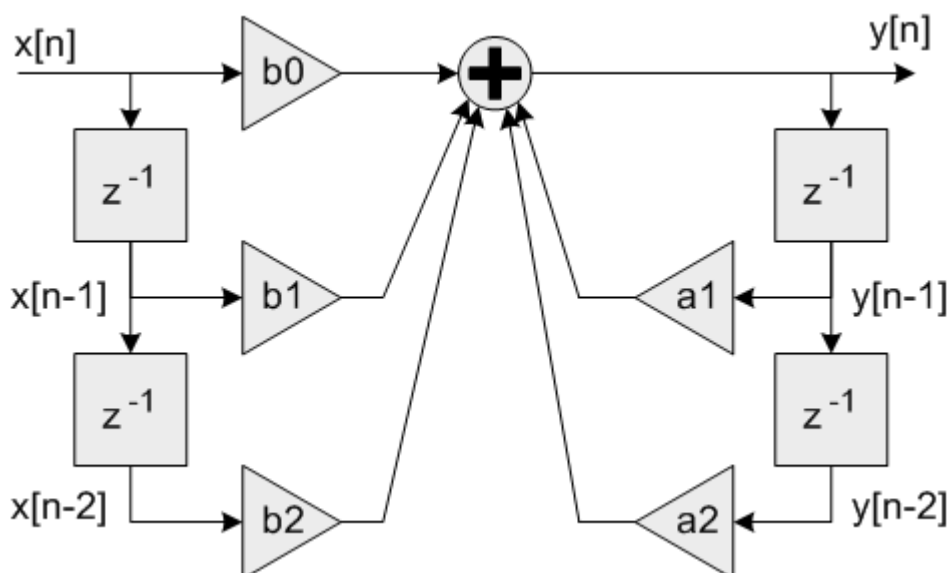
$$y[n] = b_0 * x[n] + b_1 * x[n-1] + b_2 * x[n-2] + a_1 * y[n-1] + a_2 * y[n-2]$$

A Direct Form I algorithm is used with 5 coefficients and 4 state variables per stage.

Coefficients b_0 , b_1 and b_2 multiply the input signal $x[n]$ and are referred to as the feedforward coefficients. Coefficients a_1 and a_2 multiply the output signal $y[n]$ and are referred to as the feedback coefficients. Pay careful attention to the sign of the feedback coefficients. Some design tools use the difference equation

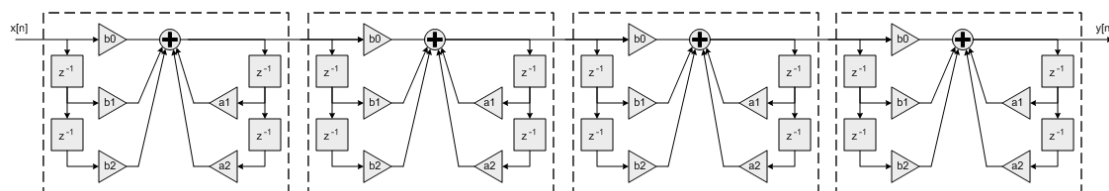
$$y[n] = b_0 * x[n] + b_1 * x[n-1] + b_2 * x[n-2] - a_1 * y[n-1] - a_2 * y[n-2]$$

In this case the feedback coefficients a_1 and a_2 must be negated when used with the NMSIS DSP Library.



Higher order filters are realized as a cascade of second order sections. `numStages` refers to the number of second order stages used. For example, an 8th order filter would be realized with `numStages=4` second order stages.

A 9th order filter would be realized with `numStages=5` second order stages with the coefficients for one of the stages configured as a first order filter ($b_2=0$ and $a_2=0$).



The `pState` points to state variables array. Each Biquad stage has 4 state variables $x[n-1]$, $x[n-2]$, $y[n-1]$, and $y[n-2]$ and each state variable in 1.63 format to improve precision. The state variables are arranged in the array as:

```
{x[n-1], x[n-2], y[n-1], y[n-2]}
```

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of `4*numStages` values of data in 1.63 format. The state variables are updated after each block of data is processed, the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared.

Init Function

There is also an associated initialization function which performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numStages`, `pCoeffs`, `postShift`, `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. For example, to statically initialize the filter instance structure use

```
riscv_biquad_cas_df1_32x64_ins_q31 S1 = {numStages, pState, pCoeffs, postShift};
```

where `numStages` is the number of Biquad stages in the filter; `pState` is the address of the state buffer; `pCoeffs` is the address of the coefficient buffer; `postShift` shift to be applied which is described in detail below.

Fixed-Point Behavior

Care must be taken while using Biquad Cascade 32x64 filter function. Following issues must be considered:

- Scaling of coefficients
- Filter gain
- Overflow and saturation

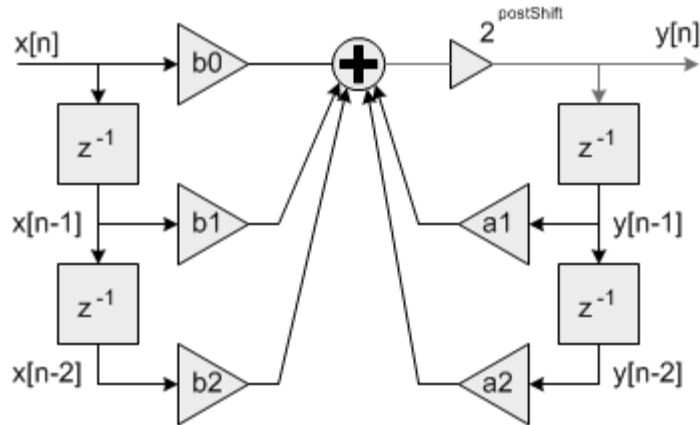
Filter coefficients are represented as fractional values and restricted to lie in the range $[-1 \text{ } +1)$. The processing function has an additional scaling parameter `postShift` which allows the filter coefficients to exceed the range $[+1 \text{ } -1)$. At the output of the filter's accumulator is a shift register which shifts the result by `postShift` bits.

This essentially scales the filter coefficients by $2^{\text{postShift}}$. For example, to realize the coefficients

```
{1.5, -0.8, 1.2, 1.6, -0.9}
```

set the Coefficient array to:

```
{0.75, -0.4, 0.6, 0.8, -0.45}
```

and set `postShift=1`

The second thing to keep in mind is the gain through the filter. The frequency response of a Biquad filter is a function of its coefficients. It is possible for the gain through the filter to exceed 1.0 meaning that the filter increases the amplitude of certain frequencies. This means that an input signal with amplitude < 1.0 may result in an output > 1.0 and these are saturated or overflowed based on the implementation of the filter. To avoid this behavior the filter needs to be scaled down such that its peak gain < 1.0 or the input signal must be scaled down so that the combination of input and filter are never overflowed.

The third item to consider is the overflow and saturation behavior of the fixed-point Q31 version. This is described in the function specific documentation below.

Functions

RISCV_DSP_ATTRIBUTE void `riscv_biquad_cas_df1_32x64_init_q31` (`riscv_biquad_cas_df1_32x64_ins_q31` *S, `uint8_t` numStages, `const` `q31_t` *pCoeffs, `q63_t` *pState, `uint8_t` postShift)

Initialization function for the Q31 Biquad cascade 32x64 filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}

where `b1x` and `a1x` are the coefficients for the first stage, `b2x` and `a2x` are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of `5*numStages` values.

The `pState` points to state variables array and size of each state variable is 1.63 format. Each Biquad stage has 4 state variables `x[n-1]`, `x[n-2]`, `y[n-1]`, and `y[n-2]`. The state variables are arranged in the state array as:

{x[n-1], x[n-2], y[n-1], y[n-2]}

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of `4*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the high precision Q31 Biquad cascade filter structure
- **numStages** – [in] number of 2nd order stages in the filter
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer
- **postShift** – [in] Shift to be applied after the accumulator. Varies according to the coefficients format

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cas_df1_32x64_q31 (const riscv_biquad_cas_df1_32x64_ins_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for the Q31 Biquad cascade 32x64 filter.

Details

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by 2 bits and lie in the range [-0.25 +0.25). After all 5 multiply-accumulates are performed, the 2.62 accumulator is shifted by `postShift` bits and the result truncated to 1.31 format by discarding the low 32 bits.

Two related functions are provided in the NMSIS DSP library.

- `riscv_biquad_cascade_df1_q31()` implements a Biquad cascade with 32-bit coefficients and state variables with a Q63 accumulator.
- `riscv_biquad_cascade_df1_fast_q31()` implements a Biquad cascade with 32-bit coefficients and state variables with a Q31 accumulator.

Parameters

- **S** – [in] points to an instance of the high precision Q31 Biquad cascade filter
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

Biquad Cascade IIR Filters Using Direct Form I Structure

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_f16 (const riscv_biquad_casd_df1_inst_f16 *S,
const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_f32 (const riscv_biquad_casd_df1_inst_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_fast_q15 (const riscv_biquad_casd_df1_inst_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_fast_q31 (const riscv_biquad_casd_df1_inst_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_f16 (riscv_biquad_casd_df1_inst_f16 *S,
uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_f32 (riscv_biquad_casd_df1_inst_f32 *S,
uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_q15 (riscv_biquad_casd_df1_inst_q15 *S,
uint8_t numStages, const q15_t *pCoeffs, q15_t *pState, int8_t postShift)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_q31 (riscv_biquad_casd_df1_inst_q31 *S,
uint8_t numStages, const q31_t *pCoeffs, q31_t *pState, int8_t postShift)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_q15 (const riscv_biquad_casd_df1_inst_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_q31 (const riscv_biquad_casd_df1_inst_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

group Biquad Cascade IIR Filters Using Direct Form I Structure

This set of functions implements arbitrary order recursive (IIR) filters. The filters are implemented as a cascade of second order Biquad sections. The functions support Q15, Q31 and floating-point data types. Fast version of Q15 and Q31 also available.

The functions operate on blocks of input and output data and each call to the function processes `blockSize` samples through the filter. `pSrc` points to the array of input data and `pDst` points to the array of output data. Both arrays contain `blockSize` values.

Algorithm

Each Biquad stage implements a second order filter using the difference equation:

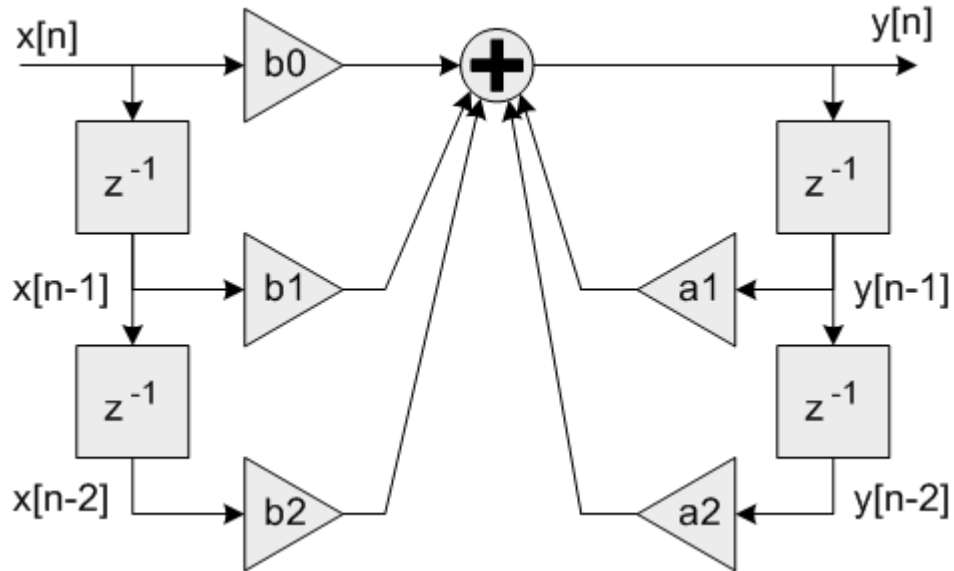
$$y[n] = b_0 * x[n] + b_1 * x[n-1] + b_2 * x[n-2] + a_1 * y[n-1] + a_2 * y[n-2]$$

A Direct Form I algorithm is used with 5 coefficients and 4 state variables per stage.

Coefficients `b0`, `b1` and `b2` multiply the input signal `x[n]` and are referred to as the feedforward coefficients. Coefficients `a1` and `a2` multiply the output signal `y[n]` and are referred to as the feedback coefficients. Pay careful attention to the sign of the feedback coefficients. Some design tools use the difference equation

$$y[n] = b_0 * x[n] + b_1 * x[n-1] + b_2 * x[n-2] - a_1 * y[n-1] - a_2 * y[n-2]$$

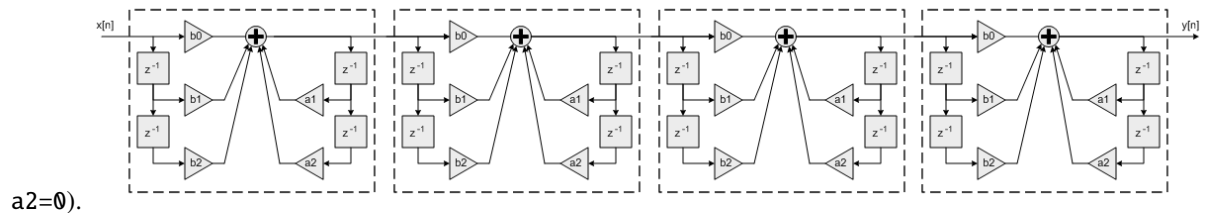
In this case the feedback coefficients `a1` and `a2` must be negated when used with the NMSIS DSP



Library.

Higher order filters are realized as a cascade of second order sections. `numStages` refers to the number of second order stages used. For example, an 8th order filter would be realized with `numStages=4` second order stages.

A 9th order filter would be realized with `numStages=5` second order stages with the coefficients for one of the stages configured as a first order filter (`b2=0` and



`a2=0`).

The `pState` points to state variables array. Each Biquad stage has 4 state variables `x[n-1]`, `x[n-2]`, `y[n-1]`, and `y[n-2]`. The state variables are arranged in the `pState` array as:

`{x[n-1], x[n-2], y[n-1], y[n-2]}`

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of `4*numStages` values. The state variables are updated after each block of data is processed, the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared. There are separate instance structure declarations for each of the 3 supported data types.

Init Function

There is also an associated initialization function for each data type. The initialization function performs following operations:

- Sets the values of the internal structure fields.

- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: numStages, pCoeffs, pState. Also set all of the values in pState to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. The code below statically initializes each of the 3 different data type filter instance structures

```
riscv_biquad_casd_df1_inst_f32 S1 = {numStages, pState, pCoeffs};
riscv_biquad_casd_df1_inst_q15 S2 = {numStages, pState, pCoeffs, postShift};
riscv_biquad_casd_df1_inst_q31 S3 = {numStages, pState, pCoeffs, postShift};
```

where numStages is the number of Biquad stages in the filter; pState is the address of the state buffer; pCoeffs is the address of the coefficient buffer; postShift shift to be applied.

Fixed-Point Behavior

Care must be taken when using the Q15 and Q31 versions of the Biquad Cascade filter functions. Following issues must be considered:

- Scaling of coefficients
- Filter gain
- Overflow and saturation

Scaling of coefficients

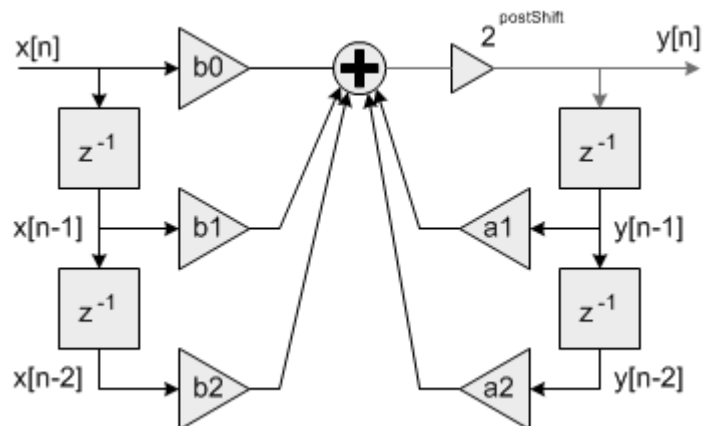
Filter coefficients are represented as fractional values and coefficients are restricted to lie in the range $[-1, +1]$. The fixed-point functions have an additional scaling parameter postShift which allow the filter coefficients to exceed the range $[-1, +1]$. At the output of the filter's accumulator is a shift register which shifts the result by postShift bits.

This essentially scales the filter coefficients by $2^{\text{postShift}}$. For example, to realize the coefficients

```
{1.5, -0.8, 1.2, 1.6, -0.9}
```

set the pCoeffs array to:

```
{0.75, -0.4, 0.6, 0.8, -0.45}
```



and set postShift=1

Filter gain

The frequency response of a Biquad filter is a function of its coefficients. It is possible for the gain through

the filter to exceed 1.0 meaning that the filter increases the amplitude of certain frequencies. This means that an input signal with amplitude < 1.0 may result in an output > 1.0 and these are saturated or overflowed based on the implementation of the filter. To avoid this behavior the filter needs to be scaled down such that its peak gain < 1.0 or the input signal must be scaled down so that the combination of input and filter are never overflowed.

Overflow and saturation

For Q15 and Q31 versions, it is described separately as part of the function specific documentation below.

Functions

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_f16 (const riscv_biquad_casd_df1_inst_f16 *S, const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)

Processing function for the floating-point Biquad cascade filter.

Parameters

- **S** – [in] points to an instance of the floating-point Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_f32 (const riscv_biquad_casd_df1_inst_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point Biquad cascade filter.

Parameters

- **S** – [in] points to an instance of the floating-point Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_fast_q15 (const riscv_biquad_casd_df1_inst_q15 *S, const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Processing function for the Q15 Biquad cascade filter (fast variant).

Fast but less precise processing function for the Q15 Biquad cascade filter for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_biquad_cascade_df1_q15()` for a slower implementation of this function which uses 64-bit accumulation to avoid wrap around distortion. Both the slow and the fast versions use the same instance structure. Use the function `riscv_biquad_cascade_df1_init_q15()` to initialize the filter structure.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision

of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around and distorts the result. In order to avoid overflows completely the input signal must be scaled down by two bits and lie in the range $[-0.25 +0.25]$. The 2.30 accumulator is then shifted by `postShift` bits and the result truncated to 1.15 format by discarding the low 16 bits.

Parameters

- **S** – [in] points to an instance of the Q15 Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process per call

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_fast_q31 (const riscv_biquad_casd_df1_inst_q31 *S, const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Processing function for the Q31 Biquad cascade filter (fast variant).

Fast but less precise processing function for the Q31 Biquad cascade filter for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_biquad_cascade_df1_q31()` for a slower implementation of this function which uses 64-bit accumulation to provide higher precision. Both the slow and the fast versions use the same instance structure. Use the function `riscv_biquad_cascade_df1_init_q31()` to initialize the filter structure.

Scaling and Overflow Behavior

This function is optimized for speed at the expense of fixed-point precision and overflow protection. The result of each 1.31×1.31 multiplication is truncated to 2.30 format. These intermediate results are added to a 2.30 accumulator. Finally, the accumulator is saturated and converted to a 1.31 result. The fast version has the same overflow behavior as the standard version and provides less precision since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signal must be scaled down by two bits and lie in the range $[-0.25 +0.25]$. Use the initialization function `riscv_biquad_cascade_df1_init_q31()` to initialize filter structure.

Parameters

- **S** – [in] points to an instance of the Q31 Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process per call

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_f16 (riscv_biquad_casd_df1_inst_f16 *S, uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)

Initialization function for the floating-point Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where $b1x$ and $a1x$ are the coefficients for the first stage, $b2x$ and $a2x$ are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of $5 \times \text{numStages}$ values.

The `pState` is a pointer to state array. Each Biquad stage has 4 state variables $x[n-1]$, $x[n-2]$, $y[n-1]$, and $y[n-2]$. The state variables are arranged in the `pState` array as:

```
{x[n-1], x[n-2], y[n-1], y[n-2]}
```

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of $4 \times \text{numStages}$ values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the floating-point Biquad cascade structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_f32 (riscv_biquad_casd_df1_inst_f32 *S, uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)

Initialization function for the floating-point Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where $b1x$ and $a1x$ are the coefficients for the first stage, $b2x$ and $a2x$ are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of $5 \times \text{numStages}$ values.

The `pState` is a pointer to state array. Each Biquad stage has 4 state variables $x[n-1]$, $x[n-2]$, $y[n-1]$, and $y[n-2]$. The state variables are arranged in the `pState` array as:

```
{x[n-1], x[n-2], y[n-1], y[n-2]}
```

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of $4 \times \text{numStages}$ values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the floating-point Biquad cascade structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.


```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_q15 (riscv_biquad_casd_df1_inst_q15 *S,
uint8_t numStages, const q15_t *pCoeffs, q15_t *pState, int8_t postShift)
```

Initialization function for the Q15 Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, 0, b11, b12, a11, a12, b20, 0, b21, b22, a21, a22, ...}
```

where `b1x` and `a1x` are the coefficients for the first stage, `b2x` and `a2x` are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of `6*numStages` values. The zero coefficient between `b1` and `b2` facilitates use of 16-bit SIMD instructions on the RISC-V Core with DSP enabled.

The state variables are stored in the array `pState`. Each Biquad stage has 4 state variables `x[n-1]`, `x[n-2]`, `y[n-1]`, and `y[n-2]`. The state variables are arranged in the `pState` array as:

```
{x[n-1], x[n-2], y[n-1], y[n-2]}
```

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of `4*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the Q15 Biquad cascade structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.
- **postShift** – [in] Shift to be applied to the accumulator result. Varies according to the coefficients format

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_init_q31 (riscv_biquad_casd_df1_inst_q31 *S,
uint8_t numStages, const q31_t *pCoeffs, q31_t *pState, int8_t postShift)
```

Initialization function for the Q31 Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where `b1x` and `a1x` are the coefficients for the first stage, `b2x` and `a2x` are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of `5*numStages` values.

The `pState` points to state variables array. Each Biquad stage has 4 state variables `x[n-1]`, `x[n-2]`, `y[n-1]`, and `y[n-2]`. The state variables are arranged in the `pState` array as:

```
{x[n-1], x[n-2], y[n-1], y[n-2]}
```

The 4 state variables for stage 1 are first, then the 4 state variables for stage 2, and so on. The state array has a total length of `4*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the Q31 Biquad cascade structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.
- **postShift** – [in] Shift to be applied after the accumulator. Varies according to the coefficients format

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_q15 (const riscv_biquad_casd_df1_inst_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

Processing function for the Q15 Biquad cascade filter.

Remark

Refer to `riscv_biquad_cascade_df1_fast_q15()` for a faster but less precise implementation of this filter.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. The accumulator is then shifted by `postShift` bits to truncate the result to 1.15 format by discarding the low 16 bits. Finally, the result is saturated to 1.15 format.

Parameters

- **S** – [in] points to an instance of the Q15 Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the location where the output result is written
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df1_q31 (const riscv_biquad_casd_df1_inst_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for the Q31 Biquad cascade filter.

Remark

Refer to `riscv_biquad_cascade_df1_fast_q31()` for a faster but less precise implementation of this filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by 2 bits and lie in the range $[-0.25 +0.25]$. After all 5 multiply-accumulates are performed, the 2.62 accumulator is shifted by `postShift` bits and the result truncated to 1.31 format by discarding the low 32 bits.

Parameters

- **S** – [in] points to an instance of the Q31 Biquad cascade structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

Biquad Cascade IIR Filters Using a Direct Form II Transposed Structure

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f16 (const riscv_biquad_cascade_df2T_instance_f16 *S,
const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f32 (const riscv_biquad_cascade_df2T_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f64 (const riscv_biquad_cascade_df2T_instance_f64 *S,
const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f16 (riscv_biquad_cascade_df2T_instance_f16 *S,
uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f32 (riscv_biquad_cascade_df2T_instance_f32 *S,
uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f64 (riscv_biquad_cascade_df2T_instance_f64 *S,
uint8_t numStages, const float64_t *pCoeffs, float64_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_f16 (const riscv_biquad_cascade_stereo_df2T_instance_f16 *S,
const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_f32 (const riscv_biquad_cascade_stereo_df2T_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_init_f16 (riscv_biquad_cascade_stereo_df2T_instance_f16 *S,
uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_init_f32 (riscv_biquad_cascade_stereo_df2T_instance_f32 *S,
uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)
```

group **Biquad Cascade IIR Filters Using a Direct Form II Transposed Structure**

This set of functions implements arbitrary order recursive (IIR) filters using a transposed direct form II structure. The filters are implemented as a cascade of second order Biquad sections. These functions provide a slight memory savings as compared to the direct form I Biquad filter functions. Only floating-point data is supported.

This function operate on blocks of input and output data and each call to the function processes `blockSize` samples through the filter. `pSrc` points to the array of input data and `pDst` points to the array of output data. Both arrays contain `blockSize` values.

Algorithm

Each Biquad stage implements a second order filter using the difference equation:

$$\begin{aligned}y[n] &= b_0 * x[n] + d_1 \\d_1 &= b_1 * x[n] + a_1 * y[n] + d_2 \\d_2 &= b_2 * x[n] + a_2 * y[n]\end{aligned}$$

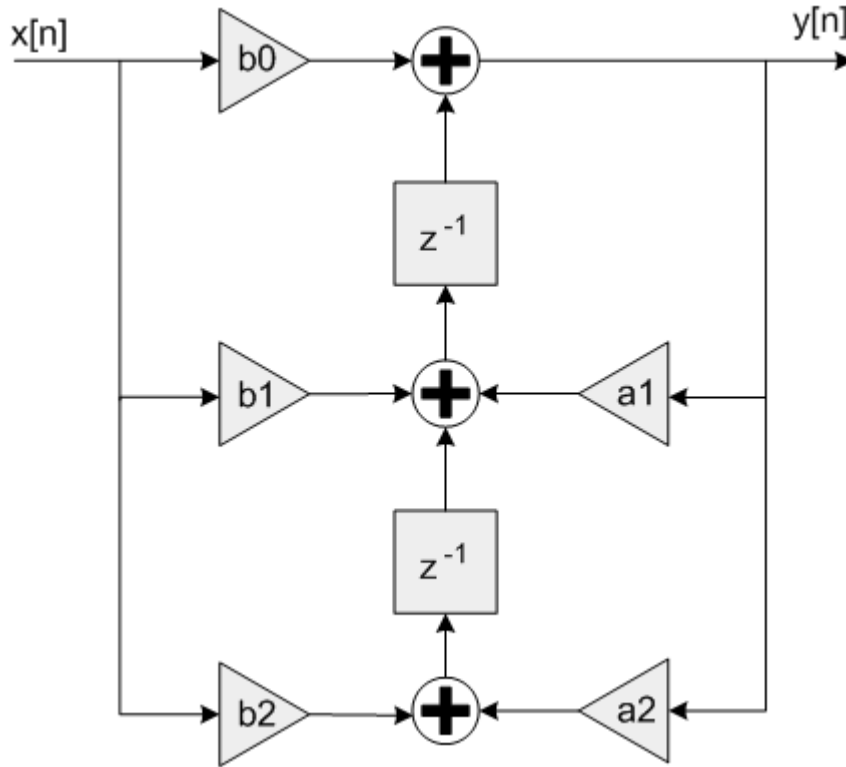
where `d1` and `d2` represent the two state values.

A Biquad filter using a transposed Direct Form II structure is shown below.

Coefficients `b0`, `b1`, and `b2` multiply the input signal `x[n]` and are referred to as the feedforward coefficients. Coefficients `a1` and `a2` multiply the output signal `y[n]` and are referred to as the feedback coefficients. Pay careful attention to the sign of the feedback coefficients. Some design tools flip the sign of the feedback coefficients:

$$\begin{aligned}y[n] &= b_0 * x[n] + d_1; \\d_1 &= b_1 * x[n] - a_1 * y[n] + d_2; \\d_2 &= b_2 * x[n] - a_2 * y[n];\end{aligned}$$

In this case the feedback coefficients `a1` and `a2` must be negated when used with the NMSIS DSP Library.



Higher order filters are realized as a cascade of second order sections. `numStages` refers to the number of second order stages used. For example, an 8th order filter would be realized with `numStages=4` second order stages. A 9th order filter would be realized with `numStages=5` second order stages with the coefficients for one of the stages configured as a first order filter (`b2=0` and `a2=0`).

`pState` points to the state variable array. Each Biquad stage has 2 state variables `d1` and `d2`. The state variables are arranged in the `pState` array as:

```
{d11, d12, d21, d22, ...}
```

where `d1x` refers to the state variables for the first Biquad and `d2x` refers to the state variables for the second Biquad. The state array has a total length of `2*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

The NMSIS library contains Biquad filters in both Direct Form I and transposed Direct Form II. The advantage of the Direct Form I structure is that it is numerically more robust for fixed-point data types. That is why the Direct Form I structure supports Q15 and Q31 data types. The transposed Direct Form II structure, on the other hand, requires a wide dynamic range for the state variables `d1` and `d2`. Because of this, the NMSIS library only has a floating-point version of the Direct Form II Biquad. The advantage of the Direct Form II Biquad is that it requires half the number of state variables, 2 rather than 4, per Biquad stage.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared.

Init Functions

There is also an associated initialization function. The initialization function performs following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: numStages, pCoeffs, pState. Also set all of the values in pState to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. For example, to statically initialize the instance structure use

```
riscv_biquad_cascade_df2T_instance_f64 S1 = {numStages, pState, pCoeffs};  
riscv_biquad_cascade_df2T_instance_f32 S1 = {numStages, pState, pCoeffs};
```

where numStages is the number of Biquad stages in the filter; pState is the address of the state buffer. pCoeffs is the address of the coefficient buffer;

Functions

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f16 (const riscv_biquad_cascade_df2T_instance_f16_t *S, const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)

Processing function for the floating-point transposed direct form II Biquad cascade filter.

Parameters

- **S** – [in] points to an instance of the filter data structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f32 (const riscv_biquad_cascade_df2T_instance_f32_t *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point transposed direct form II Biquad cascade filter.

Parameters

- **S** – [in] points to an instance of the filter data structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_f64 (const riscv_biquad_cascade_df2T_instance_f64_t *S, const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)

Processing function for the floating-point transposed direct form II Biquad cascade filter.

Parameters

- **S** – [in] points to an instance of the filter data structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data

- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f16 (riscv_biquad_cascade_df2T_instance_f16_t *instance, uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)

Initialization function for the floating-point transposed direct form II Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array pCoeffs in the following order.

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where b1x and a1x are the coefficients for the first stage, b2x and a2x are the coefficients for the second stage, and so on. The pCoeffs array contains a total of 5*numStages values.

The pState is a pointer to state array. Each Biquad stage has 2 state variables d1, and d2. The 2 state variables for stage 1 are first, then the 2 state variables for stage 2, and so on. The state array has a total length of 2*numStages values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the filter data structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f32 (riscv_biquad_cascade_df2T_instance_f32_t *instance, uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)

Initialization function for the floating-point transposed direct form II Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array pCoeffs in the following order.

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where b1x and a1x are the coefficients for the first stage, b2x and a2x are the coefficients for the second stage, and so on. The pCoeffs array contains a total of 5*numStages values.

The pState is a pointer to state array. Each Biquad stage has 2 state variables d1, and d2. The 2 state variables for stage 1 are first, then the 2 state variables for stage 2, and so on. The state array has a total length of 2*numStages values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the filter data structure.
- **numStages** – [in] number of 2nd order stages in the filter.

- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_df2T_init_f64 (riscv_biquad_cascade_df2T_instance_f64_t *pInstance, uint8_t numStages, const float64_t *pCoeffs, float64_t *pState)

Initialization function for the floating-point transposed direct form II Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array pCoeffs in the following order:

{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}

where b1x and a1x are the coefficients for the first stage, b2x and a2x are the coefficients for the second stage, and so on. The pCoeffs array contains a total of 5*numStages values.

The pState is a pointer to state array. Each Biquad stage has 2 state variables d1, and d2. The 2 state variables for stage 1 are first, then the 2 state variables for stage 2, and so on. The state array has a total length of 2*numStages values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the filter data structure
- **numStages** – [in] number of 2nd order stages in the filter
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_f16 (const riscv_biquad_cascade_stereo_df2T_instance_f16_t *pInstance, const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)

Processing function for the floating-point transposed direct form II Biquad cascade filter.

Processing function for the floating-point transposed direct form II Biquad cascade filter. 2 channels.

Parameters

- **S** – [in] points to an instance of the filter data structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_f32 (const riscv_biquad_cascade_stereo_df2T_instance_f32_t *pInstance, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point transposed direct form II Biquad cascade filter.

Processing function for the floating-point transposed direct form II Biquad cascade filter. 2 channels.

Parameters

- **S** – [in] points to an instance of the filter data structure
- **pSrc** – [in] points to the block of input data

- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_init_f16 (riscv_biquad_cascade_stereo_df2T_t *pS, uint8_t numStages, const float16_t *pCoeffs, float16_t *pState)

Initialization function for the floating-point transposed direct form II Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where `b1x` and `a1x` are the coefficients for the first stage, `b2x` and `a2x` are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of `5*numStages` values.

The `pState` is a pointer to state array. Each Biquad stage has 2 state variables `d1`, and `d2` for each channel. The 2 state variables for stage 1 are first, then the 2 state variables for stage 2, and so on. The state array has a total length of `2*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the filter data structure.
- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.

RISCV_DSP_ATTRIBUTE void riscv_biquad_cascade_stereo_df2T_init_f32 (riscv_biquad_cascade_stereo_df2T_t *pS, uint8_t numStages, const float32_t *pCoeffs, float32_t *pState)

Initialization function for the floating-point transposed direct form II Biquad cascade filter.

Coefficient and State Ordering

The coefficients are stored in the array `pCoeffs` in the following order:

```
{b10, b11, b12, a11, a12, b20, b21, b22, a21, a22, ...}
```

where `b1x` and `a1x` are the coefficients for the first stage, `b2x` and `a2x` are the coefficients for the second stage, and so on. The `pCoeffs` array contains a total of `5*numStages` values.

The `pState` is a pointer to state array. Each Biquad stage has 2 state variables `d1`, and `d2` for each channel. The 2 state variables for stage 1 are first, then the 2 state variables for stage 2, and so on. The state array has a total length of `2*numStages` values. The state variables are updated after each block of data is processed; the coefficients are untouched.

Parameters

- **S** – [inout] points to an instance of the filter data structure.

- **numStages** – [in] number of 2nd order stages in the filter.
- **pCoeffs** – [in] points to the filter coefficients.
- **pState** – [in] points to the state buffer.

Convolution

```
RISCV_DSP_ATTRIBUTE void riscv_conv_f32 (const float32_t *pSrcA, uint32_t srcALen,  
const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_fast_opt_q15 (const q15_t *pSrcA, uint32_t srcALen,  
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_fast_q15 (const q15_t *pSrcA, uint32_t srcALen,  
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_fast_q31 (const q31_t *pSrcA, uint32_t srcALen,  
const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_opt_q15 (const q15_t *pSrcA, uint32_t srcALen,  
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_opt_q7 (const q7_t *pSrcA, uint32_t srcALen,  
const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_q15 (const q15_t *pSrcA, uint32_t srcALen,  
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_q31 (const q31_t *pSrcA, uint32_t srcALen,  
const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_conv_q7 (const q7_t *pSrcA, uint32_t srcALen,  
const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst)
```

group Convolution

Convolution is a mathematical operation that operates on two finite length vectors to generate a finite length output vector. Convolution is similar to correlation and is frequently used in filtering and data analysis. The NMSIS DSP library contains functions for convolving Q7, Q15, Q31, and floating-point data types. The library also provides fast versions of the Q15 and Q31 functions.

Algorithm

Let $a[n]$ and $b[n]$ be sequences of length $srcALen$ and $srcBLen$ samples respectively. Then the convolution

$$c[n] = a[n] * b[n]$$

is defined as

$$c[n] = \sum_{k=0}^{srcALen} a[k]b[n-k]$$

Note that $c[n]$ is of length $srcALen + srcBLen - 1$ and is defined over the interval $n=0, 1, 2, \dots, srcALen + srcBLen - 2$. $pSrcA$ points to the first input vector of length $srcALen$ and $pSrcB$ points to the second input vector of length $srcBLen$. The output result is written to $pDst$ and the calling function must allocate $srcALen+srcBLen-1$ words for the result.

Conceptually, when two signals $a[n]$ and $b[n]$ are convolved, the signal $b[n]$ slides over $a[n]$. For each offset n , the overlapping portions of $a[n]$ and $b[n]$ are multiplied and summed together.

Note that convolution is a commutative operation:

$$a[n] * b[n] = b[n] * a[n].$$

This means that switching the A and B arguments to the convolution functions has no effect.

Fixed-Point Behavior

Convolution requires summing up a large number of intermediate products. As such, the Q7, Q15, and Q31 functions run a risk of overflow and saturation. Refer to the function specific documentation below for further details of the particular algorithm used.

Fast Versions

Fast versions are supported for Q31 and Q15. Cycles for Fast versions are less compared to Q31 and Q15 of conv and the design requires the input signals should be scaled down to avoid intermediate overflows.

Opt Versions

Opt versions are supported for Q15 and Q7. Design uses internal scratch buffer for getting good optimisation. These versions are optimised in cycles and consumes more memory (Scratch memory) compared to Q15 and Q7 versions

Long versions:

For convolution of long vectors, those functions are no more adapted and will be very slow. An implementation based upon FFTs should be used.

Functions

RISCV_DSP_ATTRIBUTE void riscv_conv_f32 (const float32_t *pSrcA, uint32_t srcALen, const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst)

Convolution of floating-point sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence

- **pDst** – [out] points to the location where the output result is written. Length srcALen+srcBLen-1.

RISCV_DSP_ATTRIBUTE void riscv_conv_fast_opt_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch1, q15_t *pScratch2)

Convolution of Q15 sequences (fast version).

Convolution of Q15 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_q15()` for a slower implementation of this function which uses 64-bit accumulation to avoid wrap around distortion.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down the inputs by $\log_2(\min(\text{srcALen}, \text{srcBLen}))$ ($\log_2$ is read as log to the base 2) times to avoid overflows, as maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions are carried internally. The 2.30 accumulator is right shifted by 15 bits and then saturated to 1.15 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length srcALen+srcBLen-1
- **pScratch1** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 \cdot \min(\text{srcALen}, \text{srcBLen}) - 2$
- **pScratch2** – [in] points to scratch buffer of size $\min(\text{srcALen}, \text{srcBLen})$

RISCV_DSP_ATTRIBUTE void riscv_conv_fast_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)

Convolution of Q15 sequences (fast version).

Convolution of Q15 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_q15()` for a slower implementation of this function which uses 64-bit accumulation to avoid wrap around distortion.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down the inputs by $\log_2(\min(\text{srcALen}, \text{srcBLen}))$ ($\log_2$ is read as log to the base 2) times to avoid overflows, as maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions are carried internally. The 2.30 accumulator is right shifted by 15 bits and then saturated to 1.15 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$

RISCV_DSP_ATTRIBUTE void riscv_conv_fast_q31 (const q31_t *pSrcA, uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)

Convolution of Q31 sequences (fast version).

Convolution of Q31 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_q31()` for a slower implementation of this function which uses 64-bit accumulation to provide higher precision.

Scaling and Overflow Behavior

This function is optimized for speed at the expense of fixed-point precision and overflow protection. The result of each 1.31 x 1.31 multiplication is truncated to 2.30 format. These intermediate results are accumulated in a 32-bit register in 2.30 format. Finally, the accumulator is saturated and converted to a 1.31 result.

The fast version has the same overflow behavior as the standard version but provides less precision since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signals must be scaled down. Scale down the inputs by $\log_2(\min(\text{srcALen}, \text{srcBLen}))$ ($\log_2$ is read as log to the base 2) times to avoid overflows, as maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions are carried internally.

Parameters

- **pSrcA** – [in] points to the first input sequence.
- **srcALen** – [in] length of the first input sequence.
- **pSrcB** – [in] points to the second input sequence.
- **srcBLen** – [in] length of the second input sequence.

- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.

RISCV_DSP_ATTRIBUTE void **riscv_conv_opt_q15** (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch1, q15_t *pScratch2)

Convolution of Q15 sequences.

Remark

Refer to **riscv_conv_fast_q15()** for a faster but less precise version of this function.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both inputs are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.
- **pScratch1** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.
- **pScratch2** – [in] points to scratch buffer of size $\min(\text{srcALen}, \text{srcBLen})$.

RISCV_DSP_ATTRIBUTE void **riscv_conv_opt_q7** (const q7_t *pSrcA, uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, q15_t *pScratch1, q15_t *pScratch2)

Convolution of Q7 sequences.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both the inputs are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. This approach provides 17 guard bits and there is no risk of overflow as long as $\max(\text{srcALen}, \text{srcBLen}) < 131072$. The 18.14 result is then truncated to 18.7 format by discarding the low 7 bits and then saturated to 1.7 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence

- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.
- **pScratch1** – [in] points to scratch buffer (of type `q15_t`) of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.
- **pScratch2** – [in] points to scratch buffer (of type `q15_t`) of size $\min(\text{srcALen}, \text{srcBLen})$.

RISCV_DSP_ATTRIBUTE void riscv_conv_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)

Convolution of Q15 sequences.

Remark

Refer to `riscv_conv_fast_q15()` for a faster but less precise version of this function.

Remark

Refer to `riscv_conv_opt_q15()` for a faster implementation of this function using scratch buffers.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both inputs are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.

RISCV_DSP_ATTRIBUTE void riscv_conv_q31 (const q31_t *pSrcA, uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)

Convolution of Q31 sequences.

Remark

Refer to `riscv_conv_fast_q31()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down the inputs by $\log_2(\min(\text{srcALen}, \text{srcBLen}))$ ($\log_2$ is read as log to the base 2) times to avoid overflows, as maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions are carried internally. The 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.

```
RISCV_DSP_ATTRIBUTE void riscv_conv_q7 (const q7_t *pSrcA, uint32_t srcALen,
const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst)
```

Convolution of Q7 sequences.

Remark

Refer to `riscv_conv_opt_q7()` for a faster implementation of this function.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both the inputs are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. This approach provides 17 guard bits and there is no risk of overflow as long as $\max(\text{srcALen}, \text{srcBLen}) < 131072$. The 18.14 result is then truncated to 18.7 format by discarding the low 7 bits and then saturated to 1.7 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $\text{srcALen} + \text{srcBLen} - 1$.

Partial Convolution

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_f32 (const float32_t *pSrcA,
uint32_t srcALen, const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst,
uint32_t firstIndex, uint32_t numPoints)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_opt_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex,
uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex,
uint32_t numPoints)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_q31 (const q31_t *pSrcA,
uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst, uint32_t firstIndex,
uint32_t numPoints)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_opt_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex,
uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_opt_q7 (const q7_t *pSrcA,
uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, uint32_t firstIndex,
uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex,
uint32_t numPoints)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q31 (const q31_t *pSrcA,
uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst, uint32_t firstIndex,
uint32_t numPoints)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q7 (const q7_t *pSrcA,
uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, uint32_t firstIndex,
uint32_t numPoints)
```

group Partial Convolution

Partial Convolution is equivalent to Convolution except that a subset of the output samples is generated. Each function has two additional arguments. `firstIndex` specifies the starting index of the subset of output samples. `numPoints` is the number of output samples to compute. The function computes the output in the range `[firstIndex, ..., firstIndex+numPoints-1]`. The output array `pDst` contains `numPoints` values.

The allowable range of output indices is `[0 srcALen+srcBLen-2]`. If the requested subset does not fall in this range then the functions return `RISCV_MATH_ARGUMENT_ERROR`. Otherwise the functions return `RISCV_MATH_SUCCESS`.

Fast Versions

Fast versions are supported for Q31 and Q15 of partial convolution. Cycles for Fast versions are less

compared to Q31 and Q15 of partial conv and the design requires the input signals should be scaled down to avoid intermediate overflows.

Opt Versions

Opt versions are supported for Q15 and Q7. Design uses internal scratch buffer for getting good optimisation. These versions are optimised in cycles and consumes more memory (Scratch memory) compared to Q15 and Q7 versions of partial convolution

Long versions:

For convolution of long vectors, those functions are no more adapted and will be very slow. An implementation based upon FFTs should be used.

Note

Refer to `riscv_conv_f32()` for details on fixed point behavior.

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_f32 (const float32_t *pSrcA,  
uint32_t srcALen, const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst,  
uint32_t firstIndex, uint32_t numPoints)
```

Partial convolution of floating-point sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_opt_q15 (const q15_t *pSrcA,  
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst,  
uint32_t firstIndex, uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)
```

Partial convolution of Q15 sequences (fast version).

Partial convolution of Q15 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_partial_q15()` for a slower implementation of this function which uses a 64-bit accumulator to avoid wrap around distortion.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed
- **pScratch1** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$
- **pScratch2** – [in] points to scratch buffer of size $\min(\text{srcALen}, \text{srcBLen})$

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range $[0, \text{srcALen} + \text{srcBLen} - 2]$

RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex, uint32_t numPoints)

Partial convolution of Q15 sequences (fast version).

Partial convolution of Q15 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_partial_q15()` for a slower implementation of this function which uses a 64-bit accumulator to avoid wrap around distortion.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with

- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_fast_q31 (const q31_t *pSrcA,
uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst,
uint32_t firstIndex, uint32_t numPoints)
```

Partial convolution of Q31 sequences (fast version).

Partial convolution of Q31 sequences (fast version) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_conv_partial_q31()` for a slower implementation of this function which uses a 64-bit accumulator to provide higher precision.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_opt_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst,
uint32_t firstIndex, uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)
```

Partial convolution of Q15 sequences.

Remark

Refer to `riscv_conv_partial_fast_q15()` for a faster but less precise version of this function.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed
- **pScratch1** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.
- **pScratch2** – [in] points to scratch buffer of size $\min(\text{srcALen}, \text{srcBLen})$.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range $[0, \text{srcALen} + \text{srcBLen} - 2]$

RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_opt_q7 (const q7_t *pSrcA, uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, uint32_t firstIndex, uint32_t numPoints, q15_t *pScratch1, q15_t *pScratch2)

Partial convolution of Q7 sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed
- **pScratch1** – [in] points to scratch buffer (of type q15_t) of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.
- **pScratch2** – [in] points to scratch buffer (of type q15_t) of size $\min(\text{srcALen}, \text{srcBLen})$.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range $[0, \text{srcALen} + \text{srcBLen} - 2]$

RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, uint32_t firstIndex, uint32_t numPoints)

Partial convolution of Q15 sequences.

Remark

Refer to `riscv_conv_partial_fast_q15()` for a faster but less precise version of this function.

Remark

Refer to `riscv_conv_partial_opt_q15()` for a faster implementation of this function using scratch buffers.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q31 (const q31_t *pSrcA,  
uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst,  
uint32_t firstIndex, uint32_t numPoints)
```

Partial convolution of Q31 sequences.

Remark

Refer to `riscv_conv_partial_fast_q31()` for a faster but less precise implementation of this function.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence

- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_conv_partial_q7 (const q7_t *pSrcA,
uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst,
uint32_t firstIndex, uint32_t numPoints)
```

Partial convolution of Q7 sequences.

Remark

Refer to `riscv_conv_partial_opt_q7()` for a faster implementation of this function.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written
- **firstIndex** – [in] is the first output sample to start with
- **numPoints** – [in] is the number of output points to be computed

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : requested subset is not in the range [0 srcALen+srcBLen-2]

Correlation

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_f16 (const float16_t *pSrcA, uint32_t srcALen,
const float16_t *pSrcB, uint32_t srcBLen, float16_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_f32 (const float32_t *pSrcA, uint32_t srcALen,
const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_f64 (const float64_t *pSrcA, uint32_t srcALen,
const float64_t *pSrcB, uint32_t srcBLen, float64_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_opt_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_q15 (const q15_t *pSrcA, uint32_t srcALen,
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_q31 (const q31_t *pSrcA, uint32_t srcALen,
const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_opt_q15 (const q15_t *pSrcA, uint32_t srcALen,
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_opt_q7 (const q7_t *pSrcA, uint32_t srcALen,
const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, q15_t *pScratch1, q15_t *pScratch2)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_q15 (const q15_t *pSrcA, uint32_t srcALen,
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_q31 (const q31_t *pSrcA, uint32_t srcALen,
const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_q7 (const q7_t *pSrcA, uint32_t srcALen,
const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst)
```

group **Correlation**

Correlation is a mathematical operation that is similar to convolution. As with convolution, correlation uses two signals to produce a third signal. The underlying algorithms in correlation and convolution are identical except that one of the inputs is flipped in convolution. Correlation is commonly used to measure the similarity between two signals. It has applications in pattern recognition, cryptanalysis, and searching. The NMSIS library provides correlation functions for Q7, Q15, Q31 and floating-point data types. Fast versions of the Q15 and Q31 functions are also provided.

In correlation, one of the signals is flipped in time

Algorithm

Let $a[n]$ and $b[n]$ be sequences of length $srcALen$ and $srcBLen$ samples respectively. The convolution of the two signals is denoted by

$$c[n] = a[n] * b[n]$$

$$c[n] = a[n] * b[-n]$$

and this is mathematically defined as

$$c[n] = \sum_{k=0}^{srcALen} a[k]b[k-n]$$

The pSrcA points to the first input vector of length srcALen and pSrcB points to the second input vector of length srcBLen. The result c[n] is of length $2 * \max(srcALen, srcBLen) - 1$ and is defined over the interval $n=0, 1, 2, \dots, (2 * \max(srcALen, srcBLen) - 2)$. The output result is written to pDst and the calling function must allocate $2 * \max(srcALen, srcBLen) - 1$ words for the result.

Fixed-Point Behavior

Correlation requires summing up a large number of intermediate products. As such, the Q7, Q15, and Q31 functions run a risk of overflow and saturation. Refer to the function specific documentation below for further details of the particular algorithm used.

Fast Versions

Fast versions are supported for Q31 and Q15. Cycles for Fast versions are less compared to Q31 and Q15 of correlate and the design requires the input signals should be scaled down to avoid intermediate overflows.

Opt Versions

Opt versions are supported for Q15 and Q7. Design uses internal scratch buffer for getting good optimisation. These versions are optimised in cycles and consumes more memory (Scratch memory) compared to Q15 and Q7 versions of correlate

Long versions:

For convolution of long vectors, those functions are no more adapted and will be very slow. An implementation based upon FFTs should be used.

Note

The pDst should be initialized to all zeros before being used.

Functions

RISCV_DSP_ATTRIBUTE void riscv_correlate_f16 (const float16_t *pSrcA, uint32_t srcALen, const float16_t *pSrcB, uint32_t srcBLen, float16_t *pDst)

Correlation of floating-point sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(srcALen, srcBLen) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_f32 (const float32_t *pSrcA, uint32_t srcALen, const float32_t *pSrcB, uint32_t srcBLen, float32_t *pDst)

Correlation of floating-point sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_f64 (const float64_t *pSrcA, uint32_t srcALen, const float64_t *pSrcB, uint32_t srcBLen, float64_t *pDst)

Correlation of floating-point sequences.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_opt_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst, q15_t *pScratch)

Correlation of Q15 sequences (fast version).

Remark

Refer to `riscv_correlate_q15()` for a slower implementation of this function which uses a 64-bit accumulator to avoid wrap around distortion.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down one of the inputs by $1/\min(\text{srcALen}, \text{srcBLen})$ to avoid overflow since a maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions is carried internally. The 2.30 accumulator is right shifted by 15 bits and then saturated to 1.15 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence

- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence.
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.
- **pScratch** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_q15 (const q15_t *pSrcA, uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)

Correlation of Q15 sequences (fast version).

Remark

Refer to `riscv_correlate_q15()` for a slower implementation of this function which uses a 64-bit accumulator to avoid wrap around distortion.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down one of the inputs by $1/\min(\text{srcALen}, \text{srcBLen})$ to avoid overflow since a maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions is carried internally. The 2.30 accumulator is right shifted by 15 bits and then saturated to 1.15 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_fast_q31 (const q31_t *pSrcA, uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)

Correlation of Q31 sequences (fast version).

Remark

Refer to `riscv_correlate_q31()` for a slower implementation of this function which uses 64-bit accumulation to provide higher precision.

Scaling and Overflow Behavior

This function is optimized for speed at the expense of fixed-point precision and overflow protection.

The result of each 1.31 x 1.31 multiplication is truncated to 2.30 format. These intermediate results are accumulated in a 32-bit register in 2.30 format. Finally, the accumulator is saturated and converted to a 1.31 result.

The fast version has the same overflow behavior as the standard version but provides less precision since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signals must be scaled down. The input signals should be scaled down to avoid intermediate overflows. Scale down one of the inputs by $1/\min(\text{srcALen}, \text{srcBLen})$ to avoid overflows since a maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions is carried internally.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_opt_q15 (const q15_t *pSrcA,
uint32_t srcALen, const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst,
q15_t *pScratch)
```

Correlation of Q15 sequences.

Remark

Refer to `riscv_correlate_fast_q15()` for a faster but less precise version of this function.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both inputs are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.
- **pScratch** – [in] points to scratch buffer of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_opt_q7 (const q7_t *pSrcA,
uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst, q15_t *pScratch1,
q15_t *pScratch2)
```

Correlation of Q7 sequences.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both the inputs are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. This approach provides 17 guard bits and there is no risk of overflow as long as $\max(\text{srcALen}, \text{srcBLen}) < 131072$. The 18.14 result is then truncated to 18.7 format by discarding the low 7 bits and then saturated to 1.7 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.
- **pScratch1** – [in] points to scratch buffer (of type q15_t) of size $\max(\text{srcALen}, \text{srcBLen}) + 2 * \min(\text{srcALen}, \text{srcBLen}) - 2$.
- **pScratch2** – [in] points to scratch buffer (of type q15_t) of size $\min(\text{srcALen}, \text{srcBLen})$.

```
RISCV_DSP_ATTRIBUTE void riscv_correlate_q15 (const q15_t *pSrcA, uint32_t srcALen,
const q15_t *pSrcB, uint32_t srcBLen, q15_t *pDst)
```

Correlation of Q15 sequences.

Remark

Refer to `riscv_correlate_fast_q15()` for a faster but less precise version of this function.

Remark

Refer to `riscv_correlate_opt_q15()` for a faster implementation of this function using scratch buffers.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both inputs are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Parameters

- **pSrcA** – [in] points to the first input sequence

- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_q31 (const q31_t *pSrcA, uint32_t srcALen, const q31_t *pSrcB, uint32_t srcBLen, q31_t *pDst)

Correlation of Q31 sequences.

Remark

Refer to `riscv_correlate_fast_q31()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. Scale down one of the inputs by $1/\min(\text{srcALen}, \text{srcBLen})$ to avoid overflows since a maximum of $\min(\text{srcALen}, \text{srcBLen})$ number of additions is carried internally. The 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

RISCV_DSP_ATTRIBUTE void riscv_correlate_q7 (const q7_t *pSrcA, uint32_t srcALen, const q7_t *pSrcB, uint32_t srcBLen, q7_t *pDst)

Correlation of Q7 sequences.

Remark

Refer to `riscv_correlate_opt_q7()` for a faster implementation of this function.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both the inputs are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. This approach provides 17 guard bits and there is no risk of

overflow as long as $\max(\text{srcALen}, \text{srcBLen}) < 131072$. The 18.14 result is then truncated to 18.7 format by discarding the low 7 bits and saturated to 1.7 format.

Parameters

- **pSrcA** – [in] points to the first input sequence
- **srcALen** – [in] length of the first input sequence
- **pSrcB** – [in] points to the second input sequence
- **srcBLen** – [in] length of the second input sequence
- **pDst** – [out] points to the location where the output result is written. Length $2 * \max(\text{srcALen}, \text{srcBLen}) - 1$.

Finite Impulse Response (FIR) Decimator

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_f32 (const riscv_fir_decimate_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_f64 (const riscv_fir_decimate_instance_f64 *S,
const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)
```

```
void riscv_fir_decimate_fast_q15 (const riscv_fir_decimate_instance_q15 *S, const q15_t (page 1277) *pSrc,
q15_t (page 1277) *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_fast_q31 (const riscv_fir_decimate_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_f32 (riscv_fir_decimate_instance_f32 *S,
uint16_t numTaps, uint8_t M, const float32_t *pCoeffs, float32_t *pState,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_f64 (riscv_fir_decimate_instance_f64 *S,
uint16_t numTaps, uint8_t M, const float64_t *pCoeffs, float64_t *pState,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_q15 (riscv_fir_decimate_instance_q15 *S,
uint16_t numTaps, uint8_t M, const q15_t *pCoeffs, q15_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_q31 (riscv_fir_decimate_instance_q31 *S,
uint16_t numTaps, uint8_t M, const q31_t *pCoeffs, q31_t *pState, uint32_t blockSize)
```

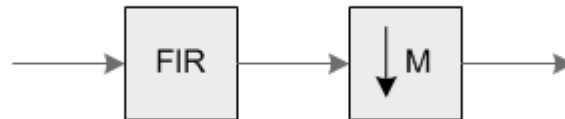
```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_q15 (const riscv_fir_decimate_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_q31 (const riscv_fir_decimate_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

group **Finite Impulse Response (FIR) Decimator**

These functions combine an FIR filter together with a decimator. They are used in multirate systems for reducing the sample rate of a signal without introducing aliasing distortion. Conceptually, the functions are equivalent to the block diagram below:

When decimating by a factor of M , the signal should be prefiltered by a lowpass filter with a normalized cutoff frequency of $1/M$ in order to prevent aliasing distortion. The user of the function is responsible for providing the



filter coefficients.

The FIR decimator functions provided in the NMSIS DSP Library combine the FIR filter and the decimator in an efficient manner. Instead of calculating all of the FIR filter outputs and discarding $M-1$ out of every M , only the samples output by the decimator are computed. The functions operate on blocks of input and output data. `pSrc` points to an array of `blockSize` input values and `pDst` points to an array of `blockSize/M` output values. In order to have an integer number of output samples `blockSize` must always be a multiple of the decimation factor M .

The library provides separate functions for Q15, Q31 and floating-point data types.

Algorithm:

The FIR portion of the algorithm uses the standard form filter:

$$y[n] = b[0] * x[n] + b[1] * x[n-1] + b[2] * x[n-2] + \dots + b[\text{numTaps}-1] * x[n-\text{numTaps}+1]$$

where, $b[n]$ are the filter coefficients.

The `pCoeffs` points to a coefficient array of size `numTaps`. Coefficients are stored in time reversed order.

$$\{b[\text{numTaps}-1], b[\text{numTaps}-2], b[\text{numTaps}-3], \dots, b[1], b[0]\}$$

`pState` points to a state array of size `numTaps + blockSize - 1`. Samples in the state buffer are stored in the order:

$$\{x[n-\text{numTaps}+1], x[n-\text{numTaps}], x[n-\text{numTaps}-1], x[n-\text{numTaps}-2], \dots, x[0], x[1], \dots, x[\text{blockSize}-1]\}$$

The state variables are updated after each block of data is processed, the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable array should be allocated separately. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer.
- Checks to make sure that the size of the input is a multiple of the decimation factor. To do this manually without calling the init function, assign the follow subfields of the instance structure: numTaps, pCoeffs, M (decimation factor), pState. Also set all of the values in pState to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. The code below statically initializes each of the 3 different data type filter instance structures

```
riscv_fir_decimate_instance_f32 S = {M, numTaps, pCoeffs, pState};
riscv_fir_decimate_instance_q31 S = {M, numTaps, pCoeffs, pState};
riscv_fir_decimate_instance_q15 S = {M, numTaps, pCoeffs, pState};
```

where M is the decimation factor; numTaps is the number of filter coefficients in the filter; pCoeffs is the address of the coefficient buffer; pState is the address of the state buffer. Be sure to set the values in the state buffer to zeros when doing static initialization.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the FIR decimate filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_f32 (const riscv_fir_decimate_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for floating-point FIR decimator.

Parameters

- **S** – [in] points to an instance of the floating-point FIR decimator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_f64 (const riscv_fir_decimate_instance_f64 *S, const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)

Processing function for floating-point FIR decimator.

Parameters

- **S** – [in] points to an instance of the floating-point FIR decimator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

```
void riscv_fir_decimate_fast_q15(const riscv_fir_decimate_instance_q15 *S, const q15_t (page 1277)
                               *pSrc, q15_t (page 1277) *pDst, uint32_t blockSize)
```

Processing function for the Q15 FIR decimator (fast variant).

Processing function for the Q15 FIR decimator (fast variant) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_fir_decimate_q15()` for a slower implementation of this function which uses 64-bit accumulation to avoid wrap around distortion. Both the slow and the fast versions use the same instance structure. Use function `riscv_fir_decimate_init_q15()` to initialize the filter structure.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around and distorts the result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits ($\log_2$ is read as log to the base 2). The 2.30 accumulator is then truncated to 2.15 format and saturated to yield the 1.15 result.

Parameters

- **S** – [in] points to an instance of the Q15 FIR decimator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process per call

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_fast_q31 (const riscv_fir_decimate_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for the Q31 FIR decimator (fast variant).

Processing function for the Q31 FIR decimator (fast variant) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_fir_decimate_q31()` for a slower implementation of this function which uses a 64-bit accumulator to provide higher precision. Both the slow and the fast versions use the same instance structure. Use function `riscv_fir_decimate_init_q31()` to initialize the filter structure.

Scaling and Overflow Behavior

This function is optimized for speed at the expense of fixed-point precision and overflow protection. The result of each 1.31 x 1.31 multiplication is truncated to 2.30 format. These intermediate results are added to a 2.30 accumulator. Finally, the accumulator is saturated and converted to a 1.31 result. The fast version has the same overflow behavior as the standard version and provides less precision since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits (where $\log_2$ is read as log to the base 2).

Parameters

- **S** – [in] points to an instance of the Q31 FIR decimator structure

- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_f32 (riscv_fir_decimate_instance_f32 *S,
uint16_t numTaps, uint8_t M, const float32_t *pCoeffs, float32_t *pState,
uint32_t blockSize)
```

Initialization function for the floating-point FIR decimator.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 words where blockSize is the number of input samples passed to riscv_fir_decimate_f32(). M is the decimation factor.

Parameters

- **S** – [inout] points to an instance of the floating-point FIR decimator structure
- **numTaps** – [in] number of coefficients in the filter
- **M** – [in] decimation factor
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process per call

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_LENGTH_ERROR** : blockSize is not a multiple of M

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_f64 (riscv_fir_decimate_instance_f64 *S,
uint16_t numTaps, uint8_t M, const float64_t *pCoeffs, float64_t *pState,
uint32_t blockSize)
```

Initialization function for the floating-point FIR decimator.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 words where blockSize is the number of input samples passed to riscv_fir_decimate_f64(). M is the decimation factor.

Parameters

- **S** – [inout] points to an instance of the floating-point FIR decimator structure
- **numTaps** – [in] number of coefficients in the filter
- **M** – [in] decimation factor
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process per call

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_LENGTH_ERROR** : blockSize is not a multiple of M

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_q15 (riscv_fir_decimate_instance_q15 *S,  
uint16_t numTaps, uint8_t M, const q15_t *pCoeffs, q15_t *pState,  
uint32_t blockSize)
```

Initialization function for the Q15 FIR decimator.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}

pState points to the array of state variables. pState is of length numTaps+blockSize-1 words where blockSize is the number of input samples to the call riscv_fir_decimate_q15(). M is the decimation factor.

Parameters

- **S** – [inout] points to an instance of the Q15 FIR decimator structure
- **numTaps** – [in] number of coefficients in the filter
- **M** – [in] decimation factor
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_LENGTH_ERROR** : blockSize is not a multiple of M

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_decimate_init_q31 (riscv_fir_decimate_instance_q31 *S,  
uint16_t numTaps, uint8_t M, const q31_t *pCoeffs, q31_t *pState,  
uint32_t blockSize)
```

Initialization function for the Q31 FIR decimator.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

`pState` points to the array of state variables. `pState` is of length `numTaps+blockSize-1` words where `blockSize` is the number of input samples passed to `riscv_fir_decimate_q31()`. `M` is the decimation factor.

Parameters

- **S** – [inout] points to an instance of the Q31 FIR decimator structure
- **numTaps** – [in] number of coefficients in the filter
- **M** – [in] decimation factor
- **pCoeffs** – [in] points to the filter coefficients
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_LENGTH_ERROR` : `blockSize` is not a multiple of `M`

RISCV_DSP_ATTRIBUTE void `riscv_fir_decimate_q15` (const `riscv_fir_decimate_instance_q15` *S, const `q15_t` *pSrc, `q15_t` *pDst, `uint32_t` blockSize)

Processing function for the Q15 FIR decimator.

Remark

Refer to `riscv_fir_decimate_fast_q15()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

Parameters

- **S** – [in] points to an instance of the Q15 FIR decimator structure
- **pSrc** – [in] points to the block of input data

- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process per call

```
RISCV_DSP_ATTRIBUTE void riscv_fir_decimate_q31 (const riscv_fir_decimate_instance_q31 *S,  
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for the Q31 FIR decimator.

Remark

Refer to `riscv_fir_decimate_fast_q31()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits (where $\log_2$ is read as log to the base 2). After all multiply-accumulates are performed, the 2.62 accumulator is truncated to 1.32 format and then saturated to 1.31 format.

Parameters

- **S** – [in] points to an instance of the Q31 FIR decimator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

Finite Impulse Response (FIR) Filters

```
RISCV_DSP_ATTRIBUTE void riscv_fir_f16 (const riscv_fir_instance_f16 *S,  
const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_f32 (const riscv_fir_instance_f32 *S,  
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_f64 (const riscv_fir_instance_f64 *S,  
const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_fast_q15 (const riscv_fir_instance_q15 *S,  
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_fast_q31 (const riscv_fir_instance_q31 *S,  
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_f16 (riscv_fir_instance_f16 *S, uint16_t numTaps,  
const float16_t *pCoeffs, float16_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_f32 (riscv_fir_instance_f32 *S, uint16_t numTaps,
const float32_t *pCoeffs, float32_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_f64 (riscv_fir_instance_f64 *S, uint16_t numTaps,
const float64_t *pCoeffs, float64_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_init_q15 (riscv_fir_instance_q15 *S,
uint16_t numTaps, const q15_t *pCoeffs, q15_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_q31 (riscv_fir_instance_q31 *S, uint16_t numTaps,
const q31_t *pCoeffs, q31_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_q7 (riscv_fir_instance_q7 *S, uint16_t numTaps,
const q7_t *pCoeffs, q7_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q15 (const riscv_fir_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q31 (const riscv_fir_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q7 (const riscv_fir_instance_q7 *S, const q7_t *pSrc,
q7_t *pDst, uint32_t blockSize)
```

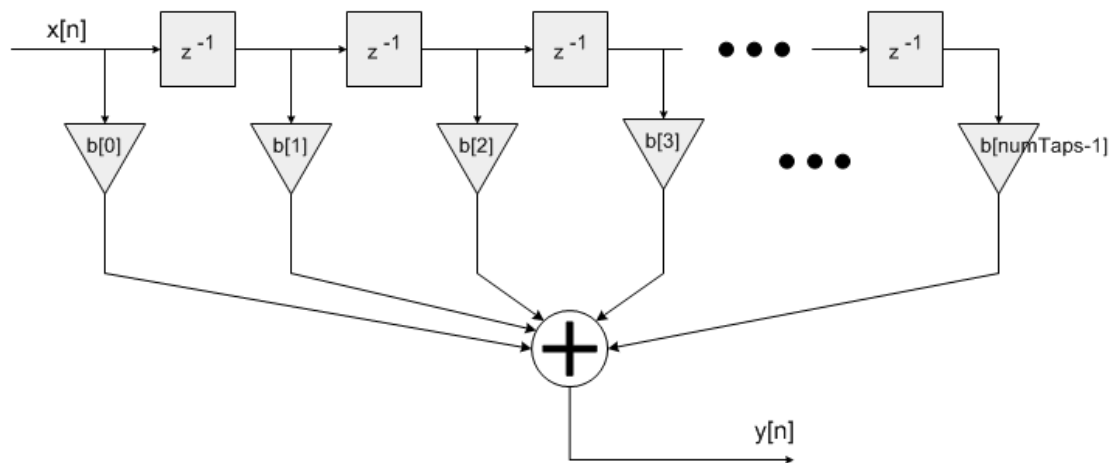
group **Finite Impulse Response (FIR) Filters**

This set of functions implements Finite Impulse Response (FIR) filters for Q7, Q15, Q31, and floating-point data types. Fast versions of Q15 and Q31 are also provided. The functions operate on blocks of input and output data and each call to the function processes `blockSize` samples through the filter. `pSrc` and `pDst` points to input and output arrays containing `blockSize` values.

Algorithm

The FIR filter algorithm is based upon a sequence of multiply-accumulate (MAC) operations. Each filter coefficient `b[n]` is multiplied by a state variable which equals a previous input sample `x[n]`.

$$y[n] = b[0] * x[n] + b[1] * x[n-1] + b[2] * x[n-2] + \dots + b[\text{numTaps}-1] * x[n-\text{numTaps}+1]$$



`pCoeffs` points to a coefficient array of size `numTaps`. Coefficients are stored in time reversed order.

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

`pState` points to a state array of size `numTaps + blockSize - 1`. Samples in the state buffer are stored in the following order.

```
{x[n-numTaps+1], x[n-numTaps], x[n-numTaps-1], x[n-numTaps-2]....  
↪ x[n](==pSrc[0]), x[n+1](==pSrc[1]), ..., x[n+blockSize-1](==pSrc[blockSize-  
↪ 1])}
```

Note that the length of the state buffer exceeds the length of the coefficient array by `blockSize-1`. The increased state buffer length allows circular addressing, which is traditionally used in the FIR filters, to be avoided and yields a significant speed improvement. The state variables are updated after each block of data is processed; the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared. There are separate instance structure declarations for each of the 4 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numTaps`, `pCoeffs`, `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance

structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. The code below statically initializes each of the 4 different data type filter instance structures

```
riscv_fir_instance_f32 S = {numTaps, pState, pCoeffs};
riscv_fir_instance_q31 S = {numTaps, pState, pCoeffs};
riscv_fir_instance_q15 S = {numTaps, pState, pCoeffs};
riscv_fir_instance_q7 S = {numTaps, pState, pCoeffs};
```

where numTaps is the number of filter coefficients in the filter; pState is the address of the state buffer; pCoeffs is the address of the coefficient buffer.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the FIR filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

RISCV_DSP_ATTRIBUTE void riscv_fir_f16 (const riscv_fir_instance_f16 *S, const float16_t *pSrc, float16_t *pDst, uint32_t blockSize)

Processing function for floating-point FIR filter.

Processing function for the floating-point FIR filter.

Parameters

- **S** – [in] points to an instance of the floating-point FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_f32 (const riscv_fir_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for floating-point FIR filter.

Processing function for the floating-point FIR filter.

Parameters

- **S** – [in] points to an instance of the floating-point FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_f64 (const riscv_fir_instance_f64 *S, const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)

Processing function for floating-point FIR filter.

Processing function for the floating-point FIR filter.

Parameters

- **S** – [in] points to an instance of the floating-point FIR filter structure

- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_fast_q15 (const riscv_fir_instance_q15 *S, const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Processing function for the Q15 FIR filter (fast version).

Processing function for the fast Q15 FIR filter (fast version).

Remark

Refer to `riscv_fir_q15()` for a slower implementation of this function which uses 64-bit accumulation to avoid wrap around distortion. Both the slow and the fast versions use the same instance structure. Use function `riscv_fir_init_q15()` to initialize the filter structure.

Scaling and Overflow Behavior

This fast version uses a 32-bit accumulator with 2.30 format. The accumulator maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around and distorts the result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits. The 2.30 accumulator is then truncated to 2.15 format and saturated to yield the 1.15 result.

Parameters

- **S** – [in] points to an instance of the Q15 FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_fast_q31 (const riscv_fir_instance_q31 *S, const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Processing function for the Q31 FIR filter (fast version).

Processing function for the fast Q31 FIR filter (fast version).

Remark

Refer to `riscv_fir_q31()` for a slower implementation of this function which uses a 64-bit accumulator to provide higher precision. Both the slow and the fast versions use the same instance structure. Use function `riscv_fir_init_q31()` to initialize the filter structure.

Scaling and Overflow Behavior

This function is optimized for speed at the expense of fixed-point precision and overflow protection. The result of each 1.31 x 1.31 multiplication is truncated to 2.30 format. These intermediate results are added to a 2.30 accumulator. Finally, the accumulator is saturated and converted to a 1.31 result. The fast version has the same overflow behavior as the standard version and provides less precision.

since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits.

Parameters

- **S** – [in] points to an instance of the Q31 structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_f16 (riscv_fir_instance_f16 *S,
uint16_t numTaps, const float16_t *pCoeffs, float16_t *pState, uint32_t blockSize)
```

Initialization function for the floating-point FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_fir_f16().

Parameters

- **S** – [inout] points to an instance of the floating-point FIR filter structure
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficients buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of samples processed per call

```
RISCV_DSP_ATTRIBUTE void riscv_fir_init_f32 (riscv_fir_instance_f32 *S,
uint16_t numTaps, const float32_t *pCoeffs, float32_t *pState, uint32_t blockSize)
```

Initialization function for the floating-point FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_fir_f32().

Parameters

- **S** – [inout] points to an instance of the floating-point FIR filter structure
- **numTaps** – [in] number of filter coefficients in the filter

- **pCoeffs** – [in] points to the filter coefficients buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of samples processed per call

RISCV_DSP_ATTRIBUTE void riscv_fir_init_f64 (riscv_fir_instance_f64 *S, uint16_t numTaps, const float64_t *pCoeffs, float64_t *pState, uint32_t blockSize)

Initialization function for the floating-point FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_fir_f64().

Parameters

- **S** – [inout] points to an instance of the floating-point FIR filter structure
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficients buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of samples processed per call

RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_init_q15 (riscv_fir_instance_q15 *S, uint16_t numTaps, const q15_t *pCoeffs, q15_t *pState, uint32_t blockSize)

Initialization function for the Q15 FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

Note that numTaps must be even and greater than or equal to 4. To implement an odd length filter simply increase numTaps by 1 and set the last coefficient to zero. For example, to implement a filter with numTaps=3 and coefficients

```
{0.3, -0.8, 0.3}
```

set numTaps=4 and use the coefficients:

```
{0.3, -0.8, 0.3, 0}.
```

Similarly, to implement a two point filter

```
{0.3, -0.3}
```

set numTaps=4 and use the coefficients:

```
{0.3, -0.3, 0, 0}.
```

pState points to the array of state variables. pState is of length numTaps+blockSize, when RISC_V_MATH_DSP is defined. Otherwise, it is of length numTaps+blockSize-1 where blockSize is the number of input samples processed by each call to riscv_fir_q15().

Parameters

- **S** – [inout] points to an instance of the Q15 FIR filter structure.
- **numTaps** – [in] number of filter coefficients in the filter. Must be even and greater than or equal to 4.
- **pCoeffs** – [in] points to the filter coefficients buffer.
- **pState** – [in] points to the state buffer.
- **blockSize** – [in] number of samples processed per call.

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : numTaps is not greater than or equal to 4 and even

```
RISC_V_DSP_ATTRIBUTE void riscv_fir_init_q31 (riscv_fir_instance_q31 *S,
uint16_t numTaps, const q31_t *pCoeffs, q31_t *pState, uint32_t blockSize)
```

Initialization function for the Q31 FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_fir_q31().

Parameters

- **S** – [inout] points to an instance of the Q31 FIR filter structure
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficients buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of samples processed

```
RISC_V_DSP_ATTRIBUTE void riscv_fir_init_q7 (riscv_fir_instance_q7 *S,
uint16_t numTaps, const q7_t *pCoeffs, q7_t *pState, uint32_t blockSize)
```

Initialization function for the Q7 FIR filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

pState points to the array of state variables. pState is of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_fir_q7().

Parameters

- **S** – [inout] points to an instance of the Q7 FIR filter structure
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficients buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of samples processed

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q15 (const riscv_fir_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

Processing function for the Q15 FIR filter.

Remark

Refer to riscv_fir_fast_q15() for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

Parameters

- **S** – [in] points to an instance of the Q15 FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q31 (const riscv_fir_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for Q31 FIR filter.

Processing function for the Q31 FIR filter.

Remark

Refer to `riscv_fir_fast_q31()` for a faster but less precise implementation of this filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits. After all multiply-accumulates are performed, the 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **S** – [in] points to an instance of the Q31 FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_fir_q7 (const riscv_fir_instance_q7 *S,
const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)
```

Processing function for Q7 FIR filter.

Processing function for the Q7 FIR filter.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both coefficients and state variables are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. The accumulator is converted to 18.7 format by discarding the low 7 bits. Finally, the result is truncated to 1.7 format.

Parameters

- **S** – [in] points to an instance of the Q7 FIR filter structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

Finite Impulse Response (FIR) Lattice Filters

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_f32 (const riscv_fir_lattice_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_f32 (riscv_fir_lattice_instance_f32 *S,
uint16_t numStages, const float32_t *pCoeffs, float32_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_q15 (riscv_fir_lattice_instance_q15 *S,
uint16_t numStages, const q15_t *pCoeffs, q15_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_q31 (riscv_fir_lattice_instance_q31 *S,
uint16_t numStages, const q31_t *pCoeffs, q31_t *pState)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_q15 (const riscv_fir_lattice_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_q31 (const riscv_fir_lattice_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

group Finite Impulse Response (FIR) Lattice Filters

Deprecated:

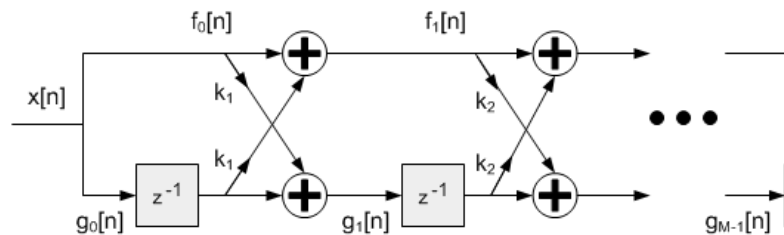
Those functions are no more tested nor maintained and will be removed in a future version.

This set of functions implements Finite Impulse Response (FIR) lattice filters for Q15, Q31 and floating-point data types. Lattice filters are used in a variety of adaptive filter applications. The filter structure is feedforward and the net impulse response is finite length. The functions operate on blocks of input and output data and each call to the function processes blockSize samples through the filter. pSrc and pDst point to input and output arrays containing blockSize values.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Algorithm



The following difference equation is implemented:

```
f0[n] = g0[n] = x[n]
fm[n] = fm-1[n] + km * gm-1[n-1] for m = 1, 2, ...M
gm[n] = km * fm-1[n] + gm-1[n-1] for m = 1, 2, ...M
y[n] = fM[n]
```


`pCoeffs` points to the array of reflection coefficients of size `numStages`. Reflection Coefficients are stored in the following order.

```
{k1, k2, ..., kM}
```

where `M` is number of stages

`pState` points to a state array of size `numStages`. The state variables (`g` values) hold previous inputs and are stored in the following order.

```
{g0[n], g1[n], g2[n] ... gM-1[n]}
```

The state variables are updated after each block of data is processed; the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numStages`, `pCoeffs`, `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros and then manually initialize the instance structure as follows:

```
riscv_fir_lattice_instance_f32 S = {numStages, pState, pCoeffs};
riscv_fir_lattice_instance_q31 S = {numStages, pState, pCoeffs};
riscv_fir_lattice_instance_q15 S = {numStages, pState, pCoeffs};
```

where `numStages` is the number of stages in the filter; `pState` is the address of the state buffer; `pCoeffs` is the address of the coefficient buffer.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the FIR Lattice filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Functions

RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_f32 (const riscv_fir_lattice_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point FIR lattice filter.

Parameters

- **S** – [in] points to an instance of the floating-point FIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_f32 (riscv_fir_lattice_instance_f32 *S, uint16_t numStages, const float32_t *pCoeffs, float32_t *pState)

Initialization function for the floating-point FIR lattice filter.

Parameters

- **S** – [in] points to an instance of the floating-point FIR lattice structure
- **numStages** – [in] number of filter stages
- **pCoeffs** – [in] points to the coefficient buffer. The array is of length numStages
- **pState** – [in] points to the state buffer. The array is of length numStages

RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_q15 (riscv_fir_lattice_instance_q15 *S, uint16_t numStages, const q15_t *pCoeffs, q15_t *pState)

Initialization function for the Q15 FIR lattice filter.

Parameters

- **S** – [in] points to an instance of the Q15 FIR lattice structure
- **numStages** – [in] number of filter stages
- **pCoeffs** – [in] points to the coefficient buffer. The array is of length numStages
- **pState** – [in] points to the state buffer. The array is of length numStages

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_init_q31 (riscv_fir_lattice_instance_q31 *S,
uint16_t numStages, const q31_t *pCoeffs, q31_t *pState)
```

Initialization function for the Q31 FIR lattice filter.

Parameters

- **S** – [in] points to an instance of the Q31 FIR lattice structure
- **numStages** – [in] number of filter stages
- **pCoeffs** – [in] points to the coefficient buffer. The array is of length numStages
- **pState** – [in] points to the state buffer. The array is of length numStages

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_q15 (const riscv_fir_lattice_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

Processing function for Q15 FIR lattice filter.

Processing function for the Q15 FIR lattice filter.

Parameters

- **S** – [in] points to an instance of the Q15 FIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_fir_lattice_q31 (const riscv_fir_lattice_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

Processing function for the Q31 FIR lattice filter.

Scaling and Overflow Behavior

In order to avoid overflows the input signal must be scaled down by $2 \cdot \log_2(\text{numStages})$ bits.

Parameters

- **S** – [in] points to an instance of the Q31 FIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

Finite Impulse Response (FIR) Sparse Filters

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_f32 (riscv_fir_sparse_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, float32_t *pScratchIn, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_f32 (riscv_fir_sparse_instance_f32 *S,
uint16_t numTaps, const float32_t *pCoeffs, float32_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q15 (riscv_fir_sparse_instance_q15 *S,
uint16_t numTaps, const q15_t *pCoeffs, q15_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q31 (riscv_fir_sparse_instance_q31 *S,
uint16_t numTaps, const q31_t *pCoeffs, q31_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q7 (riscv_fir_sparse_instance_q7 *S,
uint16_t numTaps, const q7_t *pCoeffs, q7_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q15 (riscv_fir_sparse_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, q15_t *pScratchIn, q31_t *pScratchOut,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q31 (riscv_fir_sparse_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, q31_t *pScratchIn, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q7 (riscv_fir_sparse_instance_q7 *S,
const q7_t *pSrc, q7_t *pDst, q7_t *pScratchIn, q31_t *pScratchOut, uint32_t blockSize)
```

group **Finite Impulse Response (FIR) Sparse Filters**

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

This group of functions implements sparse FIR filters. Sparse FIR filters are equivalent to standard FIR filters except that most of the coefficients are equal to zero. Sparse filters are used for simulating reflections in communications and audio applications.

There are separate functions for Q7, Q15, Q31, and floating-point data types. The functions operate on blocks of input and output data and each call to the function processes `blockSize` samples through the filter. `pSrc` and `pDst` points to input and output arrays respectively containing `blockSize` values.

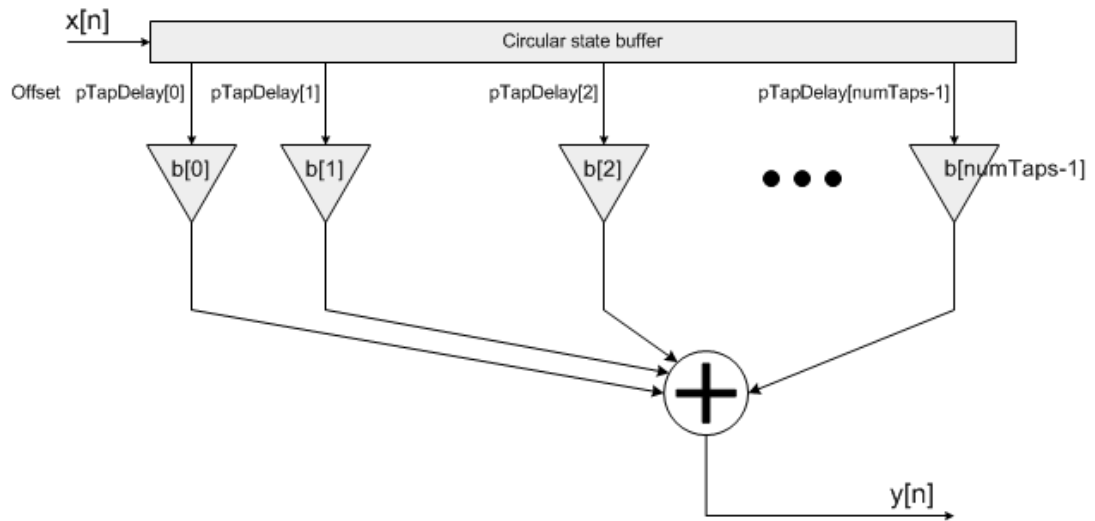
Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Algorithm

The sparse filter instant structure contains an array of tap indices `pTapDelay` which specifies the locations of the non-zero coefficients. This is in addition to the coefficient array `b`. The implementation essentially skips the multiplications by zero and leads to an efficient realization.

$$y[n] = b[0] * x[n-pTapDelay[0]] + b[1] * x[n-pTapDelay[1]] + b[2] * x[n-pTapDelay[2]] + \dots + b[numTaps-1] * x[n-pTapDelay[numTaps-1]]$$



`pCoeffs` points to a coefficient array of size `numTaps`; `pTapDelay` points to an array of nonzero indices and is also of size `numTaps`; `pState` points to a state array of size `maxDelay + blockSize`, where `maxDelay` is the largest offset value that is ever used in the `pTapDelay` array. Some of the processing functions also require temporary working buffers.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient and offset arrays may be shared among several instances while state variable arrays cannot be shared. There are separate instance structure declarations for each of the 4 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numTaps`, `pCoeffs`, `pTapDelay`, `maxDelay`, `stateIndex`, `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. The code below statically initializes each of the 4 different data type filter instance structures

```
riscv_fir_sparse_instance_f32 S = {numTaps, 0, pState, pCoeffs, maxDelay,
    ↪pTapDelay};
riscv_fir_sparse_instance_q31 S = {numTaps, 0, pState, pCoeffs, maxDelay,
    ↪pTapDelay};
riscv_fir_sparse_instance_q15 S = {numTaps, 0, pState, pCoeffs, maxDelay,
    ↪pTapDelay};
riscv_fir_sparse_instance_q7 S = {numTaps, 0, pState, pCoeffs, maxDelay,
    ↪pTapDelay};
```

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the sparse FIR filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Deprecated:

Those functions are no more tested nor maintained and will be removed in a future version.

Functions

RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_f32 (riscv_fir_sparse_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, float32_t *pScratchIn, uint32_t blockSize)

Processing function for the floating-point sparse FIR filter.

Parameters

- **S** – [in] points to an instance of the floating-point sparse FIR structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **pScratchIn** – [in] points to a temporary buffer of size blockSize
- **blockSize** – [in] number of input samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_f32 (riscv_fir_sparse_instance_f32 *S, uint16_t numTaps, const float32_t *pCoeffs, float32_t *pState, int32_t *pTapDelay, uint16_t maxDelay, uint32_t blockSize)

Initialization function for the floating-point sparse FIR filter.

Details

pCoeffs holds the filter coefficients and has length numTaps. pState holds the filter's state variables and must be of length maxDelay + blockSize, where maxDelay is the maximum number of delay line values. blockSize is the number of samples processed by the riscv_fir_sparse_f32() function.

Parameters

- **S** – [inout] points to an instance of the floating-point sparse FIR structure
- **numTaps** – [in] number of nonzero coefficients in the filter
- **pCoeffs** – [in] points to the array of filter coefficients
- **pState** – [in] points to the state buffer
- **pTapDelay** – [in] points to the array of offset times
- **maxDelay** – [in] maximum offset time supported
- **blockSize** – [in] number of samples that will be processed per block

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q15 (riscv_fir_sparse_instance_q15 *S,
uint16_t numTaps, const q15_t *pCoeffs, q15_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

Initialization function for the Q15 sparse FIR filter.

Details

pCoeffs holds the filter coefficients and has length numTaps. pState holds the filter's state variables and must be of length maxDelay + blockSize, where maxDelay is the maximum number of delay line values. blockSize is the number of words processed by riscv_fir_sparse_q15() function.

Parameters

- **S** – [inout] points to an instance of the Q15 sparse FIR structure
- **numTaps** – [in] number of nonzero coefficients in the filter
- **pCoeffs** – [in] points to the array of filter coefficients
- **pState** – [in] points to the state buffer
- **pTapDelay** – [in] points to the array of offset times
- **maxDelay** – [in] maximum offset time supported
- **blockSize** – [in] number of samples that will be processed per block

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q31 (riscv_fir_sparse_instance_q31 *S,
uint16_t numTaps, const q31_t *pCoeffs, q31_t *pState, int32_t *pTapDelay,
uint16_t maxDelay, uint32_t blockSize)
```

Initialization function for the Q31 sparse FIR filter.

Details

pCoeffs holds the filter coefficients and has length numTaps. pState holds the filter's state variables and must be of length maxDelay + blockSize, where maxDelay is the maximum number of delay line values. blockSize is the number of words processed by riscv_fir_sparse_q31() function.

Parameters

- **S** – [inout] points to an instance of the Q31 sparse FIR structure
- **numTaps** – [in] number of nonzero coefficients in the filter
- **pCoeffs** – [in] points to the array of filter coefficients

- **pState** – [in] points to the state buffer
- **pTapDelay** – [in] points to the array of offset times
- **maxDelay** – [in] maximum offset time supported
- **blockSize** – [in] number of samples that will be processed per block

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_init_q7 (riscv_fir_sparse_instance_q7 *S,  
uint16_t numTaps, const q7_t *pCoeffs, q7_t *pState, int32_t *pTapDelay,  
uint16_t maxDelay, uint32_t blockSize)
```

Initialization function for the Q7 sparse FIR filter.

Details

pCoeffs holds the filter coefficients and has length **numTaps**. **pState** holds the filter's state variables and must be of length **maxDelay + blockSize**, where **maxDelay** is the maximum number of delay line values. **blockSize** is the number of samples processed by the `riscv_fir_sparse_q7()` function.

Parameters

- **S** – [inout] points to an instance of the Q7 sparse FIR structure
- **numTaps** – [in] number of nonzero coefficients in the filter
- **pCoeffs** – [in] points to the array of filter coefficients
- **pState** – [in] points to the state buffer
- **pTapDelay** – [in] points to the array of offset times
- **maxDelay** – [in] maximum offset time supported
- **blockSize** – [in] number of samples that will be processed per block

```
RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q15 (riscv_fir_sparse_instance_q15 *S,  
const q15_t *pSrc, q15_t *pDst, q15_t *pScratchIn, q31_t *pScratchOut,  
uint32_t blockSize)
```

Processing function for the Q15 sparse FIR filter.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The 1.15 x 1.15 multiplications yield a 2.30 result and these are added to a 2.30 accumulator. Thus the full precision of the multiplications is maintained but there is only a single guard bit in the accumulator. If the accumulator result overflows it will wrap around rather than saturate. After all multiply-accumulates are performed, the 2.30 accumulator is truncated to 2.15 format and then saturated to 1.15 format. In order to avoid overflows the input signal or coefficients must be scaled down by $\log_2(\text{numTaps})$ bits.

Parameters

- **S** – [in] points to an instance of the Q15 sparse FIR structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **pScratchIn** – [in] points to a temporary buffer of size **blockSize**
- **pScratchOut** – [in] points to a temporary buffer of size **blockSize**

- **blockSize** – [in] number of input samples to process per call

RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q31 (riscv_fir_sparse_instance_q31 *S, const q31_t *pSrc, q31_t *pDst, q31_t *pScratchIn, uint32_t blockSize)

Processing function for the Q31 sparse FIR filter.

Scaling and Overflow Behavior

The function is implemented using an internal 32-bit accumulator. The 1.31 x 1.31 multiplications are truncated to 2.30 format. This leads to loss of precision on the intermediate multiplications and provides only a single guard bit. If the accumulator result overflows, it wraps around rather than saturate. In order to avoid overflows the input signal or coefficients must be scaled down by $\log_2(\text{numTaps})$ bits.

Parameters

- **S** – [in] points to an instance of the Q31 sparse FIR structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **pScratchIn** – [in] points to a temporary buffer of size blockSize
- **blockSize** – [in] number of input samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_sparse_q7 (riscv_fir_sparse_instance_q7 *S, const q7_t *pSrc, q7_t *pDst, q7_t *pScratchIn, q31_t *pScratchOut, uint32_t blockSize)

Processing function for the Q7 sparse FIR filter.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. Both coefficients and state variables are represented in 1.7 format and multiplications yield a 2.14 result. The 2.14 intermediate results are accumulated in a 32-bit accumulator in 18.14 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. The accumulator is then converted to 18.7 format by discarding the low 7 bits. Finally, the result is truncated to 1.7 format.

Parameters

- **S** – [in] points to an instance of the Q7 sparse FIR structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **pScratchIn** – [in] points to a temporary buffer of size blockSize
- **pScratchOut** – [in] points to a temporary buffer of size blockSize
- **blockSize** – [in] number of input samples to process

Infinite Impulse Response (IIR) Lattice Filters

```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_f32 (const riscv_iir_lattice_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_f32 (riscv_iir_lattice_instance_f32 *S,
uint16_t numStages, float32_t *pkCoeffs, float32_t *pvCoeffs, float32_t *pState,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_q15 (riscv_iir_lattice_instance_q15 *S,
uint16_t numStages, q15_t *pkCoeffs, q15_t *pvCoeffs, q15_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_q31 (riscv_iir_lattice_instance_q31 *S,
uint16_t numStages, q31_t *pkCoeffs, q31_t *pvCoeffs, q31_t *pState, uint32_t blockSize)
```

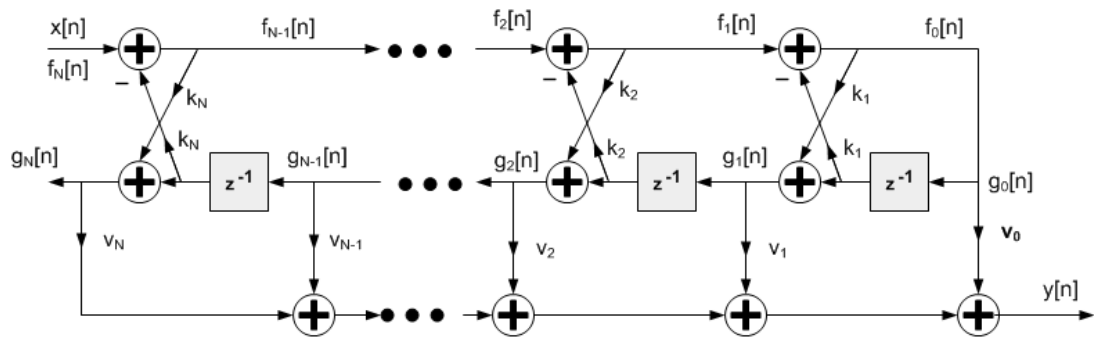
```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_q15 (const riscv_iir_lattice_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_q31 (const riscv_iir_lattice_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

group Infinite Impulse Response (IIR) Lattice Filters

This set of functions implements lattice filters for Q15, Q31 and floating-point data types. Lattice filters are used in a variety of adaptive filter applications. The filter structure has feedforward and feedback components and the net impulse response is infinite length. The functions operate on blocks of input and output data and each call to the function processes blockSize samples through the filter. pSrc and pDst point to input and output arrays containing blockSize values.

Algorithm



```
fN(n)   = x(n)
fm-1(n) = fm(n) - km * gm-1(n-1)   for m = N, N-1, ..., 1
gm(n)   = km * fm-1(n) + gm-1(n-1) for m = N, N-1, ..., 1
y(n)    = vN * gN(n) + vN-1 * gN-1(n) + ... + v0 * g0(n)
```

`pkCoeffs` points to array of reflection coefficients of size `numStages`. Reflection Coefficients are stored in time-reversed order.

```
{kN, kN-1, ..., k1}
```

`pvCoeffs` points to the array of ladder coefficients of size `(numStages+1)`. Ladder coefficients are stored in time-reversed order.

```
{vN, vN-1, ..., v0}
```

`pState` points to a state array of size `numStages + blockSize`. The state variables shown in the figure above (the `g` values) are stored in the `pState` array. The state variables are updated after each block of data is processed; the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable arrays cannot be shared. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numStages`, `pkCoeffs`, `pvCoeffs`, `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros and then manually initialize the instance structure as follows:

```
riscv_iir_lattice_instance_f32 S = {numStages, pState, pkCoeffs, pvCoeffs};
riscv_iir_lattice_instance_q31 S = {numStages, pState, pkCoeffs, pvCoeffs};
riscv_iir_lattice_instance_q15 S = {numStages, pState, pkCoeffs, pvCoeffs};
```

where `numStages` is the number of stages in the filter; `pState` points to the state buffer array; `pkCoeffs` points to array of the reflection coefficients; `pvCoeffs` points to the array of ladder coefficients.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the IIR lattice filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_f32 (const riscv_iir_lattice_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point IIR lattice filter.

Parameters

- **S** – [in] points to an instance of the floating-point IIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_f32 (riscv_iir_lattice_instance_f32 *S, uint16_t numStages, float32_t *pkCoeffs, float32_t *pvCoeffs, float32_t *pState, uint32_t blockSize)

Initialization function for the floating-point IIR lattice filter.

Parameters

- **S** – [in] points to an instance of the floating-point IIR lattice structure
- **numStages** – [in] number of stages in the filter
- **pkCoeffs** – [in] points to reflection coefficient buffer. The array is of length numStages
- **pvCoeffs** – [in] points to ladder coefficient buffer. The array is of length numStages+1
- **pState** – [in] points to state buffer. The array is of length numStages+blockSize
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_q15 (riscv_iir_lattice_instance_q15 *S, uint16_t numStages, q15_t *pkCoeffs, q15_t *pvCoeffs, q15_t *pState, uint32_t blockSize)

Initialization function for the Q15 IIR lattice filter.

Parameters

- **S** – [in] points to an instance of the Q15 IIR lattice structure
- **numStages** – [in] number of stages in the filter
- **pkCoeffs** – [in] points to reflection coefficient buffer. The array is of length numStages
- **pvCoeffs** – [in] points to ladder coefficient buffer. The array is of length numStages+1
- **pState** – [in] points to state buffer. The array is of length numStages+blockSize
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_init_q31 (riscv_iir_lattice_instance_q31 *S, uint16_t numStages, q31_t *pkCoeffs, q31_t *pvCoeffs, q31_t *pState, uint32_t blockSize)

Initialization function for the Q31 IIR lattice filter.

Parameters

- **S** – [in] points to an instance of the Q31 IIR lattice structure

- **numStages** – [in] number of stages in the filter
- **pkCoeffs** – [in] points to reflection coefficient buffer. The array is of length numStages
- **pvCoeffs** – [in] points to ladder coefficient buffer. The array is of length numStages+1
- **pState** – [in] points to state buffer. The array is of length numStages+blockSize
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_q15 (const riscv_iir_lattice_instance_q15 *S, const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Processing function for the Q15 IIR lattice filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

Parameters

- **S** – [in] points to an instance of the Q15 IIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

RISCV_DSP_ATTRIBUTE void riscv_iir_lattice_q31 (const riscv_iir_lattice_instance_q31 *S, const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Processing function for the Q31 IIR lattice filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by $2 \cdot \log_2(\text{numStages})$ bits. After all multiply-accumulates are performed, the 2.62 accumulator is saturated to 1.32 format and then truncated to 1.31 format.

Parameters

- **S** – [in] points to an instance of the Q31 IIR lattice structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process

Levinson Durbin Algorithm

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_f16 (const float16_t *phi, float16_t *a,
float16_t *err, int nbCoefs)
```

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_f32 (const float32_t *phi, float32_t *a,
float32_t *err, int nbCoefs)
```

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_q31 (const q31_t *phi, q31_t *a,
q31_t *err, int nbCoefs)
```

group Levinson Durbin Algorithm

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_f16 (const float16_t *phi,
float16_t *a, float16_t *err, int nbCoefs)
```

Levinson Durbin.

Parameters

- **phi** – [in] autocovariance vector starting with lag 0 (length is nbCoefs + 1)
- **a** – [out] autoregressive coefficients
- **err** – [out] prediction error (variance)
- **nbCoefs** – [in] number of autoregressive coefficients

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_f32 (const float32_t *phi,
float32_t *a, float32_t *err, int nbCoefs)
```

Levinson Durbin.

Parameters

- **phi** – [in] autocovariance vector starting with lag 0 (length is nbCoefs + 1)
- **a** – [out] autoregressive coefficients
- **err** – [out] prediction error (variance)
- **nbCoefs** – [in] number of autoregressive coefficients

```
RISCV_DSP_ATTRIBUTE void riscv_levinson_durbin_q31 (const q31_t *phi, q31_t *a,
q31_t *err, int nbCoefs)
```

Levinson Durbin.

Parameters

- **phi** – [in] autocovariance vector starting with lag 0 (length is nbCoefs + 1)
- **a** – [out] autoregressive coefficients
- **err** – [out] prediction error (variance)
- **nbCoefs** – [in] number of autoregressive coefficients

Least Mean Square (LMS) Filters

```
RISCV_DSP_ATTRIBUTE void riscv_lms_f32 (const riscv_lms_instance_f32 *S,
const float32_t *pSrc, float32_t *pRef, float32_t *pOut, float32_t *pErr,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_f32 (riscv_lms_instance_f32 *S, uint16_t numTaps,
float32_t *pCoeffs, float32_t *pState, float32_t mu, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_q15 (riscv_lms_instance_q15 *S, uint16_t numTaps,
q15_t *pCoeffs, q15_t *pState, q15_t mu, uint32_t blockSize, uint32_t postShift)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_q31 (riscv_lms_instance_q31 *S, uint16_t numTaps,
q31_t *pCoeffs, q31_t *pState, q31_t mu, uint32_t blockSize, uint32_t postShift)
```

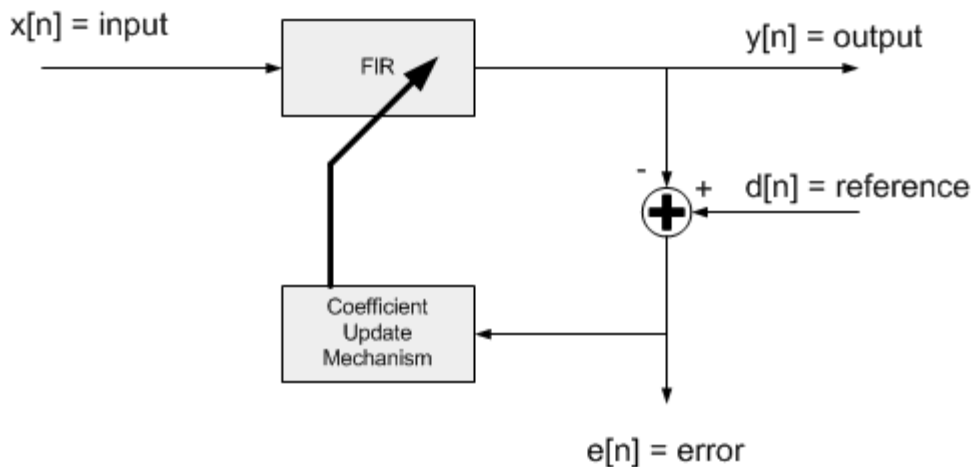
```
RISCV_DSP_ATTRIBUTE void riscv_lms_q15 (const riscv_lms_instance_q15 *S,
const q15_t *pSrc, q15_t *pRef, q15_t *pOut, q15_t *pErr, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_q31 (const riscv_lms_instance_q31 *S,
const q31_t *pSrc, q31_t *pRef, q31_t *pOut, q31_t *pErr, uint32_t blockSize)
```

group Least Mean Square (LMS) Filters

LMS filters are a class of adaptive filters that are able to “learn” an unknown transfer functions. LMS filters use a gradient descent method in which the filter coefficients are updated based on the instantaneous error signal. Adaptive filters are often used in communication systems, equalizers, and noise removal. The NMSIS DSP Library contains LMS filter functions that operate on Q15, Q31, and floating-point data types. The library also contains normalized LMS filters in which the filter coefficient adaptation is independent of the level of the input signal.

An LMS filter consists of two components as shown below. The first component is a standard transversal or FIR filter. The second component is a coefficient update mechanism. The LMS filter has two input signals. The “input” feeds the FIR filter while the “reference input” corresponds to the desired output of the FIR filter. That is, the FIR filter coefficients are updated so that the output of the FIR filter matches the reference input. The filter coefficient update mechanism is based on the difference between the FIR filter output and the reference input. This “error signal” tends towards zero as the filter adapts. The LMS processing functions accept the input and reference input signals and generate the filter output and error signal.



The functions operate on blocks of data and each call to the function processes `blockSize` samples through the filter. `pSrc` points to input signal, `pRef` points to reference signal, `pOut` points to output signal and `pErr` points to error signal. All arrays contain `blockSize` values.

The functions operate on a block-by-block basis. Internally, the filter coefficients `b[n]` are updated on a sample-by-sample basis. The convergence of the LMS filter is slower compared to the normalized LMS algorithm.

Algorithm

The output signal `y[n]` is computed by a standard FIR filter:

$$y[n] = b[0] * x[n] + b[1] * x[n-1] + b[2] * x[n-2] + \dots + b[\text{numTaps}-1] * x[n-\text{numTaps}+1]$$

The error signal equals the difference between the reference signal `d[n]` and the filter output:

$$e[n] = d[n] - y[n].$$

After each sample of the error signal is computed, the filter coefficients `b[k]` are updated on a sample-by-sample basis:

$$b[k] = b[k] + e[n] * \mu * x[n-k], \text{ for } k=0, 1, \dots, \text{numTaps}-1$$

where `mu` is the step size and controls the rate of coefficient convergence.

In the APIs, `pCoeffs` points to a coefficient array of size `numTaps`. Coefficients are stored in time reversed order.

$$\{b[\text{numTaps}-1], b[\text{numTaps}-2], b[\text{N}-2], \dots, b[1], b[0]\}$$

`pState` points to a state array of size `numTaps + blockSize - 1`. Samples in the state buffer are stored in the order:


```
{x[n-numTaps+1], x[n-numTaps], x[n-numTaps-1], x[n-numTaps-2]...x[0], x[1], ...  
↪, x[blockSize-1]}
```

Note that the length of the state buffer exceeds the length of the coefficient array by `blockSize-1` samples. The increased state buffer length allows circular addressing, which is traditionally used in FIR filters, to be avoided and yields a significant speed improvement. The state variables are updated after each block of data is processed.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter and coefficient and state arrays cannot be shared among instances. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: `numTaps`, `pCoeffs`, `mu`, `postShift` (not for f32), `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Set the values in the state buffer to zeros before static initialization. The code below statically initializes each of the 3 different data type filter instance structures

```
riscv_lms_instance_f32 S = {numTaps, pState, pCoeffs, mu};  
riscv_lms_instance_q31 S = {numTaps, pState, pCoeffs, mu, postShift};  
riscv_lms_instance_q15 S = {numTaps, pState, pCoeffs, mu, postShift};
```

where `numTaps` is the number of filter coefficients in the filter; `pState` is the address of the state buffer; `pCoeffs` is the address of the coefficient buffer; `mu` is the step size parameter; and `postShift` is the shift applied to coefficients.

Fixed-Point Behavior

Care must be taken when using the Q15 and Q31 versions of the LMS filter. The following issues must be considered:

- Scaling of coefficients
- Overflow and saturation

Scaling of Coefficients

Filter coefficients are represented as fractional values and coefficients are restricted to lie in the range $[-1, +1]$. The fixed-point functions have an additional scaling parameter `postShift`. At the output of the filter's accumulator is a shift register which shifts the result by `postShift` bits. This essentially scales the filter coefficients by $2^{\text{postShift}}$ and allows the filter coefficients to exceed the range $[-1, +1]$. The value of `postShift` is set by the user based on the expected gain through the system being modeled.

Overflow and Saturation

Overflow and saturation behavior of the fixed-point Q15 and Q31 versions are described separately as part of the function specific documentation below.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_lms_f32 (const riscv_lms_instance_f32 *S,  
const float32_t *pSrc, float32_t *pRef, float32_t *pOut, float32_t *pErr,  
uint32_t blockSize)
```

Processing function for floating-point LMS filter.

Parameters

- **S** – [in] points to an instance of the floating-point LMS filter structure
- **pSrc** – [in] points to the block of input data
- **pRef** – [in] points to the block of reference data
- **pOut** – [out] points to the block of output data
- **pErr** – [out] points to the block of error data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_f32 (riscv_lms_instance_f32 *S,  
uint16_t numTaps, float32_t *pCoeffs, float32_t *pState, float32_t mu,  
uint32_t blockSize)
```

Initialization function for floating-point LMS filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

`{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}`

The initial filter coefficients serve as a starting point for the adaptive filter. pState points to an array of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_lms_f32().

Parameters

- **S** – [in] points to an instance of the floating-point LMS filter structure
- **numTaps** – [in] number of filter coefficients
- **pCoeffs** – [in] points to coefficient buffer
- **pState** – [in] points to state buffer
- **mu** – [in] step size that controls filter coefficient updates
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_q15 (riscv_lms_instance_q15 *S,  
uint16_t numTaps, q15_t *pCoeffs, q15_t *pState, q15_t mu, uint32_t blockSize,  
uint32_t postShift)
```

Initialization function for the Q15 LMS filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

The initial filter coefficients serve as a starting point for the adaptive filter. `pState` points to the array of state variables and size of array is `numTaps+blockSize-1` samples, where `blockSize` is the number of input samples processed by each call to `riscv_lms_q15()`.

Parameters

- **S** – [in] points to an instance of the Q15 LMS filter structure.
- **numTaps** – [in] number of filter coefficients.
- **pCoeffs** – [in] points to coefficient buffer.
- **pState** – [in] points to state buffer.
- **mu** – [in] step size that controls filter coefficient updates.
- **blockSize** – [in] number of samples to process.
- **postShift** – [in] bit shift applied to coefficients.

```
RISCV_DSP_ATTRIBUTE void riscv_lms_init_q31 (riscv_lms_instance_q31 *S,
uint16_t numTaps, q31_t *pCoeffs, q31_t *pState, q31_t mu, uint32_t blockSize,
uint32_t postShift)
```

Initialization function for Q31 LMS filter.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

The initial filter coefficients serve as a starting point for the adaptive filter. `pState` points to an array of length `numTaps+blockSize-1` samples, where `blockSize` is the number of input samples processed by each call to `riscv_lms_q31()`.

Parameters

- **S** – [in] points to an instance of the Q31 LMS filter structure
- **numTaps** – [in] number of filter coefficients
- **pCoeffs** – [in] points to coefficient buffer
- **pState** – [in] points to state buffer
- **mu** – [in] step size that controls filter coefficient updates
- **blockSize** – [in] number of samples to process
- **postShift** – [in] bit shift applied to coefficients

```
RISCV_DSP_ATTRIBUTE void riscv_lms_q15 (const riscv_lms_instance_q15 *S,
const q15_t *pSrc, q15_t *pRef, q15_t *pOut, q15_t *pErr, uint32_t blockSize)
```

Processing function for Q15 LMS filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

In this filter, filter coefficients are updated for each sample and the updation of filter coefficients are saturated.

Parameters

- **S** – [in] points to an instance of the Q15 LMS filter structure
- **pSrc** – [in] points to the block of input data
- **pRef** – [in] points to the block of reference data
- **pOut** – [out] points to the block of output data
- **pErr** – [out] points to the block of error data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_q31 (const riscv_lms_instance_q31 *S,  
const q31_t *pSrc, q31_t *pRef, q31_t *pOut, q31_t *pErr, uint32_t blockSize)
```

Processing function for Q31 LMS filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clips. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits. The reference signal should not be scaled down. After all multiply-accumulates are performed, the 2.62 accumulator is shifted and saturated to 1.31 format to yield the final result. The output signal and error signal are in 1.31 format.

In this filter, filter coefficients are updated for each sample and the updation of filter coefficients are saturated.

Parameters

- **S** – [in] points to an instance of the Q31 LMS filter structure.
- **pSrc** – [in] points to the block of input data.
- **pRef** – [in] points to the block of reference data.
- **pOut** – [out] points to the block of output data.
- **pErr** – [out] points to the block of error data.
- **blockSize** – [in] number of samples to process.

Normalized LMS Filters

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_f32 (riscv_lms_norm_instance_f32 *S,
const float32_t *pSrc, float32_t *pRef, float32_t *pOut, float32_t *pErr,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_f32 (riscv_lms_norm_instance_f32 *S,
uint16_t numTaps, float32_t *pCoeffs, float32_t *pState, float32_t mu,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_q15 (riscv_lms_norm_instance_q15 *S,
uint16_t numTaps, q15_t *pCoeffs, q15_t *pState, q15_t mu, uint32_t blockSize,
uint8_t postShift)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_q31 (riscv_lms_norm_instance_q31 *S,
uint16_t numTaps, q31_t *pCoeffs, q31_t *pState, q31_t mu, uint32_t blockSize,
uint8_t postShift)
```

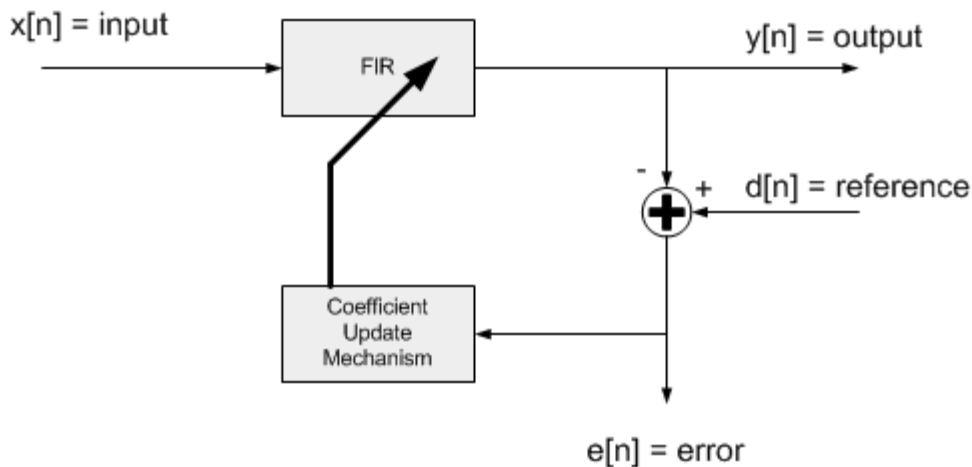
```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_q15 (riscv_lms_norm_instance_q15 *S,
const q15_t *pSrc, q15_t *pRef, q15_t *pOut, q15_t *pErr, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_q31 (riscv_lms_norm_instance_q31 *S,
const q31_t *pSrc, q31_t *pRef, q31_t *pOut, q31_t *pErr, uint32_t blockSize)
```

group Normalized LMS Filters

This set of functions implements a commonly used adaptive filter. It is related to the Least Mean Square (LMS) adaptive filter and includes an additional normalization factor which increases the adaptation rate of the filter. The NMSIS DSP Library contains normalized LMS filter functions that operate on Q15, Q31, and floating-point data types.

A normalized least mean square (NLMS) filter consists of two components as shown below. The first component is a standard transversal or FIR filter. The second component is a coefficient update mechanism. The NLMS filter has two input signals. The “input” feeds the FIR filter while the “reference input” corresponds to the desired output of the FIR filter. That is, the FIR filter coefficients are updated so that the output of the FIR filter matches the reference input. The filter coefficient update mechanism is based on the difference between the FIR filter output and the reference input. This “error signal” tends towards zero as the filter adapts. The NLMS processing functions accept the input and reference input signals and generate the filter output and error signal.



The functions operate on blocks of data and each call to the function processes `blockSize` samples through the filter. `pSrc` points to input signal, `pRef` points to reference signal, `pOut` points to output signal and `pErr` points to error signal. All arrays contain `blockSize` values.

The functions operate on a block-by-block basis. Internally, the filter coefficients `b[n]` are updated on a sample-by-sample basis. The convergence of the LMS filter is slower compared to the normalized LMS algorithm.

Algorithm

The output signal $y[n]$ is computed by a standard FIR filter:

$$y[n] = b[0] * x[n] + b[1] * x[n-1] + b[2] * x[n-2] + \dots + b[\text{numTaps}-1] * x[n-\text{numTaps}+1]$$

The error signal equals the difference between the reference signal $d[n]$ and the filter output:

$$e[n] = d[n] - y[n].$$

After each sample of the error signal is computed the instantaneous energy of the filter state variables is calculated:

$$E = x[n]^2 + x[n-1]^2 + \dots + x[n-\text{numTaps}+1]^2.$$

The filter coefficients $b[k]$ are then updated on a sample-by-sample basis:

$$b[k] = b[k] + e[n] * (\mu/E) * x[n-k], \text{ for } k=0, 1, \dots, \text{numTaps}-1$$

where μ is the step size and controls the rate of coefficient convergence.

In the APIs, `pCoeffs` points to a coefficient array of size `numTaps`. Coefficients are stored in time reversed order.

$$\{b[\text{numTaps}-1], b[\text{numTaps}-2], b[\text{numTaps}-3], \dots, b[1], b[0]\}$$

pState points to a state array of size numTaps + blockSize - 1. Samples in the state buffer are stored in the order:

```
{x[n-numTaps+1], x[n-numTaps], x[n-numTaps-1], x[n-numTaps-2]...x[0], x[1], ...  
↔, x[blockSize-1]}
```

Note that the length of the state buffer exceeds the length of the coefficient array by blockSize-1 samples. The increased state buffer length allows circular addressing, which is traditionally used in FIR filters, to be avoided and yields a significant speed improvement. The state variables are updated after each block of data is processed.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter and coefficient and state arrays cannot be shared among instances. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer. To do this manually without calling the init function, assign the follow subfields of the instance structure: numTaps, pCoeffs, mu, energy, x0, pState. Also set all of the values in pState to zero. For Q7, Q15, and Q31 the following fields must also be initialized; recipTable, postShift

Instance structure cannot be placed into a const data section and it is recommended to use the initialization function.

Fixed-Point Behavior

Care must be taken when using the Q15 and Q31 versions of the normalised LMS filter. The following issues must be considered:

- Scaling of coefficients
- Overflow and saturation

Scaling of Coefficients (fixed point versions)

Filter coefficients are represented as fractional values and coefficients are restricted to lie in the range [-1 +1). The fixed-point functions have an additional scaling parameter postShift. At the output of the filter's accumulator is a shift register which shifts the result by postShift bits. This essentially scales the filter coefficients by $2^{\text{postShift}}$ and allows the filter coefficients to exceed the range [+1 -1). The value of postShift is set by the user based on the expected gain through the system being modeled.

Overflow and Saturation (fixed point versions)

Overflow and saturation behavior of the fixed-point Q15 and Q31 versions are described separately as part of the function specific documentation below.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_f32 (riscv_lms_norm_instance_f32 *S,  
const float32_t *pSrc, float32_t *pRef, float32_t *pOut, float32_t *pErr,  
uint32_t blockSize)
```

Processing function for floating-point normalized LMS filter.

Parameters

- **S** – [in] points to an instance of the floating-point normalized LMS filter structure
- **pSrc** – [in] points to the block of input data
- **pRef** – [in] points to the block of reference data
- **pOut** – [out] points to the block of output data
- **pErr** – [out] points to the block of error data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_f32 (riscv_lms_norm_instance_f32 *S,  
uint16_t numTaps, float32_t *pCoeffs, float32_t *pState, float32_t mu,  
uint32_t blockSize)
```

Initialization function for floating-point normalized LMS filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:

`{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}`

The initial filter coefficients serve as a starting point for the adaptive filter. pState points to an array of length numTaps+blockSize-1 samples, where blockSize is the number of input samples processed by each call to riscv_lms_norm_f32().

Parameters

- **S** – [in] points to an instance of the floating-point LMS filter structure
- **numTaps** – [in] number of filter coefficients
- **pCoeffs** – [in] points to coefficient buffer
- **pState** – [in] points to state buffer
- **mu** – [in] step size that controls filter coefficient updates
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_q15 (riscv_lms_norm_instance_q15 *S,  
uint16_t numTaps, q15_t *pCoeffs, q15_t *pState, q15_t mu, uint32_t blockSize,  
uint8_t postShift)
```

Initialization function for Q15 normalized LMS filter.

Details

pCoeffs points to the array of filter coefficients stored in time reversed order:


```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

The initial filter coefficients serve as a starting point for the adaptive filter. `pState` points to the array of state variables and size of array is `numTaps+blockSize-1` samples, where `blockSize` is the number of input samples processed by each call to `riscv_lms_norm_q15()`.

Parameters

- **S** – [in] points to an instance of the Q15 normalized LMS filter structure.
- **numTaps** – [in] number of filter coefficients.
- **pCoeffs** – [in] points to coefficient buffer.
- **pState** – [in] points to state buffer.
- **mu** – [in] step size that controls filter coefficient updates.
- **blockSize** – [in] number of samples to process.
- **postShift** – [in] bit shift applied to coefficients.

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_init_q31 (riscv_lms_norm_instance_q31 *S,
uint16_t numTaps, q31_t *pCoeffs, q31_t *pState, q31_t mu, uint32_t blockSize,
uint8_t postShift)
```

Initialization function for Q31 normalized LMS filter.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

The initial filter coefficients serve as a starting point for the adaptive filter. `pState` points to an array of length `numTaps+blockSize-1` samples, where `blockSize` is the number of input samples processed by each call to `riscv_lms_norm_q31()`.

Parameters

- **S** – [in] points to an instance of the Q31 normalized LMS filter structure.
- **numTaps** – [in] number of filter coefficients.
- **pCoeffs** – [in] points to coefficient buffer.
- **pState** – [in] points to state buffer.
- **mu** – [in] step size that controls filter coefficient updates.
- **blockSize** – [in] number of samples to process.
- **postShift** – [in] bit shift applied to coefficients.

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_q15 (riscv_lms_norm_instance_q15 *S,
const q15_t *pSrc, q15_t *pRef, q15_t *pOut, q15_t *pErr, uint32_t blockSize)
```

Processing function for Q15 normalized LMS filter.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

In this filter, filter coefficients are updated for each sample and the updation of filter coefficients are saturated.

Parameters

- **S** – [in] points to an instance of the Q15 normalized LMS filter structure
- **pSrc** – [in] points to the block of input data
- **pRef** – [in] points to the block of reference data
- **pOut** – [out] points to the block of output data
- **pErr** – [out] points to the block of error data
- **blockSize** – [in] number of samples to process

```
RISCV_DSP_ATTRIBUTE void riscv_lms_norm_q31 (riscv_lms_norm_instance_q31 *S,  
const q31_t *pSrc, q31_t *pRef, q31_t *pOut, q31_t *pErr, uint32_t blockSize)
```

Processing function for Q31 normalized LMS filter.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{numTaps})$ bits. The reference signal should not be scaled down. After all multiply-accumulates are performed, the 2.62 accumulator is shifted and saturated to 1.31 format to yield the final result. The output signal and error signal are in 1.31 format.

In this filter, filter coefficients are updated for each sample and the updation of filter coefficients are saturated.

Parameters

- **S** – [in] points to an instance of the Q31 normalized LMS filter structure
- **pSrc** – [in] points to the block of input data
- **pRef** – [in] points to the block of reference data
- **pOut** – [out] points to the block of output data
- **pErr** – [out] points to the block of error data
- **blockSize** – [in] number of samples to process

Finite Impulse Response (FIR) Interpolator

```
RISCV_DSP_ATTRIBUTE void riscv_fir_interpolate_f32 (const riscv_fir_interpolate_instance_f32 *S,
const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_interpolate_init_f32 (riscv_fir_interpolate_instance_f32 *S,
uint8_t L, uint16_t numTaps, const float32_t *pCoeffs, float32_t *pState,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_interpolate_init_q15 (riscv_fir_interpolate_instance_q15 *S,
uint8_t L, uint16_t numTaps, const q15_t *pCoeffs, q15_t *pState, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_interpolate_init_q31 (riscv_fir_interpolate_instance_q31 *S,
uint8_t L, uint16_t numTaps, const q31_t *pCoeffs, q31_t *pState, uint32_t blockSize)
```

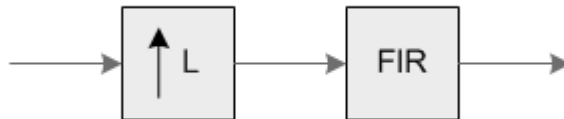
```
RISCV_DSP_ATTRIBUTE void riscv_fir_interpolate_q15 (const riscv_fir_interpolate_instance_q15 *S,
const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fir_interpolate_q31 (const riscv_fir_interpolate_instance_q31 *S,
const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)
```

group Finite Impulse Response (FIR) Interpolator

These functions combine an upsampler (zero stuffer) and an FIR filter. They are used in multirate systems for increasing the sample rate of a signal without introducing high frequency images. Conceptually, the functions are equivalent to the block diagram below:

After upsampling by a factor of L , the signal should be filtered by a lowpass filter with a normalized cutoff frequency of $1/L$ in order to eliminate high frequency copies of the spectrum. The user of the function is



responsible for providing the filter coefficients.

The FIR interpolator functions provided in the NMSIS DSP Library combine the upsampler and FIR filter in an efficient manner. The upsampler inserts $L-1$ zeros between each sample. Instead of multiplying by these zero values, the FIR filter is designed to skip them. This leads to an efficient implementation without any wasted effort. The functions operate on blocks of input and output data. `pSrc` points to an array of `blockSize` input values and `pDst` points to an array of `blockSize*L` output values.

The library provides separate functions for Q15, Q31, and floating-point data types.

Algorithm

The functions use a polyphase filter structure:

```

y[n] = b[0] * x[n] + b[L] * x[n-1] + ... + b[L*(phaseLength-1)] * x[n-
↪ phaseLength+1]
y[n+1] = b[1] * x[n] + b[L+1] * x[n-1] + ... + b[L*(phaseLength-1)+1] * x[n-
↪ phaseLength+1]
  
```

(continues on next page)

(continued from previous page)

```

...
y[n+(L-1)] = b[L-1] * x[n] + b[2*L-1] * x[n-1] + ... + b[L*(phaseLength-1)+(L-
↪1)] * x[n-phaseLength+1]

```

This approach is more efficient than straightforward upsample-then-filter algorithms. With this method the computation is reduced by a factor of $1/L$ when compared to using a standard FIR filter.

`pCoeffs` points to a coefficient array of size `numTaps`. `numTaps` must be a multiple of the interpolation factor `L` and this is checked by the initialization functions. Internally, the function divides the FIR filter's impulse response into shorter filters of length `phaseLength=numTaps/L`. Coefficients are stored in time reversed order.

```
{b[numTaps-1], b[numTaps-2], b[N-2], ..., b[1], b[0]}
```

`pState` points to a state array of size `blockSize + phaseLength - 1`. Samples in the state buffer are stored in the order:

```
{x[n-phaseLength+1], x[n-phaseLength], x[n-phaseLength-1], x[n-phaseLength-2]...
↪.x[0], x[1], ..., x[blockSize-1]}
```

The state variables are updated after each block of data is processed, the coefficients are untouched.

Instance Structure

The coefficients and state variables for a filter are stored together in an instance data structure. A separate instance structure must be defined for each filter. Coefficient arrays may be shared among several instances while state variable array should be allocated separately. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Zeros out the values in the state buffer.
- Checks to make sure that the length of the filter is a multiple of the interpolation factor. To do this manually without calling the init function, assign the follow subfields of the instance structure: `L` (interpolation factor), `pCoeffs`, `phaseLength` (`numTaps / L`), `pState`. Also set all of the values in `pState` to zero.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. The code below statically initializes each of the 3 different data type filter instance structures

```
riscv_fir_interpolate_instance_f32 S = {L, phaseLength, pCoeffs, pState};
riscv_fir_interpolate_instance_q31 S = {L, phaseLength, pCoeffs, pState};
riscv_fir_interpolate_instance_q15 S = {L, phaseLength, pCoeffs, pState};
```

where `L` is the interpolation factor; `phaseLength=numTaps/L` is the length of each of the shorter FIR

filters used internally, `pCoeffs` is the address of the coefficient buffer; `pState` is the address of the state buffer. Be sure to set the values in the state buffer to zeros when doing static initialization.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the FIR interpolate filter functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

Functions

RISCV_DSP_ATTRIBUTE `void riscv_fir_interpolate_f32 (const riscv_fir_interpolate_instance_f32 *S, const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)`

Processing function for floating-point FIR interpolator.

Processing function for the floating-point FIR interpolator.

Parameters

- **S** – [in] points to an instance of the floating-point FIR interpolator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

RISCV_DSP_ATTRIBUTE `riscv_status riscv_fir_interpolate_init_f32 (riscv_fir_interpolate_instance_f32 *S, uint8_t L, uint16_t numTaps, const float32_t *pCoeffs, float32_t *pState, uint32_t blockSize)`

Initialization function for the floating-point FIR interpolator.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

```
{b[numTaps-1], b[numTaps-2], b[numTaps-2], ..., b[1], b[0]}
```

The length of the filter `numTaps` must be a multiple of the interpolation factor `L`.

`pState` points to the array of state variables. `pState` is of length $(\text{numTaps}/L) + \text{blockSize} - 1$ words where `blockSize` is the number of input samples processed by each call to `riscv_fir_interpolate_f32()`.

Parameters

- **S** – [inout] points to an instance of the floating-point FIR interpolator structure
- **L** – [in] upsample factor
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficient buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process per call

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : filter length `numTaps` is not a multiple of the interpolation factor `L`

RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_interpolate_init_q15 (**riscv_fir_interpolate_instance_q15** `riscv_fir_interpolate_instance_q15`, `uint8_t` `L`, `uint16_t` `numTaps`, `const` `q15_t` `*pCoeffs`, `q15_t` `*pState`, `uint32_t` `blockSize`)

Initialization function for the Q15 FIR interpolator.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

`{b[numTaps-1], b[numTaps-2], b[numTaps-2], ..., b[1], b[0]}`

The length of the filter `numTaps` must be a multiple of the interpolation factor `L`.

`pState` points to the array of state variables. `pState` is of length $(\text{numTaps}/L) + \text{blockSize} - 1$ words where `blockSize` is the number of input samples processed by each call to `riscv_fir_interpolate_q15()`.

Parameters

- **S** – **[inout]** points to an instance of the Q15 FIR interpolator structure
- **L** – **[in]** upsample factor
- **numTaps** – **[in]** number of filter coefficients in the filter
- **pCoeffs** – **[in]** points to the filter coefficient buffer
- **pState** – **[in]** points to the state buffer
- **blockSize** – **[in]** number of input samples to process per call

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : filter length `numTaps` is not a multiple of the interpolation factor `L`

RISCV_DSP_ATTRIBUTE riscv_status riscv_fir_interpolate_init_q31 (**riscv_fir_interpolate_instance_q31** `riscv_fir_interpolate_instance_q31`, `uint8_t` `L`, `uint16_t` `numTaps`, `const` `q31_t` `*pCoeffs`, `q31_t` `*pState`, `uint32_t` `blockSize`)

Initialization function for the Q31 FIR interpolator.

Details

`pCoeffs` points to the array of filter coefficients stored in time reversed order:

`{b[numTaps-1], b[numTaps-2], b[numTaps-2], ..., b[1], b[0]}`

The length of the filter `numTaps` must be a multiple of the interpolation factor `L`.

pState points to the array of state variables. pState is of length (numTaps/L)+blockSize-1 words where blockSize is the number of input samples processed by each call to riscv_fir_interpolate_q31().

Parameters

- **S** – [inout] points to an instance of the Q31 FIR interpolator structure
- **L** – [in] upsample factor
- **numTaps** – [in] number of filter coefficients in the filter
- **pCoeffs** – [in] points to the filter coefficient buffer
- **pState** – [in] points to the state buffer
- **blockSize** – [in] number of input samples to process per call

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : filter length numTaps is not a multiple of the interpolation factor L

RISCV_DSP_ATTRIBUTE void riscv_fir_interpolate_q15 (const riscv_fir_interpolate_instance_q15 *S, const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Processing function for the Q15 FIR interpolator.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. There is no risk of internal overflow with this approach and the full precision of intermediate multiplications is preserved. After all additions have been performed, the accumulator is truncated to 34.15 format by discarding low 15 bits. Lastly, the accumulator is saturated to yield a result in 1.15 format.

Parameters

- **S** – [in] points to an instance of the Q15 FIR interpolator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

RISCV_DSP_ATTRIBUTE void riscv_fir_interpolate_q31 (const riscv_fir_interpolate_instance_q31 *S, const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Processing function for the Q31 FIR interpolator.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. Thus, if the accumulator result overflows it wraps around rather than clip. In order to avoid overflows completely the input signal must be scaled down by $1/(\text{numTaps}/L)$. since numTaps/L

additions occur per output sample. After all multiply-accumulates are performed, the 2.62 accumulator is truncated to 1.32 format and then saturated to 1.31 format.

Parameters

- **S** – [in] points to an instance of the Q31 FIR interpolator structure
- **pSrc** – [in] points to the block of input data
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of input samples to process

group Filtering Functions

3.3.9 Interpolation Functions

Bilinear Interpolation

float16_t **riscv_bilinear_interp_f16**(const riscv_bilinear_interp_instance_f16 *S, float16_t X, float16_t Y)

float32_t (page 1277) **riscv_bilinear_interp_f32**(const riscv_bilinear_interp_instance_f32 *S, float32_t (page 1277) X, float32_t (page 1277) Y)

q15_t (page 1277) **riscv_bilinear_interp_q15**(riscv_bilinear_interp_instance_q15 *S, q15_t (page 1277) X, q15_t (page 1277) Y)

q31_t (page 1277) **riscv_bilinear_interp_q31**(riscv_bilinear_interp_instance_q31 *S, q31_t (page 1277) X, q31_t (page 1277) Y)

q7_t (page 1277) **riscv_bilinear_interp_q7**(riscv_bilinear_interp_instance_q7 *S, q7_t (page 1277) X, q7_t (page 1277) Y)

group Bilinear Interpolation

Bilinear interpolation is an extension of linear interpolation applied to a two dimensional grid. The underlying function $f(x, y)$ is sampled on a regular grid and the interpolation process determines values between the grid points. Bilinear interpolation is equivalent to two step linear interpolation, first in the x-dimension and then in the y-dimension. Bilinear interpolation is often used in image processing to rescale images. The NMSIS DSP library provides bilinear interpolation functions for Q7, Q15, Q31, and floating-point data types.

Algorithm end of LinearInterpolate group

The instance structure used by the bilinear interpolation functions describes a two dimensional data table. For floating-point, the instance structure is defined as:

```
typedef struct
{
    uint16_t numRows;
    uint16_t numCols;
    const float32_t *pData;
} riscv_bilinear_interp_instance_f32;
```

where `numRows` specifies the number of rows in the table; `numCols` specifies the number of columns in the table; and `pData` points to an array of size `numRows*numCols` values. The data table `pTable` is organized

in row order and the supplied data values fall on integer indexes. That is, table element (x,y) is located at `pTable[x + y*numCols]` where x and y are integers.

Let (x, y) specify the desired interpolation point. Then define:

```
XF = floor(x)
YF = floor(y)
```

The interpolated output point is computed as:

```
f(x, y) = f(XF, YF) * (1-(x-XF)) * (1-(y-YF))
        + f(XF+1, YF) * (x-XF)*(1-(y-YF))
        + f(XF, YF+1) * (1-(x-XF))*(y-YF)
        + f(XF+1, YF+1) * (x-XF)*(y-YF)
```

Note that the coordinates (x, y) contain integer and fractional components. The integer components specify which portion of the table to use while the fractional components control the interpolation processor.

if (x,y) are outside of the table boundary, Bilinear interpolation returns zero output.

Functions

`float16_t riscv_bilinear_interp_f16`(const riscv_bilinear_interp_instance_f16 *S, float16_t X, float16_t Y)

Floating-point bilinear interpolation.

Parameters

- **S** – [inout] points to an instance of the interpolation structure.
- **X** – [in] interpolation coordinate.
- **Y** – [in] interpolation coordinate.

Returns

out interpolated value.

[float32_t](#) (page 1277) `riscv_bilinear_interp_f32`(const riscv_bilinear_interp_instance_f32 *S, [float32_t](#) (page 1277) X, [float32_t](#) (page 1277) Y)

Floating-point bilinear interpolation.

Parameters

- **S** – [inout] points to an instance of the interpolation structure.
- **X** – [in] interpolation coordinate.
- **Y** – [in] interpolation coordinate.

Returns

out interpolated value.

[q15_t](#) (page 1277) `riscv_bilinear_interp_q15`(riscv_bilinear_interp_instance_q15 *S, [q31_t](#) (page 1277) X, [q31_t](#) (page 1277) Y)

Q15 bilinear interpolation.

Parameters

- **S** – **[inout]** points to an instance of the interpolation structure.
- **X** – **[in]** interpolation coordinate in 12.20 format.
- **Y** – **[in]** interpolation coordinate in 12.20 format.

Returns

out interpolated value.

q31_t (page 1277) **riscv_bilinear_interp_q31**(riscv_bilinear_interp_instance_q31 *S, *q31_t* (page 1277) X, *q31_t* (page 1277) Y)

Q31 bilinear interpolation.

Parameters

- **S** – **[inout]** points to an instance of the interpolation structure.
- **X** – **[in]** interpolation coordinate in 12.20 format.
- **Y** – **[in]** interpolation coordinate in 12.20 format.

Returns

out interpolated value.

q7_t (page 1277) **riscv_bilinear_interp_q7**(riscv_bilinear_interp_instance_q7 *S, *q31_t* (page 1277) X, *q31_t* (page 1277) Y)

Q7 bilinear interpolation.

Parameters

- **S** – **[inout]** points to an instance of the interpolation structure.
- **X** – **[in]** interpolation coordinate in 12.20 format.
- **Y** – **[in]** interpolation coordinate in 12.20 format.

Returns

out interpolated value.

Linear Interpolation

float16_t **riscv_linear_interp_f16**(const riscv_linear_interp_instance_f16 *S, float16_t x)

float32_t (page 1277) **riscv_linear_interp_f32**(const riscv_linear_interp_instance_f32 *S, *float32_t* (page 1277) x)

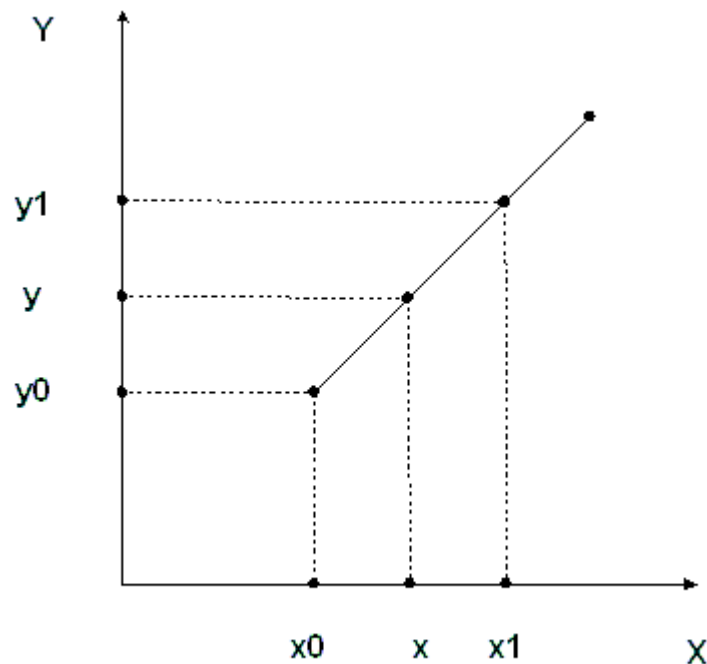
q15_t (page 1277) **riscv_linear_interp_q15**(const *q15_t* (page 1277) *pYData, *q31_t* (page 1277) x, uint32_t nValues)

q31_t (page 1277) **riscv_linear_interp_q31**(const *q31_t* (page 1277) *pYData, *q31_t* (page 1277) x, uint32_t nValues)

q7_t (page 1277) **riscv_linear_interp_q7**(const *q7_t* (page 1277) *pYData, *q31_t* (page 1277) x, uint32_t nValues)

group Linear Interpolation

Linear interpolation is a method of curve fitting using linear polynomials. Linear interpolation works by effectively drawing a straight line between two neighboring samples and returning the appropriate point along that line



A Linear Interpolate function calculates an output value(y), for the input(x) using linear interpolation of the input values x_0 , x_1 (nearest input values) and the output values y_0 and y_1 (nearest output values)

Algorithm:

```

 $y = y_0 + (x - x_0) * ((y_1 - y_0) / (x_1 - x_0))$ 
where  $x_0$ ,  $x_1$  are nearest values of input  $x$ 
       $y_0$ ,  $y_1$  are nearest values to output  $y$ 

```

This set of functions implements Linear interpolation process for Q7, Q15, Q31, and floating-point data types. The functions operate on a single sample of data and each call to the function returns a single processed value. S points to an instance of the Linear Interpolate function data structure. x is the input sample value. The functions returns the output value.

if x is outside of the table boundary, Linear interpolation returns first value of the table if x is below input range and returns last value of table if x is above range.

Functions

`float16_t riscv_linear_interp_f16(const riscv_linear_interp_instance_f16 *S, float16_t x)`

Process function for the floating-point Linear Interpolation Function.

Parameters

- **S** – [inout] is an instance of the floating-point Linear Interpolation structure
- **x** – [in] input sample to process

Returns

y processed output sample.

`float32_t` (page 1277) `riscv_linear_interp_f32(const riscv_linear_interp_instance_f32 *S, float32_t`
(page 1277) `x)`

Process function for the floating-point Linear Interpolation Function.

Parameters

- **S** – [inout] is an instance of the floating-point Linear Interpolation structure
- **x** – [in] input sample to process

Returns

y processed output sample.

`q15_t` (page 1277) `riscv_linear_interp_q15(const q15_t` (page 1277) `*pYData, q31_t` (page 1277) `x,`
`uint32_t nValues)`

Process function for the Q15 Linear Interpolation Function.

Input sample **x** is in 12.20 format which contains 12 bits for table index and 20 bits for fractional part.
This function can support maximum of table size 2^{12} .

Parameters

- **pYData** – [in] pointer to Q15 Linear Interpolation table
- **x** – [in] input sample to process
- **nValues** – [in] number of table values

Returns

y processed output sample.

`q31_t` (page 1277) `riscv_linear_interp_q31(const q31_t` (page 1277) `*pYData, q31_t` (page 1277) `x,`
`uint32_t nValues)`

Process function for the Q31 Linear Interpolation Function.

Input sample **x** is in 12.20 format which contains 12 bits for table index and 20 bits for fractional part.
This function can support maximum of table size 2^{12} .

Parameters

- **pYData** – [in] pointer to Q31 Linear Interpolation table
- **x** – [in] input sample to process

- **nValues** – [in] number of table values

Returns

y processed output sample.

q7_t (page 1277) **riscv_linear_interp_q7**(const *q7_t* (page 1277) *pYData, *q31_t* (page 1277) x, uint32_t nValues)

Process function for the Q7 Linear Interpolation Function.

Input sample x is in 12.20 format which contains 12 bits for table index and 20 bits for fractional part. This function can support maximum of table size 2^{12} .

Parameters

- **pYData** – [in] pointer to Q7 Linear Interpolation table
- **x** – [in] input sample to process
- **nValues** – [in] number of table values

Returns

y processed output sample.

Cubic Spline Interpolation

RISCV_DSP_ATTRIBUTE void **riscv_spline_f32** (riscv_spline_instance_f32 *S, const float32_t *xq, float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void **riscv_spline_init_f32** (riscv_spline_instance_f32 *S, riscv_spline_type type, const float32_t *x, const float32_t *y, uint32_t n, float32_t *coeffs, float32_t *tempBuffer)

group Cubic Spline Interpolation

Spline interpolation is a method of interpolation where the interpolant is a piecewise-defined polynomial called “spline”.

Given a function f defined on the interval [a,b], a set of n nodes $x(i)$ where $a=x(1)<x(2)<\dots<x(n)=b$ and a set of n values $y(i) = f(x(i))$, a cubic spline interpolant $S(x)$ is defined as:

Introduction

$$S(x) = \begin{matrix} S_1(x) & x(1) < x < x(2) \\ \dots & \\ S_{n-1}(x) & x(n-1) < x < x(n) \end{matrix}$$

where

$$S_i(x) = a_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3 \quad i=1, \dots, n-1$$

Having defined $h(i) = x(i+1) - x(i)$

Algorithm

$$h(i-1)c(i-1) + 2[h(i-1) + h(i)]c(i) + h(i)c(i+1) = 3/h(i)[a(i+1) - a(i)] - 3/h(i-1)[a(i) - a(i-1)] \quad i=2, \dots, n-1$$

It is possible to write the previous conditions in matrix form ($Ax=B$). In order to solve the system two boundary conditions are needed.

- Natural spline: $S_1''(x_1)=2*c(1)=0$; $S_n''(x_n)=2*c(n)=0$ In matrix form:

$$\begin{array}{cccccccc|ccc} 1 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & c(1) & & \\ \hline h(0) & 2[h(0)+h(1)] & h(1) & \dots & 0 & 0 & 0 & 0 & c(2) & & 3/h(2)[a(3)-a(2)]-3/h(1)[a(2)-a(1)] \\ \hline \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & = & \dots \\ \hline 0 & 0 & 0 & \dots & h(n-2) & 2[h(n-2)+h(n-1)] & h(n-1) & 0 & c(n-1) & & 3/h(n-1)[a(n)-a(n-1)]-3/h(n-2)[a(n-1)-a(n-2)] \\ \hline 0 & 0 & 0 & \dots & 0 & 0 & 0 & 1 & c(n) & & 0 \end{array}$$

- Parabolic runout spline: $S_1''(x_1)=2*c(1)=S_2''(x_2)=2*c(2)$; $S_{n-1}''(x_{n-1})=2*c(n-1)=S_n''(x_n)=2*c(n)$ In matrix form:

$$\begin{array}{cccccccc|ccc} 1 & -1 & 0 & \dots & 0 & 0 & 0 & 0 & c(1) & & \\ \hline h(0) & 2[h(0)+h(1)] & h(1) & \dots & 0 & 0 & 0 & 0 & c(2) & & 3/h(2)[a(3)-a(2)]-3/h(1)[a(2)-a(1)] \\ \hline \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & = & \dots \\ \hline 0 & 0 & 0 & \dots & h(n-2) & 2[h(n-2)+h(n-1)] & h(n-1) & 0 & c(n-1) & & 3/h(n-1)[a(n)-a(n-1)]-3/h(n-2)[a(n-1)-a(n-2)] \\ \hline 0 & 0 & 0 & \dots & 0 & 0 & -1 & 1 & c(n) & & 0 \end{array}$$

A is a tridiagonal matrix (a band matrix of bandwidth 3) of size $N=n+1$. The factorization algorithms ($A=LU$) can be simplified considerably because a large number of zeros appear in regular patterns. The Crout method has been used: 1) Solve $LZ=B$

$$\begin{aligned} u(1,2) &= A(1,2)/A(1,1) \\ z(1) &= B(1)/l(11) \\ \text{FOR } i=2, \dots, N-1 \\ l(i,i) &= A(i,i)-A(i,i-1)u(i-1,i) \\ u(i,i+1) &= a(i,i+1)/l(i,i) \\ z(i) &= [B(i)-A(i,i-1)z(i-1)]/l(i,i) \\ l(N,N) &= A(N,N)-A(N,N-1)u(N-1,N) \\ z(N) &= [B(N)-A(N,N-1)z(N-1)]/l(N,N) \end{aligned}$$

2) Solve $UX=Z$

```

c(N)=z(N)

FOR i=N-1, ..., 1
  c(i)=z(i)-u(i,i+1)c(i+1)

```

$c(i)$ for $i=1, \dots, n-1$ are needed to compute the $n-1$ polynomials. $b(i)$ and $d(i)$ are computed as:

- $b(i) = [y(i+1)-y(i)]/h(i)-h(i)*[c(i+1)+2*c(i)]/3$
- $d(i) = [c(i+1)-c(i)]/[3*h(i)]$ Moreover, $a(i)=y(i)$.

It is possible to compute the interpolated vector for x values outside the input range ($x_q < x(1)$; $x_q > x(n)$). The coefficients used to compute the y values for $x_q < x(1)$ are going to be the ones used for the first interval, while for $x_q > x(n)$ the coefficients used for the last interval.

Behaviour outside the given intervals

The initialization function takes as input two arrays that the user has to allocate: `coeffs` will contain the b , c , and d coefficients for the $(n-1)$ intervals (n is the number of known points), hence its size must be $3*(n-1)$; `tempBuffer` is temporally used for internal computations and its size is $n+n-1$.

Initialization function

The x input array must be strictly sorted in ascending order and it must not contain twice the same value ($x(i) < x(i+1)$).

Functions

RISCV_DSP_ATTRIBUTE void riscv_spline_f32 (riscv_spline_instance_f32 *S, const float32_t *xq, float32_t *pDst, uint32_t blockSize)

Processing function for the floating-point cubic spline interpolation.

Parameters

- **S** – [in] points to an instance of the floating-point spline structure.
- **xq** – [in] points to the x values of the interpolated data points.
- **pDst** – [out] points to the block of output data.
- **blockSize** – [in] number of samples of output data.

RISCV_DSP_ATTRIBUTE void riscv_spline_init_f32 (riscv_spline_instance_f32 *S, riscv_spline_type type, const float32_t *x, const float32_t *y, uint32_t n, float32_t *coeffs, float32_t *tempBuffer)

Initialization function for the floating-point cubic spline interpolation.

Parameters

- **S** – [inout] points to an instance of the floating-point spline structure.
- **type** – [in] type of cubic spline interpolation (boundary conditions)

- **x** – [in] points to the x values of the known data points.
- **y** – [in] points to the y values of the known data points.
- **n** – [in] number of known data points.
- **coeffs** – [in] coefficients array for b, c, and d
- **tempBuffer** – [in] buffer array for internal computations

group **Interpolation Functions**

These functions perform 1- and 2-dimensional interpolation of data. Linear interpolation is used for 1-dimensional data and bilinear interpolation is used for 2-dimensional data.

3.3.10 Matrix Functions

Householder transform of a vector

RISCV_DSP_ATTRIBUTE float16_t riscv_householder_f16 (const float16_t *pSrc,
const float16_t threshold, uint32_t blockSize, float16_t *pOut)

RISCV_DSP_ATTRIBUTE float32_t riscv_householder_f32 (const float32_t *pSrc,
const float32_t threshold, uint32_t blockSize, float32_t *pOut)

RISCV_DSP_ATTRIBUTE float64_t riscv_householder_f64 (const float64_t *pSrc,
const float64_t threshold, uint32_t blockSize, float64_t *pOut)

group **Householder transform of a vector**

Computes the Householder transform of a vector x.

The Householder transform of x is a vector v with

$$v_0 = 1$$

and a scalar β such that:

$$P = I - \beta vv^T$$

is an orthogonal matrix and

$$Px = ||x||_2 e_1$$

So P is an hyperplane reflection such that the image of x is proportional to e_1 .

e_1 is the vector of coordinates:

$$\begin{pmatrix} 1 \\ 0 \\ \vdots \end{pmatrix}$$

If x is already proportional to e_1 then the matrix P should be the identity.

Thus, β should be 0 and in this case the vector v can also be null.

But how do we detect that x is already proportional to e_1 .

If x

$$x = \begin{pmatrix} x_0 \\ xr \end{pmatrix}$$

where xr is a vector.

The algorithm is computing the norm squared of this vector:

$$||xr||^2$$

and this value is compared to a `threshold`. If the value is smaller than the `threshold`, the algorithm is returning 0 for β and the householder vector.

This `threshold` is an argument of the function.

Default values are provided in the header `dsp/matrix_functions.h` like for instance `DEFAULT_HOUSEHOLDER_THRESHOLD_F32`

Functions

RISCV_DSP_ATTRIBUTE float16_t riscv_householder_f16 (const float16_t *pSrc, const float16_t threshold, uint32_t blockSize, float16_t *pOut)

Householder transform of a half floating point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **threshold** – [in] norm2 threshold.
- **blockSize** – [in] dimension of the vector space.
- **pOut** – [out] points to the output vector.

Returns

beta return the scaling factor beta

RISCV_DSP_ATTRIBUTE float32_t riscv_householder_f32 (const float32_t *pSrc, const float32_t threshold, uint32_t blockSize, float32_t *pOut)

Householder transform of a floating point vector.

Parameters

- **pSrc** – [in] points to the input vector.

- **threshold** – [in] norm2 threshold.
- **blockSize** – [in] dimension of the vector space.
- **pOut** – [out] points to the output vector.

Returns

beta return the scaling factor beta

```
RISCV_DSP_ATTRIBUTE float64_t riscv_householder_f64 (const float64_t *pSrc,  
const float64_t threshold, uint32_t blockSize, float64_t *pOut)
```

Householder transform of a double floating point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **threshold** – [in] norm2 threshold.
- **blockSize** – [in] dimension of the vector space.
- **pOut** – [out] points to the output vector.

Returns

beta return the scaling factor beta

Matrix Addition

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_f16 (const riscv_matrix_instance_f16 *pSrcA,  
const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_f32 (const riscv_matrix_instance_f32 *pSrcA,  
const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_q15 (const riscv_matrix_instance_q15 *pSrcA,  
const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_q31 (const riscv_matrix_instance_q31 *pSrcA,  
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

group **Matrix Addition**

Adds two matrices.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} + \begin{pmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,1} & b_{3,2} & b_{3,3} \end{pmatrix} = \begin{pmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} & a_{1,3} + b_{1,3} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} & a_{2,3} + b_{2,3} \\ a_{3,1} + b_{3,1} & a_{3,2} + b_{3,2} & a_{3,3} + b_{3,3} \end{pmatrix}$$

Addition of two 3 x 3 matrices

The functions check to make sure that pSrcA, pSrcB, and pDst have the same number of rows and columns.

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_f16 (const riscv_matrix_instance_f16 *pSrcA, const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)

Floating-point matrix addition.

Parameters

- **pSrcA** – [in] points to first input matrix structure
- **pSrcB** – [in] points to second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_f32 (const riscv_matrix_instance_f32 *pSrcA, const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)

Floating-point matrix addition.

Parameters

- **pSrcA** – [in] points to first input matrix structure
- **pSrcB** – [in] points to second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_q15 (const riscv_matrix_instance_q15 *pSrcA, const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst)

Q15 matrix addition.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcA** – [in] points to first input matrix structure
- **pSrcB** – [in] points to second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful

- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_add_q31 (const riscv_matrix_instance_q31 *pSrcA,  
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

Q31 matrix addition.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcA** – [in] points to first input matrix structure
- **pSrcB** – [in] points to second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed

Cholesky and LDLT decompositions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f16 (const riscv_matrix_instance_f16 *pSrc,  
riscv_matrix_instance_f16 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f32 (const riscv_matrix_instance_f32 *pSrc,  
riscv_matrix_instance_f32 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f64 (const riscv_matrix_instance_f64 *pSrc,  
riscv_matrix_instance_f64 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_ldlt_f32 (const riscv_matrix_instance_f32 *pSrc,  
riscv_matrix_instance_f32 *pl, riscv_matrix_instance_f32 *pd, uint16_t *pp)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_ldlt_f64 (const riscv_matrix_instance_f64 *pSrc,  
riscv_matrix_instance_f64 *pl, riscv_matrix_instance_f64 *pd, uint16_t *pp)
```

group Cholesky and LDLT decompositions

Computes the Cholesky or LL^T decomposition of a matrix.

If the input matrix does not have a decomposition, then the algorithm terminates and returns error status `RISCV_MATH_DECOMPOSITION_FAILURE`.

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f16 (const riscv_matrix_instance_f16 *pSrc, riscv_matrix_instance_f16 *pDst)

Floating-point Cholesky decomposition of positive-definite matrix.

Floating-point Cholesky decomposition of Symmetric Positive Definite Matrix.

If the matrix is ill conditioned or only semi-definite, then it is better using the LDL^t decomposition. The decomposition of A is returning a lower triangular matrix U such that $A = L L^t$

The destination matrix should be set to 0 before calling the functions because the function may not overwrite all output elements.

Parameters

- **pSrc** – [in] points to the instance of the input floating-point matrix structure.
- **pDst** – [out] points to the instance of the output floating-point matrix structure.

Returns

The function returns `RISCV_MATH_SIZE_MISMATCH`, if the dimensions do not match.

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed
- `RISCV_MATH_DECOMPOSITION_FAILURE` : Input matrix cannot be decomposed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f32 (const riscv_matrix_instance_f32 *pSrc, riscv_matrix_instance_f32 *pDst)

Floating-point Cholesky decomposition of positive-definite matrix.

Floating-point Cholesky decomposition of Symmetric Positive Definite Matrix.

If the matrix is ill conditioned or only semi-definite, then it is better using the LDL^t decomposition. The decomposition of A is returning a lower triangular matrix L such that $A = L L^t$

The destination matrix should be set to 0 before calling the functions because the function may not overwrite all output elements.

Parameters

- **pSrc** – [in] points to the instance of the input floating-point matrix structure.
- **pDst** – [out] points to the instance of the output floating-point matrix structure.

Returns

The function returns `RISCV_MATH_SIZE_MISMATCH`, if the dimensions do not match.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed
- **RISCV_MATH_DECOMPOSITION_FAILURE** : Input matrix cannot be decomposed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cholesky_f64 (const riscv_matrix_instance_f64 *pSrc, riscv_matrix_instance_f64 *pDst)

Floating-point Cholesky decomposition of positive-definite matrix.

Floating-point Cholesky decomposition of Symmetric Positive Definite Matrix.

If the matrix is ill conditioned or only semi-definite, then it is better using the LDL^t decomposition. The decomposition of A is returning a lower triangular matrix L such that $A = L L^t$

The destination matrix should be set to 0 before calling the functions because the function may not overwrite all output elements.

Parameters

- **pSrc** – [in] points to the instance of the input floating-point matrix structure.
- **pDst** – [out] points to the instance of the output floating-point matrix structure.

Returns

The function returns **RISCV_MATH_SIZE_MISMATCH**, if the dimensions do not match.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed
- **RISCV_MATH_DECOMPOSITION_FAILURE** : Input matrix cannot be decomposed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_ldlt_f32 (const riscv_matrix_instance_f32 *pSrc, riscv_matrix_instance_f32 *pl, riscv_matrix_instance_f32 *pd, uint16_t *pp)

Floating-point LDL^t decomposition of positive semi-definite matrix.

Floating-point LDL decomposition of Symmetric Positive Semi-Definite Matrix.

Computes the LDL^t decomposition of a matrix A such that $P A P^t = L D L^t$.

Parameters

- **pSrc** – [in] points to the instance of the input floating-point matrix structure.
- **pl** – [out] points to the instance of the output floating-point triangular matrix structure.
- **pd** – [out] points to the instance of the output floating-point diagonal matrix structure.
- **pp** – [out] points to the instance of the output floating-point permutation vector.

Returns

The function returns `RISCV_MATH_SIZE_MISMATCH`, if the dimensions do not match.

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed
- `RISCV_MATH_DECOMPOSITION_FAILURE` : Input matrix cannot be decomposed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_ldlt_f64 (const riscv_matrix_instance_f64 *pSrc, riscv_matrix_instance_f64 *pl, riscv_matrix_instance_f64 *pd, uint16_t *pp)

Floating-point LDL^t decomposition of positive semi-definite matrix.

Floating-point LDL decomposition of Symmetric Positive Semi-Definite Matrix.

Computes the LDL^t decomposition of a matrix A such that $P A P^t = L D L^t$.

Parameters

- **pSrc** – [in] points to the instance of the input floating-point matrix structure.
- **pl** – [out] points to the instance of the output floating-point triangular matrix structure.
- **pd** – [out] points to the instance of the output floating-point diagonal matrix structure.
- **pp** – [out] points to the instance of the output floating-point permutation vector.

Returns

The function returns `RISCV_MATH_SIZE_MISMATCH`, if the dimensions do not match.

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed
- `RISCV_MATH_DECOMPOSITION_FAILURE` : Input matrix cannot be decomposed

Complex Matrix Multiplication

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_f16 (const riscv_matrix_instance_f16 *pSrcA, const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_f32 (const riscv_matrix_instance_f32 *pSrcA, const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_q15 (const riscv_matrix_instance_q15 *pSrcA, const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst, q15_t *pScratch)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_q31 (const riscv_matrix_instance_q31 *pSrcA, const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)

group **Complex Matrix Multiplication**

Complex Matrix multiplication is only defined if the number of columns of the first matrix equals the number of rows of the second matrix. Multiplying an $M \times N$ matrix with an $N \times P$ matrix results in an $M \times P$ matrix.

When matrix size checking is enabled, the functions check:

- that the inner dimensions of `pSrcA` and `pSrcB` are equal;
- that the size of the output matrix equals the outer dimensions of `pSrcA` and `pSrcB`.

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_f16 (**const riscv_matrix_instance_f16** *pSrcA, **const riscv_matrix_instance_f16** *pSrcB, **riscv_matrix_instance_f16** *pDst)

Floating-point Complex matrix multiplication.

Floating-point, complex, matrix multiplication.

Parameters

- **pSrcA** – [**in**] points to first input complex matrix structure
- **pSrcB** – [**in**] points to second input complex matrix structure
- **pDst** – [**out**] points to output complex matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_f32 (**const riscv_matrix_instance_f32** *pSrcA, **const riscv_matrix_instance_f32** *pSrcB, **riscv_matrix_instance_f32** *pDst)

Floating-point Complex matrix multiplication.

Floating-point, complex, matrix multiplication.

Parameters

- **pSrcA** – [**in**] points to first input complex matrix structure
- **pSrcB** – [**in**] points to second input complex matrix structure
- **pDst** – [**out**] points to output complex matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_q15 (**const riscv_matrix_instance_q15** *pSrcA, **const riscv_matrix_instance_q15** *pSrcB, **riscv_matrix_instance_q15** *pDst, **q15_t** *pScratch)

Q15 Complex matrix multiplication.

Q15, complex, matrix multiplication.

Conditions for optimum performance

Input, output and state buffers should be aligned by 32-bit

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The inputs to the multiplications are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Parameters

- **pSrcA** – [in] points to first input complex matrix structure
- **pSrcB** – [in] points to second input complex matrix structure
- **pDst** – [out] points to output complex matrix structure
- **pScratch** – [in] points to an array for storing intermediate results

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_mult_q31 (const riscv_matrix_instance_q31 *pSrcA, const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)

Q31 Complex matrix multiplication.

Q31, complex, matrix multiplication.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. The input is thus scaled down by $\log_2(\text{numColsA})$ bits to avoid overflows, as a total of numColsA additions are performed internally. The 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **pSrcA** – [in] points to first input complex matrix structure
- **pSrcB** – [in] points to second input complex matrix structure
- **pDst** – [out] points to output complex matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful

- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

Complex Matrix Transpose

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_f16 (const riscv_matrix_instance_f16 *pSrc,  
riscv_matrix_instance_f16 *pDst)
```

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_f32 (const riscv_matrix_instance_f32 *pSrc,  
riscv_matrix_instance_f32 *pDst)
```

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_q15 (const riscv_matrix_instance_q15 *pSrc,  
riscv_matrix_instance_q15 *pDst)
```

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_q31 (const riscv_matrix_instance_q31 *pSrc,  
riscv_matrix_instance_q31 *pDst)
```

group Complex Matrix Transpose

Transposes a complex matrix.

Transposing an $M \times N$ matrix flips it around the center diagonal and results in an $N \times M$ matrix.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix}^T = \begin{pmatrix} a_{1,1} & a_{2,1} & a_{3,1} \\ a_{1,2} & a_{2,2} & a_{3,2} \\ a_{1,3} & a_{2,3} & a_{3,3} \end{pmatrix}$$

Transpose of a 3 x 3 matrix

Functions

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_f16 (const riscv_matrix_instance_f16 *pSrc,  
riscv_matrix_instance_f16 *pDst)
```

Floating-point matrix transpose.

Floating-point complex matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_f32 (const riscv_matrix_instance_f32 *pSrc,  
riscv_matrix_instance_f32 *pDst)
```

Floating-point matrix transpose.

Floating-point complex matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_q15 (const riscv_matrix_instance_q15 *pSrc, riscv_matrix_instance_q15 *pDst)

Q15 complex matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_cmplx_trans_q31 (const riscv_matrix_instance_q31 *pSrc, riscv_matrix_instance_q31 *pDst)

Q31 complex matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

Matrix Initialization

RISCV_DSP_ATTRIBUTE void riscv_mat_init_f16 (riscv_matrix_instance_f16 *S, uint16_t nRows, uint16_t nColumns, float16_t *pData)

RISCV_DSP_ATTRIBUTE void riscv_mat_init_f32 (riscv_matrix_instance_f32 *S, uint16_t nRows, uint16_t nColumns, float32_t *pData)

RISCV_DSP_ATTRIBUTE void riscv_mat_init_f64 (riscv_matrix_instance_f64 *S, uint16_t nRows, uint16_t nColumns, float64_t *pData)

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_q15 (riscv_matrix_instance_q15 *S,  
uint16_t nRows, uint16_t nColumns, q15_t *pData)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_q31 (riscv_matrix_instance_q31 *S,  
uint16_t nRows, uint16_t nColumns, q31_t *pData)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_q7 (riscv_matrix_instance_q7 *S, uint16_t nRows,  
uint16_t nColumns, q7_t *pData)
```

group **Matrix Initialization**

Initializes the underlying matrix data structure. The functions set the numRows, numCols, and pData fields of the matrix data structure.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_f16 (riscv_matrix_instance_f16 *S,  
uint16_t nRows, uint16_t nColumns, float16_t *pData)
```

Floating-point matrix initialization.

Parameters

- **S** – [inout] points to an instance of the floating-point matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_f32 (riscv_matrix_instance_f32 *S,  
uint16_t nRows, uint16_t nColumns, float32_t *pData)
```

Floating-point matrix initialization.

Parameters

- **S** – [inout] points to an instance of the floating-point matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

```
RISCV_DSP_ATTRIBUTE void riscv_mat_init_f64 (riscv_matrix_instance_f64 *S,  
uint16_t nRows, uint16_t nColumns, float64_t *pData)
```

Floating-point matrix initialization.

Parameters

- **S** – [inout] points to an instance of the floating-point matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

RISCV_DSP_ATTRIBUTE void riscv_mat_init_q15 (riscv_matrix_instance_q15 *S,
uint16_t nRows, uint16_t nColumns, q15_t *pData)

Q15 matrix initialization.

Parameters

- **S** – [inout] points to an instance of the floating-point matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

RISCV_DSP_ATTRIBUTE void riscv_mat_init_q31 (riscv_matrix_instance_q31 *S,
uint16_t nRows, uint16_t nColumns, q31_t *pData)

Q31 matrix initialization.

Parameters

- **S** – [inout] points to an instance of the Q31 matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

RISCV_DSP_ATTRIBUTE void riscv_mat_init_q7 (riscv_matrix_instance_q7 *S,
uint16_t nRows, uint16_t nColumns, q7_t *pData)

Q7 matrix initialization.

Parameters

- **S** – [inout] points to an instance of the floating-point matrix structure
- **nRows** – [in] number of rows in the matrix
- **nColumns** – [in] number of columns in the matrix
- **pData** – [in] points to the matrix data array

Matrix Inverse

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f16 (const riscv_matrix_instance_f16 *pSrc,
riscv_matrix_instance_f16 *pDst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f32 (const riscv_matrix_instance_f32 *pSrc,
riscv_matrix_instance_f32 *pDst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f64 (const riscv_matrix_instance_f64 *pSrc,
riscv_matrix_instance_f64 *pDst)

riscv_status (page 1276) riscv_mat_solve_lower_triangular_f16(const riscv_matrix_instance_f16 *It, const
riscv_matrix_instance_f16 *a,
riscv_matrix_instance_f16 *dst)

riscv_status (page 1276) **riscv_mat_solve_lower_triangular_f32**(const riscv_matrix_instance_f32 *lt, const riscv_matrix_instance_f32 *a, riscv_matrix_instance_f32 *dst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_solve_lower_triangular_f64 (const riscv_matrix_instance_f64 *lt, const riscv_matrix_instance_f64 *a, riscv_matrix_instance_f64 *dst)

riscv_status (page 1276) **riscv_mat_solve_upper_triangular_f16**(const riscv_matrix_instance_f16 *ut, const riscv_matrix_instance_f16 *a, riscv_matrix_instance_f16 *dst)

riscv_status (page 1276) **riscv_mat_solve_upper_triangular_f32**(const riscv_matrix_instance_f32 *ut, const riscv_matrix_instance_f32 *a, riscv_matrix_instance_f32 *dst)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_solve_upper_triangular_f64 (const riscv_matrix_instance_f64 *ut, const riscv_matrix_instance_f64 *a, riscv_matrix_instance_f64 *dst)

group **Matrix Inverse**

Computes the inverse of a matrix.

The inverse is defined only if the input matrix is square and non-singular (the determinant is non-zero). The function checks that the input and output matrices are square and of the same size.

Matrix inversion is numerically sensitive and the NMSIS DSP library only supports matrix inversion of floating-point matrices.

$$\left(\begin{array}{ccc|ccc} a_{1,1} & a_{1,2} & a_{1,3} & 1 & 0 & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & 0 & 1 & 0 \\ a_{3,1} & a_{3,2} & a_{3,3} & 0 & 0 & 1 \end{array} \right) \rightarrow \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & x_{1,1} & x_{2,1} & x_{3,1} \\ 0 & 1 & 0 & x_{1,2} & x_{2,2} & x_{3,2} \\ 0 & 0 & 1 & x_{1,3} & x_{2,3} & x_{3,3} \end{array} \right)$$

Algorithm

The Gauss-Jordan method is used to find the inverse. The algorithm performs a sequence of elementary row-operations until it reduces the input matrix to an identity matrix. Applying the same sequence of elementary row-operations to an identity matrix yields the inverse matrix. If the input matrix is singular, then the algorithm terminates and returns error status **RISCV_MATH_SINGULAR**.

Matrix Inverse of a 3 x 3 matrix using Gauss-Jordan Method

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f16 (const riscv_matrix_instance_f16 *pSrc, riscv_matrix_instance_f16 *pDst)

Floating-point matrix inverse.

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed
- **RISCV_MATH_SINGULAR** : Input matrix is found to be singular (non-invertible)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f32 (const riscv_matrix_instance_f32 *pSrc, riscv_matrix_instance_f32 *pDst)

Floating-point matrix inverse.

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed
- **RISCV_MATH_SINGULAR** : Input matrix is found to be singular (non-invertible)

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_inverse_f64 (const riscv_matrix_instance_f64 *pSrc, riscv_matrix_instance_f64 *pDst)

Floating-point (64 bit) matrix inverse.

Floating-point matrix inverse.

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed
- **RISCV_MATH_SINGULAR** : Input matrix is found to be singular (non-invertible)

riscv_status (page 1276) **riscv_mat_solve_lower_triangular_f16**(const riscv_matrix_instance_f16 *lt, const riscv_matrix_instance_f16 *a, riscv_matrix_instance_f16 *dst)

Solve $LT \cdot X = A$ where LT is a lower triangular matrix.

Parameters

- **lt** – [in] The lower triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $LT \cdot X = A$

Returns

The function returns **RISCV_MATH_SINGULAR**, if the system can't be solved.

riscv_status (page 1276) **riscv_mat_solve_lower_triangular_f32**(const riscv_matrix_instance_f32 *lt,
const riscv_matrix_instance_f32 *a,
riscv_matrix_instance_f32 *dst)

Solve $LT \cdot X = A$ where LT is a lower triangular matrix.

Parameters

- **lt** – [in] The lower triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $LT \cdot X = A$

Returns

The function returns `RISCV_MATH_SINGULAR`, if the system can't be solved. Notice: The instruction `vfredusum` may introduce errors. So, if we use the V-extension implementation, we have to accept the errors that may happen in this function.

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_solve_lower_triangular_f64 (const riscv_matrix_instance_f64 *lt,
const riscv_matrix_instance_f64 *a, riscv_matrix_instance_f64 *dst)

Solve $LT \cdot X = A$ where LT is a lower triangular matrix.

Parameters

- **lt** – [in] The lower triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $LT \cdot X = A$

Returns

The function returns `RISCV_MATH_SINGULAR`, if the system can't be solved. Notice: The instruction `vfredusum` may introduce errors. So, if we use the V-extension implementation, we have to accept the errors that may happen in this function.

riscv_status (page 1276) **riscv_mat_solve_upper_triangular_f16**(const riscv_matrix_instance_f16 *ut,
const riscv_matrix_instance_f16 *a,
riscv_matrix_instance_f16 *dst)

Solve $UT \cdot X = A$ where UT is an upper triangular matrix.

Parameters

- **ut** – [in] The upper triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $UT \cdot X = A$

Returns

The function returns `RISCV_MATH_SINGULAR`, if the system can't be solved.

riscv_status (page 1276) **riscv_mat_solve_upper_triangular_f32**(const riscv_matrix_instance_f32 *ut,
const riscv_matrix_instance_f32 *a,
riscv_matrix_instance_f32 *dst)

Solve $UT \cdot X = A$ where UT is an upper triangular matrix.

Parameters

- **ut** – [in] The upper triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $UT \cdot X = A$

Returns

The function returns `RISCV_MATH_SINGULAR`, if the system can't be solved. Notice: The instruction `vfredusum` may introduce errors. So, if we use the V-extension implementation, we have to accept the errors that may happen in this function.

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_solve_upper_triangular_f64 (const riscv_matrix_instance_f64
const riscv_matrix_instance_f64 *a, riscv_matrix_instance_f64 *dst)
```

Solve $UT \cdot X = A$ where UT is an upper triangular matrix.

Parameters

- **ut** – [in] The upper triangular matrix
- **a** – [in] The matrix a
- **dst** – [out] The solution X of $UT \cdot X = A$

Returns

The function returns `RISCV_MATH_SINGULAR`, if the system can't be solved. Notice: The instruction `vfredusum` may introduce errors. So, if we use the V-extension implementation, we have to accept the errors that may happen in this function.

Matrix Multiplication

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f16 (const riscv_matrix_instance_f16 *pSrcA,
const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f32 (const riscv_matrix_instance_f32 *pSrcA,
const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f64 (const riscv_matrix_instance_f64 *pSrcA,
const riscv_matrix_instance_f64 *pSrcB, riscv_matrix_instance_f64 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_fast_q15 (const riscv_matrix_instance_q15 *pSrcA,
const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst, q15_t *pState)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_fast_q31 (const riscv_matrix_instance_q31 *pSrcA,
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_opt_q31 (const riscv_matrix_instance_q31 *pSrcA,
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst, q31_t *pState)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q15 (const riscv_matrix_instance_q15 *pSrcA,
const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst, q15_t *pState)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q31 (const riscv_matrix_instance_q31 *pSrcA,
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q7 (const riscv_matrix_instance_q7 *pSrcA,
const riscv_matrix_instance_q7 *pSrcB, riscv_matrix_instance_q7 *pDst, q7_t *pState)
```

group Matrix Multiplication

Multiplies two matrices.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} \begin{pmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,1} & b_{3,2} & b_{3,3} \end{pmatrix} = \begin{pmatrix} a_{1,1}b_{1,1} + a_{1,2}b_{2,1} + a_{1,3}b_{3,1} & a_{1,1}b_{1,2} + a_{1,2}b_{2,2} + a_{1,3}b_{3,2} & a_{1,1}b_{1,3} + a_{1,2}b_{2,3} + a_{1,3}b_{3,3} \\ a_{2,1}b_{1,1} + a_{2,2}b_{2,1} + a_{2,3}b_{3,1} & a_{2,1}b_{1,2} + a_{2,2}b_{2,2} + a_{2,3}b_{3,2} & a_{2,1}b_{1,3} + a_{2,2}b_{2,3} + a_{2,3}b_{3,3} \\ a_{3,1}b_{1,1} + a_{3,2}b_{2,1} + a_{3,3}b_{3,1} & a_{3,1}b_{1,2} + a_{3,2}b_{2,2} + a_{3,3}b_{3,2} & a_{3,1}b_{1,3} + a_{3,2}b_{2,3} + a_{3,3}b_{3,3} \end{pmatrix}$$

Multiplication of two 3x3 matrices:

Matrix multiplication is only defined if the number of columns of the first matrix equals the number of rows of the second matrix. Multiplying an $M \times N$ matrix with an $N \times P$ matrix results in an $M \times P$ matrix. When matrix size checking is enabled, the functions check: (1) that the inner dimensions of pSrcA and pSrcB are equal; and (2) that the size of the output matrix equals the outer dimensions of pSrcA and pSrcB.

Multiplies two matrices.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} a_{11}xb_{11}+a_{12}xb_{21}+a_{13}xb_{31} & a_{11}xb_{12}+a_{12}xb_{22}+a_{13}xb_{32} & a_{11}xb_{13}+a_{12}xb_{23}+a_{13}xb_{33} \\ a_{21}xb_{11}+a_{22}xb_{21}+a_{23}xb_{31} & a_{21}xb_{12}+a_{22}xb_{22}+a_{23}xb_{32} & a_{21}xb_{13}+a_{22}xb_{23}+a_{23}xb_{33} \\ a_{31}xb_{11}+a_{32}xb_{21}+a_{33}xb_{31} & a_{31}xb_{12}+a_{32}xb_{22}+a_{33}xb_{32} & a_{31}xb_{13}+a_{32}xb_{23}+a_{33}xb_{33} \end{bmatrix}$$

Matrix multiplication is only defined if the number of columns of the first matrix equals the number of rows of the second matrix. Multiplying an $M \times N$ matrix with an $N \times P$ matrix results in an $M \times P$ matrix. When matrix size checking is enabled, the functions check: (1) that the inner dimensions of pSrcA and pSrcB are equal; and (2) that the size of the output matrix equals the outer dimensions of pSrcA and pSrcB.

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f16 (const riscv_matrix_instance_f16 *pSrcA, const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)

Floating-point matrix multiplication.

Parameters

- ***pSrcA** – [in] points to the first input matrix structure
- ***pSrcB** – [in] points to the second input matrix structure
- ***pDst** – [out] points to output matrix structure

Returns

The function returns either RISCV_MATH_SIZE_MISMATCH or RISCV_MATH_SUCCESS based on the outcome of size checking.

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f32 (const riscv_matrix_instance_f32 *pSrcA, const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)

Floating-point matrix multiplication.

Parameters

- ***pSrcA** – [in] points to the first input matrix structure
- ***pSrcB** – [in] points to the second input matrix structure
- ***pDst** – [out] points to output matrix structure

Returns

The function returns either `RISCV_MATH_SIZE_MISMATCH` or `RISCV_MATH_SUCCESS` based on the outcome of size checking.

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_f64 (const riscv_matrix_instance_f64 *pSrcA, const riscv_matrix_instance_f64 *pSrcB, riscv_matrix_instance_f64 *pDst)

Floating-point matrix multiplication.

Parameters

- ***pSrcA** – [in] points to the first input matrix structure
- ***pSrcB** – [in] points to the second input matrix structure
- ***pDst** – [out] points to output matrix structure

Returns

The function returns either `RISCV_MATH_SIZE_MISMATCH` or `RISCV_MATH_SUCCESS` based on the outcome of size checking.

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_fast_q15 (const riscv_matrix_instance_q15 *pSrcA, const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst, q15_t *pState)

Q15 matrix multiplication (fast variant).

Q15 matrix multiplication (fast variant) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_mat_mult_q15()` for a slower implementation of this function which uses 64-bit accumulation to provide higher precision.

Scaling and Overflow Behavior

The difference between the function `riscv_mat_mult_q15()` and this fast variant is that the fast variant use a 32-bit rather than a 64-bit accumulator. The result of each 1.15 x 1.15 multiplication is truncated to 2.30 format. These intermediate results are accumulated in a 32-bit register in 2.30 format.

The fast version has the same overflow behavior as the standard version but provides less precision since it discards the low 16 bits of each multiplication result. In order to avoid overflows completely the input signals must be scaled down. Scale down one of the input matrices by $\log_2(\text{numColsA})$ bits to avoid overflows, as a total of `numColsA` additions are computed internally for each output element.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure
- **pState** – [in] points to the array for storing intermediate results

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful

- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_fast_q31 (const riscv_matrix_instance_q31 *pSrcA, const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)

Q31 matrix multiplication (fast variant).

Q31 matrix multiplication (fast variant) for RISC-V Core with DSP enabled.

Remark

Refer to `riscv_mat_mult_q31()` for a slower implementation of this function which uses 64-bit accumulation to provide higher precision.

Scaling and Overflow Behavior

The difference between the function `riscv_mat_mult_q31()` and this fast variant is that the fast variant use a 32-bit rather than a 64-bit accumulator. The result of each 1.31 x 1.31 multiplication is truncated to 2.30 format. These intermediate results are accumulated in a 32-bit register in 2.30 format. Finally, the accumulator is saturated and converted to a 1.31 result.

The fast version has the same overflow behavior as the standard version but provides less precision since it discards the low 32 bits of each multiplication result. In order to avoid overflows completely the input signals must be scaled down. Scale down one of the input matrices by $\log_2(\text{numColsA})$ bits to avoid overflows, as a total of `numColsA` additions are computed internally for each output element.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_opt_q31 (const riscv_matrix_instance_q31 *pSrcA, const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst, q31_t *pState)

Q31 matrix multiplication.

Remark

Refer to `riscv_mat_mult_fast_q31()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard

bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. The input is thus scaled down by $\log_2(\text{numColsA})$ bits to avoid overflows, as a total of numColsA additions are performed internally. The 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure
- **pState** – [in] points to the array for storing intermediate results

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q15 (const riscv_matrix_instance_q15 *pSrcA,
const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst,
q15_t *pState)
```

Q15 matrix multiplication.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The inputs to the multiplications are in 1.15 format and multiplications yield a 2.30 result. The 2.30 intermediate results are accumulated in a 64-bit accumulator in 34.30 format. This approach provides 33 guard bits and there is no risk of overflow. The 34.30 result is then truncated to 34.15 format by discarding the low 15 bits and then saturated to 1.15 format.

Refer to `riscv_mat_mult_fast_q15()` for a faster but less precise version of this function.

pState

pState will contain the transpose of pSrcB

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure
- **pState** – [in] points to the array for storing intermediate results

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q31 (const riscv_matrix_instance_q31 *pSrcA,
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

Q31 matrix multiplication.

Remark

Refer to `riscv_mat_mult_fast_q31()` for a faster but less precise implementation of this function.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The accumulator has a 2.62 format and maintains full precision of the intermediate multiplication results but provides only a single guard bit. There is no saturation on intermediate additions. Thus, if the accumulator overflows it wraps around and distorts the result. The input signals should be scaled down to avoid intermediate overflows. The input is thus scaled down by $\log_2(\text{numColsA})$ bits to avoid overflows, as a total of `numColsA` additions are performed internally. The 2.62 accumulator is right shifted by 31 bits and saturated to 1.31 format to yield the final result.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_mult_q7 (**const riscv_matrix_instance_q7 *pSrcA**,
const riscv_matrix_instance_q7 *pSrcB, **riscv_matrix_instance_q7 *pDst**, **q7_t *pState**)

Q7 matrix multiplication.

Scaling and Overflow Behavior:

The function is implemented using a 32-bit internal accumulator saturated to 1.7 format.

Parameters

- ***pSrcA** – [in] points to the first input matrix structure
- ***pSrcB** – [in] points to the second input matrix structure
- ***pDst** – [out] points to output matrix structure
- ***pState** – [in] points to the array for storing intermediate results (Unused in some versions)

Returns

The function returns either `RISCV_MATH_SIZE_MISMATCH` or `RISCV_MATH_SUCCESS` based on the outcome of size checking.

QR decomposition of a Matrix

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f16 (const riscv_matrix_instance_f16 *pSrc,
const float16_t threshold, riscv_matrix_instance_f16 *pOutR,
riscv_matrix_instance_f16 *pOutQ, float16_t *pOutTau, float16_t *pTmpA, float16_t *pTmpB)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f32 (const riscv_matrix_instance_f32 *pSrc,
const float32_t threshold, riscv_matrix_instance_f32 *pOutR,
riscv_matrix_instance_f32 *pOutQ, float32_t *pOutTau, float32_t *pTmpA, float32_t *pTmpB)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f64 (const riscv_matrix_instance_f64 *pSrc,
const float64_t threshold, riscv_matrix_instance_f64 *pOutR,
riscv_matrix_instance_f64 *pOutQ, float64_t *pOutTau, float64_t *pTmpA, float64_t *pTmpB)
```

group QR decomposition of a Matrix

Computes the QR decomposition of a matrix M using Householder algorithm.

$$M = QR$$

where Q is an orthogonal matrix and R is upper triangular. No pivoting strategy is used.

The returned value for R is using a format a bit similar to LAPACK : it is not just containing the matrix R but also the Householder reflectors.

The function is also returning a vector τ that is containing the scaling factor for the reflectors.

Returned value R has the structure:

$$\begin{pmatrix} r_{11} & r_{12} & \dots & r_{1n} \\ v_{12} & r_{22} & \dots & r_{2n} \\ v_{13} & v_{22} & \dots & r_{3n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{1m} & v_{2(m-1)} & \dots & r_{mn} \end{pmatrix}$$

where

$$v_1 = \begin{pmatrix} 1 \\ v_{12} \\ \vdots \\ v_{1m} \end{pmatrix}$$

is the first householder reflector.

The Householder Matrix is given by H_1

$$H_1 = I - \tau_1 v_1 v_1^T$$

The Matrix Q is the product of the Householder matrices:

$$Q = H_1 H_2 \dots H_n$$

The computation of the matrix Q by this function is optional.

And the matrix R, would be the returned value R without the householder reflectors:

$$\begin{pmatrix} r_{11} & r_{12} & \dots & r_{1n} \\ 0 & r_{22} & \dots & r_{2n} \\ 0 & 0 & \dots & r_{3n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & r_{mn} \end{pmatrix}$$

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f16 (const riscv_matrix_instance_f16 *pSrc, const float16_t threshold, riscv_matrix_instance_f16 *pOutR, riscv_matrix_instance_f16 *pOutQ, float16_t *pOutTau, float16_t *pTmpA, float16_t *pTmpB)

QR decomposition of a m x n half floating point matrix with m >= n.

QR decomposition of a m x n floating point matrix with m >= n.

pOutQ is optional:

pOutQ can be a NULL pointer. In this case, the argument will be ignored and the output Q matrix won't be computed.

f16 implementation

The f16 implementation is not very accurate.

Norm2 threshold

For the meaning of this argument please refer to the *Householder transform of a vector* (page 1026) documentation

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **threshold** – [in] norm2 threshold.
- **pOutR** – [out] points to output R matrix structure of dimension m x n
- **pOutQ** – [out] points to output Q matrix structure of dimension m x m (can be NULL)
- **pOutTau** – [out] points to Householder scaling factors of dimension n
- **pTmpA** – [inout] points to a temporary vector of dimension m.
- **pTmpB** – [inout] points to a temporary vector of dimension m.

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f32 (const riscv_matrix_instance_f32 *pSrc,
const float32_t threshold, riscv_matrix_instance_f32 *pOutR,
riscv_matrix_instance_f32 *pOutQ, float32_t *pOutTau, float32_t *pTmpA,
float32_t *pTmpB)
```

QR decomposition of a $m \times n$ floating point matrix with $m \geq n$.

pOutQ is optional:

pOutQ can be a NULL pointer. In this case, the argument will be ignored and the output Q matrix won't be computed.

Norm2 threshold

For the meaning of this argument please refer to the *Householder transform of a vector* (page 1026) documentation

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **threshold** – [in] norm2 threshold.
- **pOutR** – [out] points to output R matrix structure of dimension $m \times n$
- **pOutQ** – [out] points to output Q matrix structure of dimension $m \times m$ (can be NULL)
- **pOutTau** – [out] points to Householder scaling factors of dimension n
- **pTmpA** – [inout] points to a temporary vector of dimension m .
- **pTmpB** – [inout] points to a temporary vector of dimension m .

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_qr_f64 (const riscv_matrix_instance_f64 *pSrc,
const float64_t threshold, riscv_matrix_instance_f64 *pOutR,
riscv_matrix_instance_f64 *pOutQ, float64_t *pOutTau, float64_t *pTmpA,
float64_t *pTmpB)
```

QR decomposition of a $m \times n$ double floating point matrix with $m \geq n$.

QR decomposition of a $m \times n$ floating point matrix with $m \geq n$.

pOutQ is optional:

pOutQ can be a NULL pointer. In this case, the argument will be ignored and the output Q matrix won't be computed.

Norm2 threshold

For the meaning of this argument please refer to the *Householder transform of a vector* (page 1026) documentation

Parameters

- **pSrc** – [in] points to input matrix structure. The source matrix is modified by the function.
- **threshold** – [in] norm2 threshold.
- **pOutR** – [out] points to output R matrix structure of dimension m x n
- **pOutQ** – [out] points to output Q matrix structure of dimension m x m (can be NULL)
- **pOutTau** – [out] points to Householder scaling factors of dimension n
- **pTmpA** – [inout] points to a temporary vector of dimension m.
- **pTmpB** – [inout] points to a temporary vector of dimension m.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

Matrix Scale

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_f16 (const riscv_matrix_instance_f16 *pSrc,
float16_t scale, riscv_matrix_instance_f16 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_f32 (const riscv_matrix_instance_f32 *pSrc,
float32_t scale, riscv_matrix_instance_f32 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_q15 (const riscv_matrix_instance_q15 *pSrc,
q15_t scaleFract, int32_t shift, riscv_matrix_instance_q15 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_q31 (const riscv_matrix_instance_q31 *pSrc,
q31_t scaleFract, int32_t shift, riscv_matrix_instance_q31 *pDst)
```

group Matrix Scale

Multiplies a matrix by a scalar. This is accomplished by multiplying each element in the matrix by the scalar. For example:

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} K = \begin{pmatrix} K a_{1,1} & K a_{1,2} & K a_{1,3} \\ K a_{2,1} & K a_{2,2} & K a_{2,3} \\ K a_{3,1} & K a_{3,2} & K a_{3,3} \end{pmatrix}$$

Matrix Scaling of a 3 x 3 matrix

The function checks to make sure that the input and output matrices are of the same size.

In the fixed-point Q15 and Q31 functions, **scale** is represented by a fractional multiplication **scaleFract** and an arithmetic shift **shift**. The shift allows the gain of the scaling operation to exceed 1.0. The overall scale factor applied to the fixed-point data is

$$\text{scale} = \text{scaleFract} * 2^{\text{shift}}.$$

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_f16 (const riscv_matrix_instance_f16 *pSrc, float16_t scale, riscv_matrix_instance_f16 *pDst)

Floating-point matrix scaling.

Parameters

- **pSrc** – [in] points to input matrix
- **scale** – [in] scale factor to be applied
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_f32 (const riscv_matrix_instance_f32 *pSrc, float32_t scale, riscv_matrix_instance_f32 *pDst)

Floating-point matrix scaling.

Parameters

- **pSrc** – [in] points to input matrix
- **scale** – [in] scale factor to be applied
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_q15 (const riscv_matrix_instance_q15 *pSrc, q15_t scaleFract, int32_t shift, riscv_matrix_instance_q15 *pDst)

Q15 matrix scaling.

Scaling and Overflow Behavior

The input data *pSrc and scaleFract are in 1.15 format. These are multiplied to yield a 2.30 intermediate result and this is shifted with saturation to 1.15 format.

Parameters

- **pSrc** – [in] points to input matrix
- **scaleFract** – [in] fractional portion of the scale factor
- **shift** – [in] number of bits to shift the result by
- **pDst** – [out] points to output matrix structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_scale_q31 (const riscv_matrix_instance_q31 *pSrc, q31_t scaleFract, int32_t shift, riscv_matrix_instance_q31 *pDst)`

Q31 matrix scaling.

Scaling and Overflow Behavior

The input data `*pSrc` and `scaleFract` are in 1.31 format. These are multiplied to yield a 2.62 intermediate result which is shifted with saturation to 1.31 format.

Parameters

- **pSrc** – [in] points to input matrix
- **scaleFract** – [in] fractional portion of the scale factor
- **shift** – [in] number of bits to shift the result by
- **pDst** – [out] points to output matrix structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_SIZE_MISMATCH` : Matrix size check failed

Matrix Subtraction

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f16 (const riscv_matrix_instance_f16 *pSrcA, const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)`

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f32 (const riscv_matrix_instance_f32 *pSrcA, const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)`

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f64 (const riscv_matrix_instance_f64 *pSrcA, const riscv_matrix_instance_f64 *pSrcB, riscv_matrix_instance_f64 *pDst)`

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_q15 (const riscv_matrix_instance_q15 *pSrcA, const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst)`

`RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_q31 (const riscv_matrix_instance_q31 *pSrcA, const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)`

group Matrix Subtraction

Subtract two matrices.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix} - \begin{pmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,1} & b_{3,2} & b_{3,3} \end{pmatrix} = \begin{pmatrix} a_{1,1} - b_{1,1} & a_{1,2} - b_{1,2} & a_{1,3} - b_{1,3} \\ a_{2,1} - b_{2,1} & a_{2,2} - b_{2,2} & a_{2,3} - b_{2,3} \\ a_{3,1} - b_{3,1} & a_{3,2} - b_{3,2} & a_{3,3} - b_{3,3} \end{pmatrix}$$

The functions check to make sure that `pSrcA`, `pSrcB`, and `pDst` have the same number of rows and columns.

Subtraction of two 3 x 3 matrices

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f16 (const riscv_matrix_instance_f16 *pSrcA, const riscv_matrix_instance_f16 *pSrcB, riscv_matrix_instance_f16 *pDst)

Floating-point matrix subtraction.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f32 (const riscv_matrix_instance_f32 *pSrcA, const riscv_matrix_instance_f32 *pSrcB, riscv_matrix_instance_f32 *pDst)

Floating-point matrix subtraction.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_f64 (const riscv_matrix_instance_f64 *pSrcA, const riscv_matrix_instance_f64 *pSrcB, riscv_matrix_instance_f64 *pDst)

Floating-point matrix subtraction.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_q15 (const riscv_matrix_instance_q15 *pSrcA,  
const riscv_matrix_instance_q15 *pSrcB, riscv_matrix_instance_q15 *pDst)
```

Q15 matrix subtraction.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_sub_q31 (const riscv_matrix_instance_q31 *pSrcA,  
const riscv_matrix_instance_q31 *pSrcB, riscv_matrix_instance_q31 *pDst)
```

Q31 matrix subtraction.

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Parameters

- **pSrcA** – [in] points to the first input matrix structure
- **pSrcB** – [in] points to the second input matrix structure
- **pDst** – [out] points to output matrix structure

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_SIZE_MISMATCH** : Matrix size check failed

Matrix Transpose

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f16 (const riscv_matrix_instance_f16 *pSrc,  
riscv_matrix_instance_f16 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f32 (const riscv_matrix_instance_f32 *pSrc,  
riscv_matrix_instance_f32 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f64 (const riscv_matrix_instance_f64 *pSrc,
riscv_matrix_instance_f64 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q15 (const riscv_matrix_instance_q15 *pSrc,
riscv_matrix_instance_q15 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q31 (const riscv_matrix_instance_q31 *pSrc,
riscv_matrix_instance_q31 *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q7 (const riscv_matrix_instance_q7 *pSrc,
riscv_matrix_instance_q7 *pDst)
```

group **Matrix Transpose**

Transposes a matrix.

Transposing an $M \times N$ matrix flips it around the center diagonal and results in an $N \times M$ matrix.

$$\begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix}^T = \begin{pmatrix} a_{1,1} & a_{2,1} & a_{3,1} \\ a_{1,2} & a_{2,2} & a_{3,2} \\ a_{1,3} & a_{2,3} & a_{3,3} \end{pmatrix}$$

Transpose of a 3 x 3 matrix

Transposes a matrix.

Transposing an $M \times N$ matrix flips it around the center diagonal and results in an $N \times M$ matrix.

$$\begin{bmatrix} \overline{a_{11}} & \overline{a_{12}} & \overline{a_{13}} \\ \overline{a_{21}} & \overline{a_{22}} & \overline{a_{23}} \\ \overline{a_{31}} & \overline{a_{32}} & \overline{a_{33}} \end{bmatrix}^T = \begin{bmatrix} \overline{a_{11}} & \overline{a_{21}} & \overline{a_{31}} \\ \overline{a_{12}} & \overline{a_{22}} & \overline{a_{32}} \\ \overline{a_{13}} & \overline{a_{23}} & \overline{a_{33}} \end{bmatrix}$$

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f16 (const riscv_matrix_instance_f16 *pSrc,
riscv_matrix_instance_f16 *pDst)
```

Floating-point matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful

- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f32 (const riscv_matrix_instance_f32 *pSrc, riscv_matrix_instance_f32 *pDst)

Floating-point matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_trans_f64 (const riscv_matrix_instance_f64 *pSrc, riscv_matrix_instance_f64 *pDst)

Floating-point matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q15 (const riscv_matrix_instance_q15 *pSrc, riscv_matrix_instance_q15 *pDst)

Q15 matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q31 (const riscv_matrix_instance_q31 *pSrc, riscv_matrix_instance_q31 *pDst)

Q31 matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

RISC_V_DSP_ATTRIBUTE riscv_status riscv_mat_trans_q7 (const riscv_matrix_instance_q7 *pSrc, riscv_matrix_instance_q7 *pDst)

Q7 matrix transpose.

Parameters

- **pSrc** – [in] points to input matrix
- **pDst** – [out] points to output matrix

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_SIZE_MISMATCH : Matrix size check failed

Matrix Vector Multiplication

RISC_V_DSP_ATTRIBUTE void riscv_mat_vec_mult_f16 (const riscv_matrix_instance_f16 *pSrcMat, const float16_t *pVec, float16_t *pDst)

RISC_V_DSP_ATTRIBUTE void riscv_mat_vec_mult_f32 (const riscv_matrix_instance_f32 *pSrcMat, const float32_t *pVec, float32_t *pDst)

RISC_V_DSP_ATTRIBUTE void riscv_mat_vec_mult_q15 (const riscv_matrix_instance_q15 *pSrcMat, const q15_t *pVec, q15_t *pDst)

RISC_V_DSP_ATTRIBUTE void riscv_mat_vec_mult_q31 (const riscv_matrix_instance_q31 *pSrcMat, const q31_t *pVec, q31_t *pDst)

RISC_V_DSP_ATTRIBUTE void riscv_mat_vec_mult_q7 (const riscv_matrix_instance_q7 *pSrcMat, const q7_t *pVec, q7_t *pDst)

group **Matrix Vector Multiplication**

Multiplies a matrix and a vector.

Functions

RISCV_DSP_ATTRIBUTE void riscv_mat_vec_mult_f16 (const riscv_matrix_instance_f16 *pSrcMat, const float16_t *pVec, float16_t *pDst)

Floating-point matrix and vector multiplication.

Parameters

- ***pSrcMat** – [in] points to the input matrix structure
- ***pVec** – [in] points to input vector
- ***pDst** – [out] points to output vector

RISCV_DSP_ATTRIBUTE void riscv_mat_vec_mult_f32 (const riscv_matrix_instance_f32 *pSrcMat, const float32_t *pVec, float32_t *pDst)

Floating-point matrix and vector multiplication.

Parameters

- ***pSrcMat** – [in] points to the input matrix structure
- ***pVec** – [in] points to input vector
- ***pDst** – [out] points to output vector

RISCV_DSP_ATTRIBUTE void riscv_mat_vec_mult_q15 (const riscv_matrix_instance_q15 *pSrcMat, const q15_t *pVec, q15_t *pDst)

Q15 matrix and vector multiplication.

Parameters

- ***pSrcMat** – [in] points to the input matrix structure
- ***pVec** – [in] points to input vector
- ***pDst** – [out] points to output vector

RISCV_DSP_ATTRIBUTE void riscv_mat_vec_mult_q31 (const riscv_matrix_instance_q31 *pSrcMat, const q31_t *pVec, q31_t *pDst)

Q31 matrix and vector multiplication.

Parameters

- ***pSrcMat** – [in] points to the input matrix structure
- ***pVec** – [in] points to the input vector
- ***pDst** – [out] points to the output vector

RISCV_DSP_ATTRIBUTE void riscv_mat_vec_mult_q7 (const riscv_matrix_instance_q7 *pSrcMat, const q7_t *pVec, q7_t *pDst)

Q7 matrix and vector multiplication.

Parameters

- ***pSrcMat** – [in] points to the input matrix structure
- ***pVec** – [in] points to the input vector
- ***pDst** – [out] points to the output vector

group Matrix Functions

This set of functions provides basic matrix math operations. The functions operate on matrix data structures. For example, the type definition for the floating-point matrix structure is shown below:

```
typedef struct
{
    uint16_t numRows;    // number of rows of the matrix.
    uint16_t numCols;    // number of columns of the matrix.
    float32_t *pData;    // points to the data of the matrix.
} riscv_matrix_instance_f32;
```

There are similar definitions for Q15 and Q31 data types.

The structure specifies the size of the matrix and then points to an array of data. The array is of size `numRows X numCols` and the values are arranged in row order. That is, the matrix element (i, j) is stored at:

```
pData[i*numCols + j]
```

Init Functions

There is an associated initialization function for each type of matrix data structure. The initialization function sets the values of the internal structure fields. Refer to `riscv_mat_init_f32()`, `riscv_mat_init_q31()` and `riscv_mat_init_q15()` for floating-point, Q31 and Q15 types, respectively.

Use of the initialization function is optional. However, if initialization function is used then the instance structure cannot be placed into a const data section. To place the instance structure in a const data section, manually initialize the data structure. For example:

```
<code>riscv_matrix_instance_f32 S = {nRows, nColumns, pData};</code>
<code>riscv_matrix_instance_q31 S = {nRows, nColumns, pData};</code>
<code>riscv_matrix_instance_q15 S = {nRows, nColumns, pData};</code>
```

where `nRows` specifies the number of rows, `nColumns` specifies the number of columns, and `pData` points to the data array.

Size Checking

By default all of the matrix functions perform size checking on the input and output matrices. For example, the matrix addition function verifies that the two input matrices and the output matrix all have the same number of rows and columns. If the size check fails the functions return:

```
RISCV_MATH_SIZE_MISMATCH
```

Otherwise the functions return

```
RISCV_MATH_SUCCESS
```

There is some overhead associated with this matrix size checking. The matrix size checking is enabled via the `#define`

```
RISCV_MATH_MATRIX_CHECK
```

within the library project settings. By default this macro is defined and size checking is enabled. By changing the project settings and undefining this macro size checking is eliminated and the functions run a bit faster. With size checking disabled the functions always return `RISCV_MATH_SUCCESS`.

3.3.11 Quaternion Math Functions

Quaternion conversions

Quaternion to Rotation

```
RISCV_DSP_ATTRIBUTE void riscv_quaternion2rotation_f32 (const float32_t *pInputQuaternions,
float32_t *pOutputRotations, uint32_t nbQuaternions)
```

group Quaternion to Rotation

Conversions from quaternion to rotation.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_quaternion2rotation_f32 (const float32_t *pInputQuaternions,
float32_t *pOutputRotations, uint32_t nbQuaternions)
```

Conversion of quaternion to equivalent rotation matrix.

The quaternion $a + ib + jc + kd$ is converted into rotation matrix:

$a^2 + b^2 - c^2 - d^2$	$2bc - 2ad$	$2bd + 2ac$
$2bc + 2ad$	$a^2 - b^2 + c^2 - d^2$	$2cd - 2ab$
$2bd - 2ac$	$2cd + 2ab$	$a^2 - b^2 - c^2 + d^2$

Rotation matrix is saved in row order : R00 R01 R02 R10 R11 R12 R20 R21 R22

Format of rotation matrix

Parameters

- **pInputQuaternions** – [in] points to an array of normalized quaternions
- **pOutputRotations** – [out] points to an array of 3x3 rotations (in row order)
- **nbQuaternions** – [in] number of quaternions in the array

Rotation to Quaternion

```
RISCV_DSP_ATTRIBUTE void riscv_rotation2quaternion_f32 (const float32_t *pInputRotations,
float32_t *pOutputQuaternions, uint32_t nbQuaternions)
```

group Rotation to Quaternion

Conversions from rotation to quaternion.

Functions

RISCV_DSP_ATTRIBUTE void riscv_rotation2quaternion_f32 (const float32_t *pInputRotations, float32_t *pOutputQuaternions, uint32_t nbQuaternions)

Conversion of a rotation matrix to an equivalent quaternion.

Conversion of a rotation matrix to equivalent quaternion.

q and -q are representing the same rotation. This ambiguity must be taken into account when using the output of this function.

Parameters

- **pInputRotations** – [in] points to an array 3x3 rotation matrix (in row order)
- **pOutputQuaternions** – [out] points to an array quaternions
- **nbQuaternions** – [in] number of quaternions in the array

group Quaternion conversions

Conversions between quaternion and rotation representations.

Quaternion Conjugate

RISCV_DSP_ATTRIBUTE void riscv_quaternion_conjugate_f32 (const float32_t *pInputQuaternions, float32_t *pConjugateQuaternions, uint32_t nbQuaternions)

group Quaternion Conjugate

Compute the conjugate of a quaternion.

Functions

RISCV_DSP_ATTRIBUTE void riscv_quaternion_conjugate_f32 (const float32_t *pInputQuaternions, float32_t *pConjugateQuaternions, uint32_t nbQuaternions)

Floating-point quaternion conjugates.

Parameters

- **pInputQuaternions** – [in] points to the input vector of quaternions
- **pConjugateQuaternions** – [out] points to the output vector of conjugate quaternions
- **nbQuaternions** – [in] number of quaternions in each vector

Quaternion Inverse

RISCV_DSP_ATTRIBUTE void riscv_quaternion_inverse_f32 (const float32_t *pInputQuaternions, float32_t *pInverseQuaternions, uint32_t nbQuaternions)

group Quaternion Inverse

Compute the inverse of a quaternion.

Functions

RISCV_DSP_ATTRIBUTE void riscv_quaternion_inverse_f32 (const float32_t *pInputQuaternions, float32_t *pInverseQuaternions, uint32_t nbQuaternions)

Floating-point quaternion inverse.

Parameters

- **pInputQuaternions** – [in] points to the input vector of quaternions
- **pInverseQuaternions** – [out] points to the output vector of inverse quaternions
- **nbQuaternions** – [in] number of quaternions in each vector

Quaternion Norm

RISCV_DSP_ATTRIBUTE void riscv_quaternion_norm_f32 (const float32_t *pInputQuaternions, float32_t *pNorms, uint32_t nbQuaternions)

group Quaternion Norm

Compute the norm of a quaternion.

Functions

RISCV_DSP_ATTRIBUTE void riscv_quaternion_norm_f32 (const float32_t *pInputQuaternions, float32_t *pNorms, uint32_t nbQuaternions)

Floating-point quaternion Norm.

Parameters

- **pInputQuaternions** – [in] points to the input vector of quaternions
- **pNorms** – [out] points to the output vector of norms
- **nbQuaternions** – [in] number of quaternions in the input vector

Quaternion normalization

RISCV_DSP_ATTRIBUTE void riscv_quaternion_normalize_f32 (const float32_t *pInputQuaternions, float32_t *pNormalizedQuaternions, uint32_t nbQuaternions)

group Quaternion normalization

Compute a normalized quaternion.

Functions

RISCV_DSP_ATTRIBUTE void riscv_quaternion_normalize_f32 (const float32_t *pInputQuaternions, float32_t *pNormalizedQuaternions, uint32_t nbQuaternions)

Floating-point normalization of quaternions.

Parameters

- **pInputQuaternions** – [in] points to the input vector of quaternions
- **pNormalizedQuaternions** – [out] points to the output vector of normalized quaternions
- **nbQuaternions** – [in] number of quaternions in each vector

Quaternion Product

Elementwise Quaternion Product

RISCV_DSP_ATTRIBUTE void riscv_quaternion_product_f32 (const float32_t *qa, const float32_t *qb, float32_t *qr, uint32_t nbQuaternions)

group Elementwise Quaternion Product

Compute the elementwise product of quaternions.

Functions

RISCV_DSP_ATTRIBUTE void riscv_quaternion_product_f32 (const float32_t *qa, const float32_t *qb, float32_t *qr, uint32_t nbQuaternions)

Floating-point elementwise product two quaternions.

Parameters

- **qa** – [in] first array of quaternions
- **qb** – [in] second array of quaternions
- **qr** – [out] elementwise product of quaternions
- **nbQuaternions** – [in] number of quaternions in the array

Quaternion Product

```
RISCV_DSP_ATTRIBUTE void riscv_quaternion_product_single_f32 (const float32_t *qa,  
const float32_t *qb, float32_t *qr)
```

group Quaternion Product

Compute the product of two quaternions.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_quaternion_product_single_f32 (const float32_t *qa,  
const float32_t *qb, float32_t *qr)
```

Floating-point product of two quaternions.

Parameters

- **qa** – [in] first quaternion
- **qb** – [in] second quaternion
- **qr** – [out] product of two quaternions

group Quaternion Product

Compute the product of quaternions.

group Quaternion Math Functions

Functions to operates on quaternions and convert between a rotation and quaternion representation.

3.3.12 Statistics Functions

Absolute Maximum

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_f64 (const float64_t *pSrc, uint32_t blockSize,  
float64_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f16 (const float16_t *pSrc,  
uint32_t blockSize, float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f32 (const float32_t *pSrc,  
uint32_t blockSize, float32_t *pResult)
```



```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f64 (const float64_t *pSrc,
uint32_t blockSize, float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult, uint32_t *pIndex)
```

group **Absolute Maximum**

Computes the maximum value of absolute values of an array of data. The function returns both the maximum value and its position within the array. There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_f16 (const float16_t *pSrc,
uint32_t blockSize, float16_t *pResult, uint32_t *pIndex)
```

Maximum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_absmax_f32 (const float32_t *pSrc,
uint32_t blockSize, float32_t *pResult, uint32_t *pIndex)
```

Maximum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult, uint32_t *pIndex)

Maximum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Maximum value of absolute values of a floating-point vector.

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Maximum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Maximum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Maximum value of absolute values of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector

- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult)

Maximum value of absolute values of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult)

Maximum value of absolute values of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult, uint32_t *pIndex)

Maximum value of absolute values of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult, uint32_t *pIndex)

Maximum value of absolute values of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmax_q7 (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult, uint32_t *pIndex)

Maximum value of absolute values of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

Absolute Minimum

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_f64 (const float64_t *pSrc, uint32_t blockSize,  
float64_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f16 (const float16_t *pSrc,  
uint32_t blockSize, float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f32 (const float32_t *pSrc,  
uint32_t blockSize, float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f64 (const float64_t *pSrc,  
uint32_t blockSize, float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize,  
q7_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_absmin_q7 (const q7_t *pSrc, uint32_t blockSize,  
q7_t *pResult, uint32_t *pIndex)
```

group Absolute Minimum

Computes the minimum value of absolute values of an array of data. The function returns both the minimum value and its position within the array. There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_absmin_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Minimum value of absolute values of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Minimum value of absolute values of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult)

Minimum value of absolute values of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult)

Minimum value of absolute values of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector

- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_absmin_q7 (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult, uint32_t *pIndex)

Minimum value of absolute values of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

Accumulation functions

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

group Accumulation functions

Calculates the accumulation of the input vector. Sum is defined as the addition of the elements in the vector. The underlying algorithm is used:

```
Result = (pSrc[0] + pSrc[1] + pSrc[2] + ... + pSrc[blockSize-1]);
```

There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

accumulate value of a floating-point vector.

Sum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] sum of values in input vector.

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Accumulation value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] sum of values in input vector.

RISCV_DSP_ATTRIBUTE void riscv_accumulate_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Accumulation value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] sum of values in input vector.

Entropy

RISCV_DSP_ATTRIBUTE float16_t riscv_entropy_f16 (const float16_t *pSrcA, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE float32_t riscv_entropy_f32 (const float32_t *pSrcA, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE float64_t riscv_entropy_f64 (const float64_t *pSrcA, uint32_t blockSize)

group Entropy

Computes the entropy of a distribution

Functions

RISCV_DSP_ATTRIBUTE float16_t riscv_entropy_f16 (const float16_t *pSrcA, uint32_t blockSize)

Entropy.

Parameters

- **pSrcA** – [in] Array of input values.
- **blockSize** – [in] Number of samples in the input array.

Returns

Entropy -Sum($p \ln p$)

RISCV_DSP_ATTRIBUTE float32_t riscv_entropy_f32 (const float32_t *pSrcA, uint32_t blockSize)

Entropy.

Parameters

- **pSrcA** – [in] Array of input values.
- **blockSize** – [in] Number of samples in the input array.

Returns

Entropy -Sum($p \ln p$)

RISCV_DSP_ATTRIBUTE float64_t riscv_entropy_f64 (const float64_t *pSrcA, uint32_t blockSize)

Entropy.

Parameters

- **pSrcA** – [in] Array of input values.
- **blockSize** – [in] Number of samples in the input array.

Returns

Entropy -Sum($p \ln p$)

Kullback-Leibler divergence

RISCV_DSP_ATTRIBUTE float16_t riscv_kullback_leibler_f16 (const float16_t *pSrcA, const float16_t *pSrcB, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE float32_t riscv_kullback_leibler_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE float64_t riscv_kullback_leibler_f64 (const float64_t *pSrcA, const float64_t *pSrcB, uint32_t blockSize)

group Kullback-Leibler divergence

Computes the Kullback-Leibler divergence between two distributions

Functions

RISCV_DSP_ATTRIBUTE float16_t riscv_kullback_leibler_f16 (const float16_t *pSrcA, const float16_t *pSrcB, uint32_t blockSize)

Kullback-Leibler.

Distribution A may contain 0. Result will be right but some exception flags will be set.

Distribution B must not contain 0 probability.

Parameters

- ***pSrcA** – [in] points to an array of input values for probability distribution A.
- ***pSrcB** – [in] points to an array of input values for probability distribution B.
- **blockSize** – [in] number of samples in the input array.

Returns

Kullback-Leibler divergence $D(A \parallel B)$

RISCV_DSP_ATTRIBUTE float32_t riscv_kullback_leibler_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t blockSize)

Kullback-Leibler.

Distribution A may contain 0. Result will be right but some exception flags will be set.

Distribution B must not contain 0 probability.

Parameters

- ***pSrcA** – [in] points to an array of input values for probability distribution A.
- ***pSrcB** – [in] points to an array of input values for probability distribution B.
- **blockSize** – [in] number of samples in the input array.

Returns

Kullback-Leibler divergence $D(A \parallel B)$

RISCV_DSP_ATTRIBUTE float64_t riscv_kullback_leibler_f64 (const float64_t *pSrcA, const float64_t *pSrcB, uint32_t blockSize)

Kullback-Leibler.

Parameters

- ***pSrcA** – [in] points to an array of input values for probability distribution A.
- ***pSrcB** – [in] points to an array of input values for probability distribution B.
- **blockSize** – [in] number of samples in the input array.

Returns

Kullback-Leibler divergence $D(A \parallel B)$

LogSumExp

```
RISCV_DSP_ATTRIBUTE float16_t riscv_logsumexp_dot_prod_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, uint32_t blockSize, float16_t *pTmpBuffer)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_logsumexp_dot_prod_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, uint32_t blockSize, float32_t *pTmpBuffer)
```

```
RISCV_DSP_ATTRIBUTE float16_t riscv_logsumexp_f16 (const float16_t *in,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE float32_t riscv_logsumexp_f32 (const float32_t *in,
uint32_t blockSize)
```

group LogSumExp

LogSumExp optimizations to compute sum of probabilities with Gaussian distributions

Functions

```
RISCV_DSP_ATTRIBUTE float16_t riscv_logsumexp_dot_prod_f16 (const float16_t *pSrcA,
const float16_t *pSrcB, uint32_t blockSize, float16_t *pTmpBuffer)
```

Dot product with log arithmetic.

Vectors are containing the log of the samples

Parameters

- ***pSrcA** – [in] points to the first input vector
- ***pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in each vector
- ***pTmpBuffer** – [in] temporary buffer of length blockSize

Returns

The log of the dot product.

```
RISCV_DSP_ATTRIBUTE float32_t riscv_logsumexp_dot_prod_f32 (const float32_t *pSrcA,
const float32_t *pSrcB, uint32_t blockSize, float32_t *pTmpBuffer)
```

Dot product with log arithmetic.

Vectors are containing the log of the samples

Parameters

- ***pSrcA** – [in] points to the first input vector
- ***pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in each vector
- ***pTmpBuffer** – [in] temporary buffer of length blockSize

Returns

The log of the dot product.

RISCV_DSP_ATTRIBUTE float16_t riscv_logsumexp_f16 (const float16_t *in, uint32_t blockSize)

Computation of the LogSumExp.

In probabilistic computations, the dynamic of the probability values can be very wide because they come from gaussian functions. To avoid underflow and overflow issues, the values are represented by their log. In this representation, multiplying the original exp values is easy : their logs are added. But adding the original exp values is requiring some special handling and it is the goal of the LogSumExp function.

If the values are $x_1 \dots x_n$, the function is computing:

$\ln(\exp(x_1) + \dots + \exp(x_n))$ and the computation is done in such a way that rounding issues are minimised.

The max x_m of the values is extracted and the function is computing: $x_m + \ln(\exp(x_1 - x_m) + \dots + \exp(x_n - x_m))$

Parameters

- ***in** – [in] Pointer to an array of input values.
- **blockSize** – [in] Number of samples in the input array.

Returns

LogSumExp

RISCV_DSP_ATTRIBUTE float32_t riscv_logsumexp_f32 (const float32_t *in, uint32_t blockSize)

Computation of the LogSumExp.

In probabilistic computations, the dynamic of the probability values can be very wide because they come from gaussian functions. To avoid underflow and overflow issues, the values are represented by their log. In this representation, multiplying the original exp values is easy : their logs are added. But adding the original exp values is requiring some special handling and it is the goal of the LogSumExp function.

If the values are $x_1 \dots x_n$, the function is computing:

$\ln(\exp(x_1) + \dots + \exp(x_n))$ and the computation is done in such a way that rounding issues are minimised.

The max x_m of the values is extracted and the function is computing: $x_m + \ln(\exp(x_1 - x_m) + \dots + \exp(x_n - x_m))$

Parameters

- ***in** – [in] Pointer to an array of input values.
- **blockSize** – [in] Number of samples in the input array.

Returns

LogSumExp

Maximum

RISCV_DSP_ATTRIBUTE void riscv_max_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult, uint32_t *pIndex)

RISCV_DSP_ATTRIBUTE void riscv_max_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult, uint32_t *pIndex)

```
RISCV_DSP_ATTRIBUTE void riscv_max_f64 (const float64_t *pSrc, uint32_t blockSize,
float64_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize,
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_max_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult, uint32_t *pIndex)
```

group **Maximum**

Computes the maximum value of an array of data. The function returns both the maximum value and its position within the array. There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_max_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult, uint32_t *pIndex)
```

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult, uint32_t *pIndex)

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult, uint32_t *pIndex)

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Maximum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector

- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Maximum value of a q15 vector without index.

Maximum value of a q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult)

Maximum value of a q31 vector without index.

Maximum value of a q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult)

Maximum value of a q7 vector without index.

Maximum value of a q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult, uint32_t *pIndex)

Maximum value of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

RISCV_DSP_ATTRIBUTE void riscv_max_q31 (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult, uint32_t *pIndex)

Maximum value of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_max_q7 (const q7_t *pSrc, uint32_t blockSize,  
q7_t *pResult, uint32_t *pIndex)
```

Maximum value of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] maximum value returned here
- **pIndex** – [out] index of maximum value returned here

Mean

```
RISCV_DSP_ATTRIBUTE void riscv_mean_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mean_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mean_f64 (const float64_t *pSrc, uint32_t blockSize,  
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mean_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mean_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mean_q7 (const q7_t *pSrc, uint32_t blockSize,  
q7_t *pResult)
```

group **Mean**

Calculates the mean of the input vector. Mean is defined as the average of the elements in the vector. The underlying algorithm is used:

$$\text{Result} = (\text{pSrc}[0] + \text{pSrc}[1] + \text{pSrc}[2] + \dots + \text{pSrc}[\text{blockSize}-1]) / \text{blockSize};$$

There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_mean_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Mean value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] mean value returned here.

RISCV_DSP_ATTRIBUTE void riscv_mean_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Mean value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] mean value returned here.

RISCV_DSP_ATTRIBUTE void riscv_mean_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Mean value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] mean value returned here.

RISCV_DSP_ATTRIBUTE void riscv_mean_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Mean value of a Q15 vector.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. The input is represented in 1.15 format and is accumulated in a 32-bit accumulator in 17.15 format. There is no risk of internal overflow with this approach, and the full precision of intermediate result is preserved. Finally, the accumulator is truncated to yield a result of 1.15 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_mean_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

Mean value of a Q31 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.31 format and is accumulated in a 64-bit accumulator in 33.31 format. There is no risk of internal overflow with this approach, and the full precision of intermediate result is preserved. Finally, the accumulator is truncated to yield a result of 1.31 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_mean_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult)
```

Mean value of a Q7 vector.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. The input is represented in 1.7 format and is accumulated in a 32-bit accumulator in 25.7 format. There is no risk of internal overflow with this approach, and the full precision of intermediate result is preserved. Finally, the accumulator is truncated to yield a result of 1.7 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean value returned here

Minimum

```
RISCV_DSP_ATTRIBUTE void riscv_min_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_f64 (const float64_t *pSrc, uint32_t blockSize,
float64_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize,
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult, uint32_t *pIndex)
```

```
RISCV_DSP_ATTRIBUTE void riscv_min_q7 (const q7_t *pSrc, uint32_t blockSize,
q7_t *pResult, uint32_t *pIndex)
```

group **Minimum**

Computes the minimum value of an array of data. The function returns both the minimum value and its position within the array. There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_min_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult, uint32_t *pIndex)
```

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_min_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult, uint32_t *pIndex)
```

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector

- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_min_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult, uint32_t *pIndex)

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Maximum value of a floating-point vector.

Minimum value of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void riscv_min_no_idx_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Minimum value of a q15 vector without index.

Minimum value of a q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void **riscv_min_no_idx_q31** (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult)

Minimum value of a q31 vector without index.

Minimum value of a q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void **riscv_min_no_idx_q7** (const q7_t *pSrc, uint32_t blockSize, q7_t *pResult)

Minimum value of a q7 vector without index.

Minimum value of a q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here

RISCV_DSP_ATTRIBUTE void **riscv_min_q15** (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult, uint32_t *pIndex)

Minimum value of a Q15 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

RISCV_DSP_ATTRIBUTE void **riscv_min_q31** (const q31_t *pSrc, uint32_t blockSize, q31_t *pResult, uint32_t *pIndex)

Minimum value of a Q31 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_min_q7 (const q7_t *pSrc, uint32_t blockSize,  
q7_t *pResult, uint32_t *pIndex)
```

Minimum value of a Q7 vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] minimum value returned here
- **pIndex** – [out] index of minimum value returned here

Mean Square Error

```
RISCV_DSP_ATTRIBUTE void riscv_mse_f16 (const float16_t *pSrcA, const float16_t *pSrcB,  
uint32_t blockSize, float16_t *result)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mse_f32 (const float32_t *pSrcA, const float32_t *pSrcB,  
uint32_t blockSize, float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mse_f64 (const float64_t *pSrcA, const float64_t *pSrcB,  
uint32_t blockSize, float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mse_q15 (const q15_t *pSrcA, const q15_t *pSrcB,  
uint32_t blockSize, q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mse_q31 (const q31_t *pSrcA, const q31_t *pSrcB,  
uint32_t blockSize, q31_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_mse_q7 (const q7_t *pSrcA, const q7_t *pSrcB,  
uint32_t blockSize, q7_t *pResult)
```

group Mean Square Error

Calculates the mean square error between two vectors.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_mse_f16 (const float16_t *pSrcA,  
const float16_t *pSrcB, uint32_t blockSize, float16_t *result)
```

Mean square error between two half floating point vectors.

Mean square error between two half precision float vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector

- **result** – [out] mean square error

RISCV_DSP_ATTRIBUTE void riscv_mse_f32 (const float32_t *pSrcA, const float32_t *pSrcB, uint32_t blockSize, float32_t *pResult)

Mean square error between two floating point vectors.

Mean square error between two single precision float vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean square error

RISCV_DSP_ATTRIBUTE void riscv_mse_f64 (const float64_t *pSrcA, const float64_t *pSrcB, uint32_t blockSize, float64_t *pResult)

Mean square error between two double floating point vectors.

Mean square error between two double precision float vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean square error

RISCV_DSP_ATTRIBUTE void riscv_mse_q15 (const q15_t *pSrcA, const q15_t *pSrcB, uint32_t blockSize, q15_t *pResult)

Mean square error between two Q15 vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean square error

RISCV_DSP_ATTRIBUTE void riscv_mse_q31 (const q31_t *pSrcA, const q31_t *pSrcB, uint32_t blockSize, q31_t *pResult)

Mean square error between two Q31 vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean square error

```
RISCV_DSP_ATTRIBUTE void riscv_mse_q7 (const q7_t *pSrcA, const q7_t *pSrcB,
uint32_t blockSize, q7_t *pResult)
```

Mean square error between two Q7 vectors.

Parameters

- **pSrcA** – [in] points to the first input vector
- **pSrcB** – [in] points to the second input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] mean square error

Power

```
RISCV_DSP_ATTRIBUTE void riscv_power_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_power_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_power_f64 (const float64_t *pSrc, uint32_t blockSize,
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_power_q15 (const q15_t *pSrc, uint32_t blockSize,
q63_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_power_q31 (const q31_t *pSrc, uint32_t blockSize,
q63_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_power_q7 (const q7_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

group Power

Calculates the sum of the squares of the elements in the input vector. The underlying algorithm is used:

```
Result = pSrc[0] * pSrc[0] + pSrc[1] * pSrc[1] + pSrc[2] * pSrc[2] + ... +
↪pSrc[blockSize-1] * pSrc[blockSize-1];
```

There are separate functions for floating point, Q31, Q15, and Q7 data types.

Since the result is not divided by the length, those functions are in fact computing something which is more an energy than a power.

Functions

RISCV_DSP_ATTRIBUTE void riscv_power_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Sum of the squares of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

RISCV_DSP_ATTRIBUTE void riscv_power_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Sum of the squares of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

RISCV_DSP_ATTRIBUTE void riscv_power_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Sum of the squares of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

RISCV_DSP_ATTRIBUTE void riscv_power_q15 (const q15_t *pSrc, uint32_t blockSize, q63_t *pResult)

Sum of the squares of the elements of a Q15 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.15 format. Intermediate multiplication yields a 2.30 format, and this result is added without saturation to a 64-bit accumulator in 34.30 format. With 33 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the return result is in 34.30 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_power_q31 (const q31_t *pSrc, uint32_t blockSize,  
q63_t *pResult)
```

Sum of the squares of the elements of a Q31 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.31 format. Intermediate multiplication yields a 2.62 format, and this result is truncated to 2.48 format by discarding the lower 14 bits. The 2.48 result is then added without saturation to a 64-bit accumulator in 16.48 format. With 15 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the return result is in 16.48 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_power_q7 (const q7_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

Sum of the squares of the elements of a Q7 vector.

Scaling and Overflow Behavior

The function is implemented using a 32-bit internal accumulator. The input is represented in 1.7 format. Intermediate multiplication yields a 2.14 format, and this result is added without saturation to an accumulator in 18.14 format. With 17 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the return result is in 18.14 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] sum of the squares value returned here

Root mean square (RMS)

```
RISCV_DSP_ATTRIBUTE void riscv_rms_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_rms_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_rms_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_rms_q31 (const q31_t *pSrc, uint32_t blockSize,
q31_t *pResult)
```

group **Root mean square (RMS)**

Calculates the Root Mean Square of the elements in the input vector. The underlying algorithm is used:

```
Result = sqrt(((pSrc[0] * pSrc[0] + pSrc[1] * pSrc[1] + ... + pSrc[blockSize-1] *
↪pSrc[blockSize-1]) / blockSize));
```

There are separate functions for floating point, Q31, and Q15 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_rms_f16 (const float16_t *pSrc, uint32_t blockSize,
float16_t *pResult)
```

Root Mean Square of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] root mean square value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_rms_f32 (const float32_t *pSrc, uint32_t blockSize,
float32_t *pResult)
```

Root Mean Square of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] root mean square value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_rms_q15 (const q15_t *pSrc, uint32_t blockSize,
q15_t *pResult)
```

Root Mean Square of the elements of a Q15 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.15 format. Intermediate multiplication yields a 2.30 format, and this result is added without saturation to a 64-bit accumulator in 34.30 format. With 33 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the 34.30 result is truncated to 34.15 format by discarding the lower 15 bits, and then saturated to yield a result in 1.15 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector

- **pResult** – [out] root mean square value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_rms_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

Root Mean Square of the elements of a Q31 vector.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The input is represented in 1.31 format, and intermediate multiplication yields a 2.62 format. The accumulator maintains full precision of the intermediate multiplication results, but provides only a single guard bit. There is no saturation on intermediate additions. If the accumulator overflows, it wraps around and distorts the result. In order to avoid overflows completely, the input signal must be scaled down by $\log_2(\text{blockSize})$ bits, as a total of blockSize additions are performed internally. Finally, the 2.62 accumulator is right shifted by 31 bits to yield a 1.31 format value.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] root mean square value returned here

Standard deviation

```
RISCV_DSP_ATTRIBUTE void riscv_std_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_std_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_std_f64 (const float64_t *pSrc, uint32_t blockSize,  
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_std_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_std_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

group Standard deviation

Calculates the standard deviation of the elements in the input vector.

The float implementation is relying on `riscv_var_f32` which is using a two-pass algorithm to avoid problem of numerical instabilities and cancellation errors.

Fixed point versions are using the standard textbook algorithm since the fixed point numerical behavior is different from the float one.

Algorithm for fixed point versions is summarized below:

```

Result = sqrt((sumOfSquares - sum2 / blockSize) / (blockSize - 1))

sumOfSquares = pSrc[0] * pSrc[0] + pSrc[1] * pSrc[1] + ... + pSrc[blockSize-1] *
↪pSrc[blockSize-1]
sum = pSrc[0] + pSrc[1] + pSrc[2] + ... + pSrc[blockSize-1]

```

There are separate functions for floating point, Q31, and Q15 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_std_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Standard deviation of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] standard deviation value returned here

RISCV_DSP_ATTRIBUTE void riscv_std_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Standard deviation of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] standard deviation value returned here

RISCV_DSP_ATTRIBUTE void riscv_std_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Standard deviation of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] standard deviation value returned here

RISCV_DSP_ATTRIBUTE void riscv_std_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Standard deviation of the elements of a Q15 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.15 format. Intermediate multiplication yields a 2.30 format, and this result is added without saturation to a 64-bit accumulator in 34.30 format. With 33 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the 34.30

result is truncated to 34.15 format by discarding the lower 15 bits, and then saturated to yield a result in 1.15 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] standard deviation value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_std_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

Standard deviation of the elements of a Q31 vector.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The input is represented in 1.31 format, which is then downshifted by 8 bits which yields 1.23, and intermediate multiplication yields a 2.46 format. The accumulator maintains full precision of the intermediate multiplication results, but provides only a 16 guard bits. There is no saturation on intermediate additions. If the accumulator overflows it wraps around and distorts the result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{blockSize}) - 8$ bits, as a total of blockSize additions are performed internally. After division, internal variables should be Q18.46. Finally, the 18.46 accumulator is right shifted by 15 bits to yield a 1.31 format value.

Parameters

- **pSrc** – [in] points to the input vector.
- **blockSize** – [in] number of samples in input vector.
- **pResult** – [out] standard deviation value returned here.

Variance

```
RISCV_DSP_ATTRIBUTE void riscv_var_f16 (const float16_t *pSrc, uint32_t blockSize,  
float16_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_var_f32 (const float32_t *pSrc, uint32_t blockSize,  
float32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_var_f64 (const float64_t *pSrc, uint32_t blockSize,  
float64_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_var_q15 (const q15_t *pSrc, uint32_t blockSize,  
q15_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_var_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

group **Variance**

Calculates the variance of the elements in the input vector. The underlying algorithm used is the direct method sometimes referred to as the two-pass method:

```
Result = sum(element - meanOfElements)^2) / numElement - 1
meanOfElements = ( pSrc[0] * pSrc[0] + pSrc[1] * pSrc[1] + ... + pSrc[blockSize-1]
↪) / blockSize
```

There are separate functions for floating point, Q31, and Q15 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_var_f16 (const float16_t *pSrc, uint32_t blockSize, float16_t *pResult)

Variance of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] variance value returned here

RISCV_DSP_ATTRIBUTE void riscv_var_f32 (const float32_t *pSrc, uint32_t blockSize, float32_t *pResult)

Variance of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] variance value returned here

RISCV_DSP_ATTRIBUTE void riscv_var_f64 (const float64_t *pSrc, uint32_t blockSize, float64_t *pResult)

Variance of the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] variance value returned here

RISCV_DSP_ATTRIBUTE void riscv_var_q15 (const q15_t *pSrc, uint32_t blockSize, q15_t *pResult)

Variance of the elements of a Q15 vector.

Scaling and Overflow Behavior

The function is implemented using a 64-bit internal accumulator. The input is represented in 1.15 format. Intermediate multiplication yields a 2.30 format, and this result is added without saturation to a 64-bit accumulator in 34.30 format. With 33 guard bits in the accumulator, there is no risk of overflow, and the full precision of the intermediate multiplication is preserved. Finally, the 34.30 result is truncated to 34.15 format by discarding the lower 15 bits, and then saturated to yield a result in 1.15 format.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] variance value returned here

```
RISCV_DSP_ATTRIBUTE void riscv_var_q31 (const q31_t *pSrc, uint32_t blockSize,  
q31_t *pResult)
```

Variance of the elements of a Q31 vector.

Scaling and Overflow Behavior

The function is implemented using an internal 64-bit accumulator. The input is represented in 1.31 format, which is then downshifted by 8 bits which yields 1.23, and intermediate multiplication yields a 2.46 format. The accumulator maintains full precision of the intermediate multiplication results, and as a consequence has only 16 guard bits. There is no saturation on intermediate additions. If the accumulator overflows it wraps around and distorts the result. In order to avoid overflows completely the input signal must be scaled down by $\log_2(\text{blockSize}) - 8$ bits, as a total of blockSize additions are performed internally. After division, internal variables should be Q18.46. Finally, the 18.46 accumulator is right shifted by 15 bits to yield a 1.31 format value.

Parameters

- **pSrc** – [in] points to the input vector
- **blockSize** – [in] number of samples in input vector
- **pResult** – [out] variance value returned here

group **Statistics Functions****3.3.13 Support Functions****Typecasting**

```
__STATIC_INLINE int16_t riscv_typecast_s16_f16 (float16_t x)
```

```
__STATIC_INLINE float16_t riscv_typecast_f16_s16 (int16_t x)
```

group **Typecasting**

Functions

__STATIC_INLINE int16_t riscv_typecast_s16_f16 (float16_t x)

Interpret a f16 as an s16 value.

Description

It is a typecast. No conversion of the float to int is done. The memcpy will be optimized out by the compiler. memcpy is used to prevent type punning issues. With gcc, -fno-builtins MUST not be used or the memcpy will not be optimized out.

Parameters

x – [in] input value.

Returns

return value.

__STATIC_INLINE float16_t riscv_typecast_f16_s16 (int16_t x)

Interpret an s16 as an f16 value.

Description

It is a typecast. No conversion of the int to float is done. The memcpy will be optimized out by the compiler. memcpy is used to prevent type punning issues. With gcc, -fno-builtins MUST not be used or the memcpy will not be optimized out.

Parameters

x – [in] input value.

Returns

return value.

Barycenter

**RISCV_DSP_ATTRIBUTE void riscv_barycenter_f32 (const float32_t *in,
const float32_t *weights, float32_t *out, uint32_t nbVectors, uint32_t vecDim)**

**RISCV_DSP_ATTRIBUTE void riscv_barycenter_f16 (const float16_t *in,
const float16_t *weights, float16_t *out, uint32_t nbVectors, uint32_t vecDim)**

group Barycenter

Barycenter of weighted vectors

Unnamed Group

```
RISCV_DSP_ATTRIBUTE void riscv_barycenter_f32 (const float32_t *in,  
const float32_t *weights, float32_t *out, uint32_t nbVectors, uint32_t vecDim)
```

Barycenter.

Parameters

- ***in** – [in] List of vectors
- ***weights** – [in] Weights of the vectors
- ***out** – [out] Barycenter
- **nbVectors** – [in] Number of vectors
- **vecDim** – [in] Dimension of space (vector dimension)

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_barycenter_f16 (const float16_t *in,  
const float16_t *weights, float16_t *out, uint32_t nbVectors, uint32_t vecDim)
```

Barycenter.

Parameters

- ***in** – [in] List of vectors
- ***weights** – [in] Weights of the vectors
- ***out** – [out] Barycenter
- **nbVectors** – [in] Number of vectors
- **vecDim** – [in] Dimension of space (vector dimension)

Vector sorting algorithms

```
RISCV_DSP_ATTRIBUTE void riscv_merge_sort_f32 (const riscv_merge_sort_instance_f32 *S,  
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_merge_sort_init_f32 (riscv_merge_sort_instance_f32 *S,  
riscv_sort_dir dir, float32_t *buffer)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sort_f32 (const riscv_sort_instance_f32 *S,  
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_sort_init_f32 (riscv_sort_instance_f32 *S,  
riscv_sort_alg alg, riscv_sort_dir dir)
```

group Vector sorting algorithms

Sort the elements of a vector

There are separate functions for floating-point, Q31, Q15, and Q7 data types.

Functions

RISCV_DSP_ATTRIBUTE void riscv_bitonic_sort_f32 (const riscv_sort_instance_f32 *S, float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

RISCV_DSP_ATTRIBUTE void riscv_bubble_sort_f32 (const riscv_sort_instance_f32 *S, float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Algorithm

The bubble sort algorithm is a simple comparison algorithm that reads the elements of a vector from the beginning to the end, compares the adjacent ones and swaps them if they are in the wrong order. The procedure is repeated until there is nothing left to swap. Bubble sort is fast for input vectors that are nearly sorted.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

RISCV_DSP_ATTRIBUTE void riscv_heap_sort_f32 (const riscv_sort_instance_f32 *S, float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Algorithm

The heap sort algorithm is a comparison algorithm that divides the input array into a sorted and an unsorted region, and shrinks the unsorted region by extracting the largest element and moving it to the sorted region. A heap data structure is used to find the maximum.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed.

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_insertion_sort_f32 (const riscv_sort_instance_f32 *S,  
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

Algorithm

The insertion sort is a simple sorting algorithm that reads all the element of the input array and removes one element at a time, finds the location it belongs in the final sorted list, and inserts it there.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed.

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_merge_sort_f32 (const riscv_merge_sort_instance_f32 *S,  
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

Algorithm

The merge sort algorithm is a comparison algorithm that divide the input array in sublists and merge them to produce longer sorted sublists until there is only one list remaining.

A work array is always needed. It must be allocated by the user linked to the instance at initialization time.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_merge_sort_init_f32 (riscv_merge_sort_instance_f32 *S,  
riscv_sort_dir dir, float32_t *buffer)
```

Parameters

- **S** – [inout] points to an instance of the sorting structure.
- **dir** – [in] Sorting order.
- **buffer** – [in] Working buffer.

```
RISCV_DSP_ATTRIBUTE void riscv_quick_sort_f32 (const riscv_sort_instance_f32 *S,  
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

Algorithm

The quick sort algorithm is a comparison algorithm that divides the input array into two smaller sub-arrays and recursively sort them. An element of the array (the pivot) is chosen, all the elements with

values smaller than the pivot are moved before the pivot, while all elements with values greater than the pivot are moved after it (partition).

In this implementation the Hoare partition scheme has been used [Hoare, C. A. R. (1 January 1962). “Quicksort”. The Computer Journal. 5 (1): 10...16.] The first element has always been chosen as the pivot. The partition algorithm guarantees that the returned pivot is never placed outside the vector, since it is returned only when the pointers crossed each other. In this way it isn't possible to obtain empty partitions and infinite recursion is avoided.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed.

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [inout] points to the block of input data.
- **pDst** – [out] points to the block of output data.
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_selection_sort_f32 (const riscv_sort_instance_f32 *S,
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

Algorithm

The Selection sort algorithm is a comparison algorithm that divides the input array into a sorted and an unsorted sublist (initially the sorted sublist is empty and the unsorted sublist is the input array), looks for the smallest (or biggest) element in the unsorted sublist, swapping it with the leftmost one, and moving the sublists boundary one element to the right.

It's an in-place algorithm. In order to obtain an out-of-place function, a memcpy of the source vector is performed.

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_sort_f32 (const riscv_sort_instance_f32 *S,
float32_t *pSrc, float32_t *pDst, uint32_t blockSize)
```

Generic sorting function.

Parameters

- **S** – [in] points to an instance of the sorting structure.
- **pSrc** – [in] points to the block of input data.
- **pDst** – [out] points to the block of output data.
- **blockSize** – [in] number of samples to process.

```
RISCV_DSP_ATTRIBUTE void riscv_sort_init_f32 (riscv_sort_instance_f32 *S,  
riscv_sort_alg alg, riscv_sort_dir dir)
```

Parameters

- **S** – [inout] points to an instance of the sorting structure.
- **alg** – [in] Selected algorithm.
- **dir** – [in] Sorting order.

Vector Copy

```
RISCV_DSP_ATTRIBUTE void riscv_copy_f16 (const float16_t *pSrc, float16_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_copy_f32 (const float32_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_copy_f64 (const float64_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_copy_q15 (const q15_t *pSrc, q15_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_copy_q31 (const q31_t *pSrc, q31_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_copy_q7 (const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)
```

group Vector Copy

Copies sample by sample from source vector to destination vector.

```
pDst[n] = pSrc[n];    0 <= n < blockSize.
```

There are separate functions for floating point, Q31, Q15, and Q7 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_copy_f16 (const float16_t *pSrc, float16_t *pDst,  
uint32_t blockSize)
```

Copies the elements of a f16 vector.

Copies the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_copy_f32 (const float32_t *pSrc, float32_t *pDst, uint32_t blockSize)

Copies the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_copy_f64 (const float64_t *pSrc, float64_t *pDst, uint32_t blockSize)

Copies the elements of a floating-point vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_copy_q15 (const q15_t *pSrc, q15_t *pDst, uint32_t blockSize)

Copies the elements of a Q15 vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_copy_q31 (const q31_t *pSrc, q31_t *pDst, uint32_t blockSize)

Copies the elements of a Q31 vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_copy_q7 (const q7_t *pSrc, q7_t *pDst, uint32_t blockSize)

Copies the elements of a Q7 vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Convert 16-bit floating point value

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_f64 (const float16_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_float (const float16_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_q15 (const float16_t *pSrc, q15_t *pDst,  
uint32_t blockSize)
```

group Convert 16-bit floating point value

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_f64 (const float16_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

Converts the elements of the f16 vector to f64 vector.

Converts the elements of the 16 bit floating-point vector to 64 bit floating-point vector.

Parameters

- **pSrc** – [in] points to the f16 input vector
- **pDst** – [out] points to the f64 output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_float (const float16_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

Converts the elements of the f16 vector to f32 vector.

Converts the elements of the floating-point vector to Q31 vector.

Parameters

- **pSrc** – [in] points to the f16 input vector
- **pDst** – [out] points to the f32 output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_f16_to_q15 (const float16_t *pSrc, q15_t *pDst,  
uint32_t blockSize)
```

Converts the elements of the f16 vector to Q15 vector.

Converts the elements of the floating-point vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q15_t)(pSrc[n] * 32768); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Note

In order to apply rounding in scalar version, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- **pSrc** – [in] points to the f16 input vector
- **pDst** – [out] points to the Q15 output vector
- **blockSize** – [in] number of samples in each vector

Convert 64-bit floating point value

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_f16 (const float64_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_float (const float64_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_q15 (const float64_t *pSrc, q15_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_q31 (const float64_t *pSrc, q31_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_q7 (const float64_t *pSrc, q7_t *pDst,
uint32_t blockSize)
```

group **Convert 64-bit floating point value**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_f64_to_f16 (const float64_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

Converts the elements of the f64 vector to f16 vector.

Converts the elements of the 64 bit floating-point vector to 16 bit floating-point vector.

Parameters

- **pSrc** – [in] points to the f64 input vector
- **pDst** – [out] points to the f16 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_f64_to_float (const float64_t *pSrc, float32_t *pDst, uint32_t blockSize)

Converts the elements of the f64 vector to f32 vector.

Converts the elements of the 64 bit floating-point vector to floating-point vector.

Parameters

- **pSrc** – [in] points to the f64 input vector
- **pDst** – [out] points to the f32 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_f64_to_q15 (const float64_t *pSrc, q15_t *pDst, uint32_t blockSize)

Converts the elements of the 64 bit floating-point vector to Q15 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q15_t)(pSrc[n] * 32768); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- **pSrc** – [in] points to the 64 bit floating-point input vector
- **pDst** – [out] points to the Q15 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_f64_to_q31 (const float64_t *pSrc, q31_t *pDst, uint32_t blockSize)

Converts the elements of the 64 bit floating-point vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q31_t)(pSrc[n] * 2147483648); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range [0x80000000 0x7FFFFFFF] are saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- **pSrc** – [in] points to the 64 bit floating-point input vector
- **pDst** – [out] points to the Q31 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_f64_to_q7 (const float64_t *pSrc, q7_t *pDst, uint32_t blockSize)

Converts the elements of the 64 bit floating-point vector to Q7 vector.

Description:

The equation used for the conversion process is:

$$pDst[n] = (q7_t)(pSrc[n] * 128); \quad 0 \leq n < blockSize.$$
Scaling and Overflow Behavior:

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] will be saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- ***pSrc** – [in] points to the 64 bit floating-point input vector
- ***pDst** – [out] points to the Q7 output vector
- **blockSize** – [in] length of the input vector

Vector Fill

RISCV_DSP_ATTRIBUTE void riscv_fill_f16 (float16_t value, float16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_fill_f32 (float32_t value, float32_t *pDst, uint32_t blockSize)

```
RISCV_DSP_ATTRIBUTE void riscv_fill_f64 (float64_t value, float64_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fill_q15 (q15_t value, q15_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fill_q31 (q31_t value, q31_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_fill_q7 (q7_t value, q7_t *pDst, uint32_t blockSize)
```

group **Vector Fill**

Fills the destination vector with a constant value.

```
pDst[n] = value;    0 <= n < blockSize.
```

There are separate functions for floating point, Q31, Q15, and Q7 data types.

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_fill_f16 (float16_t value, float16_t *pDst,  
uint32_t blockSize)
```

Fills a constant value into a f16 vector.

Fills a constant value into a floating-point vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_fill_f32 (float32_t value, float32_t *pDst,  
uint32_t blockSize)
```

Fills a constant value into a floating-point vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_fill_f64 (float64_t value, float64_t *pDst,  
uint32_t blockSize)
```

Fills a constant value into a floating-point vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_fill_q15 (q15_t value, q15_t *pDst, uint32_t blockSize)

Fills a constant value into a Q15 vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_fill_q31 (q31_t value, q31_t *pDst, uint32_t blockSize)

Fills a constant value into a Q31 vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_fill_q7 (q7_t value, q7_t *pDst, uint32_t blockSize)

Fills a constant value into a Q7 vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Convert 32-bit floating point value

RISCV_DSP_ATTRIBUTE void riscv_float_to_f16 (const float32_t *pSrc, float16_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_float_to_f64 (const float32_t *pSrc, float64_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_float_to_q15 (const float32_t *pSrc, q15_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_float_to_q31 (const float32_t *pSrc, q31_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_float_to_q7 (const float32_t *pSrc, q7_t *pDst, uint32_t blockSize)

group Convert 32-bit floating point value

Functions

RISCV_DSP_ATTRIBUTE void riscv_float_to_f16 (const float32_t *pSrc, float16_t *pDst, uint32_t blockSize)

Converts the elements of the floating-point vector to f16 vector.

Converts the elements of the floating-point vector to Q31 vector.

Parameters

- **pSrc** – [in] points to the f32 input vector
- **pDst** – [out] points to the f16 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_float_to_f64 (const float32_t *pSrc, float64_t *pDst, uint32_t blockSize)

Converts the elements of the floating-point vector to f64 vector.

Converts the elements of the floating-point vector to 64 bit floating-point vector.

Parameters

- **pSrc** – [in] points to the f32 input vector
- **pDst** – [out] points to the f64 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_float_to_q15 (const float32_t *pSrc, q15_t *pDst, uint32_t blockSize)

Converts the elements of the floating-point vector to Q15 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q15_t)(pSrc[n] * 32768); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] are saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- **pSrc** – [in] points to the floating-point input vector
- **pDst** – [out] points to the Q15 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_float_to_q31 (const float32_t *pSrc, q31_t *pDst, uint32_t blockSize)

Converts the elements of the floating-point vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q31_t)(pSrc[n] * 2147483648); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior

The function uses saturating arithmetic. Results outside of the allowable Q31 range[0x80000000 0x7FFFFFFF] are saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Note

If the input float values are very big (2^{32}) then the function won't be able to saturate to the right values. If you expect very big float values in the input array then you should force those values to +1 or -1 before calling this function. For reasonable float values ($< 2^{32}$), the function will saturate correctly.

Parameters

- **pSrc** – [in] points to the floating-point input vector
- **pDst** – [out] points to the Q31 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_float_to_q7 (const float32_t *pSrc, q7_t *pDst, uint32_t blockSize)

Converts the elements of the floating-point vector to Q7 vector.

Description:

The equation used for the conversion process is:

$$pDst[n] = (q7_t)(pSrc[n] * 128); \quad 0 \leq n < blockSize.$$

Scaling and Overflow Behavior:

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] will be saturated.

Note

In order to apply rounding, the library should be rebuilt with the ROUNDING macro defined in the preprocessor section of project options.

Parameters

- ***pSrc** – [in] points to the floating-point input vector
- ***pDst** – [out] points to the Q7 output vector
- **blockSize** – [in] length of the input vector

Convert 16-bit fixed point value

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_f16 (const q15_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_f64 (const q15_t *pSrc, float64_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_float (const q15_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_q31 (const q15_t *pSrc, q31_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_q7 (const q15_t *pSrc, q7_t *pDst,
uint32_t blockSize)
```

group **Convert 16-bit fixed point value**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_f16 (const q15_t *pSrc, float16_t *pDst,
uint32_t blockSize)
```

Converts the elements of the Q15 vector to f16 vector.

Converts the elements of the floating-point vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float16_t}) \, pSrc[n] / 32768; \quad 0 \leq n < \text{blockSize}.$$
Parameters

- **pSrc** – [in] points to the Q15 input vector

- **pDst** – [out] points to the f16 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q15_to_f64 (const q15_t *pSrc, float64_t *pDst, uint32_t blockSize)

Converts the elements of the Q15 vector to 64 bit floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float64_t}) \, pSrc[n] / 32768; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q15 input vector
- **pDst** – [out] points to the 64 bit floating-point output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q15_to_float (const q15_t *pSrc, float32_t *pDst, uint32_t blockSize)

Converts the elements of the Q15 vector to floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float32_t}) \, pSrc[n] / 32768; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q15 input vector
- **pDst** – [out] points to the floating-point output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q15_to_q31 (const q15_t *pSrc, q31_t *pDst, uint32_t blockSize)

Converts the elements of the Q15 vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{q31_t}) \, pSrc[n] \ll 16; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q15 input vector
- **pDst** – [out] points to the Q31 output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_q15_to_q7 (const q15_t *pSrc, q7_t *pDst,
uint32_t blockSize)
```

Converts the elements of the Q15 vector to Q7 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q7_t) \ pSrc[n] \gg 8; \quad 0 \leq n < blockSize.$$

Parameters

- **pSrc** – [in] points to the Q15 input vector
- **pDst** – [out] points to the Q7 output vector
- **blockSize** – [in] number of samples in each vector

Convert 32-bit fixed point value

```
RISCV_DSP_ATTRIBUTE void riscv_q31_to_f64 (const q31_t *pSrc, float64_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q31_to_float (const q31_t *pSrc, float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q31_to_q15 (const q31_t *pSrc, q15_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q31_to_q7 (const q31_t *pSrc, q7_t *pDst,
uint32_t blockSize)
```

group Convert 32-bit fixed point value

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_q31_to_f64 (const q31_t *pSrc, float64_t *pDst,
uint32_t blockSize)
```

Converts the elements of the Q31 vector to 64 bit floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (float64_t) \ pSrc[n] / 2147483648; \quad 0 \leq n < blockSize.$$

Parameters

- **pSrc** – [in] points to the Q31 input vector

- **pDst** – [out] points to the 64 bit floating-point output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q31_to_float (const q31_t *pSrc, float32_t *pDst, uint32_t blockSize)

Converts the elements of the Q31 vector to floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float32_t}) \, pSrc[n] / 2147483648; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q31 input vector
- **pDst** – [out] points to the floating-point output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q31_to_q15 (const q31_t *pSrc, q15_t *pDst, uint32_t blockSize)

Converts the elements of the Q31 vector to Q15 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{q15_t}) \, pSrc[n] \gg 16; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q31 input vector
- **pDst** – [out] points to the Q15 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q31_to_q7 (const q31_t *pSrc, q7_t *pDst, uint32_t blockSize)

Converts the elements of the Q31 vector to Q7 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{q7_t}) \, pSrc[n] \gg 24; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q31 input vector
- **pDst** – [out] points to the Q7 output vector
- **blockSize** – [in] number of samples in each vector

Convert 8-bit fixed point value

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_f64 (const q7_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_float (const q7_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_q15 (const q7_t *pSrc, q15_t *pDst,  
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_q31 (const q7_t *pSrc, q31_t *pDst,  
uint32_t blockSize)
```

group Convert 8-bit fixed point value

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_f64 (const q7_t *pSrc, float64_t *pDst,  
uint32_t blockSize)
```

Converts the elements of the Q7 vector to 64 bit floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float64_t}) \text{ pSrc}[n] / 128; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q7 input vector
- **pDst** – [out] points to the 64 bit floating-point output vector
- **blockSize** – [in] number of samples in each vector

```
RISCV_DSP_ATTRIBUTE void riscv_q7_to_float (const q7_t *pSrc, float32_t *pDst,  
uint32_t blockSize)
```

Converts the elements of the Q7 vector to floating-point vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (\text{float32_t}) \text{ pSrc}[n] / 128; \quad 0 \leq n < \text{blockSize}.$$

Parameters

- **pSrc** – [in] points to the Q7 input vector
- **pDst** – [out] points to the floating-point output vector

- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q7_to_q15 (const q7_t *pSrc, q15_t *pDst, uint32_t blockSize)

Converts the elements of the Q7 vector to Q15 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q15_t) \ pSrc[n] \ll 8; \quad 0 \leq n < blockSize.$$

Parameters

- **pSrc** – [in] points to the Q7 input vector
- **pDst** – [out] points to the Q15 output vector
- **blockSize** – [in] number of samples in each vector

RISCV_DSP_ATTRIBUTE void riscv_q7_to_q31 (const q7_t *pSrc, q31_t *pDst, uint32_t blockSize)

Converts the elements of the Q7 vector to Q31 vector.

Details

The equation used for the conversion process is:

$$pDst[n] = (q31_t) \ pSrc[n] \ll 24; \quad 0 \leq n < blockSize.$$

Parameters

- **pSrc** – [in] points to the Q7 input vector
- **pDst** – [out] points to the Q31 output vector
- **blockSize** – [in] number of samples in each vector

Weighted Average

RISCV_DSP_ATTRIBUTE float16_t riscv_weighted_average_f16 (const float16_t *in, const float16_t *weights, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE float32_t riscv_weighted_average_f32 (const float32_t *in, const float32_t *weights, uint32_t blockSize)

group Weighted Average

Weighted average of values

Functions

RISCV_DSP_ATTRIBUTE float16_t riscv_weighted_average_f16 (const float16_t *in, const float16_t *weights, uint32_t blockSize)

Weighted average.

Parameters

- ***in** – [in] Array of input values.
- ***weights** – [in] Weights
- **blockSize** – [in] Number of samples in the input array.

Returns

Weighted average

RISCV_DSP_ATTRIBUTE float32_t riscv_weighted_average_f32 (const float32_t *in, const float32_t *weights, uint32_t blockSize)

Weighted average.

Parameters

- ***in** – [in] Array of input values.
- ***weights** – [in] Weights
- **blockSize** – [in] Number of samples in the input array.

Returns

Weighted average

group Support Functions

3.3.14 SVM Functions

Linear SVM

RISCV_DSP_ATTRIBUTE void riscv_svm_linear_init_f16 (riscv_svm_linear_instance_f16 *S, uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept, const float16_t *dualCoefficients, const float16_t *supportVectors, const int32_t *classes)

RISCV_DSP_ATTRIBUTE void riscv_svm_linear_init_f32 (riscv_svm_linear_instance_f32 *S, uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept, const float32_t *dualCoefficients, const float32_t *supportVectors, const int32_t *classes)

RISCV_DSP_ATTRIBUTE void riscv_svm_linear_predict_f16 (const riscv_svm_linear_instance_f16 *S, const float16_t *in, int32_t *pResult)

RISCV_DSP_ATTRIBUTE void riscv_svm_linear_predict_f32 (const riscv_svm_linear_instance_f32 *S, const float32_t *in, int32_t *pResult)

group **Linear SVM**

Linear SVM classifier

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_svm_linear_init_f16 (riscv_svm_linear_instance_f16 *S,
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,
const float16_t *dualCoefficients, const float16_t *supportVectors,
const int32_t *classes)
```

SVM linear instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] Parameters for the SVM function
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID

```
RISCV_DSP_ATTRIBUTE void riscv_svm_linear_init_f32 (riscv_svm_linear_instance_f32 *S,
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,
const float32_t *dualCoefficients, const float32_t *supportVectors,
const int32_t *classes)
```

SVM linear instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] Parameters for the SVM function
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID

```
RISCV_DSP_ATTRIBUTE void riscv_svm_linear_predict_f16 (const riscv_svm_linear_instance_f16 *S,
const float16_t *in, int32_t *pResult)
```

SVM linear prediction.

Parameters

- **S** – [in] Pointer to an instance of the linear SVM structure.

- **in** – [in] Pointer to input vector
- **pResult** – [out] Decision value

```
RISCV_DSP_ATTRIBUTE void riscv_svm_linear_predict_f32 (const riscv_svm_linear_instance_f32 *S,  
const float32_t *in, int32_t *pResult)
```

SVM linear prediction.

Parameters

- **S** – [in] Pointer to an instance of the linear SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] Decision value

Polynomial SVM

```
RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_init_f16 (riscv_svm_polynomial_instance_f16 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,  
const float16_t *dualCoefficients, const float16_t *supportVectors,  
const int32_t *classes, int32_t degree, float16_t coef0, float16_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_init_f32 (riscv_svm_polynomial_instance_f32 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,  
const float32_t *dualCoefficients, const float32_t *supportVectors,  
const int32_t *classes, int32_t degree, float32_t coef0, float32_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_predict_f16 (const riscv_svm_polynomial_instance_f16 *S,  
const float16_t *in, int32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_predict_f32 (const riscv_svm_polynomial_instance_f32 *S,  
const float32_t *in, int32_t *pResult)
```

group Polynomial SVM

Polynomial SVM classifier

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_init_f16 (riscv_svm_polynomial_instance_f16 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,  
const float16_t *dualCoefficients, const float16_t *supportVectors,  
const int32_t *classes, int32_t degree, float16_t coef0, float16_t gamma)
```

SVM polynomial instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the polynomial SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space

- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID
- **degree** – [in] Polynomial degree
- **coef0** – [in] coeff0 (scikit-learn terminology)
- **gamma** – [in] gamma (scikit-learn terminology)

RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_init_f32 (riscv_svm_polynomial_instance_f32 *S, uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept, const float32_t *dualCoefficients, const float32_t *supportVectors, const int32_t *classes, int32_t degree, float32_t coef0, float32_t gamma)

SVM polynomial instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the polynomial SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID
- **degree** – [in] Polynomial degree
- **coef0** – [in] coeff0 (scikit-learn terminology)
- **gamma** – [in] gamma (scikit-learn terminology)

RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_predict_f16 (const riscv_svm_polynomial_instance_f16 const float16_t *in, int32_t *pResult)

SVM polynomial prediction.

Parameters

- **S** – [in] Pointer to an instance of the polynomial SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] Decision value

RISCV_DSP_ATTRIBUTE void riscv_svm_polynomial_predict_f32 (const riscv_svm_polynomial_instance_f32 const float32_t *in, int32_t *pResult)

SVM polynomial prediction.

Parameters

- **S** – [in] Pointer to an instance of the polynomial SVM structure.
- **in** – [in] Pointer to input vector

- **pResult** – [out] Decision value

RBF SVM

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_init_f16 (riscv_svm_rbf_instance_f16 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,  
const float16_t *dualCoefficients, const float16_t *supportVectors,  
const int32_t *classes, float16_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_init_f32 (riscv_svm_rbf_instance_f32 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,  
const float32_t *dualCoefficients, const float32_t *supportVectors,  
const int32_t *classes, float32_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_predict_f16 (const riscv_svm_rbf_instance_f16 *S,  
const float16_t *in, int32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_predict_f32 (const riscv_svm_rbf_instance_f32 *S,  
const float32_t *in, int32_t *pResult)
```

group RBF SVM

RBFSVM classifier

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_init_f16 (riscv_svm_rbf_instance_f16 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,  
const float16_t *dualCoefficients, const float16_t *supportVectors,  
const int32_t *classes, float16_t gamma)
```

SVM radial basis function instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the polynomial SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID
- **gamma** – [in] gamma (scikit-learn terminology)

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_init_f32 (riscv_svm_rbf_instance_f32 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,  
const float32_t *dualCoefficients, const float32_t *supportVectors,  
const int32_t *classes, float32_t gamma)
```

SVM radial basis function instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the polynomial SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID
- **gamma** – [in] gamma (scikit-learn terminology)

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_predict_f16 (const riscv_svm_rbf_instance_f16 *S,
const float16_t *in, int32_t *pResult)
```

SVM rbf prediction.

Parameters

- **S** – [in] Pointer to an instance of the rbf SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] decision value

```
RISCV_DSP_ATTRIBUTE void riscv_svm_rbf_predict_f32 (const riscv_svm_rbf_instance_f32 *S,
const float32_t *in, int32_t *pResult)
```

SVM rbf prediction.

Parameters

- **S** – [in] Pointer to an instance of the rbf SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] decision value

Sigmoid SVM

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_init_f16 (riscv_svm_sigmoid_instance_f16 *S,
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,
const float16_t *dualCoefficients, const float16_t *supportVectors,
const int32_t *classes, float16_t coef0, float16_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_init_f32 (riscv_svm_sigmoid_instance_f32 *S,
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,
const float32_t *dualCoefficients, const float32_t *supportVectors,
const int32_t *classes, float32_t coef0, float32_t gamma)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_predict_f16 (const riscv_svm_sigmoid_instance_f16 *S,  
const float16_t *in, int32_t *pResult)
```

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_predict_f32 (const riscv_svm_sigmoid_instance_f32 *S,  
const float32_t *in, int32_t *pResult)
```

group **Sigmoid SVM**

Sigmoid SVM classifier

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_init_f16 (riscv_svm_sigmoid_instance_f16 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float16_t intercept,  
const float16_t *dualCoefficients, const float16_t *supportVectors,  
const int32_t *classes, float16_t coef0, float16_t gamma)
```

SVM sigmoid instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the rbf SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors
- **classes** – [in] Array of 2 classes ID
- **coef0** – [in] coef0 (scikit-learn terminology)
- **gamma** – [in] gamma (scikit-learn terminology)

```
RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_init_f32 (riscv_svm_sigmoid_instance_f32 *S,  
uint32_t nbOfSupportVectors, uint32_t vectorDimension, float32_t intercept,  
const float32_t *dualCoefficients, const float32_t *supportVectors,  
const int32_t *classes, float32_t coef0, float32_t gamma)
```

SVM sigmoid instance init function.

Classes are integer used as output of the function (instead of having -1,1 as class values).

Parameters

- **S** – [in] points to an instance of the rbf SVM structure.
- **nbOfSupportVectors** – [in] Number of support vectors
- **vectorDimension** – [in] Dimension of vector space
- **intercept** – [in] Intercept
- **dualCoefficients** – [in] Array of dual coefficients
- **supportVectors** – [in] Array of support vectors

- **classes** – [in] Array of 2 classes ID
- **coef0** – [in] coef0 (scikit-learn terminology)
- **gamma** – [in] gamma (scikit-learn terminology)

RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_predict_f16 (const riscv_svm_sigmoid_instance_f16 *S, const float16_t *in, int32_t *pResult)

SVM sigmoid prediction.

Parameters

- **S** – [in] Pointer to an instance of the rbf SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] Decision value

RISCV_DSP_ATTRIBUTE void riscv_svm_sigmoid_predict_f32 (const riscv_svm_sigmoid_instance_f32 *S, const float32_t *in, int32_t *pResult)

SVM sigmoid prediction.

Parameters

- **S** – [in] Pointer to an instance of the rbf SVM structure.
- **in** – [in] Pointer to input vector
- **pResult** – [out] Decision value

group SVM Functions

This set of functions is implementing SVM classification on 2 classes. The training must be done from scikit-learn. The parameters can be easily generated from the scikit-learn object. Some examples are given in DSP/Testing/PatternGeneration/SVM.py

If more than 2 classes are needed, the functions in this folder will have to be used, as building blocks, to do multi-class classification.

No multi-class classification is provided in this SVM folder.

3.3.15 Transform Functions

Complex FFT Functions

Complex FFT Tables

const uint16_t riscvBitRevTable[1024] RISCV_DSP_TABLE_ATTRIBUTE

const float16_t twiddleCoeff16_16[32] RISCV_DSP_TABLE_ATTRIBUTE

const uint64_t twiddleCoeff64_16[32]

const uint64_t twiddleCoeff64_32[64]

```
const uint64_t twiddleCoeff64_64[128]

const uint64_t twiddleCoeff64_128[256]

const uint64_t twiddleCoeff64_256[512]

const uint64_t twiddleCoeff64_512[1024]

const uint64_t twiddleCoeff64_1024[2048]

const uint64_t twiddleCoeff64_2048[4096]

const uint64_t twiddleCoeff64_4096[8192]

const float32_t (page 1277) twiddleCoef_16[32]

const float32_t (page 1277) twiddleCoef_32[64]

const float32_t (page 1277) twiddleCoef_64[128]

const float32_t (page 1277) twiddleCoef_128[256]

const float32_t (page 1277) twiddleCoef_256[512]

const float32_t (page 1277) twiddleCoef_512[1024]

const float32_t (page 1277) twiddleCoef_1024[2048]

const float32_t (page 1277) twiddleCoef_2048[4096]

const float32_t (page 1277) twiddleCoef_4096[8192]

const q31_t (page 1277) twiddleCoef_16_q31[24]

const q31_t (page 1277) twiddleCoef_32_q31[48]

const q31_t (page 1277) twiddleCoef_64_q31[96]

const q31_t (page 1277) twiddleCoef_128_q31[192]

const q31_t (page 1277) twiddleCoef_256_q31[384]
```

```
const q31_t (page 1277) twiddleCoef_512_q31[768]

const q31_t (page 1277) twiddleCoef_1024_q31[1536]

const q31_t (page 1277) twiddleCoef_2048_q31[3072]

const q31_t (page 1277) twiddleCoef_4096_q31[6144]

const q15_t (page 1277) twiddleCoef_16_q15[24]

const q15_t (page 1277) twiddleCoef_32_q15[48]

const q15_t (page 1277) twiddleCoef_64_q15[96]

const q15_t (page 1277) twiddleCoef_128_q15[192]

const q15_t (page 1277) twiddleCoef_256_q15[384]

const q15_t (page 1277) twiddleCoef_512_q15[768]

const q15_t (page 1277) twiddleCoef_1024_q15[1536]

const q15_t (page 1277) twiddleCoef_2048_q15[3072]

const q15_t (page 1277) twiddleCoef_4096_q15[6144]

const float16_t twiddleCoeffF16_32[64]

const float16_t twiddleCoeffF16_64[128]

const float16_t twiddleCoeffF16_128[256]

const float16_t twiddleCoeffF16_256[512]

const float16_t twiddleCoeffF16_512[1024]

const float16_t twiddleCoeffF16_1024[2048]

const float16_t twiddleCoeffF16_2048[4096]

const float16_t twiddleCoeffF16_4096[8192]
```

```
const float16_t twiddleCoeffF16_rfft_32[32]
```

```
const float16_t twiddleCoeffF16_rfft_64[64]
```

```
const float16_t twiddleCoeffF16_rfft_128[128]
```

```
const float16_t twiddleCoeffF16_rfft_256[256]
```

```
const float16_t twiddleCoeffF16_rfft_512[512]
```

```
const float16_t twiddleCoeffF16_rfft_1024[1024]
```

```
const float16_t twiddleCoeffF16_rfft_2048[2048]
```

```
const float16_t twiddleCoeffF16_rfft_4096[4096]
```

group **Complex FFT Tables**

Variables

```
const uint16_t riscvBitRevTable[1024] RISC_V_DSP_TABLE_ATTRIBUTE
```

Table for bit reversal process.

Q31 table for reciprocal.

q15 Twiddle factors Table

Q31 Twiddle factors Table.

Double Precision Floating-point Twiddle factors Table Generation.

end of CFFT_CIFFT group

Pseudo code for Generation of Bit reversal Table is

```
for (l = 1; l <= N/4; l++)
{
    for (i = 0; i < logN2; i++)
    {
        a[i] = l & (1 << i);
    }
    for (j = 0; j < logN2; j++)
    {
        if (a[j] != 0)
            y[l] += (1 << ((logN2 - 1) - j));
    }
}
```

(continues on next page)

(continued from previous page)

```

    }
    y[1] = y[1] >> 1;
}

```

where $N = 4096$, $\log_2 N = 12$

N is the maximum FFT Size supported

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/2; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(double)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(double)N);
}

```

where $N = 16$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/2; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/N);
}

```

where $N = 32$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/2; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 64$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)  
{  
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);  
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);  
}
```

where N = 128, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)  
{  
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);  
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);  
}
```

where N = 256, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)  
{  
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);  
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);  
}
```

where N = 512, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 16, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 64, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 128, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 256, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 512, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Q31 Twiddle factors Generation::

```
for(i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 16, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 64, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 128, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 256, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 512, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

Example code for q15 Twiddle factors Generation::

```
for i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 16, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 64, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 128, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 256, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 512, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

Example code for Double Precision Floating-point RFFT Twiddle factors Generation:

```
TW = exp(pi/2*i-2*pi*i*[0:L/2-1]/L) . '
```

Real and Imag values are in interleaved fashion

Example code for Floating-point RFFT Twiddle factors Generation:

```
TW = exp(pi/2*i-2*pi*i*[0:L/2-1]/L) . '
```

Real and Imag values are in interleaved fashion

Generation of realCoefA array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pATable[2 * i]      = 0.5 * ( 1.0 - sin (2 * PI / (double) (2 * n) *
    ↪(double) i));
    pATable[2 * i + 1] = 0.5 * (-1.0 * cos (2 * PI / (double) (2 * n) *
    ↪(double) i));
}
```

Generation of realCoefB array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pBTable[2 * i]      = 0.5 * (1.0 + sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pBTable[2 * i + 1] = 0.5 * (1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

Generation fixed-point realCoefAQ31 array in Q31 format:

n = 4096

```
for (i = 0; i < n; i++)
{
    pATable[2 * i]      = 0.5 * ( 1.0 - sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pATable[2 * i + 1] = 0.5 * (-1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

Convert to fixed point Q31 format round(pATable[i] * pow(2, 31))

Generation of realCoefBQ31 array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pBTable[2 * i]      = 0.5 * (1.0 + sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pBTable[2 * i + 1] = 0.5 * (1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

Convert to fixed point Q31 format round(pBTable[i] * pow(2, 31))

Example code for the generation of the floating-point sine table:

```
tableSize = 512;
for (n = 0; n < (tableSize + 1); n++)
{
    sinTable[n] = sin(2*PI*n/tableSize);
}
```

where PI value is 3.14159265358979

Table values are in Q31 (1.31 fixed-point format) and generation is done in three steps. First, generate sin values in floating point:

```
tableSize = 512;
for (n = 0; n < (tableSize + 1); n++)
{
    sinTable[n] = sin(2*PI*n/tableSize);
}
```

where PI value is 3.14159265358979

Second, convert floating-point to Q31 (Fixed point): $\text{sinTable}[i] * \text{pow}(2, 31)$

Finally, round to the nearest integer value: $\text{sinTable}[i] += (\text{sinTable}[i] > 0 ? 0.5 : -0.5);$

Table values are in Q15 (1.15 fixed-point format) and generation is done in three steps. First, generate sin values in floating point:

```
tableSize = 512;
for (n = 0; n < (tableSize + 1); n++)
{
    sinTable[n] = sin(2*PI*n/tableSize);
}
```

where PI value is 3.14159265358979

Second, convert floating-point to Q15 (Fixed point): $\text{sinTable}[i] * \text{pow}(2, 15)$

Finally, round to the nearest integer value: $\text{sinTable}[i] += (\text{sinTable}[i] > 0 ? 0.5 : -0.5);$

const float16_t twiddleCoeff16_16[32] RISC_V_DSP_TABLE_ATTRIBUTE

Floating-point Twiddle factors Table Generation.

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoeff[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoeff[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 16 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 64 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 128 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
```

(continues on next page)

(continued from previous page)

```
twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);  
}
```

where $N = 256$ and $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N/; i++)  
{  
twiddleCoef[2*i]= cos(i * 2*PI/(float)N);  
twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);  
}
```

where $N = 512$ and $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N/; i++)  
{  
twiddleCoef[2*i]= cos(i * 2*PI/(float)N);  
twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);  
}
```

where $N = 1024$ and $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N/; i++)  
{  
twiddleCoef[2*i]= cos(i * 2*PI/(float)N);  
twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);  
}
```

where $N = 2048$ and $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

Example code for Floating-point RFFT Twiddle factors Generation:

```
TW = exp(pi/2*i-2*pi*i*[0:L/2-1]/L) . '
```

Real and Imag values are in interleaved fashion

```
const uint64_t twiddleCoefF64_16[32]
```

Double Precision Floating-point Twiddle factors Table Generation.

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(double)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(double)N);
}
```

where N = 16, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_32[64]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/N);
}
```

where $N = 32$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_64[128]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 64$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_128[256]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 128$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_256[512]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 256, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_512[1024]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 512, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_1024[2048]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```

for (i = 0; i < N/; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const uint64_t twiddleCoefF64_2048[4096]
```

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const uint64_t **twiddleCoefF64_4096**[8192]

Example code for Double Precision Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_16**[32]

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 16, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_32**[64]

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 32$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_64**[128]

Example code for Floating-point Twiddle factors Generation:

```

for(i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 64$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_128**[256]

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 128$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_256**[512]

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 256, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_512**[1024]

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 512, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_1024**[2048]

Example code for Floating-point Twiddle factors Generation:

```
for (i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_2048**[4096]

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 2048$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const *float32_t* (page 1277) **twiddleCoef_4096**[8192]

Example code for Floating-point Twiddle factors Generation:

```

for (i = 0; i < N; i++)
{
    twiddleCoef[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 4096$, $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const *q31_t* (page 1277) **twiddleCoef_16_q31**[24]

Q31 Twiddle factors Table.

Example code for Q31 Twiddle factors Generation::

```

for(i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where $N = 16$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): $\text{round}(\text{twiddleCoefQ31}(i) * \text{pow}(2, 31))$

const *q31_t* (page 1277) **twiddleCoef_32_q31**[48]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

const *q31_t* (page 1277) **twiddleCoef_64_q31**[96]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 64, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

const *q31_t* (page 1277) **twiddleCoef_128_q31**[192]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```


where $N = 128$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): $\text{round}(\text{twiddleCoefQ31}(i) * \text{pow}(2, 31))$

const *q31_t* (page 1277) **twiddleCoef_256_q31**[384]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 256$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): $\text{round}(\text{twiddleCoefQ31}(i) * \text{pow}(2, 31))$

const *q31_t* (page 1277) **twiddleCoef_512_q31**[768]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 512$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): $\text{round}(\text{twiddleCoefQ31}(i) * \text{pow}(2, 31))$

const *q31_t* (page 1277) **twiddleCoef_1024_q31**[1536]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

const *q31_t* (page 1277) **twiddleCoef_2048_q31**[3072]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 2048, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

const *q31_t* (page 1277) **twiddleCoef_4096_q31**[6144]

Example code for Q31 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefQ31[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefQ31[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 4096, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to Q31(Fixed point 1.31): round(twiddleCoefQ31(i) * pow(2, 31))

const *q15_t* (page 1277) **twiddleCoef_16_q15**[24]

q15 Twiddle factors Table

Example code for q15 Twiddle factors Generation::

```
for(i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 16, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

const *q15_t* (page 1277) **twiddleCoef_32_q15**[48]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 32, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

const *q15_t* (page 1277) **twiddleCoef_64_q15**[96]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
```

(continues on next page)

(continued from previous page)

```
twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 64$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): $\text{round}(\text{twiddleCoefq15}(i) * \text{pow}(2, 15))$

const *q15_t* (page 1277) **twiddleCoef_128_q15**[192]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 128$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): $\text{round}(\text{twiddleCoefq15}(i) * \text{pow}(2, 15))$

const *q15_t* (page 1277) **twiddleCoef_256_q15**[384]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 256$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): $\text{round}(\text{twiddleCoefq15}(i) * \text{pow}(2, 15))$

const *q15_t* (page 1277) **twiddleCoef_512_q15**[768]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 512, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

const *q15_t* (page 1277) **twiddleCoef_1024_q15**[1536]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024, PI = 3.14159265358979

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): round(twiddleCoefq15(i) * pow(2, 15))

const *q15_t* (page 1277) **twiddleCoef_2048_q15**[3072]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 2048$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): $\text{round}(\text{twiddleCoefq15}(i) * \text{pow}(2, 15))$

const *q15_t* (page 1277) **twiddleCoef_4096_q15**[6144]

Example code for q15 Twiddle factors Generation::

```
for (i = 0; i < 3N/4; i++)
{
    twiddleCoefq15[2*i]   = cos(i * 2*PI/(float)N);
    twiddleCoefq15[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 4096$, $PI = 3.14159265358979$

Cos and Sin values are interleaved fashion

Convert Floating point to q15(Fixed point 1.15): $\text{round}(\text{twiddleCoefq15}(i) * \text{pow}(2, 15))$

const float16_t **twiddleCoefF16_32**[64]

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where $N = 32$ and $PI = 3.14159265358979$

Cos and Sin values are in interleaved fashion

const float16_t **twiddleCoefF16_64**[128]

Example code for Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i]= cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);
}

```

where N = 64 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_128[256]
```

Example code for Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i]= cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);
}

```

where N = 128 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_256[512]
```

Example code for Floating-point Twiddle factors Generation:

```

for(i = 0; i < N/; i++)
{
    twiddleCoef[2*i]= cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1]= sin(i * 2*PI/(float)N);
}

```

where N = 256 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_512[1024]
```

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 512 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_1024[2048]
```

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 1024 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_2048[4096]
```

Example code for Floating-point Twiddle factors Generation:

```
for(i = 0; i < N; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}
```

where N = 2048 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_4096[8192]
```

Example code for Floating-point Twiddle factors Generation:


```

for(i = 0; i < N/2; i++)
{
    twiddleCoef[2*i] = cos(i * 2*PI/(float)N);
    twiddleCoef[2*i+1] = sin(i * 2*PI/(float)N);
}

```

where N = 4096 and PI = 3.14159265358979

Cos and Sin values are in interleaved fashion

```
const float16_t twiddleCoefF16_rfft_32[32]
```

Example code for Floating-point RFFT Twiddle factors Generation:

```
TW = exp(pi/2*i-2*pi*i*[0:L/2-1]/L) . '
```

Real and Imag values are in interleaved fashion

```
const float16_t twiddleCoefF16_rfft_64[64]
```

```
const float16_t twiddleCoefF16_rfft_128[128]
```

```
const float16_t twiddleCoefF16_rfft_256[256]
```

```
const float16_t twiddleCoefF16_rfft_512[512]
```

```
const float16_t twiddleCoefF16_rfft_1024[1024]
```

```
const float16_t twiddleCoefF16_rfft_2048[2048]
```

```
const float16_t twiddleCoefF16_rfft_4096[4096]
```

Complex FFT F16

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_f16 (const riscv_cfft_instance_f16 *S, float16_t *p1,
uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f16 (riscv_cfft_instance_f16 *S,
uint16_t fftLen)
```

```
FFTINIT(EXT, SIZE)
```

CFFTINIT_F16(LEN, LENTWIDDLE)

group **Complex FFT F16**

Defines

FFTINIT(EXT, SIZE)

CFFTINIT_F16(LEN, LENTWIDDLE)

Functions

RISCV_DSP_ATTRIBUTE void riscv_cfft_f16 (const riscv_cfft_instance_f16 *S, float16_t *p1, uint8_t ifftFlag, uint8_t bitReverseFlag)

Processing function for the floating-point complex FFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – **[in]** points to an instance of the floating-point CFFT structure
- **p1** – **[inout]** points to the complex data buffer. Processing occurs in-place
- **ifftFlag** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f16 (riscv_cfft_instance_f16 *S, uint16_t fftLen)

Initialization function for the cfft f16 function with 4096 samples.

Initialization function for the cfft f16 function with 2048 samples

Initialization function for the cfft f16 function with 1024 samples

Initialization function for the cfft f16 function with 512 samples

Initialization function for the cfft f16 function with 256 samples

Initialization function for the cfft f16 function with 128 samples

Initialization function for the cfft f16 function with 64 samples

Initialization function for the cfft f16 function with 32 samples

Initialization function for the cfft f16 function with 16 samples

Generic initialization function for the cfft f16 function

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

This function should be used only if you don't know the FFT sizes that

you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes

that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **fftLen** – **[in]** fft length (number of complex samples)

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Complex FFT F32

```
RISC_V_DSP_ATTRIBUTE void riscv_cfft_f32 (const riscv_cfft_instance_f32 *S, float32_t *p1,
uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f32 (riscv_cfft_instance_f32 *S,
uint16_t fftLen)
```

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_F32(LEN, LENTWIDDLE)
```

group **Complex FFT F32**

Defines

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_F32(LEN, LENTWIDDLE)
```

Functions

```
RISC_V_DSP_ATTRIBUTE void riscv_cfft_f32 (const riscv_cfft_instance_f32 *S,
float32_t *p1, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

Processing function for the floating-point complex FFT.

RVV version

The rvv version has a different API. The input and output buffers must be different. There is an optional temporary buffer. If the temporary buffer is not used, the input buffer is modified. The bit reverse flag is not more available in RVV version.

```
void riscv_cfft_f32(
    const riscv_cfft_instance_f32 * S,
    float32_t * pIn,
    float32_t * pOut,
    float32_t * pBuffer,
    uint8_t ifftFlag);
```

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an instance of the floating-point CFFT structure
- **p1** – [inout] points to the complex data buffer. Processing occurs in-place
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f32 (riscv_cfft_instance_f32 *S, uint16_t fftLen)

Initialization function for the cfft f32 function with 4096 samples.

Initialization function for the cfft f32 function with 2048 samples

Initialization function for the cfft f32 function with 1024 samples

Initialization function for the cfft f32 function with 512 samples

Initialization function for the cfft f32 function with 256 samples

Initialization function for the cfft f32 function with 128 samples

Initialization function for the cfft f32 function with 64 samples

Initialization function for the cfft f32 function with 32 samples

Initialization function for the cfft f32 function with 16 samples

Generic initialization function for the cfft f32 function

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in riscv_const_structs.h

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in riscv_const_structs.h

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in riscv_const_structs.h

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in riscv_const_structs.h

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in riscv_const_structs.h

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT. Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **fftLen** – **[in]** fft length (number of complex samples)

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful

- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

Complex FFT F64

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_f64 (const riscv_cfft_instance_f64 *S, float64_t *p1,
uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f64 (riscv_cfft_instance_f64 *S,
uint16_t fftLen)
```

```
CFFTINIT_F64(LEN)
```

group **Complex FFT F64**

Defines

```
CFFTINIT_F64(LEN)
```

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_f64 (const riscv_cfft_instance_f64 *S,
float64_t *p1, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

Processing function for the Double Precision floating-point complex FFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – **[in]** points to an instance of the Double Precision floating-point CFFT structure
- **p1** – **[inout]** points to the complex data buffer. Processing occurs in-place
- **ifftFlag** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_f64 (riscv_cfft_instance_f64 *S,
uint16_t fftLen)
```

Initialization function for the cfft f64 function with 4096 samples.

Initialization function for the cfft f64 function with 2048 samples

Initialization function for the cfft f64 function with 1024 samples

Initialization function for the cfft f64 function with 512 samples

Initialization function for the cfft f64 function with 256 samples

Initialization function for the cfft f64 function with 128 samples

Initialization function for the cfft f64 function with 64 samples

Initialization function for the cfft f64 function with 32 samples

Initialization function for the cfft f64 function with 16 samples

Generic initialization function for the cfft f64 function

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is not mandatory. You can initialize directly the data structure using variables declared in `riscv_const_structs.h`

This function should be used only if you don't know the FFT sizes that

you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes

that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **fftLen** – **[in]** fft length (number of complex samples)

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful

- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Complex FFT Q15

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_cfft_init_q15 (riscv_cfft_instance_q15 *S,  
uint16_t fftLen)
```

```
RISC_V_DSP_ATTRIBUTE void riscv_cfft_q15 (const riscv_cfft_instance_q15 *S, q15_t *p1,  
uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_Q15(LEN, LENTWIDDLE)
```

group **Complex FFT Q15**

Defines

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_Q15(LEN, LENTWIDDLE)
```

Functions

```
RISC_V_DSP_ATTRIBUTE riscv_status riscv_cfft_init_q15 (riscv_cfft_instance_q15 *S,  
uint16_t fftLen)
```

Initialization function for the cfft q15 function for 4096 samples.

Initialization function for the cfft q15 function for 2048 samples

Initialization function for the cfft q15 function for 1024 samples

Initialization function for the cfft q15 function for 512 samples

Initialization function for the cfft q15 function for 256 samples

Initialization function for the cfft q15 function for 128 samples

Initialization function for the cfft q15 function for 64 samples

Initialization function for the cfft q15 function for 32 samples

Initialization function for the cfft q15 function for 16 samples

Generic initialization function for the cfft q15 function

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **fftLen** – **[in]** fft length (number of complex samples)

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE void riscv_cfft_q15 (const riscv_cfft_instance_q15 *S, q15_t *p1, uint8_t ifftFlag, uint8_t bitReverseFlag)

Processing function for Q15 complex FFT.

CFFT Size	Input Format	Output Format	Number of bits to upscale
16	1.15	5.11	4
64	1.15	7.9	6
256	1.15	9.7	8
1024	1.15	11.5	10

Input and Output formats for CFFT Q15

CFFT Size	Input Format	Output Format	Number of bits to upscale
16	1.15	5.11	0
64	1.15	7.9	0
256	1.15	9.7	0
1024	1.15	11.5	0

Input and Output formats for CIFFT Q15

```
void riscv_cfft_q15(
const riscv_cfft_instance_q15 * S,
const q15_t * src,
q15_t * dst,
q15_t *buffer,
uint8_t ifftFlag)
```

RVV version

The rvv version has a different API. The input and output buffers must be different. There is a temporary buffer. The bit reverse flag is not more available in rvv version.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an instance of Q15 CFFT structure
- **p1** – [inout] points to the complex data buffer. Processing occurs in-place
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Complex FFT Q31

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_q31 (riscv_cfft_instance_q31 *S,  
uint16_t fftLen)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_q31 (const riscv_cfft_instance_q31 *S, q31_t *p1,  
uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_Q31(LEN, LENTWIDDLE)
```

group **Complex FFT Q31**

Defines

```
FFTINIT(EXT, SIZE)
```

```
CFFTINIT_Q31(LEN, LENTWIDDLE)
```

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_init_q31 (riscv_cfft_instance_q31 *S,  
uint16_t fftLen)
```

Initialization function for the cfft q31 function for 4096 samples.

Initialization function for the cfft q31 function for 2048 samples

Initialization function for the cfft q31 function for 1024 samples

Initialization function for the cfft q31 function for 512 samples

Initialization function for the cfft q31 function for 256 samples

Initialization function for the cfft q31 function for 128 samples

Initialization function for the cfft q31 function for 64 samples

Initialization function for the cfft q31 function for 32 samples

Initialization function for the cfft q31 function for 16 samples

Generic initialization function for the cfft q31 function

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

Use of this function is mandatory only for the RVV versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `riscv_const_structs.h`

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure

- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **S** – **[inout]** points to an instance of the floating-point CFFT structure
- **fftLen** – **[in]** fft length (number of complex samples)

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : an error is detected

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful

- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE void riscv_cfft_q31 (const riscv_cfft_instance_q31 *S, q31_t *p1, uint8_t ifftFlag, uint8_t bitReverseFlag)

Processing function for the Q31 complex FFT.

CFFT Size	Input Format	Output Format	Number of bits to upscale
16	1.31	5.27	4
64	1.31	7.25	6
256	1.31	9.23	8
1024	1.31	11.21	10

Input and Output formats for CFFT Q31

CFFT Size	Input Format	Output Format	Number of bits to upscale
16	1.31	5.27	0
64	1.31	7.25	0
256	1.31	9.23	0
1024	1.31	11.21	0

Input and Output formats for CIFFT Q31

```
void riscv_cfft_q31(
const riscv_cfft_instance_q31 * S,
const q31_t * src,
q31_t * dst,
q31_t *buffer,
uint8_t ifftFlag
)
```

RVV version

The rvv version has a different API. The input and output buffers must be different. There is a temporary buffer. The bit reverse flag is not more available in rvv version.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an instance of the fixed-point CFFT structure
- **p1** – [inout] points to the complex data buffer. Processing occurs in-place
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Deprecated Complex FFT functions

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_f16 (const riscv_cfft_radix2_instance_f16 *S,  
float16_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_f32 (const riscv_cfft_radix2_instance_f32 *S,  
float32_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_f16 (riscv_cfft_radix2_instance_f16 *S,  
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_f32 (riscv_cfft_radix2_instance_f32 *S,  
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_q15 (riscv_cfft_radix2_instance_q15 *S,  
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_q31 (riscv_cfft_radix2_instance_q31 *S,  
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_q15 (const riscv_cfft_radix2_instance_q15 *S,  
q15_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_q31 (const riscv_cfft_radix2_instance_q31 *S,  
q31_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4by2_f16 (float16_t *pSrc, uint32_t fftLen,  
const float16_t *pCoef)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_f16 (const riscv_cfft_radix4_instance_f16 *S,
float16_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_f32 (const riscv_cfft_radix4_instance_f32 *S,
float32_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_f16 (riscv_cfft_radix4_instance_f16 *S,
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_f32 (riscv_cfft_radix4_instance_f32 *S,
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_q15 (riscv_cfft_radix4_instance_q15 *S,
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_q31 (riscv_cfft_radix4_instance_q31 *S,
uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_q15 (const riscv_cfft_radix4_instance_q15 *S,
q15_t *pSrc)
```

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_q31 (const riscv_cfft_radix4_instance_q31 *S,
q31_t *pSrc)
```

group **Deprecated Complex FFT functions**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_f16 (const riscv_cfft_radix2_instance_f16 *S,
float16_t *pSrc)
```

Radix-2 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f16` and will be removed in the future

Parameters

- **S** – [**in**] points to an instance of the floating-point Radix-2 CFFT/CIFFT structure
- **pSrc** – [**inout**] points to the complex data buffer of size `2*fftLen`. Processing occurs in-place

```
RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_f32 (const riscv_cfft_radix2_instance_f32 *S,
float32_t *pSrc)
```

Radix-2 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f32` and will be removed in the future

Parameters

- **S** – [in] points to an instance of the floating-point Radix-2 CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size `2*fftLen`. Processing occurs in-place

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_f16 (riscv_cfft_radix2_instance_f16 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the floating-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f16` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the floating-point CFFT/CIFFT structure
- **fftLen** – [in] length of the FFT
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLen` is not a supported length

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_f32 (riscv_cfft_radix2_instance_f32 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the floating-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f32` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the floating-point CFFT/CIFFT structure
- **fftLen** – [in] length of the FFT
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : `fftLen` is not a supported length

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_q15 (riscv_cfft_radix2_instance_q15 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the Q15 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q15` and will be removed

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the Q15 CFFT/CIFFT structure.
- **fftLen** – [in] length of the FFT.
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLen` is not a supported length

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix2_init_q31 (riscv_cfft_radix2_instance_q31 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the Q31 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q31` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the Q31 CFFT/CIFFT structure

- **fftLen** – [in] length of the FFT
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : **fftLen** is not a supported length

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_q15 (const riscv_cfft_radix2_instance_q15 *S, q15_t *pSrc)

Processing function for the fixed-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q15` and will be removed in the future.

Parameters

- **S** – [in] points to an instance of the fixed-point CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size $2 \times \text{fftLen}$. Processing occurs in-place

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix2_q31 (const riscv_cfft_radix2_instance_q31 *S, q31_t *pSrc)

Processing function for the fixed-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q31` and will be removed in the future.

Parameters

- **S** – [in] points to an instance of the fixed-point CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size $2 \times \text{fftLen}$. Processing occurs in-place

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4by2_f16 (float16_t *pSrc, uint32_t fftLen, const float16_t *pCoef)

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_f16 (const riscv_cfft_radix4_instance_f16 *S, float16_t *pSrc)

Processing function for the floating-point Radix-4 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f16` and will be removed in the future.

Parameters

- **S** – [in] points to an instance of the floating-point Radix-4 CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size `2*fftLen`. Processing occurs in-place

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_f32 (const riscv_cfft_radix4_instance_f32 *S, float32_t *pSrc)

Processing function for the floating-point Radix-4 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f32` and will be removed in the future.

Parameters

- **S** – [in] points to an instance of the floating-point Radix-4 CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size `2*fftLen`. Processing occurs in-place

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_f16 (riscv_cfft_radix4_instance_f16 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the floating-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_f16` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the floating-point CFFT/CIFFT structure
- **fftLen** – [in] length of the FFT
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform

- value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : fftLen is not a supported length

RISC_V_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_f32 (riscv_cfft_radix4_instance_f32 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the floating-point CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by riscv_cfft_f32 and will be removed in the future.

Details

The parameter **ifftFlag** controls whether a forward or inverse transform is computed. Set(=1) ifft-Flag for calculation of CIFFT otherwise CFFT is calculated

The parameter **bitReverseFlag** controls whether output is in normal order or bit reversed order. Set(=1) bitReverseFlag for output to be in normal order otherwise output is in bit reversed order.

The parameter **fftLen** Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the floating-point CFFT/CIFFT structure
- **fftLen** – [in] length of the FFT
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : fftLen is not a supported length

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_q15 (riscv_cfft_radix4_instance_q15 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the Q15 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q15` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [**inout**] points to an instance of the Q15 CFFT/CIFFT structure
- **fftLen** – [**in**] length of the FFT
- **ifftFlag** – [**in**] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [**in**] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : `fftLen` is not a supported length

RISCV_DSP_ATTRIBUTE riscv_status riscv_cfft_radix4_init_q31 (riscv_cfft_radix4_instance_q31 *S, uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag)

Initialization function for the Q31 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q31` and will be removed in the future.

Details

The parameter `ifftFlag` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlag` for calculation of CIFFT otherwise CFFT is calculated

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

The parameter `fftLen` Specifies length of CFFT/CIFFT process. Supported FFT Lengths are 16, 64, 256, 1024.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

Parameters

- **S** – [inout] points to an instance of the Q31 CFFT/CIFFT structure.
- **fftLen** – [in] length of the FFT.
- **ifftFlag** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : `fftLen` is not a supported length

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_q15 (const riscv_cfft_radix4_instance_q15 *S, q15_t *pSrc)

Processing function for the Q15 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q15` and will be removed in the future.

CFFT Size	Input format	Output format	Number of bits to upscale
16	1.15	5.11	4
64	1.15	7.9	6
256	1.15	9.7	8
1024	1.15	11.5	10

Input and output formats:

Internally input is downscaled by 2 for every stage to avoid saturations inside CFFT/CIFFT process. Hence the output format is different for different FFT sizes. The input and output formats for different FFT sizes and number of bits to upscale are mentioned in the tables below for CFFT and CIFFT:

CIFFT Size	Input format	Output format	Number of bits to upscale
16	1.15	5.11	0
64	1.15	7.9	0
256	1.15	9.7	0
1024	1.15	11.5	0

Parameters

- **S** – [in] points to an instance of the Q15 CFFT/CIFFT structure.
- **pSrc** – [inout] points to the complex data buffer. Processing occurs in-place.

RISCV_DSP_ATTRIBUTE void riscv_cfft_radix4_q31 (const riscv_cfft_radix4_instance_q31 *S, q31_t *pSrc)

Processing function for the Q31 CFFT/CIFFT.

Deprecated:

Do not use this function. It has been superseded by `riscv_cfft_q31` and will be removed in the future.

CFFT Size	Input format	Output format	Number of bits to upscale
16	1.31	5.27	4
64	1.31	7.25	6
256	1.31	9.23	8
1024	1.31	11.21	10

Input and output formats:

Internally input is downscaled by 2 for every stage to avoid saturations inside CFFT/CIFFT process. Hence the output format is different for different FFT sizes. The input and output formats for different FFT sizes and number of bits to upscale are mentioned in the tables below for CFFT and CIFFT:

CIFFT Size	Input format	Output format	Number of bits to upscale
16	1.31	5.27	0
64	1.31	7.25	0
256	1.31	9.23	0
1024	1.31	11.21	0

Parameters

- **S** – [in] points to an instance of the Q31 CFFT/CIFFT structure
- **pSrc** – [inout] points to the complex data buffer of size $2 \times \text{fftLen}$. Processing occurs in-place

group Complex FFT Functions

```
void riscv_cfft_f32(
    const riscv_cfft_instance_f32 * S,
    const float32_t * pIn,
    float32_t * pOut,
    float32_t * pBuffer,
    uint8_t ifftFlag);
```

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT). The FFT can be orders of magnitude faster than the DFT, especially for long lengths. The algorithms described in this section operate on complex data. A separate set of functions is devoted to handling of real sequences.

There are separate algorithms for handling floating-point, Q15, and Q31 data types. The algorithms available for each data type are described next.

The FFT functions operate in-place. That is, the array holding the input data will also be used to hold the corresponding result. The input data is complex and contains $2 * \text{fftLen}$ interleaved values as shown below.

```
{real[0], imag[0], real[1], imag[1], ...}
```

The FFT result will be contained in the same array and the frequency domain values will have the same interleaving.

Floating-point

The floating-point complex FFT uses a mixed-radix algorithm. Multiple radix-8 stages are performed along with a single radix-2 or radix-4 stage, as needed. The algorithm supports lengths of [16, 32, 64, ..., 4096] and each length uses a different twiddle factor table.

The function uses the standard FFT definition and output values may grow by a factor of fftLen when computing the forward transform. The inverse transform includes a scale of $1/\text{fftLen}$ as part of the calculation and this matches the textbook definition of the inverse FFT.

For the RVV version, the new `riscv_cfft_init_f32` initialization function is **mandatory**. **Compilation flags are available to include only the required tables for the needed FFTs**. Other FFT versions can continue to be initialized as explained below.

For versions not targeting RVV, pre-initialized data structures containing twiddle factors and bit reversal tables are provided and defined in `riscv_const_structs.h`. Include this header in your function and then pass one of the constant structures as an argument to `riscv_cfft_f32`. For example:

```
riscv_cfft_f32(riscv_cfft_sR_f32_len64, pSrc, 1, 1)
```

computes a 64-point inverse complex FFT including bit reversal. The data structures are treated as constant data and not modified during the calculation. The same data structure can be reused for multiple transforms including mixing forward and inverse transforms.

Earlier releases of the library provided separate radix-2 and radix-4 algorithms that operated on floating-point data. These functions are still provided but are deprecated. The older functions are slower and less general than the new functions.

An example of initialization of the constants for the `riscv_cfft_f32` function follows:

```
const static riscv_cfft_instance_f32 *S;
...
switch (length) {
    case 16:
        S = &riscv_cfft_sR_f32_len16;
        break;
    case 32:
        S = &riscv_cfft_sR_f32_len32;
        break;
    case 64:
        S = &riscv_cfft_sR_f32_len64;
        break;
    case 128:
        S = &riscv_cfft_sR_f32_len128;
        break;
    case 256:
        S = &riscv_cfft_sR_f32_len256;
        break;
    case 512:
        S = &riscv_cfft_sR_f32_len512;
        break;
    case 1024:
        S = &riscv_cfft_sR_f32_len1024;
        break;
    case 2048:
        S = &riscv_cfft_sR_f32_len2048;
        break;
    case 4096:
        S = &riscv_cfft_sR_f32_len4096;
        break;
}
```

The new `riscv_cfft_init_f32` can also be used.

Q15 and Q31

The floating-point complex FFT uses a mixed-radix algorithm. Multiple radix-4 stages are performed along with a single radix-2 stage, as needed. The algorithm supports lengths of [16, 32, 64, ..., 4096] and each length uses a different twiddle factor table.

The function uses the standard FFT definition and output values may grow by a factor of `fftLen` when computing the forward transform. The inverse transform includes a scale of $1/\text{fftLen}$ as part of the calculation and this matches the textbook definition of the inverse FFT.

Pre-initialized data structures containing twiddle factors and bit reversal tables are provided and defined in `riscv_const_structs.h`. Include this header in your function and then pass one of the constant structures as an argument to `riscv_cfft_q31` (except if you are targeting RVV). For example:

```
riscv_cfft_q31(riscv_cfft_sR_q31_len64, pSrc, 1, 1)
```


computes a 64-point inverse complex FFT including bit reversal. The data structures are treated as constant data and not modified during the calculation. The same data structure can be reused for multiple transforms including mixing forward and inverse transforms.

Earlier releases of the library provided separate radix-2 and radix-4 algorithms that operated on floating-point data. These functions are still provided but are deprecated. The older functions are slower and less general than the new functions.

An example of initialization of the constants for the `riscv_cfft_q31` function follows:

```
const static riscv_cfft_instance_q31 *S;
...
switch (length) {
    case 16:
        S = &riscv_cfft_sR_q31_len16;
        break;
    case 32:
        S = &riscv_cfft_sR_q31_len32;
        break;
    case 64:
        S = &riscv_cfft_sR_q31_len64;
        break;
    case 128:
        S = &riscv_cfft_sR_q31_len128;
        break;
    case 256:
        S = &riscv_cfft_sR_q31_len256;
        break;
    case 512:
        S = &riscv_cfft_sR_q31_len512;
        break;
    case 1024:
        S = &riscv_cfft_sR_q31_len1024;
        break;
    case 2048:
        S = &riscv_cfft_sR_q31_len2048;
        break;
    case 4096:
        S = &riscv_cfft_sR_q31_len4096;
        break;
}
```

RVV version

The rvv version has a different API. The input and output buffers must be different. There is a temporary buffer. The bit reverse flag is not more available in RVV version.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

DCT Type IV Functions

DCT Type IV Tables

const *float32_t* (page 1277) **Weights_128**[256]

const *float32_t* (page 1277) **cos_factors_128**[128]

const *float32_t* (page 1277) **Weights_512**[1024]

const *float32_t* (page 1277) **cos_factors_512**[512]

const *float32_t* (page 1277) **Weights_2048**[4096]

const *float32_t* (page 1277) **cos_factors_2048**[2048]

const *float32_t* (page 1277) **Weights_8192**[16384]

const *float32_t* (page 1277) **cos_factors_8192**[8192]

const *q31_t* (page 1277) **WeightsQ31_128**[256]

const *q31_t* (page 1277) **cos_factorsQ31_128**[128]

const *q31_t* (page 1277) **WeightsQ31_512**[1024]

const *q31_t* (page 1277) **cos_factorsQ31_512**[512]

const *q31_t* (page 1277) **WeightsQ31_2048**[4096]

const *q31_t* (page 1277) **cos_factorsQ31_2048**[2048]

const *q31_t* (page 1277) **WeightsQ31_8192**[16384]

const *q31_t* (page 1277) **cos_factorsQ31_8192**[8192]

group **DCT Type IV Tables**

end of RealFFT_Table group

Variables

const *float32_t* (page 1277) **Weights_128**[256]

Weights Table.

Weights tables are generated using the formula :

$$\text{weights}[n] = e^{(-j * n * \pi / (2 * N))}$$

C command to generate the table

```
for(i = 0; i < N; i++)
{
    weights[(2*i)] = cos (i*c);
    weights[(2*i)+1] = -sin (i*c);
}
```

where N is the Number of weights to be calculated and c is $\pi / (2 * N)$

In the tables below the real and imaginary values are placed alternatively, hence the array length is $2 * N$.

cosFactor tables are generated using the formula :

$$\text{cos_factors}[n] = 2 * \cos((2n+1)*\pi/(4*N))$$

C command to generate the table

```
for(i = 0; i < N; i++)
{
    cos_factors[i] = 2 * cos((2*i+1)*c/2);
}
```

where N is the number of factors to generate and c is $\pi / (2 * N)$

const *float32_t* (page 1277) **cos_factors_128**[128]

const *float32_t* (page 1277) **Weights_512**[1024]

const *float32_t* (page 1277) **cos_factors_512**[512]

const *float32_t* (page 1277) **Weights_2048**[4096]

const *float32_t* (page 1277) **cos_factors_2048**[2048]

const *float32_t* (page 1277) **Weights_8192**[16384]

const *float32_t* (page 1277) **cos_factors_8192**[8192]

const *q31_t* (page 1277) **WeightsQ31_128**[256]

Weights tables are generated using the formula :

```
weights[n] = e-j*n*pi/(2*N)
```

C command to generate the table

```
for (i = 0; i < N; i++)  
{  
    weights[(2*i)] = cos(i*c);  
    weights[(2*i)+1] = -sin(i*c);  
}
```

where N is the Number of weights to be calculated and c is $\pi/(2*N)$

Convert the output to q31 format by multiplying with 2^{31} and saturated if required.

In the tables below the real and imaginary values are placed alternatively, hence the array length is $2*N$.

cosFactor tables are generated using the formula :

```
cos_factors[n] = 2 * cos((2n+1)*pi/(4*N))
```

C command to generate the table

```
for (i = 0; i < N; i++)  
{  
    cos_factors[i] = 2 * cos((2*i+1)*c/2);  
}
```

where N is the number of factors to generate and c is $\pi/(2*N)$

Then converted to q31 format by multiplying with 2^{31} and saturated if required.

const *q31_t* (page 1277) **cos_factorsQ31_128**[128]

```

const q31_t (page 1277) WeightsQ31_512[1024]

const q31_t (page 1277) cos_factorsQ31_512[512]

const q31_t (page 1277) WeightsQ31_2048[4096]

const q31_t (page 1277) cos_factorsQ31_2048[2048]

const q31_t (page 1277) WeightsQ31_8192[16384]

const q31_t (page 1277) cos_factorsQ31_8192[8192]

```

DCT4 F32

```
RISCV_DSP_ATTRIBUTE void riscv_dct4_f32 (const riscv_dct4_instance_f32 *S,
float32_t *pState, float32_t *pInlineBuffer)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_f32 (riscv_dct4_instance_f32 *S,
riscv_rfft_instance_f32 *S_RFFT, riscv_cfft_radix4_instance_f32 *S_CFFT, uint16_t N,
uint16_t Nby2, float32_t normalize)
```

group DCT4 F32

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_dct4_f32 (const riscv_dct4_instance_f32 *S,
float32_t *pState, float32_t *pInlineBuffer)
```

Processing function for the floating-point DCT4/IDCT4.

Deprecated:

Do not use this function. It is using a deprecated version of the RFFT.

Parameters

- **S** – [in] points to an instance of the floating-point DCT4/IDCT4 structure
- **pState** – [in] points to state buffer
- **pInlineBuffer** – [inout] points to the in-place input and output buffer

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_f32 (riscv_dct4_instance_f32 *S,
riscv_rfft_instance_f32 *S_RFFT, riscv_cfft_radix4_instance_f32 *S_CFFT, uint16_t N,
uint16_t Nby2, float32_t normalize)
```

Initialization function for the floating-point DCT4/IDCT4.

Deprecated:

Do not use this function. It is using a deprecated version of the RFFT.

DCT Size	Normalizing factor value
2048	0.03125
512	0.0625
128	0.125

Normalizing factor

The normalizing factor is $\sqrt{2/N}$, which depends on the size of transform N . Floating-point normalizing factors are mentioned in the table below for different DCT sizes:

Parameters

- **S** – [**inout**] points to an instance of floating-point DCT4/IDCT4 structure
- **S_RFFT** – [**in**] points to an instance of floating-point RFFT/RIFFT structure
- **S_CFFT** – [**in**] points to an instance of floating-point CFFT/CIFFT structure
- **N** – [**in**] length of the DCT4
- **Nby2** – [**in**] half of the length of the DCT4
- **normalize** – [**in**] normalizing factor.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : N is not a supported transform length

DCT4 Q15

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_q15 (riscv_dct4_instance_q15 *S,  
riscv_rfft_instance_q15 *S_RFFT, riscv_cfft_radix4_instance_q15 *S_CFFT, uint16_t N,  
uint16_t Nby2, q15_t normalize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_dct4_q15 (const riscv_dct4_instance_q15 *S, q15_t *pState,  
q15_t *pInlineBuffer)
```

group DCT4 Q15

Functions

RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_q15 (riscv_dct4_instance_q15 *S, riscv_rfft_instance_q15 *S_RFFT, riscv_cfft_radix4_instance_q15 *S_CFFT, uint16_t N, uint16_t Nby2, q15_t normalize)

Initialization function for the Q15 DCT4/IDCT4.

Deprecated:

Do not use this function. It will be removed in future versions.

DCT Size	Normalizing factor value (hexadecimal)
2048	0x400
512	0x800
128	0x1000

Normalizing factor

The normalizing factor is $\sqrt{2/N}$, which depends on the size of transform N. Normalizing factors in 1.15 format are mentioned in the table below for different DCT sizes:

Parameters

- **S** – [inout] points to an instance of Q15 DCT4/IDCT4 structure
- **S_RFFT** – [in] points to an instance of Q15 RFFT/RIFFT structure
- **S_CFFT** – [in] points to an instance of Q15 CFFT/CIFFT structure
- **N** – [in] length of the DCT4
- **Nby2** – [in] half of the length of the DCT4
- **normalize** – [in] normalizing factor

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : N is not a supported transform length

RISCV_DSP_ATTRIBUTE void riscv_dct4_q15 (const riscv_dct4_instance_q15 *S, q15_t *pState, q15_t *pInlineBuffer)

Processing function for the Q15 DCT4/IDCT4.

Deprecated:

Do not use this function. It will be removed in future versions.

DCT Size	Input format	Output format	Number of bits to upscale
2048	1.15	11.5	10
512	1.15	9.7	8
128	1.15	7.9	6

Input an output formats

Internally inputs are downscaled in the RFFT process function to avoid overflows. Number of bits downscaled, depends on the size of the transform. The input and output formats for different DCT sizes and number of bits to upscale are mentioned in the table below:

Parameters

- **S** – [in] points to an instance of the Q15 DCT4 structure.
- **pState** – [in] points to state buffer.
- **pInlineBuffer** – [inout] points to the in-place input and output buffer.

DCT4 Q31

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_q31 (riscv_dct4_instance_q31 *S,
riscv_rfft_instance_q31 *S_RFFT, riscv_cfft_radix4_instance_q31 *S_CFFT, uint16_t N,
uint16_t Nby2, q31_t normalize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_dct4_q31 (const riscv_dct4_instance_q31 *S, q31_t *pState,
q31_t *pInlineBuffer)
```

group DCT4 Q31

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_dct4_init_q31 (riscv_dct4_instance_q31 *S,
riscv_rfft_instance_q31 *S_RFFT, riscv_cfft_radix4_instance_q31 *S_CFFT, uint16_t N,
uint16_t Nby2, q31_t normalize)
```

Initialization function for the Q31 DCT4/IDCT4.

Deprecated:

Do not use this function. It will be removed in future versions.

DCT Size	Normalizing factor value (hexadecimal)
2048	0x4000000
512	0x8000000
128	0x10000000

Normalizing factor:

The normalizing factor is $\sqrt{2/N}$, which depends on the size of transform N. Normalizing factors in 1.31 format are mentioned in the table below for different DCT sizes:

Parameters

- **S** – [inout] points to an instance of Q31 DCT4/IDCT4 structure.
- **S_RFFT** – [in] points to an instance of Q31 RFFT/RIFFT structure
- **S_CFFT** – [in] points to an instance of Q31 CFFT/CIFFT structure

- **N** – [in] length of the DCT4.
- **Nby2** – [in] half of the length of the DCT4.
- **normalize** – [in] normalizing factor.

Returns

execution status

- **RISCV_MATH_SUCCESS** : Operation successful
- **RISCV_MATH_ARGUMENT_ERROR** : N is not a supported transform length

RISCV_DSP_ATTRIBUTE void riscv_dct4_q31 (const riscv_dct4_instance_q31 *S, q31_t *pState, q31_t *pInlineBuffer)

Processing function for the Q31 DCT4/IDCT4.

Deprecated:

Do not use this function. It will be removed in future versions.

DCT Size	Input format	Output format	Number of bits to upscale
2048	2.30	12.20	11
512	2.30	10.22	9
128	2.30	8.24	7

Input an output formats

Input samples need to be downscaled by 1 bit to avoid saturations in the Q31 DCT process, as the conversion from DCT2 to DCT4 involves one subtraction. Internally inputs are downscaled in the RFFT process function to avoid overflows. Number of bits downscaled, depends on the size of the transform. The input and output formats for different DCT sizes and number of bits to upscale are mentioned in the table below:

Parameters

- **S** – [in] points to an instance of the Q31 DCT4 structure.
- **pState** – [in] points to state buffer.
- **pInlineBuffer** – [inout] points to the in-place input and output buffer.

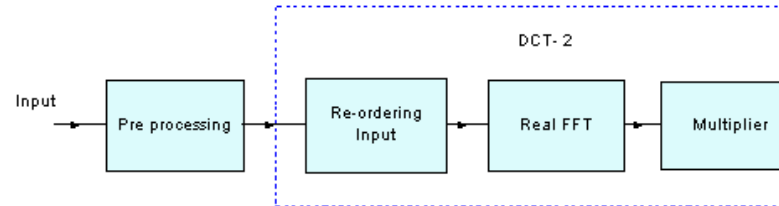
group DCT Type IV Functions

Representation of signals by minimum number of values is important for storage and transmission. The possibility of large discontinuity between the beginning and end of a period of a signal in DFT can be avoided by extending the signal so that it is even-symmetric. Discrete Cosine Transform (DCT) is constructed such that its energy is heavily concentrated in the lower part of the spectrum and is very widely used in signal and image coding applications. The family of DCTs (DCT type- 1,2,3,4) is the outcome of different combinations of homogeneous boundary conditions. DCT has an excellent energy-packing capability, hence has many applications and in data compression in particular.

DCT is essentially the Discrete Fourier Transform(DFT) of an even-extended real signal. Reordering of the input data makes the computation of DCT just a problem of computing the DFT of a real signal with a few additional operations. This approach provides regular, simple, and very efficient DCT algorithms for practical hardware and software implementations.

DCT type-II can be implemented using Fast fourier transform (FFT) internally, as the transform is applied on real values, Real FFT can be used. DCT4 is implemented using DCT2 as their implementations are similar except with some added pre-processing and post-processing. DCT2 implementation can be described in the following steps:

- Re-ordering input
- Calculating Real FFT
- Multiplication of weights and Real FFT output and getting real part from the product.



This process is explained by the block diagram below:

Algorithm

The N-point type-IV DCT is defined as a real, linear transformation by the formula:

$$X_c(k) = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} x(n) \cos \left[\left(n + \frac{1}{2} \right) \left(k + \frac{1}{2} \right) \frac{\pi}{N} \right]$$

where $k = 0, 1, 2, \dots, N-1$

Its inverse is defined as follows:

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=0}^{N-1} X_c(k) \cos \left[\left(n + \frac{1}{2} \right) \left(k + \frac{1}{2} \right) \frac{\pi}{N} \right]$$

where $n = 0, 1, 2, \dots, N-1$

The DCT4 matrices become involutory (i.e. they are self-inverse) by multiplying with an overall scale factor of $\sqrt{2/N}$. The symmetry of the transform matrix indicates that the fast algorithms for the forward and inverse transform computation are identical. Note that the implementation of Inverse DCT4 and DCT4 is same, hence same process function can be used for both.

Lengths supported by the transform:

As DCT4 internally uses Real FFT, it supports all the lengths 128, 512, 2048 and 8192. The library provides separate functions for Q15, Q31, and floating-point data types.

Instance Structure

The instances for Real FFT and FFT, cosine values table and twiddle factor table are stored in an instance data structure. A separate instance structure must be defined for each transform. There are separate instance structure declarations for each of the 3 supported data types.

Initialization Functions

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Initializes Real FFT as its process function is used internally in DCT4, by calling `riscv_rfft_init_f32()`.

Use of the initialization function is optional. However, if the initialization function is used, then the instance structure cannot be placed into a const data section. To place an instance structure into a const data section, the instance structure must be manually initialized. Manually initialize the instance structure as follows:

```
riscv_dct4_instance_f32 S = {N, Nby2, normalize, pTwiddle, pCosFactor, pRfft,
    ↪pCfft};
riscv_dct4_instance_q31 S = {N, Nby2, normalize, pTwiddle, pCosFactor, pRfft,
    ↪pCfft};
riscv_dct4_instance_q15 S = {N, Nby2, normalize, pTwiddle, pCosFactor, pRfft,
    ↪pCfft};
```

where N is the length of the DCT4; Nby2 is half of the length of the DCT4; normalize is normalizing factor used and is equal to $\sqrt{2/N}$; pTwiddle points to the twiddle factor table; pCosFactor points to the cosFactor table; pRfft points to the real FFT instance; pCfft points to the complex FFT instance; The CFFT and RFFT structures also needs to be initialized, refer to `riscv_cfft_radix4_f32()` and `riscv_rfft_f32()` respectively for details regarding static initialization.

Fixed-Point Behavior

Care must be taken when using the fixed-point versions of the DCT4 transform functions. In particular, the overflow and saturation behavior of the accumulator used in each function must be considered. Refer to the function specific documentation below for usage guidelines.

MFCC

MFCC F32

```
RISCV_DSP_ATTRIBUTE void riscv_mfcc_f32 (const riscv_mfcc_instance_f32 *S,
float32_t *pSrc, float32_t *pDst, float32_t *pTmp)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_f32 (riscv_mfcc_instance_f32 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs, const float32_t *dctCoefs,
const uint32_t *filterPos, const uint32_t *filterLengths, const float32_t *filterCoefs,
const float32_t *windowCoefs)
```

```
MFCC_INIT_F32(LEN)
```

group MFCC F32

Defines

```
MFCC_INIT_F32(LEN)
```

Functions

RISCV_DSP_ATTRIBUTE void riscv_mfcc_f32 (const riscv_mfcc_instance_f32 *S, float32_t *pSrc, float32_t *pDst, float32_t *pTmp)

MFCC F32.

The source buffer is modified by this function.

Description

The number of input samples if the FFT length used when initializing the instance data structure.

RVV implementation

The RVV implementation has a different API. There is an additional temporary buffer pTmp2. The source buffer is modified.

```
void riscv_mfcc_f32(  
    const riscv_mfcc_instance_f32 * S,  
    float32_t *pSrc,  
    float32_t *pDst,  
    float32_t *pTmp,  
    float32_t *pTmp2  
);
```

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to the mfcc instance structure
- **pSrc** – [in] points to the input samples
- **pDst** – [out] points to the output MFCC values
- **pTmp** – [inout] points to a temporary buffer of complex

RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_f32 (riscv_mfcc_instance_f32 *S, uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs, const float32_t *dctCoefs, const uint32_t *filterPos, const uint32_t *filterLengths, const float32_t *filterCoefs, const float32_t *windowCoefs)

Generic initialization of the MFCC F32 instance structure.

window coefficients can describe (for instance) a Hamming window. The array has the same size as the FFT length.

Description

The matrix of Mel filter coefficients is sparse. Most of the coefficients are zero. To avoid multiplying the spectrogram by those zeros, the filter is applied only to a given position in the spectrogram and on a given number of FFT bins (the filter length). It is the reason for the arrays filterPos and filterLengths.

The folder Scripts is containing a Python script which can be used to generate the filter, dct and window arrays.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the MFCC sizes that you need at build time, then it is better to use the initialization functions defined for each MFCC size.

Parameters

- **S** – [out] points to the mfcc instance structure
- **fftLen** – [in] fft length
- **nbMelFilters** – [in] number of Mel filters
- **nbDctOutputs** – [in] number of Dct outputs
- **dctCoefs** – [in] points to an array of DCT coefficients
- **filterPos** – [in] points of the array of filter positions
- **filterLengths** – [in] points to the array of filter lengths
- **filterCoefs** – [in] points to the array of filter coefficients
- **windowCoefs** – [in] points to the array of window coefficients

Returns

error status

MFCC F16

```
RISCV_DSP_ATTRIBUTE void riscv_mfcc_f16 (const riscv_mfcc_instance_f16 *S,
float16_t *pSrc, float16_t *pDst, float16_t *pTmp)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_f16 (riscv_mfcc_instance_f16 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs, const float16_t *dctCoefs,
const uint32_t *filterPos, const uint32_t *filterLengths, const float16_t *filterCoefs,
const float16_t *windowCoefs)
```

```
MFCC_INIT_F16(LEN)
```

group MFCC F16

Defines

```
MFCC_INIT_F16(LEN)
```

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_mfcc_f16 (const riscv_mfcc_instance_f16 *S,  
float16_t *pSrc, float16_t *pDst, float16_t *pTmp)
```

MFCC F16.

The source buffer is modified by this function.

Description

The number of input samples if the FFT length used when initializing the instance data structure.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to the mfcc instance structure
- **pSrc** – [in] points to the input samples
- **pDst** – [out] points to the output MFCC values
- **pTmp** – [inout] points to a temporary buffer of complex

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_f16 (riscv_mfcc_instance_f16 *S,  
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs,  
const float16_t *dctCoefs, const uint32_t *filterPos, const uint32_t *filterLengths,  
const float16_t *filterCoefs, const float16_t *windowCoefs)
```

Generic initialization of the MFCC F16 instance structure.

window coefficients can describe (for instance) a Hamming window. The array has the same size as the FFT length.

Description

The matrix of Mel filter coefficients is sparse. Most of the coefficients are zero. To avoid multiplying the spectrogram by those zeros, the filter is applied only to a given position in the spectrogram and on a given number of FFT bins (the filter length). It is the reason for the arrays filterPos and filterLengths.

The folder Scripts is containing a Python script that can be used to generate the filter, dct and window arrays.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the MFCC sizes that you need at build time, then it is better to use the initialization functions defined for each MFCC size.

Parameters

- **S** – [out] points to the mfcc instance structure

- **fftLen** – [in] fft length
- **nbMelFilters** – [in] number of Mel filters
- **nbDctOutputs** – [in] number of Dct outputs
- **dctCoefs** – [in] points to an array of DCT coefficients
- **filterPos** – [in] points of the array of filter positions
- **filterLengths** – [in] points to the array of filter lengths
- **filterCoefs** – [in] points to the array of filter coefficients
- **windowCoefs** – [in] points to the array of window coefficients

Returns

error status

MFCC Q15

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_q15 (riscv_mfcc_instance_q15 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs, const q15_t *dctCoefs,
const uint32_t *filterPos, const uint32_t *filterLengths, const q15_t *filterCoefs,
const q15_t *windowCoefs)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_q15 (const riscv_mfcc_instance_q15 *S,
q15_t *pSrc, q15_t *pDst, q31_t *pTmp)
```

```
MFCC_INIT_Q15(LEN)
```

group MFCC Q15

Defines

```
MFCC_INIT_Q15(LEN)
```

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_q15 (riscv_mfcc_instance_q15 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs,
const q15_t *dctCoefs, const uint32_t *filterPos, const uint32_t *filterLengths,
const q15_t *filterCoefs, const q15_t *windowCoefs)
```

Generic initialization of the MFCC Q15 instance structure.

window coefficients can describe (for instance) a Hamming window. The array has the same size as the FFT length.

Description

The matrix of Mel filter coefficients is sparse. Most of the coefficients are zero. To avoid multiplying the spectrogram by those zeros, the filter is applied only to a given position in the spectrogram and on a given number of FFT bins (the filter length). It is the reason for the arrays filterPos and filterLengths.

The folder Scripts is containing a Python script which can be used to generate the filter, dct and window arrays.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the MFCC sizes that you need at build time, then it is better to use the initialization functions defined for each MFCC size.

Parameters

- **S** – [out] points to the mfcc instance structure
- **fftLen** – [in] fft length
- **nbMelFilters** – [in] number of Mel filters
- **nbDctOutputs** – [in] number of Dct outputs
- **dctCoefs** – [in] points to an array of DCT coefficients
- **filterPos** – [in] points of the array of filter positions
- **filterLengths** – [in] points to the array of filter lengths
- **filterCoefs** – [in] points to the array of filter coefficients
- **windowCoefs** – [in] points to the array of window coefficients

Returns

error status

RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_q15 (const riscv_mfcc_instance_q15 *S, q15_t *pSrc, q15_t *pDst, q31_t *pTmp)

MFCC Q15.

The source buffer is modified by this function.

Description

The number of input samples is the FFT length used when initializing the instance data structure.

The function may saturate. If the FFT length is too big and the number of MEL filters too small then the fixed point computations may saturate.

```
riscv_status riscv_mfcc_q15(  
const riscv_mfcc_instance_q15 * S,  
q15_t *pSrc,  
q15_t *pDst,  
q31_t *pTmp,  
q15_t *pTmp_rfft  
)
```


RVV implementation

There is an additional temporary buffer used for the RFFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to the mfcc instance structure
- **pSrc** – [in] points to the input samples in Q15
- **pDst** – [out] points to the output MFCC values in q8.7 format
- **pTmp** – [inout] points to a temporary buffer of complex

Returns

error status

MFCC Q31

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_q31 (riscv_mfcc_instance_q31 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs, const q31_t *dctCoefs,
const uint32_t *filterPos, const uint32_t *filterLengths, const q31_t *filterCoefs,
const q31_t *windowCoefs)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_q31 (const riscv_mfcc_instance_q31 *S,
q31_t *pSrc, q31_t *pDst, q31_t *pTmp)
```

```
MFCC_INIT_Q31(LEN)
```

group MFCC Q31

Defines

```
MFCC_INIT_Q31(LEN)
```

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_init_q31 (riscv_mfcc_instance_q31 *S,
uint32_t fftLen, uint32_t nbMelFilters, uint32_t nbDctOutputs,
const q31_t *dctCoefs, const uint32_t *filterPos, const uint32_t *filterLengths,
const q31_t *filterCoefs, const q31_t *windowCoefs)
```

Generic initialization of the MFCC Q31 instance structure.

window coefficients can describe (for instance) a Hamming window. The array has the same size as the FFT length.

Description

The matrix of Mel filter coefficients is sparse. Most of the coefficients are zero. To avoid multiplying the spectrogram by those zeros, the filter is applied only to a given position in the spectrogram and on a given number of FFT bins (the filter length). It is the reason for the arrays `filterPos` and `filterLengths`.

The folder `Scripts` is containing a Python script which can be used to generate the filter, dct and window arrays.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the MFCC sizes that you need at build time, then it is better to use the initialization functions defined for each MFCC size.

Parameters

- **S** – [out] points to the mfcc instance structure
- **fftLen** – [in] fft length
- **nbMelFilters** – [in] number of Mel filters
- **nbDctOutputs** – [in] number of Dct outputs
- **dctCoefs** – [in] points to an array of DCT coefficients
- **filterPos** – [in] points of the array of filter positions
- **filterLengths** – [in] points to the array of filter lengths
- **filterCoefs** – [in] points to the array of filter coefficients
- **windowCoefs** – [in] points to the array of window coefficients

Returns

error status

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_mfcc_q31 (const riscv_mfcc_instance_q31 *S,  
q31_t *pSrc, q31_t *pDst, q31_t *pTmp)
```

MFCC Q31.

The source buffer is modified by this function.

Description

The number of input samples is the FFT length used when initializing the instance data structure.

The function may saturate. If the FFT length is too big and the number of MEL filters too small then the fixed point computations may saturate.

```
riscv_status riscv_mfcc_q31(  
const riscv_mfcc_instance_q31 * S,  
q31_t *pSrc,  
q31_t *pDst,
```

(continues on next page)

(continued from previous page)

```

q31_t *pTmp,
q31_t *pTmp_rfft
)

```

RVV implementation

There is an additional temporary buffer used for the RFFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to the mfcc instance structure
- **pSrc** – [in] points to the input samples in Q31
- **pDst** – [out] points to the output MFCC values in q8.23 format
- **pTmp** – [inout] points to a temporary buffer of complex

Returns

error status

group **MFCC**

MFCC Transform

There are separate functions for floating-point, Q15, and Q31 data types.

Real FFT Functions**Real FFT F16 Functions**

```
RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f16 (const riscv_rfft_fast_instance_f16 *S,
float16_t *p, float16_t *pOut, uint8_t ifftFlag)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f16 (riscv_rfft_fast_instance_f16 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f16 (riscv_rfft_fast_instance_f16 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f16 (riscv_rfft_fast_instance_f16 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f16 (riscv_rfft_fast_instance_f16 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f16 (riscv_rfft_fast_instance_f16 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f16 (riscv_rfft_fast_instance_f16 *S)
```

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f16 (riscv_rfft_fast_instance_f16 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f16 (riscv_rfft_fast_instance_f16 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f16 (riscv_rfft_fast_instance_f16 *S,
uint16_t fftLen)

group Real FFT F16 Functions

Functions

RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f16 (const riscv_rfft_fast_instance_f16 *S,
float16_t *p, float16_t *pOut, uint8_t ifftFlag)

Processing function for the floating-point real FFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an riscv_rfft_fast_instance_f16 structure
- **p** – [in] points to input buffer (Source buffer is modified by this function.)
- **pOut** – [in] points to output buffer
- **ifftFlag** – [in]
 - value = 0: RFFT
 - value = 1: RIFFT

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 32pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 64pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 128pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 256pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 512pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 1024pt floating-point real FFT.

Parameters

S – [inout] points to an riscv_rfft_fast_instance_f16 structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 2048pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f16` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f16 (riscv_rfft_fast_instance_f16 *S)

Initialization function for the 4096pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f16` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f16 (riscv_rfft_fast_instance_f16 *S, uint16_t fftLen)

Generic initialization function for the floating-point real FFT.

Description

The parameter `fftLen` specifies the length of RFFT/CIFFT process. Supported FFT Lengths are 32, 64, 128, 256, 512, 1024, 2048, 4096.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – [inout] points to an `riscv_rfft_fast_instance_f16` structure
- **fftLen** – [in] length of the Real Sequence

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLen` is not a supported length

Real FFT F64 Functions

RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f64 (riscv_rfft_fast_instance_f64 *S,
float64_t *p, float64_t *pOut, uint8_t ifftFlag)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f64 (riscv_rfft_fast_instance_f64 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f64 (riscv_rfft_fast_instance_f64 *S,
uint16_t fftLen)

group Real FFT F64 Functions

Functions

RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f64 (riscv_rfft_fast_instance_f64 *S,
float64_t *p, float64_t *pOut, uint8_t ifftFlag)

Processing function for the Double Precision floating-point real FFT.

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an riscv_rfft_fast_instance_f64 structure
- **p** – [in] points to input buffer (Source buffer is modified by this function.)
- **pOut** – [in] points to output buffer
- **ifftFlag** – [in]
 - value = 0: RFFT
 - value = 1: RIFFT

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 32pt double precision floating-point real FFT.

Parameters

S – **[inout]** points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 64pt Double Precision floating-point real FFT.

Parameters

S – **[inout]** points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 128pt Double Precision floating-point real FFT.

Parameters

S – **[inout]** points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 256pt Double Precision floating-point real FFT.

Parameters

S – **[inout]** points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 512pt Double Precision floating-point real FFT.

Parameters

S – **[inout]** points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 1024pt Double Precision floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 2048pt Double Precision floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f64 (riscv_rfft_fast_instance_f64 *S)

Initialization function for the 4096pt Double Precision floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f64` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f64 (riscv_rfft_fast_instance_f64 *S, uint16_t fftLen)

Generic initialization function for the Double Precision floating-point real FFT.

Description

The parameter `fftLen` specifies the length of RFFT/CIFFT process. Supported FFT Lengths are 32, 64, 128, 256, 512, 1024, 2048, 4096.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – [inout] points to an `riscv_rfft_fast_instance_f64` structure
- **fftLen** – [in] length of the Real Sequence

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLen` is not a supported length

Real FFT Q15 Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_q15 (riscv_rfft_instance_q15 *S,  
uint32_t fftLenReal, uint32_t ifftFlagR, uint32_t bitReverseFlag)
```

```
RISCV_DSP_ATTRIBUTE void riscv_rfft_q15 (const riscv_rfft_instance_q15 *S, q15_t *pSrc,  
q15_t *pDst)
```

```
RFFTINIT_Q15(LEN, CFFTLEN, TWIDMOD)
```

group **Real FFT Q15 Functions**

Defines

```
RFFTINIT_Q15(LEN, CFFTLEN, TWIDMOD)
```

Functions

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_q15 (riscv_rfft_instance_q15 *S,  
uint32_t fftLenReal, uint32_t ifftFlagR, uint32_t bitReverseFlag)
```

Initialization function for the 8192 pt Q15 real FFT.

Initialization function for the 4096 pt Q15 real FFT.

Initialization function for the 2048 pt Q15 real FFT.

Initialization function for the 1024 pt Q15 real FFT.

Initialization function for the 512 pt Q15 real FFT.

Initialization function for the 256 pt Q15 real FFT.

Initialization function for the 128 pt Q15 real FFT.

Initialization function for the 64 pt Q15 real FFT.

Initialization function for the 32 pt Q15 real FFT.

Generic initialization function for the Q15 RFFT/RIFFT.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

Details

The parameter `fftLenReal` specifies length of RFFT/RIFFT Process. Supported FFT Lengths are 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform

- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q15 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q15 RFFT/RIFFT structure
- **fftLenReal** – [in] length of the FFT
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform

- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : fftLenReal is not a supported length

RISC_V_DSP_ATTRIBUTE void riscv_rfft_q15 (const riscv_rfft_instance_q15 *S, q15_t *pSrc, q15_t *pDst)

Processing function for the Q15 RFFT/RIFFT.

RFFT Size	Input Format	Output Format	Number of bits to upscale
32	1.15	6.10	5
64	1.15	7.9	6
128	1.15	8.8	7
256	1.15	9.7	8
512	1.15	10.6	9
1024	1.15	11.5	10
2048	1.15	12.4	11
4096	1.15	13.3	12
8192	1.15	14.2	13

Input and output formats

Internally input is downscaled by 2 for every stage to avoid saturations inside CFFT/CIFFT process. Hence the output format is different for different RFFT sizes. The input and output formats for different RFFT sizes and number of bits to upscale are mentioned in the tables below for RFFT and RIFFT:

Input and Output formats for RFFT Q15

RIFFT Size	Input Format	Output Format	Number of bits to upscale
32	1.15	6.10	0
64	1.15	7.9	0
128	1.15	8.8	0
256	1.15	9.7	0
512	1.15	10.6	0
1024	1.15	11.5	0
2048	1.15	12.4	0
4096	1.15	13.3	0
8192	1.15	14.2	0

Input and Output formats for RIFFT Q15

```
void riscv_rfft_q15(
const riscv_rfft_instance_q15 * S,
const q15_t * pSrc,
q15_t * pDst,
```

(continues on next page)

(continued from previous page)

```

    q15_t *tmp,
    uint8_t ifftFlag
)

```

RVV implementation

A temporary buffer is required

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an instance of the Q15 RFFT/RIFFT structure
- **pSrc** – [in] points to input buffer (Source buffer is modified by this function.)
- **pDst** – [out] points to output buffer

Real FFT Q31 Functions

```

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_q31 (riscv_rfft_instance_q31 *S,
uint32_t fftLenReal, uint32_t ifftFlagR, uint32_t bitReverseFlag)

```

```

RISCV_DSP_ATTRIBUTE void riscv_rfft_q31 (const riscv_rfft_instance_q31 *S, q31_t *pSrc,
q31_t *pDst)

```

```

RFFTINIT_Q31(LEN, CFFTLEN, TWIDMOD)

```

group **Real FFT Q31 Functions**

Defines

```

RFFTINIT_Q31(LEN, CFFTLEN, TWIDMOD)

```

Functions

```

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_q31 (riscv_rfft_instance_q31 *S,
uint32_t fftLenReal, uint32_t ifftFlagR, uint32_t bitReverseFlag)

```

Initialization function for the 8192 pt Q31 real FFT.

Initialization function for the 4096 pt Q31 real FFT.

Initialization function for the 2048 pt Q31 real FFT.

Initialization function for the 1024 pt Q31 real FFT.

Initialization function for the 512 pt Q31 real FFT.

Initialization function for the 256 pt Q31 real FFT.

Initialization function for the 128 pt Q31 real FFT.

Initialization function for the 64 pt Q31 real FFT.

Initialization function for the 32 pt Q31 real FFT.

Generic initialization function for the Q31 RFFT/RIFFT.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

Details

The parameter `fftLenReal` specifies length of RFFT/RIFFT Process. Supported FFT Lengths are 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – **[inout]** points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – **[in]** flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – **[in]** flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – **[inout]** points to an instance of the Q31 RFFT/RIFFT structure

- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q31 RFFT/RIFFT structure
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output
- **S** – [inout] points to an instance of the Q31 RFFT/RIFFT structure
- **fftLenReal** – [in] length of the FFT

- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISCV_MATH_SUCCESS : Operation successful
- RISCV_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : `fftLenReal` is not a supported length

RISC_V_DSP_ATTRIBUTE void riscv_rfft_q31 (const riscv_rfft_instance_q31 *S, q31_t *pSrc, q31_t *pDst)

Processing function for the Q31 RFFT/RIFFT.

RFFT Size	Input Format	Output Format	Number of bits to upscale
32	1.31	6.26	5
64	1.31	7.25	6
128	1.31	8.24	7
256	1.31	9.23	8
512	1.31	10.22	9
1024	1.31	11.21	10
2048	1.31	12.20	11
4096	1.31	13.19	12
8192	1.31	14.18	13

Input and output formats

Internally input is downscaled by 2 for every stage to avoid saturations inside CFFT/CIFFT process. Hence the output format is different for different RFFT sizes. The input and output formats for different RFFT sizes and number of bits to upscale are mentioned in the tables below for RFFT and RIFFT:

Input and Output formats for RFFT Q31

RIFFT Size	Input Format	Output Format	Number of bits to upscale
32	1.31	6.26	0
64	1.31	7.25	0
128	1.31	8.24	0
256	1.31	9.23	0
512	1.31	10.22	0
1024	1.31	11.21	0
2048	1.31	12.20	0
4096	1.31	13.19	0
8192	1.31	14.18	0

Input and Output formats for RIFFT Q31

```
void riscv_rfft_q31(  
const riscv_rfft_instance_q31 * S,  
    const q31_t * pSrc,  
    q31_t * pDst,  
    q31_t * tmp,  
    uint8_t ifftFlag  
)
```

RVV implementation

A temporary buffer is needed

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an instance of the Q31 RFFT/RIFFT structure
- **pSrc** – [in] points to input buffer (Source buffer is modified by this function)
- **pDst** – [out] points to output buffer

Real FFT Tables

```
const q15_t __ALIGNED (4)
```

```
const float32_t (page 1277) realCoefA[8192]
```

```
const float32_t (page 1277) realCoefB[8192]
```

```
const q31_t (page 1277) realCoefAQ31[8192]
```

```
const q31_t (page 1277) realCoefBQ31[8192]
```

group **Real FFT Tables**

Functions

```
const q15_t __ALIGNED (4)
```

Weights Table.

Q15 table for reciprocal.

end of DCT4_IDCT4_Table group

Generation fixed-point realCoefAQ15 array in Q15 format:

n = 4096

```
for (i = 0; i < n; i++)
{
    pATable[2 * i]      = 0.5 * ( 1.0 - sin (2 * PI / (double) (2 * n) *
    ↪(double) i));
    pATable[2 * i + 1] = 0.5 * (-1.0 * cos (2 * PI / (double) (2 * n) *
    ↪(double) i));
}
```

Convert to fixed point Q15 format round(pATable[i] * pow(2, 15))

Generation of real_CoefB array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pBTable[2 * i]      = 0.5 * (1.0 + sin (2 * PI / (double) (2 * n) *
    ↪(double) i));
    pBTable[2 * i + 1] = 0.5 * (1.0 * cos (2 * PI / (double) (2 * n) *
    ↪(double) i));
}
```

Convert to fixed point Q15 format round(pBTable[i] * pow(2, 15))

Weights tables are generated using the formula :

$$\text{weights}[n] = e^{(-j * n * \pi / (2 * N))}$$

C command to generate the table

```
for(i = 0; i < N; i++)
{
    weights[(2*i)]   = cos(i*c);
    weights[(2*i)+1] = -sin(i*c);
}
```

where N is the Number of weights to be calculated and c is $\pi / (2 * N)$

Converted the output to q15 format by multiplying with 2^{31} and saturated if required.

In the tables below the real and imaginary values are placed alternatively, hence the array length is $2 * N$.

cosFactor tables are generated using the formula :

```
cos_factors[n] = 2 * cos((2n+1)*pi/(4*N))
```

C command to generate the table

```
for (i = 0; i < N; i++)
{
    cos_factors[i] = 2 * cos((2*i+1)*c/2);
}
```

where N is the number of factors to generate and c is $\pi/(2*N)$

Then converted to q15 format by multiplying with 2^{31} and saturated if required.

Variables

const *float32_t* (page 1277) **realCoefA**[8192]

Generation of realCoefA array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pATable[2 * i]      = 0.5 * ( 1.0 - sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pATable[2 * i + 1] = 0.5 * (-1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

const *float32_t* (page 1277) **realCoefB**[8192]

Generation of realCoefB array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pBTable[2 * i]      = 0.5 * (1.0 + sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pBTable[2 * i + 1] = 0.5 * (1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

const *q31_t* (page 1277) **realCoefAQ31**[8192]

Generation fixed-point realCoefAQ31 array in Q31 format:

n = 4096

```
for (i = 0; i < n; i++)
{
    pATable[2 * i]      = 0.5 * ( 1.0 - sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pATable[2 * i + 1] = 0.5 * (-1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

Convert to fixed point Q31 format round(pATable[i] * pow(2, 31))

const *q31_t* (page 1277) **realCoefBQ31**[8192]

Generation of realCoefBQ31 array:

n = 4096

```
for (i = 0; i < n; i++)
{
    pBTable[2 * i]      = 0.5 * (1.0 + sin (2 * PI / (double) (2 * n) *
↪(double) i));
    pBTable[2 * i + 1] = 0.5 * (1.0 * cos (2 * PI / (double) (2 * n) *
↪(double) i));
}
```

Convert to fixed point Q31 format round(pBTable[i] * pow(2, 31))

Real FFT F32 Functions

RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f32 (const riscv_rfft_fast_instance_f32 *S,
float32_t *p, float32_t *pOut, uint8_t ifftFlag)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f32 (riscv_rfft_fast_instance_f32 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f32 (riscv_rfft_fast_instance_f32 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f32 (riscv_rfft_fast_instance_f32 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f32 (riscv_rfft_fast_instance_f32 *S)

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f32 (riscv_rfft_fast_instance_f32 *S)

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f32 (riscv_rfft_fast_instance_f32 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f32 (riscv_rfft_fast_instance_f32 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f32 (riscv_rfft_fast_instance_f32 *S)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f32 (riscv_rfft_fast_instance_f32 *S,  
uint16_t fftLen)
```

group Real FFT F32 Functions

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_rfft_fast_f32 (const riscv_rfft_fast_instance_f32 *S,  
float32_t *p, float32_t *pOut, uint8_t ifftFlag)
```

Processing function for the floating-point real FFT.

RVV version

The rvv version has a different API. The input and output buffers must be different. There is a temporary buffer.

The bit reverse flag is not more available in rvv version.

```
void riscv_rfft_fast_f32(  
    const riscv_rfft_fast_instance_f32 * S,  
    const float32_t * p,  
    float32_t * pOut,  
    float32_t * tmpbuf,  
    uint8_t ifftFlag);
```

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Parameters

- **S** – [in] points to an `riscv_rfft_fast_instance_f32` structure
- **p** – [in] points to input buffer (Source buffer is modified by this function.)
- **pOut** – [in] points to output buffer
- **ifftFlag** – [in]
 - value = 0: RFFT
 - value = 1: RIFFT

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_32_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 32pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_64_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 64pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_128_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 128pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_256_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 256pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : an error is detected

RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_512_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 512pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_1024_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 1024pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_2048_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 2048pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_4096_f32 (riscv_rfft_fast_instance_f32 *S)

Initialization function for the 4096pt floating-point real FFT.

Parameters

S – [inout] points to an `riscv_rfft_fast_instance_f32` structure

Returns

execution status

- RISC_V_MATH_SUCCESS : Operation successful
- RISC_V_MATH_ARGUMENT_ERROR : an error is detected

RISC_V_DSP_ATTRIBUTE riscv_status riscv_rfft_fast_init_f32 (riscv_rfft_fast_instance_f32 *S, uint16_t fftLen)

Generic initialization function for the floating-point real FFT.

Description

The parameter `fftLen` specifies the length of RFFT/CIFFT process. Supported FFT Lengths are 32, 64, 128, 256, 512, 1024, 2048, 4096.

This Function also initializes Twiddle factor table pointer and Bit reversal table pointer.

This function should be used only if you don't know the FFT sizes that you'll need at build time. The use of this function will prevent the linker from removing the FFT tables that are not needed and the library code size will be bigger than needed.

If you use NMSIS-DSP as a static library, and if you know the FFT sizes that you need at build time, then it is better to use the initialization functions defined for each FFT size.

Parameters

- **S** – [inout] points to an `riscv_rfft_fast_instance_f32` structure
- **fftLen** – [in] length of the Real Sequence

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLen` is not a supported length

Deprecated Real FFT Functions

```
RISCV_DSP_ATTRIBUTE void riscv_rfft_f32 (const riscv_rfft_instance_f32 *S,
float32_t *pSrc, float32_t *pDst)
```

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_f32 (riscv_rfft_instance_f32 *S,
riscv_cfft_radix4_instance_f32 *S_CFFT, uint32_t fftLenReal, uint32_t ifftFlagR,
uint32_t bitReverseFlag)
```

group **Deprecated Real FFT Functions**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_rfft_f32 (const riscv_rfft_instance_f32 *S,
float32_t *pSrc, float32_t *pDst)
```

Processing function for the floating-point RFFT/RIFFT. Source buffer is modified by this function.

Deprecated:

Do not use this function. It has been superceded by `riscv_rfft_fast_f32` and will be removed in the future.

Parameters

- **S** – [in] points to an instance of the floating-point RFFT/RIFFT structure
- **pSrc** – [in] points to the input buffer
- **pDst** – [out] points to the output buffer

```
RISCV_DSP_ATTRIBUTE riscv_status riscv_rfft_init_f32 (riscv_rfft_instance_f32 *S,
riscv_cfft_radix4_instance_f32 *S_CFFT, uint32_t fftLenReal, uint32_t ifftFlagR,
uint32_t bitReverseFlag)
```

Initialization function for the floating-point RFFT/RIFFT.

Deprecated:

Do not use this function. It has been superceded by `riscv_rfft_fast_init_f32` and will be removed in the future.

Description

The parameter `fftLenReal` specifies length of RFFT/RIFFT Process. Supported FFT Lengths are 128, 512, 2048.

The parameter `ifftFlagR` controls whether a forward or inverse transform is computed. Set(=1) `ifftFlagR` to calculate RIFFT, otherwise RFFT is calculated.

The parameter `bitReverseFlag` controls whether output is in normal order or bit reversed order. Set(=1) `bitReverseFlag` for output to be in normal order otherwise output is in bit reversed order.

This function also initializes Twiddle factor table.

Parameters

- **S** – [inout] points to an instance of the floating-point RFFT/RIFFT structure
- **S_CFFT** – [inout] points to an instance of the floating-point CFFT/CIFFT structure
- **fftLenReal** – [in] length of the FFT.
- **ifftFlagR** – [in] flag that selects transform direction
 - value = 0: forward transform
 - value = 1: inverse transform
- **bitReverseFlag** – [in] flag that enables / disables bit reversal of output
 - value = 0: disables bit reversal of output
 - value = 1: enables bit reversal of output

Returns

execution status

- `RISCV_MATH_SUCCESS` : Operation successful
- `RISCV_MATH_ARGUMENT_ERROR` : `fftLenReal` is not a supported length

group **Real FFT Functions**

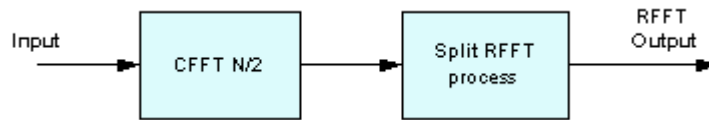
For f16, the functions are `riscv_rfft_fast_f16()` and `riscv_rfft_fast_init_f16()`. For f64, the functions are `riscv_rfft_fast_f64()` and `riscv_rfft_fast_init_f64()`.

The bit reverse flag is not more available in rvv version.

The NMSIS DSP library includes specialized algorithms for computing the FFT of real data sequences. The FFT is defined over complex data but in many applications the input is real. Real FFT algorithms take advantage of the symmetry properties of the FFT and have a speed advantage over complex algorithms of the same length.

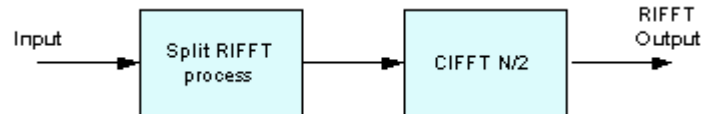
The Fast RFFT algorithm relays on the mixed radix CFFT that save processor usage.

The real length N forward FFT of a sequence is computed using the steps shown below.



The real sequence is initially treated as if it were complex to perform a CFFT. Later, a processing stage reshapes the data to obtain half of the frequency spectrum in complex format.

The input for the inverse RFFT should keep the same format as the output of the forward RFFT. A first processing stage pre-process the data to later perform an inverse CFFT.



The algorithms for floating-point, Q15, and Q31 data are slightly different and we describe each algorithm in turn.

Floating-point

The main functions are `riscv_rfft_fast_f32()` and `riscv_rfft_fast_init_f32()`.

The FFT of a real N -point sequence has even symmetry in the frequency domain. The second half of the data equals the conjugate of the first half flipped in frequency. This conjugate part is not computed by the float RFFT. As consequence, the output of a N point real FFT should be a $N/2 + 1$ complex numbers so $N + 2$ floats.

It happens that the first complex of number of the RFFT output is actually all real. Its real part represents the DC offset. The value at Nyquist frequency is also real.

Those two complex numbers can be encoded with 2 floats rather than using two numbers with an imaginary part set to zero.

The implementation is using a trick so that the output buffer can be N float : the last real is packaged in the imaginary part of the first complex (since this imaginary part is not used and is zero).

The real FFT functions pack the frequency domain data in this fashion. The forward transform outputs the data in this form and the inverse transform expects input data in this form. The function always performs the needed bitreversal so that the input and output data is always in normal order. The functions support lengths of [32, 64, 128, ..., 4096] samples.

Q15 and Q31

The real algorithms are defined in a similar manner and utilize $N/2$ complex transforms behind the scenes.

But warning, contrary to the float version, the fixed point implementation RFFT is also computing the conjugate part (except for RVV version) so the output buffer must be bigger. Also the fixed point RFFTs

are not using any trick to pack the DC and Nyquist frequency in the same complex number. The RIFFT is not using the conjugate part but it is still using the Nyquist frequency value. The details are given in the documentation for the functions.

The complex transforms used internally include scaling to prevent fixed-point overflows. The overall scaling equals $1/(\text{fftLen}/2)$. Due to the use of complex transform internally, the source buffer is modified by the rfft.

A separate instance structure must be defined for each transform used but twiddle factor and bit reversal tables can be reused.

There is also an associated initialization function for each data type. The initialization function performs the following operations:

- Sets the values of the internal structure fields.
- Initializes twiddle factor table and bit reversal table pointers.
- Initializes the internal complex FFT data structure.

Use of the initialization function is optional **except for RVV versions where it is mandatory**. If you don't use the initialization functions, then the structures should be initialized with code similar to the one below:

```
riscv_rfft_instance_q31 S = {fftLenReal, fftLenBy2, ifftFlagR, bitReverseFlagR,
↪twidCoefRModifier, pTwiddleAReal, pTwiddleBReal, pCfft};
riscv_rfft_instance_q15 S = {fftLenReal, fftLenBy2, ifftFlagR, bitReverseFlagR,
↪twidCoefRModifier, pTwiddleAReal, pTwiddleBReal, pCfft};
```

where `fftLenReal` is the length of the real transform; `fftLenBy2` length of the internal complex transform (`fftLenReal/2`). `ifftFlagR` Selects forward (`=0`) or inverse (`=1`) transform. `bitReverseFlagR` Selects bit reversed output (`=0`) or normal order output (`=1`). `twidCoefRModifier` stride modifier for the twiddle factor table. The value is based on the FFT length; `pTwiddleAReal` points to the A array of twiddle coefficients; `pTwiddleBReal` points to the B array of twiddle coefficients; `pCfft` points to the CFFT Instance structure. The CFFT structure must also be initialized.

Note that with RVV versions you can't initialize instance structures directly and **must use the initialization function**.

RVV version

The rvv version has a different API. The input and output buffers must be different. There is a temporary buffer that is not optional.

```
void riscv_rfft_fast_f32(
    const riscv_rfft_fast_instance_f32 * S,
    float32_t * p,
    float32_t * pOut,
    float32_t * tmpbuf,
    uint8_t ifftFlag);
```

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

Transform Buffer Sizes

Buffer Sizes for CFFT

int32_t **riscv_cfft_tmp_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

int32_t **riscv_cfft_output_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

int32_t **riscv_cifft_output_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

group **Buffer Sizes for CFFT**

Functions

int32_t **riscv_cfft_tmp_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

Calculate required length for the temporary buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples

Returns

Length in datatype elements (real numbers) for the temporary buffer

int32_t **riscv_cfft_output_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

Calculate required length for the output buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples in the input data

Returns

Length in datatype elements (real numbers) for the output buffer

`int32_t riscv_cifft_output_buffer_size(riscv_math_datatype (page 1275) dt, uint32_t nb_samples)`

Calculate required length for the output buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples in the input data

Returns

Length in datatype elements (real numbers) for the output buffer

Buffer Sizes for RFFT

`int32_t riscv_rfft_tmp_buffer_size(riscv_math_datatype (page 1275) dt, uint32_t nb_samples)`

`int32_t riscv_rfft_output_buffer_size(riscv_math_datatype (page 1275) dt, uint32_t nb_samples)`

`int32_t riscv_rifft_input_buffer_size(riscv_math_datatype (page 1275) dt, uint32_t nb_samples)`

group **Buffer Sizes for RFFT**

end of CFFTBuffers group

Functions

`int32_t riscv_rfft_tmp_buffer_size(riscv_math_datatype (page 1275) dt, uint32_t nb_samples)`

Calculate required length for the temporary buffer for both RFFT and RIFFT.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples in the input data

Returns

Length in datatype elements (real numbers) for the temporary buffer

int32_t **riscv_rfft_output_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

Calculate required length for the output buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples in the input data

Returns

Length in datatype elements (real numbers) for the output buffer

int32_t **riscv_rifft_input_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples)

Calculate required length for the input buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] RFFT length in samples

Returns

Length in datatype elements (real numbers) for the input buffer

Buffer Sizes for MFCC

int32_t **riscv_mfcc_tmp_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples, uint32_t buf_id, uint32_t use_cfft)

group **Buffer Sizes for MFCC**
end of RFFTBuffers group

Functions

int32_t **riscv_mfcc_tmp_buffer_size**(*riscv_math_datatype* (page 1275) dt, uint32_t nb_samples, uint32_t buf_id, uint32_t use_cfft)

Calculate required length for the temporary buffer.

Note

0 means not applicable (temporary buffer not needed)

Note

-1 means error : configuration not supported

Note

The define RISCV_MFCC_USE_CFFT can be checked to know the current MFCC mode (CFFT or RFFT) for the build

Parameters

- **dt** – [in] Data type of the input data
- **nb_samples** – [in] Number of samples in the input data
- **buf_id** – [in] Identification for the temporary buffer
- **use_cfft** – [in] 1 if implementation uses CFFT, 0 if RFFT

Returns

Length in datatype elements (real numbers) for the temporary buffer

group **Transform Buffer Sizes**

Functions to get the sizes of the buffers in the different transform functions according to the datatype and target architecture.

The current architecture for the build is defined in RISCV_MATH_DEFAULT_TARGET_ARCH

Note

All returned lengths are in real numbers and never in complex. So, for a complex buffer of S samples, the returned length will be 2*S.

group **Transform Functions**

NMSIS-DSP provides CFFT and RFFT for different architectures. The implementation of those transforms may be different for the different architectures : different algorithms, different capabilities of the instruction set.

All those variants are not giving exactly the same results but they are passing the same tests with same SNR checks and same threshold for the sample errors.

The APIs for the RISC-V Vector (RVV) variants are different : some additional temporary buffers are required.

Float16 versions are provided but they are not very accurate and should only be used for small FFTs.

There are several ways to initialize the transform functions. Below explanations are using q15 as example but the same explanations apply to other datatypes.

Transform initializations

For the scalar version, you can just use a pre-initialized constant data structure and use it in the `riscv_cfft_...` function. For instance, `&riscv_cfft_sR_q15_len256` for a 256 Q15 CFFT.

If your CPU platform support RISC-V Vector extension, you *must* use an initialization function. If you know the size of your FFT (or other transform) you can use a specific initialization function for this size only : like `riscv_cfft_init_256_q15` for a 256 Q15 complex FFT.

By using a specific initialization function, you give an hint to the linker and it will be able to remove all unused initialization tables (some compilation and link flags must be used for the linker to be able to do this optimization. It is compiler dependent).

If you don't know the size you'll need at runtime, you need to use a function like `riscv_cfft_init_q15`. If you use such a function, all the tables for all FFT sizes (up to the NMSIS-DSP maximum of 4096) will be included in the build !

On RVV, there is another possibility. You can use `riscv_cfft_init_dynamic_q15`. This function will allocate a buffer at runtime and compute at runtime all the tables that are required for a specific FFT size. This initialization is also supported by RFFT. The computation to initialize all the tables can take lot of cycles (since several cos and sin must be computed)

Size of buffers according to the target architecture and datatype:

They are described on the page transform buffers.

3.3.16 Window Functions**Flat-top window functions****Hft116d window function (116.8 dB)**

```
RISCV_DSP_ATTRIBUTE void riscv_hft116d_f32 (float32_t *pDst, uint32_t blockSize)
```

RISCV_DSP_ATTRIBUTE void riscv_hft116d_f64 (float64_t *pDst, uint32_t blockSize)

group Hft116d window function (116.8 dB)

Functions

RISCV_DSP_ATTRIBUTE void riscv_hft116d_f32 (float32_t *pDst, uint32_t blockSize)

Hft116d window generating function (f32).

Hft116d window (float).

Parameter	Value
Peak sidelobe level	116.8 dB
Normalized equivalent noise bandwidth	4.2186 bins
3 dB bandwidth	4.1579 bins
Flatness	-0.0028 dB
Recommended overlap	78.2 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hft116d_f64 (float64_t *pDst, uint32_t blockSize)

Hft116d window generating function (f64).

Hft116d window (double).

Parameter	Value
Peak sidelobe level	116.8 dB
Normalized equivalent noise bandwidth	4.2186 bins
3 dB bandwidth	4.1579 bins
Flatness	-0.0028 dB
Recommended overlap	78.2 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft144d window function (144.1 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft144d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft144d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft144d window function (144.1 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft144d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft144d window generating function (f32).

Hft144d window (float).

Parameter	Value
Peak sidelobe level	144.1 dB
Normalized equivalent noise bandwidth	4.5386 bins
3 dB bandwidth	4.4697 bins
Flatness	0.0021 dB
Recommended overlap	79.9 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hft144d_f64 (float64_t *pDst, uint32_t blockSize)

Hft144d window generating function (f64).

Hft144d window (double).

Parameter	Value
Peak sidelobe level	144.1 dB
Normalized equivalent noise bandwidth	4.5386 bins
3 dB bandwidth	4.4697 bins
Flatness	0.0021 dB
Recommended overlap	79.9 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft169d window function (169.5 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft169d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft169d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft169d window function (169.5 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft169d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft169d window generating function (f32).

Hft169d window (float).

Parameter	Value
Peak sidelobe level	169.5 dB
Normalized equivalent noise bandwidth	4.8347 bins
3 dB bandwidth	4.7588 bins
Flatness	0.0017 dB
Recommended overlap	81.2 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_hft169d_f64 (float64_t *pDst, uint32_t blockSize)
```

Hft169d window generating function (f64).

Hft169d window (double).

Parameter	Value
Peak sidelobe level	169.5 dB
Normalized equivalent noise bandwidth	4.8347 bins
3 dB bandwidth	4.7588 bins
Flatness	0.0017 dB
Recommended overlap	81.2 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft196d window function (196.2 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft196d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft196d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft196d window function (196.2 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft196d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft196d window generating function (f32).

Hft196d window (float).

Parameter	Value
Peak sidelobe level	196.2 dB
Normalized equivalent noise bandwidth	5.1134 bins
3 dB bandwidth	5.0308 bins
Flatness	0.0013 dB
Recommended overlap	82.3 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hft196d_f64 (float64_t *pDst, uint32_t blockSize)

Hft196d window generating function (f64).

Hft196d window (double).

Parameter	Value
Peak sidelobe level	196.2 dB
Normalized equivalent noise bandwidth	5.1134 bins
3 dB bandwidth	5.0308 bins
Flatness	0.0013 dB
Recommended overlap	82.3 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft223d window function (223.0 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft223d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft223d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft223d window function (223.0 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft223d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft223d window generating function (f32).

Hft223d window (float).

Parameter	Value
Peak sidelobe level	223.0 dB
Normalized equivalent noise bandwidth	5.3888 bins
3 dB bandwidth	5.3000 bins
Flatness	-0.0011 dB
Recommended overlap	83.3 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_hft223d_f64 (float64_t *pDst, uint32_t blockSize)
```

Hft223d window generating function (f64).

Hft223d window (double).

Parameter	Value
Peak sidelobe level	223.0 dB
Normalized equivalent noise bandwidth	5.3888 bins
3 dB bandwidth	5.3000 bins
Flatness	0.0011 dB
Recommended overlap	83.3 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft248d window function (248.4 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft248d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft248d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft248d window function (248.4 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft248d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft248d window generating function (f32).

Hft248d window (float).

Parameter	Value
Peak sidelobe level	248.4 dB
Normalized equivalent noise bandwidth	5.6512 bins
3 dB bandwidth	5.5567 bins
Flatness	0.0009 dB
Recommended overlap	84.1 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hft248d_f64 (float64_t *pDst, uint32_t blockSize)

Hft248d window generating function (f64).

Hft248d window (double).

Parameter	Value
Peak sidelobe level	248.4 dB
Normalized equivalent noise bandwidth	5.6512 bins
3 dB bandwidth	5.5567 bins
Flatness	0.0009 dB
Recommended overlap	84.1 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft90d window function (90.2 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft90d_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft90d_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft90d window function (90.2 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft90d_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft90d window generating function (f32).

Hft90d window (float).

Parameter	Value
Peak sidelobe level	90.2 dB
Normalized equivalent noise bandwidth	3.8832 bins
3 dB bandwidth	3.8320 bins
Flatness	-0.0039 dB
Recommended overlap	76.0 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_hft90d_f64 (float64_t *pDst, uint32_t blockSize)
```

Hft90d window generating function (f64).

Hft90d window (double).

Parameter	Value
Peak sidelobe level	90.2 dB
Normalized equivalent noise bandwidth	3.8832 bins
3 dB bandwidth	3.8320 bins
Flatness	-0.0039 dB
Recommended overlap	76.0 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hft95 window function (95.0 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_hft95_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_hft95_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Hft95 window function (95.0 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_hft95_f32 (float32_t *pDst, uint32_t blockSize)
```

Hft95 window generating function (f32).

Hft95 window (float).

Parameter	Value
Peak sidelobe level	95.0 dB
Normalized equivalent noise bandwidth	3.8112 bins
3 dB bandwidth	3.7590 bins
Flatness	0.0044 dB
Recommended overlap	75.6 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hft95_f64 (float64_t *pDst, uint32_t blockSize)

Hft95 window generating function (f64).

Hft95 window (double).

Parameter	Value
Peak sidelobe level	95.0 dB
Normalized equivalent noise bandwidth	3.8112 bins
3 dB bandwidth	3.7590 bins
Flatness	0.0044 dB
Recommended overlap	75.6 %

Parameters of the window

Included in NMSIS-DSP with authorization from professor Gerhard Heinzel.

Original article:

Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.

Authors:

G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut für Gravitationsphysik (Albert-Einstein-Institut) Teilinstitut Hannover

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

group **Flat-top window functions**

Use those windows when you need to estimate the amplitude of a tone.

If just need to detect a tone or estimate the noise, you can use the regular windows.

Regular window functions

Bartlett window function (26.5 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_bartlett_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_bartlett_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Bartlett window function (26.5 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_bartlett_f32 (float32_t *pDst, uint32_t blockSize)
```

Bartlett window generating function (f32).

Bartlett window (float).

Parameter	Value
Peak sidelobe level	26.5 dB
Normalized equivalent noise bandwidth	1.3333 bins
3 dB bandwidth	1.2736 bins
Flatness	-1.8242 dB
Recommended overlap	50.0 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_bartlett_f64 (float64_t *pDst, uint32_t blockSize)
```

Bartlett window generating function (f64).

Bartlett window (double).

Parameter	Value
Peak sidelobe level	26.5 dB
Normalized equivalent noise bandwidth	1.3333 bins
3 dB bandwidth	1.2736 bins
Flatness	-1.8242 dB
Recommended overlap	50.0 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Blackman Harris window function (92 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_blackman_harris_92db_f32 (float32_t *pDst,
uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_blackman_harris_92db_f64 (float64_t *pDst,
uint32_t blockSize)
```

group **Blackman Harris window function (92 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_blackman_harris_92db_f32 (float32_t *pDst,
uint32_t blockSize)
```

92 dB Blackman Harris window generating function (f32).

92 db blackman harris window (float).

Parameter	Value
Peak sidelobe level	92.0 dB
Normalized equivalent noise bandwidth	2.0044 bins
3 dB bandwidth	1.8962 bins
Flatness	-0.8256 dB
Recommended overlap	66.1 %

Parameters of the window**Parameters**

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_blackman_harris_92db_f64 (float64_t *pDst,
uint32_t blockSize)
```

92 dB Blackman Harris window generating function (f64).

92 db blackman harris window (double).

Parameter	Value
Peak sidelobe level	92.0 dB
Normalized equivalent noise bandwidth	2.0044 bins
3 dB bandwidth	1.8962 bins
Flatness	-0.8256 dB
Recommended overlap	66.1 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hamming window function (42.7 dB)

RISCV_DSP_ATTRIBUTE void riscv_hamming_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_hamming_f64 (float64_t *pDst, uint32_t blockSize)

group Hamming window function (42.7 dB)

Functions

RISCV_DSP_ATTRIBUTE void riscv_hamming_f32 (float32_t *pDst, uint32_t blockSize)

Hamming window generating function (f32).

Hamming window (float).

Parameter	Value
Peak sidelobe level	42.7 dB
Normalized equivalent noise bandwidth	1.3628 bins
3 dB bandwidth	1.3008 bins
Flatness	-1.7514 dB
Recommended overlap	50 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hamming_f64 (float64_t *pDst, uint32_t blockSize)

Hamming window generating function (f64).

Hamming window (double).

Parameter	Value
Peak sidelobe level	42.7 dB
Normalized equivalent noise bandwidth	1.3628 bins
3 dB bandwidth	1.3008 bins
Flatness	-1.7514 dB
Recommended overlap	50 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Hanning window function (31.5 dB)

RISCV_DSP_ATTRIBUTE void riscv_hanning_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_hanning_f64 (float64_t *pDst, uint32_t blockSize)

group **Hanning window function (31.5 dB)**

Functions

RISCV_DSP_ATTRIBUTE void riscv_hanning_f32 (float32_t *pDst, uint32_t blockSize)

Hanning window generating function (f32).

Hanning window (float).

Parameter	Value
Peak sidelobe level	31.5 dB
Normalized equivalent noise bandwidth	1.5 bins
3 dB bandwidth	1.4382 bins
Flatness	-1.4236 dB
Recommended overlap	50 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_hanning_f64 (float64_t *pDst, uint32_t blockSize)

Hanning window generating function (f64).

Hanning window (double).

Parameter	Value
Peak sidelobe level	31.5 dB
Normalized equivalent noise bandwidth	1.5 bins
3 dB bandwidth	1.4382 bins
Flatness	-1.4236 dB
Recommended overlap	50 %

Parameters of the window**Parameters**

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall3 window function (46.7 dB)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3_f64 (float64_t *pDst, uint32_t blockSize)

group **Nuttall3 window function (46.7 dB)**

Functions

RISCV_DSP_ATTRIBUTE void riscv_nuttall3_f32 (float32_t *pDst, uint32_t blockSize)

Nuttall3 window generating function (f32).

Nuttall3 window (float).

Parameter	Value
Peak sidelobe level	46.7 dB
Normalized equivalent noise bandwidth	1.9444 bins
3 dB bandwidth	1.8496 bins
Flatness	-0.8630 dB
Recommended overlap	64.7 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_nuttall3_f64 (float64_t *pDst, uint32_t blockSize)

Nuttall3 window generating function (f64).

Nuttall3 window (double).

Parameter	Value
Peak sidelobe level	46.7 dB
Normalized equivalent noise bandwidth	1.9444 bins
3 dB bandwidth	1.8496 bins
Flatness	-0.863 dB
Recommended overlap	64.7 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall3a window function (64.2 dB)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3a_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3a_f64 (float64_t *pDst, uint32_t blockSize)

group **Nuttall3a window function (64.2 dB)**

Functions

RISCV_DSP_ATTRIBUTE void riscv_nuttall3a_f32 (float32_t *pDst, uint32_t blockSize)

Nuttall3a window generating function (f32).

Nuttall3a window (float).

Parameter	Value
Peak sidelobe level	64.2 dB
Normalized equivalent noise bandwidth	1.7721 bins
3 dB bandwidth	1.6828 bins
Flatness	-1.0453 dB
Recommended overlap	61.2 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_nuttall3a_f64 (float64_t *pDst, uint32_t blockSize)

Nuttall3a window generating function (f64).

Nuttall3a window (double).

Parameter	Value
Peak sidelobe level	64.2 dB
Normalized equivalent noise bandwidth	1.7721 bins
3 dB bandwidth	1.6828 bins
Flatness	-1.0453 dB
Recommended overlap	61.2 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall3b window function (71.5 dB)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3b_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_nuttall3b_f64 (float64_t *pDst, uint32_t blockSize)

group **Nuttall3b window function (71.5 dB)**

Functions

RISCV_DSP_ATTRIBUTE void riscv_nuttall3b_f32 (float32_t *pDst, uint32_t blockSize)

Nuttall3b window generating function (f32).

Nuttall3b window (float).

Parameter	Value
Peak sidelobe level	71.5 dB
Normalized equivalent noise bandwidth	1.7037 bins
3 dB bandwidth	1.6162 bins
Flatness	-1.1352 dB
Recommended overlap	59.8 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_nuttall3b_f64 (float64_t *pDst, uint32_t blockSize)

Nuttall3b window generating function (f64).

Nuttall3b window (double).

Parameter	Value
Peak sidelobe level	71.5 dB
Normalized equivalent noise bandwidth	1.7037 bins
3 dB bandwidth	1.6162 bins
Flatness	-1.1352 dB
Recommended overlap	59.8 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall4 window function (60.9 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4_f64 (float64_t *pDst, uint32_t blockSize)
```

group **Nuttall4 window function (60.9 dB)**

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4_f32 (float32_t *pDst, uint32_t blockSize)
```

Nuttall4 window generating function (f32).

Nuttall4 window (float).

Parameter	Value
Peak sidelobe level	60.9 dB
Normalized equivalent noise bandwidth	2.31 bins
3 dB bandwidth	2.1884 bins
Flatness	-0.6184 dB
Recommended overlap	70.5 %

Parameters of the window**Parameters**

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4_f64 (float64_t *pDst, uint32_t blockSize)
```

Nuttall4 window generating function (f64).

Nuttall4 window (double).

Parameter	Value
Peak sidelobe level	60.9 dB
Normalized equivalent noise bandwidth	2.31 bins
3 dB bandwidth	2.1884 bins
Flatness	-0.6184 dB
Recommended overlap	70.5 %

Parameters of the window**Parameters**

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall4a window function (82.6 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4a_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4a_f64 (float64_t *pDst, uint32_t blockSize)
```

group Nuttall4a window function (82.6 dB)

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4a_f32 (float32_t *pDst, uint32_t blockSize)
```

Nuttall4a window generating function (f32).

Nuttall4a window (float).

Parameter	Value
Peak sidelobe level	82.6 dB
Normalized equivalent noise bandwidth	2.1253 bins
3 dB bandwidth	2.0123 bins
Flatness	-0.7321 dB
Recommended overlap	68.0 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4a_f64 (float64_t *pDst, uint32_t blockSize)
```

Nuttall4a window generating function (f64).

Nuttall4a window (double).

Parameter	Value
Peak sidelobe level	82.6 dB
Normalized equivalent noise bandwidth	2.1253 bins
3 dB bandwidth	2.0123 bins
Flatness	-0.7321 dB
Recommended overlap	68.0 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall4b window function (93.3 dB)

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4b_f32 (float32_t *pDst, uint32_t blockSize)
```

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4b_f64 (float64_t *pDst, uint32_t blockSize)
```

group Nuttall4b window function (93.3 dB)

Functions

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4b_f32 (float32_t *pDst, uint32_t blockSize)
```

Nuttall4b window generating function (f32).

Nuttall4b window (float).

Parameter	Value
Peak sidelobe level	93.3 dB
Normalized equivalent noise bandwidth	2.0212 bins
3 dB bandwidth	1.9122 bins
Flatness	-0.8118 dB
Recommended overlap	66.3 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

```
RISCV_DSP_ATTRIBUTE void riscv_nuttall4b_f64 (float64_t *pDst, uint32_t blockSize)
```

Nuttall4b window generating function (f64).

Nuttall4b window (double).

Parameter	Value
Peak sidelobe level	93.3 dB
Normalized equivalent noise bandwidth	2.0212 bins
3 dB bandwidth	1.9122 bins
Flatness	-0.8118 dB
Recommended overlap	66.3 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Nuttall4c window function (98.1 dB)

RISCV_DSP_ATTRIBUTE void riscv_nuttall4c_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_nuttall4c_f64 (float64_t *pDst, uint32_t blockSize)

group Nuttall4c window function (98.1 dB)

Functions

RISCV_DSP_ATTRIBUTE void riscv_nuttall4c_f32 (float32_t *pDst, uint32_t blockSize)

Nuttall4c window generating function (f32).

Nuttall4c window (float).

Parameter	Value
Peak sidelobe level	98.1 dB
Normalized equivalent noise bandwidth	1.9761 bins
3 dB bandwidth	1.8687 bins
Flatness	-0.8506 dB
Recommended overlap	65.6 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_nuttall4c_f64 (float64_t *pDst, uint32_t blockSize)

Nuttall4c window generating function (f64).

Nuttall4c window (double).

Parameter	Value
Peak sidelobe level	98.1 dB
Normalized equivalent noise bandwidth	1.9761 bins
3 dB bandwidth	1.8687 bins
Flatness	-0.8506 dB
Recommended overlap	65.6 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

Welch window function (21.3 dB)

RISCV_DSP_ATTRIBUTE void riscv_welch_f32 (float32_t *pDst, uint32_t blockSize)

RISCV_DSP_ATTRIBUTE void riscv_welch_f64 (float64_t *pDst, uint32_t blockSize)

group **Welch window function (21.3 dB)**

Functions

RISCV_DSP_ATTRIBUTE void riscv_welch_f32 (float32_t *pDst, uint32_t blockSize)

Welch window generating function (f32).

Welch window (float).

Parameter	Value
Peak sidelobe level	21.3 dB
Normalized equivalent noise bandwidth	1.2 bins
3 dB bandwidth	1.1535 bins
Flatness	-2.2248 dB
Recommended overlap	29.3 %

Parameters of the window

Parameters

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

RISCV_DSP_ATTRIBUTE void riscv_welch_f64 (float64_t *pDst, uint32_t blockSize)

Welch window generating function (f64).

Welch window (double).

Parameter	Value
Peak sidelobe level	21.3 dB
Normalized equivalent noise bandwidth	1.2 bins
3 dB bandwidth	1.1535 bins
Flatness	-2.2248 dB
Recommended overlap	29.3 %

Parameters of the window**Parameters**

- **pDst** – [out] points to the output generated window
- **blockSize** – [in] number of samples in the window

group **Regular window functions**

Regular window functions that can be used for detecting tones or estimating the noise. If you need to estimate the amplitude of a tones, prefer a flat top window.

group **Window Functions****3.3.17 Generic Types****Enumerations for transform buffer size functions**

enum **riscv_math_datatype**

Values:

enumerator **RISCV_MATH_F16**

enumerator **RISCV_MATH_F32**

enumerator **RISCV_MATH_F64**

enumerator **RISCV_MATH_Q7**

enumerator **RISCV_MATH_Q15**

enumerator **RISCV_MATH_Q31**

group **Enumerations for transform buffer size functions**

Enums

enum **riscv_math_datatype**

Datatype identifier.

Values:

enumerator **RISCV_MATH_F16**

f16 datatype identifier

enumerator **RISCV_MATH_F32**

f32 datatype identifier

enumerator **RISCV_MATH_F64**

f64 datatype identifier

enumerator **RISCV_MATH_Q7**

Q7 datatype identifier

enumerator **RISCV_MATH_Q15**

Q15 datatype identifier

enumerator **RISCV_MATH_Q31**

Q31 datatype identifier

enum **riscv_status**

Values:

enumerator **RISCV_MATH_SUCCESS**

enumerator **RISCV_MATH_ARGUMENT_ERROR**

enumerator **RISCV_MATH_LENGTH_ERROR**

enumerator **RISCV_MATH_SIZE_MISMATCH**

enumerator **RISCV_MATH_NANINF**

enumerator **RISCV_MATH_SINGULAR**

enumerator **RISCV_MATH_TEST_FAILURE**

enumerator **RISCV_MATH_DECOMPOSITION_FAILURE**

typedef int8_t **q7_t**

typedef int16_t **q15_t**

typedef int32_t **q31_t**

typedef int64_t **q63_t**

typedef float **float32_t**

typedef double **float64_t**

group **Generic Types**

Typedefs

typedef int8_t **q7_t**

8-bit fractional data type in 1.7 format.

typedef int16_t **q15_t**

16-bit fractional data type in 1.15 format.

typedef int32_t **q31_t**

32-bit fractional data type in 1.31 format.

typedef int64_t **q63_t**

64-bit fractional data type in 1.63 format.

typedef float **float32_t**

32-bit floating-point type definition.

typedef double **float64_t**

64-bit floating-point type definition.

Enums

enum **riscv_status**

vector types

Error status returned by some functions in the library.

Values:

enumerator **RISCV_MATH_SUCCESS**

No error

enumerator **RISCV_MATH_ARGUMENT_ERROR**

One or more arguments are incorrect

enumerator **RISCV_MATH_LENGTH_ERROR**

Length of data buffer is incorrect

enumerator **RISCV_MATH_SIZE_MISMATCH**

Size of matrices is not compatible with the operation

enumerator **RISCV_MATH_NANINF**

Not-a-number (NaN) or infinity is generated

enumerator **RISCV_MATH_SINGULAR**

Input matrix is singular and cannot be inverted

enumerator **RISCV_MATH_TEST_FAILURE**

Test Failed

enumerator **RISCV_MATH_DECOMPOSITION_FAILURE**

Decomposition Failed

3.4 Changelog

Note

NMSIS DSP Library changelog will not maintain here in future, please check [Changelog](#) (page 1435) here.

3.4.1 latest

See [Changelog](#) (page 1435)

3.4.2 V1.3.1

This is release 1.3.1 version of NMSIS-DSP library.

- Update get started guide for build NMSIS-DSP library
- Optimize fredsum, use less fredsum and more fmac for rvv implementation
- Sync cmake file changes

3.4.3 V1.3.0

This is release 1.3.0 version of NMSIS-DSP library.

- Sync with CMSIS-DSP Library v1.16.2
- Add initial version of NMSIS DSP Benchmark
- Fix bug found for ComplexMathFunctions when RISCV_MATH_LOOPUNROLL not defined

3.4.4 V1.2.1

This is release 1.2.1 version of NMSIS-DSP library.

- Sync with CMSIS-DSP library(CMSIS-DSP v1.15.0, commit id:8f8bb8d5)
- Add a few Nuclei DSP N2 intrinsic APIs in `core_feature_dsp.h`, they are: `__RV_DPACK32`, `__RV_DSUNPKD810`, `__RV_DSUNPKD820`, `__RV_DSUNPKD830`, `__RV_DSUNPKD831`, `__RV_DSUNPKD832`, `__RV_DZUNPKD810`, `__RV_DZUNPKD820`, `__RV_DZUNPKD830`, `__RV_DZUNPKD831`, `__RV_DZUNPKD832`. And change `__RV_DPKBB32` to `__RV_DPACK32` for better performance
- Unified instruction naming style in `core_feature_dsp.h`, such as: change `DSMMULU` to `DSMMUL.u`, change `DKWMMULU` to `DKWMMUL.u`, change `DKMMACU` to `DKMMAC.u`, change `DKMMSBU` to `DKMMSB.u`, change `DSMAQASU` to `DSMAQA.SU`, change `DDSMAQASU` to `DDSMAQA.SU`
- Optimize the rvv implementation of some MatMult and Complex MatMult functions
- Fixed some doc erros in `core_feature_dsp.h`
- Fixed `riscv_lms_q31.c` rvv bug

3.4.5 V1.2.0

Note

- This 1.2.0 version will no longer support old gcc 10 verison, and it now only support Nuclei Toolchain 2023.10(gcc13 and clang17) or later. The major changes that can be felt are as follows:
 - The prefix of toolchain has changed from `riscv-nuclei-elf-` to `riscv64-unknown-elf-`
 - The `-march` option has changed a lot, for example:
 - `b` extension changed to `_zba_zbb_zbc_zbs` extension,
 - `p` extension changed to `_xxldsp` or `_xxldspn1x` or `_xxldspn2x` or `_xxldspn3x` extensions which means stardard DSP extension, Nuclei N1, N2, N3 DSP extensions
 - `v` extension changed to `v` or `_zve32f` or `_zve64f` extensions according to the riscv cpu isa used

These extensions need be combined in a certain order to get a correct arch name to match the prebuilt library name, please be cautious

- The name of libraries changed due to `-march` in `gcc13` updated, for example, the library named `libnmsis_dsp_rv32imacb.a` is now named `libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs.a` since `b` extension changed to `_zba_zbb_zbc_zbs`

This is release 1.2.0 version of NMSIS-DSP library.

- Defined `NUCLEI_DSP_DEFAULT`, `NUCLEI_DSP_N1`, `NUCLEI_DSP_N2`, `NUCLEI_DSP_N3` in `riscv_math_types.h` according to gcc options. This means that if compile with `--march=rv32imafc_xxldspn1x`, the `NUCLEI_DSP_N1` will defined, if compile with `--march=rv32imafc_xxldspn2x`, the `NUCLEI_DSP_N1` and `NUCLEI_DSP_N2` will defined, and so on
- Optimize some functions with DSP N1/N2/N3 (such as `FilteringFunctions`, `TransformFunctions`, `ComplexMathFunctions`)
- RVV intrinsic APIs is update to v0.12.0
- Add f16 support(include f16 rvv extension support)
- Fix the use of `expd80` instruction(Nuclei default dsp instruction)
- Fix some testcases bugs(such as `MatrixFunctions`, `TransformFunctions`)

3.4.6 V1.1.1

This is release 1.1.1 version of NMSIS-DSP library.

- Sync changes from CMSIS DSP commit 1d9e38a, version after v1.14.4
- Optimized more for RVP/RVV
- Add support for RV32 Vector
- Some bugfix(`riscv_mat_inverse_f32.c` rvv fix, `riscv_offset_q15.c` p fix, `riscv_fir_q15.c` rvv fix etc.)

3.4.7 V1.1.0

This is release 1.1.0 version of NMSIS-DSP library.

- Sync changes from CMSIS 5.9.0 release
- Optimized more for RVP/RVV
- Add experimental support for RV32 Vector

3.4.8 V1.0.3

This is release 1.0.3 version of NMSIS-DSP library.

- Update build system for NMSIS-DSP library
- Rename `RISCV_VECTOR` to `RISCV_MATH_VECTOR` in header file and source code
- Using new python script to generate NMSIS-DSP library
- Fix `riscv_float_to_q31` function for `rv64imafcv` target
- Change `vfredsum` to `vfredusum` when using vector intrinsic function due to vector spec 1.0

- Support Nuclei RISC-V GCC 10.2

3.4.9 V1.0.2

This is release 1.0.2 version of NMSIS-DSP library.

- Sync up to CMSIS DSP library 1.9.0
- Adding initial support for RISC-V vector extension support
- **Caution:** `riscv_math.h` is separated into several header files. Extra `PrivateInclude` folder is included as header folder.

3.4.10 V1.0.1

This is release V1.0.1 version of NMSIS-DSP library.

- Both Nuclei RISC-V 32 and 64 bit cores are supported now.
- Libraries are optimized for RISC-V 32 and 64 bit DSP instructions.
- The NN examples are now using Nuclei SDK as running environment.

3.4.11 V1.0.0

This is the first version of NMSIS-DSP library.

We adapt the CMSIS-DSP v1.6.0 library to use RISC-V DSP instructions, all the API names now are renamed from `arm_` to `riscv_`.

4.1 Overview

4.1.1 Introduction

This user manual describes the NMSIS NN software library, a collection of efficient neural network kernels developed to maximize the performance and minimize the memory footprint of neural networks on Nuclei N/NX/UX Class Processors cores.

The library is divided into a number of functions each covering a specific category:

- Activation Functions
- BasicMath Functions
- Concatenation Functions
- Convolution Functions
- Fully-Connected Functions
- LSTM Functions
- NNSupport Functions
- Pooling Functions
- Reshape Functions
- Softmax Functions
- SVD Functions

The library has separate functions for operating on different weight and activation data types including 8-bit integers (q7_t) and 16-bit integers (q15_t). The description of the kernels are included in the function description.

The implementation details are also described in this paper [CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs](https://arxiv.org/abs/1801.06601)³⁷.

³⁷ <https://arxiv.org/abs/1801.06601>

4.1.2 Block Diagram

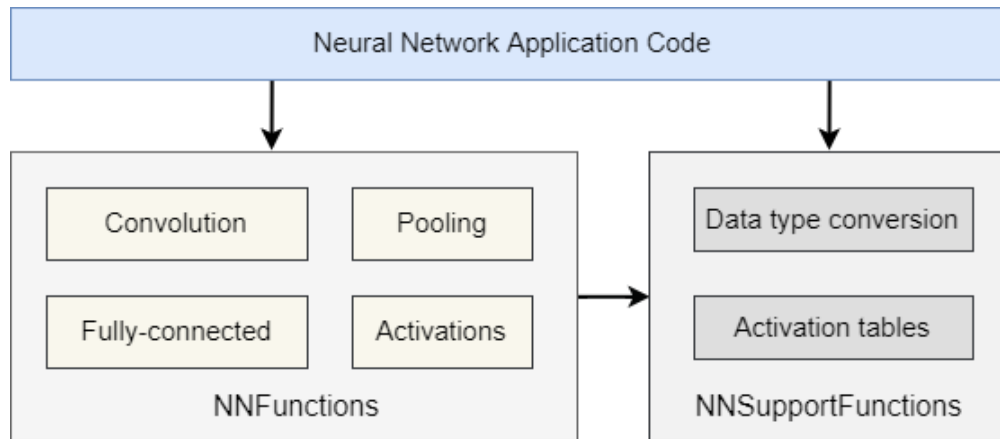


Fig. 1: NMSIS NN Block Diagram

4.1.3 Examples

The library ships with a number of examples which demonstrate how to use the library functions.

- *Convolutional Neural Network Example* (page 1289)
- *Gated Recurrent Unit Example* (page 1290)

4.1.4 Pre-processor Macros

Each library project have different pre-processor macros controlled via CMakeLists.txt.

This library is only built for little endian targets.

4.2 Using NMSIS-NN

Here we will describe how to run the nmsis nn examples in Nuclei QEMU.

4.2.1 Preparation

- Nuclei SDK, master branch(>= 0.9.0 release)
- Nuclei RISC-V GNU Toolchain 2025.10
- Nuclei QEMU 2025.10
- CMake >= 3.14
- Python 3 and pip package requirements located in
 - <nuclei-sdk>/tools/scripts/requirements.txt
 - <NMSIS>/NMSIS/Scripts/requirements.txt

4.2.2 Tool Setup

1. Export **PATH** correctly for qemu and riscv64-unknown-elf-gcc

```
export PATH=/path/to/qemu/bin:/path/to/gcc/bin:$PATH
```

4.2.3 Build NMSIS NN Library

1. Download or clone NMSIS source code into **NMSIS** directory.
2. cd to *NMSIS/NMSIS/* directory, if you want to add or remove more arches to be build, you can modify *Scripts/Build/nmsis_nn.json* file
3. Build NMSIS NN library and strip debug information using `make gen_nn_lib`
4. The nn library will be generated into *./Library/NN/GCC* folder
5. The nn libraries will be look like this:

```
$ ls -lhG Library/NN/GCC/
total 64M
-rw-rw-r-- 1 619K Oct 20 11:55 libnmsis_nn_rv32imac.a
-rw-rw-r-- 1 983K Oct 20 11:55 libnmsis_nn_rv32imac_xxldsp.a
-rw-rw-r-- 1 986K Oct 20 11:55 libnmsis_nn_rv32imac_xxldspn1x.a
-rw-rw-r-- 1 989K Oct 20 11:55 libnmsis_nn_rv32imac_xxldspn2x.a
-rw-rw-r-- 1 992K Oct 20 11:55 libnmsis_nn_rv32imac_xxldspn3x.a
-rw-rw-r-- 1 607K Oct 20 11:55 libnmsis_nn_rv32imac_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 961K Oct 20 11:55 libnmsis_nn_rv32imac_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 964K Oct 20 11:55 libnmsis_nn_rv32imac_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 966K Oct 20 11:55 libnmsis_nn_rv32imac_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 969K Oct 20 11:55 libnmsis_nn_rv32imac_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 620K Oct 20 11:55 libnmsis_nn_rv32imafc.a
-rw-rw-r-- 1 984K Oct 20 11:55 libnmsis_nn_rv32imafc_xxldsp.a
-rw-rw-r-- 1 987K Oct 20 11:55 libnmsis_nn_rv32imafc_xxldspn1x.a
-rw-rw-r-- 1 990K Oct 20 11:55 libnmsis_nn_rv32imafc_xxldspn2x.a
-rw-rw-r-- 1 993K Oct 20 11:55 libnmsis_nn_rv32imafc_xxldspn3x.a
-rw-rw-r-- 1 608K Oct 20 11:55 libnmsis_nn_rv32imafc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 962K Oct 20 11:55 libnmsis_nn_rv32imafc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 965K Oct 20 11:55 libnmsis_nn_rv32imafc_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 968K Oct 20 11:55 libnmsis_nn_rv32imafc_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 970K Oct 20 11:55 libnmsis_nn_rv32imafc_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 716K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f.a
-rw-rw-r-- 1 848K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_xxldsp.a
-rw-rw-r-- 1 851K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_xxldspn1x.a
-rw-rw-r-- 1 854K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_xxldspn2x.a
-rw-rw-r-- 1 856K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_xxldspn3x.a
-rw-rw-r-- 1 695K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 825K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 828K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 831K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 834K Oct 20 11:55 libnmsis_nn_rv32imafc_zve32f_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 621K Oct 20 11:55 libnmsis_nn_rv32imafdc.a
-rw-rw-r-- 1 985K Oct 20 11:55 libnmsis_nn_rv32imafdc_xxldsp.a
-rw-rw-r-- 1 988K Oct 20 11:55 libnmsis_nn_rv32imafdc_xxldspn1x.a
```

(continues on next page)

(continued from previous page)

```

-rw-rw-r-- 1 991K Oct 20 11:55 libnmsis_nn_rv32imafdc_xxldspn2x.a
-rw-rw-r-- 1 994K Oct 20 11:55 libnmsis_nn_rv32imafdc_xxldspn3x.a
-rw-rw-r-- 1 609K Oct 20 11:55 libnmsis_nn_rv32imafdc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 963K Oct 20 11:55 libnmsis_nn_rv32imafdc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 966K Oct 20 11:55 libnmsis_nn_rv32imafdc_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 969K Oct 20 11:55 libnmsis_nn_rv32imafdc_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 972K Oct 20 11:55 libnmsis_nn_rv32imafdc_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 718K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f.a
-rw-rw-r-- 1 847K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_xxldsp.a
-rw-rw-r-- 1 850K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_xxldspn1x.a
-rw-rw-r-- 1 852K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_xxldspn2x.a
-rw-rw-r-- 1 855K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_xxldspn3x.a
-rw-rw-r-- 1 697K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 824K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 827K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn1x.a
-rw-rw-r-- 1 830K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn2x.a
-rw-rw-r-- 1 832K Oct 20 11:55 libnmsis_nn_rv32imafdc_zve32f_zba_zbb_zbc_zbs_xxldspn3x.a
-rw-rw-r-- 1 852K Oct 20 11:55 libnmsis_nn_rv64imac.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imac_xxldsp.a
-rw-rw-r-- 1 826K Oct 20 11:55 libnmsis_nn_rv64imac_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imac_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 854K Oct 20 11:55 libnmsis_nn_rv64imafc.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imafc_xxldsp.a
-rw-rw-r-- 1 827K Oct 20 11:55 libnmsis_nn_rv64imafc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imafc_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 965K Oct 20 11:55 libnmsis_nn_rv64imafc_zve64f.a
-rw-rw-r-- 1 1.2M Oct 20 11:55 libnmsis_nn_rv64imafc_zve64f_xxldsp.a
-rw-rw-r-- 1 932K Oct 20 11:55 libnmsis_nn_rv64imafc_zve64f_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 1.2M Oct 20 11:55 libnmsis_nn_rv64imafc_zve64f_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 855K Oct 20 11:55 libnmsis_nn_rv64imafdc.a
-rw-rw-r-- 1 972K Oct 20 11:55 libnmsis_nn_rv64imafdcv.a
-rw-rw-r-- 1 1.2M Oct 20 11:55 libnmsis_nn_rv64imafdcv_xxldsp.a
-rw-rw-r-- 1 939K Oct 20 11:55 libnmsis_nn_rv64imafdcv_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 1.2M Oct 20 11:55 libnmsis_nn_rv64imafdcv_zba_zbb_zbc_zbs_xxldsp.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imafdc_xxldsp.a
-rw-rw-r-- 1 828K Oct 20 11:55 libnmsis_nn_rv64imafdc_zba_zbb_zbc_zbs.a
-rw-rw-r-- 1 1.4M Oct 20 11:55 libnmsis_nn_rv64imafdc_zba_zbb_zbc_zbs_xxldsp.a

```

7. library name with extra `_xxldsp` `_xxldspn1x` `_xxldspn2x` `_xxldspn3x` is built with RISC-V DSP enabled

The examples are as follows:

- `libnmsis_nn_rv32imac.a`: Build for **RISCV_ARCH=rv32imac** without DSP
- `libnmsis_nn_rv32imac_xxldsp.a`: Build for **RISCV_ARCH=rv32imac_xxldsp** with Nuclei DSP enabled
- `libnmsis_nn_rv32imac_xxldspn1x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn1x** with Nuclei N1 DSP extension enabled
- `libnmsis_nn_rv32imac_xxldspn2x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn2x** with Nuclei N1/N2 DSP extension enabled
- `libnmsis_nn_rv32imac_xxldspn3x.a`: Build for **RISCV_ARCH=rv32imac_xxldspn3x** with Nuclei N1/N2/N3 DSP extension enabled

8. library name with extra `_zve32f _zve64f v` is built with RISC-V Vector enabled

The examples are as follows:

- `libnmsis_nn_rv32imafc_zve32f.a`: Build for **RISCV_ARCH=rv32imafc_zve32f** with Vector enabled
- `libnmsis_nn_rv32imafdc_zve32f.a`: Build for **RISCV_ARCH=rv32imafdc_zve32f** with Vector enabled
- `libnmsis_nn_rv64imafc_zve64f.a`: Build for **RISCV_ARCH=rv64imafc_zve64f** with Vector enabled
- `libnmsis_nn_rv64imafdcv.a`: Build for **RISCV_ARCH=rv64imafdcv** with Vector enabled

Note

- This NMSIS 1.2.0 is a big change version, will no longer support old gcc 10 version, and it now only support Nuclei Toolchain 2023.10. The `--march` option has changed a lot, such as:
 - `b` extension changed to `_zba_zbb_zbc_zbs` extension,
 - `p` extension changed to `_xxldsp`, `_xxldspn1x`, `_xxldspn2x`, `_xxldspn3x` extensions which means standard DSP extension, Nuclei N1, N2, N3 DSP extensions
 - `v` extension changed to `v`, `_zve32f`, `_zve64f` extensions
- The name of libraries has changed with `-march`, for examples, the library named `libnmsis_nn_rv32imacb.a` is now named `libnmsis_nn_rv32imac_zba_zbb_zbc_zbs.a` since `b` extension changed to `_zba_zbb_zbc_zbs`
- `_xxldspn1x` `_xxldspn2x` `_xxldspn3x` only valid for RISC-V 32bit processor. `_xxldsp` is valid for RISC-V 32/64 bit processor
- You can also directly build both DSP and NN library using `make gen`
- You can strip the generated DSP and NN library using `make strip`
- DSP and Vector extension can be combined, such as `_xxldsp, v` and `v_xxldsp`, should notice the extension order
- Vector extension currently enabled for RISC-V 32/64 bit processor
- NN library has no float16 data type, so here have no need to build float16 library

4.2.4 How to run

1. Set environment variables `NUCLEI_SDK_ROOT` and `NUCLEI_SDK_NMSIS`, and set Nuclei SDK SoC to `evalsoc`, and change `ilm/dlm` size from 64K to 512K.

```
export NUCLEI_SDK_ROOT=/path/to/nuclei_sdk
export NUCLEI_SDK_NMSIS=/path/to/NMSIS/NMSIS
# Setup SDK development environment
cd $NUCLEI_SDK_ROOT
source setup.sh
cd -
# !!!!Take Care!!!!
# change this link script will make compiled example can only run on bitstream which has
↪ 512K ILM/DLM
```

(continues on next page)

(continued from previous page)

```
# For Nuclei SDK < 0.7.0
sed -i "s/64K/512K/g" $NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_
↪evalsoc_ilm.ld
# For Nuclei SDK >= 0.7.0
sed -i 's/\([ID]LM_MEMORY_SIZE\).*\/\1 = 0x80000;/ ' $NUCLEI_SDK_ROOT/SoC/evalsoc/Board/
↪nuclei_fpga_eval/Source/GCC/evalsoc.memory
export SOC=evalsoc
```

2. Due to many of the examples could not be placed in 64K ILM and 64K DLM, and we are running using qemu, the ILM/DLM size in it are set to be 32MB, so we can change ilm/dlm to 512K/512K in the link script \$NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld or \$NUCLEI_SDK_ROOT/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/evalsoc.memory

```
--- a/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld
+++ b/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld
@@ -30,8 +30,8 @@ __HEAP_SIZE = 2K;

MEMORY
{
-   ilm (rxa!w) : ORIGIN = 0x80000000, LENGTH = 64K
-   ram (wxa!r) : ORIGIN = 0x90000000, LENGTH = 64K
+   ilm (rxa!w) : ORIGIN = 0x80000000, LENGTH = 512K
+   ram (wxa!r) : ORIGIN = 0x90000000, LENGTH = 512K
}
```

3. Let us take cifar10 for example:

```
cd $NUCLEI_SDK_NMSIS/NN/Examples/RISCV/cifar10/
```

4. Run with RISCV DSP enabled and Vector enabled NMSIS-NN library for CORE nx900fd

```
# Clean project
make ARCH_EXT=v_xldsp CORE=nx900fd clean
# Build project, enable ``v`` and ``xldsp`` optimize
make ARCH_EXT=v_xldsp CORE=nx900fd all
# Run application using qemu
make ARCH_EXT=v_xldsp CORE=nx900fd run_qemu
```

5. Run with RISCV DSP disabled and Vector disabled NMSIS-NN library for CORE nx900fd

```
make ARCH_EXT= CORE=nx900fd clean
make ARCH_EXT= CORE=nx900fd all
make ARCH_EXT= CORE=nx900fd run_qemu
```

Note

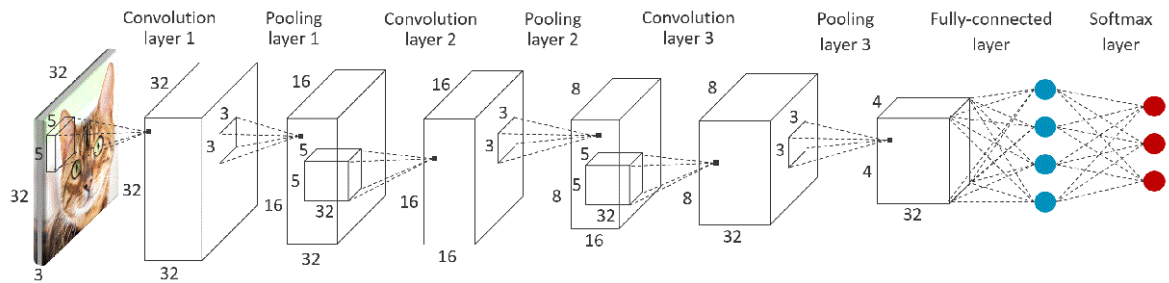
- You can easily run this example in your hardware, if you have enough memory to run it, just modify the SOC to the one you are using in step 1.

4.3 NMSIS NN API

If you want to access doxygen generated NMSIS NN API, please click [NMSIS NN API Doxygen Documentation](#).

4.3.1 Convolutional Neural Network Example

group Convolutional Neural Network Example



Description:

Demonstrates a convolutional neural network (CNN) example with the use of convolution, ReLU activation, pooling and fully-connected functions.

Model definition:

The CNN used in this example is based on CIFAR-10 example from Caffe [1]. The neural network consists of 3 convolution layers interspersed by ReLU activation and max pooling layers, followed by a fully-connected layer at the end. The input to the network is a 32x32 pixel color image, which will be classified into one of the 10 output classes. This example model implementation needs 32.3 KB to store weights, 40 KB for activations and 3.1 KB for storing the `im2col` data.

Refer `riscv_nnexamples_cifar10.c`

Variables Description:

- `conv1_wt`, `conv2_wt`, `conv3_wt` are convolution layer weight matrices
- `conv1_bias`, `conv2_bias`, `conv3_bias` are convolution layer bias arrays
- `ip1_wt`, `ip1_bias` point to fully-connected layer weights and biases
- `input_data` points to the input image data
- `output_data` points to the classification output
- `col_buffer` is a buffer to store the `im2col` output
- `scratch_buffer` is used to store the activation data (intermediate layer outputs)

NMSIS NN Software Library Functions Used:

- `riscv_convolve_HWC_q7_RGB()`
- `riscv_convolve_HWC_q7_fast()`

- `riscv_relu_q7()`
- `riscv_maxpool_q7_HWC()`
- `riscv_avepool_q7_HWC()`
- `riscv_fully_connected_q7_opt()`
- `riscv_fully_connected_q7()`

[1] <https://github.com/BVLC/caffe>

4.3.2 Gated Recurrent Unit Example

group **Gated Recurrent Unit Example**

Refer `riscv_nnexamples_gru.c`

Description:

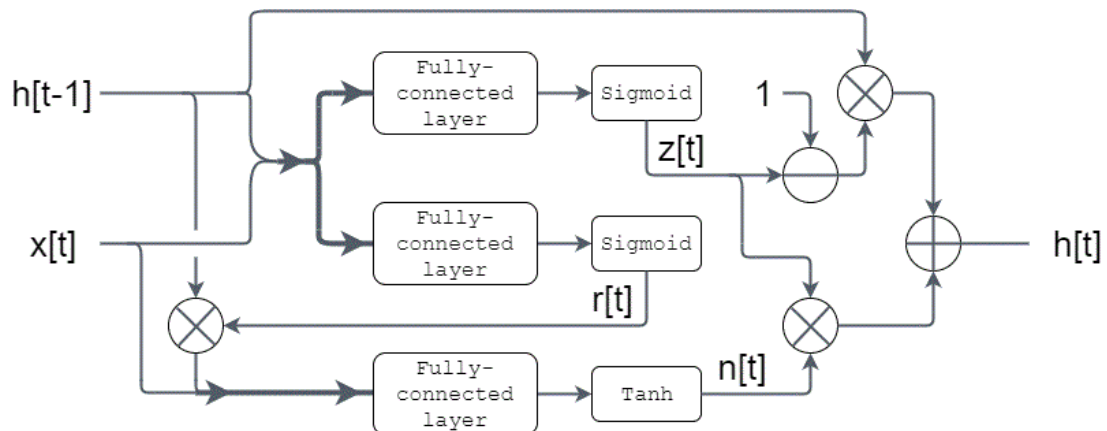
Demonstrates a gated recurrent unit (GRU) example with the use of fully-connected, Tanh/Sigmoid activation functions.

Model definition:

GRU is a type of recurrent neural network (RNN). It contains two sigmoid gates and one hidden state.

The computation can be summarized as:

```
z[t] = sigmoid( W_z &sdot; {h[t-1], x[t]} )
r[t] = sigmoid( W_r &sdot; {h[t-1], x[t]} )
n[t] = tanh( W_n &sdot; [r[t] &times; {h[t-1], x[t]}] )
h[t] = (1 - z[t]) &times; h[t-1] + z[t] &times; n[t]
```



Variables Description:

- `update_gate_weights`, `reset_gate_weights`, `hidden_state_weights` are weights corresponding to update gate (W_z), reset gate (W_r), and hidden state (W_n).

- `update_gate_bias`, `reset_gate_bias`, `hidden_state_bias` are layer bias arrays
- `test_input1`, `test_input2`, `test_history` are the inputs and initial history

The buffer is allocated as:

| reset | input | history | update | hidden_state |

In this way, the concatenation is automatically done since (reset, input) and (input, history) are physically concatenated in memory.

The ordering of the weight matrix should be adjusted accordingly.

NMSIS NN Software Library Functions Used:

- `riscv_fully_connected_mat_q7_vec_q15_opt()`
- `riscv_nn_activations_direct_q15()`
- `riscv_mult_q15()`
- `riscv_offset_q15()`
- `riscv_sub_q15()`
- `riscv_copy_q15()`

4.3.3 Public

Structure Types

enum **riscv_nn_activation_type**

Values:

enumerator **RISCV_SIGMOID**

enumerator **RISCV_TANH**

enum **riscv_nmsis_nn_status**

Values:

enumerator **RISCV_NMSIS_NN_SUCCESS**

enumerator **RISCV_NMSIS_NN_ARG_ERROR**

enumerator **RISCV_NMSIS_NN_NO_IMPL_ERROR**

enumerator **RISCV_NMSIS_NN_FAILURE**

struct **nmsis_nn_tile**

struct **nmsis_nn_context**

struct **nmsis_nn_bias_data**

struct **nmsis_nn_dims**

struct **nmsis_nn_lstm_dims**

struct **nmsis_nn_per_channel_quant_params**

struct **nmsis_nn_per_tensor_quant_params**

struct **nmsis_nn_quant_params**

struct **nmsis_nn_activation**

struct **nmsis_nn_conv_params**

struct **nmsis_nn_transpose_conv_params**

struct **nmsis_nn_dw_conv_params**

struct **nmsis_nn_pool_params**

struct **nmsis_nn_fc_params**

struct **nmsis_nn_bmm_params**

struct **nmsis_nn_transpose_params**

struct **nmsis_nn_svd_f_params**

struct **nmsis_nn_softmax_lut_s16**

struct **nmsis_nn_scaling**

struct **nmsis_nn_lstm_gate**

struct **nmsis_nn_lstm_params**

struct **nmsis_nn_lstm_context**

group **Structure Types**

Enums and Data Structures used in public API.

Enums

enum **riscv_nn_activation_type**

Enum for specifying activation function types

Values:

enumerator **RISCV_SIGMOID**

Sigmoid activation function

enumerator **RISCV_TANH**

Tanh activation function

enum **riscv_nmsis_nn_status**

Function return codes

Values:

enumerator **RISCV_NMSIS_NN_SUCCESS**

No error

enumerator **RISCV_NMSIS_NN_ARG_ERROR**

One or more arguments are incorrect

enumerator **RISCV_NMSIS_NN_NO_IMPL_ERROR**

No implementation available

enumerator **RISCV_NMSIS_NN_FAILURE**

Logical error

struct **nmsis_nn_tile**

#include <riscv_nn_types.h> NMSIS-NN object to contain the width and height of a tile

struct **nmsis_nn_context**

#include <riscv_nn_types.h> NMSIS-NN object used for the function context.

struct **nmsis_nn_bias_data**

#include <riscv_nn_types.h> NMSIS-NN object used to hold bias data for int16 variants.

struct **nmsis_nn_dims**

#include <riscv_nn_types.h> NMSIS-NN object to contain the dimensions of the tensors

```
struct nmsis_nn_lstm_dims
    #include <riscv_nn_types.h> NMSIS-NN object to contain LSTM specific input parameters related to
    dimensions

struct nmsis_nn_per_channel_quant_params
    #include <riscv_nn_types.h> NMSIS-NN object for the per-channel quantization parameters

struct nmsis_nn_per_tensor_quant_params
    #include <riscv_nn_types.h> NMSIS-NN object for the per-tensor quantization parameters

struct nmsis_nn_quant_params
    #include <riscv_nn_types.h> NMSIS-NN object for quantization parameters. This struct supports both
    per-tensor and per-channels requantization and is recommended for new operators.

struct nmsis_nn_activation
    #include <riscv_nn_types.h> NMSIS-NN object for the quantized Relu activation

struct nmsis_nn_conv_params
    #include <riscv_nn_types.h> NMSIS-NN object for the convolution layer parameters

struct nmsis_nn_transpose_conv_params
    #include <riscv_nn_types.h> NMSIS-NN object for the transpose convolution layer parameters

struct nmsis_nn_dw_conv_params
    #include <riscv_nn_types.h> NMSIS-NN object for the depthwise convolution layer parameters

struct nmsis_nn_pool_params
    #include <riscv_nn_types.h> NMSIS-NN object for pooling layer parameters

struct nmsis_nn_fc_params
    #include <riscv_nn_types.h> NMSIS-NN object for Fully Connected layer parameters

struct nmsis_nn_bmm_params
    #include <riscv_nn_types.h> NMSIS-NN object for Batch Matmul layer parameters

struct nmsis_nn_transpose_params
    #include <riscv_nn_types.h> NMSIS-NN object for Transpose layer parameters

struct nmsis_nn_svd_f_params
    #include <riscv_nn_types.h> NMSIS-NN object for SVDF layer parameters

struct nmsis_nn_softmax_lut_s16
    #include <riscv_nn_types.h> NMSIS-NN object for Softmax s16 layer parameters

struct nmsis_nn_scaling
    #include <riscv_nn_types.h> NMSIS-NN object for quantization parameters
```

```

struct nmsis_nn_lstm_gate
    #include <riscv_nn_types.h> NMSIS-NN object for LSTM gate parameters

struct nmsis_nn_lstm_params
    #include <riscv_nn_types.h> NMSIS-NN object for LSTM parameters

struct nmsis_nn_lstm_context
    #include <riscv_nn_types.h> NMSIS-NN object for LSTM scratch buffers

```

Activation Functions

riscv_nmsis_nn_status (page 1291) **riscv_nn_activation_s16**(const int16_t *input, int16_t *output, const int32_t size, const int32_t left_shift, const *riscv_nn_activation_type* (page 1291) type)

void **riscv_nn_activations_direct_q15**(*q15_t* (page 1277) *data, uint16_t size, uint16_t int_width, *riscv_nn_activation_type* (page 1291) type)

void **riscv_nn_activations_direct_q7**(*q7_t* (page 1277) *data, uint16_t size, uint16_t int_width, *riscv_nn_activation_type* (page 1291) type)

void **riscv_relu6_s8**(int8_t *data, uint16_t size)

void **riscv_relu_q15**(int16_t *data, uint16_t size)

void **riscv_relu_q7**(int8_t *data, uint16_t size)

group Activation Functions

Perform activation layers, including ReLU (Rectified Linear Unit), sigmoid and tanh

Functions

riscv_nmsis_nn_status (page 1291) **riscv_nn_activation_s16**(const int16_t *input, int16_t *output, const int32_t size, const int32_t left_shift, const *riscv_nn_activation_type* (page 1291) type)

s16 neural network activation function using direct table look-up

Supported framework: TensorFlow Lite for Microcontrollers. This activation function must be bit precise congruent with the corresponding TFLM tanh and sigmoid activation functions

Parameters

- **input** – [in] pointer to input data
- **output** – [out] pointer to output
- **size** – [in] number of elements
- **left_shift** – [in] bit-width of the integer part, assumed to be smaller than 3.
- **type** – [in] type of activation functions

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

void **riscv_nn_activations_direct_q15**(*q15_t* (page 1277) *data, uint16_t size, uint16_t int_width, *riscv_nn_activation_type* (page 1291) type)

neural network activation function using direct table look-up

Q15 neural network activation function using direct table look-up.

Note

Refer header file for details.

void **riscv_nn_activations_direct_q7**(*q7_t* (page 1277) *data, uint16_t size, uint16_t int_width, *riscv_nn_activation_type* (page 1291) type)

Q7 neural network activation function using direct table look-up.

This is the direct table look-up approach.

Assume here the integer part of the fixed-point is ≤ 3 . More than 3 just not making much sense, makes no difference with saturation followed by any of these activation functions.

Parameters

- **data** – [inout] pointer to input
- **size** – [in] number of elements
- **int_width** – [in] bit-width of the integer part, assume to be smaller than 3
- **type** – [in] type of activation functions

void **riscv_relu6_s8**(int8_t *data, uint16_t size)

s8 ReLU6 function

Parameters

- **data** – [inout] pointer to input
- **size** – [in] number of elements

void **riscv_relu_q15**(int16_t *data, uint16_t size)

Q15 RELU function.

Parameters

- **data** – [inout] pointer to input
- **size** – [in] number of elements

void **riscv_relu_q7**(int8_t *data, uint16_t size)

Q7 RELU function.

Parameters

- **data** – [inout] pointer to input
- **size** – [in] number of elements

Elementwise Functions

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_add_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_1_mult, const int32_t input_1_shift, const int32_t input_2_offset, const int32_t input_2_mult, const int32_t input_2_shift, const int32_t left_shift, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_add_s8**(const int8_t *input_1_vect, const int8_t *input_2_vect, const int32_t input_1_offset, const int32_t input_1_mult, const int32_t input_1_shift, const int32_t input_2_offset, const int32_t input_2_mult, const int32_t input_2_shift, const int32_t left_shift, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_acc_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16_batch_offset**(const int16_t *input_1_vect, const int16_t *input_2_vect, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t block_size, const int32_t batch_size, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s8**(const int8_t *input_1_vect, const int8_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

group **Elementwise Functions**

Elementwise add and multiplication functions.

Functions

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_add_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_1_mult, const int32_t input_1_shift, const int32_t input_2_offset, const int32_t input_2_mult, const int32_t input_2_shift, const int32_t left_shift, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

s16 elementwise add of two vectors

Parameters

- **input_1_vect** – [in] pointer to input vector 1
- **input_2_vect** – [in] pointer to input vector 2
- **input_1_offset** – [in] offset for input 1. Not used.
- **input_1_mult** – [in] multiplier for input 1
- **input_1_shift** – [in] shift for input 1
- **input_2_offset** – [in] offset for input 2. Not used.
- **input_2_mult** – [in] multiplier for input 2
- **input_2_shift** – [in] shift for input 2
- **left_shift** – [in] input left shift
- **output** – [inout] pointer to output vector
- **out_offset** – [in] output offset. Not used.
- **out_mult** – [in] output multiplier
- **out_shift** – [in] output shift
- **out_activation_min** – [in] minimum value to clamp output to. Min: -32768
- **out_activation_max** – [in] maximum value to clamp output to. Max: 32767

- **block_size** – [in] number of samples

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_add_s8**(const int8_t *input_1_vect, const int8_t *input_2_vect, const int32_t input_1_offset, const int32_t input_1_mult, const int32_t input_1_shift, const int32_t input_2_offset, const int32_t input_2_mult, const int32_t input_2_shift, const int32_t left_shift, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

s8 elementwise add of two vectors

Parameters

- **input_1_vect** – [in] pointer to input vector 1
- **input_2_vect** – [in] pointer to input vector 2
- **input_1_offset** – [in] offset for input 1. Range: -127 to 128
- **input_1_mult** – [in] multiplier for input 1
- **input_1_shift** – [in] shift for input 1
- **input_2_offset** – [in] offset for input 2. Range: -127 to 128
- **input_2_mult** – [in] multiplier for input 2
- **input_2_shift** – [in] shift for input 2
- **left_shift** – [in] input left shift
- **output** – [inout] pointer to output vector
- **out_offset** – [in] output offset. Range: -128 to 127
- **out_mult** – [in] output multiplier
- **out_shift** – [in] output shift
- **out_activation_min** – [in] minimum value to clamp output to. Min: -128
- **out_activation_max** – [in] maximum value to clamp output to. Max: 127
- **block_size** – [in] number of samples

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_acc_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

s16 element wise accumulative multiplication of two vectors

s16 elementwise multiplication. The result of the multiplication is accumulated to the passed result buffer.

Note

Refer header file for details.

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16**(const int16_t *input_1_vect, const int16_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

s16 element wise multiplication of two vectors

s16 elementwise multiplication

Note

Refer header file for details.

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16_batch_offset**(const int16_t *input_1_vect, const int16_t *input_2_vect, int16_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t block_size, const int32_t batch_size, const int32_t batch_offset)

s16 element wise multiplication of batches of two vectors

s16 elementwise multiplication with s16 output

Note

Refer header file for details.

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s8**(const int8_t *input_1_vect, const int8_t *input_2_vect, const int32_t input_1_offset, const int32_t input_2_offset, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t out_activation_min, const int32_t out_activation_max, const int32_t block_size)

s8 elementwise multiplication

Supported framework: TensorFlow Lite micro

Parameters

- **input_1_vect** – [in] pointer to input vector 1
- **input_2_vect** – [in] pointer to input vector 2
- **input_1_offset** – [in] offset for input 1. Range: -127 to 128
- **input_2_offset** – [in] offset for input 2. Range: -127 to 128
- **output** – [inout] pointer to output vector
- **out_offset** – [in] output offset. Range: -128 to 127
- **out_mult** – [in] output multiplier
- **out_shift** – [in] output shift
- **out_activation_min** – [in] minimum value to clamp output to. Min: -128
- **out_activation_max** – [in] maximum value to clamp output to. Max: 127
- **block_size** – [in] number of samples

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

MinimumMaximum

Warning

doxygengroup: Cannot find group “MinimumMaximum” in doxygen xml output for project “nmsis_nn” from directory: ../build/html/doxygen/nn/xml

Warning

doxygengroup: Cannot find group “MinimumMaximum” in doxygen xml output for project “nmsis_nn” from directory: ../build/html/doxygen/nn/xml

Concatenation Functions

```
void riscv_concatenation_s8_w(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
    uint16_t input_z, const uint16_t input_w, int8_t *output, const uint32_t
    offset_w)
```

```
void riscv_concatenation_s8_x(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
    uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
    output_x, const uint32_t offset_x)
```

```
void riscv_concatenation_s8_y(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
    uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
    output_y, const uint32_t offset_y)
```

```
void riscv_concatenation_s8_z(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
    uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
    output_z, const uint32_t offset_z)
```

group Concatenation Functions

Functions

```
void riscv_concatenation_s8_w(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
    uint16_t input_z, const uint16_t input_w, int8_t *output, const uint32_t
    offset_w)
```

int8/uint8 concatenation function to be used for concatenating N-tensors along the W axis (Batch size) This function should be called for each input tensor to concatenate. The argument offset_w will be used to store the input tensor in the correct position in the output tensor

i.e. offset_w = 0 for(i = 0 i < num_input_tensors; ++i) { riscv_concatenation_s8_w(&input[i], ..., &output, ..., ..., offset_w) offset_w += input_w[i] }

This function assumes that the output tensor has:

- The same width of the input tensor
- The same height of the input tensor
- The same number o channels of the input tensor

Unless specified otherwise, arguments are mandatory.

Note

This function, data layout independent, can be used to concatenate either int8 or uint8 tensors because it does not involve any arithmetic operation

Parameters

- **input** – [in] Pointer to input tensor
- **input_x** – [in] Width of input tensor
- **input_y** – [in] Height of input tensor
- **input_z** – [in] Channels in input tensor

- **input_w** – [in] Batch size in input tensor
- **output** – [out] Pointer to output tensor. Expected to be at least $\text{input_x} * \text{input_y} * \text{input_z} * \text{input_w}$ bytes.
- **offset_w** – [in] The offset on the W axis to start concatenating the input tensor It is user responsibility to provide the correct value

```
void riscv_concatenation_s8_x(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
                             uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
                             output_x, const uint32_t offset_x)
```

int8/uint8 concatenation function to be used for concatenating N-tensors along the X axis This function should be called for each input tensor to concatenate. The argument offset_x will be used to store the input tensor in the correct position in the output tensor

i.e. `offset_x = 0` for `(i = 0 i < num_input_tensors; ++i) { riscv_concatenation_s8_x(&input[i], ..., &output, ..., ..., offset_x) offset_x += input_x[i] }`

This function assumes that the output tensor has:

- The same height of the input tensor
- The same number of channels of the input tensor
- The same batch size of the input tensor

Unless specified otherwise, arguments are mandatory.

Input constraints offset_x is less than output_x

Note

This function, data layout independent, can be used to concatenate either int8 or uint8 tensors because it does not involve any arithmetic operation

Parameters

- **input** – [in] Pointer to input tensor. Input tensor must not overlap with the output tensor.
- **input_x** – [in] Width of input tensor
- **input_y** – [in] Height of input tensor
- **input_z** – [in] Channels in input tensor
- **input_w** – [in] Batch size in input tensor
- **output** – [out] Pointer to output tensor. Expected to be at least $(\text{input_x} * \text{input_y} * \text{input_z} * \text{input_w}) + \text{offset_x}$ bytes.
- **output_x** – [in] Width of output tensor
- **offset_x** – [in] The offset (in number of elements) on the X axis to start concatenating the input tensor It is user responsibility to provide the correct value

```
void riscv_concatenation_s8_y(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
                             uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
                             output_y, const uint32_t offset_y)
```

int8/uint8 concatenation function to be used for concatenating N-tensors along the Y axis This function should be called for each input tensor to concatenate. The argument offset_y will be used to store the input tensor in the correct position in the output tensor

i.e. `offset_y = 0` for(`i = 0` `i < num_input_tensors`; `++i`) { `riscv_concatenation_s8_y(&input[i], ..., &output, ..., ..., offset_y)` `offset_y += input_y[i]` }

This function assumes that the output tensor has:

- The same width of the input tensor
- The same number of channels of the input tensor
- The same batch size of the input tensor

Unless specified otherwise, arguments are mandatory.

Input constraints `offset_y` is less than `output_y`

Note

This function, data layout independent, can be used to concatenate either `int8` or `uint8` tensors because it does not involve any arithmetic operation

Parameters

- **input** – [in] Pointer to input tensor. Input tensor must not overlap with the output tensor.
- **input_x** – [in] Width of input tensor
- **input_y** – [in] Height of input tensor
- **input_z** – [in] Channels in input tensor
- **input_w** – [in] Batch size in input tensor
- **output** – [out] Pointer to output tensor. Expected to be at least $(\text{input_z} * \text{input_w} * \text{input_x} * \text{input_y}) + \text{offset_y}$ bytes.
- **output_y** – [in] Height of output tensor
- **offset_y** – [in] The offset on the Y axis to start concatenating the input tensor It is user responsibility to provide the correct value

```
void riscv_concatenation_s8_z(const int8_t *input, const uint16_t input_x, const uint16_t input_y, const
                             uint16_t input_z, const uint16_t input_w, int8_t *output, const uint16_t
                             output_z, const uint32_t offset_z)
```

`int8/uint8` concatenation function to be used for concatenating N-tensors along the Z axis This function should be called for each input tensor to concatenate. The argument `offset_z` will be used to store the input tensor in the correct position in the output tensor

i.e. `offset_z = 0` for(`i = 0` `i < num_input_tensors`; `++i`) { `riscv_concatenation_s8_z(&input[i], ..., &output, ..., ..., offset_z)` `offset_z += input_z[i]` }

This function assumes that the output tensor has:

- The same width of the input tensor
- The same height of the input tensor
- The same batch size of the input tensor

Unless specified otherwise, arguments are mandatory.

Input constraints `offset_z` is less than `output_z`

Note

This function, data layout independent, can be used to concatenate either int8 or uint8 tensors because it does not involve any arithmetic operation

Parameters

- **input** – [in] Pointer to input tensor. Input tensor must not overlap with output tensor.
- **input_x** – [in] Width of input tensor
- **input_y** – [in] Height of input tensor
- **input_z** – [in] Channels in input tensor
- **input_w** – [in] Batch size in input tensor
- **output** – [out] Pointer to output tensor. Expected to be at least $(\text{input_x} * \text{input_y} * \text{input_z} * \text{input_w}) + \text{offset_z}$ bytes.
- **output_z** – [in] Channels in output tensor
- **offset_z** – [in] The offset on the Z axis to start concatenating the input tensor It is user responsibility to provide the correct value

Convolution Functions**GetBufferSizeNNConv**

```
int32_t riscv_convolve_s16_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims, const
                                           nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_convolve_wrapper_s16_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                    *conv_params, const nmsis_nn_dims (page 1292)
                                                    *input_dims, const nmsis_nn_dims (page 1292)
                                                    *filter_dims, const nmsis_nn_dims (page 1292)
                                                    *output_dims)
```

```
int32_t riscv_convolve_wrapper_s16_get_buffer_size_dsp(const nmsis_nn_conv_params (page 1292)
                                                         *conv_params, const nmsis_nn_dims
                                                         (page 1292) *input_dims, const nmsis_nn_dims
                                                         (page 1292) *filter_dims, const nmsis_nn_dims
                                                         (page 1292) *output_dims)
```

```
int32_t riscv_convolve_s4_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims, const
                                           nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_convolve_1_x_n_s4_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                  *conv_params, const nmsis_nn_dims (page 1292)
                                                  *input_dims, const nmsis_nn_dims (page 1292)
                                                  *filter_dims, const nmsis_nn_dims (page 1292)
                                                  *output_dims)
```

```
int32_t riscv_convolve_1x1_s4_fast_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims)
```

```
int32_t riscv_convolve_wrapper_s4_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                *conv_params, const nmsis_nn_dims (page 1292)
                                                *input_dims, const nmsis_nn_dims (page 1292)
                                                *filter_dims, const nmsis_nn_dims (page 1292)
                                                *output_dims)
```

```
int32_t riscv_convolve_wrapper_s4_get_buffer_size_dsp(const nmsis_nn_conv_params (page 1292)
                                                      *conv_params, const nmsis_nn_dims
                                                      (page 1292) *input_dims, const nmsis_nn_dims
                                                      (page 1292) *filter_dims, const nmsis_nn_dims
                                                      (page 1292) *output_dims)
```

```
__STATIC_INLINE int32_t riscv_convolve_1x1_s8_fast_get_buffer_size_dsp (const nmsis_nn_dims *input_dims,
```

```
int32_t riscv_convolve_s8_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims, const
                                           nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_convolve_1_x_n_s8_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                  *conv_params, const nmsis_nn_dims (page 1292)
                                                  *input_dims, const nmsis_nn_dims (page 1292)
                                                  *filter_dims, const nmsis_nn_dims (page 1292)
                                                  *output_dims)
```

```
int32_t riscv_convolve_1x1_s8_fast_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims)
```

```
int32_t riscv_convolve_wrapper_s8_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                    *conv_params, const nmsis_nn_dims (page 1292)
                                                    *input_dims, const nmsis_nn_dims (page 1292)
                                                    *filter_dims, const nmsis_nn_dims (page 1292)
                                                    *output_dims)
```

```
int32_t riscv_convolve_wrapper_s8_get_buffer_size_dsp(const nmsis_nn_conv_params (page 1292)
                                                        *conv_params, const nmsis_nn_dims
                                                        (page 1292) *input_dims, const nmsis_nn_dims
                                                        (page 1292) *filter_dims, const nmsis_nn_dims
                                                        (page 1292) *output_dims)
```

```
__STATIC_INLINE int32_t riscv_depthwise_conv_fast_s16_get_buffer_size_dsp (const nmsis_nn_dims *input_d,
const nmsis_nn_dims *filter_dims)
```

```
int32_t riscv_depthwise_conv_fast_s16_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims,
                                                         const nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_depthwise_conv_wrapper_s16_get_buffer_size(const nmsis_nn_dw_conv_params
                                                         (page 1292) *dw_conv_params, const
                                                         nmsis_nn_dims (page 1292) *input_dims,
                                                         const nmsis_nn_dims (page 1292)
                                                         *filter_dims, const nmsis_nn_dims
                                                         (page 1292) *output_dims)
```



```
int32_t riscv_depthwise_conv_wrapper_s16_get_buffer_size_dsp(const nmsis_nn_dw_conv_params
    (page 1292) *dw_conv_params, const
    nmsis_nn_dims (page 1292)
    *input_dims, const nmsis_nn_dims
    (page 1292) *filter_dims, const
    nmsis_nn_dims (page 1292)
    *output_dims)
```

```
int32_t riscv_depthwise_conv_s4_opt_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims,
    const nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_depthwise_conv_wrapper_s4_get_buffer_size(const nmsis_nn_dw_conv_params (page 1292)
    *dw_conv_params, const nmsis_nn_dims
    (page 1292) *input_dims, const
    nmsis_nn_dims (page 1292) *filter_dims,
    const nmsis_nn_dims (page 1292)
    *output_dims)
```

```
int32_t riscv_depthwise_conv_wrapper_s4_get_buffer_size_dsp(const nmsis_nn_dw_conv_params
    (page 1292) *dw_conv_params, const
    nmsis_nn_dims (page 1292)
    *input_dims, const nmsis_nn_dims
    (page 1292) *filter_dims, const
    nmsis_nn_dims (page 1292)
    *output_dims)
```

```
int32_t riscv_depthwise_conv_s8_opt_get_buffer_size_dsp(const nmsis_nn_dims (page 1292)
    *input_dims, const nmsis_nn_dims
    (page 1292) *filter_dims)
```

```
int32_t riscv_depthwise_conv_s8_opt_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims,
    const nmsis_nn_dims (page 1292) *filter_dims)
```

```
int32_t riscv_depthwise_conv_wrapper_s8_get_buffer_size(const nmsis_nn_dw_conv_params (page 1292)
    *dw_conv_params, const nmsis_nn_dims
    (page 1292) *input_dims, const
    nmsis_nn_dims (page 1292) *filter_dims,
    const nmsis_nn_dims (page 1292)
    *output_dims)
```

```
int32_t riscv_depthwise_conv_wrapper_s8_get_buffer_size_dsp(const nmsis_nn_dw_conv_params
    (page 1292) *dw_conv_params, const
    nmsis_nn_dims (page 1292)
    *input_dims, const nmsis_nn_dims
    (page 1292) *filter_dims, const
    nmsis_nn_dims (page 1292)
    *output_dims)
```

```
int32_t riscv_transpose_conv_s8_get_buffer_size(const nmsis_nn_transpose_conv_params (page 1292)
    *transpose_conv_params, const nmsis_nn_dims
    (page 1292) *input_dims, const nmsis_nn_dims
    (page 1292) *filter_dims, const nmsis_nn_dims
    (page 1292) *out_dims)
```

int32_t **riscv_transpose_conv_s8_get_reverse_conv_buffer_size**(const *nmsis_nn_transpose_conv_params* (page 1292) *transpose_conv_params, const *nmsis_nn_dims* (page 1292) *input_dims, const *nmsis_nn_dims* (page 1292) *filter_dims)

group **GetBufferSizeNNConv**

Functions

int32_t **riscv_convolve_s16_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *input_dims, const *nmsis_nn_dims* (page 1292) *filter_dims)

Get the required buffer size for s16 convolution function.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions

Returns

The function returns required buffer size(bytes)

int32_t **riscv_convolve_wrapper_s16_get_buffer_size**(const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_dims* (page 1292) *input_dims, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims)

Get the required buffer size for riscv_convolve_wrapper_s16.

Parameters

- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). conv_params->input_offset : Not used conv_params->output_offset : Not used
- **input_dims** – [in] Input (activation) dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

int32_t **riscv_convolve_wrapper_s16_get_buffer_size_dsp**(const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_dims* (page 1292) *input_dims, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims)

Get the required buffer size for riscv_convolve_wrapper_s16 for processors with DSP extension. Refer to riscv_convolve_wrapper_s16_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use `riscv_convolve_wrapper_s16_get_buffer_size()`.

`int32_t riscv_convolve_s4_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims, const nmsis_nn_dims (page 1292) *filter_dims)`

Get the required buffer size for s4 convolution function.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions

Returns

The function returns required buffer size(bytes)

`int32_t riscv_convolve_1x_n_s4_get_buffer_size(const nmsis_nn_conv_params (page 1292) *conv_params, const nmsis_nn_dims (page 1292) *input_dims, const nmsis_nn_dims (page 1292) *filter_dims, const nmsis_nn_dims (page 1292) *output_dims)`

Get the required additional buffer size for 1xn convolution.

Parameters

- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `conv_params->input_offset` : [-127, 128] Range of `conv_params->output_offset` : [-128, 127]
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, WK, C_IN] where WK is the horizontal spatial filter dimension
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

`int32_t riscv_convolve_1x1_s4_fast_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims)`

Get the required buffer size for `riscv_convolve_1x1_s4_fast`.

Parameters

input_dims – [in] Input (activation) dimensions

Returns

The function returns the required buffer size in bytes

`int32_t riscv_convolve_wrapper_s4_get_buffer_size(const nmsis_nn_conv_params (page 1292) *conv_params, const nmsis_nn_dims (page 1292) *input_dims, const nmsis_nn_dims (page 1292) *filter_dims, const nmsis_nn_dims (page 1292) *output_dims)`

Get the required buffer size for `riscv_convolve_wrapper_s4`.

Parameters

- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **input_dims** – [in] Input (activation) dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

```
int32_t riscv_convolve_wrapper_s4_get_buffer_size_dsp(const nmsis_nn_conv_params (page 1292)
                                                    *conv_params, const nmsis_nn_dims
                                                    (page 1292) *input_dims, const
                                                    nmsis_nn_dims (page 1292) *filter_dims,
                                                    const nmsis_nn_dims (page 1292)
                                                    *output_dims)
```

Get the required buffer size for riscv_convolve_wrapper_s4 for processors with DSP extension. Refer to riscv_convolve_wrapper_s4_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_convolve_wrapper_s4_get_buffer_size().

```
__STATIC_INLINE int32_t riscv_convolve_1x1_s8_fast_get_buffer_size_dsp (const nmsis_nn_dims *input,
```

```
int32_t riscv_convolve_s8_get_buffer_size(const nmsis_nn_dims (page 1292) *input_dims, const
                                          nmsis_nn_dims (page 1292) *filter_dims)
```

Get the required buffer size for s8 convolution function.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions

Returns

The function returns required buffer size(bytes)

```
int32_t riscv_convolve_1_x_n_s8_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                *conv_params, const nmsis_nn_dims (page 1292)
                                                *input_dims, const nmsis_nn_dims (page 1292)
                                                *filter_dims, const nmsis_nn_dims (page 1292)
                                                *output_dims)
```

Get the required additional buffer size for 1xn convolution.

Parameters

- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]

- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, WK, C_IN] where WK is the horizontal spatial filter dimension
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

```
int32_t riscv_convolve_1x1_s8_fast_get_buffer_size(const nmsis_nn_dims (page 1292)
                                                    *input_dims)
```

Get the required buffer size for riscv_convolve_1x1_s8_fast.

Parameters

input_dims – [in] Input (activation) dimensions

Returns

The function returns the required buffer size in bytes

```
int32_t riscv_convolve_wrapper_s8_get_buffer_size(const nmsis_nn_conv_params (page 1292)
                                                    *conv_params, const nmsis_nn_dims
                                                    (page 1292) *input_dims, const nmsis_nn_dims
                                                    (page 1292) *filter_dims, const nmsis_nn_dims
                                                    (page 1292) *output_dims)
```

Get the required buffer size for riscv_convolve_wrapper_s8.

Parameters

- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **input_dims** – [in] Input (activation) dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

```
int32_t riscv_convolve_wrapper_s8_get_buffer_size_dsp(const nmsis_nn_conv_params (page 1292)
                                                       *conv_params, const nmsis_nn_dims
                                                       (page 1292) *input_dims, const
                                                       nmsis_nn_dims (page 1292) *filter_dims,
                                                       const nmsis_nn_dims (page 1292)
                                                       *output_dims)
```

Get the required buffer size for riscv_convolve_wrapper_s8 for processors with DSP extension. Refer to riscv_convolve_wrapper_s8_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_convolve_wrapper_s8_get_buffer_size().

```
__STATIC_INLINE int32_t riscv_depthwise_conv_fast_s16_get_buffer_size_dsp (const nmsis_nn_dims *inp
const nmsis_nn_dims *filter_dims)
```

```
int32_t riscv_depthwise_conv_fast_s16_get_buffer_size(const nmsis_nn_dims (page 1292)
                                                    *input_dims, const nmsis_nn_dims
                                                    (page 1292) *filter_dims)
```

Get the required buffer size for optimized s16 depthwise convolution function with constraint that in_channel equals out_channel.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [1, H, W, C_IN] Batch argument N is not used.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]

Returns

The function returns required buffer size in bytes

```
int32_t riscv_depthwise_conv_wrapper_s16_get_buffer_size(const nmsis_nn_dw_conv_params
                                                         (page 1292) *dw_conv_params, const
                                                         nmsis_nn_dims (page 1292)
                                                         *input_dims, const nmsis_nn_dims
                                                         (page 1292) *filter_dims, const
                                                         nmsis_nn_dims (page 1292)
                                                         *output_dims)
```

Get size of additional buffer required by riscv_depthwise_conv_wrapper_s16()

Parameters

- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) Range of dw_conv_params->input_offset : Not used Range of dw_conv_params->input_offset : Not used
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]

Returns

Size of additional memory required for optimizations in bytes.

```
int32_t riscv_depthwise_conv_wrapper_s16_get_buffer_size_dsp(const
                                                            nmsis_nn_dw_conv_params
                                                            (page 1292) *dw_conv_params,
                                                            const nmsis_nn_dims (page 1292)
                                                            *input_dims, const
                                                            nmsis_nn_dims (page 1292)
                                                            *filter_dims, const
                                                            nmsis_nn_dims (page 1292)
                                                            *output_dims)
```

Get size of additional buffer required by riscv_depthwise_conv_wrapper_s16() for processors with DSP extension. Refer to riscv_depthwise_conv_wrapper_s16_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_depthwise_conv_wrapper_s16_get_buffer_size().

```
int32_t riscv_depthwise_conv_s4_opt_get_buffer_size(const nmsis_nn_dims (page 1292)
                                                    *input_dims, const nmsis_nn_dims
                                                    (page 1292) *filter_dims)
```

Get the required buffer size for optimized s4 depthwise convolution function with constraint that in_channel equals out_channel.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [1, H, W, C_IN] Batch argument N is not used.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]

Returns

The function returns required buffer size in bytes

```
int32_t riscv_depthwise_conv_wrapper_s4_get_buffer_size(const nmsis_nn_dw_conv_params
                                                         (page 1292) *dw_conv_params, const
                                                         nmsis_nn_dims (page 1292)
                                                         *input_dims, const nmsis_nn_dims
                                                         (page 1292) *filter_dims, const
                                                         nmsis_nn_dims (page 1292)
                                                         *output_dims)
```

Get size of additional buffer required by riscv_depthwise_conv_wrapper_s4()

Parameters

- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) Range of dw_conv_params->input_offset : [-127, 128] Range of dw_conv_params->input_offset : [-128, 127]
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]

Returns

Size of additional memory required for optimizations in bytes.

```
int32_t riscv_depthwise_conv_wrapper_s4_get_buffer_size_dsp(const nmsis_nn_dw_conv_params
                                                             (page 1292) *dw_conv_params,
                                                             const nmsis_nn_dims (page 1292)
                                                             *input_dims, const nmsis_nn_dims
                                                             (page 1292) *filter_dims, const
                                                             nmsis_nn_dims (page 1292)
                                                             *output_dims)
```

Get size of additional buffer required by riscv_depthwise_conv_wrapper_s4() for processors with DSP extension. Refer to riscv_depthwise_conv_wrapper_s4_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_depthwise_conv_wrapper_s4_get_buffer_size().

```
int32_t riscv_depthwise_conv_s8_opt_get_buffer_size_dsp(const nmsis_nn_dims (page 1292)
                                                          *input_dims, const nmsis_nn_dims
                                                          (page 1292) *filter_dims)
```

Get the required buffer size for optimized s8 depthwise convolution function with constraint that in_channel equals out_channel. This is for processors with DSP extension. Refer to riscv_depthwise_conv_s8_opt_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_depthwise_conv_s8_opt_get_buffer_size(). Note also this is a support function, so not recommended to call directly even on Host.

```
int32_t riscv_depthwise_conv_s8_opt_get_buffer_size(const nmsis_nn_dims (page 1292)
                                                    *input_dims, const nmsis_nn_dims
                                                    (page 1292) *filter_dims)
```

Get the required buffer size for optimized s8 depthwise convolution function with constraint that in_channel equals out_channel.

Parameters

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [1, H, W, C_IN] Batch argument N is not used.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]

Returns

The function returns required buffer size in bytes

```
int32_t riscv_depthwise_conv_wrapper_s8_get_buffer_size(const nmsis_nn_dw_conv_params
                                                         (page 1292) *dw_conv_params, const
                                                         nmsis_nn_dims (page 1292)
                                                         *input_dims, const nmsis_nn_dims
                                                         (page 1292) *filter_dims, const
                                                         nmsis_nn_dims (page 1292)
                                                         *output_dims)
```

Get size of additional buffer required by riscv_depthwise_conv_wrapper_s8()

Parameters

- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) Range of dw_conv_params->input_offset : [-127, 128] Range of dw_conv_params->input_offset : [-128, 127]
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]

Returns

Size of additional memory required for optimizations in bytes.

```
int32_t riscv_depthwise_conv_wrapper_s8_get_buffer_size_dsp(const nmsis_nn_dw_conv_params
                                                             (page 1292) *dw_conv_params,
                                                             const nmsis_nn_dims (page 1292)
                                                             *input_dims, const nmsis_nn_dims
                                                             (page 1292) *filter_dims, const
                                                             nmsis_nn_dims (page 1292)
                                                             *output_dims)
```


Get size of additional buffer required by `riscv_depthwise_conv_wrapper_s8()` for processors with DSP extension. Refer to `riscv_depthwise_conv_wrapper_s8_get_buffer_size()` for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use `riscv_depthwise_conv_wrapper_s8_get_buffer_size()`.

```
int32_t riscv_transpose_conv_s8_get_buffer_size(const nmsis_nn_transpose_conv_params
                                                (page 1292) *transpose_conv_params, const
                                                nmsis_nn_dims (page 1292) *input_dims, const
                                                nmsis_nn_dims (page 1292) *filter_dims, const
                                                nmsis_nn_dims (page 1292) *out_dims)
```

Get the required buffer size for ctx in s8 transpose conv function.

Parameters

- **transposed_conv_params** – [in] Transposed convolution parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **out_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

Returns

The function returns required buffer size(bytes)

```
int32_t riscv_transpose_conv_s8_get_reverse_conv_buffer_size(const nm-
                                                             sis_nn_transpose_conv_params
                                                             (page 1292)
                                                             *transpose_conv_params, const
                                                             nmsis_nn_dims (page 1292)
                                                             *input_dims, const
                                                             nmsis_nn_dims (page 1292)
                                                             *filter_dims)
```

Get the required buffer size for output_ctx in s8 transpose conv function.

Parameters

- **transposed_conv_params** – [in] Transposed convolution parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions

Returns

The function returns required buffer size(bytes)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1_x_n_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1_x_n_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_HWC_q7_fast_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s4_fast**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s8_fast**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_even_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *packed_filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_basic**(const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_fast**(const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_fast_nonsquare**(const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_basic**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_basic_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_fast**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_fast_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_RGB**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const *nmsis_nn_bias_data* (page 1292) *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *packed_filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *upscale_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const *nmsis_nn_bias_data* (page 1292) *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_3x3_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_fast_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

static void __attribute__((unused))

static void depthwise_conv_s16_generic_s16(const int16_t *input, const uint16_t input_batches, const uint16_t input_x, const uint16_t input_y, const uint16_t input_ch, const int8_t *kernel, const uint16_t ch_mult, const uint16_t kernel_x, const uint16_t kernel_y, const uint16_t pad_x, const uint16_t pad_y, const uint16_t stride_x, const uint16_t stride_y, const int64_t *bias, int16_t *output, const int32_t *output_shift, const int32_t *output_mult, const uint16_t output_x, const uint16_t output_y, const int32_t output_activation_min, const int32_t output_activation_max, const uint16_t dilation_x, const uint16_t dilation_y)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

static void **depthwise_conv_s4_generic**(const int8_t *input, const int32_t input_batches, const int32_t input_x, const int32_t input_y, const int32_t input_ch, const int8_t *kernel, const int32_t output_ch, const int32_t ch_mult, const int32_t kernel_x, const int32_t kernel_y, const int32_t pad_x, const int32_t pad_y, const int32_t stride_x, const int32_t stride_y, const int32_t *bias, int8_t *output, const int32_t *output_shift, const int32_t *output_mult, const int32_t output_x, const int32_t output_y, const int32_t output_offset, const int32_t input_offset, const int32_t output_activation_min, const int32_t output_activation_max, const int32_t dilation_x, const int32_t dilation_y)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s4_opt**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

static void **depthwise_conv_s8_mult_4**(const int8_t *input, const int32_t input_x, const int32_t input_y, const int32_t input_ch, const int8_t *kernel, const int32_t output_ch, const int32_t ch_mult, const int32_t kernel_x, const int32_t kernel_y, const int32_t pad_x, const int32_t pad_y, const int32_t stride_x, const int32_t stride_y, const int32_t *bias, int8_t *output, const int32_t *output_shift, const int32_t *output_mult, const int32_t output_x, const int32_t output_y, const int32_t output_offset, const int32_t input_offset, const int32_t output_activation_min, const int32_t output_activation_max)

```
static void depthwise_conv_s8_generic(const int8_t *input, const uint16_t input_batches, const uint16_t
    input_x, const uint16_t input_y, const uint16_t input_ch, const int8_t
    *kernel, const uint16_t output_ch, const uint16_t ch_mult, const
    uint16_t kernel_x, const uint16_t kernel_y, const uint16_t pad_x, const
    uint16_t pad_y, const uint16_t stride_x, const uint16_t stride_y, const
    int32_t *bias, int8_t *output, const int32_t *output_shift, const int32_t
    *output_mult, const uint16_t output_x, const uint16_t output_y, const
    int32_t output_offset, const int32_t input_offset, const int32_t
    output_activation_min, const int32_t output_activation_max, const
    uint16_t dilation_x, const uint16_t dilation_y)
```

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s8_opt**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_separable_conv_HWC_q7**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_separable_conv_HWC_q7_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

riscv_nmsis_nn_status (page 1291) **riscv_transpose_conv_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_transpose_conv_params* (page 1292) *transpose_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_transpose_conv_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *reverse_conv_ctx, const *nmsis_nn_transpose_conv_params* (page 1292) *transpose_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

group Convolution Functions

Collection of convolution, depthwise convolution functions and their variants.

The convolution is implemented in 2 steps: im2col and General Matrix Multiplication(GEMM)

im2col is a process of converting each patch of image data into a column. After im2col, the convolution is computed as matrix-matrix multiplication.

To reduce the memory footprint, the im2col is performed partially. Each iteration, only a few column (i.e., patches) are generated followed by GEMM.

Functions

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1_x_n_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

1xn convolution for s4 weights

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. stride.w * input_dims->c is a multiple of 4
 - b. Explicit constraints(since it is for 1xN convolution) -## input_dims->h equals 1 -## output_dims->h equals 1 -## filter_dims->h equals 1

Todo:

Remove constraint on output_dims->w to make the function generic.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. riscv_convolve_1_x_n_s4_get_buffer_size will return the buffer_size if required The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, WK, C_IN] where WK is the horizontal spatial filter dimension
- **filter_data** – [in] Filter data pointer. Data type: int8 as packed int4
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISCV_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISCV_NMSIS_NN_SUCCESS on successful completion.

```
riscv_nmsis_nn_status (page 1291) riscv_convolve_1_x_n_s8(const nmsis_nn_context (page 1292) *ctx,
                                                         const nmsis_nn_conv_params (page 1292)
                                                         *conv_params, const
                                                         nmsis_nn_per_channel_quant_params
                                                         (page 1292) *quant_params, const
                                                         nmsis_nn_dims (page 1292) *input_dims,
                                                         const int8_t *input_data, const
                                                         nmsis_nn_dims (page 1292) *filter_dims,
                                                         const int8_t *filter_data, const
                                                         nmsis_nn_dims (page 1292) *bias_dims,
                                                         const int32_t *bias_data, const
                                                         nmsis_nn_dims (page 1292) *output_dims,
                                                         int8_t *output_data)
```

1xn convolution

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. input_dims->n equals 1
 - b. output_dims->w is a multiple of 4

- c. Explicit constraints(since it is for 1xN convolution) -## input_dims->h equals 1 -## output_dims->h equals 1 -## filter_dims->h equals 1

Todo:

Remove constraint on output_dims->w to make the function generic.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. riscv_convolve_1_x_n_s8_get_buffer_size will return the buffer_size if required The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, WK, C_IN] where WK is the horizontal spatial filter dimension
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

```

riscv_nmsis_nn_status (page 1291) riscv_convolve_1x1_HWC_q7_fast_nonsquare(const q7_t
                                                                    (page 1277) *Im_in,
                                                                    const uint16_t
                                                                    dim_im_in_x, const
                                                                    uint16_t
                                                                    dim_im_in_y, const
                                                                    uint16_t ch_im_in,
                                                                    const q7_t
                                                                    (page 1277) *wt,
                                                                    const uint16_t
                                                                    ch_im_out, const
                                                                    uint16_t
                                                                    dim_kernel_x, const
                                                                    uint16_t
                                                                    dim_kernel_y, const
                                                                    uint16_t padding_x,
                                                                    const uint16_t
                                                                    padding_y, const
                                                                    uint16_t stride_x,
                                                                    const uint16_t
                                                                    stride_y, const q7_t
                                                                    (page 1277) *bias,
                                                                    const uint16_t
                                                                    bias_shift, const
                                                                    uint16_t out_shift,
                                                                    q7_t (page 1277)
                                                                    *Im_out, const
                                                                    uint16_t
                                                                    dim_im_out_x, const
                                                                    uint16_t
                                                                    dim_im_out_y,
                                                                    q15_t (page 1277)
                                                                    *bufferA, q7_t
                                                                    (page 1277)
                                                                    *bufferB)

```

Fast Q7 version of 1x1 convolution (non-square shape)

This function is optimized for convolution with 1x1 kernel size (i.e., dim_kernel_x=1 and dim_kernel_y=1). It can be used for the second half of MobileNets [1] after depthwise separable convolution.

This function is the version with full list of optimization tricks, but with some constraints: ch_im_in is multiple of 4 ch_im_out is multiple of 2

[1] MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications <https://arxiv.org/abs/1704.04861>

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in_x** – [in] input tensor dimension x
- **dim_im_in_y** – [in] input tensor dimension y
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights

- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel_x** – [in] filter kernel size x
- **dim_kernel_y** – [in] filter kernel size y
- **padding_x** – [in] padding size x
- **padding_y** – [in] padding size y
- **stride_x** – [in] convolution stride x
- **stride_y** – [in] convolution stride y
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out_x** – [in] output tensor dimension x
- **dim_im_out_y** – [in] output tensor dimension y
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s4 version for 1x1 convolution with support for non-unity stride values

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. conv_params->padding.w = conv_params->padding.h = 0

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. None is required by this function.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]

- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, 1, C_IN]
- **filter_data** – [in] Filter data pointer. Data type: int8 packed with 2x int4
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s4_fast**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Fast s4 version for 1x1 convolution (non-square shape)

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. conv_params->padding.w = conv_params->padding.h = 0
 - b. conv_params->stride.w = conv_params->stride.h = 1

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_1x1_s4_fast_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `conv_params->input_offset` : [-127, 128] Range of `conv_params->output_offset` : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]

- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, 1, C_IN]
- **filter_data** – [in] Filter data pointer. Data type: int8 packed with 2x int4
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either `RISCV_NMSIS_NN_ARG_ERROR` if argument constraints fail. or, `RISCV_NMSIS_NN_SUCCESS` on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s8 version for 1x1 convolution with support for non-unity stride values

- Supported framework : TensorFlow Lite Micro
- The following constrains on the arguments apply
 - a. conv_params->padding.w = conv_params->padding.h = 0

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. None is required by this function.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, 1, C_IN]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32

- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_1x1_s8_fast**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Fast s8 version for 1x1 convolution (non-square shape)

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. conv_params->padding.w = conv_params->padding.h = 0
 - b. conv_params->stride.w = conv_params->stride.h = 1
- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. input_dims->c is a multiple of 4
 - b. conv_params->padding.w = conv_params->padding.h = 0
 - c. conv_params->stride.w = conv_params->stride.h = 1

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. *riscv_convolve_1x1_s8_fast_get_buffer_size* will return the buffer_size if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, 1, C_IN]

- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8
- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_1x1_s8_fast_get_buffer_size` will return the `buffer_size` if required
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `conv_params->input_offset` : [-127, 128] Range of `conv_params->output_offset` : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, 1, 1, C_IN]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either `RISCV_NMSIS_NN_ARG_ERROR` if argument constraints fail. or, `RISCV_NMSIS_NN_SUCCESS` on successful completion.

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` if argument constraints fail. or, `RISCV_NMSIS_NN_SUCCESS` on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_even_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *packed_filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_basic**(const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Basic Q15 convolution function.

Buffer size:

bufferA size: ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

This basic version is designed to work for any input tensor and weight dimension.

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_fast**(const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Fast Q15 convolution function.

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

Input dimension constraints:

ch_im_in is multiple of 2

ch_im_out is multiple of 2

dim_im_out is a multiple of 2

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either **RISCV_NMSIS_NN_SIZE_MISMATCH** or **RISCV_NMSIS_NN_SUCCESS** based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q15_fast_nonsquare** (const *q15_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q15_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q15_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q15_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Fast Q15 convolution function (non-square shape)

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

Input dimension constraints:

ch_im_in is multiple of 2

ch_im_out is multiple of 2

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in_x** – [in] input tensor dimension x
- **dim_im_in_y** – [in] input tensor dimension y
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel_x** – [in] filter kernel size x
- **dim_kernel_y** – [in] filter kernel size y
- **padding_x** – [in] padding size x
- **padding_y** – [in] padding size y
- **stride_x** – [in] convolution stride x

- **stride_y** – [in] convolution stride y
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out_x** – [in] output tensor dimension x
- **dim_im_out_y** – [in] output tensor dimension y
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_basic**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Basic Q7 convolution function.

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

This basic version is designed to work for any input tensor and weight dimension.

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output

- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_basic_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Basic Q7 convolution function (non-square shape)

Basic Q7 convolution function (non-square shape)

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in_x** – [in] input tensor dimension x
- **dim_im_in_y** – [in] input tensor dimension y
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel_x** – [in] filter kernel size x
- **dim_kernel_y** – [in] filter kernel size y
- **padding_x** – [in] padding size x
- **padding_y** – [in] padding size y

- **stride_x** – [in] convolution stride x
- **stride_y** – [in] convolution stride y
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out_x** – [in] output tensor dimension x
- **dim_im_out_y** – [in] output tensor dimension y
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_fast**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Fast Q7 convolution function.

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

Input dimension constraints:

ch_im_in is multiple of 4 (because of the SIMD32 read and swap)

ch_im_out is multiple of 2 (because 2x2 mat_mult kernel)

The im2col converts the Q7 tensor input into Q15 column, which is stored in bufferA. There is reordering happening during this im2col process with riscv_q7_to_q15_reordered_no_shift. For every four elements, the second and third elements are swapped.

The computation kernel riscv_nn_mat_mult_kernel_q7_q15_reordered does the GEMM computation with the reordered columns.

To speed-up the determination of the padding condition, we split the computation into 3x3 parts, i.e., {top, mid, bottom} X {left, mid, right}. This reduces the total number of boundary condition checks and improves the data copying performance.

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension

- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_fast_nonsquare**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in_x, const uint16_t dim_im_in_y, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel_x, const uint16_t dim_kernel_y, const uint16_t padding_x, const uint16_t padding_y, const uint16_t stride_x, const uint16_t stride_y, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out_x, const uint16_t dim_im_out_y, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Fast Q7 convolution function (non-square shape)

This function is the version with full list of optimization tricks, but with some constraints: ch_im_in is multiple of 4 ch_im_out is multiple of 2

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in_x** – [in] input tensor dimension x
- **dim_im_in_y** – [in] input tensor dimension y
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel_x** – [in] filter kernel size x
- **dim_kernel_y** – [in] filter kernel size y
- **padding_x** – [in] padding size x
- **padding_y** – [in] padding size y
- **stride_x** – [in] convolution stride x
- **stride_y** – [in] convolution stride y
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out_x** – [in] output tensor dimension x
- **dim_im_out_y** – [in] output tensor dimension y
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_HWC_q7_RGB**(const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Q7 convolution function for RGB image.

Q7 version of convolution for RGB image.

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

Input dimension constraints:

ch_im_in equals 3

This kernel is written exclusively for convolution with ch_im_in equals 3. This applies on the first layer of CNNs which has input image with RGB format.

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const *nmsis_nn_bias_data* (page 1292) *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output_data)

Basic s16 convolution function.

- a. Supported framework: TensorFlow Lite micro
- b. Additional memory is required for optimization. Refer to argument 'ctx' for details.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_s16_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). `conv_params->input_offset` : Not used `conv_params->output_offset` : Not used
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int16
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Struct with optional bias data pointer. Bias data type can be int64 or int32 depending flag in struct.
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int16

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS` if successful or `RISCV_NMSIS_NN_ARG_ERROR` if incorrect arguments or `RISCV_NMSIS_NN_NO_IMPL_ERROR`

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *packed_filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Basic s4 convolution function.

- Supported framework: TensorFlow Lite micro
- Additional memory is required for optimization. Refer to argument 'ctx' for details.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_s4_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `conv_params->input_offset` : [-127, 128] Range of `conv_params->output_offset` : [-128, 127]

- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Packed Filter data pointer. Data type: int8 packed with 2x int4
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_convolve_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *upscale_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Basic s8 convolution function.

- Supported framework: TensorFlow Lite micro
- Additional memory is required for optimization. Refer to argument 'ctx' for details.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. *riscv_convolve_s8_get_buffer_size* will return the buffer_size if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of conv_params->input_offset : [-127, 128] Range of conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8

- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, CK] where HK, WK and CK are the spatial filter dimensions. CK != C_IN is used for grouped convolution, in which case the required conditions are C_IN = N * CK and C_OUT = N * M for N groups of size M.
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **upscale_dims** – [in] Inserts zeroes to upscale the input in h/w dimensions if set to 2. This is used for tranposed convolution.
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS if successful or RISC_V_NMSIS_NN_ARG_ERROR if incorrect arguments or RISC_V_NMSIS_NN_NO_IMPL_ERROR

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const *nmsis_nn_bias_data* (page 1292) *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output_data)

s16 convolution layer wrapper function with the main purpose to call the optimal kernel available in nmsis-nn to perform the convolution.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_wrapper_s8_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). `conv_params->input_offset` : Not used `conv_params->output_offset` : Not used
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int16
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]

- **bias_data** – [in] Struct with optional bias data pointer. Bias data type can be int64 or int32 depending flag in struct.
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int16

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s4 convolution layer wrapper function with the main purpose to call the optimal kernel available in nmsis-nn to perform the convolution.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_convolve_wrapper_s4_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer ,if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `conv_params->input_offset` : [-127, 128] Range of `conv_params->output_offset` : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8 packed with 2x int4
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_convolve_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s8 convolution layer wrapper function with the main purpose to call the optimal kernel available in nmsis-nn to perform the convolution.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. *riscv_convolve_wrapper_s8_get_buffer_size* will return the *buffer_size* if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of *conv_params->input_offset* : [-127, 128] Range of *conv_params->output_offset* : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either *RISCV_NMSIS_NN_ARG_ERROR* if argument constraints fail. or, *RISCV_NMSIS_NN_SUCCESS* on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_3x3_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nm-sis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Optimized s8 depthwise convolution function for 3x3 kernel size with some constraints on the input arguments(documented below). Refer `riscv_depthwise_conv_s8()` for function argument details.

- Supported framework : TensorFlow Lite Micro
- The following constraints on the arguments apply
 - a. Number of input channel equals number of output channels
 - b. Filter height and width equals 3
 - c. Padding along x is either 0 or 1.

Returns

The function returns one of the following RISC_V_NMSIS_NN_ARG_ERROR - Unsupported dimension of tensors

- Unsupported pad size along the x axis RISC_V_NMSIS_NN_SUCCESS - Successful operation

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_fast_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nm-sis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

Optimized s16 depthwise convolution function with constraint that in_channel equals out_channel. Refer `riscv_depthwise_conv_s16()` for function argument details.

RISC_V_NMSIS_NN_SUCCESS - Successful operation

- Supported framework: TensorFlow Lite
- The following constraints on the arguments apply
 - a. Number of input channel equals number of output channels or `ch_mult` equals 1
- Recommended when number of channels is 4 or greater.

Returns

The function returns one of the following `RISCV_NMSIS_NN_ARG_ERROR` - `ctx-buff == NULL` and `riscv_depthwise_conv_fast_s16_get_buffer_size() > 0` or `input channel != output channel` or `ch_mult != 1`

static void __attribute__((unused))

static void depthwise_conv_s16_generic_s16(const int16_t *input, const uint16_t input_batches, const uint16_t input_x, const uint16_t input_y, const uint16_t input_ch, const int8_t *kernel, const uint16_t ch_mult, const uint16_t kernel_x, const uint16_t kernel_y, const uint16_t pad_x, const uint16_t pad_y, const uint16_t stride_x, const uint16_t stride_y, const int64_t *bias, int16_t *output, const int32_t *output_shift, const int32_t *output_mult, const uint16_t output_x, const uint16_t output_y, const int32_t output_activation_min, const int32_t output_activation_max, const uint16_t dilation_x, const uint16_t dilation_y)

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

Basic s16 depthwise convolution function that doesn't have any constraints on the input dimensions.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function `{API}_get_buffer_size()` provides the buffer size if an additional buffer is required. exists if additional memory is. The caller is expected to clear the buffer, if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) `conv_params->input_offset` : Not used `conv_params->output_offset` : Not used
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel

- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Batch argument N is not used.
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int64
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [inout] Output data pointer. Data type: int16

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

```
static void depthwise_conv_s4_generic(const int8_t *input, const int32_t input_batches, const int32_t
    input_x, const int32_t input_y, const int32_t input_ch, const int8_t
    *kernel, const int32_t output_ch, const int32_t ch_mult, const
    int32_t kernel_x, const int32_t kernel_y, const int32_t pad_x,
    const int32_t pad_y, const int32_t stride_x, const int32_t stride_y,
    const int32_t *bias, int8_t *output, const int32_t *output_shift,
    const int32_t *output_mult, const int32_t output_x, const int32_t
    output_y, const int32_t output_offset, const int32_t input_offset,
    const int32_t output_activation_min, const int32_t
    output_activation_max, const int32_t dilation_x, const int32_t
    dilation_y)
```

```
riscv_nmsis_nn_status (page 1291) riscv_depthwise_conv_s4(const nmsis_nn_context (page 1292) *ctx,
    const nmsis_nn_dw_conv_params
    (page 1292) *dw_conv_params, const
    nmsis_nn_per_channel_quant_params
    (page 1292) *quant_params, const
    nmsis_nn_dims (page 1292) *input_dims,
    const int8_t *input, const nmsis_nn_dims
    (page 1292) *filter_dims, const int8_t
    *kernel, const nmsis_nn_dims (page 1292)
    *bias_dims, const int32_t *bias, const
    nmsis_nn_dims (page 1292) *output_dims,
    int8_t *output)
```

Basic s4 depthwise convolution function that doesn't have any constraints on the input dimensions.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required exists if additional memory is. The caller is expected to clear the buffer ,if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) dw_conv_params->dilation is not used. Range of dw_conv_params->input_offset : [-127, 128] Range of dw_conv_params->input_offset : [-128, 127]

- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Batch argument N is not used.
- **input** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **kernel** – [in] Filter data pointer. Data type: int8_t packed 4-bit weights, e.g four sequential weights [0x1, 0x2, 0x3, 0x4] packed as [0x21, 0x43].
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output** – [inout] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s4_opt**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Optimized s4 depthwise convolution function with constraint that in_channel equals out_channel. Refer riscv_depthwise_conv_s4() for function argument details.

- Supported framework: TensorFlow Lite
- The following constraints on the arguments apply
 - a. Number of input channel equals number of output channels or ch_mult equals 1
- Recommended when number of channels is 4 or greater.

Returns

The function returns one of the following RISC_V_NMSIS_NN_ARG_ERROR - input channel != output channel or ch_mult != 1 RISC_V_NMSIS_NN_SUCCESS - Successful operation

```
static void depthwise_conv_s8_mult_4(const int8_t *input, const int32_t input_x, const int32_t input_y,
                                     const int32_t input_ch, const int8_t *kernel, const int32_t
                                     output_ch, const int32_t ch_mult, const int32_t kernel_x, const
                                     int32_t kernel_y, const int32_t pad_x, const int32_t pad_y, const
                                     int32_t stride_x, const int32_t stride_y, const int32_t *bias, int8_t
                                     *output, const int32_t *output_shift, const int32_t *output_mult,
                                     const int32_t output_x, const int32_t output_y, const int32_t
                                     output_offset, const int32_t input_offset, const int32_t
                                     output_activation_min, const int32_t output_activation_max)
```

```
static void depthwise_conv_s8_generic(const int8_t *input, const uint16_t input_batches, const uint16_t
                                     input_x, const uint16_t input_y, const uint16_t input_ch, const
                                     int8_t *kernel, const uint16_t output_ch, const uint16_t ch_mult,
                                     const uint16_t kernel_x, const uint16_t kernel_y, const uint16_t
                                     pad_x, const uint16_t pad_y, const uint16_t stride_x, const
                                     uint16_t stride_y, const int32_t *bias, int8_t *output, const
                                     int32_t *output_shift, const int32_t *output_mult, const uint16_t
                                     output_x, const uint16_t output_y, const int32_t output_offset,
                                     const int32_t input_offset, const int32_t output_activation_min,
                                     const int32_t output_activation_max, const uint16_t dilation_x,
                                     const uint16_t dilation_y)
```

```
riscv_nmsis_nn_status (page 1291) riscv_depthwise_conv_s8(const nmsis_nn_context (page 1292) *ctx,
                                                           const nmsis_nn_dw_conv_params
                                                           (page 1292) *dw_conv_params, const
                                                           nmsis_nn_per_channel_quant_params
                                                           (page 1292) *quant_params, const
                                                           nmsis_nn_dims (page 1292) *input_dims,
                                                           const int8_t *input, const nmsis_nn_dims
                                                           (page 1292) *filter_dims, const int8_t
                                                           *kernel, const nmsis_nn_dims (page 1292)
                                                           *bias_dims, const int32_t *bias, const
                                                           nmsis_nn_dims (page 1292) *output_dims,
                                                           int8_t *output)
```

Basic s8 depthwise convolution function that doesn't have any constraints on the input dimensions.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required exists if additional memory is. The caller is expected to clear the buffer, if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) dw_conv_params->dilation is not used. Range of dw_conv_params->input_offset : [-127, 128] Range of dw_conv_params->input_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Batch argument N is not used.
- **input_data** – [in] Input (activation) data pointer. Data type: int8

- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_s8_opt**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Optimized s8 depthwise convolution function with constraint that in_channel equals out_channel. Refer riscv_depthwise_conv_s8() for function argument details.

- Supported framework: TensorFlow Lite
- The following constraints on the arguments apply
 - a. Number of input channel equals number of output channels or ch_mult equals 1
- Recommended when number of channels is 4 or greater.

Returns

The function returns one of the following RISC_V_NMSIS_NN_ARG_ERROR - input channel != output channel or ch_mult != 1 RISC_V_NMSIS_NN_SUCCESS - Successful operation

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

Wrapper function to pick the right optimized s16 depthwise convolution function.

- Supported framework: TensorFlow Lite
- Picks one of the the following functions
 - a. `riscv_depthwise_conv_s16()`
 - b. `riscv_depthwise_conv_fast_s16()` - RISC-V CPUs with DSP extension only

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) `dw_conv_params->dilation` is not used. Range of `dw_conv_params->input_offset` : Not used Range of `dw_conv_params->output_offset` : Not used
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **input_data** – [in] Input (activation) data pointer. Data type: int16
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int64
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]
- **output_data** – [inout] Output data pointer. Data type: int16

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS` - Successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Wrapper function to pick the right optimized s4 depthwise convolution function.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if required. The caller is expected to clear the buffer ,if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) dw_conv_params->dilation is not used. Range of dw_conv_params->input_offset : [-127, 128] Range of dw_conv_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **filter_data** – [in] Filter data pointer. Data type: int8_t packed 4-bit weights, e.g four sequential weights [0x1, 0x2, 0x3, 0x4] packed as [0x21, 0x43].
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS - Successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_conv_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_dw_conv_params* (page 1292) *dw_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Wrapper function to pick the right optimized s8 depthwise convolution function.

- Supported framework: TensorFlow Lite
- Picks one of the the following functions
 - a. `riscv_depthwise_conv_s8()`
 - b. `riscv_depthwise_conv_3x3_s8()` - RISC-V CPUs with DSP extension only
 - c. `riscv_depthwise_conv_s8_opt()`
- Check details of `riscv_depthwise_conv_s8_opt()` for potential data that can be accessed outside of the boundary.

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **dw_conv_params** – [in] Depthwise convolution parameters (e.g. strides, dilations, pads,...) `dw_conv_params->dilation` is not used. Range of `dw_conv_params->input_offset` : [-127, 128] Range of `dw_conv_params->output_offset` : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN] Batch argument N is not used and assumed to be 1.
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [1, H, W, C_OUT]
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [1, H, W, C_OUT]

- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS - Successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_separable_conv_HWC_q7** (const *q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const *q7_t* (page 1277) *wt, const uint16_t ch_im_out, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const *q7_t* (page 1277) *bias, const uint16_t bias_shift, const uint16_t out_shift, *q7_t* (page 1277) *Im_out, const uint16_t dim_im_out, *q15_t* (page 1277) *bufferA, *q7_t* (page 1277) *bufferB)

Q7 depthwise separable convolution function.

Buffer size:

bufferA size: 2*ch_im_in*dim_kernel*dim_kernel

bufferB size: 0

Input dimension constraints:

ch_im_in equals ch_im_out

Implementation: There are 3 nested loop here: Inner loop: calculate each output value with MAC instruction over an accumulator Mid loop: loop over different output channel Outer loop: loop over different output (x, y)

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias

- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_depthwise_separable_conv_HWC_q7_nonsquare**(const *q7_t*
 (page 1277)
 *Im_in,
 const
 uint16_t
 dim_im_in_x,
 const
 uint16_t
 dim_im_in_y,
 const
 uint16_t
 ch_im_in,
 const *q7_t*
 (page 1277)
 *wt, const
 uint16_t
 ch_im_out,
 const
 uint16_t
 dim_kernel_x,
 const
 uint16_t
 dim_kernel_y,
 const
 uint16_t
 padding_x,
 const
 uint16_t
 padding_y,
 const
 uint16_t
 stride_x,
 const
 uint16_t
 stride_y,
 const *q7_t*
 (page 1277)
 *bias, const
 uint16_t
 bias_shift,
 const
 uint16_t
 out_shift,
q7_t
 (page 1277)
 *Im_out,
 const
 uint16_t
 dim_im_out_x,
 const
 uint16_t
 dim_im_out_y,
q15_t
 (page 1277)
 *bufferA,
q7_t
 (page 1277)
 *bufferB)

Q7 depthwise separable convolution function (non-square shape)

This function is the version with full list of optimization tricks, but with some constraints: `ch_im_in` is equal to `ch_im_out`

Parameters

- **Im_in** – [in] pointer to input tensor
- **dim_im_in_x** – [in] input tensor dimension x
- **dim_im_in_y** – [in] input tensor dimension y
- **ch_im_in** – [in] number of input tensor channels
- **wt** – [in] pointer to kernel weights
- **ch_im_out** – [in] number of filters, i.e., output tensor channels
- **dim_kernel_x** – [in] filter kernel size x
- **dim_kernel_y** – [in] filter kernel size y
- **padding_x** – [in] padding sizes x
- **padding_y** – [in] padding sizes y
- **stride_x** – [in] convolution stride x
- **stride_y** – [in] convolution stride y
- **bias** – [in] pointer to bias
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **Im_out** – [inout] pointer to output tensor
- **dim_im_out_x** – [in] output tensor dimension x
- **dim_im_out_y** – [in] output tensor dimension y
- **bufferA** – [inout] pointer to buffer space for input
- **bufferB** – [inout] pointer to buffer space for output

Returns

The function returns either `RISCV_NMSIS_NN_SIZE_MISMATCH` or `RISCV_NMSIS_NN_SUCCESS` based on the outcome of size checking.

riscv_nmsis_nn_status (page 1291) **riscv_transpose_conv_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_transpose_conv_params* (page 1292) *transpose_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Basic s8 transpose convolution function.

- Supported framework: TensorFlow Lite micro
- Additional memory is required for optimization. Refer to arguments 'ctx' and 'output_ctx' for details.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. *riscv_transpose_conv_s8_get_buffer_size* will return the *buffer_size* if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **output_ctx** – [inout] Temporary scratch buffer. The size required size is: output width * output height * output channel * 4 The caller is expected to clear the buffer, if applicable, for security reasons.
- **transpose_conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of *transpose_conv_params->input_offset* : [-127, 128] Range of *transpose_conv_params->output_offset* : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each out channel.
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]
- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either *RISCV_NMSIS_NN_ARG_ERROR* if argument constraints fail. or, *RISCV_NMSIS_NN_SUCCESS* on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_transpose_conv_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *reverse_conv_ctx, const *nmsis_nn_transpose_conv_params* (page 1292) *transpose_conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Wrapper to select optimal transposed convolution algorithm depending on parameters.

- a. Supported framework: TensorFlow Lite micro
- b. Additional memory is required for optimization. Refer to arguments 'ctx' and 'output_ctx' for details.

Parameters

- **ctx** – [inout] Function context that contains the additional buffer if required by the function. `riscv_transpose_conv_s8_get_buffer_size` will return the `buffer_size` if required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **output_ctx** – [inout] Temporary scratch buffer. The size required size is: output width * output height * output channel * 4 The caller is expected to clear the buffer, if applicable, for security reasons.
- **transpose_conv_params** – [in] Convolution parameters (e.g. strides, dilations, pads,...). Range of `transpose_conv_params->input_offset` : [-127, 128] Range of `transpose_conv_params->output_offset` : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each out channel.
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [C_OUT, HK, WK, C_IN] where HK and WK are the spatial filter dimensions
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT]
- **bias_data** – [in] Optional bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, H, W, C_OUT]

- **output_data** – [out] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

Fully-connected Layer Functions

GetBufferSizeFC

int32_t **riscv_fully_connected_s16_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *filter_dims)

int32_t **riscv_fully_connected_s16_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *filter_dims)

int32_t **riscv_fully_connected_s8_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *filter_dims)

int32_t **riscv_fully_connected_s8_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *filter_dims)

group **GetBufferSizeFC**

Functions

int32_t **riscv_fully_connected_s16_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *filter_dims)

Get size of additional buffer required by riscv_fully_connected_s16().

Parameters

filter_dims – [in] dimension of filter

Returns

The function returns required buffer size in bytes

int32_t **riscv_fully_connected_s16_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *filter_dims)

Get size of additional buffer required by riscv_fully_connected_s16() for processors with DSP extension. Refer to riscv_fully_connected_s16_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Arm target, use riscv_fully_connected_s16_get_buffer_size().

int32_t **riscv_fully_connected_s8_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *filter_dims)

Get size of additional buffer required by riscv_fully_connected_s8() for processors with DSP extension. Refer to riscv_fully_connected_s8_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use riscv_fully_connected_s8_get_buffer_size().

int32_t **riscv_fully_connected_s8_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *filter_dims)

Get size of additional buffer required by `riscv_fully_connected_s8()`. See also `riscv_vector_sum_s8`, which is required if buffer size is > 0.

Parameters

filter_dims – [in] dimension of filter

Returns

The function returns required buffer size in bytes

riscv_nmsis_nn_status (page 1291) **riscv_batch_matmul_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_bmm_params* (page 1292) *bmm_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_lhs_dims, const int16_t *input_lhs, const *nmsis_nn_dims* (page 1292) *input_rhs_dims, const int16_t *input_rhs, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_batch_matmul_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_bmm_params* (page 1292) *bmm_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_lhs_dims, const int8_t *input_lhs, const *nmsis_nn_dims* (page 1292) *input_rhs_dims, const int8_t *input_rhs, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_mat_q7_vec_q15**(const *q15_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_mat_q7_vec_q15_opt**(const *q15_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_per_channel_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q15**(const *q15_t* (page 1277) *pV, const *q15_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q15_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q15_opt**(const *q15_t* (page 1277) *pV, const *q15_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q15_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q7**(const *q7_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q7_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q7_opt**(const *q7_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q7_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_s4**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_wrapper_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *filter_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_vector_sum_s8**(int32_t *vector_sum_buf, const int32_t vector_cols, const int32_t vector_rows, const int8_t *vector_data, const int32_t lhs_offset, const int32_t rhs_offset, const int32_t *bias_data)

riscv_nmsis_nn_status (page 1291) **riscv_vector_sum_s8_s64**(int64_t *vector_sum_buf, const int32_t vector_cols, const int32_t vector_rows, const int8_t *vector_data, const int32_t lhs_offset, const int64_t *bias_data)

group Fully-connected Layer Functions

Collection of fully-connected and matrix multiplication functions.

Fully-connected layer is basically a matrix-vector multiplication with bias. The matrix is the weights and the input/output vectors are the activation values. Supported {weight, activation} precisions include {8-bit, 8-bit} and {8-bit, 16-bit}

Functions

riscv_nmsis_nn_status (page 1291) **riscv_batch_matmul_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_bmm_params* (page 1292) *bmm_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_lhs_dims, const int16_t *input_lhs, const *nmsis_nn_dims* (page 1292) *input_rhs_dims, const int16_t *input_rhs, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

Batch matmul function with 16 bit input and output.

- a. Supported framework: TensorFlow Lite Micro
- b. Performs row * row matrix multiplication with the RHS transposed.

Parameters

- **ctx** – [in] Temporary scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons. Optional function `riscv_fully_connected_s8_get_buffer_size()` provides the buffer size if an additional buffer is required.
- **bmm_params** – [in] Batch matmul Parameters Adjoint flags are currently unused.
- **quant_params** – [in] Quantization parameters
- **input_lhs_dims** – [in] Input lhs tensor dimensions. This should be NHWC where LHS.C = RHS.C
- **input_lhs** – [in] Pointer to input tensor
- **input_rhs_dims** – [in] Input rhs tensor dimensions. This is expected to be transposed so should be NHWC where LHS.C = RHS.C
- **input_rhs** – [in] Pointer to transposed input tensor
- **output_dims** – [in] Output tensor dimensions
- **output** – [out] Pointer to the output tensor

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

riscv_nmsis_nn_status (page 1291) **riscv_batch_matmul_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_bmm_params* (page 1292) *bmm_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_lhs_dims, const int8_t *input_lhs, const *nmsis_nn_dims* (page 1292) *input_rhs_dims, const int8_t *input_rhs, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Batch matmul function with 8 bit input and output.

- a. Supported framework: TensorFlow Lite Micro
- b. Performs row * row matrix multiplication with the RHS transposed.

Parameters

- **ctx** – [in] Temporary scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons. Optional function *riscv_fully_connected_s8_get_buffer_size*() provides the buffer size if an additional buffer is required.
- **bmm_params** – [in] Batch matmul Parameters Adjoint flags are currently unused.
- **quant_params** – [in] Quantization parameters
- **input_lhs_dims** – [in] Input lhs tensor dimensions. This should be NHWC where lhs C = rhs C
- **input_lhs** – [in] Pointer to input tensor
- **input_rhs_dims** – [in] Input rhs tensor dimensions. This is expected to be transposed so should be NHWC where lhs C = rhs C
- **input_rhs** – [in] Pointer to transposed input tensor
- **output_dims** – [in] Output tensor dimensions
- **output** – [out] Pointer to the output tensor

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_mat_q7_vec_q15**(const *q15_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Mixed Q15-Q7 fully-connected layer function.

Buffer size:

vec_buffer size: 0

Q7_Q15 version of the fully connected layer

Weights are in q7_t and Activations are in q15_t

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_mat_q7_vec_q15_opt**(const *q15_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Mixed Q15-Q7 opt fully-connected layer function.

Buffer size:

vec_buffer size: 0

Q7_Q15 version of the fully connected layer

Weights are in q7_t and Activations are in q15_t

Limitation: x4 version requires weight reordering to work

Here we use only one pointer to read 4 rows in the weight matrix. So if the original q7_t matrix looks like this:

```
| a11 | a12 | a13 | a14 | a15 | a16 | a17 |
| a21 | a22 | a23 | a24 | a25 | a26 | a27 |
| a31 | a32 | a33 | a34 | a35 | a36 | a37 |
| a41 | a42 | a43 | a44 | a45 | a46 | a47 |
```

| a51 | a52 | a53 | a54 | a55 | a56 | a57 |

| a61 | a62 | a63 | a64 | a65 | a66 | a67 |

We operate on multiple-of-4 rows, so the first four rows becomes

| a11 | a21 | a12 | a22 | a31 | a41 | a32 | a42 |

| a13 | a23 | a14 | a24 | a33 | a43 | a34 | a44 |

| a15 | a25 | a16 | a26 | a35 | a45 | a36 | a46 |

The column left over will be in-order. which is: | a17 | a27 | a37 | a47 |

For the left-over rows, we do 1x1 computation, so the data remains as its original order.

So the stored weight matrix looks like this:

| a11 | a21 | a12 | a22 | a31 | a41 |

| a32 | a42 | a13 | a23 | a14 | a24 |

| a33 | a43 | a34 | a44 | a15 | a25 |

| a16 | a26 | a35 | a45 | a36 | a46 |

| a17 | a27 | a37 | a47 | a51 | a52 |

| a53 | a54 | a55 | a56 | a57 | a61 |

| a62 | a63 | a64 | a65 | a66 | a67 |

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_per_channel_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

Basic s8 Fully Connected function using per channel quantization.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **fc_params** – [in] Fully Connected layer parameters. Range of fc_params->input_offset : [-127, 128] fc_params->filter_offset : 0 Range of fc_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel quantization info. It contains the multiplier and shift values to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Input dimension is taken as Nx(H * W * C_IN)
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Two dimensional filter dimensions. Format: [N, C] N : accumulation depth and equals (H * W * C_IN) from input_dims C : output depth and equals C_OUT in output_dims H & W : Not used
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT] N, H, W : Not used
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, C_OUT] N : Batches C_OUT : Output depth H & W : Not used.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns either `RISCV_NMSIS_NN_ARG_ERROR` if argument constraints fail. or, `RISCV_NMSIS_NN_SUCCESS` on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q15**(const *q15_t* (page 1277) *pV, const *q15_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q15_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Q15 opt fully-connected layer function.

Q15 basic fully-connected layer function.

Buffer size:

vec_buffer size: 0

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q15_opt**(const *q15_t* (page 1277) *pV, const *q15_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q15_t* (page 1277) *bias, *q15_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Q15 opt fully-connected layer function.

Buffer size:

vec_buffer size: 0

Here we use only one pointer to read 4 rows in the weight matrix. So if the original matrix looks like this:

```
| a11 | a12 | a13 |  
| a21 | a22 | a23 |  
| a31 | a32 | a33 |
```

```
| a41 | a42 | a43 |
| a51 | a52 | a53 |
| a61 | a62 | a63 |
```

We operate on multiple-of-4 rows, so the first four rows becomes

```
| a11 | a12 | a21 | a22 | a31 | a32 | a41 | a42 |
| a13 | a23 | a33 | a43 |
```

Remaining rows are kept the same original order.

So the stored weight matrix looks like this:

```
| a11 | a12 | a21 | a22 | a31 | a32 | a41 | a42 |
| a13 | a23 | a33 | a43 | a51 | a52 | a53 | a61 |
| a62 | a63 |
```

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q7**(const *q7_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q7_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Q7 basic fully-connected layer function.

Buffer size:

vec_buffer size: dim_vec

This basic function is designed to work with regular weight matrix without interleaving.

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix

- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_q7_opt**(const *q7_t* (page 1277) *pV, const *q7_t* (page 1277) *pM, const uint16_t dim_vec, const uint16_t num_of_rows, const uint16_t bias_shift, const uint16_t out_shift, const *q7_t* (page 1277) *bias, *q7_t* (page 1277) *pOut, *q15_t* (page 1277) *vec_buffer)

Q7 opt fully-connected layer function.

Buffer size:

vec_buffer size: dim_vec

This opt function is designed to work with interleaved weight matrix. The vector input is assumed in *q7_t* format, we call *riscv_q7_to_q15_no_shift_shuffle* function to expand into *q15_t* format with certain weight re-ordering, refer to the function comments for more details. Here we use only one pointer to read 4 rows in the weight matrix. So if the original *q7_t* matrix looks like this:

```
| a11 | a12 | a13 | a14 | a15 | a16 | a17 |
| a21 | a22 | a23 | a24 | a25 | a26 | a27 |
| a31 | a32 | a33 | a34 | a35 | a36 | a37 |
| a41 | a42 | a43 | a44 | a45 | a46 | a47 |
| a51 | a52 | a53 | a54 | a55 | a56 | a57 |
| a61 | a62 | a63 | a64 | a65 | a66 | a67 |
```

We operate on multiple-of-4 rows, so the first four rows become

```
| a11 | a21 | a13 | a23 | a31 | a41 | a33 | a43 |
| a12 | a22 | a14 | a24 | a32 | a42 | a34 | a44 |
| a15 | a25 | a35 | a45 | a16 | a26 | a36 | a46 |
```

So within the kernel, we first read the re-ordered vector in as:

```
| b1 | b3 | and | b2 | b4 |
```

the four *q15_t* weights will look like

```
| a11 | a13 |, | a21 | a23 |, | a31 | a33 |, | a41 | a43 |
| a12 | a14 |, | a22 | a24 |, | a32 | a34 |, | a42 | a44 |
```

The column left over will be in-order. which is:

```
| a17 | a27 | a37 | a47 |
```

For the left-over rows, we do 1x1 computation, so the data remains as its original order.

So the stored weight matrix looks like this:

```
| a11 | a21 | a13 | a23 | a31 | a41 |
| a33 | a43 | a12 | a22 | a14 | a24 |
| a32 | a42 | a34 | a44 | a15 | a25 |
| a35 | a45 | a16 | a26 | a36 | a46 |
| a17 | a27 | a37 | a47 | a51 | a52 |
| a53 | a54 | a55 | a56 | a57 | a61 |
| a62 | a63 | a64 | a65 | a66 | a67 |
```

Parameters

- **pV** – [in] pointer to input vector
- **pM** – [in] pointer to matrix weights
- **dim_vec** – [in] length of the vector
- **num_of_rows** – [in] number of rows in weight matrix
- **bias_shift** – [in] amount of left-shift for bias
- **out_shift** – [in] amount of right-shift for output
- **bias** – [in] pointer to bias
- **pOut** – [inout] pointer to output vector
- **vec_buffer** – [inout] pointer to buffer space for input

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int64_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *output)

Basic s16 Fully Connected function.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.

- **fc_params** – [in] Fully Connected layer parameters. fc_params->input_offset : 0
fc_params->filter_offset : 0 fc_params->output_offset : 0
- **quant_params** – [in] Per-tensor quantization info. It contains the multiplier and shift value to be applied to the output tensor.
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Input dimension is taken as Nx(H * W * C_IN)
- **input_data** – [in] Input (activation) data pointer. Data type: int16
- **filter_dims** – [in] Two dimensional filter dimensions. Format: [N, C] N : accumulation depth and equals (H * W * C_IN) from input_dims C : output depth and equals C_OUT in output_dims H & W : Not used
- **filter_data** – [in] Filter data pointer. Data type: int8
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT] N, H, W : Not used
- **bias_data** – [in] Bias data pointer. Data type: int64
- **output_dims** – [in] Output tensor dimensions. Format: [N, C_OUT] N : Batches C_OUT : Output depth H & W : Not used.
- **output_data** – [inout] Output data pointer. Data type: int16

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

```
riscv_nmsis_nn_status (page 1291) riscv_fully_connected_s4(const nmsis_nn_context (page 1292) *ctx,
const nmsis_nn_fc_params (page 1292)
*fc_params, const
nmsis_nn_per_tensor_quant_params
(page 1292) *quant_params, const
nmsis_nn_dims (page 1292) *input_dims,
const int8_t *input, const nmsis_nn_dims
(page 1292) *filter_dims, const int8_t
*kernel, const nmsis_nn_dims (page 1292)
*bias_dims, const int32_t *bias, const
nmsis_nn_dims (page 1292)
*output_dims, int8_t *output)
```

Basic s4 Fully Connected function.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer ,if applicable, for security reasons.
- **fc_params** – [in] Fully Connected layer parameters. Range of fc_params->input_offset : [-127, 128] fc_params->filter_offset : 0 Range of fc_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-tensor quantization info. It contains the multiplier and shift value to be applied to the output tensor.
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Input dimension is taken as Nx(H * W * C_IN)

- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Two dimensional filter dimensions. Format: [N, C] N : accumulation depth and equals (H * W * C_IN) from input_dims C : output depth and equals C_OUT in output_dims H & W : Not used
- **filter_data** – [in] Filter data pointer. Data type: int8_t packed 4-bit weights, e.g four sequential weights [0x1, 0x2, 0x3, 0x4] packed as [0x21, 0x43].
- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT] N, H, W : Not used
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, C_OUT] N : Batches C_OUT : Output depth H & W : Not used.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_fully_connected_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_fc_params* (page 1292) *fc_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input, const *nmsis_nn_dims* (page 1292) *filter_dims, const int8_t *kernel, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output)

Basic s8 Fully Connected function.

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **fc_params** – [in] Fully Connected layer parameters. Range of fc_params->input_offset : [-127, 128] fc_params->filter_offset : 0 Range of fc_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-tensor quantization info. It contains the multiplier and shift value to be applied to the output tensor.
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Input dimension is taken as Nx(H * W * C_IN)
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Two dimensional filter dimensions. Format: [N, C] N : accumulation depth and equals (H * W * C_IN) from input_dims C : output depth and equals C_OUT in output_dims H & W : Not used
- **filter_data** – [in] Filter data pointer. Data type: int8

- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT] N, H, W : Not used
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, C_OUT] N : Batches C_OUT : Output depth H & W : Not used.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

```
riscv_nmsis_nn_status (page 1291) riscv_fully_connected_wrapper_s8(const nmsis_nn_context  
                                                                    (page 1292) *ctx, const  
                                                                    nmsis_nn_fc_params  
                                                                    (page 1292) *fc_params, const  
                                                                    nmsis_nn_quant_params  
                                                                    (page 1292) *quant_params,  
                                                                    const nmsis_nn_dims  
                                                                    (page 1292) *input_dims, const  
                                                                    int8_t *input_data, const  
                                                                    nmsis_nn_dims (page 1292)  
                                                                    *filter_dims, const int8_t  
                                                                    *filter_data, const  
                                                                    nmsis_nn_dims (page 1292)  
                                                                    *bias_dims, const int32_t  
                                                                    *bias_data, const  
                                                                    nmsis_nn_dims (page 1292)  
                                                                    *output_dims, int8_t  
                                                                    *output_data)
```

s8 Fully Connected layer wrapper function

- Supported framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **fc_params** – [in] Fully Connected layer parameters. Range of fc_params->input_offset : [-127, 128] fc_params->filter_offset : 0 Range of fc_params->output_offset : [-128, 127]
- **quant_params** – [in] Per-channel or per-tensor quantization info. Check struct defintion for details. It contains the multiplier and shift value(s) to be applied to each output channel
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN] Input dimension is taken as Nx(H * W * C_IN)
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Two dimensional filter dimensions. Format: [N, C] N : accumulation depth and equals (H * W * C_IN) from input_dims C : output depth and equals C_OUT in output_dims H & W : Not used
- **filter_data** – [in] Filter data pointer. Data type: int8

- **bias_dims** – [in] Bias tensor dimensions. Format: [C_OUT] N, H, W : Not used
- **bias_data** – [in] Bias data pointer. Data type: int32
- **output_dims** – [in] Output tensor dimensions. Format: [N, C_OUT] N : Batches C_OUT : Output depth H & W : Not used.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_vector_sum_s8**(int32_t *vector_sum_buf, const int32_t vector_cols, const int32_t vector_rows, const int8_t *vector_data, const int32_t lhs_offset, const int32_t rhs_offset, const int32_t *bias_data)

Calculate the sum of each row in vector_data, multiply by lhs_offset and optionally add s32 bias_data.

Parameters

- **vector_sum_buf** – [inout] Buffer for vector sums
- **vector_cols** – [in] Number of vector columns
- **vector_rows** – [in] Number of vector rows
- **vector_data** – [in] Vector of weights data
- **lhs_offset** – [in] Constant multiplied with each sum
- **rhs_offset** – [in] Constant added to each vector element before sum
- **bias_data** – [in] Vector of bias data, added to each sum.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS - Successful operation

riscv_nmsis_nn_status (page 1291) **riscv_vector_sum_s8_s64**(int64_t *vector_sum_buf, const int32_t vector_cols, const int32_t vector_rows, const int8_t *vector_data, const int32_t lhs_offset, const int64_t *bias_data)

Calculate the sum of each row in vector_data, multiply by lhs_offset and optionally add s64 bias_data.

Parameters

- **vector_sum_buf** – [inout] Buffer for vector sums
- **vector_cols** – [in] Number of vector columns
- **vector_rows** – [in] Number of vector rows
- **vector_data** – [in] Vector of weights data
- **lhs_offset** – [in] Constant multiplied with each sum
- **bias_data** – [in] Vector of bias data, added to each sum.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS - Successful operation

LSTM Layer Functions

riscv_nmsis_nn_status (page 1291) **riscv_lstm_unidirectional_s16**(const int16_t *input, int16_t *output, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers)

riscv_nmsis_nn_status (page 1291) **riscv_lstm_unidirectional_s8**(const int8_t *input, int8_t *output, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers)

group LSTM Layer Functions

Functions

riscv_nmsis_nn_status (page 1291) **riscv_lstm_unidirectional_s16**(const int16_t *input, int16_t *output, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers)

LSTM unidirectional function with 16 bit input and output and 16 bit gate output, 64 bit bias.

- a. Supported framework: TensorFlow Lite Micro

Parameters

- **input** – [in] Pointer to input data
- **output** – [out] Pointer to output data
- **params** – [in] Struct containing all information about the lstm operator, see *riscv_nn_types*.
- **buffers** – [in] Struct containing pointers to all temporary scratch buffers needed for the lstm operator, see *riscv_nn_types*.

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

riscv_nmsis_nn_status (page 1291) **riscv_lstm_unidirectional_s8**(const int8_t *input, int8_t *output, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers)

LSTM unidirectional function with 8 bit input and output and 16 bit gate output, 32 bit bias.

- a. Supported framework: TensorFlow Lite Micro

Parameters

- **input** – [in] Pointer to input data
- **output** – [out] Pointer to output data

- **params** – [in] Struct containing all information about the lstm operator, see `riscv_nn_types`.
- **buffers** – [in] Struct containing pointers to all temporary scratch buffers needed for the lstm operator, see `riscv_nn_types`.

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

Pad Layer Functions:

riscv_nmsis_nn_status (page 1291) **riscv_pad_s8**(const int8_t *input, int8_t *output, const int8_t pad_value, const *nmsis_nn_dims* (page 1292) *input_size, const *nmsis_nn_dims* (page 1292) *pre_pad, const *nmsis_nn_dims* (page 1292) *post_pad)

group Pad Layer Functions:

Functions

riscv_nmsis_nn_status (page 1291) **riscv_pad_s8**(const int8_t *input, int8_t *output, const int8_t pad_value, const *nmsis_nn_dims* (page 1292) *input_size, const *nmsis_nn_dims* (page 1292) *pre_pad, const *nmsis_nn_dims* (page 1292) *post_pad)

Expands the size of the input by adding constant values before and after the data, in all dimensions.

Parameters

- **input** – [in] Pointer to input data
- **output** – [out] Pointer to output data
- **pad_value** – [in] Value to pad with
- **input_size** – [in] Input tensor dimensions
- **pre_pad** – [in] Padding to apply before data in each dimension
- **post_pad** – [in] Padding to apply after data in each dimension

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

Pooling Functions

GetBufferSizePooling

int32_t **riscv_avgpool_s16_get_buffer_size**(const int output_x, const int ch_src)

int32_t **riscv_avgpool_s16_get_buffer_size_dsp**(const int output_x, const int ch_src)

int32_t **riscv_avgpool_s8_get_buffer_size**(const int output_x, const int ch_src)

int32_t **riscv_avgpool_s8_get_buffer_size_dsp**(const int output_x, const int ch_src)

group GetBufferSizePooling

Functions

int32_t **riscv_avgpool_s16_get_buffer_size**(const int output_x, const int ch_src)

Get the required buffer size for S16 average pooling function.

Parameters

- **dim_dst_width** – [in] output tensor dimension
- **ch_src** – [in] number of input tensor channels

Returns

The function returns required buffer size in bytes

int32_t **riscv_avgpool_s16_get_buffer_size_dsp**(const int output_x, const int ch_src)

Get the required buffer size for S16 average pooling function for processors with DSP extension. Refer to `riscv_avgpool_s16_get_buffer_size()` for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use `riscv_avgpool_s16_get_buffer_size()`.

int32_t **riscv_avgpool_s8_get_buffer_size**(const int output_x, const int ch_src)

Get the required buffer size for S8 average pooling function.

Parameters

- **dim_dst_width** – [in] output tensor dimension
- **ch_src** – [in] number of input tensor channels

Returns

The function returns required buffer size in bytes

int32_t **riscv_avgpool_s8_get_buffer_size_dsp**(const int output_x, const int ch_src)

Get the required buffer size for S8 average pooling function for processors with DSP extension. Refer to `riscv_avgpool_s8_get_buffer_size()` for function argument details.

Note

Intended for compilation on Host. If compiling for an Riscv target, use `riscv_avgpool_s8_get_buffer_size()`.

riscv_nmsis_nn_status (page 1291) **riscv_avgpool_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *dst)

riscv_nmsis_nn_status (page 1291) **riscv_avgpool_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *dst)

riscv_nmsis_nn_status (page 1291) **riscv_max_pool_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *dst)

riscv_nmsis_nn_status (page 1291) **riscv_max_pool_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *dst)

void **riscv_maxpool_q7_HWC**(*q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const uint16_t dim_im_out, *q7_t* (page 1277) *bufferA, *q7_t* (page 1277) *Im_out)

void **riscv_avepool_q7_HWC**(*q7_t* (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in, const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const uint16_t dim_im_out, *q7_t* (page 1277) *bufferA, *q7_t* (page 1277) *Im_out)

group Pooling Functions

Perform pooling functions, including max pooling and average pooling

Perform max and average pooling operations

Functions

riscv_nmsis_nn_status (page 1291) **riscv_avgpool_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *dst)

s16 average pooling function.

- Supported Framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **pool_params** – [in] Pooling parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int16
- **filter_dims** – [in] Filter tensor dimensions. Format: [H, W] Argument N and C are not used.

- **output_dims** – [in] Output tensor dimensions. Format: [H, W, C_OUT] Argument N is not used. C_OUT equals C_IN.
- **output_data** – [inout] Output data pointer. Data type: int16

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS - Successful operation
RISC_V_NMSIS_NN_ARG_ERROR - In case of invalid arguments

riscv_nmsis_nn_status (page 1291) **riscv_avgpool_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *dst)

s8 average pooling function.

- Supported Framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **pool_params** – [in] Pooling parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [H, W] Argument N and C are not used.
- **output_dims** – [in] Output tensor dimensions. Format: [H, W, C_OUT] Argument N is not used. C_OUT equals C_IN.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_max_pool_s16**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int16_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int16_t *dst)

s16 max pooling function.

- Supported Framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **pool_params** – [in] Pooling parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN]
- **src** – [in] Input (activation) data pointer. The input tensor must not overlap with the output tensor. Data type: int16
- **filter_dims** – [in] Filter tensor dimensions. Format: [H, W] Argument N and C are not used.
- **output_dims** – [in] Output tensor dimensions. Format: [H, W, C_OUT] Argument N is not used. C_OUT equals C_IN.
- **dst** – [inout] Output data pointer. Data type: int16

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_max_pool_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_pool_params* (page 1292) *pool_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *src, const *nmsis_nn_dims* (page 1292) *filter_dims, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *dst)

s8 max pooling function.

- Supported Framework: TensorFlow Lite

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function {API}_get_buffer_size() provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.
- **pool_params** – [in] Pooling parameters
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [H, W, C_IN]
- **input_data** – [in] Input (activation) data pointer. The input tensor must not overlap with the output tensor. Data type: int8
- **filter_dims** – [in] Filter tensor dimensions. Format: [H, W] Argument N and C are not used.
- **output_dims** – [in] Output tensor dimensions. Format: [H, W, C_OUT] Argument N is not used. C_OUT equals C_IN.
- **output_data** – [inout] Output data pointer. Data type: int8

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

```
void riscv_maxpool_q7_HWC(q7_t (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in,  
                        const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const  
                        uint16_t dim_im_out, q7_t (page 1277) *bufferA, q7_t (page 1277) *Im_out)
```

Q7 max pooling function.

The pooling function is implemented as split x-pooling then y-pooling.

This pooling function is input-destructive. Input data is undefined after calling this function.

Parameters

- **Im_in** – [inout] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] Not used
- **Im_out** – [inout] pointer to output tensor

```
void riscv_avepool_q7_HWC(q7_t (page 1277) *Im_in, const uint16_t dim_im_in, const uint16_t ch_im_in,  
                        const uint16_t dim_kernel, const uint16_t padding, const uint16_t stride, const  
                        uint16_t dim_im_out, q7_t (page 1277) *bufferA, q7_t (page 1277) *Im_out)
```

Q7 average pooling function.

Buffer size:

bufferA size: 2*dim_im_out*ch_im_in

The pooling function is implemented as split x-pooling then y-pooling.

This pooling function is input-destructive. Input data is undefined after calling this function.

Parameters

- **Im_in** – [inout] pointer to input tensor
- **dim_im_in** – [in] input tensor dimension
- **ch_im_in** – [in] number of input tensor channels
- **dim_kernel** – [in] filter kernel size
- **padding** – [in] padding sizes
- **stride** – [in] convolution stride
- **dim_im_out** – [in] output tensor dimension
- **bufferA** – [inout] pointer to buffer space for input
- **Im_out** – [inout] pointer to output tensor

Reshape Functions

void **riscv_reshape_s8**(const int8_t *input, int8_t *output, const uint32_t total_size)

group **Reshape Functions**

Functions

void **riscv_reshape_s8**(const int8_t *input, int8_t *output, const uint32_t total_size)

Reshape a s8 vector into another with different shape.

Note

The output is expected to be in a memory area that does not overlap with the input's

Parameters

- **input** – [**in**] points to the s8 input vector
- **output** – [**out**] points to the s8 output vector
- **total_size** – [**in**] total size of the input and output vectors in bytes

Softmax Functions

void **riscv_softmax_q15**(const [q15_t](#) (page 1277) *vec_in, const uint16_t dim_vec, [q15_t](#) (page 1277) *p_out)

void **riscv_softmax_q7**(const [q7_t](#) (page 1277) *vec_in, const uint16_t dim_vec, [q7_t](#) (page 1277) *p_out)

[riscv_nmsis_nn_status](#) (page 1291) **riscv_softmax_s16**(const int16_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const [nmsis_nn_softmax_lut_s16](#) (page 1292) *softmax_params, int16_t *output)

void **riscv_softmax_s8**(const int8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, int8_t *output)

void **riscv_softmax_s8_s16**(const int8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, int16_t *output)

void **riscv_softmax_u8**(const uint8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, uint8_t *output)

void **riscv_softmax_with_batch_q7**(const [q7_t](#) (page 1277) *vec_in, const uint16_t nb_batches, const uint16_t dim_vec, [q7_t](#) (page 1277) *p_out)

group **Softmax Functions**

Functions

void **riscv_softmax_q15**(const *q15_t* (page 1277) *vec_in, const uint16_t dim_vec, *q15_t* (page 1277) *p_out)

Q15 softmax function.

Here, instead of typical e based softmax, we use 2-based softmax, i.e.,:

$$y_i = 2^{(x_i)} / \sum(2^{x_j})$$

The relative output will be different here. But mathematically, the gradient will be the same with a log(2) scaling factor.

Parameters

- **vec_in** – [in] pointer to input vector
- **dim_vec** – [in] input vector dimension
- **p_out** – [out] pointer to output vector

void **riscv_softmax_q7**(const *q7_t* (page 1277) *vec_in, const uint16_t dim_vec, *q7_t* (page 1277) *p_out)

Q7 softmax function.

Here, instead of typical natural logarithm e based softmax, we use 2-based softmax here, i.e.,:

$$y_i = 2^{(x_i)} / \sum(2^{x_j})$$

The relative output will be different here. But mathematically, the gradient will be the same with a log(2) scaling factor.

Parameters

- **vec_in** – [in] pointer to input vector
- **dim_vec** – [in] input vector dimension
- **p_out** – [out] pointer to output vector

riscv_nmsis_nn_status (page 1291) **riscv_softmax_s16**(const int16_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const *nmsis_nn_softmax_lut_s16* (page 1292) *softmax_params, int16_t *output)

S16 softmax function.

Note

Supported framework: TensorFlow Lite micro (bit-accurate)

Parameters

- **input** – [in] Pointer to the input tensor
- **num_rows** – [in] Number of rows in the input tensor
- **row_size** – [in] Number of elements in each input row
- **mult** – [in] Input quantization multiplier

- **shift** – [in] Input quantization shift within the range [0, 31]
- **softmax_params** – [in] Softmax s16 layer parameters with two pointers to LUTs specified below. For indexing the high 9 bits are used and 7 remaining for interpolation. That means 512 entries for the 9-bit indexing and 1 extra for interpolation, i.e. 513 values for each LUT.
 - Lookup table for $\exp(x)$, where x uniform distributed between $[-10.0, 0.0]$
 - Lookup table for $1 / (1 + x)$, where x uniform distributed between $[0.0, 1.0]$
- **output** – [out] Pointer to the output tensor

Returns

The function returns `RISCV_NMSIS_NN_ARG_ERROR` Argument error check failed
`RISCV_NMSIS_NN_SUCCESS` - Successful operation

```
void riscv_softmax_s8(const int8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t
    mult, const int32_t shift, const int32_t diff_min, int8_t *output)
```

S8 softmax function.

Note

Supported framework: TensorFlow Lite micro (bit-accurate)

Parameters

- **input** – [in] Pointer to the input tensor
- **num_rows** – [in] Number of rows in the input tensor
- **row_size** – [in] Number of elements in each input row
- **mult** – [in] Input quantization multiplier
- **shift** – [in] Input quantization shift within the range [0, 31]
- **diff_min** – [in] Minimum difference with max in row. Used to check if the quantized exponential operation can be performed
- **output** – [out] Pointer to the output tensor

```
void riscv_softmax_s8_s16(const int8_t *input, const int32_t num_rows, const int32_t row_size, const
    int32_t mult, const int32_t shift, const int32_t diff_min, int16_t *output)
```

S8 to s16 softmax function.

Note

Supported framework: TensorFlow Lite micro (bit-accurate)

Parameters

- **input** – [in] Pointer to the input tensor
- **num_rows** – [in] Number of rows in the input tensor
- **row_size** – [in] Number of elements in each input row
- **mult** – [in] Input quantization multiplier
- **shift** – [in] Input quantization shift within the range [0, 31]

- **diff_min** – [in] Minimum difference with max in row. Used to check if the quantized exponential operation can be performed
- **output** – [out] Pointer to the output tensor

void **riscv_softmax_u8**(const uint8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, uint8_t *output)

U8 softmax function.

Note

Supported framework: TensorFlow Lite micro (bit-accurate)

Parameters

- **input** – [in] Pointer to the input tensor
- **num_rows** – [in] Number of rows in the input tensor
- **row_size** – [in] Number of elements in each input row
- **mult** – [in] Input quantization multiplier
- **shift** – [in] Input quantization shift within the range [0, 31]
- **diff_min** – [in] Minimum difference with max in row. Used to check if the quantized exponential operation can be performed
- **output** – [out] Pointer to the output tensor

void **riscv_softmax_with_batch_q7**(const *q7_t* (page 1277) *vec_in, const uint16_t nb_batches, const uint16_t dim_vec, *q7_t* (page 1277) *p_out)

Q7 softmax function with batch parameter.

Here, instead of typical natural logarithm e based softmax, we use 2-based softmax here, i.e.,:

$$y_i = 2^{(x_i)} / \sum(2^{x_j})$$

The relative output will be different here. But mathematically, the gradient will be the same with a log(2) scaling factor.

Parameters

- **vec_in** – [in] pointer to input vector
- **nb_batches** – [in] number of batches
- **dim_vec** – [in] input vector dimension
- **p_out** – [out] pointer to output vector

SVDF Functions

GetBufferSizeSVDF

int32_t **riscv_svdf_s8_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *weights_feature_dims)

int32_t **riscv_svdf_s8_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *weights_feature_dims)

group **GetBufferSizeSVDF**

Functions

int32_t **riscv_svdf_s8_get_buffer_size_dsp**(const *nmsis_nn_dims* (page 1292) *weights_feature_dims)

Get size of additional buffer required by riscv_svdf_s8() for processors with DSP extension. Refer to riscv_svdf_s8_get_buffer_size() for function argument details.

Note

Intended for compilation on Host. If compiling for an Arm target, use riscv_svdf_s8_get_buffer_size().

int32_t **riscv_svdf_s8_get_buffer_size**(const *nmsis_nn_dims* (page 1292) *weights_feature_dims)

Get size of additional buffer required by riscv_svdf_s8().

Parameters

filter_dims – [in] dimension of filter

Returns

The function returns required buffer size in bytes

riscv_nmsis_nn_status (page 1291) **riscv_svdf_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *input_ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_svdf_params* (page 1292) *svdf_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *input_quant_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *output_quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *state_dims, int8_t *state_data, const *nmsis_nn_dims* (page 1292) *weights_feature_dims, const int8_t *weights_feature_data, const *nmsis_nn_dims* (page 1292) *weights_time_dims, const int8_t *weights_time_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

riscv_nmsis_nn_status (page 1291) **riscv_svdf_state_s16_s8**(const *nmsis_nn_context* (page 1292) *input_ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_svdf_params* (page 1292) *svdf_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *input_quant_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *output_quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *state_dims, int16_t *state_data, const *nmsis_nn_dims* (page 1292) *weights_feature_dims, const int8_t *weights_feature_data, const *nmsis_nn_dims* (page 1292) *weights_time_dims, const int16_t *weights_time_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

group SVDF Functions

Functions

riscv_nmsis_nn_status (page 1291) **riscv_svdf_s8**(const *nmsis_nn_context* (page 1292) *ctx, const *nmsis_nn_context* (page 1292) *input_ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_svdf_params* (page 1292) *svdf_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *input_quant_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *output_quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *state_dims, int8_t *state_data, const *nmsis_nn_dims* (page 1292) *weights_feature_dims, const int8_t *weights_feature_data, const *nmsis_nn_dims* (page 1292) *weights_time_dims, const int8_t *weights_time_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s8 SVDF function with 8 bit state tensor and 8 bit time weights

- a. Supported framework: TensorFlow Lite micro

Parameters

- **ctx** – [inout] Function context (e.g. temporary buffer). Check the function definition file to see if an additional buffer is required. Optional function `riscv_fully_connected_s8_get_buffer_size()` provides the buffer size if an additional buffer is required. The caller is expected to clear the buffer, if applicable, for security reasons.

- **input_ctx** – [in] Temporary scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons.
- **output_ctx** – [in] Temporary output scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons.
- **svdf_params** – [in] SVDF Parameters Range of svdf_params->input_offset : [-128, 127]
Range of svdf_params->output_offset : [-128, 127]
- **input_quant_params** – [in] Input quantization parameters
- **output_quant_params** – [in] Output quantization parameters
- **input_dims** – [in] Input tensor dimensions
- **input_data** – [in] Pointer to input tensor
- **state_dims** – [in] State tensor dimensions
- **state_data** – [in] Pointer to state tensor
- **weights_feature_dims** – [in] Weights (feature) tensor dimensions
- **weights_feature_data** – [in] Pointer to the weights (feature) tensor
- **weights_time_dims** – [in] Weights (time) tensor dimensions
- **weights_time_data** – [in] Pointer to the weights (time) tensor
- **bias_dims** – [in] Bias tensor dimensions
- **bias_data** – [in] Pointer to bias tensor
- **output_dims** – [in] Output tensor dimensions
- **output_data** – [out] Pointer to the output tensor

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

riscv_nmsis_nn_status (page 1291) **riscv_svdf_state_s16_s8**(const *nmsis_nn_context* (page 1292) *input_ctx, const *nmsis_nn_context* (page 1292) *output_ctx, const *nmsis_nn_svdf_params* (page 1292) *svdf_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *input_quant_params, const *nmsis_nn_per_tensor_quant_params* (page 1292) *output_quant_params, const *nmsis_nn_dims* (page 1292) *input_dims, const int8_t *input_data, const *nmsis_nn_dims* (page 1292) *state_dims, int16_t *state_data, const *nmsis_nn_dims* (page 1292) *weights_feature_dims, const int8_t *weights_feature_data, const *nmsis_nn_dims* (page 1292) *weights_time_dims, const int16_t *weights_time_data, const *nmsis_nn_dims* (page 1292) *bias_dims, const int32_t *bias_data, const *nmsis_nn_dims* (page 1292) *output_dims, int8_t *output_data)

s8 SVDF function with 16 bit state tensor and 16 bit time weights

a. Supported framework: TensorFlow Lite micro

Parameters

- **input_ctx** – [in] Temporary scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons.
- **output_ctx** – [in] Temporary output scratch buffer The caller is expected to clear the buffer, if applicable, for security reasons.
- **svdf_params** – [in] SVDF Parameters Range of svdf_params->input_offset : [-128, 127] Range of svdf_params->output_offset : [-128, 127]
- **input_quant_params** – [in] Input quantization parameters
- **output_quant_params** – [in] Output quantization parameters
- **input_dims** – [in] Input tensor dimensions
- **input_data** – [in] Pointer to input tensor
- **state_dims** – [in] State tensor dimensions
- **state_data** – [in] Pointer to state tensor
- **weights_feature_dims** – [in] Weights (feature) tensor dimensions
- **weights_feature_data** – [in] Pointer to the weights (feature) tensor
- **weights_time_dims** – [in] Weights (time) tensor dimensions
- **weights_time_data** – [in] Pointer to the weights (time) tensor
- **bias_dims** – [in] Bias tensor dimensions
- **bias_data** – [in] Pointer to bias tensor
- **output_dims** – [in] Output tensor dimensions
- **output_data** – [out] Pointer to the output tensor

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

Transpose Functions

static *riscv_nmsis_nn_status* (page 1291) **riscv_transpose_s8_nhcw**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const int32_t *const in_strides, const int32_t *const out_strides)

static *riscv_nmsis_nn_status* (page 1291) **riscv_transpose_s8_default**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const int32_t *const in_strides, const int32_t *const out_strides)

riscv_nmsis_nn_status (page 1291) **riscv_transpose_s8**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const *nmsis_nn_dims* (page 1292) *const output_dims, const *nmsis_nn_transpose_params* (page 1292) *const transpose_params)

group Transpose Functions

Functions

static *riscv_nmsis_nn_status* (page 1291) **riscv_transpose_s8_nhcw**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const int32_t *const in_strides, const int32_t *const out_strides)

static *riscv_nmsis_nn_status* (page 1291) **riscv_transpose_s8_default**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const int32_t *const in_strides, const int32_t *const out_strides)

riscv_nmsis_nn_status (page 1291) **riscv_transpose_s8**(const int8_t *input, int8_t *const output, const *nmsis_nn_dims* (page 1292) *const input_dims, const *nmsis_nn_dims* (page 1292) *const output_dims, const *nmsis_nn_transpose_params* (page 1292) *const transpose_params)

Basic transpose function.

Parameters

- **input_data** – [in] Input (activation) data pointer. Data type: int8
- **output_data** – [out] Output data pointer. Data type: int8
- **input_dims** – [in] Input (activation) tensor dimensions. Format: [N, H, W, C_IN]
- **output_dims** – [in] Output tensor dimensions. Format may be arbitrary relative to input format. The output dimension will depend on the permutation dimensions. In other words the out dimensions are the result of applying the permutation to the input dimensions.
- **transpose_params** – [in] Transpose parameters. Contains permutation dimensions.

Returns

The function returns either RISC_V_NMSIS_NN_ARG_ERROR if argument constraints fail. or, RISC_V_NMSIS_NN_SUCCESS on successful completion.

group Public

A collection of functions to perform basic operations for neural network layers. Functions with a _s8 suffix support TensorFlow Lite framework.

4.3.4 Private

Structure Types

```
__STATIC_FORCEINLINE q31_t nn_clip_q63_to_q31 (q63_t x)

__STATIC_FORCEINLINE int32_t __NN_QADD (int32_t x, int32_t y)

__STATIC_FORCEINLINE uint32_t __NN_QADD16 (uint32_t x, uint32_t y)

__STATIC_FORCEINLINE uint32_t __NN_SXTB16 (uint32_t x)

__NN_PKHBT(ARG1, ARG2, ARG3)

__NN_PKHTB(ARG1, ARG2, ARG3)

__NN_PACKq7(v0, v1, v2, v3)
```

union **riscv_nnword**

Public Members

int32_t **word**

int16_t **half_words**[2]

int8_t **bytes**[4]

struct **riscv_nn_double**

union **riscv_nn_long_long**

Public Members

int64_t **long_long**

struct *riscv_nn_double* (page 1398) **word**

group Structure Types

Data structure types used by private functions.

Defines

__NN_PKHBT(ARG1, ARG2, ARG3)
definition to pack two 16 bit values.

__NN_PKHTB(ARG1, ARG2, ARG3)

__NN_PACKq7(v0, v1, v2, v3)
definition to pack four 8 bit values.

Functions

__STATIC_FORCEINLINE q31_t **nn_clip_q63_to_q31** (q63_t x)
Clips Q63 to Q31 values.

__STATIC_FORCEINLINE int32_t **__NN_QADD** (int32_t x, int32_t y)

__STATIC_FORCEINLINE uint32_t **__NN_QADD16** (uint32_t x, uint32_t y)

__STATIC_FORCEINLINE uint32_t **__NN_SXTB16** (uint32_t x)

union **riscv_nnword**

#include <riscv_nnsupportfunctions.h> Union for SIMD access of q31/s16/s8 types.

Public Members

int32_t **word**
q31 type

int16_t **half_words**[2]
s16 type

int8_t **bytes**[4]
s8 type

struct **riscv_nn_double**

#include <riscv_nnsupportfunctions.h> Union for data type long long.

union **riscv_nn_long_long**

#include <riscv_nnsupportfunctions.h>

Public Members

int64_t **long_long**

struct *riscv_nn_double* (page 1398) **word**

Convolution

int16_t ***riscv_nn_mat_mult_kernel_s16**(const int8_t *input_a, const int16_t *input_b, const int32_t output_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t activation_min, const int32_t activation_max, const int32_t num_col_a, const *nmsis_nn_bias_data* (page 1292) *const bias_data, int16_t *out_0)

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_padded_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

int16_t ***riscv_nn_depthwise_conv_nt_t_s16**(const int16_t *lhs, const int8_t *rhs, const uint16_t num_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int64_t *const output_bias, int16_t *out)

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_s4**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mul_core_1x_s4**(int32_t row_elements, const int32_t skipped_row_elements, const int8_t *row_base_ref, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mul_core_1x_s8**(int32_t row_elements, const int32_t skipped_row_elements, const int8_t *row_base_ref, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

int8_t ***riscv_nn_mat_mul_core_4x_s8**(const int32_t row_elements, const int32_t offset, const int8_t *row_base, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_interleaved_t_even_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t lhs_cols_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s16**(const int16_t *lhs, const int8_t *rhs, const *nmsis_nn_bias_data* (page 1292) *bias_data, int16_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t activation_min, const int32_t activation_max)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t lhs_cols_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t row_address_offset, const int32_t lhs_cols_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s8_s32**(const int8_t *lhs, const int8_t *rhs, int32_t *dst, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_idx_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_transpose_conv_row_s8_s32**(const int8_t *lhs, const int8_t *rhs, int32_t *output_start, const int32_t output_index, const int32_t output_max, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t input_channels, const int32_t output_channels, const int32_t lhs_offset, const int32_t row_offset, const int32_t input_x, const int32_t stride_x, const int32_t skip_rows_top, const int32_t skip_rows_bottom)

group Convolution

Support functions for Convolution and DW Convolution

Functions

int16_t ***riscv_nn_mat_mult_kernel_s16**(const int8_t *input_a, const int16_t *input_b, const int32_t output_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t activation_min, const int32_t activation_max, const int32_t num_col_a, const *nmsis_nn_bias_data* (page 1292) *const bias_data, int16_t *out_0)

Matrix-multiplication function for convolution with per-channel requantization for 16 bits convolution.

Parameters

- **input_a** – [in] pointer to operand A

- **input_b** – [in] pointer to operand B, always consists of 2 vectors.
- **output_ch** – [in] number of rows of A
- **out_shift** – [in] pointer to per output channel requantization shift parameter.
- **out_mult** – [in] pointer to per output channel requantization multiplier parameter.
- **activation_min** – [in] minimum value to clamp the output to. Range : int16
- **activation_max** – [in] maximum value to clamp the output to. Range : int16
- **num_col_a** – [in] number of columns of A
- **bias_data** – [in] pointer to struct with bias vector. The length of this vector is equal to the number of output columns (or RHS input rows). The vector can be int32 or int64 indicated by a flag in the struct.
- **out_0** – [inout] pointer to output

Returns

The function returns one of the two

- The incremented output pointer for a successful operation or
- NULL if implementation is not available.

This function does the matrix multiplication of weight matrix for all output channels with 2 columns from im2col and produces two elements/output_channel. The outputs are clamped in the range provided by activation min and max. Supported framework: TensorFlow Lite micro.

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_padded_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

Depthwise convolution of transposed rhs matrix with 4 lhs matrices. To be used in padded cases where the padding is -lhs_offset(Range: int8). Dimensions are the same for lhs and rhs.

Note

If number of channels is not a multiple of 4, upto 3 elements outside the boundary will be read out for the following.

- Output shift
- Output multiplier

- Output bias
- rhs

Parameters

- **lhs** – [in] Input left-hand side matrix
- **rhs** – [in] Input right-hand side matrix (transposed)
- **lhs_offset** – [in] LHS matrix offset(input offset). Range: -127 to 128
- **active_ch** – [in] Subset of total_ch processed
- **total_ch** – [in] Number of channels in LHS/RHS
- **out_shift** – [in] Per channel output shift. Length of vector is equal to number of channels
- **out_mult** – [in] Per channel output multiplier. Length of vector is equal to number of channels
- **out_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **row_x_col** – [in] (row_dimension * col_dimension) of LHS/RHS matrix
- **output_bias** – [in] Per channel output bias. Length of vector is equal to number of channels
- **out** – [in] Output pointer

Returns

The function returns one of the two

- Updated output pointer if an implementation is available
- NULL if no implementation is available.

```
int16_t *riscv_nn_depthwise_conv_nt_t_s16(const int16_t *lhs, const int8_t *rhs, const uint16_t
    num_ch, const int32_t *out_shift, const int32_t *out_mult,
    const int32_t activation_min, const int32_t
    activation_max, const uint16_t row_x_col, const int64_t
    *const output_bias, int16_t *out)
```

Depthwise convolution of transposed rhs matrix with 4 lhs matrices. To be used in non-padded cases. Dimensions are the same for lhs and rhs.

Note

If number of channels is not a multiple of 4, upto 3 elements outside the boundary will be read out for the following.

- Output shift
- Output multiplier
- Output bias
- rhs

Parameters

- **lhs** – [in] Input left-hand side matrix
- **rhs** – [in] Input right-hand side matrix (transposed)
- **num_ch** – [in] Number of channels in LHS/RHS
- **out_shift** – [in] Per channel output shift. Length of vector is equal to number of channels.
- **out_mult** – [in] Per channel output multiplier. Length of vector is equal to number of channels.
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **row_x_col** – [in] (row_dimension * col_dimension) of LHS/RHS matrix
- **output_bias** – [in] Per channel output bias. Length of vector is equal to number of channels.
- **out** – [in] Output pointer

Returns

The function returns one of the two

- Updated output pointer if an implementation is available
- NULL if no implementation is available.

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_s4**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

Depthwise convolution of transposed rhs matrix with 4 lhs matrices. To be used in non-padded cases. rhs consists of packed int4 data. Dimensions are the same for lhs and rhs.

Note

If number of channels is not a multiple of 4, upto 3 elements outside the boundary will be read out for the following.

- Output shift
- Output multiplier
- Output bias
- rhs

Parameters

- **lhs** – [in] Input left-hand side matrix

- **rhs** – [in] Input right-hand side matrix (transposed). Consists of int4 data packed in an int8 buffer.
- **lhs_offset** – [in] LHS matrix offset(input offset). Range: -127 to 128
- **active_ch** – [in] Subset of total_ch processed
- **total_ch** – [in] Number of channels in LHS/RHS
- **out_shift** – [in] Per channel output shift. Length of vector is equal to number of channels.
- **out_mult** – [in] Per channel output multiplier. Length of vector is equal to number of channels.
- **out_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **row_x_col** – [in] (row_dimension * col_dimension) of LHS/RHS matrix
- **output_bias** – [in] Per channel output bias. Length of vector is equal to number of channels.
- **out** – [in] Output pointer

Returns

The function returns one of the two

- Updated output pointer if an implementation is available
- NULL if no implementation is available.

riscv_nmsis_nn_status (page 1291) **riscv_nn_depthwise_conv_nt_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t input_offset, const int32_t active_ch, const int32_t total_ch, const int32_t *out_shift, const int32_t *out_mult, const int32_t out_offset, const int32_t activation_min, const int32_t activation_max, const uint16_t row_x_col, const int32_t *const output_bias, int8_t *out)

Depthwise convolution of transposed rhs matrix with 4 lhs matrices. To be used in non-padded cases. Dimensions are the same for lhs and rhs.

Note

If number of channels is not a multiple of 4, upto 3 elements outside the boundary will be read out for the following.

- Output shift
- Output multiplier
- Output bias
- rhs

Parameters

- **lhs** – [in] Input left-hand side matrix
- **rhs** – [in] Input right-hand side matrix (transposed)
- **lhs_offset** – [in] LHS matrix offset(input offset). Range: -127 to 128
- **active_ch** – [in] Subset of total_ch processed
- **total_ch** – [in] Number of channels in LHS/RHS
- **out_shift** – [in] Per channel output shift. Length of vector is equal to number of channels.
- **out_mult** – [in] Per channel output multiplier. Length of vector is equal to number of channels.
- **out_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **row_x_col** – [in] (row_dimension * col_dimension) of LHS/RHS matrix
- **output_bias** – [in] Per channel output bias. Length of vector is equal to number of channels.
- **out** – [in] Output pointer

Returns

The function returns one of the two

- Updated output pointer if an implementation is available
- NULL if no implementation is available.

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mul_core_1x_s4**(int32_t row_elements, const int32_t skipped_row_elements, const int8_t *row_base_ref, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

General Vector by Matrix multiplication with requantization, storage of result and int4 weights packed into an int8 buffer.

Pseudo-code as int8 example. Int4 filter data will be unpacked. *output = 0 sum_col = 0 for (j = 0; j < out_ch; j++) for (i = 0; i < row_elements; i++) *output += row_base_ref[i] * col_base_ref[i] sum_col += col_base_ref[i] scale sum_col using quant_params and bias store result in 'output'

Parameters

- **row_elements** – [in] number of row elements
- **skipped_row_elements** – [in] number of row elements skipped due to padding.
row_elements + skipped_row_elements = (kernel_x * kernel_y) * input_ch
- **row_base_ref** – [in] pointer to row operand
- **col_base_ref** – [in] pointer to col operand as packed int4
- **out_ch** – [out] Number of output channels

- **conv_params** – [in] Pointer to convolution parameters like offsets and activation values
- **quant_params** – [in] Pointer to per-channel quantization parameters
- **bias** – [in] Pointer to optional per-channel bias
- **output** – [out] Pointer to output where int8 results are stored.

Returns

The function performs matrix(row_base_ref) multiplication with vector(col_base_ref) and scaled result is stored in memory.

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mul_core_1x_s8**(int32_t row_elements, const int32_t skipped_row_elements, const int8_t *row_base_ref, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

General Vector by Matrix multiplication with requantization and storage of result.

Pseudo-code *output = 0 sum_col = 0 for (j = 0; j < out_ch; j++) for (i = 0; i < row_elements; i++) *output += row_base_ref[i] * col_base_ref[i] sum_col += col_base_ref[i] scale sum_col using quant_params and bias store result in 'output'

Parameters

- **row_elements** – [in] number of row elements
- **skipped_row_elements** – [in] number of row elements skipped due to padding.
row_elements + skipped_row_elements = (kernel_x * kernel_y) * input_ch
- **row_base_ref** – [in] pointer to row operand
- **col_base_ref** – [in] pointer to col operand
- **out_ch** – [out] Number of output channels
- **conv_params** – [in] Pointer to convolution parameters like offsets and activation values
- **quant_params** – [in] Pointer to per-channel quantization parameters
- **bias** – [in] Pointer to optional per-channel bias
- **output** – [out] Pointer to output where int8 results are stored.

Returns

The function performs matrix(row_base_ref) multiplication with vector(col_base_ref) and scaled result is stored in memory.

int8_t ***riscv_nn_mat_mul_core_4x_s8**(const int32_t row_elements, const int32_t offset, const int8_t *row_base, const int8_t *col_base_ref, const int32_t out_ch, const *nmsis_nn_conv_params* (page 1292) *conv_params, const *nmsis_nn_per_channel_quant_params* (page 1292) *quant_params, const int32_t *bias, int8_t *output)

Matrix-multiplication with requantization & activation function for four rows and one column.

Compliant to TFLM int8 specification.

Parameters

- **row_elements** – [in] number of row elements
- **offset** – [in] offset between rows. Can be the same as row_elements. For e.g, in a 1x1 conv scenario with stride as 1.
- **row_base** – [in] pointer to row operand
- **col_base** – [in] pointer to col operand
- **out_ch** – [in] Number of output channels
- **conv_params** – [in] Pointer to convolution parameters like offsets and activation values
- **quant_params** – [in] Pointer to per-channel quantization parameters
- **bias** – [in] Pointer to per-channel bias
- **output** – [out] Pointer to output where int8 results are stored.

Returns

The function returns the updated output pointer or NULL if implementation is not available.

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_interleaved_t_even_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t lhs_cols_offset)

General Matrix-multiplication function with per-channel requantization. This function assumes:

- LHS input matrix NOT transposed (nt)
- RHS input matrix transposed (t)
- RHS is int8 packed with 2x int4
- LHS is int8
- LHS/RHS input columns must be even numbered
- LHS must be interleaved. Compare to riscv_nn_mat_mult_nt_t_s4 where LHS is not interleaved.

Note

This operation also performs the broadcast bias addition before the requantization

Parameters

- **lhs** – [in] Pointer to the LHS input matrix
- **rhs** – [in] Pointer to the RHS input matrix
- **bias** – [in] Pointer to the bias vector. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst** – [out] Pointer to the output matrix with “m” rows and “n” columns
- **dst_multipliers** – [in] Pointer to the multipliers vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst_shifts** – [in] Pointer to the shifts vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **lhs_rows** – [in] Number of LHS input rows
- **rhs_rows** – [in] Number of RHS input rows
- **rhs_cols** – [in] Number of LHS/RHS input columns. Note this must be even.
- **lhs_offset** – [in] Offset to be applied to the LHS input value
- **dst_offset** – [in] Offset to be applied the output result
- **activation_min** – [in] Minimum value to clamp down the output. Range : int8
- **activation_max** – [in] Maximum value to clamp up the output. Range : int8
- **lhs_cols_offset** – [in] Column offset between subsequent lhs_rows

Returns

The function returns RISCv_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s16**(const int16_t *lhs, const int8_t *rhs, const *nmsis_nn_bias_data* (page 1292) *bias_data, int16_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t activation_min, const int32_t activation_max)

General Matrix-multiplication function with per-channel requantization and int16 input (LHS) and output. This function assumes:

- LHS input matrix NOT transposed (nt)
- RHS input matrix transposed (t)

Note

This operation also performs the broadcast bias addition before the requantization

Parameters

- **lhs** – [in] Pointer to the LHS input matrix
- **rhs** – [in] Pointer to the RHS input matrix
- **bias_data** – [in] Pointer to struct with bias vector. The length of this vector is equal to the number of output columns (or RHS input rows). The vector can be int32 or int64 indicated by a flag in the struct.
- **dst** – [out] Pointer to the output matrix with “m” rows and “n” columns
- **dst_multipliers** – [in] Pointer to the multipliers vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst_shifts** – [in] Pointer to the shifts vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **lhs_rows** – [in] Number of LHS input rows
- **rhs_rows** – [in] Number of RHS input rows
- **rhs_cols** – [in] Number of LHS/RHS input columns
- **activation_min** – [in] Minimum value to clamp down the output. Range : int16
- **activation_max** – [in] Maximum value to clamp up the output. Range : int16

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS or RISC_V_NMSIS_NN_NO_IMPL_ERROR

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t lhs_cols_offset)

General Matrix-multiplication function with per-channel requantization. This function assumes:

- LHS input matrix NOT transposed (nt)
- RHS input matrix transposed (t)
- RHS is int8 packed with 2x int4
- LHS is int8

Note

This operation also performs the broadcast bias addition before the requantization

Parameters

- **lhs** – [in] Pointer to the LHS input matrix
- **rhs** – [in] Pointer to the RHS input matrix
- **bias** – [in] Pointer to the bias vector. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst** – [out] Pointer to the output matrix with “m” rows and “n” columns
- **dst_multipliers** – [in] Pointer to the multipliers vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst_shifts** – [in] Pointer to the shifts vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **lhs_rows** – [in] Number of LHS input rows
- **rhs_rows** – [in] Number of RHS input rows
- **rhs_cols** – [in] Number of LHS/RHS input columns
- **lhs_offset** – [in] Offset to be applied to the LHS input value
- **dst_offset** – [in] Offset to be applied the output result
- **activation_min** – [in] Minimum value to clamp down the output. Range : int8
- **activation_max** – [in] Maximum value to clamp up the output. Range : int8
- **lhs_cols_offset** – [in] Column offset between subsequent lhs_rows

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *bias, int8_t *dst, const int32_t *dst_multipliers, const int32_t *dst_shifts, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_offset, const int32_t activation_min, const int32_t activation_max, const int32_t row_address_offset, const int32_t lhs_cols_offset)

General Matrix-multiplication function with per-channel requantization. This function assumes:

- LHS input matrix NOT transposed (nt)
- RHS input matrix transposed (t)

Note

This operation also performs the broadcast bias addition before the requantization

Parameters

- **lhs** – [in] Pointer to the LHS input matrix
- **rhs** – [in] Pointer to the RHS input matrix
- **bias** – [in] Pointer to the bias vector. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst** – [out] Pointer to the output matrix with “m” rows and “n” columns
- **dst_multipliers** – [in] Pointer to the multipliers vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **dst_shifts** – [in] Pointer to the shifts vector needed for the per-channel requantization. The length of this vector is equal to the number of output columns (or RHS input rows)
- **lhs_rows** – [in] Number of LHS input rows
- **rhs_rows** – [in] Number of RHS input rows
- **rhs_cols** – [in] Number of LHS/RHS input columns
- **lhs_offset** – [in] Offset to be applied to the LHS input value
- **dst_offset** – [in] Offset to be applied the output result
- **activation_min** – [in] Minimum value to clamp down the output. Range : int8
- **activation_max** – [in] Maximum value to clamp up the output. Range : int8
- **row_address_offset** – [in] Address offset between rows in output.
- **lhs_cols_offset** – [in] Column offset between subsequent lhs_rows

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_mat_mult_nt_t_s8_s32**(const int8_t *lhs, const int8_t *rhs, int32_t *dst, const int32_t lhs_rows, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t lhs_offset, const int32_t dst_idx_offset)

General Matrix-multiplication function with int8 input and int32 output. This function assumes:

- LHS input matrix NOT transposed (nt)
- RHS input matrix transposed (t)

Note

Dst/output buffer must be zeroed out before calling this function.

Parameters

- **lhs** – [in] Pointer to the LHS input matrix
- **rhs** – [in] Pointer to the RHS input matrix
- **dst** – [out] Pointer to the output matrix with “m” rows and “n” columns
- **lhs_rows** – [in] Number of LHS input rows
- **rhs_rows** – [in] Number of LHS input columns/RHS input rows
- **rhs_cols** – [in] Number of RHS input columns
- **lhs_offset** – [in] Offset to be applied to the LHS input value
- **dst_idx_offset** – [in] Offset between subsequent output results

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_transpose_conv_row_s8_s32**(const int8_t *lhs, const int8_t *rhs, int32_t *output_start, const int32_t output_index, const int32_t output_max, const int32_t rhs_rows, const int32_t rhs_cols, const int32_t input_channels, const int32_t output_channels, const int32_t lhs_offset, const int32_t row_offset, const int32_t input_x, const int32_t stride_x, const int32_t skip_rows_top, const int32_t skip_rows_bottom)

Row of s8 scalars multiplied with a s8 matrix and accumulated into a s32 rolling scratch buffer. Helper function for transposed convolution.

Note

Rolling buffer refers to how the function wraps around the scratch buffer, e.g. it starts writing at [output_start + output_index], writes to [output_start + output_max] and then continues at [output_start] again.

Parameters

- **lhs** – [in] Input left-hand side scalars
- **rhs** – [in] Input right-hand side matrix
- **output_start** – [out] Output buffer start
- **output_index** – [in] Output buffer current index
- **output_max** – [in] Output buffer size
- **rhs_rows** – [in] Number of rows in rhs matrix
- **rhs_cols** – [in] Number of columns in rhs matrix
- **input_channels** – [in] Number of input channels

- **output_channels** – [in] Number of output channels
- **lhs_offset** – [in] Offset added to lhs before multiplication
- **row_offset** – [in] Address offset between each row of data output
- **input_x** – [in] Length of lhs scalar row.
- **stride_x** – [in] Address offset between each scalar-matrix multiplication result.
- **skip_row_top** – [in] Skip rows on top of the filter, used for padding.
- **skip_row_bottom** – [in] Skip rows in the bottom of the filter, used for padding.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

LSTM

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_calculate_gate_s16**(const int16_t *data_in, const int16_t *hidden_in, const *nmsis_nn_lstm_gate* (page 1292) *gate, const *nmsis_nn_lstm_params* (page 1292) *params, int16_t *output, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_calculate_gate_s8_s16**(const int8_t *data_in, const int8_t *hidden_in, const *nmsis_nn_lstm_gate* (page 1292) *gate, const *nmsis_nn_lstm_params* (page 1292) *params, int16_t *output, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_step_s16**(const int16_t *data_in, const int16_t *hidden_in, int16_t *hidden_out, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_step_s8**(const int8_t *data_in, const int8_t *hidden_in, int8_t *hidden_out, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mul_result_acc_s8_s16**(const int8_t *lhs, const int8_t *rhs, const int32_t *effective_bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t batches, const int32_t batch_offset)

group LSTM

Support functions for LSTM

Functions

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_calculate_gate_s16**(const int16_t *data_in, const int16_t *hidden_in, const *nmsis_nn_lstm_gate* (page 1292) *gate, const *nmsis_nn_lstm_params* (page 1292) *params, int16_t *output, const int32_t batch_offset)

Updates a LSTM gate for an iteration step of LSTM function, int16x8_16 version.

Parameters

- **data_in** – [in] Data input pointer
- **hidden_in** – [in] Hidden state/ recurrent input pointer
- **gate_data** – [in] Struct containing all information about the gate calculation, see *riscv_nn_types*.
- **params** – [in] Struct containing all information about the lstm_operation, see *riscv_nn_types*
- **output** – [out] Hidden state/ recurrent output pointer
- **batch_offset** – [in] Number of timesteps between consecutive batches, see *riscv_nn_lstm_step_s16*.

Returns

The function returns **RISCV_NMSIS_NN_SUCCESS**

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_calculate_gate_s8_s16**(const int8_t *data_in, const int8_t *hidden_in, const *nmsis_nn_lstm_gate* (page 1292) *gate, const *nmsis_nn_lstm_params* (page 1292) *params, int16_t *output, const int32_t batch_offset)

Updates a LSTM gate for an iteration step of LSTM function, int8x8_16 version.

Parameters

- **data_in** – [in] Data input pointer
- **hidden_in** – [in] Hidden state/ recurrent input pointer
- **gate_data** – [in] Struct containing all information about the gate calculation, see *riscv_nn_types*.
- **params** – [in] Struct containing all information about the lstm_operation, see *riscv_nn_types*
- **output** – [out] Hidden state/ recurrent output pointer
- **batch_offset** – [in] Number of timesteps between consecutive batches, see *riscv_nn_lstm_step_s8*.

Returns

The function returns **RISCV_NMSIS_NN_SUCCESS**

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_step_s16**(const int16_t *data_in, const int16_t *hidden_in, int16_t *hidden_out, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers, const int32_t batch_offset)

Update LSTM function for an iteration step using s16 input and output, and s16 internally.

Parameters

- **data_in** – [in] Data input pointer
- **hidden_in** – [in] Hidden state/ recurrent input pointer
- **hidden_out** – [out] Hidden state/ recurrent output pointer
- **params** – [in] Struct containing all information about the lstm operator, see *riscv_nn_types*.
- **buffers** – [in] Struct containing pointers to all temporary scratch buffers needed for the lstm operator, see *riscv_nn_types*.
- **batch_offset** – [in] Number of timesteps between consecutive batches. E.g for *params->timing_major* = true, all batches for t=0 are stored sequentially, so batch offset = 1. For *params->time_major* = false, all time steps are stored continuously before the next batch, so batch offset = *params->time_steps*.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_lstm_step_s8**(const int8_t *data_in, const int8_t *hidden_in, int8_t *hidden_out, const *nmsis_nn_lstm_params* (page 1292) *params, *nmsis_nn_lstm_context* (page 1292) *buffers, const int32_t batch_offset)

Update LSTM function for an iteration step using s8 input and output, and s16 internally.

Parameters

- **data_in** – [in] Data input pointer
- **hidden_in** – [in] Hidden state/ recurrent input pointer
- **hidden_out** – [out] Hidden state/ recurrent output pointer
- **params** – [in] Struct containing all information about the lstm operator, see *riscv_nn_types*.
- **buffers** – [in] Struct containing pointers to all temporary scratch buffers needed for the lstm operator, see *riscv_nn_types*.
- **batch_offset** – [in] Number of timesteps between consecutive batches. E.g for *params->timing_major* = true, all batches for t=0 are stored sequentially, so batch offset = 1. For *params->time_major* = false, all time steps are stored continuously before the next batch, so batch offset = *params->time_steps*.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mul_result_acc_s8_s16**(const int8_t *lhs, const int8_t *rhs, const int32_t *effective_bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t batches, const int32_t batch_offset)

The result of the multiplication is accumulated to the passed result buffer. Multiplies a matrix by a “batched” vector (i.e. a matrix with a batch dimension composed by input vectors independent from each other).

Parameters

- **lhs** – [in] Batched vector
- **rhs** – [in] Weights - input matrix (H(Rows)xW(Columns))
- **effective_bias** – [in] Bias + lhs_offset * kernel_sum term precalculated into a constant vector.
- **dst** – [out] Output
- **dst_multiplier** – [in] Multiplier for quantization
- **dst_shift** – [in] Shift for quantization
- **rhs_cols** – [in] Vector/matrix column length
- **rhs_rows** – [in] Row count of matrix
- **batches** – [in] Batch size
- **batch_offset** – [in] Number of timesteps between consecutive batches in input, see *riscv_nn_lstm_step_s8*. Note that the output is always stored with sequential batches.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

Fully Connected

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mul_result_acc_s16**(const int16_t *lhs, const int8_t *rhs, const int64_t *effective_bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t batches, const int32_t batch_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_per_ch_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *kernel_sum, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t *dst_multiplier, const int32_t *dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max, const int32_t address_offset, const int32_t rhs_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s16**(const int16_t *lhs, const int8_t *rhs, const int64_t *bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s16_s16**(const int16_t *lhs, const int16_t *rhs, const int64_t *bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *kernel_sum, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max, const int32_t address_offset, const int32_t rhs_offset)

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_svdf_s8**(const int8_t *lhs, const int8_t *rhs, int16_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

group Fully Connected

Support functions for Fully Connected

Functions

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mul_result_acc_s16**(const int16_t *lhs, const int8_t *rhs, const int64_t *effective_bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t batches, const int32_t batch_offset)

The result of the multiplication is accumulated to the passed result buffer. Multiplies a matrix by a “batched” vector (i.e. a matrix with a batch dimension composed by input vectors independent from each other).

Parameters

- **lhs** – [in] Batched vector
- **rhs** – [in] Weights - input matrix (H(Rows)xW(Columns))
- **effective_bias** – [in] Bias + lhs_offset * kernel_sum term precalculated into a constant vector.
- **dst** – [out] Output
- **dst_multiplier** – [in] Multiplier for quantization
- **dst_shift** – [in] Shift for quantization
- **rhs_cols** – [in] Vector/matrix column length
- **rhs_rows** – [in] Row count of matrix
- **batches** – [in] Batch size
- **batch_offset** – [in] Number of timesteps between consecutive batches in input, see *riscv_nn_lstm_step_s16*. Note that the output is always stored with sequential batches.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_per_ch_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *kernel_sum, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t *dst_multiplier, const int32_t *dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max, const int32_t address_offset, const int32_t rhs_offset)

s8 Vector by Matrix (transposed) multiplication using per channel quantization for output

Parameters

- **lhs** – [in] Input left-hand side vector
- **rhs** – [in] Input right-hand side matrix (transposed)
- **kernel_sum** – [in] Kernel sums of the kernels (rhs). See `riscv_vector_sum_s8` for more info.
- **bias** – [in] Input bias
- **dst** – [out] Output vector
- **lhs_offset** – [in] Offset to be added to the input values of the left-hand side vector. Range: -127 to 128
- **dst_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **dst_multiplier** – [in] Output multipliers
- **dst_shift** – [in] Output shifts
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix
- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **address_offset** – [in] Memory position offset for dst. First output is stored at 'dst', the second at 'dst + address_offset' and so on. Default value is typically 1.
- **rhs_offset** – [in] Offset to be added to the input values of the right-hand side vector. Range: -127 to 128

Returns

The function returns `RISCV_NMSIS_NN_SUCCESS`

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s16**(const int16_t *lhs, const int8_t *rhs, const int64_t *bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

s16 Vector by s8 Matrix (transposed) multiplication

Parameters

- **lhs** – [in] Input left-hand side vector
- **rhs** – [in] Input right-hand side matrix (transposed)
- **bias** – [in] Input bias
- **dst** – [out] Output vector
- **dst_multiplier** – [in] Output multiplier
- **dst_shift** – [in] Output shift
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix

- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int16
- **activation_max** – [in] Maximum value to clamp the output to. Range: int16

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s16_s16**(const int16_t *lhs, const int16_t *rhs, const int64_t *bias, int16_t *dst, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

s16 Vector by s16 Matrix (transposed) multiplication

Parameters

- **lhs** – [in] Input left-hand side vector
- **rhs** – [in] Input right-hand side matrix (transposed)
- **bias** – [in] Input bias
- **dst** – [out] Output vector
- **dst_multiplier** – [in] Output multiplier
- **dst_shift** – [in] Output shift
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix
- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int16
- **activation_max** – [in] Maximum value to clamp the output to. Range: int16

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s4**(const int8_t *lhs, const int8_t *packed_rhs, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

s4 Vector by Matrix (transposed) multiplication

Parameters

- **lhs** – [in] Input left-hand side vector
- **packed_rhs** – [in] Input right-hand side matrix (transposed)
- **bias** – [in] Input bias
- **dst** – [out] Output vector

- **lhs_offset** – [in] Offset to be added to the input values of the left-hand side vector. Range: -127 to 128
- **dst_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **dst_multiplier** – [in] Output multiplier
- **dst_shift** – [in] Output shift
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix
- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_s8**(const int8_t *lhs, const int8_t *rhs, const int32_t *kernel_sum, const int32_t *bias, int8_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max, const int32_t address_offset, const int32_t rhs_offset)

s8 Vector by Matrix (transposed) multiplication

Parameters

- **lhs** – [in] Input left-hand side vector
- **rhs** – [in] Input right-hand side matrix (transposed)
- **kernel_sum** – [in] Kernel sums of the kernels (rhs). See *riscv_vector_sum_s8* for more info.
- **bias** – [in] Input bias
- **dst** – [out] Output vector
- **lhs_offset** – [in] Offset to be added to the input values of the left-hand side vector. Range: -127 to 128
- **dst_offset** – [in] Offset to be added to the output values. Range: -127 to 128
- **dst_multiplier** – [in] Output multiplier
- **dst_shift** – [in] Output shift
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix
- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int8
- **activation_max** – [in] Maximum value to clamp the output to. Range: int8
- **address_offset** – [in] Memory position offset for dst. First output is stored at 'dst', the second at 'dst + address_offset' and so on. Default value is typically 1.

- **rhs_offset** – [in] Offset to be added to the input values of the right-hand side vector.
Range: -127 to 128

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

riscv_nmsis_nn_status (page 1291) **riscv_nn_vec_mat_mult_t_svd_s8**(const int8_t *lhs, const int8_t *rhs, int16_t *dst, const int32_t lhs_offset, const int32_t dst_offset, const int32_t dst_multiplier, const int32_t dst_shift, const int32_t rhs_cols, const int32_t rhs_rows, const int32_t activation_min, const int32_t activation_max)

s8 Vector by Matrix (transposed) multiplication with s16 output

Parameters

- **lhs** – [in] Input left-hand side vector
- **rhs** – [in] Input right-hand side matrix (transposed)
- **dst** – [out] Output vector
- **lhs_offset** – [in] Offset to be added to the input values of the left-hand side vector.
Range: -127 to 128
- **scatter_offset** – [in] Address offset for dst. First output is stored at 'dst', the second at 'dst + scatter_offset' and so on.
- **dst_multiplier** – [in] Output multiplier
- **dst_shift** – [in] Output shift
- **rhs_cols** – [in] Number of columns in the right-hand side input matrix
- **rhs_rows** – [in] Number of rows in the right-hand side input matrix
- **activation_min** – [in] Minimum value to clamp the output to. Range: int16
- **activation_max** – [in] Maximum value to clamp the output to. Range: int16

Returns

The function returns RISCV_NMSIS_NN_SUCCESS

Softmax

void **riscv_nn_softmax_common_s8**(const int8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, const bool int16_output, void *output)

group **Softmax**

Support functions for Softmax

Functions

void **riscv_nn_softmax_common_s8**(const int8_t *input, const int32_t num_rows, const int32_t row_size, const int32_t mult, const int32_t shift, const int32_t diff_min, const bool int16_output, void *output)

Common softmax function for s8 input and s8 or s16 output.

Note

Supported framework: TensorFlow Lite micro (bit-accurate)

Parameters

- **input** – [in] Pointer to the input tensor
- **num_rows** – [in] Number of rows in the input tensor
- **row_size** – [in] Number of elements in each input row
- **mult** – [in] Input quantization multiplier
- **shift** – [in] Input quantization shift within the range [0, 31]
- **diff_min** – [in] Minimum difference with max in row. Used to check if the quantized exponential operation can be performed
- **int16_output** – [in] Indicating s8 output if 0 else s16 output
- **output** – [out] Pointer to the output tensor

BasicMath

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16_s8**(const int16_t *input_1_vect, const int16_t *input_2_vect, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t block_size, const int32_t batch_size, const int32_t batch_offset)

group **BasicMath**

Functions

riscv_nmsis_nn_status (page 1291) **riscv_elementwise_mul_s16_s8**(const int16_t *input_1_vect, const int16_t *input_2_vect, int8_t *output, const int32_t out_offset, const int32_t out_mult, const int32_t out_shift, const int32_t block_size, const int32_t batch_size, const int32_t batch_offset)

s16 elementwise multiplication with s8 output

Supported framework: TensorFlow Lite micro

Parameters

- **input_1_vect** – [in] pointer to input vector 1
- **input_2_vect** – [in] pointer to input vector 2
- **output** – [inout] pointer to output vector
- **out_offset** – [in] output offset
- **out_mult** – [in] output multiplier
- **out_shift** – [in] output shift
- **block_size** – [in] number of samples per batch
- **batch_size** – [in] number of samples per batch
- **batch_offset** – [in] Number of timesteps between consecutive batches in output, see riscv_nn_lstm_step_s8. Note that it is assumed that the input is stored with sequential batches.

Returns

The function returns RISC_V_NMSIS_NN_SUCCESS

Basic Math Functions for Neural Network Computation

void **riscv_nn_accumulate_q7_to_q15**([q15_t](#) (page 1277) *pDst, const [q7_t](#) (page 1277) *pSrc, uint32_t length)

void **riscv_nn_add_q7**(const [q7_t](#) (page 1277) *input, [q31_t](#) (page 1277) *output, uint32_t block_size)

void **riscv_nn_mult_q15**([q15_t](#) (page 1277) *pSrcA, [q15_t](#) (page 1277) *pSrcB, [q15_t](#) (page 1277) *pDst, const uint16_t out_shift, uint32_t blockSize)

void **riscv_nn_mult_q7**([q7_t](#) (page 1277) *pSrcA, [q7_t](#) (page 1277) *pSrcB, [q7_t](#) (page 1277) *pDst, const uint16_t out_shift, uint32_t blockSize)

group **Basic Math Functions for Neural Network Computation**

Basic Math Functions for Neural Network Computation

Functions

void **riscv_nn_accumulate_q7_to_q15**([q15_t](#) (page 1277) *pDst, const [q7_t](#) (page 1277) *pSrc, uint32_t length)

Converts the elements from a q7 vector and accumulate to a q15 vector.

The equation used for the conversion process is:

Description:

```
dst[n] += (q15_t) src[n] ; 0 <= n < block_size.
```

Parameters

- ***src** – [in] points to the q7 input vector
- ***dst** – [out] points to the q15 output vector

- **block_size** – [in] length of the input vector

void **riscv_nn_add_q7**(const *q7_t* (page 1277) *input, *q31_t* (page 1277) *output, uint32_t block_size)

Non-saturating addition of elements of a q7 vector.

2²⁴ samples can be added without saturating the result.

Description:

The equation used for the conversion process is:

```
sum = input[0] + input[1] + .. + input[block_size - 1]
```

Parameters

- ***input** – [in] Pointer to the q7 input vector
- ***output** – [out] Pointer to the q31 output variable.
- **block_size** – [in] length of the input vector

void **riscv_nn_mult_q15**(*q15_t* (page 1277) *pSrcA, *q15_t* (page 1277) *pSrcB, *q15_t* (page 1277) *pDst, const uint16_t out_shift, uint32_t blockSize)

Q7 vector multiplication with variable output shifts.

q7 vector multiplication with variable output shifts

Scaling and Overflow Behavior:

The function uses saturating arithmetic. Results outside of the allowable Q15 range [0x8000 0x7FFF] will be saturated.

Parameters

- ***pSrcA** – [in] pointer to the first input vector
- ***pSrcB** – [in] pointer to the second input vector
- ***pDst** – [out] pointer to the output vector
- **out_shift** – [in] amount of right-shift for output
- **blockSize** – [in] number of samples in each vector

void **riscv_nn_mult_q7**(*q7_t* (page 1277) *pSrcA, *q7_t* (page 1277) *pSrcB, *q7_t* (page 1277) *pDst, const uint16_t out_shift, uint32_t blockSize)

Q7 vector multiplication with variable output shifts.

q7 vector multiplication with variable output shifts

Scaling and Overflow Behavior:

The function uses saturating arithmetic. Results outside of the allowable Q7 range [0x80 0x7F] will be saturated.

Parameters

- ***pSrcA** – [in] pointer to the first input vector

- ***pSrcB** – [in] pointer to the second input vector
- ***pDst** – [out] pointer to the output vector
- **out_shift** – [in] amount of right-shift for output
- **blockSize** – [in] number of samples in each vector

Copy

void **riscv_nn_copy_q15**(const *q15_t* (page 1277) *pSrc, *q15_t* (page 1277) *pDst, uint32_t blockSize)

void **riscv_nn_copy_q7**(const *q7_t* (page 1277) *pSrc, *q7_t* (page 1277) *pDst, uint32_t blockSize)

group **Copy**

Functions

void **riscv_nn_copy_q15**(const *q15_t* (page 1277) *pSrc, *q15_t* (page 1277) *pDst, uint32_t blockSize)

Copies the elements of a Q15 vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Returns

none

void **riscv_nn_copy_q7**(const *q7_t* (page 1277) *pSrc, *q7_t* (page 1277) *pDst, uint32_t blockSize)

Copies the elements of a Q7 vector.

Parameters

- **pSrc** – [in] points to input vector
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Returns

none

Fill

void **riscv_nn_fill_q15**(*q15_t* (page 1277) value, *q15_t* (page 1277) *pDst, uint32_t blockSize)

void **riscv_nn_fill_q7**(*q7_t* (page 1277) value, *q7_t* (page 1277) *pDst, uint32_t blockSize)

group **Fill**

Functions

void **riscv_nn_fill_q15**(*q15_t* (page 1277) value, *q15_t* (page 1277) *pDst, uint32_t blockSize)

Fills a constant value into a Q15 vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Returns

none

void **riscv_nn_fill_q7**(*q7_t* (page 1277) value, *q7_t* (page 1277) *pDst, uint32_t blockSize)

Fills a constant value into a Q7 vector.

Parameters

- **value** – [in] input value to be filled
- **pDst** – [out] points to output vector
- **blockSize** – [in] number of samples in each vector

Returns

none

Nndata_convert

Warning

doxygengroup: Cannot find group “Nndata_convert” in doxygen xml output for project “nmsis_nn” from directory: ../build/html/doxygen/nn/xml

Warning

doxygengroup: Cannot find group “Nndata_convert” in doxygen xml output for project “nmsis_nn” from directory: ../build/html/doxygen/nn/xml

Data Conversion

void **riscv_q7_to_q15_with_offset**(const int8_t *src, int16_t *dst, int32_t block_size, int16_t offset)

group Data Conversion

// end group groupPrivTypes

Perform data type conversion in-between neural network operations

Functions

void **riscv_q7_to_q15_with_offset**(const int8_t *src, int16_t *dst, int32_t block_size, int16_t offset)

Converts the elements from a s8 vector to a s16 vector with an added offset.

Output elements are ordered. The equation used for the conversion process is:

Description:

$$\text{dst}[n] = (\text{int16_t}) \text{src}[n] + \text{offset}; \quad 0 \leq n < \text{block_size}.$$

Parameters

- **src** – [in] pointer to the s8 input vector
- **dst** – [out] pointer to the s16 output vector
- **block_size** – [in] length of the input vector
- **offset** – [in] s16 offset to be added to each input vector element.

group Private

Internal Support functions. Not intended to be called directly by a NMSIS-NN user.

4.4 Changelog

Note

NMSIS NN Library changelog will not maintain here in future, please check [Changelog](#) (page 1435) here.

4.4.1 latest

See [Changelog](#) (page 1435)

4.4.2 V1.3.1

This is release 1.3.1 version of NMSIS-NN library.

- Update get started guide for build NMSIS-NN library
- Fix testcases in NN/Tests/Cases/full: `riscv_elementwise_mul_acc_s16` and `riscv_elementwise_mul_s16_batch_offset`
- Sync cmake file changes

4.4.3 V1.3.0

This is release 1.3.0 version of NMSIS-NN library.

- Sync with CMSIS-NN library v6.0.0
- Add initial version of NMSIS NN benchmark
- Roll back p-ext optimized version for function `riscv_nn_vec_mat_mult_t_svd_s8`

4.4.4 V1.2.1

This is release 1.2.1 version of NMSIS-NN library.

- Sync with CMSIS-NN library(CMSIS-NN v5.0.0, commit id:bfc54edb)
- Optimize some functions with Nuclei DSP N3 instruction(such as: `riscv_convolve_s8`, `riscv_fully_connected_s8`, etc.)
- Optimize the rvv implementation of fullyconnect s8 function
- Fixed `riscv_nn_softmax_common_s8.c` rvv bug

4.4.5 V1.2.0

Note

- This 1.2.0 version will no longer support old gcc 10 version, and it now only support Nuclei Toolchain 2023.10(gcc13 and clang17) or later. The major changes that can be felt are as follows:
- The prefix of toolchain has changed from `riscv-nuclei-elf-` to `riscv64-unknown-elf-`
- The `-march` option has changed a lot, for example:
 - b extension changed to `_zba_zbb_zbc_zbs` extension,
 - p extension changed to `_xxldsp` or `_xxldspn1x` or `_xxldspn2x` or `_xxldspn3x` extensions which means standard DSP extension, Nuclei N1, N2, N3 DSP extensions
 - v extension changed to `v` or `_zve32f` or `_zve64f` extensions according to the riscv cpu isa used

These extensions need be combined in a certain order to get a correct arch name to match the prebuilt library name, please be cautious
- The name of libraries changed due to `-march` in gcc13 updated, for example, the library named `libnmsis_nn_rv32imacb.a` is now named `libnmsis_nn_rv32imac_zba_zbb_zbc_zbs.a` since b extension changed to `_zba_zbb_zbc_zbs`

This is release 1.2.0 version of NMSIS-NN library.

- Defined `NUCLEI_DSP_DEFAULT`, `NUCLEI_DSP_N1`, `NUCLEI_DSP_N2`, `NUCLEI_DSP_N3` in `riscv_nn_math_types.h` according to gcc options now, no longer define it in cmake files.
- RVV intrinsic APIs is update to v0.12.0
- Clean code(nnref lib)

4.4.6 V1.1.1

This is release 1.1.1 version of NMSIS-NN library.

- Sync changes from CMSIS NN v4.1.0
- Optimized more for RVP/RVV
- Add support for RV32 Vector
- Some bugfix that make tflite-micro test successfully

4.4.7 V1.1.0

This is release 1.1.0 version of NMSIS-NN library.

- Sync changes from CMSIS 5.9.0 release
- Optimized more for RVP/RVV
- Add experimental support for RV32 Vector

4.4.8 V1.0.3

This is release 1.0.3 version of NMSIS-NN library.

- Update build system for NMSIS-NN library
- Rename RISCV_VECTOR to RISCV_MATH_VECTOR in header file and source code
- Using new python script to generate NMSIS-NN library
- Support Nuclei RISC-V GCC 10.2

4.4.9 V1.0.2

This is release 1.0.2 version of NMSIS-NN library.

- Sync up to CMSIS NN library 3.0.0
- Initial support for RISC-V vector extension support

4.4.10 V1.0.1

This is release V1.0.1 version of NMSIS-DSP library.

- Both Nuclei RISC-V 32 and 64 bit cores are supported now.
- Libraries are optimized for RISC-V 32 and 64 bit DSP instructions.
- The DSP examples are now using Nuclei SDK as running environment.

4.4.11 V1.0.0

This is the first version of NMSIS-NN library.

We adapt the CMSIS-NN v1.0.0 library to use RISC-V DSP instructions, all the API names now are renamed from `arm_` to `riscv_`.

CHANGELOG

5.1 V1.6.0

This is the version of V1.6.0 release.

Important

- **Breaking Change:** The 32-bit AMO intrinsic APIs in `core_feature_base.h` (`__AMOADD_W`, `__AMOAND_W`, `__AMOOR_W`, `__AMOXOR_W`, `__AMOMAXU_W`, `__AMOMAX_W`, `__AMOMINU_W`, `__AMOMIN_W`, `__AMOSWAP_W`, `__CAS_W`) have been completely reimplemented. The previous implementation had concurrency issues (returning memory value instead of the result register) and incorrect instruction suffixes (`.d` instead of `.w`), which did not conform to the RISC-V intrinsic API semantics. The new implementation fixes all of these issues. Users relying on the old behavior should review and update their code accordingly.
- **Breaking Change:** The `RISCV_MATH_VECTOR_FLOAT16` macro in `riscv_math_types_f16.h` has been renamed to `RISCV_MATH_VECTOR_ZVFH`. Users who directly referenced the old macro name will need to update their code.
- The RVV macro control has been refined. The single `RISCV_MATH_VECTOR` macro is now replaced by more fine-grained capability macros (`RISCV_MATH_VECTOR_ZVFH`, `RISCV_MATH_VECTOR_ZVE32X`, etc.), which are derived from the `__riscv_zvfh`, `__riscv_zve32x`, etc. macros automatically defined by the compiler based on the `--march` option. See the **NMSIS-DSP** and **NMSIS-NN** sections below for details. All DSP and NN source files have been adapted to use the appropriate fine-grained macros.

Warning

VPU agnostic change: RVV Intrinsic APIs have been updated to adapt to the VPU agnostic feature. Pay attention to cases where tail elements and non-mask elements are used in RVV calculation.

- **NMSIS-Core**
 - Completely reimplement 32-bit AMO intrinsic APIs in `core_feature_base.h` to fix concurrency issues and incorrect instruction suffixes (see important notice above)
 - Fix `__ECLIC_SetModeIRQ` in `core_feature_eclic.h` using wrong register (ECLIC->SCTRL corrected to ECLIC->CTRL)
 - Clear pending error status (RESC/FESC/BESC) before issuing Cluster Cache commands in `MFlushCCache`, `SFlushCCache`, and `UFlushCCache` in `core_feature_cache.h`
 - Replace `MLockDCacheLine` with `__LW` + `__RWMB` in DCache ECC injection functions in `core_feature_ecc.h`

- Add precise ECC injection support in `core_feature_iinfo.h`: new `IINFO_ECC_INJ_WAY_Type` union, `IINFO_IsPreciseECCInjSupported()` and `IINFO_SetPreciseECCInjWay()` APIs
- Extend `core_feature_smpcc.h` with new SMPCC register types (`CC_INV_RANGE_Type`, `CC_ECC_INJ_WAY_Type`, `CC_ECC_INJ_ADDR_Type`, `CC_ECC_INJ_DATA_Type`, `IOCP_ATTR_RMP_Type`), new `SMPCC_ClearBusErrPending()` API, and update `SMPCC_CCacheTramErrInject()` / `SMPCC_CCacheDramErrInject()` to support precise ECC injection with an additional way parameter
- Fix instruction descriptions in `core_feature_dsp.h`
- Add boundary check in `__CTZ` in `core_compatible.h` to return `__RISCV_XLEN` when input is 0, preventing an infinite loop
- Fix typos in CSR field comments in `core_feature_base.h`: `accesse` to `access`, `priviled` to `privilege`, `fisrt` to `first`, `nestting` to `nesting`
- Fix `DisableSUCCM` in `core_feature_cache.h` to use direct bitmask clearing (`SMPCC_CMD_CTRL_SUP_EN_Msk | SMPCC_CMD_CTRL_USE_EN_Msk`) instead of incorrect `_VAL2FLD` approach
- Fix `ECC_IsTLBSupportECC` in `core_feature_ecc.h` to remove incorrect PLIC dependency, now directly checks `mtlbcfginfo.b.ecc`
- Fix `ECC_IsTLBSingleBitErrorOccured` and `ECC_IsTLBDoubleBitErrorOccured` in `core_feature_ecc.h` to use `ECC_ERROR_RAMID_MASK_TLB` instead of `ECC_ERROR_RAMID_MASK_DLM`
- Fix typos in `core_feature_ecc.h`: `Maintainence` to `Maintenance`, `supported` to `supported`
- Add `__ECLIC_VER` macro in `core_feature_eclic.h` with default value of 1 for ECLIC version identification
- Fix `ssnpm` field comment in `core_feature_iinfo.h` to correctly reference `Ssnpm` extension instead of `Smpnpm`
- Improve PLIC memory map documentation and fix `PLIC_SetThreshold` macro signature (removed incorrect source parameter) and `PLIC_CompleteContextInterrupt` parameter description in `core_feature_plic.h`
- Fix indentation of `#include "core_feature_smp.h"` in `nmsis_core.h`

• NMSIS-DSP

- Introduce fine-grained RVV capability macros to replace the single `RISCV_MATH_VECTOR` macro:
 - * `RISCV_MATH_VECTOR_ZVFH` when defined(`__riscv_zvfh`)
 - * `RISCV_MATH_VECTOR_ZVE32X` when defined(`__riscv_zve32x`)
 - * `RISCV_MATH_VECTOR_ZVE32F` when defined(`__riscv_zve32f`)
 - * `RISCV_MATH_VECTOR_ZVE64X` when defined(`__riscv_zve64x`)
 - * `RISCV_MATH_VECTOR_ZVE64F` when defined(`__riscv_zve64f`)
 - * `RISCV_MATH_VECTOR_ZVE64D` when defined(`__riscv_zve64d`)
- Add RVV optimization for fast functions, including `ComplexMathFunctions`, `FilteringFunctions`, and `MatrixFunctions`
- Adapt all DSP source files (~325 files) to use the appropriate fine-grained RVV macros instead of the single `RISCV_MATH_VECTOR` macro, ensuring each function only compiles RVV code for the minimum required vector extension

• NMSIS-NN

- Add fine-grained RVV capability macros in `riscv_nn_math_types.h`, consistent with NMSIS-DSP
- Adapt all NN source files (~56 files) to use the appropriate fine-grained RVV macros
- Narrow the RVV guard for `riscv_nn_requantize` RVV helper functions in `riscv_nnsupportfunctions.h` from `RISCV_MATH_VECTOR` to `RISCV_MATH_VECTOR_ZVE32X`, so these functions are only enabled when ZVE32X or higher is available

5.2 V1.5.0

This is the version of V1.5.0 release.

Important

- Please use with Nuclei RISC-V Toolchain 2025.10.
- CFFT/RFFT functions for Q15/Q31/F16/F32 have changed function signatures when the RVV extension is present. You need to provide an extra temporary buffer when calling these functions with RVV.
- Avoid using functions listed in [Deprecated List](#)³⁸, as they will be removed in future releases.
- DCT4 functions are also listed in the [Deprecated List](#)³⁹. The upstream removed these functions in commit: [4404d3e](#)⁴⁰. However, we continue to provide these functions, but only when the RVV extension is not present.

Warning

NMSIS-NN Q-format APIs Deprecation Notice: In the next version of NMSIS, all Q-format APIs in NMSIS-NN will be removed, no longer maintained, and will no longer be available. Please migrate to alternative APIs if you are currently using Q-format functions.

• NMSIS-CORE

- Update `core_feature_base.h` with new CSR structures (`CSR_MSUBM_Type`, `CSR_MMISCCTRL_Type`, `CSR_MILMCTL_Type`, `CSR_MECC_CTL_Type`, `CSR_MDLMCTL_Type`, `CSR_MMISC_CTL1_Type`), `rv_counter_t` type for n100, and fence barrier before CSR reads
- Update `core_feature_eclic.h` with ECLICv2 support (shadow register APIs, S-mode pending control APIs, `SAVE_SSUBM_VAR/RESTORE_SSUBM_VAR` macros) and rename `__TEE_PRESENT` to `__SMODE_PRESENT`
- Update `core_feature_timer.h` with n100 support, **HDBG** bit position fix (4 to 3), **MTIME_SRC** bit position, and rename `__TEE_PRESENT` to `__SMODE_PRESENT`
- Update `core_feature_pmp.h` to support up to 64 PMP entries (changed from 16)
- Update `core_feature_fpu.h` with FCSR save/restore in `SAVE_FPU_CONTEXT` and `RESTORE_FPU_CONTEXT` macros
- Update `core_feature_cache.h` with Cluster Cache coherency APIs and SMPCC support
- Update `core_feature_dsp.h` to fix variable shadowing (`result` to `__res`)
- Update `core_feature_pma.h` with proper PMA region setup sequence and fence

³⁸ <https://doc.nucleisys.com/nmsis/dxygen/dsp/html/deprecated.html>

³⁹ <https://doc.nucleisys.com/nmsis/dxygen/dsp/html/deprecated.html>

⁴⁰ <https://github.com/ARM-software/CMSIS-DSP/commit/4404d3e65bd3>

- Update `riscv_encoding.h` with ECLICv2 CSR bit definitions (MSUBM/SSUBM, CSR_PUSHXSUBM, CSR_MCFG_INFO feature flags)
- Update `nmsis_bench.h` for n100 support and correct L2 cache event definitions
- Add `core_feature_smpcc.h` to support SMP and Cluster Cache related operations
- Add `core_feature_ecc.h` to support ECC related operations
- Add `core_feature_iinfo.h` to support IREGION INFO related operations

- **NMSIS-DSP**

- Add missing functions `riscv_fir_decimate_f64`, `riscv_fir_decimate_init_f64` and `riscv_mat_init_f64`
- Add a new test framework for DSP library validation, see `NMSIS/DSP/Testing/`
- Fix `riscv_abs_q15` which can't handle -32768 when using the P extension
- Fix `riscv_cmplx_conj` and `riscv_quaternion_conjugate_f32` that mixed up the element count with the complex/quaternion number count.
- Fix `riscv_correlate` producing incorrect results when `len(srcB) < len(srcA)` with the Vector extension
- Fix `riscv_absmin_no_idx` to use the correct `blkCnt`
- Fix `riscv_fir_decimate_q15` using the wrong variable
- Fix `riscv_mat_mult_q15` to use the correct rows of matrix
- Fix `riscv_mat_solve_upper_triangular_f64` to use the correct `bstride`
- Fix `riscv_conv_partial_f32` to use the correct `vblkCnt`
- Fix `riscv_quaternion_conjugate_f32` to use the correct element count
- Fix `riscv_cfft_radix4_q31.c` producing incorrect results
- Fix `riscv_power_q31` output scaling, which should be in Q2.48 format
- Fix `riscv_cfft_q15` using the wrong variable
- Fix `riscv_scale_q15` functional error when optimized with rv64 P extension
- Add `riscv_vec_math.h` and `riscv_vec_math_f16.h` providing f16/f32 RVV-accelerated math functions (exp, log, sin, cos, pow, tanh, etc.)
- Add multiple library build configurations for Nuclei n100 and n200 CPU
- Synchronize with the CMSIS-DSP v1.17.0 release, incorporating the following updates:
 - * Increase sine table precision from 9 to 10 decimal places in `riscv_common_tables.c`.
 - * Add `riscv_transform_buffer_sizes.c` to get the buffer sizes needed for transform functions.
 - * Introduce new macros: `RISCV_MATH_AUTOVECTORIZE`, `RISCV_MFCC_USE_CFFT` and `RISCV_DSP_TABLE_ATTRIBUTE`.
 - * Remove redundant test in `riscv_atan2_q15`.
 - * Fix an overflow risk of `riscv_div_int64_to_int32` and add result saturation.
 - * Miscellaneous minor modifications, mostly code formatting, which do not affect the functionality but can ease future maintenance.
- Add RVV optimization for CFFT/RCFFT functions, and also change the function signature when RVV extension is present.

- DCT4 functions are no longer supported when RVV extension is present due to the function signature of RFFT has changed.
- Add new test suite TransformRVVC Q15/Q31/F16/F32 for new RVV CFFT functions and skip the old incompatible tests.
- **NMSIS-NN**
 - Add multiple library build configurations for Nuclei n100 and n200 CPU
- **Build System**
 - Enable different toolchain support to build DSP and NN library, you can pass **TOOLCHAIN=nuclei_llvm** or **TOOLCHAIN=terapines** to build for different toolchain when generate library
 - Gitlab CI introduced support for build library for **nuclei_gnu**, **nuclei_llvm** and **terapines** toolchain
- **Tools**
 - Add `pyparsing` pip packages required by DSP Testing framework

5.3 V1.4.1

This is the version of V1.4.1 release.

- **NMSIS-Core**
 - Replace `__ASSEMBLY__` with `__ASSEMBLER__` to match with gcc predefined asm macro
 - Optimize `__STATIC_FORCEINLINE` to `__STATIC_INLINE` in NMSIS Core header files to avoid unnecessary inline to save code size

5.4 V1.4.0

This is the version of V1.4.0 release.

- **NMSIS-Core**
 - Fix wrong macro `PLIC_GetThreshold` & `PLIC_GetThreshold_S` implementation for `core_feature_plic.h`
 - Add `MTIME_SRW_CTRL` bitfields in **SysTimer_Type** structure for `core_feature_timer.h`
 - Optimize ECLIC API for better code performance in `core_feature_eclic.h`
 - Add SSTC support in `core_feature_timer.h`, a new macro called `__SSTC_PRESENT` is added
 - Update and add more CSR Union types
 - Add more CSR macros such `shartid` csr, `worldguard` csrs, and related csr bitfield macro
 - Add the `BENCH_XLEN_MODE` macro to enable more accurate cycle and HPM counter measurements for **RV32**, when `BENCH_XLEN_MODE` is enabled, the cycle/instret/time/hpm_counter will be 32 bits for rv32 and 64 bits for rv64.
 - Fix return type error of `__get_hpm_counter`
 - Add new APIs to read cycle/instret/time/hpm_counter with XLEN bits:
 - * `unsigned long __read_cycle_csr()`
 - * `unsigned long __read_instret_csr()`

- * unsigned long __read_time_csr()
- * unsigned long __read_hpm_counter(unsigned long idx)
- Fix __clear_core_irq_pending and __clear_core_irq_pending_s implementation in core_feature_base.h
- Fix __enable_sw_irq_s implementation in core_feature_base.h
- Add PMA(Physical Memory Attribute) APIs for managing attribute type(Device/Non-Cacheable/Cacheable) of memory regions when **__PMA_PRESENT=1**
- Fix and update HPM v1 event name macros in nmsis_bench.h, especially EVENT_INSTRUCTION_COMMIT_JAL_PREDICTION_FAIL not existed in hardware due to document error
- Add new PMU v1 and v2 event macros in nmsis_bench.h
- Add flushpipe and fence in each ccm operation api in core_feature_cache.h
- **NMSIS-NN**
 - Sync with CMSIS-NN v7.0.0
 - Add new testcases for NN library testing
 - Add script to generate reflib by renaming symbols
 - Fix data overflow when rhs_offset=128 in riscv_nn_vec_mat_mult_t_s8
 - Add new test framework for NN library testing, see NMSIS/NN/Tests/UnitTest/
 - Update prebuilt NN Library and example IAR workbench projects and remove NN reference library IAR project
- **NMSIS-DSP**
 - Fix undefined blkCnt when LOOPUNROLL=OFF
 - Update prebuilt DSP Library IAR workbench projects
- **Documentation**
 - Update Core/DSP/NN documentation to match latest update.
 - Device templates are directly copied from [nuclei sdk evalsoc](https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc)⁴¹
 - Device templates and its documentation will be deprecated, please refer to *nuclei sdk evalsoc* implementation

5.5 V1.3.1

This is the version of V1.3.1 release.

- **NMSIS-Core**
 - Fix Cache CCM related API compile fail using c++ compiler
 - **mfp16mode** csr is renamed to **mmisc_ctl1** due to hw changes
- **NMSIS-NN**
 - Update get started guide for build NMSIS-NN library

⁴¹ <https://github.com/Nuclei-Software/nuclei-sdk/tree/master/SoC/evalsoc/Common>

- Fix testcases in NN/Tests/Cases/full: `riscv_elementwise_mul_acc_s16` and `riscv_elementwise_mul_s16_batch_offset`
- Sync cmake file changes
- Update IAR prebuilt NN library and example projects

- **NMSIS-DSP**

- Optimize fredsum, use less fredsum and more fmacc for rvv implementation
- Sync cmake file changes
- Update IAR prebuilt DSP library and example projects

5.6 V1.3.0

This is the version of V1.3.0 release.

- **NMSIS-Core**

- Add more ECC related macros for `milm_ctl/mdlm_ctl/mcache_ctl` csr
- Add plic related API support in `core_feature_plic.h`
- Fix `core_feature_cidu.h` which wrongly included `math.h` which is not required

- **Build System**

- Change `TOOLCHAIN` to `LIBTYPE` in `Makefile.build` and `Makefile.nmsis` located in `NMSIS/Scripts/Build/`

- **NMSIS-DSP**

- Sync with CMSIS-DSP Library v1.16.2
- Add initial version of NMSIS DSP Benchmark
- Fix bug found for ComplexMathFunctions when `RISCV_MATH_LOOPUNROLL` not defined

- **NMSIS-NN**

- Sync with CMSIS-NN library v6.0.0
- Add initial version of NMSIS NN benchmark
- Roll back p-ext optimized version for function `riscv_nn_vec_mat_mult_t_svd_s8`

- **Nuclei AI Library**

- Introduced RVV optimized AI operators implementation which can be found in <https://github.com/Nuclei-Software/nuclei-ai-library>

5.7 V1.2.1

This is the version V1.2.1 release.

- **NMSIS-Core**

- Fix and update many CSR type define structure defined in `core_feature_base.h`
- Fix protection type error in PMP/sPMP `__set_PMPENTRYx/__set_sPMPENTRYx` API
- Add SMPU support in `core_feature_smp.h`
- Fix wrong CLIC macros defined in `core_feature_eclic.h`
- Update RISC-V DSP intrinsic API and its comments

- **NMSIS-DSP**

- Sync with CMSIS-DSP library(CMSIS-DSP v1.15.0, commit id:8f8bb8d5)
- Add a few Nuclei DSP N2 intrinsic APIs in `core_feature_dsp.h`, they are: `__RV_DPACK32`, `__RV_DSUNPKD810`, `__RV_DSUNPKD820`, `__RV_DSUNPKD830`, `__RV_DSUNPKD831`, `__RV_DSUNPKD832`, `__RV_DZUNPKD810`, `__RV_DZUNPKD820`, `__RV_DZUNPKD830`, `__RV_DZUNPKD831`, `__RV_DZUNPKD832`. And change `__RV_DPKBB32` to `__RV_DPACK32` for better performance
- Unified instruction naming style in `core_feature_dsp.h`, such as: change `DSMMULU` to `DSMMUL.u`, change `DKWMMULU` to `DKWMMUL.u`, change `DKMMACU` to `DKMMAC.u`, change `DKMMSBU` to `DKMMSB.u`, change `DSMAQASU` to `DSMAQA.SU`, change `DDSMAQASU` to `DDSMAQA.SU`
- Optimize the rvv implementation of some MatMult and Complex MatMult functions
- Fixed some doc erros in `core_feature_dsp.h`
- Fixed `riscv_lms_q31.c` rvv bug

- **NMSIS-NN**

- Sync with CMSIS-NN library(CMSIS-NN v5.0.0, commit id:bfc54edb)
- Optimize some functions with Nuclei DSP N3 instruction(such as: `riscv_convolve_s8`, `riscv_fully_connected_s8`, etc.)
- Optimize the rvv implementation of fullyconnect s8 function
- Fixed `riscv_nn_softmax_common_s8.c` rvv bug

- **CI**

- Nuclei SDK use 0.6.0 release version, SOC still use evalsoc.

- **Documentation**

- Update sphinx and doxygen document version to 1.2.1

5.8 V1.2.0

This is the version V1.2.0 release.

Note

- This 1.2.0 version will no longer support old gcc 10 version, and it now only support Nuclei Toolchain 2023.10 (gcc13 and clang17) or later. The major changes that can be felt are as follows:
- The prefix of toolchain has changed from `riscv-nuclei-elf-` to `riscv64-unknown-elf-`
- The `-march` option has changed a lot, see <https://github.com/riscv-non-isa/riscv-toolchain-conventions/pull/26>, for examples:
 - `b` extension changed to `_zba_zbb_zbc_zbs` extension,
 - `p` extension changed to `_xxldsp`, `_xxldspn1x`, `_xxldspn2x`, `_xxldspn3x` extensions which means standard DSP extension, Nuclei N1, N2, N3 DSP extensions
 - `v` extension changed to `v`, `_zve32f`, `_zve64f` extensions

These extensions also can be combined in a certain order, please be cautious

- The name of Libraries has changed with `-march`, for examples, the library named `libnmsis_dsp_rv32imacb.a` is now named `libnmsis_dsp_rv32imac_zba_zbb_zbc_zbs.a` since `b` extension changed to `_zba_zbb_zbc_zbs`
- NMSIS v1.2.0 should be used with Nuclei SDK v0.5.0 or later
- RVV intrinsic APIs is update to v0.12.0, please visit [rvv-intrinsic-doc](https://github.com/riscv-non-isa/rvv-intrinsic-doc)⁴²

• NMSIS-Core

- Add more Nuclei DSP N1/N2/N3 intrinsic APIs and fix some intrinsic API definition and descriptions in `core_feature_dsp.h`
- Add basic IAR support for NMSIS Core header files and device template, for sample usage, see Nuclei SDK 0.5.0 release
- Fix missing break in `__set_hpm_event` function API in `core_feature_base.h`, which affected the `nmsis_bench.h`
- Use IAR custom instruction and IAR P-ext 0.5.0 support to support Nuclei DSP extension based on P-ext 0.5.4, see changes made in `core_feature_dsp.h`, only `Xxldsp` is supported, no N1/N2/N3 supported, and some instructions can't be supported using custom instruction, but in future, we will cooperate with IAR to do full Nuclei DSP support
- Add more CSRs definition according to Nuclei ISA updates such as `Zc/stack check`
- No more bitmanip extension intrinsic header `<rvintrin.h>` for gcc13
- Fix `__RV_CLAMP` macro and add `__MACHINE/SUPERVISOR/USER_INTERRUPT` macros
- Add `__get_hart_index` and `SysTimer_GetHartID` and modify `__get_hart_id` API
- In `<Device.h>`, we introduced `__HARTID_OFFSET` and `__SYSTIMER_HARTID` macro to represent timer hart index relation with cpu hartid for AMP SoC
- Clean compiler warning of NMSIS-Core header files
- Fix Cache CCM API missing return value in some case

⁴² <https://github.com/riscv-non-isa/rvv-intrinsic-doc/releases/tag/v0.12.0>

- **NMSIS-DSP**

- Defined `NUCLEI_DSP_DEFAULT`, `NUCLEI_DSP_N1`, `NUCLEI_DSP_N2`, `NUCLEI_DSP_N3` in `riscv_math_types.h` according to gcc options. This means that if compile with `--march=rv32imafc_xxldspn1x`, the `NUCLEI_DSP_N1` will defined, if compile with `--march=rv32imafc_xxldspn2x`, the `NUCLEI_DSP_N1` and `NUCLEI_DSP_N2` will defined, and so on
- Optimize some functions with DSP N1/N2/N3 (such as `FilteringFunctions`, `TransformFunctions`, `Complex-MathFunctions`)
- RVV intrinsic APIs is update to v0.12.0
- Add f16 support(include f16 rvv extension support)
- Fix the use of `expd80` instruction(Nuclei default dsp instruction)
- Fix some testcases bugs(such as `MatrixFunctions`, `TransformFunctions`)

- **NMSIS-NN**

- Defined `NUCLEI_DSP_DEFAULT`, `NUCLEI_DSP_N1`, `NUCLEI_DSP_N2`, `NUCLEI_DSP_N3` in `riscv_nn_math_types.h` according to gcc options
- RVV intrinsic APIs is update to v0.12.0
- Clean code(nnref lib)

- **Build System**

- Toolchain change to gcc13, The prefix of toolchain has changed to `riscv64-unknown-elf-`, old gcc10 `riscv-nuclei-elf-gcc` changed to `gcc 13 riscv64-unknown-elf-gcc`
- Add ci configurations to support different instruction combinations, please check `Scripts/Build/nmsis_dsp.json` and `Scripts/Build/nmsis_nn.json`
- Library naming scheme changed due to march changes, which means the library name will not be compatible with previous release, check dsp/nn get started guide for details
- F16 library build is supported now when `zfh/zvfh` extension enabled

- **CI**

- Change NMSIS to use Nuclei SDK `evalsoc` as ci run target, `demosoc` is removed in 0.5.0 Nuclei SDK release.
- Spilt DSP and NN test jobs to reduce ci running time
- Build DSP/NN library in one job now, since N1/N2/N3 library naming are different, and library build speed for risc-v vector increased now

5.9 V1.1.1

This is the version V1.1.1 release.

- **NMSIS-Core**

- Add CIDU support via `core_feature_cidu.h`, and `__CIDU_PRESENT` macro is required in `<Device>.h` to represent CIDU present or not
- Add macros of HPM m/s/u event enable, events type, events idx
- Fix define error of `HPM_INIT` macro

- Update systimer/pmp/spmp/eclic API comment and implementation
- Add Cache ECC related APIs
- Due to mhartid csr update, two new API added called `__get_hart_id` and `__get_cluster_id`
 - * mhartid in Nuclei RISC-V processor are now used to present cluster id and hart id
 - * bit 0-7 is used for hart id in current cluster
 - * bit 8-15 is used for cluster id of current cluster
- **NMSIS-DSP**
 - Sync with CMSIS-DSP library(branch:main, commit id:1d9e38a, after CMSIS-DSP v1.14.4)
 - Optimize some functions with RVV(such as: ComplexMathFunctions, FilteringFunctions, MatrixFunctions, StatisticsFunctions, etc.)
 - Some bugfix(riscv_mat_inverse_f32.c rvv fix, riscv_offset_q15.c p fix, riscv_fir_q15.c rvv fix etc.)
- **NMSIS-NN**
 - Sync with CMSIS-NN library(branch:main, commit id:61d1bb6, CMSIS-NN v4.1.0)
 - Compile independent, no longer depend on NMSIS-DSP
 - Optimize some functions with RVV(such as: ActivationFunctions, FullyConnectedFunctions, PoolingFunctions, etc.)
 - Some bugfix that make tflite-micro test successfully
- **Documentation**
 - Update sphinx and doxygen document version to 1.1.1
 - Use mathjax to render latex formulas instead of latex, which can avoid strange compile error
 - Change dsp/nn sphinx rst document structure to match dsp/nn doxygen documentation update
- **NPK**
 - Add `nmsis_dsp_nn` choice for `nmsislibsel`, it will select nmsis dsp and nn library
 - `nmsis_nn` library no longer select dsp library, since now it can live without dsp library

5.10 V1.1.0

This is the version V1.1.0 release of Nuclei MCU Software Interface Standard(NMSIS).

- **NMSIS-Core**
 - Add `nmsis_bench.h` for benchmark and hpm helper functions.
 - Add hpm related API
 - Update `riscv_encoding.h` for latest riscv changes.
 - Add `core_feature_spmp.h` for TEE/sPMP unit.
 - Add more Nuclei DSP N1/N2/N3 intrinsic APIs in `core_feature_dsp.h`
 - Bring SMP/AMP support in `core_feature_eclic.h` and `core_feature_timer.h`
- **NMSIS-DSP**
 - Sync with DSP library in CMSIS 5.9.0 release.

- Add experimental RV32 Vector support.
- Optimize with RVP/RVV for DSP library.
- **NMSIS-NN**
 - Sync with NN library in CMSIS 5.9.0 release.
 - Add experimental RV32 Vector support.
 - Optimize with RVP/RVV for NN library.
- **Build System**
 - **DSP64** is removed, and replaced by **NUCLEI_DSP_N1**, which means Nuclei DSP N1 extension present.
 - **NUCLEI_DSP_N2** and **NUCLEI_DSP_N3** are introduced to standard for Nuclei DSP N2/N3 extension present.
 - Now you build different DSP/NN library optimized Nuclei DSP N1/N2/N3 via command such as `make NUCLEI_DSP=N1 gen`
 - Add `nmsis_help` make target to show help message to build `nmsis dsp/nn` library.
 - Add `check_build` and `check_run` make target for locally build or run on a small test suite configuration.
 - Add fpga related test script located in `Scripts/Configs/fpga/`.
 - Fix bugs found in `nlbuild.py` script.
- **Device Templates**
 - Update Device templates to support SMP/AMP and new linker script changes to align with Nuclei SDK 0.4.0
- **CI**
 - Misc changes for github and gitlab ci, see commit history
 - gitlab ci will now test `NUCLEI_DSP=N0/N1/N2/N3` cases and also check rv32 with VPU for DSP/NN test cases
- **Documentation**
 - Update Core/DSP/NN documentation
- **Misc**
 - Nuclei SDK 0.4.0 will use NMSIS 1.1.0

5.11 V1.0.4

This is the version V1.0.4 release of Nuclei MCU Software Interface Standard(NMSIS).

- **NMSIS-Core**
 - add `__CCM_PRESENT` macro in NMSIS-Core, if CCM hardware unit is present in your CPU, `__CCM_PRESENT` macro need to be set to 1 in `<Device>.h`
 - Fixed `mtvec` related api comment in `core_feature_ecllic.h`
 - Add safely write `mtime/mtimecmp` register for 32bit risc-v processor
 - rearrange `#include` header files for all NMSIS Core header files
 - removed some not good `#pragma gcc` diagnostic lines in `nmsis_gcc.h`

- **NMSIS-DSP**
 - Add initial bitmainp extension support
 - Fix bug in riscv_cmplx_mult_cmplx_q15 function when XLEN=64
- **NMSIS-NN**
 - Add initial bitmainp extension support
 - Change riscv_maxpool_q7_HWC implementation for rvv
 - Re-org NN_Lib_Tests to Tests
- **Build System**
 - Change minimal version of cmake to 3.14
 - Add REBUILD=0 to reuse previous generated Makefile
- **Device Templates**
 - Fix bss section lma and vma not aligned and tbss space not reserved
- **CI**
 - Change NMSIS to use Nuclei SDK demosoc as ci run target
 - only run ci on master/develop branch
- **Documentation**
 - Update get started guide for dsp/nn library

5.12 V1.0.3

This is the official release version V1.0.3 release of Nuclei MCU Software Interface Standard(NMSIS).

This release is only supported by Nuclei GNU Toolchain 2022.01 and its later version, since it required intrinsic header files in RISC-V GCC for B/P/V extensions.

The following changes has been made since V1.0.2.

- **Documentation**
 - Update NMSIS Core/DSP/NN related documentation
- **Device Templates**
 - Add __INC_INTRINSIC_API, __BITMANIP_PRESENT and __VECTOR_PRESENT in <Device>.h
 - Add more REG/ADDR/BIT access macros in <Device>.h
 - Update linker script for <Device>.ld for Nuclei C Runtime Library
 - Add tp register initialization and add early exception setup during startup in startup_<Device>.S
 - Adding support for Nuclei C Runtime library
- **NMSIS-Core**
 - Update core_feature_ecllic.h, core_feature_timer.h and core_feature_dsp.h
 - Added core_feature_vector.h and core_feature_bitmainp.h
 - Add more nuclei customized csr in riscv_encoding.h
 - Include rvb/rvp/rvv header files when __INC_INTRINSIC_API = 1

- **NMSIS-DSP/NN**

- Add support for Nuclei GNU Toolchain 2021.12
- Add new build system to generate NMSIS DSP and NN library
- Update cmake files for both DSP and NN library
- No need to define `__RISCV_FEATURE_DSP` and `__RISCV_FEATURE_VECTOR` when using DSP or NN library, it will be defined in `riscv_math_types.h` via the predefined macros in Nuclei RISC-V gcc 10.2
- Rename `RISCV_VECTOR` to `RISCV_MATH_VECTOR`
- Fix `FLEN` and `XLEN` mis-usage in library

5.13 V1.0.2

This is the official release version V1.0.2 release of Nuclei MCU Software Interface Standard(NMSIS).

The following changes has been made since V1.0.1.

- **Documentation**

- Update NMSIS Core/DSP/NN related documentation

- **Device Templates**

- `DOWNLOAD_MODE_XXX` macros are removed from `riscv_encoding.h`, it is now defined as enum in `<Device.h>`, and can be customized by soc vendor.
- startup code now don't rely on `DOWNLOAD_MODE` macro, instead it now rely on a new macro called `VECTOR_TABLE_REMAPPED`, when `VECTOR_TABLE_REMAPPED` is defined, it means the vector table's lma != vma, such as vector table need to be copied from flash to ilm when boot up
- Add more customized csr of Nuclei RISC-V Core
- Add **BIT**, **BITS**, **REG**, **ADDR** related macros in `<Device.h>`

- **NMSIS-Core**

- Nuclei Cache CCM operation APIs are now introduced in `core_feature_cache.h`
- Update NMSIS-Core header files

- **NMSIS-DSP/NN**

- Merged the official CMSIS 5.8.0 release, CMSIS-DSP 1.9.0, CMSIS-NN 3.0.0
- RISC-V Vector extension and P-extension support for DSP/NN libraries are added

5.14 V1.0.2-RC2

This is the release candidate version V1.0.2-RC2 release of Nuclei MCU Software Interface Standard(NMSIS).

The following changes has been made since V1.0.2-RC1.

- **Documentation**

- Update NMSIS Core/DSP/NN related documentation

5.15 V1.0.2-RC1

This is the release candidate version V1.0.2-RC1 release of Nuclei MCU Software Interface Standard(NMSIS).

The following changes has been made since V1.0.1.

- **Device Templates**
 - DOWNLOAD_MODE_XXX macros are removed from riscv_encoding.h, it is now defined as enum in <Device.h>, and can be customized by soc vendor.
 - startup code now don't rely on DOWNLOAD_MODE macro, instead it now rely on a new macro called VECTOR_TABLE_REMAPPED, when VECTOR_TABLE_REMAPPED is defined, it means the vector table's lma != vma, such as vector table need to be copied from flash to ilm when boot up
 - Add **BIT**, **BITS**, **REG**, **ADDR** related macros in <Device.h>
- **NMSIS-Core**
 - Nuclei Cache CCM operation APIs are now introduced in core_feature_cache.h
- **NMSIS-DSP/NN**
 - Merged the official CMSIS 5.8.0 release, CMSIS-DSP 1.9.0, CMSIS-NN 3.0.0
 - RISC-V Vector extension and P-extension support for DSP/NN libraries are added

5.16 V1.0.1

This is the official V1.0.1 release of Nuclei MCU Software Interface Standard(NMSIS).

The following changes has been maded since V1.0.1-RC1.

- **Device Templates**
 - I/D Cache enable assemble code in startup_<Device>.S are removed now
 - Cache control updates in System_<Device>.c
 - * I-Cache will be enabled if __ICACHE_PRESENT = 1 defined in <Device.h>
 - * D-Cache will be enabled if __DCACHE_PRESENT = 1 defined in <Device.h>

5.17 V1.0.1-RC1

This is release candidate version V1.0.1-RC1 of NMSIS.

- **NMSIS-Core**
 - Add RISC-V DSP 64bit intrinsic functions in core_feature_dsp.h
 - Add more CSR definitions in riscv_encoding.h
 - Update arm compatible functions for RISC-V dsp instruction cases in core_compatible.h
- **NMSIS-DSP**
 - Optimize RISC-V 32bit DSP library implementation
 - Add support for Nuclei RISC-V 64bit DSP SIMD instruction for DSP library
 - Add test cases used for DSP library testing, mainly for internal usage

- Change the examples and tests to use Nuclei SDK as running environment
- **NMSIS-NN**
 - Add support for Nuclei RISC-V 64bit DSP SIMD instruction for NN library
 - Change the examples and tests to use Nuclei SDK as running environment
- **Device Templates**
 - Add DDR `DOWNLOAD_MODE` in device templates
 - Modifications to `startup_<Device>.S` files
 - * `_premain_init` is added to replace `_init`
 - * `_postmain_fini` is added to replace `_fini`
 - If you have implemented your init or de-init functions through `_init` or `_fini`, please use `_premain_init` and `_postmain_fini` functions defined `system_<Device>.c` now

5.18 V1.0.0-beta1

Main changes in release **V1.0.0-beta1**.

- **NMSIS-Core**
 - Fix `SysTick_Reload` implementation
 - Update `ECLIC_Register_IRQ` implementation to allow handler == NULL
 - Fix MTH offset from 0x8 to 0xB, this will affect function of `ECLIC_GetMth` and `ECLIC_SetMth`
 - Fix wrong macro check in cache function
 - Add missing `SOC_INT_MAX` enum definition in Device template
 - In `System_<Device>.c`, `ECLIC_NLBits` set to `__ECLIC_INTCTLBITS`, which means all the bits are for level, no bits for priority

5.19 V1.0.0-beta

Main changes in release **V1.0.0-beta**.

- **NMSIS-Core**
 - Fix error typedef of `CSR_MCAUSE_Type`
 - Change `CSR_MCACHE_CTL_DE` to future value `0x00010000`
 - Fix names in CSR naming, `CSR_SCRATCHCSW` -> `CSR_MSCRATCHCSW`, and `CSR_SCRATCHCSWL` -> `CSR_MSCRATCHCSWL`
 - Add macros in `riscv_encoding.h`: `MSTATUS_FS_INITIAL`, `MSTATUS_FS_CLEAN`, `MSTATUS_FS_DIRTY`
- **Documentation**
 - Fix an typo in `core_template_intexc.rst`
 - Add cross references of Nuclei ISA Spec
 - Update appendix
 - Refines tables and figures

5.20 V1.0.0-alpha.1

API changes has been made to system timer.

- Start from Nuclei N core version 1.4, MSTOP register is renamed to MTIMECTL to provide more features
- Changes made to NMSIS/Core/core_feature_timer.h
 - MSTOP register name changed to MTIMECTL due to core spec changes
 - SysTimer_SetMstopValue renamed to SysTimer_SetControlValue
 - SysTimer_GetMstopValue renamed to SysTimer_GetControlValue
 - Add SysTimer_Start and SysTimer_Stop to start or stop system timer counter
 - SysTick_Reload function is introduced to reload system timer
 - Macro names started with SysTimer_xxx are changed, please check in the code.
- Removed unused lines of code in DSP and NN library source code which has unused macros which will not work for RISC-V cores.
- Fix some documentation issues, mainly typos and invalid cross references.

5.21 V1.0.0-alpha

This is the V1.0.0-alpha release of Nuclei MCU Software Interface Standard(NMSIS).

In this release, we have release three main components:

- **NMSIS-Core:** Standardized API for the Nuclei processor core and peripherals.
- **NMSIS-DSP:** DSP library collection optimized for the Nuclei Processors which has RISC-V SIMD instruction set.
- **NMSIS-NN:** Efficient neural network library developed to maximize the performance and minimize the memory footprint Nuclei Processors which has RISC-V SIMD instruction set.

We also released totally new [Nuclei-SDK](https://github.com/Nuclei-Software/nuclei-sdk)⁴³ which is an SDK implementation based on the **NMSIS-Core** for Nuclei N/NX evaluation cores running on HummingBird Evaluation Kit.

⁴³ <https://github.com/Nuclei-Software/nuclei-sdk>

GLOSSARY**API**

(Application Program Interface) A defined set of routines and protocols for building application software.

DSP

(Digital Signal Processing) is the use of digital processing, such as by computers or more specialized digital signal processors, to perform a wide variety of signal processing operations.

ISR

(Interrupt Service Routine) Also known as an interrupt handler, an ISR is a callback function whose execution is triggered by a hardware interrupt (or software interrupt instructions) and is used to handle high-priority conditions that require interrupting the current code executing on the processor.

NN

(Neural Network) is a network or circuit of neurons, or in a modern sense, an artificial neural network, composed of artificial neurons or nodes.

XIP

(eXecute In Place) a method of executing programs directly from long term storage rather than copying it into RAM, saving writable memory for dynamic data and not the static program code.

APPENDIX

- **Nuclei Tools and Documents:** <https://nucleisys.com/download.php>
- **Nuclei Software Opensource Organization:** <https://github.com/Nuclei-Software>
- **RISC-V MCU Opensource Organization:** <https://github.com/riscv-mcu>
- **Nuclei Toolchain Repo:** <https://github.com/riscv-mcu/riscv-gnu-toolchain>
- **Nuclei OpenOCD Repo:** <https://github.com/riscv-mcu/riscv-openocd>
- **Nuclei QEMU Repo:** <https://github.com/riscv-mcu/qemu>
- **Nuclei SDK:** <https://github.com/Nuclei-Software/nuclei-sdk>
- **NMSIS:** <https://github.com/Nuclei-Software/NMSIS>
- **Nuclei AI Library:** <https://github.com/Nuclei-Software/nuclei-ai-library>
- **Nuclei RISC-V IP Products:** <https://www.nucleisys.com/product.php>
- **Nuclei Tools Documentation:** https://doc.nucleisys.com/nuclei_tools
- **Nuclei Studio Supply Documents:** <https://github.com/Nuclei-Software/nuclei-studio>
- **RISC-V MCU Community Website:** <https://www.riscv-mcu.com/>
- **Nuclei RISC-V CPU Spec:** https://doc.nucleisys.com/nuclei_spec
- **RISC-V ISA Specifications(Ratified):** <https://riscv.org/technical/specifications>
- **RISC-V ISA Specification(Latest):** <https://github.com/riscv/riscv-isa-manual/releases>
- **RISC-V Architecture Profiles:** <https://github.com/riscv/riscv-profiles>
- **RISC-V Bitmanip(B) Extension Spec:** <https://github.com/riscv/riscv-bitmanip>
- **RISC-V Packed SIMD(P) Extension Spec:** <https://github.com/riscv/riscv-p-spec>
- **RISC-V Cryptography(K) Extension Spec:** <https://github.com/riscv/riscv-crypto>
- **RISC-V Vector(V) Extension Spec:** <https://github.com/riscv/riscv-v-spec>
- **RISC-V Vector Intrinsic API Spec:** <https://github.com/riscv-non-isa/rvv-intrinsic-doc>
- **RISC-V ISA Extension Spec Status:** <https://wiki.riscv.org/display/HOME/Specification+Status>
- **Nuclei Bumblebee Core Document:** https://github.com/nucleisys/Bumblebee_Core_Doc

INDICES AND TABLES

- `genindex`
- `search`

Symbols

__FLD2VAL (*C macro*), 611, 612
 __VAL2FLD (*C macro*), 611
 __ALIGNED (*C macro*), 74
 __ASM (*C macro*), 73
 __CLZ (*C macro*), 820
 __COMPILER_BARRIER (*C macro*), 74
 __CPU_RELAX (*C macro*), 77
 __DMB (*C macro*), 819
 __DSB (*C macro*), 819
 __ECLIC_VER (*C macro*), 164
 __EXPD_BYTE (*C macro*), 820
 __FENCE (*C macro*), 76
 __I (*C macro*), 611
 __IM (*C macro*), 611
 __INLINE (*C macro*), 73
 __INTERRUPT (*C macro*), 74
 __IO (*C macro*), 611
 __IOM (*C macro*), 611
 __ISB (*C macro*), 819
 __LDRBT (*C macro*), 819
 __LDRHT (*C macro*), 819
 __LDRT (*C macro*), 819
 __MACHINE_INTERRUPT (*C macro*), 74
 __NMSIS_VERSION (*C macro*), 72
 __NMSIS_VERSION_MAJOR (*C macro*), 72
 __NMSIS_VERSION_MINOR (*C macro*), 72
 __NMSIS_VERSION_PATCH (*C macro*), 72
 __NN_PACKq7 (*C macro*), 1398, 1399
 __NN_PKHBT (*C macro*), 1398, 1399
 __NN_PKHTB (*C macro*), 1398, 1399
 __NO_RETURN (*C macro*), 73
 __NUCLEI_CPU_REV (*C macro*), 71
 __NUCLEI_CPU_SERIES (*C macro*), 71
 __NUCLEI_NX_REV (*C macro*), 71
 __NUCLEI_N_REV (*C macro*), 71
 __O (*C macro*), 611
 __OM (*C macro*), 611
 __PACKED (*C macro*), 73
 __PACKED_STRUCT (*C macro*), 73
 __PACKED_UNION (*C macro*), 73
 __RARELY (*C macro*), 74
 __RBIT (*C macro*), 820
 __RESTRICT (*C macro*), 74
 __RISCV_FLEN (*C macro*), 690
 __RISCV_XLEN (*C macro*), 135
 __RMB (*C macro*), 76
 __RV_BITREVI (*C macro*), 337, 338
 __RV_CSR_CLEAR (*C macro*), 76
 __RV_CSR_READ (*C macro*), 75
 __RV_CSR_READ_CLEAR (*C macro*), 76
 __RV_CSR_READ_SET (*C macro*), 75
 __RV_CSR_SET (*C macro*), 76
 __RV_CSR_SWAP (*C macro*), 75
 __RV_CSR_WRITE (*C macro*), 75
 __RV_DSCLIP16 (*C macro*), 519, 520
 __RV_DSCLIP32 (*C macro*), 519, 520
 __RV_DSCLIP8 (*C macro*), 519
 __RV_FLD (*C macro*), 692
 __RV_FLOAD (*C macro*), 693
 __RV_FLW (*C macro*), 691
 __RV_FSD (*C macro*), 692
 __RV_FSTORE (*C macro*), 693
 __RV_FSW (*C macro*), 691
 __RV_INSB (*C macro*), 337, 338
 __RV_KSLLI16 (*C macro*), 242, 243
 __RV_KSLLI32 (*C macro*), 458
 __RV_KSLLI8 (*C macro*), 255
 __RV_KSLLIW (*C macro*), 317
 __RV_SCLIP16 (*C macro*), 289, 290
 __RV_SCLIP32 (*C macro*), 393, 394
 __RV_SCLIP8 (*C macro*), 296, 297
 __RV_SLLI16 (*C macro*), 242, 243
 __RV_SLLI32 (*C macro*), 458, 459
 __RV_SLLI8 (*C macro*), 255, 256
 __RV_SRAI16 (*C macro*), 242, 244
 __RV_SRAI16_U (*C macro*), 242, 245
 __RV_SRAI32 (*C macro*), 458, 459
 __RV_SRAI32_U (*C macro*), 458, 460
 __RV_SRAI8 (*C macro*), 255, 256
 __RV_SRAI8_U (*C macro*), 255, 257
 __RV_SRAIW_U (*C macro*), 499
 __RV_SRAI_U (*C macro*), 337, 339
 __RV_SRLI16 (*C macro*), 242, 246

__RV_SRLI16_U (C macro), 242, 247
 __RV_SRLI32 (C macro), 458, 461
 __RV_SRLI32_U (C macro), 458, 462
 __RV_SRLI8 (C macro), 255, 258
 __RV_SRLI8_U (C macro), 255, 259
 __RV_UCLIP16 (C macro), 289, 290
 __RV_UCLIP32 (C macro), 393, 394
 __RV_UCLIP8 (C macro), 296, 297
 __RV_WEXTI (C macro), 337, 340
 __RWMB (C macro), 76
 __SMP_RMB (C macro), 77
 __SMP_RWMB (C macro), 77
 __SMP_WMB (C macro), 77
 __SSAT (C macro), 819
 __STATIC_FORCEINLINE (C macro), 73
 __STATIC_INLINE (C macro), 73
 __STRBT (C macro), 819
 __STRHT (C macro), 819
 __STRT (C macro), 819
 __SUPERVISOR_INTERRUPT (C macro), 74
 __UNALIGNED_UINT16_READ (C macro), 73
 __UNALIGNED_UINT16_WRITE (C macro), 73
 __UNALIGNED_UINT32_READ (C macro), 73
 __UNALIGNED_UINT32_WRITE (C macro), 73
 __USAT (C macro), 819
 __USED (C macro), 73
 __USER_INTERRUPT (C macro), 74
 __USUALLY (C macro), 74
 __VECTOR_SIZE (C macro), 73
 __WEAK (C macro), 73
 __WMB (C macro), 77
 __disable_FPU (C macro), 691
 __enable_FPU (C macro), 691
 __get_FCSR (C macro), 690
 __get_FFLAGS (C macro), 691
 __get_FRM (C macro), 690
 __get_SMPUADDRx (C macro), 701
 __get_SMPUCFGx (C macro), 701
 __get_SMPUENTRYx (C macro), 701, 702
 __get_SMPUxCFG (C macro), 701
 __has_builtin (C macro), 73
 __set_FCSR (C macro), 690
 __set_FFLAGS (C macro), 691
 __set_FRM (C macro), 691
 __set_SMPUADDRx (C macro), 701, 702
 __set_SMPUCFGx (C macro), 701
 __set_SMPUENTRYx (C macro), 701, 702
 __set_SMPUxCFG (C macro), 701
 __sync_harts (C++ function), 809
 _fini (C++ function), 810
 _init (C++ function), 810
 _postmain_fini (C++ function), 809
 _premain_init (C++ function), 809

A

API, 1453

B

BENCH_DECLARE_VAR (C macro), 822
 BENCH_END (C macro), 822
 BENCH_ERROR (C macro), 823
 BENCH_GET_LPCNT (C macro), 823
 BENCH_GET_SUMCYC (C macro), 823
 BENCH_GET_USECYC (C macro), 823
 BENCH_INIT (C macro), 822
 BENCH_RESET (C macro), 822
 BENCH_SAMPLE (C macro), 822
 BENCH_START (C macro), 822
 BENCH_STAT (C macro), 823
 BENCH_STATUS (C macro), 823
 BENCH_STOP (C macro), 822
 BREAKPOINT (C macro), 133

C

CacheInfo_Type (C++ struct), 713, 716
 CAUSE_BREAKPOINT (C macro), 132
 CAUSE_FAULT_FETCH (C macro), 132
 CAUSE_FAULT_LOAD (C macro), 132
 CAUSE_FAULT_STORE (C macro), 132
 CAUSE_FETCH_PAGE_FAULT (C macro), 133
 CAUSE_HYPERVISOR_ECALL (C macro), 133
 CAUSE_ILLEGAL_INSTRUCTION (C macro), 132
 CAUSE_LOAD_PAGE_FAULT (C macro), 133
 CAUSE_MACHINE_ECALL (C macro), 133
 CAUSE_MISALIGNED_FETCH (C macro), 132
 CAUSE_MISALIGNED_LOAD (C macro), 132
 CAUSE_MISALIGNED_STORE (C macro), 132
 CAUSE_STORE_PAGE_FAULT (C macro), 133
 CAUSE_SUPERVISOR_ECALL (C macro), 132
 CAUSE_USER_ECALL (C macro), 132
 CC_CFG_Type (C++ union), 181
 CC_CFG_Type::b (C++ member), 181
 CC_CFG_Type::w (C++ member), 181
 CC_CMD_Type (C++ union), 184
 CC_CMD_Type::b (C++ member), 184
 CC_CMD_Type::w (C++ member), 184
 CC_CTRL_Type (C++ union), 182
 CC_CTRL_Type::b (C++ member), 183
 CC_CTRL_Type::w (C++ member), 183
 CC_ECC_INJ_WAY_Type (C++ union), 194
 CC_ECC_INJ_WAY_Type::b (C++ member), 194
 CC_ECC_INJ_WAY_Type::w (C++ member), 194
 CC_ERR_INJ_Type (C++ union), 184
 CC_ERR_INJ_Type::b (C++ member), 185
 CC_ERR_INJ_Type::w (C++ member), 185
 CC_FATAL_CNT_Type (C++ union), 186
 CC_FATAL_CNT_Type::b (C++ member), 186

CC_FATAL_CNT_Type: :w (C++ member), 186
 CC_FATAL_THV_Type (C++ union), 186
 CC_FATAL_THV_Type: :b (C++ member), 187
 CC_FATAL_THV_Type: :w (C++ member), 187
 CC_INV_RANGE_Type (C++ union), 194
 CC_INV_RANGE_Type: :b (C++ member), 194
 CC_INV_RANGE_Type: :w (C++ member), 194
 CC_INVALID_ALL_Type (C++ union), 189
 CC_INVALID_ALL_Type: :b (C++ member), 189
 CC_INVALID_ALL_Type: :w (C++ member), 189
 CC_RECV_CNT_Type (C++ union), 185
 CC_RECV_CNT_Type: :b (C++ member), 185
 CC_RECV_CNT_Type: :w (C++ member), 186
 CC_RECV_THV_Type (C++ union), 186
 CC_RECV_THV_Type: :b (C++ member), 186
 CC_RECV_THV_Type: :w (C++ member), 186
 CCM_CMD_Type (C++ enum), 712, 714
 CCM_CMD_Type: :CCM_CC_LOCK (C++ enumerator), 713, 715
 CCM_CMD_Type: :CCM_CC_UNLOCK (C++ enumerator), 713, 715
 CCM_CMD_Type: :CCM_DC_INVALID (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_INVALID_ALL (C++ enumerator), 712, 715
 CCM_CMD_Type: :CCM_DC_LOCK (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_UNLOCK (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_WB (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_WB_ALL (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_WBINVALID (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_DC_WBINVALID_ALL (C++ enumerator), 712, 714
 CCM_CMD_Type: :CCM_IC_INVALID (C++ enumerator), 712, 715
 CCM_CMD_Type: :CCM_IC_INVALID_ALL (C++ enumerator), 713, 715
 CCM_CMD_Type: :CCM_IC_LOCK (C++ enumerator), 712, 715
 CCM_CMD_Type: :CCM_IC_UNLOCK (C++ enumerator), 713, 715
 CCM_COMMAND_COMMAND (C macro), 127
 CCM_DATA_DATA (C macro), 127
 CCM_OP_FINFO_Type (C++ enum), 712, 714
 CCM_OP_FINFO_Type: :CCM_OP_ECC_ERR (C++ enumerator), 712, 714
 CCM_OP_FINFO_Type: :CCM_OP_EXCEED_ERR (C++ enumerator), 712, 714
 CCM_OP_FINFO_Type: :CCM_OP_PERM_CHECK_ERR (C++ enumerator), 712, 714
 CCM_OP_FINFO_Type: :CCM_OP_REFILL_BUS_ERR (C++ enumerator), 712, 714
 CCM_OP_FINFO_Type: :CCM_OP_SUCCESS (C++ enumerator), 712, 714
 CCM_SUEN_SUEN (C macro), 127
 CCM_SUEN_SUEN_Msk (C macro), 713
 CFFTINIT_F16 (C macro), 1163, 1164
 CFFTINIT_F32 (C macro), 1167
 CFFTINIT_F64 (C macro), 1171
 CFFTINIT_Q15 (C macro), 1174
 CFFTINIT_Q31 (C macro), 1178
 CIDU_BASE (C macro), 680, 681
 CIDU_CORE_INT_STATUS_ADDR (C macro), 680, 682
 CIDU_CORE_INT_STATUS_OFS (C macro), 680, 681
 CIDU_CORE_NUM_ADDR (C macro), 680, 682
 CIDU_CORE_NUM_OFS (C macro), 680, 681
 CIDU_ICI_SEND_CORE_ID_POS (C macro), 681, 682
 CIDU_ICI_SHADOW_ADDR (C macro), 680, 682
 CIDU_ICI_SHADOW_OFS (C macro), 680, 681
 CIDU_INT_INDICATOR_ADDR (C macro), 680, 682
 CIDU_INT_INDICATOR_OFS (C macro), 680, 681
 CIDU_INT_MASK_ADDR (C macro), 680, 682
 CIDU_INT_MASK_OFS (C macro), 680, 681
 CIDU_INT_NUM_ADDR (C macro), 681, 682
 CIDU_INT_NUM_OFS (C macro), 680, 681
 CIDU_RECEIVE_INTERRUPT_EN (C macro), 680, 681
 CIDU_SEMAPHORE_ADDR (C macro), 680, 682
 CIDU_SEMAPHORE_OFS (C macro), 680, 681
 CLIC_CLICCFG_NLBIT_Msk (C macro), 163
 CLIC_CLICCFG_NLBIT_Pos (C macro), 163
 CLIC_CLICINFO_CTLBIT_Msk (C macro), 163
 CLIC_CLICINFO_CTLBIT_Pos (C macro), 163
 CLIC_CLICINFO_NUM_Msk (C macro), 163
 CLIC_CLICINFO_NUM_Pos (C macro), 163
 CLIC_CLICINFO_SHD_NUM_Msk (C macro), 163
 CLIC_CLICINFO_SHD_NUM_Pos (C macro), 163
 CLIC_CLICINFO_VER_Msk (C macro), 163
 CLIC_CLICINFO_VER_Pos (C macro), 163
 CLIC_CTRL_Type (C++ struct), 166
 CLIC_INTATTR_MODE_Msk (C macro), 164
 CLIC_INTATTR_MODE_Pos (C macro), 164
 CLIC_INTATTR_SHV_Msk (C macro), 164
 CLIC_INTATTR_SHV_Pos (C macro), 164
 CLIC_INTATTR_TRIG_Msk (C macro), 164
 CLIC_INTATTR_TRIG_Pos (C macro), 164
 CLIC_INTIE_IE_Msk (C macro), 164
 CLIC_INTIE_IE_Pos (C macro), 164
 CLIC_INTIP_IP_Msk (C macro), 164
 CLIC_INTIP_IP_Pos (C macro), 164
 CLIC_Type (C++ struct), 166
 CLICCFG_Type (C++ union), 165
 CLICCFG_Type: :b (C++ member), 165
 CLICCFG_Type: :w (C++ member), 165
 CLICINFO_Type (C++ union), 166

CLICINFO_Type: :b (C++ member), 166
 CLIENT_ERR_STATUS_Type (C++ union), 187
 CLIENT_ERR_STATUS_Type: :b (C++ member), 187
 CLIENT_ERR_STATUS_Type: :w (C++ member), 187
 CLIENT_WAY_MASK_Type (C++ union), 193
 CLIENT_WAY_MASK_Type: :b (C++ member), 194
 CLIENT_WAY_MASK_Type: :w (C++ member), 194
 CLINT_Interrupt_Init (C++ function), 806
 CLINT_MSIP (C macro), 805
 CLM_ADDR_BASE_Type (C++ union), 188
 CLM_ADDR_BASE_Type: :clm32 (C++ member), 189
 CLM_ADDR_BASE_Type: :clm_base64 (C++ member), 189
 CLM_WAY_EN_Type (C++ union), 189
 CLM_WAY_EN_Type: :b (C++ member), 189
 CLM_WAY_EN_Type: :w (C++ member), 189
 core_exception_handler (C++ function), 812
 core_exception_handler_s (C++ function), 815
 core_interrupt_handler (C++ function), 811
 core_interrupt_handler_s (C++ function), 813
 Core_Register_IRQ (C++ function), 807
 Core_Register_IRQ_S (C++ function), 807
 cos_factors_128 (C++ member), 1196, 1197
 cos_factors_2048 (C++ member), 1196, 1197
 cos_factors_512 (C++ member), 1196, 1197
 cos_factors_8192 (C++ member), 1196, 1198
 cos_factorsQ31_128 (C++ member), 1196, 1198
 cos_factorsQ31_2048 (C++ member), 1196, 1199
 cos_factorsQ31_512 (C++ member), 1196, 1199
 cos_factorsQ31_8192 (C++ member), 1196, 1199
 CpuIRegionBase (C++ member), 810
 CSR_CCM_FPIPE (C macro), 112
 CSR_CCM_MBEGINADDR (C macro), 112
 CSR_CCM_MCOMMAND (C macro), 112
 CSR_CCM_MDATA (C macro), 112
 CSR_CCM_SBEGINADDR (C macro), 112
 CSR_CCM_SCOMMAND (C macro), 112
 CSR_CCM_SDATA (C macro), 112
 CSR_CCM_SUEN (C macro), 112
 CSR_CCM_UBEGINADDR (C macro), 112
 CSR_CCM_UCOMMAND (C macro), 112
 CSR_CCM_UDATA (C macro), 112
 CSR_CYCLE (C macro), 86
 CSR_CYCLEH (C macro), 100
 CSR_DCSR (C macro), 97
 CSR_DILM_CTL_Type (C++ type), 136
 CSR_DPC (C macro), 97
 CSR_DSCRATCH0 (C macro), 97
 CSR_DSCRATCH1 (C macro), 97
 CSR_FCSR (C macro), 86
 CSR_FFLAGS (C macro), 86
 CSR_FRM (C macro), 86
 CSR_HCONTEXT (C macro), 90
 CSR_HCOUNTEREN (C macro), 90
 CSR_HEDELEG (C macro), 90
 CSR_HENVCFG (C macro), 90
 CSR_HENVCFGH (C macro), 100
 CSR_HGATP (C macro), 90
 CSR_HGEIE (C macro), 90
 CSR_HGEIP (C macro), 90
 CSR_HIDELEG (C macro), 90
 CSR_HIE (C macro), 90
 CSR_HIP (C macro), 90
 CSR_HPMCounter10 (C macro), 87
 CSR_HPMCounter10H (C macro), 101
 CSR_HPMCounter11 (C macro), 87
 CSR_HPMCounter11H (C macro), 101
 CSR_HPMCounter12 (C macro), 87
 CSR_HPMCounter12H (C macro), 101
 CSR_HPMCounter13 (C macro), 87
 CSR_HPMCounter13H (C macro), 101
 CSR_HPMCounter14 (C macro), 87
 CSR_HPMCounter14H (C macro), 101
 CSR_HPMCounter15 (C macro), 87
 CSR_HPMCounter15H (C macro), 101
 CSR_HPMCounter16 (C macro), 87
 CSR_HPMCounter16H (C macro), 101
 CSR_HPMCounter17 (C macro), 87
 CSR_HPMCounter17H (C macro), 101
 CSR_HPMCounter18 (C macro), 87
 CSR_HPMCounter18H (C macro), 101
 CSR_HPMCounter19 (C macro), 87
 CSR_HPMCounter19H (C macro), 101
 CSR_HPMCounter20 (C macro), 87
 CSR_HPMCounter20H (C macro), 101
 CSR_HPMCounter21 (C macro), 87
 CSR_HPMCounter21H (C macro), 101
 CSR_HPMCounter22 (C macro), 87
 CSR_HPMCounter22H (C macro), 101
 CSR_HPMCounter23 (C macro), 87
 CSR_HPMCounter23H (C macro), 101
 CSR_HPMCounter24 (C macro), 87
 CSR_HPMCounter24H (C macro), 101
 CSR_HPMCounter25 (C macro), 87
 CSR_HPMCounter25H (C macro), 101
 CSR_HPMCounter26 (C macro), 87
 CSR_HPMCounter26H (C macro), 101
 CSR_HPMCounter27 (C macro), 87
 CSR_HPMCounter27H (C macro), 102
 CSR_HPMCounter28 (C macro), 87
 CSR_HPMCounter28H (C macro), 102
 CSR_HPMCounter29 (C macro), 88
 CSR_HPMCounter29H (C macro), 102
 CSR_HPMCounter3 (C macro), 86
 CSR_HPMCounter30 (C macro), 88
 CSR_HPMCounter30H (C macro), 102
 CSR_HPMCounter31 (C macro), 88
 CSR_HPMCounter31H (C macro), 102

CSR_HPMCounter3H (*C macro*), 100
 CSR_HPMCounter4 (*C macro*), 86
 CSR_HPMCounter4H (*C macro*), 100
 CSR_HPMCounter5 (*C macro*), 86
 CSR_HPMCounter5H (*C macro*), 100
 CSR_HPMCounter6 (*C macro*), 86
 CSR_HPMCounter6H (*C macro*), 101
 CSR_HPMCounter7 (*C macro*), 86
 CSR_HPMCounter7H (*C macro*), 101
 CSR_HPMCounter8 (*C macro*), 87
 CSR_HPMCounter8H (*C macro*), 101
 CSR_HPMCounter9 (*C macro*), 87
 CSR_HPMCounter9H (*C macro*), 101
 CSR_HSTATEEN0 (*C macro*), 90
 CSR_HSTATEEN0H (*C macro*), 100
 CSR_HSTATEEN1 (*C macro*), 90
 CSR_HSTATEEN1H (*C macro*), 100
 CSR_HSTATEEN2 (*C macro*), 90
 CSR_HSTATEEN2H (*C macro*), 100
 CSR_HSTATEEN3 (*C macro*), 90
 CSR_HSTATEEN3H (*C macro*), 100
 CSR_HSTATUS (*C macro*), 89
 CSR_HTIMEDELTA (*C macro*), 90
 CSR_HTIMEDELTAH (*C macro*), 100
 CSR_HTINST (*C macro*), 90
 CSR_HTVAL (*C macro*), 90
 CSR_HVIP (*C macro*), 90
 CSR_INSTRET (*C macro*), 86
 CSR_INSTRETH (*C macro*), 100
 CSR_IRQCIE (*C macro*), 113
 CSR_IRQCIP (*C macro*), 113
 CSR_JALMNXTI (*C macro*), 108
 CSR_JALSNXTI (*C macro*), 111
 CSR_JALXNXTI (*C macro*), 135
 CSR_JVT (*C macro*), 86
 CSR_LSTEPFORC (*C macro*), 108
 CSR_MARCHID (*C macro*), 100
 CSR_MATTRIO_BASE (*C macro*), 109
 CSR_MATTRIO_MASK (*C macro*), 109
 CSR_MATTRI1_BASE (*C macro*), 109
 CSR_MATTRI1_MASK (*C macro*), 109
 CSR_MATTRI2_BASE (*C macro*), 109
 CSR_MATTRI2_MASK (*C macro*), 109
 CSR_MATTRI3_BASE (*C macro*), 109
 CSR_MATTRI3_MASK (*C macro*), 109
 CSR_MATTRI4_BASE (*C macro*), 109
 CSR_MATTRI4_MASK (*C macro*), 109
 CSR_MATTRI5_BASE (*C macro*), 109
 CSR_MATTRI5_MASK (*C macro*), 109
 CSR_MATTRI6_BASE (*C macro*), 109
 CSR_MATTRI6_MASK (*C macro*), 109
 CSR_MATTRI7_BASE (*C macro*), 110
 CSR_MATTRI7_MASK (*C macro*), 110
 CSR_MBADADDR (*C macro*), 92

CSR_MCACHE_CTL (*C macro*), 107
 CSR_MCACHE_CTL_DE (*C macro*), 121
 CSR_MCACHE_CTL_IE (*C macro*), 121
 CSR_MCACHE_CTL_Type (*C++ type*), 136
 CSR_MCACHECTL_Type (*C++ union*), 147
 CSR_MCACHECTL_Type::_reserved0 (*C++ member*), 147
 CSR_MCACHECTL_Type::_reserved1 (*C++ member*), 148
 CSR_MCACHECTL_Type::b (*C++ member*), 148
 CSR_MCACHECTL_Type::d (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_burst_type (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_ecc_chk_en (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_ecc_en (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_ecc_excp_en (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_en (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_prefetch_en (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_rwdecc (*C++ member*), 148
 CSR_MCACHECTL_Type::dc_rwtecc (*C++ member*), 148
 CSR_MCACHECTL_Type::ic_burst_type (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_cancel_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_ecc_chk_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_ecc_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_ecc_excp_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_pf_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_prefetch_en (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_rwdecc (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_rwtecc (*C++ member*), 147
 CSR_MCACHECTL_Type::ic_scpd_mod (*C++ member*), 147
 CSR_MCAUSE (*C macro*), 92
 CSR_MCAUSE_Type (*C++ union*), 142
 CSR_MCAUSE_Type::_reserved0 (*C++ member*), 142
 CSR_MCAUSE_Type::_reserved1 (*C++ member*), 142
 CSR_MCAUSE_Type::b (*C++ member*), 142
 CSR_MCAUSE_Type::d (*C++ member*), 143
 CSR_MCAUSE_Type::exccode (*C++ member*), 142
 CSR_MCAUSE_Type::interrupt (*C++ member*), 142

CSR_MCAUSE_Type::minhv (C++ member), 142
 CSR_MCAUSE_Type::mpie (C++ member), 142
 CSR_MCAUSE_Type::mpil (C++ member), 142
 CSR_MCAUSE_Type::mpp (C++ member), 142
 CSR_MCFG_INFO (C macro), 111
 CSR_MCFG_INFO_Type (C++ type), 136
 CSR_MCFGINFO_Type (C++ union), 151
 CSR_MCFGINFO_Type::_reserved1 (C++ member), 153
 CSR_MCFGINFO_Type::b (C++ member), 153
 CSR_MCFGINFO_Type::clic (C++ member), 152
 CSR_MCFGINFO_Type::d (C++ member), 153
 CSR_MCFGINFO_Type::dcache (C++ member), 152
 CSR_MCFGINFO_Type::dlm (C++ member), 152
 CSR_MCFGINFO_Type::dsp_n1 (C++ member), 152
 CSR_MCFGINFO_Type::dsp_n2 (C++ member), 152
 CSR_MCFGINFO_Type::dsp_n3 (C++ member), 152
 CSR_MCFGINFO_Type::ecc (C++ member), 152
 CSR_MCFGINFO_Type::etrace (C++ member), 153
 CSR_MCFGINFO_Type::fio (C++ member), 152
 CSR_MCFGINFO_Type::icache (C++ member), 152
 CSR_MCFGINFO_Type::ilm (C++ member), 152
 CSR_MCFGINFO_Type::iregion (C++ member), 153
 CSR_MCFGINFO_Type::nice (C++ member), 152
 CSR_MCFGINFO_Type::plic (C++ member), 152
 CSR_MCFGINFO_Type::ppi (C++ member), 152
 CSR_MCFGINFO_Type::safety_mecha (C++ member), 153
 CSR_MCFGINFO_Type::sec_mode (C++ member), 153
 CSR_MCFGINFO_Type::smp (C++ member), 152
 CSR_MCFGINFO_Type::sstc (C++ member), 153
 CSR_MCFGINFO_Type::tee (C++ member), 152
 CSR_MCFGINFO_Type::vnice (C++ member), 153
 CSR_MCFGINFO_Type::vpu_degree (C++ member), 153
 CSR_MCFGINFO_Type::xlcz (C++ member), 153
 CSR_MCFGINFO_Type::zc_xlcz (C++ member), 152
 CSR_MCFGINFO_Type::zilsd (C++ member), 153
 CSR_MCLICBASE (C macro), 107
 CSR_MCONFIGPTR (C macro), 100
 CSR_MCONTEXT (C macro), 97
 CSR_MCOUNTEREN (C macro), 92
 CSR_MCOUNTINHIBIT (C macro), 92
 CSR_MCOUNTINHIBIT_Type (C++ union), 143
 CSR_MCOUNTINHIBIT_Type::_reserved0 (C++ member), 143
 CSR_MCOUNTINHIBIT_Type::_reserved1 (C++ member), 143
 CSR_MCOUNTINHIBIT_Type::b (C++ member), 143
 CSR_MCOUNTINHIBIT_Type::cy (C++ member), 143
 CSR_MCOUNTINHIBIT_Type::d (C++ member), 143
 CSR_MCOUNTINHIBIT_Type::ir (C++ member), 143
 CSR_MCYCLE (C macro), 97
 CSR_MCYCLEH (C macro), 103
 CSR_MDCAUSE (C macro), 107
 CSR_MDCAUSE_Type (C++ union), 144
 CSR_MDCAUSE_Type::_reserved0 (C++ member), 144
 CSR_MDCAUSE_Type::b (C++ member), 144
 CSR_MDCAUSE_Type::d (C++ member), 144
 CSR_MDCAUSE_Type::mdcause (C++ member), 144
 CSR_MDCFG_INFO (C macro), 111
 CSR_MDCFG_INFO_Type (C++ type), 136
 CSR_MDCFGINFO_Type (C++ union), 154
 CSR_MDCFGINFO_Type::_reserved0 (C++ member), 155
 CSR_MDCFGINFO_Type::_reserved1 (C++ member), 155
 CSR_MDCFGINFO_Type::b (C++ member), 155
 CSR_MDCFGINFO_Type::d (C++ member), 155
 CSR_MDCFGINFO_Type::ecc (C++ member), 155
 CSR_MDCFGINFO_Type::lm_ecc (C++ member), 155
 CSR_MDCFGINFO_Type::lm_size (C++ member), 155
 CSR_MDCFGINFO_Type::lsize (C++ member), 155
 CSR_MDCFGINFO_Type::set (C++ member), 155
 CSR_MDCFGINFO_Type::way (C++ member), 155
 CSR_MDEVB (C macro), 109
 CSR_MDEVM (C macro), 109
 CSR_MDLM_CTL (C macro), 107
 CSR_MDLMCTL_Type (C++ union), 150
 CSR_MDLMCTL_Type::_reserved0 (C++ member), 151
 CSR_MDLMCTL_Type::b (C++ member), 151
 CSR_MDLMCTL_Type::d (C++ member), 151
 CSR_MDLMCTL_Type::dis_lsu_dlm (C++ member), 151
 CSR_MDLMCTL_Type::dlm_bpa (C++ member), 151
 CSR_MDLMCTL_Type::dlm_ecc_chk_en (C++ member), 151
 CSR_MDLMCTL_Type::dlm_ecc_en (C++ member), 151
 CSR_MDLMCTL_Type::dlm_ecc_excp_en (C++ member), 151
 CSR_MDLMCTL_Type::dlm_en (C++ member), 151
 CSR_MDLMCTL_Type::dlm_rwecc (C++ member), 151
 CSR_MDLMCTL_Type::dlm_va_en (C++ member), 151
 CSR_MECC_CODE (C macro), 107
 CSR_MECC_CODE_Type (C++ type), 136
 CSR_MECC_CTL (C macro), 111
 CSR_MECC_CTL_Type (C++ union), 159
 CSR_MECC_CTL_Type::_reserved0 (C++ member), 160
 CSR_MECC_CTL_Type::b (C++ member), 160
 CSR_MECC_CTL_Type::d (C++ member), 160
 CSR_MECC_CTL_Type::dc_acc_msk (C++ member), 159
 CSR_MECC_CTL_Type::dc_ccm_msk (C++ member), 159
 CSR_MECC_CTL_Type::dc_cpbk_msk (C++ member), 160

CSR_MECC_CTL_Type::d1m_acc_msk (C++ member), 159
 CSR_MECC_CTL_Type::d1m_ext_msk (C++ member), 159
 CSR_MECC_CTL_Type::ic_ccm_msk (C++ member), 159
 CSR_MECC_CTL_Type::ic_fch_msk (C++ member), 159
 CSR_MECC_CTL_Type::ilm_acc_msk (C++ member), 159
 CSR_MECC_CTL_Type::ilm_ext_msk (C++ member), 159
 CSR_MECC_CTL_Type::ilm_fch_msk (C++ member), 159
 CSR_MECC_CTL_Type::io_prot_chk_en (C++ member), 160
 CSR_MECC_LOCK (C macro), 108
 CSR_MECC_LOCK_Type (C++ type), 136
 CSR_MECC_STATUS (C macro), 111
 CSR_MECC_STATUS_Type (C++ union), 160
 CSR_MECC_STATUS_Type::_reserved0 (C++ member), 161
 CSR_MECC_STATUS_Type::b (C++ member), 161
 CSR_MECC_STATUS_Type::d (C++ member), 161
 CSR_MECC_STATUS_Type::dc_acc_err (C++ member), 160
 CSR_MECC_STATUS_Type::dc_ccm_err (C++ member), 161
 CSR_MECC_STATUS_Type::dc_cpbk_err (C++ member), 161
 CSR_MECC_STATUS_Type::d1m_acc_err (C++ member), 160
 CSR_MECC_STATUS_Type::d1m_ext_err (C++ member), 160
 CSR_MECC_STATUS_Type::ic_ccm_err (C++ member), 160
 CSR_MECC_STATUS_Type::ic_fch_err (C++ member), 160
 CSR_MECC_STATUS_Type::ilm_acc_err (C++ member), 160
 CSR_MECC_STATUS_Type::ilm_ext_err (C++ member), 160
 CSR_MECC_STATUS_Type::ilm_fch_err (C++ member), 160
 CSR_MECCCODE_Type (C++ union), 158
 CSR_MECCCODE_Type::_reserved0 (C++ member), 158
 CSR_MECCCODE_Type::_reserved1 (C++ member), 158
 CSR_MECCCODE_Type::_reserved2 (C++ member), 159
 CSR_MECCCODE_Type::b (C++ member), 159
 CSR_MECCCODE_Type::code (C++ member), 158
 CSR_MECCCODE_Type::d (C++ member), 159
 CSR_MECCCODE_Type::ecc_inj_mode (C++ member), 159
 CSR_MECCCODE_Type::ramid (C++ member), 158
 CSR_MECCCODE_Type::sramid (C++ member), 158
 CSR_MECCLOCK_Type (C++ union), 158
 CSR_MECCLOCK_Type::_reserved0 (C++ member), 158
 CSR_MECCLOCK_Type::b (C++ member), 158
 CSR_MECCLOCK_Type::d (C++ member), 158
 CSR_MECCLOCK_Type::ecc_lock (C++ member), 158
 CSR_MECLIC_CTL (C macro), 108
 CSR_MEDELEG (C macro), 91
 CSR_MENVCFG (C macro), 92
 CSR_MENVCFGH (C macro), 92
 CSR_MEPC (C macro), 92
 CSR_MFIOCFG_INFO (C macro), 108
 CSR_MFIOCFG_INFO_Type (C++ type), 136
 CSR_MFIOCFGINFO_Type (C++ union), 157
 CSR_MFIOCFGINFO_Type::_reserved0 (C++ member), 157
 CSR_MFIOCFGINFO_Type::_reserved1 (C++ member), 157
 CSR_MFIOCFGINFO_Type::b (C++ member), 158
 CSR_MFIOCFGINFO_Type::d (C++ member), 158
 CSR_MFIOCFGINFO_Type::fio_bpa (C++ member), 157
 CSR_MFIOCFGINFO_Type::fio_size (C++ member), 157
 CSR_MFP16MODE (C macro), 108
 CSR_MHARTID (C macro), 100
 CSR_MHPMCOUNTER10 (C macro), 97
 CSR_MHPMCOUNTER10H (C macro), 104
 CSR_MHPMCOUNTER11 (C macro), 97
 CSR_MHPMCOUNTER11H (C macro), 104
 CSR_MHPMCOUNTER12 (C macro), 97
 CSR_MHPMCOUNTER12H (C macro), 104
 CSR_MHPMCOUNTER13 (C macro), 97
 CSR_MHPMCOUNTER13H (C macro), 104
 CSR_MHPMCOUNTER14 (C macro), 97
 CSR_MHPMCOUNTER14H (C macro), 104
 CSR_MHPMCOUNTER15 (C macro), 98
 CSR_MHPMCOUNTER15H (C macro), 104
 CSR_MHPMCOUNTER16 (C macro), 98
 CSR_MHPMCOUNTER16H (C macro), 104
 CSR_MHPMCOUNTER17 (C macro), 98
 CSR_MHPMCOUNTER17H (C macro), 104
 CSR_MHPMCOUNTER18 (C macro), 98
 CSR_MHPMCOUNTER18H (C macro), 104
 CSR_MHPMCOUNTER19 (C macro), 98
 CSR_MHPMCOUNTER19H (C macro), 104
 CSR_MHPMCOUNTER20 (C macro), 98
 CSR_MHPMCOUNTER20H (C macro), 104
 CSR_MHPMCOUNTER21 (C macro), 98
 CSR_MHPMCOUNTER21H (C macro), 104

CSR_MHPMCOUNTER22 (*C macro*), 98
 CSR_MHPMCOUNTER22H (*C macro*), 104
 CSR_MHPMCOUNTER23 (*C macro*), 98
 CSR_MHPMCOUNTER23H (*C macro*), 104
 CSR_MHPMCOUNTER24 (*C macro*), 98
 CSR_MHPMCOUNTER24H (*C macro*), 105
 CSR_MHPMCOUNTER25 (*C macro*), 98
 CSR_MHPMCOUNTER25H (*C macro*), 105
 CSR_MHPMCOUNTER26 (*C macro*), 98
 CSR_MHPMCOUNTER26H (*C macro*), 105
 CSR_MHPMCOUNTER27 (*C macro*), 98
 CSR_MHPMCOUNTER27H (*C macro*), 105
 CSR_MHPMCOUNTER28 (*C macro*), 98
 CSR_MHPMCOUNTER28H (*C macro*), 105
 CSR_MHPMCOUNTER29 (*C macro*), 98
 CSR_MHPMCOUNTER29H (*C macro*), 105
 CSR_MHPMCOUNTER3 (*C macro*), 97
 CSR_MHPMCOUNTER30 (*C macro*), 98
 CSR_MHPMCOUNTER30H (*C macro*), 105
 CSR_MHPMCOUNTER31 (*C macro*), 98
 CSR_MHPMCOUNTER31H (*C macro*), 105
 CSR_MHPMCOUNTER3H (*C macro*), 104
 CSR_MHPMCOUNTER4 (*C macro*), 97
 CSR_MHPMCOUNTER4H (*C macro*), 104
 CSR_MHPMCOUNTER5 (*C macro*), 97
 CSR_MHPMCOUNTER5H (*C macro*), 104
 CSR_MHPMCOUNTER6 (*C macro*), 97
 CSR_MHPMCOUNTER6H (*C macro*), 104
 CSR_MHPMCOUNTER7 (*C macro*), 97
 CSR_MHPMCOUNTER7H (*C macro*), 104
 CSR_MHPMCOUNTER8 (*C macro*), 97
 CSR_MHPMCOUNTER8H (*C macro*), 104
 CSR_MHPMCOUNTER9 (*C macro*), 97
 CSR_MHPMCOUNTER9H (*C macro*), 104
 CSR_MHPMEVENT10 (*C macro*), 99
 CSR_MHPMEVENT10H (*C macro*), 102
 CSR_MHPMEVENT11 (*C macro*), 99
 CSR_MHPMEVENT11H (*C macro*), 102
 CSR_MHPMEVENT12 (*C macro*), 99
 CSR_MHPMEVENT12H (*C macro*), 102
 CSR_MHPMEVENT13 (*C macro*), 99
 CSR_MHPMEVENT13H (*C macro*), 102
 CSR_MHPMEVENT14 (*C macro*), 99
 CSR_MHPMEVENT14H (*C macro*), 103
 CSR_MHPMEVENT15 (*C macro*), 99
 CSR_MHPMEVENT15H (*C macro*), 103
 CSR_MHPMEVENT16 (*C macro*), 99
 CSR_MHPMEVENT16H (*C macro*), 103
 CSR_MHPMEVENT17 (*C macro*), 99
 CSR_MHPMEVENT17H (*C macro*), 103
 CSR_MHPMEVENT18 (*C macro*), 99
 CSR_MHPMEVENT18H (*C macro*), 103
 CSR_MHPMEVENT19 (*C macro*), 99
 CSR_MHPMEVENT19H (*C macro*), 103

CSR_MHPMEVENT20 (*C macro*), 99
 CSR_MHPMEVENT20H (*C macro*), 103
 CSR_MHPMEVENT21 (*C macro*), 99
 CSR_MHPMEVENT21H (*C macro*), 103
 CSR_MHPMEVENT22 (*C macro*), 99
 CSR_MHPMEVENT22H (*C macro*), 103
 CSR_MHPMEVENT23 (*C macro*), 99
 CSR_MHPMEVENT23H (*C macro*), 103
 CSR_MHPMEVENT24 (*C macro*), 99
 CSR_MHPMEVENT24H (*C macro*), 103
 CSR_MHPMEVENT25 (*C macro*), 99
 CSR_MHPMEVENT25H (*C macro*), 103
 CSR_MHPMEVENT26 (*C macro*), 99
 CSR_MHPMEVENT26H (*C macro*), 103
 CSR_MHPMEVENT27 (*C macro*), 99
 CSR_MHPMEVENT27H (*C macro*), 103
 CSR_MHPMEVENT28 (*C macro*), 100
 CSR_MHPMEVENT28H (*C macro*), 103
 CSR_MHPMEVENT29 (*C macro*), 100
 CSR_MHPMEVENT29H (*C macro*), 103
 CSR_MHPMEVENT3 (*C macro*), 98
 CSR_MHPMEVENT30 (*C macro*), 100
 CSR_MHPMEVENT30H (*C macro*), 103
 CSR_MHPMEVENT31 (*C macro*), 100
 CSR_MHPMEVENT31H (*C macro*), 103
 CSR_MHPMEVENT3H (*C macro*), 102
 CSR_MHPMEVENT4 (*C macro*), 98
 CSR_MHPMEVENT4H (*C macro*), 102
 CSR_MHPMEVENT5 (*C macro*), 98
 CSR_MHPMEVENT5H (*C macro*), 102
 CSR_MHPMEVENT6 (*C macro*), 98
 CSR_MHPMEVENT6H (*C macro*), 102
 CSR_MHPMEVENT7 (*C macro*), 99
 CSR_MHPMEVENT7H (*C macro*), 102
 CSR_MHPMEVENT8 (*C macro*), 99
 CSR_MHPMEVENT8H (*C macro*), 102
 CSR_MHPMEVENT9 (*C macro*), 99
 CSR_MHPMEVENT9H (*C macro*), 102
 CSR_MICFG_INFO (*C macro*), 111
 CSR_MICFG_INFO_Type (*C++ type*), 136
 CSR_MICFGINFO_Type (*C++ union*), 153
 CSR_MICFGINFO_Type::_reserved0 (*C++ member*),
 154
 CSR_MICFGINFO_Type::_reserved1 (*C++ member*),
 154
 CSR_MICFGINFO_Type::b (*C++ member*), 154
 CSR_MICFGINFO_Type::d (*C++ member*), 154
 CSR_MICFGINFO_Type::ecc (*C++ member*), 154
 CSR_MICFGINFO_Type::i_share_dlm (*C++ member*),
 154
 CSR_MICFGINFO_Type::lm_ecc (*C++ member*), 154
 CSR_MICFGINFO_Type::lm_size (*C++ member*), 154
 CSR_MICFGINFO_Type::lm_xonly (*C++ member*), 154
 CSR_MICFGINFO_Type::lsize (*C++ member*), 154

CSR_MICFGINFO_Type::set (C++ member), 154
 CSR_MICFGINFO_Type::way (C++ member), 154
 CSR_MIDELEG (C macro), 91
 CSR_MIE (C macro), 92
 CSR_MILM_CTL (C macro), 107
 CSR_MILM_CTL_Type (C++ type), 136
 CSR_MILMCTL_Type (C++ union), 149
 CSR_MILMCTL_Type::_reserved0 (C++ member), 150
 CSR_MILMCTL_Type::b (C++ member), 150
 CSR_MILMCTL_Type::d (C++ member), 150
 CSR_MILMCTL_Type::dis_lsu_ilm (C++ member), 150
 CSR_MILMCTL_Type::ilm_bpa (C++ member), 150
 CSR_MILMCTL_Type::ilm_ecc_chk_en (C++ member), 150
 CSR_MILMCTL_Type::ilm_ecc_en (C++ member), 150
 CSR_MILMCTL_Type::ilm_ecc_excp_en (C++ member), 150
 CSR_MILMCTL_Type::ilm_en (C++ member), 150
 CSR_MILMCTL_Type::ilm_rwecc (C++ member), 150
 CSR_MILMCTL_Type::ilm_va_en (C++ member), 150
 CSR_MIMPID (C macro), 100
 CSR_MINSTRET (C macro), 97
 CSR_MINSTRETH (C macro), 103
 CSR_MINTSTATUS (C macro), 91
 CSR_MIP (C macro), 92
 CSR_MIRGB_INFO (C macro), 110
 CSR_MIRGB_INFO_Type (C++ union), 161
 CSR_MIRGB_INFO_Type::_reserved0 (C++ member), 161
 CSR_MIRGB_INFO_Type::_reserved1 (C++ member), 161
 CSR_MIRGB_INFO_Type::b (C++ member), 161
 CSR_MIRGB_INFO_Type::d (C++ member), 161
 CSR_MIRGB_INFO_Type::iregion_base (C++ member), 161
 CSR_MIRGB_INFO_Type::iregion_size (C++ member), 161
 CSR_MISA (C macro), 91
 CSR_MISA_Type (C++ union), 136
 CSR_MISA_Type::_reserved0 (C++ member), 138
 CSR_MISA_Type::a (C++ member), 137
 CSR_MISA_Type::b (C++ member), 137, 138
 CSR_MISA_Type::c (C++ member), 137
 CSR_MISA_Type::d (C++ member), 137
 CSR_MISA_Type::e (C++ member), 137
 CSR_MISA_Type::f (C++ member), 137
 CSR_MISA_Type::g (C++ member), 137
 CSR_MISA_Type::h (C++ member), 137
 CSR_MISA_Type::i (C++ member), 137
 CSR_MISA_Type::j (C++ member), 137
 CSR_MISA_Type::k (C++ member), 137
 CSR_MISA_Type::l (C++ member), 137
 CSR_MISA_Type::m (C++ member), 137
 CSR_MISA_Type::mxl (C++ member), 138
 CSR_MISA_Type::n (C++ member), 137
 CSR_MISA_Type::o (C++ member), 138
 CSR_MISA_Type::p (C++ member), 138
 CSR_MISA_Type::q (C++ member), 138
 CSR_MISA_Type::r (C++ member), 138
 CSR_MISA_Type::s (C++ member), 138
 CSR_MISA_Type::t (C++ member), 138
 CSR_MISA_Type::u (C++ member), 138
 CSR_MISA_Type::v (C++ member), 138
 CSR_MISA_Type::w (C++ member), 138
 CSR_MISA_Type::x (C++ member), 138
 CSR_MISA_Type::y (C++ member), 138
 CSR_MISA_Type::z (C++ member), 138
 CSR_MLWID (C macro), 112
 CSR_MMACRO_CA_EN (C macro), 109
 CSR_MMACRO_DEV_EN (C macro), 109
 CSR_MMACRO_NOC_EN (C macro), 109
 CSR_MMISC_CTL (C macro), 107
 CSR_MMISC_CTL1 (C macro), 108
 CSR_MMISC_CTL1_Type (C++ union), 146
 CSR_MMISC_CTL1_Type::_reserved0 (C++ member), 146
 CSR_MMISC_CTL1_Type::b (C++ member), 146
 CSR_MMISC_CTL1_Type::d (C++ member), 147
 CSR_MMISC_CTL1_Type::fp16mode (C++ member), 146
 CSR_MMISC_CTL1_Type::rvv_v1_0_cmpt (C++ member), 146
 CSR_MMISC_CTL1_Type::vlm_path_en (C++ member), 146
 CSR_MMISC_CTL1_Type::vlsu_cof_en (C++ member), 146
 CSR_MMISC_CTL1_Type::vlsu_ooo_4k_mode (C++ member), 146
 CSR_MMISC_CTL1_Type::vlsu_ooo_en (C++ member), 146
 CSR_MMISC_CTL1_Type::vlsu_ooo_force_va_4k (C++ member), 146
 CSR_MMISC_CTL_Type (C++ type), 136
 CSR_MMISCCTL_Type (C++ type), 136
 CSR_MMISCCTRL_Type (C++ union), 144
 CSR_MMISCCTRL_Type::_reserved0 (C++ member), 144
 CSR_MMISCCTRL_Type::_reserved1 (C++ member), 144
 CSR_MMISCCTRL_Type::_reserved2 (C++ member), 145
 CSR_MMISCCTRL_Type::_reserved3 (C++ member), 145
 CSR_MMISCCTRL_Type::_reserved4 (C++ member), 145
 CSR_MMISCCTRL_Type::_reserved5 (C++ member), 145

CSR_MMISCCTRL_Type::_reserved6 (C++ member), 146
 CSR_MMISCCTRL_Type::b (C++ member), 146
 CSR_MMISCCTRL_Type::bpu (C++ member), 145
 CSR_MMISCCTRL_Type::core_buserr (C++ member), 145
 CSR_MMISCCTRL_Type::csr_excl_enable (C++ member), 145
 CSR_MMISCCTRL_Type::d (C++ member), 146
 CSR_MMISCCTRL_Type::dbg_sec (C++ member), 145
 CSR_MMISCCTRL_Type::hw_auto_context (C++ member), 146
 CSR_MMISCCTRL_Type::imreturn_en (C++ member), 145
 CSR_MMISCCTRL_Type::ldspec_en (C++ member), 145
 CSR_MMISCCTRL_Type::lsu_allow_diff_en (C++ member), 145
 CSR_MMISCCTRL_Type::misalign (C++ member), 145
 CSR_MMISCCTRL_Type::nmi_cause (C++ member), 145
 CSR_MMISCCTRL_Type::sijump_en (C++ member), 145
 CSR_MMISCCTRL_Type::zclsd_en (C++ member), 144
 CSR_MMISCCTRL_Type::zcmr_zcmp (C++ member), 145
 CSR_MNOCB (C macro), 109
 CSR_MNOCM (C macro), 109
 CSR_MNVEC (C macro), 107
 CSR_MNXTI (C macro), 91
 CSR_MPPICFG_INFO (C macro), 108
 CSR_MPPICFG_INFO_Type (C++ type), 136
 CSR_MPPICFGINFO_Type (C++ union), 157
 CSR_MPPICFGINFO_Type::_reserved0 (C++ member), 157
 CSR_MPPICFGINFO_Type::_reserved1 (C++ member), 157
 CSR_MPPICFGINFO_Type::b (C++ member), 157
 CSR_MPPICFGINFO_Type::d (C++ member), 157
 CSR_MPPICFGINFO_Type::ppi_bpa (C++ member), 157
 CSR_MPPICFGINFO_Type::ppi_en (C++ member), 157
 CSR_MPPICFGINFO_Type::ppi_size (C++ member), 157
 CSR_MSAVECAUSE1 (C macro), 108
 CSR_MSAVECAUSE2 (C macro), 108
 CSR_MSAVEDCAUSE1 (C macro), 108
 CSR_MSAVEDCAUSE2 (C macro), 108
 CSR_MSAVEEPC1 (C macro), 107
 CSR_MSAVEEPC2 (C macro), 108
 CSR_MSAVESTATUS (C macro), 107
 CSR_MSAVESTATUS_Type (C++ union), 148
 CSR_MSAVESTATUS_Type::_reserved0 (C++ member), 149
 CSR_MSAVESTATUS_Type::_reserved1 (C++ member), 149
 CSR_MSAVESTATUS_Type::_reserved2 (C++ member), 149
 CSR_MSAVESTATUS_Type::b (C++ member), 149
 CSR_MSAVESTATUS_Type::mpie1 (C++ member), 149
 CSR_MSAVESTATUS_Type::mpie2 (C++ member), 149
 CSR_MSAVESTATUS_Type::mpp1 (C++ member), 149
 CSR_MSAVESTATUS_Type::mpp2 (C++ member), 149
 CSR_MSAVESTATUS_Type::ptyp1 (C++ member), 149
 CSR_MSAVESTATUS_Type::ptyp2 (C++ member), 149
 CSR_MSAVESTATUS_Type::w (C++ member), 149
 CSR_MSCONTEXT (C macro), 97
 CSR_MSCRATCH (C macro), 92
 CSR_MSCRATCHCSW (C macro), 91
 CSR_MSCRATCHCSWL (C macro), 91
 CSR_MSECCFG (C macro), 96
 CSR_MSECCFGH (C macro), 103
 CSR_MSHADGPRLVL0 (C macro), 108
 CSR_MSHADGPRLVL1 (C macro), 108
 CSR_MSIP (C macro), 113
 CSR_MSMPCFG_INFO (C macro), 110
 CSR_MSTACK_BASE (C macro), 112
 CSR_MSTACK_BOUND (C macro), 112
 CSR_MSTACK_CTL (C macro), 112
 CSR_MSTACK_CTL_Type (C++ union), 161
 CSR_MSTACK_CTL_Type::_reserved0 (C++ member), 162
 CSR_MSTACK_CTL_Type::b (C++ member), 162
 CSR_MSTACK_CTL_Type::d (C++ member), 162
 CSR_MSTACK_CTL_Type::mode (C++ member), 162
 CSR_MSTACK_CTL_Type::ovf_track_en (C++ member), 162
 CSR_MSTACK_CTL_Type::udf_en (C++ member), 162
 CSR_MSTACK_CTRL (C macro), 112
 CSR_MSTATEEN0 (C macro), 92
 CSR_MSTATEEN0H (C macro), 102
 CSR_MSTATEEN1 (C macro), 92
 CSR_MSTATEEN1H (C macro), 102
 CSR_MSTATEEN2 (C macro), 92
 CSR_MSTATEEN2H (C macro), 102
 CSR_MSTATEEN3 (C macro), 92
 CSR_MSTATEEN3H (C macro), 102
 CSR_MSTATUS (C macro), 91
 CSR_MSTATUS_Type (C++ union), 139
 CSR_MSTATUS_Type::_reserved0 (C++ member), 139
 CSR_MSTATUS_Type::_reserved1 (C++ member), 139
 CSR_MSTATUS_Type::_reserved2 (C++ member), 139
 CSR_MSTATUS_Type::_reserved3 (C++ member), 140
 CSR_MSTATUS_Type::b (C++ member), 140
 CSR_MSTATUS_Type::d (C++ member), 140
 CSR_MSTATUS_Type::fs (C++ member), 139
 CSR_MSTATUS_Type::mie (C++ member), 139
 CSR_MSTATUS_Type::mpie (C++ member), 139

CSR_MSTATUS_Type::mvp (C++ member), 139
 CSR_MSTATUS_Type::mprv (C++ member), 140
 CSR_MSTATUS_Type::mxr (C++ member), 140
 CSR_MSTATUS_Type::sd (C++ member), 140
 CSR_MSTATUS_Type::sdt (C++ member), 140
 CSR_MSTATUS_Type::sie (C++ member), 139
 CSR_MSTATUS_Type::spelp (C++ member), 140
 CSR_MSTATUS_Type::spie (C++ member), 139
 CSR_MSTATUS_Type::spp (C++ member), 139
 CSR_MSTATUS_Type::sum (C++ member), 140
 CSR_MSTATUS_Type::tsr (C++ member), 140
 CSR_MSTATUS_Type::tvm (C++ member), 140
 CSR_MSTATUS_Type::tw (C++ member), 140
 CSR_MSTATUS_Type::ube (C++ member), 139
 CSR_MSTATUS_Type::vs (C++ member), 139
 CSR_MSTATUS_Type::xs (C++ member), 139
 CSR_MSTATUSH (C macro), 102
 CSR_MSTATUSH_Type (C++ union), 140
 CSR_MSTATUSH_Type::_reserved0 (C++ member), 141
 CSR_MSTATUSH_Type::_reserved1 (C++ member), 141
 CSR_MSTATUSH_Type::_reserved5 (C++ member), 141
 CSR_MSTATUSH_Type::b (C++ member), 141
 CSR_MSTATUSH_Type::d (C++ member), 141
 CSR_MSTATUSH_Type::gva (C++ member), 141
 CSR_MSTATUSH_Type::mbe (C++ member), 141
 CSR_MSTATUSH_Type::mdt (C++ member), 141
 CSR_MSTATUSH_Type::mpelp (C++ member), 141
 CSR_MSTATUSH_Type::mpv (C++ member), 141
 CSR_MSTATUSH_Type::sbe (C++ member), 141
 CSR_MSTOP (C macro), 113
 CSR_MSUBM (C macro), 107
 CSR_MSUBM_Type (C++ union), 143
 CSR_MSUBM_Type::_reserved0 (C++ member), 143
 CSR_MSUBM_Type::_reserved1 (C++ member), 144
 CSR_MSUBM_Type::b (C++ member), 144
 CSR_MSUBM_Type::d (C++ member), 144
 CSR_MSUBM_Type::gpridx (C++ member), 143
 CSR_MSUBM_Type::pgpridx (C++ member), 144
 CSR_MSUBM_Type::ptyp (C++ member), 143
 CSR_MSUBM_Type::typ (C++ member), 143
 CSR_MTIME (C macro), 113
 CSR_MTIMECMP (C macro), 113
 CSR_MTINST (C macro), 92
 CSR_MTLB_CTL (C macro), 108
 CSR_MTLB_CTL_Type (C++ union), 162
 CSR_MTLB_CTL_Type::_reserved0 (C++ member), 162
 CSR_MTLB_CTL_Type::_reserved1 (C++ member), 162
 CSR_MTLB_CTL_Type::b (C++ member), 163
 CSR_MTLB_CTL_Type::d (C++ member), 163
 CSR_MTLB_CTL_Type::napot_en (C++ member), 162
 CSR_MTLB_CTL_Type::tlb_dram_ecc_inj_en (C++ member), 162
 CSR_MTLB_CTL_Type::tlb_ecc_chk_en (C++ member), 162
 CSR_MTLB_CTL_Type::tlb_ecc_en (C++ member), 162
 CSR_MTLB_CTL_Type::tlb_ecc_excpt_en (C++ member), 162
 CSR_MTLB_CTL_Type::tlb_tram_ecc_inj_en (C++ member), 162
 CSR_MTLBCFG_INFO (C macro), 111
 CSR_MTLBCFG_INFO_Type (C++ type), 136
 CSR_MTLBCFGINFO_Type (C++ union), 155
 CSR_MTLBCFGINFO_Type::_reserved0 (C++ member), 156
 CSR_MTLBCFGINFO_Type::_reserved1 (C++ member), 156
 CSR_MTLBCFGINFO_Type::_reserved2 (C++ member), 156
 CSR_MTLBCFGINFO_Type::b (C++ member), 156
 CSR_MTLBCFGINFO_Type::d (C++ member), 156
 CSR_MTLBCFGINFO_Type::d_size (C++ member), 156
 CSR_MTLBCFGINFO_Type::ecc (C++ member), 156
 CSR_MTLBCFGINFO_Type::i_size (C++ member), 156
 CSR_MTLBCFGINFO_Type::lsize (C++ member), 156
 CSR_MTLBCFGINFO_Type::mapping (C++ member), 156
 CSR_MTLBCFGINFO_Type::napot (C++ member), 156
 CSR_MTLBCFGINFO_Type::nb (C++ member), 156
 CSR_MTLBCFGINFO_Type::set (C++ member), 156
 CSR_MTLBCFGINFO_Type::way (C++ member), 156
 CSR_MTSP (C macro), 108
 CSR_MTSPCSW (C macro), 107
 CSR_MTVAL (C macro), 92
 CSR_MTVAL2 (C macro), 92
 CSR_MTVEC (C macro), 92
 CSR_MTVEC_Type (C++ union), 141
 CSR_MTVEC_Type::addr (C++ member), 142
 CSR_MTVEC_Type::b (C++ member), 142
 CSR_MTVEC_Type::d (C++ member), 142
 CSR_MTVEC_Type::mode (C++ member), 142
 CSR_MTVT (C macro), 91
 CSR_MTVT2 (C macro), 108
 CSR_MVENDORID (C macro), 100
 CSR_MWIDDELEG (C macro), 112
 CSR_PMPADDR0 (C macro), 93
 CSR_PMPADDR1 (C macro), 93
 CSR_PMPADDR10 (C macro), 94
 CSR_PMPADDR11 (C macro), 94
 CSR_PMPADDR12 (C macro), 94
 CSR_PMPADDR13 (C macro), 94
 CSR_PMPADDR14 (C macro), 94
 CSR_PMPADDR15 (C macro), 94

CSR_PMPADDR16 (*C macro*), 94
CSR_PMPADDR17 (*C macro*), 94
CSR_PMPADDR18 (*C macro*), 94
CSR_PMPADDR19 (*C macro*), 94
CSR_PMPADDR2 (*C macro*), 93
CSR_PMPADDR20 (*C macro*), 94
CSR_PMPADDR21 (*C macro*), 94
CSR_PMPADDR22 (*C macro*), 94
CSR_PMPADDR23 (*C macro*), 94
CSR_PMPADDR24 (*C macro*), 94
CSR_PMPADDR25 (*C macro*), 94
CSR_PMPADDR26 (*C macro*), 94
CSR_PMPADDR27 (*C macro*), 94
CSR_PMPADDR28 (*C macro*), 95
CSR_PMPADDR29 (*C macro*), 95
CSR_PMPADDR3 (*C macro*), 93
CSR_PMPADDR30 (*C macro*), 95
CSR_PMPADDR31 (*C macro*), 95
CSR_PMPADDR32 (*C macro*), 95
CSR_PMPADDR33 (*C macro*), 95
CSR_PMPADDR34 (*C macro*), 95
CSR_PMPADDR35 (*C macro*), 95
CSR_PMPADDR36 (*C macro*), 95
CSR_PMPADDR37 (*C macro*), 95
CSR_PMPADDR38 (*C macro*), 95
CSR_PMPADDR39 (*C macro*), 95
CSR_PMPADDR4 (*C macro*), 93
CSR_PMPADDR40 (*C macro*), 95
CSR_PMPADDR41 (*C macro*), 95
CSR_PMPADDR42 (*C macro*), 95
CSR_PMPADDR43 (*C macro*), 95
CSR_PMPADDR44 (*C macro*), 95
CSR_PMPADDR45 (*C macro*), 95
CSR_PMPADDR46 (*C macro*), 95
CSR_PMPADDR47 (*C macro*), 95
CSR_PMPADDR48 (*C macro*), 95
CSR_PMPADDR49 (*C macro*), 96
CSR_PMPADDR5 (*C macro*), 93
CSR_PMPADDR50 (*C macro*), 96
CSR_PMPADDR51 (*C macro*), 96
CSR_PMPADDR52 (*C macro*), 96
CSR_PMPADDR53 (*C macro*), 96
CSR_PMPADDR54 (*C macro*), 96
CSR_PMPADDR55 (*C macro*), 96
CSR_PMPADDR56 (*C macro*), 96
CSR_PMPADDR57 (*C macro*), 96
CSR_PMPADDR58 (*C macro*), 96
CSR_PMPADDR59 (*C macro*), 96
CSR_PMPADDR6 (*C macro*), 93
CSR_PMPADDR60 (*C macro*), 96
CSR_PMPADDR61 (*C macro*), 96
CSR_PMPADDR62 (*C macro*), 96
CSR_PMPADDR63 (*C macro*), 96
CSR_PMPADDR7 (*C macro*), 94
CSR_PMPADDR8 (*C macro*), 94
CSR_PMPADDR9 (*C macro*), 94
CSR_PMPCFG0 (*C macro*), 92
CSR_PMPCFG1 (*C macro*), 92
CSR_PMPCFG10 (*C macro*), 93
CSR_PMPCFG11 (*C macro*), 93
CSR_PMPCFG12 (*C macro*), 93
CSR_PMPCFG13 (*C macro*), 93
CSR_PMPCFG14 (*C macro*), 93
CSR_PMPCFG15 (*C macro*), 93
CSR_PMPCFG2 (*C macro*), 93
CSR_PMPCFG3 (*C macro*), 93
CSR_PMPCFG4 (*C macro*), 93
CSR_PMPCFG5 (*C macro*), 93
CSR_PMPCFG6 (*C macro*), 93
CSR_PMPCFG7 (*C macro*), 93
CSR_PMPCFG8 (*C macro*), 93
CSR_PMPCFG9 (*C macro*), 93
CSR_POPXRET (*C macro*), 111
CSR_PUSHMCAUSE (*C macro*), 108
CSR_PUSHMEPC (*C macro*), 108
CSR_PUSHMSUBM (*C macro*), 108
CSR_PUSHSCAUSE (*C macro*), 111
CSR_PUSHSEPC (*C macro*), 111
CSR_PUSHSSUBM (*C macro*), 111
CSR_PUSHXCAUSE (*C macro*), 135
CSR_PUSHXEPC (*C macro*), 135
CSR_PUSHXSUBM (*C macro*), 135
CSR_SAFETY_CRC_CTL (*C macro*), 112
CSR_SAFETY_STL_STATUS (*C macro*), 112
CSR_SATP (*C macro*), 89
CSR_SATTRI0_BASE (*C macro*), 110
CSR_SATTRI0_MASK (*C macro*), 110
CSR_SATTRI1_BASE (*C macro*), 110
CSR_SATTRI1_MASK (*C macro*), 110
CSR_SATTRI2_BASE (*C macro*), 110
CSR_SATTRI2_MASK (*C macro*), 110
CSR_SATTRI3_BASE (*C macro*), 110
CSR_SATTRI3_MASK (*C macro*), 110
CSR_SATTRI4_BASE (*C macro*), 110
CSR_SATTRI4_MASK (*C macro*), 110
CSR_SATTRI5_BASE (*C macro*), 110
CSR_SATTRI5_MASK (*C macro*), 110
CSR_SATTRI6_BASE (*C macro*), 110
CSR_SATTRI6_MASK (*C macro*), 110
CSR_SATTRI7_BASE (*C macro*), 110
CSR_SATTRI7_MASK (*C macro*), 110
CSR_SCAUSE (*C macro*), 89
CSR_SCONTEXT (*C macro*), 89
CSR_SCOUNTEREN (*C macro*), 88
CSR_SCOUNTOVF (*C macro*), 90
CSR_SDCAUSE (*C macro*), 111
CSR_SECLIC_CTL (*C macro*), 111
CSR_SEDELEG (*C macro*), 88

CSR_SEED (*C macro*), 86
 CSR_SENVCFG (*C macro*), 88
 CSR_SEPC (*C macro*), 89
 CSR_SHARTID (*C macro*), 112
 CSR_SIDELEG (*C macro*), 88
 CSR_SIE (*C macro*), 88
 CSR_SINTSTATUS (*C macro*), 91
 CSR_SIP (*C macro*), 89
 CSR_SLEEPVALUE (*C macro*), 110
 CSR_SLWID (*C macro*), 112
 CSR_SMPUADDR0 (*C macro*), 106
 CSR_SMPUADDR1 (*C macro*), 106
 CSR_SMPUADDR10 (*C macro*), 107
 CSR_SMPUADDR11 (*C macro*), 107
 CSR_SMPUADDR12 (*C macro*), 107
 CSR_SMPUADDR13 (*C macro*), 107
 CSR_SMPUADDR14 (*C macro*), 107
 CSR_SMPUADDR15 (*C macro*), 107
 CSR_SMPUADDR2 (*C macro*), 106
 CSR_SMPUADDR3 (*C macro*), 106
 CSR_SMPUADDR4 (*C macro*), 106
 CSR_SMPUADDR5 (*C macro*), 106
 CSR_SMPUADDR6 (*C macro*), 106
 CSR_SMPUADDR7 (*C macro*), 106
 CSR_SMPUADDR8 (*C macro*), 106
 CSR_SMPUADDR9 (*C macro*), 106
 CSR_SMPUCFG0 (*C macro*), 106
 CSR_SMPUCFG1 (*C macro*), 106
 CSR_SMPUCFG2 (*C macro*), 106
 CSR_SMPUCFG3 (*C macro*), 106
 CSR_SMPUSWITCH0 (*C macro*), 107
 CSR_SMPUSWITCH1 (*C macro*), 107
 CSR_SNXTI (*C macro*), 91
 CSR_SPMPADDR0 (*C macro*), 105
 CSR_SPMPADDR1 (*C macro*), 105
 CSR_SPMPADDR10 (*C macro*), 106
 CSR_SPMPADDR11 (*C macro*), 106
 CSR_SPMPADDR12 (*C macro*), 106
 CSR_SPMPADDR13 (*C macro*), 106
 CSR_SPMPADDR14 (*C macro*), 106
 CSR_SPMPADDR15 (*C macro*), 106
 CSR_SPMPADDR2 (*C macro*), 105
 CSR_SPMPADDR3 (*C macro*), 105
 CSR_SPMPADDR4 (*C macro*), 105
 CSR_SPMPADDR5 (*C macro*), 105
 CSR_SPMPADDR6 (*C macro*), 105
 CSR_SPMPADDR7 (*C macro*), 105
 CSR_SPMPADDR8 (*C macro*), 105
 CSR_SPMPADDR9 (*C macro*), 106
 CSR_SPMPCFG0 (*C macro*), 105
 CSR_SPMPCFG1 (*C macro*), 105
 CSR_SPMPCFG2 (*C macro*), 105
 CSR_SPMPCFG3 (*C macro*), 105
 CSR_SSCRATCH (*C macro*), 89
 CSR_SSCRATCHCSW (*C macro*), 91
 CSR_SSCRATCHCSWL (*C macro*), 91
 CSR_SSHADGPRLVL0 (*C macro*), 111
 CSR_SSHADGPRLVL1 (*C macro*), 111
 CSR_SSTATEEN0 (*C macro*), 88
 CSR_SSTATEEN1 (*C macro*), 88
 CSR_SSTATEEN2 (*C macro*), 88
 CSR_SSTATEEN3 (*C macro*), 88
 CSR_SSTATUS (*C macro*), 88
 CSR_SSUBM (*C macro*), 111
 CSR_STIMECMP (*C macro*), 89
 CSR_STIMECMPH (*C macro*), 89
 CSR_STSP (*C macro*), 111
 CSR_STSPCSW (*C macro*), 111
 CSR_STVAL (*C macro*), 89
 CSR_STVEC (*C macro*), 88
 CSR_STVT (*C macro*), 88
 CSR_STVT2 (*C macro*), 111
 CSR_TCONTROL (*C macro*), 97
 CSR_TDATA1 (*C macro*), 96
 CSR_TDATA2 (*C macro*), 96
 CSR_TDATA3 (*C macro*), 96
 CSR_TIME (*C macro*), 86
 CSR_TIMEH (*C macro*), 88
 CSR_TINFO (*C macro*), 96
 CSR_TSELECT (*C macro*), 96
 CSR_TXEVT (*C macro*), 111
 CSR_UCODE (*C macro*), 107
 CSR_UINTSTATUS (*C macro*), 91
 CSR_UNXTI (*C macro*), 90
 CSR_USCRATCHCSW (*C macro*), 91
 CSR_USCRATCHCSWL (*C macro*), 91
 CSR_USTATUS (*C macro*), 86
 CSR_UTVT (*C macro*), 90
 CSR_VCSR (*C macro*), 86
 CSR_VL (*C macro*), 88
 CSR_VLENB (*C macro*), 88
 CSR_VSATP (*C macro*), 89
 CSR_VSCAUSE (*C macro*), 89
 CSR_VSEPC (*C macro*), 89
 CSR_VSIE (*C macro*), 89
 CSR_VSIP (*C macro*), 89
 CSR_VSSCRATCH (*C macro*), 89
 CSR_VSSTATUS (*C macro*), 89
 CSR_VSTART (*C macro*), 86
 CSR_VSTIMECMP (*C macro*), 89
 CSR_VSTIMECMPH (*C macro*), 100
 CSR_VSTVAL (*C macro*), 89
 CSR_VSTVEC (*C macro*), 89
 CSR_VTYPE (*C macro*), 88
 CSR_VXRM (*C macro*), 86
 CSR_VXSAT (*C macro*), 86
 CSR_WFE (*C macro*), 111
 CSR_XCAUSE (*C macro*), 134

CSR_XCOUNTEREN (*C macro*), 134
CSR_XDCAUSE (*C macro*), 134
CSR_XECLIC_CTL (*C macro*), 134
CSR_XENVCFG (*C macro*), 134
CSR_XEPC (*C macro*), 134
CSR_XIE (*C macro*), 134
CSR_XINTSTATUS (*C macro*), 135
CSR_XIP (*C macro*), 134
CSR_XNXTI (*C macro*), 135
CSR_XSCRATCH (*C macro*), 134
CSR_XSCRATCHCSW (*C macro*), 134
CSR_XSCRATCHCSWL (*C macro*), 134
CSR_XSTATUS (*C macro*), 134
CSR_XSUBM (*C macro*), 134
CSR_XTSP (*C macro*), 134
CSR_XTVAL (*C macro*), 134
CSR_XTVEC (*C macro*), 134
CSR_XTVT (*C macro*), 134
CSR_XTVT2 (*C macro*), 134

D

DCAUSE_FAULT_FETCH_INST (*C macro*), 133
DCAUSE_FAULT_FETCH_PMP (*C macro*), 133
DCAUSE_FAULT_LOAD_INST (*C macro*), 133
DCAUSE_FAULT_LOAD_NICE (*C macro*), 134
DCAUSE_FAULT_LOAD_PMP (*C macro*), 133
DCAUSE_FAULT_STORE_INST (*C macro*), 134
DCAUSE_FAULT_STORE_PMP (*C macro*), 134
DCSR_CAUSE (*C macro*), 116
DCSR_CAUSE_DEBUGINT (*C macro*), 116
DCSR_CAUSE_HALT (*C macro*), 116
DCSR_CAUSE_HWBP (*C macro*), 116
DCSR_CAUSE_NONE (*C macro*), 116
DCSR_CAUSE_STEP (*C macro*), 116
DCSR_CAUSE_SWBP (*C macro*), 116
DCSR_DEBUGINT (*C macro*), 116
DCSR_EBREAKH (*C macro*), 116
DCSR_EBREAKM (*C macro*), 116
DCSR_EBREAKS (*C macro*), 116
DCSR_EBREAKU (*C macro*), 116
DCSR_FULLRESET (*C macro*), 116
DCSR_HALT (*C macro*), 116
DCSR_NDRESET (*C macro*), 116
DCSR_PRV (*C macro*), 116
DCSR_STEP (*C macro*), 116
DCSR_STOPCYCLE (*C macro*), 116
DCSR_STOPTIME (*C macro*), 116
DCSR_XDEBUGVER (*C macro*), 116
default_intexc_handler (*C++ function*), 805
depthwise_conv_s16_generic_s16 (*C++ function*),
1322, 1351
depthwise_conv_s4_generic (*C++ function*), 1323,
1352

depthwise_conv_s8_generic (*C++ function*), 1324,
1354
depthwise_conv_s8_mult_4 (*C++ function*), 1323,
1353
DFF_PROT_Type (*C++ union*), 191
DFF_PROT_Type::b (*C++ member*), 192
DFF_PROT_Type::w (*C++ member*), 192
DSP, 1453

E

ECC_ERR_MSK_Type (*C++ union*), 192
ECC_ERR_MSK_Type::b (*C++ member*), 192
ECC_ERR_MSK_Type::w (*C++ member*), 192
ECC_ERROR_RAMID_MASK_DCACHE (*C macro*), 794, 795
ECC_ERROR_RAMID_MASK_DLM (*C macro*), 794, 795
ECC_ERROR_RAMID_MASK_ICACHE (*C macro*), 794, 795
ECC_ERROR_RAMID_MASK_ILM (*C macro*), 794, 795
ECC_ERROR_RAMID_MASK_TLB (*C macro*), 794, 795
ECC_GenerateECCCodeU32 (*C++ function*), 794, 804
ECC_GenerateECCCodeU64 (*C++ function*), 794, 804
ECLIC (*C macro*), 165
ECLIC_BASE (*C macro*), 165
ECLIC_ClearPendingIRQ (*C macro*), 640, 643
ECLIC_ClearPendingIRQ_S (*C macro*), 641, 644
ECLIC_DisableIRQ (*C macro*), 640, 643
ECLIC_DisableIRQ_S (*C macro*), 642, 644
ECLIC_EnableIRQ (*C macro*), 640, 643
ECLIC_EnableIRQ_S (*C macro*), 642, 644
ECLIC_GetCfgNlbits (*C macro*), 640, 642
ECLIC_GetCtrlIRQ (*C macro*), 641, 643
ECLIC_GetCtrlIRQ_S (*C macro*), 641, 644
ECLIC_GetEnableIRQ (*C macro*), 640, 643
ECLIC_GetEnableIRQ_S (*C macro*), 642, 644
ECLIC_GetInfoCtlbits (*C macro*), 640, 642
ECLIC_GetInfoNum (*C macro*), 640, 642
ECLIC_GetInfoShadowNum (*C macro*), 640, 642
ECLIC_GetInfoVer (*C macro*), 640, 642
ECLIC_GetLevelIRQ (*C macro*), 641, 643
ECLIC_GetLevelIRQ_S (*C macro*), 641, 644
ECLIC_GetMth (*C macro*), 640, 643
ECLIC_GetPendingIRQ (*C macro*), 640, 643
ECLIC_GetPendingIRQ_S (*C macro*), 641, 644
ECLIC_GetPriorityIRQ (*C macro*), 641, 643
ECLIC_GetPriorityIRQ_S (*C macro*), 641, 644
ECLIC_GetShvIRQ (*C macro*), 640, 643
ECLIC_GetShvIRQ_S (*C macro*), 641, 644
ECLIC_GetSth (*C macro*), 641, 643
ECLIC_GetTrigIRQ (*C macro*), 640, 643
ECLIC_GetTrigIRQ_S (*C macro*), 641, 644
ECLIC_GetVector (*C macro*), 642, 644
ECLIC_GetVector_S (*C macro*), 642, 644
ECLIC_Interrupt_Init (*C++ function*), 805
ECLIC_MAX_NLBITS (*C macro*), 164
ECLIC_MODE_MTVEC_Msk (*C macro*), 164

ECLIC_NON_VECTOR_INTERRUPT (*C macro*), 164
 ECLIC_Register_IRQ (*C++ function*), 806
 ECLIC_Register_IRQ_S (*C++ function*), 809
 ECLIC_SetCfgNlbits (*C macro*), 640, 642
 ECLIC_SetCtrlIRQ (*C macro*), 640, 643
 ECLIC_SetCtrlIRQ_S (*C macro*), 641, 644
 ECLIC_SetLevelIRQ (*C macro*), 641, 643
 ECLIC_SetLevelIRQ_S (*C macro*), 641, 644
 ECLIC_SetModeIRQ (*C macro*), 641, 643
 ECLIC_SetMth (*C macro*), 640, 642
 ECLIC_SetPendingIRQ (*C macro*), 640, 643
 ECLIC_SetPendingIRQ_S (*C macro*), 641, 643
 ECLIC_SetPriorityIRQ (*C macro*), 641, 643
 ECLIC_SetPriorityIRQ_S (*C macro*), 641, 644
 ECLIC_SetShvIRQ (*C macro*), 640, 643
 ECLIC_SetShvIRQ_S (*C macro*), 641, 644
 ECLIC_SetSth (*C macro*), 641, 643
 ECLIC_SetTrigIRQ (*C macro*), 640, 643
 ECLIC_SetTrigIRQ_S (*C macro*), 641, 644
 ECLIC_SetVector (*C macro*), 642, 644
 ECLIC_SetVector_S (*C macro*), 642, 644
 ECLIC_TRIGGER_Type (*C++ enum*), 165
 ECLIC_TRIGGER_Type::ECLIC_LEVEL_TRIGGER (*C++ enumerator*), 165
 ECLIC_TRIGGER_Type::ECLIC_MAX_TRIGGER (*C++ enumerator*), 165
 ECLIC_TRIGGER_Type::ECLIC_NEGATIVE_EDGE_TRIGGER (*C++ enumerator*), 165
 ECLIC_TRIGGER_Type::ECLIC_POSTIVE_EDGE_TRIGGER (*C++ enumerator*), 165
 ECLIC_VECTOR_INTERRUPT (*C macro*), 164
 eclic_xsip_handler (*C macro*), 135
 eclic_xtip_handler (*C macro*), 135
 EVENT_INSTRUCTION_COMMIT_ATOMIC_MEMORY_OPERATION (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_CONDITIONAL_BRANCH (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_CONDITIONAL_BRANCH_PREDICTION_FAIL (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_CONTROL_TRANSFER (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_CYCLE_COUNT (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_ECALL_INSTRUCTION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_EXCEPTION_INSTRUCTION (*C macro*), 825
 EVENT_INSTRUCTION_COMMIT_FENCE_INSTRUCTION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FENCEI_INSTRUCTION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_ADDITION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_DIVISION_OR_SQUARE (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_FUSED_MULTIPLY_ADD (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_LOAD (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_MULTIPLICATION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_FLOATING_POINT_STORE (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_INTEGER_COMPUTATIONAL (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_INTEGER_DIVISION_REMAINDER (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_INTEGER_LOAD (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_INTEGER_MULTIPLICATION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_INTEGER_STORE (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_INTERRUPT_INSTRUCTION (*C macro*), 825
 EVENT_INSTRUCTION_COMMIT_JAL (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_JALR (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_JALR_PREDICTION_FAIL (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_OTHER_FLOATING_POINT_INSTRUCTION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_POP_PREDICTION_FAIL (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_RETIRED_COUNT (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_RETURN (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_SFENCE_INSTRUCTION (*C macro*), 824
 EVENT_INSTRUCTION_COMMIT_SYSTEM (*C macro*), 823
 EVENT_INSTRUCTION_COMMIT_TAKEN_CONDITIONAL_BRANCH (*C macro*), 824
 EVENT_MEMORY_ACCESS_DCACHE_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_DTLB_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_EXU_STALL_CYCLE (*C macro*), 825
 EVENT_MEMORY_ACCESS_ICACHE_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_IFU_STALL_CYCLE (*C macro*), 825
 EVENT_MEMORY_ACCESS_ITLB_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_L2_CACHE_ACCESS (*C macro*), 825
 EVENT_MEMORY_ACCESS_L2_CACHE_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_MAIN_DTLB_MISS (*C macro*), 825
 EVENT_MEMORY_ACCESS_MAIN_TLB_MISS (*C macro*), 825

EVENT_MEMORY_ACCESS_MEMORY_BUS_REQUEST (C macro), 825
 EVENT_MEMORY_ACCESS_TIMER (C macro), 825
 EVENT_SEL_INSTRUCTION_COMMIT (C macro), 823
 EVENT_SEL_MEMORY_ACCESS (C macro), 823
 EVENT_SEL_TYPE_0 (C macro), 823
 EVENT_SEL_TYPE_1 (C macro), 823
 EVENT_SEL_TYPE_2 (C macro), 823
 EVENT_SEL_TYPE_3 (C macro), 823
 EVENT_TYPE_0_ATOMIC_MEMORY_OPERATION (C macro), 825
 EVENT_TYPE_0_CONDITIONAL_BRANCH (C macro), 826
 EVENT_TYPE_0_CONDITIONAL_BRANCH_PREDICTION_FAIL (C macro), 826
 EVENT_TYPE_0_CONTROL_TRANSFER (C macro), 826
 EVENT_TYPE_0_CYCLE_COUNT (C macro), 825
 EVENT_TYPE_0_ECALL_INSTRUCTION (C macro), 827
 EVENT_TYPE_0_EXCEPTION_INSTRUCTION (C macro), 827
 EVENT_TYPE_0_FENCE_INSTRUCTION (C macro), 826
 EVENT_TYPE_0_FENCEI_INSTRUCTION (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_ADDITION_SUBTRACTION (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_DIVISION_OR_SQUARE_ROOT (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_FUSED_MULTIPLY_ADD (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_LOAD (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_MULTIPLICATION (C macro), 826
 EVENT_TYPE_0_FLOATING_POINT_STORE (C macro), 826
 EVENT_TYPE_0_INTEGER_COMPUTATIONAL (C macro), 825
 EVENT_TYPE_0_INTEGER_DIVISION_REMAINDER (C macro), 826
 EVENT_TYPE_0_INTEGER_LOAD (C macro), 825
 EVENT_TYPE_0_INTEGER_MULTIPLICATION (C macro), 826
 EVENT_TYPE_0_INTEGER_STORE (C macro), 825
 EVENT_TYPE_0_INTERRUPT_INSTRUCTION (C macro), 827
 EVENT_TYPE_0_JAL (C macro), 826
 EVENT_TYPE_0_JALR (C macro), 826
 EVENT_TYPE_0_JALR_PREDICTION_FAIL (C macro), 826
 EVENT_TYPE_0_OTHER_FLOATING_POINT_INSTRUCTION (C macro), 826
 EVENT_TYPE_0_POP_PREDICTION_FAIL (C macro), 826
 EVENT_TYPE_0_RETIRED_COUNT (C macro), 825
 EVENT_TYPE_0_RETURN (C macro), 826
 EVENT_TYPE_0_SFENCE_INSTRUCTION (C macro), 826
 EVENT_TYPE_0_SYSTEM (C macro), 825
 EVENT_TYPE_0_TAKEN_CONDITIONAL_BRANCH (C macro), 825
 EVENT_TYPE_1_DCACHE_RW_MISS (C macro), 827
 EVENT_TYPE_1_DTLB_RW_MISS (C macro), 827
 EVENT_TYPE_1_EXU_STALL_CYCLE (C macro), 827
 EVENT_TYPE_1_ICACHE_READ_MISS (C macro), 827
 EVENT_TYPE_1_IFU_STALL_CYCLE (C macro), 827
 EVENT_TYPE_1_ITLB_READ_MISS (C macro), 827
 EVENT_TYPE_1_L2_CACHE_ACCESS (C macro), 827
 EVENT_TYPE_1_L2_CACHE_MISS (C macro), 827
 EVENT_TYPE_1_MAIN_TLB_MISS (C macro), 827
 EVENT_TYPE_1_MEMORY_BUS_REQUEST (C macro), 827
 EVENT_TYPE_1_TIMER (C macro), 827
 EVENT_TYPE_2_BRANCH_INSTRUCTION_COMMIT (C macro), 827
 EVENT_TYPE_2_BRANCH_PREDICT_FAIL_COMMIT (C macro), 827
 EVENT_TYPE_3_BTBT_READ (C macro), 828
 EVENT_TYPE_3_BTBT_READ_MISS (C macro), 828
 EVENT_TYPE_3_BTBT_WRITE (C macro), 828
 EVENT_TYPE_3_BTBT_WRITE_MISS (C macro), 828
 EVENT_TYPE_3_DCACHE_PREFETCH (C macro), 827
 EVENT_TYPE_3_DCACHE_PREFETCH_MISS (C macro), 828
 EVENT_TYPE_3_DCACHE_READ (C macro), 827
 EVENT_TYPE_3_DCACHE_READ_MISS (C macro), 827
 EVENT_TYPE_3_DCACHE_WRITE (C macro), 827
 EVENT_TYPE_3_DCACHE_WRITE_MISS (C macro), 827
 EVENT_TYPE_3_DTLB_READ (C macro), 828
 EVENT_TYPE_3_DTLB_READ_MISS (C macro), 828
 EVENT_TYPE_3_DTLB_WRITE (C macro), 828
 EVENT_TYPE_3_DTLB_WRITE_MISS (C macro), 828
 EVENT_TYPE_3_ICACHE_PREFETCH (C macro), 828
 EVENT_TYPE_3_ICACHE_PREFETCH_MISS (C macro), 828
 EVENT_TYPE_3_ICACHE_READ (C macro), 828
 EVENT_TYPE_3_ITLB_READ (C macro), 828
 EVENT_TYPE_3_L2_CACHE_PREFETCH_HIT (C macro), 828
 EVENT_TYPE_3_L2_CACHE_PREFETCH_MISS (C macro), 828
 EVENT_TYPE_3_L2_CACHE_READ (C macro), 828
 EVENT_TYPE_3_L2_CACHE_READ_MISS (C macro), 828
 EVENT_TYPE_3_L2_CACHE_WRITE (C macro), 828
 EVENT_TYPE_3_L2_CACHE_WRITE_MISS (C macro), 828
 exc_entry (C++ function), 805
 exc_entry_s (C++ function), 805
 EXC_HANDLER (C++ type), 811
 Exception_DumpFrame (C++ function), 813
 Exception_Get_EXC (C++ function), 815
 Exception_Get_EXC_S (C++ function), 816
 Exception_Init (C++ function), 815
 Exception_Register_EXC (C++ function), 814
 Exception_Register_EXC_S (C++ function), 816

F

FAULT_FETCH (C macro), 133
 FAULT_LOAD (C macro), 133
 FAULT_STORE (C macro), 133
 FETCH_PAGE_FAULT (C macro), 133
 FFLAGS_AE_DZ (C macro), 130
 FFLAGS_AE_NV (C macro), 130
 FFLAGS_AE_NX (C macro), 130
 FFLAGS_AE_OF (C macro), 130
 FFLAGS_AE_UF (C macro), 130
 FFTINIT (C macro), 1163, 1164, 1167, 1174, 1178
 float32_t (C++ type), 1277
 float64_t (C++ type), 1277
 FREG (C macro), 130
 FRM_RNDMODE_DYN (C macro), 130
 FRM_RNDMODE_RDN (C macro), 129
 FRM_RNDMODE_RMM (C macro), 130
 FRM_RNDMODE_RNE (C macro), 129
 FRM_RNDMODE_RTZ (C macro), 129
 FRM_RNDMODE_RUP (C macro), 129

H

HPM_DECLARE_VAR (C macro), 829
 HPM_END (C macro), 829
 HPM_EVENT (C macro), 829
 HPM_GET_LPCNT (C macro), 829
 HPM_GET_SUMCYC (C macro), 829
 HPM_GET_USECYC (C macro), 829
 HPM_INIT (C macro), 829
 HPM_RESET (C macro), 829
 HPM_SAMPLE (C macro), 829
 HPM_SEL_ENABLE (C macro), 829
 HPM_SEL_EVENT (C macro), 829
 HPM_START (C macro), 829
 HPM_STAT (C macro), 829
 HPM_STOP (C macro), 829

I

ILLEGAL_INSTRUCTION (C macro), 133
 INT_HANDLER (C++ type), 811
 Interrupt_Get_CoreIRQ (C++ function), 813
 Interrupt_Get_CoreIRQ_S (C++ function), 816
 Interrupt_Get_ExtIRQ (C++ function), 814
 Interrupt_Get_ExtIRQ_S (C++ function), 817
 Interrupt_Init (C++ function), 806
 Interrupt_Register_CoreIRQ (C++ function), 813
 Interrupt_Register_CoreIRQ_S (C++ function), 816
 Interrupt_Register_ExtIRQ (C++ function), 814
 Interrupt_Register_ExtIRQ_S (C++ function), 817
 IOCP_ATTR_RMP_Type (C++ union), 195
 IOCP_ATTR_RMP_Type::b (C++ member), 195
 IOCP_ATTR_RMP_Type::w (C++ member), 195
 IREGION_DEBUG_OFS (C macro), 127

IREGION_DPREFETCH_OFS (C macro), 127
 IREGION_ECLIC_OFS (C macro), 127
 IREGION_IDU_OFS (C macro), 127
 IREGION_IINFO_OFS (C macro), 127
 IREGION_PL2_OFS (C macro), 127
 IREGION_PLIC_OFS (C macro), 128
 IREGION_SMP_OFS (C macro), 127
 IREGION_TIMER_OFS (C macro), 127
 IRQ_COP (C macro), 129
 irq_entry (C++ function), 805
 irq_entry_s (C++ function), 805
 IRQ_H_EXT (C macro), 129
 IRQ_H_SOFT (C macro), 129
 IRQ_H_TIMER (C macro), 129
 IRQ_HOST (C macro), 129
 IRQ_M_EXT (C macro), 129
 IRQ_M_SOFT (C macro), 129
 IRQ_M_TIMER (C macro), 129
 IRQ_S_EXT (C macro), 129
 IRQ_S_SOFT (C macro), 129
 IRQ_S_TIMER (C macro), 129
 IRQn_Type (C++ enum), 636, 646
 IRQn_Type::FirstDeviceSpecificInterrupt_IRQn
 (C++ enumerator), 637, 647
 IRQn_Type::Reserved0_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved10_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved11_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved12_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved13_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved14_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved15_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved16_IRQn (C++ enumerator),
 637, 647
 IRQn_Type::Reserved1_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved2_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved3_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved4_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved5_IRQn (C++ enumerator),
 636, 646
 IRQn_Type::Reserved6_IRQn (C++ enumerator),
 637, 646
 IRQn_Type::Reserved7_IRQn (C++ enumerator),
 637, 646

IRQn_Type::Reserved8_IRQn (C++ *enumerator*), 637, 646
 IRQn_Type::Reserved9_IRQn (C++ *enumerator*), 637, 646
 IRQn_Type::SOC_INT_MAX (C++ *enumerator*), 637, 647
 IRQn_Type::SysTimer_IRQn (C++ *enumerator*), 636, 646
 IRQn_Type::SysTimerSW_IRQn (C++ *enumerator*), 636, 646
 ISR, 1453

L

LOAD_PAGE_FAULT (C *macro*), 133

M

MAX_SYSTEM_EXCEPTION_NUM (C *macro*), 811
 MCACHE_CTL_DC_CMO_PF_EN (C *macro*), 124
 MCACHE_CTL_DC_DRAM_ECC_INJ_EN (C *macro*), 124
 MCACHE_CTL_DC_ECC_CHK_EN (C *macro*), 124
 MCACHE_CTL_DC_ECC_EN (C *macro*), 124
 MCACHE_CTL_DC_ECC_EXCP_EN (C *macro*), 124
 MCACHE_CTL_DC_EN (C *macro*), 124
 MCACHE_CTL_DC_RWDECC (C *macro*), 124
 MCACHE_CTL_DC_RWTECC (C *macro*), 124
 MCACHE_CTL_DC_TRAM_ECC_INJ_EN (C *macro*), 124
 MCACHE_CTL_IC_CANCEL_EN (C *macro*), 124
 MCACHE_CTL_IC_CMO_PF_EN (C *macro*), 124
 MCACHE_CTL_IC_DRAM_ECC_INJ_EN (C *macro*), 123
 MCACHE_CTL_IC_ECC_CHK_EN (C *macro*), 124
 MCACHE_CTL_IC_ECC_EN (C *macro*), 123
 MCACHE_CTL_IC_ECC_EXCP_EN (C *macro*), 123
 MCACHE_CTL_IC_EN (C *macro*), 123
 MCACHE_CTL_IC_PF_EN (C *macro*), 124
 MCACHE_CTL_IC_RWDECC (C *macro*), 123
 MCACHE_CTL_IC_RWTECC (C *macro*), 123
 MCACHE_CTL_IC_SCPD_MOD (C *macro*), 123
 MCACHE_CTL_IC_TRAM_ECC_INJ_EN (C *macro*), 123
 MCAUSE_CAUSE (C *macro*), 119
 MCAUSE_INTR (C *macro*), 119
 MCFG_INFO_CLIC (C *macro*), 124
 MCFG_INFO_DCACHE (C *macro*), 125
 MCFG_INFO_DLM (C *macro*), 125
 MCFG_INFO_DSP_N1 (C *macro*), 125
 MCFG_INFO_DSP_N2 (C *macro*), 125
 MCFG_INFO_DSP_N3 (C *macro*), 125
 MCFG_INFO_ECC (C *macro*), 124
 MCFG_INFO_ETRACE (C *macro*), 125
 MCFG_INFO_FIO (C *macro*), 124
 MCFG_INFO_ICACHE (C *macro*), 125
 MCFG_INFO_ILM (C *macro*), 125
 MCFG_INFO_IREGION_EXIST (C *macro*), 125
 MCFG_INFO_NICE (C *macro*), 125
 MCFG_INFO_PLIC (C *macro*), 124
 MCFG_INFO_PPI (C *macro*), 124
 MCFG_INFO_SAFETY_MECHANISM (C *macro*), 125
 MCFG_INFO_SEC_MODE (C *macro*), 125
 MCFG_INFO_SMP (C *macro*), 125
 MCFG_INFO_SSTC (C *macro*), 125
 MCFG_INFO_TEE (C *macro*), 124
 MCFG_INFO_VNICE (C *macro*), 125
 MCFG_INFO_VP (C *macro*), 125
 MCFG_INFO_XLCZ (C *macro*), 125
 MCFG_INFO_ZC_XLCZ_EXT (C *macro*), 125
 MCFG_INFO_ZILSD (C *macro*), 125
 MCONTROL_ACTION (C *macro*), 117
 MCONTROL_ACTION_DEBUG_EXCEPTION (C *macro*), 117
 MCONTROL_ACTION_DEBUG_MODE (C *macro*), 117
 MCONTROL_ACTION_TRACE_EMIT (C *macro*), 117
 MCONTROL_ACTION_TRACE_START (C *macro*), 117
 MCONTROL_ACTION_TRACE_STOP (C *macro*), 117
 MCONTROL_CHAIN (C *macro*), 117
 MCONTROL_DMODE (C *macro*), 117
 MCONTROL_EXECUTE (C *macro*), 117
 MCONTROL_H (C *macro*), 117
 MCONTROL_LOAD (C *macro*), 117
 MCONTROL_M (C *macro*), 117
 MCONTROL_MASKMAX (C *macro*), 117
 MCONTROL_MATCH (C *macro*), 117
 MCONTROL_MATCH_EQUAL (C *macro*), 117
 MCONTROL_MATCH_GE (C *macro*), 118
 MCONTROL_MATCH_LT (C *macro*), 118
 MCONTROL_MATCH_MASK_HIGH (C *macro*), 118
 MCONTROL_MATCH_MASK_LOW (C *macro*), 118
 MCONTROL_MATCH_NAPOT (C *macro*), 118
 MCONTROL_S (C *macro*), 117
 MCONTROL_SELECT (C *macro*), 117
 MCONTROL_STORE (C *macro*), 117
 MCONTROL_TIMING (C *macro*), 117
 MCONTROL_TYPE (C *macro*), 116
 MCONTROL_TYPE_MATCH (C *macro*), 117
 MCONTROL_TYPE_NONE (C *macro*), 117
 MCONTROL_U (C *macro*), 117
 MCOUNTEREN_CY (C *macro*), 121
 MCOUNTEREN_CY_SHIFT (C *macro*), 121
 MCOUNTEREN_IR (C *macro*), 121
 MCOUNTEREN_IR_SHIFT (C *macro*), 121
 MCOUNTEREN_TIME (C *macro*), 121
 MCOUNTEREN_TIME_SHIFT (C *macro*), 121
 MCOUNTINHIBIT_CY (C *macro*), 121
 MCOUNTINHIBIT_IR (C *macro*), 121
 MDCAUSE_MDCAUSE (C *macro*), 122
 MDCFG_DC_ECC (C *macro*), 126
 MDCFG_DC_LSIZE (C *macro*), 126
 MDCFG_DC_SET (C *macro*), 126
 MDCFG_DC_WAY (C *macro*), 126
 MDCFG_DLM_ECC (C *macro*), 126
 MDCFG_DLM_SIZE (C *macro*), 126

MDLM_CTL_DLM_BPA (C macro), 122
 MDLM_CTL_DLM_ECC_CHK_EN (C macro), 122
 MDLM_CTL_DLM_ECC_EN (C macro), 122
 MDLM_CTL_DLM_ECC_EXCP_EN (C macro), 122
 MDLM_CTL_DLM_ECC_INJ_EN (C macro), 122
 MDLM_CTL_DLM_EN (C macro), 122
 MDLM_CTL_DLM_RWECC (C macro), 122
 MECC_CODE_CODE (C macro), 126
 MECC_CODE_RAMID (C macro), 126
 MECC_CODE_RAMID_DC (C macro), 127
 MECC_CODE_RAMID_DLM (C macro), 127
 MECC_CODE_RAMID_IC (C macro), 127
 MECC_CODE_RAMID_ILM (C macro), 127
 MECC_CODE_RAMID_TLB (C macro), 127
 MECC_CODE_SRAMID (C macro), 126
 MECC_CODE_SRAMID_DC (C macro), 127
 MECC_CODE_SRAMID_DLM (C macro), 127
 MECC_CODE_SRAMID_IC (C macro), 127
 MECC_CODE_SRAMID_ILM (C macro), 127
 MECC_CODE_SRAMID_TLB (C macro), 127
 MECC_LOCK_ECC_LOCK (C macro), 126
 MECLIC_CTL_FEAT_EN (C macro), 123
 MECLIC_CTL_SHADOW_EN (C macro), 123
 MECLIC_CTL_TSP_EN (C macro), 123
 MENVCFG_ADUE (C macro), 120
 MENVCFG_CBCFE (C macro), 120
 MENVCFG_CBIE (C macro), 120
 MENVCFG_CBIE_EN (C macro), 119
 MENVCFG_CBIE_FLUSH (C macro), 119
 MENVCFG_CBIE_INVALID (C macro), 119
 MENVCFG_CBZE (C macro), 120
 MENVCFG_DTE (C macro), 120
 MENVCFG_FIOM (C macro), 120
 MENVCFG_LPE (C macro), 120
 MENVCFG_PBMTE (C macro), 120
 MENVCFG_PMM (C macro), 120
 MENVCFG_SSE (C macro), 120
 MENVCFG_STCE (C macro), 120
 MENVCFGH_ADUE (C macro), 120
 MENVCFGH_DTE (C macro), 120
 MENVCFGH_PBMTE (C macro), 120
 MENVCFGH_STCE (C macro), 120
 MEVENT_EN (C macro), 828
 MFCC_INIT_F16 (C macro), 1207
 MFCC_INIT_F32 (C macro), 1205
 MFCC_INIT_Q15 (C macro), 1209
 MFCC_INIT_Q31 (C macro), 1211
 MFIOCFG_INFO_FIO_BPA (C macro), 126
 MFIOCFG_INFO_FIO_SIZE (C macro), 126
 MFlushDCacheCCacheLine (C macro), 733, 735
 MFlushDCacheCCacheLines (C macro), 733, 735
 MFlushInvalidCacheCCacheLine (C macro), 733, 737
 MFlushInvalidCacheCCacheLines (C macro), 733, 737
 MICFG_IC_ECC (C macro), 126
 MICFG_IC_LSIZE (C macro), 126
 MICFG_IC_SET (C macro), 125
 MICFG_IC_WAY (C macro), 125
 MICFG_ILM_ECC (C macro), 126
 MICFG_ILM_SIZE (C macro), 126
 MICFG_ILM_XONLY (C macro), 126
 MIDELEG_SEIE (C macro), 119
 MIDELEG_SSIE (C macro), 119
 MIDELEG_STIE (C macro), 119
 MIE_HEIE (C macro), 119
 MIE_HSIE (C macro), 118
 MIE_HTIE (C macro), 118
 MIE_MEIE (C macro), 119
 MIE_MSIE (C macro), 118
 MIE_MTIE (C macro), 118
 MIE_SEIE (C macro), 118
 MIE_SSIE (C macro), 118
 MIE_STIE (C macro), 118
 MILM_CTL_ILM_BPA (C macro), 121
 MILM_CTL_ILM_ECC_CHK_EN (C macro), 121
 MILM_CTL_ILM_ECC_EN (C macro), 122
 MILM_CTL_ILM_ECC_EXCP_EN (C macro), 122
 MILM_CTL_ILM_ECC_INJ_EN (C macro), 121
 MILM_CTL_ILM_EN (C macro), 122
 MILM_CTL_ILM_RWECC (C macro), 121
 MInvalidCacheCCacheLine (C macro), 733
 MInvalidCacheCCacheLines (C macro), 733, 734
 MIP_HEIP (C macro), 118
 MIP_HSIP (C macro), 118
 MIP_HTIP (C macro), 118
 MIP_MEIP (C macro), 118
 MIP_MSIP (C macro), 118
 MIP_MTIP (C macro), 118
 MIP_SEIP (C macro), 118
 MIP_SSIP (C macro), 118
 MIP_STIP (C macro), 118
 MIRGB_INFO_IREGION_SIZE_BOFS (C macro), 126
 MIRGB_INFO_IRG_BASE_ADDR_BOFS (C macro), 126
 MISALIGNED_FETCH (C macro), 133
 MISALIGNED_LOAD (C macro), 133
 MISALIGNED_STORE (C macro), 133
 MMISC_CTL_BPU (C macro), 123
 MMISC_CTL_CODE_BUS_ERR (C macro), 123
 MMISC_CTL_HW_AUTO_CONTEXT (C macro), 122
 MMISC_CTL_IMRETURN_ENABLE (C macro), 123
 MMISC_CTL_LDSPEC_ENABLE (C macro), 122
 MMISC_CTL_MISALIGN (C macro), 123
 MMISC_CTL_NMI_CAUSE_FFF (C macro), 123
 MMISC_CTL_SIJUMP_ENABLE (C macro), 123
 MMISC_CTL_ZC (C macro), 123
 MPPICFG_INFO_PPI_BPA (C macro), 126
 MPPICFG_INFO_PPI_SIZE (C macro), 126
 MSTACK_CTRL_MODE (C macro), 128
 MSTACK_CTRL_OVF_TRACK_EN (C macro), 128

MSTACK_CTRL_UDF_EN (*C macro*), 128
MSTATUS32_SD (*C macro*), 114
MSTATUS64_SD (*C macro*), 114
MSTATUS_FS (*C macro*), 114
MSTATUS_FS_CLEAN (*C macro*), 114
MSTATUS_FS_DIRTY (*C macro*), 114
MSTATUS_FS_INITIAL (*C macro*), 114
MSTATUS_GVA (*C macro*), 114
MSTATUS_HIE (*C macro*), 113
MSTATUS_MBE (*C macro*), 114
MSTATUS_MIE (*C macro*), 113
MSTATUS_MPIE (*C macro*), 113
MSTATUS_MPP (*C macro*), 113
MSTATUS_MPRV (*C macro*), 114
MSTATUS_MPV (*C macro*), 114
MSTATUS_MXR (*C macro*), 114
MSTATUS_SBE (*C macro*), 114
MSTATUS_SIE (*C macro*), 113
MSTATUS_SPIE (*C macro*), 113
MSTATUS_SPP (*C macro*), 113
MSTATUS_SUM (*C macro*), 114
MSTATUS_SXL (*C macro*), 114
MSTATUS_TSR (*C macro*), 114
MSTATUS_TVM (*C macro*), 114
MSTATUS_TW (*C macro*), 114
MSTATUS_UBE (*C macro*), 113
MSTATUS_UIE (*C macro*), 113
MSTATUS_UPIE (*C macro*), 113
MSTATUS_UXL (*C macro*), 114
MSTATUS_VS (*C macro*), 113
MSTATUS_VS_CLEAN (*C macro*), 114
MSTATUS_VS_DIRTY (*C macro*), 115
MSTATUS_VS_INITIAL (*C macro*), 114
MSTATUS_XS (*C macro*), 114
MSTATUSH_GVA (*C macro*), 115
MSTATUSH_MBE (*C macro*), 115
MSTATUSH_MPV (*C macro*), 115
MSTATUSH_SBE (*C macro*), 115
MSU_EVENT_ENABLE (*C macro*), 828
MSUBM_GPRIDX (*C macro*), 122
MSUBM_PGPRIDX (*C macro*), 122
MSUBM_PTyp (*C macro*), 122
MSUBM_Typ (*C macro*), 122
MTVT2_COMMON_CODE_ENTRY (*C macro*), 124
MTVT2_MTVT2EN (*C macro*), 124

N

nmsis_nn_activation (*C++ struct*), 1292, 1294
nmsis_nn_bias_data (*C++ struct*), 1292, 1293
nmsis_nn_bmm_params (*C++ struct*), 1292, 1294
nmsis_nn_context (*C++ struct*), 1292, 1293
nmsis_nn_conv_params (*C++ struct*), 1292, 1294
nmsis_nn_dims (*C++ struct*), 1292, 1293
nmsis_nn_dw_conv_params (*C++ struct*), 1292, 1294

nmsis_nn_fc_params (*C++ struct*), 1292, 1294
nmsis_nn_lstm_context (*C++ struct*), 1292, 1295
nmsis_nn_lstm_dims (*C++ struct*), 1292, 1293
nmsis_nn_lstm_gate (*C++ struct*), 1292, 1294
nmsis_nn_lstm_params (*C++ struct*), 1292, 1295
nmsis_nn_per_channel_quant_params (*C++ struct*), 1292, 1294
nmsis_nn_per_tensor_quant_params (*C++ struct*), 1292, 1294
nmsis_nn_pool_params (*C++ struct*), 1292, 1294
nmsis_nn_quant_params (*C++ struct*), 1292, 1294
nmsis_nn_scaling (*C++ struct*), 1292, 1294
nmsis_nn_softmax_lut_s16 (*C++ struct*), 1292, 1294
nmsis_nn_svd_params (*C++ struct*), 1292, 1294
nmsis_nn_tile (*C++ struct*), 1291, 1293
nmsis_nn_transpose_conv_params (*C++ struct*), 1292, 1294
nmsis_nn_transpose_params (*C++ struct*), 1292, 1294
NMSIS_TEST_FAIL (*C macro*), 829
NMSIS_TEST_PASS (*C macro*), 829
NN, 1453
NS_RG_Type (*C++ union*), 192
NS_RG_Type::b (*C++ member*), 193

P

PLIC_BASE (*C macro*), 167
PLIC_CLAIM_OFFSET (*C macro*), 167
PLIC_CLAIM_REGADDR (*C macro*), 168
PLIC_CLAIM_SHIFT_PER_CONTEXT (*C macro*), 167
PLIC_ClaimInterrupt (*C macro*), 672, 673
PLIC_ClaimInterrupt_S (*C macro*), 672, 673
PLIC_COMPLETE_REGADDR (*C macro*), 168
PLIC_CompleteInterrupt (*C macro*), 672, 673
PLIC_CompleteInterrupt_S (*C macro*), 672, 673
PLIC_DisableInterrupt (*C macro*), 672, 673
PLIC_DisableInterrupt_S (*C macro*), 672, 673
PLIC_ENABLE_OFFSET (*C macro*), 167
PLIC_ENABLE_REGADDR (*C macro*), 168
PLIC_ENABLE_SHIFT_PER_CONTEXT (*C macro*), 167
PLIC_EnableInterrupt (*C macro*), 672, 673
PLIC_EnableInterrupt_S (*C macro*), 672, 673
PLIC_GetHartID (*C macro*), 167
PLIC_GetHartID_S (*C macro*), 167
PLIC_GetHartMContextID (*C macro*), 168
PLIC_GetHartSContextID (*C macro*), 168
PLIC_GetInterruptEnable (*C macro*), 672, 673
PLIC_GetInterruptEnable_S (*C macro*), 672, 673
PLIC_GetThreshold (*C macro*), 672, 673
PLIC_GetThreshold_S (*C macro*), 672, 673
PLIC_Init (*C macro*), 672, 673
PLIC_Init_S (*C macro*), 672, 673
PLIC_Interrupt_Init (*C++ function*), 806
PLIC_PENDING_OFFSET (*C macro*), 167

PLIC_PENDING_REGADDR (C macro), 168
 PLIC_PENDING_SHIFT_PER_SOURCE (C macro), 167
 PLIC_PRIORITY_OFFSET (C macro), 167
 PLIC_PRIORITY_REGADDR (C macro), 168
 PLIC_PRIORITY_SHIFT_PER_SOURCE (C macro), 167
 PLIC_Register_IRQ (C++ function), 808
 PLIC_Register_IRQ_S (C++ function), 808
 PLIC_SetThreshold (C macro), 672, 673
 PLIC_SetThreshold_S (C macro), 672, 673
 PLIC_THRESHOLD_OFFSET (C macro), 167
 PLIC_THRESHOLD_REGADDR (C macro), 168
 PLIC_THRESHOLD_SHIFT_PER_CONTEXT (C macro), 167
 pma_config (C++ struct), 706, 712
 PMA_REGION_DIS (C macro), 706, 707
 PMA_REGION_ENA (C macro), 706, 707
 PMA_REGION_TYPE_CA (C macro), 706
 PMA_REGION_TYPE_DEV (C macro), 706
 PMA_REGION_TYPE_NC (C macro), 706
 PMA_REGION_TYPE_SEC SHARE (C macro), 706
 PMP_A (C macro), 130
 PMP_A_NA4 (C macro), 130
 PMP_A_NAPOT (C macro), 130
 PMP_A_TOR (C macro), 130
 pmp_config (C++ struct), 695, 700
 PMP_COUNT (C macro), 131
 PMP_L (C macro), 130
 PMP_R (C macro), 130
 PMP_SHIFT (C macro), 130
 PMP_W (C macro), 130
 PMP_X (C macro), 130
 PMPADDR_LIST (C macro), 695, 697
 PMPADDR_LIST_16_31 (C macro), 695, 696
 PMPADDR_LIST_32_63 (C macro), 695, 697
 PMPADDR_LIST_8_15 (C macro), 695, 696
 PMPADDR_LIST_BASE (C macro), 695, 696
 PMPCFG_LIST (C macro), 695, 696
 PMPCFG_LIST_RV32_0_7 (C macro), 695, 696
 PMPCFG_LIST_RV32_8_15 (C macro), 695, 696
 PMPCFG_LIST_RV64_0_6 (C macro), 695, 696
 PMPCFG_LIST_RV64_8_14 (C macro), 695, 696
 PRV_H (C macro), 128
 PRV_M (C macro), 128
 PRV_S (C macro), 128
 PRV_U (C macro), 128
 PTE_A (C macro), 132
 PTE_D (C macro), 132
 PTE_G (C macro), 132
 PTE_PPN_SHIFT (C macro), 132
 PTE_R (C macro), 132
 PTE_SOFT (C macro), 132
 PTE_TABLE (C macro), 132
 PTE_U (C macro), 132
 PTE_V (C macro), 132
 PTE_W (C macro), 132

PTE_X (C macro), 132

Q

q15_t (C++ type), 1277
 q31_t (C++ type), 1277
 q63_t (C++ type), 1277
 q7_t (C++ type), 1277

R

READ_CYCLE (C macro), 822
 READ_HPM_COUNTER (C macro), 829
 realCoefA (C++ member), 1234, 1236
 realCoefAQ31 (C++ member), 1234, 1236
 realCoefB (C++ member), 1234, 1236
 realCoefBQ31 (C++ member), 1234, 1237
 RESTORE_FPU_CONTEXT (C macro), 694
 RESTORE_IRQ_CSR_CONTEXT (C macro), 642, 645
 RESTORE_IRQ_CSR_CONTEXT_S (C macro), 642, 645
 RESTORE_SSUBM_VAR (C macro), 642, 644
 RFFTINIT_Q15 (C macro), 1220
 RFFTINIT_Q31 (C macro), 1227
 riscv_avepool_q7_HWC (C++ function), 1385, 1388
 riscv_avgpool_s16 (C++ function), 1384, 1385
 riscv_avgpool_s16_get_buffer_size (C++ function), 1383, 1384
 riscv_avgpool_s16_get_buffer_size_dsp (C++ function), 1383, 1384
 riscv_avgpool_s8 (C++ function), 1384, 1386
 riscv_avgpool_s8_get_buffer_size (C++ function), 1383, 1384
 riscv_avgpool_s8_get_buffer_size_dsp (C++ function), 1383, 1384
 riscv_batch_matmul_s16 (C++ function), 1366, 1369
 riscv_batch_matmul_s8 (C++ function), 1366, 1370
 riscv_bilinear_interp_f16 (C++ function), 1018, 1019
 riscv_bilinear_interp_f32 (C++ function), 1018, 1019
 riscv_bilinear_interp_q15 (C++ function), 1018, 1019
 riscv_bilinear_interp_q31 (C++ function), 1018, 1020
 riscv_bilinear_interp_q7 (C++ function), 1018, 1020
 riscv_cfft_output_buffer_size (C++ function), 1245
 riscv_cfft_tmp_buffer_size (C++ function), 1245
 riscv_cifft_output_buffer_size (C++ function), 1245, 1246
 riscv_concatenation_s8_w (C++ function), 1302
 riscv_concatenation_s8_x (C++ function), 1302, 1303
 riscv_concatenation_s8_y (C++ function), 1302, 1303

[riscv_concatenation_s8_z](#) (C++ function), 1302, 1304
[riscv_convolve_1x1_n_s4](#) (C++ function), 1315, 1327
[riscv_convolve_1x1_n_s4_get_buffer_size](#) (C++ function), 1305, 1309
[riscv_convolve_1x1_n_s8](#) (C++ function), 1316, 1328
[riscv_convolve_1x1_n_s8_get_buffer_size](#) (C++ function), 1306, 1310
[riscv_convolve_1x1_HWC_q7_fast_nonsquare](#) (C++ function), 1316, 1329
[riscv_convolve_1x1_s4](#) (C++ function), 1316, 1331
[riscv_convolve_1x1_s4_fast](#) (C++ function), 1317, 1332
[riscv_convolve_1x1_s4_fast_get_buffer_size](#) (C++ function), 1305, 1309
[riscv_convolve_1x1_s8](#) (C++ function), 1317, 1333
[riscv_convolve_1x1_s8_fast](#) (C++ function), 1317, 1334
[riscv_convolve_1x1_s8_fast_get_buffer_size](#) (C++ function), 1306, 1311
[riscv_convolve_even_s4](#) (C++ function), 1317, 1335
[riscv_convolve_HWC_q15_basic](#) (C++ function), 1318, 1335
[riscv_convolve_HWC_q15_fast](#) (C++ function), 1318, 1336
[riscv_convolve_HWC_q15_fast_nonsquare](#) (C++ function), 1318, 1337
[riscv_convolve_HWC_q7_basic](#) (C++ function), 1319, 1339
[riscv_convolve_HWC_q7_basic_nonsquare](#) (C++ function), 1319, 1340
[riscv_convolve_HWC_q7_fast](#) (C++ function), 1319, 1341
[riscv_convolve_HWC_q7_fast_nonsquare](#) (C++ function), 1320, 1342
[riscv_convolve_HWC_q7_RGB](#) (C++ function), 1320, 1343
[riscv_convolve_s16](#) (C++ function), 1320, 1344
[riscv_convolve_s16_get_buffer_size](#) (C++ function), 1305, 1308
[riscv_convolve_s4](#) (C++ function), 1320, 1345
[riscv_convolve_s4_get_buffer_size](#) (C++ function), 1305, 1309
[riscv_convolve_s8](#) (C++ function), 1321, 1346
[riscv_convolve_s8_get_buffer_size](#) (C++ function), 1306, 1310
[riscv_convolve_wrapper_s16](#) (C++ function), 1321, 1347
[riscv_convolve_wrapper_s16_get_buffer_size](#) (C++ function), 1305, 1308
[riscv_convolve_wrapper_s16_get_buffer_size_dspriscv_depthwise_separable_conv_HWC_q7](#) (C++ function), 1305, 1308
[riscv_convolve_wrapper_s4](#) (C++ function), 1321, 1348
[riscv_convolve_wrapper_s4_get_buffer_size](#) (C++ function), 1305, 1309
[riscv_convolve_wrapper_s4_get_buffer_size_dsp](#) (C++ function), 1306, 1310
[riscv_convolve_wrapper_s8](#) (C++ function), 1321, 1348
[riscv_convolve_wrapper_s8_get_buffer_size](#) (C++ function), 1306, 1311
[riscv_convolve_wrapper_s8_get_buffer_size_dsp](#) (C++ function), 1306, 1311
[riscv_depthwise_conv_3x3_s8](#) (C++ function), 1322, 1349
[riscv_depthwise_conv_fast_s16](#) (C++ function), 1322, 1350
[riscv_depthwise_conv_fast_s16_get_buffer_size](#) (C++ function), 1306, 1311
[riscv_depthwise_conv_s16](#) (C++ function), 1322, 1351
[riscv_depthwise_conv_s4](#) (C++ function), 1323, 1352
[riscv_depthwise_conv_s4_opt](#) (C++ function), 1323, 1353
[riscv_depthwise_conv_s4_opt_get_buffer_size](#) (C++ function), 1307, 1312
[riscv_depthwise_conv_s8](#) (C++ function), 1324, 1354
[riscv_depthwise_conv_s8_opt](#) (C++ function), 1324, 1355
[riscv_depthwise_conv_s8_opt_get_buffer_size](#) (C++ function), 1307, 1314
[riscv_depthwise_conv_s8_opt_get_buffer_size_dsp](#) (C++ function), 1307, 1313
[riscv_depthwise_conv_wrapper_s16](#) (C++ function), 1324, 1355
[riscv_depthwise_conv_wrapper_s16_get_buffer_size](#) (C++ function), 1306, 1312
[riscv_depthwise_conv_wrapper_s16_get_buffer_size_dsp](#) (C++ function), 1306, 1312
[riscv_depthwise_conv_wrapper_s4](#) (C++ function), 1324, 1357
[riscv_depthwise_conv_wrapper_s4_get_buffer_size](#) (C++ function), 1307, 1313
[riscv_depthwise_conv_wrapper_s4_get_buffer_size_dsp](#) (C++ function), 1307, 1313
[riscv_depthwise_conv_wrapper_s8](#) (C++ function), 1325, 1357
[riscv_depthwise_conv_wrapper_s8_get_buffer_size](#) (C++ function), 1307, 1314
[riscv_depthwise_conv_wrapper_s8_get_buffer_size_dsp](#) (C++ function), 1307, 1314

`riscv_depthwise_separable_conv_HWC_q7_nonsquare` (C++ function), 1325, 1360
`riscv_elementwise_add_s16` (C++ function), 1297, 1298
`riscv_elementwise_add_s8` (C++ function), 1297, 1299
`riscv_elementwise_mul_acc_s16` (C++ function), 1297, 1299
`riscv_elementwise_mul_s16` (C++ function), 1297, 1300
`riscv_elementwise_mul_s16_batch_offset` (C++ function), 1297, 1300
`riscv_elementwise_mul_s16_s8` (C++ function), 1425
`riscv_elementwise_mul_s8` (C++ function), 1297, 1301
`riscv_fir_decimate_fast_q15` (C++ function), 969, 972
`riscv_fully_connected_mat_q7_vec_q15` (C++ function), 1366, 1370
`riscv_fully_connected_mat_q7_vec_q15_opt` (C++ function), 1366, 1371
`riscv_fully_connected_per_channel_s8` (C++ function), 1366, 1372
`riscv_fully_connected_q15` (C++ function), 1367, 1374
`riscv_fully_connected_q15_opt` (C++ function), 1367, 1374
`riscv_fully_connected_q7` (C++ function), 1367, 1375
`riscv_fully_connected_q7_opt` (C++ function), 1367, 1376
`riscv_fully_connected_s16` (C++ function), 1367, 1377
`riscv_fully_connected_s16_get_buffer_size` (C++ function), 1365
`riscv_fully_connected_s16_get_buffer_size_dsp` (C++ function), 1365
`riscv_fully_connected_s4` (C++ function), 1368, 1378
`riscv_fully_connected_s8` (C++ function), 1368, 1379
`riscv_fully_connected_s8_get_buffer_size` (C++ function), 1365
`riscv_fully_connected_s8_get_buffer_size_dsp` (C++ function), 1365
`riscv_fully_connected_wrapper_s8` (C++ function), 1368, 1380
`riscv_linear_interp_f16` (C++ function), 1020, 1022
`riscv_linear_interp_f32` (C++ function), 1020, 1022
`riscv_linear_interp_q15` (C++ function), 1020, 1022
`riscv_linear_interp_q31` (C++ function), 1020, 1022
`riscv_linear_interp_q7` (C++ function), 1020, 1023
`riscv_lstm_unidirectional_s16` (C++ function), 1382
`riscv_lstm_unidirectional_s8` (C++ function), 1382
`riscv_mat_solve_lower_triangular_f16` (C++ function), 1039, 1041
`riscv_mat_solve_lower_triangular_f32` (C++ function), 1039, 1041
`riscv_mat_solve_upper_triangular_f16` (C++ function), 1040, 1042
`riscv_mat_solve_upper_triangular_f32` (C++ function), 1040, 1042
`riscv_math_datatype` (C++ enum), 1275, 1276
`riscv_math_datatype::RISC_V_MATH_F16` (C++ enumerator), 1275, 1276
`riscv_math_datatype::RISC_V_MATH_F32` (C++ enumerator), 1275, 1276
`riscv_math_datatype::RISC_V_MATH_F64` (C++ enumerator), 1275, 1276
`riscv_math_datatype::RISC_V_MATH_Q15` (C++ enumerator), 1275, 1276
`riscv_math_datatype::RISC_V_MATH_Q31` (C++ enumerator), 1276
`riscv_math_datatype::RISC_V_MATH_Q7` (C++ enumerator), 1275, 1276
`riscv_max_pool_s16` (C++ function), 1385, 1386
`riscv_max_pool_s8` (C++ function), 1385, 1387
`riscv_maxpool_q7_HWC` (C++ function), 1385, 1387
`riscv_mfcc_tmp_buffer_size` (C++ function), 1248
`riscv_nmsis_nn_status` (C++ enum), 1291, 1293
`riscv_nmsis_nn_status::RISC_V_NMSIS_NN_ARG_ERROR` (C++ enumerator), 1291, 1293
`riscv_nmsis_nn_status::RISC_V_NMSIS_NN_FAILURE` (C++ enumerator), 1291, 1293
`riscv_nmsis_nn_status::RISC_V_NMSIS_NN_NO_IMPL_ERROR` (C++ enumerator), 1291, 1293
`riscv_nmsis_nn_status::RISC_V_NMSIS_NN_SUCCESS` (C++ enumerator), 1291, 1293
`riscv_nn_accumulate_q7_to_q15` (C++ function), 1426
`riscv_nn_activation_s16` (C++ function), 1295
`riscv_nn_activation_type` (C++ enum), 1291, 1293
`riscv_nn_activation_type::RISC_V_SIGMOID` (C++ enumerator), 1291, 1293
`riscv_nn_activation_type::RISC_V_TANH` (C++ enumerator), 1291, 1293
`riscv_nn_activations_direct_q15` (C++ function), 1295, 1296
`riscv_nn_activations_direct_q7` (C++ function), 1295, 1296
`riscv_nn_add_q7` (C++ function), 1426, 1427

riscv_nn_copy_q15 (C++ function), 1428
 riscv_nn_copy_q7 (C++ function), 1428
 riscv_nn_depthwise_conv_nt_t_padded_s8 (C++ function), 1400, 1403
 riscv_nn_depthwise_conv_nt_t_s16 (C++ function), 1400, 1404
 riscv_nn_depthwise_conv_nt_t_s4 (C++ function), 1400, 1405
 riscv_nn_depthwise_conv_nt_t_s8 (C++ function), 1400, 1406
 riscv_nn_double (C++ struct), 1398, 1399
 riscv_nn_fill_q15 (C++ function), 1428, 1429
 riscv_nn_fill_q7 (C++ function), 1428, 1429
 riscv_nn_long_long (C++ union), 1398, 1399
 riscv_nn_long_long::long_long (C++ member), 1398, 1400
 riscv_nn_long_long::word (C++ member), 1398, 1400
 riscv_nn_lstm_calculate_gate_s16 (C++ function), 1415, 1416
 riscv_nn_lstm_calculate_gate_s8_s16 (C++ function), 1415, 1416
 riscv_nn_lstm_step_s16 (C++ function), 1415, 1416
 riscv_nn_lstm_step_s8 (C++ function), 1415, 1417
 riscv_nn_mat_mul_core_1x_s4 (C++ function), 1400, 1407
 riscv_nn_mat_mul_core_1x_s8 (C++ function), 1401, 1408
 riscv_nn_mat_mul_core_4x_s8 (C++ function), 1401, 1408
 riscv_nn_mat_mult_kernel_s16 (C++ function), 1400, 1402
 riscv_nn_mat_mult_nt_interleaved_t_even_s4 (C++ function), 1401, 1409
 riscv_nn_mat_mult_nt_t_s16 (C++ function), 1401, 1410
 riscv_nn_mat_mult_nt_t_s4 (C++ function), 1401, 1411
 riscv_nn_mat_mult_nt_t_s8 (C++ function), 1402, 1412
 riscv_nn_mat_mult_nt_t_s8_s32 (C++ function), 1402, 1413
 riscv_nn_mult_q15 (C++ function), 1426, 1427
 riscv_nn_mult_q7 (C++ function), 1426, 1427
 riscv_nn_softmax_common_s8 (C++ function), 1424, 1425
 riscv_nn_transpose_conv_row_s8_s32 (C++ function), 1402, 1414
 riscv_nn_vec_mat_mul_result_acc_s16 (C++ function), 1418, 1420
 riscv_nn_vec_mat_mul_result_acc_s8_s16 (C++ function), 1415, 1417
 riscv_nn_vec_mat_mult_t_per_ch_s8 (C++ function), 1418, 1420
 riscv_nn_vec_mat_mult_t_s16 (C++ function), 1419, 1421
 riscv_nn_vec_mat_mult_t_s16_s16 (C++ function), 1419, 1422
 riscv_nn_vec_mat_mult_t_s4 (C++ function), 1419, 1422
 riscv_nn_vec_mat_mult_t_s8 (C++ function), 1419, 1423
 riscv_nn_vec_mat_mult_t_svd_s8 (C++ function), 1419, 1424
 riscv_nnword (C++ union), 1398, 1399
 riscv_nnword::bytes (C++ member), 1398, 1399
 riscv_nnword::half_words (C++ member), 1398, 1399
 riscv_nnword::word (C++ member), 1398, 1399
 riscv_pad_s8 (C++ function), 1383
 riscv_pid_instance_f32 (C++ struct), 895, 899
 riscv_pid_instance_q15 (C++ struct), 895, 899
 riscv_pid_instance_q31 (C++ struct), 895, 899
 riscv_q7_to_q15_with_offset (C++ function), 1429, 1430
 riscv_relu6_s8 (C++ function), 1295, 1296
 riscv_relu_q15 (C++ function), 1295, 1296
 riscv_relu_q7 (C++ function), 1295, 1296
 riscv_reshape_s8 (C++ function), 1389
 riscv_rfft_output_buffer_size (C++ function), 1246, 1247
 riscv_rfft_tmp_buffer_size (C++ function), 1246
 riscv_rifft_input_buffer_size (C++ function), 1246, 1247
 riscv_scale_q31 (C++ function), 873, 874
 riscv_softmax_q15 (C++ function), 1389, 1390
 riscv_softmax_q7 (C++ function), 1389, 1390
 riscv_softmax_s16 (C++ function), 1389, 1390
 riscv_softmax_s8 (C++ function), 1389, 1391
 riscv_softmax_s8_s16 (C++ function), 1389, 1391
 riscv_softmax_u8 (C++ function), 1389, 1392
 riscv_softmax_with_batch_q7 (C++ function), 1389, 1392
 riscv_sqrt_q15 (C++ function), 927
 riscv_sqrt_q31 (C++ function), 927
 riscv_status (C++ enum), 1276, 1278
 riscv_status::RISCV_MATH_ARGUMENT_ERROR (C++ enumerator), 1276, 1278
 riscv_status::RISCV_MATH_DECOMPOSITION_FAILURE (C++ enumerator), 1277, 1278
 riscv_status::RISCV_MATH_LENGTH_ERROR (C++ enumerator), 1276, 1278
 riscv_status::RISCV_MATH_NANINF (C++ enumerator), 1276, 1278
 riscv_status::RISCV_MATH_SINGULAR (C++ enumerator), 1276, 1278
 riscv_status::RISCV_MATH_SIZE_MISMATCH (C++ enumerator), 1276, 1278

riscv_status::RISCV_MATH_SUCCESS (C++ *enumerator*), 1276, 1278
 riscv_status::RISCV_MATH_TEST_FAILURE (C++ *enumerator*), 1277, 1278
 riscv_svdf_s8 (C++ *function*), 1393, 1394
 riscv_svdf_s8_get_buffer_size (C++ *function*), 1393
 riscv_svdf_s8_get_buffer_size_dsp (C++ *function*), 1393
 riscv_svdf_state_s16_s8 (C++ *function*), 1393, 1395
 riscv_transpose_conv_s8 (C++ *function*), 1326, 1362
 riscv_transpose_conv_s8_get_buffer_size (C++ *function*), 1307, 1315
 riscv_transpose_conv_s8_get_reverse_conv_buffer_size (C++ *function*), 1307, 1315
 riscv_transpose_conv_wrapper_s8 (C++ *function*), 1326, 1363
 riscv_transpose_s8 (C++ *function*), 1396, 1397
 riscv_transpose_s8_default (C++ *function*), 1396, 1397
 riscv_transpose_s8_nhwc (C++ *function*), 1396, 1397
 riscv_vector_sum_s8 (C++ *function*), 1368, 1381
 riscv_vector_sum_s8_s64 (C++ *function*), 1368, 1381
 rv_counter_t (C++ *type*), 136
 rv_csr_t (C++ *type*), 136
 rv_fpu_t (C++ *type*), 694

S

SATP32_ASID (C *macro*), 128
 SATP32_MODE (C *macro*), 128
 SATP32_PPN (C *macro*), 128
 SATP64_ASID (C *macro*), 128
 SATP64_MODE (C *macro*), 128
 SATP64_PPN (C *macro*), 128
 SATP_MODE_OFF (C *macro*), 128
 SATP_MODE_SV32 (C *macro*), 129
 SATP_MODE_SV39 (C *macro*), 129
 SATP_MODE_SV48 (C *macro*), 129
 SATP_MODE_SV57 (C *macro*), 129
 SATP_MODE_SV64 (C *macro*), 129
 SAVE_FPU_CONTEXT (C *macro*), 693
 SAVE_IRQ_CSR_CONTEXT (C *macro*), 642, 644
 SAVE_IRQ_CSR_CONTEXT_S (C *macro*), 642, 645
 SAVE_SSUBM_VAR (C *macro*), 642, 644
 SCAUSE_CAUSE (C *macro*), 119
 SCAUSE_INTR (C *macro*), 119
 SECLIC_CTL_FEAT_EN (C *macro*), 123
 SECLIC_CTL_SHADOW_EN (C *macro*), 123
 SECLIC_CTL_TSP_EN (C *macro*), 123
 SENVCFG_CBCFE (C *macro*), 121
 SENVCFG_CBIE (C *macro*), 120
 SENVCFG_CBIE_EN (C *macro*), 119
 SENVCFG_CBIE_FLUSH (C *macro*), 120
 SENVCFG_CBIE_INVALID (C *macro*), 120
 SENVCFG_CBZE (C *macro*), 121
 SENVCFG_FIOM (C *macro*), 120
 SENVCFG_LPE (C *macro*), 120
 SENVCFG_PMM (C *macro*), 121
 SENVCFG_SSE (C *macro*), 120
 SEVENT_EN (C *macro*), 829
 SFlushDCacheCCacheLine (C *macro*), 733, 735
 SFlushDCacheCCacheLines (C *macro*), 733, 736
 SFlushInvalDCacheCCacheLine (C *macro*), 733, 737
 SFlushInvalDCacheCCacheLines (C *macro*), 733, 737
 SIE_SEIE (C *macro*), 119
 SIE_SEIE (C *macro*), 119
 SIE_STIE (C *macro*), 119
 SInvalDCacheCCacheLine (C *macro*), 733, 734
 SInvalDCacheCCacheLines (C *macro*), 733, 734
 SIP_SEIP (C *macro*), 119
 SIP_SSIP (C *macro*), 119
 SIP_STIP (C *macro*), 119
 SLEEPVALUE_SLEEPVALUE (C *macro*), 121
 SMP_CFG_Type (C++ *union*), 180
 SMP_CFG_Type::b (C++ *member*), 181
 SMP_CTRLREG (C *macro*), 805
 SMP_ENB_Type (C++ *union*), 181
 SMP_ENB_Type::b (C++ *member*), 182
 SMP_ENB_Type::w (C++ *member*), 182
 SMP_PMON_SEL_Type (C++ *union*), 193
 SMP_PMON_SEL_Type::b (C++ *member*), 193
 SMP_PMON_SEL_Type::w (C++ *member*), 193
 SMP_VER_Type (C++ *union*), 180
 SMP_VER_Type::b (C++ *member*), 180
 SMPCC (C *macro*), 179
 SMPCC_BASE (C *macro*), 179
 SMPCC_CLIERRSTS_CC_SCU_ECC_ERR_Msk (C *macro*), 175
 SMPCC_CLIERRSTS_CC_SCU_ECC_ERR_Pos (C *macro*), 175
 SMPCC_CLIERRSTS_IOC_P_BUS_ERR_Msk (C *macro*), 175
 SMPCC_CLIERRSTS_IOC_P_BUS_ERR_Pos (C *macro*), 175
 SMPCC_CLIERRSTS_READ_BUS_ERR_Msk (C *macro*), 175
 SMPCC_CLIERRSTS_READ_BUS_ERR_Pos (C *macro*), 175
 SMPCC_CLIERRSTS_WRITE_BUS_ERR_Msk (C *macro*), 175
 SMPCC_CLIERRSTS_WRITE_BUS_ERR_Pos (C *macro*), 175
 SMPCC_CMD (C *macro*), 757, 763
 SMPCC_CMD_BASE (C *macro*), 757, 763
 SMPCC_CMD_CTRL_SUP_EN_DISABLE (C *macro*), 756, 761
 SMPCC_CMD_CTRL_SUP_EN_ENABLE (C *macro*), 756, 761
 SMPCC_CMD_CTRL_SUP_EN_Msk (C *macro*), 756, 761

SMPCC_CMD_CTRL_SUP_EN_Pos (*C macro*), 756, 761
 SMPCC_CMD_CTRL_USE_EN_DISABLE (*C macro*), 756, 762
 SMPCC_CMD_CTRL_USE_EN_ENABLE (*C macro*), 756, 762
 SMPCC_CMD_CTRL_USE_EN_Msk (*C macro*), 756, 762
 SMPCC_CMD_CTRL_USE_EN_Pos (*C macro*), 756, 762
 SMPCC_CMD_INVALID_ALL_Msk (*C macro*), 757, 763
 SMPCC_CMD_INVALID_ALL_Pos (*C macro*), 757, 763
 SMPCC_CMD_Type (*C++ struct*), 757, 763
 SMPCC_CMD_xCMD_CMD_Msk (*C macro*), 757, 762
 SMPCC_CMD_xCMD_CMD_Pos (*C macro*), 756, 762
 SMPCC_CMD_xCMD_CMD_WB_ALL (*C macro*), 757, 762
 SMPCC_CMD_xCMD_CMD_WBINVAL_ALL (*C macro*), 757, 762
 SMPCC_CMD_xCMD_COMPLETE_Msk (*C macro*), 757, 763
 SMPCC_CMD_xCMD_COMPLETE_Pos (*C macro*), 757, 763
 SMPCC_CMD_xCMD_RESULT_CPBACK_BUS_ERROR (*C macro*), 757, 763
 SMPCC_CMD_xCMD_RESULT_ECC_ERROR (*C macro*), 757, 763
 SMPCC_CMD_xCMD_RESULT_ENTRY_EXCEED_LIMIT (*C macro*), 757, 762
 SMPCC_CMD_xCMD_RESULT_Msk (*C macro*), 757, 762
 SMPCC_CMD_xCMD_RESULT_Pos (*C macro*), 757, 762
 SMPCC_CMD_xCMD_RESULT_REFILL_BUS_ERROR (*C macro*), 757, 762
 SMPCC_CMD_xCMD_RESULT_SUCCESS (*C macro*), 757, 762
 SMPCC_CMD_xCMD_STATUS_Msk (*C macro*), 757, 762
 SMPCC_CMD_xCMD_STATUS_Pos (*C macro*), 757, 762
 SMPCC_CTRL_BUS_ERR_IRQ_EN_DISABLE (*C macro*), 170
 SMPCC_CTRL_BUS_ERR_IRQ_EN_ENABLE (*C macro*), 170
 SMPCC_CTRL_BUS_ERR_IRQ_EN_Msk (*C macro*), 170
 SMPCC_CTRL_BUS_ERR_IRQ_EN_Pos (*C macro*), 170
 SMPCC_CTRL_BUS_ERR_PEND_Msk (*C macro*), 170
 SMPCC_CTRL_BUS_ERR_PEND_Pos (*C macro*), 170
 SMPCC_CTRL_CC_ECC_EN_DISABLE (*C macro*), 169
 SMPCC_CTRL_CC_ECC_EN_ENABLE (*C macro*), 168
 SMPCC_CTRL_CC_ECC_EN_Msk (*C macro*), 168
 SMPCC_CTRL_CC_ECC_EN_Pos (*C macro*), 168
 SMPCC_CTRL_CC_ECC_EXCP_EN_DISABLE (*C macro*), 169
 SMPCC_CTRL_CC_ECC_EXCP_EN_ENABLE (*C macro*), 169
 SMPCC_CTRL_CC_ECC_EXCP_EN_Msk (*C macro*), 169
 SMPCC_CTRL_CC_ECC_EXCP_EN_Pos (*C macro*), 169
 SMPCC_CTRL_CC_EN_DISABLE (*C macro*), 168
 SMPCC_CTRL_CC_EN_ENABLE (*C macro*), 168
 SMPCC_CTRL_CC_EN_Msk (*C macro*), 168
 SMPCC_CTRL_CC_EN_Pos (*C macro*), 168
 SMPCC_CTRL_CLM_ECC_CHK_EN_DISABLE (*C macro*), 172
 SMPCC_CTRL_CLM_ECC_CHK_EN_ENABLE (*C macro*), 172
 SMPCC_CTRL_CLM_ECC_CHK_EN_Msk (*C macro*), 172
 SMPCC_CTRL_CLM_ECC_CHK_EN_Pos (*C macro*), 172
 SMPCC_CTRL_CLM_ECC_EN_DISABLE (*C macro*), 171
 SMPCC_CTRL_CLM_ECC_EN_ENABLE (*C macro*), 171
 SMPCC_CTRL_CLM_ECC_EN_Msk (*C macro*), 171
 SMPCC_CTRL_CLM_ECC_EN_Pos (*C macro*), 171
 SMPCC_CTRL_CLM_EXCP_EN_DISABLE (*C macro*), 171
 SMPCC_CTRL_CLM_EXCP_EN_ENABLE (*C macro*), 171
 SMPCC_CTRL_CLM_EXCP_EN_Msk (*C macro*), 171
 SMPCC_CTRL_CLM_EXCP_EN_Pos (*C macro*), 171
 SMPCC_CTRL_EARLY_WR_ERR_Msk (*C macro*), 173
 SMPCC_CTRL_EARLY_WR_ERR_Pos (*C macro*), 173
 SMPCC_CTRL_ECC_CHK_EN_DISABLE (*C macro*), 171
 SMPCC_CTRL_ECC_CHK_EN_ENABLE (*C macro*), 171
 SMPCC_CTRL_ECC_CHK_EN_Msk (*C macro*), 171
 SMPCC_CTRL_ECC_CHK_EN_Pos (*C macro*), 171
 SMPCC_CTRL_FATAL_ERR_IRQ_EN_DISABLE (*C macro*), 170
 SMPCC_CTRL_FATAL_ERR_IRQ_EN_ENABLE (*C macro*), 170
 SMPCC_CTRL_FATAL_ERR_IRQ_EN_Msk (*C macro*), 170
 SMPCC_CTRL_FATAL_ERR_IRQ_EN_Pos (*C macro*), 170
 SMPCC_CTRL_I_SNOOP_D_EN_DISABLE (*C macro*), 173
 SMPCC_CTRL_I_SNOOP_D_EN_ENABLE (*C macro*), 173
 SMPCC_CTRL_I_SNOOP_D_EN_Msk (*C macro*), 173
 SMPCC_CTRL_I_SNOOP_D_EN_Pos (*C macro*), 173
 SMPCC_CTRL_IOCC_ERR_Msk (*C macro*), 173
 SMPCC_CTRL_IOCC_ERR_Pos (*C macro*), 173
 SMPCC_CTRL_LOCK_ECC_CFG_LOCK (*C macro*), 169
 SMPCC_CTRL_LOCK_ECC_CFG_Msk (*C macro*), 169
 SMPCC_CTRL_LOCK_ECC_CFG_Pos (*C macro*), 169
 SMPCC_CTRL_LOCK_ECC_ERR_INJ_LOCK (*C macro*), 169
 SMPCC_CTRL_LOCK_ECC_ERR_INJ_Msk (*C macro*), 169
 SMPCC_CTRL_LOCK_ECC_ERR_INJ_Pos (*C macro*), 169
 SMPCC_CTRL_PF_BIU_OUTS_EN_DISABLE (*C macro*), 173
 SMPCC_CTRL_PF_BIU_OUTS_EN_ENABLE (*C macro*), 172
 SMPCC_CTRL_PF_BIU_OUTS_EN_Msk (*C macro*), 172
 SMPCC_CTRL_PF_BIU_OUTS_EN_Pos (*C macro*), 172
 SMPCC_CTRL_PF_L2_EARLY_EN_DISABLE (*C macro*), 172
 SMPCC_CTRL_PF_L2_EARLY_EN_ENABLE (*C macro*), 172
 SMPCC_CTRL_PF_L2_EARLY_EN_Msk (*C macro*), 172
 SMPCC_CTRL_PF_L2_EARLY_EN_Pos (*C macro*), 172
 SMPCC_CTRL_PF_NO_WB_DISABLE (*C macro*), 173
 SMPCC_CTRL_PF_NO_WB_ENABLE (*C macro*), 173
 SMPCC_CTRL_PF_NO_WB_Msk (*C macro*), 173
 SMPCC_CTRL_PF_NO_WB_Pos (*C macro*), 173
 SMPCC_CTRL_PF_SH_CL_EN_DISABLE (*C macro*), 172
 SMPCC_CTRL_PF_SH_CL_EN_ENABLE (*C macro*), 172
 SMPCC_CTRL_PF_SH_CL_EN_Msk (*C macro*), 172
 SMPCC_CTRL_PF_SH_CL_EN_Pos (*C macro*), 172
 SMPCC_CTRL_RECV_ERR_IRQ_EN_DISABLE (*C macro*), 169

SMPCC_CTRL_RECV_ERR_IRQ_EN_ENABLE (C macro), 169
 SMPCC_CTRL_RECV_ERR_IRQ_EN_Msk (C macro), 169
 SMPCC_CTRL_RECV_ERR_IRQ_EN_Pos (C macro), 169
 SMPCC_CTRL_SUP_CMD_EN_DISABLE (C macro), 170
 SMPCC_CTRL_SUP_CMD_EN_ENABLE (C macro), 170
 SMPCC_CTRL_SUP_CMD_EN_Msk (C macro), 170
 SMPCC_CTRL_SUP_CMD_EN_Pos (C macro), 170
 SMPCC_CTRL_USE_CMD_EN_DISABLE (C macro), 171
 SMPCC_CTRL_USE_CMD_EN_ENABLE (C macro), 171
 SMPCC_CTRL_USE_CMD_EN_Msk (C macro), 171
 SMPCC_CTRL_USE_CMD_EN_Pos (C macro), 170
 SMPCC_DFF_PROT_CHK_EN_DISABLE (C macro), 177
 SMPCC_DFF_PROT_CHK_EN_ENABLE (C macro), 177
 SMPCC_DFF_PROT_CHK_EN_Msk (C macro), 177
 SMPCC_DFF_PROT_CHK_EN_Pos (C macro), 177
 SMPCC_ERR_INJ_CS_Msk (C macro), 175
 SMPCC_ERR_INJ_CS_Pos (C macro), 174
 SMPCC_ERR_INJ_CS_START (C macro), 175
 SMPCC_ERR_INJ_INJCLM_DISABLE (C macro), 174
 SMPCC_ERR_INJ_INJCLM_ENABLE (C macro), 174
 SMPCC_ERR_INJ_INJCLM_Msk (C macro), 174
 SMPCC_ERR_INJ_INJCLM_Pos (C macro), 174
 SMPCC_ERR_INJ_INJDATA_DISABLE (C macro), 174
 SMPCC_ERR_INJ_INJDATA_ENABLE (C macro), 174
 SMPCC_ERR_INJ_INJDATA_Msk (C macro), 173
 SMPCC_ERR_INJ_INJDATA_Pos (C macro), 173
 SMPCC_ERR_INJ_INJECCCODE_Msk (C macro), 175
 SMPCC_ERR_INJ_INJECCCODE_Pos (C macro), 175
 SMPCC_ERR_INJ_INJMODE_DIRECT (C macro), 174
 SMPCC_ERR_INJ_INJMODE_Msk (C macro), 174
 SMPCC_ERR_INJ_INJMODE_Pos (C macro), 174
 SMPCC_ERR_INJ_INJMODE_XOR (C macro), 174
 SMPCC_ERR_INJ_INJTAG_DISABLE (C macro), 174
 SMPCC_ERR_INJ_INJTAG_ENABLE (C macro), 174
 SMPCC_ERR_INJ_INJTAG_Msk (C macro), 174
 SMPCC_ERR_INJ_INJTAG_Pos (C macro), 174
 SMPCC_NS_RG_CFG_DISABLE (C macro), 178
 SMPCC_NS_RG_CFG_Msk (C macro), 178
 SMPCC_NS_RG_CFG_NACL (C macro), 178
 SMPCC_NS_RG_CFG_NAPOT (C macro), 178
 SMPCC_NS_RG_CFG_Pos (C macro), 178
 SMPCC_PMON_CLIENT_SEL_Msk (C macro), 179
 SMPCC_PMON_CLIENT_SEL_Pos (C macro), 179
 SMPCC_PMON_EVENT (C macro), 179
 SMPCC_PMON_EVENT_DATA_READ_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DATA_READ_HIT_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DATA_READ_MISS_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DATA_READ_REPLACE_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DATA_WRITE_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DATA_WRITE_REPLACE_COUNT (C macro), 178
 SMPCC_PMON_EVENT_DISABLE (C macro), 178
 SMPCC_PMON_EVENT_INSTR_READ_COUNT (C macro), 178
 SMPCC_PMON_EVENT_INSTR_READ_HIT_COUNT (C macro), 179
 SMPCC_PMON_EVENT_INSTR_READ_MISS_COUNT (C macro), 179
 SMPCC_PMON_EVENT_INSTR_READ_REPLACE_COUNT (C macro), 179
 SMPCC_PMON_EVENT_SEL_Msk (C macro), 178
 SMPCC_PMON_EVENT_SEL_Pos (C macro), 178
 SMPCC_STMCFG_RD_BYTE_THRE_Msk (C macro), 177
 SMPCC_STMCFG_RD_BYTE_THRE_Pos (C macro), 177
 SMPCC_STMCFG_RD_DEGREE_Msk (C macro), 177
 SMPCC_STMCFG_RD_DEGREE_Pos (C macro), 177
 SMPCC_STMCFG_RD_DISTANCE_Msk (C macro), 177
 SMPCC_STMCFG_RD_DISTANCE_Pos (C macro), 177
 SMPCC_STMCFG_WR_BYTE_THRE_Msk (C macro), 177
 SMPCC_STMCFG_WR_BYTE_THRE_Pos (C macro), 177
 SMPCC_STMCTRL_CROSS_EN_DISABLE (C macro), 177
 SMPCC_STMCTRL_CROSS_EN_ENABLE (C macro), 177
 SMPCC_STMCTRL_CROSS_EN_Msk (C macro), 177
 SMPCC_STMCTRL_CROSS_EN_Pos (C macro), 176
 SMPCC_STMCTRL_RD_MERGE_EN_DISABLE (C macro), 176
 SMPCC_STMCTRL_RD_MERGE_EN_ENABLE (C macro), 176
 SMPCC_STMCTRL_RD_MERGE_EN_Msk (C macro), 176
 SMPCC_STMCTRL_RD_MERGE_EN_Pos (C macro), 176
 SMPCC_STMCTRL_RD_STM_EN_DISABLE (C macro), 176
 SMPCC_STMCTRL_RD_STM_EN_ENABLE (C macro), 176
 SMPCC_STMCTRL_RD_STM_EN_Msk (C macro), 175
 SMPCC_STMCTRL_RD_STM_EN_Pos (C macro), 175
 SMPCC_STMCTRL_TRANS_ALLOC_DISABLE (C macro), 176
 SMPCC_STMCTRL_TRANS_ALLOC_ENABLE (C macro), 176
 SMPCC_STMCTRL_TRANS_ALLOC_Msk (C macro), 176
 SMPCC_STMCTRL_TRANS_ALLOC_Pos (C macro), 176
 SMPCC_STMCTRL_WR_STM_EN_DISABLE (C macro), 176
 SMPCC_STMCTRL_WR_STM_EN_ENABLE (C macro), 176
 SMPCC_STMCTRL_WR_STM_EN_Msk (C macro), 176
 SMPCC_STMCTRL_WR_STM_EN_Pos (C macro), 176
 SMPCC_Type (C++ struct), 195
 SMPU_A (C macro), 131
 SMPU_A_NA4 (C macro), 131
 SMPU_A_NAPOT (C macro), 131
 SMPU_A_TOR (C macro), 131
 smpu_config (C++ type), 700, 702
 SMPU_R (C macro), 131
 SMPU_S (C macro), 131
 SMPU_SHIFT (C macro), 131
 SMPU_W (C macro), 131
 SMPU_X (C macro), 131

SNOOP_PENDING_Type (C++ union), 187
 SNOOP_PENDING_Type::b (C++ member), 188
 SPMP_A (C macro), 131
 SPMP_A_NA4 (C macro), 131
 SPMP_A_NAPOT (C macro), 131
 SPMP_A_TOR (C macro), 131
 spmp_config (C++ struct), 701, 705
 SPMP_COUNT (C macro), 131
 SPMP_L (C macro), 131
 SPMP_R (C macro), 131
 SPMP_SHIFT (C macro), 131
 SPMP_U (C macro), 131
 SPMP_W (C macro), 131
 SPMP_X (C macro), 131
 SSTATUS32_SD (C macro), 115
 SSTATUS64_SD (C macro), 115
 SSTATUS_FS (C macro), 115
 SSTATUS_MXR (C macro), 115
 SSTATUS_SIE (C macro), 115
 SSTATUS_SPIE (C macro), 115
 SSTATUS_SPP (C macro), 115
 SSTATUS_SUM (C macro), 115
 SSTATUS_UBE (C macro), 115
 SSTATUS_UIE (C macro), 115
 SSTATUS_UPIE (C macro), 115
 SSTATUS_UXL (C macro), 115
 SSTATUS_VS (C macro), 115
 SSTATUS_XS (C macro), 115
 SSUBM_GPRIDX (C macro), 122
 SSUBM_PGPRIDX (C macro), 122
 SSUBM_PTyp (C macro), 122
 SSUBM_Typ (C macro), 122
 STM_CFG_Type (C++ union), 190
 STM_CFG_Type::b (C++ member), 191
 STM_CFG_Type::w (C++ member), 191
 STM_CTRL_Type (C++ union), 190
 STM_CTRL_Type::b (C++ member), 190
 STM_CTRL_Type::w (C++ member), 190
 STM_TIMEOUT_Type (C++ union), 191
 STM_TIMEOUT_Type::b (C++ member), 191
 STM_TIMEOUT_Type::w (C++ member), 191
 STORE_PAGE_FAULT (C macro), 133
 system_core_interrupt_handler (C++ member), 818
 system_core_interrupt_handler_s (C++ member), 818
 SYSTEM_CORE_INTNUM (C macro), 811
 system_default_exception_handler (C++ function), 817
 system_default_exception_handler_s (C++ function), 817
 system_default_interrupt_handler (C++ function), 813
 system_default_interrupt_handler_s (C++ function), 812
 system_mmode_extirq_handler (C++ function), 811
 system_smode_extirq_handler (C++ function), 812
 SystemBannerPrint (C++ function), 805
 SystemCoreClock (C++ member), 810
 SystemCoreClockUpdate (C++ function), 805
 SystemCoreInterruptHandlers (C++ member), 818
 SystemCoreInterruptHandlers_S (C++ member), 818
 SystemExceptionHandlerHandlers (C++ member), 818
 SystemExceptionHandlerHandlers_S (C++ member), 818
 SystemInit (C++ function), 805
 SystemMExtInterruptHandlers (C++ member), 818
 SystemSExtInterruptHandlers (C++ member), 818
 SysTimer (C macro), 197
 SysTimer_BASE (C macro), 197
 SysTimer_CLINT_MSIP_BASE (C macro), 197
 SysTimer_CLINT_MSIP_OFS (C macro), 197
 SysTimer_CLINT_MTIME_BASE (C macro), 198
 SysTimer_CLINT_MTIME_OFS (C macro), 197
 SysTimer_CLINT_MTIMECMP_BASE (C macro), 198
 SysTimer_CLINT_MTIMECMP_OFS (C macro), 197
 SysTimer_CLINT_SSIP_BASE (C macro), 198
 SysTimer_CLINT_SSIP_OFS (C macro), 197
 SysTimer_GetHartID (C macro), 614, 615
 SysTimer_GetHartID_S (C macro), 614, 615
 SysTimer_MSFRST_KEY (C macro), 197
 SysTimer_MSFRST_Msk (C macro), 197
 SysTimer_MSIP_MSIP_Msk (C macro), 196
 SysTimer_MSIP_MSIP_Pos (C macro), 196
 SysTimer_MSIP_Msk (C macro), 197
 SysTimer_MTIME_SRW_CTRL_SRW_Msk (C macro), 197
 SysTimer_MTIME_SRW_CTRL_SRW_Pos (C macro), 197
 SysTimer_MTIMECTL_CLKSRC_Msk (C macro), 196
 SysTimer_MTIMECTL_CLKSRC_Pos (C macro), 196
 SysTimer_MTIMECTL_CMPCLREN_Msk (C macro), 196
 SysTimer_MTIMECTL_CMPCLREN_Pos (C macro), 196
 SysTimer_MTIMECTL_HDBG_Msk (C macro), 196
 SysTimer_MTIMECTL_HDBG_Pos (C macro), 196
 SysTimer_MTIMECTL_Msk (C macro), 197
 SysTimer_MTIMECTL_MTIME_SRC_Msk (C macro), 196
 SysTimer_MTIMECTL_MTIME_SRC_Pos (C macro), 196
 SysTimer_MTIMECTL_TIMESTOP_Msk (C macro), 196
 SysTimer_MTIMECTL_TIMESTOP_Pos (C macro), 196
 SysTimer_MTIMER_Msk (C macro), 197
 SysTimer_MTIMERCMP_Msk (C macro), 197
 SysTimer_SSIP_SSIP_Msk (C macro), 196
 SysTimer_SSIP_SSIP_Pos (C macro), 196
 SysTimer_Type (C++ struct), 198

T

T_UINT16_READ (C++ member), 74
 T_UINT16_WRITE (C++ member), 74

T_UINT32_READ (C++ member), 74
 T_UINT32_WRITE (C++ member), 74
 TRANS_PENDING_Type (C++ union), 188
 TRANS_PENDING_Type::b (C++ member), 188
 Trap_Init (C++ function), 809
 twiddleCoef_1024 (C++ member), 1128, 1152
 twiddleCoef_1024_q15 (C++ member), 1129, 1159
 twiddleCoef_1024_q31 (C++ member), 1129, 1155
 twiddleCoef_128 (C++ member), 1128, 1151
 twiddleCoef_128_q15 (C++ member), 1129, 1158
 twiddleCoef_128_q31 (C++ member), 1128, 1154
 twiddleCoef_16 (C++ member), 1128, 1150
 twiddleCoef_16_q15 (C++ member), 1129, 1157
 twiddleCoef_16_q31 (C++ member), 1128, 1153
 twiddleCoef_2048 (C++ member), 1128, 1152
 twiddleCoef_2048_q15 (C++ member), 1129, 1159
 twiddleCoef_2048_q31 (C++ member), 1129, 1156
 twiddleCoef_256 (C++ member), 1128, 1151
 twiddleCoef_256_q15 (C++ member), 1129, 1158
 twiddleCoef_256_q31 (C++ member), 1128, 1155
 twiddleCoef_32 (C++ member), 1128, 1150
 twiddleCoef_32_q15 (C++ member), 1129, 1157
 twiddleCoef_32_q31 (C++ member), 1128, 1153
 twiddleCoef_4096 (C++ member), 1128, 1153
 twiddleCoef_4096_q15 (C++ member), 1129, 1160
 twiddleCoef_4096_q31 (C++ member), 1129, 1156
 twiddleCoef_512 (C++ member), 1128, 1152
 twiddleCoef_512_q15 (C++ member), 1129, 1158
 twiddleCoef_512_q31 (C++ member), 1128, 1155
 twiddleCoef_64 (C++ member), 1128, 1151
 twiddleCoef_64_q15 (C++ member), 1129, 1157
 twiddleCoef_64_q31 (C++ member), 1128, 1154
 twiddleCoefF16_1024 (C++ member), 1129, 1162
 twiddleCoefF16_128 (C++ member), 1129, 1161
 twiddleCoefF16_2048 (C++ member), 1129, 1162
 twiddleCoefF16_256 (C++ member), 1129, 1161
 twiddleCoefF16_32 (C++ member), 1129, 1160
 twiddleCoefF16_4096 (C++ member), 1129, 1162
 twiddleCoefF16_512 (C++ member), 1129, 1161
 twiddleCoefF16_64 (C++ member), 1129, 1160
 twiddleCoefF16_rfft_1024 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_128 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_2048 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_256 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_32 (C++ member), 1129, 1163
 twiddleCoefF16_rfft_4096 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_512 (C++ member), 1130, 1163
 twiddleCoefF16_rfft_64 (C++ member), 1130, 1163

twiddleCoefF64_1024 (C++ member), 1128, 1149
 twiddleCoefF64_128 (C++ member), 1128, 1148
 twiddleCoefF64_16 (C++ member), 1127, 1147
 twiddleCoefF64_2048 (C++ member), 1128, 1149
 twiddleCoefF64_256 (C++ member), 1128, 1148
 twiddleCoefF64_32 (C++ member), 1127, 1147
 twiddleCoefF64_4096 (C++ member), 1128, 1150
 twiddleCoefF64_512 (C++ member), 1128, 1149
 twiddleCoefF64_64 (C++ member), 1127, 1148
 TXEVT_TXEVT (C macro), 121

U

UCODE_OV (C macro), 121
 UEVENT_EN (C macro), 829
 UFlushDCacheCCacheLine (C macro), 733, 736
 UFlushDCacheCCacheLines (C macro), 733, 736
 UFlushInvalDCacheCCacheLine (C macro), 733, 738
 UFlushInvalDCacheCCacheLines (C macro), 733, 738
 UInvalDCacheCCacheLine (C macro), 733, 734
 UInvalDCacheCCacheLines (C macro), 733, 735
 USER_ECALL (C macro), 133
 USTATUS_UIE (C macro), 115
 USTATUS_UPIE (C macro), 115

V

vector_base (C++ member), 810
 VM_MBARE (C macro), 128
 VM_MBB (C macro), 128
 VM_MBBID (C macro), 128
 VM_SV32 (C macro), 128
 VM_SV39 (C macro), 128
 VM_SV48 (C macro), 128

W

Weights_128 (C++ member), 1196, 1197
 Weights_2048 (C++ member), 1196, 1197
 Weights_512 (C++ member), 1196, 1197
 Weights_8192 (C++ member), 1196, 1198
 WeightsQ31_128 (C++ member), 1196, 1198
 WeightsQ31_2048 (C++ member), 1196, 1199
 WeightsQ31_512 (C++ member), 1196, 1198
 WeightsQ31_8192 (C++ member), 1196, 1199
 WFE_WFE (C macro), 121
 WFI_SleepMode_Type (C++ enum), 198, 201
 WFI_SleepMode_Type::WFI_DEEP_SLEEP (C++ enumerator), 198, 201
 WFI_SleepMode_Type::WFI_SHALLOW_SLEEP (C++ enumerator), 198, 201

X

x_exc_entry (C macro), 135
 x_irq_entry (C macro), 135
 XECLIC_CTL_TSP_EN (C macro), 135

XIP, [1453](#)

XRET (*C macro*), [135](#)

XSTATUS_XIE (*C macro*), [135](#)